

Е. А. НИКУЛИН

КОМПЬЮТЕРНАЯ ГРАФИКА. МОДЕЛИ И АЛГОРИТМЫ

Издание второе, стереотипное

РЕКОМЕНДОВАНО

*Ученым советом Нижегородского государственного
технического университета им. Р. Е. Алексеева
в качестве учебного пособия
для студентов направления подготовки
«Информатика и вычислительная техника»*



САНКТ-ПЕТЕРБУРГ
МОСКВА
КРАСНОДАР
2018

ББК 32.81я73
Н 65

Никулин Е. А.

Н 65 Компьютерная графика. Модели и алгоритмы: Учебное пособие. — 2-е изд., стер. — СПб.: Издательство «Лань», 2018. — 708 с.: ил. — (Учебники для вузов. Специальная литература).

ISBN 978-5-8114-2505-1

В книге детально излагаются математические и алгоритмические основы современной компьютерной графики: модели графических объектов на плоскости и в пространстве (точки, векторы, линии и поверхности, включая составные, полиэдры, сплошные и воксельные объекты), геометрические задачи визуализации — комплекс алгоритмов 2d- и 3d-отсечения и удаления, алгоритмы аффинных и проективных преобразований, методы изображения поверхностей, включая текстурирование. Материал сопровождается большим числом иллюстраций, блок-схем алгоритмов и примеров их реализации.

Настоящее пособие предназначено для студентов направления подготовки «Информатика и вычислительная техника». Также может быть полезно аспирантам, преподавателям вузов и всем специалистам, как постигающим основы компьютерной графики, так и разрабатывающим новые алгоритмы и прикладные графические программы.

ББК 32.81я73

Рецензенты:

В. Е. ТУРЛАПОВ — доктор технических наук, профессор кафедры математического обеспечения и суперкомпьютерных технологий Института информационных технологий, математики и механики Нижегородского государственного университета им. Н. И. Лобачевского;

В. В. КОНДРАТЬЕВ — член-корреспондент РАН, профессор, зав. кафедрой вычислительных систем и технологий Нижегородского государственного технического университета им. Р. Е. Алексеева.

Обложка

Е. А. ВЛАСОВА

© Издательство «Лань», 2018

© Е. А. Никулин, 2018

© Издательство «Лань»,

художественное оформление, 2018

СОДЕРЖАНИЕ

ОСНОВНЫЕ ОБОЗНАЧЕНИЯ И СООТНОШЕНИЯ.....	6
АББРЕВИАТУРЫ.....	10
ГЛАВА 1. ГЕОМЕТРИЧЕСКИЕ ОСНОВЫ КОМПЬЮТЕРНОЙ ГРАФИКИ.....	11
1.1. Графические элементы на плоскости.....	17
1.1.1. Модели прямой линии на плоскости.....	18
1.1.2. Взаимное положение графических элементов на плоскости.....	22
1.1.3. Тесты свойств графических элементов на плоскости.....	31
1.1.4. Тесты локализации объектов на плоскости.....	37
1.1.5. Площадь и геометрический центр полигона.....	47
1.1.6. Алгоритмы генерирования случайных полигонов.....	49
1.1.7. Квадратичные линии.....	57
1.1.8. Параметрические линии.....	60
1.1.9. Геометрические преобразования неявных линий.....	64
1.2. Сопряжение на плоскости.....	66
1.2.1. Задачи общего назначения.....	66
1.2.2. Сопряжения с окружностями.....	69
1.2.3. Скругление углов.....	72
1.2.4. Эквидистантные линии.....	74
1.3. Операции со списками.....	77
1.3.1. Объединение списков отрезков.....	79
1.3.2. Пересечение списков отрезков.....	81
1.3.3. Исключение списков отрезков.....	83
1.3.4. Соединение отрезков в контуры.....	85
1.4. Отсечение на плоскости.....	91
1.4.1. Отсечение отрезков.....	92
1.4.2. Отсечение полигонов.....	100
1.4.3. Отсечение полуплоскостями.....	104
1.4.4. Разрезание полигонов.....	108
1.4.5. Триангуляция полигонов.....	114
1.5. Графические элементы в пространстве.....	126
1.5.1. Модели плоской поверхности.....	128
1.5.2. Модели линий в пространстве.....	132
1.5.3. Взаимное положение графических элементов в пространстве.....	135
1.5.4. Модель полиэдра.....	143
1.5.5. Тесты свойств графических элементов в пространстве.....	151
1.5.6. Тесты локализации объектов в пространстве.....	154
1.5.7. Объём и центр тяжести полиэдра.....	167
1.5.8. Выпуклая полиэдральная оболочка массива точек.....	168
1.5.9. Квадратичные поверхности.....	173
1.5.10. Параметрические поверхности.....	178
1.5.11. Геометрические преобразования неявных поверхностей.....	183

1.6. Отсечение в пространстве.....	187
1.6.1. Отсечение отрезков.....	188
1.6.2. Отсечение многоугольников и многогранников.....	192
1.6.3. Отсечение полупространствами.....	200
1.6.4. Сечение многогранника плоскостью.....	207
1.6.5. Разрезание многогранника.....	213
1.7. Удаление в пространстве.....	217
1.7.1. Классификация методов удаления.....	218
1.7.2. Обзор типовых методов удаления.....	222
1.7.3. Удаление невидимых линий.....	233
1.7.4. Удаление невидимых граней.....	238
 ГЛАВА 2. МАТЕМАТИЧЕСКИЕ МОДЕЛИ ЛИНИЙ.....	 243
2.1. Полиномиальные линии.....	246
2.1.1. Интерполяция.....	248
2.1.2. Аппроксимация и приближение по МНК.....	261
2.1.3. Линии Безье.....	267
2.1.4. Конгруэнтные преобразования моделей линий.....	287
2.2. Составные линии.....	294
2.2.1. Сопряжения интерполяционных сегментов.....	295
2.2.2. Сопряжение с перекрытием сегментов.....	304
2.2.3. Составные линии Безье.....	309
2.2.4. Рациональные составные линии Безье.....	321
2.3. Сплайновые линии.....	327
2.3.1. Интерполяция.....	327
2.3.2. Аппроксимация и приближение.....	338
2.3.3. Сплайны с конечными носителями.....	341
 ГЛАВА 3. ГЕОМЕТРИЧЕСКИЕ ПРЕОБРАЗОВАНИЯ.....	 353
3.1. Аффинные преобразования.....	354
3.1.1. Основные понятия и методы преобразований.....	354
3.1.2. Элементарные аффинные преобразования.....	368
3.1.3. Сложные аффинные преобразования.....	377
3.2. Проективные преобразования.....	410
3.2.1. Ортографические проекции.....	411
3.2.2. Аксонометрические проекции.....	412
3.2.3. Косоугольные проекции.....	419
3.2.4. Центральные (перспективные) проекции.....	423
3.2.5. Проективные преобразования.....	436
3.3. Проективные алгоритмы сложных преобразований.....	439
3.3.1. Проецирование пространственных линий на плоскость.....	440
3.3.2. Стереографические проекции.....	444
3.3.3. Картографические проекции.....	448
3.3.4. Построение сцены с подвижным наблюдателем.....	461

ГЛАВА 4. МАТЕМАТИЧЕСКИЕ МОДЕЛИ ПОВЕРХНОСТЕЙ.....	465
4.1. Классификация поверхностей.....	466
4.2. Методы изображения поверхностей.....	469
4.2.1. Выбор проекции изображения.....	470
4.2.2. Каркасные сети.....	471
4.2.3. Полигональные сети.....	475
4.2.4. Текстуры.....	485
4.3. Кинематические поверхности.....	513
4.3.1. Поверхности элементарных преобразований.....	515
4.3.2. Линейчатые поверхности.....	522
4.3.3. Нелинейные поверхности.....	533
4.4. Полиномиальные поверхности.....	553
4.4.1. Интерполяция граничных элементов.....	554
4.4.2. Интерполяция узловых элементов.....	565
4.4.3. Аппроксимация и приближение по МНК.....	573
4.4.4. Поверхности Безье.....	576
4.4.5. Конгруэнтные преобразования моделей поверхностей.....	597
4.4.6. Остовные поверхности.....	600
4.5. Составные поверхности.....	609
4.5.1. Сопряжение интерполяционных сегментов.....	610
4.5.2. Составные поверхности Безье.....	618
4.5.3. Рациональные составные поверхности Безье.....	627
4.5.4. Эквидистантные поверхности.....	635
4.6. Сплайновые поверхности.....	641
4.6.1. Бикубические сплайны.....	641
4.6.2. Сплайны с перекрытием параболических сегментов.....	646
4.7. Дискретные поверхности.....	650
ГЛАВА 5. МОДЕЛИ ОБЪЕКТОВ В ПРОСТРАНСТВЕ.....	656
5.1. Методы трехмерного моделирования.....	657
5.2. Каркасные модели.....	658
5.2.1. Модели полиэдров.....	660
5.2.2. Преобразования моделей полиэдров.....	664
5.3. Граничные модели.....	668
5.4. Сплошные модели.....	671
5.4.1. Воксельные модели.....	672
5.4.2. Конструктивные модели.....	678
5.5. Оптимизация визуализации.....	684
5.5.1. Выбор мировой системы координат.....	684
5.5.2. Отбраковка нелицевых граней и нормировка векторов.....	687
5.5.3. Построение охватывающих оболочек.....	690
5.5.4. Разбиение граней.....	697
БИБЛИОГРАФИЯ.....	703

ОСНОВНЫЕ ОБОЗНАЧЕНИЯ И СООТНОШЕНИЯ

☞ — символ начала алгоритма или примера.

☐ — символ конца алгоритма или примера.

$\mathbb{R}^n$ — n -мерное евклидово пространство **чисел**.

x^0, y^0 и z^0 — ортогональные векторы единичной длины (**орты**), направленные по осям декартовой системы координат хуз.

f, h и p — фронтальная ху, горизонтальная хз и профильная уз **координатные плоскости** декартовой системы координат.

$V = [V_x \ V_y] = V_x x^0 + V_y y^0 \in \mathbb{R}^2$ — **вектор** в плоскости ху с координатами V_x, V_y и **модулем** (нормой, длиной) $|V| = \sqrt{V_x^2 + V_y^2}$.

$V = [V_x \ V_y \ V_z] = V_x x^0 + V_y y^0 + V_z z^0 \in \mathbb{R}^3$ — вектор в пространстве хуз с координатами V_x, V_y, V_z и модулем $|V| = \sqrt{V_x^2 + V_y^2 + V_z^2}$.

V^n — n -мерное евклидово пространство **векторов**.

M^n — n -мерное евклидово пространство **матриц**.

$O_n \in \mathbb{R}^n$ — нулевой вектор или блок вектора размерности n .

$O_{n,m} \in \mathbb{R}^{n \times m}$ — нулевая матрица или блок матрицы размерности $n \times m$.

O — нулевой вектор или матрица контекстно определённой размерности.

$\overset{o}{p} = [\omega p \ \omega] \in \mathbb{R}^{n+1}$ — радиус-вектор точки p в **однородных координатах**.

$\tilde{p} = [p \ 1] \in \mathbb{R}^{n+1}$ — расширение координат **точки** p единицей.

$\tilde{V} = [V \ 0] \in \mathbb{R}^{n+1}$ — расширение координат **вектора** V нулём.

$\bar{V} = V/|V|$ — **нормированный** вектор V единичной длины.

$V \parallel W$ и $V \# W$ — параллельные и непараллельные векторы V и W .

$V \perp W$ и $V \nperp W$ — ортогональные и неортогональные векторы V и W .

$\angle VW$ или $\angle(V, W)$ — угол между векторами V и W .

$V \circ W = V_x W_x + V_y W_y + V_z W_z$ — формула вычисления **скалярного произведения** векторов V и W . Вторая формула $V \circ W = |V| \cdot |W| \cdot \cos(\angle VW)$ позволяет найти косинус угла между векторами V и W как

$$\cos(\angle VW) = \bar{V} \circ \bar{W}.$$

$V \circ V = 0$ — **тест нулевого вектора** V .

$V \circ W = 0$ — **тест ортогональности** векторов V и W .

$\bar{V} \circ \bar{W} = \pm 1$ — **тесты параллельности** (знак «+») и **антипараллельности** (знак «-») векторов V и W .

$$N = V \times W = \begin{vmatrix} x^0 & y^0 & z^0 \\ V_x & V_y & V_z \\ W_x & W_y & W_z \end{vmatrix} = N_x x^0 + N_y y^0 + N_z z^0 \text{ — } \textbf{векторное произведение}$$

векторов $V \in \mathbb{R}^3$ и $W \in \mathbb{R}^3$ с координатами

$$N_x = \begin{vmatrix} V_y & V_z \\ W_y & W_z \end{vmatrix}, \quad N_y = \begin{vmatrix} V_z & V_x \\ W_z & W_x \end{vmatrix}, \quad N_z = \begin{vmatrix} V_x & V_y \\ W_x & W_y \end{vmatrix}$$

и модулем $|N| = |V| \cdot |W| \cdot \sin(\angle VW)$. Синус угла между векторами находится как

$$\sin(\angle VW) = \frac{|V \times W|}{|V| \cdot |W|} = \frac{|\overline{V} \times \overline{W}|}{|V| \cdot |W|}.$$

$$\begin{vmatrix} V_x & V_y \\ W_x & W_y \end{vmatrix} = 0 \text{ — тест параллельности либо антипараллельности 2d-векто-}$$

ров $V \in \mathbb{R}^2$ и $W \in \mathbb{R}^2$.

$|V \times W| = 0$ — тест параллельности либо антипараллельности 3d-векторов $V \in \mathbb{R}^3$ и $W \in \mathbb{R}^3$.

$$\text{pr}_B V = |V| \cos(\angle VB) \overline{B} = \frac{V \circ B}{B \circ B} B = (V \circ \overline{B}) \overline{B} \text{ — проекция вектора } V \text{ на базовый}$$

вектор B . Длина вектора проекции $|\text{pr}_B V| = |V \circ \overline{B}|$ равна нулю при $V \perp B$.

$\text{or}_B V = V - \text{pr}_B V = V - (V \circ \overline{B}) \overline{B}$ — ортогональное дополнение вектора V к базовому вектору B . Длина вектора дополнения $|\text{or}_B V| = \sqrt{V \circ V - (V \circ \overline{B})^2}$ равна нулю при $V \parallel B$.

$A \bullet B$ — операция **декартова** (поэлементного) **произведения равноразмерных** векторов A и B либо матриц A и B , возвращающая вектор либо матрицу с элементами $a_i b_i$ либо $a_{ij} b_{ij}$.

$p_1 p_2 \dots p_n$ — ломаная линия (**полилиния**), заданная списком из n точек в порядке их соединения $p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_n$.

$p_1 p_2 \dots p_n p_1$ — многоугольник (**полигон**), заданный списком из $n+1$ -ой точек-вершин в порядке обхода по замкнутому контуру $p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_n \rightarrow p_1$.

$\overrightarrow{P} = p_n p_1 p_2 \dots p_{n-1} p_n$ — правый и $\overleftarrow{P} = p_2 p_3 \dots p_n p_1 p_2$ — левый циклические сдвиги полигона $P = p_1 p_2 \dots p_n p_1$.

$\overleftrightarrow{P}$ — реверсивный полигон $p_1 p_n \dots p_2 p_1$ с противоположным направлением обхода вершин, чем в полигоне $P = p_1 p_2 \dots p_n p_1$.

$\overline{ab}$ — дуга кривой между точками a и b .

$\nabla f(V) = \frac{df}{dV}$ — вектор градиента скалярной функции f векторного аргумента V .

$\text{lin}(A, B, t) = (1-t)A + tB$ — функция линейной интерполяции между объектами A и B с равномерным изменением параметра $t \in [0, 1]$.

$\text{line}(a, b) = \text{lin}(a, b, t)$ — функция вывода на экран отрезка прямой ab .

$\text{diag}[M_{11} \dots M_{nn}]$ — квадратная матрица $M \in \mathbb{R}^{n \times n}$ с диагональными элементами M_{ij} и недиагональными элементами $M_{ij} = 0 \quad \forall i \neq j$.

$E_n = \text{diag}[1 \dots 1]$ — единичная $n \times n$ -матрица.

$E_{n,m}$ — прямоугольная $n \times m$ -матрица с единичной главной диагональю, принимающая в зависимости от размерностей n и m следующие формы:

- E_n при $n=m$;
- $[E_n \mid O_{n,m-n}]$ при $n < m$;
- $\begin{bmatrix} E_m \\ O_{n-m,m} \end{bmatrix}$ при $n > m$.

$\tilde{M} = \begin{bmatrix} M & O_{n,1} \\ O_{1,m} & I \end{bmatrix} \in R^{(n+1) \times (m+1)}$ — расширение матрицы $M \in R^{n \times m}$ нулевой

нижней строкой, нулевым правым столбцом и правой нижней единицей.

$\text{tr}(M) = \sum_{i=1}^n M_{ii}$ — *след* квадратной матрицы $M \in R^{n \times n}$.

$\text{rang}[M]$ — *ранг* матрицы M , равный числу её линейно независимых строк или столбцов.

$\{cond\} = \begin{cases} 1, & \text{если выполняется условие } cond; \\ 0, & \text{если не выполняется условие } cond. \end{cases}$

$\delta_i^j = \{i=j\} = \begin{cases} 1 & \text{при } i=j \\ 0 & \text{при } i \neq j \end{cases}$ — символ Кронекера.

$\text{if}(cond: a, b) = \begin{cases} a & \text{при } cond \neq 0 \\ b & \text{при } cond = 0 \end{cases}$ — функция ветвления на 2 направления.

$\text{if3}(cond: b, r, m) = \begin{cases} b & \text{при } cond > 0 \\ r & \text{при } cond = 0 \\ m & \text{при } cond < 0 \end{cases}$ — функция ветвления на 3 направления.

$\text{sgn}(x) = \text{if3}(x: 1, 0, -1)$ — функция *знака* действительного числа x .

$\text{mod}(a, b)$ — функция вычисления остатка от деления действительного числа a на действительное число $b \neq 0$. Знак остатка равен знаку числа a . Дробная часть числа x находится как $\{x\} = \text{mod}(x, 1)$.

$x = \max\{x, 0\}$ — неотрицательное *усечение* числа x .

$\lfloor x \rfloor$ — *округление* числа x до *максимального* целого числа $m \leq x$.

$\lceil x \rceil$ — округление числа x до *минимального* целого числа $M \geq x$.

$[x]$ — округление числа x до *ближайшего* целого числа $\lfloor x + 0.5 \rfloor$.

$\text{rnd}(x)$ — функция генерирования случайного числа, равномерно распределённого в интервале $(0, x)$. Случайное число в интервале (a, b) генерируется по формуле $a + \text{rnd}(b - a)$. Случайное *целое* число в интервале с целочисленными границами $[n, m]$ генерируется как $n + [\text{rnd}(m - n)]$.

$++i$ и $--i$ — префиксные формы инкремента и декремента числа i .

$i++$ и $i--$ — постфиксные формы инкремента и декремента числа i .

$L = \{l_1, \dots, l_n\}$ — *список* из n скалярных переменных.

$Lp = \{p_1, \dots, p_n\}$ — список из n несвязанных точек $p_1 \div p_n$.

$Ls = \{a_1b_1, \dots, a_nb_n\}$ — список из n отрезков, элементом которого a_ib_i является список или матрица из двух векторов концевых точек отрезка a_i и b_i .

$LP = \{P_1, \dots, P_n\}$ — список из n полигонов, элемент которого представляет собой полигон $P_i = p_{i1}p_{i2} \dots p_{in}p_{i1}$.

$G = \{G_1, \dots, G_n\}$ — **группа n списков**, элемент которой G_i есть список более низкого уровня иерархии типа L , Lp , Ls или LP .

$size(S)$ — функция расчёта числа элементов структурного объекта S .

$\emptyset$ — символ пустого списка.

$e \rightarrow L$ — добавление элемента e в конец списка L .

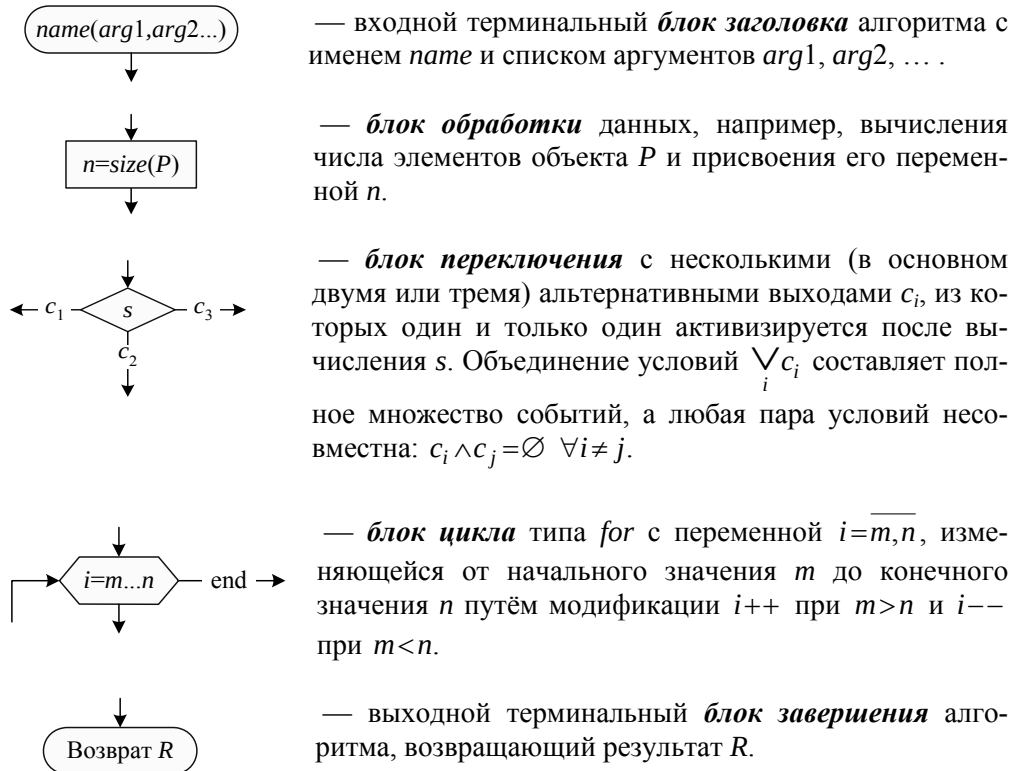
$L - e$ — удаление элемента e из списка L .

$\vec{L} = \{l_n, l_1, \dots, l_{n-1}\}$ — правый и $\overleftarrow{L} = \{l_2, \dots, l_n, l_1\}$ — левый циклические сдвиги списка $L = \{l_1, l_2, \dots, l_{n-1}, l_n\}$.

$mean(L)$ — функция расчёта среднего арифметического значения элементов списка L .

$sort_c(L)$ — функция сортировки элементов списка L по критерию c .

Элементы блок-схем алгоритмов:



АББРЕВИАТУРЫ

АП — аффинное преобразование.
БП — базовая поверхность.
БФТ — билинейная фильтрация текстуры.
ВБФ — вектор базисных функций.
ВМНК — взвешенный метод наименьших квадратов.
ВПК — вектор полиномиальных коэффициентов.
ВФ — весовая функция.
ГКШ — генератор клеточного шума.
ГСЧ — генератор случайных чисел.
ИП — интерполяционный полином.
ИПЭ — интерполяционный полином Эрмита.
КП — картинная плоскость.
МВК — матрица весовых коэффициентов.
МКВ — метод координатных вращений.
МНК — метод наименьших квадратов.
МПК — матрица полиномиальных коэффициентов.
МПТ — метод парных точек.
МСК — мировая система координат.
МЭВ — метод экономичного вращения.
НФ — неявная форма (функция).
ОСК — объектная система координат.
ПС — полигональная сеть.
ПФ — параметрическая форма (функция).
РП — расширенное пространство.
РРСЧ — равномерно распределённое случайное число.
СК — система координат.
СКН — система координат наблюдателя.
СКС — система координат сцены.
СКТ — система координат текстуры.
СЛАУ — система линейных алгебраических уравнений.
ФЗФ — функция закрашки Фонга.
ФКП — фронтальная картинная плоскость.
ХП — характеристическая полилиния.
ХПС — характеристическая полигональная сеть.
ЦП — центральная проекция.
ЭДП — эквидистантная поверхность.
ЭП — элементарное преобразование.
ЭСК — экранная система координат.

Реалистичность восприятия человеком сложного компьютерного изображения определяется умением разработчика *математической модели* изображаемого объекта или процесса достоверно повторить на экране его развитие в пространстве и во времени. Модель включает в себя комплекс уравнений и алгоритмов их реализации. Математической основой построения модели являются зависимости, описывающие форму и движение объектов. Всё многообразие геометрических объектов является комбинацией различных *примитивов* — простейших фигур, которые, в свою очередь, состоят из *графических элементов* — точек, линий, поверхностей и элементов объёма.

Положение *точки* в R^n задаётся *радиус-вектором* $p = [p_1 \ p_2 \ \dots \ p_n]$, имеющим n *координат* $p_1 \div p_n$ и разложение

$$p = p_1 e_1 + p_2 e_2 + \dots + p_n e_n = \sum_{i=1}^n p_i e_i \quad (1.1)$$

по n линейно независимым *базисным векторам* $e_1 \div e_n$. Совокупность базисных векторов и единиц измерения расстояний вдоль этих векторов составляет *систему координат* (СК). В пространствах с $n \leq 3$ вместо цифр используются синонимы — буквенные обозначения координат. Например, первой координате можно сопоставить букву x , второй — букву y , а третьей — букву z . Если при этом векторы $e_1 \equiv x^0$, $e_2 \equiv y^0$ и $e_3 \equiv z^0$ нормированы и взаимно ортогональны, то xyz является декартовой, наиболее распространённой системой координат.

Для описания формы графических объектов, задания расположения объектов в пространстве и создания их экранных проекций используются различные СК, наиболее удобные в каждом конкретном случае. Перечислим системы, применяемые в настоящей книге, и кратко охарактеризуем их назначение:

- неподвижная *мировая система координат* (МСК) xyz содержит точку отсчёта (*начало координат*) и линейно независимый *базис*, благодаря которым становится возможным цифровое описание геометрических свойств любого графического объекта в абсолютных единицах;

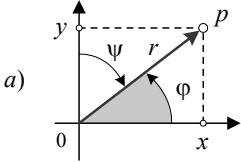
- *экранная система координат* (ЭСК) $x_3 y_3 z_3$, в которой задаётся положение *проекций* геометрических объектов на экране *дисплея* — устройства отображения графической информации. Проекция точки в ЭСК имеет аппликату $z_3 = 0$. Тем не менее её не следует отбрасывать, поскольку МСК и ЭСК часто выбираются совпадающими друг с другом и, кроме того, вектор проекции $[x_3 \ y_3 \ 0]$ может участвовать в преобразованиях с тремя координатами;

- *система координат сцены* (СКС) $x_c y_c z_c$, в которой описывается положение всех объектов *сцены* — некоторой части мирового пространства с собственными началом отсчёта и базисом, используемыми для описания положения объектов независимо от МСК;

• **объектная** (локальная) **система координат (ОСК)** $x_0y_0z_0$, связанная с конкретным объектом и совершающая с ним все движения в СКС или МСК.

По умолчанию будем полагать все координаты заданными в МСК.

В пространстве R^2 наиболее распространены ортогональная **декартова** (x, y) и **полярная** (r, φ) системы (рис. 1.1, а), связанные следующими соотно-



шениями координат:

$$\begin{cases} x = r \cos(\varphi), y = r \sin(\varphi); \\ r = \sqrt{x^2 + y^2}, \operatorname{tg}(\varphi) = \frac{y}{x}. \end{cases} \quad (1.2)$$

В R^3 наиболее распространены ортогональная **декартова** (x, y, z) , **цилиндрическая** (ρ, φ, ψ) и **сферическая** (r, φ, ψ) системы (рис. 1.1, б), связанные друг с другом следующими соотношениями координат:

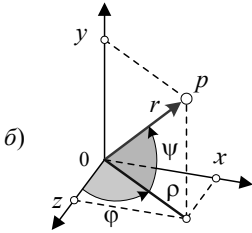


Рис. 1.1

• декартова СК с цилиндрической СК:

$$\begin{cases} x = \rho \sin(\varphi), z = \rho \cos(\varphi); \\ \rho = \sqrt{x^2 + z^2}, \operatorname{tg}(\varphi) = \frac{x}{z}; \end{cases} \quad (1.3a)$$

• декартова СК со сферической СК:

$$\begin{cases} x = r \sin(\varphi) \cos(\psi), y = r \sin(\varphi) \sin(\psi), z = r \cos(\varphi); \\ r = \sqrt{x^2 + y^2 + z^2}, \operatorname{tg}(\varphi) = x/z, \sin(\psi) = y/\sqrt{x^2 + y^2 + z^2}; \end{cases} \quad (1.3б)$$

• цилиндрическая СК со сферической СК:

$$\begin{cases} \rho = r \cos(\psi), y = r \sin(\psi); \\ r = \sqrt{\rho^2 + y^2}, \sin(\psi) = y/\sqrt{\rho^2 + y^2}. \end{cases} \quad (1.3в)$$

Для единообразия терминов и определённости алгоритмов мы будем работать в **правой** декартовой системе координат, оси которой ориентированы в пространстве так, что вращения ортов $x^0 \rightarrow y^0$, $y^0 \rightarrow z^0$ и $z^0 \rightarrow x^0$ происходят в положительном направлении, т. е. **против часовой стрелки** с точки зрения наблюдателя, смотрящего с конца третьего свободного орта.

При вычислении угловых координат φ и ψ в (1.3) необходимо сопоставлять интервалы их изменения с интервалами углов, возвращаемых обратными тригонометрическими функциями $\arcsin(x) \in [-90^\circ, 90^\circ]$, $\arccos(x) \in [0^\circ, 180^\circ]$ и $\operatorname{arctg}(x) \in (-90^\circ, 90^\circ)$. Анализируя интервалы изменения углов $\varphi \in (-180^\circ, 180^\circ]$ и $\psi \in [-90^\circ, 90^\circ]$ на рис. 1.1, б, заключаем, что правильно вычисляется только угол ψ в (1.3б) и в (1.3в) с помощью функции $\arcsin$, тогда как угол φ не определяется верно по отдельности ни одной из функций $\arcsin$, $\arccos$ или arctg .

Определим новую **функцию вычисления аргумента точки** $p = [x \ y] \in R^2$ как угла **кратчайшего** вращения от оси x до радиус-вектора точки p с учётом знаков её координат x и y :

$$\boxed{\arg(p) = \text{if}(y \geq 0: 1, -1) \cdot \arccos\left(\frac{x}{|p|}\right) \in (-\pi, \pi]}. \quad (1.4)$$

При $y \geq 0$ неотрицательный угол вычисляется как $\arccos(x/|p|) \in [0, \pi]$, а при отрицательной ординате $y < 0$ и той же абсциссе x у такого же значения угла инвертируется знак. С помощью функции (1.4) стало возможным правильное вычисление углов $\varphi = \arg(p)$ в (1.2) и $\varphi = \arg([z \ x])$ в (1.3).

Взаимосвязь координат графического элемента может быть описана **неявным уравнением** $f(p)=0$ и **параметрической функцией** $p(t)$. Частным случаем неявного описания является явная форма, в которой одна из координат выражена через другие. Это возможно лишь для некоторых простейших линий и поверхностей. **Не все графические элементы могут быть описаны неявными функциями.** Наиболее универсальным является параметрическое описание каждой из n координат графического элемента соответствующей функцией времени, угла поворота или других параметров.

Целью настоящего краткого экскурса в вычислительную геометрию, не претендующего на исчерпывающую полноту и строгость, является систематизация формул, необходимых для работы с графическими объектами, и приведение их к **алгоритмическому** виду. Это отличает компьютерную графику от обычной, например, бумажной или наскальной, в которой свойства и взаимосвязь графических объектов визуально воспринимаются глазами человека и интеллектуально интерпретируются его мозгом. С методами решения задач классической геометрии можно познакомиться, например, в [7, 8].

Компьютерная или вычислительная геометрия оперирует не рисунками, а числами (координатами точек и векторов, матрицами преобразований, списками данных) и отношениями между ними, выраженными также в цифровой форме. Исходная информация, обработанная специальными алгоритмами, преобразуется в другие цифровые данные, интерпретируемые как искомый результат — модель объекта, его экранное изображение, отношения между объектами сцены и т. п. Подробное изложение основ вычислительной геометрии и геометрического моделирования приведено в [3, 13, 17, 20, 28, 48, 56].

В этой главе рассматриваются основные соотношения между графическими элементами на плоскости R^2 и в пространстве R^3 . Все наиболее важные формулы записываются в координатной и векторной формах. Векторная форма компактнее координатной и предпочтительнее для понимания и программирования алгоритмов, хотя и требует несколько больших машинных ресурсов — объёма памяти и времени счёта. Увеличение ресурсов объясняется наличием в векторах и матрицах фиксированных элементов (единиц и нулей), с которыми иногда выполняются бесполезные арифметические операции. Тем не менее координатные зависимости, аккумулирующие несколько векторно-матричных преобразований, целесообразно использовать в окончательном программировании отлаженных алгоритмов для ускорения их работы ценой разбухания программного кода.

Выше приведены основные обозначения и соотношения векторно-матричной алгебры. Если используемая для создания графического проекта среда программирования не содержит никаких средств из этой области математики, программист вынужден будет создать собственную библиотеку отлаженных функций для работы с векторами и матрицами.

В реализации простейших действий с объектами данного класса (сложение, вычитание, умножение, транспонирование, нормирование, вычисление следа) обычно не возникает никаких проблем. Затруднения начинаются при программировании самых трудоёмких операций — обращения матрицы и расчёта её определителя. Многие известные алгоритмы их выполнения [9, 10, 12, 21, 59] нередко ставят разработчика библиотеки перед мучительным выбором компромисса между математической надёжностью методов, их вычислительной эффективностью и затратами времени на программирование и отладку.

Автор рекомендует выполнять обращение квадратной $n \times n$ -матрицы M и расчёт её определителя с максимально возможной точностью по простому в программировании рекуррентному алгоритму Леверрье – Фаддеева [12]:

$$\begin{cases} L_1 = E_n, \quad \alpha_i = \frac{\text{tr}(ML_i)}{i}, \quad L_{i+1} = ML_i - \alpha_i E_n \quad \forall i = \overline{1, n}, \\ M^{-1} = \frac{L_n}{\alpha_n}, \quad |M| = (-1)^{n-1} \alpha_n. \end{cases} \quad (1.5)$$

Без применения специальных приёмов оптимизации алгоритм потребляет вспомогательную память объёмом $2n^2 + 1$ для хранения матриц L , $M \cdot L$ и числа α . При отладке алгоритма полезно знать его свойство $L_{n+1} = O_{n,n}$.

На примере третьей нетривиальной матричной операции — вычисления ранга прямоугольной $n \times m$ -матрицы M методом ортогонализации Грама – Шмидта [10, 12, 24] — познакомимся с принятым в книге стилем оформления **блок-схем** алгоритмов — мощных изобразительных средств усвоения логики вычислений. На рис. 1.2,а построена блок-схема алгоритма $\text{rang}(M)$, возвращающего через имя rang максимальное количество её линейно независимых строк, которое по определению равно **рангу матрицы** M .

Инициализируем ранг $r=0$ и, рассматривая n строк матрицы как векторы $V_i \in \mathbb{R}^m$ в m -мерном пространстве, организуем цикл $i = \overline{1, n}$ формирования **ортонормированного базиса** векторов $\{e_1, e_2, \dots, e_r\}$ по следующим правилам:

- первый базисный вектор e_1 вычисляем нормировкой первого встретившегося **ненулевого** вектора V_i , после чего устанавливаем значение ранга $r=1$;
- очередной вектор e_{r+1} получаем нормировкой **ненулевого** вектора

$$N = V_i - \sum_{j=1}^r (V_i \circ e_j) e_j$$

ортogonalного дополнения вектора V_i к текущему базисному пространству $E = e_1 \times e_2 \times \dots \times e_r$, после чего значение r инкрементируется.

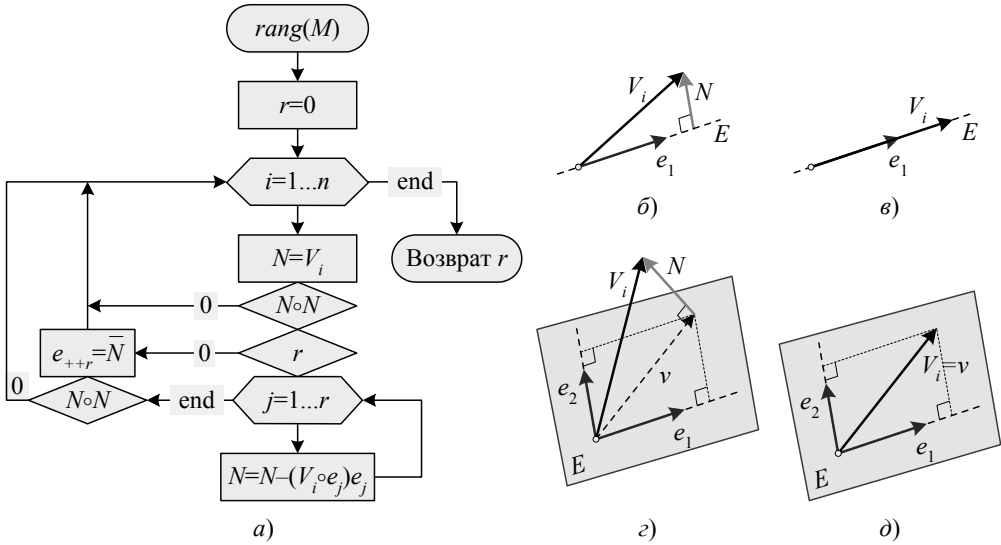


Рис. 1.2

Поясним приведённую формулу читателю, подзабывшему векторную алгебру, на частных примерах.

1. При $r=1$ базисное пространство — прямая линия — образовано одним вектором e_1 единичной длины, а проекция V_i на E равна $\alpha_1 e_1$, где $\alpha_1 = V_i \circ e_1$. Ортогональное дополнение $N = V_i - \alpha_1 e_1 \perp e_1$ (рис. 1.2,б) даёт направление второго базисного вектора $e_2 = N/|N|$. Однако, при $V_i \parallel e_1$ вектор $V_i = \alpha_1 e_1 \in E$ не имеет ортогонального дополнения к E (рис. 1.2,в) и не увеличивает ранга матрицы.

2. При $r=2$ базисная плоскость натянута на два вектора $e_1 \perp e_2$, а проекция V_i на E равна $v = \alpha_1 e_1 + \alpha_2 e_2$, где $\alpha_2 = V_i \circ e_2$. Ортогональное дополнение $N = V_i - v \perp E$ (рис. 1.2,г) задаёт направление третьего базисного вектора $e_3 = \bar{N}$. Однако, при $V_i \in E$ ортогональное дополнение отсутствует (рис. 1.2,д) и увеличения ранга матрицы не происходит.

Таково **теоретическое** описание алгоритма расчёта ранга матрицы. В реальной же вычислительной практике неизбежны различные погрешности [10], изменяющие как числа, так и условия принятия решений. Например, в алгоритме *rang* решение об увеличении ранга и расширении базисного пространства принимается в зависимости от длины вектора N . Но поскольку его координаты вычисляются с неизбежными погрешностями, то условие $N \circ N = 0$ никогда не выполнится и ранг матрицы вычислится неверно.

Для **огрубления** алгоритма (снижения его чувствительности к ошибкам округления) введём пороговое значение фильтрации погрешностей $\varepsilon \approx 10^{-12} \div 10^{-9}$. Изменив решающие условия на $N \circ N \leq \varepsilon$, получим устойчивый алгоритм вычисления ранга матрицы. Задав $\varepsilon=0$, всегда можно вернуться к его теоретическому варианту. Аналогичные меры по повышению грубости всех алгоритмов и тестов, **теоретически** точные варианты которых рассматриваются в настоящей

книге, следует принять в логических формулах, сравнивающих некоторую переменную v , вычисленную с *плавающей точкой*, с заданным числом c :

- строгое неравенство $v < c$ заменить на $v < c - \varepsilon$;
- нестрогое неравенство $v \leq c$ заменить на $v \leq c + \varepsilon$;
- сравнение $v = c$ заменить на неравенство $|v - c| \leq \varepsilon$;
- сравнение $v \neq c$ заменить на условие $|v - c| > \varepsilon$;
- нестрогое неравенство $v \geq c$ заменить на $v \geq c - \varepsilon$;
- строгое неравенство $v > c$ заменить на $v > c + \varepsilon$.

☞ Многие геометрические задачи компьютерной графики связаны с решением той или иной *системы линейных алгебраических уравнений* (СЛАУ), записываемой в векторно-матричной форме как

$$p \cdot A = B, \quad p \in \mathbb{R}^n, \quad A \in \mathbb{R}^{n \times m}, \quad B \in \mathbb{R}^m, \quad (1.6a)$$

где n — число неизвестных элементов вектора p , а m — число скалярных уравнений. В зависимости от соотношения между этими числами и **при условии полного ранга матрицы системы** $\text{rang}[A] = \min\{n, m\}$ тип системы и методы её решения таковы [9, 10, 12, 54, 59]:

- при $n = m$ СЛАУ полностью *определена* и имеет *единственное* решение

$$\boxed{p = BA^{-1}}; \quad (1.6б)$$

- при $n < m$ система (1.6a) *переопределена* и при $\text{rang}[A^T] < \text{rang}[A^T \ B^T]$ *несовместна*. Составив из квадрата длины вектора невязки $pA - B$ функционал

$$\mathfrak{J}(p) = |pA - B|^2 = (pA - B)(pA - B)^T = pAA^T p^T - 2BA^T p^T + BB^T$$

и минимизируя его по вектору p , получим *приближённое* решение по **методу наименьших квадратов** (МНК):

$$2pAA^T = 2BA^T \Rightarrow \boxed{p = BA^T(AA^T)^{-1}}, \quad (1.6в)$$

которое становится *точным* при равенстве рангов матриц A^T и $[A^T \ B^T]$;

- при $n > m$ система *недоопределена* и имеет *бесконечное множество решений*, из которых представляет интерес решение с *минимальной нормой* $|p|$.

Сформировав *функционал Лагранжа* с вектором множителей $\lambda \in \mathbb{R}^m$

$$\mathfrak{J}(p, \lambda) = p \circ p + 2\lambda \circ (pA - B)$$

и минимизируя его по p и λ , получим условное МНК-решение задачи:

$$\begin{cases} \partial \mathfrak{J} / \partial p = 2p + 2\lambda A^T = O_n \\ \partial \mathfrak{J} / \partial \lambda = 2(pA - B) = O_m \end{cases} \Rightarrow \boxed{p = B(A^T A)^{-1} A^T}. \quad (1.6г)$$

Отметим, что все обращаемые в (1.6) матрицы невырождены, а при $n = m$ оба решения (1.6в) и (1.6г) переходят в (1.6б).

1.1. ГРАФИЧЕСКИЕ ЭЛЕМЕНТЫ НА ПЛОСКОСТИ

Точка имеет нулевую топологическую размерность. **Расстояние между двумя точками** $p_1=[x_1 \ y_1] \equiv [r_1 \ \varphi_1]$ и $p_2=[x_2 \ y_2] \equiv [r_2 \ \varphi_2]$ равно:

$$d = \begin{cases} |p_1 - p_2| = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} & \text{— в декартовой СК,} \\ \sqrt{r_1^2 + r_2^2 - 2r_1r_2 \cos(\varphi_1 - \varphi_2)} & \text{— в полярной СК.} \end{cases}$$

Линия имеет одну степень свободы. **Модели линии** в неявной (**НФ**) и параметрической (**ПФ**) формах имеют следующий вид:

$$\text{НФ:} \quad f(x, y) = 0 \quad \text{или} \quad f(p) = 0; \quad (1.7a)$$

$$\text{ПФ:} \quad p(t) = [x(t) \ y(t)]. \quad (1.7b)$$

В любой регулярной (гладкой и не кратной) точке на линии $q = [q_x \ q_y] \equiv p(t_q)$ возможна **линеаризация** кривой (рис. 1.3), т. е. проведение к ней **касательной прямой**:

$$\text{НФ:} \quad \begin{cases} N_x(x - q_x) + N_y(y - q_y) = 0, \\ N_x = \left. \frac{df(x, y)}{dx} \right|_{q_x, q_y}, \quad N_y = \left. \frac{df(x, y)}{dy} \right|_{q_x, q_y} \end{cases} \quad \text{или} \quad \begin{cases} N \circ (p - q) = 0, \\ N = \nabla f(p)|_q; \end{cases} \quad (1.8a)$$

$$\text{ПФ:} \quad \begin{cases} x(t) = q_x + V_x t, \quad y(t) = q_y + V_y t, \\ V_x = \left. \frac{dx(t)}{dt} \right|_{t_q}, \quad V_y = \left. \frac{dy(t)}{dt} \right|_{t_q} \end{cases} \quad \text{или} \quad \begin{cases} p(t) = q + Vt, \\ V = \left. \frac{dp(t)}{dt} \right|_{t_q}. \end{cases} \quad (1.8b)$$

Вектор **нормали** $N = [N_x \ N_y]$ ортогонален к линии и по свойству градиента направлен в ту сторону, где $f(p) > 0$. При замене уравнения линии на $-f(p) = 0$ направление нормали становится противоположным.

Направляющий вектор линии $V = [V_x \ V_y]$ выходит из точки q в направлении касательной к $p(t)$ в сторону **увеличения** параметра t . Для перехода от одной формы (1.8) к другой необходимо выбрать одно из двух ортогональных направлений нормали относительно направляющего вектора. Из условия ортогональности $N \circ V = 0$, т. е. $N_x V_x + N_y V_y = 0$, получаем следующий способ быстрых преобразований $V \leftrightarrow N$:

- координаты исходного вектора переставляются местами;
- у одной из координат полученного вектора меняется знак.

Для примера возьмём вектор $V = [1 \ 2]$. После перестановки координат он становится равным $[2 \ 1]$, а после изменения знака, например, у **первой** координаты получим вектор $N = [-2 \ 1]$, направленный **влево** от V . Проверка показывает, что векторы V и N ортогональны: $V \circ N = 1 \cdot (-2) + 2 \cdot 1 = 0$.

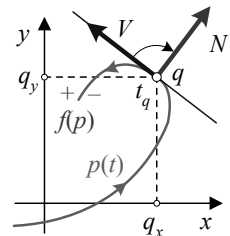


Рис. 1.3

В целях единообразия примем **правую ориентацию нормали**, означающую, что при движении точки по линии вектор N направлен перпендикулярно вправо от вектора V . Определим для дальнейшего использования две **функции правого $Rrot(V)$ и левого $Lrot(V)$ ортогональных вращений вектора V** :

$$Rrot(V) = [V_y \ -V_x], \quad Lrot(V) = [-V_y \ V_x]. \quad (1.9)$$

С их помощью векторы нормали и направления прямой на плоскости становятся связанными друг с другом соотношениями $N = Rrot(V)$ и $V = Lrot(N)$.

Направление и изгиб плоской параметрической линии в точке $p(t)$ определяются скоростью изменения наклона касательного вектора $V(t) = p'(t)$. Определённая в дифференциальной геометрии [8, 21] **кривизна линии** с учётом соотношения $N = Rrot(V)$ может быть вычислена по следующим формулам:

$$\kappa(t) = \frac{|V(t)|}{|V'(t)|^3} = \frac{|N(t)|}{|N'(t)|^3} = \frac{V(t) \circ N'(t)}{|V(t)|^3}. \quad (1.10a)$$

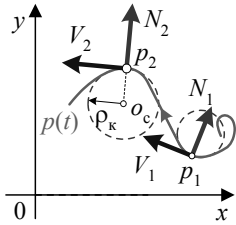


Рис. 1.4

Знак кривизны совпадает с направлением вращения вектора $V(t)$ при движении по линии в сторону увеличения параметра t . При **положительной** кривизне $\kappa > 0$ условно **выпуклая** линия изгибается **влево**, а её правая нормаль направлена в сторону, противоположную изгибу (рис. 1.4). На вогнутом участке линии нормаль обращена в сторону правого изгиба. На прямых участках линии кривизна обращается в нуль.

Обратное значение κ^{-1} того же знака составляет **радиус кривизны** ρ_k , отложив который от точки p вдоль вектора $-N$, получим центр **соприкасающейся окружности** o_c :

$$\rho_k = \frac{1}{\kappa}, \quad o_c = p - \rho_k \overline{N} = p - \frac{V \circ V}{V \circ N'} N. \quad (1.10б)$$

На выпуклых участках эта окружность расположена слева от линии, а на вогнутых — справа. На прямых отрезках с нулевой кривизной радиус соприкасающейся окружности бесконечен, а положение её центра не определено.

1.1.1. Модели прямой линии на плоскости

Прямая линия является наиболее простой и востребованной в компьютерной графике алгебраической линией первого порядка. Рассмотрим различные формы её задания.

Неявное уравнение прямой

Неявное уравнение прямой, заданной коэффициентами A , B и D , составляющими вектор $F = [A \ B \ D]$, имеет координатный и векторный виды:

$$\text{НФ:} \quad Ax + By + D = 0 \quad \text{или} \quad \boxed{[p \ 1] \circ F = \tilde{p} \circ F = 0}. \quad (1.11)$$

Хотя бы одно из чисел A или B должно быть ненулевым. Если оба коэффициента $A \neq 0$ и $B \neq 0$, то прямая проходит наклонно к осям координат и пересекается с ними в двух точках $p_x = [-D/A \ 0]$ и $p_y = [0 \ -D/B]$ (рис. 1.5,а).

При $A=0$ и $B \neq 0$ уравнение $By + D = 0$ описывает горизонтальную, а уравнение $Ax + D = 0$ при $A \neq 0$ и $B=0$ — вертикальную прямые. Отметим ограниченность «школьного» способа приведения неявного координатного уравнения к явному виду $y(x) = -(A/B)x - (D/B)$ и построения линии по точкам в интервале $x \in [x_{\min}, x_{\max}]$. Этот способ неприменим для вертикальных прямых с $B=0$ и единственной координатой $x = -D/A$ для всех точек таких прямых.

Прямая $f(p) \equiv \tilde{p} \circ F = 0$ проходит через начало координат при $f(O_2) \equiv D = 0$. Благодаря её свойству разделять пространство плоскости на два подпространства с противоположными знаками $f(p)$, неявная функция позволяет с минимальными вычислительными затратами (за два умножения и два сложения) определить по условию $f(q) = 0$ принадлежность произвольной точки q прямой линии, а также положение двух точек a и b относительно прямой (рис. 1.5,а):

- при $f(a)f(b) > 0$ точки расположены по одну сторону от прямой;
- при $f(a)f(b) < 0$ точки a и b лежат по разные стороны от прямой.

Для построения прямой по уравнению $Ax + By + D = 0$ нужно иметь либо две несовпадающие точки o и p_1 , через которые она проходит, либо точку o и направляющий вектор V , с помощью которого вторая точка вычисляется как $p_1 = o + V$. Сравнивая (1.11) с (1.8а), заключаем, что **коэффициенты неявного уравнения A и B можно принять координатами нормали $N = [A \ B]$** , а с помощью (1.9) получить направляющий вектор прямой $V = Lrot(N) = [-B \ A]$.

Рассмотрим два способа выбора точки o .

1. Первое, что приходит на ум, — выбрать её там, где прямая пересекается с осью x при $A \neq 0$ или (только горизонтальная прямая) с осью y при $A=0$:

$$o = \text{if}(A: p_x, p_y). \quad (1.12a)$$

Несмотря на лёгкость понимания, этот способ не самый лучший для программирования из-за условности выбора точки в зависимости от коэффициента A .

2. Найдём **ближайшую к началу системы координат** точку o (рис. 1.5,а) как МНК-решение (1.6г) недоопределённого уравнения прямой $pN^T = -D$:

$$o = \frac{-D}{N \circ N} N = \frac{-D}{A^2 + B^2} [A \ B]. \quad (1.12б)$$

Данное решение в сравнении с (1.12а) безальтернативно, более удобно в программировании и эффективнее в вычислении.

Таким образом, координатная и векторная параметрические формы прямой, заданной вектором $F = [A \ B \ D]$, принимают вид

$$\begin{cases} x(t) = \frac{-AD}{A^2+B^2} - Bt \\ y(t) = \frac{-BD}{A^2+B^2} + At \end{cases} \quad \text{или} \quad p(t) = \frac{-D}{A^2+B^2} [A \ B] + [-B \ A] t. \quad (1.13)$$

Нормальная модель прямой

Нормальное уравнение прямой $\{o, N\}$, заданной точкой o и вектором нормали N , выводится из условия ортогональности векторов N и $p-o$ для всех точек p , принадлежащих прямой (рис. 1.5,б):

$$\text{НФ:} \quad N_x(x-o_x) + N_y(y-o_y) = 0 \quad \text{или} \quad \boxed{(p-o) \circ N = 0}; \quad (1.14a)$$

$$\text{ПФ:} \quad x(t) = o_x - N_y t, \quad y(t) = o_y + N_x t \quad \text{или} \quad p(t) = o + Vt, \quad (1.14б)$$

где $V = Lrot(N) = [-N_y \ N_x]$. Сравнивая (1.14a) с (1.11), получим зависимость вектора неявного уравнения от параметров нормального уравнения:

$$F = [N \ -o \circ N]. \quad (1.15)$$

Неявная функция $f(p) = (p-o) \circ N$ позволяет оценить положение точки p относительно прямой не только по знаку возвращаемого ею значения, но также и относительно направления нормали:

- при $f(a) > 0$ точка a лежит в том полупространстве, в которое направлена нормаль, так что угол $\angle(a-o, N)$ острый;
- при $f(b) < 0$ угол $\angle(b-o, N)$ тупой, а точка b и нормаль находятся по разные стороны от прямой.

Параметрическая модель прямой

Параметрические функции прямой $\{o, V\}$, проходящей через точку o в направлении вектора V (рис. 1.5,б), и эквивалентная им НФ имеют вид:

$$\text{ПФ:} \quad x(t) = o_x + V_x t, \quad y(t) = o_y + V_y t \quad \text{или} \quad \boxed{p(t) = o + Vt}; \quad (1.16a)$$

$$\text{НФ:} \quad V_y x - V_x y + D = 0, \quad D = V_x o_y - V_y o_x \Rightarrow F = [N \ -o \circ N], \quad (1.16б)$$

где $N = Rrot(V) = [V_y \ -V_x]$. **Параметрическая модель удобна для задания и построения частей прямой** — отрезков и лучей. Для этого в (1.16a) необходимо указать пределы изменения параметра t . Например:

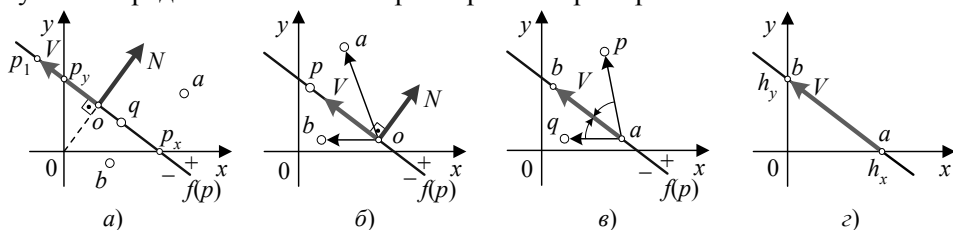


Рис. 1.5

- бесконечный интервал $-\infty < t < \infty$ не ограничивает протяжённость бесконечной **прямой** в обоих направлениях V и $-V$;
- задание $t \geq 0$ даёт **луч**, идущий из точки o в бесконечность в направлении вектора V ;
- конечный интервал $t_0 \leq t \leq t_1$ определяет **отрезок** прямой между точками $o + Vt_0$ и $o + Vt_1$.

Благодаря левой ориентации направляющего вектора V относительно вектора нормали N эквивалентная нормальной форме параметрическая функция

$$f(p) = (p - o) \circ [V_y \ -V_x] = \left| \frac{p - o}{V} \right|,$$

равная определителю матрицы, составленной из векторов $p - o$ и V , позволяет определить положение точки относительно направления движения по прямой:

- при $f(a) > 0$ точка a лежит справа, так что угол $\angle(a - o, V)$ положительный;
- при $f(b) < 0$ угол $\angle(b - o, V)$ отрицательный, а точка b расположена слева.

Модели прямой, проходящей через две точки

Неявная форма **уравнения прямой, проходящей через точки** $a = [a_x \ a_y]$ и $b = [b_x \ b_y]$ (рис. 1.5, в), выводится из условия принадлежности прямой этих двух точек и точки $p = [x \ y]$:

$$\text{НФ: } \begin{bmatrix} \tilde{p} \\ \tilde{b} \\ \tilde{a} \end{bmatrix} F^T = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \begin{vmatrix} x & y & 1 \\ b_x & b_y & 1 \\ a_x & a_y & 1 \end{vmatrix} = 0 \Rightarrow \underbrace{\begin{vmatrix} b_y & 1 \\ a_y & 1 \end{vmatrix}}_{N_x} x + \underbrace{\begin{vmatrix} 1 & b_x \\ 1 & a_x \end{vmatrix}}_{N_y} y + \underbrace{\begin{vmatrix} b_x & b_y \\ a_x & a_y \end{vmatrix}}_D = 0. \quad (1.17a)$$

Выбрав направление движения по прямой от точки a к точке b , определим направляющий вектор прямой как $V = b - a$ и её параметрическую модель в координатной и векторной формах:

$$\text{ПФ: } x(t) = a_x + (b_x - a_x)t, \ y(t) = a_y + (b_y - a_y)t \text{ или } \boxed{p(t) = a + (b - a)t}. \quad (1.17б)$$

Условие существования прямой очевидно: $|V| \neq 0$, т. е. $a \neq b$ — несовпадение точек, через которые проходит прямая.

При изменении параметра от $i=0$ до $i=1$ движение точки происходит по отрезку ab от точки a к точке b . Ниже рассматриваются свойства плоских многоугольников (**полигонов**), определяемые ориентацией векторов нормалей к их сторонам. С учётом принятого выше соглашения о правой ориентации нормали к прямой **внешняя ориентация нормалей к сторонам полигона обеспечивается при его обходе против часовой стрелки**. Соответствующая функция

$$f(p) = (p - a) \circ [b_y - a_y \ a_x - b_x] = \left| \frac{p - a}{b - a} \right|$$

позволяет определить нахождение точки p относительно прямой линии при движении от точки a к точке b . При $f(p) > 0$ точка p лежит справа от прямой, а при $f(q) < 0$ точка q находится от неё слева.

В частности, двухточечную прямую можно провести через точки $a=[h_x \ 0]$ и $b=[0 \ h_y]$, отсекающие на осях координат отрезки h_x и h_y , длины которых одновременно не равны нулю (рис. 1.5,з). Подставив в (1.11) значения $A=-D/h_x$ и $B=-D/h_y$, получим неявное **уравнение прямой в отрезках**:

$$\text{НФ:} \quad \frac{x}{h_x} + \frac{y}{h_y} - 1 = 0 \Rightarrow h_y x + h_x y - h_x h_y = 0 \Rightarrow F = [h_y \ h_x \ -h_x h_y]. \quad (1.18a)$$

По вектору нормали $N=[h_y \ h_x]$ с помощью (1.9) получим направляющий вектор прямой $V=Lrot(N)=[-h_x \ h_y]$ и её параметрические функции:

$$\text{ПФ:} \quad x(t)=h_x(1-t), \ y(t)=h_y t \text{ или } p(t)=[h_x(1-t) \ h_y t]. \quad (1.18б)$$

Итак, в зависимости от способа задания прямой мы получили разные формы и способы вычисления её НФ $f(p)$. Введём **синонимы** этой функции nf^* , где «*» заменяет третий символ её имени «F», «N», «V» или «2»:

$$nfF(F,p)=\tilde{p} \circ F; \quad (1.19a)$$

$$nfN(o,N,p)=(p-o) \circ N; \quad (1.19б)$$

$$nfV(o,V,p)=\left| \begin{smallmatrix} p-o \\ V \end{smallmatrix} \right| = (p-o) \circ Rrot(V); \quad (1.19в)$$

$$nf2(a,b,p)=\left| \begin{smallmatrix} p-a \\ b-a \end{smallmatrix} \right| = (p-a) \circ Rrot(b-a). \quad (1.19г)$$

1.1.2. Взаимное положение графических элементов на плоскости

Рассмотрим некоторые **полезные формулы, тесты и алгоритмы**, описывающие свойства и взаимное расположение точек, прямых и полигонов на плоскости. Для получения необходимых результатов активно используются тесты ортогональности и параллельности векторов, приведённые в перечне основных обозначений и соотношений.

Коллинеарность точек

Три точки p_1 , p_2 и p_3 коллинеарны, т. е. лежат на одной прямой (рис. 1.6,а), при коллинеарности векторов p_2-p_1 и p_3-p_1 :

$$\left| \begin{smallmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{smallmatrix} \right| = 0 \text{ или } \left| \begin{smallmatrix} p_2-p_1 \\ p_3-p_1 \end{smallmatrix} \right| = 0. \quad (1.20a)$$

Обобщение (1.20a) на произвольное число точек $p_1 \div p_n$:

$$\text{rang} \begin{bmatrix} \tilde{p}_1 \\ \dots \\ \tilde{p}_n \end{bmatrix} = 2 \quad \text{или} \quad \text{rang} \begin{bmatrix} p_2 - p_1 \\ \dots \\ p_n - p_1 \end{bmatrix} = 1. \quad (1.206)$$

Точка p лежит на отрезке ab при нулевом угле между векторами $p-a$ и $b-p$, построенными на трёх точках (рис. 1.6,б):

$$(p-a) \circ (b-p) = |p-a| \cdot |b-p|. \quad (1.21)$$

Значение параметра t , соответствующее положению точки $p = o + Vt$ на прямой $\{o, V\}$ с вектором направления V (рис. 1.6,в), вычисляется с помощью скалярного произведения векторов $p-o$ и V :

$$(p-o) \circ V = Vt \circ V = |V|^2 t \Rightarrow t = \frac{(p-o) \circ V}{V \circ V}. \quad (1.22)$$

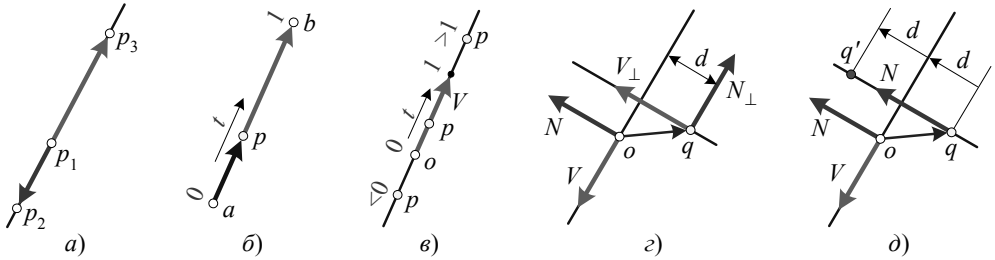


Рис. 1.6

Точка и прямая

Модели перпендикуляра, опущенного из точки $q = [q_x \ q_y]$ на прямую (рис. 1.6,з), заданную в НФ или ПФ, выглядят следующим образом:

$$\text{НФ:} \quad N_y(x-q_x) - N_x(y-q_y) = 0 \Rightarrow N_{\perp} = [N_y \ -N_x], \quad F_{\perp} = [N_{\perp} \ -N_{\perp} \circ q]; \quad (1.23a)$$

$$\text{ПФ:} \quad p_{\perp}(t) = q + Nt \quad \text{или} \quad p_{\perp}(t) = q + V_{\perp}t, \quad \text{где} \quad V_{\perp} = [V_y \ -V_x]. \quad (1.236)$$

Расстояние от точки q до прямой — это длина вектора проекции $q-o$ на N :

$$\text{НФ:} \quad d = \frac{|f(q)|}{|N|} = |\text{pr}_N(q-o)| = |(q-o) \circ \overline{N}|; \quad (1.24a)$$

$$\text{ПФ:} \quad d = \frac{|f(q)|}{|V|} = \left| \frac{q-o}{V} \right| = |q-o - ((q-o) \circ \overline{V})\overline{V}|. \quad (1.246)$$

Зеркальное отражение точки q от прямой лежит на перпендикуляре, опущенном из точки на прямую, на двойном расстоянии (1.24a), отложенном от точки q против проекции вектора $q-o$ на нормаль N (рис. 1.6,д):

$$\text{НФ:} \quad q' = q - 2\text{pr}_N(q-o) = q - 2((q-o) \circ \overline{N})\overline{N}; \quad (1.25a)$$

$$\text{ПФ:} \quad q' = q - 2\text{or}_V(q-o) = 2o - q + 2((q-o) \circ \overline{V})\overline{V}. \quad (1.256)$$

Пример 1.1. Отобразить треугольник $p_1p_2p_3$ с вершинами $p_1=[0 \ 0]$, $p_2=[0 \ 1]$ и $p_3=[2 \ 0]$ от прямой линии, проходящей через точки $a=[3 \ 0]$ и $b=[2 \ 2]$ (рис. 1.7).

Решение. Находим направляющий вектор прямой $V=b-a=[-1 \ 2]$, с помощью (1.9) нормаль $N=[2 \ 1]$ и квадрат её длины $N \circ N=5$. Пусть $o=a$, тогда по (1.25а) последовательно

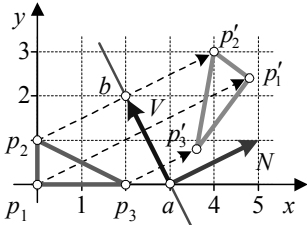


Рис. 1.7

для точек p_1, p_2 и p_3 вычисляем отражённые точки

$$p'_i = p_i - \frac{2(p_i - a) \circ N}{N \circ N} N;$$

$$p'_1 = [0 \ 0] - 0.4([-3 \ 0] \circ [2 \ 1])[2 \ 1] = [4.8 \ 2.4],$$

$$p'_2 = [0 \ 1] - 0.4([-3 \ 1] \circ [2 \ 1])[2 \ 1] = [4 \ 3],$$

$$p'_3 = [2 \ 0] - 0.4([-1 \ 0] \circ [2 \ 1])[2 \ 1] = [3.6 \ 0.8].$$

Прямая, минимально удалённая от совокупности точек

Рассмотрим одну интересную задачу, связанную с понятием расстояния от точки до объекта как длины перпендикуляра, опущенного из точки на объект. Допустим, в $\mathbb{R}^n$ (при решении задачи на плоскости xy будем полагать $n=2$) задано $m \geq 3$ произвольных точек $p_1 \div p_m$. Сформируем из них матрицу

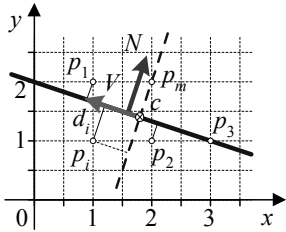


Рис. 1.8

$$P = \begin{bmatrix} p_1 \\ \dots \\ p_m \end{bmatrix} \in \mathbb{R}^{m \times n}. \quad (1.26)$$

Так как в общем случае точки не коллинеарны, то не существует ни одной прямой $f(p)=0$, для которой все значения $d_i = f(p_i)$ обращаются в нуль. Другими словами, расстояния $|d_i|/|N|$ от точек p_i до прямой с нормалью N , вычисленные по (1.24а), не могут быть одновременно все нулевыми (рис. 1.8).

Построим *аппроксимирующую прямую* $\{c, N\}$, минимально удалённую от совокупности точек p_i , путём решения экстремальной задачи

$$\sum_{i=1}^m d_i^2 \rightarrow \min.$$

Для устранения неопределённости введём ограничение $|N|=1$, т. к. ориентация прямой линии зависит лишь от направления, но не длины вектора нормали. При выполнении данного ограничения числа d_i являются *алгебраическими* (со знаками) расстояниями от точек до прямой. Из пяти рассмотренных выше моделей прямой линии выберем неявную форму (1.11) с вектором $F=[N \ D]$, имеющим минимальное число скалярных параметров, равное $n+1=3$. При таком выборе m -вектор $d \in \mathbb{R}^m$ расстояний от точек до прямой принимает вид

$$d = F\tilde{P}^T = NP^T + D \underbrace{[1 \ \dots \ 1]}_m,$$

а квадрат его длины равен

$$\begin{aligned} |d|^2 &= dd^T = (NP^T + D[1 \dots 1])(PN^T + D[1 \dots 1]^T) = \\ &= NP^T PN^T + 2D \left(\sum_{i=1}^m p_i \right) N^T + mD^2 = NP^T PN^T + 2mDcN^T + mD^2, \end{aligned}$$

где точка

$$c = \text{mean}(P) = \frac{1}{m} \sum_{i=1}^m p_i \quad (1.27)$$

является **геометрическим центром** массива точек p_i . Задача аппроксимации решается по МНК путём минимизации **функции Лагранжа**

$$\mathfrak{Z}(N, D, \lambda) = |d|^2 + \lambda (|N|^2 - 1) = NP^T PN^T + 2mDcN^T + mD^2 + \lambda (NN^T - 1). \quad (1.28a)$$

Из условий её экстремума

$$\begin{cases} \partial \mathfrak{Z} / \partial N = 2NP^T P + 2mDc + 2\lambda N = O_n, \\ \partial \mathfrak{Z} / \partial D = 2m(N \circ c) + 2mD = 0, \quad \partial \mathfrak{Z} / \partial \lambda = NN^T - 1 = 0 \end{cases} \quad (1.28б)$$

и свойства $(a \circ b) \cdot c = a \cdot (b^T c)$ произведения скалярного произведения двух векторов на третий вектор следует, что

$$D = -N \circ c, \quad N(P^T P - mc^T c + \lambda E_n) = O_n. \quad (1.28в)$$

Сравнение выражений для коэффициента D в (1.28в) и (1.15) убеждает, что все прямые, дающие экстремум функции $\mathfrak{Z}(N, D, \lambda)$, проходят через точку $o=c$.

Второе соотношение в (1.28в) представляет собой однородную СЛАУ относительно координат вектора N . Она имеет нетривиальное решение $N \neq O_n$ при нулевом определителе матрицы $\lambda E_n - M$, где

$$M = mc^T c - P^T P \in \mathbb{R}^{n \times n}. \quad (1.28г)$$

Следовательно, множитель Лагранжа λ есть одно из n **собственных чисел** (значений), а N — соответствующий ему нормированный **собственный вектор** матрицы M . Число собственных значений и векторов квадратной матрицы равно её размерности. В случае $n=2$ симметричная матрица M имеет две действительные собственные пары $\{\lambda_1, N_1\}$ и $\{\lambda_2, N_2\}$, причём $N_1 \perp N_2$ при $\lambda_1 \neq \lambda_2$. Это означает, что прямые, доставляющие экстремум функции $\mathfrak{Z}(N, D, \lambda)$, **ортгональны**. Вычисление собственных чисел и векторов матрицы — хорошо отработанная процедура вычислительной математики, описанная, например, в [10, 12, 21, 54, 59]. Следует позаботиться о нормировании собственных векторов в том случае, если алгоритм их вычисления не делает эту операцию сам.

Возникает закономерный вопрос: какой из полученных векторов N_1 или N_2 следует взять в качестве нормали искомой прямой? Ответ очевиден: тот, чья прямая доставляет **минимум** функции Лагранжа (1.28а). Ведь условия (1.28б) дают лишь экстремальные значения, которые при $D = -N \circ c$ и $|N|=1$ равны

$$\mathfrak{Z} = NP^T PN^T - m(N \circ c)^2 = -NMN^T = -\lambda NN^T = -\lambda \geq 0.$$

Отсюда следует, что все собственные числа матрицы M не положительны, а минимум функции Лагранжа доставляет собственная пара $\{\lambda, N\}$ с **максимальным** (ближним к нулю) значением $\lambda = \max\{\lambda_1, \lambda_2\}$ и соответствующим ему вектором N — нормалью искомой прямой $\{c, N\}$. Для удобства построения найденной прямой линии можно перейти от её нормальной формы (1.14а) к параметрической (1.14б) с направляющим вектором $V = \text{Lrot}(N) = [-N_y \ N_x]$.

Оставшаяся собственная пара матрицы M с минимальным собственным числом даёт максимум функции Лагранжа и ортогональную (при $\lambda_1 \neq \lambda_2$) прямую (на рис. 1.8 она изображена штрихом), пересекающуюся с оптимальной прямой в точке c .

Пример 1.2. Построить прямую линию, наименее отклонённую от точек $p_1 = [1 \ 2]$, $p_2 = [2 \ 1]$, $p_3 = [3 \ 1]$, $p_4 = [1 \ 1]$ и $p_5 = [2 \ 2]$ (рис. 1.8).

Решение. Из заданных точек составим матрицу (1.26) и найдём их центр (1.27):

$$P = \begin{bmatrix} 1 & 2 & 3 & 1 & 2 \\ 2 & 1 & 1 & 1 & 2 \end{bmatrix}^T, \quad c = \frac{p_1 + p_2 + p_3 + p_4 + p_5}{5} = [1.8 \ 1.4].$$

Вычислим по (1.28г) матрицу M и с помощью MathCAD её собственные параметры:

$$M = 5 \begin{bmatrix} 1.8 \\ 1.4 \end{bmatrix} \begin{bmatrix} 1.8 & 1.4 \end{bmatrix} - \begin{bmatrix} 1 & 2 & 3 & 1 & 2 \\ 2 & 1 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 1 & 2 \\ 2 & 1 & 1 & 1 & 2 \end{bmatrix}^T = \begin{bmatrix} -2.8 & 0.6 \\ 0.6 & -1.2 \end{bmatrix},$$

$$\lambda_1 = -3, \quad \boxed{\lambda_2 = -1}, \quad N_1 = [-0.949 \ 0.316], \quad N_2 = [0.316 \ 0.949].$$

Отделив максимальное значение $\lambda = \lambda_2$, получим параметры неявной формы искомой прямой: $N = [0.316 \ 0.949]$, $D = -N \circ c = -1.897$.

Таким образом, аппроксимирующая прямая описывается неявным уравнением $0.316x + 0.949y - 1.897 = 0$ или параметрическими координатами $x(t) = 1.8 - 0.949t$ и $y(t) = 1.4 + 0.316t$. Алгебраические расстояния от точек до этой прямой равны $d_1 = 0.316$, $d_2 = -0.316$, $d_3 = 0$, $d_4 = -0.632$ и $d_5 = 0.632$, а минимум функции Лагранжа составляет $\mathfrak{S}_{\min} = 1$. Для сравнения на рис. 1.8 построена штриховая прямая, соответствующая собственной паре $\{\lambda_1, N_1\}$. Расстояния от неё до заданных точек равны $d_1 = 0.949$, $d_2 = -0.316$, $d_3 = -1.265$, $d_4 = 0.632$ и $d_5 = 0$, а значение функции Лагранжа $\mathfrak{S}_{\max} = 3 > \mathfrak{S}_{\min}$ подтверждает правильность решения задачи МНК-аппроксимации. $\square$

Две прямые

Две прямые совпадают (рис. 1.9,а), если

$$\text{НФ:} \quad F_1 \times F_2 = O_3; \quad \text{ПФ:} \quad \left\{ \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = 0 \right\} \wedge \left\{ \begin{bmatrix} V_1 \\ o_2 - o_1 \end{bmatrix} = 0 \right\}. \quad (1.29a)$$

Две прямые параллельны (рис. 1.9,б), если

$$\text{НФ:} \quad \begin{bmatrix} N_1 \\ N_2 \end{bmatrix} = 0; \quad \text{ПФ:} \quad \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = 0; \quad \text{НФ и ПФ:} \quad N_1 \circ V_2 = N_2 \circ V_1 = 0. \quad (1.29б)$$

Расстояние между параллельными прямыми равно:

$$\text{НФ:} \quad d = |\text{pr}_{N_1}(o_2 - o_1)| = |(o_2 - o_1) \circ \overline{N_1}|;$$

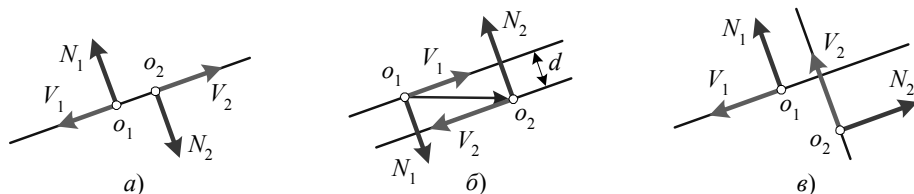


Рис. 1.9

ПФ:
$$d = \left\| \frac{o_2 - o_1}{\bar{V}_1} \right\| = |\text{or}_{V_1}(o_2 - o_1)| = |o_2 - o_1 - ((o_2 - o_1) \circ \bar{V}_1) \bar{V}_1|.$$

Две прямые ортогональны (рис. 1.9, в), если

НФ: $N_1 \circ N_2 = 0;$ ПФ: $V_1 \circ V_2 = 0.$ (1.30)

Пересечение двух прямых

Важнейшей задачей геометрии на плоскости является расчёт *точки с пересечения двух прямых* (рис. 1.10, а), который в зависимости от формы их задания выполняется следующим образом:

НФ: $\{c \circ N_1 = o_1 \circ N_1\} \wedge \{c \circ N_2 = o_2 \circ N_2\} \Rightarrow c = [o_1 \circ N_1 \quad o_2 \circ N_2] \begin{bmatrix} N_1^T & N_2^T \end{bmatrix}^{-1};$ (1.31а)

ПФ: $o_1 + V_1 t_1 = o_2 + V_2 t_2 \Rightarrow [t_1 \quad t_2] = (o_2 - o_1) \begin{bmatrix} V_1 \\ -V_2 \end{bmatrix}^{-1} \Rightarrow c = o_1 + V_1 t_1.$ (1.31б)

Условия существования решений в (1.31) $|N_1^T \quad N_2^T| \neq 0$ и $|V_1^T \quad V_2^T| \neq 0$ равносильны *непараллельности прямых*.

Несмотря на двухэтапность программирования параметрического варианта задачи оба метода в (1.31) имеют одинаковую вычислительную сложность — 17 арифметических операций (+, −, · и /) без учёта пересылок и инвертирования чисел. Неравноценность методов обнаруживается при нахождении точки пересечения *частей* прямых — лучей или отрезков:

- **нормальная форма** прямой $\{o, N\}$ не содержит информации о расположении на ней отрезка ab или луча $a \rightarrow b \rightarrow \infty$, поэтому не может быть использована для оценки положения точки на отдельных частях прямой. Даже если известны, например, концевые точки отрезка $a = [a_x \quad a_y]$ и $b = [b_x \quad b_y]$, то принадлежность точки $c = [x \quad y]$ отрезку, включая его концы (рис. 1.10, б), определяется решением системы координатных неравенств

$$\{\min(a_x, b_x) \leq x \leq \max(a_x, b_x)\} \wedge \{\min(a_y, b_y) \leq y \leq \max(a_y, b_y)\}$$

ценой ещё восьми арифметических операций. Строгое же положение точки c между точками a и b , вопреки распространённому заблуждению, не всегда определяется решением аналогичной системы *строгих* неравенств: например, для вертикального отрезка с координатами $a_x = b_x = x$ неравенство $a_x < x < b_x$ несовместно, что вынуждает программиста придумывать более изощрённый тест;

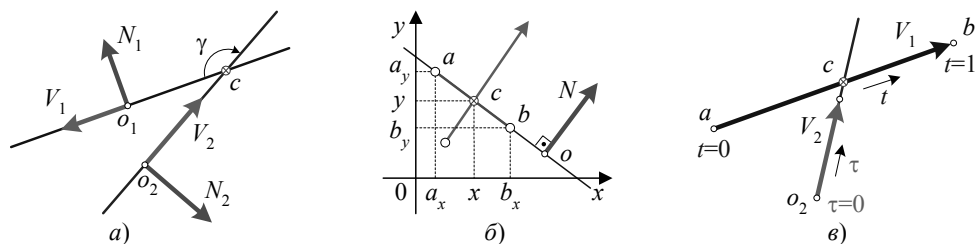


Рис. 1.10

• **параметрическая форма** части прямой $p(t)=o+Vt$ обязательно включает интервал изменения параметра t : для луча $t \in [0, \infty)$, а для отрезка $t \in [0, 1]$ при задании точки $a=o$ и вектора $V=b-a$ (рис. 1.10, в). При известном параметре t определение места точки $c=o+Vt$ на прямой линии элементарно: она принадлежит лучу при $t \geq 0$, отрезку при $0 \leq t \leq 1$ и внутренности отрезка при $0 < t < 1$.

Из девяти задач на пересечение двух прямых, каждая из которых может быть неограниченной, односторонне либо двусторонне ограниченной, наиболее востребованной является задача **пересечения двух отрезков** прямых. Составим **алгоритм расчёта точки q и параметров t и τ пересечения отрезков ab и cd** , возвращающий **признак расположения отрезков**, равный:

- -1, если прямые отрезков параллельны либо совпадают и точка пересечения не определена;
- 0, если непараллельные отрезки не пересекаются;
- 1, если отрезки касаются или пересекаются.

При непараллельности отрезков возвращаются параметры и точка пересечения прямых, несущих отрезки. Возврат значения $cross_segm=1$ означает, что эта точка к тому же является точкой пересечения отрезков.

✎ $cross_segm(ab, cd, t, \tau, q)$

$\{V=b-a, M=\begin{bmatrix} V \\ c-d \end{bmatrix};$ // матрица направляющих векторов

если $|M|=0$, то **возврат** -1; // тест параллельности прямых

$[t \ \tau]=(c-a)M^{-1};$ // параметры пересечения прямых

$q=a+Vt;$ // точка пересечения прямых

возврат $\{0 \leq t \leq 1\} \wedge \{0 \leq \tau \leq 1\};$ // тест пересечения либо касания отрезков

} □

Максимальная трудоёмкость алгоритма составляет 11 операций сложения/вычитания и 10 операций умножения/деления. Сравним эти затраты со вторым вариантом того же алгоритма, основанным на использовании неявной функции (1.19г) и вычислении по ней параметров t и τ .

✎ $cross_segm(ab, cd, t, \tau, q)$

$\{f_a=nf2(c, d, a), f_b=nf2(c, d, b);$ // НФ точек a и b относительно cd

$f_c=nf2(a, b, c), f_d=nf2(a, b, d);$ // НФ точек c и d относительно ab

если $f_a = f_b$, то **возврат** -1 ; // тест параллельности прямых
 $t = f_a / (f_a - f_b)$, $\tau = f_c / (f_c - f_d)$; // параметры пересечения прямых
 $q = a + (b - a)t$; // точка пересечения прямых
возврат $\{0 \leq t \leq 1\} \wedge \{0 \leq \tau \leq 1\}$; // тест пересечения либо касания отрезков
 } $\square$

Несмотря на кажущуюся простоту второго варианта, его трудоёмкость вдвое выше — 41 арифметическая операция: 27 сложений/вычитаний и 14 умножений/делений. Впредь имя *cross_segmt* будет ассоциироваться с более эффективным первым вариантом функции расчёта пересечения отрезков.

☞ Далее введём **функцию вычисления угла между векторами** V и W

$$\boxed{\text{ang}(V, W) = \text{if} \left(\left| \frac{V}{W} \right| \geq 0: 1, -1 \right) \cdot \arccos \left(\frac{V \circ W}{|V| \cdot |W|} \right) \in (-\pi, \pi]} \quad (1.32)$$

как угла **кратчайшего** вращения от первого вектора V ко второму W , в общем случае **не равного** разности углов $\arg(W) - \arg(V)$, вычисленных по (1.4). По свойству скалярного произведения векторов модуль угла $\angle VW$ равен $\arccos(\overline{V} \circ \overline{W}) \in [0, \pi]$, но при вращении от первого вектора ко второму по часовой стрелке определитель Δ матрицы, составленной из них в той же последовательности, становится отрицательным (почему?), как и знак угла.

При $\Delta = 0$ векторы V и W линейно зависимы. Угол между ними правильно вычисляется функцией $\arccos(\overline{V} \circ \overline{W})$, возвращающей 0 при $V \uparrow \uparrow W$ и π при $V \uparrow \downarrow W$. Для защиты алгоритма от вычислительных погрешностей, приводящих к теоретически невозможному значению $|\overline{V} \circ \overline{W}| > 1$, рекомендуется взять от полученного **комплексного** угла (1.32) действительную часть.

Таким образом, **угол пересечения двух прямых** как кратчайший угол между их нормальными либо направлениями (рис. 1.10, а) находится как

$$\gamma = \text{ang}(N_1, N_2) \quad \text{или} \quad \gamma = \text{ang}(V_1, V_2). \quad (1.33)$$

Равенство углов в (1.33) имеет место лишь при условии одинаковой (либо правой, либо левой) ориентации обоих векторов нормалей относительно направляющих векторов.

Пучок прямых и биссектриса угла

Модели пучка прямых, заключённых между двумя **непараллельными** прямыми (рис. 1.11, а), выводятся методом линейной интерполяции:

$$\text{НФ:} \quad \begin{cases} f_1(p) = \tilde{p} \circ F_1, \\ f_2(p) = \tilde{p} \circ F_2 \end{cases} \Rightarrow \begin{cases} f_{\lambda+1}(p) = \text{lin}(f_1(p), f_2(p), \lambda), \\ F_{\lambda+1} = \text{lin}(F_1, F_2, \lambda); \end{cases} \quad (1.34a)$$

$$\text{ПФ:} \quad \begin{cases} p_1(t) = o_1 + V_1 t, \\ p_2(t) = o_2 + V_2 t \end{cases} \stackrel{(1.31б)}{\Rightarrow} c \Rightarrow \begin{cases} p_{\lambda+1}(t) = c + V_{\lambda+1} t, \\ V_{\lambda+1} = \text{lin}(V_1, V_2, \lambda). \end{cases} \quad (1.34б)$$

Изменение параметра пучка в интервале $0 \leq \lambda \leq 1$ даёт такие промежуточные прямые, что вращение векторов $N_{\lambda+1}$ и $V_{\lambda+1}$ происходит по **кратчайшим** углам от начальных положений N_1 и V_1 до конечных положений N_2 и V_2 . В (1.34б) зависимость $p_{\lambda+1}(t)$ с вектором $V_{\lambda+1} = (1-\lambda)V_1 + \lambda V_2$ является параметрической с параметром λ моделью прямой линии $p_1 + (p_2 - p_1)\lambda$, проходящей через точки исходных прямых $p_1 = c + V_1 t$ и $p_2 = c + V_2 t$ при фиксированном значении t .

На основе этой трактовки несложно и без вычисления углов решается задача построения **дуги окружности** радиуса r в створе **неразвёрнутого** угла между векторами V_1 и V_2 , выходящими из точки c (рис. 1.11,б). Точки дуги $d_{\lambda+1}$, соответствующие параметру $\lambda \in [0, 1]$, вычисляются по формуле

$$d_{\lambda+1} = c + r \bar{V}_{\lambda+1}, \quad (1.35)$$

в которой нормируемый вектор $V_{\lambda+1}$ находится по (1.34б). Для изображения дуги в виде ломаной линии, состоящей из m отрезков между точками d_1 и d_2 , нужно разбить диапазон изменения параметра λ на m равных интервалов (например, дуга на рис. 1.11,б построена при $m=5$), сформировать набор значений $\lambda = i/m \quad \forall i=0, m$, рассчитать векторы $V_{\lambda+1}$, точки дуги $d_{\lambda+1}$ и соединить последние отрезками прямых. Полученная таким способом дуга имеет **неравномерный** угловой шаг, максимальный в середине и минимальный на её краях.

Векторы **биссектрисы угла** между двумя прямыми получаются по моделям (1.34) с параметром $\lambda=0.5$ только при $|N_1|=|N_2|$ или $|V_1|=|V_2|$, когда треугольник с вершиной в точке c и сторонами V_1 и V_2 является равнобедренным, а его биссектриса совпадает с медианой. В противном случае можно искусственно создать равнобедренный треугольник со сторонами $|V_2|V_1$ и $|V_1|V_2$ равной длины $|V_1| \cdot |V_2|$. В результате параметры биссектрисы вычисляются по формулам

$$F_{\text{бис}} = |N_2|F_1 + |N_1|F_2, \quad p_{\text{бис}}(t) = c + V_{\text{бис}} t, \quad V_{\text{бис}} = |V_2|V_1 + |V_1|V_2. \quad (1.36a)$$

Уравнять длины складываемых векторов можно также путём их нормировки, а вектор биссектрисы **неразвёрнутого** угла $\angle abc$ вычислять как диагональ ромба с вершиной в точке b и сторонами $V = a - b$ и $W = c - b$ (рис. 1.11,з):

$$B = \overline{a - b} + \overline{c - b}. \quad (1.36б)$$

Биссектрисы бывают нужны, например, в задаче построения **окружности, вписанной в треугольник** (рис. 1.11,в). Как известно, центр такой окружности лежит в точке пересечения любых двух биссектрис **внутренних** углов тре-

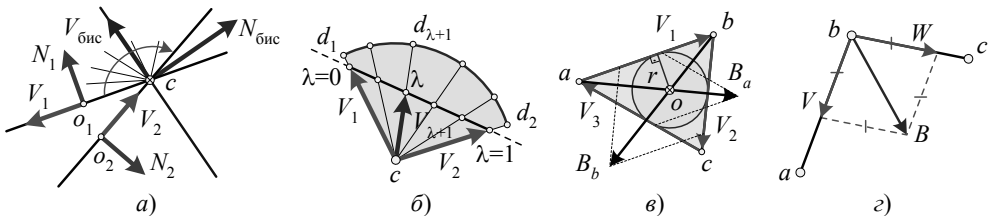


Рис. 1.11

угольника. При построении биссектрисы внутреннего угла следует задавать оба вектора сторон выходящими из вершины этого угла, что и сделано в (1.36б). При несоблюдении этого правила в (1.36а) будет получена биссектриса внешнего угла треугольника, а окружность окажется внеписанной.

Пусть заданы вершины треугольника a, b и c . Вычислив векторы сторон $V_1=b-a$, $V_2=c-b$ и $V_3=a-c$, найдём биссектрисы двух внутренних углов $B_a=\bar{V}_1-\bar{V}_3$ и $B_b=\bar{V}_2-\bar{V}_1$. Тогда центр вписанной окружности o лежит на пересечении параметрических прямых $\{a, B_a\}$, $\{b, B_b\}$ и находится по (1.31б), а её радиус r вычисляется по (1.24б) как расстояние от точки o до прямой $\{a, V_1\}$.

1.1.3. Тесты свойств графических элементов на плоскости

Изначением **теста** — ответить на поставленный вопрос ценой **минимальных** вычислительных затрат. Расчётная задача требует большего объёма вычислений, например, точек пересечения объектов. Похожий тест интересует лишь факт наличия точек пересечения, а не их расположение. Изучаемые здесь тесты основаны на **свойстве сепарабельности прямой** $f(p)=0$ разделять плоскость на два подпространства с противоположными знаками функции $f(p)$.

□ Тест расположения точки p относительно прямой $f(p)=0$ проверяет знак числа $f(p)$, для вычисления которого в зависимости от способа задания прямой в п. 1.1.1 используется соответствующая неявная функция nfF , nfN , nfV или $nf2$, определённая в (1.19).

Все точки q , находящиеся справа от прямой, т. е. со стороны её нормали, дают каждой из этих функций значения $nf*>0$, а левые точки дают $nf*<0$. Для доказательства данного факта подставим, например, в функцию (1.19б) точку $q=o+Vt+\lambda N$, **правую** при $\lambda>0$ и векторе $V=Lrot(N)\perp N$ (рис. 1.12):

$$nfN(o, N, q) = (Vt + \lambda N) \circ N = (V \circ N)t + \lambda(N \circ N) = 0 \cdot t + \lambda|N|^2 > 0$$

Параметры $\{t, \lambda\}$ есть координаты точки q в системе координат $\{o, V, N\}$. Так как $f(q)$ не зависит от координаты t , отсчитываемой по вектору V , то все **линии постоянных уровней** $f(p)=\text{const}$ проходят параллельно прямой $f(p)=0$ и также являются прямыми. Модуль координаты λ пропорционален расстоянию $d=|\lambda|N|$ от точки до прямой, поэтому

$$d = \frac{|nfF(F, q)|}{A^2 + B^2} = \frac{|nfN(o, N, q)|}{|N|} = \frac{|nfV(o, V, q)|}{|V|} = \frac{|nf2(a, b, q)|}{|b-a|}.$$

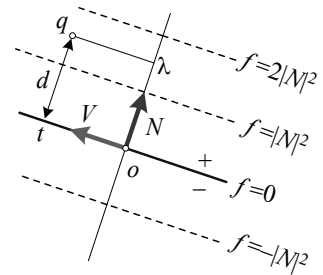
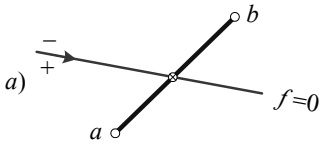


Рис. 1.12

Простая в расчёте функция $f(p)$, реализуемая одной из функций-синонимов (1.19), идеально подходит для сортировки точек относительно прямой на правые, левые и на ней лежащие. Тест расположения точки относительно прямой применяется в алгоритмах, связанных с обработкой точек, вершин полигонов и во многих других задачах вычислительной геометрии.

□ **Тест пересечения отрезка ab с прямой линией $f(p)=0$:**



$$f(a)f(b) < 0. \quad (1.37a)$$

Прямая **пересекает** отрезок, если значения неявной функции $f(p)$ на его концах **разнозначны**, т. е. точки a и b лежат по разные стороны прямой (рис. 1.13,а).

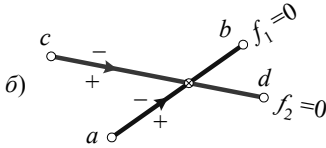


Рис. 1.13

□ **Тест пересечения отрезков ab и cd :**

$$\{f_1(c)f_1(d) < 0\} \wedge \{f_2(a)f_2(b) < 0\}. \quad (1.37б)$$

Отрезки **пересекаются**, если неявные функции этих прямых $f_1(p)$ и $f_2(p)$ принимают **разнозначные значения на концах отрезков**,

т. е. точки c и d лежат по разные стороны от прямой отрезка ab , а точки a и b лежат по разные стороны от прямой отрезка cd (рис. 1.13,б).

□ **Тест пересечения прямой линии $f(p)=0$ с полигоном $P=p_1p_2\dots p_np_1$, имеющим произвольное направление обхода вершин, выполняется с помощью следующей функции:**

$$cross2(f, P) = \{\exists 1 \leq j < i \leq n: f(p_i)f(p_j) < 0\}. \quad (1.38)$$

Прямая **пересекает** полигон, если существует хотя бы одна пара вершин $\{p_i, p_j\}$, лежащих от неё по разные стороны (рис. 1.14,а).

Обсудим некоторые подходы к реализации теста в виде конкретного алгоритма, стремясь получить его наиболее эффективным в смысле минимальной трудоёмкости. Следует ясно понимать, что нужно сравнивать положение относительно прямой линии не только **смежных** вершин полигона p_i и p_{i+1} , а **всех** различных пар вершин $\{p_i, p_j\}$, т. к. прямая может, пересекая полигон только в вершинах, не пересечь ни одного ребра (рис. 1.14,б). Свойство сепарабельности (1.37а) позволяет по разным знакам чисел $f(p_i)$ и $f(p_j)$ обнаружить вершины p_i и p_j по разные стороны от пересекающей полигон прямой. Если же нет ни одной пары разносторонних вершин, то прямая с полигоном не пересекается.

Общее число различных пар из n точек равно $n(n+1)/2$. Если решать задачу алгоритмизации теста (1.38) «в лоб» — с двумя вычислениями $f(p)$ для каждой пары вершин, — то при больших значениях n (например, когда некоторая замкнутая кривая линия аппроксимируется полигоном с числом сторон порядка нескольких тысяч) трудоёмкость теста будет пропорциональна n^2 .

Достижение линейной сложности от числа сторон полигона возможно в алгоритме теста (рис. 1.14,в), использующем три флажка, сигнализирующих о наличии вершин полигона слева (флажок l), справа (флажок r) и точно на прямой (флажок e). Вначале все флажки сбрасываем в нуль. В цикле $i=\overline{1, n}$ вычисляем **скалярную** переменную $s=f(p_i)$ и устанавливаем один из флажков: $l=1$ при $s < 0$, $e=1$ при $s=0$ или $r=1$ при $s > 0$. И тут же по условию $l \cdot r \neq 0$ проверяем наличие **любой** пары вершин полигона по разные стороны от прямой. Если это

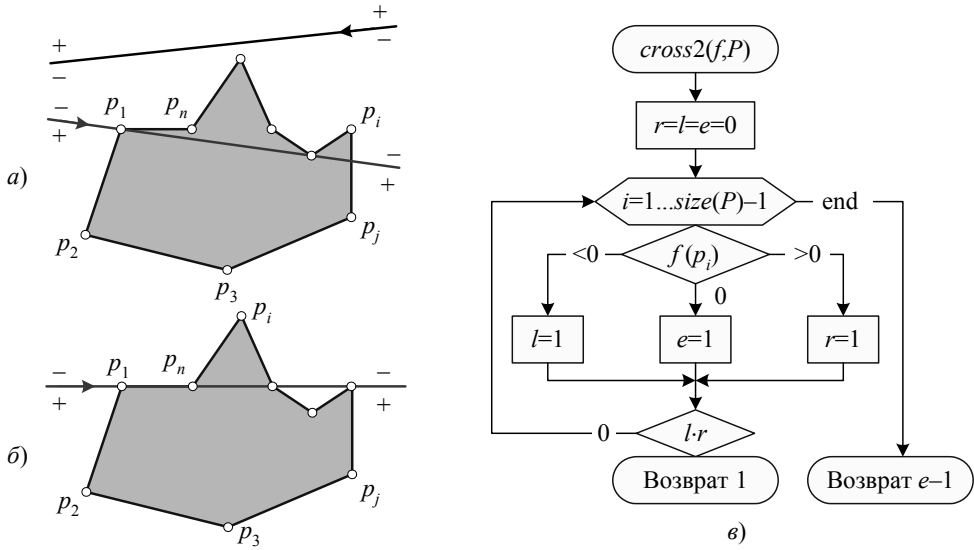


Рис. 1.14

условие выполняется, то тест немедленно завершается, возвращая 1 как признак **пересечения** прямой с полигоном. После нормального окончания цикла обхода вершин алгоритм возвращает значение $e-1$, равное 0 при касании или -1 в отсутствие пересечения прямой с полигоном.

□ **Тест выпуклости полигона** $P = p_1 p_2 \dots p_n p_1$:

$$conv2(P) = \{ \text{sgn}(nf2(p_{i-1}, p_i, p_{i+1})) = \text{const} \forall i = \overline{1, n} \}, \text{ где } p_0 \equiv p_n, p_{n+1} \equiv p_1. \quad (1.39)$$

У выпуклого (convex) полигона все углы $\angle p_{i-1} p_i p_{i+1}$ **одного знака**. При обходе выпуклого полигона по замкнутому контуру в **произвольном** направлении каждая вершина p_{i+1} расположена относительно ребра $p_{i-1} p_i$ одинаково для всех значений i : слева при положительном направлении обхода и справа при отрицательном (рис. 1.15, а). Свойство сепарабельности прямой позволяет об-

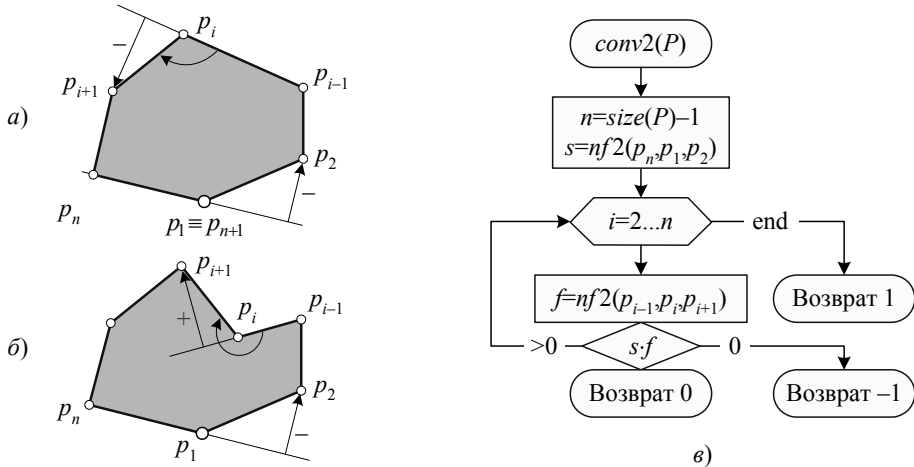


Рис. 1.15

наружить вершину p_i , в которой изменилось направление обхода **невыпуклого** полигона, по инверсии знака числа $nf2(p_{i-1}, p_i, p_{i+1})$, как показано на рис. 1.15,б.

Блок-схема алгоритма теста приведена на рис. 1.15,в. Сначала находится значение $s = nf2(p_n, p_1, p_2)$, фиксирующее направление поворота в вершине p_1 . Затем в цикле $i=2, n$ вычисляемые значения $f = nf2(p_{i-1}, p_i, p_{i+1})$ сравниваются с числом s . При $s \cdot f < 0$ обнаружена инверсия направления **локального** поворота в вершине p_i , по которой тест завершается с возвратом числа -1 — признака невыпуклости полигона. Если обход полигона будет завершён со всеми произведениями $s \cdot f > 0$, то возвращается 1 — признак его выпуклости. В сравнении с (1.39) алгоритм блок-схемы может по **нулевым** значениям $s=0$ либо $f=0$ обнаружить выпуклый полигон с **развёрнутым** углом при вершине p_1 либо p_i .

Функция $conv2$ возвращает 1 для выпуклого, 0 — для невыпуклого и -1 — для полигона, имеющего хотя бы один развёрнутый угол.

□ **Тест самопересечения полигона** $P = p_1 p_2 \dots p_n p_1$:

$$self_test(P) = \{ \exists i, j: \{ f_{ij} \neq f_{i,j+1} \} \wedge \{ f_{ij} f_{i,j+1} \leq 0 \} \wedge \{ f_{ji} f_{j,i+1} \leq 0 \} \}, \quad (1.40a)$$

где $f_{ij} = nf2(p_i, p_{i+1}, p_j)$ — значение неявной функции прямой ребра $p_i p_{i+1}$ в вершине p_j . Блок-схема функции $self_test(P)$, возвращающей 1 при самопересечении либо самокасании полигона P , приведена на рис. 1.16,а.

Полигон с числом сторон $n > 3$ называется **самопересекающимся**, если у него есть хотя бы одна пара пересекающихся рёбер $p_i p_{i+1}$ и $p_j p_{j+1}$ (рис. 1.16,б).

Два отрезка пересекаются, если концы одного лежат по разные стороны от прямой другого, и наоборот. Согласно (1.37б) это обнаруживается по **разно-**

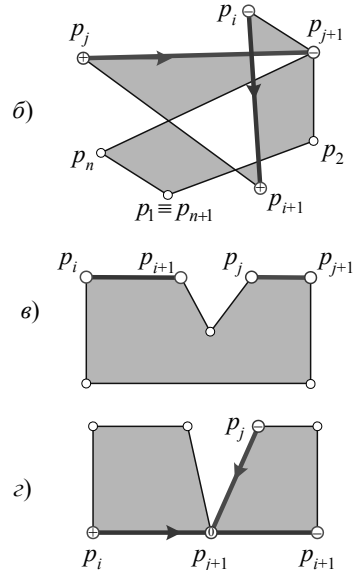
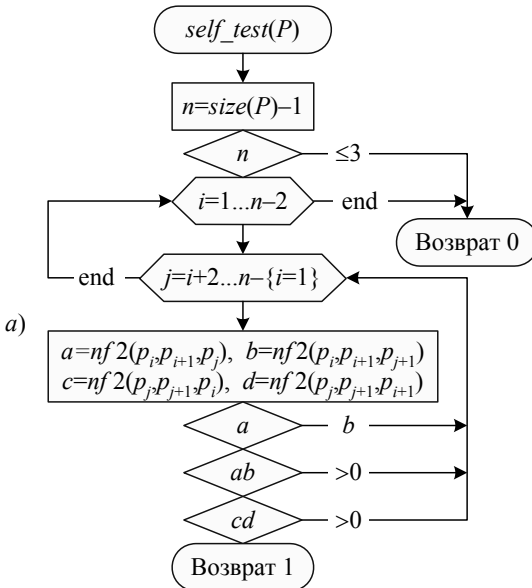


Рис. 1.16

значным числам в парах $\{f_{ij}, f_{i,j+1}\}$ и $\{f_{ji}, f_{j,i+1}\}$. При касании рёбер полигона в (1.40а) срабатывают знаки « $\Rightarrow$ ». Из проверки исключаются параллельные, в том числе коллинеарные (рис. 1.16,в), рёбра полигона, для которых $f_{ij} = f_{i,j+1}$.

Для исключения повторных и недопустимых проверок интервалы изменения индексов i и j выбираются так, чтобы были **однократно** протестированы **все** пары **несмежных** рёбер полигона:

- индекс i изменяется от 1 до $n-2$;
- индекс j изменяется от $i+2$ до $n-1$ при $i=1$ и от $i+2$ до n при $i>1$.

Для обнаружения только **самокасания** несмежных рёбер полигона (рис. 1.16,з) используемое в (1.40а) условие заменяется логической формулой

$$\{f_{ij} \neq f_{i,j+1}\} \wedge \{f_{ij} f_{i,j+1} \leq 0\} \wedge \{f_{ji} f_{j,i+1} = 0\} \vee \{f_{ij} f_{i,j+1} = 0\} \wedge \{f_{ji} f_{j,i+1} \leq 0\}. \quad (1.40б)$$

□ **Тест минимальности полигона** $P = p_1 p_2 \dots p_n p_1$:

$$\min2_test(P) = \{nf2(p_{i-1}, p_i, p_{i+1}) \neq 0 \forall i = \overline{1, n}\}, \text{ где } p_0 \equiv p_n, p_{n+1} \equiv p_1. \quad (1.41)$$

У **минимального** полигона все вершины p_i **простые**, а углы $\angle p_{i-1} p_i p_{i+1}$ — **неразвёрнутые**. Другими словами, среди **смежных** вершин полигона отсутствуют, во-первых, **кратные** $p_i = p_{i+1}$ (рис. 1.17,а), а во-вторых, — **коллинеарные** $\{p_{i-1}, p_i, p_{i+1}\}$ (рис. 1.17,б) точки.

Неминимальные полигоны не рекомендуется использовать в силу ряда причин как терминологического, так и вычислительного характера:

- совпадение вершин $p_i = p_{i+1}$ делает одну из них явно лишней;
- обнуление вектора направления ребра $V_i = p_{i+1} - p_i$ и его нормали даёт сбой или ошибочные результаты тестов и алгоритмов, полагающих векторы направлений по умолчанию ненулевыми и потому не проверяющих кратность вершин;
- коллинеарность трёх смежных вершин $\{p_{i-1}, p_i, p_{i+1}\}$ делает точку p_i фиктивной вершиной полигона, а ребро $p_i p_{i+1}$ — продолжением ребра $p_{i-1} p_i$, в результате чего их можно заменить одним ребром $p_{i-1} p_{i+1}$;

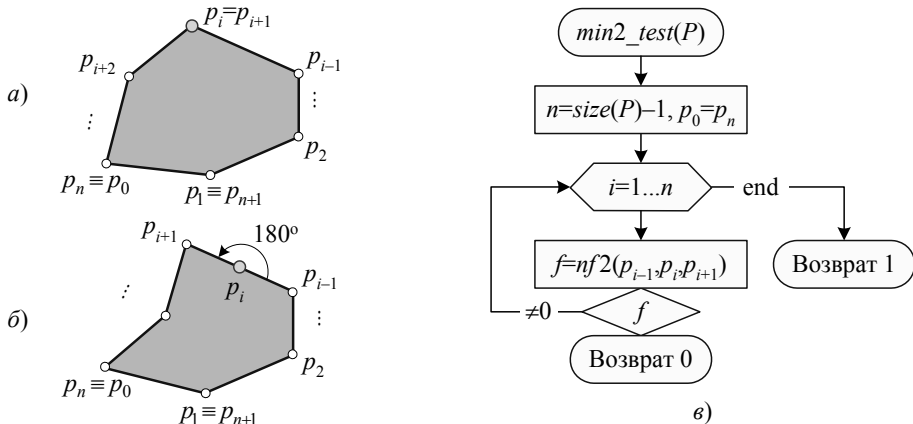


Рис. 1.17

После инициализации индекса $m=1$ в цикле $i=\overline{2, size(P)}$ проверяется способность точки p_i быть вершиной минимального полигона. При нулевом векторе $V_i = p_i - p_m$ кратная точка $p_i = p_m$ пропускается, а некратные смежные вершины обрабатываются следующим образом:

- при $m=1$ точка p_i копируется в p_{++m} с инкрементированным индексом m ;
- для всех следующих значений $m \geq 2$ анализируется расположение точки p_i относительно ребра $p_{m-1}p_m$. Если точки p_{m-1} , p_m и p_i не коллинеарны, т. е. удовлетворяют условию $nf2(p_{m-1}, p_m, p_i) \neq 0$, то точка p_i становится новой вершиной минимального полигона p_{++m} . Коллинеарная же точка p_i может находиться относительно ребра $p_{m-1}p_m$ с направляющим вектором $W = p_m - p_{m-1}$ в одном из трёх положений:

◊ при однонаправленных векторах $V \uparrow \uparrow W$, что проверяется неравенством $V \circ W > 0$, **передняя** точка p_i заменяет лишнюю точку p_m (рис. 1.18,б);

◊ при $V \uparrow \downarrow W$ и $(p_i - p_{m-1}) \circ W \leq 0$ лишняя точка p_{m-1} заменяется на p_m , а точка p_m — на **заднюю** точку p_i (рис. 1.18,в);

◊ при $V \uparrow \downarrow W$ и $(p_i - p_{m-1}) \circ W > 0$ пропускается лишняя **внутренняя** точка p_i (рис. 1.18,г).

По завершении цикла в начале массива P накоплены m вершин минимального полигона $p_1 \dots p_{m-1}p_m$ с $p_m = p_1$, который возвращается при неколлинеарности точек p_{m-1} , p_1 и p_2 , проверяемой по условию $nf2(p_{m-1}, p_1, p_2) \neq 0$. В противном случае возвращается полигон $p_2 \dots p_{m-1}p_2$ без лишней вершины p_1 .

1.1.4. Тесты локализации объектов на плоскости

Важное место в компьютерной геометрии занимают тесты взаимного расположения графических элементов. Рассмотрим сначала группу тестов локализации **точки** $q = [x \ y]$ относительно полигона $P = p_1 p_2 \dots p_n p_1$.

□ Выпуклый тест

$$conv2_test(P, q) = \begin{cases} -1, & \text{если } \text{sgn}(f_i) = \text{const } \forall i = \overline{1, n}; \\ 1, & \text{если } \exists 1 \leq j < i \leq n: f_i f_j < 0; \\ 0, & \text{если не } \pm 1 \end{cases} \quad (1.42)$$

определяет **внутреннее** положение точки $q_{\text{вну}}$ относительно **выпуклого** полигона при **одинаковых** знаках чисел $f_i = nf2(p_i, p_{i+1}, q_{\text{вну}})$, вычисленных функцией (1.19г) для **всех** рёбер $p_i p_{i+1}$ (рис. 1.19,а). Любую **внешнюю** точку $q_{\text{вне}}$ тест обнаруживает по **разным** знакам чисел f_i и f_j , соответствующих рёбрам $p_i p_{i+1}$ и $p_j p_{j+1}$ (рис. 1.19,б). Как и в тесте *cross2*, здесь также нужно сравнивать положение точки q не только относительно смежных рёбер $p_i p_{i+1}$ и $p_{i+1} p_{i+2}$, а **всех** различных пар рёбер $p_i p_{i+1}$ и $p_j p_{j+1}$. Наконец, все точки, идентифицированные как не внутренние и не внешние, считаются **граничными** точками $q_{\text{гр}}$.

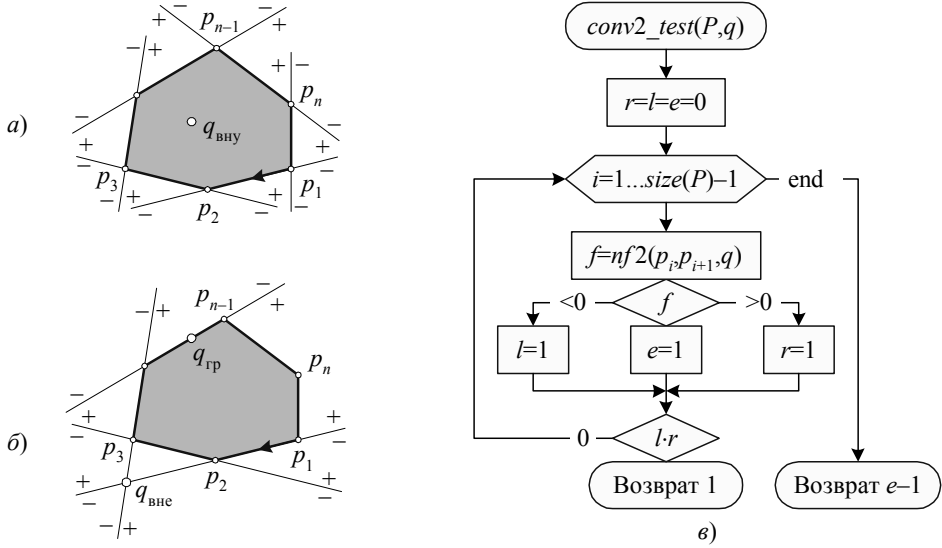


Рис. 1.19

Тесты *cross2* и *conv2_test* имеют много общего и фактически являются двойственными формами общего метода обработки точек и прямых на основе свойства сепарабельности прямой. Схожесть тестов наглядно иллюстрируется блок-схемой алгоритма *conv2_test*, изображённой на рис. 1.19,в.

В соответствии со знаком числа 1, возвращаемым тестом для внешних точек, назовём внешнее подпространство полигона **положительным**. Внутреннее пространство полигона будем считать **отрицательным** согласно знаку возвращаемого значения -1 . Все точки $q_{\text{гр}}$ на границе полигона (в вершинах или на рёбрах) дают возвращаемое значение $\text{conv2_test}(P, q_{\text{гр}}) = 0$.

Если тест (1.42) проводится в экранной системе координат, то все расчёты являются **целочисленными** и максимально быстрыми.

Среди выпуклых полигонов особое место занимают треугольники и параллелограммы. Из этих простейших фигур часто формируются полигоны сложной формы, в том числе и не плоские, что облегчает решение с ними различных геометрических и физических задач.

Рассмотрим **вспомогательную задачу** вычисления методом Грама [10, 12] **координат точки q в косоугольной системе координат $\{o, V, W\}$** с началом в точке o и базисными векторами V и W (рис. 1.20,а). Отметим, что здесь **отсутствуют ограничения на размерность пространства**: базисные векторы могут быть заданы как на плоскости, так и в трёхмерном пространстве. Записав аналогично (1.1) разложение вектора $q - o = Vt + W\tau$ и умножив его скалярно на векторы V и W , составим систему двух линейных уравнений с двумя неизвестными координатами t и τ :

$$\begin{cases} (q-o) \circ V = (V \circ V)t + (W \circ V)\tau, \\ (q-o) \circ W = (V \circ W)t + (W \circ W)\tau. \end{cases} \quad (1.43a)$$

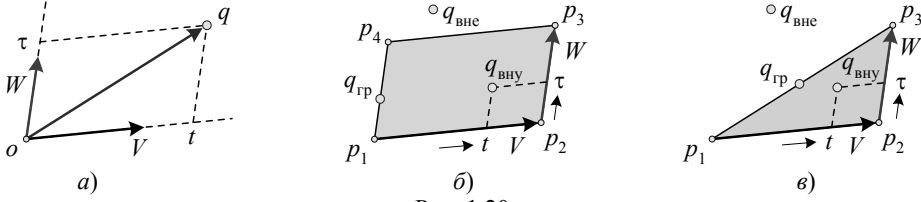


Рис. 1.20

Решение этой системы

$$[t \ \tau] = [(q-o) \circ V \ (q-o) \circ W] \begin{bmatrix} V \circ V & V \circ W \\ W \circ V & W \circ W \end{bmatrix}^{-1} \quad (1.43б)$$

существует при невырожденности обращаемой матрицы, определитель которой

$$\Delta = (V \circ V)(W \circ W) - (V \circ W)(W \circ V) = |V|^2 |W|^2 \sin^2(\angle VW)$$

не равен нулю при **линейной независимости базисных векторов V и W** .

В $\mathbb{R}^2$ (на плоскости xy) решение (1.43б) упрощается до следующего вида:

$$[t \ \tau] = (q-o) \begin{bmatrix} V^T & W^T \end{bmatrix} \left(\begin{bmatrix} V \\ W \end{bmatrix} \begin{bmatrix} V^T & W^T \end{bmatrix} \right)^{-1} = (q-o) \begin{bmatrix} V \\ W \end{bmatrix}^{-1}. \quad (1.43в)$$

Построим на трёх первых вершинах параллелограмма $p_1 p_2 p_3 p_4$ (рис. 1.20,б) либо треугольника $p_1 p_2 p_3$ (рис. 1.20,в) базисные векторы $V = p_2 - p_1$ и $W = p_3 - p_1$, после чего вычислим по (1.43б) при $o = p_1$ координаты тестируемой точки q . Элементарные соображения подсказывают, что:

- точка q лежит строго внутри **параллелограмма** при выполнении условий

$$\{0 < t < 1\} \wedge \{0 < \tau < 1\}, \quad (1.44а)$$

на его границе при нестрогих неравенствах $\{0 \leq t \leq 1\} \wedge \{0 \leq \tau \leq 1\}$ и снаружи при остальных значениях косоугольных координат $\{t < 0\} \vee \{t > 1\} \vee \{\tau < 0\} \vee \{\tau > 1\}$;

- точка q лежит строго внутри **треугольника** при выполнении неравенств

$$0 < \tau < t < 1, \quad (1.44б)$$

на его границе при нестрогих неравенствах $0 \leq \tau \leq t \leq 1$ и снаружи при остальных значениях координат $\{\tau < 0\} \vee \{\tau < t\} \vee \{t > 1\}$.

□ Габаритный тест

$$gab2_test(P, q) = \{x < p_{x\min}\} \vee \{x > p_{x\max}\} \vee \{y < p_{y\min}\} \vee \{y > p_{y\max}\} \quad (1.45а)$$

определяет по возвращаемому значению $gab2_test=1$ **гарантированную непринадлежность** точки q **произвольному полигону** P путём сравнения её координат с **габаритами** полигона — минимальными и максимальными координатами его вершин $p_i \ \forall i = \overline{1, n}$:

$$p_{x\min} = \min\{p_{ix}\}, \ p_{x\max} = \max\{p_{ix}\}, \ p_{y\min} = \min\{p_{iy}\}, \ p_{y\max} = \max\{p_{iy}\}. \quad (1.45б)$$

Полностью габаритный тест задачу локализации не решает, возвращая значение 1 только для тех внешних точек $q_{\text{вне}}$, которые расположены вне габ-

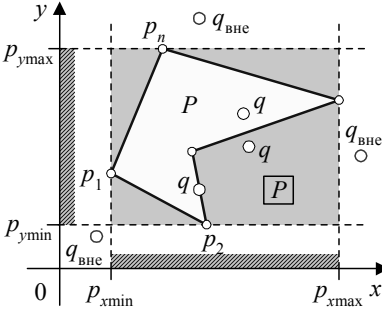


Рис. 1.21

ритного прямоугольника $[P]$ (рис. 1.21). Результат $gab2_test(P, q) = 0$ означает, что точка находится внутри габаритного прямоугольника, но не может быть однозначно идентифицирована как внешняя, внутренняя либо граничная. Тем не менее благодаря своей простоте габаритный тест применяется во многих алгоритмах для быстрого обнаружения заведомо непересекающихся геометрических объектов, что освобождает от необходимости решения трудоёмких задач отсечения.

□ **Угловой тест** основан на вычислении и анализе *алгебраической* суммы углов $\delta_i = \angle(V_i, V_{i+1})$ между смежными векторами $V_i = p_i - q$, соединяющими точку q с вершинами p_i , при обходе *произвольного* полигона P по замкнутому контуру согласно нумерации его вершин. Тест основан на следующем опыте: наблюдатель, просматривающий вершины полигона из *внутренней* точки $q_{вну}$, совершает вокруг себя полный оборот (рис. 1.22, а), а из любой *внешней* точки $q_{вне}$ — ни одного оборота (рис. 1.22, б).

Воспользуемся для вычисления углов между векторами функцией ang , определённой в (1.32) и возвращающей радианную меру угла. Тогда при всех ненулевых векторах V_i **радианный вариант углового теста** примет вид

$$\left| \sum_{i=1}^n ang(V_i, V_{i+1}) \right| = \begin{cases} 0 < \pi \Rightarrow q_{вне} \notin P, \\ 2\pi > \pi \Rightarrow q_{вну} \in P. \end{cases} \quad (1.46)$$

Ввиду неизбежных погрешностей суммирования углов, вычисляемых *приближённой* функцией ang , в (1.46) нельзя сравнивать результат ни с 0, ни с 2π : он

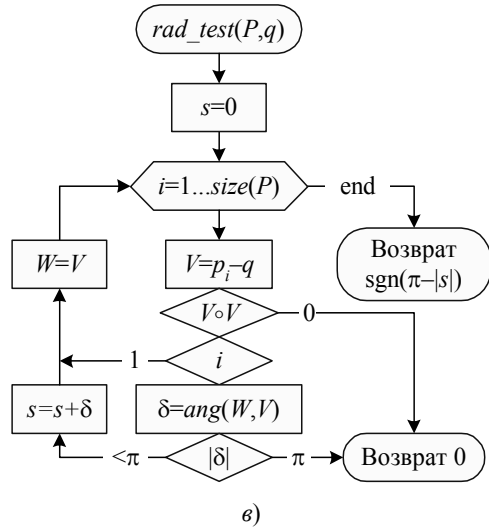
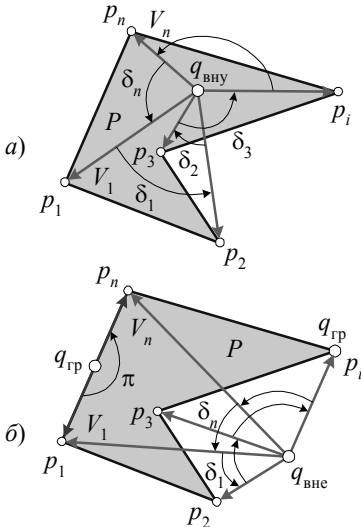


Рис. 1.22

всегда будет отличаться от этих чисел. Предлагается взять пороговое значение в виде числа π : оно ни при каких погрешностях не даст неверного результата тестирования.

Граничная точка $q_{\text{гр}}$ обнаруживается по следующим признакам:

- если при расчёте векторов будет получен вектор нулевой длины $|V_i|=0$, то тестируемая точка совпадает с вершиной p_i ;
- если при расчёте углов δ_i будет получен развёрнутый угол с модулем $|\delta_i|=\pi$, то тестируемая точка лежит на ребре $p_i p_{i+1}$ (рис. 1.22,б).

Резюмируя сказанное, получим алгоритм $rad_test(P, q)$ радианного углового теста локализации точки q относительно произвольного полигона P (рис. 1.22,в). Поскольку для вычисления угла δ_i нужна лишь пара смежных векторов V_i и V_{i+1} , то при обходе вершин полигона можно вычислить текущий вектор V , а в конце цикла запомнить его как W для использования в следующей итерации, экономя тем самым память. Алгоритм возвращает 1 для внешних точек, -1 для внутренних и 0 для граничных точек.

Ввиду важности места, занимаемого угловым тестом в компьютерной графике, обсудим как явные, так и скрытые **недостатки** алгоритма rad_test :

- необходимо нормировать все ненулевые векторы V_i , что связано с операциями деления и вычисления квадратного корня, которые относятся к нецелочисленным операциям с **плавающей точкой**, выполняющимся гораздо медленнее арифметических операций с целыми числами;
- вычисление в (1.32) угла тригонометрической функцией $\arccos$ производится путём суммирования бесконечного степенного ряда до сходимости по точности, предусмотренной компилятором. Этот **итерационный** процесс с плавающей точкой также не отличается высоким быстродействием;
- для обнаружения граничной точки $q \in p_i p_{i+1}$ сравнением $|\delta_i|=\pi$ нужно число π максимальной точности, а не как обычно его задают $PI=3.1415926$. Если оно отсутствует в используемом языке программирования, то должно быть один раз **вычислено** с машинной точностью. Для этого можно воспользоваться абсолютно, но медленно сходящимся за $\lceil 1/8\varepsilon \rceil$ итераций степенным рядом

$$4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = \pi.$$

Модифицируем радианный угловой тест так, чтобы расчёт углов δ_i с высокой точностью стал не обязателен. Будем считать углы в **целочисленных октантах** — восьмь частях круга. Согласно неравенствам на рис. 1.23 сопоставим вектору $V=[x \ y]$ **октантный угол** $1 \leq v \leq 8$ и **функцию вычисления октантного угла** $oct(V)$, возвращающую целое число $v \in \{1, 2, \dots, 8\}$ при $|V| \neq 0$ и 0 при $V=[0 \ 0]$. Нулевое значение $v_i = oct(p_i - q)$

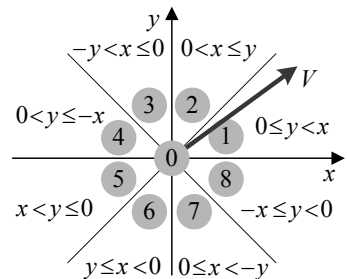


Рис. 1.23

означает расположение точки q на **границе** полигона, а именно в его вершине p_i , что ведёт к немедленному завершению теста с возвратом нуля.

При $q \neq p_i \ \forall i=\overline{1,n}$ найдём приращения октантных углов $\Delta_i = v_{i+1} - v_i$ на переходах $p_i \rightarrow p_{i+1}$. Для получения **кратчайших** вращений при октантных приращениях с $|\Delta_i| \geq 4$ необходима **коррекция** величин Δ_i :

- если $\Delta_i > 4$, то $\Delta_i = \Delta_i - 8$ (например, при переходе из первого октанта в восьмой $\Delta_i = -1$, а не $\Delta_i = 7$);
- если $\Delta_i < -4$, то $\Delta_i = \Delta_i + 8$ (например, при переходе из седьмого октанта во второй $\Delta_i = 3$, а не $\Delta_i = -5$);
- если $\Delta_i = \pm 4$ и $f = nf^2(p_i, p_{i+1}, q) \neq 0$, то $\Delta_i = -4\text{sgn}(f)$.

Последнюю коррекцию рассмотрим подробнее. Значение $\Delta_i = 4$ возможно в трёх случаях расположения тестируемой точки в окрестности ребра $p_i p_{i+1}$, в которых угол $\angle p_i q p_{i+1}$ близок к развёрнутому или равен ему:

- точка q находится точно на ребре $p_i p_{i+1}$ (**на границе** полигона P , рис. 1.24,а). Этот факт устанавливается проверкой $f=0$ и завершает тест возвратом граничного признака 0;
- при $f > 0$ точка q лежит справа от отрезка $p_i p_{i+1}$, а $\Delta_i = -4$ (рис. 1.24,б);
- при $f < 0$ точка q лежит слева от отрезка $p_i p_{i+1}$, а $\Delta_i = 4$ (рис. 1.24,в).

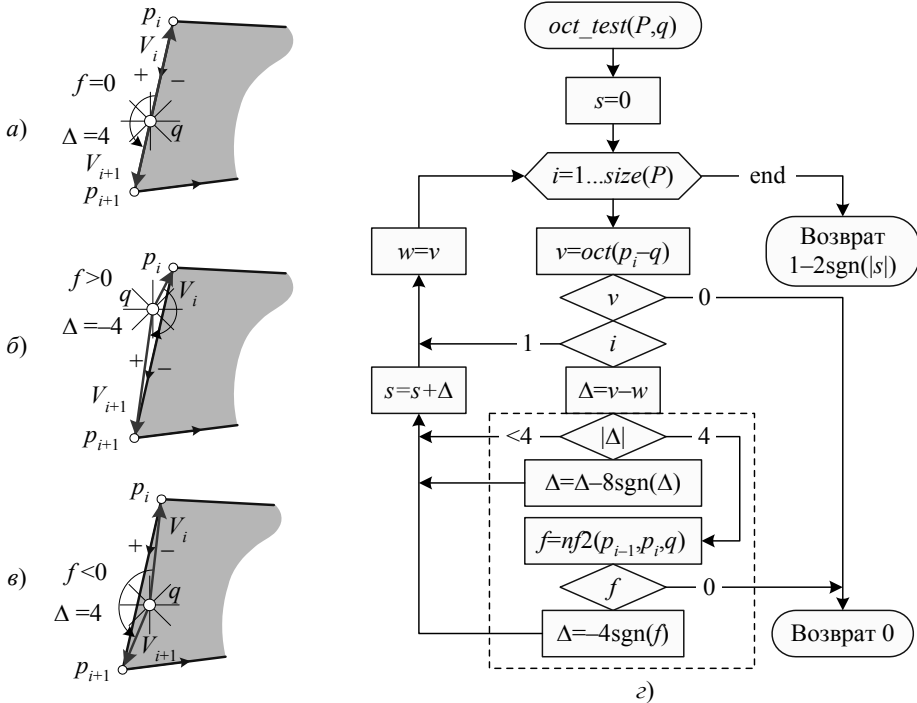


Рис. 1.24

Обозначим $oct_cor(a, b, q)$ — функцию вычисления и коррекции октантной меры угла $\angle aqb$. Тогда **целочисленный октантный тест**, аналогичный радианному тесту (1.46), принимает вид

$$\sum_{i=1}^n oct_cor(p_i, p_{i+1}, q) = \begin{cases} 0 \Rightarrow q \notin P, \\ \pm 8 \neq 0 \Rightarrow q \in P. \end{cases} \quad (1.47)$$

Точка q является **внешней** относительно полигона P , если суммарный октантный угол поворота вектора $V_i = p_i - q$ при обходе вершин по замкнутому контуру равен нулю. Точка лежит **внутри** полигона, если этот угол ненулевой. Наконец, точка q является **граничной**, если либо при $oct(p_i - q) = 0$ она совпадает с вершиной p_i , либо при $|\Delta_i| = 4$ и $nf2(p_i, p_{i+1}, q) = 0$ она лежит на ребре $p_i p_{i+1}$.

Обозначим $oct_test(P, q)$ — функцию октантного углового теста локализации точки q относительно плоского полигона $P = p_1 p_2 \dots p_n p_1$, возвращающую -1 для точки внутри полигона, 1 — вне полигона и 0 — на его границе. Блок-схема октантного теста построена на рис. 1.24, з. Пунктиром выделена часть алгоритма, выполняющая коррекцию приращения октантного угла.

Пример 1.3. Определить расположение трёх точек $q_1 = [4.5 \ 3]$, $q_2 = [2.5 \ 1]$ и $q_3 = [1.5 \ 1.5]$ относительно невыпуклого полигона с вершинами $p_1 = [1 \ 2]$, $p_2 = [2.5 \ 0.5]$, $p_3 = [2 \ 2.5]$, $p_4 = [4 \ 0.5]$, $p_5 = [5 \ 4]$, $p_6 = [3 \ 3]$ и $p_7 = [1.5 \ 4.5]$ (рис. 1.25).

Решение. Для первой тестируемой точки $q_1 = [x_1 \ y_1]$ вычислим октантные углы векторов $V_i = p_i - q_1$ и их приращения:

$$v_1=5, v_2=6, v_3=5, v_4=6, v_5=2, v_6=5, v_7=4, v_8=5;$$

$$\Delta_1=1, \Delta_2=-1, \Delta_3=1, \Delta_4=-4, \Delta_5=3, \Delta_6=-1, \Delta_7=1.$$

Без коррекции приращения угла на ребре $p_4 p_5$ сумма октантных приращений равна нулю, что не соответствует внутреннему положению точки. Подставив координаты точки q_1 в функцию прямой (1.19г) ребра $p_4 p_5$ $nf2(p_4, p_5, q_1) = 3.5x_1 - y_1 - 13.5 = -0.75 < 0$, получим скорректированное значение $\Delta_4=4$ и скорректированную сумму $s=8$. Согласно тесту (1.47) точка q_1 лежит внутри полигона.

Для второй точки $q_2 = [x_2 \ y_2]$ октантные углы и их приращения равны:

$$v_1=4, v_2=7, v_3=3, v_4=8, v_5=2, v_6=2, v_7=3, v_8=v_1=4;$$

$$\Delta_1=3, \Delta_2=-4, \Delta_3=5, \Delta_4=-3, \Delta_5=-6, \Delta_6=1, \Delta_7=1.$$

Значение функции прямой ребра $p_2 p_3$ в точке q_2 $nf2(p_2, p_3, q_2) = 2x_2 + 0.5y_2 - 5.25 = 7.75 > 0$ положительно, поэтому $\Delta_2=-4$, а сумма приращений октантных углов $s=0$. Из (1.47) следует, что точка q_2 является внешней.

Наконец, для точки $q_3 = [x_3 \ y_3]$ на первом ребре $p_1 p_2$ получим $\Delta_1=8-4=4$. Подставив координаты точки в функцию ребра $p_1 p_2$, по значению $nf2(p_1 p_2, q_3) = 1.5x_3 + 1.5y_3 - 4.5 = 0$ заключаем, что точка q_3 принадлежит границе полигона. $\square$

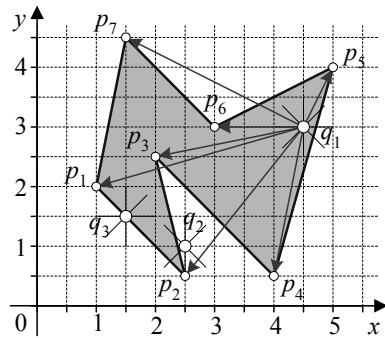


Рис. 1.25

Суммированием углов можно **алгоритмически** определить направление обхода вершин полигона. Установив **границную** точку $q_{\text{гp}} = 0.5(p_1 + p_n)$ в середине последнего ребра $p_n p_1$, вычислим алгебраическую сумму углов вращения векторов $V_i = p_i - q_{\text{гp}}$ при обходе вершин от p_1 до p_n (см. рис. 1.22,б). Если общее направление обхода вершин полигона против часовой стрелки, то этот суммарный угол равен π рад или $+4$ октанта. При обходе вершин по часовой стрелке (в отрицательном направлении) знак суммарного угла отрицателен.

Отсюда получаем два варианта **теста** $\text{dir_test}(P)$ **направления обхода вершин полигона** $P = p_1 p_2 \dots p_n p_1$, возвращающего знак направления (1 — против часовой стрелки либо -1 — по часовой стрелке):

- **радианный** тест направления обхода

$$\text{dir_test}(P) = \text{sgn} \left(\sum_{i=1}^{n-1} \text{ang}(p_i - q_{\text{гp}}, p_{i+1} - q_{\text{гp}}) \right); \quad (1.48a)$$

- **октантный** тест направления обхода

$$\text{dir_test}(P) = \text{sgn} \left(\sum_{i=1}^{n-1} \text{oct_cor}(p_i, p_{i+1}, q_{\text{гp}}) \right) \quad (1.48б)$$

эффективнее радианного благодаря целочисленности.

□ **Лучевой тест** локализации точки q относительно произвольного полигона $P = p_1 p_2 \dots p_n p_1$ заключается в выпускании из неё луча, $t \geq 0$, в **случайном** направлении $V = [\cos(\varphi) \sin(\varphi)]$ под углом $\varphi = \text{rnd}(2\pi)$ и подсчёте числа k его пересечений с рёбрами полигона $p_i p_{i+1} \forall i = \overline{1, n}$. Параметры пересечения луча с отрезком $p_i + (p_{i+1} - p_i)\tau$, $0 \leq \tau \leq 1$, **непараллельным** вектору V , находятся по (1.31б):

$$[t_i \ \tau_i] = (p_i - q) \left[\begin{array}{c} V \\ p_i - p_{i+1} \end{array} \right]^{-1}. \quad (1.49)$$

Анализ параметрических пар $\{t_i, \tau_i\}$ даёт следующие **критерии локализации** точки относительно полигона:

- точка является **внешней** $q_{\text{вне}}$, если число пар параметров $\{t_i > 0, 0 < \tau_i < 1\}$ **чётно** либо равно нулю (рис. 1.26,а);
- у **внутренней** точки $q_{\text{вну}}$ число таких пар **нечётно** (рис. 1.26,б);
- точка $q_{\text{гp}}$ лежит на **границе** полигона, если существует пара $\{t_i = 0, 0 \leq \tau_i \leq 1\}$, соответствующая началу луча на ребре $p_i p_{i+1}$ (рис. 1.26,в).

Вместо анализа чётности числа пересечений луча с рёбрами можно первоначально установить специальный флажок в состояние $f=1$, соответствующее внешней точке, а после каждого пересечения делать инверсию $f=-f$. Тогда по окончании обхода рёбер полигона флажок автоматически получит знак того полупространства, в котором находится неграницная точка q .

Особенности лучевого теста:

- неопределённость локализации точки q при прохождении выпущенного из неё луча точно через вершину p_i . Так, луч l с полигоном касается, но не пересе-

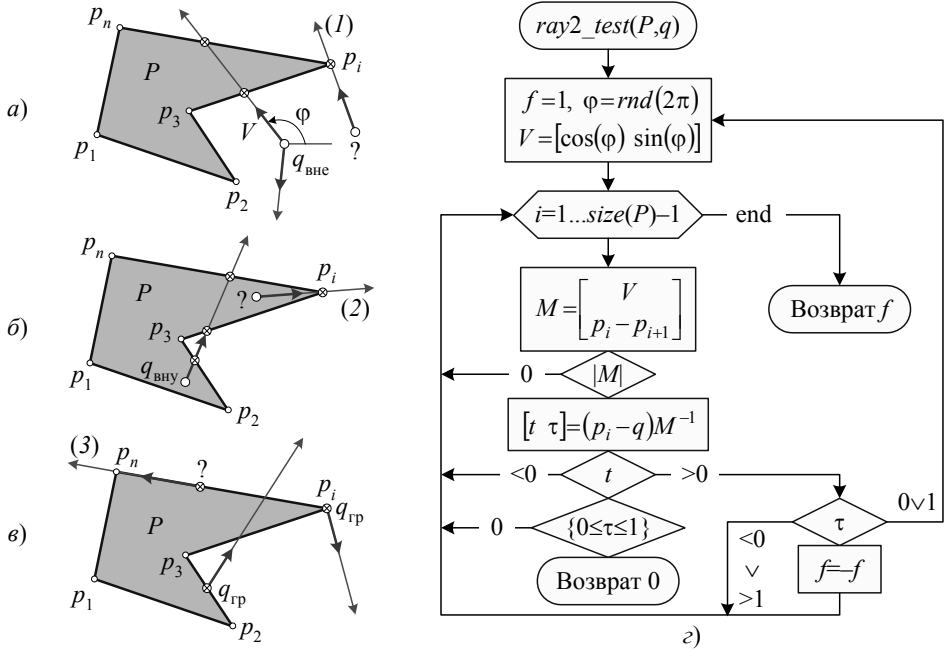


Рис. 1.26

кается, а луч 2 пересекает его один раз, тогда как расчёт даст два пересечения с отрезочными параметрами $\tau_{i-1}=1$ и $\tau_i=0$. У луча 3, выходящего из граничной точки точно по ребру, что, правда, очень маловероятно, обнаружится одно пересечение при $\tau_n=0$. **Решение проблемы:** начать тест заново с другим случайным направлением луча V ;

• в (1.49) идёт **приближённый** расчёт с плавающей точкой и обращением матрицы. Поэтому рекомендуется заменить сравнения параметров t_i и τ_i с точными числами 0 и 1 сравнениями с их ε -окрестностями (см. начало гл. 1).

Оформим сказанное в виде блок-схемы **алгоритма** $ray2_test(P,q)$ **лучевого теста локализации точки q относительно плоского полигона P** (рис. 1.26,з), возвращающего число -1 , если точка находится внутри полигона, 0 для точки на границе полигона и 1 , если точка лежит вне полигона.

Вторая группа тестов занимается проверкой расположения **отрезка ab и полигона $Q=q_1q_2\dots q_mq_1$** относительно полигона $P=p_1p_2\dots p_np_1$. Поскольку полигон можно интерпретировать как связный список отрезков (рёбер), будем строить тесты локализации для объекта общего типа, каким является полигон.

□ Точечный габаритный тест (1.45) легко обобщается, как показано на рис. 1.27, на **произвольный** полигон Q с габаритными координатами

$$q_{x\min}=\min\{q_{ix}\}, q_{x\max}=\max\{q_{ix}\}, q_{y\min}=\min\{q_{iy}\}, q_{y\max}=\max\{q_{iy}\}. \quad (1.50a)$$

Габаритный тест расположения полигона Q относительно полигона P выполняется функцией

$$gab2_poly(P,Q) = \{q_{x\max} < p_{x\min}\} \vee \{q_{x\min} > p_{x\max}\} \vee \{q_{y\max} < p_{y\min}\} \vee \{q_{y\min} > p_{y\max}\}, \quad (1.506)$$

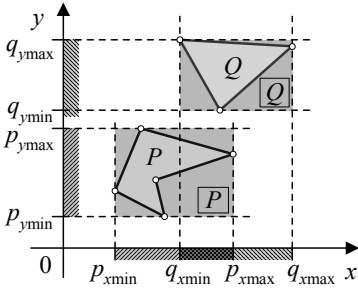


Рис. 1.27

возвращающей значение 1 при **гарантированном непересечении** прямоугольных габаритных оболочек полигонов $\boxed{Q}$ и $\boxed{P}$. Нулевой результат теста означает, что оболочка $\boxed{Q}$ либо полностью находится внутри оболочки $\boxed{P}$, либо с ней пересекается и не может быть однозначно идентифицирована как внешняя или внутренняя. Таким образом, **габаритный тест полностью задачу взаимной локализации полигонов не решает.**

□ Следующий тест $coll2_test(P,Q)$ **столкновения полигонов** $P=p_1p_2\dots p_np_1$ и $Q=q_1q_2\dots q_mq_1$ обнаруживает это событие (*collision*) по касанию (рис. 1.28,а) либо пересечению (рис. 1.28,б) хотя бы одной пары рёбер $\{p_ip_{i+1}, q_jq_{j+1}\}$, возвращая при этом 1 согласно блок-схеме на рис. 1.28,в. Проверка пересечения отрезков выполняется как и в алгоритме *self_test* (см. рис. 1.16,а) с помощью четырёх значений неявных функций $a \div d$. Параллельные и коллинеарные пары рёбер, для которых $a=b$, из проверки исключаются.

В отсутствие столкновения тест возвращает 0. При этом полигон Q может быть целиком расположен как снаружи, так и внутри полигона P . Различению этих двух случаев посвящён следующий тест.

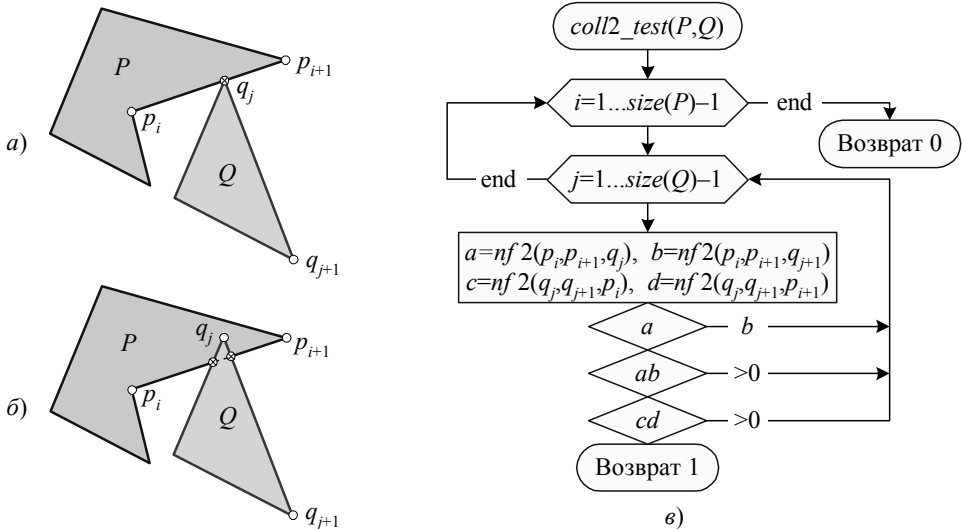


Рис. 1.28

□ Последний тест $surr2_test(P,Q)$ **окружения полигона** $Q=q_1q_2\dots q_mq_1$ **полигоном** $P=p_1p_2\dots p_np_1$ обнаруживает это событие (*surrounding*) по расположению всех точек q_i внутри полигона P и отсутствию столкновения полигонов

друг с другом (рис. 1.29,а), возвращая при этом число 1 согласно блок-схеме на рис. 1.29,б. В отсутствие охватывания тест возвращает 0.

Если полигон P выпуклый, то предпочтительнее вместо тестовой функции $ray2_test$ использовать её выпуклый аналог $conv2_test$, а тест столкновения можно опустить, возвратив число 1 сразу по окончании цикла.

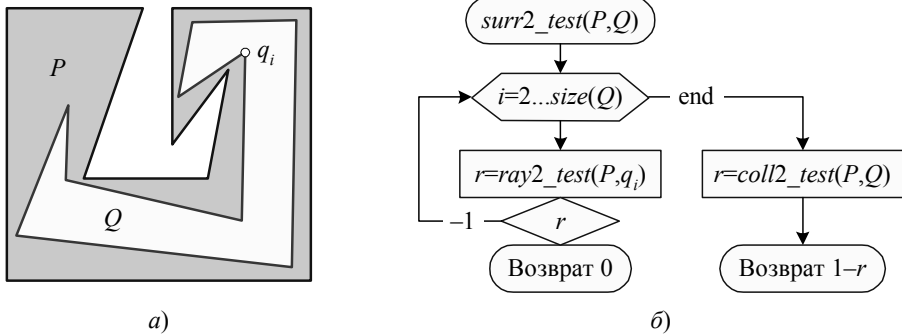


Рис. 1.29

Необходимость в тестах столкновения и окружения возникает при разработке анимационных программ с «твердыми» объектами, которые при столкновении, происшедшем в ходе движения, должны либо остановиться, либо упруго оттолкнуться по физическим законам без проникновения друг в друга.

1.1.5. Площадь и геометрический центр полигона

Рассмотрим методы вычисления площади S и геометрического центра g полигона $P=p_1p_2\dots p_np_1$.

□ Простейшим полигоном является **треугольник** abc (рис. 1.30). Его **площадь** может быть найдена несколькими способами [21]:

- расчёт по формуле **Герона**, открытой Архимедом,

$$S_{\Delta} = \sqrt{\rho(\rho - |a-b|)(\rho - |b-c|)(\rho - |c-a|)}$$

требует нахождения длин сторон треугольника $|a-b|$, $|b-c|$, $|c-a|$ и полупериметра $\rho = 0.5(|a-b| + |b-c| + |c-a|)$ ценой четырёх итерационных и **приближённых** вычислений квадратного корня. Эти затраты делают формулу Герона наименее эффективной в расчёте площади треугольника;

- известная из школьной тригонометрии формула площади треугольника

$$S_{\Delta} = 0.5|b-a| \cdot |c-a| \cdot \sin(\angle bac)$$

как половины произведения длины стороны ab на длину высоты h , опущенной на неё из вершины c (рис. 1.30), также содержит приближённые расчёты длин векторов $V = b-a$, $W = c-a$ и синуса угла между ними;

- сравнение последней формулы с нормой векторного произведения векторов $\vec{V} = [V_x \ V_y \ 0] \in xy$ и $\vec{W} = [W_x \ W_y \ 0] \in xy$, позволяет записать её в виде

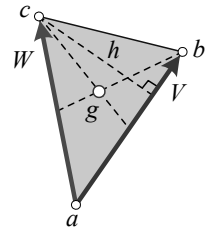


Рис. 1.30

$$S_{\Delta} = 0.5 |\vec{V} \times \vec{W}| = 0.5 \left| \begin{vmatrix} b-a \\ c-a \end{vmatrix} \right|.$$

Введём **функцию алгебраической площади треугольника abc**

$$\sigma(a, b, c) = 0.5 \left| \begin{vmatrix} b-a \\ c-a \end{vmatrix} \right| = 0.5 n f 2(a, c, b), \quad (1.51)$$

сводящуюся к вычислению значения НФ прямой $\{a, c\}$ в точке b . Знак алгебраической площади $\sigma(a, b, c)$ совпадает с ориентацией вершины b относительно стороны ac и с направлением обхода треугольника. Окончательное выражение площади треугольника принимает вид

$$S_{\Delta} = |\sigma(a, b, c)|. \quad (1.52a)$$

Геометрический центр треугольника расположен в точке пересечения медиан и делит каждую из них в отношении 2:1. Отсюда получаем

$$g = a + \frac{2}{3} \left(\frac{b+c}{2} - a \right) = \frac{a+b+c}{3}. \quad (1.52b)$$

□ **Выпуклый полигон** $p_1 p_2 \dots p_n p_1$ после разрезания диагоналями, проведёнными из первой вершины p_1 , представляет собой объединение $n-2$ -х непересекающихся треугольников $p_1 p_2 p_3 p_1 \div p_1 p_{n-1} p_n p_1$ (рис. 1.31), площади которых S_i и геометрические центры g_i вычисляются по (1.52):

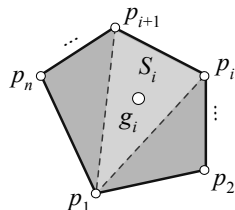


Рис. 1.31

$$S_i = |\sigma(p_1, p_i, p_{i+1})|, \quad g_i = \frac{p_1 + p_i + p_{i+1}}{3}. \quad (1.53a)$$

Представляя совокупность треугольников системой точечных масс S_i , закреплённых в точках g_i [22], получим площадь и центр выпуклого полигона:

$$S = \sum_{i=2}^{n-1} S_i, \quad g = \sum_{i=2}^{n-1} S_i g_i / S. \quad (1.53b)$$

□ Площадь **невыпуклого полигона** можно вычислить по (1.53б) после его **триангуляции** — разрезания на непересекающиеся треугольники (рис. 1.32,а). Задача триангуляции произвольного полигона достаточно сложна. Ряд алгоритмов триангуляции рассматривается в п. 1.4.5. После небольшой модификации метод (1.53) может работать и с невыпуклыми полигонами (рис. 1.32,б):

$$S = \left| \sum_{i=2}^{n-1} \sigma_i \right|, \quad g = \sum_{i=2}^{n-1} \sigma_i g_i / \sum_{i=2}^{n-1} \sigma_i, \quad \text{где } \sigma_i = 0.5 n f 2(p_1, p_{i+1}, p_i). \quad (1.54)$$

Знаки площадей σ_i треугольников $p_1 p_i p_{i+1} p_1$ совпадают с направлениями их обхода. Фрагменты треугольников, лежащие вне полигона, суммируются в (1.54) с противоположными знаками чётное число раз, в результате чего их площади взаимно сокращаются. Фрагменты треугольников внутри полигона суммируются с противоположными знаками нечётное число раз. Таким обра-

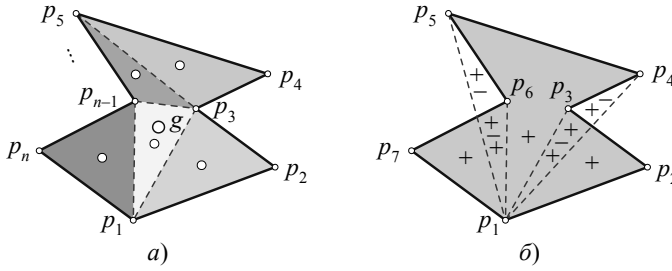


Рис. 1.32

зом, **алгебраическая площадь полигона включает площади только внутренних фрагментов** и только однократно. Геометрический центр полигона также является взвешенной **алгебраической** суммой центров треугольников g_i с учётом их алгебраических площадей σ_i , вычисленных по (1.54).

Знак алгебраической площади полигона совпадает с общим направлением обхода его вершин. В итоге получен **тест направления обхода полигона**

$$\boxed{dir_test(P) = \operatorname{sgn} \left(\sum_{i=2}^{n-1} n f 2(p_1, p_{i+1}, p_i) \right)}, \quad (1.55)$$

трудоемкость которого составляет $8n-15$ арифметических операций. По эффективности он даст сто очков вперёд полученным в (1.48) радианному и октантному вариантам теста направления обхода полигона.

1.1.6. Алгоритмы генерирования случайных полигонов

В процессе создания и отладки алгоритмов компьютерной графики необходимы образцы различных графических объектов, служащие исходными данными для моделирования. Принципиально различаются два типа данных:

- задаваемые **вручную** и позволяющие добиться верного решения задачи при априорно известном ответе. Сложность ручного задания — в умении заранее предвидеть все нюансы задачи, поставить нужные цифры в нужное место так, чтобы за минимальное число испытаний пройти по всем ветвям алгоритма и добиться его правильной работы;
- задаваемые **случайно** и позволяющие повысить статистическую достоверность работы алгоритма. При большом числе испытаний со случайными данными и хорошо продуманной системой диагностики рано или поздно обнаружатся все имеющиеся в алгоритме логические и вычислительные ошибки.

Имея в распоряжении функцию $rnd(x)$ — **генератор равномерно распределённых случайных чисел (РРСЧ)** в интервале $(0, x)$, можно построить генераторы других типов случайных данных:

- $a + rnd(b-a)$ — генератор РРСЧ в интервале (a, b) ;
- $n + [rnd(m-n)]$ — генератор **целых** РРСЧ в интервале $[n, m]$;
- $V_i = a + rnd(b-a) \quad \forall i = \overline{1, n}$ — генератор случайного вектора $V \in \mathbb{R}^n$, элементы которого есть РРСЧ в интервале (a, b) ;

- $V = [\cos(\varphi) \sin(\varphi)]$ — генератор единичного вектора на плоскости со случайным углом направления $\varphi = \text{rnd}(2\pi)$;
- $V = [\sin(\varphi)\cos(\psi) \sin(\psi) \cos(\varphi)\cos(\psi)]$ — генератор единичного вектора в трёхмерном пространстве со случайными углами азимута $\varphi = \text{rnd}(2\pi)$ и места $\psi = \text{rnd}(\pi) - \pi/2$ (см. рис. 1.1,б).

Разработаем два алгоритма генерирования **случайных полигонов** — системы **случайного** (не менее трёх) числа **случайных** точек на плоскости, последовательно соединённых друг с другом **самонепересекающейся** замкнутой ломаной линией со **случайным** направлением обхода вершин.

□ **Случайный звёздный** [48]) **полигон** создаётся путём испускания лучей случайной длины из некоторой, возможно, тоже случайной точки в процессе кругового вращения со случайными приращениями углов (рис. 1.33,а).

📁 **Шаг 1.** Из заданной точки T проводим лучи под углами φ_n к оси x . Начальный угол $\varphi_1 = 0^\circ$, а следующие углы $\varphi_{n+1} = \varphi_n + \Delta\varphi_n$ накапливают случайные приращения $\Delta\varphi_n = \theta + \text{rnd}(\Theta - \theta) \in (\theta, \Theta)$ до тех пор, пока $\varphi_n < 360^\circ$.

Шаг 2. Вдоль лучей откладываем расстояния $r_n = a + \text{rnd}(b - a)$, генерируемые как РРСЧ в интервале (a, b) . Получаем вершины полигона

$$p_n = T + r_n [\cos(\varphi_n) \sin(\varphi_n)], \quad n = 1, 2, \dots$$

Шаг 3. Генерируем РРСЧ $\text{rnd}(2) \in (0, 2)$, равномерно попадающее как в интервал $(0, 1)$ (возвращается полигон $P = p_1 p_2 \dots p_n p_1$), так и в интервал $[1, 2)$ (возвращается реверсивный полигон $\vec{P} = p_1 p_n \dots p_1$). □

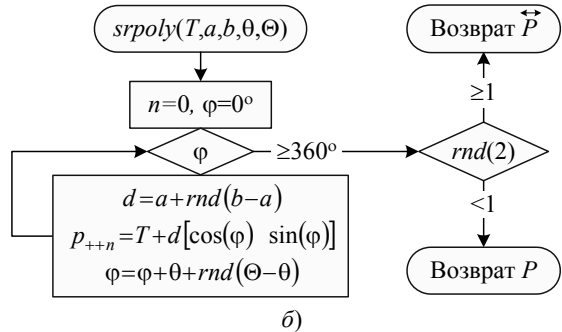
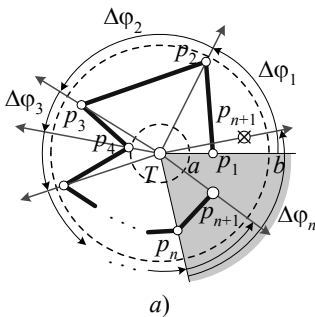


Рис. 1.33

Оформим алгоритм как **функцию** $\text{srpoly}(T, a, b, \theta, \Theta)$ **генератора звёздного случайного полигона**, лучи которого удалены от точки T (её тоже можно задавать случайной) на случайные расстояния от a до b , а векторы направлений состоят на случайные углы от θ° до Θ° (рис. 1.33,б).

Статистический анализ генерируемых полигонов в зависимости от параметра $\Theta \in \{90^\circ, 120^\circ, 150^\circ, 180^\circ\}$ при $\theta = 0^\circ$ на выборке в 10 000 испытаний дал гистограммы числа сторон полигонов, показанные на рис. 1.34. Зависимости приближённо описываются законом распределения Рэлея. Ненулевая вероят-

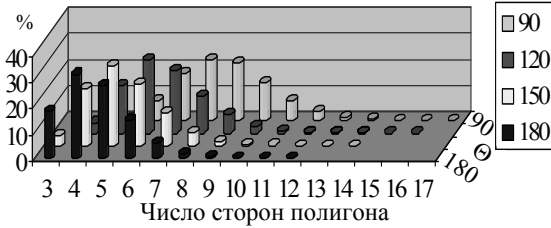


Рис. 1.34

гоны лишь ограниченного *звёздного* класса, имеющие *ядро* [23, 48], содержащее точку T . Для получения только *выпуклых* полигонов следует задавать *равные* значения $a=b$, при которых все вершины полигона $p_i = T + a[\cos(\varphi_i) \sin(\varphi_i)]$ лежат на описанной окружности радиуса a и видны из её центра T под случайными углами.

□ **Абсолютно случайный полигон**, как выпуклый, так и невыпуклый, как с ядром, так и без ядра, с произвольными углами и направлением обхода строится методом имитации процесса шагания по плоскости в случайных направлениях без пересечения пройденного пути и с возвратом в исходную точку [39].

Блок-схема функции $arpoly(T, a, b, m, M)$ генератора абсолютно случайного полигона, реализующей этот процесс, приведена на рис. 1.35,а. Первый векторный аргумент T задаёт расположение начальной вершины $p_1 = T$, пара следующих аргументов a и b определяет диапазон длины шага $d \in (a, b)$, четвёртый аргумент $m \geq 3$ задаёт номер вершины, с которой начинается проверка возможности замыкания полигона (при задании $m=3$ генерируются только треугольники), а пятый аргумент $M \geq m$ ограничивает максимальное число шагов хаотического блуждания для предотвращения его заклинивания.

Главные действия алгоритма происходят в цикле $n = \overline{1, M}$:

- поиск следующей вершины p_{n+1} начинаем с откладывания от точки p_n отрезка случайной длины $d = a + rnd(b - a)$ под случайным углом $\varphi = rnd(2\pi)$. На конце отрезка получаем *пробную точку* $q = p_n + d[\cos(\varphi) \sin(\varphi)]$;
- при $n \leq 2$, запоминая точки $p_{n+1} = q$, создаём полилинию $p_1 p_2 p_3$;
- при $n \geq 3$ проверяем пересечение отрезка $p_n q$ с уже построенными *несмежными* рёбрами $p_1 p_2 \div p_{n-2} p_{n-1}$ (рис. 1.35,б). Обнаружив пересечение, повторяем генерацию новой пробной точки q до тех пор, пока она не станет новой вершиной полигона. Во избежание заклинивания ограничим число пробных повторений большим числом, например, 100;
- при $n \geq m$ проверяем полилинию $p_1 p_2 \dots p_n$ на возможность замыкания ребром $p_n p_1$, не пересекающимся ни с одним из *несмежных* рёбер $p_2 p_3 \div p_{n-2} p_{n-1}$ (рис. 1.35,в). При удаче алгоритм **возвращает** готовый полигон $P = p_1 p_2 \dots p_n p_1$;
- если за все M шагов случайного блуждания полилиния $p_1 p_2 \dots p_M$ не может быть замкнута без самопересечения (рис. 1.35,г), то, проинициализировав пустой список $P = \emptyset$, повторим процесс шагания от точки $p_1 = T$ заново до успешно-

ность генерации полигонов, начиная с треугольников, гарантируется при угле $\Theta \in (120^\circ, 180^\circ]$. Соотношение положительного и отрицательного направлений обхода полигонов близко к 1:1.

Алгоритм способен генерировать произвольные поли-

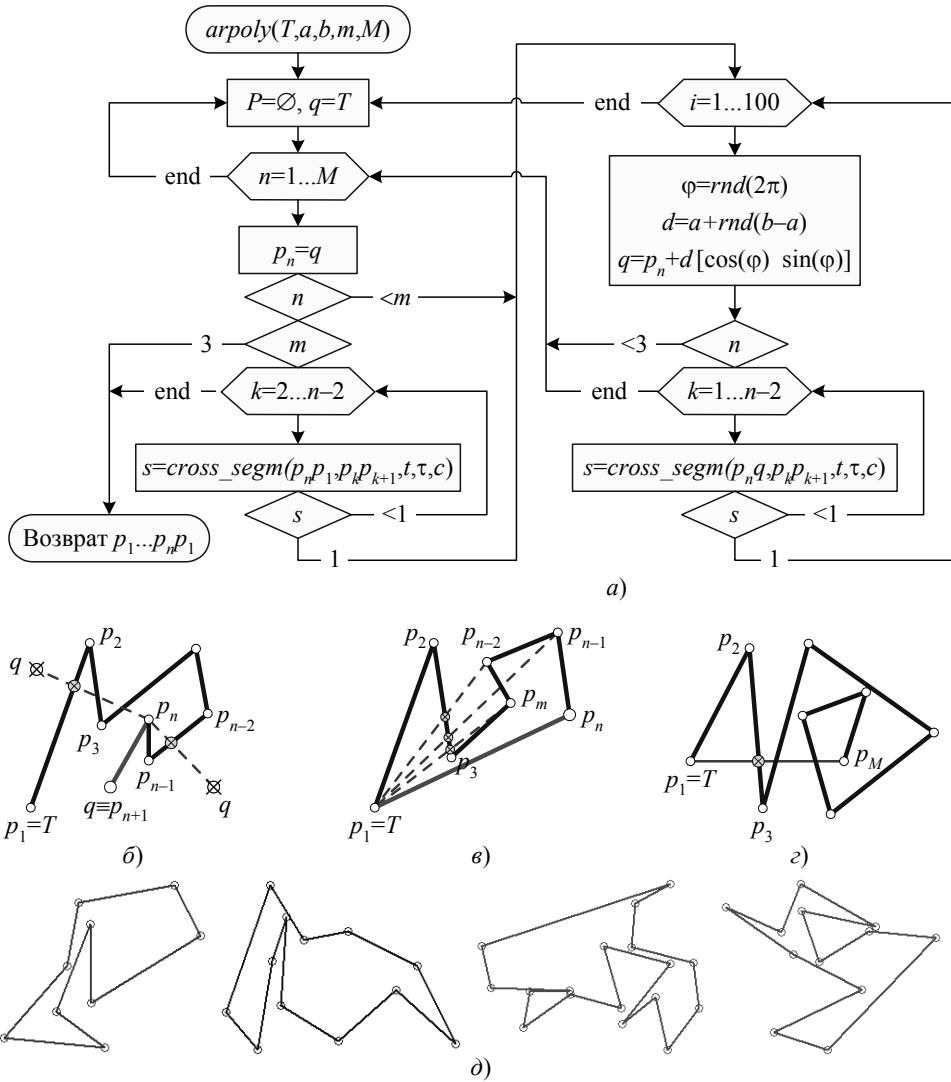


Рис. 1.35

го получения $n \in [m, M]$ -стороннего полигона $P = p_1 p_2 \dots p_n p_1$. Такой же повтор следует делать после 100 неудачных попыток генерации пробных точек.

На рис. 1.35, д показано несколько случайных полигонов, сгенерированных алгоритмом $arpoly(T, a, b, m, M)$ с параметрами $a=2, b=5, m=5$ и $M=20$.

Рассмотрим ещё два метода построения полигонов на основе массива $P = \{p_1, p_2, \dots, p_n\}$ из $n > 2$ -х точек $p_i = [x_i \ y_i]$, $i \in [1, n]$. При случайном характере генератора точек получаемые полигоны также будут случайными.

□ **Выпуклая полигональная оболочка массива точек** есть объединение всех полигонов, построенных на точках этого массива как на вершинах. Её можно также представить как выпуклый полигон минимальной площади, охватывающий все точки массива (рис. 1.36).

По отношению к оболочке точки массива делятся на **внутренние** и **граничные**, принадлежащие сторонам полигона оболочки. Граничные точки, образующие вершины этого полигона, назовём **выступающими**. На отрезки между выступающими вершинами могут попасть другие граничные точки. Задача построения выпуклой оболочки заключается в нахождении выступающих точек и соединении их замкнутым контуром.

Алгоритм $cover2(P)$, решающий задачу построения выпуклой оболочки, представлен блок-схемой на рис. 1.37. Он воплощает простой метод построения ограды вокруг рощи деревьев, по которому сначала выбирается некоторое «крайнее» дерево (**нижняя левая** точка на рис. 1.36), от которого строится первая сторона ограды в направлении V_1 до второго выступающего дерева под минимальным углом $\phi_1 \geq 0$ (на рис. 1.36 этот угол нулевой). Если в этом направлении растёт несколько деревьев, то ограда строится до самого дальнего из них. Опираясь на эту точку, вектор направления V_2 вращается на минимальный угол ϕ_2 , под которым видно третье выступающее дерево. Процесс огораживания продолжается до тех пор, пока полигон не замкнётся в начальной точке. В итоге окажется, что контур ограды соединяет выступающие деревья отрезками прямых и при этом все деревья расположены слева от отрезков ограды либо на них. Следовательно, в соответствии с тестом $conv2$, определённым в (1.39), полученная полигональная оболочка является выпуклой.

Переходя к алгоритмизации процесса построения полигональной оболочки в **положительном** направлении обхода, найдём в массиве точек первую высту-

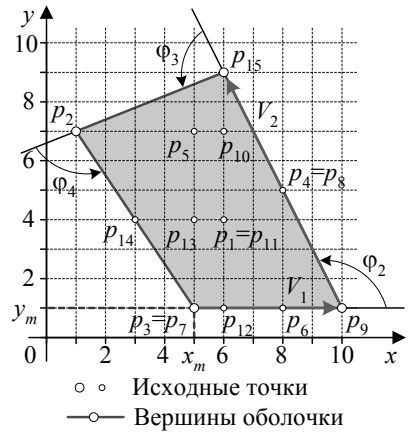


Рис. 1.36

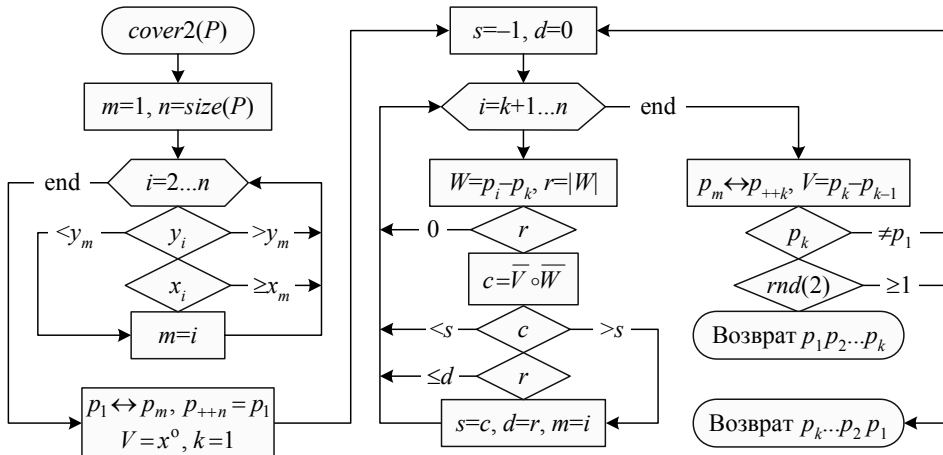


Рис. 1.37

пающую точку p_m с минимальной ординатой y_m . Если таких точек несколько, то выберем из них точку с минимальной абсциссой x_m (см. рис. 1.36). Путём обмена $p_1 \leftrightarrow p_m$ поставим найденную точку в начало массива, а её копию добавим в его конец в роли точки p_{++n} , необходимой для проверки замыкания оболочки.

Зададим начальное направление вектора охватывающей линии $V = x^0$ вправо по оси x и число найденных точек оболочки $k=1$.

Далее организуем внешний цикл $k=1, 2, \dots$ поиска следующих выступающих точек p_m и накопление их в начале массива. Во внутреннем цикле $i=\overline{k+1, n}$ для каждой точки p_i вычисляем **пробный** вектор $W = p_i - p_k$. При его ненулевой длине $r=|W| \neq 0$ (данная проверка позволяет отсеять кратные точки, не входящие в оболочку) находим скалярное произведение $c = \cos(\varphi) = \overline{V} \circ \overline{W} \in (-1, 1]$. Перебрав точки $p_i \forall i > k$, находим точку p_m , видимую из точки p_k под минимальным углом $\varphi = \angle(V, W)$, т. е. при **максимальном** значении c . Если таких точек несколько, то выбираем из них самую дальнюю от p_k . Инкрементировав индекс $++k$, сделаем её путём обмена $p_m \leftrightarrow p_k$ новой выступающей точкой оболочки. Новый вектор направления положим равным $V = p_k - p_{k-1}$.

Внешний цикл и построение оболочки заканчиваются при совпадении последней точки p_k с начальной точкой p_1 . В результате получен субмассив вершин оболочки $p_1 \dots p_k$ с положительным направлением обхода, который равновероятно возвращается либо в таком, либо в реверсивном $p_k \dots p_1$ виде.

Функция $cover2(P)$ является быстрым **генератором выпуклых полигонов**, дополняющим описанный выше алгоритм $srpoly$ с аргументами $a=b$. Статистическое моделирование со случайными выборками случайного числа точек показало его высокую надёжность и быстроту работы. В сравнении с рассмотренным в [23] алгоритмом $giftwrapHull$, основанным на методе «заворачивания подарка» [48], алгоритм $cover2$ правильно строит оболочку и в том случае, когда в массиве есть кратные и коллинеарные точки.

Расчёт выпуклой оболочки может быть полезен при построении тени **выпуклого** сплошного полиэдра P на некоторой плоскости f . Для решения этой задачи совсем не обязательно строить тени каждой грани полиэдра, которые потом всё равно сольются в одно выпуклое пятно. Достаточно рассчитать проекции p_{if} вершин полиэдра p_i , построить выпуклую оболочку этих точек (то самое пятно) и закрасить её цветом тени.

 **Пример 1.4.** Построить выпуклую оболочку массива точек $p_1 \div p_{15}$, из которых кратными являются $p_1 \equiv p_{11}$, $p_3 \equiv p_7$ и $p_4 \equiv p_8$ (см. рис. 1.36).

Решение. Анализируя координаты точек p_i , находим самую нижнюю левую точку p_3 — первую выступающую вершину оболочки. Переставив её с точкой p_1 и добавив в конец массива, получим (нумерация точек для наглядности сохранена в исходном виде, хотя в действительности она меняется при обмене точек) $P = p_3 \mid p_2 p_1 p_4 p_5 \dots p_{15} p_3$. Вертикальный пунктир отделяет вершины построенной части оболочки от остальных точек.

Вторую выступающую точку ищем по принципу уменьшения положительного угла между векторами $V = [1 \ 0]$ и $W = p_i - p_3$. Просматривая точки p_2, p_1, p_4 и p_6 , лежащие на луче с

углом $\varphi_1=0$, находим, наконец, самую дальнюю точку p_9 . После перестановки p_9 с p_2 массив точек принимает следующий вид:

$$P = p_3 p_9 \mid p_1 p_4 \dots p_8 p_2 p_{10} \dots p_{15} p_3.$$

Следующий угол φ_2 между векторами $V = p_9 - p_3$ и $W = p_1 - p_9$ будет минимальным для выступающей точки p_{15} . Обменяв её с p_1 , получим следующий состав массива и в его начале — построенную часть оболочки:

$$P = p_3 p_9 p_{15} \mid p_4 \dots p_8 p_2 p_{10} \dots p_{14} p_1 p_3.$$

После нахождения четвёртой выступающей точки p_2 и обмена её с точкой p_4 получим следующий массив:

$$P = p_3 p_9 p_{15} p_2 \mid p_5 p_6 p_7 p_8 p_4 p_{10} p_{11} p_{12} p_{13} p_{14} p_1 p_3.$$

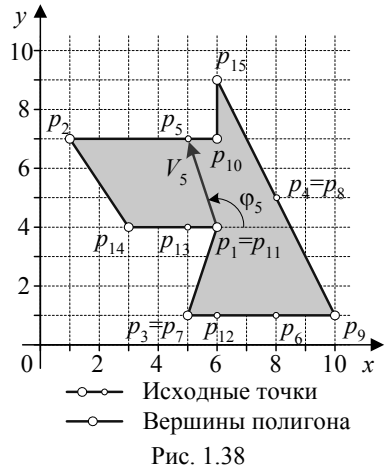
Обнаружение выступающей точки p_7 , совпадающей с первой вершиной оболочки p_3 , завершает работу алгоритма со значением $k=5$. Как видно, благодаря совпадению точка p_3 , специально помещённая в конец массива, не пригодилась. Но если бы точка p_7 находилась в другом месте, то алгоритм закончил бы работу при обнаружении совпадения начальной точки p_3 с её копией в конце массива. Окончательный состав оболочки следующий:

$$P = p_3 p_9 p_{15} p_2 p_7.$$



□ **Полигонализация массива точек** $P = \{p_1, p_2, \dots, p_n\}$ заключается в проведении через **все** точки замкнутого полигонального **непересекающегося** контура. Понятно, что если соединить точки $p_1 \div p_n$ в порядке их нумерации в массиве, то, скорее всего, получится самопересекающаяся ломаная линия. Нужно найти такой критерий сортировки точек, чтобы после их перенумерации новая замкнутая полилиния $p_1 p_2 \dots p_n p_1$ не имела самопересечений.

Свойство видимости всех вершин **звёздного** полигона из любой точки его ядра [48] наводит на мысль о возможности такой сортировки в полярной системе координат с началом в любой точке массива, например, в p_1 (рис. 1.38). Описываемый **алгоритм полигонализации массива точек** $poly_point(P)$, блок-схема которого приведена на рис. 1.39, основан на сортировке точек по возрастанию угла вращения луча $p_1 \rightarrow p_i \forall i = \overline{2, n}$ при $p_i \neq p_1$, не допуская пересечения рёбер строящегося полигона друг с другом и исключая из списка вершин кратные и содержащие развёрнутый угол точки.



Сначала в цикле $i = \overline{2, n}$ формируется массив $Q = \{q_1, \dots, q_m\}$ из $m < n$ точек $p_i \neq p_1$ и по (1.4) вычисляются углы $\varphi_i = \arg(V_i) \in (-\pi, \pi]$ векторов $V_i = q_i - p_1$. Затем точки q_i сортируются по возрастанию углов φ_i и по краям полученной ломаной линии $q_1 q_2 \dots q_m$ добавляется точка p_1 .

Будет ли замкнутый полигон $P = p_1 q_1 \dots q_m p_1$ решением задачи полигонализации массива точек, как это утверждается в [23]? Ответ утвердительный **почти**

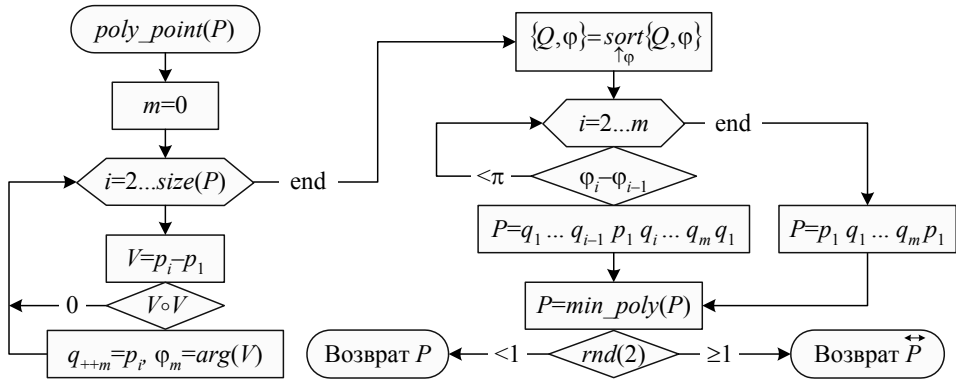


Рис. 1.39

всегда для произвольного набора точек. Однако, есть случаи, нарушающие данный порядок формирования полигона. Соответственно, дальнейшие действия следующие:

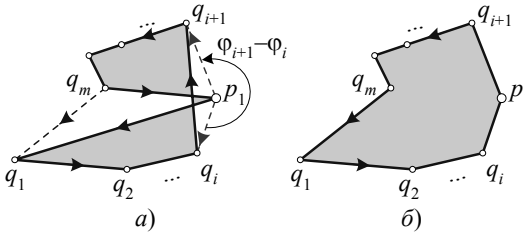


Рис. 1.40

- если существуют соседние лучи $p_1 \rightarrow q_i$ и $p_1 \rightarrow q_{i+1}$ с углом вращения $\phi_{i+1} - \phi_i > \pi$, то полигон $P = p_1 q_1 \dots q_m p_1$ станет самопересекающимся (рис. 1.40,а). Тогда следует сформировать замкнутый полигон $P = q_1 \dots q_i p_1 q_{i+1} \dots q_m q_1$ с началом в точке q_1 (рис. 1.40,б);

- если массив имеет на отрезке прямой группу смежных точек, то вершинами полигона должны быть только концевые точки этого отрезка. То же касается и избыточности кратных точек в задаче полигонализации. Удаление из неминимального полигона лишних вершин выполняется алгоритмом формирования минимального полигона $P = \min_poly(P)$ (см. рис. 1.18,а).

В заключение алгоритм генерирует случайное число $d = \text{rnd}(2) \in (0, 2)$ и равновероятно возвращает либо полигон $P = p_1 p_2 \dots p_k p_1$ (при $d < 1$), либо реверсивный полигон $\vec{P} = p_1 p_k \dots p_2 p_1$ (при $d \geq 1$).

Функция *poly_points* может быть полезна в качестве **генератора произвольных звёздных полигонов** взамен приведённого выше алгоритма *srpoly*. Такой генератор можно использовать как для проверки работоспособности различных тестов и алгоритмов на плоскости, так и для решения задач со случайными полигонами.

Пример 1.5. Построить полигон на точках $p_1 \div p_{15}$ из примера 1.4.

Решение. Вычислим в градусах углы видимости каждой, кроме p_{11} , точки массива из точки p_1 (см. рис. 1.38):

p_i	p_2	p_3	p_4	p_5	p_6	p_7	p_8	p_9	p_{10}	p_{12}	p_{13}	p_{14}	p_{15}
ϕ_i	149°	-108°	26°	108°	-56°	-108°	26°	-37°	90°	-90°	180°	180°	90°

Угловая сортировка, выполненная MathCAD-программой, расположила точки массива Q в следующем порядке:

q_i	p_3	p_7	p_{12}	p_6	p_9	p_4	p_8	p_{15}	p_{10}	p_5	p_2	p_{13}	p_{14}
φ_i	-108°	-108°	-90°	-56°	-37°	26°	26°	90°	90°	108°	149°	180°	180°

Так как приращения углов между каждой парой точек меньше 180° , то замкнутый контур обхода точек массива имеет следующий вид:

$$P = p_1 p_3 p_7 p_{12} p_6 p_9 p_4 p_8 p_{15} p_{10} p_5 p_2 p_{13} p_{14} p_1.$$

На этапе минимизации этого массива отбираем точки p_1, p_3 , пропускаем кратную точку $p_7 \equiv p_3$ и добавляем p_{12} . Следующая точка p_6 коллинеарна отрезку $p_3 p_{12}$ и лежит впереди него, поэтому p_{12} заменяем точкой p_6 . Она, в свою очередь, заменяется коллинеарной точкой p_9 , лежащей впереди отрезка $p_3 p_6$. Далее к списку добавляется вершина p_4 , а кратная точка $p_8 \equiv p_4$ пропускается. Вершина p_{15} впереди отрезка $p_9 p_4$ заменяет точку p_4 .

Следующими вершинами минимального полигона становятся точки p_{10}, p_2 (она заменила точку p_5 , т. к. лежит впереди коллинеарного отрезка $p_{10} p_5$), p_{13} и p_{14} . Последняя точка p_1 лежит сзади коллинеарного отрезка $p_{13} p_{14}$, поэтому согласно рис. 1.18,б последние звенья минимального полигона $\dots p_2 p_{13} p_{14} p_1$ заменяем на $\dots p_2 p_{14} p_1$. В результате получаем полигон

$$P = p_1 p_3 p_9 p_{15} p_{10} p_2 p_{14} p_1.$$



1.1.7. Квадратичные линии

Алгебраические линии второго порядка (**квадратичные линии**) на плоскости описываются неявным двухкоординатным уравнением (**квадрикой**)

$$f(x, y) = Ax^2 + By^2 + 2Exy + 2Kx + 2Ly + D = 0 \quad (1.56a)$$

или **квадратичной формой** с векторным аргументом $p \in \mathbb{R}^2$:

$$f(p) = \tilde{p} F \tilde{p}^T = 0, \text{ где } F = \begin{bmatrix} A & E & K \\ E & B & L \\ K & L & D \end{bmatrix} = \begin{bmatrix} F_{11} & F_{21}^T \\ F_{21} & D \end{bmatrix}, \quad F_{11}^T = F_{11}. \quad (1.56b)$$

Вектор нормали к квадратичной линии в точке $q \in f(p)$ вычисляется по (1.8a) путём дифференцирования (1.56б):

$$N = 2 \begin{bmatrix} q_x & q_y & 1 \end{bmatrix} \begin{bmatrix} A & E \\ E & B \\ K & L \end{bmatrix} = 2 \begin{bmatrix} q & 1 \end{bmatrix} \begin{bmatrix} F_{11} \\ F_{21} \end{bmatrix} = 2(q F_{11} + F_{21}). \quad (1.57)$$

Для любой линии второго порядка три величины

$$I_1 = A + B, \quad I_2 = |F_{11}| = AB - E^2, \quad I_3 = |F| \quad (1.58a)$$

называются **инвариантами** [13, 21], не изменяющимися при невырожденных преобразованиях переноса и вращения, изучаемых в гл. 3. Число

$$I'_2 = \begin{vmatrix} A & K \\ K & D \end{vmatrix} + \begin{vmatrix} B & L \\ L & D \end{vmatrix} = I_1 D - K^2 - L^2 \quad (1.58b)$$

является **полуинвариантом** линии второго порядка относительно преобразования вращения вокруг начала системы координат. Знание легко вычисляемых инвариантов и полуинварианта позволяет быстро определить по уравнениям (1.56) любой из девяти возможных типов и **каноническое уравнение** квадратичной линии с помощью табл. 1.1.

Таблица 1.1

$I_2 \backslash I_3$		Невырожденные кривые $I_3 \neq 0$		Вырожденные кривые $I_3 = 0$
Центральные кривые $I_2 \neq 0$	$I_2 > 0$	$I_1 I_3 < 0$	Действительный эллипс $x^2/a^2 + y^2/b^2 - 1 = 0$	Точка вырожденного эллипса $x^2/a^2 + y^2/b^2 = 0$
		$I_1 I_3 > 0$	Мнимый эллипс $x^2/a^2 + y^2/b^2 + 1 = 0$	
	$I_2 < 0$	Гипербола $x^2/a^2 - y^2/b^2 - 1 = 0$		Две пересекающиеся прямые $x^2/a^2 - y^2/b^2 = 0$
Нецентральные кривые $I_2 = 0$		Парабола $x^2/a^2 - y/b = 0$		$I'_2 < 0$ Две параллельные прямые $x^2/a^2 - 1 = 0$
				$I'_2 = 0$ Одна двойная прямая $x^2/a^2 = 0$
				$I'_2 > 0$ Мнимая прямая $x^2/a^2 + 1 = 0$

Неявная форма неудобна для построения квадратичных линий и используется для анализа их свойств по инвариантам и расчёта пересечений. Исключением являются редкие случаи явных функций $y(x)$ или $x(y)$.

Рассмотрим варианты **взаимного расположения прямых и квадратичных линий**. Подставим параметрическую функцию прямой $p(t) = q + Vt$ в неявное векторное уравнение квадратичной линии (1.56б):

$$[q + Vt \ 1] \begin{bmatrix} F_{11} & F_{21}^T \\ F_{21} & D \end{bmatrix} \begin{bmatrix} q^T + V^T t \\ 1 \end{bmatrix} = 0 \Rightarrow (q + Vt)F_{11}(q + Vt)^T + 2(q + Vt)F_{21}^T + D = 0. \quad (1.59)$$

После упрощения получим квадратное уравнение относительно параметра t

$$at^2 + 2bt + c = 0 \quad (1.60a)$$

с коэффициентами

$$a = VF_{11}V^T, \quad b = (qF_{11} + F_{21})V^T, \quad c = qF_{11}q^T + 2qF_{21}^T + D = f(q). \quad (1.60б)$$

В зависимости от их значений и дискриминанта

$$\delta = b^2 - ac \quad (1.60в)$$

возможны следующие варианты решения задачи пересечения прямой с квадратичной линией (рис. 1.41,а):

- при $\{a \neq 0\} \wedge \{\delta > 0\}$ прямая (l) **пересекает** квадрику в двух различных точках $p_1 = q + Vt_1$ и $p_2 = q + Vt_2$ с параметрами

$$t_1 = (-b - \sqrt{\delta})/a < t_2 = (-b + \sqrt{\delta})/a; \quad (1.61a)$$

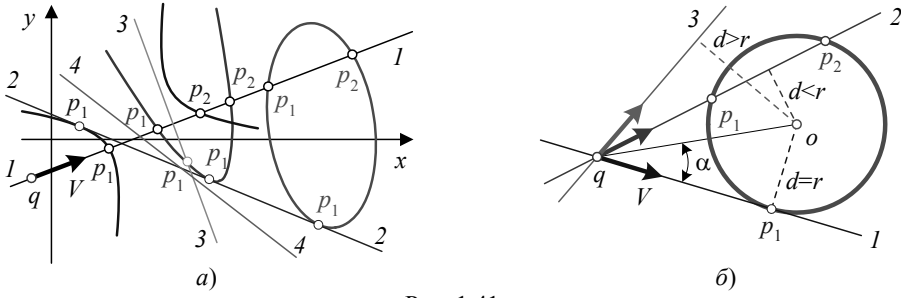


Рис. 1.41

• при $\{a \neq 0\} \wedge \{\delta = 0\}$ прямая (2) **касается** с квадратичной линией в **единственной** точке $p_1 = q + Vt_1$ с параметром

$$t_1 = -b/a; \quad (1.61б)$$

• при $\{a = 0\} \wedge \{b \neq 0\}$ прямая (3) **пересекает** квадрику в **единственной** точке $p_1 = q + Vt_1$ с параметром

$$t_1 = -c/2b; \quad (1.61в)$$

• при $\{a = 0\} \wedge \{b = 0\} \wedge \{c = 0\}$ прямая **совпадает** с **вырожденной** прямой линией второго порядка в **бесконечном множестве точек**;

• в остальных случаях прямая (4) не имеет с квадратичной линией ни одной общей точки.

Исследуем в частном случае возможные решения **задачи пересечения прямой линии с окружностью** $\{o, r\}$ радиуса r и центром в точке o (рис. 1.41, б). Подставив в неявную функцию окружности

$$f(p) = |p - o|^2 - r^2 = p \circ p - 2p \circ o + o \circ o - r^2$$

модель прямой $p(t) = q + Vt$

$$(Vt + q - o) \circ (Vt + q - o) - r^2 = (V \circ V)t^2 + (q - o) \circ V + V \circ (q - o) + (q - o) \circ (q - o) - r^2,$$

получим коэффициенты квадратного уравнения (1.60а):

$$a = V \circ V \neq 0, \quad b = (q - o) \circ V, \quad c = (q - o) \circ (q - o) - r^2. \quad (1.62)$$

Условие **касания** прямой (1) с окружностью — нулевой дискриминант

$$\delta = b^2 - ac = |q - o|^2 |V|^2 (\cos^2(\alpha) - 1) + r^2 |V|^2 = (r^2 - d^2) \cdot |V|^2,$$

где d — вычисляемое по (1.24б) расстояние от центра окружности до прямой. При $d = r$ получим значение параметра (1.61б) и точку касания

$$p_1 = q + \frac{(o - q) \circ V}{V \circ V} V = q + ((o - q) \circ \bar{V}) \bar{V}. \quad (1.63а)$$

Если $d < r$, то прямая (2) **пересекает** окружность в двух точках

$$p_{1,2} = q + \frac{(o - q) \circ V \pm |V| \sqrt{r^2 - d^2}}{V \circ V} V = q + ((o - q) \circ \bar{V} \pm \sqrt{r^2 - d^2}) \bar{V}. \quad (1.63б)$$

Наконец, при $d > r$ прямая (3) и окружность не пересекаются и не касаются.

1.1.8. Параметрические линии

Параметрическая форма $p(t)$ как самая гибкая и универсальная наиболее распространена для описания и построения линий на плоскости и особенно в пространстве благодаря разнообразию выбора функций, задающих движение точки по направлениям её степеней свободы. С независимым параметром t , изменяющим координатные функции, в большинстве случаев ассоциируется конкретное геометрическое (угол) или физическое (время) понятие, что превращает абстрактные формулы в легко представимую траекторию движения точки.

Декартова параметризация линии $p(t) \equiv [x(t) \ y(t)]$ из других форм её описания может быть выполнена следующими способами.

1. **Заменой аргументов x и y неявного уравнения $f(x, y) = 0$** подходящими функциями одного параметра t . Явные формы $x = \zeta(y)$ или $y = \psi(x)$ всегда можно параметризовать как $\{x(t) = \zeta(t), y(t) = t\}$ или $\{x(t) = t, y(t) = \psi(t)\}$.

Часто для поиска подстановок удобно использовать известные тождества $\cos^2(t) + \sin^2(t) = 1$ и $\text{ch}^2(t) - \text{sh}^2(t) = 1$. К примеру, параметризации одной из простейших линий — центрированной в начале координат **окружности** единичного радиуса с каноническим уравнением $x^2 + y^2 - 1 = 0$ — удовлетворяют восемь пар тригонометрических функций, описывающих гармонические колебания с произвольными частотой ω и фазовым углом φ :

$$\begin{aligned} \{x(t) = \pm \cos(\omega t + \varphi), y(t) = \pm \sin(\omega t + \varphi)\}, \\ \{x(t) = \pm \sin(\omega t + \varphi), y(t) = \pm \cos(\omega t + \varphi)\}. \end{aligned}$$

Выберем одну из этих пар для дальнейшего широкого использования и оформим в виде векторной функции

$$c(t) = [\cos(t) \ \sin(t)] \quad \forall t \in [0, 2\pi]. \quad (1.64)$$

Она описывает равномерное вращение точки вокруг начала координат с единичным радиусом в положительном направлении (против часовой стрелки) из начального положения $\{x(0) = 1, y(0) = 0\}$ и периодом обращения 2π секунд.

Составим подходящие **параметрические модели канонических линий второго порядка** из табл. 1.1:

- **действительного эллипса** (рис. 1.42, а):

$$p(t) = [a \cos(t) \ b \sin(t)] \quad \forall t \in [0, 2\pi]; \quad (1.65a)$$

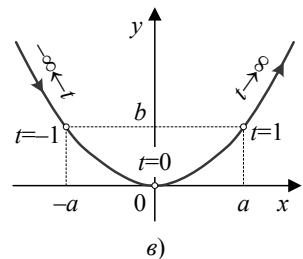
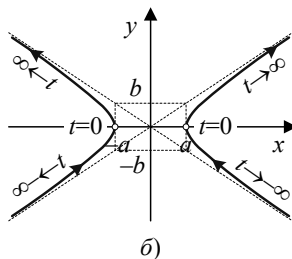
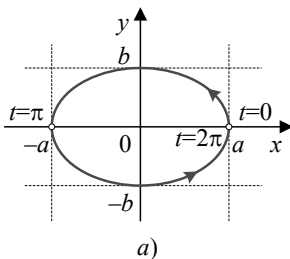


Рис. 1.42

- правой (+) и левой (–) ветвей **гиперболы** (рис. 1.42,б):

$$p(t)=[\pm \operatorname{ach}(t) \operatorname{bsh}(t)] \quad \forall t \in (-\infty, \infty); \quad (1.65б)$$

- **параболы** (рис. 1.42,в):

$$p(t)=[at \ bt^2] \quad \forall t \in (-\infty, \infty). \quad (1.65в)$$

2. **Преобразованием полярной модели** $r=r(\varphi)$ с помощью соотношений (1.2) между полярными и декартовыми координатами:

$$p(\varphi)=r(\varphi)[\cos(\varphi) \ \sin(\varphi)]. \quad (1.66а)$$

Здесь угол φ откладывается в положительном направлении (против часовой стрелки) от оси x . Если же откладывать угол вправо от оси y , как, например, ψ на рис. 1.1,а, то вектор декартовых координат точки $r(\psi)$ должен вычисляться по формуле

$$p(\psi)=r(\psi)[\sin(\psi) \ \cos(\psi)]. \quad (1.66б)$$

Полярными моделями хорошо описываются линии, **равномерно** обходящие начало координат с функциональным изменением радиуса r от угла φ . Постоянному радиусу $r=1$ соответствует окружность (1.64). Приведём примеры некоторых полярных линий, полезных в практических приложениях:

- разнообразные **спирали** с монотонным изменением радиуса $r(\varphi)$ по формулам, в которых коэффициенты $r_0=r(0)$ и r_1 задают начальный радиус и характер его изменения за один оборот — приращение угла φ на 2π (рис. 1.43):

◇ спираль **Архимеда** (а) с линейно растущим радиусом $r(\varphi)=r_0+r_1\varphi$;

◇ спираль **Бернулли** (б) с экспоненциальной функцией $r(\varphi)=r_0 e^{\eta\varphi}$;

◇ **гиперболическую** спираль (в) с обратной зависимостью $r(\varphi)=a/\varphi$;

◇ **эвольвенту окружности** (г) — траекторию движения конца нити, намотанной на окружность, в процессе её разматывания при $r(\varphi)=r_0\sqrt{1+\varphi^2}$;

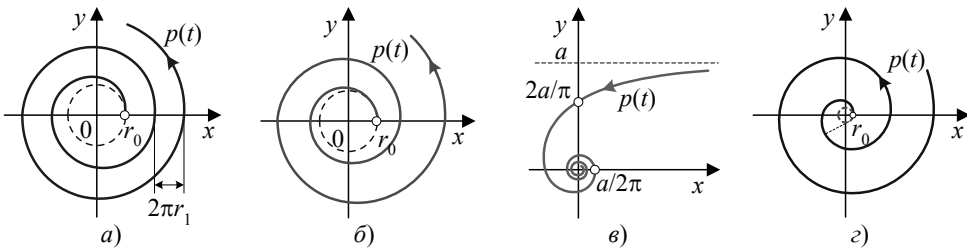


Рис. 1.43

- гармоническое изменение радиуса вращения по законам $r(\varphi)=a\pm b\cos(n\varphi)$ или $r(\varphi)=a\pm b\sin(n\varphi)$ в диапазоне $r\in[a-b, a+b]$ с n циклами колебаний за один оборот вокруг начала координат даёт линии, некоторые из которых показаны на рис. 1.44 (читателю предлагается самостоятельно подобрать для них подходящую гармоническую функцию $r(\varphi)$ или $r(\psi)$): (а) при $a\gg b>0$ и $n=3$, (б) при $a>b>0$ и $n=7$, (в) при $0<a<b$ и $n=4$, а при $a\geq b>0$ и $n=1$ получается **улитка Паскаля** и её частный случай **кардиоида** (г) при $a=b$;

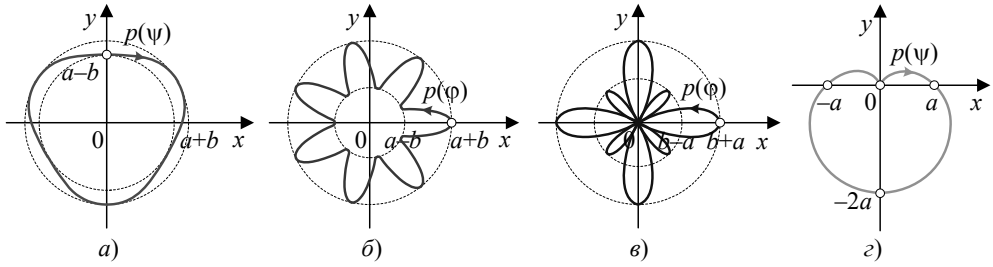


Рис. 1.44

• модульные функции $r(\varphi) = a \pm b|\cos(n\varphi)|^m$ или $r(\varphi) = a \pm b|\sin(n\varphi)|^m$ подвергают радиус вращения точки $p(\varphi)$ **негладким** колебаниям с двойной частотой $2n$ и обострёнными экстремумами при $0 < m < 1$ или $m > 1$. Некоторые полярные линии с такими эффектами модуляции построены на рис. 1.45 (подберите для них подходящие модульно-гармонические функции $r(\varphi)$ или $r(\psi)$): (а) **цветок** при $a > 0$, $b > 0$, $n=3$ и $m=1$; (б) **звезда** при $a > b > 0$, $n=2.5$ и $m=0.5$; (в) **вентилятор** при $a > b > 0$, $n=1.5$ и $m=10$; (з) **яйцо** при $b=2a$, $n=0.5$ и $m=7$;

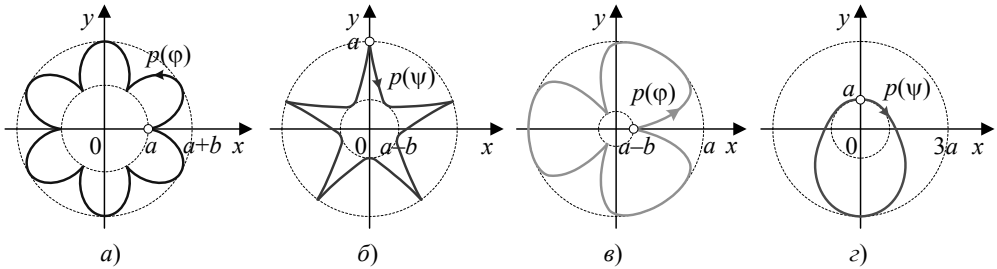


Рис. 1.45

• множество полярных линий создаётся с помощью **нелинейных** и **немонотонных** зависимостей угла $\varphi(t)$ от свободного параметра t (рис. 1.46).

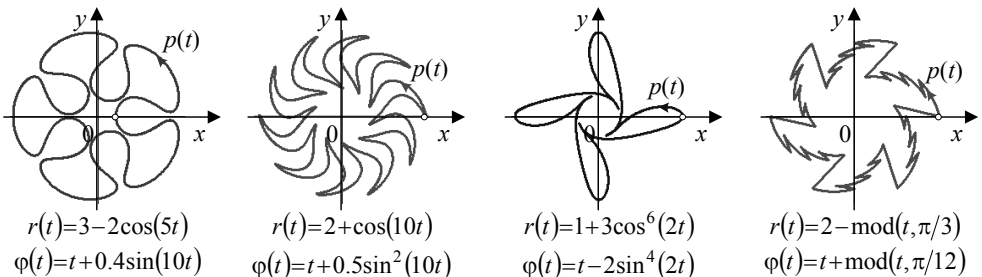


Рис. 1.46

3. **Суперпозицией** (наложением) **одновременных движений точки вдоль координатных осей** из заданного начального положения $\{x(0), y(0)\}$. Например, сложение осевых гармонических колебаний $\cos(t)x^0$ и $\sin(t)y^0$, записываемое как векторная функция (1.64), даёт при $t=0, 2\pi$ вращение точки по окружности из начального положения $x(0)=1$, $y(0)=0$ (рис. 1.47,а). Направление вычисленного по формуле (1.8б) вектора скорости

$$V(t) = [-\sin(t) \cos(t)],$$

лево-ортогонального радиус-вектору $c(t)$, соответствует положительному направлению вращения.

Фазовый сдвиг одного из колебаний, например, вдоль оси y , на угол φ даёт в суперпозиции $x(t) = \cos(t)$ и $y(t) = \sin(t + \varphi)$ эллипс с

ортогональными полуосями длиной $a = \sqrt{1 + \sin(\varphi)}$ и $b = \sqrt{1 - \sin(\varphi)}$, наклонёнными на углы $\pm 45^\circ$ (рис. 1.47, б).

Вращательное движение лежит в основе построения и более сложных линий: рассмотренных выше полярных, фигур Лиссажу и циклических линий:

• **фигуры Лиссажу** образуются суперпозицией **разночастотных** колебаний $x(t) = \cos(\omega_x t)$ и $y(t) = \sin(\omega_y t)$ вдоль ортов СК:

$$p(t) = [\cos(\omega_x t) \sin(\omega_y t)].$$

Одна из таких фигур с некратными частотами $\omega_x = 4$, $\omega_y = 7$ и двумя точками возврата построена на рис. 1.48, а в интервале изменения параметра $t \in [0, 2\pi]$. При кратных частотах, например, $\omega_x = 4$ и $\omega_y = 12$, получаются гладкие замкнутые фигуры Лиссажу (рис. 1.48, б);

• простейшие **циклические линии** описываются параметрически как

$$p(t) = Rc(t) + rc(\omega t), \quad (1.67)$$

где R — радиус окружности, по которой равномерно вращается центр второй окружности радиуса r , по которой одновременно вращается точка с угловой скоростью ω . На рис. 1.48, в построена циклическая линия при $R=3$, $r=2$ и $\omega=3.5$ в интервале изменения параметра $t \in [0, 4\pi]$.

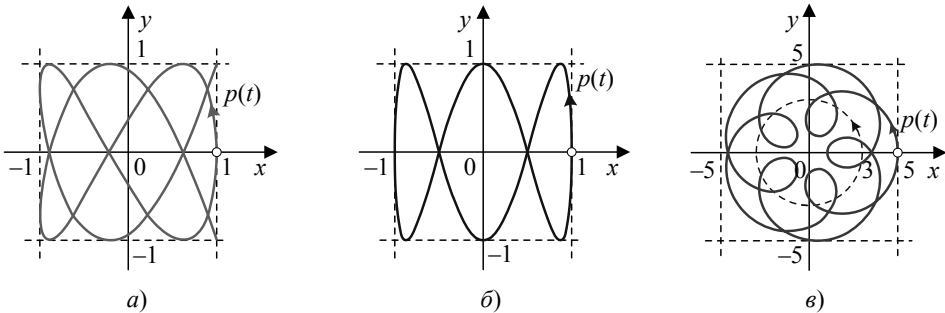


Рис. 1.48

Дальнейшее изучение плоских и пространственных циклических линий будет продолжено в гл. 3, где мы познакомимся с получением сложных параметрических линий кинематическим методом геометрических преобразований.

1.1.9. Геометрические преобразования неявных линий

Неявные модели линий в виде уравнений с векторным $f(p)=0$ либо скалярными $f(x,y)=0$ переменными востребованы во многих задачах компьютерной графики. К ним, несомненно, относится расчёт пересечения неявной линии с прямой $q+U\theta$, сводящийся к решению **скалярного** уравнения $f(q+U\theta)=0$ относительно параметра θ . При параметрическом же описании линии векторной функцией $p(t)$ приходится решать более сложное векторное нелинейное уравнение $p(t)=q+U\theta$ относительно двух неизвестных параметров t и θ .

Другое ценное свойство неявной формы **самонепересекающейся** линии заключается в простоте проверки ориентации относительно неё некоторой точки q . Индикатором расположения этой точки на линии служит значение $f(q)=0$. Для двух точек a и b знак произведения $f(a)f(b)$ сигнализирует об их одностороннем (+) либо разностороннем (–) расположении.

Выбор неявного описания линий весьма ограничен. Без дополнительных трудоёмких изысканий быстро составляются координатные функции $f(x,y)$, пожалуй, лишь для прямых (1.11), (1.14а) и канонических форм линий второго порядка из табл. 1.1. Расширим возможности получения неявных функций линий $\phi(x,y)$ **геометрическими преобразованиями** некоторых исходных графических объектов, также заданных в неявной форме $f(x,y)=0$. Формирование функции ϕ производится **методом обратного преобразования** аргументов функции f . Конкретнее, последовательность действий с координатами исходной линии $\{x, y\}$ и преобразованной линии $\{x', y'\}$ такова:

- составляются уравнения прямого преобразования $x'=\xi(x,y)$ и $y'=\psi(x,y)$;
- данная система разрешается относительно исходных координат, реализуя **обратное преобразование** $x=\xi'(x',y')$ и $y=\psi'(x',y')$;
- подставив полученные зависимости в $f(x,y)$, получим преобразованную НФ $\phi(x',y')=f(\xi'(x',y'),\psi'(x',y'))$, равносильную

$$\phi(x,y)=f(\xi'(x,y),\psi'(x,y)).$$

Примеры элементарных преобразований в декартовой СК на плоскости:

- параллельный **перенос** всех точек линии на вектор $B=[b_x \ b_y]$ даёт новые координаты $x'=x+b_x$, $y'=y+b_y$ (рис. 1.49,а) и неявную функцию

$$\phi(x,y)=f(x-b_x,y-b_y); \quad (1.68a)$$

- **масштабирование** (растяжение, сжатие, отражение) всех точек линии вдоль координатных осей с ненулевыми коэффициентами m_x и m_y даёт преобразованные координаты $x'=m_x x$, $y'=m_y y$ (рис. 1.49,б) и новую функцию

$$\phi(x,y)=f(x/m_x,y/m_y); \quad (1.68б)$$

- при **вращении** линии вокруг начала СК на угол φ (рис. 1.49,в) координаты точки $[x \ y]$ изменяются по формулам

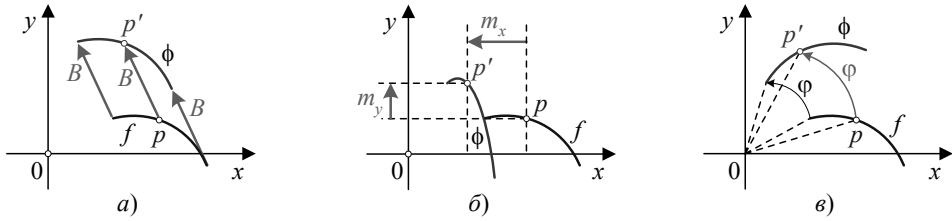


Рис. 1.49

$$x' = x \cos(\varphi) - y \sin(\varphi), \quad y' = x \sin(\varphi) + y \cos(\varphi),$$

а неявная функция преобразуется в

$$\phi(x, y) = f(x \cos(\varphi) + y \sin(\varphi), -x \sin(\varphi) + y \cos(\varphi)). \quad (1.68\text{в})$$

Если над линией $f(x, y) = 0$ совершается последовательность (будем называть её **цепочкой**) преобразований, каждое из которых изменяет её неявную функцию, то конечная НФ $\phi(x, y)$ находится **методом суперпозиции** (наложения). Возьмём, например, центрированную окружность единичного радиуса с НФ $x^2 + y^2 - 1 = 0$ и выполним следующую цепочку преобразований (рис. 1.50):

- **растяжение** вдоль оси x с масштабными коэффициентами $m_x = 2$ и $m_y = 1$ даёт по (1.68б) неявное уравнение $0.25x^2 + y^2 - 1 = 0$;

- **вращение** полученного эллипса на угол $\varphi = 45^\circ$ изменяет неявную функцию линии согласно (1.68в):

$$0.25 \left(\frac{x}{\sqrt{2}} + \frac{y}{\sqrt{2}} \right)^2 + \left(-\frac{x}{\sqrt{2}} + \frac{y}{\sqrt{2}} \right)^2 - 1 = \frac{5}{8}x^2 + \frac{5}{8}y^2 - \frac{3}{4}xy - 1;$$

- **перенос** вдоль оси x на расстояние $b_x = 1$ даёт по (1.68а) конечную НФ

$$\phi(x, y, z) = \frac{5}{8}(x-1)^2 - \frac{3(x-1)y}{4} + \frac{5}{8}y^2 - 1 = \frac{5}{8}x^2 + \frac{5}{8}y^2 - \frac{3}{4}xy - \frac{5}{4}x + \frac{3}{4}y - \frac{3}{8}.$$

Проверка $\phi(\sqrt{2}+1, \sqrt{2}) = 0$ доказывает, что точка $q = [\sqrt{2}+1, \sqrt{2}]$ действительно лежит на линии конечного эллипса, что ясно видно на последнем графике.

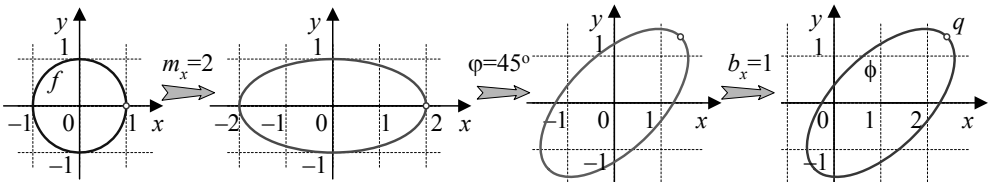


Рис. 1.50

Аналогичным преобразованиям могут быть подвергнуты линии, описываемые явными координатными функциями, например, $y = \psi(x)$. Для этого достаточно составить неявную функцию $f(x, y) = \psi(x) - y$ и следовать правилам (1.68) получения НФ преобразованной линии $\phi(x, y)$.

1.2. СОПРЯЖЕНИЕ НА ПЛОСКОСТИ

Рассмотрим ряд задач конструирования линий, составленных из гладко сопряжённых отрезков прямых и дуг окружностей. Такие геометрические конструкции, изучаемые со времён Древней Греции, широко применяются:

- в разнообразных графических программах для построения составных линий типа «отрезок → дуга → дуга → отрезок → дуга...». Такими линиями рисуют эскизные схемы различных механизмов, контуры деталей, блок-схемы алгоритмов, планы местностей и помещений, символы векторных шрифтов;
- в программах проектирования микросхем и печатных плат на этапе размещения электронных компонент и проводников тока с учётом их тепловыделения, помехоустойчивости и минимально допустимых технологических зазоров;
- в машиностроительных САПР при расчёте траекторий движения обрабатывающих инструментов с учётом дальнейшей механической обработки изделий и изображении контуров деталей после чистовой обработки — шлифования, фрезерования, скругления углов и т. п.;
- при раскрое листовых материалов с учётом припусков на швы, сгибы и чистовую обработку деталей;
- в архитектурном проектировании для скругления углов контуров зданий, окружения строений санитарными зонами, плавного сопряжения линий коммуникаций, построения гладких охватывающих оболочек и т. п.

1.2.1. Задачи общего назначения

Многие задачи на пересечение и сопряжение плоских графических объектов основаны на «школьных» построениях с помощью линейки и циркуля [7]. Тот результат, что достаточно просто достигается вручную, может быть получен и аналитически путём составления геометрических моделей объектов и решения необходимых уравнений. Этому и будет посвящён наш краткий экскурс в компьютерное решение задач на геометрические построения.

□ Первыми рассмотрим **три школьные задачи построения треугольника ABC** по необходимому и достаточному числу элементов (рис. 1.51). Общим элементом всех задач возьмём первую сторону AB , заданную длиной $c = |A-B|$ и углом направления $\varphi = \angle(x^0, AB)$. Тогда, задав положение точки A , вычислим вторую вершину треугольника

$$B = A + c[\cos(\varphi) \sin(\varphi)].$$

В каждой задаче существуют два положения третьей вершины C , симметричные относительно стороны AB : левое C_1 и правое C_2 . Они находятся методом, соответствующим заданию двух других элементов треугольника:

- если заданы длина второй стороны $b = |A-C|$ и угол $\alpha = \angle BAC$ между первой и второй сторонами (рис. 1.51, а), то

$$C_1 = A + b[\cos(\varphi + \alpha) \sin(\varphi + \alpha)], \quad C_2 = A + b[\cos(\varphi - \alpha) \sin(\varphi - \alpha)];$$

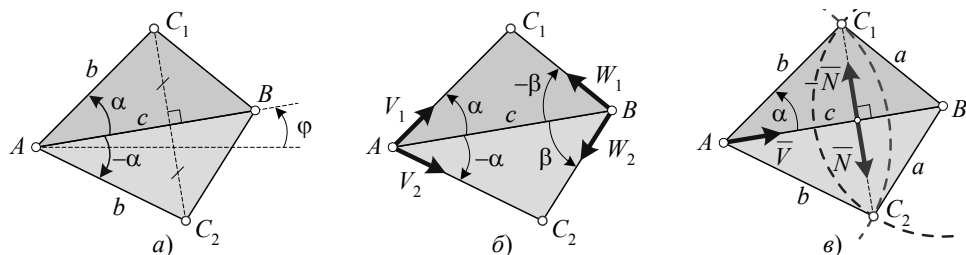


Рис. 1.51

• если заданы примыкающие к стороне AB углы $\alpha = \angle BAC$ и $\beta = \angle ABC$, то вершина C лежит в рассчитанной по (1.31б) точке пересечения параметрических **лучей** $A+Vt$ и $B+W\tau$ с направляющими векторами V и W (рис. 1.51,б):

$$V_1 = [\cos(\varphi + \alpha) \sin(\varphi + \alpha)], \quad W_1 = -[\cos(\varphi - \beta) \sin(\varphi - \beta)] \Rightarrow \{t_1, \tau_1\} \Rightarrow C_1 = A + V_1 t_1;$$

$$V_2 = [\cos(\varphi - \alpha) \sin(\varphi - \alpha)], \quad W_2 = -[\cos(\varphi + \beta) \sin(\varphi + \beta)] \Rightarrow \{t_2, \tau_2\} \Rightarrow C_2 = A + V_2 t_2.$$

Условие существования решения $\{t > 0\} \wedge \{\tau > 0\}$ выполняется при однозначности углов, т. е. при $\alpha > 0$, и модуле их суммы $|\alpha + \beta| < 180^\circ$;

• если кроме первой стороны в треугольнике заданы длины двух других сторон $a = |B - C|$ и $b = |A - C|$, то по теореме косинусов

$$a^2 = b^2 + c^2 - 2bc \cos(\alpha), \quad b^2 = a^2 + c^2 - 2accos(\beta)$$

вычисляются углы

$$\alpha = \arccos((b^2 + c^2 - a^2)/2bc) > 0, \quad \beta = \arccos((a^2 + c^2 - b^2)/2ac) > 0,$$

после чего данная задача сводится к предыдущей.

Возможно решение и без расчёта углов и пересечений лучей (рис. 1.51,в). Вычислив значение $\cos(\alpha) = (b^2 + c^2 - a^2)/2bc$ и отложив от точки A в направлении $V = B - A$ расстояние $b \cos(\alpha)$, а в нормальных направлениях $N = R \text{rot}(V)$ и $-N$ расстояния $b \sin(\alpha)$, где $\sin(\alpha) = \sqrt{1 - \cos^2(\alpha)}$, приходим в точки C_1 и C_2 .

С учётом сказанного составим **функцию решения треугольника**

$$\text{triang}(A, B, a, b) = A + bd\bar{V} + b\sqrt{1 - d^2} \bar{N}, \quad (1.69a)$$

$$\text{где } V = B - A, \quad \bar{N} = R \text{rot}(\bar{V}), \quad c = |V|, \quad d = (b^2 + c^2 - a^2)/2bc.$$

Она возвращает точку C_2 , расположенную **справа** от отрезка AB на расстояниях b от точки A и a от точки B . Симметричная левая вершина C_1 может быть получена по (1.25a) зеркальным отражением $C_1 = C_2 - 2((C_2 - A) \circ \bar{N}) \bar{N}$, либо с помощью функции (1.69a) как $C_1 = \text{triang}(B, A, b, a)$, либо $C_1 = \text{triang}(A, B, a, -b)$.

Условие существования решения $|d| < 1$ эквивалентно системе неравенств

$$\{a + b > c\} \wedge \{a + c > b\} \wedge \{b + c > a\}. \quad (1.69б)$$

Если она совместна, то существует **два** симметричных решения задачи треугольника ABC_1 и ABC_2 . При $|d| = 1$ сумма длин двух сторон равна длине третьей стороны, двойная точка $C_1 = C_2$ лежит на прямой AB , а треугольник вырожден в отрезок. Наконец, при $|d| > 1$ задача не имеет действительного решения.

□ Вторая общая задача расчёта **пересечения двух окружностей** $\{o_1, r_1\}$ и $\{o_2, r_2\}$ с центрами o_1, o_2 и радиусами r_1, r_2 сводится к третьей задаче треугольника (рис. 1.51,б) и при условии (1.69б) с $a=r_2, b=r_1$ и $c=|o_1-o_2|$ имеет решения

$$C_1 = \text{triang}(o_2, o_1, r_1, r_2), \quad C_2 = \text{triang}(o_1, o_2, r_2, r_1).$$

□ Третья общая задача — **построение дуги окружности** (рис. 1.52). Как и любая кривая, она создаётся в виде полилинии $D=d_0 d_1 \dots d_m$ с такой плотностью

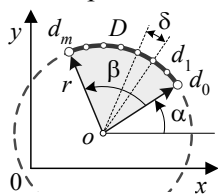


Рис. 1.52

размещения вершин, чтобы, с одной стороны, не были заметны изломы на стыках рёбер, а с другой — чтобы смежные слишком близкие вершины не попали в один и тот же пиксел экрана. Визуальный опыт показывает, что дуга окружности выглядит практически кривой, когда угловой шаг между смежными вершинами полилинии D заключен в интервале $\Delta \in [1^\circ, 5^\circ]$.

В параметрической модели единичной центрированной окружности (1.64) параметр $t \in [0, 2\pi]$ играет роль угла вращения радиус-вектора $c(t)$, отсчитываемого от оси x (см. рис. 1.47,а). Для вывода произвольной дуги окружности нужно сделать ряд преобразований примитива (1.64):

- изменение радиуса до значения r делается масштабированием $rc(t)$;
- смещение центра окружности в точку o реализуется переносом $rc(t)+o$;
- интервал изменения параметра дуги $t \in [\alpha, \gamma]$ можно, казалось бы, задать

начальным углом α и конечным углом γ . Это, однако, не рационально, т. к. потребуется задать ещё третий параметр выбора одной из двух дуг, составляющих полную окружность. Оптимальным является набор всего **двух** угловых параметров дуги: α и **угла раствора** β — любых чисел произвольных знаков;

• если угол β не кратен **рекомендуемому** угловому шагу Δ , то нужен такой **расчётный** шаг δ , чтобы число шагов $m = |\beta|/\delta \geq 1$ было целым и минимальным.

Составим **алгоритм** $\text{arc}(o, r, \alpha, \beta)$ расчёта дуги окружности с центром в точке o , радиусом r , начальным углом α и углом раствора β , возвращающий полилинию $d_0 d_1 \dots d_m$, вершины которой равномерно распределены по дуге с максимальным угловым шагом $|\delta| \leq \Delta$. Для универсальности в начало алгоритма добавлено возвращение одной точки o при $r=0$ либо d_0 при $\beta=0$.

☞ $\text{arc}(o, r, \alpha, \beta)$

{если $r\beta=0$, то **возврат** $o+r[\cos(\alpha) \sin(\alpha)]$; // вырожденная в точку дуга

$m = \lceil |\beta|/\Delta \rceil$, $\delta = \beta/m$; // число шагов и угловой шаг

$\forall i = \overline{0, m}$

// цикл расчёта узловых точек дуги

{ $t = \alpha + i\delta$;

// угол радиус-вектора

$d_i = o + r[\cos(t) \sin(t)]$;

// узловая точка полилинии

возврат $d_0 d_1 \dots d_m$;

// полилиния дуги

} ☐

1.2.2. Сопряжения с окружностями

Изучим геометрию гладкого сопряжения разных объектов с окружностями.

□ Начнём с задачи проведения из точки q **касательной прямой к окружности** $\{o, r\}$ с центром в точке o и радиусом r (рис. 1.53). Нужно найти на окружности точки p , где векторы $p-o$ и $p-q$ ортогональны.

Вычислив расстояния $d=|q-o|$ и $h=\sqrt{d^2-r^2}$, сведём расчёт точек касания к построению двух прямоугольных треугольников с гипотенузой oq и длинами катетов r и h :

$$p_{1,2} = \text{triang}(o, q, \sqrt{|q-o|^2 - r^2}, \pm r). \quad (1.70)$$

Условие существования решения $d \geq r$ означает, что **точка q не должна лежать внутри окружности $\{o, r\}$** . Из внешней точки можно провести **два** касательных отрезка в точки p_1 и p_2 , зеркально симметричные относительно отрезка qo . При $d=r$ точка q лежит на окружности, отрезок qp вырождается в точку $p=q$, а у **единственной** касательной линии $q+V_{\text{кас}}t$ направляющий вектор $V_{\text{кас}} = Lrot(N_{\text{кас}})$ находится вращением по (1.9) вектора её нормали $N_{\text{кас}} = q-o$.

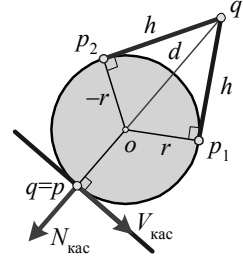


Рис. 1.53

□ Вторая задача **сопряжения двух окружностей** $\{o_1, r_1\}$ и $\{o_2, r_2\}$ **касательной прямой** pq разрешима при расстоянии между центрами $d=|o_2-o_1| \geq |r_1-r_2|$. Если $d > r_1+r_2$, то окружности не пересекаются и существует **четыре** касательных отрезка $p_1q_1 \div p_4q_4$ (рис. 1.54,а), получаемых на основе решений (1.70):

- построив из центра **меньшей** окружности o_2 (при $r_1 < r_2$ следует обменять параметры окружностей) касательные отрезки $o_2p'_1$ и $o_2p'_2$ к окружности $\{o_1, r_1-r_2\}$, раздвигаем их на расстояние r_2 — первый отрезок по нормали $N_1 = Rrot(\overline{o_2-p'_1})$, а второй по нормали $N_2 = Lrot(\overline{o_2-p'_2})$ (рис. 1.54,б):

$$p_1 = p'_1 + r_2 N_1, \quad q_1 = o_2 + r_2 N_1, \quad p_2 = p'_2 + r_2 N_2, \quad q_2 = o_2 + r_2 N_2; \quad (1.71a)$$

- построив из точки o_2 касательные отрезки $o_2p'_3$ и $o_2p'_4$ к окружности $\{o_1, r_1+r_2\}$, сдвигаем их на расстояние r_2 — первый по нормали $N_3 = Lrot(\overline{o_2-p'_3})$, а второй по нормали $N_4 = Rrot(\overline{o_2-p'_4})$ (рис. 1.54,в):

$$p_3 = p'_3 + r_2 N_3, \quad q_3 = o_2 + r_2 N_3, \quad p_4 = p'_4 + r_2 N_4, \quad q_4 = o_2 + r_2 N_4. \quad (1.71б)$$

При $d = r_1 + r_2$ существует **два** касательных отрезка p_1q_1 , p_2q_2 и **одна** касательная прямая $p_3+V_{\text{кас}}t$, где $p_3 = (r_1o_2 + r_2o_1)/d$, $V_{\text{кас}} = Lrot(o_2-o_1)$ (рис. 1.54,г).

Если $|r_1-r_2| < d < r_1+r_2$, то к пересекающимся окружностям можно построить только **два** касательных отрезка p_1q_1 и p_2q_2 (рис. 1.54,д).

При $d = |r_1-r_2| > 0$ есть только **одна** прямая внутреннего касания $p_1+V_{\text{кас}}t$, где $p_1 = (r_1o_2 - r_2o_1)/(r_1-r_2)$ и $V_{\text{кас}} = Lrot(o_2-o_1)$ (рис. 1.54,е).

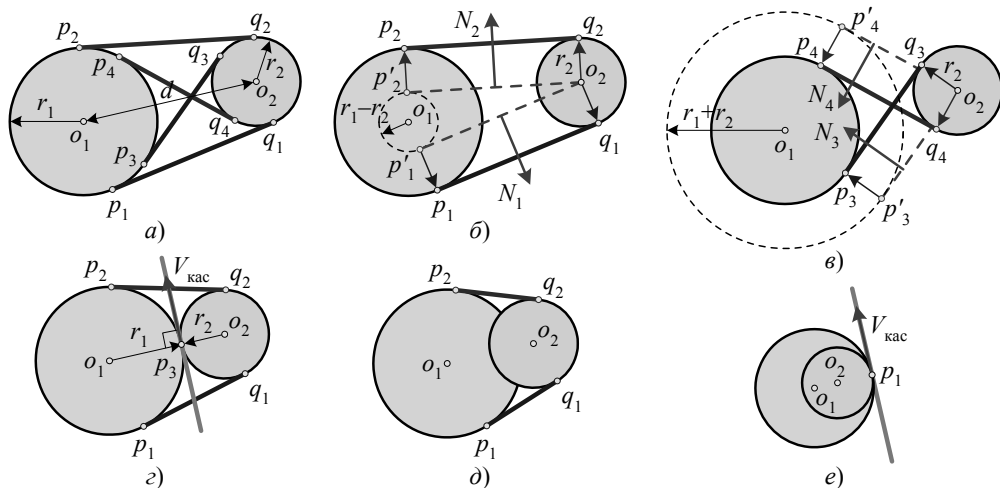


Рис. 1.54

Наконец, при $d < |r_1 - r_2|$ (меньшая окружность находится внутри большей) либо $d = |r_1 - r_2| = 0$ (окружности совпадают) сопряжение отсутствует.

В следующих задачах сопрягающим элементом является **дуга окружности**.

□ Первой идёт задача **сопряжения двух окружностей** $\{c_1, r_1\}$ и $\{c_2, r_2\}$ **дугой** радиуса r . Пусть $d = |c_1 - c_2|$ — расстояние между центрами окружностей. В одном из вариантов сопряжения **внешней вогнутой** дугой (рис. 1.55,а) нужно найти такое положение центра дуги o , чтобы расстояния от него до центров окружностей $|o - c_1|$ и $|o - c_2|$ составляли величины $d_1 = r_1 + r$ и $d_2 = r_2 + r$.

Рассмотрим различные расположения окружностей:

- при $|d_1 - d_2| < d < d_1 + d_2$ существует **два** центра внешних сопрягающих дуг o_1 и o_2 , получаемых аналогично (1.70):

$$o_i = \text{triang}(c_1, c_2, r_2 + r, (-1)^{i-1}(r_1 + r)) \quad \forall i = \overline{1, 2}. \quad (1.72a)$$

Вычислив угловые параметры сопрягающих дуг

$$\alpha_i = \arg(c_1 - o_i), \quad \beta_i = \arg(c_2 - o_i, c_2 - o_i), \quad (1.72б)$$

получим полилинии сопрягающих дуг:

$$D_i = \text{arc}(o_i, r, \alpha_i, \beta_i); \quad (1.72в)$$

- при $d = d_1 + d_2$ кратчайшее расстояние между окружностями $d - r_1 - r_2 = 2r$, а **одна** сопрягающая полуокружность имеет центр $o = (d_1 c_2 + d_2 c_1) / d \in c_1 c_2$;
- при $d > d_1 + d_2$ сопряжение невозможно, т. к. в минимальный промежуток между окружностями $d - r_1 - r_2 > 2r$ окружность дуги проходит с зазором;
- при $d \leq |d_1 - d_2|$ меньшая окружность лежит внутри большей, а любое внешнее сопряжение отсутствует.

Возможны и другие варианты сопряжения окружностей дугами, кроме рассмотренного внешнего вогнутого. Так, из внешних **выпуклых** гладко сопряжён-

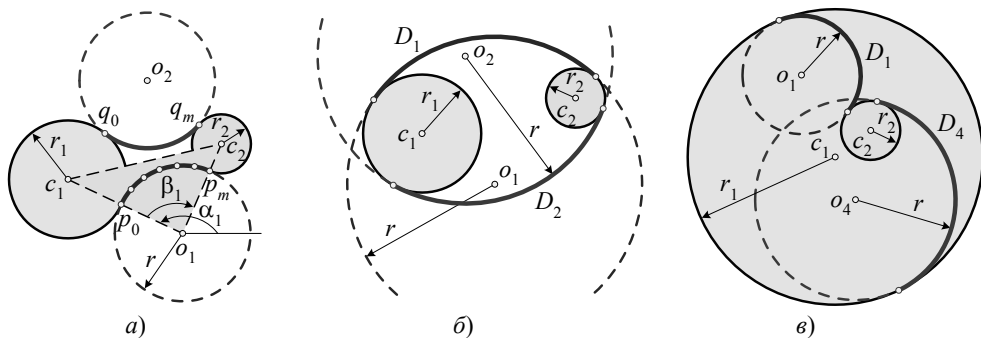


Рис. 1.55

ных дуг строятся фигуры овальной формы (рис. 1.55, б), а при нахождении одной окружности внутри другой возможны две пары **внутренних** сопряжений двух видов (рис. 1.55, в). Читателю предлагается самостоятельно получить условия существования и алгоритмы построения всех вариантов таких дуг.

□ Вторая задача заключается в **сопряжении окружности** $\{o, R\}$ с **прямой** $\{q, V\}$ **внешней дугой** радиуса r . Из геометрических построений на рис. 1.56, а видно, что центр сопрягающей окружности лежит на пересечении **внешней** вспомогательной окружности $\{o, R+r\}$ с любой из вспомогательных прямых $\{h_1, V\}$ или $\{h_2, V\}$, проходящих на расстоянии r параллельно и по разные стороны от прямой $\{q, V\}$ [7]. Вычислив нормированную нормаль $N = R \text{rot}(\bar{V})$, получим точки $h_1 = q + rN$ и $h_2 = q - rN$.

Задача пересечения прямой $\{h, \bar{V}\}$ с окружностью $\{o, R+r\}$ решается аналогично (1.59) подстановкой в неявное уравнение $(p-o) \circ (p-o) - (R+r)^2 = 0$ векторной функции $p(t) = h + \bar{V}t$ и сводится к решению квадратного уравнения (1.60а) со следующими коэффициентами (1.62):

$$a = \bar{V} \circ \bar{V} = 1, \quad b = (h-o) \circ \bar{V}, \quad c = (h-o) \circ (h-o) - (R+r)^2. \quad (1.73a)$$

При $b^2 > c$ будут две разные, а при $b^2 = c$ — совпадающие точки

$$p_{1,2} = h - (b \pm \sqrt{b^2 - c}) \bar{V}. \quad (1.73б)$$

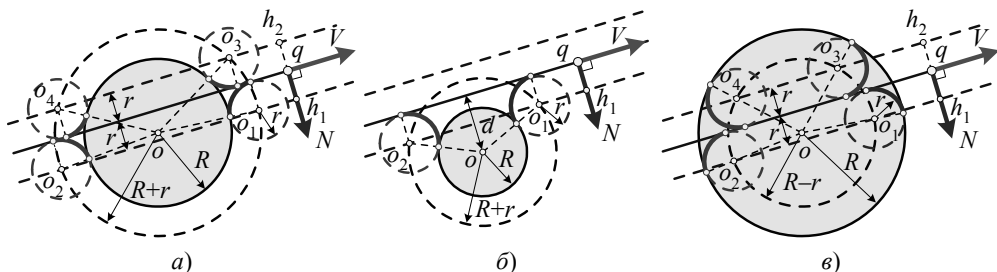


Рис. 1.56

В зависимости от расстояния $d = |(q-o) \circ N|$ между точкой o и прямой $\{q, V\}$ число k сопрягающих дуг равно числу пересечений вспомогательной окружности $\{o, R+r\}$ с прямыми $\{h_1, V\}$ и $\{h_2, V\}$: четырём при $d < R$, двум при $R \leq d \leq R+2r$ (рис. 1.56,б) либо нулю при $d > R+2r$. Получив по (1.73) центр i -й ($i \in [1, k]$) сопрягающей окружности o_i , вычислим углы дуги $\alpha_i = \arg(o - o_i)$, $\beta_i = \arg(o - o_i, ((q - o_i) \circ N)N)$ и, наконец, полилинию $D_i = \text{arc}(o_i, r, \alpha_i, \beta_i)$.

Читателю предлагается самостоятельно решить задачу **внутреннего** сопряжения прямой $\{q, V\}$ с окружностью $\{o, R\}$ дугами радиуса $r < R$, центры которых находятся в точках пересечения внутренней окружности $\{o, R-r\}$ с параллельными прямыми $\{h_1, V\}$ или $\{h_2, V\}$ (рис. 1.56,в).

1.2.3. Скругление углов

Рассмотрим задачу **скругления угла** $\angle abc$ дугой окружности радиуса r . Из геометрических построений на рис. 1.57,а видно, что скруглённый угол представляет собой линию aDc , составленную из трёх сегментов — отрезка ad_0 , дуги окружности $D = d_0d_1 \dots d_m$ и отрезка $d_m c$. Центр скругляющей дуги o лежит на биссектрисе угла (1.36б) на расстоянии $r/\cos(\varphi)$ от точки b . Угол φ составляет половину угла между векторами $V = \overrightarrow{b-a}$ и $W = \overrightarrow{c-b}$.

Скругление угла будет корректным только при попадании начальной точки дуги d_0 на отрезок ab , а конечной точки d_m — на отрезок bc , иначе вся дуга или её часть выйдут из контура треугольника $abca$ (рис. 1.57,б). Для устранения этой ошибки, случающейся при скруглении углов либо очень острых, либо с короткими сторонами, нужно уменьшить радиус дуги r до такого значения, при котором точки d_0 и d_m удовлетворяют условиям $|d_0 - b| \leq \rho|a - b|$ и $|d_m - b| \leq \rho|c - b|$, где $0 \leq \rho \leq 1$ — задаваемый пользователем **коэффициент скругления** — максимально допустимая скругляемая часть стороны угла. К примеру, дуга на рис. 1.57,в построена при $\rho = 1$, а на рис. 1.57,г — при $\rho = 0.5$. **Значение $\rho = 0.5$ рекомендуется использовать для скругления последовательности углов в полигонах и полилиниях.** Оно обеспечивает гладкое сопряжение смежных дуг с максимальными радиусами, не превышающими значения r .

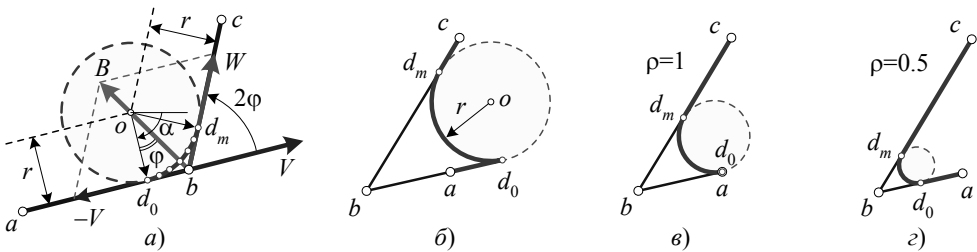


Рис. 1.57

Оформим сказанное в виде **алгоритма** $\text{round}(abc, r, \rho)$ **скругления угла** $\angle abc$ дугой максимального радиуса r с коэффициентом скругления $\rho \in [0, 1]$.

$\square$ $round(abc, r, \rho)$
 $\{V = \overline{b-a}, W = \overline{c-b}, \varphi = 0.5 \text{ang}(V, W);$ // векторы сторон угла
если $\rho \neq 0$, **то** **возврат** b ; // отсутствие дуги
 $r = \min\{r, \rho|a-b| \text{ctg}(|\varphi|), \rho|c-b| \text{ctg}(|\varphi|)\};$ // коррекция радиуса
 $o = b + r \cdot \sec(\varphi) \cdot \overline{W-V}, \alpha = \arg(V) - \text{sgn}(\varphi)\pi/2;$ // параметры дуги
возврат $arc(o, r, \alpha, 2\varphi);$ // полилиния дуги
 $\}$ $\square$

Таким образом, задача скругления угла $\angle abc$ решается формированием **логического** объединения $L = a \cup round(abc, r, \rho) \cup c$, исключающего кратные вершины в получаемой полилинии.

$\square$ В цикле последовательного объединения концевых точек и дуг смежных углов производится **скругление углов полилинии** $p_1 p_2 \dots p_n$ (рис. 1.58, а):

$$L = p_1 \cup round(p_1 p_2 p_3, r, \rho) \cup \dots \cup round(p_{n-2} p_{n-1} p_n, r, \rho) \cup p_n. \quad (1.74)$$

$\square$ Алгоритм $round(abc, r, \rho)$ органично вписывается и в задачу **скругления углов полигона** $P = p_1 p_2 \dots p_n p_1$ (рис. 1.58, б). Для получения желаемой составной линии достаточно обойти вершины $p_i \forall i=1, n$, сформировать объединение

$$S = \bigcup_{i=1}^n round(p_{i-1} p_i p_{i+1}, r, \rho), \text{ где } p_0 = p_n \text{ и } p_{n+1} \equiv p_1, \quad (1.75a)$$

и выполнить замыкание $S = S \cup S_1$.

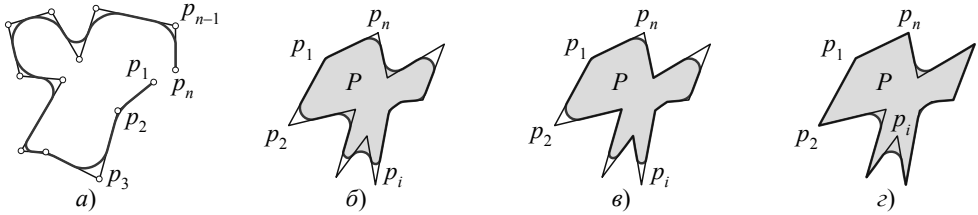


Рис. 1.58

Немного сложнее задача скругления только **выпуклых** углов полигона (рис. 1.58, в). Определив по (1.55) направление обхода вершин $d = \text{dir_test}(P)$, включим в объединение S нескруглённую вершину p_i , если угол $p_{i-1} p_i p_{i+1}$ невыпуклый, иначе дугу $round(p_{i-1} p_i p_{i+1}, r)$:

$$S = \bigcup_{i=1}^n \text{if}(d \cdot \text{nf} 2(p_{i-1}, p_i, p_{i+1}) \geq 0: p_i, round(p_{i-1} p_i p_{i+1}, r, \rho)), S = S \cup S_1. \quad (1.75б)$$

Если требуется сделать скругление только **вогнутых** углов полигона (рис. 1.58, г), то объединение элементов составной линии формируется как

$$S = \bigcup_{i=1}^n \text{if}(d \cdot \text{nf} 2(p_{i-1}, p_i, p_{i+1}) \leq 0: p_i, round(p_{i-1} p_i p_{i+1}, r, \rho)), S = S \cup S_1. \quad (1.75в)$$

Все варианты скругления (1.75) легко объединяются в один **алгоритм** $\text{round_poly}(P, r, \rho, \text{mod})$ **скругления углов полигона** $P=p_1p_2\dots p_n p_1$ с аргументом $\text{mod} \in \{-1, 0, 1\}$, задающим режим операции: -1 — скругление только вогнутых, 0 — всех и 1 — только выпуклых углов. При $\rho=0$ скругление отменяется.

1.2.4. Эквилидистантные линии

Линия, все точки которой отстоят от некоторой **базовой линии** на заданном расстоянии (**дистанте**) r , называется **эквилидистантой** [13]. Равноотстоящие линии широко используются в разных конструкторских задачах (некоторые из них перечислены в начале разд. 1.2). Алгоритмизация их построения позволяет автоматизировать этот процесс в системах интерактивного проектирования. Например, обнаружив тестом coll2_test (см. рис. 1.28) столкновение внешних эквидистантных контуров корпусов микросхем, можно заблокировать перемещение, нарушающее нормы минимально допустимых зазоров на печатной плате.

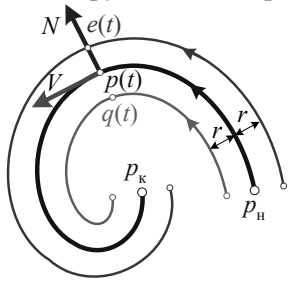


Рис. 1.59

□ У **эквилидистанты гладкой линии** $p(t)$, определённой в параметрическом интервале $t \in [t_n, t_k]$, есть правая $e(t)$ и левая $q(t)$ составляющие (рис. 1.59), получаемые откладыванием от каждой точки p расстояния r вдоль и, соответственно, против направления вектора правой нормали $N(t) = R\text{rot}(p'(t))$:

$$\begin{aligned} e(t) &= p(t) + r\bar{N}(t), \\ q(t) &= p(t) - r\bar{N}(t) = 2p(t) - e(t). \end{aligned} \quad (1.76)$$

Обозначив концевые точки базовой линии как $p_n = p(t_n)$ и $p_k = p(t_k)$, проанализируем вид получаемых эквидистант:

- если базовая линия **не замкнута**, т. е. $p_n \neq p_k$, то её полная эквидистанта $e(t) \cup D_k \cup q(t_n + t_k - t) \cup D_n \quad \forall t \in [t_n, t_k]$ состоит из линии $e(t)$, инверсной (в обратном направлении) линии $q(t_k + t_n - t)$ и соединяющих их дуг полуокружностей $D_n = \text{arc}(p_n, r, \arg(q(t_n) - p_n), \pi)$ и $D_k = \text{arc}(p_k, r, \arg(e(t_k) - p_k), \pi)$ (рис. 1.60, а);

- у гладкой **замкнутой** базовой линии с совпадающими концами $p_n = p_k$ и концевыми направлениями $p'_n = p'_k$ полная эквидистанта состоит из отдельных замкнутых линий $e(t)$ и $q(t)$ (рис. 1.60, б);

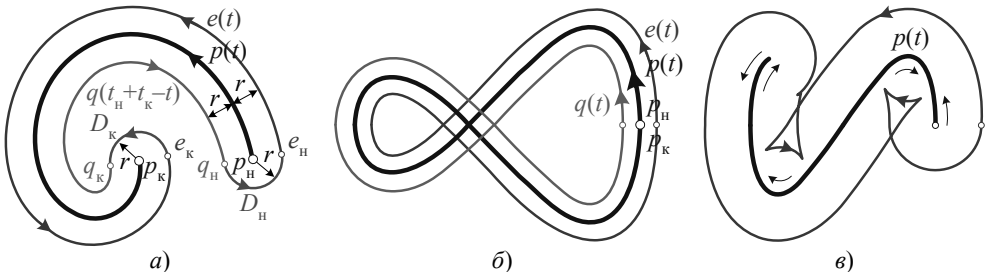


Рис. 1.60

• эквидистанта, проходящая с вогнутой стороны участка базовой линии с радиусами кривизны $\rho_k < r$, образует «петлю» с попятным движением (рис. 1.60,в), а при $\rho_k = r$ вырождается в особую точку.

□ Возьмём теперь замкнутую базовую полилинию $P = p_1 p_2 \dots p_n p_1$. Все точки **эквидистанты полигона**, состоящей из отрезков прямых и дуг окружностей, отстоят от всех базовых точек на кратчайшем расстоянии r (рис. 1.61,а):

• **внешняя** эквидистанта образуется скруглением **выпуклых** углов полигона $E = e_1 e_2 \dots e_n e_1$ по алгоритму $\text{round_poly}(E, r, \rho, 1)$;

• **внутренний** эквидистантный контур есть результат скругления **вогнутых** углов полигона $Q = q_1 q_2 \dots q_n q_1$ по алгоритму $\text{round_poly}(Q, r, \rho, -1)$.

Вершины скругляемых полигонов e_i и q_i лежат на биссектрисах углов $\angle p_{i-1} p_i p_{i+1}$ (по-прежнему $p_0 \equiv p_n$ и $p_{n+1} \equiv p_1$) и с учётом направления обхода полигона $d = \text{dir_test}(P) \in \{-1, 1\}$ находятся следующим образом:

• вычисляются векторы направлений сторон угла $V_i = \overline{p_{i+1} - p_i}$, **правых** биссектрис $B_i = R\text{rot}(V_{i-1} + V_i)$ и половины углов поворотов $\phi_i = 0.5 \text{ang}(V_{i-1}, V_i)$;

• при $d=1$ внешние точки e_i лежат на правых биссектрисах, внутренние точки q_i — на их левых продолжениях, а при $d=-1$ — всё наоборот:

$$e_i = p_i + d r \sec(\phi_i) \cdot \overline{B_i}, \quad q_i = p_i - d r \sec(\phi_i) \cdot \overline{B_i} = 2p_i - e_i. \quad (1.77)$$

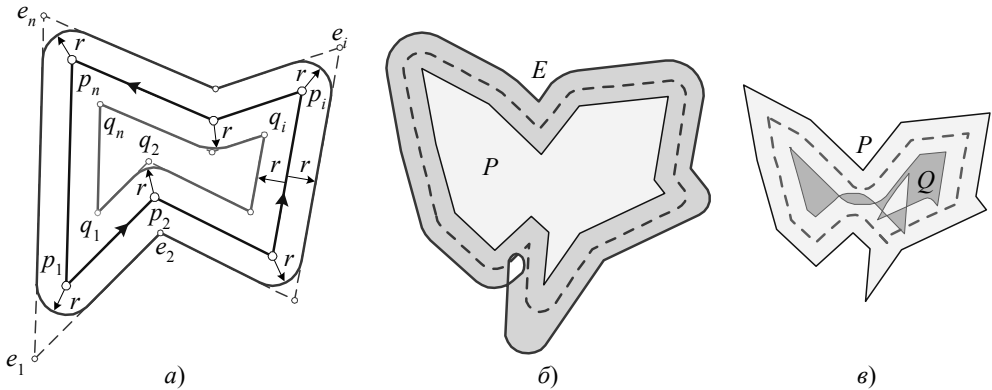


Рис. 1.61

Сведём оба варианта в **алгоритм** $\text{equi_poly}(P, r, \rho, \text{mod})$ **построения эквидистанты полигона** $P = p_1 p_2 \dots p_n p_1$ в режиме $\text{mod} \in \{-1, 1\}$, задающем скругляемый полигон: -1 — внутренний Q , а 1 — внешний E . При задании коэффициента $\rho=0$ всякое скругление отменяется.

При построении как внешнего (рис. 1.61,б), так и внутреннего (рис. 1.61,в) эквидистантных контуров возможно их **самопересечение** из-за несоответствия дистанты r размерам полигона и длинам его рёбер. В практической реализации алгоритма рекомендуется перед скруглением углов выполнить проверку самопересечения полигонов E и Q тестом self_test (см. рис. 1.16). Если оно обнаружено, то значение r автоматически либо интерактивно уменьшается до получения приемлемого решения (см. штриховые контуры на рис. 1.61).

□ Как построить **эквидистанту полилинии** $P=p_1p_2\dots p_n$, где $n \geq 2$, окружающую её со всех сторон? Ведь из-за незамкнутости у такого объекта P не определено направление обхода d , необходимое в (1.77) для вычисления вершин эквидистанты. Если попытаться составить полигон $p_1p_2\dots p_{n-1}p_np_{n-1}\dots p_2p_1$ **нулевой площади**, то по (1.55) получим $d=0$, а по формуле $\varphi_i=0.5ang(V_{i-1}, V_i)$ в углах $\angle p_{n-1}p_np_{n-1}$ и $\angle p_2p_1p_2$ будут получены значения $\varphi_1=\varphi_n=\pi/2$. Возникшая неопределённость $d \cdot \sec(\varphi)=0 \cdot \infty$ делает положения точек e_1, q_1, e_n и q_n непредсказуемыми.

Предлагается несложный способ разрешения проблемы (рис. 1.62). Построим справа от базовой полилинии P с нормированными векторами рёбер $V_i=p_{i+1}-p_i$ полилинию $E=e_1e_2\dots e_n$ (а), а слева — полилинию $Q=q_1q_2\dots q_n$ (б) со следующими вершинами:

- первые вершины располагаются на расстоянии $r\sqrt{2}$ от точки p_1 в направлениях $\pm 135^\circ$ от вектора V_1 :

$$e_1 = p_1 + r(Rrot(V_1) - V_1), \quad q_1 = p_1 + Rrot(e_1 - p_1); \quad (1.78a)$$

- промежуточные, существующие при $n > 2$, вершины линии E лежат на правых, а линии Q — на левых биссектрисах углов в точках пересечения прямых, эквидистантных рёбрам полилинии P :

$$e_i = p_i + r \sec(0.5ang(V_{i-1}, V_i)) \cdot Rrot(\overline{V_i + V_{i-1}}), \quad q_i = 2p_i - e_i \quad \forall i=2, n-1; \quad (1.78б)$$

- последние вершины находятся на расстоянии $r\sqrt{2}$ от точки p_n в направлениях $\pm 45^\circ$ от вектора V_{n-1} :

$$e_n = p_n + r(Rrot(V_{n-1}) + V_{n-1}), \quad q_n = p_n + Lrot(e_n - p_n). \quad (1.78в)$$

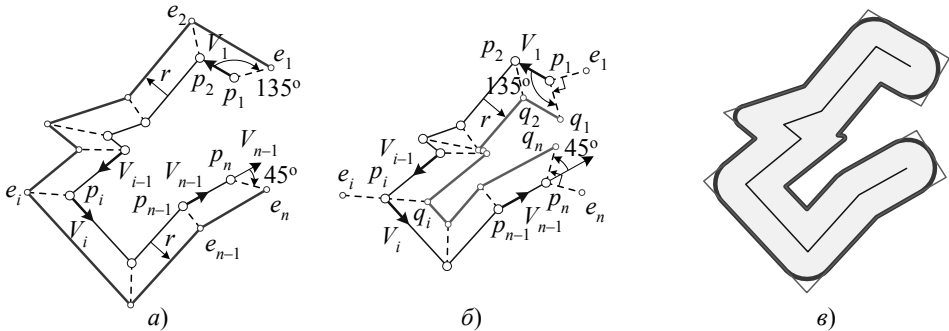


Рис. 1.62

Соединив две полилинии в один замкнутый контур $Lp=e_1e_2\dots e_nq_n\dots q_2q_1e_1$, скруглим его углы функцией $round_poly(Lp, r, p, 1)$. В результате получим составную линию, все точки которой при $p=0.5$ отстоят от заданной полилинии P на расстояние r (рис. 1.62, в). При задании коэффициента $p=0$ получим полигон $E \cup Q \cup e_1$ без скруглённых углов.

1.3. ОПЕРАЦИИ СО СПИСКАМИ

Для работы с переменным объёмом данных используются динамические массивы — **списки** различных типов [6, 19, 57]. Примером списка связанных элементов является перечень вершин полигона $\{p_1, p_2, \dots, p_n, p_1\}$ в порядке их соединения рёбрами.

Перечислим **операции со списками**, выполняемые в разных алгоритмах:

- вычисление **размера** списка $size(L)$ — числа входящих в него элементов;
- **инициализация** пустого списка $L=\emptyset$ с размером $size(\emptyset)=0$;
- **добавление** $e \rightarrow L$ нового элемента e в конец списка L с увеличением размера списка $L=\{L, e\}$ на единицу;
- **удаление** $L \leftarrow e$ из списка L элемента, равного e . Если $e \in L$, то после удаления размер списка уменьшается на единицу. Возможен вариант удаления из списка **всех** элементов, равных e , с уменьшением $size(L)$ на их количество. При $e \notin L$ список не изменяется. Если $size(L)=0$, то список остается пустым;
- **левый циклический сдвиг** $\overleftarrow{L} = \{e_2, \dots, e_n, e_1\}$. Если $size(L) \leq 1$, то $\overleftarrow{L} = L$;
- **сортировка** $\underset{c}{sort}(L)$ элементов списка L по критерию c , которым может

быть возрастание или убывание элементов самого списка L либо какого-нибудь другого списка, связанного с L . Одним из видов сортировки является преобразование списка не связанных элементов в один или несколько списков связанных элементов. Примером такой операции служит сборка из произвольно пронумерованных отрезков одного или нескольких замкнутых полигонов;

• **соединение (конкатенация)** [4]) двух списков L_1 и L_2 в список $L = \{L_1, L_2\}$, включающий все элементы обоих списков независимо от их кратности в соединённом списке L . Размер конкатенации $size(\{L_1, L_2\}) = size(L_1) + size(L_2)$;

• **объединение (дизъюнкция)** двух списков L_1 и L_2 в список $L = L_1 \cup L_2$ минимальной длины $size(L_1 \cup L_2) \leq size(L_1) + size(L_2)$ путём отбора в него не совпадающих элементов списков L_1 и L_2 ;

• **пересечение (конъюнкция)** двух списков L_1 и L_2 в список $L = L_1 \cap L_2$ минимальной длины $size(L_1 \cap L_2) \leq \min\{size(L_1), size(L_2)\}$ путём отбора в него совпадающих и не повторяющихся элементов списков L_1 и L_2 ;

• **исключение** $L_1 \setminus L_2 = L_1 \cap (\neg L_2)$ элементов списка L_2 из списка L_1 с минимальной длиной $size(L_1 \setminus L_2) \leq size(L_1) - size(L_1 \cap L_2)$ формируется отбором не повторяющихся элементов списка L_1 , не совпадающих с элементами списка L_2 .

Логические операции со списками скалярных (L) и векторных (Lp) переменных имеют переборный характер. Рассмотрим их выполнение на примере списков **скалярных** данных $L_1 = \{a_1, \dots, a_n\}$ и $L_2 = \{b_1, \dots, b_m\}$.

□ **Алгоритм соединения** $L = \{L_1, L_2\}$ заключается в создании списка $L = L_1$ и поочерёдном добавлении $b_j \rightarrow L \forall j = \overline{1, m}$ в него **всех** элементов списка L_2 .

□ **Алгоритм объединения** $L=L_1 \cup L_2$ можно реализовать **аддитивным** (с добавлением элементов) и **субтрактивным** (с удалением элементов) методами:

- после инициализации пустого списка $L=\emptyset$ в него последовательно **добавляются** элементы $a_i \forall i=\overline{1,n}$ и $b_j \forall j=\overline{1,m}$, отсутствующие в текущем списке L ;
- из конкатенации $L=\{L_1, L_2\}$ **удаляются** все кратные элементы.

Удаление из произвольного списка L всех кратных элементов выполняется с помощью операции $L=L \cup \emptyset$:

- по **аддитивному** методу в буферный список $L'=\emptyset$ добавляются некрратные элементы списка L , после чего L' переименовывается в L ;
- по **субтрактивному** методу элементы списка L попарно сравниваются друг с другом и кратные элементы e удаляются: $L=L-e$.

□ **Алгоритм пересечения** $L=L_1 \cap L_2$ также реализуется двумя методами:

- после инициализации списка $L=\emptyset$ в него **добавляются** некрратные элементы $a_i=b_j$, общие для обоих списков-сомножителей;
- из списка $L=L_1 \cup \emptyset$ **исключаются** кратные элементы и элементы, не совпадающие ни с одним из элементов b_j .

□ **Алгоритм исключения** $L=L_1 - L_2$ также двойственен:

- после инициализации списка $L=\emptyset$ в него **добавляются** некрратные элементы первого списка $a_i \neq b_j$, отсутствующие во втором списке;
- из списка $L=L_1 \cup \emptyset$ **исключаются** элементы, совпадающие хотя бы с одним из элементов b_j .

☞ Теперь рассмотрим порядок выполнения **логических операций со списками отрезков** $LS_1=\{a_1b_1, \dots, a_nb_n\}$ и $LS_2=\{c_1d_1, \dots, c_md_m\}$, расположенных на плоскости или в пространстве. В отличие от списков типа L и Lp , элементы которых являются объектами нулевой длины, в списках типа LS возможно частичное либо полное совпадение отрезков a_ib_i и c_jd_j , **лежащих на одной прямой**. Эти отрезки можно обнаружить **универсальными тестами** (1.206), работающими в пространствах R^r любой размерности $r \in \{2, 3\}$:

- тестом **коллинеарности** четырёх точек a_i, b_i, c_j и d_j :

$$\tilde{P} = \begin{bmatrix} a_i & 1 \\ b_i & 1 \\ c_j & 1 \\ d_j & 1 \end{bmatrix} \in R^{4 \times (r+1)}, \quad \text{rang}[\tilde{P}] = 2; \quad (1.79a)$$

- тестом **параллельности** трёх векторов $b_i - a_i, d_j - c_j$ и $c_j - a_i$:

$$M = \begin{bmatrix} b_i - a_i \\ d_j - c_j \\ c_j - a_i \end{bmatrix} \in R^{3 \times r}, \quad \text{rang}[M] = 1. \quad (1.79б)$$

В изучаемых ниже алгоритмах эксплуатируется более эффективный вариант теста (1.79б), работающий с матрицей меньшей размерности. Вычисление её ранга возможно как методом ортогонализации Грама – Шмидта (см. рис. 1.2), так и с помощью миноров [9, 10, 12, 21]. Последний метод свободен от приближённых операций деления, извлечения корней и нормировки векторов — источников вычислительных погрешностей, тогда как расчёт определителей не требует операций деления и может безошибочно проводиться непосредственно с экранными координатами точек.

Наиболее эффективные алгоритмы обработки списков отрезков основаны на комбинированном *аддитивно-субтрактивном* методе.

1.3.1. Объединение списков отрезков

Изображённый на рис. 1.63,а алгоритм объединения $Ls_1 \cup Ls_2$ списков отрезков Ls_1 и Ls_2 начинается с соединения исходных списков в конкатенацию $Ls = \{Ls_1, Ls_2\} \equiv \{u_1 w_1, \dots, u_l w_l\}$ с $l = n + m$ элементами

$$u_i w_i = a_i b_i \quad \forall i = \overline{1, n}, \quad u_{n+i} w_{n+i} = c_i d_i \quad \forall i = \overline{1, m}.$$

Определим направляющий вектор i -го отрезка $V_i = w_i - u_i \quad \forall i = \overline{1, l}$. До тех пор, пока $l \geq 2$, итеративно выполняются следующие действия.

Шаг 1. Перебираем пары отрезков $u_i w_i \quad \forall i = \overline{2, l}$ и $u_j w_j \quad \forall j = \overline{1, i-1}$, пока не будет найдена коллинеарная пара. Два отрезка с номерами i и j коллинеарны (рис. 1.63,б) при выполнении условия (1.79б):

$$\text{rang} \begin{bmatrix} V_i \\ V_j \\ u_j - u_i \end{bmatrix} = 1.$$

Если ни одна пара элементов Ls ему не удовлетворяет, то алгоритм **возвращает** текущий список Ls и заканчивает свою работу.

Шаг 2. Для найденной пары отрезков $\{u_i w_i, u_j w_j\}$ по (1.22) вычисляем координаты t и τ точек u_j и w_j на их общей прямой с началом координат в точке u_i и единицей в точке w_i (рис. 1.63,в). Если окажется, что $t > \tau$, то обмен параметров $t \leftrightarrow \tau$ обеспечит сонаправленность объединяемых отрезков (рис. 1.63,з).

Шаг 3. Тестируем *перекрывание* отрезков $u_i w_i$ и $u_j w_j$ по логическому пересечению числовых интервалов $[0, 1]$ и $[t, \tau]$. При $t > 1$ или $\tau < 0$ отрезки не имеют ни одной общей точки и остаются в списке без изменения (рис. 1.63,з). В противном случае при $t \leq 1$ (рис. 1.63,д) и $\tau \geq 0$ (рис. 1.63,е) вычисляем дизъюнкцию $pq = u_i w_i \cup u_j w_j$ — новый отрезок, у которого концевые точки p и q имеют в масштабе отрезка $u_i w_i$ координаты $\min\{0, t\}$ и $\max\{1, \tau\}$ соответственно.

Шаг 4. Исключаем элементы $u_i w_i$ и $u_j w_j$ из списка Ls , а отрезок pq в него добавляем. Определяем новый размер списка $l = \text{size}(Ls)$ и при $l \geq 2$ переходим на повторение цикла с **шага 1**. $\square$

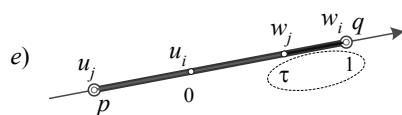
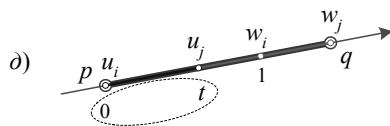
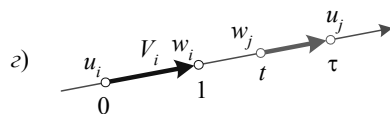
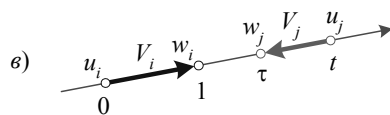
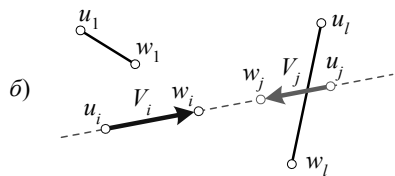
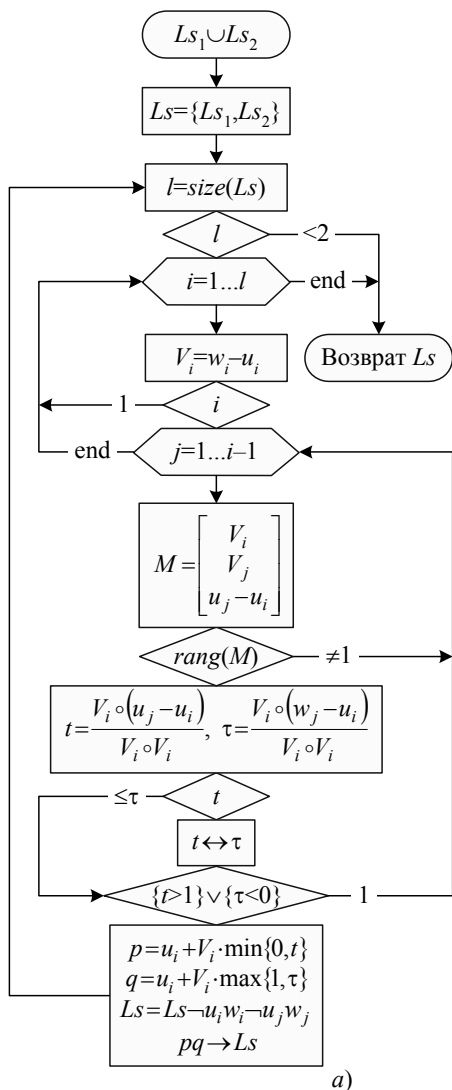


Рис. 1.63

Внешний цикл алгоритма заканчивается, когда в списке Ls не останется ни одной пары перекрывающихся коллинеарных отрезков либо размер списка l не станет равным единице. Возвращаемый алгоритмом список содержит все попарно неколлинеарные отрезки списков Ls_1 и Ls_2 , а также все дизъюнкции коллинеарных отрезков.

Особенностью изложенного алгоритма является первоначальное формирование конкатенации $Ls = \{Ls_1, Ls_2\}$, в которой все отрезки являются равноправными независимо от их принадлежности исходным спискам. Следовательно, алгоритм объединения списков отрезков можно применять для дизъюнкции отрезков одного списка Ls выполнением операции $Ls = Ls \cup \emptyset$.

Пример 1.6. Рассчитать объединение двух списков отрезков $Ls_1 = \{a_1b_1, a_2b_2, a_3b_3\}$ и $Ls_2 = \{c_1d_1, c_2d_2, c_3d_3\}$ (рис. 1.64,а). Для наглядности коллинеарные отрезки немного смещены.

Решение. Формируем конкатенацию из $l=6$ отрезков, обозначенных на рис. 1.64,б направлениями и номерами в списке Ls . В поиске коллинеарных перекрывающихся отрезков перебираем пары $\{2,1\}$ (не коллинеарны), $\{3,1\}$ (коллинеарны, но не перекрываются), $\{3,2\}$, $\{4,1\}$, пока не находим нужную пару отрезков $\{4,2\}$, для которых значения параметров $0 < t < 1$ и $\tau > 1$. Следовательно, концы объединённого отрезка равны

$$p = c_1 + (d_1 - c_1) \cdot \min\{0, t\} = c_1, \quad q = c_1 + (d_1 - c_1) \cdot \max\{1, \tau\} = b_2.$$

После удаления из списка Ls второго, четвёртого отрезков и добавления нового отрезка pq нумерация элементов становится такой, что изображена на рис. 1.64,в.

Повторяя поиск из $l=5$ элементов списка, находим пару перекрывающихся отрезков $\{3,2\}$ с параметрами $t > 1$ и $\tau < 0$. Так как $t > \tau$, то после обмена получим $t < 0$, $\tau > 1$ и концевые точки $p=d_2$ и $q=c_2$. Удалив второй, третий и добавив новый отрезок pq , получим размер списка $l=4$ и новую нумерацию его элементов (рис. 1.64,з).

Аналогично, при объединении в этом списке третьего и второго перекрывающихся отрезков получим $t > 1$ и $0 < \tau < 1$. После обмена параметров получим нужное соотношение $0 < t < 1 < \tau$ и концевые точки объединённого отрезка $p=c_3$ и $q=c_1$. Удаление двух отрезков и добавление отрезка pq даёт список из $l=3$ элементов (рис. 1.64,д).

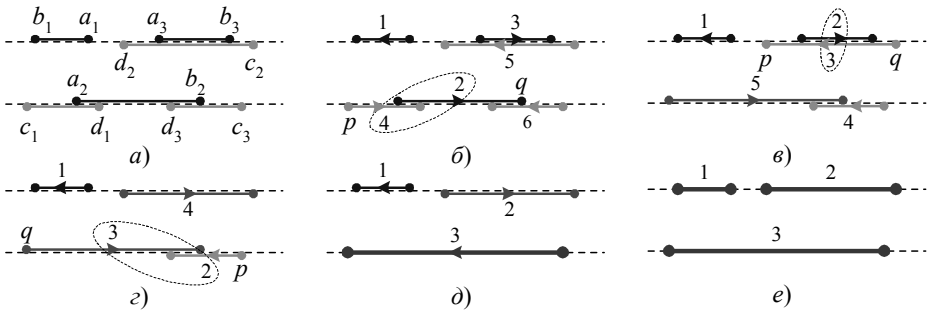


Рис. 1.64

Анализ расположения этих отрезков не находит ни одной перекрывающейся пары. Таким образом, мы получили окончательный результат $\{a_1b_1, d_2c_2, c_3c_1\}$ объединения списков отрезков Ls_1 и Ls_2 , показанный на рис. 1.64,е. $\square$

1.3.2. Пересечение списков отрезков

Алгоритм пересечения $Ls_1 \cap Ls_2$ списков отрезков Ls_1 и Ls_2 (рис. 1.65,а) построен по аддитивному методу. Сначала инициализируется список $Ls = \emptyset$, в который затем добавляются попарные пересечения $pq = a_i b_i \cap c_j d_j$. Если любой из исходных списков пустой, алгоритм **возвращает** пустой результат.

Пересечение отрезков $a_i b_i$ и $c_j d_j$ не пусто, если они:

- коллинеарны, т. е. удовлетворяют условию (1.79б):

$$\text{rang} \begin{bmatrix} V \\ d_j - c_j \\ c_j - a_i \end{bmatrix} = 1,$$

где $V = b_i - a_i$ — вектор направления отрезка $a_i b_i$;

• перекрывают друг друга при $\{t < 1\} \wedge \{\tau > 0\}$, где t и $\tau > t$ — вычисленные по (1.22) координаты $t = V \circ (c_j - a_i) / V \circ V$ и $\tau = V \circ (d_j - a_i) / V \circ V$ точек c_j и d_j в масштабе отрезка $a_i b_i$. Если окажется $t > \tau$, то обмен $t \leftrightarrow \tau$ сделает отрезки сонаправленными.

Концевые точки конъюнкции pq имеют в масштабе отрезка $a_i b_i$ координаты $\max\{0, t\}$ (рис. 1.65, б) и $\min\{1, \tau\}$ (рис. 1.65, в) соответственно.

Пример 1.7. Рассчитать пересечение двух списков отрезков $Ls_1 = \{a_1 b_1, a_2 b_2, a_3 b_3\}$ и $Ls_2 = \{c_1 d_1, c_2 d_2, c_3 d_3\}$. Взаимное расположение отрезков показано на рис. 1.65, з.

Решение. Инициализируем список $Ls = \emptyset$. В поиске коллинеарных перекрывающихся отрезков $\{a_i b_i, c_j d_j\}$ первой находим пару $\{a_2 b_2, c_1 d_1\}$, для которой значения параметров $t < 0$ и $0 < \tau < 1$. Следовательно, первый отрезок, помещённый в список Ls , имеет концевые точки, изображённые на рис. 1.65, д:

$$p_1 = a_2 + (b_2 - a_2) \cdot \max\{0, t\} = a_2, \quad q_1 = a_2 + (b_2 - a_2) \cdot \min\{1, \tau\} = d_1.$$

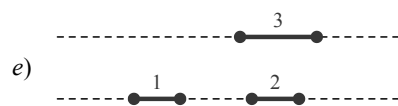
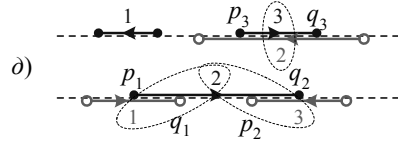
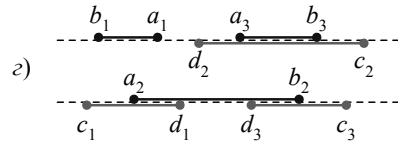
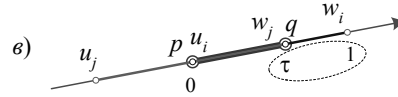
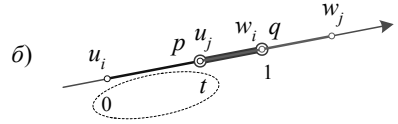
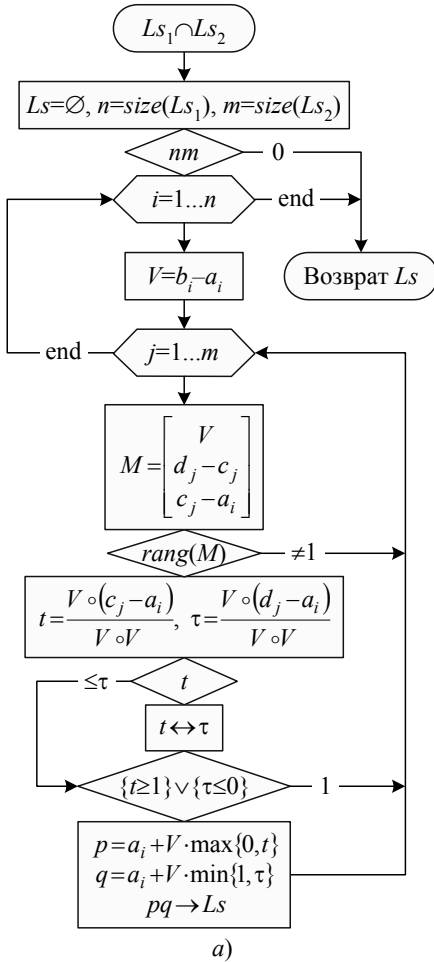


Рис. 1.65

Вторая пара отрезков $\{a_2b_2, c_3d_3\}$ характеризуется соотношением параметров $0 < \tau < 1 < t$. После обмена $0 < t < 1 < \tau$ получим второй отрезок списка Ls с концевыми точками $p_2=d_3$ и $q_2=b_2$. Наконец, третья пара $\{a_3b_3, c_2d_2\}$ с переставленными параметрами $t < 0$ и $\tau > 1$ добавляет в Ls отрезок $p_3q_3 \equiv a_3b_3$.

Окончательный состав списка $Ls = \{a_2d_1, d_3b_2, a_3b_3\}$ изображён на рис. 1.65,е.



1.3.3. Исключение списков отрезков

Алгоритм исключения $Ls_1 - Ls_2$ **отрезков** списка $Ls_2 = \{c_1d_1, \dots, c_md_m\}$ из отрезков списка $Ls_1 = \{a_1b_1, \dots, a_nb_n\}$ приведён на рис. 1.66,а. Принципиальное отличие операции исключения отрезков от других логических операций с отрезками и от исключения одного списка скаляров или точек из другого списка заключается в возможности получения из одного отрезка двух при полном перекрытии отрезка c_jd_j отрезком a_ib_i .

Вначале формируется список $Ls = Ls_1 = \{u_1w_1, \dots, u_lw_l\}$ с $l=n$ элементами $u_iw_i = a_ib_i$. Если список Ls_2 пуст, т. е. $m=0$, то алгоритм возвращает список Ls и заканчивает работу. Иначе, пока размер исключаемого списка $l = \text{size}(Ls) > 0$, производится аналогичный предыдущим алгоритмам поиск пары перекрывающихся отрезков $\{u_iw_i, c_jd_j\} \quad \forall i = \overline{1, l}, \quad j = \overline{1, m}$. В зависимости от их ориентации, определяемой значениями координат t и $\tau > t$ концевых точек c_j и d_j в масштабе отрезка u_iw_i (при $\tau < t$ выполняются обмены параметров $t \leftrightarrow \tau$ и точек $c_j \leftrightarrow d_j$), возможны следующие варианты действий:

- при $t \leq 0$ и $\tau \geq 1$ отрезок u_iw_i целиком перекрывается отрезком c_jd_j (рис. 1.66,б);
- при $0 < t < 1$ не перекрытая часть u, c_j отрезка u_iw_i добавляется в Ls (рис. 1.66,в);
- при $0 < \tau < 1$ не перекрытая часть d_jw_i отрезка u_iw_i добавляется в Ls (рис. 1.66,г).

После этого отрезок u_iw_i из списка Ls удаляется. Из-за изменений размера Ls и нумерации его элементов алгоритм повторяет анализ отрезков с начала.

Завершение работы алгоритма происходит либо при $m=0$, либо при получении пустого списка $Ls = \emptyset$, когда все отрезки a_ib_i перекрыты отрезками c_jd_j , либо в отсутствие пар перекрывающихся отрезков $\{u_iw_i, c_jd_j\}$.

Пример 1.8. Рассчитать исключение отрезков списка $Ls_2 = \{c_1d_1, c_2d_2, c_3d_3\}$ из отрезков списка $Ls_1 = \{a_1b_1, a_2b_2, a_3b_3\}$. Взаимное расположение отрезков показано на рис. 1.66,д.

Решение. Создаем список Ls с тремя элементами $u_iw_i = a_ib_i$ и ищем пары перекрывающихся отрезков $\{u_iw_i, c_jd_j\}$. Первой находим пару $\{u_2w_2, c_1d_1\}$, для которой $0 < t < \tau < 1$ (рис. 1.66,е). Исключив из Ls отрезок u_2w_2 и добавив два отрезка a_2c_1 и d_1b_2 , получим конфигурацию, показанную на рис. 1.66,ж.

В следующей паре отрезок u_2w_2 (под ним теперь скрывается a_3b_3) полностью перекрыт отрезком c_2d_2 и поэтому удаляется из Ls (рис. 1.66,з).

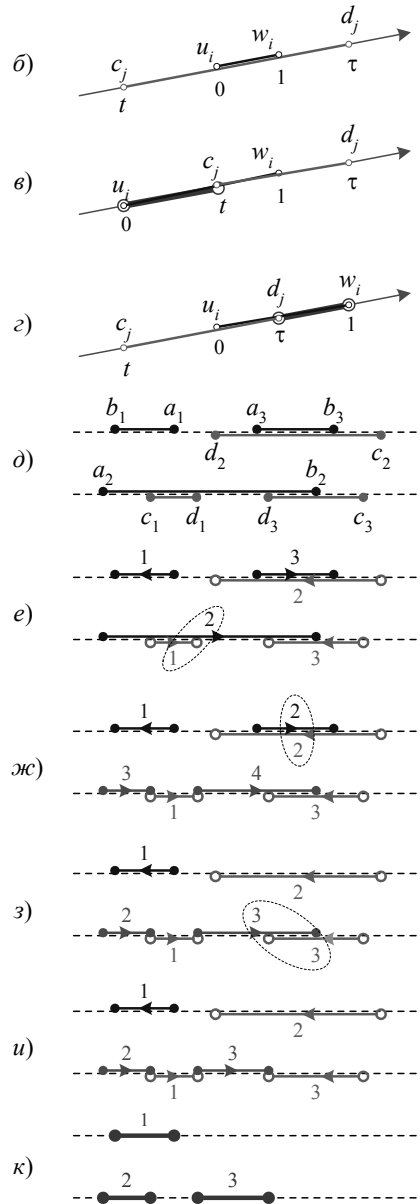
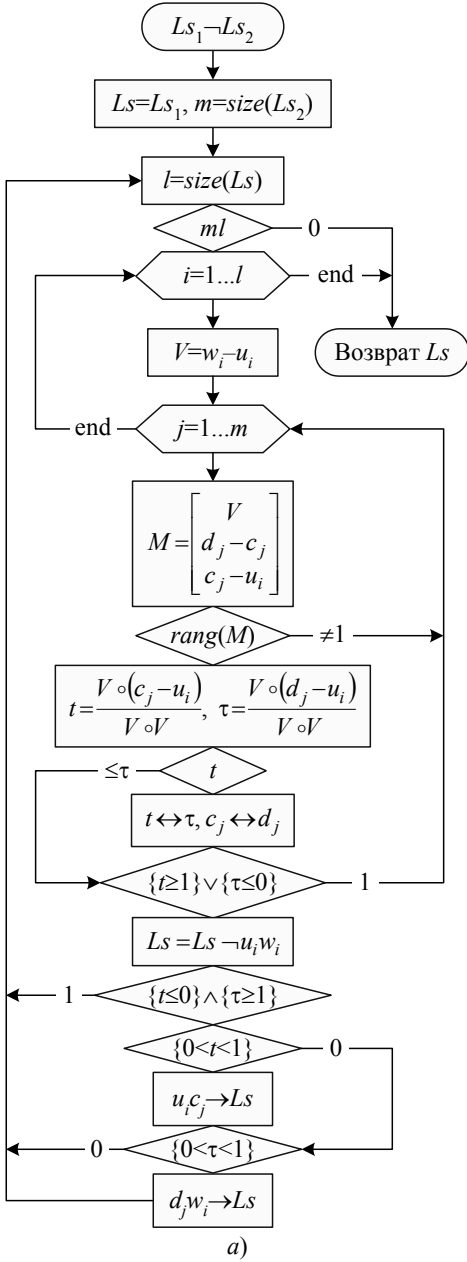


Рис. 1.66

Анализируя третью пару отрезков $\{u_3 w_3, c_3 d_3\}$ с соотношением параметров после обмена $0 < t < 1 < \tau$, удаляем из Ls отрезок $u_3 w_3$ и добавляем туда $d_1 d_3$. В получившейся конфигурации отрезков (рис. 1.66, и) нет ни одной перекрывающейся пары.

Окончательный состав списка $Ls = \{a_1 b_1, a_2 c_1, d_1 d_3\}$ изображён на рис. 1.66, к.



1.3.4. Соединение отрезков в контуры

Завершим изучение операций с отрезками непростым действием **сборки замкнутых контуров** из отрезков, составляющих список Ls . Сборка завершает все логические операции с полигонами (рис. 1.67) и заключается в соединении отрезков совпадающими концами. Количество собранных контуров может быть:

- **нулевым** при пустом списке Ls , получающемся (а) при расчёте пересечения $A \cap B$ непересекающихся полигонов и (б) в операции исключения $A - B$ с полигоном A , расположенным целиком внутри полигона B ;
- **один** контур C будет получен при расчёте (в) пересечения $A \cap B$ выпуклых полигонов или объединения $A \cup B$ пересекающихся выпуклых полигонов;
- **несколько** контуров $C_1 \div C_k$ в других логических операциях как с выпуклыми, так и с невыпуклыми полигонами, например, в расчёте (а) объединения $A \cup B$ или (з) пересечения $A \cap B$.

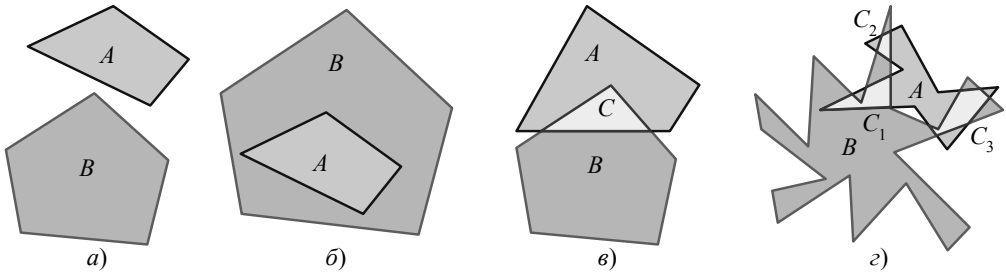


Рис. 1.67

□ Начнём освоение операции сборки с простейшего случая, когда **все** отрезки списка Ls образуют только **один** замкнутый контур. Блок-схема **алгоритма** $cont1(Ls)$ **сборки одного замкнутого контура** $C = c_1c_2 \dots c_{m-1}c_m$, где $c_m = c_1$, **из всех отрезков списка** $Ls = \{a_1b_1, \dots, a_nb_n\}$ приведена на рис. 1.68,а. Порядок следования отрезков в списке произволен — главное, чтобы их было не менее трёх, соединялись они друг с другом только попарно и среди отрезков не было кратных и тупиковых, в том числе одиночных (рис. 1.68,б).

Начало контура формируем из двух концевых точек первого отрезка $a_1b_1 \equiv Ls_1$, а счётчик вершин контура устанавливаем в $m=2$. В процессе **многократного** сканирования списка Ls подвергаем проверке на возможность стыковки с текущей ломаной линией $C = c_1c_2 \dots c_m$ только отрезки a_ib_i с флажками $f_i=1$ (рис. 1.68,в). **Условием стыковки** является совпадение точки c_m либо с началом отрезка a_i , либо с его концом b_i . Тогда следующим концом ломаной линии C становится точка b_i либо a_i , счётчик m инкрементируется, а флажок f_i сбрасывается в ноль, блокируя дальнейшее использование отрезка a_ib_i .

При выполнении **условия замыкания** $c_m = c_1$ — совпадения нового конца цепочки c_m с начальной точкой c_1 — формирование замкнутого контура C заканчивается. Рекомендуются подвергнуть возвращаемый полигон **минимизации** алгоритмом min_poly (см. рис. 1.18,а) для удаления вершин с развёрнутыми углами, которые появляются при соединении коллинеарных отрезков.

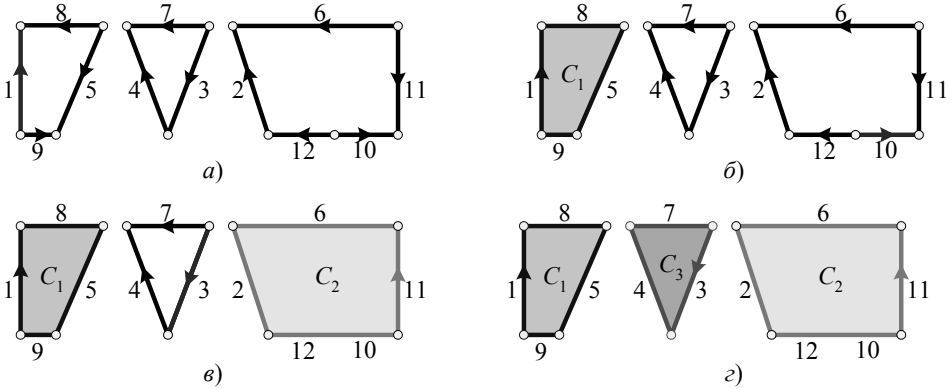


Рис. 1.70

Проходы 3, 4. Присоединение к цепочке отрезков 6, 2 и 12 делает её замкнутой. Минимальный полигон $C_2 = \{11, 6, 2, 12 \cup 10\}$ добавляется в список $LP = \{C_1, C_2\}$ (рис. 1.70, в).

Проход 5. Из трёх оставшихся в списке Ls активных отрезков собираем замкнутый контур $C_3 = \{3, 4, 7\}$ (рис. 1.70, г). В итоге получаем искомый список $LP = \{C_1, C_2, C_3\}$.

□

□ Описанный метод сборки не рассчитан на списки, содержащие несвязанные и тупиковые отрезки, которые могут появиться при граничном отсечении. Третий алгоритм $cont(Ls)$ сборки замкнутых контуров $C_1 \div C_k$ из подходящих отрезков списка $Ls = \{a_1 b_1, \dots, a_n b_n\}$ построен на рис. 1.71, а. Все преобразования отрезков списка Ls и вершин контура C ведутся напрямую, без использования флажков, с помощью списковых операций добавления, удаления и сдвига.

Вначале инициализируются счётчик вершин $m=0$ и пустой список полигонов $LP = \emptyset$. Затем, пока $Ls \neq \emptyset$, выполняется цикл обработки всегда **первого** элемента $Ls \equiv ab$. В отсутствие лишних и тупиковых отрезков это выглядит так:

- при $m=0$ начинается сборка нового контура $C=ab$ путём присвоения счётчику вершин значения $m=2$ и удаления отрезка ab из верхушки списка Ls ;
- при $m \neq 0$ проверяется совпадение a и b с концом полилинии $C = c_1 c_2 \dots c_m$:
 - ◊ если $c_m = a$, то точка b , а если $c_m = b$, то точка a добавляется в конец C с инкрементом счётчика $m = m+1$ и удалением отрезка ab из списка Ls ;
 - ◊ при невозможности присоединения отрезка ab к полилинии C список Ls подвергается левому циклическому сдвигу;
 - ◊ пока контур C не замкнулся, цикл его сборки повторяется, а при $m \geq 4$ и выполнении условия замыкания $c_m = c_1$ собранный полигон C минимизируется алгоритмом min_poly и добавляется в список LP .

Если же в исходном списке Ls есть лишние, тупиковые или цепочки таких отрезков, то возможны ошибки сборки, устранимые следующими действиями:

- поочерёдным удалением из построенной цепочки $C = c_1 \dots c_{j-1} c_j \dots c_m$ **начальных** вершин $c_1 \div c_{j-1}$ при замыкании контура в вершине c_j с индексом $j \in [2, m-3]$ (рис. 1.71, б). Удалённые звенья цепочки добавляются в список Ls ,

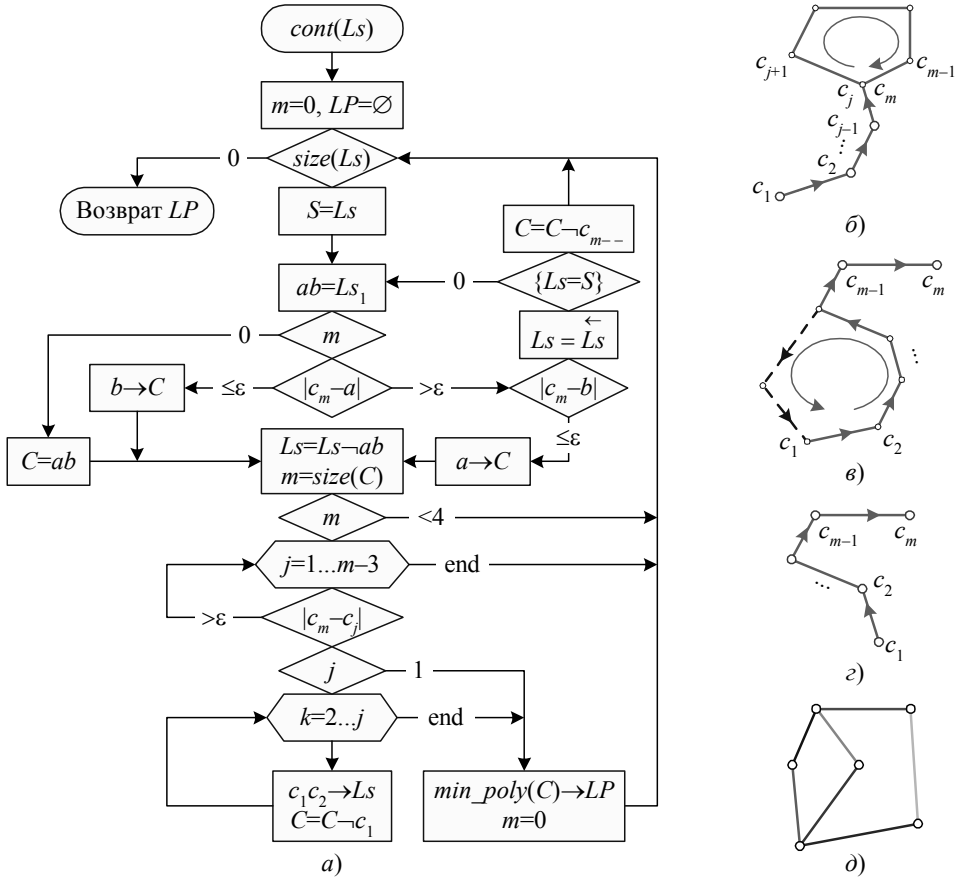


Рис. 1.71

чтобы их можно было использовать для построения других контуров. Оставшийся контур $c_j \dots c_m$ минимизируется и добавляется в список полигонов LP ;

- удалением **конечной** вершины c_m из цепочки $C=c_1 c_2 \dots c_m$ в отсутствие в списке Ls хотя бы одного отрезка с любой концевой точкой, совпадающей с c_m (рис. 1.71, ϵ). Достоверно тупиковый отрезок $c_{m-1} c_m$ обнаруживается сравнением циклически сдвинутого списка $\overleftarrow{Ls}$ с копией S , создаваемой из Ls после каждого изменения C . Идентичность списков $\overleftarrow{Ls}=S$ означает, что ни один из отрезков в Ls не стыкуется с текущей цепочкой C и её надо укоротить на одно звено. Процесс укорачивания идёт до полного удаления всей тупиковой ветви. Аналогично удаляется обособленная тупиковая цепочка (рис. 1.71, ζ).

Благодаря циклическому сдвигу динамического списка Ls упрощается доступ к проверяемому (всегда первому) отрезку ab . Использование для этого вектора флажков f неэффективно из-за необходимости поиска восстанавливаемых в списке Ls отрезков и повторной активации их флажков. Принятое в алгоритме $cont$ решение о прямой работе со списками вполне оправдано и обогащает читателя полезными навыками обработки структурированных данных.

Отметим, что, к сожалению, данный алгоритм не способен собрать все контуры, касающиеся друг друга рёбрами, как показано на рис. 1.71,д.

Пример 1.10. Собрать возможные замкнутые контуры из тринадцати пронумерованных направленных отрезков.

Решение. Составим из номеров отрезков список $Ls = \{1, 2, 3, \dots, 12, 13\}$ и начнём работу алгоритма *cont* с инициализации пустого списка полигонов $LP = \emptyset$ начала цепочки $C = \{1\}$ из отрезка 1 (рис. 1.72,а), удаления его из Ls и шести левых циклических сдвигов списка Ls для того, чтобы первым в нём оказался отрезок 8, сцепляемый с концом цепочки. Переносим его из $Ls = \{8, 9, \dots, 12, 13, 2, \dots, 7\}$ в $C = \{1, 8\}$, после чего $Ls = \{9, 10, 11, 12, 13, 2, 3, 4, 5, 6, 7\}$.

Дальнейший ход процесса сборки контуров схематически выглядит так:

- $Ls = \{5, 6, 7, 9, 10, 11, 12, 13, 2, 3, 4\} \Rightarrow C = \{1, 8, 5\}$, $Ls = \{6, 7, 9, 10, 11, 12, 13, 2, 3, 4\}$;
- $Ls = \{9, 10, 11, 12, 13, 2, 3, 4, 6, 7\} \Rightarrow C = \{1, 8, 5, 9\}$, $Ls = \{10, 11, 12, 13, 2, 3, 4, 6, 7\}$;
- $c_5 = c_1 \Rightarrow C_1 = \{1, 8, 5, 9\} \rightarrow LP$ (рис. 1.72,б);
- начало новой цепочки $C = \{10\} \Rightarrow Ls = \{1, 12, 13, 2, 3, 4, 6, 7\} \Rightarrow C = \{10, 11\}$, $Ls = \{12, 13, 2, 3, 4, 6, 7\}$;
- $Ls = \{6, 7, 12, 13, 2, 3, 4\} \Rightarrow C = \{10, 11, 6\}$, $Ls = \{7, 12, 13, 2, 3, 4\} \Rightarrow C = \{10, 11, 6, 7\}$, $Ls = \{12, 13, 2, 3, 4\}$;
- $Ls = \{4, 12, 13, 2, 3\} \Rightarrow C = \{10, 11, 6, 7, 4\}$, $Ls = \{12, 13, 2, 3\}$;
- $Ls = \{13, 2, 3, 12\} \Rightarrow C = \{10, 11, 6, 7, 4, 13\}$, $Ls = \{2, 3, 12\}$;
- отрезок 13 — тупиковый $\Rightarrow C = \{10, 11, 6, 7, 4\}$, $Ls = \{2, 3, 12\}$;
- $Ls = \{3, 12, 2\} \Rightarrow C = \{10, 11, 6, 7, 4, 3\}$, $Ls = \{12, 2\}$;
- $c_7 = c_4 \Rightarrow C_2 = \{7, 4, 3\} \rightarrow LP$ (рис. 1.72,в), а три начальных отрезка 10, 11 и 6 возвращаются в список $Ls = \{12, 2, 10, 11, 6\}$;

- начало новой цепочки $C = \{12\} \Rightarrow Ls = \{2, 10, 11, 6\} \Rightarrow C = \{12, 2\}$, $Ls = \{10, 11, 6\}$;
- $Ls = \{6, 10, 11\} \Rightarrow C = \{12, 2, 6\}$, $Ls = \{10, 11\}$;
- $Ls = \{11, 10\} \Rightarrow C = \{12, 2, 6, 11\}$, $Ls = \{10\} \Rightarrow C = \{12, 2, 6, 11, 10\}$, $Ls = \emptyset$;
- $c_6 = c_1 \Rightarrow$ минимальный полигон $C_3 = \{2, 6, 11, 10 \cup 12\} \rightarrow LP$ (рис. 1.72,г).

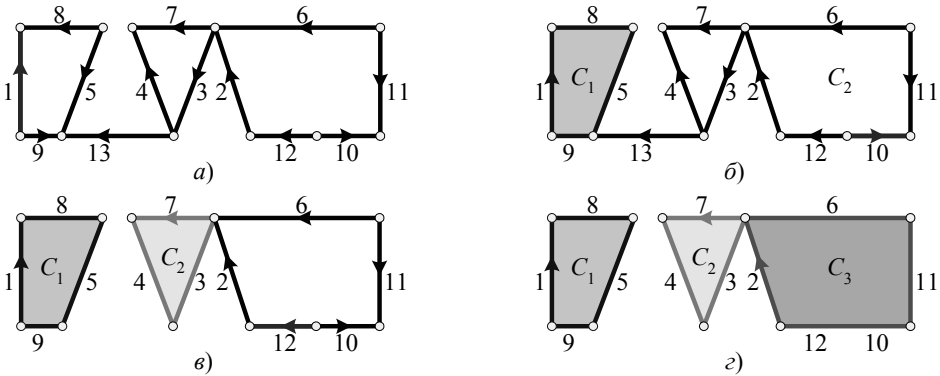


Рис. 1.72

По достижении пустого списка $Ls = \emptyset$ функция *cont* свою работу заканчивает, возвращая решение задачи — список $LP = \{C_1, C_2, C_3\}$, элементами которого являются полигоны $C_1 = \{1, 8, 5, 9\}$, $C_2 = \{7, 4, 3\}$ и $C_3 = \{12, 2, 6, 11, 10\}$. □

1.4. ОТСЕЧЕНИЕ НА ПЛОСКОСТИ

Отсечение (*clipping*) — это процедура обнаружения и исключения из дальнейшего анализа тех элементов сцены (линий, поверхностей, объёмных фигур), которые не попадают в видимую область. Эта операция выполняется на ранних этапах визуализации, позволяя существенно снизить трудоёмкость расчёта пространственно протяжённой сцены за счёт исключения из дальнейшей обработки многих ограниченно видимых объектов. В отличие от тестов отсечение связано с полным расчётом элементов взаимного пересечения объектов и дальнейшим формированием из них неотсечённых фрагментов.

Помимо визуализации алгоритмы отсечения используются в системах конструктивного моделирования для выполнения логических операций с плоскими и объёмными объектами и построения сечений, в разных тестах локализации, при построении оптических эффектов — теней, отражений и преломлений — на ограниченных поверхностях, для создания многооконного интерфейса прикладных программ со сложной, т. е. *не прямоугольной*, формой окон и т. д.

Объект, устанавливающий границы области отсечения, называется *отсекателем*. В зависимости от размерности, области действия, сложности формы и средств реализации отсекатели бывают [28, 50, 65]:

- *плоскими* (2d), например, экранное окно, в котором строится изображение сцены. Такое несложное отсечение производится на конечной стадии отображения объёмной сцены, многие элементы которой, возможно, и не попадут в окно;
- *объёмными* (3d), например, призма (при параллельном) или пирамида (при центральном проецировании) видимости. Более трудоёмкое в сравнении с плоским, отсечение по объёму исключает заведомо невидимые элементы сцены из дальнейшего анализа. Это даёт значительную экономию машинных ресурсов;
- *внешними*, удаляющими элементы сцены вне отсекателя, играющего роль ограниченного окна, через которое наблюдатель видит объекты сцены;
- *внутренними*, отбрасывающими элементы сцены внутри отсекателя, играющего роль ширмы, загораживающей объекты сцены от наблюдателя;
- *регулярными*, имеющими форму плоской (прямоугольник) и объёмной (параллелепипед) фигур, ориентированных по координатным осям. Для этих простейших форм отсекателей разработаны специальные упрощённые и более быстрые алгоритмы отсечения;
- *нерегулярными*, имеющими произвольную форму. Алгоритмы, работающие с такими отсекателями, более сложные и ресурсоёмкие;
- *аппаратными*, реализованными микропрограммными средствами центрального процессора, контроллеров, графических ускорителей и т. д. Обычно аппаратные отсекатели ограничиваются простейшими — регулярными или выпуклыми — 2d- и 3d-формами;
- *программными*, создаваемыми в отсутствие аппаратного отсекателя или в нестандартных ситуациях отсечения. Программные алгоритмы часто имеют недостаточное быстродействие в приложениях, ориентированных на процессы, протекающие в реальном времени.

Физической моделью плоского отсечения служит образование **тени** (области низкой освещённости) одного полигона 3d-сцены на другой. Сравним **две тактические схемы тенеобразования** на примере построения экранной проекции $P_{тэ}$ тени полигона P на плоскую подложку, **ограниченную** полигоном O , от точечного источника света S (рис. 1.73). Тень точки p на **неограниченную** плоскость подложки рассчитывается как точка пересечения с нею луча $S+(p-S)t$ при $t>1$. Тени вершин полигона P образуют теневой полигон $P_{т}$.

Рассматриваемые схемы тенеобразования отличаются порядком выполнения операций отсечения и проецирования:

- в схеме (а) первым делается **отсечение в плоскости подложки**, для чего координаты объектов $P_{т}$ и O переводятся во вспомогательную СК $x'y'z'$ с фронтальной плоскостью $x'y' \supset O$. Затем неотсечённая часть полигона $P_{т}$ пересчитывается в исходную СК и проецируется на экранную плоскость x_3y_3 ;
- в схеме (б) сначала выполняется проецирование полигонов $P_{т}$ и O на плоскость x_3y_3 , после чего делается **отсечение в экранной плоскости** полигона $P_{тэ}$ полигональным отсекателем O_3 .

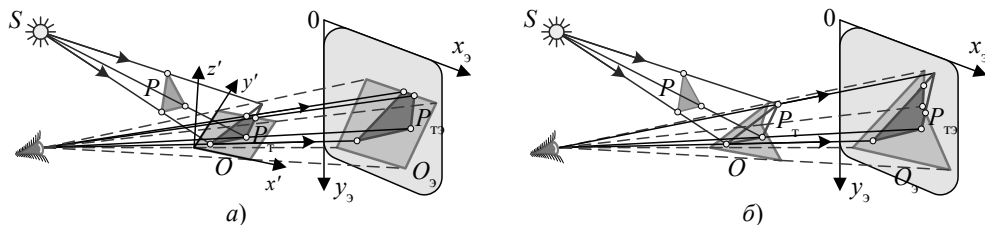


Рис. 1.73

Таким образом, **построение тени выполняется быстрее по второй схеме**, не требующей преобразований СК. Это имеет место и при построении **отражения** объекта в полигональном зеркале. Выигрыш достигается за счёт исключения расчёта промежуточных точек на плоскости тени или зеркала, преобразования СК и получения результата отсечения сразу в виде экранной формы.

1.4.1. Отсечение отрезков

Операции отсечения отрезков разными графическими объектами — наиболее востребованные расчётные задачи. Их решением являются неотсечённые фрагменты, из которых затем собираются полигоны, служащие объектами многих задач компьютерной графики, например, сборки объёмных полиэдров.

Регулярное отсечение отрезка

Простейшим отсекателем является **прямоугольное** окно $O = o_{лн} o_{пн} o_{пв} o_{лв} o_{лн}$, заданное координатами сторон $x_л, x_п, y_н, y_в$ и вершинами

$$o_{лн} = [x_л \ y_н], \quad o_{пн} = [x_п \ y_н], \quad o_{пв} = [x_п \ y_в], \quad o_{лв} = [x_л \ y_в].$$

С отсекателями такой формы работают многие задачи селективной визуализации. К примеру, расчёт оптических эффектов на прямоугольных подложках по-

сле преобразования СК сводится к внешнему регулярному отсечению (рис. 1.74,а). Внутренний тип регулярного отсекаателя (рис. 1.74,б) востребован при программировании оконного интерфейса операционных систем.

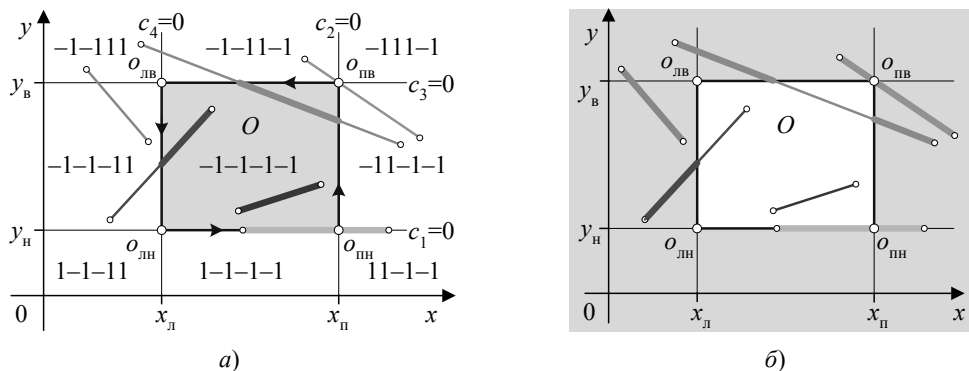


Рис. 1.74

Будучи нацеленными на аппаратную реализацию, алгоритмы регулярного отсечения должны максимально быстро решать две задачи:

- оценивать расположение отрезка относительно окна и возвращать пустой результат либо исходный отрезок без расчёта его пересечения с отсекателем;
- рассчитывать пересечение отрезка со сторонами отсекаателя по простейшим формулам, адаптированным к регулярной форме полигона O.

В первой задаче нужно отделить заведомо внутренние и внешние отрезки, а для остальных запомнить стороны окна, с которыми *потенциально возможно* их пересечение. Для этого каждой точке плоскости $p=[x \ y]$ поставим в соответствие *троичный логический код*

$$C=c_1c_2c_3c_4, \quad c_i \in \{-1,0,1\} \quad \forall i=\overline{1,4}, \quad (1.80a)$$

четыре разряда которого описывают её расположение относительно сторон регулярного окна в порядке обхода «нижняя → правая → верхняя → левая»:

$$c_1=\text{sgn}(y_N - y), \quad c_2=\text{sgn}(x - x_P), \quad c_3=\text{sgn}(y - y_B), \quad c_4=\text{sgn}(x_L - x). \quad (1.80б)$$

В отличие от двоичного кодирования в методе Козна – Сазерленда [50], используемая здесь *троичная логика* имеет не два, а три состояния координаты относительно ребра отсекаателя, включая граничное. У точки, лежащей *слева* от i -го ребра, разряд $c_i=-1$ у правой точки $c_i=1$, а у точки на линии ребра $c_i=0$. В результате девять областей между прямыми сторон регулярного отсекаателя получают логические коды, показанные на рис. 1.74,а.

Логические операции инверсии, конъюнкции и дизъюнкции с троичными числами и переменными $A \in \{-1,0,1\}$ и $B \in \{-1,0,1\}$ выполняются по следующим формулам и *таблицам истинности* (табл. 1.2):

$$\begin{aligned} \neg(-1) &= 1, \quad \neg 0 = 0, \quad \neg(1) = -1; \\ A \wedge B &= \min\{A, B\}; \\ A \vee B &= \max\{A, B\}. \end{aligned} \quad (1.81)$$

Таблица 1.2

A	B	$\neg A$	$A \wedge B$	$A \vee B$
-1	-1	1	-1	-1
	0	1	-1	0
	1	1	-1	1
0	-1	0	-1	0
	0	0	0	0
	1	0	0	1
1	-1	-1	-1	1
	0	-1	0	1
	1	-1	1	1

Пусть выражение $C*a$ с символом $* \in \{=, <, \leq\}$ означает, что **все разряды кода C удовлетворяют условию c_i*a** . Тогда положение точки p относительно регулярного окна определяется следующим логическим кодом:

- у **внутренней** точки $C=-1$;
- в коде **граничной** точки $C \leq 0$ один либо два нулевых разряда $c_i=c_j=0$ означают её расположение на ребре либо в вершине полигона отсекаателя;

- в коде **внешней** точки существует хотя бы

один разряд $c_i=1$. Число таких разрядов может быть не более двух (рис. 1.74,а).

Определим выражение C_1*C_2 с символом $* \in \{\wedge, \vee\}$ как логическую операцию с кодами C_1 и C_2 , выполняемую **поразрядно** по табл. 1.2. Теперь можно сформировать **признаки гарантированного расположения** отрезка p_1p_2 с логическими кодами концевых точек C_1 и C_2 относительно регулярного окна:

- оба конца **внутреннего** отрезка лежат слева от всех сторон отсекаателя:

$$c_{1i}=c_{2i}=-1 \quad \forall i=\overline{1,4} \Leftrightarrow C_1 \vee C_2 = -1; \quad (1.82a)$$

- оба конца **внутренне-граничного** отрезка лежат не справа от всех сторон:

$$c_{ki} \leq 0 \quad \forall k=\overline{1,2}, i=\overline{1,4} \Leftrightarrow C_1 \vee C_2 \leq 0; \quad (1.82б)$$

- оба конца **внешнего** отрезка лежат справа от какой-нибудь стороны окна:

$$\exists i \in [1, 4]: c_{1i}=c_{2i}=1 \Leftrightarrow \neg\{C_1 \wedge C_2 \leq 0\}; \quad (1.82в)$$

- оба конца **внешне-граничного** отрезка лежат не слева от какой-нибудь стороны окна:

$$\exists i \in [1, 4]: c_{1i} \geq 0, c_{2i} \geq 0 \Leftrightarrow \neg\{C_1 \wedge C_2 = -1\}. \quad (1.82г)$$

Отрезок, не прошедший эти тесты, содержит один или два внешних конца. Со стороны внешнего конца p_k с логическим кодом C_k он **может** пересечь не более одной из двух сторон окна, для которых $c_{ki}=1$.

Быстрое решение второй задачи выполняется упрощенным алгоритмом *cross_seg* (см. п. 1.1.2), оформленным в виде **функции** *reg_cross2(ab,cd,q)* **расчёта пересечения отрезка ab с регулярным** (горизонтальным либо вертикальным) **отрезком cd** , возвращающей в своём имени признак расположения отрезков (-1, 0 либо 1), а в аргументе q — точку их пересечения.

☞ *reg_cross2(ab,cd,q)*

$\{V=b-a, U=c-a\};$

если $\{V_x=0\} \wedge \{V_y=0\}$, то **возврат** -1; // вырожденный отрезок ab

если $\{c_y=d_y\} \wedge \{V_y \neq 0\}$, то $q=[a_x+V_x U_y/V_y, c_y]$; // горизонтальный cd

если $\{c_x=d_x\} \wedge \{V_x \neq 0\}$, то $q=[c_x, a_y+V_y U_x/V_x]$; // вертикальный cd

возврат $\{(q_x - c_x)(d_x - q_x) \geq 0\} \wedge \{(q_y - c_y)(d_y - q_y) \geq 0\}$; // тест пересечения
 $\}$ $\square$

На рис. 1.75,а представлен разработанный на основе метода логических кодов алгоритм $reg_outclip2(p_1p_2, O, Ls, *)$ **внешнего отсечения расположенных на плоскости отрезка p_1p_2 регулярным окном $O = o_1o_2o_3o_4o_1$** . Символьный аргумент $* \in \{<, \leq\}$ задаёт тип внешнего отсечения:

- $\leq$ — внешнее, сохраняющее внутренние и граничные части отрезка;
- $<$ — внешне-граничное, сохраняющее внутренние части отрезка.

Алгоритм возвращает **признак видимости отрезка в окне**:

- 0 — отрезок полностью отсечён окном и невидим;
- 1 — отрезок видим в окне целиком или частично.

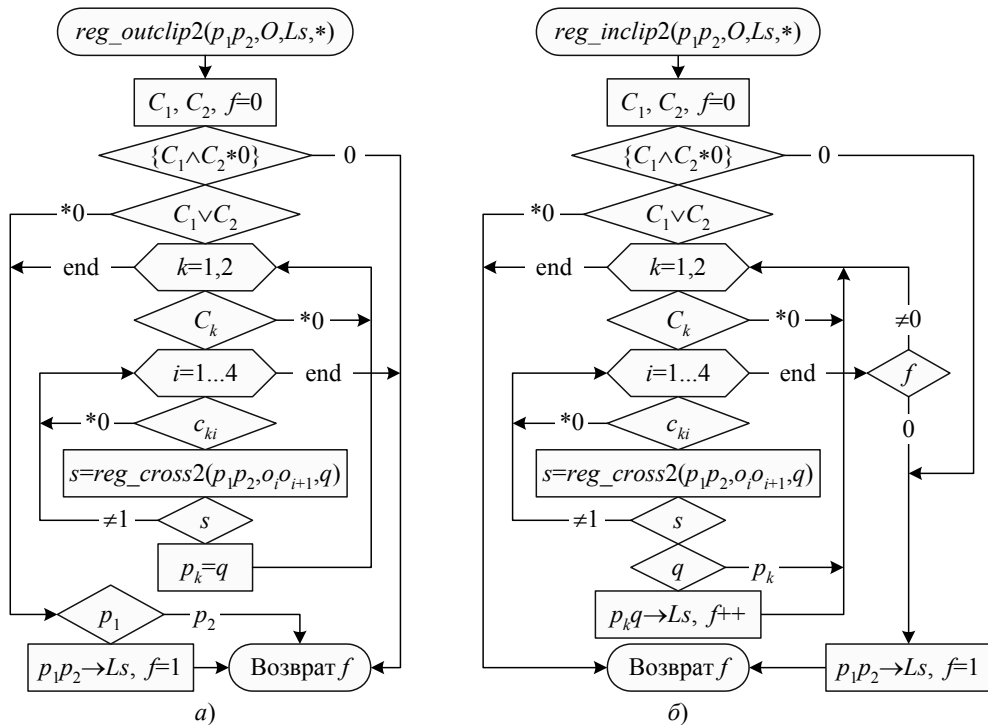


Рис. 1.75

На рис. 1.75,б приведена блок-схема алгоритма $reg_inclip2(p_1p_2, O, Ls, *)$ **внутреннего отсечения отрезка p_1p_2 регулярным окном $O = o_1o_2o_3o_4o_1$** . Символ $* \in \{<, \leq\}$ задаёт тип внутреннего отсечения:

- $<$ — внутреннее, сохраняющее внешние и граничные части отрезка;
- $\leq$ — внутренне-граничное, сохраняющее внешние части отрезка.

Алгоритм возвращает **признак видимости отрезка вне окна**:

- 0 — отрезок полностью отсечён окном и невидим;
- 1 — отрезок видим вне окна **одним** фрагментом целиком или частично;
- 2 — отрезок видим частично **двумя** фрагментами.

В обоих алгоритмах по (1.80) формируются логические коды концов отрезка C_1 и C_2 , после чего видимые **не вырожденные в точку** фрагменты **добавляются** в список отрезков Ls , который перед первым обращением к функциям $reg_outclip2$ и $reg_inclip2$ нужно инициализировать пустым. Накопленное содержимое этого списка может быть использовано другими алгоритмами, например, сборки замкнутых контуров $cont1$, $conts$ или $cont$ (см. п. 1.3.4).

Отсечение отрезка выпуклым полигоном

Распространим метод логических кодов на задачу отсечения отрезка p_1p_2 **выпуклым** полигоном $O=o_1o_2\dots o_n o_1$. Подобные отсекатели возникают, например, при расчёте тени или отражения отрезка на выпуклом участке плоскости.

Приняв за основу алгоритмы регулярного отсечения, внесём в них и формулы (1.80), (1.82) изменения, реализованные в блок-схемах на рис. 1.76:

- при $dir_test(O)=-1$ заменим полигон O реверсивным полигоном $\leftrightarrow O = o_1 o_n \dots o_1$ для создания у отсекателя положительного направления обхода;
- цикл $i=1\dots 4$ заменим циклом $i=1\dots n$;

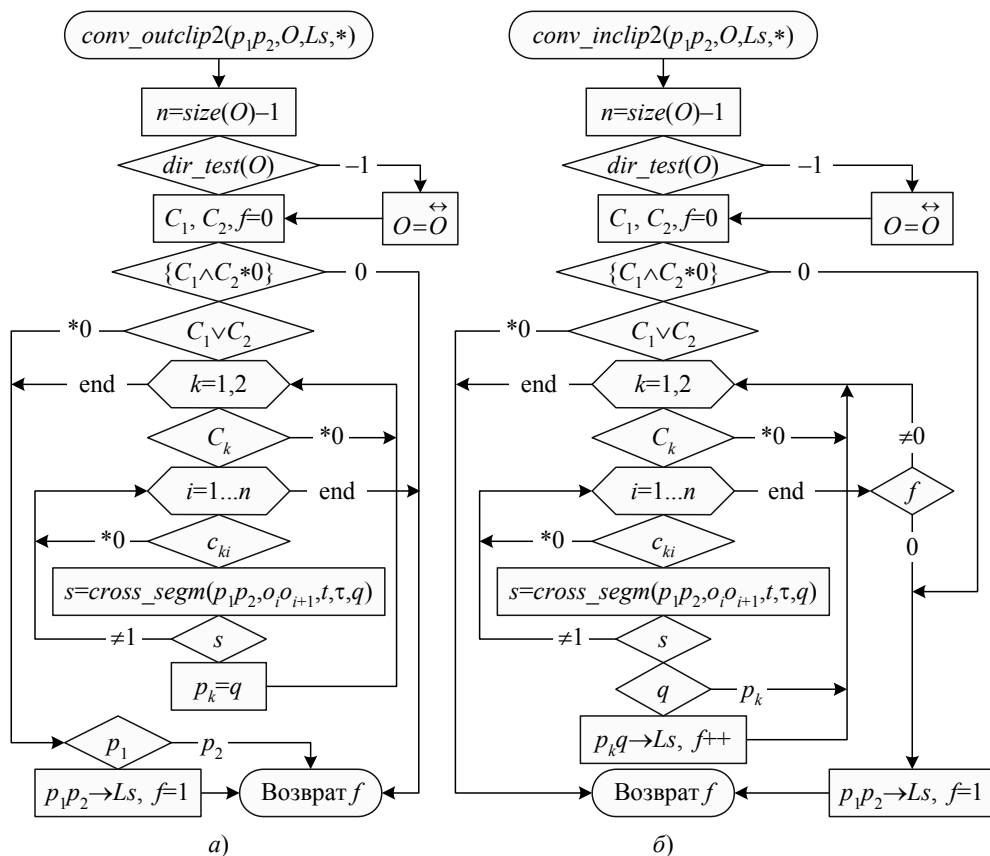


Рис. 1.76

• изменим правила расчёта n -разрядного логического кода C ориентации точки p относительно сторон **выпуклого** окна. Используя знаковые свойства не-явной функции (1.19г), можно найти разряды логического кода точки как

$$c_i = \text{sgn}(\text{nf}2(o_i, o_{i+1}, p)) \quad \forall i = \overline{1, n}. \quad (1.83a)$$

Второй способ их вычисления заключается в построении по (1.9) векторов правых нормалей сторон окна $N_i = \text{Rrot}(o_{i+1} - o_i)$, которые при $\text{dir_test}(O) = 1$ будут **внешними**. Тогда значение разряда c_i определяется **остротой угла** между вектором $p - o_i$ и правой нормалью N_i (рис. 1.77):

$$c_i = \text{sgn}((p - o_i) \circ N_i) \quad \forall i = \overline{1, n}; \quad (1.83б)$$

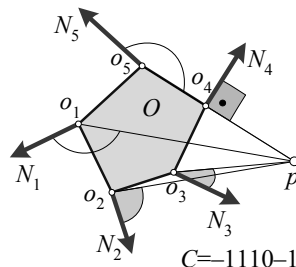


Рис. 1.77

• регулярную функцию `reg_cross2` заменим функцией `cross_segmt` расчёта пересечения произвольных отрезков (см. п. 1.1.2).

В результате получим **алгоритмы внешнего** `conv_outclip2(p1p2, O, Ls, *)` и **внутреннего** `conv_inclip2(p1p2, O, Ls, *)` **отсечений отрезка p_1p_2 выпуклым полигоном** $O = o_1o_2 \dots o_n o_1$ с прежним смыслом символьного аргумента $* \in \{<, \leq\}$. Обе функции возвращают в своих именах число видимых фрагментов отрезка p_1p_2 , добавленных в возвращаемый список Ls .

☞ Можно создать алгоритм отсечения отрезка ab полигоном O без расчёта логических кодов. Секрет фрагментации отрезка между его концами и точками пересечения с рёбрами полигона прост: **каждый фрагмент расположен так же, как его средняя точка**, локализуемая подходящим тестом из п. 1.1.4.

Первая реализация этой идеи представлена на рис. 1.78,а блок-схемой **алгоритма** `clip2_test(ab, O, Ls, *)` **внешнего отсечения отрезка ab выпуклым полигоном** $O = o_1o_2 \dots o_n o_1$ (рис. 1.78,б). Символ $* \in \{>, \geq\}$ задаёт тип отсечения:

- $>$ — внешнее, сохраняющее внутренние и граничные части отрезка;
- $\geq$ — внешне-граничное, сохраняющее внутренние части отрезка.

Работа алгоритма начинается с вычисления вектора отрезка $V = b - a$ и инициализации списка $L = \{0, 1\}$ из двух концевых значений параметра $t \in [0, 1]$ модели $p(t) = a + Vt$. Затем в цикле $i = \overline{1, n}$ обхода рёбер отсекаателя рассчитывается пересечение отрезков ab и $o_i o_{i+1}$ по алгоритму `cross_segmt` (см. п. 1.1.2). Если отрезки пересекаются либо касаются, то параметр t_i добавляется в L **логически**, пропуская кратные точки пересечения с вершинами (рис. 1.78,в). По достижению размера списка $\text{size}(L) = 4$ цикл обхода рёбер прекращается.

После сортировки по возрастанию список $L = \{L_1 = 0, L_2, \dots, L_{M-1}, L_M = 1\}$ разбивает отрезок ab точками $q_k = p(L_k)$ на $M - 1 \leq 3$ соединённых друг с другом фрагментов $[q_k, q_{k+1}]$ с номерами $k = \overline{1, M - 1}$. Выпуклым тестом локализации `conv2_test` (см. рис. 1.19,в) определяем расположение всего k -го фрагмента по его средней точке $m_k = 0.5(q_k + q_{k+1}) = a + 0.5(L_k + L_{k+1})V$. Фрагмент, **не удовлетво-**

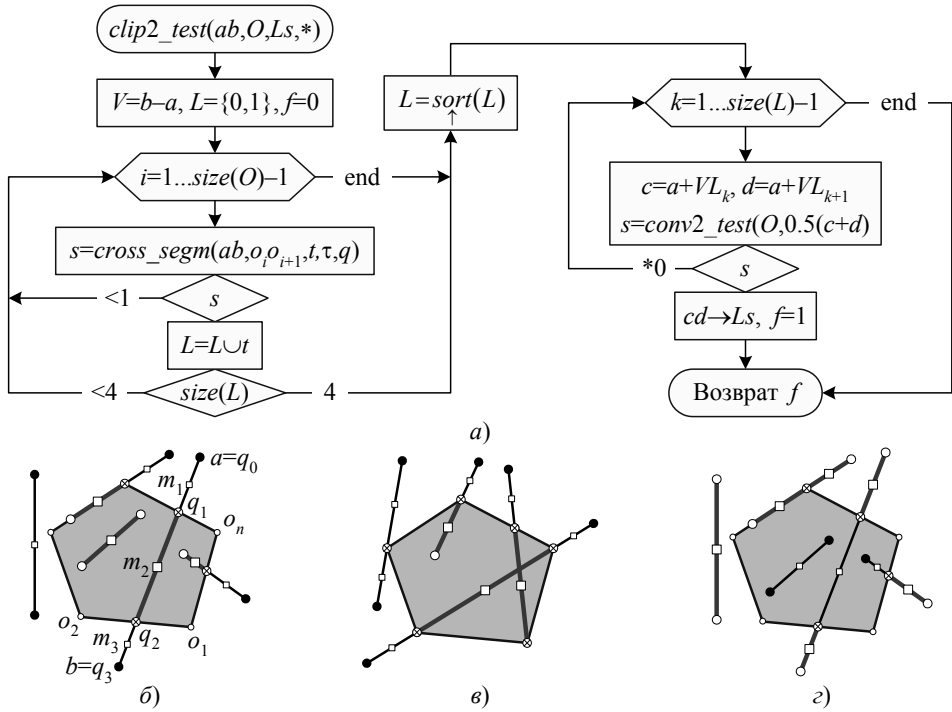


Рис. 1.78

ряющий условию $\text{conv2_test}(O, m_k) * 0$, добавляется в возвращаемый список отрезков Ls , а имя функции возвращает 1. При полном отсечении отрезка возвращаются исходный список Ls и нулевое число добавленных в него отрезков.

Идея локализации фрагмента отрезка лишь по одной его средней точке удачно вписывается в задачи с разными типами отсечения и с произвольной формой отсекаемых. Две из них — **внутреннее** ($= <$) и **внутренне-граничное** ($= \leq$) отсечения **выпуклым** полигоном по условию $\text{conv2_test}(O, m_k) * 0$ (рис. 1.78, з). Число видимых фрагментов может быть от нуля (весь отрезок закрыт внутренней областью полигона) до **двух** (экранирован средний фрагмент отрезка). Поэтому алгоритм `clip2_test` должен проверять локализацию **всех** фрагментов отрезка, а видимые накапливать в возвращаемом списке Ls .

Отсечение отрезка произвольным полигоном

При отсечении отрезка **невывуклым** полигоном число видимых фрагментов может быть самым разным, а для их обнаружения нужно использовать угловые `rad_test`, `oct_test` или лучевой `ray2_test` тесты локализации. В зависимости от расположения отсекаемых фрагментов определим **типы отсечений**:

- **внешнее** (отрезков ab на рис. 1.79, а);
- **внутреннее** (отрезков ab на рис. 1.79, б);
- **внешне-граничное** (отрезка cd на рис. 1.79, а);
- **внутренне-граничное** (отрезка cd на рис. 1.79, б);

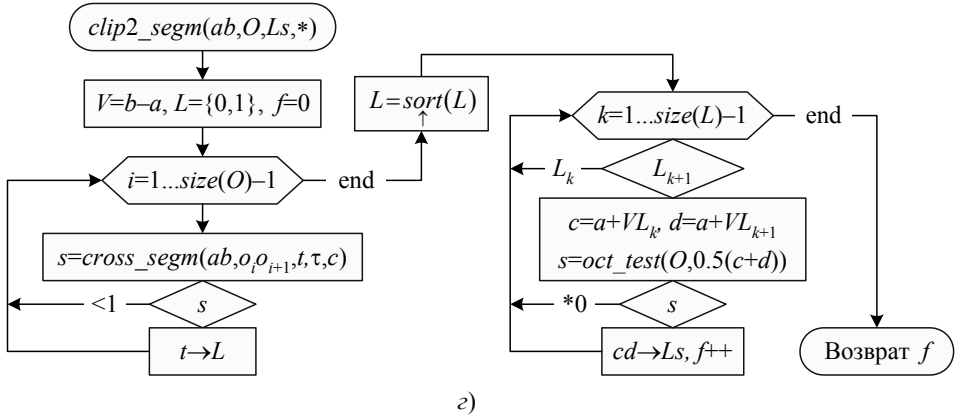
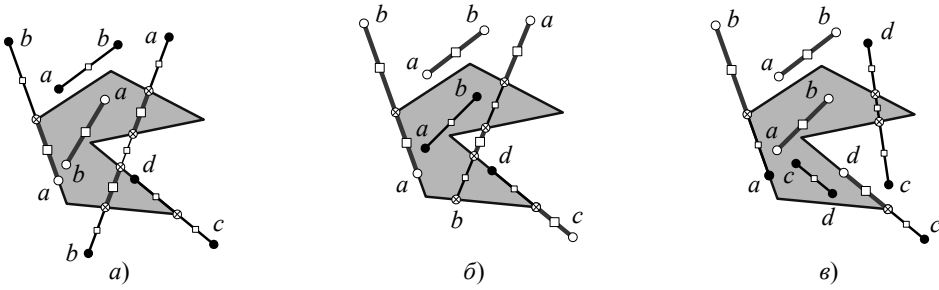


Рис. 1.79

- **граничное** (отрезков ab на рис. 1.79,а);
- **внеграницное** (отрезков cd на рис. 1.79,в).

Разработаем **универсальный алгоритм** $clip2_seg(ab, O, Ls, *)$ **плоского** ***-отсечения отрезка ab произвольным полигоном O** (рис. 1.79,з), накапливающий видимые фрагменты отрезка в возвращаемом списке Ls , а в имени функции — число добавленных отрезков. Тип отсечения закодирован символом $* \in \{>, <, \geq, \leq, =, \neq\}$, подставляемым в **условие отсечения** $s*0$ по результату $s \in \{-1, 0, 1\}$ теста локализации средней точки фрагмента. Фрагменты $cd \subseteq ab$, **не удовлетворяющие** условию $s*0$, добавляются в список отрезков Ls .

Другие отличия алгоритма $clip2_seg$ от алгоритма $clip2_test$:

- логическое добавление параметра t_i в точке пересечения отрезков ab и $o_i o_{i+1}$ заменено простым его добавлением в конец списка L с учётом будущего пропуска k -го фрагмента нулевой длины с концевыми параметрами $L_k = L_{k+1}$;
- снято ограничение размера списка L в четыре элемента, т. к. **невыпуклый** полигон может пересекаться с отрезком в более чем двух точках;
- вместо выпуклого теста локализации $conv2_test$ применён универсальный целочисленный oct_test (см. рис. 1.24,з). Подходят также rad_test и $ray2_test$.

Статус списка Ls как входного и выходного позволяет многократно накапливать в нём фрагменты разных отрезков. **Перед первым обращением к функции $clip2_seg$ нужно инициализировать пустой список $Ls = \emptyset$.**

Ради единообразия обозначений введём синонимы алгоритмов внешне-граничного, внутренне-граничного и внеграничного отсечений отрезка ab произвольным полигоном O :

$$\begin{aligned} outclip2(ab, O, Ls) &\equiv clip2_segm(ab, O, Ls, \geq), \\ inclip2(ab, O, Ls) &\equiv clip2_segm(ab, O, Ls, \leq), \\ neclip2(ab, O, Ls) &\equiv clip2_segm(ab, O, Ls, \neq). \end{aligned} \quad (1.84)$$

Пример 1.11. Выполнить внешне-граничное, внутренне-граничное и внеграничное отсечения отрезка ab невыпуклым окном $O = \{o_1 \dots o_{12} o_1\}$ (рис. 1.80, а).

Решение. Инициализируем список $L = \{0, 1\}$ и добавим в него параметры t_i точек пересечения отрезка со сторонами окна:

- $cross_segm(ab, o_1 o_2, t_1, \tau, c) = 1$, $t_1 = 0 \Rightarrow L = \{0, 1, 0\}$;
- $cross_segm(ab, o_2 o_3, t_2, \tau, c) = 0$, $t_2 = 1.09$;
- $cross_segm(ab, o_3 o_4, t_3, \tau, c) = 0$, $t_3 = 0.55$;
- $cross_segm(ab, o_4 o_5, t_4, \tau, c) = 0$, $t_4 = 0.91$;
- $cross_segm(ab, o_5 o_6, t_5, \tau, c) = 1$, $t_5 = 0.82 \Rightarrow L = \{0, 1, 0, t_5\}$;
- $cross_segm(ab, o_6 o_7, t_6, \tau, c) = -1$;
- $cross_segm(ab, o_7 o_8, t_7, \tau, c) = 1$, $t_7 = 0.55 \Rightarrow L = \{0, 1, 0, t_5, t_7\}$;
- $cross_segm(ab, o_8 o_9, t_8, \tau, c) = 1$, $t_8 = 0.36 \Rightarrow L = \{0, 1, 0, t_5, t_7, t_8\}$;
- $cross_segm(ab, o_9 o_{10}, t_9, \tau, c) = -1$;
- $cross_segm(ab, o_{10} o_{11}, t_{10}, \tau, c) = 1$, $t_{10} = 0.18 \Rightarrow L = \{0, 1, 0, t_5, t_7, t_8, t_{10}\}$;
- $cross_segm(ab, o_{11} o_{12}, t_{11}, \tau, c) = 0$, $t_{11} = 0.82$;
- $cross_segm(ab, o_{12} o_1, t_{12}, \tau, c) = 1$, $t_{12} = 0 \Rightarrow L = \{0, 1, 0, t_5, t_7, t_8, t_{10}, 0\}$.

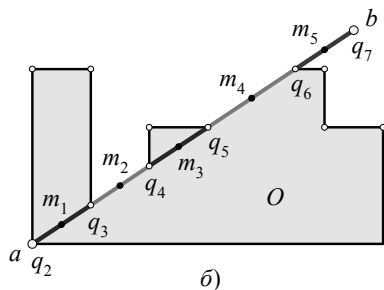
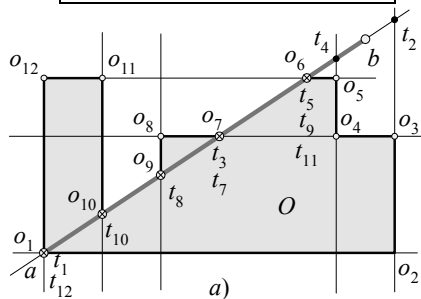


Рис. 1.80

Отсортируем по возрастанию список параметров $L = \{0, 0, 0, t_{10}, t_8, t_7, t_5, 1\}$ и зафиксируем концы фрагментов $q_2 = a$, $q_3 = o_{10}$, $q_4 = o_9$, $q_5 = o_7$, $q_6 = o_6$ и $q_7 = b$ (рис. 1.80, б). Тест расположения их средних точек $m_1 \div m_5$ даёт значения $s_1 = -1$, $s_2 = 0$, $s_3 = -1$, $s_4 = 0$ и $s_5 = 1$, по которым составляем списки внешне-граничного $Ls^{out} = \{q_2 q_3, q_4 q_5\}$, внутренне-граничного $Ls^{in} = \{q_6 q_7\}$ и внеграничного $Ls^{ne} = \{q_3 q_4, q_5 q_6\}$ отсечений. $\square$

1.4.2. Отсечение полигонов

На основе разработанного универсального метода отсечения отрезка и реализующих его функций (1.84) приступим к решению **главной задачи плоского**

отсечения произвольного полигона P **произвольным** полигональным окном O . Объектами отсечения полигонов являются их рёбра. Из видимых фрагментов рёбер, накопленных в предварительно инициализированном списке $Ls = \emptyset$, собираются желаемые графические объекты. В случае невыпуклости любого из взаимодействующих полигонов, а также при внутреннем отсечении, возможна **фрагментация** видимых элементов, т. е. расчленение отсекаемого объекта на несвязные фрагменты, которые алгоритм отсечения должен уметь формировать.

На рис. 1.81 приведены примеры фрагментарного отсечения полигона $P = p_1 p_2 p_3 p_4 p_1$ полигональным окном $O = o_1 o_2 \dots o_7 o_1$. В результате внешнего отсечения (а) получаются фрагменты $P_1 = p_1 q_1 o_6 q_6 p_1$ и $P_2 = q_2 q_3 q_4 q_5 q_2$, а внутреннего (б) — фрагменты $P_3 = p_2 q_2 q_5 p_4 q_6 o_6 q_1 p_2$ и $P_4 = p_3 q_4 q_3 p_3$. Границы этих полигонов составляют:

- отрезки $p_1 q_1$, $q_2 q_3$, $q_4 q_5$ и $q_6 p_1$ — фрагменты рёбер полигона P , лежащие **внутри** полигона O . Они добавляются в список Ls в цикле **внешне-границного** отсечения рёбер P окном O :

$$outclip2(p_i p_{i+1}, O, Ls) \quad \forall i = \overline{1, size(P)-1}; \quad (1.85a)$$

- отрезки $q_3 q_4$, $q_5 q_2$, $q_1 o_6$ и $o_6 q_6$ — фрагменты рёбер полигона O , лежащие **внутри** полигона P . Они добавляются в список Ls в цикле **внешне-границного** отсечения рёбер O окном P :

$$outclip2(o_i o_{i+1}, P, Ls) \quad \forall i = \overline{1, size(O)-1}; \quad (1.85б)$$

- отрезки $q_1 p_2$, $p_2 q_2$, $q_3 p_3$, $p_3 q_4$, $q_5 p_4$ и $p_4 q_6$ — фрагменты рёбер полигона P , лежащие **вне** полигона O . Они добавляются в список Ls в цикле **внутренне-границного** отсечения рёбер P окном O :

$$inclip2(p_i p_{i+1}, O, Ls) \quad \forall i = \overline{1, size(P)-1}. \quad (1.85в)$$

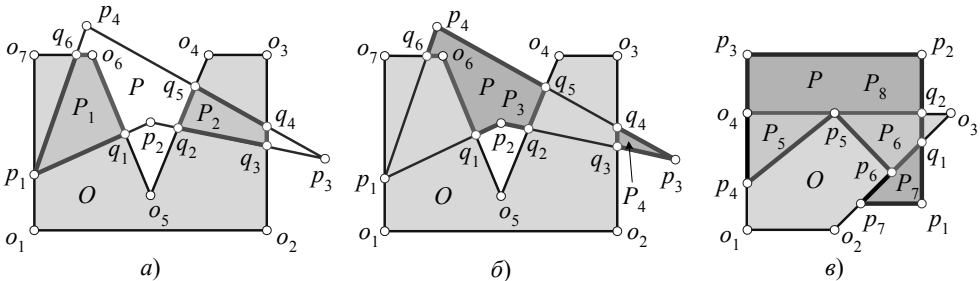


Рис. 1.81

Сборка отрезков может быть сделана разработанными в п. 1.3.4 алгоритмами *cont1* и *conts* (см. рис. 1.68, 1.69). Они правильно формируют один и множество **обособленных** контуров при условии, что в списке Ls нет **некорректных** (одиночных, тупиковых и кратных) отрезков и цепочек отрезков.

Более сложен расчёт отсечения объектов, имеющих общие границы. Например, полигоны $P = p_1 \dots p_7 p_1$ и $O = o_1 \dots o_4 o_1$ на рис. 1.81,в граничат друг с другом по двум отрезкам $o_4 p_4$ и $p_6 p_7$. Оба отрезка добавляются в список Ls в цикле **внешне-границного** отсечения рёбер полигона P окном O :

$$neclip2(p_i p_{i+1}, O, Ls) \quad \forall i = \overline{1, size(P)-1}. \quad (1.85\Gamma)$$

Сложность сборки контуров из накопленных отрезков заключается в **избыточности** списка Ls за счёт некоторых граничных отрезков. Например, в рёбрах фрагментов внешнего отсечения $\{P_5, P_6\}$ нет отрезка $p_6 p_7$, а в рёбрах внутреннего отсечения $\{P_7, P_8\}$ отсутствует отрезок $o_4 p_4$. Поэтому **сборку контуров можно доверить только функции** $cont(Ls)$ (см. рис. 1.71), способной «справиться» с некорректным списком отрезков Ls и касающимися контурами.

Оформим **алгоритмы внешнего** $outclip2_poly(P, O, LP)$ (**внутреннего** $inclip2_poly(P, O, LP)$) **отсечений полигона** $P = p_1 p_2 \dots p_n p_1$ **полигональным окном** $O = o_1 \dots o_m o_1$, возвращающие в своих именах список видимых отрезков Ls , а в аргументе LP — список контуров, собранных из этих отрезков.

Шаг 1. Инициализация пустого списка отрезков $Ls = \emptyset$ и расчёт рёберных размеров полигонов $n = size(P) - 1$ и $m = size(O) - 1$.

Шаг 2. Внешне-граничное $outclip2(p_i p_{i+1}, O, Ls)$ (внутренне-граничное $inclip2(p_i p_{i+1}, O, Ls)$) $\forall i = \overline{1, n}$ отсечение рёбер полигона P полигоном O . В результате внутренние (внешние) фрагменты отсекаемых рёбер добавляются в список Ls .

Шаг 3. Внешне-граничное $outclip2(o_j o_{j+1}, P, Ls)$ $\forall j = \overline{1, m}$ отсечение рёбер полигона O полигоном P . При этом внутренние фрагменты отсекаемых рёбер добавляются в список Ls .

Шаг 4. Внеграничное $neclip2(p_i p_{i+1}, O, Ls)$ $\forall i = \overline{1, n}$ отсечение рёбер полигона P полигоном O . Общие граничные фрагменты рёбер добавляются в Ls .

Шаг 5. Формирование видимых контуров в списке $LP = cont(Ls)$.

Шаг 6. Возврат Ls .



Особенность операции **внутреннего** отсечения полигонов — возможность получения списка LP , в котором один контур, назовём его Q , охватывает другой контур, например, O (рис. 1.82). Такие контуры обнаруживаются тестом окружения $surr2_test(Q, O) = 1$ (см. рис. 1.29). Фактически они представляют собой один **полигон** Q с **отверстием** O . Сформируем замкнутый контур такого полигона с помощью двух **фиктивных рёбер** $q_1 o_1$ и $o_1 q_1$, положение которых закодируем особым символом, например, «Ф»:

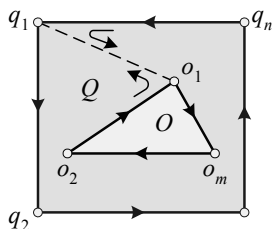


Рис. 1.82

$$P = q_1 q_2 \dots q_n q_1 \Phi o_1 o_m \dots o_2 o_1 \Phi q_1. \quad (1.86)$$

Таким образом, для правильного изображения полигонов следует каждый элемент списка LP протестировать на окружение другим и заменить внешний и внутренний контуры одним полигоном с отверстием, устроенным по (1.86).

Логические операции с полигонами

Логические операции с полигонами занимают важное место в инструментальном арсенале компьютерной графики. Они позволяют реализовать эффекты слияния и взаимного экранирования площадных объектов без применения многослойной технологии визуализации.

Рассмотрим выполнение трёх логических операций с полигонами A и B . Анализ рис. 1.83 убеждает, что внешнее отсечение (a) полигона A окном B функцией $outclip2_poly(A, B, LP)$ выполняет логическую операцию **пересечения** (конъюнкции, произведения) $A \cap B$, а внутреннее отсечение (b) функцией $inclip2_poly(A, B, LP)$ реализует логическую операцию **исключения** (вычитания) $A - B = A \cap (\neg B)$. Каждая функция возвращает результат в выходном списке полигонов LP . Оформим эти логические операции в виде следующих функций.

```


✂



$mul2(A, B)$



$\{ outclip2\_poly(A, B, LP);$   

возврат  $LP;$   

 $\}$  □



// пересечение полигонов  $A \cap B$


```

```


✂



$sub2(A, B)$



$\{ inclip2\_poly(A, B, LP);$   

возврат  $LP;$   

 $\}$  □



// исключение полигона  $A - B$


```

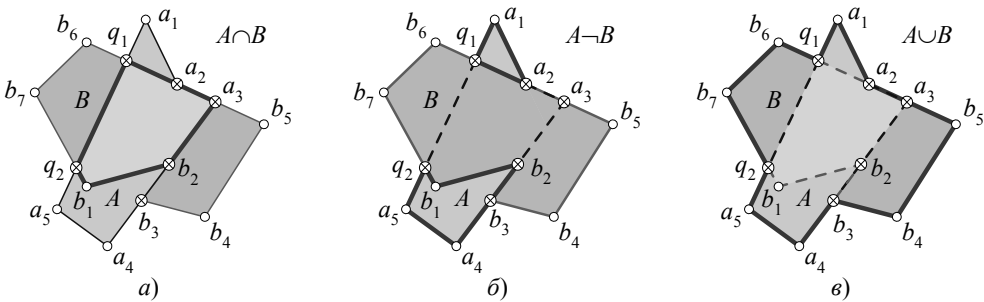


Рис. 1.83

Третья логическая операция **объединения** (дизъюнкции, суммы) полигонов $A \cup B$ (рис. 1.83, в) выполняется путём накопления фрагментов рёбер полигона A , **внешних** относительно окна B (a_1a_2 , b_3a_4 , a_4a_5 , a_5q_2 , q_1a_1), фрагментов рёбер B **внешних** относительно A (b_3b_4 , b_4b_5 , b_5a_3 , q_1b_6 , b_6b_7 , b_7q_2), и общих **граничных** фрагментов рёбер (a_2a_3 , b_2b_3) с последующей сборкой из них замкнутых контуров и запоминанием их в списке полигонов LP .

```


✂



$add2(A, B)$



$\{ Ls = \emptyset, n = size(A) - 1, m = size(B) - 1;$   

 $inclip2(a_i a_{i+1}, B, Ls) \forall i = \overline{1, n};$



// объединение полигонов  $A \cup B$



// инициализация списка отрезков



// внешние рёбра  $A$


```

```

inclip2( $b_j, b_{j+1}, A, Ls$ )  $\forall j = \overline{1, m}$ ; // внешние рёбра  $B$ 
neclip2( $a_i, a_{i+1}, B, Ls$ )  $\forall i = \overline{1, n}$ ; // граничные рёбра  $A$  и  $B$ 
возврат  $cont(Ls)$ ; // сборка контуров
} □

```

Результатом объединения непересекающихся полигонов будет список из тех же полигонов. При объединении пересекающихся **невыпуклых** полигонов возможно получение **полигона с отверстием** (см. рис. 1.82), выявляемого тестом окружения $surr2_test$ и формируемого аналогично (1.86).

1.4.3. Отсечение полуплоскостями

Как известно, прямая линия разделяет плоскость на две неограниченные области с противоположными знаками НФ прямой $f(p)$. Далее **полуплоскостью** будем называть **отрицательно** определённую область с $f(p) < 0$.

Рассмотрим задачу вычисления области пересечения полуплоскостей. Если эта область не пуста, то она безусловно **выпукла** как пересечение **выпуклых** объектов. Способ решения задачи заключается в последовательном внешнем отсечении текущей выпуклой области очередной полуплоскостью (рис. 1.84). В результате после каждого отсечения оставшаяся (заштрихованная) область:

- сужается при пересечении с полуплоскостью (*а*);
- не изменяется, если вся она лежит в отсекающей полуплоскости (*б*);
- становится пустой, если не пересекается с полуплоскостью (*в*).

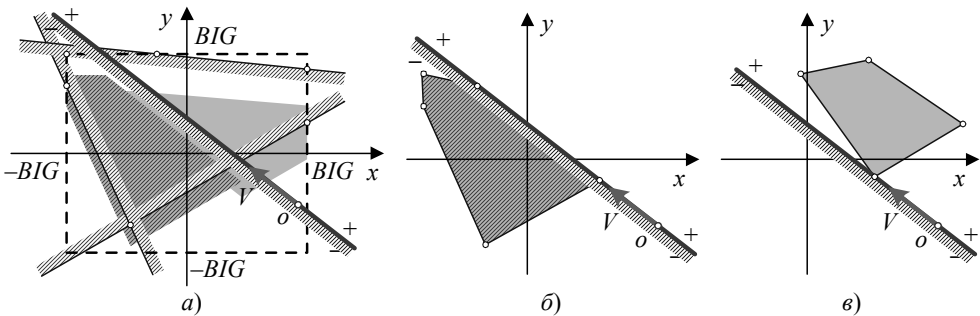


Рис. 1.84

Выберем наилучший способ задания граничной линии полуплоскости. Сравнивая модели прямой в п. 1.1.1 и варианты её неявной формы $f(p)$ в (1.19), видим, что наиболее практичны ПФ $\{o, V\}$ и её функция $nfV(o, V, p)$. Для описания прямой этим методом нужно 4 числа, а для вычисления значения $f(p)$ по (1.19в) — 5 арифметических операций. Такие же затраты ресурсов имеют нормальная форма $\{o, N\}$ и её функция $nfN(o, N, p)$, но ПФ предпочтительнее для задания направления прямой вектором V .

Если отсекаемая область не является замкнутым полигоном, возникает естественный вопрос о способе описания такой **неограниченной** области. Для преодоления численной неопределённости зададим начальный полигон

$$big_box = p_1 p_2 p_3 p_4 p_1 \cdot BIG, \quad (1.87)$$

$$\text{где } p_1 = [-1 \ -1], p_2 = [1 \ -1], p_3 = [1 \ 1], p_4 = [-1 \ 1],$$

в виде выпуклой **ограниченной** области в форме квадрата с габаритными координатами $|x|=BIG$ и $|y|=BIG$, где BIG — максимально возможное **конечное** число, обеспечивающее надёжное пересечение области big_box с любой прямой (рис. 1.84,а). Задача пересечения неограниченных полуплоскостей сводится к поочерёдному отсечению ими частей большого, но всё же ограниченного выпуклого полигона (1.87), расположенных **справа** от граничных прямых.

Отсечение выпуклого полигона

На рис. 1.85,а построен алгоритм $clip_line(P, o, V)$ отсечения **выпуклого** полигона $P = p_1 p_2 \dots p_n p_1$ **полуплоскостью**, лежащей **слева** от граничной прямой $\{o, V\}$ с вектором направления V (при $|V|=0$ отсечение отменяется и возвращается исходный полигон P). Работа начинается с расчёта расположения вершин полигона $p_i \ \forall i = \overline{1, n}$ относительно прямой (рис. 1.85,б), для чего вычисляются значения НФ $f_i = nfV(o, V, p_i)$ и анализируются максимальный $f_{\max} = \max\{f\}$ и минимальный $f_{\min} = \min\{f\}$ элементы вектора f :

- при $f_{\max} \leq 0$ весь полигон P , лежащий в отсекающей полуплоскости, возможно, касаясь её границы, возвращается алгоритмом без изменения;

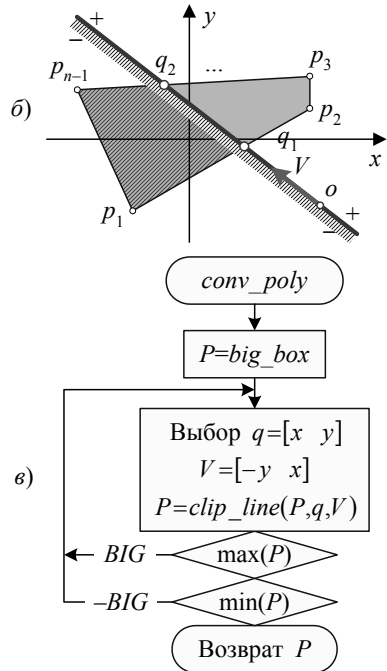
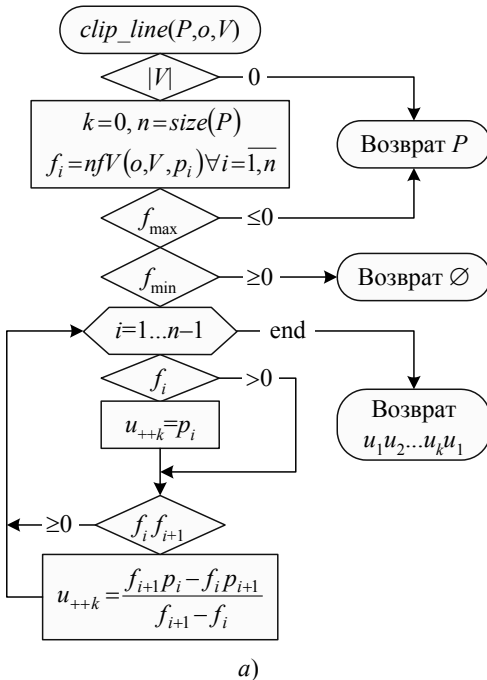


Рис. 1.85

- при $f_{\min} \geq 0$ весь полигон расположен справа от прямой или касается её вершиной либо ребром. В этом случае алгоритм возвращает пустой результат;
- оставшееся условие $\{f_{\min} < 0\} \wedge \{f_{\max} > 0\}$ означает, что существует хотя бы одна пара вершин $\{p_i, p_j\}$, лежащих по разные стороны от прямой $\{o, V\}$. Так как полигон выпуклый, то прямая пересекает его ровно в **двух** точках q_1 и q_2 . Отрезок между этими точками вместе с частью полигона, расположенной левее отсекающей прямой, составляет искомым полигон $U = u_1 u_2 \dots u_k u_1$.

Для нахождения этого полигона инициализируем значение $k=0$ и организуем цикл $i=\overline{1, n}$ обхода рёбер $p_i p_{i+1}$ с добавлением в список U :

- вершин p_i , лежащих слева или на граничной прямой. Эти точки обнаруживаются по неположительным значениям $f_i \leq 0$;
- точек $q_i = p_i(1 - \tau_i) + p_{i+1}\tau_i$ с параметрами $\tau_i = f_i / (f_i - f_{i+1})$ пересечения прямой $\{o, V\}$ с отрезками $p_i p_{i+1}$ при условии $f_i f_{i+1} < 0$ расположения концов рёбер по разные стороны от прямой.

Добавление точки в список U производится путём создания нового элемента u_{++k} . Поскольку конечная вершина полигона $p_{n+1} \equiv p_1$ анализу принадлежности полуплоскости не подвергалась, то по окончании цикла список содержит всегда не замкнутую ломаную линию $u_1 u_2 \dots u_k$. Завершающее действие алгоритма состоит в замыкании этой полилинии путём добавления к ней точки u_1 и возвращения итогового полигона $U = u_1 u_2 \dots u_k u_1$. Заметим, что **направления обхода вершин полигонов U и P совпадают**.

На основе описанного метода отсечения работает **алгоритм генерирования случайного выпуклого полигона *conv_poly*** (рис. 1.85, в), полезный для отладки тестов и алгоритмов. Алгоритм выполняет поочерёдное отсечение определённого в (1.87) выпуклого полигона $P = \text{big_box}$ случайно сгенерированными полуплоскостями. Секрет получения всегда не пустого решения — в способе выбора отсекающих линий, у которых начало системы координат всегда лежит слева от векторов направлений, т. е. в отрицательных полупространствах.

На плоскости задаётся либо случайно генерируется точка $q = [x \ y] \neq O_2$, через которую проводится прямая $\{q, N\}$ с векторами нормали $N = q$ и направления $V = [-y \ x]$. Отрицательное значение неявной функции

$$nfN(q, q, O_2) = -q \circ q < 0$$

доказывает, что начало СК находится слева от любой прямой, построенной таким способом. Генерирование полуплоскостей и отсечение ими полигона P продолжается до тех пор, пока в координатах его вершин остаются габариты исходного полигона — числа BIG либо $-BIG$. При полном отсечении от P всех границ объекта *big_box* алгоритм возвращает полученный полигон как решение задачи генерирования случайного выпуклого полигона.

Область применения метода отсечения выпуклого полигона полуплоскостями обширна: это целый круг задач, связанных с изображением сцен, отдельных объектов и оптических эффектов внутри **ограниченных и выпуклых** отсе-

кателей — граней, окон, зеркал... Задача невыпуклого отсечения сводится к объединению выпуклых решений после разрезания произвольных фигур на выпуклые фрагменты. Алгоритмы выполнения этих операций изучаются далее.

Расчёт ядра полигона

Ядро полигона — это выпуклое множество точек K (рис. 1.86,а), из которых видны все его вершины [23, 48]. Две точки полигона $a \in P$ и $b \in P$ друг другу видны, если полигону принадлежит весь отрезок $ab \subset P$. Любой выпуклый полигон совпадает со своим ядром (рис. 1.86,б), но не у каждого невыпуклого полигона есть ядро, что видно из рис. 1.86,в. Невыпуклые полигоны, имеющие ядро, относятся к подклассу *звёздных* [23, 48], алгоритмы генерирования которых *srpoly* и *poly_point* рассмотрены в п. 1.1.6 (см. рис. 1.33, 1.39).

В обиходном смысле ядро K полигона P — это то, что останется от исходной неограниченной плоскости после отрезания от неё всего, что находится по одну сторону от линий разреза — сторон полигона P — по завершении обхода его рёбер. Выбор направления этого обхода положительным приводит к отсечению плоскости отрицательными (левыми) полуплоскостями рёбер полигона, что позволяет использовать для этой операции алгоритм *clip_line*.

Заметим, что описанная процедура вовсе не равносильна отсечению полигональной фигуры **в процессе** обхода её сторон. В полигоне с вогнутыми вершинами последовательное отсечение удаляет существенные рёбра, формирующие ядро, в результате чего от полигона остаётся нечто другое (рис. 1.86,г).

Алгоритм $kern_poly(P)$ расчёта ядра полигона $P=p_1p_2\dots p_np_1$ работает согласно блок-схеме на рис. 1.86,д. Вначале с помощью функции *dir_test*(P), определённой в (1.55), находится направление обхода $d \in \{+1, -1\}$ исходного полигона P и устанавливается начальный отсекаемый полигон $K=big_box$. Затем

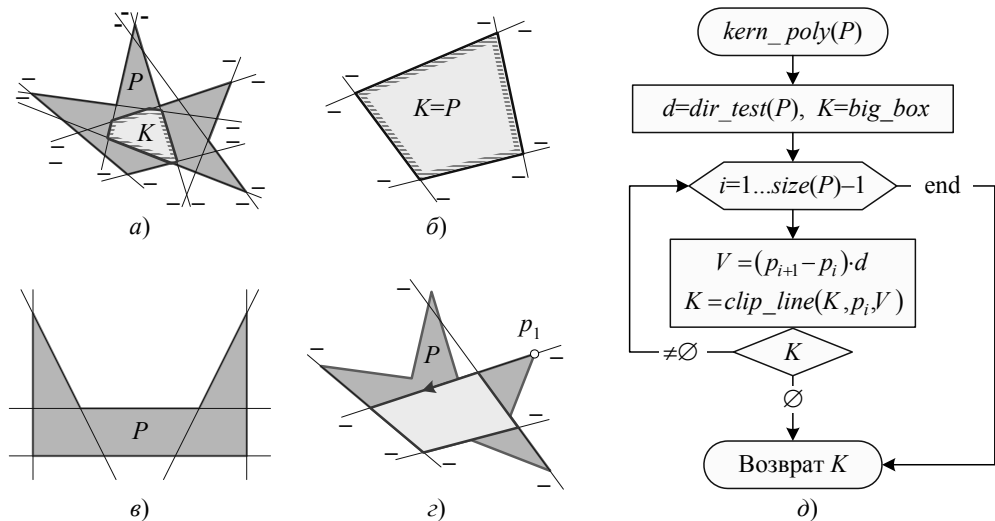


Рис. 1.86

для каждого ребра $p_i p_{i+1}$ вычисляется вектор направления $V_i = p_{i+1} - p_i$ и выполняется отсечение K прямой этого ребра: справа при положительном ($d=1$) либо слева при отрицательном ($d=-1$) направлении обхода. Если после очередного отсечения окажется $K=\emptyset$, то у полигона P ядро отсутствует. В противном случае по завершении цикла обхода всех рёбер полигона P возвращается его ядро K с **положительным** направлением обхода.

Пересечение выпуклых полигонов

Близкой к нахождению ядра полигона является реализация логической функции $A \cap B$ пересечения **выпуклых** полигонов A и B . Если в алгоритме *kern_poly* заменить P на A , K на B и отсекать от полигона B фрагменты, не принадлежащие полуплоскостям сторон полигона A , то получим полигон, принадлежащий как A , так и B , т. е. их пересечению $A \cap B$, если, конечно, оно существует. Пересечение лежит в ядре объединения выпуклых полигонов $A \cup B$.

На рис. 1.87,а построена блок-схема **алгоритма** *cross_poly(A,B)* **расчёта пересечения выпуклых полигонов** $A=a_1 \dots a_n a_1$ и $B=b_1 \dots b_m b_1$. Алгоритм возвращает выпуклый полигон $A \cap B$, если он, конечно, существует (рис. 1.87,б), либо пустой результат, если A и B касаются или не пересекаются (рис. 1.87,в). В алгоритме используется результат $d = \text{dir_test}(A)$ теста обхода полигона A , благодаря чему отсечение полигона B всегда выполняется с **внешней** стороны рёбер полигона A , т. е. справа при $d=1$ либо слева при $d=-1$.

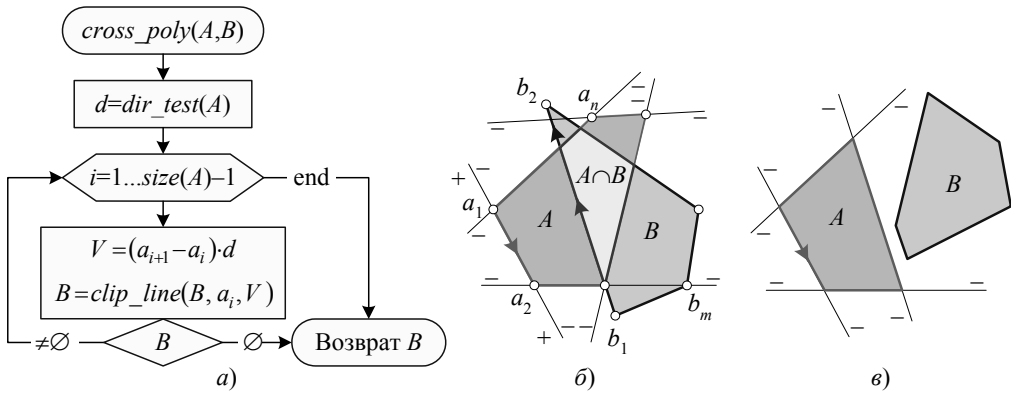


Рис. 1.87

Изложенный алгоритм решает задачу пересечения **выпуклых** полигонов гораздо эффективнее, чем универсальный алгоритм *mul2(A,B)* из п. 1.4.2. Его рекомендуется использовать совместно с простым тестом выпуклости полигона (см. рис. 1.15), если тот вернёт значения $\text{conv2}(A)=1$ и $\text{conv2}(B)=1$.

1.4.4. Разрезание полигонов

Операция **разрезания полигонов** широко распространена в различных прикладных программах и системах автоматизированного проектирования. Она ис-

пользуется в следующих целях (список, возможно, далеко не полон):

- для разделения полигона на независимые фрагменты, с которыми далее будут производиться индивидуальные преобразования;
- для замены невыпуклого полигона объединением выпуклых фрагментов, с которыми все тестовые и расчётные задачи выполняются проще и быстрее;
- для представления произвольных полигонов объединением однотипных простейших фигур — треугольников — с целью унификации и максимального упрощения полигональных операций;
- разрезание плоского полигона является составной частью трёхмерной задачи сечения многогранника плоскостью, в которой производится разрезание граней на фрагменты, из которых затем собираются многогранники.

Разрезание может быть **ручным** и **алгоритмическим**. Ручное разрезание невыпуклого объекта, даёт, естественно, минимальное число фрагментов за счёт интеллектуальных способностей человека. Если же объект **вычисляется** в процессе работы программы и заранее неизвестны его конфигурация и даже сам факт выпуклости, то необходимы тест проверки выпуклости и **алгоритмы** разрезания невыпуклого объекта на выпуклые фрагменты.

Разрезание прямой линией

Разрезание полигона $P=p_1p_2\dots p_np_1$ бесконечной **прямой** линией $\{o,V\}$ заключается в получении списка $LP=\{P_1,P_2,\dots,P_k\}$ полигонов $P_i\subseteq P$, лежащих по **обе** стороны от прямой (рис. 1.88). Если прямая и полигон не пересекаются, то итогом разрезания является исходный полигон, целиком лежащий по одну сторону от прямой.

Для разрезания **выпуклого** полигона можно дважды использовать алгоритм *clip_line* (см. рис. 1.85,а):

- левый выпуклый полигон $P_{\text{л}}=\text{clip_line}(P,o,V)$ образуется отсечением полигона P полупространством прямой $\{o,V\}$;
- правый выпуклый полигон $P_{\text{п}}=\text{clip_line}(P,o,-V)$ образуется отсечением полигона P полупространством прямой обратного направления $\{o,-V\}$;
- если $\{P_{\text{л}}\neq\emptyset\}\wedge\{P_{\text{п}}\neq\emptyset\}$, то $LP=\{P_{\text{л}},P_{\text{п}}\}$, иначе $LP=\{P\}$.

Недостатком данного подхода является двойной расчёт одних и тех же точек пересечения секущей прямой с рёбрами полигона. Попытаемся реализовать заложенную в алгоритме *clip_line* идею для **однопроходного** получения этих точек и разделения пересекаемых рёбер **произвольного** полигона на левые и правые отрезки. Как видно из рис. 1.88, рёбра отрезанных полигонов собраны из

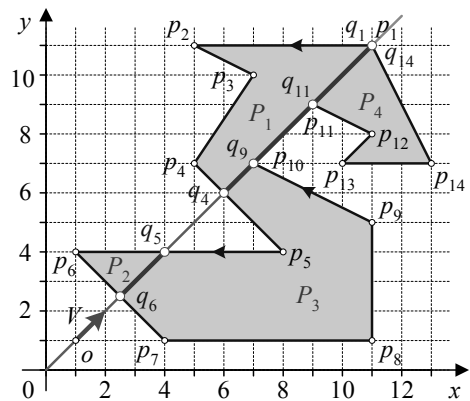


Рис. 1.88

Если же $f_{\min}f_{\max} < 0$, то разрезание возможно и начинается оценка расположения каждого ребра полигона $p_i p_{i+1}$, $i \in [1, n]$, относительно прямой $\{o, V\}$ и выполнение соответствующих действий (см. рис. 1.88):

- при $f_i = f_{i+1} = 0$ ребро, лежащее на секущей прямой (например, $p_{10}p_{11}$), пропускается;
- при $f_i f_{i+1} > 0$ левое от прямой ребро с $f_i < 0$ (например, p_3p_4) добавляется в список Ll , а правое с $f_i > 0$ (p_7p_8 , p_8p_9 , $p_{12}p_{13}$, $p_{13}p_{14}$) — в список Lr ;
- остальные рёбра с $f_i f_{i+1} \leq 0$ имеют с прямой по одной точке пересечения или касания $q_i = (p_i f_{i+1} - p_{i+1} f_i) / (f_{i+1} - f_i)$. Левая часть ребра $p_i q_i$ с $f_i < 0$ (отрезки p_4q_4 , p_6q_6) или $q_i p_{i+1}$ с $f_{i+1} < 0$ (отрезки q_1p_2 , q_5p_6) добавляется в список Ll , правая часть ребра $p_i q_i$ с $f_i > 0$ (отрезки p_9q_9 , $p_{14}q_{14}$) или $q_i p_{i+1}$ с $f_{i+1} > 0$ (отрезки q_4p_5 , q_6p_7 , $q_{11}p_{12}$) — в список Lr , а параметр $t_i = (q_i - o) \circ V$ расположения точки q_i на прямой $o + Vt$ добавляется в список L .

По окончании обхода рёбер списки левых Ll и правых Lr отрезков полностью сформированы. Сортировка списка параметров L упорядочивает точки q_i вдоль прямой, что позволяет разделить её на фрагменты $c_k d_k$. Применяя тест локализации к средним точкам $0.5(c_k + d_k)$, отбираем в списки Ll и Lr только граничные и внутренние отрезки (q_1q_{11} , $q_{11}q_9$, q_9q_4 и q_5q_6).

В завершение с помощью функции *cont* (см. рис. 1.71, а) выполняется сборка левых (в список LL) и правых (в список LR) полигонов из списков отрезков Ll и Lr . По окончании возвращается соединённый список полигонов $LP = \{LL, LR\}$. Заметим, что минимизация полученных контуров выполняется в алгоритме сборки, так что дополнительно её делать не нужно.

Пример 1.12. Разрезать полигон P на рис 1.88 прямой линией, проходящей через точку $o = [1 \ 1]$ в направлении $V = [1 \ 1]$.

Решение. Разные знаки крайних значений неявной функции в вершинах полигона $f_{\min} \equiv f_2 = -6 < 0$ и $f_{\max} \equiv f_8 = 10 > 0$ означают существование пересечения полигона P с прямой $\{o, V\}$. В цикле обхода рёбер вычисляем точки их пересечения с прямой $q_1 \equiv p_1$, $q_4 = [6 \ 6]$, $q_5 = [4 \ 4]$, $q_6 = [2.5 \ 2.5]$, $q_9 \equiv p_{10}$, $q_{11} \equiv p_{11}$, $q_{14} \equiv p_1$ и формируем следующие списки отрезков и параметров:

$$\begin{aligned} Ll &= \{p_1p_2, p_2p_3, p_3p_4, p_4q_4, q_5p_6, p_6q_6\}, \\ Lr &= \{q_4p_5, p_5q_5, q_6p_7, p_7p_8, p_8p_9, p_9p_{10}, p_{11}p_{12}, p_{12}p_{13}, p_{13}p_{14}, p_{14}p_1\}, \\ L &= \{t_1=10, t_4=5, t_5=3, t_6=2, t_9=6, t_{11}=8, t_{14}=10\}. \end{aligned}$$

После сортировки по возрастанию вектора параметров $L = \{t_6, t_5, t_4, t_9, t_{11}, t_1, t_{14}\}$ и локализации средних точек $o + 0.5(t_k + t_{k+1})V$ с $t_k < t_{k+1}$ получаем список внутренних и граничных отрезков разрезающей прямой $Ls = \{q_6q_5, q_4p_{10}, p_{10}p_{11}, p_{11}p_1\}$.

Сборка замкнутых контуров из отрезков соединённого списка

$$\{Ll, Ls\} = \{p_1p_2, p_2p_3, p_3p_4, p_4q_4, q_5p_6, p_6q_6, q_6q_5, q_4p_{10}, p_{10}p_{11}, p_{11}p_1\}$$

даёт левые полигоны $P_1 = p_1p_2p_3p_4q_4p_1$, $P_2 = q_5p_6q_6q_5$ (из полигона P_1 при минимизации были удалены коллинеарные вершины p_{10} и p_{11}), а из отрезков списка $\{Lr, Ls\}$ — правые полигоны

$P_3 = q_4 p_5 q_5 q_6 p_7 p_8 p_9 p_{10} q_4$, $P_4 = p_{11} p_{12} p_{13} p_{14} p_{11}$. При сборке полигона P_4 был обнаружен и удалён тупиковый отрезок $p_{10} p_{11}$.

Таким образом, решением задачи разрезания полигона P прямой линией $\{o, V\}$ является список полигонов $LP = \{P_1, P_2, P_3, P_4\}$. $\square$

Разрезание на выпуклые фрагменты

Данная операция заключается в проведении внутри произвольного полигона прямолинейных границ, по которым он распадается на выпуклые непересекающиеся фрагменты. Желательно, чтобы их число было минимальным.

Разработаем метод разрезания полигона $P = p_1 p_2 \dots p_n p_1$ *хордами* — продолжениями его рёбер (рис. 1.90,а). Ясно, что будучи выпуклым, в частности треугольным, полигон не нуждается в разрезании, т. к. продолжения рёбер проходят снаружи него. Невыпуклый же полигон можно начинать резать с любого *вогнутого* угла. Поэтому сначала исходный полигон P необходимо проверить по (1.39) тестом выпуклости $conv2(P)$. Идейная основа этого теста — одностороннее расположение вершин выпуклого полигона от каждого ребра — пригодна для обнаружения *вогнутого* угла $\angle p_i p_{i+1} p_{i+2}$ при вершине p_{i+1} по *совпадению* знака числа $nf2(p_i, p_{i+1}, p_{i+2})$ с направлением обхода полигона $d = dir_test(P)$.

Далее нужно рассчитать все пересечения луча $p_i \rightarrow p_{i+1}$ за точкой p_{i+1} с рёбрами $p_{i+2} p_{i+3}, \dots, p_n p_1, \dots, p_{i-2} p_{i-1}$ и найти *минимальный* параметр пересечения $\theta > 1$, номер k ближайшего пересечённого ребра, ближайшую точку пересечения $q = p_i + (p_{i+1} - p_i)\theta$ и линию разреза $p_{i+1} q$, по которой полигон P разделяется на два фрагмента — левый P_1 и правый P_2 .

Для простоты формирования списков вершин фрагментов будем после каждого обнаружения выпуклого угла делать *левый циклический сдвиг списка* вершин полигона $P = p_1 p_2 \dots p_n p_1$ (рис. 1.90,б) функцией

$$LCShift(P) = p_2 p_3 \dots p_n p_1 p_2. \quad (1.88)$$

Благодаря сдвигам тестируемая на выпуклость вершина имеет постоянный номер p_2 , луч $p_1 p_2$ проверяется на пересечение с рёбрами $p_3 p_4, \dots, p_{n-1} p_n$, а полигон разрезается на два фрагмента $P_1 = p_1 q p_{k+1} \dots p_n p_1$ и $P_2 = p_2 p_3 \dots p_k q p_2$.

При совпадении найденной точки q с вершиной полигона P у одного из фрагментов появятся кратные вершины, которые должны быть удалены. Возможно также совпадение луча с какой-либо стороной разрезаемого полигона (рис. 1.90,а), в результате чего один из фрагментов будет иметь вершину с развёрнутым углом. Поэтому фрагменты перед дальнейшим разрезанием необходимо минимизировать алгоритмом *min_poly* (см. рис. 1.18,а).

Описанная процедура разрезания невыпуклого полигона на два фрагмента рекурсивно применяется для полигонов P_1 и P_2 . Примерное дерево рекурсии, терминальные вершины которого есть искомые выпуклые полигоны, изображено на рис. 1.90,в. Выход из рекурсии с добавлением P в список полигонов LP осуществляется при условии возвращения тестом выпуклости полигона результирующего значения $conv2(P) = 1$.

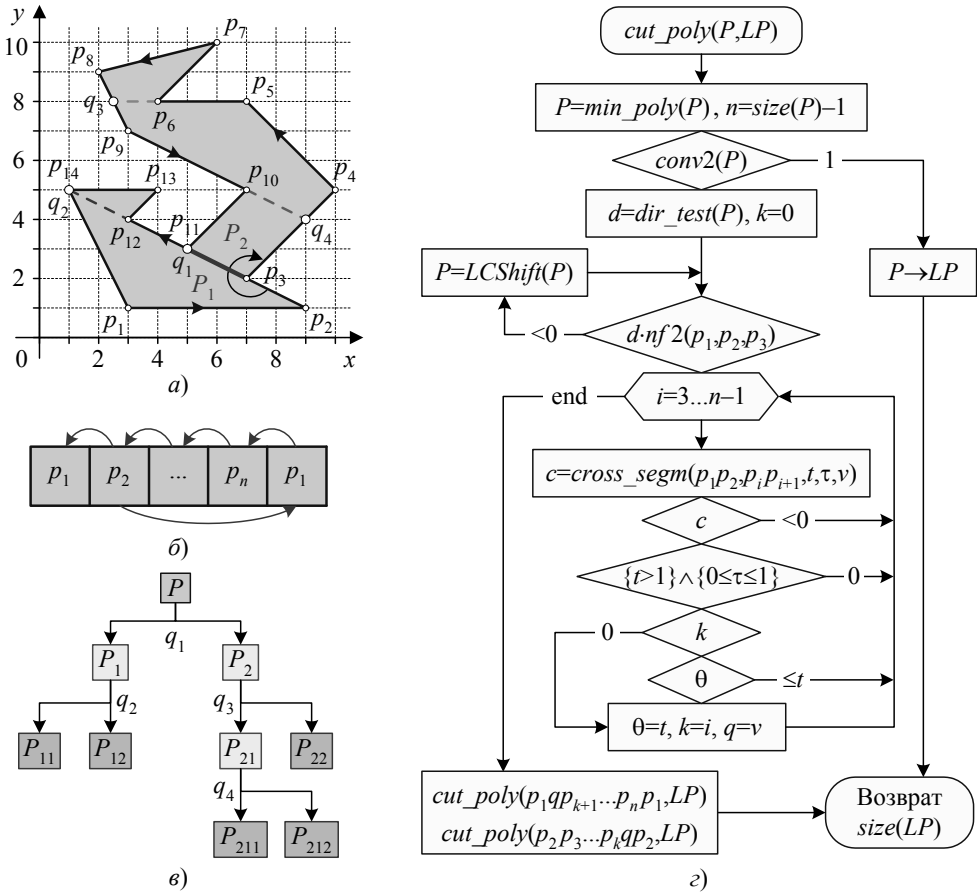


Рис. 1.90

Реализация всего сказанного в виде рекурсивной **функции** $cut_poly(P, LP)$ **разрезания полигона** P изображена на рис. 1.90, д. Функция формирует список LP выпуклых фрагментов и возвращает их количество. Перед внешним обращением к функции необходимо инициализировать пустой список полигонов $LP = \emptyset$, в который внутри функции будут добавляться выпуклые фрагменты, отрезанные от исходного полигона.

Необходимость расчёта множества пересечений прямых полностью уничтожает все преимущества описанного **алгоритмического** метода разрезания. Ручной метод разрезания невыпуклого полигона имеет меньшую трудоёмкость, чем алгоритмический. Тем не менее алгоритм cut_poly незаменим в задачах разрезания полигонов заранее неизвестной конфигурации, которая может быть решена только алгоритмически, а не вручную.

Пример 1.13. Проиллюстрируем работу алгоритма cut_poly на примере разрезания невыпуклого полигона $P = p_1 p_2 \dots p_{14} p_1$, изображённого на рис. 1.90, а. Тест (1.55) даёт положительное направление обхода $d = dir_test(P) = 1$. Дерево рекурсивного разрезания полигона построено на рис. 1.90, в.

Начинаем первый уровень рекурсии с проверки выпуклости угла $\angle p_1 p_2 p_3$. Так как $nf2(p_1, p_2, p_3) = -6 < 0$, то с учётом знака числа $d > 0$ заключаем, что этот угол выпуклый. После левого циклического сдвига вершин полигон принимает вид $P = p_2 p_3 \dots p_{14} p_1 p_2$. Вот теперь из равенства знаков чисел d и $nf2(p_2, p_3, p_4) = 8 > 0$ следует, что вершина p_3 является вогнутой. Для удобства анализа работы алгоритма номера вершин оставлены без изменения.

Луч $p_2 \rightarrow p_3$ пересекается с рёбрами $p_{10} p_{11}$, $p_{12} p_{13}$, $p_{13} p_{14}$ и $p_{14} p_1$. Из всех точек пересечения ближайшей к p_3 является точка q_1 , случайно совпавшая с вершиной p_{11} . Таким образом, полигон P разрезался по отрезку $p_3 q_1$ на два полигона:

$$P_1 = p_2 q_1 p_{11} p_{12} p_{13} p_{14} p_1 p_2, \quad P_2 = p_3 p_4 p_5 p_6 p_7 p_8 p_9 p_{10} q_1 p_3.$$

На втором уровне рекурсии полигон P_1 , имеющий кратные вершины $q_1 \equiv p_{11}$ и лежащие на одной прямой соседние вершины p_2 , q_1 и p_{12} , минимизируется алгоритмом *min_poly* до вида $P_1 = p_2 p_{12} p_{13} p_{14} p_1 p_2$. Далее полигон P_1 разрезается на составные части

$$P_{11} = p_2 q_2 p_{14} p_1 p_2 \equiv p_2 q_2 p_1 p_2, \quad P_{12} = p_{12} p_{13} q_2 p_{12},$$

а другой полигон P_2 — на фрагменты

$$P_{21} = p_5 q_3 p_9 p_{10} q_1 p_3 p_4 p_5, \quad P_{22} = p_6 p_7 p_8 q_3 p_6.$$

На третьем уровне рекурсии с помощью теста *conv2* обнаруживается, что полигоны P_{11} , P_{12} и P_{22} — выпуклые и в дальнейшем разрезании не нуждаются.

Оставшийся полигон P_{21} на четвёртом уровне разрезается на составляющие части

$$P_{211} = p_9 q_4 p_4 p_5 q_3 p_9, \quad P_{212} = p_{10} q_1 p_3 q_4 p_{10},$$

которые на последнем пятом уровне тестируются как выпуклые.

Таким образом, исходный полигон P оказался разрезан на пять выпуклых фрагментов, которые сохранились в списке полигонов $LP = \{P_{11}, P_{12}, P_{211}, P_{212}, P_{22}\}$.

□

1.4.5. Триангуляция полигонов

Триангуляцией называется операция разрезания полигона на треугольники. Она часто используется для сведения решения типовых задач (пересечения, отсечения, удаления, визуализации и т. п.) в области, имеющей сложную конфигурацию, к совокупности задач, решаемых в области треугольника — простейшей полигональной фигуры, важнейшими свойствами которой являются планарность и выпуклость. Что даёт замена многоугольника набором треугольников?

1. Существенно упрощается тест локализации точки q относительно треугольника $p_1 p_2 p_3 p_1$ (см. рис. 1.20,б). Представив её как $q = o + Vt + W\tau$ в косоугольной системе координат $\{o, V, W\}$ с началом в точке $o = p_1$ и направлениями осей $V = p_2 - p_1$ и $W = p_3 - p_2$, вычислим методом Грама (1.43) вектор координат

$$[t \ \tau] = (q - p_1) \begin{bmatrix} p_2 - p_1 \\ p_3 - p_2 \end{bmatrix}^{-1}$$

и тогда из (1.44б) следует, что при $0 < \tau < t < 1$ точка q лежит **внутри** треугольника $p_1 p_2 p_3 p_1$, при $\{\tau < 0\} \vee \{t < \tau\} \vee \{t > 1\}$ она расположена **вне** треугольника, а при $0 \leq \tau \leq t \leq 1$ — на его **границе**. Обращаемая в (1.43в) матрица невырождена при непараллельных векторах V и W , т. е. при неколлинеарных вершинах треугольника и отсутствии у него нулевых и развёрнутых углов.

2. Благодаря минимальному числу сторон треугольного полигона быстро выполняется их перебор в операциях отсечения и удаления. Априорная выпуклость треугольника позволяет существенно упростить все алгоритмы обработки полигонов путём использования только их выпуклых вариантов.

3. Структура данных для хранения m треугольников может иметь вид регулярной трёхмерной $3 \times 4 \times m$ -матрицы, а не более сложно организованных массивов, структур и списков.

4. Обобщение алгоритма триангуляции плоского полигона на криволинейную поверхность позволяет заменить её сеткой треугольников, для которых значительно проще решаются все задачи визуализации, в том числе закрашивания всех точек треугольной грани в постоянный цвет благодаря её планарности.

Триангуляция хордами

В принципе триангуляция произвольного полигона может быть выполнена **модифицированным алгоритмом** $cut_poly(P, LP)$, завершающая часть которого изображена на рис. 1.91,а. Суть модификации состоит в том, что в список LP вместо выпуклого n -угольника $P = p_1 \dots p_n p_1$ добавляется $n-2$ треугольников

$$p_1 p_2 p_3 p_1, p_1 p_3 p_4 p_1, \dots, p_1 p_{n-1} p_n p_1,$$

полученных разрезанием полигона диагоналями $p_1 p_3, \dots, p_1 p_{n-1}$ (рис. 1.91,б).

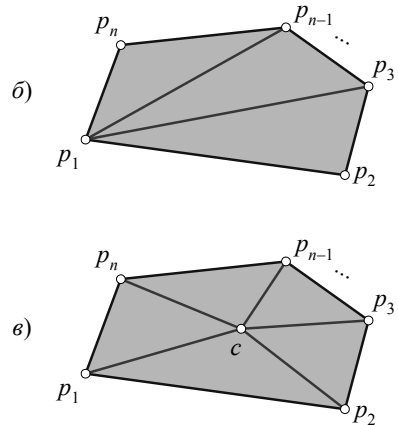
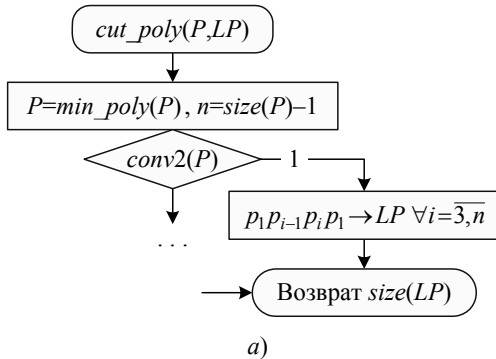


Рис. 1.91

Выпуклый полигон можно разрезать и другим способом — на n треугольников $p_1 p_2 c p_1, p_2 p_3 c p_2, \dots, p_m p_1 c p_n$ (рис. 1.85,в) с дополнительно вычисленной вершиной — точкой среднearифметического центра полигона $c = mean(P)$. Хотя при этом и получается большее число треугольных фрагментов, но зато они имеют менее вытянутую форму, чем при разрезании диагоналями.

Недостаток триангуляции алгоритмом cut_poly в том, что полигон разрезается **хордами**, пересечение которых с рёбрами создаёт новые точки. Да и число полученных треугольников далеко от минимального.

Триангуляция диагоналями

Рассмотрим более быстрый метод триангуляции, в котором полигон рекурсивно разрезается на части **диагоналями** (рис. 1.92,а). Задача заключается в поиске разрезающей диагонали $p_i p_j$, удовлетворяющей следующим условиям:

- является **внутренней** диагональю полигона;
- не пересекает и не касается не смежных с ней сторон полигона.

Рекурсивный алгоритм $tri_poly(P, LP)$ **диагональной триангуляции полигона** $P=p_1 p_2 \dots p_n p_1$, реализующий эти условия, построен на рис. 1.92,б. Его работа начинается с минимизации исходного полигона, исключаяющей из него кратные и коллинеарные вершины. Если полученный полигон оказался треугольником ($n=3$), то он добавляется в возвращаемый список LP .

Дальнейшие действия выполняются с не треугольным полигоном. В случае его выпуклости, что быстро проверяется тестом $conv2(P)=1$, полигон можно сразу резать диагональю $p_1 p_3$ на фрагменты $p_1 p_2 p_3 p_1$ и $p_1 p_3 \dots p_n p_1$.

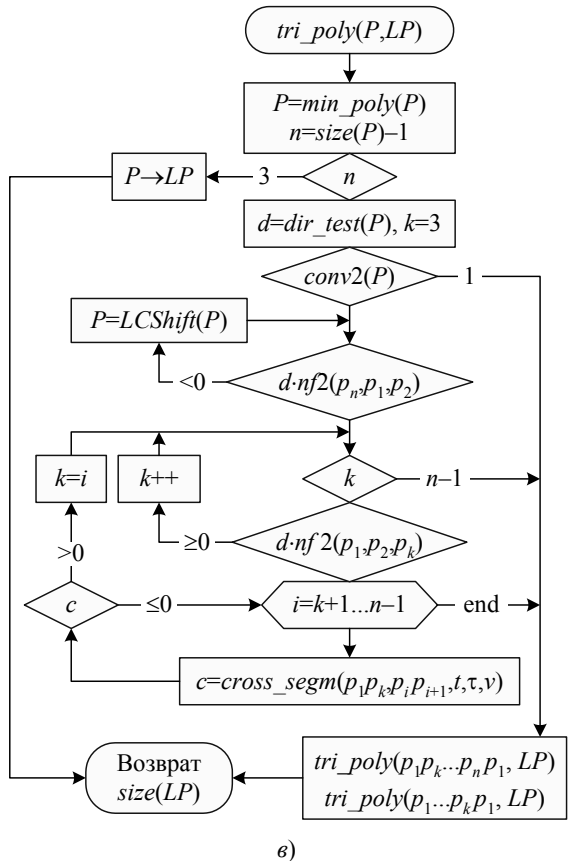
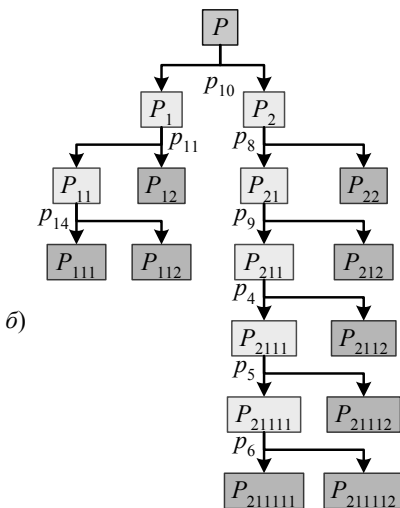
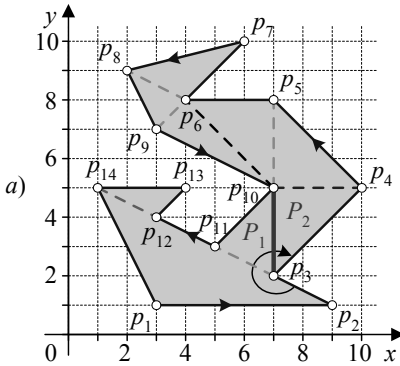


Рис. 1.92

У невыпуклого полигона сначала ищется вершина, из которой можно провести разрезающую диагональ. Для этого годится не каждая выпуклая вершина, чему пример — точки p_7 и p_{13} на рис. 1.92,а. В то же время из **любой вогнутой** вершины p_i напрямую видна хотя бы одна несмежная с ней вершина p_j . Поиск вогнутой вершины выполняется путём **циклического сдвига** вершин полигона функцией $LCShift$, определённой в (1.88), до тех пор, пока значение $nf2(p_n, p_1, p_2)$ не станет того же знака, что и направление обхода полигона $d = dir_test(P)$. Благодаря сдвигам вогнутая вершина и начало разрезающей диагонали всегда будут находиться в точке p_1 , а доступ к остальным вершинам полигона становится более простым.

Второй конец диагонали p_k с номером $k \in [3, n-1]$ должен, во-первых, лежать относительно ребра p_1p_2 с внутренней стороны полигона, где значения d и $nf2(p_1, p_2, p_k)$ равнозначны. Во-вторых, отрезок p_1p_k не должен пересекаться или касаться ни с одной из следующих сторон полигона $p_i p_{i+1}$, несмежных с вершинами p_1 и p_k , откуда следует, что $k+1 \leq i \leq n-1$. Благодаря выбору начальной точки p_1 в вогнутом угле такая точка p_k и разрезающая диагональ p_1p_k существуют всегда. Для ускорения процесса поиска после обнаружения пересечения диагональю стороны $p_i p_{i+1}$ значение k приравнивается индексу i .

Разрезание полигона диагональю p_1p_k реализуется рекурсивным обращением к функции tri_poly с аргументами-полигонами

$$P_1 = p_1 p_k \dots p_n p_1, \quad P_2 = p_1 \dots p_k p_1.$$

Примерное бинарное дерево рекурсии, терминальные вершины которого представляют собой искомые треугольники, изображено на рис. 1.92,б. Функция формирует список LP , содержащий треугольные фрагменты разрезанного полигона, и возвращает их количество. Перед внешним обращением к функции необходимо инициализировать пустой список $LP = \emptyset$, в который внутри функции tri_poly будут добавляться отрезанные треугольники.

Пример 1.14. Проследим работу алгоритма tri_poly на примере триангуляции диагоналями невыпуклого полигона $P = p_1 \dots p_{14} p_1$ (рис. 1.92,а). Тест направления обхода полигона (1.55) даёт значение $d = dir_test(P) = 1$. Дерево рекурсивного процесса разрезания полигона изображено на рис. 1.92,б.

На первом уровне рекурсии после двух циклических сдвигов по условиям $d > 0$ и $nf2(p_2, p_3, p_4) = 8 > 0$ находим вогнутую вершину p_3 , ставшую первой в $P = p_3 p_4 \dots p_{14} p_1 p_2 p_3$. Для удобства анализа работы алгоритма номера вершин оставлены без изменения.

Последовательная проверка точек p_k на левое (при $d=1$) расположение от ребра $p_3 p_4$ и поиск диагонали $p_3 p_k$, не пересекающейся с остальными рёбрами, начиная с $p_{k+1} p_{k+2}$, даёт значение $k=10$. Таким образом, исходный полигон P разрезался диагональю $p_3 p_{10}$ на два фрагмента:

$$P_1 = p_3 p_{10} \dots p_{14} p_1 p_2 p_3, \quad P_2 = p_3 \dots p_{10} p_3.$$

На втором уровне рекурсии у полигона P_1 сразу находим вогнутую вершину p_3 и диагональ $p_3 p_{11}$, которой он разрезается на $P_{11} = p_3 p_{11} \dots p_{14} p_1 p_2 p_3$ и $P_{12} = p_3 p_{10} p_{11} p_3$. У полигона P_2 находим ближайшую вогнутую вершину p_6 и диагональ $p_6 p_8$, которой он разрезается на фрагменты $P_{21} = p_6 p_8 p_9 p_{10} p_3 p_4 p_5 p_6$ и $P_{22} = p_6 p_7 p_8 p_6$.

На третьем уровне обнаруживаем, что полигоны P_{12} и P_{22} являются треугольниками и дальнейшему разрезанию не подлежат. Сохраняем их в предварительно инициализированном списке $LP=\emptyset$. Полигон P_{11} , имеющий четыре вершины p_2, p_3, p_{11} и p_{12} на одной прямой, минимизируем до $P_{11}=p_{12}p_{13}p_{14}p_2p_{12}$ и разрезаем диагональю $p_{12}p_{14}$ на фрагменты

$$P_{111}=p_{12}p_{14}p_1p_2p_{12}, \quad P_{112}=p_{12}p_{13}p_{14}p_{12}.$$

Полигон P_{21} разрезаем диагональю p_6p_9 на следующие фрагменты:

$$P_{211}=p_6p_9p_{10}p_3p_4p_5p_6, \quad P_{212}=p_6p_8p_9p_6.$$

На четвертом уровне рекурсии полигон P_{111} , имеющий три вершины p_2, p_{12}, p_{14} на одной прямой, минимизируем до треугольника $P_{111}=p_{14}p_1p_2p_{14}$ и сохраняем в списке LP . Полигоны P_{112} и P_{212} также являются треугольниками и добавляются в LP . Оставшийся полигон P_{211} разрезаем диагональю $p_{10}p_4$ на фрагменты

$$P_{2111}=p_{10}p_4p_5p_6p_9p_{10}, \quad P_{2112}=p_{10}p_3p_4p_{10}.$$

На пятом уровне обнаруживаем, что полигон P_{2112} является треугольником, и записываем его в LP . Полигон P_{2111} разрезаем диагональю $p_{10}p_5$ на

$$P_{21111}=p_{10}p_5p_6p_9p_{10}, \quad P_{21112}=p_{10}p_4p_5p_{10}.$$

На шестом уровне рекурсии треугольник P_{21112} добавляем в LP , а P_{21111} разрезаем диагональю $p_{10}p_6$ на фрагменты

$$P_{211111}=p_{10}p_6p_9p_{10}, \quad P_{211112}=p_{10}p_5p_6p_{10},$$

которые, как треугольники, на последнем седьмом уровне добавляем в LP .

В результате работы алгоритма полигон P оказался разрезан на девять треугольников, которые записались в список LP в следующей последовательности:

$$LP=\{P_{111}, P_{112}, P_{12}, P_{211111}, P_{211112}, P_{21112}, P_{2112}, P_{212}, P_{22}\}.$$

□

После небольшой модификации алгоритм $tri_poly(P)$ можно использовать для разрезания произвольного полигона P диагоналями на **выпуклые** и не обязательно треугольные фрагменты. Суть модификации в том, что проверка треугольности полигона $\{n=3\}$ заменяется на проверку его выпуклости

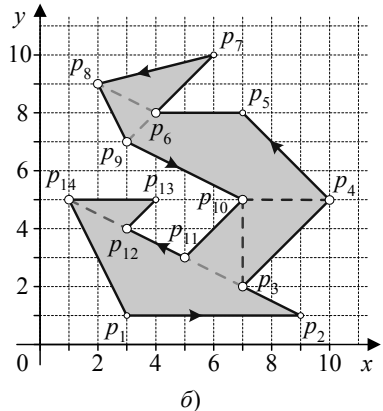
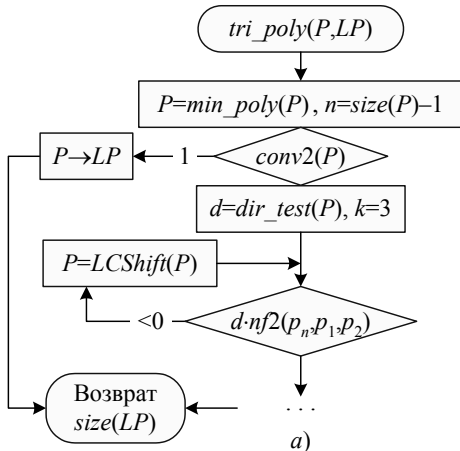


Рис. 1.93

$\{conv2(P)=1\}$, а следующее после вычисления $d=dir_test(P)$ тестирование $conv2(P)$ исключается (рис. 1.93,а). При разрезании полигона диагоналями не возникает новых точек, как это происходит при разрезании хордами в алгоритме cut_poly (см. рис. 1.90,а).

Моделирование показывает, что обычно невыпуклый полигон разрезается диагоналями на несколько треугольников и один выпуклый не треугольный фрагмент. Например, полигон P , изображённый на рис. 1.93,б, разрезался на шесть треугольников

$$p_{14}p_1p_2p_{14}, p_{12}p_{13}p_{14}p_{12}, p_3p_{10}p_{11}p_3, p_{10}p_3p_4p_{10}, p_6p_8p_9p_6, p_6p_7p_8p_6$$

и один пятиугольник $p_{10}p_4p_5p_6p_9p_{10}$.

Итерационная триангуляция

Триангуляцию произвольного полигона можно осуществить не только рекурсивным методом, но и поочерёдным отрезанием от полигона выступающих треугольников до тех пор, пока не останется последний треугольник. В основе предлагаемого метода лежит следующее свойство любого **минимального** (без кратных вершин и развёрнутых углов) полигона P : **всегда найдётся такой выпуклый угол $\angle p_i p_{i+1} p_{i+2}$, что все остальные вершины полигона расположены вне треугольника $p_i p_{i+1} p_{i+2} p_i$** [48]. Следовательно, этот треугольник может быть отрезан от полигона диагональю $p_i p_{i+2}$ (рис. 1.94,а).

Блок-схема алгоритма $tri_it_poly(P)$ итерационной триангуляции полигона P приведена на рис. 1.94,б. Вначале инициализируется пустой список полигонов $LP=\emptyset$ и вычисляется направление обхода полигона $d=dir_test(P)$, не изменяющееся после отрезания от него треугольников. Затем полигон минимизируется функцией min_poly . Обозначим результат минимизации как $P=p_1p_2\dots p_np_1$. Если это треугольник ($n=3$), то он заносится в возвращаемый список LP , и алгоритм завершает работу.

Для полигона с числом вершин $n \geq 4$ ищется подходящий для отрезания треугольник $p_i p_{i+1} p_{i+2} p_i$, где $i=1,2,\dots,n$. Вместо инкремента индекса i удобнее делать левый циклический сдвиг номеров вершин полигона функцией $LCShift$, определённой в (1.88). Тогда все операции будут совершаться с фиксированным треугольником $T=p_1p_2p_3$, годным для отрезания при двух условиях:

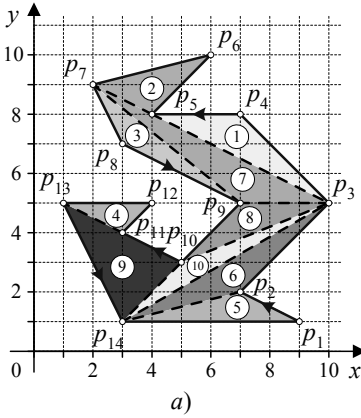
- угол $\angle p_1 p_2 p_3$ — **выпуклый**, т. е.

$$d \cdot nf2(p_1, p_2, p_3) < 0;$$

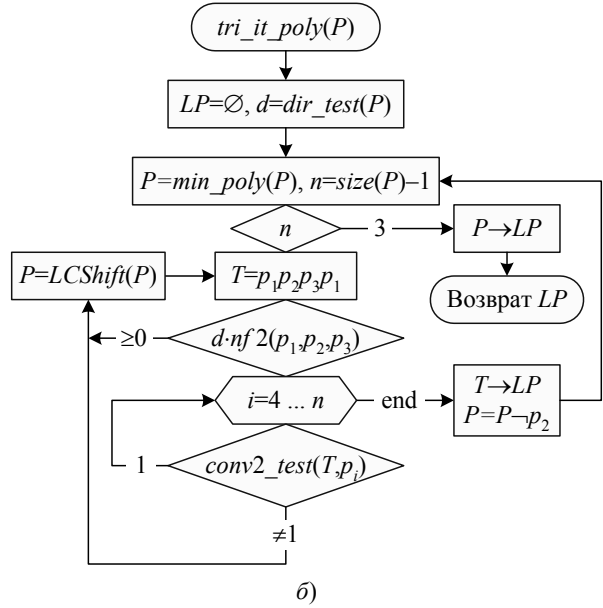
- все точки $p_4 \div p_n$ лежат **вне** треугольника, т. е.

$$conv2_test(T, p_i) = 1 \quad \forall i = \overline{4, n}.$$

Отрезанный треугольник T добавляется в список LP , а оставшийся после удаления вершины p_2 полигон $P=p_1p_3\dots p_np_1$ минимизируется алгоритмом min_poly и поступает на следующую итерацию. По достижении размера полигона $size(P)=4$ последний треугольник добавляется в список LP , возвращаемый функцией tri_it_poly как результат триангуляции.



а)



б)

Рис. 1.94

Пример 1.15. Проследим работу алгоритма *tri_it_poly* на примере триангуляции диагоналями невыпуклого полигона $P=p_1p_2\dots p_{14}p_1$ (рис. 1.94,а), тест направления обхода которого даёт значение переменной $d = \text{dir_test}(P) = 1$.

Поскольку первая тройка вершин полигона составляет вогнутый угол $\angle p_1p_2p_3$, о чём свидетельствует значение $f_3 = \text{nf}2(p_1, p_2, p_3) = 9 > 0$, при котором $d \cdot f_3 > 0$, то переходим к определению типа угла $\angle p_2p_3p_4$. Он является выпуклым ($f_4 = -18$), но одна из вершин полигона p_9 оказывается лежащей не вне (точнее, на границе) треугольника $p_2p_3p_4$, поэтому последний не годится для отрезания.

Следующий угол $\angle p_3p_4p_5$ является выпуклым ($f_5 = -9$), и все точки $p_6\dots p_{14}p_1$ лежат вне треугольника $T_1 = p_3p_4p_5p_3$, поэтому тот отрезается от полигона первым.

Оставшийся полигон $P = p_3p_5\dots p_{14}p_1p_2p_3$ является минимальным, но не треугольным, и первая тройка его вершин составляет вогнутый угол ($f_6 = 18 > 0$). Зато вторая тройка образует угол выпуклый и такой, что в треугольник $T_2 = p_5p_6p_7p_5$ не попадает ни одной вершины полигона. Поэтому этот треугольник отрезается вторым.

Оставшийся полигон $P = p_5p_7\dots p_{14}p_1p_2p_3p_5$ с тремя коллинеарными вершинами p_5, p_7 и p_3 **не является** минимальным. После исключения вершины p_5 получается минимальный полигон $P = p_7\dots p_{14}p_1p_2p_3p_7$ с первым выпуклым углом $\angle p_7p_8p_9$, в площадь которого не попадает ни одной вершины P . Вывод: третьим отрезается треугольник $T_3 = p_7p_8p_9p_7$.

В новом минимальном полигоне $P = p_7p_9\dots p_{14}p_1p_2p_3p_7$ от ближайшего выпуклого угла $\angle p_{11}p_{12}p_{13}$ отрезается треугольник $T_4 = p_{11}p_{12}p_{13}p_{11}$. После этой операции остаётся полигон с коллинеарными вершинами p_{11}, p_{13} и p_{10} , минимизация которого даёт полигон $P = p_{13}p_{14}p_1p_2p_3p_7p_9p_{10}p_{13}$. Его первый треугольник $p_{13}p_{14}p_1p_{13}$ не пригоден для отрезания, поскольку на его границе лежит вершина p_{10} . Зато следующий треугольник при вогнутом угле $T_5 = p_{14}p_1p_2p_{14}$ удовлетворяет всем условиям пополнения списка LP .

Следующим от обновлённого полигона $P=p_{14}p_2p_3p_7p_9p_{10}p_{13}p_{14}$ отрезается треугольник $T_6=p_{14}p_2p_3p_{14}$, от остатка $P=p_{14}p_3p_7p_9p_{10}p_{13}p_{14}$ — треугольник $T_7=p_3p_7p_9p_3$, а от следующего остатка $P=p_3p_9p_{10}p_{13}p_{14}p_3$ — треугольник $T_8=p_3p_9p_{10}p_3$.

Дело близится к концу: от четырёхугольника $P=p_3p_{10}p_{13}p_{14}p_3$ отрезается треугольник $T_9=p_{10}p_{13}p_{14}p_{10}$, после чего оставшийся треугольник $T_{10}=p_{10}p_{14}p_3p_{10}$ замыкает список триангуляции $LP=\{T_1, \dots, T_{10}\}$. $\square$

Рассмотренные алгоритмы разрезания *cut_poly*, *tri_poly* и все их модификации обладают общим **недостатком**: получаемые фрагменты имеют заранее непредсказуемые размеры и форму. Большое различие в габаритах и сильная вытянутость отдельных фрагментов затрудняют решение некоторых задач визуализации, например сортировки граней по удалённости от наблюдателя, используемой в алгоритмах удаления невидимых элементов.

Контролируемая триангуляция

Разработаем метод триангуляции произвольного полигона $P=p_1p_2\dots p_np_1$ с **контролируемыми размерами** получаемых треугольников, у которых длины сторон не превышают заданного значения h . Идея метода, эскизно и без детализации намеченная в [26], заключается в поиске у полигона **выпуклой** вершины p_m с **минимальным** внутренним углом (рис. 1.95,а)

$$\psi \equiv |\varphi_m| = \min_i \{|\varphi_i|\}, \text{ где } \varphi_i = \angle(p_{i-1}-p_i, p_{i+1}-p_i),$$

и последующем отрезании от этого угла в зависимости от его величины ψ одного (рис. 1.95,б), двух (рис. 1.95,в) либо трёх (рис. 1.95,г) треугольников с длинами сторон, не превышающими h [34]. Рациональнее устроить поиск вершины с минимальным выпуклым углом ψ на n циклических сдвигах вершин по (1.88)

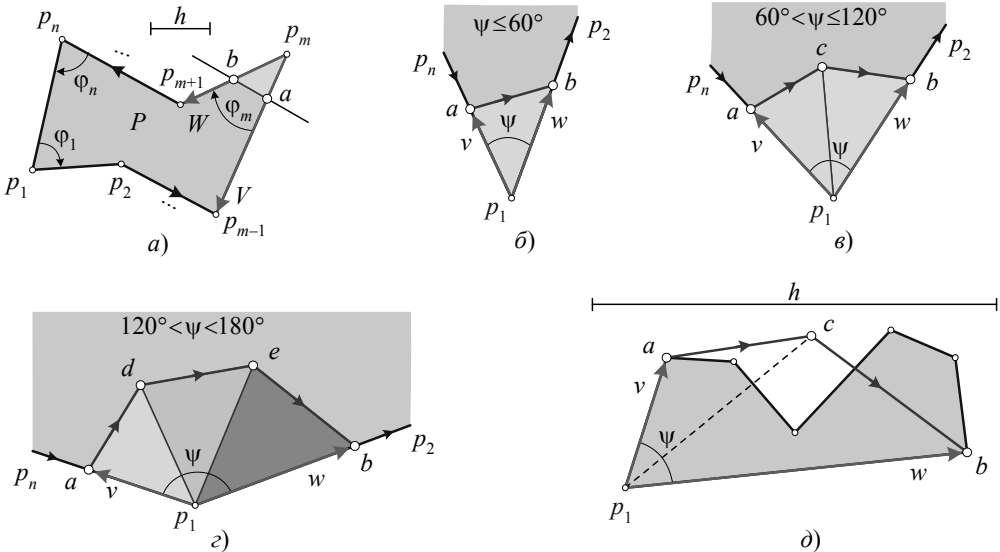


Рис. 1.95

для того, чтобы работать с фиксированным индексом $m=1$, а при уменьшении модуля угла $|\varphi_1|$ запоминать его и текущую конфигурацию полигона P .

В эффективном алгоритме контролируемой триангуляции число отрезанных треугольников должно быть минимальным, а их форма приближалась к правильной с длинами сторон, максимально близкими к h , и внутренними углами, близкими к 60° . Для реализации перечисленных пожеланий расположим точки a и b на концах векторов $v \uparrow \downarrow p_1 - p_n$ и $w \uparrow \uparrow p_2 - p_1$ с длинами $|v| \leq h$ и $|w| \leq h$, а точки c , d и e разместим между этими векторами. Равномерность размещения достигается помощью параметрической функции

$$u(v, w, \lambda, \psi) = \text{lin}(|v|, |w|, \lambda) \bar{v} R(-\lambda \psi),$$

возвращающей вектор, у которого длина $r(\lambda) = (1 - \lambda)|v| + \lambda|w|$ и угол $\gamma = -\lambda\psi$ отклонения от вектора v являются линейными функциями параметра λ . При изменении последнего в интервале $\lambda \in [0, 1]$ вектор u переходит из положения v в положение w , равномерно увеличивая угол вращения от 0 до ψ и длину от $|v|$ до $|w|$. Таким образом, с учётом индикатора направления обхода полигона $D \in \{1, -1\}$ вершины отрезаемых треугольников лежат в точках

$$a = p_1 + v, \quad b = p_1 + w; \quad (1.89a)$$

$$c = p_1 + u(v, w, 0.5, D\psi); \quad (1.89б)$$

$$d = p_1 + u(v, w, 1/3, D\psi), \quad e = p_1 + u(v, w, 2/3, D\psi). \quad (1.89в)$$

Отрезанные треугольники добавляются в предварительно инициализированный пустой список полигонов LP , а оставшаяся часть полигона подвергается следующей итерации разрезания. Для предотвращения излишнего дробления и для уменьшения общего числа треугольников целесообразно по достижении расстояния $|a - b| \leq h$ отрезать всегда один треугольник $p_1 b a p_1$ независимо от величины угла ψ . Так как при этом $|v| \leq h$ и $|w| \leq h$, то он удовлетворяет габаритному ограничению и не нуждается в разрезании.

В зависимости от формы и размеров полигона возможны ситуации, когда отрезок ab или полилинии acb , $adeb$ пересекают другие рёбра полигона либо вычисленные по (1.89) точки c , d или e оказываются за его пределами (рис. 1.95, д). В результате новый полигон перестаёт быть частью старого. Он становится самопересекающимся и может изменить направление обхода. Понятно, что дальнейшая триангуляция такого полигона невозможна.

Предлагается следующее решение этой проблемы. Перед отрезанием треугольников сохраним копии полигона $Q = P$ и списка отрезанных треугольников $LQ = LP$, а после отрезания проверим новый полигон P на самопересечение или самокасание тестом (1.40a). При $\text{self_test}(P) = 1$ либо $\text{dir_test}(P) \neq D$ восстановим сохранённые в Q и LQ данные, уменьшим длины векторов v и w , а затем будем отрезать треугольники с новыми вершинами (1.89) до тех пор, пока полигон P не перестанет самопересекаться или менять направление обхода.

На рис. 1.96 построена блок-схема алгоритма $contri_poly(P, h)$ контролируемой триангуляции полигона P с габаритным параметром h . Работа алгоритма начинается с инициализации пустого списка $LP = \emptyset$, в который будут добавляться списки вершин отрезанных треугольников, и вычисления направления обхода полигона $D = dir_test(P)$, необходимого в (1.89) и для пропуска вогнутых углов по условию $D\varphi_1 > 0$.

Дальнейший процесс выполняется циклически до отрезания последнего треугольника. После каждого отрезания полигон изменяет свою конфигурацию, число вершин n и их список $P = p_1 p_2 \dots p_n p_1$.

Рассмотрим действия, выполняемые в одном цикле алгоритма.

Шаг 1. Исключив из полигона кратные и коллинеарные вершины с помощью функции $min_poly(P)$, зафиксируем число $n = size(P) - 1$ сторон минимального полигона $P = p_1 p_2 \dots p_n p_1$, сохраним его копию $Q = O$ и установим начальное значение угла $\psi = \pi$.

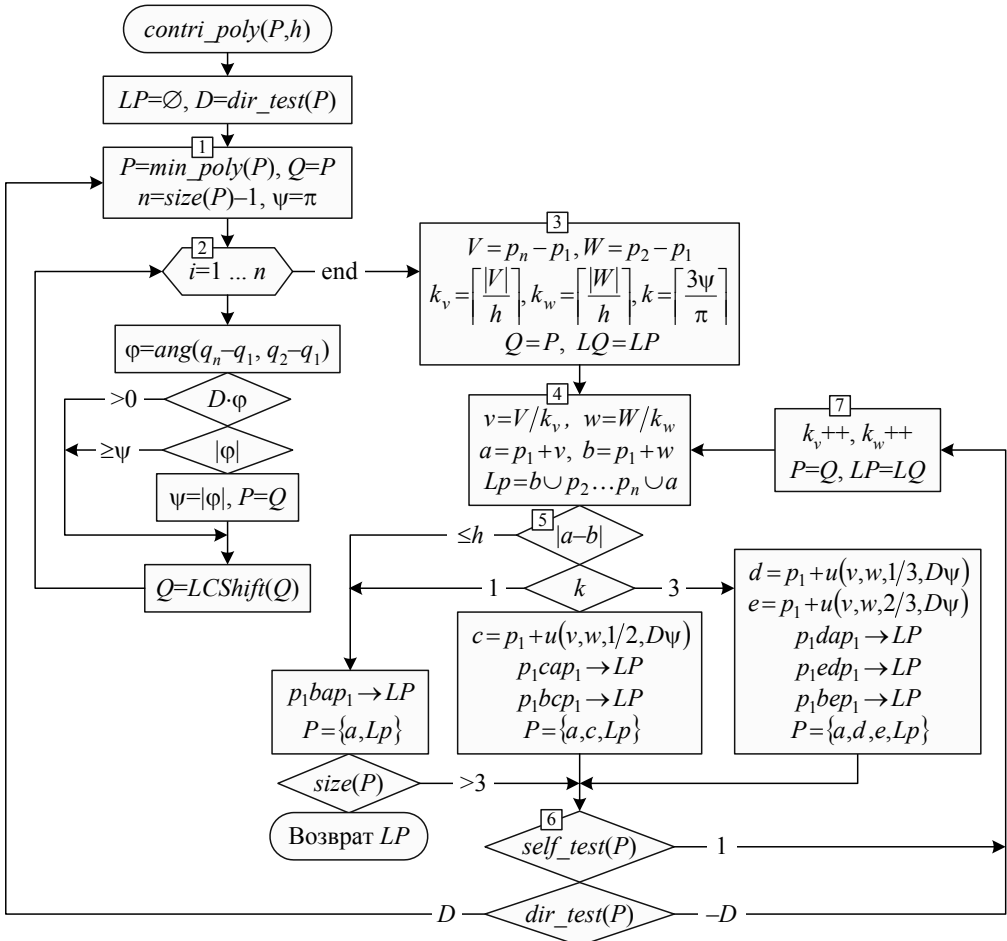


Рис. 1.96

Шаг 2. В цикле $i=\overline{1, n}$ вычисляем с помощью функции (1.32) внутренний угол $\varphi_1 = \text{ang}(q_n - q_1, q_2 - q_1)$ при первой вершине полигона Q . Если он выпукл ($D\varphi_1 < 0$) и минимален ($|\varphi_1| < \psi$), то запоминаем текущий минимум $\psi = |\varphi_1|$ и полигон $P=Q$. Далее выполняется циклический сдвиг полигона $Q = LCShift(Q)$.

Теперь разместим на сторонах угла $p_n p_1 p_2$ точки a и b , отстоящие от вершины p_1 не дальше, чем на расстояние h . Проще всего отложить от точки p_1 отрезки длиной h , но тогда в конце отрезания ребра, длина которого не кратна h , может остаться очень короткий отрезок, что, в свою очередь, даст мелкий треугольник. Да и длины сторон угла могут быть много меньше h .

Процедура **равномерного** разбиения рёбер полигона состоит из начальных установок некоторых величин и их циклических настроек.

Шаг 3. Установочная фаза включает:

- вычисление векторов сторон $p_1 p_n$ и $p_1 p_2$: $V = p_n - p_1$, $W = p_2 - p_1$;
- расчёт целых кратностей длин сторон к числу h : $k_v = |V|/h$, $k_w = |W|/h$;
- расчёт целой кратности угла ψ к углу $60^\circ \equiv \pi/3$: $k = \lceil 3\psi/\pi \rceil$;
- сохранение копий полигона и списка треугольников: $Q=P$, $LQ=LP$.

Шаг 4. Циклическая фаза разбиения рёбер $p_1 p_n$ и $p_1 p_2$ включает:

- вычисление векторов сторон треугольников, длины которых не превышают значения h : $v = V/k_v$, $w = W/k_w$;
- вычисление по (1.89а) вершин треугольников, лежащих на рёбрах, примыкающих к вершине p_1 : $a = p_1 + v$, $b = p_1 + w$;
- формирование списка **некратных** вершин полилинии, проходящей между точками b и a с учётом возможного совпадения точек $b=p_2$ и $a=p_n$:

$$Lp = b \cup p_2 \dots p_n \cup a. \quad (1.90a)$$

В программировании логические объединения можно заменить конкатенациями с проверками совпадений точек, возникающих при $k_w=1$ или $k_v=1$:

$$\begin{cases} Lp = p_2 p_3 \dots p_n; \\ Lp = \text{if}(k_w=1: Lp, \{b, Lp\}); \\ Lp = \text{if}(k_v=1: Lp, \{Lp, a\}). \end{cases} \quad (1.90б)$$

Шаг 5. В зависимости от длины отрезка ab и коэффициента k , вычисленного на **шаге 3**, возможны три варианта отрезания треугольников и формирования списка вершин нового полигона P (см. рис. 1.95,б–г):

- при $|a-b| < h$ либо $k=1$, т. е. $\psi \leq 60^\circ$, добавляем в список LP **один** треугольник $p_1 b a p_1$, а новый полигон P формируем соединением точки a и списка Lp , полученного в (1.90). После добавления в LP последнего отрезанного треугольника полигон вырождается в двунаправленный отрезок $P = p_1 p_2 p_1$ и по условию $\text{size}(P) \leq 3$ алгоритм заканчивает работу, **возвращая** список LP ;
- при $k=2$, т. е. $60^\circ < \psi \leq 120^\circ$, вычисляем по (1.89б) точку c , добавляем в список LP **два** треугольника $p_1 c a p_1$ и $p_1 b c p_1$, а полигон P формируем соединением точек a , c и списка Lp ;

• при $k=3$, т. е. $120^\circ < \psi < 180^\circ$, вычисляем по (1.89в) точки d и e , добавляем в список LP **три** треугольника p_1dap_1 , p_1edp_1 и p_1ber_1 , а полигон P формируем соединением точек a , d , e и списка LP .

Шаг 6. Тестом (1.40а) проверяем самопересечение либо самокасание полигона P : $s = self_test(P)$. При $s=0$ и $dir_test(P)=D$ **переходим** на **шаг 1**.

Шаг 7. В противном случае:

- инкрементируем коэффициенты разбиения рёбер: k_v++ , k_w++ ;
- восстанавливаем копии полигона и списка треугольников: $P=Q$, $LP=LQ$;
- **переходим** на **шаг 4**.



По окончании все возвращённые в списке LP треугольники имеют такое же направление обхода, что и у исходного полигона P . Это обеспечивается порядком следования вершин треугольников, добавляемых в LP на **шаге 5**.

В заключение проиллюстрируем работу алгоритма на рис. 1.97. Полигон верхнего ряда взят из примера 1.13 и разрезан с различными значениями габаритного параметра h на 714, 166, 49 и 10 треугольников. Очередность отрезания треугольников от наименьшего выпуклого угла полигона можно проследить на двух последних рисунках.

Три случайных полигона нижнего ряда сгенерированы алгоритмом *srpoly* (см. рис. 1.33), а в четвёртом полигоне внутреннее отверстие соединено с внешним контуром двумя рёбрами, разделёнными ε -зазором во избежание их совпадения и касания несмежных рёбер.

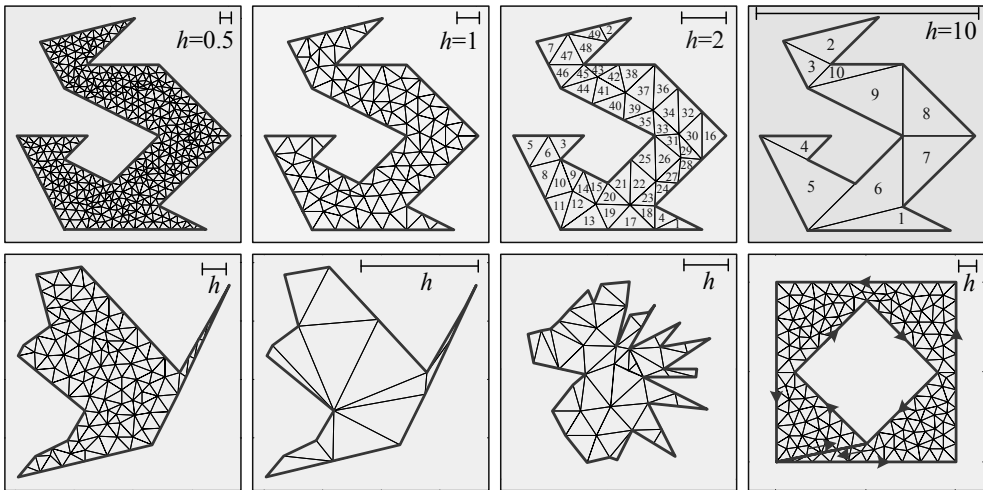


Рис. 1.97

Отметим, что при задании параметра h заведомо бóльшим габаритов полигона размеры отрезаемых треугольников ограничиваются лишь длинами его рёбер. Поэтому функцию *contri_poly* можно использовать для разрезания полигона на максимально крупные треугольники, что иллюстрировано на рис. 1.97 решением задачи триангуляции диагоналями с габаритным параметром $h=10$.

1.5. ГРАФИЧЕСКИЕ ЭЛЕМЕНТЫ В ПРОСТРАНСТВЕ

Точка в пространстве имеет нулевую топологическую размерность. **Расстояние** $|p_1 - p_2|$ **между точками** p_1 и p_2 в разных системах координат вычисляется следующим образом:

- в декартовой системе координат (x, y, z)

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2};$$

- в цилиндрической системе координат (ρ, φ, y)

$$d = \sqrt{\rho_1^2 + \rho_2^2 - 2\rho_1\rho_2\cos(\varphi_1 - \varphi_2) + (y_1 - y_2)^2};$$

- в сферической системе координат (r, φ, ψ)

$$d = \sqrt{r_1^2 + r_2^2 - 2r_1r_2(\cos(\psi_1)\cos(\psi_2)\cos(\varphi_1 - \varphi_2) + \sin(\psi_1)\sin(\psi_2))}.$$

У точек **поверхности** две степени свободы. **Модели поверхности** в неявной и параметрической формах имеют следующий вид:

$$\text{НФ:} \quad f(x, y, z) = 0 \text{ или } f(p) = 0; \quad (1.91a)$$

$$\text{ПФ:} \quad p(t, \tau) = [x(t, \tau) \ y(t, \tau) \ z(t, \tau)]. \quad (1.91b)$$

Любая реальная поверхность ограничена в пространстве либо в силу своей замкнутости (сфера, эллипсоид, тор и т. п.), либо благодаря системе неравенств, наложенных на её координаты или параметры:

$$\text{НФ:} \quad \eta(x, y, z) \leq 0 \text{ или } \eta(p) \leq 0; \quad \text{ПФ:} \quad \eta(t, \tau) \leq 0. \quad (1.92)$$

Особенность ограниченной поверхности в том, что при расчёте её пересечения с другим графическим объектом могут быть получены точки или линии, лежащие на поверхности (1.91), но не удовлетворяющие условиям (1.92). Следовательно, **результат пересечения должен всегда тестироваться по всем необходимым условиям существования**.

В регулярной точке поверхности $q = [q_x \ q_y \ q_z] \equiv p(t_0, \tau_0)$ функция $p(t, \tau)$ непрерывно дифференцируема по своим аргументам и обладает свойством [21]

$$\begin{vmatrix} x'_t & y'_t \\ x'_\tau & y'_\tau \end{vmatrix}^2 + \begin{vmatrix} y'_t & z'_t \\ y'_\tau & z'_\tau \end{vmatrix}^2 + \begin{vmatrix} z'_t & x'_t \\ z'_\tau & x'_\tau \end{vmatrix}^2 \neq 0.$$

В ней существует единственная **касательная плоскость** (рис. 1.98), описываемая координатными и векторными моделями в НФ и ПФ:

$$\text{НФ:} \quad \begin{cases} N_x(x - q_x) + N_y(y - q_y) + N_z(z - q_z) = 0, \\ N_\omega = f'_\omega|_{q_x, q_y, q_z} \quad \forall \omega \in \{x, y, z\} \end{cases} \text{ или } \begin{cases} N \circ (p - q) = 0, \\ N = \nabla f(p)|_q; \end{cases} \quad (1.93a)$$

$$\text{ПФ:} \quad \begin{cases} x(t, \tau) = q_x + V_x t + W_x \tau, \\ y(t, \tau) = q_y + V_y t + W_y \tau, \\ z(t, \tau) = q_z + V_z t + W_z \tau, \\ V_\omega = \omega'_t|_{t_q, \tau_q}, \quad W_\omega = \omega'_\tau|_{t_q, \tau_q} \\ \forall \omega \in \{x, y, z\} \end{cases} \text{ или } \begin{cases} p(t, \tau) = q + Vt + W\tau, \\ V = \frac{\partial p}{\partial t}\bigg|_{t_q, \tau_q}, \quad W = \frac{\partial p}{\partial \tau}\bigg|_{t_q, \tau_q}, \\ V \times W \neq O_3. \end{cases} \quad (1.93b)$$

Вектор **нормали** $N = [N_x \ N_y \ N_z]$ направлен в ту сторону поверхности (назовём её **нормальной**), где значение $f(p) > 0$. **Направляющие векторы** $V \perp N$ и $W \perp N$ должны быть **непараллельными**. Они составляют с нормалью

$$N = V \times W = \begin{bmatrix} V_y & V_z \\ W_y & W_z \end{bmatrix} \begin{bmatrix} V_z & V_x \\ W_z & W_x \end{bmatrix} \begin{bmatrix} V_x & V_y \\ W_x & W_y \end{bmatrix} \quad (1.94)$$

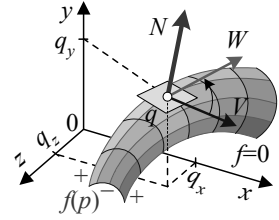


Рис. 1.98

правую тройку векторов, в которой вращения от V к W , от W к N и от N к V происходят в положительном направлении — **против часовой стрелки** — с точки зрения наблюдателя, находящегося на конце третьего вектора N , V и W .

Все плоскости, проходящие через точку $p(t, \tau)$ и вектор нормали $N(t, \tau)$, называются **нормальными**, а линии пересечения поверхности с ними — **нормальными сечениями**. Сформируем в касательной плоскости вектор $U = \alpha V + \beta W$, где $\alpha^2 + \beta^2 \neq 0$. Каждое нормальное сечение, лежащее в плоскости векторов N и U (рис. 1.99), имеет в точке p кривизну $\kappa(\alpha, \beta)$, вычисляемую по (1.10а), где вектор V заменён вектором U . Точка поверхности с постоянной кривизной для всех проходящих через неё нормальных сечений называется **омбилической**. В каждой неомбилической точке существуют два **главных нормальных сечения** с минимальной $\kappa_{\min}$ и максимальной $\kappa_{\max}$ кривизной. По теореме Эйлера [21] плоскости этих сечений ортогональны.

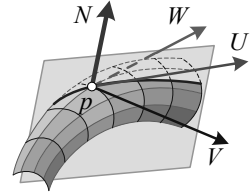


Рис. 1.99

Особый интерес в задачах анализа и практических применениях представляет **гауссова кривизна** поверхности в точке $p(t, \tau)$, определяемая и вычисляемая следующим образом [21, 51]:

$$K = \kappa_{\min} \kappa_{\max} = \frac{(N \circ V'_t)(N \circ W'_\tau) - (N \circ V'_\tau)^2}{(N \circ N)^2}. \quad (1.95)$$

В зависимости от знака гауссовой кривизны поверхность имеет в точке $p(t, \tau)$ следующие **локальные** свойства:

- **эллиптические** при $K > 0$ — все нормальные сечения либо выпуклы ($\kappa_{\min} > 0$), либо вогнуты ($\kappa_{\max} < 0$) и односторонне касаются плоскости $\{p, N\}$ в единственной точке p ;
- **гиперболические** при $K < 0$ — существуют как выпуклые ($\kappa_{\max} > 0$), так и вогнутые ($\kappa_{\min} < 0$) нормальные сечения, разносторонне касающиеся плоскости $\{p, N\}$ в **седловой точке** поверхности p ;
- **параболические** при $K = 0$ — существует хотя бы одно нормальное сечение с нулевой кривизной $\kappa_{\min} = 0$ или $\kappa_{\max} = 0$ в форме прямой **линии касания** локально цилиндрического, конического или плоского участка поверхности с плоскостью $\{p, N\}$.

1.5.1. Модели плоской поверхности

Наиболее простыми и распространёнными поверхностями трёхмерной графики являются плоскости. Рассмотрим различные способы их описания.

Неявное уравнение плоскости

Неявное уравнение плоскости, заданной четырьмя коэффициентами A, B, C и D , составляющими вектор $F = [A \ B \ C \ D]$, имеет следующие координатный и векторный виды:

$$\text{НФ:} \quad Ax + By + Cz + D = 0 \quad \text{или} \quad \boxed{[p \ 1] \circ F = \tilde{p} \circ F = 0}. \quad (1.96)$$

Хотя бы одно из чисел A, B или C должно быть ненулевым. Если все коэффициенты $A \neq 0, B \neq 0$ и $C \neq 0$, то плоскость не параллельна ни одной оси координат и пересекается с ними в следующих точках (рис. 1.100, а):

$$p_x = [-D/A \ 0 \ 0], \quad p_y = [0 \ -D/B \ 0], \quad p_z = [0 \ 0 \ -D/C].$$

Но, например, при $A=0, B \neq 0$ и $C \neq 0$ уравнение $By + Cz + D = 0$ описывает плоскость, параллельную оси x и ортогональную координатной плоскости yz ; при $A \neq 0, B=0$ и $C=0$ уравнению $Ax + D = 0$ соответствует плоскость, ортогональная оси x и параллельная плоскости yz .

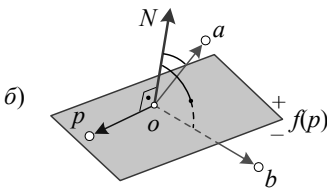
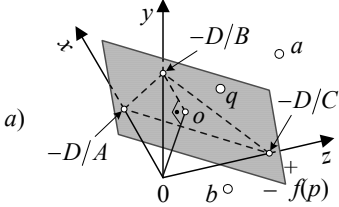


Рис. 1.100

Плоскость $f(p) \equiv \tilde{p} \circ F = 0$ проходит через начало координат при $f(O_3) \equiv D = 0$. Благодаря её свойству разделять пространство на два подпространства с противоположными знаками $f(p)$ неявное уравнение позволяет с минимальными вычислительными затратами — за три умножения и три сложения — определить по условию $f(q) = 0$ принадлежность произвольной точки q плоскости, а также ориентацию двух точек a и b относительно плоскости:

- при $f(a)f(b) > 0$ обе точки лежат по одну сторону от плоскости;
- при $f(a)f(b) < 0$ — по разные стороны от плоскости.

Для перехода от неявного описания плоскости к следующему — нормальному — описанию необходимо найти любую её точку o и вектор нормали N . Сравнивая (1.96) с (1.93а), заключаем, что **координатами вектора нормали можно принять коэффициенты неявного уравнения**, т. е. $N = [A \ B \ C]$.

Приведём два способа получения точки o .

1. Выберем её в месте пересечения плоскости с осью координат, соответствующей ненулевому коэффициенту в (1.96):

$$o = \text{if}(A: p_x, \text{if}(B: p_y, p_z)). \quad (1.97a)$$

Несмотря на легкость понимания, этот способ не самый лучший для программирования из-за условности выбора точки в зависимости от чисел A и B .

2. Вычислим *ближайшую к началу координат* точку o (рис. 1.100,а) путём решения МНК-задачи $|p|^2 = p \circ p \rightarrow \min$ с ограничением $p \circ N + D = 0$. Аналогично (1.126) решение имеет вид

$$o = \frac{-D}{N \circ N} N = \frac{-D}{A^2 + B^2 + C^2} [A \ B \ C]. \quad (1.976)$$

Нормальная модель плоскости

Нормальное уравнение плоскости $\{o, N\}$, заданной точкой o и вектором нормали N , совпадает с (1.14а) и выводится из условия ортогональности векторов N и $p - o$ для всех точек p , принадлежащих плоскости (рис. 1.100,б):

$$\text{НФ:} \quad N_x(x - o_x) + N_y(y - o_y) + N_z(z - o_z) = 0 \quad \text{или} \quad \boxed{(p - o) \circ N = 0}. \quad (1.98)$$

Зависимость вектора неявного уравнения от параметров нормального уравнения совпадает с (1.15):

$$F = [N \ -o \circ N]. \quad (1.99)$$

С помощью функции

$$f(p) = (p - o) \circ N \quad (1.100)$$

можно определить ориентацию точки p относительно плоскости:

- при $f(a) > 0$ точка a лежит в том полупространстве, куда направлена нормаль, а угол $\angle(a - o, N)$ острый;
- при $f(b) < 0$ угол $\angle(b - o, N)$ тупой, а точка b и нормаль находятся по разные стороны от плоскости.

Таким образом, плоскость является **двухсторонней** границей, разделяющей пространство R^3 на положительное и отрицательное полупространства согласно знаку функции $f(p)$. Назовём сторону плоскости, обращённую в положительное полупространство, **внешней**, а обратную сторону — внутренней.

Параметрическая модель плоскости

Параметрические функции плоскости $\{o, V, W\}$, заданной точкой o и линейно независимыми направляющими векторами V и W (рис. 1.101,а), имеют следующие координатные и векторную формы:

$$\text{ПФ:} \quad \begin{cases} x(t, \tau) = o_x + V_x t + W_x \tau \\ y(t, \tau) = o_y + V_y t + W_y \tau \\ z(t, \tau) = o_z + V_z t + W_z \tau \end{cases} \quad \text{или} \quad \boxed{p(t, \tau) = o + Vt + W\tau}. \quad (1.101)$$

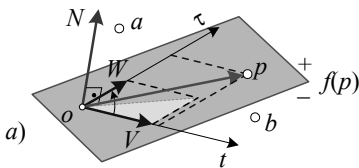
Условие линейной независимости направляющих векторов $V \times W \neq O_3$ эквивалентно их непараллельности.

Параметрическая модель удобна для задания как всей бесконечной плоскости, так и её полубесконечных и конечных частей. Для этого в (1.101) указываются соответствующие интервалы изменения параметров t и τ . Например:

- бесконечные интервалы $-\infty < t < \infty$ и $-\infty < \tau < \infty$ не ограничивают протяжённость бесконечной во всех направлениях **плоскости**;
- задание бесконечного $-\infty < t < \infty$ и полубесконечного $\tau \geq 0$ интервалов определяет **полуплоскость**, находящуюся от прямой $o+Vt$ по одну сторону с W ;
- бесконечный $-\infty < t < \infty$ и конечный $0 \leq \tau \leq 1$ интервалы задают бесконечную **полосу** шириной $|W \times \bar{V}|$ между прямыми $o+Vt$ и $o+W+Vt$;
- конечные интервалы $0 \leq t \leq 1$ и $0 \leq \tau \leq 1$ определяют **параллелограмм** между четырьмя точками o , $o+V$, $o+W$ и $o+V+W$;
- конечные интервалы $0 \leq t \leq 1$ и $0 \leq \tau \leq t$ дают **треугольник** с вершинами в точках o , $o+V$ и $o+V+W$.

Переход от параметрического описания плоскости $\{o, V, W\}$ к нормальному $\{o, N\}$ и неявному $\{F\}$ описаниям заключается в вычислении по (1.94) вектора нормали N , а по (1.99) — вектора F :

$$N = V \times W, \quad F = [N \quad -o \circ N]. \quad (1.102)$$



Ортогональность векторного произведения $V \times W$ к векторам-сомножителям позволяет использовать его в качестве нормали. Эквивалентная (1.100) функция

$$f(p) = (p - o) \circ (V \times W) \quad (1.103)$$

определяет ориентацию точки p относительно плоскости $\{o, V, W\}$ (рис. 1.101, а):

- при $f(a) > 0$ точка a лежит в положительном полупространстве, откуда вращение от вектора V до вектора W выглядит положительным (против часовой стрелки). При этом угол $\angle(a - o, N)$ острый;

- при $f(b) < 0$ угол $\angle(b - o, N)$ тупой, а точка b находится в отрицательном полупространстве (с внутренней стороны плоскости).

Более сложной является неоднозначная обратная задача нахождения пары направляющих векторов по единственному вектору нормали (рис. 1.101, б). Выберем **произвольный** вектор H , не параллельный N , например один из координатных ортов $\omega^0 \# N$, где $\omega \in \{z, y\}$, и по свойству ортогональности векторов в векторном произведении получим желаемое решение:

$$H = \text{if}(N \parallel z^0: y^0, z^0), \quad V = H \times N \neq O_3, \quad W = N \times V \neq O_3. \quad (1.104)$$

Модели плоскости, проходящей через три точки

Неявное **уравнение плоскости, проходящей через три точки** a , b и c (рис. 1.102, а), выводится из условия их компланарности с точкой $p = [x \ y \ z]$

$$\begin{vmatrix} \tilde{p} \\ \tilde{a} \\ \tilde{b} \\ \tilde{c} \end{vmatrix} \equiv \begin{vmatrix} x & y & z & 1 \\ a_x & a_y & a_z & 1 \\ b_x & b_y & b_z & 1 \\ c_x & c_y & c_z & 1 \end{vmatrix} = 0:$$

$$\text{НФ: } \underbrace{\begin{vmatrix} 1 & a_y & a_z \\ 1 & b_y & b_z \\ 1 & c_y & c_z \end{vmatrix}}_{A \equiv N_x} x + \underbrace{\begin{vmatrix} a_x & 1 & a_z \\ b_x & 1 & b_z \\ c_x & 1 & c_z \end{vmatrix}}_{B \equiv N_y} y + \underbrace{\begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix}}_{C \equiv N_z} z + \underbrace{\begin{vmatrix} a_z & a_y & a_x \\ b_z & b_y & b_x \\ c_z & c_y & c_x \end{vmatrix}}_D = 0. \quad (1.105a)$$

Выбрав направляющие векторы плоскости $V=b-a$ и $W=c-b$, получим **параметрическую модель плоскости, проходящей через три точки**:

$$\text{ПФ: } \boxed{p(t, \tau) = a + (b-a)t + (c-b)\tau}. \quad (1.105b)$$

Условия существования плоскости в (1.105) $|N| \neq 0$ или $(b-a) \times (c-b) \neq 0_3$ означают, что три точки не должны лежать на одной прямой.

Эквивалентная (1.103) функция **смешанного произведения**

$$f(p) = (p-a) \circ ((b-a) \times (c-b)) = \begin{vmatrix} p-a \\ b-a \\ c-b \end{vmatrix} \quad (1.106)$$

позволяет вычислить ориентацию точки p относительно плоскости $\{a, b, c\}$. При $f(p) > 0$ точка p лежит в положительном полупространстве, откуда обход треугольника $abca$ выглядит против часовой стрелки.

Ниже рассматриваются свойства многогранников (**полиэдров**), обусловленные направлением векторов нормалей граней. При выборе векторов V и W по первым двум рёбрам грани полиэдра **внешняя ориентация нормали к грани обеспечивается при её обходе против часовой стрелки**, если смотреть на грань из внешнего полупространства.

В частном случае плоскость (1.105) можно провести через три точки $a=h_x x^0$, $b=h_y y^0$ и $c=h_z z^0$, отсекающие на осях координат отрезки h_x , h_y и h_z (рис. 1.102, б). Аналогично (1.18a) получим **уравнение плоскости в отрезках**:

$$\text{НФ: } \frac{x}{h_x} + \frac{y}{h_y} + \frac{z}{h_z} - 1 = 0 \Rightarrow F = [h_y h_z \quad h_x h_z \quad h_x h_y \quad -h_x h_y h_z]. \quad (1.107a)$$

Выбрав направляющие векторы $V=b-a=[-h_x \ h_y \ 0]$ и $W=c-b=[0 \ -h_y \ h_z]$, в (1.101) получим координатные и векторную параметрические функции:

$$\text{ПФ: } \begin{cases} x(t, \tau) = h_x(1-t), & y(t, \tau) = h_y(t-\tau), & z(t, \tau) = h_z \tau, \\ p(t) = [h_x(1-t) \ h_y(t-\tau) \ h_z \tau]. \end{cases} \quad (1.107b)$$

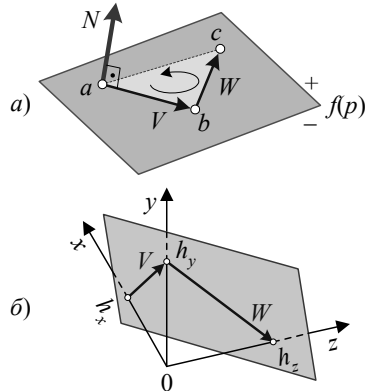


Рис. 1.102

Итак, в зависимости от способа задания плоскости получены разные формы и способы вычисления её НФ $f(p)$. Введём **синонимы** этой функции nf^* , где «*» заменяет третий символ её имени «F», «N», «VW» или «3»:

$$nfF(F, p) = \tilde{p} \circ F; \quad (1.108a)$$

$$nfN(o, N, p) = (p - o) \circ N; \quad (1.108б)$$

$$nfVW(o, V, W, p) = (p - o) \circ (V \times W) = \begin{vmatrix} p - o \\ V \\ W \end{vmatrix}; \quad (1.108в)$$

$$nf3(a, b, c, p) = (p - a) \circ ((b - a) \times (c - b)) = \begin{vmatrix} p - a \\ b - a \\ c - b \end{vmatrix}. \quad (1.108г)$$

В дополнение к (1.108а–г) определим ещё одну неявную функцию плоскости полигона $P = p_1 p_2 p_3 \dots p_n p_1 \in \mathbb{R}^3$:

$$nfP(P, p) = (p - p_1) \circ ((p_2 - p_1) \times (p_3 - p_2)) = \begin{vmatrix} p - p_1 \\ p_2 - p_1 \\ p_3 - p_2 \end{vmatrix}. \quad (1.108д)$$

1.5.2. Модели линий в пространстве

Линия в пространстве имеет одну степень свободы и является либо результатом пересечения двух поверхностей (рис. 1.103,а), либо кинематическим следом движения некоторой точки в пространстве (рис. 1.103,б). Соответственно, имеем две формы **модели пространственных линий**:

$$\text{НФ:} \quad \{f_1(p)=0\} \cap \{f_2(p)=0\}; \quad \text{ПФ:} \quad p(t) = [x(t) \ y(t) \ z(t)].$$

Определение координат точек пересечения поверхностей, заданных в **неявной форме**, требует решения системы двух в общем случае нелинейных уравнений с тремя неизвестными x , y и z . Ввиду недоопределённости пусть даже и совместной системы её решением является не единственная точка пересечения

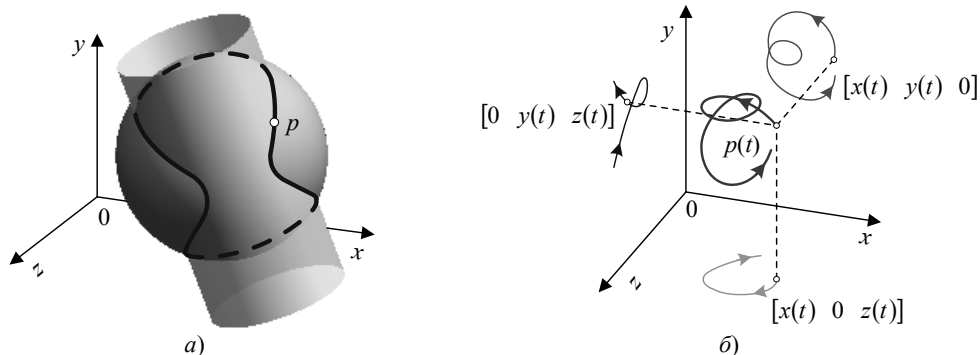


Рис. 1.103

поверхностей, а бесконечное множество точек $p(t)=[x(t) \ y(t) \ z(t)]$, изменение координат которых зависит от **одного** свободного параметра t . Этим параметром, в частности, может быть и одна из координат x , y или z :

- если хотя бы одно неявное уравнение, например $f_1(x,y,z)=0$, разрешимо относительно некоторой координаты, например $x=\varphi_1(y,z)$, а второе уравнение после подстановки $f_2(\varphi_1(y,z),y,z)=0$ разрешимо относительно другой координаты, например $y=\varphi_2(z)$, то, параметризируя третью координату $z=t$, получим **аналитическую** модель линии пересечения $p(t)=[\varphi_1(\varphi_2(t),t) \ \varphi_2(t) \ t]$;

- в отсутствие явных форм уравнений обеих поверхностей точки линии $p(t)=[x(t) \ y(t) \ t]$ рассчитываются для каждого фиксированного параметра $t=z$, при котором система двух уравнений $f_1(x,y,t)=0$ и $f_2(x,y,t)=0$ **численно** разрешима относительно двух неизвестных $x(t)$ и $y(t)$.

Решим геометрическую **задачу пересечения двух плоскостей** (рис. 1.104), актуальную в ряде алгоритмов 3d-графики, связанных с созданием моделей многогранников и их визуализацией. Плоскости зададим в неявной форме векторами $F_1=[N_1 \ D_1]$ и $F_2=[N_2 \ D_2]$, а результат — модель прямой линии $\{o, V\}$ — будем искать в **параметрической форме** $p(t)=o+Vt$, совпадающей с моделью (1.16а), дополненной функцией изменения аппликаты $z(t)=o_z+V_z t$.

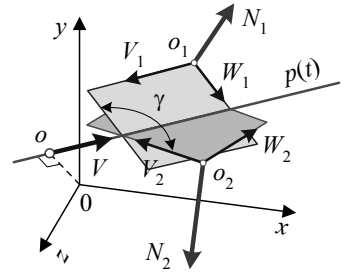


Рис. 1.104

Направляющий вектор прямой, ортогональный к нормальям обеих плоскостей N_1 и N_2 , наиболее просто вычисляется как $V=N_1 \times N_2$. Задача нахождения второго элемента параметрической модели прямой — точки o — не имеет однозначного решения ввиду **недоопределённости** системы двух скалярных уравнений плоскостей относительно трёх координат вектора p :

$$\begin{bmatrix} p & 1 \end{bmatrix} \begin{bmatrix} N_1^T & N_2^T \\ D_1 & D_2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix} \Rightarrow p \begin{bmatrix} N_1^T & N_2^T \end{bmatrix} = \begin{bmatrix} -D_1 & -D_2 \end{bmatrix}. \quad (1.109a)$$

Из (1.6г) следует МНК-решение СЛАУ с минимальной нормой:

$$o = -\begin{bmatrix} D_1 & D_2 \end{bmatrix} (M^T M)^{-1} M^T, \text{ где } M = \begin{bmatrix} N_1^T & N_2^T \end{bmatrix} \in \mathbb{R}^{3 \times 2}. \quad (1.109б)$$

Таким образом, полученная **точка o лежит на прямой пересечения плоскостей и является ближайшей к началу координат**. Геометрически это означает ортогональность радиус-вектора o направляющему вектору V . И действительно, поскольку $VM = (N_1 \times N_2) \begin{bmatrix} N_1^T & N_2^T \end{bmatrix} = [(N_1 \times N_2) \circ N_1 \ (N_1 \times N_2) \circ N_2] = \begin{bmatrix} 0 & 0 \end{bmatrix}$, то $o \circ V = o V^T = -\begin{bmatrix} D_1 & D_2 \end{bmatrix} (M^T M)^{-1} (VM)^T = 0$, что есть условие ортогональности.

В качестве упражнения читателю рекомендуется получить решение методом наименьших квадратов, минимизируя функцию Лагранжа

$$\mathfrak{J}(p, \lambda_1, \lambda_2) = p \circ p + \lambda_1 (p \circ N_1 + D_1) + \lambda_2 (p \circ N_2 + D_2).$$

Условие существования решения есть условие невырожденности обращаемой в (1.109б) матрицы:

$$\begin{aligned} |M^T M| &= \begin{vmatrix} N_1 \circ N_1 & N_1 \circ N_2 \\ N_2 \circ N_1 & N_2 \circ N_2 \end{vmatrix} = |N_1|^2 |N_2|^2 - (N_1 \circ N_2)^2 = \\ &= |N_1|^2 |N_2|^2 (1 - \cos^2(\angle N_1 N_2)) = |N_1|^2 |N_2|^2 \sin^2(\angle N_1 N_2) \neq 0. \end{aligned} \quad (1.109в)$$

Это означает, что нормали к плоскостям, а следовательно, и сами **плоскости не должны быть параллельны**. Если это так, то определена не только точка прямой o , но и её направляющий вектор $V = N_1 \times N_2 \neq O_3$.

К **недостаткам** поверхностных методов описания 3d-линий относятся:

- необходимость подбора подходящих поверхностей для получения желаемой линии их пересечения;
- неоднозначность, сложность или невозможность явного разрешения уравнений поверхностей;
- сложность определения законов изменения координатного параметра t и составления линии $p(t)$ из отдельных ветвей.

Поскольку результат пересечения поверхностей не одноточечный и зависит от некоторого свободного параметра, то практичнее сразу задавать модель линии в параметрической форме $p(t)$ с имеющим определённый геометрический или физический смысл параметром t , **монотонно изменяющимся** в известных границах. Если рассматривать параметр t как равномерно текущее время, а линию — как след, оставляемый движущейся точкой в пространстве, то направляющему вектору параметрической линии соответствует вычисляемая по (1.8б) скорость $V(t) = p'(t)$, касательная к линии в точке $p(t)$.

В каждой точке **гладкой** линии $p(t)$ можно построить систему плоскостей

$\{\alpha, \beta, \gamma\}$, называемую подвижным **трёхгранником Френé** (рис. 1.105):

- плоскость $\alpha = \{p, V\}$, ортогональная вектору $V(t)$, называется **нормальной**. Она содержит все нормали к линии в точке $p(t)$;

- предельное положение плоскости, проходящей через три точки $p(t)$, $p(t+\varepsilon)$ и $p(t-\varepsilon)$ при $\varepsilon \rightarrow 0$, называется **соприкасающейся** плоскостью β . Найдём вектор её нормали, ограничиваясь квадратичными приближениями векторных функций

$$p(t \pm \varepsilon) \approx p(t) \pm \varepsilon V(t) + 0.5 \varepsilon^2 V'(t):$$

$$\begin{aligned} (p(t) - p(t-\varepsilon)) \times (p(t+\varepsilon) - p(t)) &\approx (\varepsilon V(t) - 0.5 \varepsilon^2 V'(t)) \times (\varepsilon V(t) + 0.5 \varepsilon^2 V'(t)) = \\ &= \varepsilon^2 V(t) \times V(t) + 0.5 \varepsilon^3 (V(t) \times V'(t) - V'(t) \times V(t)) - 0.25 \varepsilon^4 V'(t) \times V'(t) = \varepsilon^3 V(t) \times V'(t). \end{aligned}$$

Таким образом, соприкасающаяся плоскость описывается нормальной моделью $\beta = \{p, B\}$ с вектором $B = V(t) \times V'(t)$ **бинормали** к линии в точке $p(t)$;

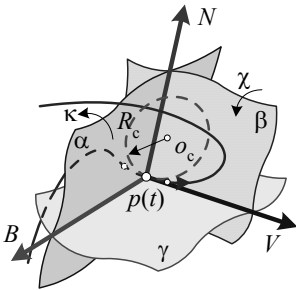


Рис. 1.105

• плоскость $\gamma = \{p, N\}$, ортогональная вектору $N(t) = B(t) \times V(t)$ **главной нормали** к линии в точке $p(t)$, называется **спрямляющей**.

Правая тройка векторов $\{\bar{V}, \bar{N}, \bar{B}\}$, приложенных к точке $p(t)$, называется **рёпером Френэ**. У точек, лежащих на прямолинейном участке кривой или на прямой линии в целом, векторы скорости V и ускорения V' параллельны, а плоскости β, γ и нормаль $N(t)$ не определены из-за нулевой бинормали $|B| = 0$.

Изгиб линии в точке p может происходить в двух ортогональных направлениях и описываться тремя числовыми характеристиками [8, 21] — **кривизной** κ , **кручением** χ и **полной кривизной** K :

$$\kappa = \frac{|V \times V'|}{|V|^3}, \quad \chi = \frac{V \circ (V' \times V'')}{|V \times V'|^2}, \quad K = \frac{\kappa^2 \chi}{\sqrt{\kappa^2 \chi^2 + (\kappa')^2}}. \quad (1.110a)$$

Кривизна является угловой скоростью вращения нормальной плоскости α вокруг мгновенного положения бинормали B , а кручение представляет собой угловую скорость вращения соприкасающейся плоскости β вокруг касательного вектора V . Полная кривизна обратна радиусу R_c большого круга соприкасающейся сферы с центром в точке

$$o_c = p + \frac{1}{\kappa} \bar{N} - \frac{\kappa'}{\kappa^2 \chi} \bar{B}. \quad (1.110b)$$

1.5.3. Взаимное положение графических элементов в пространстве

Рассмотрим некоторые **полезные тесты, формулы и алгоритмы**, описывающие свойства и взаимное расположение точек, прямых, плоскостей и полиэдров в пространстве. Для получения необходимых результатов активно используются тесты ортогональности и параллельности векторов из перечня основных обозначений и соотношений.

Коллинеарность точек

Три точки p_1, p_2 и p_3 коллинеарны, т. е. лежат на одной прямой, если

$$(p_2 - p_1) \times (p_3 - p_1) = O_3 \quad \text{или} \quad |(p_2 - p_1) \circ (p_3 - p_1)| = |p_2 - p_1| \cdot |p_3 - p_1|. \quad (1.111a)$$

Обобщение (1.111a) на произвольное количество точек $p_1 \div p_n$:

$$\text{rang} \begin{bmatrix} \tilde{p}_1 \\ \tilde{p}_2 \\ \dots \\ \tilde{p}_n \end{bmatrix} = 2 \quad \text{или} \quad \text{rang} \begin{bmatrix} p_2 - p_1 \\ \dots \\ p_n - p_1 \end{bmatrix} = 1. \quad (1.111b)$$

Проверка принадлежности точки p отрезку ab выполняется тестом (1.21), а значение параметра расположения точки $p = a + (b - a)t$ на отрезке ab вычисляется по (1.22) при $V = b - a$ как $t = (p - a) \circ V / V \circ V$, тогда $p \in ab$ при $t \in [0, 1]$.

Компланарность точек

Четыре точки p_1, p_2, p_3 и p_4 компланарны, т. е. лежат в одной плоскости, если

$$\begin{vmatrix} \tilde{p}_1 \\ \dots \\ \tilde{p}_4 \end{vmatrix} = 0, \text{ или } \begin{vmatrix} p_2 - p_1 \\ p_3 - p_1 \\ p_4 - p_1 \end{vmatrix} = 0, \text{ или } ((p_2 - p_1) \times (p_3 - p_1)) \circ (p_4 - p_1) = 0. \quad (1.112a)$$

Размерность пространства R^m , натянутого на n точек $p_1 \div p_n$, вычисляется по определению ранга матрицы как число её линейно независимых строк:

$$m = \text{rang} \begin{bmatrix} \tilde{p}_1 \\ \dots \\ \tilde{p}_n \end{bmatrix} - 1 = \text{rang} \begin{bmatrix} p_2 - p_1 \\ \dots \\ p_n - p_1 \end{bmatrix} \leq \min\{3, n-1\}. \quad (1.112б)$$

Точка и прямая

Проведём от точки o **прямой** $p(t) = o + Vt$ к **точке** q вектор $W = q - o$, составляющий с вектором V угол $\alpha = \arccos(\bar{V} \circ \bar{W})$ (рис. 1.106). Построим от точки $c = q - N$ векторы бинормали $B = \bar{V} \times W$ и нормали $N = \text{or}_V W = B \times \bar{V}$, длины которых равны **расстоянию от точки q до прямой** $\{o, V\}$:

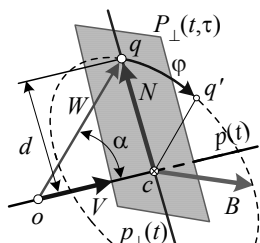


Рис. 1.106

$$d = |W| \sin(\alpha) = |N| = |B| = |\bar{V} \times (q - o)|. \quad (1.113)$$

Точка принадлежит прямой при $d=0$.

Модель перпендикуляра, опущенного из точки на прямую в направлении её нормали N , имеет вид

$$p_{\perp}(t) = q + Nt. \quad (1.114)$$

Модели плоскости, проходящей через точку ортогонально прямой, принимают следующие формы:

НФ: $\tilde{P}_{\perp} \circ F_{\perp} = 0$, где $F_{\perp} = [V \ -q \circ V]$; (1.115a)

ПФ: $P_{\perp}(t, \tau) = q + Nt + B\tau$. (1.115б)

Вращение точки q вокруг прямой $\{o, V\}$ на угол φ происходит в плоскости $P_{\perp}$ вокруг точки c . Приняв положительным угол поворота правого винта, заворачиваемого в направлении V , запишем положение точки q' как суперпозицию ортогональных гармонических колебаний точки q вокруг центра c :

$$q' = c + N \cos(\varphi) + B \sin(\varphi) = q + N(\cos(\varphi) - 1) + B \sin(\varphi). \quad (1.116)$$

Точка и плоскость

Рассмотрим расположение **точки q** и **плоскости**, заданной в неявной и параметрической формах (рис. 1.107).

Точка принадлежит плоскости, если $f(q)=0$:

$$\tilde{q} \circ F = (q - o) \circ N = (q - o) \circ (V \times W) = 0. \quad (1.117)$$

Модели перпендикуляра, опущенного из точки q на плоскость:

$$p_{\perp}(t) = q + Nt = q + (V \times W)t. \quad (1.118)$$

Расстояние от точки до плоскости находится аналогично (1.24a):

$$d = |\text{pr}_N(q - o)| = |(q - o) \circ \bar{N}| = |(q - o) \circ \overline{V \times W}|. \quad (1.119)$$

Зеркальное отражение точки q от плоскости $\{o, N\}$ строится путём продления перпендикуляра (1.118) на расстояние d от точки пересечения. Формула вычисления зеркальной точки q' полностью совпадает с (1.25a):

$$q' = q - 2((q - o) \circ \bar{N})\bar{N}.$$

Модели параллельной плоскости, проходящей через точку q :

$$\text{НФ: } \tilde{q} \circ F_{\parallel} = 0, F_{\parallel} = [N - q \circ N]; \quad \text{ПФ: } p_{\parallel}(t, \tau) = q + Vt + W\tau. \quad (1.120)$$

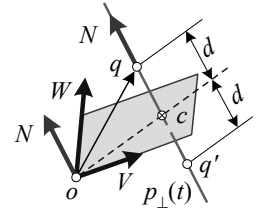


Рис. 1.107

Плоскость, наименее удалённая от совокупности точек

Рассмотрим задачу, связанную с понятием расстояния от точки до плоскости. Допустим, в $\mathbb{R}^3$ задано $m \geq 4$ произвольных точек $p_1 \div p_m$. Так как в общем случае точки не компланарны, то не существует ни одной плоскости $f(p)=0$, для которой все числа $d_i = f(p_i)$ обращаются в нуль. Другими словами, расстояния $|d_i|/|N|$ от точек p_i до этой плоскости, вычисленные по (1.119) и пропорциональные d_i , не могут быть одновременно все нулевыми (рис. 1.108).

Алгоритм построения аппроксимирующей плоскости $\{c, N\}$, наименее удалённой от совокупности точек p_i , аналогичен двумерному алгоритму (1.27), (1.28). Применительно к размерности пространства $n=3$ он имеет следующий вид.

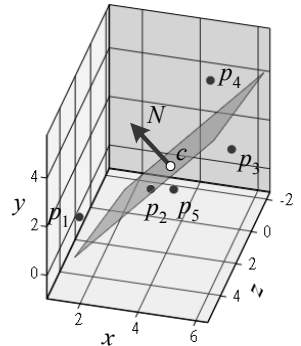


Рис. 1.108

Шаг 1. Сформируем из точек $p_1 \div p_m$ $m \times 3$ -матрицу P и найдём их геометрический центр c :

$$P = \begin{bmatrix} p_1 \\ \dots \\ p_m \end{bmatrix}, \quad c = \text{mean}(P) = \frac{1}{m} \sum_{i=1}^m p_i. \quad (1.121a)$$

Шаг 2. Вычислим собственные значения $\lambda_1 \div \lambda_3$ и соответствующие им **нормированные** собственные векторы $N_1 \div N_3$ симметричной 3×3 -матрицы

$$M = mc^T c - P^T P. \quad (1.121b)$$

Шаг 3. Определив собственную пару $\{\lambda_k, N_k\}$ с **максимальным** собственным значением $\lambda_k = \max\{\lambda_i\}$, получим элементы вектора $F = [N \ D]$ неявного уравнения искомой плоскости (1.96):

$$N = N_k, \quad D = -N \circ c. \quad (1.121в)$$



Вместо неявной формы плоскости можно использовать её нормальное уравнение $(p-c) \circ N = 0$. Для построения более удобна параметрическая форма $p(t, \tau) = c + Vt + W\tau$, направляющие векторы которой V и W находятся по (1.104).

Оставшиеся две пары собственных значений и векторов матрицы M дают плоскости, ортогональные оптимальной плоскости (при различных собственных числах) и пересекающиеся с ней в геометрическом центре точек c .

Пример 1.16. Построить плоскость, наименее отклонённую от точек $p_1 = [1 \ 0 \ 2]$, $p_2 = [4 \ 3 \ 4]$, $p_3 = [6 \ 3 \ 1]$, $p_4 = [5 \ 5 \ 0]$ и $p_5 = [4 \ 1 \ 1]$ (рис. 1.108).

Решение. Следуем шагам алгоритма (1.121).

$$\text{Шаг 1. } P = \begin{bmatrix} 1 & 4 & 6 & 5 & 4 \\ 0 & 3 & 3 & 5 & 1 \\ 2 & 4 & 1 & 0 & 1 \end{bmatrix}^T, \quad c = \frac{1}{5} \sum_{i=1}^5 p_i = [4 \ 2.4 \ 1.6].$$

$$\text{Шаг 2. } M = 5c^T c - P^T P = \begin{bmatrix} -14 & -11 & 4 \\ -11 & -15.2 & 3.2 \\ 4 & 3.2 & -9.2 \end{bmatrix}, \quad \boxed{\lambda_1 = -3.475}, \quad N_1 = [-0.74 \ 0.65 \ -0.15],$$

$$\lambda_2 = -7.865, \quad N_2 = [0.073 \ 0.304 \ 0.95], \quad \lambda_3 = -27.06, \quad N_3 = [-0.67 \ -0.69 \ 0.27].$$

$$\text{Шаг 3. } k=1, \quad N = [-0.74 \ 0.65 \ -0.15], \quad D = 1.64.$$

Таким образом, аппроксимирующая плоскость описывается неявным уравнением $-0.74x + 0.65y - 0.15z + 1.64 = 0$. Для получения направляющих векторов её параметрической формы выберем вектор $H = [0 \ 0 \ 1]$, не параллельный N . Тогда по (1.104) получим векторы $V = [-0.65 \ -0.74 \ 0]$, $W = [-0.11 \ 0.1 \ 0.98]$ и параметрические функции плоскости

$$x(t, \tau) = 4 - 0.65t - 0.11\tau, \quad y(t, \tau) = 2.4 - 0.74t + 0.1\tau, \quad z(t, \tau) = 1.6 + 0.98\tau.$$

Алгебраические расстояния от заданных точек до этой плоскости равны $d_1 = 0.59$, $d_2 = 0.026$ (точка p_2 почти лежит в плоскости), $d_3 = -1$, $d_4 = 1.2$ и $d_5 = -0.82$, а минимум функции Лагранжа равен $\mathfrak{Z}_{\min} = 3.475$. $\square$

Две прямые

Исследуем расположение в пространстве **двух прямых** $p_1(t) = o_1 + V_1 t_1$ и $p_2(t_2) = o_2 + V_2 t_2$ (рис. 1.109).

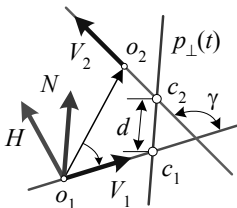


Рис. 1.109

Прямые совпадают, если

$$\{N = O_3\} \wedge \{H = O_3\},$$

$$\text{где } N = V_1 \times V_2, \quad H = (o_2 - o_1) \times V_1.$$

$$(1.122)$$

Прямые параллельны, если $N = O_3$. Расстояние между параллельными прямыми равно

$$d = |(o_2 - o_1) \times \overline{V_1}|.$$

Модели плоскости, проходящей через две параллельные несовпадающие прямые, имеют вид:

$$\text{НФ: } \tilde{p} \circ F_{||} = 0, \quad F_{||} = [H - o_1 \circ H]; \quad \text{ПФ: } p(t, \tau) = o_1 + (o_2 - o_1)t + V_1\tau. \quad (1.123)$$

Прямые компланарны при выполнении условия $(o_2 - o_1) \circ (V_1 \times V_2) = 0$ расположения вектора $o_2 - o_1$ и направляющих векторов V_1 и V_2 в одной плоскости. Один из **углов пересечения прямых**

$$\gamma_{кр} = \arccos(\bar{V}_1 \circ \bar{V}_2) \in [0, 180^\circ] \quad (1.124)$$

есть угол **кратчайшего** вращения от вектора V_1 к вектору V_2 . Вторым углом пересечения прямых является смежный угол $\gamma_{см} = 180^\circ - \gamma_{кр} = \arccos(-\bar{V}_1 \circ \bar{V}_2)$.

Модели плоскости, проходящей через две пересекающиеся прямые:

$$\text{НФ: } \tilde{p} \circ F_{\times} = 0, \quad \text{где } F_{\times} = [N - o_1 \circ N]; \quad \text{ПФ: } p(t, \tau) = o_1 + V_1t + V_2\tau. \quad (1.125)$$

Прямые ортогональны, если $V_1 \circ V_2 = 0$.

Прямая, соединяющая скрещивающиеся прямые по кратчайшему пути, проходит ортогонально каждой прямой через точки $c_1 = o_1 + V_1t_1$ и $c_2 = o_2 + V_2t_2$. Параметры t_1 и t_2 находятся из условий ортогональности

$$\begin{cases} (c_2 - c_1) \circ V_1 = 0 \\ (c_1 - c_2) \circ V_2 = 0 \end{cases} \Rightarrow [t_1 \ t_2] = (o_2 - o_1) [V_1^T \ -V_2^T] \begin{bmatrix} V_1 \circ V_1 & -V_1 \circ V_2 \\ -V_1 \circ V_2 & V_2 \circ V_2 \end{bmatrix}^{-1}. \quad (1.126)$$

Модель кратчайшей прямой есть $p_{\perp}(t) = c_1 + (V_1 \times V_2)t$. **Кратчайшее расстояние между скрещивающимися прямыми** равно

$$d = |c_1 - c_2| = |(o_2 - o_1) \circ \bar{N}|.$$

При совпадении точек c_1 и c_2 получаем $d=0$ и одну **точку пересечения компланарных прямых** $c = o_1 + V_1t_1 = o_2 + V_2t_2$.

Прямая и плоскость

Рассмотрим расположение **прямой** $p(\theta) = q + U\theta$ и **плоскости**, заданной в неявной $\{o, N\}$ или параметрической $\{o, V, W\}$ формах (рис. 1.110).

Прямая принадлежит плоскости, если

$$\text{НФ: } \{U \circ N = 0\} \wedge \{\tilde{q} \circ F = 0\}; \quad (1.127a)$$

$$\text{ПФ: } \{U \circ (V \times W) = 0\} \wedge \{(q - o) \circ (V \times W) = 0\}. \quad (1.127b)$$

Прямая параллельна плоскости, если

$$\text{НФ: } U \circ N = 0; \quad \text{ПФ: } U \circ (V \times W) = 0. \quad (1.128)$$

Расстояние от прямой до параллельной ей плоскости

$$d = |(q - o) \circ \bar{N}| = |(q - o) \circ \bar{V} \times \bar{W}|.$$

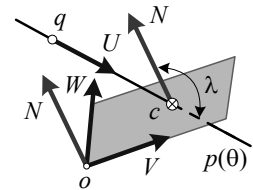


Рис. 1.110

Прямая ортогональна плоскости, если

$$\text{НФ: } U \times N = O_3; \quad \text{ПФ: } U \times (V \times W) = O_3. \quad (1.129)$$

Одной из основных задач трёхмерной графики является расчёт **точки пересечения прямой** $\{q, U\}$ **с плоскостью**, аналогичный расчётам (1.31) пересечения двух прямых на плоскости xy :

$$\text{НФ: } \begin{cases} c = q + U\theta \\ (c - o) \circ N = 0 \end{cases} \Rightarrow \theta = \frac{(o - q) \circ N}{U \circ N} \Rightarrow \boxed{c = q + \frac{(o - q) \circ N}{U \circ N} U}; \quad (1.130a)$$

$$\text{ПФ: } q + U\theta = o + Vt + W\tau \Rightarrow [\theta \ t \ \tau] = (o - q) \begin{bmatrix} U \\ -V \\ -W \end{bmatrix}^{-1} \Rightarrow c = q + U\theta. \quad (1.130б)$$

Условия существования решения: $U \circ N \neq 0$ или $U \circ (V \times W) \neq 0$ — **прямая и плоскость не параллельны.**

Ввиду частого использования формулы (1.130a) определим **функцию** $\boxed{\text{cross_line}(q, U, o, N)}$ **пересечения прямой** $\{q, U\}$ **с плоскостью** $\{o, N\}$, возвращающую точку пересечения этих геометрических объектов при $U \circ N \neq 0$.

Вычислительная сложность задачи (1.130б) составляет 77 арифметических операций в сравнении с 20 операциями в (1.130a). Однако её решение более информативно и позволяет найти точку пересечения **частей** прямой и плоскости, включая их границы, что затруднительно сделать по прямой формуле (1.130a):

- если часть прямой представляет собой **луч** либо **отрезок**, то найденное значение параметра θ должно удовлетворять условию $\theta \geq 0$ либо $0 \leq \theta \leq 1$;
- если часть плоскости является **параллелограммом** $P = p_1 p_2 p_3 p_4 p_1$ с векторами $V = p_2 - p_1$ и $W = p_3 - p_2$, то при выборе точки $o = p_1$ параметры t и τ должны согласно (1.44a) лежать в единичных интервалах $\{0 \leq t \leq 1\} \wedge \{0 \leq \tau \leq 1\}$;
- если часть плоскости имеет форму **треугольника** $P = p_1 p_2 p_3 p_1$ с теми же векторами $V = p_2 - p_1$ и $W = p_3 - p_2$, то при $o = p_1$ параметры t и τ должны аналогично (1.44б) удовлетворять неравенствам $0 \leq \tau \leq t \leq 1$.

Таким образом, точка c пересечения прямой или любой её части с гранями полиэдра в форме этих двух плоских фигур находится достаточно просто **без преобразования системы координат**. В общем случае плоского **полигона** $P = p_1 p_2 p_3 \dots p_n p_1$ с векторами сторон $V = p_2 - p_1$ и $W = p_3 - p_2$, а также при использовании нормального метода (1.130a), принадлежность $c \in P$ может быть (пока) определена любым из плоских тестов локализации во вспомогательной СК $x'y'z'$, чья фронтальная плоскость $x'y'$ совпадает с плоскостью $\{p_1, V, W\}$.

Угол между прямой и нормалью плоскости находится по (1.124) как

$$\lambda = \arccos(\overline{U \circ N}) = \arccos(\overline{U \circ V \times W}). \quad (1.131)$$

Модели плоскости, проходящей через прямую $q + U\theta$ **ортогонально заданной плоскости**, имеют вид:

$$\text{НФ: } \tilde{\rho} \circ F_{\perp} = 0, \text{ где } F_{\perp} = [N_{\perp} \ -q \circ N_{\perp}], \ N_{\perp} = U \times N; \quad (1.132a)$$

$$\text{ПФ: } p(t, \tau) = q + Ut + (V \times W)\tau. \quad (1.132б)$$

Условие единственности решения: $U \times N \neq O_3$ — прямая не ортогональна плоскости, т. е. не параллельна её нормали.

Две плоскости

Исследуем расположение в пространстве *двух плоскостей*, заданных в неявной и параметрической формах (см. рис. 1.104).

Плоскости совпадают, если

$$\text{НФ:} \quad \{N_1 \times N_2 = O_3\} \wedge \{(o_2 - o_1) \circ N_2 = 0\}; \quad (1.133a)$$

$$\text{ПФ:} \quad \{(V_1 \times W_1) \times (V_2 \times W_2) = O_3\} \wedge \{(o_2 - o_1) \circ (V_2 \times W_2) = 0\}. \quad (1.133б)$$

Плоскости параллельны, если

$$\text{НФ:} \quad N_1 \times N_2 = O_3; \quad \text{ПФ:} \quad (V_1 \times W_1) \times (V_2 \times W_2) = O_3. \quad (1.134)$$

Расстояние между параллельными плоскостями равно

$$d = |(o_2 - o_1) \circ \overline{N_1}| = |(o_2 - o_1) \circ \overline{V_1 \times W_1}|.$$

Плоскости ортогональны, если

$$\text{НФ:} \quad N_1 \circ N_2 = 0; \quad \text{ПФ:} \quad (V_1 \times W_1) \circ (V_2 \times W_2) = 0. \quad (1.135)$$

Важной задачей 3d-графики является расчёт *прямой* $p(t) = o + Vt$ *пересечения двух плоскостей* (см. рис. 1.104), выполненный в (1.109):

$$o = [o_1 \circ N_1 \quad o_2 \circ N_2] \begin{bmatrix} N_1 \circ N_1 & N_1 \circ N_2 \\ N_1 \circ N_2 & N_2 \circ N_2 \end{bmatrix}^{-1} \begin{bmatrix} N_1 \\ N_2 \end{bmatrix}, \quad V = N_1 \times N_2.$$

Аналогично (1.124) дополним его формулами вычисления кратчайшего и смежного ему *углов пересечения плоскостей*:

$$\gamma_{\text{кр}} = \arccos(\overline{N_1} \circ \overline{N_2}), \quad \gamma_{\text{см}} = 180^\circ - \gamma_{\text{кр}} = \arccos(-\overline{N_1} \circ \overline{N_2}). \quad (1.136)$$

Модели плоскости, проходящей через точку о ортогонально двум плоскостям, имеют следующий вид:

$$\text{НФ:} \quad \tilde{p} \circ F_{\perp} = 0, \quad \text{где } F_{\perp} = [N_{\perp} \quad -o \circ N_{\perp}], \quad N_{\perp} = N_1 \times N_2; \quad (1.137a)$$

$$\text{ПФ:} \quad p(t, \tau) = o + (V_1 \times W_1)t + (V_2 \times W_2)\tau. \quad (1.137б)$$

Пучок плоскостей и биссекторная плоскость

Модели пучка плоскостей, заключённых между двумя плоскостями (рис. 1.111, а), имеют вид:

$$\text{НФ:} \quad \tilde{p} \circ F_{\lambda+1} = 0, \quad \text{где } F_{\lambda+1} = \text{lin}(F_1, F_2, \lambda); \quad (1.138a)$$

$$\text{ПФ:} \quad p_{\lambda+1}(t, \tau) = \text{lin}(o_1 + V_1 t + W_1 \tau, o_2 + V_2 t + W_2 \tau, \lambda). \quad (1.138б)$$

Изменение параметра пучка в интервале $0 \leq \lambda \leq 1$ даёт такие положения плоскостей, что вращение нормалей $N_{\lambda+1} = \text{lin}(N_1, N_2, \lambda)$ происходит *по крат-*

чайшему углу от N_1 до N_2 . Модели (1.138) являются линейными комбинациями точек плоскостей, соответствующих равным значениям параметров t и τ .

Параметры **биссекторной плоскости** получаются из (1.138) при $\lambda=0.5$ и уравнивании длин векторов, определяющих исходные плоскости:

$$F_{\text{бис}} = |N_2|F_1 + |N_1|F_2, \quad V_{\text{бис}} = |V_2|V_1 + |V_1|V_2, \quad W_{\text{бис}} = |W_2|W_1 + |W_1|W_2. \quad (1.139)$$

Расчёт биссекторных плоскостей бывает необходим, например, при построении сферы, вписанной в четырёхгранник. Её центр o лежит в точке пересечения трёх биссекторных плоскостей **внутренних** телесных углов, примыкающих к одной из треугольных граней, а радиус r вычисляется как расстояние (1.119) от точки o до плоскости любой грани, например, $\{o_1, N_1\}$.

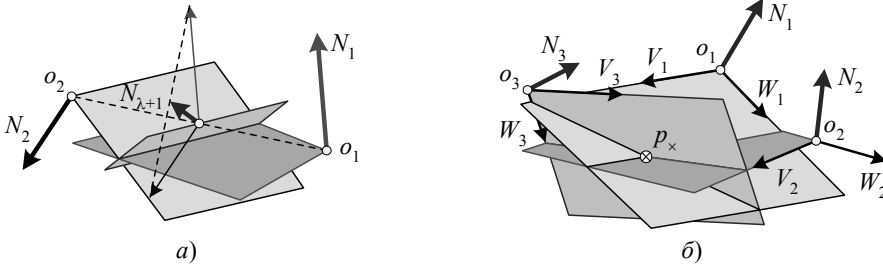


Рис. 1.111

Пересечение плоскостей

Точка пересечения трёх плоскостей (рис. 1.111,б) определяется из условия её принадлежности этим плоскостям:

$$\text{НФ:} \quad [p_x \ 1] [F_1^T \ F_2^T \ F_3^T] = [0 \ 0 \ 0] \Rightarrow p_x = -[D_1 \ D_2 \ D_3] [N_1^T \ N_2^T \ N_3^T]^{-1}; \quad (1.140a)$$

$$\text{ПФ:} \quad p_x = [o_1 \circ N_1 \ o_2 \circ N_2 \ o_3 \circ N_3] [N_1^T \ N_2^T \ N_3^T]^{-1}, \quad \text{где } N_i = V_i \times W_i. \quad (1.140б)$$

Условие существования и единственности решения $|N_1^T \ N_2^T \ N_3^T| \neq 0$ — линейная независимость нормалей плоскостей, что равносильно существованию и попарной непараллельности линий пересечения плоскостей.

По данным формулам рассчитываются вершины полиэдра как точки пересечения плоскостей любых трёх смежных граней.

Пересечение n плоскостей в общей точке p_x , вычисляемой по (1.140а,б) на основе параметров первых трёх плоскостей, возможно при условии

$$\text{rang}[F_1^T \ F_2^T \ \dots \ F_n^T] = 3, \quad (1.140в)$$

аккумулирующем требования **совместности** системы линейных уравнений, предъявляемое теоремой **Кронекера – Капелли** [9, 10, 12] о равенстве рангов матриц $[F_1^T \ F_2^T \ \dots \ F_n^T]$ и $[N_1^T \ N_2^T \ \dots \ N_n^T]$, и **единственности** её решения, определяемой полнотой ранга матрицы $[N_1^T \ N_2^T \ \dots \ N_n^T]$, который должен быть равен трём.

1.5.4. Модель полиэдра

Переходим к изучению **полиэдров** (многогранников) — замкнутых трёхмерных объектов, ограниченных плоскостями. Помимо того, что существует множество собственно полиэдральных объектов (параллелепипеды, призмы, пирамиды, кристаллы и т. п.), часто замкнутые поверхности аппроксимируют системой полигональных граней, что превращает гладкие тела в полиэдры.

К графическим элементам полиэдра относятся:

- **грани** — ограниченные плоскости, отделяющие внутреннее пространство полиэдра от внешнего;
- **рёбра** — линии пересечения граней;
- **вершины** — точки пересечения рёбер.

Вершины, **инцидентные** (присущие) одному и тому же ребру, как и грани, связанные общим ребром, являются **смежными**. Благодаря планарности всех граничных поверхностей полиэдра существенно упрощается расчёт его пересечений с другими полиэдрами и другими графическими объектами.

Количественное соотношение между числом вершин n_v , граней **без отверстий** n_r и рёбер n_p **замкнутого** полиэдра устанавливается удивительно простой с виду, но сложной в выводе, **формулой Эйлера** [8, 13, 21]:

$$\boxed{n_v + n_r - n_p = 2}. \quad (1.141a)$$

К примеру, невыпуклый полиэдр, изображённый на рис. 1.112,а, имеет 14 вершин, 9 граней без отверстий и 21 ребро, что точно соответствует (1.141a).

Если полиэдр имеет $n_{го}$ граней с **глухими отверстиями** либо **шипами**, которые можно интерпретировать как вывернутые наружу глухие отверстия, то вместо (1.141a) справедливо иное соотношение:

$$n_v + n_r - n_p = 2 + n_{го}. \quad (1.141б)$$

Его можно доказать с позиций конструктивного моделирования. Полиэдр с параметрами $\{n_v, n_p, n_r\}$ образуется путём исключения полиэдра углубления $\{n_{v2}, n_{p2}, n_{r2}\}$ из «недырявого» полиэдра $\{n_{v1}, n_{p1}, n_{r1}\}$, либо путём объединения полиэдра $\{n_{v1}, n_{p1}, n_{r1}\}$ с полиэдром шипа $\{n_{v2}, n_{p2}, n_{r2}\}$. Обе тройки параметров полиэдров удовлетворяют формуле Эйлера: $n_{v1} + n_{r1} - n_{p1} = 2$ и $n_{v2} + n_{r2} - n_{p2} = 2$. В результате логической операции соприкасающаяся грань первого объекта приобретает $n_{го}=1$ отверстие, а такая же грань второго объекта теряется, т. е. $n_r = n_{r1} + n_{r2} - 1$. В то же время $n_v = n_{v1} + n_{v2}$ и $n_p = n_{p1} + n_{p2}$. Тогда, действительно, получаем (1.141б):

$$n_v + n_r - n_p = n_{v1} + n_{r1} - n_{p1} + n_{v2} + n_{r2} - n_{p2} - 1 = 3 = 2 + n_{го}.$$

Образование у полиэдра **сквозного отверстия** можно представить как прорыв глухим отверстием тонкой стенки противоположной грани, при котором теряется внутренняя грань отверстия. Таким образом, полиэдр со сквозным отверстием подчиняется формуле (1.141б). Обозначив как $n_o = n_{го} + 2n_{co}$ общее число

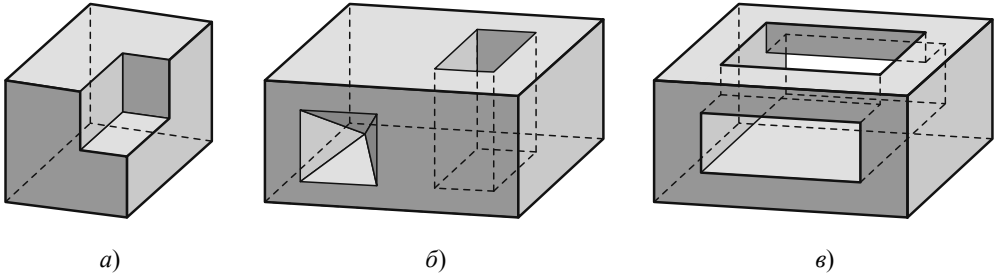


Рис. 1.112

граней с $n_{го}$ глухими и $n_{со}$ сквозными отверстиями, получим альтернативный и иногда более удобный вариант этой формулы:

$$n_b + n_r - n_p = 2 + n_o - 2n_{co}. \quad (1.141в)$$

Убедимся в истинности соотношений (1.141) на следующих примерах:

- полиэдр с одним сквозным и одним глухим отверстием (рис. 1.112,б) имеет $n_b=21$ вершину, $n_r=14$ граней и $n_p=32$ ребра, так что $n_b + n_r - n_p = 3$. По (1.141б) также получим $2 + n_{го} = 3$. Поскольку общее число граней с отверстиями составляет $n_o=3$, то в (1.141в) имеем $2 + n_o - 2n_{co} = 3$;
- более сложный полиэдр, изображённый на рис. 1.112,в, характеризуется параметрами $n_b=24$, $n_r=13$, $n_p=36 \Rightarrow n_b + n_r - n_p = 1$. Оба отверстия являются сквозными, так что при $n_o=3$ по (1.141в) получаем $2 + n_o - 2n_{co} = 1$.

Подробное изучение различных моделей и свойств полиэдров продолжим в гл. 5, а здесь приведём наиболее удачную и удобную в использовании **модель «вершины + грани»** $H = \{P, G\}$, состоящую из двух структур данных:

- координатного **списка** $n=n_b$ **вершин** $P = \{p_1, \dots, p_{n_b}\}$, пронумерованных в произвольном порядке. Благодаря однородности элементов списка удобнее организовать и использовать вершины полиэдра в виде **матрицы вершин** в обычной или расширенной формах (последняя применяется для унификации геометрических преобразований, изучаемых в гл. 3):

$$P = \begin{bmatrix} p_1 \\ \dots \\ p_n \end{bmatrix} \equiv \begin{bmatrix} x_1 & y_1 & z_1 \\ \dots & \dots & \dots \\ x_n & y_n & z_n \end{bmatrix} \in \mathbb{R}^{n \times 3}, \quad \tilde{P} = \begin{bmatrix} \tilde{p}_1 \\ \dots \\ \tilde{p}_n \end{bmatrix} \equiv \begin{bmatrix} x_1 & y_1 & z_1 & 1 \\ \dots & \dots & \dots & \dots \\ x_n & y_n & z_n & 1 \end{bmatrix} \in \mathbb{R}^{n \times 4}; \quad (1.142)$$

- **топологической группы списков** n_r **граней** $G = \{G_1, \dots, G_{n_r}\}$. Элемент группы $G_i = \{g_{i1}, \dots, g_{in_i}, g_{i1}\}$ представляет собой **список номеров** (целочисленных индексов) n_i вершин полигона i -й грани, перечисленных в порядке их обхода по замкнутому контуру.

В таком методе организации структуры данных полиэдра доступ к его конкретной вершине с номером g_{ij} осуществляется в два этапа.

1. Из списка G_i считывается j -й элемент $v = g_{ij}$.
2. Из списка P считывается элемент p_v .

Группа G содержит сведения о порядке соединения рёбрами вершин полиэдра, хранящихся в списке (матрице) P . При этом описание каждого ребра, соединяющего вершины p_i и p_j , **дважды** встречается в списках граней как одна из последовательностей индексов $\{..., i, j, ...\}$ либо $\{..., j, i, ...\}$.

Для иллюстрации сказанного построим модель полиэдра, изображённого на рис. 1.113 и состоящего из $n_v=11$ вершин и $n_r=8$ граней:

$$P = \begin{bmatrix} 0 & 0 & 3 & 3 & 4 & 0 & 4 & 4 & 4 & 0 \\ 5 & 2 & 2 & 2 & 5 & 0 & 0 & 0 & 1 & 1 \\ 0 & 6 & 6 & 0 & 0 & 6 & 6 & 0 & 0 & 6 \end{bmatrix}^T, \quad G = \{G_1, G_2, \dots, G_8\}.$$

Опишем грани полиэдра следующими списками:

$$\begin{aligned} G_1 &= \{1, 2, 3, 5, 1\}, & G_2 &= \{2, 3, 10, 7, 6, 2\}, \\ G_3 &= \{3, 4, 5, 3\}, & G_4 &= \{3, 4, 9, 10, 3\}, \\ G_5 &= \{7, 8, 9, 10, 7\}, & G_6 &= \{5, 4, 9, 8, 11, 1, 5\}, \\ G_7 &= \{1, 2, 6, 11, 1\}, & G_8 &= \{11, 8, 7, 6, 11\}. \end{aligned}$$

Для работы с полиэдрами необходимы две функции, возвращающие:

- полигон вершин i -й грани полиэдра $\{P, G\}$:

$$side(P, G, i) = p_{g_{i1}} \dots p_{g_{in_i}} p_{g_{i1}}; \quad (1.143)$$

- вектор нормали к плоскости полигона $P = p_1 p_2 p_3 \dots p_n p_1 \in \mathbb{R}^3$:

$$norm(P) = (p_2 - p_1) \times (p_3 - p_1). \quad (1.144)$$

Пространственная ориентация плоскости полигона задаётся направляющими векторами $V = p_2 - p_1$, $W = p_3 - p_1$ и нормалью $N = V \times W$ по первым трём **неколлинеарным** вершинам p_1 , p_2 и p_3 . Со стороны **внешнего полупространства**, в которое направлена нормаль, обход вершин полигона выглядит **положительным**, происходящим **против часовой стрелки** (см. рис. 1.102, а, 1.113).

Для обеспечения внешней ориентации нормали i -й грани полиэдра

$$N_i = V_i \times W_i = norm(side(P, G, i)) \quad (1.145)$$

нужно руководствоваться следующими **правилами составления списка G_i** :

- **вершины в списке грани должны перечисляться в положительном направлении** (против часовой стрелки) с точки зрения внешнего наблюдателя;
- если полигон грани не выпуклый, то **первые три вершины должны составлять выпуклый угол** с положительным направлением поворота;
- если направление обхода вершин в списке G_i оказалось отрицательным, то нужно перенумеровать их в обратном порядке: $G_i = \{g_{i1}, g_{in_i}, \dots, g_{i2}, g_{i1}\}$;
- если нормаль к грани не вычисляется по формуле (1.145), а задаётся внешней вручную, то направление обхода вершин в списке этой грани безразлично.

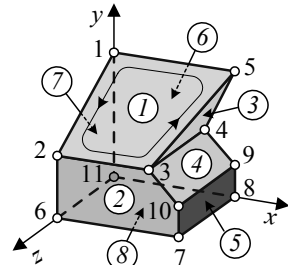


Рис. 1.113

С учётом сказанного скорректируем неверные списки граней полиэдра на рис. 1.113 так, чтобы их нормали (1.145) стали внешними:

$$G_2 = \{2, 6, 7, 10, 3, 2\}, \quad G_4 = \{3, 10, 9, 4, 3\}, \quad G_6 = \{1, 5, 4, 9, 8, 11, 1\}, \quad G_7 = \{1, 11, 6, 2, 1\}.$$

Платоновы тела

Из всех полиэдров особо выделяются пять правильных многогранников, называемых ещё **платоновыми телами** (рис. 1.114). Количество этих фигур определяется ограниченным числом способов соединения в каждой вершине полиэдра **одинакового** числа **однородных** и **правильных** полигонов.

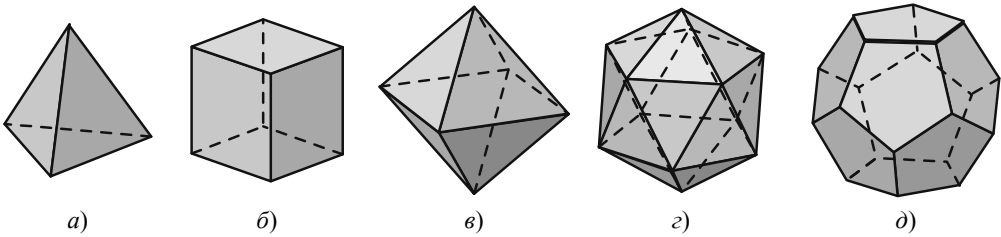


Рис. 1.114

Пусть к вершине примыкает $m \geq 3$ правильных n -угольников, где $n \geq 3$. Сумма их внутренних углов, каждый из которых равен $\psi_n = 180^\circ - 360^\circ/n$, должна быть меньше 360° , т. к. угол при вершине не должен быть плоским. Из условия $m\psi_n < 360^\circ$ и естественных геометрических соображений получаем систему неравенств

$$\begin{cases} m(n-2) < 2n, \\ n \geq 3, m \geq 3, \end{cases}$$

имеющую всего пять **целочисленных** решений $\{n, m\}$, показанных на рис. 1.115:

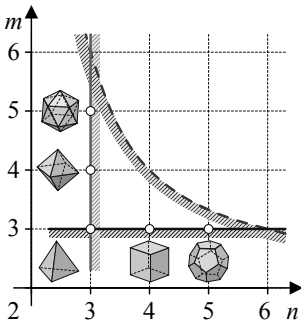


Рис. 1.115

- $\{3, 3\}$ — тетраэдр;
- $\{4, 3\}$ — гексаэдр;
- $\{3, 4\}$ — октаэдр;
- $\{3, 5\}$ — икосаэдр;
- $\{5, 3\}$ — додекаэдр.

Приставка перед греческим корнем «эдр» означает число граней n_r . У любого n_r -эдра число рёбер и вершин равно, соответственно, $n_p = n_r n / 2$ и $n_v = n_r n / m$. С помощью формулы Эйлера (1.141a) нетрудно получить зависимости главных параметров платоновых тел от чисел n и m :

$$n_r = \frac{4m}{2n+2m-nm}, \quad n_v = \frac{4n}{2n+2m-nm}, \quad n_p = \frac{2nm}{2n+2m-nm}.$$

Рассмотрим **экономичные**, т. е. с минимумом ручных расчётов, **методы построения платоновых тел**, у которых координаты n_v вершин отсчитываются от начала локальной СК, находящейся в геометрическом центре симметрии. По-

сколько группа G состоит из n_r списков равной длины $n+1$, то будем задавать её $n_r \times (n+1)$ -матрицей. Добавим к параметрам полиэдра длину ребра d , радиус описанной сферы R_c и длину ребра описанного куба d_k . С помощью этих величин можно управлять относительными размерами платоновых тел:

- деление матрицы P на R_c даёт фигуры, вписанные в сферу единичного радиуса $R_c=1$ (см. рис. 1.114);

- деление матрицы на d_k даёт тела, вписанные в куб с единичными рёбрами;

- деление матрицы P на d даёт тела с рёбрами единичной длины $d=1$.

□ **Тетраэдр** ($n=3$, $n_r=4$, $n_p=6$, $n_b=4$) — правильный полиэдр с минимальным числом граней, построение которого в привычном виде (см. рис. 1.114,а) — совсем не тривиальная задача из-за иррациональности координат вершин. Вращая вершины тетраэдра, вписанного в куб, вокруг осей ОСК на углы $\varphi_y = -45^\circ$ и $\varphi_x = \arctg(\sqrt{2}) \approx 54.7^\circ$ (рис. 1.116) путём умножения матрицы координат четырёх вершин куба на произведение матриц вращений $A = R_y(\varphi_y)R_x(\varphi_x)$, которые будут определены в п. 3.1.2, получим желаемые параметры тетраэдра

$$P4 = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & 1 \\ -1 & -1 & 1 \\ 1 & -1 & -1 \end{bmatrix}, A = \begin{bmatrix} 0 & \sqrt{3} & 0 \\ 0 & -1/\sqrt{3} & 2\sqrt{2/3} \\ -\sqrt{2} & -1/\sqrt{3} & -\sqrt{2/3} \\ \sqrt{2} & -1/\sqrt{3} & -\sqrt{2/3} \end{bmatrix}, G4 = \begin{bmatrix} 1 & 3 & 2 & 1 \\ 1 & 2 & 4 & 1 \\ 1 & 4 & 3 & 1 \\ 2 & 3 & 4 & 2 \end{bmatrix},$$

$$d = 2\sqrt{2}, R_c = \sqrt{3}, d_k = 2.$$

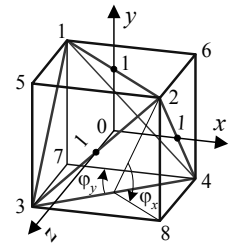


Рис. 1.116

□ **Гексаэдр** ($n=4$, $n_r=6$, $n_p=12$, $n_b=8$) или в обиходе куб (см. рис. 1.114,б) прост в построении модели (рис. 1.116):

$$P6 = \begin{bmatrix} -1 & 1 & -1 & 1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & -1 & 1 & -1 & -1 & 1 \end{bmatrix}^T, G6 = \begin{bmatrix} 1 & 1 & 1 & 2 & 2 & 3 \\ 5 & 7 & 6 & 8 & 5 & 7 \\ 2 & 3 & 4 & 4 & 3 & 4 \\ 6 & 5 & 7 & 6 & 8 & 8 \\ 1 & 1 & 1 & 2 & 2 & 3 \end{bmatrix}^T, d=2, R_c = \sqrt{3}, d_k=2.$$

□ **Октаэдр** ($n=3$, $n_r=8$, $n_p=12$, $n_b=6$), изображённый на рис. 1.114,в, также просто строится на основе куба (рис. 1.117) и имеет параметры

$$P8 = \begin{bmatrix} 0 & 1 & 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 & -1 & 0 \end{bmatrix}^T, G8 = \begin{bmatrix} 1 & 1 & 1 & 1 & 2 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 & 3 & 6 & 4 & 5 \\ 5 & 2 & 3 & 4 & 6 & 5 & 6 & 6 \\ 1 & 1 & 1 & 1 & 2 & 2 & 3 & 4 \end{bmatrix}^T,$$

$$d = \sqrt{2}, R_c = 1, d_k = 2.$$

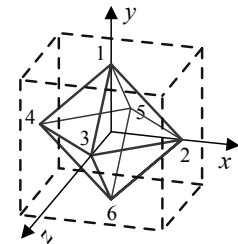


Рис. 1.117

□ **Икосаэдр** ($n=3$, $n_r=20$, $n_p=30$, $n_b=12$), показанный на рис. 1.114,г, конструируется по следующему алгоритму [1].

Шаг 1. В двух окружностях горизонтальных сечений цилиндра единичного радиуса $x^2 + z^2 = 1$ строятся правильные пятиугольники 1-3-5-7-9-1 и 2-4-6-8-10-2 (рис. 1.118,а) с координатами вершин

$$x_i = \cos(36^\circ i), \quad z_i = \sin(36^\circ i), \quad y_i = (-1)^{i-1} h \quad \forall i = \overline{1, 10}, \quad h = 0.5.$$

Шаг 2. Соединяя вершины 1-2-3-4-5-6-7-8-9-10-1, получим пояс из десяти средних треугольных граней икосаэдра с длиной ребер

$$d = |p_1 - p_9| = 2z_1 = \sqrt{(5 - \sqrt{5})/2}.$$

Из условия получения равно-
сторонних треугольников

$$\begin{aligned} |p_1 - p_{10}| &= \sqrt{4h^2 + z_1^2 + (1 - x_1)^2} = \\ &= \sqrt{4h^2 + (3 - \sqrt{5})/2} = d \end{aligned}$$

получим значение $h = 0.5$.

Шаг 3. Вычислив радиус описанной сферы $R_c = |p_{10}| = \sqrt{x_{10}^2 + h^2} = \sqrt{5}/2$, построим две вершины $p_{11} = [0 \ R_c \ 0]$ и $p_{12} = [0 \ -R_c \ 0]$. Соединяя их с вершинами $p_1 \div p_{10}$ (рис. 1.118,б), получим пять верхних и нижних граней икосаэдра. $\square$

В результате конструирования сформировались следующие параметры кар-
касной модели икосаэдра:

$$P20 = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 & x_9 & x_{10} & 0 & 0 \\ h & -h & h & -h & h & -h & h & -h & h & -h & R_c & -R_c \\ z_1 & z_2 & z_3 & z_4 & z_5 & z_6 & z_7 & z_8 & z_9 & z_{10} & 0 & 0 \end{bmatrix}^T,$$

$$G20 = \begin{bmatrix} 1 & 3 & 5 & 7 & 9 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 2 & 4 & 6 & 8 & 10 \\ 11 & 11 & 11 & 11 & 11 & 3 & 3 & 5 & 5 & 7 & 7 & 9 & 9 & 1 & 1 & 12 & 12 & 12 & 12 & 12 \\ 3 & 5 & 7 & 9 & 1 & 2 & 4 & 4 & 6 & 6 & 8 & 8 & 10 & 10 & 2 & 10 & 2 & 4 & 6 & 8 \\ 1 & 3 & 5 & 7 & 9 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 2 & 4 & 6 & 8 & 10 \end{bmatrix}^T,$$

$$h = 0.5, \quad R_c = \sqrt{5}/2, \quad d = \sqrt{(5 - \sqrt{5})/2}, \quad d_k = \sqrt{(5 + \sqrt{5})/2}.$$

$\square$ **Додекаэдр** ($n=5$, $n_r=12$, $n_p=30$, $n_b=20$) — наиболее сложное в конструировании платоново тело (см. рис. 1.114,д). Оптимальный метод его построения заключается в создании модели икосаэдра и соединении центров его смежных граней ребрами (рис. 1.119). Получаем формулу расчёта строк $P12^{(i)}$ матрицы координат вершин додекаэдра на основе матриц икосаэдра $P20$ и $G20$:

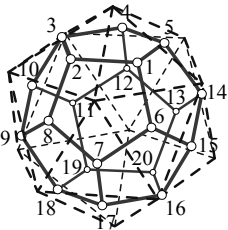


Рис. 1.119

$$P12^{(i)} = \frac{1}{3} \sum_{k=1}^3 P20^{(G20_{ik})} \quad \forall i = \overline{1, 20}.$$

Матрица граней додекаэдра и остальные его параметры равны

$$G_{12} = \begin{bmatrix} 1 & 1 & 2 & 3 & 4 & 5 & 7 & 9 & 11 & 13 & 15 & 16 \\ 5 & 2 & 3 & 4 & 5 & 1 & 8 & 10 & 12 & 14 & 6 & 17 \\ 4 & 8 & 10 & 12 & 14 & 6 & 9 & 11 & 13 & 15 & 7 & 18 \\ 3 & 7 & 9 & 11 & 13 & 15 & 18 & 19 & 20 & 16 & 17 & 19 \\ 2 & 6 & 8 & 10 & 12 & 14 & 17 & 18 & 19 & 20 & 16 & 20 \\ 1 & 1 & 2 & 3 & 4 & 5 & 7 & 9 & 11 & 13 & 15 & 16 \end{bmatrix}^T, \quad \begin{aligned} R_c &= \sqrt{(2\sqrt{5}+5)/12}, \\ d &= \sqrt{(5+\sqrt{5})/18}, \\ d_k &= \sqrt{(25+11\sqrt{5})/18}. \end{aligned}$$

Сфероидные и звёздные полиэдры

Путём построения дополнительных надстроек на гранях платоновых тел создаются различные полиэдры сфероидной и звёздной форм. В первой группе объектов все вершины полигонов надстроек лежат на описанной сфере; во второй группе надстройки над гранями полиэдра имеют форму объёмных лучей.

□ Для получения **сфероидного полиэдра** на платоновом теле с параметрами $\{n, n_B, n_T, P, G, d, R_c\}$ каждая из n_T граней (например, грань $p_1 p_2 \dots p_n p_1$ на рис. 1.120, а) заменяется надстройкой из $n+1$ -й грани. Вершины центральной грани $p_m \div p_{m+n-1}$, где $m > n_B$, лежат на прямых, соединяющих центр полиэдра с серединами рёбер грани $\{p_1 p_2 \dots p_n p_1\}$, на расстоянии R_c от центра:

$$p_m = (\overline{p_1 + p_2}) \cdot R_c, \quad p_{m+1} = (\overline{p_2 + p_3}) \cdot R_c, \quad \dots, \quad p_{m+n-1} = (\overline{p_n + p_1}) \cdot R_c.$$

Боковые грани $p_1 p_m p_{m+n-1} p_1$, $p_2 p_{m+1} p_m p_2$, $\dots$, $p_n p_{m+n-1} p_{m+n-2} p_n$ (равнобедренные треугольники) соединяются с соседними надстройками. Замена одной старой n -угольной грани на $n+1$ новую грань выполняется так:

- $m = n_B + 1$, $n_B = n_B + n$, а в координатную матрицу P добавляется n строк координат новых вершин $p_m \div p_{m+n-1}$;
- $n_T = n_T + n$, а из группы G удаляется список заменяемой грани и вставляется $n+1$ список новых граней.

Совершив преобразования всех граней исходного полиэдра, получим модель сфероидного полиэдра, вершины которого лежат на поверхности описан-

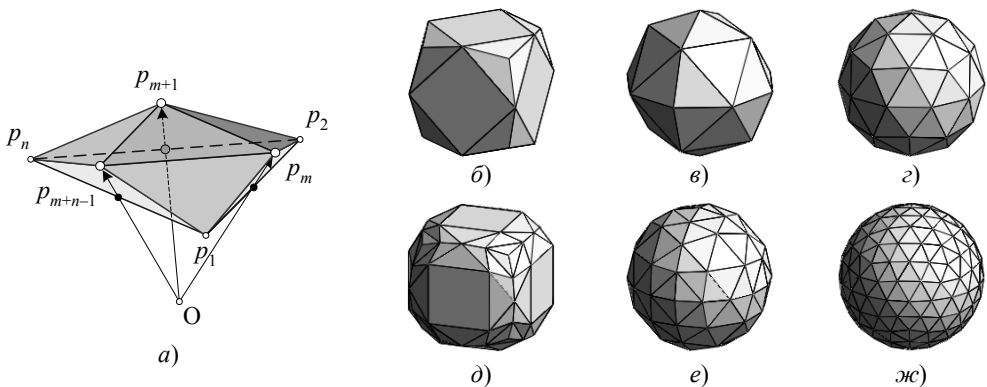


Рис. 1.120

ной сферы. На рис. 1.120 изображены объекты этого класса, построенные на гексаэдре (б), октаэдре (в) и икосаэдре (г). Последний полиэдр *близок* к правильному, но таковым не является: в его вершинах соединено разное число треугольников, что отчётливо видно по неравномерности каркасной сетки.

Возможно итерационное применение описанного алгоритма к граням выпуклого полиэдра. На рис. 1.120 показан результат двукратного надстраивания граней на гексаэдре (д), октаэдре (е) и икосаэдре (ж). При задании исходного платонова тела с треугольными гранями (тетраэдр, октаэдр и икосаэдр) решается задача *триангуляции* поверхности сферы — разбиения её на треугольники. Наибольшая однородность и близость граней сфероидного полиэдра к правильным треугольникам достигается при выборе в качестве базовой фигуры *икосаэдра*, что хорошо видно из рис. 1.120,г,ж.

□ **Звёздный полиэдр** на платоновом теле конструируется путём надстройки прямой пирамиды над каждой его гранью. Например, у грани $p_1p_2 \dots p_n p_1$ на рис. 1.121,а вершина пирамиды находится в точке

$$P_m = \alpha s, \text{ где } s = \frac{1}{n} \sum_{i=1}^n p_i.$$

Коэффициент α регулирует высоту луча в единицах расстояния от центра полиэдра до его грани. При $\alpha < 1$ лучи направлены внутрь объекта, а при $\alpha > 1$ — наружу. Для получения вершины p_m на поверхности описанной сферы нужно задать значение $\alpha = |p_1|/|s| > 1$. Это ещё один метод триангуляции сферы, однако, получаемые при этом сфероидные объекты состоят из тупоугольных треугольников и выглядят некрасиво, в чём читатель может убедиться самостоятельно.

Замена одной старой n -угольной грани на n новых треугольных граней алгоритмически выполняется следующим образом:

- $m = ++n_{\text{в}}$, а в матрицу P добавляется строка координат новой точки p_m ;
- $n_{\text{г}} = n_{\text{г}} + n - 1$, а в группу G вместо списка вершин заменяемой грани вставляется n списков новых граней.

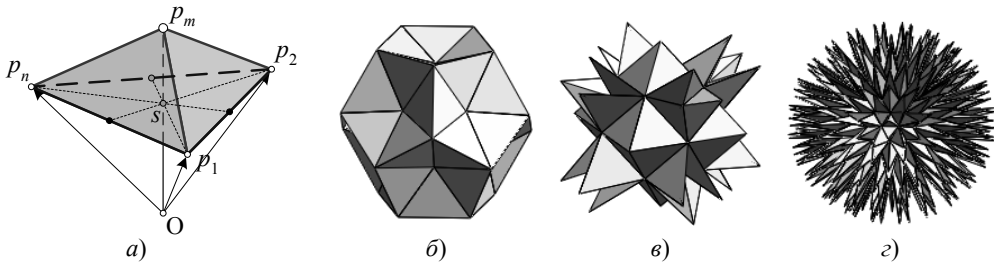


Рис. 1.121

Выполнив аналогичные замены всех граней полиэдра, получим модель звёздного полиэдра. На рис. 1.121 изображены объекты данного класса, один из которых (б) построен на додекаэдре с коэффициентом $\alpha = 0.75$, а два других (в, г) — на сфероидных объектах при $\alpha = 2$.

1.5.5. Тесты свойств графических элементов в пространстве

Рассмотрим тесты, основанные на *свойстве сепарабельности плоскости* $f(p)=0$ разделять пространство на два полупространства с противоположными знаками функции $f(p)$.

□ **Тест расположения точки p относительно плоскости $f(p)=0$** проверяет знак числа $f(p)$, для вычисления которого в зависимости от способа задания плоскости в п. 1.5.1 используется соответствующая неявная функция nfF , nfN , $nfVW$, $nf3$ или nfP , определённая в (1.108).

Все точки q , находящиеся с внешней (нормальной) стороны плоскости, дают значения каждой из этих функций $nf* > 0$. Для доказательства данного факта построим к плоскости $\{o, V, W\}$ вектор нормали $N = V \times W$, выберем точку $q = o + Vt + W\tau + \lambda N$ с параметром $\lambda > 0$ и найдём в ней знак функции (1.108б):

$$nfN(o, N, q) = (Vt + W\tau + \lambda N) \circ N = (V \circ N)t + (W \circ N)\tau + \lambda(N \circ N) = 0 + 0 + \lambda|N|^2 > 0.$$

Параметры $\{t, \tau, \lambda\}$ есть координаты точки q в системе координат $\{o, V, W, N\}$. Так как значение $f(q)$ не зависит от координат t и τ вдоль векторов V и W , то все **поверхности постоянных уровней** $f(p) = \text{const}$ проходят параллельно плоскости $f(p) = 0$ и также являются плоскостями (рис. 1.122). Значение $|\lambda|$ пропорционально расстоянию $d = |\lambda| |N|$ от точки до плоскости, откуда следуют формулы вычисления последнего, обобщающие (1.119):

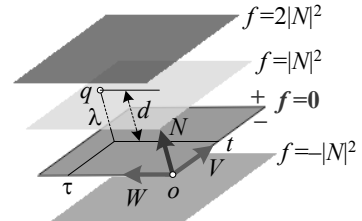


Рис. 1.122

$$d = \frac{|nf(F, q)|}{A^2 + B^2 + C^2} = \frac{|nfN(o, N, q)|}{|N|} = \frac{|nfVW(o, V, W, q)|}{|V \times W|} = \frac{|nf3(a, b, c, q)|}{|(b-a) \times (c-b)|}.$$

Ввиду простоты вычисления функция $f(p)$, реализуемая одной из функций $nf*$ в (1.108), идеально подходит для сортировки точек по сторонам плоскости. Тест применяется в алгоритмах, связанных с обработкой точек, вершин полигонов и во многих других 3d-задачах вычислительной геометрии.

□ **Тест пересечения плоскости $f(p)=0$ и полиэдра $H = \{P, G\}$** со списком вершин $P = \{p_1, \dots, p_n\}$ идентичен 2d-аналогу (1.38) с точностью до имени функции и размерности пространства:

$$cross(f, P) = \{\exists 1 \leq j < i \leq n: f(p_i)f(p_j) < 0\}. \quad (1.146)$$

После замены имени функции и выражения $size(P)-1$ на $size(P)$ блок-схема теста $cross3$ совпадает со схемой на рис. 1.14,в и потому здесь не приводится.

Плоскость пересекает полиэдр, если существует хотя бы одна пара вершин $\{p_i, p_j\}$, лежащих по разные стороны от плоскости. Свойство сепарабельности плоскости позволяет обнаружить вершины полиэдра p_i и p_j по разные стороны от пересекающей его плоскости по разным знакам значений $f(p_i)$

и $f(p_j)$. Если же не найдётся ни одной пары разносторонних вершин, то пересечение полиэдра с плоскостью отсутствует. Функция (1.146) возвращает 1 как признак пересечения. В отсутствие оного тест возвращает -1 , а при касании полиэдра с плоскостью вершиной, ребром или гранью — значение 0.

□ **Тест выпуклости полиэдра** $H=\{P, G\}$, заданного списком вершин $P=\{p_1, \dots, p_n\}$ и группой списков граней $G=\{G_1, \dots, G_m\}$:

$$\text{conv3}(P, G) = \text{sgn}(\text{nfP}(\text{side}(P, G, i), p_j)) = \text{const} \quad \forall i = \overline{1, m}, j = \overline{1, n}: j \notin G_i. \quad (1.147)$$

У выпуклого полиэдра не существует ни одной пары вершин, лежащих по разные стороны от любой не включающей их грани (рис. 1.123,а). Напротив, в невыпуклом полигоне всегда найдутся грань G_i и пара вершин $\{p_j, p_k\}$, находящихся по разные стороны от плоскости этой грани (рис. 1.123,б).

Свойство сепарабельности плоскости позволяет обнаружить у полиэдра вогнутую грань Q по противоположным знакам чисел $\text{nfP}(Q, p_j)$ и $\text{nfP}(Q, p_k)$ в вершинах $p_j \notin Q$ и $p_k \notin Q$, не входящих в состав этой грани.

Наилучший, по мнению автора, **алгоритм проверки выпуклости полиэдра** $\text{conv3}(P, G)$ представлен блок-схемой на рис. 1.123,в и использует два флажка, показывающих наличие у полиэдра вершин внутри (флажок l) и снаружи (флажок r) плоскости i -й грани $Q = \text{side}(P, G, i)$. В начале внешнего цикла $i = \overline{1, m}$ обхода граней G_i оба флажка обнуляем. Во внутреннем цикле обхода вершин p_j вычисляем значение $f = \text{nfP}(Q, p_j)$. Вершину с $f=0$, лежащую в плос-

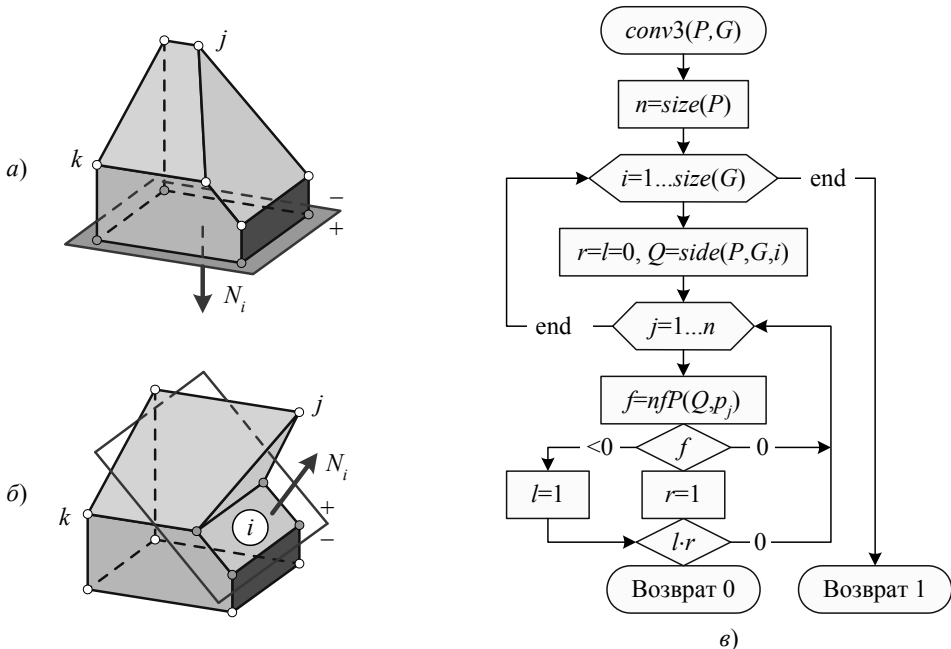


Рис. 1.123

кости грани, пропускаем, а при $f \neq 0$ инкрементируем соответствующий флажок: l при $f < 0$ либо r при $f > 0$. Тут же по условию $l \cdot r \neq 0$ проверяем наличие *любой* пары вершин по разные стороны от плоскости грани. Если оно выполняется, то тест немедленно завершается, возвращая 0 как признак невыпуклости полиэдра. После окончания внешнего цикла перебора вершин алгоритм *conv3* возвращает значение 1 как признак выпуклости полиэдра.

□ **Тест минимальности полиэдра** $H = \{P, G\}$ со списком вершин $P = \{p_1, \dots, p_n\}$ и группой списков граней $G = \{G_1, \dots, G_m\}$, блок-схема которого построена на рис. 1.124, позволяет обнаружить:

- **кратные** вершины $p_i = p_j$ при $i \neq j$, возникающие в неоптимизированных алгоритмах обработки полиэдров либо по недосмотру при ручном формировании их моделей. Существование таких вершин в списке P ничем не оправдано из-за лишней путаницы и дополнительных затрат машинной памяти;

- **лишние** вершины p_i , номеров которых $g_{jk} = i$ нет ни в одном из списков $G_j = \{g_{j1}, \dots, g_{jn_j}, g_{j1}\} \forall j = 1, m$. Такие вершины могут оказаться в списке P , содержащем координаты вершин не только полиэдра H , но и других объектов сцены, а с помощью теста данный факт можно обнаружить.

Функция *min3_test* возвращает 1 для минимального полиэдра и 0 — для неминимального, содержащего кратные или лишние вершины. Тест легко модифицировать так, чтобы он возвращал разные признаки неминимальности полиэдра, например, 0 при обнаружении кратных и -1 — лишних вершин.

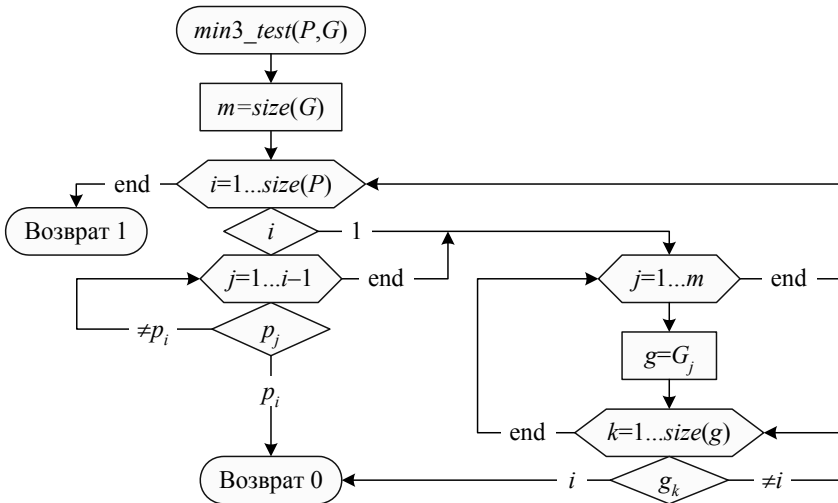


Рис. 1.124

При обнаружении неминимальности полиэдра следует удалить из списка P повинные в этом вершины. Эту работу выполняет приведённый на рис. 1.125 алгоритм *min_hedr(P, G)* **минимизации топологии полиэдра**, заданного списками вершин $P = \{p_1, \dots, p_n\}$ и граней $G = \{G_1, \dots, G_m\}$.

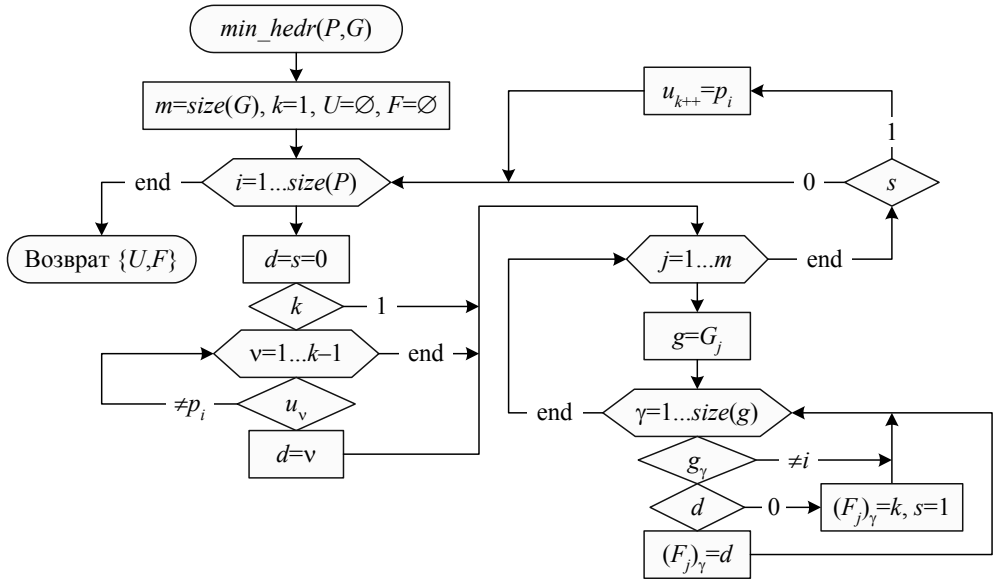


Рис. 1.125

Работа алгоритма начинается с инициализации списка вершин $U=\emptyset$, группы списков граней $F=\emptyset$ и начального индекса $k=1$ записи **простых** вершин в список U . Затем начинается цикл сканирования вершин $p_i \forall i=\overline{1, n}$, в котором при $k>1$ проверяется их совпадение с уже накопленными в U точками $u_v \forall v=\overline{1, k-1}$. Если совпадение найдено, то в заранее сброшенной переменной d запоминается индекс простой вершины u_v , которой кратна вершина p_i .

Далее производится сканирование списков граней $g=G_j \forall j=\overline{1, m}$ в поиске всех индексов $g_\gamma=i$, где $\gamma \in [1, \text{size}(g)]$. При каждом успехе поиска γ -у элементу списка F_j присваивается одно из следующих значений:

- d при $d>0$, что означает удаление из списков граней кратных вершин;
- k при $d=0$ с установкой флажка $s=1$, означающей наличие в списках граней G_j одной или нескольких ссылок на простую вершину p_i .

После просмотра всех списков граней G_j при условии $s=1$ добавляем p_i в список U и инкрементируем счётчик $k++$. При данном порядке формирования новых списков граней F_j все лишние вершины p_i , на которые нет ни одной ссылки $g_\gamma=i$, остаются с флажками $s=0$, что исключает их попадание в U .

По окончании перебора вершин p_i алгоритм возвращает минимизированную модель полиэдра $\{U, F\}$.

1.5.6. Тесты локализации объектов в пространстве

В программах 3d-графики широко применяются тесты локализации одних ограниченных в пространстве объектов (точек, отрезков, полигонов, полиэдров) относительно других объектов (полигонов, полиэдров). Они анализируют вза-

имное расположение объектов в пространстве сцены, позволяя управлять работой программы по результату тестирования и избежать трудоёмких расчётов.

□ Сначала рассмотрим тесты локализации графических объектов, **целиком расположенных в произвольной плоскости** $\{o, N\}$. Это могут быть, например, точки пересечения световых лучей с плоскостью грани полиэдра в задаче построения на его поверхности тени некоторого объекта.

Возможны **два подхода к работе с компланарными объектами**.

1. Первый подход заключается в укладывании плоскости $\{o, N\}$ вместе с находящимися в ней объектами на координатную плоскость xu и использовании 2d-тестов и алгоритмов из п.п. 1.1.3, 1.1.4. Оставим плоскость и все объекты сцены в покое, а систему xuz подвергнем пассивному преобразованию перевода в положение $x'y'z'$ с началом в точке o и направлением оси z' по вектору нормали N (рис. 1.126). Различные алгоритмы расчёта матрицы такого преобразования СК будут получены в разд. 3.1. Они отличаются направлением осей x' и y' в плоскости $\{o, N\}$. Лучший, по мнению автора, алгоритм реализован в функции *MP2*, с помощью которой вычисляется 4×4 -матрица

$$C_K = MP2(o, \bar{N}, O_3, z^0).$$

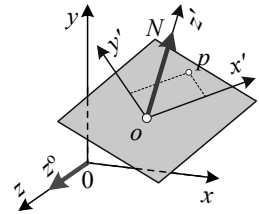


Рис. 1.126

Далее координаты любой точки p в новой системе координат $x'y'z'$ находятся линейным преобразованием $\tilde{p}' = \tilde{p}C_K$, при этом для точки $p \in \{o, N\}$ автоматически получится $z' = 0$. Также одной операцией матричного умножения $\tilde{P}' = \tilde{P}C_K$ выполняется **групповое преобразование** сразу всех точек сцены, расширенные координаты которых собраны по (1.142) в единую матрицу $\tilde{P}$.

Поскольку 2d-тесты и расчётные задачи работают на плоскости, а реализующие их формулы могут быть невычислимы с 3d-данными (например, определители **неквадратных** 2×3 -матриц в (1.19в,г), (1.24б), (1.29), (1.32)), то все компланарные объекты должны быть спроецированы из пространства $x'y'z'$ на плоскость $x'y'$ с сохранением координат x' и y' с помощью матрицы $E_{3,2}$:

$$[x' \ y' \ z']E_{3,2} = [x' \ y']. \quad (1.148a)$$

Для возвращения в исходное пространство R^3 с добавлением в точку $[x' \ y']$ координаты $z' = 0$ используется преобразование с матрицей $E_{2,3}$:

$$[x' \ y']E_{2,3} = [x' \ y' \ 0]. \quad (1.148б)$$

2. Второй подход, не требующий преобразования координат, состоит в создании тестов, работающих непосредственно с плоскостью $\{o, N\}$ и расположенными на ней объектами в 3d-пространстве. Для этого нужно тщательно проверить все формулы из разд. 1.1 – 1.4 на пригодность и вычислимость в пространстве, а в отсутствие оных попытаться создать их 3d-варианты. Например, все варианты НФ прямой $f(p)$ в (1.19) непригодны в пространстве R^3 , поскольку в

нём отсутствует само понятие неявной функции прямой линии. Мало того: в каждой точке 3d-прямой число нормалей бесконечно, нет ни «правой», ни «левой» сторон от направляющего вектора прямой, а определители **неквадратных** матриц невычислимы. Всё это делает невозможным создание 3d-вариантов тестов, плоские аналоги которых используют неявные функции прямой.

Рассмотрим теперь введенную в (1.32) функцию $ang(V, W)$, в которой определитель матрицы, составленной из 2d-векторов V и W , даёт **знак** угла между ними. Определитель **неквадратной** матрицы из двух 3d-векторов становится не вычислимым, да и смысл знака 3d-угла не однозначен: с разных сторон плоскости $\{o, V, W\}$ вращение от вектора V к вектору W выглядит в противоположных направлениях. Для определённости будем отслеживать это вращение, глядя со стороны **заданной** нормали плоскости $\{o, N\}$. Вычислим векторное произведение $H = V \times W$ и, оценив взаимные направления параллельных векторов N и H , присвоим модулю угла $\delta = \arccos(\bar{V} \circ \bar{W}) \in [0, \pi]$ знак $\text{sgn}(N \circ H)$ (рис. 1.127):

- положительный при сонаправленности $N \uparrow \uparrow H$ (а);
- отрицательный при антинаправленности $N \uparrow \downarrow H$ (б);
- нулевой при $|H| = 0$, что соответствует параллельности векторов $V \parallel W$.

С учётом сказанного на основе 2d-функции rad_test (см. рис. 1.22, в) разработан плоско-объёмный **радианный тест** $rad23_test(P, q)$ **локализации точки** $q \in \mathbb{R}^3$ **относительно плоского 3d-полигона** $P = p_1 \dots p_n p_1 \in \mathbb{R}^3$ (рис. 1.127, в). Вектор нормали N к плоскости полигона P , необходимый для определения знака угла $\delta = \arccos(W \circ V)$, не передаётся через аргументы функции, а вычисляется внутри неё с помощью функции (1.144) как $N = \text{norm}(P)$.

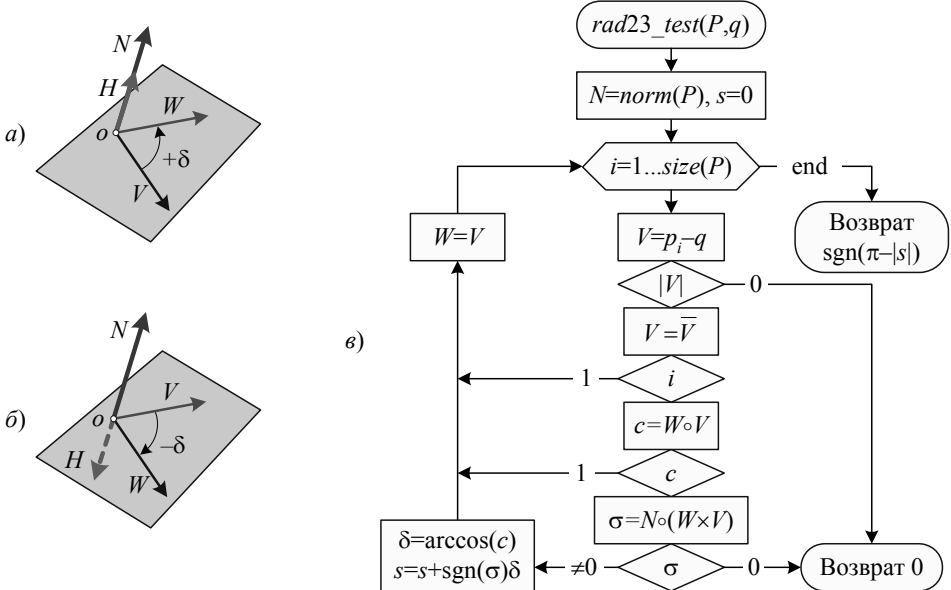


Рис. 1.127

Как и все тесты локализации, функция *rad23_test* традиционно возвращает признак, равный 1 для внешнего, -1 для внутреннего и 0 для граничного (на рёбрах или в вершинах) расположения точки q относительно полигона P .

Тест будет использован в расчётах пересечения графических объектов в пространстве. Он позволяет напрямую, без преобразования системы координат, по условиям $\{\theta > 0\} \wedge \{rad23_test(P, c) \leq 0\}$ обнаружить **пересечение луча** $q + U\theta$ с **произвольным полигоном** $P \in \mathbb{R}^3$ в вычисленной по (1.130) точке c .

Составим крайне полезный алгоритм $\boxed{cross_side(ab, P, t, c)}$ расчёта линейного параметра t и точки c пересечения отрезка ab с произвольно расположенным в пространстве плоским полигоном $P = p_1 p_2 \dots p_n p_1$ (рис. 1.128, а).

☞ $cross_side(ab, P, t, c)$

$\{U = b - a, \quad N = norm(P)\};$

если $U \circ N = 0$, то **возврат** -1;

// тест параллельности (1.128)

$t = ((p_1 - a) \circ N) / (U \circ N), \quad c = a + Ut;$

// параметр и точка пересечения

возврат $\{0 \leq t \leq 1\} \wedge \{rad23_test(P, c) \leq 0\};$

// тест пересечения ab с P

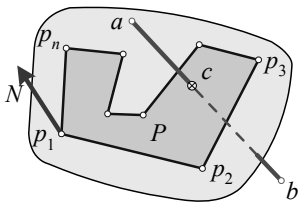
}

☐

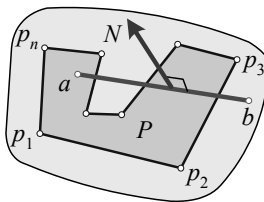
Функция возвращает через своё имя **признак расположения отрезка**:

- -1 — отрезок **параллелен** плоскости полигона и пересечения нет либо весь лежит в ней и точка пересечения не определена (рис. 1.128, б);
- 0 — **непараллельный** отрезок **не пересекается** с полигоном (рис. 1.128, в);
- 1 — **непараллельные** отрезок и полигон **касаются** либо **пересекаются**.

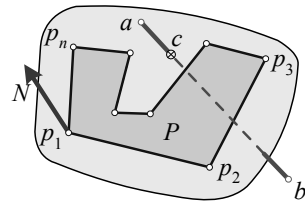
В двух последних случаях через аргументы функции возвращаются линейный параметр t и точка пересечения c .



а)



б)



в)

Рис. 1.128

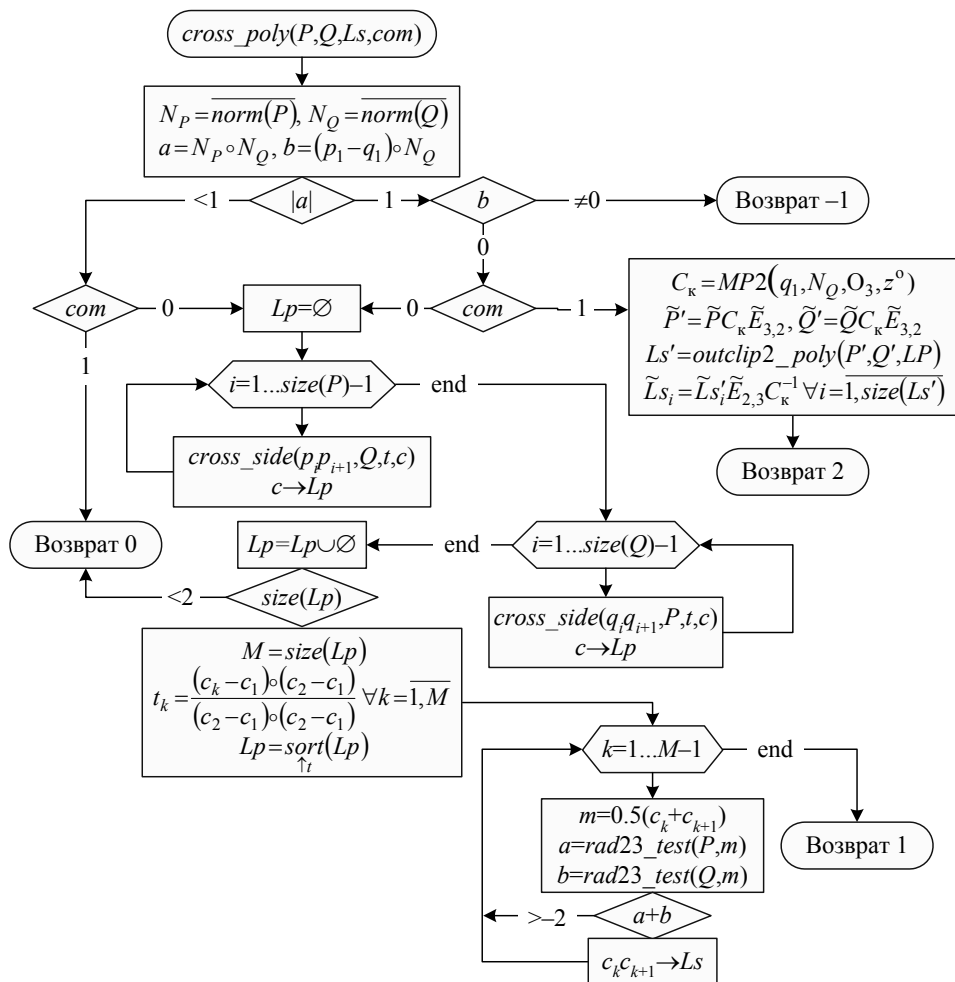
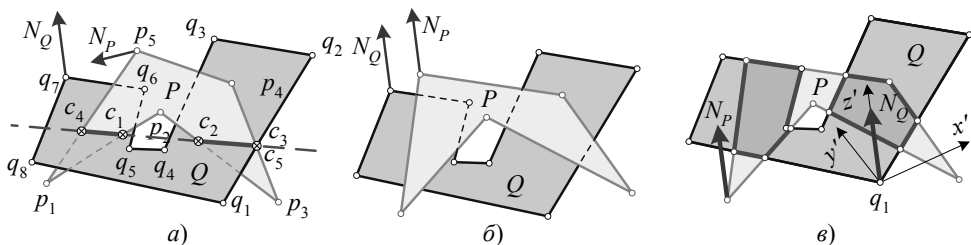
Ещё одно применение *rad23_test* находит в расчёте пересечения двух произвольно расположенных полигонов P и Q . Требуется найти все отрезки, по которым они пересекаются (рис. 1.129, а).

Составим алгоритм $\boxed{cross_poly(P, Q, Ls, com)}$ расчёта отрезков пересечения произвольных плоских полигонов $P = p_1 p_2 \dots p_n p_1$ и $Q = q_1 q_2 \dots q_m q_1$. Аргумент $com \in \{0, 1\}$ управляет обработкой полигонов с учётом их компланарности:

- 0 — рассчитывается пересечение только **некомпланарных** полигонов;
- 1 — обрабатываются только **компланарные** полигоны.

Функция возвращает в своём имени **признак расположения** полигонов:

- -1 — плоскости **непересекающихся** полигонов **параллельны**, но не компланарны (рис. 1.129,б);
- 0 — полигоны лежат в **непараллельных** плоскостях, но друг с другом **не пересекаются**;



з)

Рис. 1.129

- 1 — полигоны в **непараллельных** плоскостях **пересекаются** по коллинеарным отрезкам, добавленным в возвращаемый список Ls (рис. 1.129,а);
- 2 — плоскости полигонов **компланарны**, а в список Ls добавляются рёбра полигонов логической операции пересечения $P \cap Q$ (рис. 1.129,в).

Алгоритм с блок-схемой на рис. 1.129,г работает в следующем порядке.

Шаг 1. Вычислив нормали к полигонам $N_P = \overline{\text{norm}(P)}$ и $N_Q = \overline{\text{norm}(Q)}$, проверяем взаимную ориентацию плоскостей P и Q :

- если $\{|N_P \circ N_Q| = 1\} \wedge \{(p_1 - q_1) \circ N_Q \neq 0\}$, то **возврат** -1;
- если $\{|N_P \circ N_Q| = 1\} \wedge \{(p_1 - q_1) \circ N_Q = 0\} \wedge \{com = 1\}$, то рассчитываем матрицу $C_K = MP2(q_1, N_Q, O_3, z^0)$ перевода координатной плоскости $x'y'$ на плоскость полигонов и выполняем внешнее отсечение полигона P отсекателем Q :

$$\{P, Q\} \xrightarrow{C_K} \{P', Q'\}, \text{ outclip2_poly}(P', Q', LP) \xrightarrow{C_K^{-1}} Ls,$$

после чего следует **возврат** 2;

- если $com = 1$, то **возврат** 0.

Шаг 2. Инициализируем пустой список точек $Lp = \emptyset$ и добавляем в него точки пересечения отрезков $p_i p_{i+1}$ с полигоном Q :

$$\text{cross_side}(p_i p_{i+1}, Q, t, c) > 0 \Rightarrow c \rightarrow Lp \quad \forall i = \overline{1, \text{size}(P) - 1}.$$

Шаг 3. Добавляем в Lp точки пересечения отрезков $q_i q_{i+1}$ с полигоном P :

$$\text{cross_side}(q_i q_{i+1}, P, t, c) > 0 \Rightarrow c \rightarrow Lp \quad \forall i = \overline{1, \text{size}(Q) - 1}.$$

Минимизируем список $Lp = Lp \cup \emptyset$. Если $\text{size}(Lp) < 2$, то **возврат** 0.

Шаг 4. Сортируем **некратные** точки списка $Lp = \{c_1, \dots, c_M\}$ вдоль прямой пересечения плоскостей полигонов:

$$t_k = \frac{(c_k - c_1) \circ (c_2 - c_1)}{(c_2 - c_1) \circ (c_2 - c_1)} \quad \forall k = \overline{1, M} \Rightarrow Lp = \underset{\uparrow t}{\text{sort}}(Lp).$$

Шаг 5. Добавляем в список Ls отрезки $c_k c_{k+1}$, обнаруженные внутри **обоих** полигонов тестом локализации средних точек $m_k = 0.5(c_k + c_{k+1})$:

$$\text{rad23_test}(P, m_k) + \text{rad23_test}(Q, m_k) = -2 \Rightarrow c_k c_{k+1} \rightarrow Ls \quad \forall k \in [1, M - 1]$$

и **возврат** 1. $\square$

Переходим к построению тестов взаимной ориентации графических объектов, **произвольно** расположенных в пространстве. Начнём с группы **тестов локализации точки** q относительно **полиэдра** $H = \{P, G\}$, заданного списками вершин $P = \{p_1, \dots, p_n\}$ и граней $G = \{G_1, \dots, G_m\}$ (см. п. 1.5.4).

$\square$ **Выпуклый тест**, реализуемый функцией

$$\text{conv3_test}(P, G, q) = \begin{cases} -1, & \text{если } f_i < 0 \quad \forall i = \overline{1, m}, \\ 1, & \text{если } \exists i \in [1, m]: f_i > 0, \\ 0, & \text{если не } \pm 1, \end{cases} \quad (1.149)$$

аналогичен плоскому тесту (1.42) и определяет положение точки q относительно **выпуклого** полиэдра H с **внешними** нормальными граней по знакам неявной функции (1.108д) $f_i = nfP(Q_i, q)$ для граней $Q_i = \text{side}(P, G, i)$.

Все **внутренние** точки $q_{\text{вну}}$ лежат с внутренних сторон всех граней Q_i . Для любой **внешней** точки $q_{\text{вне}}$ всегда найдется пара граней Q_i и Q_j , относительно которых она расположена с разных сторон. Наконец, все точки, не идентифицированные как внутренние или внешние, считаются **граничными** точками $q_{\text{гр}}$, расположенными на гранях, рёбрах или в вершинах выпуклого полиэдра.

Блок-схема **алгоритма выпуклого теста** $\text{conv3_test}(P, G, q)$ приведена на рис. 1.130,а. В сравнении с плоским вариантом conv2_test (см. рис. 1.19,в) получено упрощение и ускорение работы этого 3d-теста благодаря изначально **внешней** ориентации нормалей к граням полиэдра.

В соответствии со знаком числа 1, возвращаемого тестом для внешних точек, назовём внешнее пространство полиэдра **положительным**. Внутреннее пространство будем считать **отрицательным** по знаку числа -1 , возвращаемого алгоритмом для всех точек этого пространства. Все точки на границе полиэдра (в вершинах, на рёбрах или гранях) дают нулевой результат тестирования.

Среди выпуклых полиэдров особое место занимают треугольные пирамиды и параллелепипеды. Из этих простых фигур часто складываются полиэдры сложной формы, в том числе и невыпуклые. Поочерёдная проверка позволяет определить принадлежность точки составной фигуре по её принадлежности любой из частей без использования сложных универсальных тестов.

Записав представление точки $q = o + Vt + W\tau + U\theta$ в **трёхмерной** косоугольной системе координат $\{o, V, W, U\}$ с началом в точке o и базисными векторами V, W и U , получим её координаты:

$$\begin{bmatrix} t & \tau & \theta \end{bmatrix} \begin{bmatrix} V \\ W \\ U \end{bmatrix} = q - o \Rightarrow \begin{bmatrix} t & \tau & \theta \end{bmatrix} = (q - o) \begin{bmatrix} V \\ W \\ U \end{bmatrix}^{-1}. \quad (1.150)$$

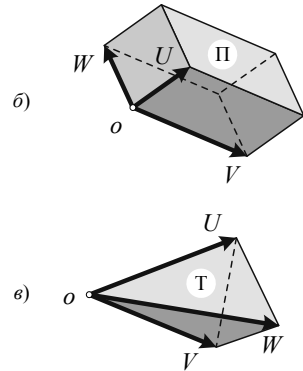
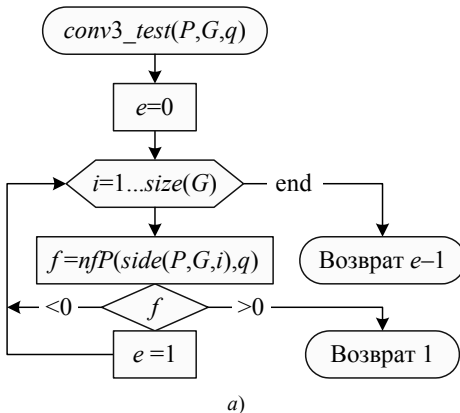


Рис. 1.130

Условие существования решения — невырожденность обращаемой матрицы, определитель которой не равен нулю при **линейной независимости базисных векторов** V, W и U , т. е. их некомпланарности.

Проведём из любой вершины параллелепипеда (рис. 1.130,б) либо призмы (рис. 1.130,в) базисные векторы V, W и U , после чего вычислим по (1.150) координаты тестируемой точки q . Элементарные соображения, аналогичные (1.44), подсказывают, что:

- точка q лежит внутри **параллелепипеда** Π при выполнении условий

$$\{0 < t < 1\} \wedge \{0 < \tau < 1\} \wedge \{0 < \theta < 1\}, \quad (1.151a)$$

на его границе при $\{0 \leq t \leq 1\} \wedge \{0 \leq \tau \leq 1\} \wedge \{0 \leq \theta \leq 1\}$ и снаружи при всех остальных значениях косоугольных координат $\{t < 0\} \vee \{t > 1\} \vee \{\tau < 0\} \vee \{\tau > 1\} \vee \{\theta < 0\} \vee \{\theta > 1\}$;

- точка q лежит внутри **треугольной призмы** T при выполнении условий

$$\{t > 0\} \wedge \{\tau > 0\} \wedge \{\theta > 0\} \wedge \{t + \tau + \theta < 1\}, \quad (1.151б)$$

на его границе при $\{t \geq 0\} \wedge \{\tau \geq 0\} \wedge \{\theta \geq 0\} \wedge \{t + \tau + \theta \leq 1\}$ и снаружи при всех остальных значениях координат $\{t < 0\} \vee \{\tau < 0\} \vee \{\theta < 0\} \vee \{t + \tau + \theta > 1\}$.

□ Габаритный тест

$$gab3_test(P, q) = \{x \notin [x_{\min}, x_{\max}]\} \vee \{y \notin [y_{\min}, y_{\max}]\} \vee \{z \notin [z_{\min}, z_{\max}]\} \quad (1.152)$$

определяет по возвращаемому значению $gab3_test=1$ **гарантированную непринадлежность точки** $q=[x \ y \ z]$ **полиэдру** $H=\{P, G\}$ при непопадании её координат в габаритную параллелепипедную оболочку $\boxed{H}$. Иной результат теста не даёт однозначного ответа о расположении точки и полиэдра.

□ **Лучевой 3d-тест локализации** $ray3_test(P, G, q)$, блок-схема которого приведена на рис. 1.131, аналогично 2d-варианту (см. рис. 1.26,з) основан на выпускании из точки q луча $p(\theta)=q+U\theta \ \forall \theta \geq 0$ и анализе чётности числа его пересечений с гранями полиэдра $H=\{P, G\}$. Ввиду неопределённости этого чис-

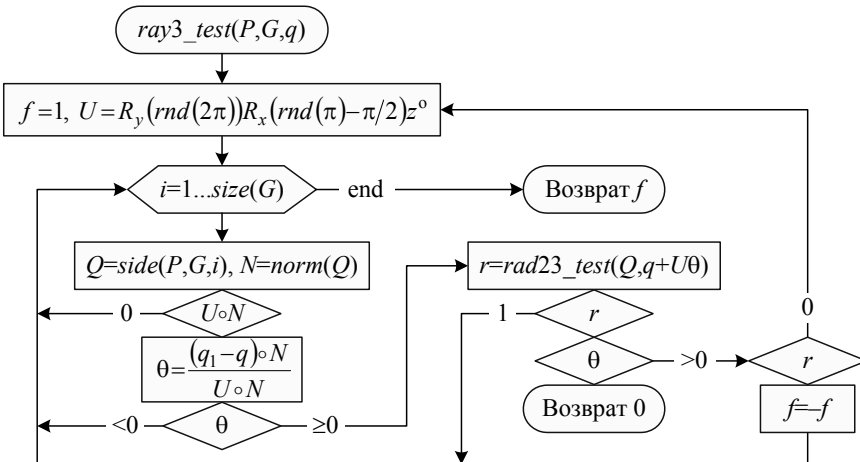


Рис. 1.131

ла в случаях касания луча с рёбрами или вершинами полиэдра организуем следующий порядок подсчёта пересечений:

- установим флажок пересечений $f=1$ и сгенерируем вектор U со *случайными* углами азимута $\varphi = \text{rnd}(2\pi)$ и места $\psi = \text{rnd}(\pi) - \pi/2$ (см. рис. 1.1,б);
- в цикле $i = \overline{1, \text{size}(G)}$ рассчитаем для i -й грани $Q = \text{side}(P, G, i) \equiv q_1 \dots q_n q_1$ нормаль $N = \text{norm}(Q)$ и по (1.130а) при $U \perp N$ параметр $\theta = (q_1 - q) \circ N / U \circ N$ и точку $c = q + U\theta$ пересечения луча с *не параллельной* ему плоскостью полигона;
- при $\theta \geq 0$ локализуем по числу $r \in \{-1, 0, 1\}$ точку c относительно границ полигона Q (как это лучше всего сделать — чуть ниже) и принимаем решение:
 - ◊ при $\{r \leq 0\} \wedge \{\theta = 0\}$ луч *начинается* внутри или на границе полигона, что означает *граничную* локализацию точки q и завершение теста с **возвратом** числа 0;
 - ◊ при $\{r = -1\} \wedge \{\theta > 0\}$ луч *пересекает* грань внутри её границ, что влечёт инвертирование флажка $f = -f$ и переход к следующей грани;
 - ◊ при $\{r = 0\} \wedge \{\theta > 0\}$ луч *касается* грани по ребру или вершине. Цикл перебора граней прекращается, заново устанавливается флажок $f=1$, генерируется новый случайный вектор U и тест проводится для нового луча.

Рассмотрим подробнее варианты реализации теста локализации точки c относительно компланарного с ней полигона $Q = q_1 q_2 \dots q_n q_1$:

- в двух частных случаях простейшей формы грани вычислим направляющие векторы её плоскости $V = q_2 - q_1$, $W = q_3 - q_2$ и по (1.43б) при $o = q_1$ найдём параметры t и τ , после чего:

- ◊ если $Q = q_1 q_2 q_3 q_4 q_1$ — *параллелограмм*, то в соответствии с (1.44а)

$$r = \text{if}(\{0 < t < 1\} \wedge \{0 < \tau < 1\}: -1, \text{if}(\{0 \leq t \leq 1\} \wedge \{0 \leq \tau \leq 1\}: 0, 1)); \quad (1.153а)$$

- ◊ если $Q = q_1 q_2 q_3 q_1$ — *треугольник*, то по (1.44б)

$$r = \text{if}(0 < \tau < t < 1: -1, \text{if}(0 \leq \tau \leq t \leq 1: 0, 1)); \quad (1.153б)$$

- в общем случае *произвольного* полигона Q , реализованном в блок-схеме на рис. 1.131, тест локализации выполняется разработанной выше плоско-объёмной функцией rad23_test (см. рис. 1.127,в):

$$r = \text{rad23_test}(Q, c). \quad (1.153в)$$

Вторая группа тестов предназначена для проверки расположения *полиэдра* $O = \{Q, F\}$ со списками вершин Q и граней F относительно *полиэдра* $H = \{P, G\}$ со списками вершин P и граней G . Поскольку модель полиэдра обобщает модели плоского полигона и отрезка, разработанные ниже алгоритмы могут быть модифицированы на эти частные случаи. Читателю предлагается сделать это самостоятельно в качестве упражнений.

□ Точечный габаритный тест (1.152) легко расширяется на случай полиэдра $O = \{Q, F\}$ с габаритными координатами

$$\begin{aligned} q_{x\min} &= \min\{q_{ix}\}, & q_{x\min} &= \min\{q_{ix}\}, & q_{z\min} &= \min\{q_{iz}\}, \\ q_{x\max} &= \max\{q_{ix}\}, & q_{y\max} &= \max\{q_{iy}\}, & q_{z\max} &= \max\{q_{iz}\}. \end{aligned} \quad (1.154а)$$

Габаритный тест локализации полиэдра О относительно полиэдра Н выполняется функцией

$$gab3_poly(P, Q) = \bigvee_{\omega \in \{x, y, z\}} (\{q_{\omega \max} < \omega_{\min}\} \vee \{q_{\omega \min} > \omega_{\max}\}), \quad (1.1546)$$

возвращающей 1 при **гарантированном непересечении** габаритных 3d-оболочек $\boxed{O}$ и $\boxed{H}$. Нулевой результат означает, что оболочка $\boxed{O}$ либо полностью лежит внутри оболочки $\boxed{H}$, либо с ней пересекается, и не может быть однозначно идентифицирована как внешняя либо внутренняя. Таким образом, **габаритный тест полностью задачу взаимной локализации полиэдров не решает**.

Нелишне будет ещё раз напомнить предназначение габаритных тестов — быстрое обнаружение заведомого **непересечения** графических объектов, которое можно подтвердить другими, но более сложными тестами.

□ Следующий **тест $coll3_test(P, G, Q, F)$ столкновения полиэдров** $H = \{P, G\}$ и $O = \{Q, F\}$ фиксирует это событие возвращением числа 1, найдя хотя бы одну точку с пересечения (рис. 1.132, а) либо касания (рис. 1.132, б) любой грани одного полиэдра с любым ребром другого. Блок-схема теста приведена на рис. 1.132, в. Тест имеет очень важное значение в моделировании движения

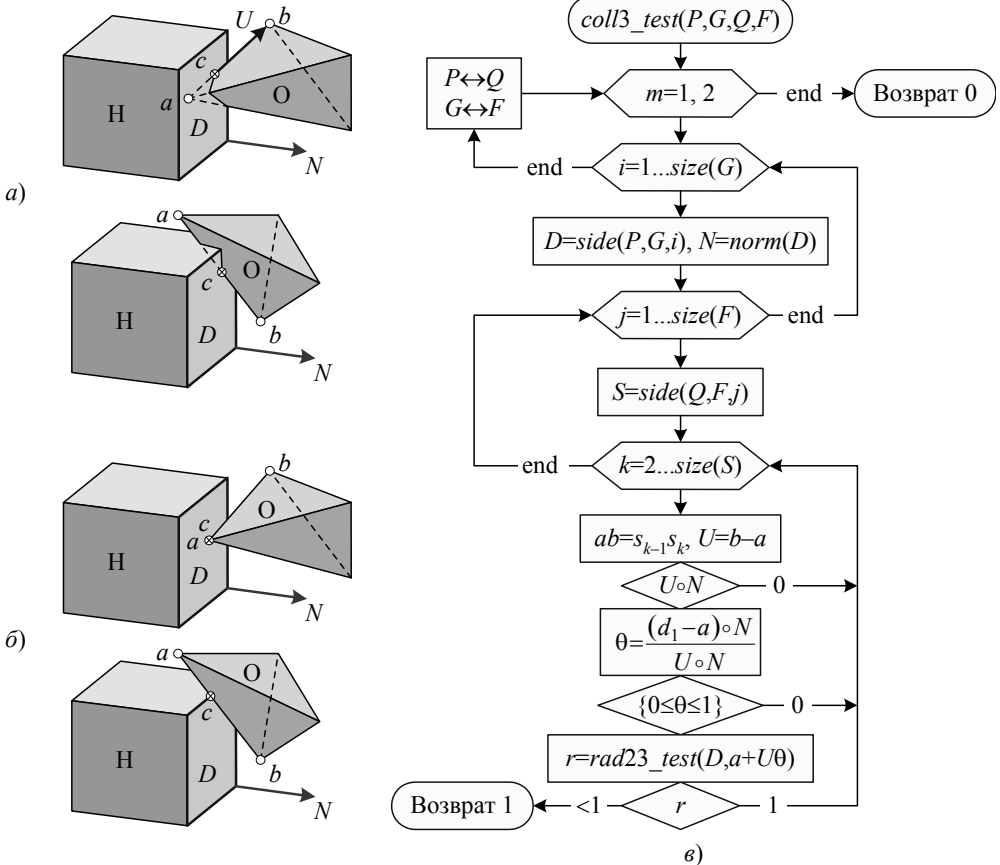


Рис. 1.132

твёрдых объектов, при столкновении которых нужно принять решение об их дальнейшем поведении: либо остановке обоих тел, либо упругом отскоке по законам сохранения импульса и энергии, либо какой-то другой реакции.

Алгоритм теста сначала перебирает каждую грань полиэдра H и каждое ребро полиэдра O , а затем наоборот, что реализовано в цикле $m=\overline{1,2}$, в котором после первого прохода производится обмен параметрами полиэдров $H \leftrightarrow O$.

Расчёт пересечения плоскости i -й грани $D = \text{side}(P, G, i) \equiv d_1 d_2 \dots d_{n_i} d_1$ с прямой k -го ребра $ab = s_k s_{k+1}$ j -й грани $S = \text{side}(Q, F, j) \equiv s_1 s_2 \dots s_{m_j} s_1$ выполняется нормальным методом (1.130а) при условии $U \perp N$ непараллельности направляющего вектора отрезка $U = b - a$ и плоскости грани D . Далее найденная точка $c = a + U\theta$ проверяется на принадлежность отрезку ab и полигону D .

Первая проверка проста: $c \in ab$ при $0 \leq \theta \leq 1$. Второй тест $c \in D$ делается функцией *rad23_test*, возвращающей $r \leq 0$ при расположении точки c в границах полигона D , а сам тест возвращает 1 — **признак столкновения полиэдров**.

По окончании перебора всех пар граней и рёбер полиэдров H и O , но обнаружении их пересечения, тест возвращает нулевой признак столкновения. При этом один из полиэдров может быть целиком расположен как снаружи, так и внутри другого. Различению этих двух случаев посвящён следующий тест.

□ Третий тест *surr3_test*(P, G, Q, F) **окружения полиэдра** $O = \{Q, F\}$ **полиэдром** $H = \{P, G\}$ (рис. 1.133) обнаруживает это событие по расположению всех

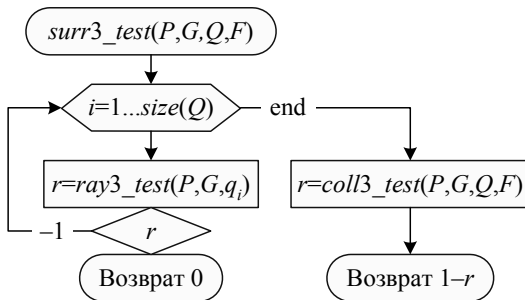


Рис. 1.133

точек списка Q внутри полиэдра H и отсутствию столкновения полиэдров, возвращая 1 как **признак окружения полиэдров**. В отсутствие окружения тест возвращает 0.

В случае выпуклости полиэдра H предпочтительнее вместо *ray3_test* использовать функцию *conv3_test*, а тест столкновения *coll3_test* отменить, возвратив по окончании цикла значение *surr3_test*=1.

□ Напоследок составим полезный для многих алгоритмов **тест ориентации нормалей полиэдра** $H = \{P, G\}$ со списками вершин $P = \{p_1, \dots, p_n\}$ и граней $G = \{G_1, \dots, G_m\}$, предназначенный для обнаружения и коррекции списков граней, чьи нормали направлены внутрь полиэдра.

Ручное задание и определение ориентации нормалей граней, а также (при необходимости) корректировка списков G_i , не представляет особых проблем. Однако, у полиэдров, полученных при решении многих задач отсечения, разрезания, объединения и т. п., могут появиться грани с внутренними нормальными. Это случится при нумерации вершин грани в отрицательном направлении и расчёте нормали по (1.145). В результате не защищённые от этого явления алго-

ритмы будут работать неверно. В первую очередь это касается правильного изображения передних (*лицевых*) и задних (*нелицевых*) граней алгоритмами, требующими строго внешней ориентации нормалей (см. п. 1.7.2).

Алгоритмический поиск и коррекция грани $G_i = \{g_{i1}, \dots, g_{in_i}, g_{i1}\}$, с внутренней нормалью могут быть выполнены **методом пробной точки**:

- формируется полигон i -й грани $Q = \text{side}(P, G, i)$ с первой вершиной q_1 и находится вектор нормали $N = \text{norm}(Q)$;
- в **близкой окрестности** грани G_i выбирается пробная точка q_n с локализацией $r \in \{1, -1\}$ относительно полиэдра (1 — внешней, -1 — внутренней);
- вычисляется значение неявной функции в пробной точке $f = nfN(q_1, N, q_n)$;
- при $r \cdot f < 0$ список G_i заменяется на реверсивный $\{g_{i1}, g_{in_i}, \dots, g_{i2}, g_{i1}\}$.

Единственная неформальность описанной процедуры — это выбор точки q_n с известной или вычисляемой локализацией. В случае **выпуклого** полиэдра решение элементарно (рис. 1.134,а): гарантированно **внутренней** точкой для всех его граней является **геометрический центр** вершин $\text{mean}(P)$, а свойство выпуклости снимает требование близости пробной точки $q_{\text{вну}} = \text{mean}(P)$ к тестируемой грани.

Один из возможных подходов к выбору пробной точки q_n для грани G_i **невыпуклого** полиэдра (рис. 1.134,б) включает следующие действия:

- в плоскости $\{q_1, N\}$ задаётся точка c_i **внутри** полигона грани Q . У выпуклого полигона такой точкой является его геометрический центр. У невыпуклой грани СК $x'y'z'$ преобразуется в положение $x''y''z''$, где плоскость полигона Q' становится фронтальной (см. гл. 3), находится любая диагональ $q'_j q'_k$ с **внутренней** средней точкой c' и восстанавливается положение точки c в исходной системе координат;

- берётся пробная точка $q_n = c + \varepsilon \bar{N}$, отстоящая от плоскости грани на расстоянии $0 < \varepsilon < 1$, гарантированно **малом** в сравнении с расстояниями до других граней полиэдра;

- вычисляется локализация пробной точки $r = \text{ray3_test}(P, G, q_n) \in \{1, -1\}$ и значение неявной функции $f = nfN(q_1, N, q_n)$.

С учётом всего сказанного получаем **алгоритм тестирования нормалей и коррекции граней полиэдра** $\text{norm_test}(P, G)$ с блок-схемой на рис. 1.135. Ввиду значительной сложности обработки невыпуклого полиэдра алгоритм проводит его тестирование $s = \text{conv3}(P, G)$ и при $s=1$ выполняет более простые операции, соответствующие выпуклому полиэдру.

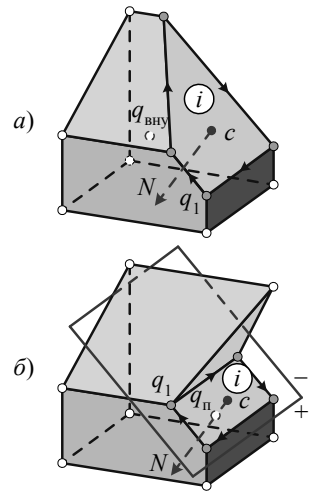


Рис. 1.134

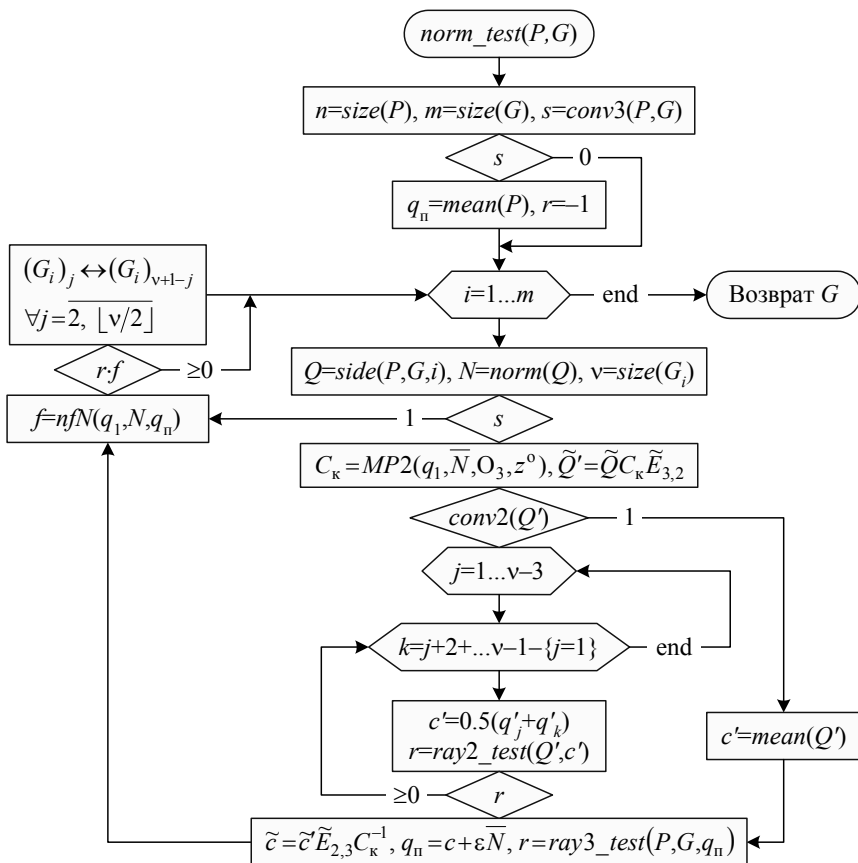


Рис. 1.135

Коррекция списка грани G_i с обнаруженной внутренней нормалью заключается в его **реверсировании** — перестановке $v=size(G_i)$ элементов списка в обратном порядке. Это реализуется либо путём создания вспомогательного списка L с элементами $L_j=(G_i)_{v+1-j} \forall j=\overline{1, v}$ с последующей заменой им старого списка G_i , либо прямым обменом элементов $(G_i)_j \leftrightarrow (G_i)_{v+1-j} \forall j=2, \lfloor v/2 \rfloor$. Вторым способ вдвое экономичнее как по времени выполнения, так и по затратам машинной памяти. По окончании проверки и коррекции списков граней с внутренними нормальными алгоритм *norm_test* возвращает обновлённую группу списков граней G , не имеющих внутренних нормалей.

Тестирование нормалей полиэдра и коррекцию списков его граней нужно проводить перед применением методов и алгоритмов, критичных к направлению нормалей, например, при отборе лицевых граней (см. п. 1.7.2). Для исключения лишних проверок в модель полиэдра можно ввести динамический флажок, устанавливаемый после тестирования нормалей и сбрасываемый после выполнения операций, способных изменить направление нормалей.

1.5.7. Объём и центр тяжести полиэдра

Рассмотрим методы вычисления объёма V и центра тяжести g **сплошного** (не пустотелого) полиэдра $H=\{P,G\}$. Эти параметры имеют большое прикладное значение в системах трёхмерного проектирования и моделирования, позволяя найти массу полиэдра и организовать его движение в пространстве по физическим законам, требующим знания расположения центра масс.

□ Начнём с изучения **пирамиды** с вершиной в точке o и основанием в виде произвольного полигона $P=p_1p_2\dots p_np_1$, лежащего в плоскости xy (рис. 1.136). Вычислим по (1.54) его площадь S_0 и центр тяжести g_0 , а по (1.119) — высоту пирамиды $h=|o_z|$. Проведём на расстоянии $0\leq z\leq h$ от плоскости основания параллельную секущую плоскость и на основе зависимостей

$$s(z)=S_0\left(1-\frac{z}{h}\right)^2, \quad c(z)=c_0+(o-c_0)\frac{z}{h}$$

площади и центра полигона сечения получим известные из [7, 21] формулы расчёта объёма и центра тяжести пирамиды:

$$V=\int_0^h s(z)dz=\frac{1}{3}S_0h, \quad (1.155a)$$

$$g=\frac{1}{V}\int_0^h s(z)c(z)dz=\frac{o+3c_0}{4}.$$

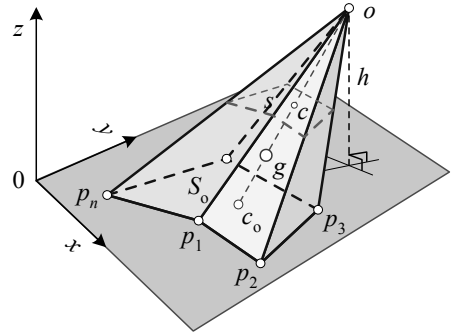


Рис. 1.136

При произвольной ориентации пирамиды в пространстве для вычисления площади и центра её основания S_0 и c_0 используются формулы (1.54), в которых алгебраическая площадь i -го треугольника с центром $c_i=(p_1+p_i+p_{i+1})/3$ находится с помощью векторного произведения его сторон [7, 21]:

$$S_0=\left|\sum_{i=2}^{n-1}\sigma_i\right|, \quad c_0=\sum_{i=2}^{n-1}\sigma_i c_i / \sum_{i=2}^{n-1}\sigma_i, \quad \sigma_i=0.5|(p_i-p_1)\times(p_{i+1}-p_1)|. \quad (1.155b)$$

□ **Выпуклый полиэдр** со списком вершин $P=\{p_1,\dots,p_n\}$ и группой списков граней $G=\{G_1,\dots,G_m\}$ представляем объединением t пирамид с полигонами оснований $Q_i=side(P,G,i)$, вершинами в любой **внутренней** точке, например, $o=mean(P)$ (рис. 1.137,a), объёмами и центрами тяжести V_i и g_i . Получаем аналогичные (1.53б) формулы расчёта объёма и центра выпуклого полиэдра:

$$V=\sum_i V_i, \quad g=\sum_i V_i g_i / V. \quad (1.156)$$

□ Объём и центр **невыпуклого полиэдра** со списком вершин $P=\{p_1,\dots,p_n\}$ и группой списков граней $G=\{G_1,\dots,G_m\}$ также могут быть вычислены по формуле (1.156), в которой V_i и g_i обозначают объёмы и центры тяжести **выпуклых** полиэдров, на которые **разрезан** исходный полиэдр $\{P,G\}$. Реализующий этот

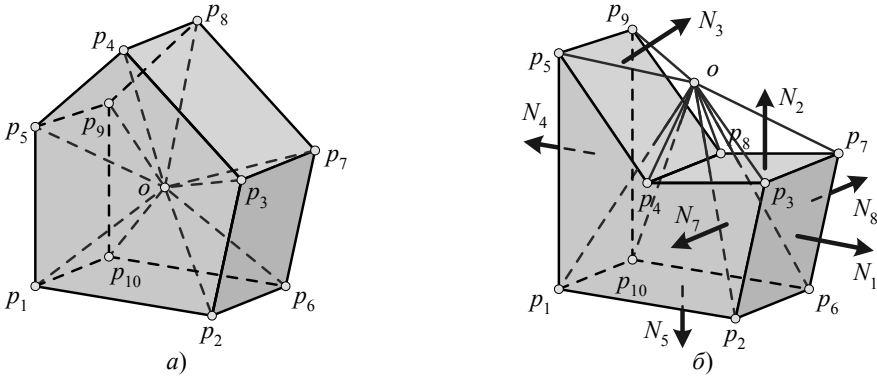


Рис. 1.137

метод алгоритм разрезания, разработанный в конце главы, весьма непрост и цена расчёта геометрических параметров полиэдра неоправданно большая.

После небольшой модификации описанный выше аддитивный метод способен работать и с невыпуклыми полиэдрами без их разрезания на выпуклые фрагменты. Потребуем *внешнюю* направленность нормалей N_i у всех граней полиэдра. Автоматическое тестирование нормалей и коррекция списков граней осуществляются алгоритмом *norm_test* (см. рис. 1.135). Соединив *любую* точку в окрестности полиэдра, например $o = \text{mean}(P)$, со всеми вершинами p_i , получим m пирамид (рис. 1.137,б) с центрами в точках c_i . Припишем объёму i -й пирамиды $\{Q_i, o\}$ с внешней нормалью основания $N_i = \text{norm}(Q_i)$ обратный знак НФ вершины $-nfN(q_{i1}, N_i, o)$, тогда в расчётных соотношениях

$$V_i = -\frac{1}{3} S_i \cdot nfN(q_{i1}, \bar{N}_i, o) \Rightarrow V = \sum_{i=1}^m V_i, \quad g_i = c_i + \frac{1}{4}(o - c_i) \Rightarrow g = \frac{1}{V} \sum_{i=1}^m V_i g_i \quad (1.157)$$

объёмы внутренних пирамид суммируются, а объёмы внешних — вычитаются, благодаря чему формируется алгебраическая сумма объёмов пирамид.

1.5.8. Выпуклая полиэдральная оболочка массива точек

Метод генерирования случайных выпуклых полигонов путём построения выпуклой оболочки массива случайных точек с помощью алгоритма *cover2* (см. рис. 1.37) имеет естественное трёхмерное обобщение. Допустим, в пространстве задано произвольное число n точек $p_i \forall i=1, n$. **Выпуклая полиэдральная оболочка массива точек** $P = \{p_1, \dots, p_n\}$ представляет собой выпуклый полиэдр минимального объёма, охватывающий все точки массива наподобие замкнутого покрова из эластичной ткани. По отношению к оболочке точки массива делятся на *внутренние* и *граничные*, принадлежащие вершинам, рёбрам или граням оболочки. Граничные точки, образующие вершины, назовём *выступающими*. Задача построения оболочки заключается в отборе из массива $\{p_i\}$ всех выступающих точек и построении на них полигонов граней с одновременным формированием матрицы вершин оболочки U и группы списков её граней G .

Алгоритм $cover3(P)$ построения выпуклой полиэдральной оболочки представлен блок-схемой на рис. 1.138. Сначала он проверяет **возможность** существования оболочки с помощью теста (1.112б). Если размерность пространства, натянутого на точки массива P , меньше трёх, то $n \times 4$ -матрица вершин $\tilde{P}$, построенная по (1.142), имеет неполный ранг, меньший четырёх. При этом оболочка, обтягивающая точки, не имеет объёма, поэтому алгоритм возвращает пустой результат $\{\emptyset, \emptyset\}$. Если же принципиальная возможность построения оболочки существует, то её формирование выполняется в три этапа.

1. На **первом этапе** ищутся три неколлинеарные точки $\{p_i, p_j, p_k\}$, плоскость которых не пересекает оболочку и потому способна быть плоскостью её первой грани. Условие неколлинеарности точек по (1.111а) — существование вектора нормали $N = (p_j - p_i) \times (p_k - p_j) \neq O_3$. Для исключения повторных сочетаний индексы точек берутся в интервалах $i = \overline{1, n-2}$, $j = \overline{i+1, n-1}$ и $k = \overline{j+1, n}$.

Искомая плоскость будет найдена тогда, когда во всём массиве P не найдется ни одной пары точек, лежащих от неё по разные стороны. Это легко проверяется тестом $cross3$, определённым в (1.146). В качестве параметра f — имени функции локализации точки относительно плоскости — наиболее экономично использовать неявную 3d-функцию $f(p) = nfN(p_i, N, p)$, определённую в (1.108б). Отметим, что полного прохода цикла $i = \overline{1, n-2}$ не происходит.

2. На **втором этапе** формируется полигон первой грани полиэдральной оболочки как выпуклой полигональной оболочки точек в плоскости $\{p_i, N\}$.

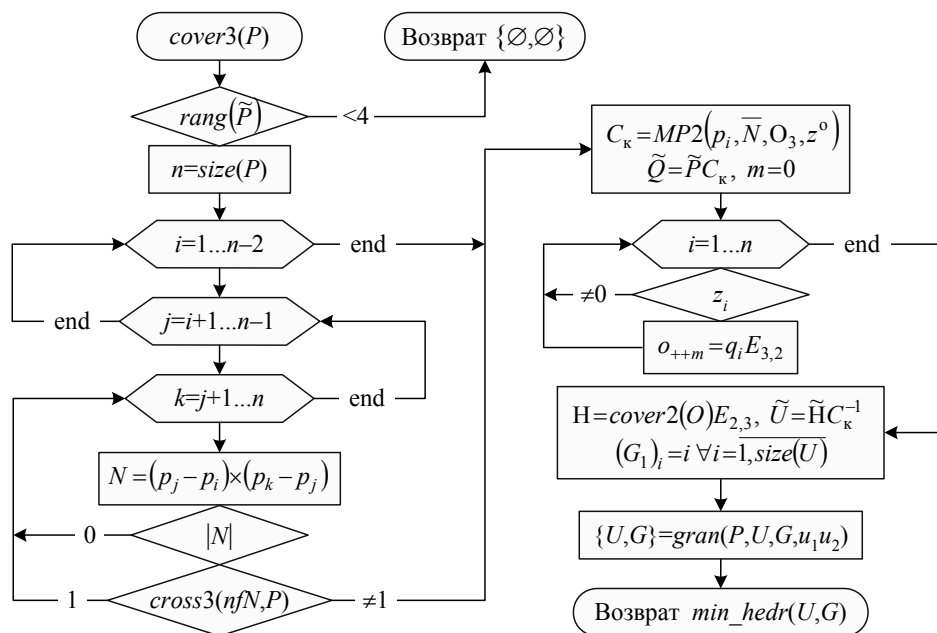


Рис. 1.138

Вычислив по (3.25б) матрицу преобразования СК $C_k = MP2(p_i, \bar{N}, O_3, z^0)$, рассчитаем координаты $\tilde{Q} = \tilde{P}C_k$ всех точек массива P в новой СК с фронтальной плоскостью $\{p_i, N\}$. Затем отберём в матрицу O лежащие в этой плоскости точки $q_i = [x_i \ y_i \ 0]$. Из них по алгоритму *cover2* (см. рис. 1.37) сформируем выпуклую оболочку $H = \text{cover2}(O)$. Обратным преобразованием $\tilde{U} = \tilde{H}C_k^{-1}$ получаем список $U = \{u_1, \dots, u_m\}$ вершин первой грани оболочки в исходной СК, а списком $G_1 = \{1, \dots, m\}$ начинаем формирование группы списков G .

Для корректного обращения с функцией *cover2*, работающей с точками на плоскости xu , в алгоритме *cover3* используются матрицы $E_{2,3}$ и $E_{3,2}$ преобразований (1.148) из R^2 в R^3 и обратно с сохранением координат x и y .

3. На **третьем этапе** запускается поиск второй грани оболочки, примыкающей к первому ребру первой грани u_1u_2 . Затем к ней присоединяются новые примыкающие грани до замыкания оболочки. По окончании формирования полиэдра оболочки алгоритм минимизирует его топологию функцией *min_hedr* (см. рис. 1.125) и возвращает параметры полиэдральной оболочки $\{U, G\}$.

Процесс присоединения новых граней оболочки к уже существующим имеет **рекурсивный** характер и реализует простой способ включения света во всех комнатах здания, соединённых друг с другом коридорами. По этому способу при входе в тёмную комнату нужно включить в ней свет и начать обход по периметру, нумеруя со второй двери, открываемые для входа в комнаты следующего уровня. Войдя в освещённую комнату, следует вернуться назад и перейти к следующей неотмеченной двери. В конце обхода всех дверей текущей комнаты нужно вернуться через её первую дверь в комнату предыдущего уровня и продолжить обход её неотмеченных дверей. Процесс освещения всего дома автоматически завершится, когда будут отмечены все двери первой комнаты.

На рис. 1.139 построен рекурсивный алгоритм *gran(P, U, G, ab)* **формирования из точек списка P грани, примыкающей к отрезку ab** . Кроме аргументов $P = \{p_1, \dots, p_n\}$ и ab он получает список вершин $U = \{u_1, \dots, u_m\}$ и группу списков граней $G = \{G_1, \dots, G_\gamma\}$ текущей оболочки, а возвращает её обновлённую модель $\{U, G\}$. Методика нахождения примыкающей грани следующая.

□ Сначала ищем в списке P такую точку $p_i, i \in [1, n]$, что плоскость $\{a, N\}$ с нормалью $N = (b-a) \times (p_i-a) \neq O_3$ удовлетворяет двум условиям:

- во-первых, не совпадает ни с одной из уже имеющихся граней $G_j \ \forall j = \overline{1, \gamma}$. Совпадение плоскости $\{a, N\}$ с плоскостью j -й грани $Q = \text{side}(U, G, j)$ с нормалью $H = \text{norm}(Q)$ проверяется тестом (1.133а). При $|N \times H| = 0$ и $N \circ (q_1 - a) = 0$ плоскость $\{a, N\}$, совпадающая с плоскостью Q , не может нести новую грань;

- во-вторых, не пересекается с оболочкой, т. е. среди элементов списка P нет ни одной пары разносторонних точек. Это легко проверяется тестом *cross3* с аргументом $f(p) = n f N(a, N, p)$. При *cross3*=1 плоскость $\{a, N\}$ пересекает оболочку и не может нести грань выпуклого полиэдра.

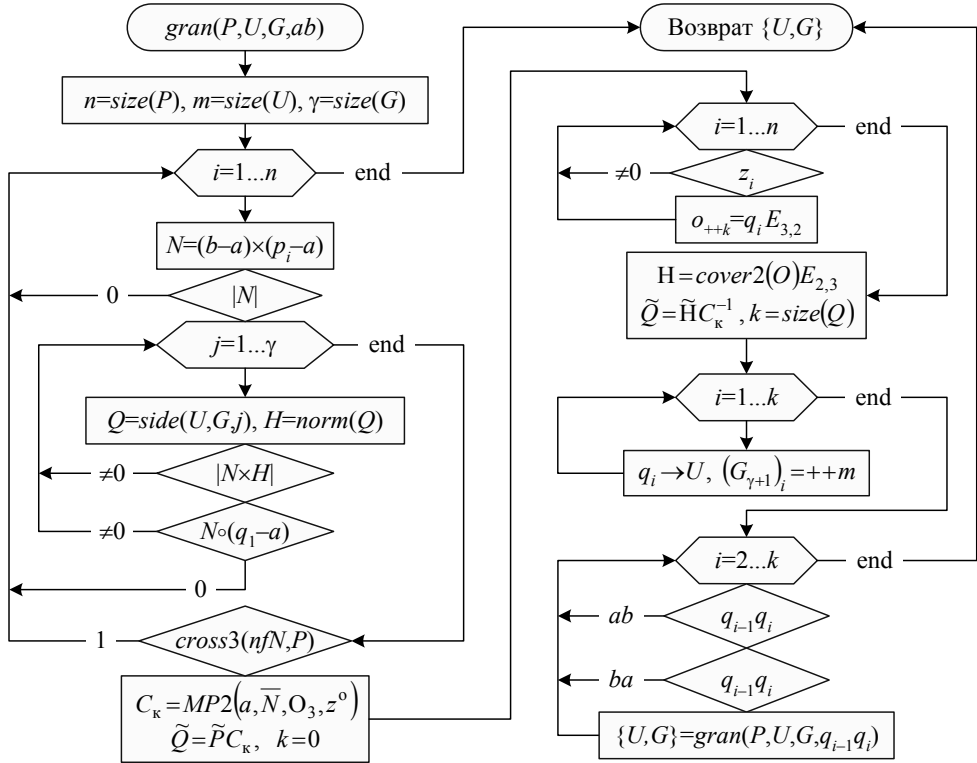


Рис. 1.139

Если после просмотра списка P не будет найдено ни одной плоскости $\{a, b, p_i\}$, удовлетворяющей перечисленным условиям, то функция $gran$ возвращает параметры полиэдра без изменения. Это **первое условие выхода из рекурсии** — по отсутствию новой грани, примыкающей к отрезку ab .

□ Далее формируем полигон $\gamma+1$ -й грани полиэдральной оболочки как выпуклую полигональную оболочку точек, лежащих в плоскости $\{a, N\}$. Вычислив матрицу преобразования $C_k = MP2(a, \bar{N}, O_3, z^0)$, найдём координаты $\tilde{Q} = \tilde{P} C_k$ всех точек массива P в новой СК $x'y'z'$, где плоскость $\{a, N\}$ является фронтальной. Затем отберём в список O точки $q_i = [x_i \ y_i \ 0] \in x'y'$. Из этих точек с помощью алгоритма $cover2(O)$ формируем выпуклую оболочку H . Обратным преобразованием $\tilde{Q} = \tilde{H} C_k^{-1}$ получаем полигон $Q = q_1 q_2 \dots q_k$ вершин новой грани в исходной СК xuz . Точки q_i добавляем в список U , а из их индексов в нём создаём список вершин грани $G_{\gamma+1} = \{m+1, \dots, m+k\}$.

□ В заключение организуем рекурсивный вызов функции $gran$ в форме

$$\{U, G\} = gran(P, U, G, q_i q_{i+1}) \forall i = \overline{1, k-1}.$$

Это означает разветвление дерева рекурсии на $k-1$ направление по числу сторон полигона Q . Некоторые из этих ветвей окажутся **тупиковыми** по первому

условию выхода из рекурсии. Другие ветви, возможно, сами будут ветвиться. Для исключения захода в грань предыдущего уровня рекурсии проверяем несовпадение отрезка $q_i q_{i+1}$ с входным отрезком в прямом ab либо обратном ba порядке следования концевых точек. По **второму условию выхода из рекурсии** — завершению цикла перебора рёбер грани — функция *gran* возвращает обновлённую модель оболочки $\{U, G\}$.

Для корректного обращения к функции *cover2*, строящей выпуклую оболочку на плоскости xu , в алгоритме *gran* используются матрицы $E_{3,2}$ и $E_{2,3}$ преобразований (1.148) из R^3 в R^2 и обратно с сохранением координат x и u .

Пример 1.17. Проследим работу алгоритма *cover3* на примере построения оболочки массива семи точек $p_1 \div p_7$, из которых p_2 и $p_4 \div p_7$ являются выступающими, p_3 — внутренней, p_1 и $p_4 \div p_7$ — компланарными, а $p_1 \div p_3$ — коллинеарными (рис. 1.140, а).

Перебирая индексы $i=\overline{1,5}$, $j=\overline{i+1,6}$ и $k=\overline{j+1,7}$, тестируем тройки $\{p_i, p_j, p_k\}$ на способность нести плоскость грани выпуклого полиэдра:

- тройка точек $\{p_1, p_2, p_3\}$ коллинеарна;
- плоскости всех троек $\{p_1, p_2, p_4\} \div \{p_1, p_2, p_7\}$ и $\{p_1, p_3, p_4\} \div \{p_1, p_3, p_7\}$ проходят так, что точки множества P лежат относительно них по разные стороны;
- наконец, находим подходящую тройку точек $\{p_1, p_4, p_5\}$. В её плоскости лежат ещё две точки p_6 и p_7 , так что полигональная оболочка пяти точек, формируемая алгоритмом *cover2* во вспомогательной системе координат, имеет вид $U=p_7 p_6 p_4 p_5 p_1 \equiv u_1 u_2 u_3 u_4 u_5$, а список первой четырёхугольной грани есть $G_1=\{1, 2, 3, 4, 5\}$.

Начиная поиск грани, примыкающей к первому ребру первой грани $p_7 p_6$, функция *gran* находит точку p_2 , а алгоритм *cover2* формирует полигон $p_6 p_2 p_7 p_6 \equiv u_6 u_7 u_8 u_9$ второй треугольной грани $G_2=\{6, 7, 8, 9\}$. На втором переходе ищется третья грань, примыкающая к первому ребру второй грани $p_6 p_2$. Результат поиска — точка p_4 и плоская оболочка трёх точек $p_4 p_6 p_2 p_4 \equiv u_{10} u_{11} u_{12} u_{13}$ треугольной грани $G_3=\{10, 11, 12, 13\}$.

Третий переход на уже существующую грань G_1 , примыкающую к первому ребру третьей грани $p_4 p_6$, является тупиковым, а следующий переход даёт примыкающую к ребру $p_2 p_4$ четвёртую грань $p_4 p_5 p_2 p_4 \equiv u_{14} u_{15} u_{16} u_{17}$ со списком вершин $G_4=\{14, 15, 16, 17\}$. Аналогично, пятый переход через ребро $p_4 p_5$ на грань G_1 является тупиковым, а шестой переход через ребро $p_5 p_2$ открывает пятую треугольную грань $p_5 p_2 p_7 p_5 \equiv u_{18} u_{19} u_{20} u_{21}$ со списком вершин $G_5=\{18, 19, 20, 21\}$. Оба перехода с этой треугольной грани на G_2 и G_1 являются тупиковыми.

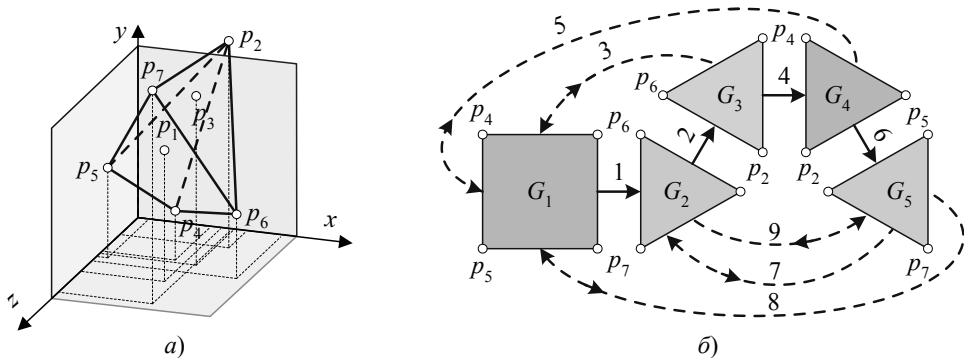


Рис. 1.140

Они завершают обход G_5 и возвращают алгоритм через четвёртую и третью грани (их обход также оказался завершённым) на грань G_2 .

Оставшийся тупиковый переход через ребро p_2p_7 на грань G_5 возвращает поиск на первую грань G_1 , а алгоритм — в функцию *cover2* со списком вершин

$$U = p_7p_6p_4p_5p_7 | p_6p_2p_7p_6 | p_4p_6p_2p_4 | p_4p_5p_2p_4 | p_5p_2p_7p_5 \equiv u_1 \dots u_{21}$$

и списками граней $G_1 \div G_5$. После минимизации топологии этого полиэдра получим следующую модель выпуклой полиэдральной оболочки (рис. 1.140,б):

$$U = p_7p_6p_4p_5p_2 \equiv u_1u_2u_3u_4u_5, \quad G = \{G_1, G_2, G_3, G_4, G_5\}, \\ G_1 = \{1, 4, 3, 2, 1\}, \quad G_2 = \{2, 5, 1, 2\}, \quad G_3 = \{3, 5, 2, 3\}, \quad G_4 = \{3, 4, 5, 3\}, \quad G_5 = \{4, 1, 5, 4\}.$$

□

1.5.9. Квадратичные поверхности

Алгебраические поверхности второго порядка (*квадратичные поверхности*) описываются неявным трёхкоординатным уравнением (*квадрикой*)

$$f(x, y, z) = Ax^2 + By^2 + Cz^2 + 2Exy + 2Gxz + 2Hyz + 2Kx + 2Ly + 2Mz + D = 0 \quad (1.158a)$$

или *квадратичной формой* с векторным аргументом $p \in \mathbb{R}^3$:

$$f(p) = \tilde{p} F \tilde{p}^T = 0, \quad \text{где}$$

$$F = \left[\begin{array}{ccc|c} A & E & G & K \\ E & B & H & L \\ G & H & C & M \\ \hline K & L & M & D \end{array} \right] = \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & D \end{bmatrix}, \quad F_{11}^T = F_{11}, \quad F_{12} = F_{21}^T. \quad (1.158б)$$

Нормаль и вектор параметров касательной плоскости в точке $q \in f(p)$ в соответствии с (1.93а) и (1.99) равны

$$N = 2[q \ 1] \begin{bmatrix} F_{11} \\ F_{21} \end{bmatrix} = 2(qF_{11} + F_{21}), \quad F_{\text{кас}} = [N \ -q \circ N]. \quad (1.159)$$

Для любой квадратичной поверхности четыре величины

$$I_1 = A + B + C, \quad I_2 = \begin{vmatrix} A & E \\ E & B \end{vmatrix} + \begin{vmatrix} B & H \\ H & C \end{vmatrix} + \begin{vmatrix} A & G \\ G & C \end{vmatrix}, \quad I_3 = |F_{11}|, \quad I_4 = |F| \quad (1.160a)$$

являются *инвариантами* [12] относительно переноса и вращения, а числа

$$I'_2 = \begin{vmatrix} A & K \\ K & D \end{vmatrix} + \begin{vmatrix} B & L \\ L & D \end{vmatrix} + \begin{vmatrix} C & M \\ M & D \end{vmatrix}, \quad I'_3 = \begin{vmatrix} B & H & L \\ H & C & M \\ L & M & D \end{vmatrix} + \begin{vmatrix} A & G & K \\ G & C & M \\ K & M & D \end{vmatrix} + \begin{vmatrix} A & E & K \\ E & B & L \\ K & L & D \end{vmatrix} \quad (1.160б)$$

представляют собой *полуинварианты* относительно преобразования вращения. Анализ знаков и соотношений инвариантов и полуинвариантов уравнений (1.158) позволяет быстро определить тип и составить *каноническое уравнение* поверхности второго порядка с помощью табл. 1.3, 1.4.

Неявная форма неудобна для построения поверхностей и используется при анализе их свойств по инвариантам и решении задач пересечения.

Таблица 1.3

$\frac{I_3}{I_4}$		Невырожденные поверхности $I_4 \neq 0$		Вырожденные поверхности $I_4 = 0$
		$I_4 > 0$	$I_4 < 0$	
Центральные поверхности $I_3 \neq 0$	$I_1 I_3 > 0$ и $I_2 > 0$	Мнимый эллипсоид $x^2/a^2 + y^2/b^2 + z^2/c^2 + 1 = 0$	Действительный эллипсоид $x^2/a^2 + y^2/b^2 + z^2/c^2 - 1 = 0$	Точка вырожденного эллипсоида $x^2/a^2 + y^2/b^2 + z^2/c^2 = 0$
	$I_1 I_3 \leq 0$ или $I_2 \leq 0$	Однополостный гиперболоид $x^2/a^2 - y^2/b^2 + z^2/c^2 - 1 = 0$	Двуполостный гиперболоид $x^2/a^2 - y^2/b^2 + z^2/c^2 + 1 = 0$	Конус вырожденного гиперболоида $x^2/a^2 - y^2/b^2 + z^2/c^2 = 0$
Нецентральные поверхности $I_3 = 0$		Гиперболический параболоид $x^2/a^2 - y/b - z^2/c^2 = 0$	Эллиптический параболоид $x^2/a^2 - y/b + z^2/c^2 = 0$	Цилиндры и пары плоскостей (табл. 1.4)

Таблица 1.4

$\frac{I'_3}{I_2}$	$I_2 > 0$	$I_2 < 0$	$I_2 = 0$
Цилиндры $I'_3 \neq 0$	$I_1 I'_3 < 0$	гиперболический $x^2/a^2 - z^2/c^2 - 1 = 0$	параболический $x^2/a^2 - z/c = 0$
	$I_1 I'_3 > 0$		
Пары плоскостей $I'_3 = 0$	вырожденных в прямую линию $x^2/a^2 + z^2/c^2 = 0$	пересекающихся $x^2/a^2 - z^2/c^2 = 0$	$I'_2 < 0$ параллельных $x^2/a^2 - 1 = 0$
			$I'_2 = 0$ совпадающих $x^2/a^2 = 0$
			$I'_2 > 0$ мнимых $x^2/a^2 + 1 = 0$

Исследуем *взаимное расположение квадратичной поверхности $\{F\}$ с прямой линией $\{q, V\}$ и плоскостью $\{q, V, W\}$.*

□ Первая задача решена в п. 1.1.7. Анализируя варианты квадратного уравнения $at^2 + 2bt + c = 0$ из (1.60) с коэффициентами $a = VF_{11}V^T$, $b = (qF_{11} + F_{21})V^T$, $c = qF_{11}q^T + 2qF_{21}^T + D = f(q)$ и дискриминантом $\delta = b^2 - ac$, заключаем:

- при $\{a \neq 0\} \wedge \{\delta > 0\}$ прямая **пересекает** поверхность в **двух различных** точках $p_1 = q + Vt_1$ и $p_2 = q + Vt_2$ с параметрами t_1 и t_2 из (1.61а);
- при $\{a \neq 0\} \wedge \{\delta = 0\}$ прямая **касается** поверхности в **единственной** точке $p_1 = q + Vt_1$ с параметром t_1 из (1.61б);
- при $\{a = 0\} \wedge \{b \neq 0\}$ прямая **пересекает** поверхность в **единственной** точке $p_1 = q + Vt_1$ с параметром t_1 из (1.61в);
- при $\{a = 0\} \wedge \{b = 0\} \wedge \{c = 0\}$ прямая **касается** нецентральной поверхности в **бесконечном множестве точек**;
- в остальных случаях прямая не имеет с квадратичной поверхностью ни одной общей точки.

□ Для оценки взаимного положения квадратичной поверхности и плоскости подставим ПФ плоскости $p(t, \tau) = q + Vt + W\tau$ в неявное векторное уравнение квадратичной поверхности с блочной матрицей F из (1.158б):

$$(q + Vt + W\tau)F_{11}(q + Vt + W\tau)^T + 2(q + Vt + W\tau)F_{21}^T + D = 0. \quad (1.161)$$

После перемножения и приведения подобных членов получим квадратичное уравнение связи параметров t и τ в скалярном и матричном видах

$$at^2 + b\tau^2 + 2et\tau + 2kt + 2l\tau + d = 0, \quad \theta(t, \tau) = \begin{bmatrix} t & \tau \end{bmatrix} \Theta \begin{bmatrix} t \\ \tau \end{bmatrix} = 0 \quad (1.162a)$$

с симметричной матрицей Θ и коэффициентами a, b, d, e, k и l :

$$\Theta = \begin{bmatrix} a & e & k \\ e & b & l \\ k & l & d \end{bmatrix}, \quad \begin{aligned} a &= VF_{11}V^T, & b &= WF_{11}W^T, & d &= qF_{11}q^T + 2qF_{21}^T + D, \\ e &= VF_{11}W^T, & k &= (qF_{11} + F_{21})V^T, & l &= (qF_{11} + F_{21})W^T. \end{aligned} \quad (1.162б)$$

Налицо полная аналогия (1.162) с уравнениями (1.56) квадратичных линий на плоскости. Вычислив по (1.58) инварианты и полуинвариант матрицы Θ

$$I_1 = a + b, \quad I_2 = ab - e^2, \quad I_3 = |\Theta|, \quad I'_2 = \begin{vmatrix} a & k \\ k & d \end{vmatrix} + \begin{vmatrix} b & l \\ l & d \end{vmatrix} = I_1 d - k^2 - l^2,$$

определим по табл. 1.1 *тип линии пересечения* 3d-квадрики с плоскостью:

- невырожденные линии (при $I_3 \neq 0$): **действительный эллипс** (при $I_2 > 0$ и $I_1 I_3 < 0$), **гипербола** (при $I_2 < 0$) и **парабола** (при $I_2 = 0$) образуются в большинстве случаев **сечения криволинейной** квадратичной поверхности плоскостью;
- **мнимый эллипс** (при $I_2 > 0$ и $I_1 I_3 > 0$) и **мнимая прямая** (при $I_2 = I_3 = I'_2 = 0$) означают **отсутствие пересечений и касаний** поверхности с плоскостью;
- **точка вырожденного эллипса** (при $I_2 > 0$ и $I_3 = 0$) получается при **касании** поверхности с плоскостью в одной точке;

- две асимптоты **вырожденной гиперболы** (при $I_2 < 0$ и $I_3 = 0$) образуются при **сечении конуса плоскостью**, проходящей через его вершину;
- **две параллельные прямые** (при $I_2 = I_3 = 0$ и $I'_2 < 0$) возникают при **сечении цилиндра плоскостью**, проходящей параллельно его оси;
- **одна двойная прямая** (при $I_2 = I_3 = 0$ и $I'_2 > 0$) образуется в результате **касания плоскости с линейчатыми поверхностями** параболоидов, конуса или цилиндров.

В тех случаях, когда неявное уравнение в (1.162а) может быть приведено к явному виду $t = \varphi(\tau)$ или $\tau = \psi(t)$, возможно получение явных ПФ $p(\tau) = q + V\varphi(\tau) + W\tau$ или $p(t) = q + Vt + W\psi(t)$ пространственных линий пересечения плоскости с квадратичной поверхностью. Иначе нужно найти зависимости $t = \varphi(\sigma)$ и $\tau = \psi(\sigma)$, тогда линией сечения будет $p(\sigma) = q + V\varphi(\sigma) + W\psi(\sigma)$.

Пример 1.18. Классифицировать поверхность, описываемую неявным уравнением $f(x, y, z) = 2x^2 + y^2 - 3z^2 + xz - 2y + z - 1 = 0$, и линию пересечения этой поверхности с плоскостью, заданной отрезками $h_x = 3$, $h_y = 2$ и $h_z = 1$, отсекаемыми ею на осях координат.

Решение. Составим по (1.158) матрицу поверхности и вычислим её инварианты:

$$F = \begin{bmatrix} 4 & 0 & 1 & 0 \\ 0 & 2 & 0 & -2 \\ 1 & 0 & -6 & 1 \\ 0 & -2 & 1 & -2 \end{bmatrix}, I_1 = 0, I_2 = \begin{vmatrix} 4 & 0 \\ 0 & 2 \end{vmatrix} + \begin{vmatrix} 2 & 0 \\ 0 & -6 \end{vmatrix} + \begin{vmatrix} 4 & 1 \\ 1 & -6 \end{vmatrix} = -29, I_3 = 2 \begin{vmatrix} 4 & 1 \\ 1 & -6 \end{vmatrix} = -50, I_4 = |F| = 192.$$

По классификации в табл. 1.3 такие инварианты соответствуют поверхности **однополостного гиперболоида**, плоскими сечениями которого могут быть эллипс, парабола или гипербола.

Сформируем по (1.107б) параметры плоскости $o = [3 \ 0 \ 0]$, $V = [-3 \ 2 \ 0]$, $W = [0 \ -2 \ 1]$ и вычислим по (1.162) коэффициенты квадратичного уравнения $a = 44$, $b = 2$, $e = -11$, $g = -40$, $j = 8$, $d = 34$. По инвариантам квадратичной линии $I_1 = 46 > 0$, $I_2 = -33 < 0$, $I_3 = -98 < 0$ с помощью табл. 1.1 определяем, что линия пересечения, описываемая неявным уравнением $44t^2 + 2\tau^2 - 22t\tau - 80t + 16\tau + 34 = 0$, является **гиперболой**.

Задав значение $h_x = 4$, получим в сечении гиперболоида плоскостью **параболу**, удовлетворяющую неявному уравнению $72t^2 + 2\tau^2 - 24t\tau - 136t + 18\tau + 62 = 0$ с инвариантами $I_1 = 74 > 0$, $I_2 = 0$, $I_3 = -392 < 0$. При $h_x = 5$ в сечении образуется **эллипс** с уравнением $108t^2 + 2\tau^2 - 26t\tau - 208t + 20\tau + 98 = 0$ и инвариантами $I_1 = 110 > 0$, $I_2 = 47 > 0$ и $I_3 = -786 < 0$. $\square$

Пример 1.19. Построить тень каркаса пирамиды с вершинами $p_1 = [3 \ 4 \ -1]$, $p_2 = [2 \ 4 \ 2]$, $p_3 = [5 \ 3 \ 2]$, $p_4 = [2 \ 2 \ 0]$ на поверхности эллипсоида с полуосями $r_x = 2$, $r_y = 1$, $r_z = 3$

и центром в точке $o = [-2 \ 1 \ -2]$. Источник света удалён в бесконечность в направлении вектора $L = [4 \ 2 \ 1]$ (рис. 1.141).

Решение. По каноническому уравнению эллипсоида из табл. 1.3 и заданным параметрам получим его неявное уравнение

$$\left(\frac{x-o_x}{r_x}\right)^2 + \left(\frac{y-o_y}{r_y}\right)^2 + \left(\frac{z-o_z}{r_z}\right)^2 - 1 = 0 \Rightarrow 9x^2 + 36y^2 + 4z^2 + 36x - 72y + 16z + 52 = 0,$$

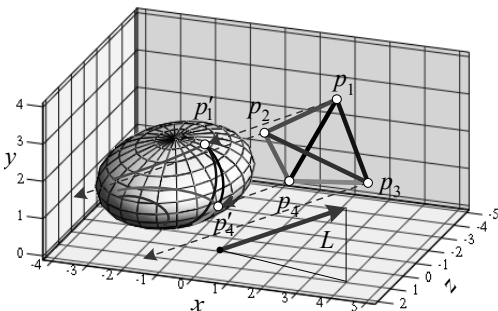


Рис. 1.141

а по (1.158) следующую матрицу квадратичной формы и её блоки:

$$F = \begin{bmatrix} 9 & 0 & 0 & 18 \\ 0 & 36 & 0 & -36 \\ 0 & 0 & 4 & 8 \\ 18 & -36 & 8 & 52 \end{bmatrix}, \quad F_{11} = \begin{bmatrix} 9 & 0 & 0 \\ 0 & 36 & 0 \\ 0 & 0 & 4 \end{bmatrix}, \quad F_{12} = \begin{bmatrix} 18 \\ -36 \\ 8 \end{bmatrix}, \quad F_{21} = [18 \ -36 \ 8], \quad D=52.$$

Так как в квадратном уравнении (1.60) коэффициент $a(p)=LF_{11}L^T=292 \neq 0$ и не зависит от расположения точки p , то возможность пересечения луча $p-Lt$ с квадратичной поверхностью определяется знаком дискриминанта $\delta(p)=b^2(p)-a(p)c(p)$ с коэффициентами

$$b(p)=-(pF_{11}+F_{21})L^T=-8-34x-72y-4z,$$

$$c(p)=pF_{11}p^T+2pF_{21}^T+D=9x^2+36y^2+4z^2+36x-72y+16z+52.$$

Вычислим дискриминанты и действительные точки тени для четырёх вершин пирамиды

$$p'_i = p_i + \frac{b(p_i) + \sqrt{\delta(p_i)}}{a(p_i)}L \quad \forall i=\overline{1,4}:$$

$$b(p_1)=-400, \quad c(p_1)=517, \quad \delta(p_1)=9036>0 \Rightarrow p'_1=[-1.177 \ 1.911 \ -2.044];$$

$$b(p_2)=-376, \quad c(p_2)=496, \quad \delta(p_2)=-3456<0;$$

$$b(p_3)=-412, \quad c(p_3)=613, \quad \delta(p_3)=-9252<0;$$

$$b(p_4)=-224, \quad c(p_4)=160, \quad \delta(p_4)=3456>0 \Rightarrow p'_4=[-0.263 \ 0.868 \ -0.566].$$

Отрицательные значения $\delta(p_2)$ и $\delta(p_3)$ означают, что лучи, проходящие через вершины p_2 и p_3 , не пересекаются с поверхностью эллипсоида. Следовательно, рёбра пирамиды p_1p_2 , p_1p_3 , p_2p_3 , p_2p_4 и p_3p_4 , инцидентные этим вершинам, не могут полностью проецироваться на поверхность. $\square$

В общем случае тень прямолинейного отрезка $p_i p_j$ на криволинейной поверхности может быть одной линией, разрываться на несколько фрагментов либо вообще отсутствовать. Концы отрезка могут не иметь тени, а его внутренние точки могут, как, например, ребро p_2p_3 на рис. 1.141. Предлагается следующий **численный метод расчёта криволинейной тени** каркасного объекта на квадратичной поверхности от источника света, бесконечно удалённого в направлении L . Метод обеспечивает достоверное построение тени произвольного каркаса на произвольной квадратичной поверхности, а при большой частоте разбиения криволинейных рёбер каркаса — визуальную гладкость их теней.

 **Шаг 1.** Рёбра каркаса разбиваются на достаточно малые отрезки прямых.

Шаг 2. Для каждой концевой точки отрезка по (1.60б,в) при $q=p_i$ и $V=-L$ вычисляются коэффициенты a_i , b_i , c_i и дискриминант δ_i .

Шаг 3. Если для отрезка $p_i p_j$ выполняется условие $\{\delta_i \geq 0\} \wedge \{\delta_j \geq 0\}$, то на основе (1.61а) рассчитываются ближайшие к источнику света точки

$$p'_i = p_i + \frac{b_i + \sqrt{\delta_i}}{a_i}L, \quad p'_j = p_j + \frac{b_j + \sqrt{\delta_j}}{a_j}L$$

пересечения лучей p_i-Lt и p_j-Lt с квадратичной поверхностью. В противном случае полная тень отрезка отсутствует.

Шаг 4. Строится изображение отрезка прямой $p'_i p'_j$ на экране дисплея. $\square$

1.5.10. Параметрические поверхности

Параметрическая форма $p(t, \tau)$ как самая гибкая и универсальная наиболее распространена для описания и построения поверхностей благодаря разнообразию выбора функций, задающих движения точки и линии в доступных направлениях. С независимыми параметрами t и τ часто ассоциируются конкретные геометрические или физические понятия, например, угол и время, что превращает абстрактные формулы в легко представимые фигуры. Обязательными элементами ПФ являются интервалы изменения параметров t и τ , **необходимые и достаточные** для описания точек желаемого фрагмента поверхности.

Декартова параметризация поверхности $p(t, \tau) = [x(t, \tau) \ y(t, \tau) \ z(t, \tau)]$ из других форм её описания может быть выполнена следующими способами.

1. **Заменой аргументов** x , y и z **канонического уравнения** $f(x, y, z) = 0$ функциями двух параметров t и τ . Явное уравнение, например, $y = \psi(x, z)$, всегда можно параметризовать как $\{x(t, \tau) = t, \ y(t, \tau) = \psi(t, \tau), \ z(t, \tau) = \tau\}$. В основном **метод подстановок** применяется для параметризации неявных канонических моделей квадратичных поверхностей (см. табл. 1.3, 1.4) с помощью известных тригонометрических тождеств (см. п. 1.1.8).

Неявная модель простейшей квадратичной поверхности — **сферы единичного радиуса** с центром в начале координат и каноническим уравнением $x^2 + y^2 + z^2 - 1 = 0$ — параметризуется с помощью любой из 96 (!) троек подстановок $\{x(t, \tau), \ y(t, \tau), \ z(t, \tau)\}$, использующих тригонометрические функции $\pm \sin(t)$, $\pm \cos(t)$, $\pm \sin(\tau)$ и $\pm \cos(\tau)$. Вот одна из таких подстановок:

$$x(t, \tau) = \sin(t), \ y(t, \tau) = \cos(t) \sin(\tau), \ z(t, \tau) = \cos(t) \cos(\tau).$$

Путём подбора подходящих подстановок в канонические уравнения на рис. 1.142 построены все криволинейные квадратичные поверхности при масштабных коэффициентах $a=1$, $b=2$ и $c=3$:

- **действительный эллипсоид** $(x/a)^2 + (y/b)^2 + (z/c)^2 - 1 = 0$ (рис. 1.142, а):

$$p(t, \tau) = [a \sin(t) \sin(\tau) \ b \cos(t) \ c \sin(t) \cos(\tau)] \quad \forall t \in [0, \pi], \ \tau \in [0, 2\pi]. \quad (1.163a)$$

Как образец неудачной подстановки приведём другую модель эллипсоида

$$p(t, \tau) = [a \sqrt{1-t^2} \sin(\tau) \ bt \ c \sqrt{1-t^2} \cos(\tau)] \quad \forall t \in [-1, 1], \ \tau \in [0, 2\pi], \quad (1.163б)$$

у которой частная производная

$$p'_t(t, \tau) = \left[\frac{-a t \sin(\tau)}{\sqrt{1-t^2}} \ b \ \frac{-c t \cos(\tau)}{\sqrt{1-t^2}} \right]$$

не вычислима в точках $[0 \ \pm b \ 0]$ при $t = \pm 1$, хотя нормаль

$$N(t, \tau) = -[bc \sqrt{1-t^2} \sin(\tau) \ act \ ab \sqrt{1-t^2} \cos(\tau)]$$

существует во всех точках поверхности эллипсоида, а при $t = \pm 1$ вычисляется **вручную** разрешением неопределённости типа 0/0 по правилу Лопиталя [21];

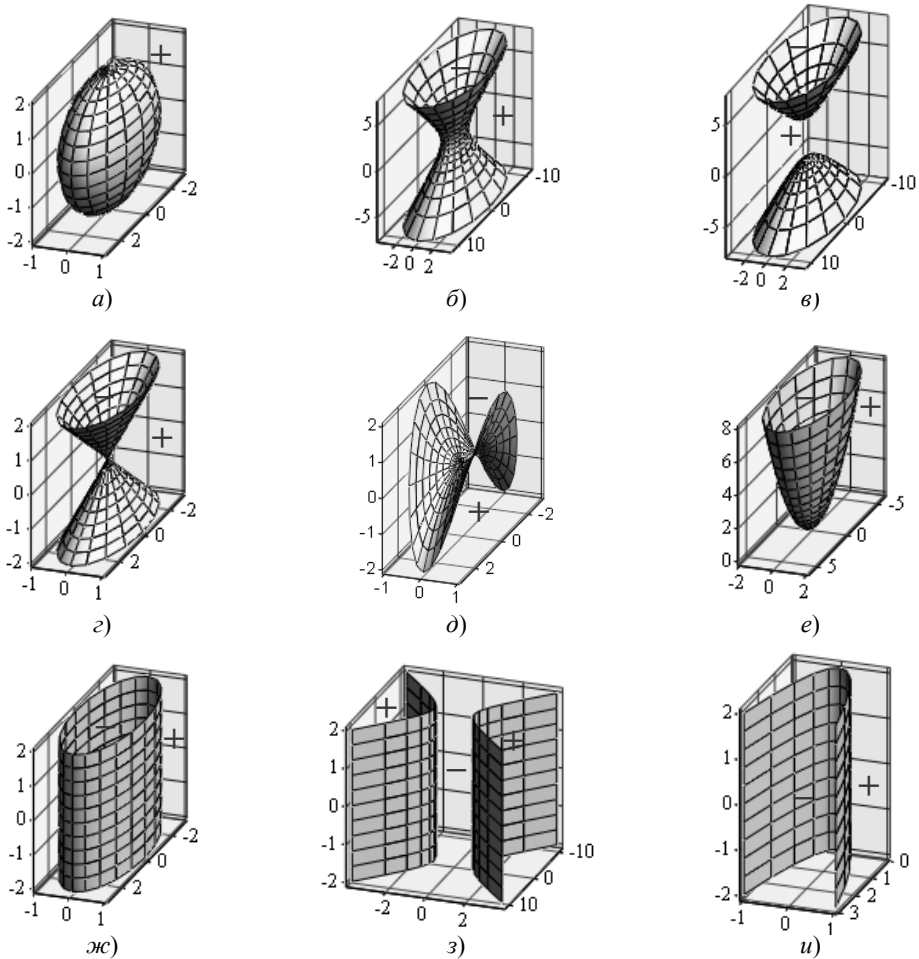


Рис. 1.142

- **однополостный гиперboloид** $(x/a)^2 - (y/b)^2 + (z/c)^2 - 1 = 0$ (рис. 1.142,б):

$$p(t, \tau) = [ach(t)\sin(\tau) \quad bsh(t) \quad cch(t)\cos(\tau)] \quad \forall t \in (-\infty, \infty), \tau \in [0, 2\pi]; \quad (1.164)$$

- **двуполостный гиперboloид**, заданный единым каноническим уравнением $(x/a)^2 - (y/b)^2 + (z/c)^2 + 1 = 0$, описывается двумя параметрическими функциями верхней (+) и нижней (-) полостей (рис. 1.142,в):

$$p(t, \tau) = [ash(t)\sin(\tau) \quad \pm bch(t) \quad csh(t)\cos(\tau)] \quad \forall t \in [0, \infty), \tau \in [0, 2\pi]; \quad (1.165)$$

- **эллиптический конус** $(x/a)^2 - (y/b)^2 + (z/c)^2 = 0$ (рис. 1.142,г):

$$p(t, \tau) = [at\sin(\tau) \quad bt \quad ct\cos(\tau)] \quad \forall t \in (-\infty, \infty), \tau \in [0, 2\pi]; \quad (1.166)$$

- седловая поверхность **гиперболического параболоида** (рис. 1.142,д), заданного неявным уравнением $(x/a)^2 - y/b - (z/c)^2 = 0$, описывается в непрерывной тригонометрической форме как

$$p(t, \tau) = [at \sin(\tau) \quad -bt^2 \cos(2\tau) \quad ct \cos(\tau)] \quad \forall t \in [0, \infty), \tau \in [0, 2\pi]. \quad (1.167a)$$

В учебниках и справочниках [21, 51] недостаточно обдуманно приводятся координатные функции этой фигуры, имеющие при $a=b=c=1$ и принятой на рис. 1.142,д ориентации вид $x(t, \tau) = t \operatorname{ch}(\tau)$, $y(t, \tau) = t^2$, $z(t, \tau) = t \operatorname{sh}(\tau)$ и описывающие поверхность лишь в верхнем полупространстве $y \geq 0$. Правильнее задавать верхние и нижние пары полостей отдельными выражениями:

$$p(t, \tau) = \begin{cases} [t \operatorname{ch}(\tau) \quad t^2 \quad t \operatorname{sh}(\tau)] & \text{— верхние левая и правая;} \\ [t \operatorname{sh}(\tau) \quad -t^2 \quad t \operatorname{ch}(\tau)] & \text{— нижние передняя и задняя.} \end{cases} \quad (1.167б)$$

Эти полости соединяются друг с другом лишь в пределах $t \rightarrow \pm\infty$ на вычислительно недоступных асимптотических прямых $|x/a| = |z/c|$, что не позволяет изобразить всю поверхность как единое целое (рис. 1.143). Следовательно, модель гиперболического параболоида (1.167б) является неудачной и во всём проигрывает модели (1.167а);

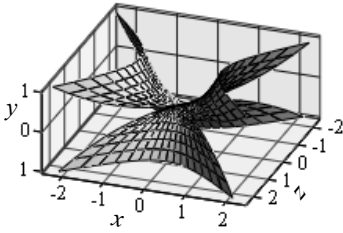


Рис. 1.143

• **эллиптический параболоид** (рис. 1.142,е) с неявным уравнением $(x/a)^2 - y/b + (z/c)^2 = 0$:

$$p(t, \tau) = [at \sin(\tau) \quad bt^2 \quad ct \cos(\tau)] \quad \forall t \in [0, \infty), \tau \in [0, 2\pi]; \quad (1.168)$$

• **эллиптический цилиндр** $(x/a)^2 + (z/c)^2 - 1 = 0$ (рис. 1.142,ж):

$$p(t, \tau) = [a \sin(\tau) \quad t \quad c \cos(\tau)] \quad \forall t \in (-\infty, \infty), \tau \in [0, 2\pi]; \quad (1.169)$$

• две полости **гиперболического цилиндра** (рис. 1.142,з), описываемые единым неявным уравнением $(x/a)^2 - (z/c)^2 - 1 = 0$:

$$p(t, \tau) = [\pm a \operatorname{ch}(\tau) \quad t \quad c \operatorname{sh}(\tau)] \quad \forall t \in (-\infty, \infty), \tau \in (-\infty, \infty). \quad (1.170)$$

Правой полости соответствует знак «+», а левой знак «-»;

• **параболический цилиндр** $(x/a)^2 - z/c = 0$ (рис. 1.142,и):

$$p(t, \tau) = [a\tau \quad t \quad c\tau^2] \quad \forall t \in (-\infty, \infty), \tau \in (-\infty, \infty). \quad (1.171)$$

На всех графиках квадратичных поверхностей проставлены знаки неявной функции $f(p)$ в областях её знакопостоянства.

Гораздо более сложен поиск подстановок для произвольного уравнения второго порядка (1.158а). Данная задача решается в гл. 4 путём расчёта матрицы геометрических преобразований канонической формы поверхности в заданную форму общего вида.

Основными трудностями метода подстановок в задаче синтеза параметрических моделей квадратичных поверхностей являются:

• заранее непредсказуемая ориентация каркасной сетки $\{p(t_i, \tau), p(t, \tau_j)\}$, выясняемая только после фактического построения поверхности;

- неочевидность выбора интервалов изменения параметров t и τ , обеспечивающих *существование* и *единственность* поверхности в желаемой области;
- невозможность описания двуполостной поверхности одной векторной функцией из-за знакопостоянства функций $\text{ch}(t) > 0$ и $t^2 \geq 0$;
- многовариантность выбора функций подстановки, обеспечивающих существование и вычислимость вектора нормали $N(t, \tau) = p'_t(t, \tau) \times p'_\tau(t, \tau)$ во *всех* регулярных точках поверхности.

2. *Наложением одновременных движений точки вдоль координатных направлений* из определённого начального положения, что составляет кинематическую сущность **метода суперпозиции**. Например, совмещение гармонических колебаний $x(t) = r \sin(t)$ и $z(t) = r \cos(t)$ с амплитудой r вдоль осей x и z с независимым от них равномерным движением $y(\tau) = \tau$ вдоль оси y даёт боковую поверхность **круглого цилиндра** радиуса r , описываемую параметрической моделью $p(t, \tau) = [r \sin(t) \quad \tau \quad r \cos(t)]$ (рис. 1.144, а). Если же вертикальное движение сделать зависимым от параметра t , например, по линейному закону $y(t) = bt$, то получится графический объект с одной степенью свободы — **цилиндрическая винтовая линия** $p(t) = [r \sin(t) \quad bt \quad r \cos(t)]$ радиуса r с шагом навивки $h = 2\pi b$.

Разновидностью суперпозиции является **метод модуляции высоты**, удобный для получения параметрических моделей поверхностей

$$p(t, \tau) = [t \quad y(t, \tau) \quad \tau] \quad (1.172a)$$

с явной функцией $y(x, z)$, модулирующей высоту рельефа в каждой точке прямоугольного горизонтального плана с независимыми координатами $x(t, \tau) = t$ и $z(t, \tau) = \tau$. Например, волнистая поверхность с гармоническими колебаниями, затухающими по гауссовскому закону, создаётся модулирующей функцией

$$y(x, z) = A e^{-\gamma_x x^2 - \gamma_z z^2} \cos(\omega_x x + \varphi_x) \cos(\omega_z z + \varphi_z), \quad (1.172б)$$

параметры которой имеют следующее назначение:

- пары $\{\omega_x, \varphi_x\}$ и $\{\omega_z, \varphi_z\}$ задают частоты и фазовые сдвиги гармонических колебаний высоты вдоль горизонтальных осей x и z . Полезно помнить, что частоте ω соответствует период колебаний $2\pi/\omega$;

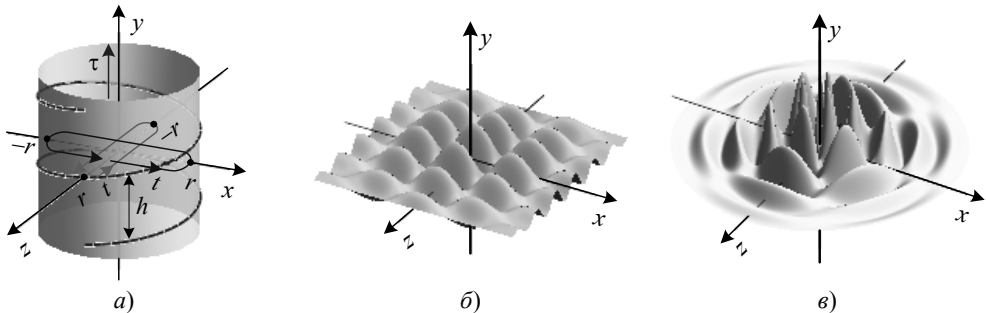


Рис. 1.144

- коэффициент A задаёт максимальное значение экспоненты и амплитуду колебаний в начале координат $x=z=0$;
- коэффициенты $\gamma_x \geq 0$ и $\gamma_z \geq 0$ регулируют затухание колебаний по нормальному закону вдоль каждой из горизонтальных осей координат.

Поверхность, построенная на рис. 1.144,б в интервалах $-2\pi \leq x \leq 2\pi$ и $-2\pi \leq z \leq 2\pi$, создана модулирующей функцией (1.172б) с параметрами $A=2$, $\omega_x=1$, $\varphi_x=0$, $\omega_z=3$, $\varphi_z=\pi/2$, $\gamma_x=0.02$ и $\gamma_z=0.05$.

При конструировании поверхности, обладающей радиальной симметрией на плоскости xz , логично использовать цилиндрические координаты ρ , φ и модулирующую функцию высоты

$$y(\rho, \varphi) = A \exp(-\gamma \rho^2) \cos(\omega_\rho \rho) \cos(\omega_\varphi \varphi). \quad (1.172в)$$

Преобразование цилиндрических координат в декартовы производится по (1.3а) как $x(\rho, \varphi) = \rho \sin(\varphi)$ и $z(\rho, \varphi) = \rho \cos(\varphi)$. Образец рельефа, заданного функцией (1.172в) с параметрами $A=8$, $\gamma=0.02$, $\omega_\rho=2$ и $\omega_\varphi=3$, показан на рис. 1.144,в.

Более обстоятельное изучение кинематического метода построения моделей параметрических поверхностей будет продолжено в разд. 4.2.

Реалистичное изображение поверхности на экране дисплея — достаточно сложная задача, состоящая в передаче объёмной формы на плоскости посредством правильного расчёта освещённости видимых точек. Упрощенное изображение поверхности заключается в построении на экране её **каркаса** — совокупности линий, изгибы которых создают ощущение формы поверхности. Для этого производится **квантование** непрерывных параметров

$$t = \{t_0, t_1, \dots, t_n\}, \quad \tau = \{\tau_0, \tau_1, \dots, \tau_m\} \quad (1.173)$$

и выполняются два вложенных цикла поочерёдного вывода на экран двух пучков каркасных линий, которые можно условно назвать:

- **продольными** t -линиями $\{p(t, \tau_j) \mid \forall t \in [t_0, t_n]\} \forall j = \overline{0, m}$, проходящими вдоль изменения параметра t ;
- **поперечными** τ -линиями $\{p(t_i, \tau) \mid \forall \tau \in [\tau_0, \tau_m]\} \forall i = \overline{0, n}$, проходящими вдоль изменения параметра τ .

Система каркасных линий образует **полигональную сеть**, состоящую из четырёхугольных **ячеек**, которые при совпадении двух смежных вершин становятся треугольными, как на полюсах глобуса. В простейшем методе визуализации ячейки считаются прозрачным, а все каркасные линии видимыми, из-за чего может возникать неоднозначность восприятия формы поверхности.

Наиболее простым методом показа непрозрачной поверхности является **плоская закрашка** всех точек видимой ячейки постоянным цветом, соответствующим освещённости усреднённой точки ячейки, например, её центра. Существуют и более сложные методы окрашивания ячеек поверхностей в зависимости от их расположения относительно наблюдателя и источников света. Некоторые из этих методов будут рассмотрены в п. 4.2.3.

1.5.11. Геометрические преобразования неявных поверхностей

Представление поверхностей в виде неявных уравнений с векторным $f(p)=0$ либо скалярными $f(x,y,z)=0$ переменными востребовано во многих задачах компьютерной графики. К ним, несомненно, относится расчёт пересечения неявной поверхности с прямой $q+U\theta$, сводящийся к решению **скалярного** уравнения $f(q+U\theta)=0$ относительно параметра θ . Применительно к плоской поверхности это сделано в (1.130а). При параметрическом же описании поверхности векторной функцией $p(t,\tau)$ приходится решать более сложное векторное нелинейное уравнение $p(t,\tau)=q+U\theta$ относительно трёх неизвестных параметров t , τ и θ , что чаще всего возможно лишь численными методами.

Другое ценное свойство неявной формы **самонепересекающейся** поверхности заключается в простоте локализации относительно неё любой точки q . Индикатором её расположения на поверхности служит значение $f(q)=0$. Для двух точек a и b знак произведения $c=f(a)f(b)$ сигнализирует об одностороннем (при $c>0$) либо разностороннем (при $c<0$) их расположении.

Выбор неявного описания поверхностей весьма ограничен. Без дополнительных трудоёмких изысканий быстро составляются координатные функции $f(x,y,z)$, пожалуй, лишь для плоскостей и канонических форм поверхностей второго порядка (см. табл. 1.3, 1.4). Расширим возможности получения неявных функций поверхностей $\phi(x,y,z)$ **геометрическими преобразованиями** некоторых графических объектов, также заданных в НФ.

Вначале рассмотрим, как преобразования поверхности изменяют её неявную модель. Формирование функции ϕ производится **методом обратного преобразования** аргументов функции f (см. п. 1.1.9):

- параллельный **перенос** всех точек поверхности на один и тот же вектор $B=[b_x \ b_y \ b_z]$ даёт новые координаты $x'=x+b_x$, $y'=y+b_y$, $z'=z+b_z$ и НФ

$$\phi(x,y,z)=f(x-b_x,y-b_y,z-b_z); \quad (1.174a)$$

- **масштабирование** (растяжение, сжатие, отражение) всех точек поверхности вдоль координатных осей с ненулевыми коэффициентами m_x , m_y и m_z даёт преобразованные координаты $x'=m_x x$, $y'=m_y y$, $z'=m_z z$ и новую функцию

$$\phi(x,y,z)=f(x/m_x,y/m_y,z/m_z); \quad (1.174б)$$

- три преобразования **вращения** поверхности вокруг координатных осей:

- ◊ при вращении вокруг оси x на угол φ_x координаты точки $[x \ y \ z]$ изменяются по формулам $x'=x$, $y'=y\cos(\varphi_x)-z\sin(\varphi_x)$ и $z'=y\sin(\varphi_x)+z\cos(\varphi_x)$, а неявная функция поверхности превращается в

$$\phi(x,y,z)=f(x,y\cos(\varphi_x)+z\sin(\varphi_x),-y\sin(\varphi_x)+z\cos(\varphi_x)); \quad (1.174в)$$

- ◊ вращение вокруг оси y изменяет координаты точки по формулам $x'=x\cos(\varphi_y)+z\sin(\varphi_y)$, $y'=y$ и $z'=-x\sin(\varphi_y)+z\cos(\varphi_y)$, а неявная функция поверхности принимает вид

$$\phi(x, y, z) = f(x \cos(\varphi_y) - z \sin(\varphi_y), y, x \sin(\varphi_y) + z \cos(\varphi_y)); \quad (1.174\Gamma)$$

◇ после поворота поверхности вокруг оси z координаты её точек становятся равными $x' = x \cos(\varphi_z) - y \sin(\varphi_z)$, $y' = x \sin(\varphi_z) + y \cos(\varphi_z)$ и $z' = z$, а неявная функция поверхности преобразуется в

$$\phi(x, y, z) = f(x \cos(\varphi_z) + y \sin(\varphi_z), -x \sin(\varphi_z) + y \cos(\varphi_z), z). \quad (1.174\Delta)$$

Если над поверхностью $f(x, y, z) = 0$ совершается **цепочка преобразований**, каждое из которых изменяет её неявную функцию, то конечная НФ $\phi(x, y, z)$ находится **методом суперпозиции**. Возьмём, например, центрированную сферу единичного радиуса $x^2 + y^2 + z^2 - 1 = 0$ и выполним следующую цепочку преобразований (рис. 1.145):

- **растяжение** вдоль оси x с масштабным коэффициентом $m_x = 2$ при $m_y = m_z = 1$ даёт по (1.174Б) неявное уравнение $0.25x^2 + y^2 + z^2 - 1 = 0$;

- **вращение** полученного эллипсоида вокруг оси y на угол $\varphi_y = 90^\circ$ изменяет неявную функцию поверхности согласно (1.174Г):

$$0.25(0x - 1z)^2 + y^2 + (1x + 0z)^2 - 1 = x^2 + y^2 + 0.25z^2 - 1;$$

- **перенос** вдоль оси x на расстояние $b_x = 2$ даёт по (1.174А) конечную НФ:

$$\phi(x, y, z) = (x - 2)^2 + y^2 + 0.25z^2 - 1 = x^2 + y^2 + 0.25z^2 - 4x + 3.$$

Проверка $\phi(2, 0, 2) = 0$ доказывает, что точка $q = [2 \ 0 \ 2]$ действительно лежит на поверхности конечного эллипсоида, что ясно видно на последнем графике.

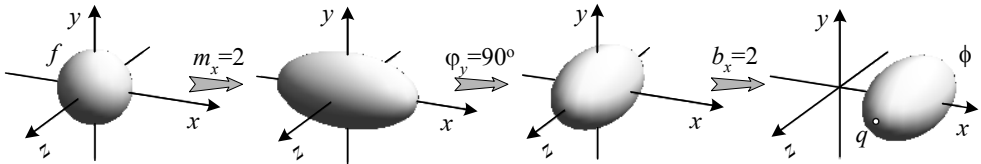


Рис. 1.145

Аналогичным преобразованиям может быть подвергнута поверхность, описываемая явным координатным уравнением, например, $y = \psi(x, z)$, которому подходит неявная функция $f(x, y, z) = \psi(x, z) - y$.

На очереди задачи синтеза неявных функций $\phi(x, y, z)$ поверхностей, замечаемых движением линий, заданных в координатных плоскостях неявными функциями $\zeta(x, y)$, $\psi(x, z)$ и $\xi(y, z)$.

□ Поверхность, образованная **прямыми**, проходящими через некоторую линию в направлении постоянного вектора V , называется **цилиндрической**. Допустим, в плоскости $z = 0$ неявно определена линия $\zeta(x, y) = 0$. Взяв на ней точку $p = [x \ y \ 0]$, переместим её в положение $p' = p + Vt$ (рис. 1.146) с координатами

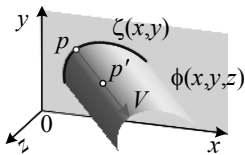


Рис. 1.146

$$x' = x + V_x t, \quad y' = y + V_y t, \quad z' = V_z t.$$

Подставив в первые две координаты $t=z'/V_z$, получим

$$x'=x+(V_x/V_z)z', \quad y'=y+(V_y/V_z)z'.$$

По принципу обратного преобразования получим НФ поверхности в виде

$$\phi(x,y,z)=\zeta\left(x-(V_x/V_z)z, y-(V_y/V_z)z\right) \text{ при } V_z \neq 0. \quad (1.175a)$$

При нарушении условия $V_z \neq 0$ вектор V становится параллелен плоскости xu определения направляющей линии $\zeta(x,y)=0$, а поверхность с ней совпадает.

Аналогично выводятся НФ цилиндрических поверхностей, образованных параллельным переносом координатных линий $\psi(x,z)=0$ и $\xi(y,z)=0$:

$$\phi(x,y,z)=\psi\left(x-(V_x/V_y)y, z-(V_z/V_y)y\right) \text{ при } V_y \neq 0; \quad (1.175б)$$

$$\phi(x,y,z)=\xi\left(y-(V_y/V_x)x, z-(V_z/V_x)x\right) \text{ при } V_x \neq 0. \quad (1.175в)$$

□ Поверхность, образованная **прямыми**, проходящими через некоторую линию и неподвижную точку, называется **конической**. Задав в плоскости $z=0$ неявную линию $\zeta(x,y)=0$ и двигая её точку $p=[x \ y \ 0]$ по прямой $p+(o-p)t$ до точки o , получим координаты точки конической поверхности p' (рис. 1.147):

$$x'=x+(o_x-x)t, \quad y'=y+(o_y-y)t, \quad z'=o_z t.$$

Подставим параметр $t=z'/o_z$ в первые две координаты:

$$x'=x+(o_x-x)z'/o_z, \quad y'=y+(o_y-y)z'/o_z.$$

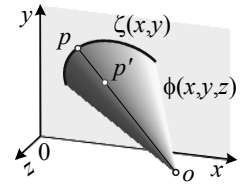


Рис. 1.147

Обратное преобразование $p' \rightarrow p$ даёт НФ конической поверхности:

$$\phi(x,y,z)=\zeta\left(\frac{o_z x - o_x z}{o_z - z}, \frac{o_z y - o_y z}{o_z - z}\right) \text{ при } o_z \neq 0. \quad (1.176a)$$

Аналогично выводятся НФ конических поверхностей, образованных движением прямых по координатным линиям $\psi(x,z)=0$ и $\xi(y,z)=0$ через точку o :

$$\phi(x,y,z)=\psi\left(\frac{o_y x - o_x y}{o_y - y}, \frac{o_y z - o_z y}{o_y - y}\right) \text{ при } o_y \neq 0; \quad (1.176б)$$

$$\phi(x,y,z)=\xi\left(\frac{o_x y - o_y x}{o_x - x}, \frac{o_x z - o_z x}{o_x - x}\right) \text{ при } o_x \neq 0. \quad (1.176в)$$

Составим для примера неявную модель боковой поверхности прямого кругового конуса с высотой h и радиусом r основания окружности, лежащей в плоскости xz (рис. 1.148). Составим уравнение линии основания $\psi(x,z)=x^2+z^2-r^2=0$, зададим точку вершины $o=[0 \ h \ 0]$ и по (1.176б) получим неявную функцию

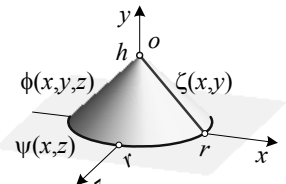


Рис. 1.148

$$\phi(x,y,z)=\left(\frac{hx}{h-y}\right)^2 + \left(\frac{hz}{h-y}\right)^2 - r^2 = \frac{h^2 x^2 - r^2 (y-h)^2 + h^2 z^2}{(y-h)^2},$$

альтернативную неявному уравнению поверхности конуса (ср. с табл. 1.3)

$$(x/r)^2 - (y/h-1)^2 + (z/r)^2 = 0.$$

□ Поверхность, образованная **вращением** линии вокруг некоторой прямой, например, любой из координатных осей, называется **циклической**. Допустим, в плоскости $z=0$ задана неявная линия $\zeta(x,y)=0$. Повернём её точку $p=[x \ y \ 0]$ вокруг оси x по дуге радиуса y в положение $p'=[x \ y' \ z']$ (рис. 1.149). Из равенства радиусов $y=\sqrt{y'^2+z'^2}$ по принципу обратного преобразования получим неявную функцию циклической поверхности в виде $\phi(x,y,z)=\zeta\left(x,\sqrt{y^2+z^2}\right)$.

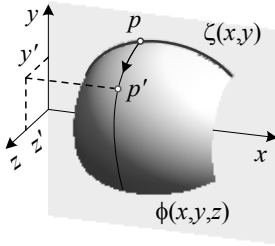


Рис. 1.149

Аналогично создаётся неявная функция $\phi(x,y,z)=\zeta\left(\sqrt{x^2+z^2},y\right)$ поверхности вращения линии $\zeta(x,y)=0$ вокруг оси y и ещё две пары неявных функций поверхностей вращения неявных линий $\psi(x,z)=0$ и $\xi(y,z)=0$. Все шесть вариантов НФ циклических поверхностей сведены в табл. 1.5.

Таблица 1.5

Неявная линия	Ось вращения		
	x	y	z
$\zeta(x,y)=0$	$\zeta\left(x,\sqrt{y^2+z^2}\right)$	$\zeta\left(\sqrt{x^2+z^2},y\right)$	—
$\psi(x,z)=0$	$\psi\left(x,\sqrt{y^2+z^2}\right)$	—	$\psi\left(\sqrt{x^2+y^2},z\right)$
$\xi(y,z)=0$	—	$\xi\left(y,\sqrt{x^2+z^2}\right)$	$\xi\left(\sqrt{x^2+y^2},z\right)$

К примеру, поверхность **тора** замечается при вращении образующей окружности радиуса r с неявной функцией

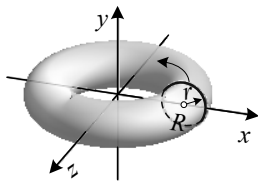


Рис. 1.150

$\zeta(x,y)=(x-R)^2+y^2-r^2$ вокруг оси y по осевой линии радиуса R (рис. 1.150). По табл. 1.5 неявная функция поверхности тора получается заменой аргумента x в функции $\zeta(x,z)$ выражением $\sqrt{x^2+z^2}$:

$$\phi(x,y,z)=\left(\sqrt{x^2+z^2}-R\right)^2+y^2-r^2.$$

Изображённую на рис. 1.148 боковую поверхность **конуса** также можно получить вращением вокруг оси y прямой с НФ $\zeta(x,y)=x/r+y/h-1$. По табл. 1.5 заменой $x \rightarrow \sqrt{x^2+z^2}$ в функции $\zeta(x,z)$ получаем неявную функцию

$$\phi(x,y,z)=\sqrt{x^2+z^2}/r+y/h-1,$$

альтернативную приведённому выше неявному уравнению поверхности конуса

$$(x/r)^2 - (y/h-1)^2 + (z/r)^2 = 0.$$

1.6. ОТСЕЧЕНИЕ В ПРОСТРАНСТВЕ

Обобщим методы отсечения на плоскости, изученные в разд. 1.4, на трёхмерное пространство. Здесь собрана небольшая коллекция алгоритмов объёмного отсечения, полезных при решении различных задач трёхмерной компьютерной графики. В отличие от тестов, отсечение требует полного расчёта пересечения объектов друг с другом.

Одной из задач, где выполняется объёмное отсечение, является показ на экране видимых элементов сцены. Наиболее распространённые формы 3d-отсекателей (рис. 1.151) — *призма* (а) и *пирамида* (б) *видимости* — возник-

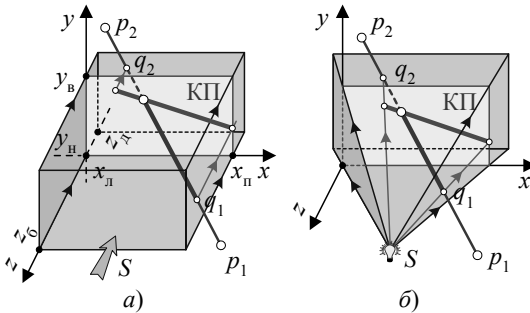


Рис. 1.151

ают, соответственно, при параллельном и центральном проектировании. В сечении этих полиэдров картинной плоскостью (КП) образуется окно, куда помещается лишь *часть* сцены, находящаяся внутри отсекаателя. Боковые грани отсекаателя ограничивают наблюдателю обзор по сторонам пространства, а передняя и задние грани могут не участвовать в отсечении, т. е. быть пассивными.

Цель объёмного отсечения — выявление активных объектов на ранней стадии расчёта сцены до её проектирования на картинную плоскость. Это существенно экономит машинные ресурсы и не загружает компьютер бесполезной работой с заведомо невидимыми объектами.

Второе применение методы 3d-отсечения находят в выполнении логических операций со сплошными телами A и B (см. 2d-аналог на рис. 1.83):

- границы объекта **пересечения** $A \cap B$ образуются внутренними фрагментами границ объектов A и B относительно друг друга и общими фрагментами границ их внутреннего касания;
- границы объекта **исключения** $A - B$ состоят из внешних фрагментов границ тела A относительно тела B , внутренних фрагментов границ тела B относительно A и общих фрагментов границ их внешнего касания;
- границы объекта **объединения** $A \cup B$ включают внешние фрагменты границ объектов A и B относительно друг друга и общие фрагменты границ внутреннего касания объектов.

Рассмотрим последовательно задачи 3d-отсечения отрезка прямой линии, плоского полигона и полиэдра. Взяв за основу алгоритмы плоского отсечения, внесём в них необходимые изменения. Для работы с полиэдрами нам потребуются определённые в (1.143) и (1.144) функции $side(P, G, i)$ и $norm(P)$, позволяющие по (1.145) вычислить нормаль к i -й грани полиэдра $H = \{P, G\}$ как $N = norm(side(P, G, i))$.

1.6.1. Отсечение отрезков

Операции отсечения отрезков различными объёмными объектами относятся к наиболее востребованным расчётным задачам компьютерной 3d-геометрии [50, 53, 58]. Результатом отсечения является список отрезков, из которых собираются полигоны граней полиэдров, являющихся решением задач 3d-отсечения графических объектов.

Регулярное и выпуклое отсечение отрезка

Простейшим **регулярным 3d-отсекателем** является параллелепипед с гранями, параллельными координатным плоскостям (рис. 1.151,а). Перечислим изменения, которые нужно внести в функции регулярного 2d-отсечения (см. рис. 1.75) для получения **алгоритмов внешнего** $reg_outclip3(p_1p_2, O, G, Ls, *)$ и **внутреннего** $reg_inclip3(p_1p_2, O, G, Ls, *)$ **отсечений отрезка p_1p_2 регулярным параллелепипедом** $\{O, G\}$, заданным списком вершин $O = \{o_1, o_2, \dots, o_8\}$ и группой списков граней $G = \{G_1, G_2, \dots, G_6\}$ с порядком обхода «нижняя $\rightarrow$ правая $\rightarrow$ верхняя $\rightarrow$ левая $\rightarrow$ ближняя $\rightarrow$ дальняя»:

- описать положение точки $p = [x \ y \ z]$ относительно шести граней отсекателя логическим кодом $C = c_1 \dots c_4 c_5 c_6$, четыре первых разряда которого определены в (1.80б), а два последних разряда

$$c_5 = \text{sgn}(z - z_6), \quad c_6 = \text{sgn}(z_d - z) \quad (1.177)$$

ориентируют точку по глубине параллелепипеда относительно его ближней и дальней граней с аппликатами z_6 и z_d :

- заменить цикл $i = 1 \dots 4$ на $i = 1 \dots 6$;
- заменить отрезок $o_i o_{i+1}$ на полигон i -й грани $P_i = \text{side}(O, G, i)$;
- для оценки расположения и расчёта пересечения отрезка p_1p_2 с гранью P_i использовать функцию $\text{cross_side}(p_1p_2, P_i, t, q)$, возвращающую в своём имени признак их взаимного расположения (-1 , 0 либо 1), а в аргументе q — точку пересечения.

Расширим метод логических кодов также и на задачу отсечения отрезка p_1p_2 **выпуклым** полиэдром $\{O, G\}$, заданным списком вершин $O = \{o_1, o_2, \dots, o_n\}$ и группой списков граней $G = \{G_1, G_2, \dots, G_m\}$, путём небольших изменений алгоритмов регулярного отсечения:

- заменить цикл $i = 1 \dots 6$ на $i = 1 \dots m$;
- вместо (1.80б) и (1.177) формировать m -разрядный логический код точки p с помощью определённой в (1.108д) неявной функции i -й грани:

$$c_i = \text{sgn}(nfP(P_i, p)) \quad \forall i = \overline{1, m}. \quad (1.178)$$

Правильные значения разрядов обеспечиваются при **внешней** ориентации нормали грани, для чего **вершины в списке G_i должны быть пронумерованы в положительном направлении** (см. п. 1.5.4).

В итоге получаем алгоритмы внешнего $\text{conv_outclip3}(p_1p_2, O, G, Ls, *)$ и внутреннего $\text{conv_inclip3}(p_1p_2, O, G, Ls, *)$ отсечений отрезка p_1p_2 выпуклым полиэдром $\{O, G\}$ с соответствующими символами $* \in \{<, \leq\}$ (рис. 1.152):

- $\leq$ — строго внешнее и внутренне-граничное отсечение;
- $<$ — внешне-граничное и строго внутреннее отсечение.

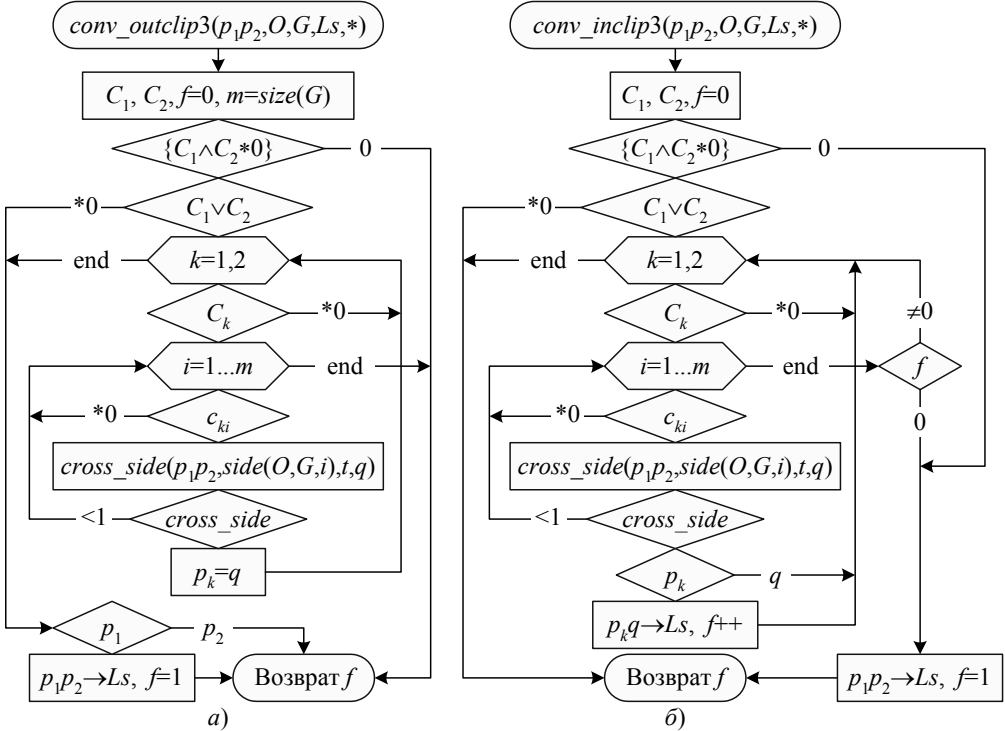


Рис. 1.152

Обе функции возвращают в своих именах число видимых фрагментов отрезка p_1p_2 , добавленных в возвращаемый список Ls . Если отрезок совпадает с плоскостью какой-либо грани (при этом возвращается $\text{cross_side}=-1$), то решать задачу плоского отсечения отрезка контуром этой грани не требуется, т. к. прямая отрезка в этом случае пересекает другие, не параллельные ей грани.

Возможен второй подход к решению задачи внешнего отсечения, основанный на использовании теста локализации. Он реализован алгоритмом $\text{clip3_test}(ab, O, G, Ls, *)$ внешнего отсечения отрезка ab выпуклым полиэдром $\{O, G\}$ (рис. 1.153), устроенным аналогично 2d-алгоритму clip2_test (см. рис. 1.78) и возвращающим такие же результаты, в связи с чем он здесь подробно не комментируется. Отметим лишь отличия 3d-алгоритма:

- в цикле $i=1, \text{size}(G)$ перебираются не рёбра полигона, а грани полиэдра;
- пересечение отрезка ab с i -й гранью $P=\text{side}(O, G, i)$ рассчитывается функцией $\text{cross_side}(ab, P, t, c)$, возвращающей в случае пересечения признак $\text{cross_side}=1$ и параметр $t \in [0, 1]$;

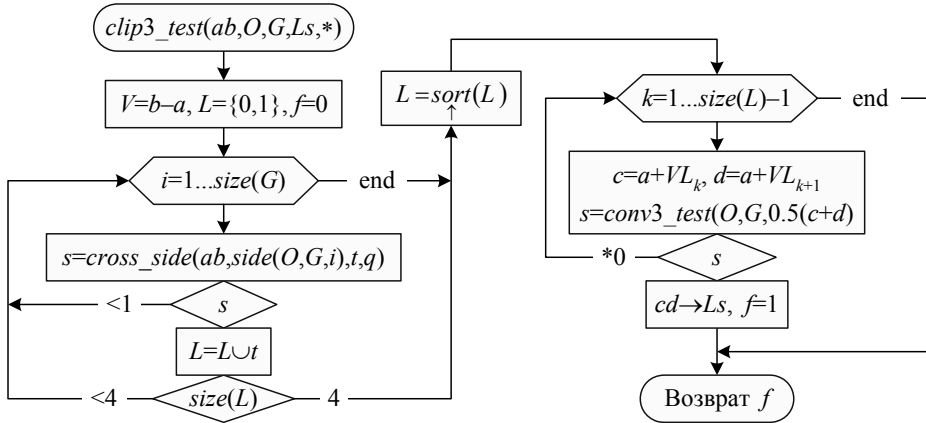


Рис. 1.153

• используется определённый в (1.149) 3d-тест $conv3_test$ локализации точки относительно выпуклого полиэдра (см. рис. 1.130,а).

Отсечение отрезка произвольным полиэдром

Несмотря на то, что алгоритм $clip3_test$ имеет большую трудоёмкость в сравнении с алгоритмом $conv_outclip3$, его идейная основа лучше подходит для решения других задач отсечения (рис. 1.154):

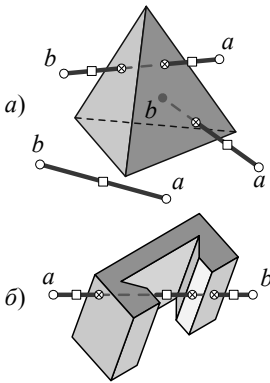


Рис. 1.154

• при **внутренне-граничном** отсечении отрезка **выпуклым** полиэдром (а) отбор видимых фрагментов производится по условию их **внешней** локализации $conv3_test(P, G, m_k) > 0$. Таких фрагментов может быть несколько (весь отрезок лежит внутри или на границе полиэдра), один (один конец отрезка лежит внутри, а другой — снаружи полиэдра) или два (средняя часть отрезка погружена в полиэдр);

• при отсечении отрезка **невыпуклым** полиэдром (б) нужно вместо выпуклого теста $conv3_test$ использовать лучевой тест $ray3_test$, единственно способный определить положение точки относительно **произвольного** полиэдра. При внешне-граничном отсечении отбираются фрагменты, средние точки которых лежат внутри полиэдра, а при внутренне-граничном — фрагменты со средними точками вне полиэдра.

Составим, наконец, **универсальный алгоритм** $clip3_segm(ab, O, G, Ls, *)$ **объёмного *-отсечения отрезка ab произвольным полиэдром** $\{O, G\}$, накапливающий видимые фрагменты $cd \subseteq ab$ в возвращаемом списке отрезков Ls . Блок-схема алгоритма, схожего с $clip2_segm$, приведена на рис. 1.155. В зависимости от символьного аргумента $* \in \{>, <, \geq, \leq, =, \neq\}$, задающего **условие отсечения** $s \neq 0$ по результату $s \in \{-1, 0, 1\}$ лучевого теста локализации средней точки фрагмента, алгоритм выполняет следующие **типы отсечений**:

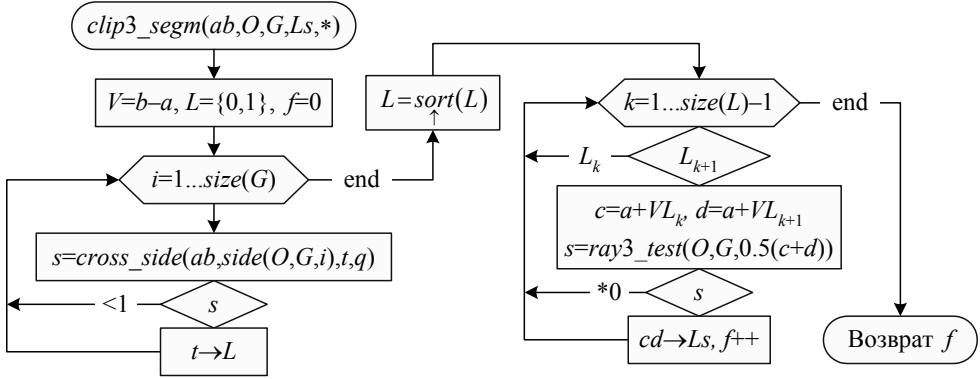


Рис. 1.155

- $>$ — внешнее;
- $<$ — внутреннее;
- $\geq$ — внешне-граничное;
- $\leq$ — внутренне-граничное;
- $=$ — граничное;
- $\neq$ — внеграничное.

Фрагменты cd , не удовлетворяющие условию $s \neq 0$, добавляются в возвращаемый список отрезков Ls . Перечислим другие отличия алгоритма $clip3_segm$ от $clip3_test$:

- логическое добавление параметра t в точку пересечения отрезка ab с гранью $side(O, G, i)$ заменено его добавлением в конец списка L с учётом будущего пропуска k -го фрагмента нулевой длины с концевыми параметрами $L_k = L_{k+1}$;
- исключено ограничение размера списка L в четыре элемента, т. к. **невыпуклый** полиэдр может пересекаться с отрезком более чем в двух точках;
- вместо выпуклого теста локализации $conv3_test$ применён универсальный **лучевой** тест $ray3_test$. Его результат $s \in \{-1, 0, 1\}$ используется для принятия решения об отсечении либо сохранении фрагмента $cd \subseteq ab$.

Статус аргумента Ls как входного, так и выходного позволяет многократно накапливать в одном списке не отсечённые полиэдром фрагменты разных отрезков. **Перед первым обращением к функции $clip3_segm$ нужно инициализировать пустой список $Ls = \emptyset$.**

Ради единообразия обозначений введём синонимы алгоритмов внешне-граничного, внутренне-граничного и внеграничного отсечений отрезка ab произвольным полиэдром $\{O, G\}$:

$$\begin{aligned}
 outclip3(ab, O, G, Ls) &\equiv clip3_segm(ab, O, G, Ls, \geq), \\
 inclip3(ab, O, G, Ls) &\equiv clip3_segm(ab, O, G, Ls, \leq), \\
 neclip3(ab, O, G, Ls) &\equiv clip3_segm(ab, O, G, Ls, \neq).
 \end{aligned}$$

1.6.2. Отсечение полигонов и полиэдров

Завершим изучение темы 3d-отсечения двумя задачами, в которых объектами отсечения полиэдром выступают полигон и полиэдр.

На основе определённых выше функций *outclip3* и *inclip3*, а также функции *cross_poly*, блок-схема которой приведена в п. 1.5.6 (см. рис. 1.129,з), составим алгоритмы внешнего *outclip3_poly(P,O,G,U,F)* либо внутреннего *inclip3_poly(P,O,G,U,F)* отсечений плоского полигона $P=p_1p_2\dots p_np_1$ произвольным полиэдром $\{O,G\}$, заданным списком координат вершин O и группой списков граней G . Последними аргументами функции являются координатный список вершин U и группа списков граней F , в которые должны добавляться данные о неотсечённых фрагментах полигона P .

Шаг 1. Инициализация пустого списка $Ls=\emptyset$ и накопление в нём отрезков пересечения полигона P с каждой **некомпланарной** гранью полиэдра O :

$$cross_poly(P, side(O,G,i), Ls, 0) \quad \forall i=\overline{1, size(G)}.$$

Шаг 2. Добавление в список Ls результатов внешнего либо внутреннего отсечений рёбер P полиэдром $\{O,G\}$ соответствующими функциями

$$outclip3(p_ip_{i+1}, O, G, Ls) \text{ либо } inclip3(p_ip_{i+1}, O, G, Ls) \quad \forall i=\overline{1, size(P)-1}.$$

Шаг 3. Формирование видимых контуров в списке $LP=cont(Ls)$.

Шаг 4. Добавление данных о полигонах, составляющих список LP , в списки U и F функцией *addLP(LP,U,F)* и **возврат** LP . $\square$

Результат отсечения — видимые фрагменты полигона P , возвращаемые списком LP в именах функций. Одновременно в аргументах U и F возвращаются координаты и индексы вершин этих фрагментов, добавленные алгоритмом на **шаге 4** с помощью специально предназначенной для этого функции *addLP*.

Разработаем алгоритм *addLP(LP,U,F)* добавления координат и индексов

вершин полигонов списка LP в списки вершин $U=\{u_1,\dots,u_n\}$ и граней $F=\{F_1,\dots,F_m\}$ (рис. 1.156).

В цикле перебора полигонов

$$LP_i \equiv P = p_1p_2\dots p_l p_1 \quad \forall i=\overline{1, size(LP)}$$

добавляются следующие данные:

- в координатный список U — **некратные**, т. е. отсутствующие в нём точки $p_j \notin U \quad \forall j=\overline{1, l}$;

- в группу F — списки граней F_i , сформированные из индексов элементов списка U , представляющих вершины полигона P .

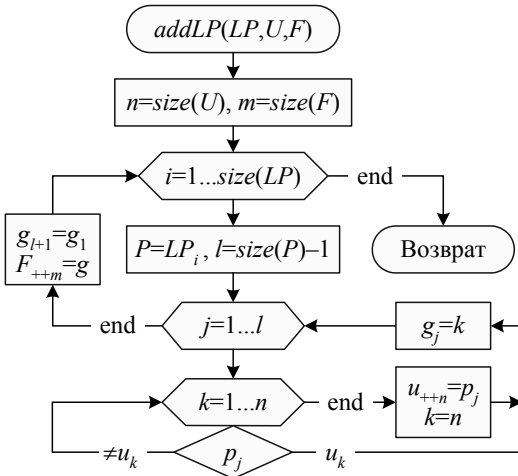


Рис. 1.156

Перед началом накопления координат и индексов вершин списки U и F должны быть инициализированы пустыми либо нужными данными.

Пример 1.20. Рассчитать внешнее и внутреннее отсечения треугольника $P=p_1p_2p_3p_1$ с вершинами $p_1=[0 \ 5 \ 0]$, $p_2=[1 \ 1 \ 5]$ и $p_3=[8 \ 0 \ 1]$ невыпуклым полиэдром $\Theta=\{O, G\}$ с вершинами

$$o_1=[0 \ 0 \ 0], \quad o_2=[4 \ 0 \ 0], \quad o_3=[4 \ 0 \ 2], \quad o_4=[2 \ 0 \ 2], \quad o_5=[2 \ 0 \ 6], \quad o_6=[0 \ 0 \ 6], \\ o_7=[0 \ 2 \ 0], \quad o_8=[0 \ 2 \ 6], \quad o_9=[2 \ 2 \ 6], \quad o_{10}=[2 \ 2 \ 2], \quad o_{11}=[4 \ 2 \ 2], \quad o_{12}=[4 \ 2 \ 0]$$

и топологической группой $G=\{G_1, \dots, G_8\}$ со следующими списками граней (рис. 1.157):

$$G_1=\{1, 2, 3, 4, 5, 6, 1\}, \quad G_2=\{7, 8, 9, 10, 11, 12, 7\}, \quad G_3=\{6, 5, 9, 8, 6\}, \quad G_4=\{5, 4, 10, 9, 5\}, \\ G_5=\{4, 3, 11, 10, 4\}, \quad G_6=\{3, 2, 12, 11, 3\}, \quad G_7=\{2, 1, 7, 12, 2\}, \quad G_8=\{1, 6, 8, 7, 1\}.$$

Решение. На первом шаге получаем список $LS_1=\{c_1c_2, c_3c_4, c_5c_6, c_7c_8, c_9c_{10}\}$ отрезков пересечения полигона P со второй по шестую гранями отсекателя Θ . С остальными гранями полигон не пересекается. Координаты точек $c_1 \div c_{10}$ следующие:

$$c_1=[0.75 \ 2 \ 3.75], \quad c_2=[2 \ 2 \ 2.78], \quad c_3=[3 \ 2 \ 2], \quad c_4=[4 \ 2 \ 1.22], \quad c_5=[2 \ 0.86 \ 4.43], \\ c_6=[2 \ 2 \ 2.78], \quad c_7=[4 \ 1.46 \ 2], \quad c_8=[3 \ 2 \ 2], \quad c_9=[4 \ 2 \ 1.22], \quad c_{10}=[4 \ 1.46 \ 2].$$

На втором шаге формируем два списка: внутренних $LS_2=\{q_1p_2, p_2q_2\}$ и внешних $LS_3=\{p_1q_1, q_2p_3, p_3p_1\}$ фрагментов рёбер полигона с точками $q_1=[0.75 \ 2 \ 3.75]$ и $q_2=[2 \ 0.86 \ 4.43]$ их пересечения с гранями отсекателя. В результате получаем списки внутренне-границных $LS^{in}=LS_1 \cup LS_2$ и внешне-границных $LS^{out}=LS_1 \cup LS_3$ отрезков.

На третьем шаге с учётом совпадения точек $c_1 \equiv q_1$, $c_2 \equiv c_6$, $c_3 \equiv c_8$, $c_4 \equiv c_9$, $c_5 \equiv q_2$ и $c_7 \equiv c_{10}$ формируем два списка контуров: внутренних $LP^{in}=\{c_1c_2c_5p_2c_1, c_3c_4c_7c_3\}$ и внешних $LP^{out}=\{c_1c_2c_5p_3p_1c_1, c_3c_4c_7c_3\}$. Заметим, что контур $c_3c_4c_7c_3$ входит в оба списка.

На четвёртом шаге инициализируем пустые списки $U=\emptyset$, $F=\emptyset$ и формируем списки вершин и граней внутренних и внешних фрагментов:

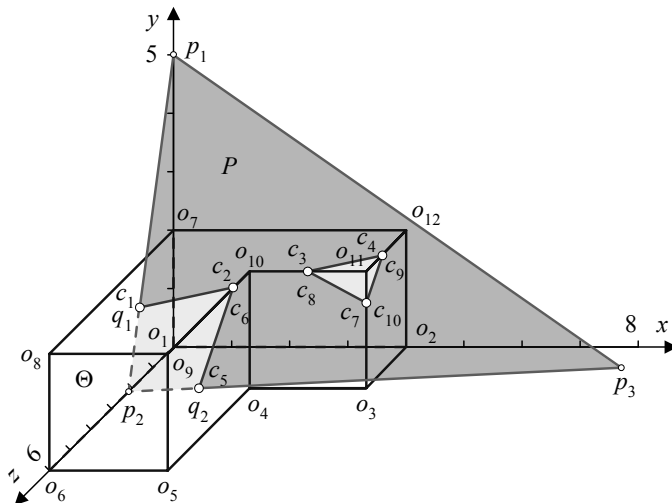


Рис. 1.157

$$U^{in} = \{c_1, c_2, c_5, p_2, c_3, c_4, c_7\}, \quad F^{in} = \{\{1, 2, 3, 4, 1\}, \{5, 6, 7, 5\}\};$$

$$U^{out} = \{c_1, c_2, c_5, p_3, p_1, c_3, c_4, c_7\}, \quad F^{out} = \{\{1, 2, 3, 4, 5, 1\}, \{6, 7, 8, 6\}\}.$$

Для правильного изображения результатов внешнего и внутреннего отсечений проверим полигоны в списках LP^{in} и LP^{out} на окружение в плоскости полигона P . Внутренние полигоны друг друга не охватывают и выводятся на экран поодиночке. Проверая два внешних полигона, тест *surr2_test* обнаруживает окружение второго первым, поэтому новый список $LP^{out} = \{c_1 c_2 c_5 p_3 p_1 c_3 c_4 c_7 c_3 c_4 c_3 c_1\}$ состоит из одного полигона с треугольным отверстием. $\square$

Переходим к последней и **главной задаче 3d-отсечения произвольного** полиэдра $H_1 = \{P_1, G_1\}$ **произвольным** полиэдром $H_2 = \{P_2, G_2\}$. Исходные объекты заданы списками координат вершин P_1, P_2 и топологии граней G_1, G_2 .

Основная идея объёмного **внешнего (внутреннего)** отсечения аналогична 2d-вариантам этих задач, реализованным в п. 1.4.2 алгоритмами *outclip2_poly* и *inclip2_poly*: накопление граней объекта H_1 , находящихся **внутри (снаружи)** объекта H_2 , граней объекта H_2 находящихся внутри H_1 , и пересечения компланарных граней H_1 и H_2 . В результате формируются список вершин U и **неупорядоченный** список F видимых фрагментов граней.

В процессе 3d-отсечения также возможно расчленение H_1 на несвязные полиэдры. Алгоритм отсечения должен сформировать все эти фрагменты — описать каждый из них аналогично описанию исходного полиэдра. Это означает упорядочение группы F путём разделения её на группы списков граней **замкнутых** полиэдров. **Принцип фрагментации** — сцепление граней друг с другом по общим рёбрам до получения замкнутого объекта. Результат фрагментации представляет собой группу $GL = \{GL_1, \dots, GL_{n_\phi}\}$ списков граней замкнутых поли-

эдров — неотсечённых фрагментов исходного полиэдра. Значение n_ϕ равно числу полученных фрагментов, а k -й элемент группы представляет собой список $GL_k = \{G_{k1}, \dots, G_{km_k}\}$ граней k -го замкнутого m_k -гранного полиэдра.

Функция $frag(G)$ формирования замкнутых полиэдральных фрагментов из вершин списка P и граней списка G , возвращающая группу списков граней фрагментов GL , работает по следующему алгоритму.

Шаг 1. Инициализация пустой группы списков $GL = \emptyset$ и числа фрагментов $k=0$. При $G=\emptyset$ видимые фрагменты отсутствуют и следует **возврат** GL .

Шаг 2. Первая грань в списке G принимается за начало $++k$ -го фрагмента и её список G_1 переносится в список GL_k .

Шаг 3. Для каждого ребра из GL_k в списках G ищется совпадающее ребро с номерами вершин G_{ij} и $G_{i,j+1}$. При удаче весь список G_i переносится в GL_k . Формирование фрагмента GL_k заканчивается после просмотра всех рёбер.

Шаг 4. Если $G=\emptyset$, то **перейти** на **шаг 2**, иначе **возврат** GL . $\square$

Составим **алгоритмы внешнего $outclip3_hedr(P_1, G_1, P_2, G_2)$ либо внутреннего $inclip3_hedr(P_1, G_1, P_2, G_2)$ отсечения полиэдра $H_1 = \{P_1, G_1\}$ полиэдром $H_2 = \{P_2, G_2\}$** , возвращающие в именах функций список вершин неотсечённых фрагментов U и группу списков их граней GL .

Шаг 1. Инициализация пустых списков $U=\emptyset$, $F=\emptyset$ и расчёт размеров полиэдров $n_1=size(G_1)$ и $n_2=size(G_2)$.

Шаг 2. Внешне-граничное $outclip3_poly(\Pi_i, P_2, G_2, U, F)$ либо внутренне-граничное $inclip3_poly(\Pi_i, P_2, G_2, U, F)$ отсечение n_1 граней $\Pi_i = side(P_1, G_1, i)$ $\forall i = \overline{1, n_1}$ полиэдра H_1 отсекателем H_2 . При этом вершины и списки внутренних либо внешних фрагментов граней H_1 добавляются в списки U и F .

Шаг 3. Внешне-граничное $outclip3_poly(\Theta_j, P_1, G_1, U, F)$ отсечение n_2 граней $\Theta_j = side(P_2, G_2, j)$ $\forall j = \overline{1, n_2}$ полиэдра H_2 отсекателем H_1 . При этом вершины и списки внутренних фрагментов граней H_2 добавляются в списки U и F .

Шаг 4. Для каждой пары индексов $i = \overline{1, n_1}$ и $j = \overline{1, n_2}$ с помощью процедуры

$$Ls = \emptyset, \quad c = cross_poly(\Pi_i, \Theta_j, Ls, 1)$$

при $c=2$ получаем список Ls отрезков пересечения **компланарных** граней Π_i и Θ_j . Из них формируем список $LP = cont(Ls)$ полигонов пересечения этих граней, параметры которых добавляем в списки U и F : $addLP(LP, U, F)$.

Шаг 5. Формируем видимые фрагменты в группе списков $GL = frag(F)$, после чего следует **возврат** структуры списков $\{U, GL\}$.



Пример 1.21. Рассчитать внешнее и внутреннее отсечения треугольной пирамиды $H = \{P, G\}$, заданной списками вершин и граней

$$P = \{p_1, p_2, p_3, p_4\} = \{[3 \ 5 \ 3], [1 \ 1 \ 4], [5 \ 1 \ 0], [5 \ 0 \ 6]\},$$

$$G = \{G_1, G_2, G_3, G_4\} = \{\{1, 3, 2, 1\}, \{1, 2, 4, 1\}, \{1, 4, 3, 1\}, \{2, 3, 4, 2\}\},$$

отсекателем $\Theta = \{O, G\}$ из примера 1.20 с вершинами $O = \{o_1, \dots, o_{12}\}$ и гранями $G = \{G_1, \dots, G_8\}$.

Решение. На рис. 1.158 приведён каркасный эскиз сцены с выделенными отрезками, по которым грани полиэдров H и Θ пересекаются друг с другом:

- $\{c_1 c_2, c_3 c_4\}$, $\{c_5 c_2\}$, $\{c_6 c_3\}$ и $\{c_7 c_4\}$ — отрезки пересечения грани G_1 с гранями G_2, G_4, G_5 и G_6 ;
- $\{c_8 c_1\}$ и $\{c_9 c_8\}$ — отрезки пересечения грани G_2 с G_2 и G_4 ;
- $\{c_5 c_9\}$, $\{c_6 c_{10}\}$ и $\{c_7 c_{10}\}$ — отрезки пересечения G_4 с G_4, G_5 и G_6 .

Вот координаты точек $c_1 \div c_{10}$:

$$c_1 = [1.5 \ 2 \ 3.75], \quad c_2 = [2 \ 2 \ 3.25],$$

$$c_3 = [3.25 \ 2 \ 2], \quad c_4 = [4 \ 2 \ 1.25],$$

$$c_5 = [2 \ 1 \ 3], \quad c_6 = [3 \ 1 \ 2],$$

$$c_7 = [4 \ 1 \ 1], \quad c_8 = [2 \ 2 \ 3.94],$$

$$c_9 = [2 \ 0.75 \ 4.5], \quad c_{10} = [4 \ 0.83 \ 2].$$

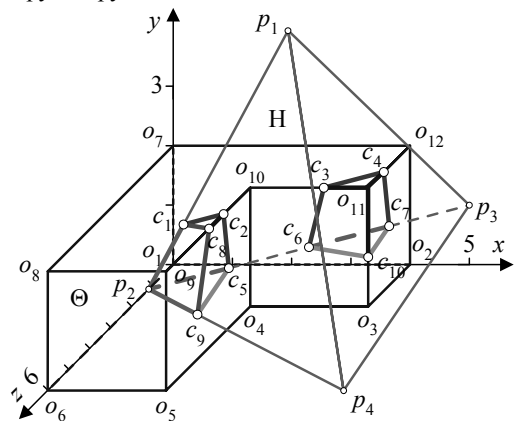


Рис. 1.158

В начале расчёта отсечения полиэдра H полиэдром Θ создаём пустые списки вершин $U^{in} = U^{out} = \emptyset$ и группу списков граней $F^{in} = F^{out} = \emptyset$.

Внешне-граничное отсечение граней $\Pi_i = \text{side}(P, \Gamma, i) \quad \forall i = \overline{1,4}$ алгоритмом $\text{outclip3_poly}(\Pi_i, O, G, U^{in}, F^{in})$ добавляет в группу F^{in} внутренние фрагменты

$$F_1 = c_1 c_2 c_5 p_2 c_1, \quad F_2 = c_3 c_4 c_7 c_6 c_3, \quad F_3 = c_8 c_1 p_2 c_9 c_8, \quad F_4 = c_5 c_9 p_2 c_5, \quad F_5 = c_6 c_{10} c_7 c_6.$$

Внутренне-граничное отсечение тех же граней $\text{inclip3_poly}(\Pi_i, O, G, U^{out}, F^{out})$ добавляет в группу F^{out} внешние фрагменты

$$F_6 = p_1 p_3 c_7 c_4 c_3 c_6 c_5 c_2 c_1 p_1, \quad F_7 = p_1 c_1 c_8 c_9 p_4 p_1, \quad F_8 = p_1 p_4 p_3 p_1, \quad F_9 = c_5 c_6 c_{10} c_7 p_3 p_4 c_9 c_5.$$

Внешне-граничное отсечение граней $Q_j = \text{side}(O, G, j) \quad \forall j = \overline{1,8}$ полиэдром Н с помощью функций $\text{outclip3_poly}(Q_j, P, \Gamma, U^{in}, F^{in})$ и $\text{outclip3_poly}(Q_j, P, \Gamma, U^{out}, F^{out})$ добавляет в группы F^{in} и F^{out} фрагменты

$$F_{10} = c_2 c_1 c_8 c_2, \quad F_{11} = c_4 c_3 o_{11} c_4, \quad F_{12} = c_8 c_9 c_5 c_2 c_8, \quad F_{13} = c_3 c_6 c_{10} o_{11} c_3, \quad F_{14} = c_{10} c_7 c_4 o_{11} c_{10}.$$

Ввиду попарной некомпланарности граней полиэдров пересечение их границ отсутствует. В результате имеем два списка вершин:

$$U^{in} = \{c_1, c_2, c_5, p_2, c_3, c_4, c_7, c_6, c_8, c_9, c_{10}, o_{11}\}, \quad U^{out} = \{p_1, p_3, c_7, c_4, c_3, c_6, c_5, c_2, c_1, c_8, c_9, p_4, c_{10}, o_{11}\},$$

а фрагментация списков F^{in} и F^{out} функцией $\text{frag}(F)$ создаёт две группы списков граней

$$\Gamma L^{in} = \{\{F_1, F_{10}, F_{12}, F_4, F_3\}, \{F_2, F_{11}, F_{14}, F_5, F_{13}\}\}, \quad \Gamma L^{out} = \{\{F_6, F_8, F_9, F_{14}, F_{11}, F_{13}, F_{12}, F_{10}, F_7\}\}.$$

□

Логические операции с полиэдрами

Логические операции с полиэдрами занимают важное место в инструментальном арсенале компьютерной графики [13, 20]. Они позволяют реализовать эффекты перекрытия, слияния и вытеснения объёмных объектов, составляющие суть конструктивного моделирования сложных фигур. На рис. 1.159 показаны результаты логических операций объединения $A \cup B$, пересечения $A \cap B$ и двух исключений $A \setminus B$ и $B \setminus A$ с двумя 3d-объектами: пирамидой A и сферой B .

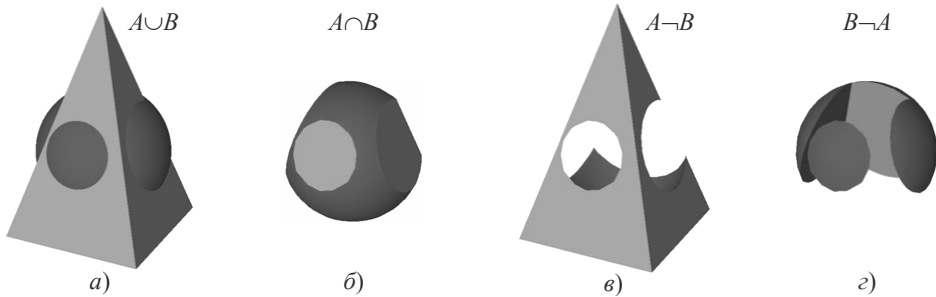


Рис. 1.159

Изучим порядок выполнения этих логических операций с полиэдрами $A = \{P_A, G_A\}$ и $B = \{P_B, G_B\}$. Аналогично полигональным операциям, изображённым на рис. 1.83, внешнее отсечение $\text{outclip3_hedr}(P_A, G_A, P_B, G_B, U, \Gamma L)$ фактически выполняет логическую операцию **пересечения** (конъюнкции, произведения) $A \cap B$, а внутреннее отсечение $\text{inclip3_hedr}(P_A, G_A, P_B, G_B, U, \Gamma L)$ реализует логическую операцию **исключения** (вычитания) $A \setminus B = A \cap (\neg B)$. Каждая из этих функций возвращает результат в выходной структуре списков $\{U, \Gamma L\}$. Оформим эти логические операции в виде следующих функций.


 $\boxed{mul3(P_A, G_A, P_B, G_B)}$ // пересечение полиэдров $A \cap B$
 $\{U, GL\} = outclip3_hedr(P_A, G_A, P_B, G_B);$
возврат $\{U, GL\};$
 $\}$ 


 $\boxed{sub3(P_A, G_A, P_B, G_B)}$ // исключение полиэдров $A - B$
 $\{U, GL\} = inclip3_hedr(P_A, G_A, P_B, G_B);$
возврат $\{U, GL\};$
 $\}$ 

Третья операция **объединения** (дизъюнкции, суммы) полиэдров $A \cup B$ выполняется путём накопления фрагментов граней полигона A , **внешних** относительно отсекающей B , фрагментов граней B , **внешних** относительно отсекающей A , и общих **граничных** фрагментов граней A и B с последующей сборкой из них замкнутых полиэдров и сохранением их в структуре списков $\{U, GL\}$.


 $\boxed{add3(P_A, G_A, P_B, G_B)}$ // объединение полиэдров $A \cup B$
 $\{U = \emptyset, F = \emptyset, n = size(G_A), m = size(G_B);$ // инициализация списков
 $\Pi_i = side(P_A, G_A, i), \Theta_j = side(P_B, G_B, j);$ // грани полиэдров
 $inclip3_poly(\Pi_i, P_B, G_B, U, F) \forall i = \overline{1, n};$ // внешние грани A
 $inclip3_poly(\Theta_j, P_A, G_A, U, F) \forall j = \overline{1, m};$ // внешние грани B
 $\forall i = \overline{1, n}, j = \overline{1, m};$ // граничные грани A и B
 $\{Ls = \emptyset, c = cross_poly(\Pi_i, \Theta_j, Ls, 1);$
если $c = 2$, **то** $LP = cont(Ls), \{U, F\} = addLP(LP, U, F);$
 $\}$ $GL = frag(F);$ // сборка полиэдров
возврат $\{U, GL\};$
 $\}$ 

 **Пример 1.22.** Из простых полиэдров A, B и C , показанных на рис. 1.160, a каркасами со списками вершин

$$P_A = \{[0 \ 0 \ 0], [8 \ 0 \ 0], [8 \ 0 \ 6], [0 \ 0 \ 6], [0 \ 6 \ 0], [8 \ 6 \ 0], [8 \ 6 \ 6], [0 \ 6 \ 6]\} \equiv \{a_1, a_2, \dots, a_8\},$$

$$P_B = \{[0 \ 0 \ 0], [8 \ 0 \ 0], [8 \ 0 \ 8], [0 \ 0 \ 8], [0 \ 8 \ 0], [8 \ 8 \ 0]\} \equiv \{b_1, b_2, \dots, b_6\},$$

$$P_C = \{[2 \ 2 \ 2], [6 \ 2 \ 0], [6 \ 2 \ 6], [0 \ 2 \ 6], [2 \ 6 \ 2], [6 \ 6 \ 2], [6 \ 6 \ 6], [2 \ 6 \ 6]\} \equiv \{c_1, c_2, \dots, c_8\},$$

сконструировать сложный объект O по логической формуле $O = A \cap B - C$.

Решение. Составив для объектов $A = \{P_A, G_A\}$, $B = \{P_B, G_B\}$ и $C = \{P_C, G_C\}$ списки граней

$$G_A = \{\{1, 2, 3, 4, 1\}, \{5, 8, 7, 6, 5\}, \{4, 3, 7, 8, 4\}, \{3, 2, 6, 7, 3\}, \{2, 1, 5, 6, 2\}, \{1, 4, 8, 5, 1\}\},$$

$$G_B = \{\{1, 2, 3, 4, 1\}, \{4, 3, 6, 5, 4\}, \{3, 2, 6, 3\}, \{2, 1, 5, 6, 2\}, \{1, 4, 5, 1\}\}, \quad G_C = G_A,$$

инициализируем пустые список вершин $P_D = \emptyset$, группу списков граней $G_D = \emptyset$ и начнём с расчёта пересечения $D = A \cap B$ по алгоритму $outclip3_hedr(P_A, G_A, P_B, G_B)$. Выполняемая в нём последовательность отсечений и наполнения списков P_D и G_D показана в табл. 1.6. Четыре точки пересечения рёбер объекта A с гранями объекта B имеют координаты $q_1 = [0 \ 6 \ 2]$, $q_2 = [8 \ 6 \ 2]$, $q_3 = [8 \ 2 \ 6]$ и $q_4 = [0 \ 2 \ 6]$.

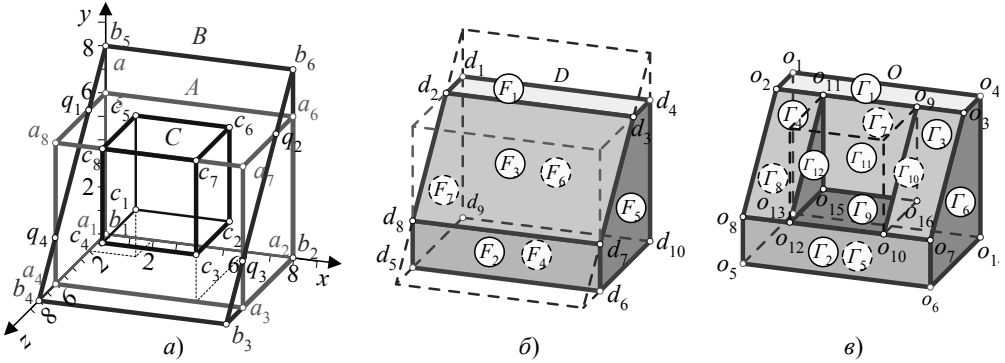


Рис. 1.160

В результате получаем модель полиэдра $D = \{P_D, G_D\}$, состоящую из списка десяти вершин $P_D = \{d_1, d_2, \dots, d_{10}\}$ и группы списков семи граней $G_D = \{F_1, F_2, \dots, F_7\}$. Фрагментация $frag(G_D)$ соединяет все грани в **один** 3d-фрагмент (рис. 1.160,б).

Вторая операция $O = D - C$ выполняется функцией $inclip3_hedr(P_D, G_D, P_C, G_C)$, возвращающей координатный список шестнадцати вершин $P_O = \{o_1, o_2, \dots, o_{16}\}$ и группу списков двенадцати граней $G_O = \{\Gamma_1, \Gamma_2, \dots, \Gamma_{12}\}$. Фрагментация $frag(G_O)$ соединяет эти грани в **один** 3d-фрагмент (рис. 1.160,в). $\square$

Таблица 1.6

Грани	Отсечение	Списки P_D и P_O	Списки G_D и G_O
$a_5 a_8 a_7 a_6 a_5$	$a_5 q_1 q_2 a_6 a_5$	$a_5 \equiv d_1, q_1 \equiv d_2, q_2 \equiv d_3, a_6 \equiv d_4$	$F_1 = \{1, 2, 3, 4, 1\}$
$a_4 a_3 a_7 a_8 a_4$	$a_4 a_3 q_3 q_4 a_4$	$a_4 \equiv d_5, a_3 \equiv d_6, q_3 \equiv d_7, q_4 \equiv d_8$	$F_2 = \{5, 6, 7, 8, 5\}$
$b_4 b_3 b_6 b_5 b_4$	$q_3 q_2 q_1 q_4 q_3$		$F_3 = \{7, 3, 2, 8, 7\}$
$a_1 a_2 a_3 a_4 a_1$	$a_1 a_2 a_3 a_4 a_1$	$a_1 \equiv d_9, a_2 \equiv d_{10}$	$F_4 = \{9, 10, 6, 5, 9\}$
$a_3 a_2 a_6 a_7 a_3$	$a_3 a_2 a_6 q_2 q_3 a_3$		$F_5 = \{6, 10, 4, 3, 7, 6\}$
$a_2 a_1 a_5 a_6 a_2$	$a_2 a_1 a_5 a_6 a_2$		$F_6 = \{10, 9, 1, 4, 10\}$
$a_1 a_4 a_8 a_5 a_1$	$a_1 a_4 q_4 q_1 a_5 a_1$		$F_7 = \{9, 5, 8, 2, 1, 9\}$
$d_1 d_2 d_3 d_4 d_1$	$d_1 d_2 d_3 d_4 d_1$	$d_1 \equiv o_1, d_2 \equiv o_2, d_3 \equiv o_3, d_4 \equiv o_4$	$\Gamma_1 = \{1, 2, 3, 4, 1\}$
$d_5 d_6 d_7 d_8 d_5$	$d_5 d_6 d_7 d_8 d_5$	$d_5 \equiv o_5, d_6 \equiv o_6, d_7 \equiv o_7, d_8 \equiv o_8$	$\Gamma_2 = \{5, 6, 7, 8, 5\}$
$d_7 d_3 d_2 d_8 d_7$	$d_7 d_3 c_6 c_3 d_7, \\ c_5 d_2 d_8 c_4 c_5$	$c_6 \equiv o_9, c_3 \equiv o_{10}, c_5 \equiv o_{11}, c_4 \equiv o_{12}$	$\Gamma_3 = \{7, 3, 9, 10, 7\}, \\ \Gamma_4 = \{11, 2, 8, 12, 11\}$
$d_9 d_{10} d_6 d_5 d_9$	$d_9 d_{10} d_6 d_5 d_9$	$d_9 \equiv o_{13}, d_{10} \equiv o_{14}$	$\Gamma_5 = \{13, 14, 6, 5, 13\}$
$d_6 d_{10} d_4 d_3 d_7 d_6$	$d_6 d_{10} d_4 d_3 d_7 d_6$		$\Gamma_6 = \{6, 14, 4, 3, 7, 6\}$
$d_{10} d_9 d_1 d_4 d_{10}$	$d_{10} d_9 d_1 d_4 d_{10}$		$\Gamma_7 = \{14, 13, 1, 4, 14\}$
$d_9 d_5 d_8 d_2 d_1 d_9$	$d_9 d_5 d_8 d_2 d_1 d_9$		$\Gamma_8 = \{13, 5, 8, 2, 1, 13\}$
$c_1 c_2 c_3 c_4 c_1$	$c_1 c_2 c_3 c_4 c_1$	$c_1 \equiv o_{15}, c_2 \equiv o_{16}$	$\Gamma_9 = \{15, 16, 10, 12, 15\}$
$c_3 c_2 c_6 c_7 c_3$	$c_3 c_2 c_6 c_3$		$\Gamma_{10} = \{10, 16, 9, 10\}$
$c_2 c_1 c_5 c_6 c_2$	$c_2 c_1 c_5 c_6 c_2$		$\Gamma_{11} = \{16, 15, 11, 9, 16\}$
$c_1 c_4 c_8 c_5 c_1$	$c_1 c_4 c_5 c_1$		$\Gamma_{12} = \{15, 12, 11, 15\}$

С учётом усилий, предпринятых в разд. 1.4, разработан модульно организованный **комплекс алгоритмов отсечения** (рис. 1.161). Форма модулей отсечения условно отражает форму отсекаателя: плоская, объёмная, регулярная, выпуклая, произвольная. Изображённые перпендикулярно модулям фигуры показывают тип отсекаемых объектов: отрезки, полигоны или полиэдры. Модули округлой формы выполняют вспомогательные функции, необходимые для работы основных алгоритмов. Стрелки, соединяющие модули, означают использование модуля-источника модулем-приёмником. Число модулей, связанных с модулем-приёмником, достаточно объективно характеризует сложность решаемой последней задачи, её ресурсоёмкость и потенциальное быстродействие.

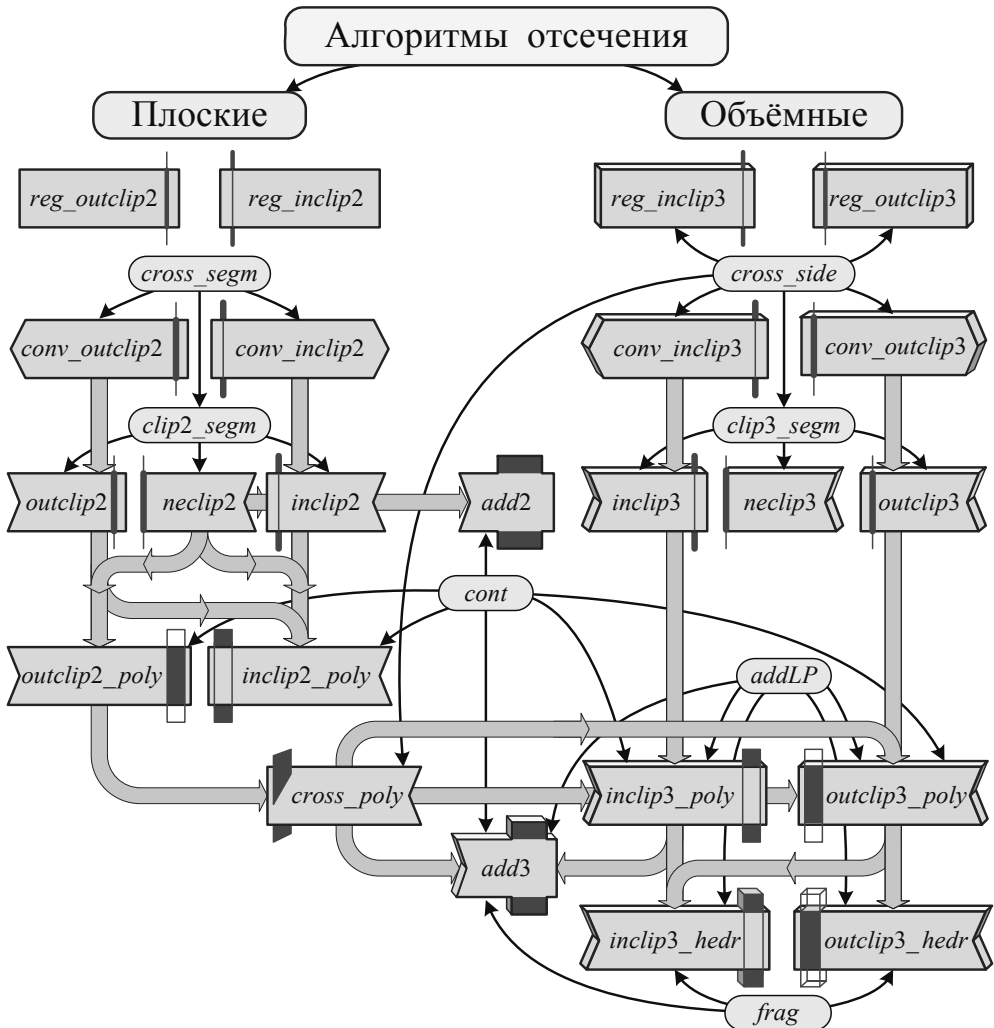


Рис. 1.161

1.6.3. Отсечение полупространствами

Аналогично определению полуплоскости, данному в п. 1.4.3, назовём **полупространством** ту часть пространства, во всех точках которой неявная функция плоскости **отрицательна**, т. е. $f(p) < 0$. Обращённую в полупространство сторону плоскости назовём **внутренней**.

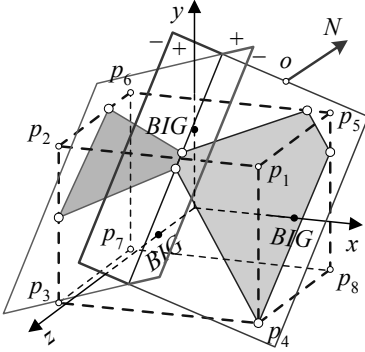


Рис. 1.162

Рассмотрим задачу нахождения области пересечения полупространств. Если эта область не пуста, то она безусловно выпукла как пересечение выпуклых 3d-областей. Решение задачи заключается в последовательном отсечении текущей выпуклой области очередным полупространством (рис. 1.162). В итоге после каждого шага область пересечения либо не изменяется — когда она целиком лежит в полупространстве, — либо сужается, либо становится пустой — когда пересечения с полупространством нет, — после чего обработка оставшихся полупространств прекращается.

Выберем способ задания границы полупространства. Сравнивая методы описания плоскости в п. 1.5.1 и варианты (1.108) вычисления $f(p)$, заключаем, что наиболее практичны нормальная форма $\{o, N\}$ (шесть чисел) и функция $\eta f N(o, N, p)$ (восемь арифметических операций). Параметрическая (1.101) и трёхточечная (1.105б) модели плоскости и соответствующие им функции (1.108в) и (1.108г) имеют большую ресурсоёмкость.

Если отсекаемая область не является замкнутым полиэдром, возникает естественный вопрос о способе описания такой **неограниченной** области. Для преодоления численной неопределённости зададим начальный полиэдр

$$BIG_BOX = \{P_{\square}, BIG, G_{\square}\} \quad (1.179a)$$

в виде выпуклой **ограниченной** кубической области, показанной на рис. 1.162 пунктиром, с габаритными координатами $|x|=|y|=|z|=BIG$, где BIG — максимально большое **конечное** число, надёжно обеспечивающее пересечение области BIG_BOX с любой возможной плоскостью. Матрица координат вершин гексаэдра $P_{\square}$ и группа списков его граней $G_{\square}$ следующие:

$$P_{\square} = \begin{bmatrix} 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \end{bmatrix}^T, \quad \begin{aligned} G_{\square} &= \{G_1, G_2, G_3, G_4, G_5, G_6\}, \\ G_1 &= \{1, 2, 3, 4, 1\}, \quad G_2 = \{1, 4, 8, 5, 1\}, \\ G_3 &= \{1, 5, 6, 2, 1\}, \quad G_4 = \{2, 6, 7, 3, 2\}, \\ G_5 &= \{3, 7, 8, 4, 3\}, \quad G_6 = \{5, 8, 7, 6, 5\}. \end{aligned} \quad (1.179б)$$

Таким образом, задача пересечения неограниченных полупространств сводится к поочерёднему отсечению ими от ограниченного выпуклого полиэдра частей, лежащих с внешних сторон граничных плоскостей.

На рис. 1.163 построена блок-схема алгоритма $clip_side(P, G, o, N)$ отсе-
 чения полиэдра $H = \{P, G\}$ полупространством с граничной плоскостью $\{o, N\}$,
 заданной точкой o и нормалью N . Список $P = \{p_1, \dots, p_n\}$ содержит координаты n
 вершин, а группа $G = \{G_1, \dots, G_m\}$ — списки m граней полиэдра H . Работа начи-
 нается с инициализации пустых списка $U = \emptyset$ и группы $F = \emptyset$, в которых будет
 накапливаться информация о вершинах и гранях отсекаемого полиэдра, а также
 установки значения $k=0$ индекса последнего записанного в U элемента.

В связи с повышенной сложностью 3d-расчётов нужно быстрее установить
 факт пересечения плоскости с полиэдром, как это делает тест $cross3$, опреде-
 лённый в (1.146). Для этого вычисляются значения неявной функции
 $f_i = n f N(o, N, p_i)$ во всех вершинах полиэдра и производится анализ экстре-
 мальных $f_{\min} = \min\{f\}$ и $f_{\max} = \max\{f\}$ значений элементов вектора f .

- при $f_{\min} \geq 0$ весь полиэдр расположен над плоскостью либо касается её
 вершиной, ребром или гранью, а алгоритм возвращает пустое решение;

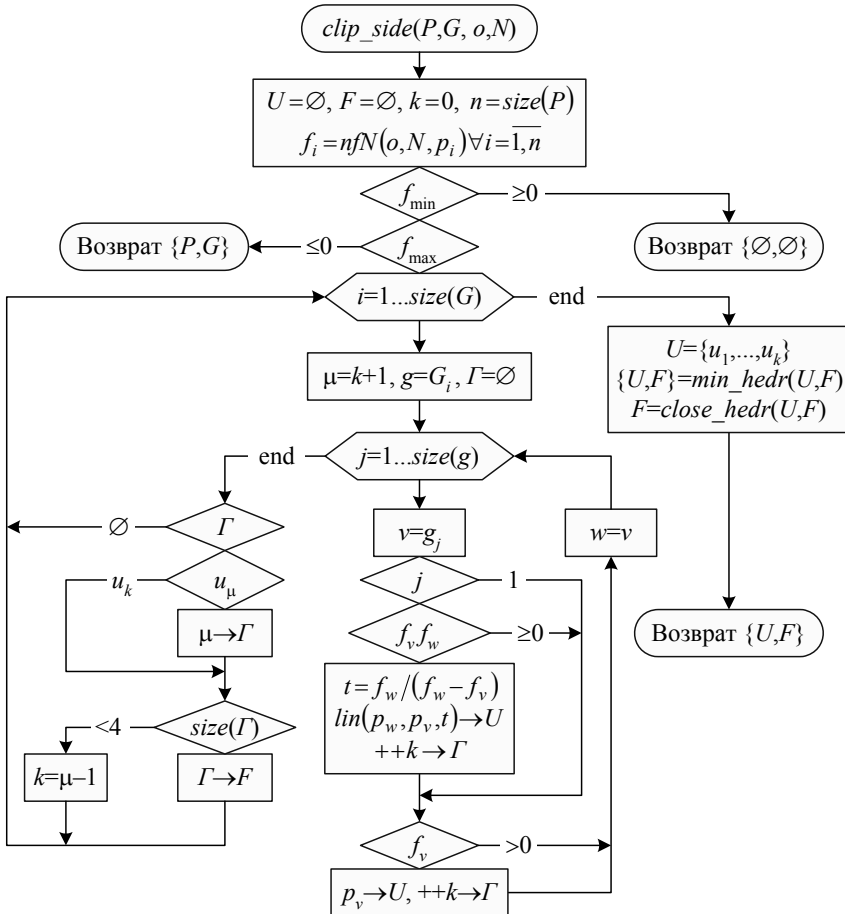


Рис. 1.163

- при $f_{\max} \leq 0$ весь полиэдр находится в отсекающем полупространстве, возможно касаясь его границы, и возвращается алгоритмом без изменения;
- оставшееся условие $\{f_{\min} < 0\} \wedge \{f_{\max} > 0\}$ означает, что существует хотя бы одна пара вершин $\{p_i, p_j\}$, лежащих по разные стороны от плоскости $\{o, N\}$. Так как полиэдр выпуклый, то плоскость $\{o, N\}$ пересекает его минимум по трём граням. Не отсечённые полупространством фрагменты граней должны составить искомым полиэдр $\{U, F\}$. Организуем цикл $i = \overline{1, m}$ обхода граней G_i , в котором будем добавлять в список U координаты, а в группу F — списки граней, оставшихся у полиэдра после отсечения. Перед началом обработки i -й грани, во-первых, установим в списке U индекс начальной вершины этой грани в $\mu = k + 1$ и, во-вторых, инициализируем пустой список $\Gamma = \emptyset$ индексов неотсечённых вершин.

В центральной части алгоритма отсечения грани полупространством производится обход её вершин согласно списку $g = G_i$ и формирование неотсечённого контура с внутренней стороны секущей плоскости $\{o, N\}$. В этот контур включаются вершины p_{g_j} , $j \in [1, \text{size}(g)]$, лежащие в полупространстве, а также точки пересечения рёбер с плоскостью. Найденные точки добавляются в список U , а инкрементированный индекс $++k$ — в список Γ .

Завершив обход грани G_i и убедившись в наличии содержимого в Γ (при $\Gamma = \emptyset$ вся грань отсечена плоскостью), проверяем замкнутость полученного контура. При $u_k \neq u_\mu$ добавляем в Γ индекс μ . Затем проверяем число неотсечённых вершин грани. Для того, чтобы грань была хотя бы треугольной, число элементов её списка должно быть более трёх. В этом случае добавляем список Γ в группу F . Число новых точек в списке U , меньшее четырёх, бывает при отсечении грани, касающейся плоскости с внешней стороны одной вершиной или одним ребром. Для аннулирования этих вершин устанавливаем исходное состояние индекса $k = \mu - 1$, которое он имел перед началом отсечения грани.

По окончании обхода всех граней исходного полиэдра из списка U отбираются первые k элементов — вершины искомого полиэдра. Группа F содержит списки его граней, **но не всех!** На завершающем этапе алгоритма *clip_side* выполняются две важные операции со списками U и F :

- **минимизация модели полиэдра.** После отсечения координатный список полиэдра всегда имеет **кратные** точки, т. к. каждая вершина входит в состав не менее трёх смежных граней. Устранением кратных вершин занимается показанный на рис. 1.125 алгоритм *min_hedr(P, G)*;

- **замыкание топологии полиэдра.** Дело в том, что отсечение 3d-объекта полупространством по алгоритму *clip_side* не даёт **новой** грани, лежащей в сечении полиэдра плоскостью. Вершины этой грани рассчитываются независимо друг от друга и в отдельный список не собираются. В результате искомым полиэдр оказывается не замкнутым — без одного списка грани в группе G .

Идея поиска **единственной** отсутствующей грани основана на следующей закономерности. Если в замкнутом полиэдре некоторое ребро $p_k p_l$ соединяет две

грани с номерами i и j , то индексы вершин на его концах входят в состав *двух* списков G_i и G_j как $\{\dots, k, l, \dots\}$ либо $\{\dots, l, k, \dots\}$. Ребро, находящееся на краю незамкнутого полиэдра, входит лишь в *один* список грани. Это свойство позволяет найти все крайние рёбра и соединить их совпадающими концами в замкнутый контур, составляющий требуемую замыкающую грань.

На рис. 1.164 представлен алгоритм $close_hedr(P, G)$ топологического замыкания полиэдра с формальными параметрами — списком вершин $P = \{p_1, \dots, p_n\}$ и группой списков граней $G = \{G_1, \dots, G_m\}$. Работа алгоритма начинается с задания нулевой квадратной *матрицы смежности вершин* полиэдра $S \in R^{n \times n}$ и счётчика рёбер замыкающего полигона $k=0$. Поочерёдно сканируя содержимое списков $g = G_i \ \forall i = \overline{1, m}$, инкрементируем (g_{j-1}, g_j) -е элементы матрицы $S \ \forall j = \overline{2, size(g)}$, ведя таким образом подсчёт кратностей рёбер, соединяющих вершины $p_{g_{j-1}}$ и p_{g_j} . По окончании сканирования сумма симметричных элементов матрицы смежности $s_{ij} + s_{ji}$ равна кратности ребра $p_i p_j$.

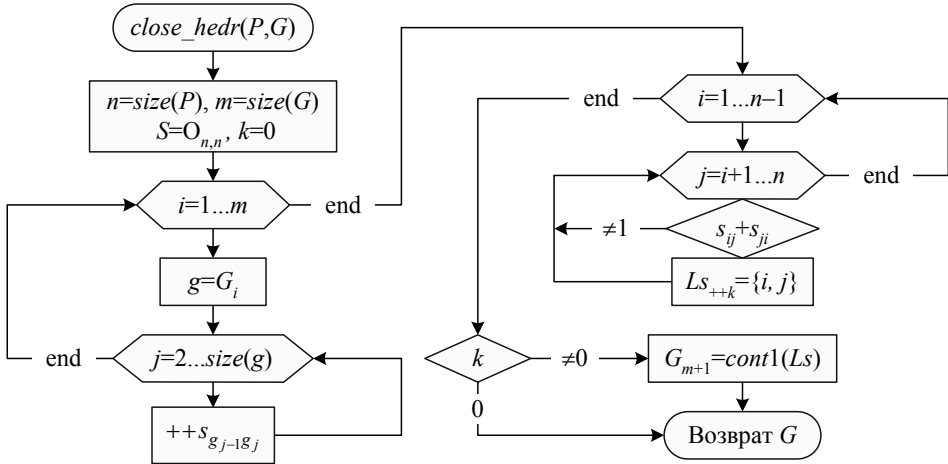


Рис. 1.164

Для иллюстрации сказанного допустим, что при отсечении шестигранника, изображённого на рис. 1.165, получены следующие списки вершин и граней:

$$P = \{p_1, p_2, p_3, p_4\}, \quad G = \{G_1, G_2, G_3\}, \\ G_1 = \{1, 2, 3, 1\}, \quad G_2 = \{1, 2, 4, 1\}, \quad G_3 = \{1, 3, 4, 1\}.$$

Тогда матрица смежности принимает вид

$$S = \begin{bmatrix} 0 & 2 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 2 & 0 & 0 & 0 \end{bmatrix}. \quad (1.180)$$

Второй этап алгоритма замыкания — формирование списка Ls из индексов вершин рёбер единичной кратности. Сканируя наддиагональные элементы

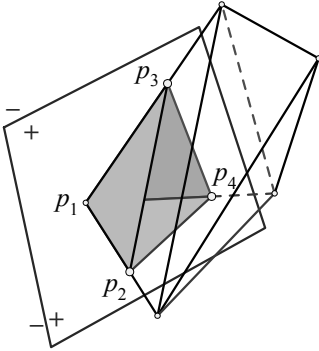


Рис. 1.165

$s_{ij} \forall i=1, n-1, j=i+1, n$, добавляем в Ls пары индексов $\{i, j\}$, для которых выполняются условия $s_{ij} + s_{ji} = 1$ простоты рёбер $p_i p_j$.

Просматривая наддиагональный треугольник матрицы смежности (1.180), получим список $Ls = \{\{2, 3\}, \{2, 4\}, \{3, 4\}\}$. Следовательно, грань сечения полиэдра составляют рёбра единичной кратности $p_2 p_3, p_2 p_4$ и $p_3 p_4$.

На последнем третьем этапе неупорядоченные рёбра сечения соединяются вершинами с **совпадающими индексами** в полигон. Подобная задача с поиском совпадающих точек на плоскости решается функцией *cont1* (см. рис. 1.68,а). Эту же методику можно применить для сборки замкнутых контуров из индексных отрезков $\{i, j\}$, что и сделано в алгоритме *close_hedr*. Сформированный список индексов вершин новой грани добавляется в группу G .

В отсутствие пересечения полиэдра с плоскостью число рёбер полигона сечения остаётся нулевым, а изменения G не происходит. На этом алгоритм *close_hedr* заканчивает работу, возвращая обновлённую либо исходную группу списков граней G .

Завершая решение задачи отсечения на рис. 1.165, получаем новую грань $G_4 = \{2, 3, 4, 2\}$, описывающую полигон $p_2 p_3 p_4 p_2$.

Область применения метода отсечения выпуклого полиэдра полупространством обширна: это задачи отсечения объектов **ограниченными** областями видимости (призмы или конуса), генерирования случайных выпуклых полиэдров, построения сечений выпуклых объектов плоскостями, расчёта ядра полиэдра, пересечения выпуклых полиэдров и т. п.

Генератор случайных выпуклых полиэдров

На основе разработанного метода отсечения полиэдра полупространством составлен **алгоритм *conv_hedr* построения выпуклого полиэдра** (рис. 1.166,а), полезный в различных задачах компьютерной графики, например, для отладки 3d-тестов и алгоритмов на случайных выпуклых полиэдрах. Подобно каменотёсу, алгоритм поочерёдно отсекает от исходного выпуклого гексаэдра $BIG_BOX = \{P_{\square} \cdot BIG, G_{\square}\}$ части, внешние относительно случайно сгенерированных плоскостей. Для получения всегда непустого решения выберем **способ генерирования отсекающих плоскостей**, у которых начало координат всегда находится в отрицательных полупространствах.

Пусть в пространстве задана или случайно сгенерирована точка $q \neq O_3$. Проведя отсекающую плоскость $\{q, N\}$ с нормалью $N=q$, получим **отрицательное** значение $f(O_3) = -q \circ q < 0$ неявной функции плоскости в начале координат. Поэтому отсечение полиэдра H полупространством с граничной плоскостью $\{q, q\}$ сохраняет начало системы координат внутри H . Процесс генерирования плоско-

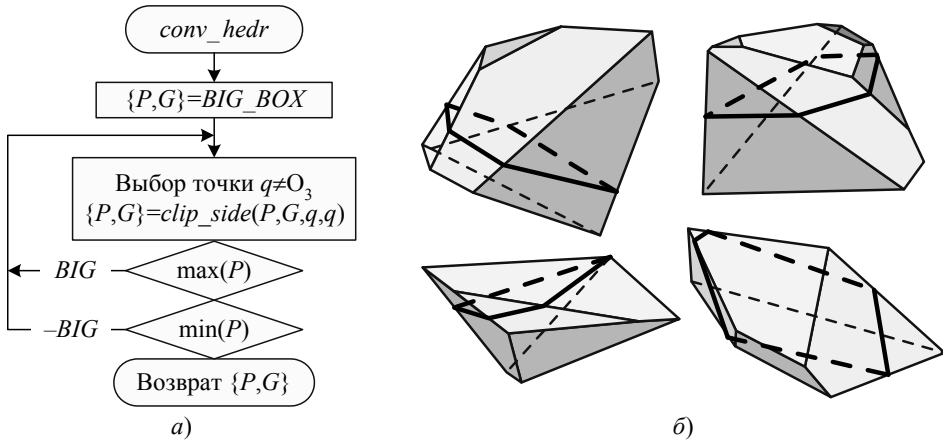


Рис. 1.166

стей и отсечения ими полиэдра продолжается до тех пор, пока координаты вершин получающегося полиэдра имеют числа *BIG* либо $-BIG$. При полном отсечении от объекта *BIG_BOX* его исходных границ функция возвращает искомым выпуклый полиэдр. На рис. 1.166,б показано несколько случайных полиэдров, сгенерированных алгоритмом *conv_hedr*.

Расчёт ядра полиэдра

Ядро полиэдра *H* — это множество точек *K*, из которых видны все вершины полиэдра (рис. 1.167,а). Любой выпуклый полиэдр совпадает со своим ядром (рис. 1.167,б), но не у каждого невыпуклого полиэдра есть ядро (рис. 1.167,в).

Конструктивно ядро создаётся отсечением неограниченного пространства полупространства плоскостей граней полиэдра. Блок-схема **алгоритма**

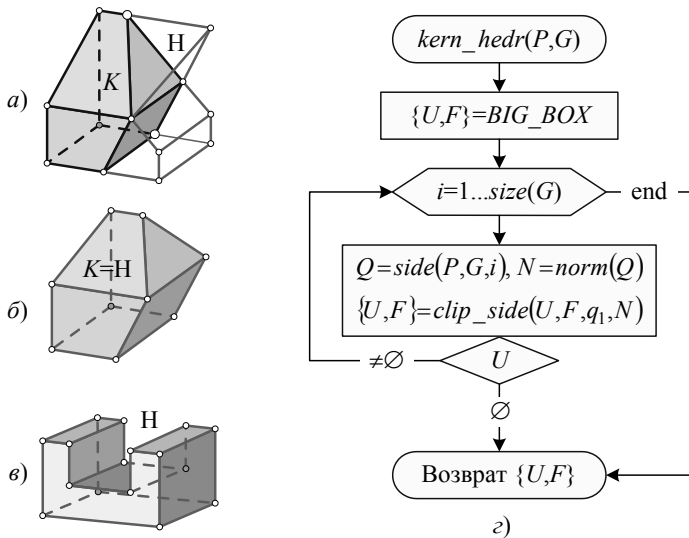


Рис. 1.167

$kern_hedr(P, G)$ **расчёта ядра полиэдра** $H = \{P, G\}$ с матрицей вершин P и списками граней G приведена на рис. 1.167, з.

Вначале устанавливаются исходные параметры (1.179) отсекаемого полиэдра: $K = BIG_BOX$. Затем в цикле $i = \overline{1, size(G)}$ выполняется внешнее отсечение выпуклого полиэдра K плоскостями всех граней полиэдра H :

- формируется полигон i -й грани полиэдра $Q = side(P, G, i)$ и по его первым трём точкам q_1, q_2 и q_3 вычисляется вектор нормали $N = norm(Q)$;
- выполняется отсечение полиэдра $K = \{U, F\}$ полупространством этой грани с помощью функции $clip_side(U, F, \Pi_1, N)$;
- проверяется размер координатного списка U . Если на некотором шаге результат отсечения окажется пустым, то ядро полиэдра отсутствует, а алгоритм возвращает пустой результат $U = \emptyset$.

Для того чтобы отсечение плоскостью грани выполнялось с внешней стороны, **нормаль к грани должна быть всегда внешней**. Это достигается выбором **положительного** направления нумерации вершин грани, если смотреть на неё снаружи полиэдра. Дополнительное условие предъявляется к описанию грани, имеющей форму невыпуклого полигона (например, задней шестиугольной грани полиэдра H на рис. 1.167, а). В её списке G_i **первые три вершины должны составлять выпуклый угол**. Только тогда определённая по ним нормаль (1.145) будет внешней.

Пересечение выпуклых полиэдров

Аналогично плоской задаче пересечения полигонов реализация булевой функции $A \cap B$ пересечения **выпуклых** полиэдров A и B основана на отсечении объекта B полупространствами граней объекта A (либо наоборот). В результате получим объект $A \cap B$, принадлежащий как A , так и B , т. е. пересечение этих объектов.

Из этих соображений на рис. 1.168, а построена блок-схема **алгоритма** $cross_hedr(P_A, G_A, P_B, G_B)$ **расчёта пересечения выпуклых полиэдров** $A = \{P_A, G_A\}$ и $B = \{P_B, G_B\}$. Алгоритм возвращает полиэдр пересечения объектов $A \cap B$, если он существует (рис. 1.168, б), либо структуру $\{\emptyset, G_B\}$ с пустым списком координат, если полиэдры A и B лишь касаются друг друга или вообще не пересекаются. Принципиальное требование к объекту A , плоскостями граней которого выполняется отсечение объекта B , является **внешняя ориентация нормалей** ко всем граням G_{Ai} . Аналогичное требование к нормальям граней объекта B не обязательно.

Идентификация направления нормалей граней основана на **методе пробной точки**, описанном в п. 1.5.6 (см. рис. 1.134). Выбрав гарантированно **внутреннюю** точку **выпуклого** полиэдра со списком вершин $P_A = \{a_1, \dots, a_n\}$ — геометрический центр $c = mean(P_A)$ — и сформировав по (1.143) полигон i -й грани $Q = side(P_A, G_A, i)$ с вычисляемой по (1.145) нормалью $N = norm(Q)$, определим

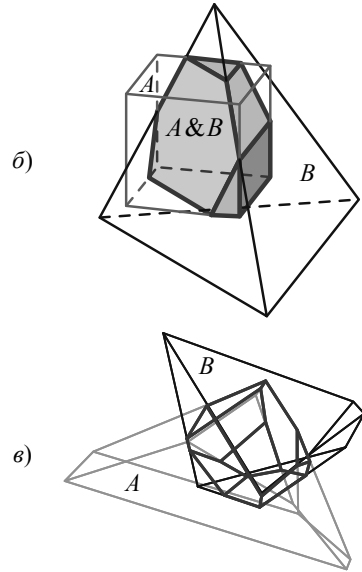
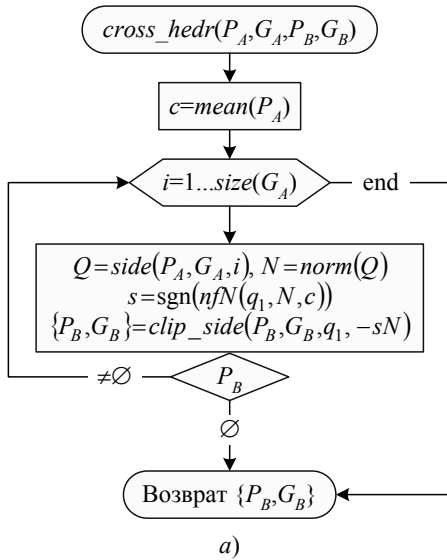


Рис. 1.168

знак s числа $nfN(q_1, N, c)$, где q_1 — первая вершина полигона Q . При $s=-1$ точка c находится в отрицательном полупространстве грани, её нормаль — внешняя, а отсечение объекта B плоскостью $\{q_1, N\}$ будет выполняться с внешней стороны. В противном случае $s=1$, а для отсечения должна быть использована антинормаль $-N$.

Если окажется, что после очередного отсечения список вершин полиэдра B стал пустым, то дальнейший перебор граней объекта A теряет смысл, и алгоритм заканчивает работу, возвращая пустой результат пересечения.

Для иллюстрации работоспособности алгоритма на рис. 1.168, в построен каркасный полиэдр $A \cap B$ — пересечение двух случайных выпуклых полиэдров A и B , сгенерированных алгоритмом *conv_hedr*.

1.6.4. Сечение полиэдра плоскостью

Построение **сечения** (*section*) полиэдра плоскостью представляет собой типовую операцию многих промышленных САПР и заключается в формировании одного или нескольких полигонов, по рёбрам которых секущая плоскость пересекает грани полиэдра.

Рассмотрим сначала задачу сечения **выпуклого** полиэдра $\{P, G\}$ бесконечной плоскостью $\{o, N\}$. Выпуклость рассекаемого полиэдра обуславливает следующие **свойства полигона сечения** C :

- если сечение существует, то оно состоит из **одного** фрагмента;
- полигон сечения C **выпуклый**;
- каждая выпуклая грань полиэдра рассекается плоскостью не более чем по **одному** отрезку.

Фактически расчёт выпуклого сечения делается на последнем этапе алгоритма *clip_side* (см. рис. 1.163), где функция $close_hedr(U, F)$ формирует полигон грани, замыкающей топологию неотсечённой части полиэдра. Возвращаемый этим алгоритмом результат в виде совокупности списков $\{U, F\}$ можно использовать для выделения из него полигона C . Если плоскость с полиэдром пересекаются, то вершины полигона C возвращаются в матрице $U \neq P$ среди других вершин неотсечённого полиэдра, а список вершин сечения находится в конце группы $F \neq G$. В противном случае сечение отсутствует, а алгоритм возвращает либо $\{\emptyset, \emptyset\}$, либо $\{P, G\}$. Так или иначе, для получения полигона C нужна дополнительная обработка списков, возвращаемых в U и F .

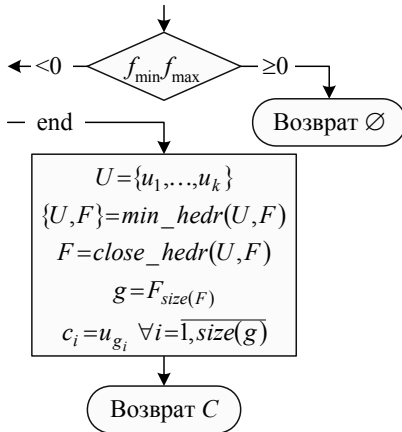


Рис. 1.169

На рис. 1.169 показано, как нужно модифицировать заключительную часть алгоритма *clip_side* (правую часть блок-схемы на рис. 1.163), чтобы он возвращал полигон сечения C . В отсутствие пересечения плоскости с полиэдром при $f_{\min} f_{\max} \geq 0$ возвращается пустой результат. В противном случае выполняется отсечение полиэдра полупространством плоскости $\{o, N\}$, после чего список $g = F_{size(F)}$, записанный в группу F последним, используется для формирования полигона C из точек $u_{g_i} \forall i = 1, size(g)$.

Задача, казалось бы, решена. Однако, то обстоятельство, что модифицированный алгоритм *clip_side* выполняет избыточные вычисления вершин и граней, не принадлежащих сечению, побуждает к разработке самостоятельного алгоритма, находящего только отрезки пересечения полиэдра с плоскостью, составляющие рёбра полигона C .

Такой алгоритм *sect_conv_hedr(P, G, o, N)* построения полигона сечения выпуклого полиэдра $\{P, G\}$ плоскостью $\{o, N\}$ представлен на рис. 1.170. Его работа начинается с тестирования пересечения полиэдра с плоскостью. Вычислив значения неявной функции $f_i = nfN(o, N, p_i)$ во всех n вершинах полиэдра, принимаем решение о существовании сечения:

- при $f_{\min} f_{\max} \geq 0$ сечение отсутствует, а алгоритм возвращает пустой результат $C = \emptyset$;
- если числа $f_{\min}$ и $f_{\max}$ разных знаков, то вершины полиэдра находятся по разные стороны секущей его плоскости и алгоритм приступает к расчёту непустого решения.

Полигон сечения C собирается из отрезков, по которым плоскость пересекает выпуклые полигональные грани полиэдра. По мере нахождения будем добавлять отрезки в список Ls . Перед началом обработки i -й грани $g = G_i$, где $i \in [1, size(G)]$, инициализируем пустой список точек $U = \emptyset$. В процессе обхода

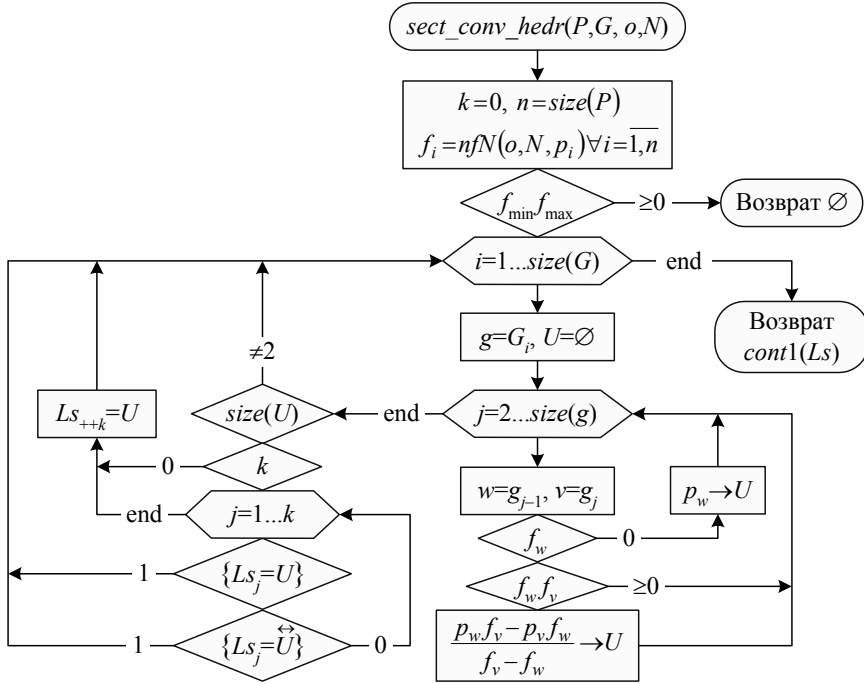


Рис. 1.170

рёбер $p_w p_v$ с индексами вершин $w = g_{j-1}$ и $v = g_j$, где $j \in [2, \text{size}(g)]$, добавляем в список U **некратные** точки касания (со значением $f_w = 0$) или пересечения (с произведением $f_w f_v < 0$) этих отрезков с плоскостью $\{o, N\}$.

По завершении обхода грани при условиях, что в списке $U = \{u_1, u_2\}$ ровно **две** точки и они в любом сочетании U либо $\bar{U} = \{u_2, u_1\}$ не встречаются ни в одном из элементов $L_{s_j} \forall j = \overline{1, k}$, где $k = \text{size}(L_s)$, запоминаем U как новый простой отрезок $L_{s_{++k}}$. Кратность отрезка в сечении полиэдра появляется при прохождении секущей плоскости по общему ребру двух смежных граней. А если какая-нибудь грань касается плоскости $\{o, N\}$ единственной вершиной, то её список $U = \{u_1\}$ будет состоять всего из одной точки. Для исключения некорректных данных обработка таких граней завершается без изменения списка L_s .

На последнем этапе алгоритма из отрезков **непустого** списка L_s (благодаря замкнутости полиэдра число этих отрезков не менее трёх) с помощью функции cont1 (см. рис. 1.68,а) собирается полигон сечения.

Работа алгоритма иллюстрируется построенными на рис. 1.166,б сечениями случайных выпуклых полиэдров случайными плоскостями.

Переходя к созданию алгоритма построения сечения **произвольного**, в том числе и невыпуклого, полиэдра, отметим **свойства невыпуклого решения** (рис. 1.171,а):

- возможно существование **нескольких** $k > 1$ фрагментов сечения;

- грань, имеющая форму невыпуклого полигона, может рассекаться плоскостью по **нескольким** отрезкам;
- полигоны сечения $C_1, \dots, C_k$ могут быть **невыпуклыми**.

Как и в алгоритме *sect_conv_hedr*, полигоны сечения невыпуклого полиэдра $\{P, G\}$ собираются из отрезков, по которым плоскость $\{o, N\}$ пересекает или касается его граней. Отметим, что в отличие от выпуклого решения **не все рёбра произвольного полиэдра, лежащие в секущей плоскости, могут входить в состав полигонов сечения**. Это существенно затрудняет отбор необходимых отрезков в цикле обработки граней полиэдра. К примеру, сечение на рис. 1.171,б состоит из двух фрагментов — полигонов $C_1 = p_1 p_6 p_{11} p_{13} p_1$ и $C_2 = p_5 q_1 q_2 p_{10} p_5$. Однако рёбра $p_5 p_6$ передней и $p_{10} p_{11}$ задней граней, входящие также в состав грани $p_5 p_{10} p_{11} p_6 p_5$, не принадлежат полигонам сечения. Необходим алгоритм, который бы правильно отбирал отрезки пересечения граней с секущей плоскостью и собирал из них полигоны сечения.

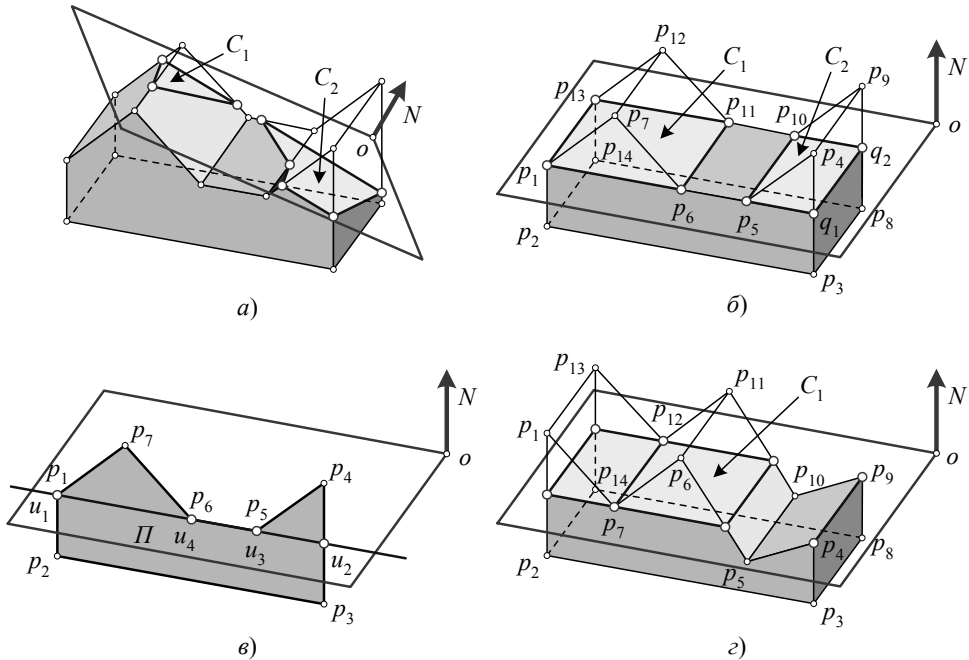


Рис. 1.171

Именно такой алгоритм $\text{sect_hedr}(P, G, o, N)$ построения сечения плоскостью $\{o, N\}$ произвольного полиэдра $\{P, G\}$, заданного списком вершин $P = \{p_1, \dots, p_n\}$ и группой списков граней $G = \{G_1, \dots, G_m\}$, представлен на рис. 1.172. Его работа начинается с инициализации пустого списка отрезков $Ls = \emptyset$ и счётчика их числа $k = 0$.

Далее, как и в алгоритме *sect_conv_hedr*, вычисляются значения неявной функции $f_i = \eta f N(o, N, p_i)$ во всех n вершинах полиэдра p_i , после чего, анализируя их экстремальные значения, принимается решение о существовании сече-

ния. При $f_{\min}f_{\max} \geq 0$ сечение отсутствует, а алгоритм немедленно возвращает пустой список полигонов $LP=\emptyset$. В противном случае числа $f_{\min}$ и $f_{\max}$ имеют разные знаки, вершины полиэдра находятся по разные стороны секущей его плоскости и алгоритм приступает к расчёту непустого решения.

Сформировав полигон i -й грани $Q=side(P,G,i)$ с первой вершиной q_1 , вычисляем её нормаль $H=norm(Q)$ и при условии $|H \times N| \neq 0$ непараллельности грани и секущей плоскости приступаем к расчёту их пересечения. Обходя рёбра грани $g=G_i$ по замкнутому контуру, добавляем **некратные** точки их пересечения с секущей плоскостью в предварительно проинициализированный пустой спи-

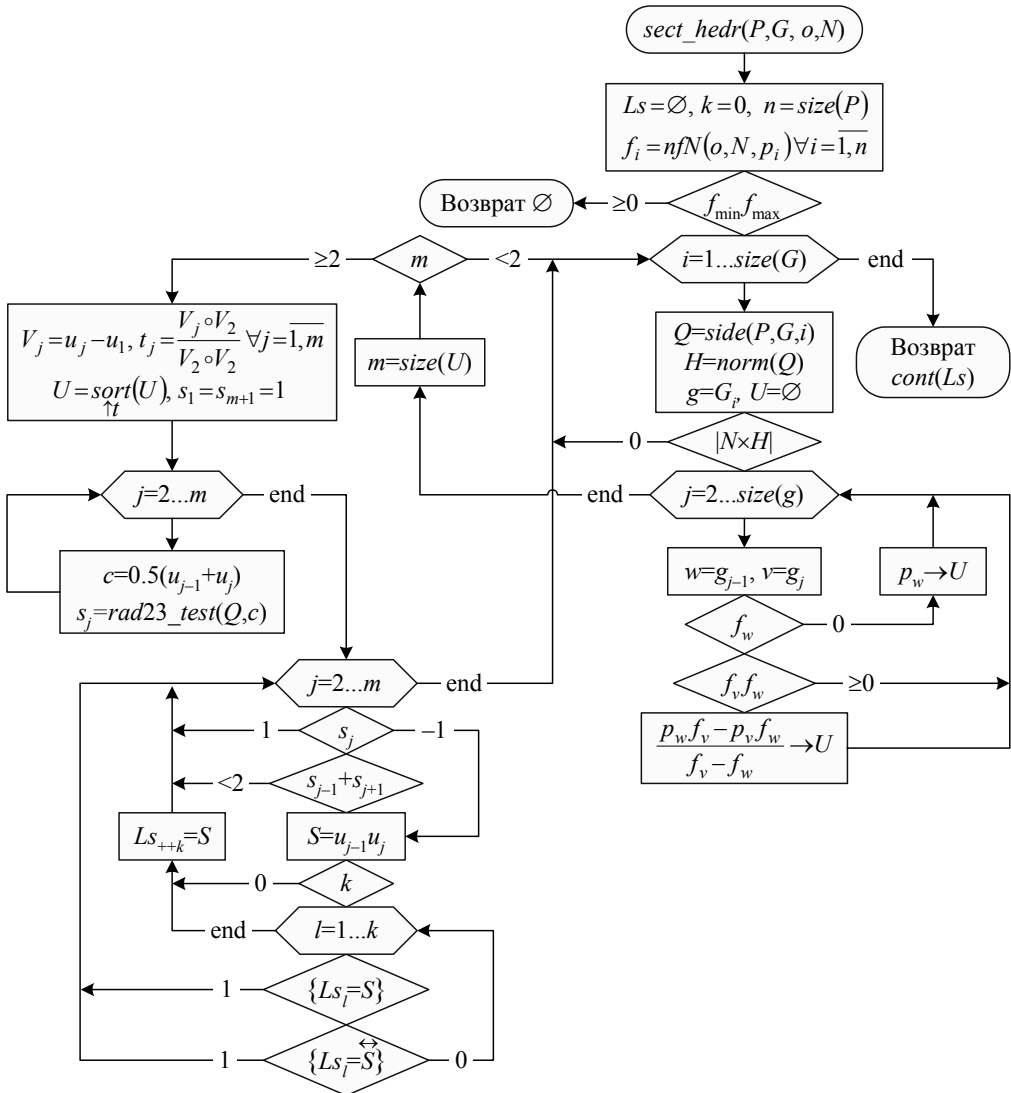


Рис. 1.172

сок $U=\emptyset$. По окончании обхода проверяем размер полученного списка $U=\{u_1, \dots, u_m\}$. Если в нём менее двух точек, то грань Q не пересекается с плоскостью $\{o, N\}$ либо касается её лишь одной вершиной, что также означает отсутствие пересечения. В противном случае необходимо решить, какие отрезки между *геометрически* (а не в порядке очерёдности в списке U) соседними вершинами могут формировать полигоны сечения (см. рис. 1.171,б).

Для этого отсортируем точки $u_1 \div u_m$ вдоль несущей их прямой по критерию возрастания вычисленных по (1.22) значений параметра

$$t_j = (V_j \circ V_2) / (V_2 \circ V_2) \quad \forall j = \overline{1, m}, \text{ где } V_j = u_j - u_1.$$

После геометрического упорядочивания точек u_j по возрастанию значений t_j получаем набор отрезков $u_1 u_2, u_2 u_3, \dots, u_{m-1} u_m$, из которых в состав сечения входят лишь те, что лежат либо внутри полигона грани, либо на его границе, но окружены внешними отрезками.

Организуем проверку расположения отрезков с помощью плоскостного объёмного теста локализации $rad23_test(Q, c)$ (см. рис. 1.127,в). Присвоив переменным $s_j \in \{-1, 0, 1\} \quad \forall j = \overline{2, m}$ результаты тестирования средних точек отрезков $c = 0.5(u_{j-1} + u_j)$ и зафиксировав значения $s_1 = s_{m+1} = 1$ внешней локализации полубесконечных концов линии пересечения, сформулируем вышеназванные **условия отбора отрезков** $u_{j-1} u_j \quad \forall j = \overline{2, m}$ в математической форме:

$$\{s_j = -1\} \vee \{s_j = 0\} \wedge \{s_{j-1} = 1\} \wedge \{s_{j+1} = 1\}.$$

Если отобранный отрезок в любом варианте $S = u_{j-1} u_j$ либо $\overleftrightarrow{S} = u_j u_{j-1}$ не совпадает ни с одним из элементов $Ls_i \quad \forall i = \overline{1, k}$, то он добавляется в Ls . Кратный отрезок в сечении полиэдра появляется при касании двух смежных граней с секущей плоскостью по их общему ребру, как отрезок $p_1 p_{13}$ на рис. 1.171,б.

На завершающем этапе алгоритма из отрезков, накопленных в списке Ls , формируются фрагменты сечения полиэдра, составляющие список полигонов LP . В отличие от выпуклого решения, в котором для сборки единственного полигона сечения S используются **все** отрезки, входящие в Ls , теперь возможны следующие нетипичные, но имеющие право на существование ситуации (рис. 1.171,г):

- имеются одиночные отрезки (как $p_4 p_9$) или незамкнутые цепочки отрезков в местах одностороннего касания граней полиэдра с секущей плоскостью;
- существуют тупиковые отрезки (как $p_7 p_{12}$) или разветвляющиеся цепочки отрезков, не имеющие продолжения или замыкания;
- формируются полигоны сечения с более чем двумя коллинеарными вершинами, как, например, полигон C_1 .

Все эти проблемы успешно решаются функцией *cont* (см. рис. 1.71), корректно обрабатывающей одиночные и тупиковые отрезки и цепочки отрезков, а также минимизирующей возвращаемые полигоны.

1.6.5. Разрезание полиэдра

В отличие от отсечения с односторонней ориентацией результата операция **разрезания полиэдра** $H=\{P,G\}$ разделяет его на фрагменты, находящиеся с обеих сторон разрезающей плоскости. Каждый из отрезанных фрагментов представляет собой полиэдр с моделью $H_i=\{P_i,G_i\}$, состоящей из матрицы координат вершин P_i и группы списков граней G_i . Он является самостоятельным объектом сцены, существующим независимо от других объектов.

Разрезание плоскостью

Результатом разрезания полиэдра $H=\{P,G\}$ **одной** бесконечной плоскостью $\{o,N\}$ являются правые (со стороны нормали) и левые полиэдральные фрагменты (рис. 1.173). В совокупности k таких полиэдров составляют иерархический список $LH=\{H_1,H_2,\dots,H_k\}$, элемент которого $H_i=\{P_i,G_i\}$ представляет собой модель i -го фрагмента, состоящую из списка его вершин P_i и группы списков граней G_i . Если плоскость с полиэдром не пересекается, то итогом разрезания логично считать исходный полиэдр H .

Для разрезания **выпуклого** полиэдра можно воспользоваться алгоритмом *clip_side* (см. рис. 1.163):

- левый выпуклый полиэдр

$H_{\text{л}}=\text{clip_side}(P,G,o,N)$ образуется при отсечении полиэдра H полупространством плоскости $\{o,N\}$;

• правый выпуклый полиэдр $H_{\text{п}}=\text{clip_side}(P,G,o,-N)$ образуется отсечением полиэдра H полупространством плоскости $\{o,-N\}$ с инверсной нормалью;

- если $\{H_{\text{л}} \neq \{\emptyset, \emptyset\}\} \wedge \{H_{\text{п}} \neq \{\emptyset, \emptyset\}\}$, то $LH=\{H_{\text{л}}, H_{\text{п}}\}$, иначе $LH=\{H\}$.

Недостатком данного подхода является двойной расчёт одних и тех же отрезков пересечения секущей плоскости с гранями полиэдра. Попробуем реализовать заложенную в алгоритме *sect_hedr* идею их **однопроходного** получения в рамках алгоритма *cut_side(P,G,o,N)* **разрезания произвольного полиэдра** $\{P,G\}$ **плоскостью** $\{o,N\}$ с блок-схемой на рис. 1.174. Затем из этих отрезков собираются полигоны граней отрезанных полиэдров, заполняющие три списка:

- Pl — левых граней, лежащих в полупространстве плоскости $\{o,N\}$;
- Pr — правых граней, лежащих в полупространстве плоскости $\{o,-N\}$;
- Ps — внутренних и граничных граней в секущей плоскости $\{o,N\}$.

Работа алгоритма начинается с инициализации списков $Pl=Pr=\emptyset$ и формирования функцией *sect_hedr* (см. рис. 1.172) списка Ps из полигонов сечения полиэдра плоскостью. Если последние не пересекаются, то $Ps=\emptyset$, а алгоритм возвращает список элементов исходного полиэдра $\{P,G\}$.

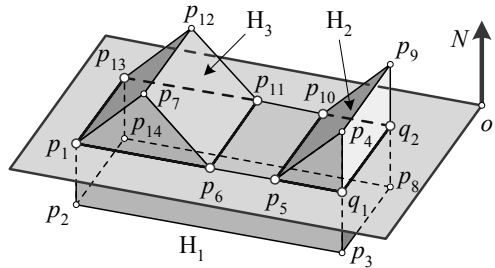


Рис. 1.173

Если пересечение существует, то запускается последовательная обработка граней полиэдра G_i . Сначала по (1.143) формируется полигон грани $Q \equiv q_1 q_2 \dots q_n q_1$ и по (1.144) вычисляется вектор её нормали $H = \text{norm}(Q)$. Для проверки параллельности плоскостей $\{o, N\}$ и $\{q_1, H\}$ находится вектор $V = N \times H$. Если $|V| = 0$, то единственной линии пересечения **параллельных** плоскостей не существует, но при условии $nfN(o, N, q_1) = 0$ грань Q **компланарна** секущей плоскости и целиком добавляется в список Ps .

Метод расчёта параметров прямой $p(t) = b + Vt$ пересечения **непараллельных** плоскостей $\{o, N\}$ и $\{q_1, H\}$ изложен в п. 1.5.2. Направляющий вектор этой линии равен $V = N \times H$, а ближайшая к началу СК точка b по (1.109б) равна

$$b = [o \circ N \quad q_1 \circ H] \begin{bmatrix} N \circ N & N \circ H \\ N \circ H & H \circ H \end{bmatrix}^{-1} \begin{bmatrix} N \\ H \end{bmatrix}.$$

Вычислив матрицу преобразования системы координат $C_K = MP2(q_1, \bar{H}, O_3, z^0)$, переводим объекты Q , b и V на плоскость $x'y'$, после чего с помощью алгоритма *cut_line* разрезаем полигон прямой линией на полигональные фрагменты и собираем их в списке L . Одностороннюю ориентацию каждого фрагмента относи-

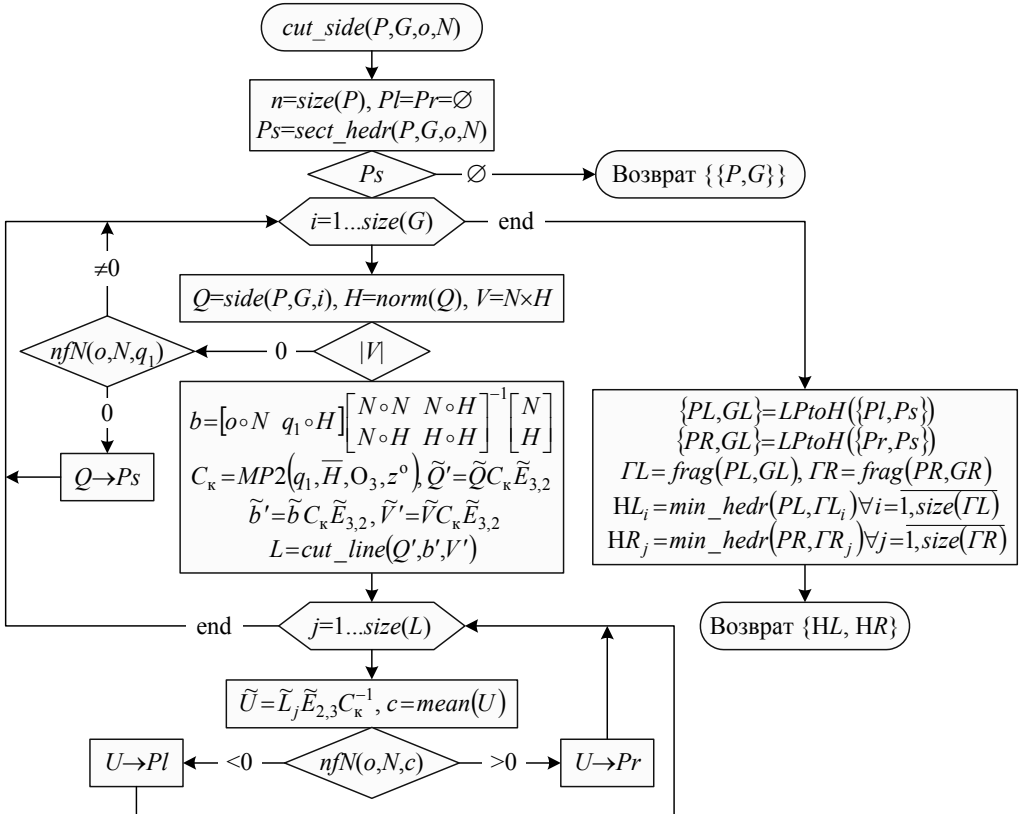


Рис. 1.174

тельно секущей плоскости определяем по значению $f_c = nfN(o, N, c)$ неявной функции средней точки фрагмента c :

- при $f_c < 0$ фрагмент является левым и заносится в список Pl ;
- при $f_c > 0$ фрагмент расположен справа и заносится в список Pr .

По окончании обработки граней полиэдра преобразуем соединённые списки полигонов $\{Pl, Ps\}$ и $\{Pr, Ps\}$ с помощью алгоритма LPH (см. рис. 5.8) в списки вершин и индексов левых $\{PL, GL\}$ и правых $\{PR, GR\}$ граней, после чего производим фрагментацию граней в группы списков GL и GR .

В последней операции функция min_hedr (см. рис. 1.125) формирует модели левых HL_i и правых HR_j полиэдров, каждая из которых включает **индивидуальные** списки вершин и граней отрезанного фрагмента. В завершение алгоритм возвращает соединённый список полиэдров $\{HL, HR\}$.

Разрезание на выпуклые полиэдры

Большинство методов и алгоритмов трёхмерной графики предназначено для работы с **выпуклыми** телами. Это обусловлено меньшей сложностью выполняемых операций и более тщательной продуманностью алгоритмов, таких как локализации точки $conv3_test$ (см. рис. 1.130), отсечения отрезка $clip3_test$ (см. рис. 1.153), вычисления объёма полиэдра по (1.156), визуализации методом Робертса (см. п. 1.7.2), отсечения полиэдра полупространством $clip_side$ (см. рис. 1.163), пересечения полиэдров $cross_hedr$ (см. рис. 1.168,а), сечения плоскостью $sect_conv_hedr$ (см. рис. 1.170) и т. п. В результате часто бывает выгоднее один раз представить сложный невыпуклый полиэдр объединением выпуклых, чем многократно нагружать компьютер сложными и, возможно, несовершенными универсальными алгоритмами обработки произвольных фигур.

Ручное выполнение декомпозиции невыпуклых полиэдров связано с многочисленными расчётами пересечения секущих плоскостей с полигонами граней, требует отлично развитого пространственного воображения и практически реализуемо лишь для не очень сложных тел, составленных из параллелепипедов, пирамид, призм и других 3d-примитивов.

Алгоритм $cut_hedr(P, G, LH)$ рекурсивного разрезания невыпуклого полиэдра $\{P, G\}$ на выпуклые фрагменты, блок-схема которого построена на рис. 1.175, во многом сходен с 2d-алгоритмом cut_poly разрезания невыпуклого полигона хордами (см. рис. 1.90,з) и использует умение описанного выше алгоритма cut_side разрезать полиэдр произвольной плоскостью.

В первую очередь необходимо найти у полиэдра любую вогнутую грань, по которой должна пройти разрезающая плоскость. Это делается методом, в точности совпадающим с тестом выпуклости полиэдра $conv3$ (см. рис. 1.123,в):

- в цикле $i = 1, size(G)$ формируется полигон грани $Q_i = side(P, G, i)$ с первой вершиной q_1 и сбрасываются флажки левого $l=0$ и правого $r=0$ расположения точек относительно плоскости $\{q_1, N_i\}$, где $N_i = norm(Q_i)$ — нормаль грани, **не обязательно внешняя**;

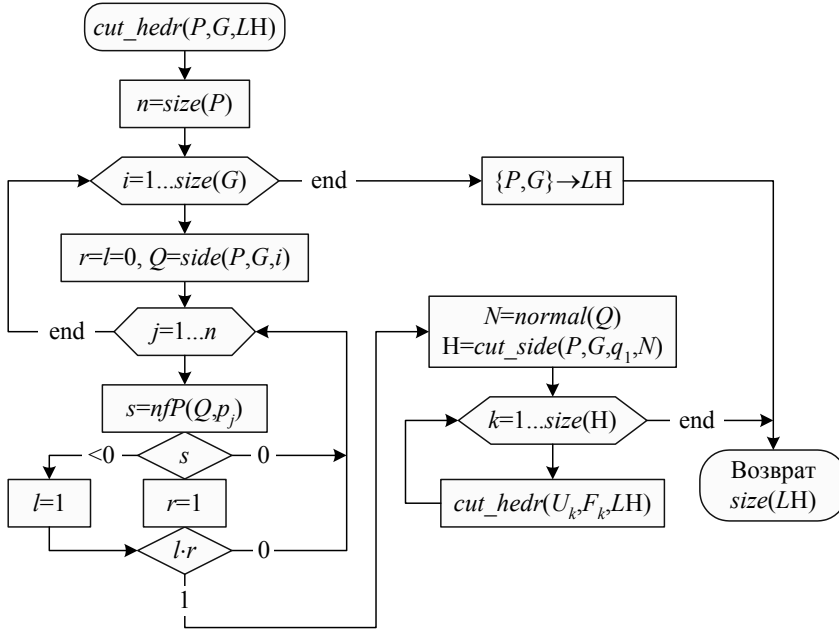


Рис. 1.175

• во вложенном цикле $j = \overline{1, \text{size}(P)}$ определяется расположение вершин полиэдра p_j и устанавливаются соответствующие флажки: $l=1$ при значении неявной функции $f(p_j) < 0$ и $r=1$ при $f(p_j) > 0$. В качестве тестовой функции $f(p)$ можно использовать функции $\text{nfN}(q_1, N_i, p)$ или $\text{nfP}(Q_i, p)$ из (1.108);

• если для некоторой грани G_i оба флажка окажутся установленными в $l=r=1$, то эта грань является вогнутой и алгоритм переходит на разрезание полиэдра её плоскостью $\{q_1, N_i\}$;

• при $l \cdot r = 0 \forall i$ выпуклый полиэдр $\{P, G\}$ не нуждается в разрезании и целиком добавляется в список полиэдров LH .

Разрезание выполняется с помощью алгоритма $\text{cut_side}(P, G, q_1, N_i)$, возвращающего группу полиэдров H . Её элемент представляет собой структуру $H_k = \{U_k, F_k\}$, включающую список вершин U_k и группу списков граней F_k k -го полиэдрального фрагмента. Рекурсивно разрезая каждый невыпуклый полиэдр $\{U_k, F_k\}$, алгоритм накапливает, таким образом, в списке LH выпуклые фрагменты с **индивидуальным** описанием вершин и граней благодаря использованию в функции cut_side алгоритма минимизации min_hedr (см. рис. 1.125).

Перед внешним обращением к функции cut_hedr с параметрами исходного полиэдра $\{P, G\}$ необходимо инициализировать пустой список $LH = \emptyset$, в который внутри функции будут добавляться выпуклые фрагменты, отрезанные от исходного полиэдра.

1.7. УДАЛЕНИЕ В ПРОСТРАНСТВЕ

Удаление невидимых линий и поверхностей является, пожалуй, наиболее сложной задачей визуализации сложных сцен с большим числом объектов. Она заключается в построении на экране дисплея изображений лишь тех графических объектов либо их фрагментов, которые видны наблюдателю с его точки зрения [6, 28, 46, 50, 56, 58, 61, 65].

Полная либо частичная невидимость элементов сцены обусловлена двумя причинами:

- одни из них находятся вне поля зрения наблюдателя — сбоку или сзади от призмы или пирамиды видимости (см. рис. 1.151). Исключением этих элементов из списка визуализации занимаются задачи отсечения;
- объекты второй группы заслонены (**экранированы**) другими непрозрачными объектами, более близкими к наблюдателю. Обнаружение таких объектов и их реалистичное изображение на экране с учётом эффекта экранирования составляет **задачу удаления невидимых элементов**.

Важность решения задачи удаления легко объясняется на примере перехода от каркасной модели фигуры, у которой видны все рёбра независимо от их расположения, к поверхностной модели с непрозрачными гранями, способными экранировать некоторые дальние рёбра. Это существенно повышает информативность восприятия расположения объекта. Например, привычное изображение куба каркасом из двенадцати рёбер (рис. 1.176,а) не позволяет однозначно судить о его действительной ориентации в пространстве. Удаление невидимых рёбер позволяет избавиться от неоднозначности восприятия и изображает истинное расположение объекта (рис. 1.176,б,в), даже если его грани бесцветны или монохромны.

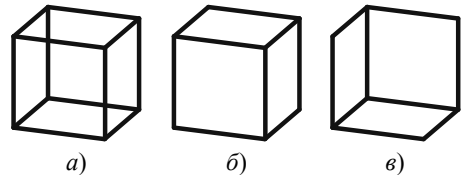


Рис. 1.176

Эффект экранирования дальних объектов ближними легко объясним с позиций прямого и обратного **методов трассировки лучей** [28, 35]. Пусть вдоль некоторой определённой прямой в глаз наблюдателя попадают **прямые** лучи, излучённые источниками либо отражённые поверхностями, играющими роль источников вторичных, третичных и т. д. прямых лучей (рис. 1.177,а). Лишь луч от самого **ближнего** к наблюдателю источника не встретит на своем пути пре-

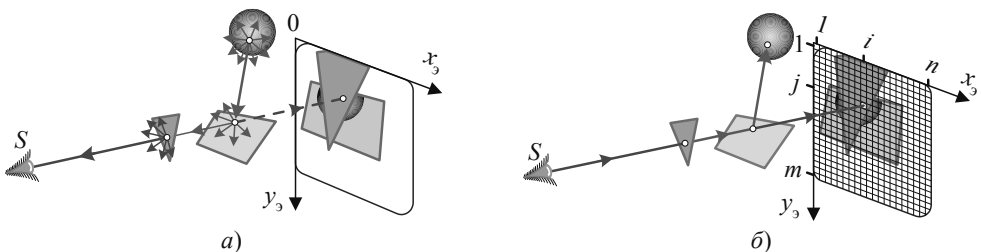


Рис. 1.177

град и благополучно достигнет своей цели, а глаз увидит навстречу данному направлению точку, окрашенную в цвет ближайшего источника света. Остальные прямые лучи того же направления поглотятся или рассеются на поверхностях встреченных объектов.

Полагая, что из глаза наблюдателя вдоль той же прямой выпущен **обратный** луч, проследим его путь до **первого** пересечения с **ближайшей** непрозрачной поверхностью (рис. 1.177,б). Точки пересечения с остальными объектами, «нанизанные» на продолжение обратного луча, оказываются экранированными, поэтому глаз их не видит и они подлежат удалению из общего списка.

1.7.1. Классификация методов удаления

Несмотря на кажущуюся очевидность и простоту, удаление невидимых линий и поверхностей является достаточно сложной задачей визуализации, требующей расчёта множества пересечений графических объектов с привлечением больших машинных ресурсов. Поэтому существует целый ряд алгоритмов её решения, включая и методы, опирающиеся на аппаратные средства.

Методы удаления невидимых элементов сцены классифицируются по следующим признакам (рис. 1.178).

❑ Во-первых, по **типу объектов**. Здесь водораздел проходит между объектами с разным числом степеней свободы, т. е. между **линиями** и **поверхностями**. Необходимость в удалении невидимых линий возникает при работе как с сугубо одномерными 3d-объектами, например, тонкими нитями или осями координат, так и каркасными поверхностями с непрозрачными ячейками, пересекающимися по рёбрам, некоторые из которых могут быть невидимы (см. рис. 1.176,б,в). Конечно, подавляющее число объектов машинной графики обладает некоторой поверхностью, отчего и методы, ориентированные только на удаление линий, представляют меньшинство в общей массе алгоритмов данного назначения.

❑ Второе, чем различаются алгоритмы удаления, — это используемые **модели объектов**, из которых основными являются **аналитические** (явные и неявные), **параметрические** и **полигональные**. Аналитическое описание поверхно-

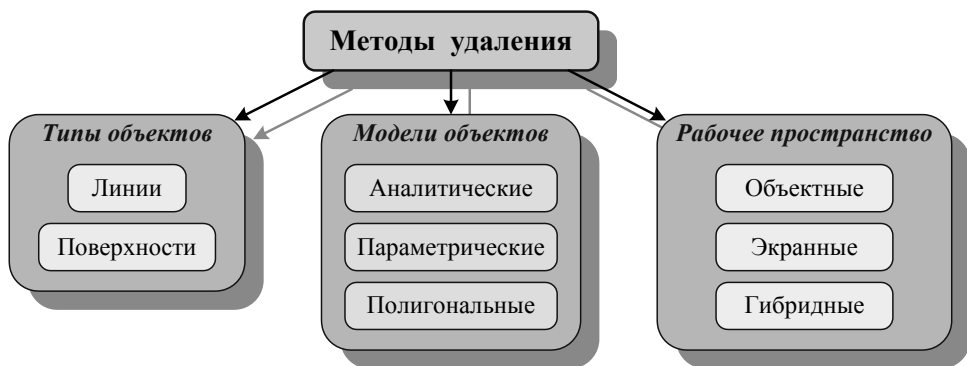


Рис. 1.178

сти неявным уравнением $f(p)=0$ полезно для вычисления по (1.93а) вектора нормали $N(p)$ с целью определения видимости точки p , а также применяется в расчёте ориентации и пересечения в рамках общей задачи удаления. Большинство же алгоритмов удаления работает с поверхностями, представленными системой полигональных граней, в основном, треугольников и четырёхугольников (см. рис. 1.142). Такой регулярно упорядоченной **полигональной сетью** (*mesh*) с узловыми вершинами $u_{ij} = p(t_i, \tau_j)$ представляется параметрическая поверхность $p(t, \tau)$ с помощью квантования параметров t и τ .

□ Третий признак классификации — по выбору **рабочего пространства**:

- **объектные** алгоритмы работают в физической системе координат сцены. Элементы объектов — вершины, рёбра, грани, объёмы — расчленяются на фрагменты, которые затем упорядочиваются по удалению от наблюдателя. Результат представляет собой список видимых элементов, вычисленных с машинной точностью, не связанной с растровыми свойствами дисплея. В связи с этим объектные алгоритмы часто называют **непрерывными**. Изображение видимых объектов, полученное в объектном окне, создаётся на экране путём несложного пересчёта объектных координат в экранные пиксели.

Трудоёмкость объектных алгоритмов, сравнивающих каждый из N_0 объектов с каждым оставшимся, оценивается зависимостью N_0^2 . Отсюда и главный недостаток непрерывных методов — их достаточно высокая сложность;

- **экранные** алгоритмы удаляют проекции невидимых элементов в ЭСК. К примеру, при переборе по строкам, столбцам или в ином порядке **конечного** числа пикселей экрана находятся точки пересечения сканирующего луча со всеми объектами сцены (см. рис. 1.177,б). На луче, соединяющем наблюдателя с (i, j) -м пикселем, формируется набор конкурирующих точек. Пиксел закрашивается цветом точки, ближайшей к наблюдателю. Степень детализации генерируемого изображения ограничена разрешающей способностью экрана (n пикселей по горизонтали и m по вертикали), из-за чего экранные алгоритмы называют **дискретными** или **растровыми**. Им свойственны ошибки квантования, особенно заметные при масштабном увеличении полученного изображения.

Ресурсоёмкость простейшего дискретного алгоритма, сравнивающего проекцию каждой из N_0 граней с каждым из nm пикселей, пропорциональна произведению $N_0 nm$ и, например, при $n=m \approx 512$ намного превышает трудоёмкость объектного алгоритма с числом объектов $N_0 \approx 1000$. В связи с бурным развитием технологии СБИС и резким удешевлением электронной памяти вопрос ресурсов практически снят с повестки дня. Это позволило относительно простым дискретным алгоритмам визуализации (не только удаления, но также закрашивания пикселей, геометрического и цветового сглаживания) внедриться в программное обеспечение видеоадаптера каждого современного компьютера;

- **гибридные** алгоритмы работают как с объектами в пространстве, так и с их экранными проекциями, объединяя преимущества и минимизируя недостатки объектных и экранных алгоритмов, неся в себе ряд оптимизаций.

Большинство алгоритмов удаления включает в себя *сортировку*. Главная сортировка ведется *по геометрическому расстоянию* от точки наблюдения до поверхности, ребра или точки объекта. Смысл сортировки по глубине заключается в том, что чем дальше расположен объект от наблюдателя, тем больше вероятность того, что он будет полностью либо частично заслонён другим объектом, более близким к наблюдателю. Дополнительная сортировка объектов по горизонтали и/или вертикали позволяет оценить действительную возможность перекрытия экранных проекций объектов. Весьма эффективно в этом смысле применение габаритных тестов и охватывающих оболочек различной степени иерархии — от оболочек отдельных объектов до оболочек *кластеров* — групп пространственно связанных объектов.

Для повышения эффективности визуализации путём учёта предыстории состояния сцены часто используется свойство её *когерентности*, т. е. тенденции неизменяемости свойств видимости объектов в малой области рабочего пространства. Выделяют несколько *видов когерентности*:

- когерентность *в картинной плоскости*: если данный пиксел соответствует проекции некоторой грани, то, скорее всего, соседние пикселы также соответствуют точкам этой же грани;
- когерентность *в объектном пространстве*: если данная грань видима, то соседняя грань также видима;
- *временная когерентность* при создании анимации: свойства видимости объектов в соседних кадрах, вероятнее всего, сохраняются.

Громадный объём вычислений предъявляет жёсткие требования к аппаратному обеспечению компьютера — процессору, оперативной памяти и системной шине, по которой данные передаются в видеоадаптер, а также к самому видеоадаптеру. Сравнительно недавно появились специализированные устройства обработки изображений — *графические ускорители*, решающие задачи визуализации на аппаратном уровне и делающие экранные алгоритмы предпочтительнее объектных. Поскольку центральный процессор не всегда способен достаточно быстро обработать весь поток данных, эта задача перекладывается непосредственно на специализированную плату-акселератор, имеющую «на борту» собственный графический процессор и дополнительную память для буферизации данных. Применение графических ускорителей позволяет распараллелить и тем самым ускорить процесс обработки информации.

Взаимодействие «ускоряемой» программы с тем или иным набором микросхем осуществляется с помощью *драйверов*, а на программном уровне — *графических библиотек*. Из наиболее популярных библиотек можно назвать Direct3D в составе пакета DirectX, OpenGL и QuickTime3D. Графические 3d-библиотеки, или так называемые *интерфейсы программных приложений* (API — *Application Programming Interface*), предназначены для того, чтобы с помощью некоторых общих методов обработки графических данных облегчить и ускорить процесс создания приложений, использующих трёхмерную графику — объёмные объекты, свет, оптические эффекты, перспективу, деформации и т. д. При работе над тем или иным графическим эффектом программисту необ-

ходимо обратиться к библиотечной функции, передать ей требуемые параметры и получить готовый результат. Многие графические библиотеки, например, Direct3D, рассчитаны на комбинированную работу как с акселератором, так и с центральным процессором. Если акселератор в системе не найден или не поддерживает данную функцию, то библиотека «переложит» всю работу на центральный процессор и эффект всё равно будет достигнут, правда, за счёт несколько большего времени.

В зависимости от сложности сцены (числа её элементов), типа объектов (выпуклые или не выпуклые), наличия пересечения объектов друг с другом, степени компромисса между желаемой скоростью и реалистичностью визуализации существующие алгоритмы удаления более или менее учитывают эти и другие особенности. Следует ясно понимать, что **не придумано ещё универсального метода решения задачи удаления**.

Разработка нового алгоритма, аккумулирующего достижения отдельных методов, нередко выливается в чрезвычайно громоздкую процедуру, медленно работающую даже с простыми сценами. Понятно, что для обработки информации в реальном времени с частотой более 30 кадров в секунду, например, в программах авиатренажёров или «крутых» компьютерных игр (между прочим, двигателей компьютерного прогресса), такие алгоритмы не годятся. С другой стороны в компьютерном кинематографе временной фактор отходит на второй план: здесь более важна глубокая проработка оптических эффектов, позволяющая достичь небывалой реалистичности или, наоборот, фантастичности изображения. По мере создания более быстрых алгоритмов и способных их реализовать видеокарт всегда будут возникать новые потребности в скорости, детальности и минимальной стоимости обработки данных.

Переходим к изучению **методов удаления невидимых элементов**. Разработано великое множество алгоритмов удаления. Часть их описана в книгах [1, 6, 17, 23, 47, 50, 56, 58, 61, 62, 64, 65], другая часть — в библиографии к ним, третья — на специализированных сайтах и форумах. Прочие менее известные методы оказались вне пределов упоминания, что, однако, не лишает их права на существование. Одни алгоритмы просты, но применимы только в частных случаях; другие — более универсальны, но и трудны для понимания, сложны в программировании и требуют больших машинных ресурсов.

Многообразие методов удаления, их названий (функциональных и именных) и продолжающаяся разработка новых методов свидетельствует об активности и перспективности исследований в этом направлении. Наиболее обстоятельное, полное, детальное (объём списка эпитетов стремится в бесконечность) изложение различных методов удаления приведено в книге Д. Роджерса [50]. Вместо подробного пересказа хорошо всем известных алгоритмов здесь будет приведён лишь краткий обзор их работы. Основное же внимание будет уделено развитию **геометрического** подхода к решению задач удаления. Не претендуя на изобретение велосипеда, автор всего лишь предпринимает попытки оригинального решения подобных задач в надежде, что его алгоритмы будут востребованы в вычислительной практике.

1.7.2. Обзор типовых методов удаления

Самые известные и практически востребованные алгоритмы визуализации основаны на сортировке элементов сцены по определённым факторам.

□ Простейший в реализации, но часто вполне достаточный, объектный **метод Робертса** работает с **выпуклыми** полиэдрами и выводит на экран только **лицевые** грани, обращённые внешней стороной к наблюдателю. Идентификация i -й грани заключается в оценке **угла видимости** $\psi_i = \angle N_i H_i$ между вектором **внешней** нормали N_i и **вектором наблюдателя** H_i (рис. 1.179). Отметим, что в сцене с близким наблюдателем (а) векторы H_i от точек разных граней c_i **различны**, а при удалении наблюдателя в бесконечность (б) он видит сцену в параллельных лучах и все векторы H_i **одинаковы**. Благодаря планарности грани вся её поверхность обладает константной видимостью, поэтому точка c_i может быть взята, например, в **центре грани** либо первой в списке её вершин.

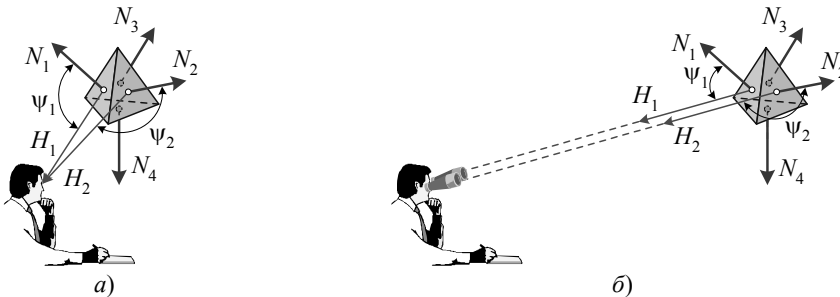


Рис. 1.179

Грань является лицевой при остром угле видимости, что быстро определяется **лицевым тестом** знака скалярного произведения векторов

$$N_i \circ H_i > 0 \quad (1.181)$$

без вычисления самого угла ψ_i . Более подробно вопросы отбраковки нелицевых граней и выбора системы координат, в которой эта процедура выполняется максимально быстро, рассматриваются в разд. 5.5.

Достоинство метода Робертса — отсутствие сортировки точек поверхности по удалению от наблюдателя и **произвольный порядок вывода лицевых граней**. Иначе говоря, очередность изображения граней, прошедших лицевой тест (1.181), зависит только от их нумерации в группе списков граней G либо от порядка обработки ячеек параметрической поверхности между узлами u_{ij} .

Для изображения **невыпуклых** полигональных объектов **Р. Шумейкером** предложена модификация метода Робертса, названная **методом приоритетного списка** и заключающаяся в присвоении каждой грани определённого **приоритета** вывода по следующим правилам (рис. 1.180):

- приоритеты граней выпуклого полиэдра равны;
- граням, не способным экранировать друг друга, также назначается одинаковый приоритет;

• чем внутреннее находится грань и больше вероятность её экранирования другими гранями, тем бóльшим задаётся приоритет её вывода.

Значения **статических** приоритетов граней полиэдра задают в процессе формирования структуры G (см. п. 1.5.4), нумеруя в ней списки граней G_i в порядке **уменьшения** приоритетов. Порядок следования граней с равными приоритетами произволен. Расширим список i -й грани атрибутом видимости, динамически устанавливаемым в значение $\{N_i \circ H_i > 0\}$ на ка-

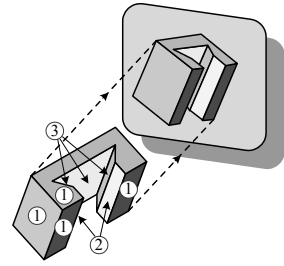


Рис. 1.180

ждом кадре визуализации. Если выводить на экран только лицевые грани в порядке их перечисления в списке G , то автоматически обеспечится корректное экранирование проекций внутренних граней проекциями внешних.

Незамкнутая поверхность (см. рис. 1.165) также может изображаться методом приоритетного списка, если определять видимость внутренних сторон граней с **внешними** нормальными N_i и высшим приоритетом по правилу $N_i \circ H_i < 0$.

□ Ещё один объектный метод приоритетного вывода граней полигональных объектов известен как **алгоритм художника**. Он предназначен для изображения **произвольных** поверхностей и выводит на экран **все** ячейки, независимо от их видимости, по мере **приближения** к наблюдателю. Проекция ближних граней могут частично или полностью наложиться на ранее построенные проекции дальних граней подобно тому, как художник наносит на холст новый мазок поверх ранее нанесённого мазка другого цвета, тем самым скрывая его от зрителя.

Метод основан на вычислении **динамических** приоритетов — **глубин** (дальностей) d_i центров граней $c_i = \text{mean}(P_i)$ (в разных вариантах алгоритма используются и другие опорные точки — самые близкие либо самые дальние вершины полигонов $P_i = \text{side}(P, G, i)$) к наблюдателю, сортировке и выводе ячеек в порядке **уменьшения** элементов вектора d .

При ближнем наблюдателе, находящемся в конечно удалённой **точке** S (рис. 1.181,а), глубина точки c_i равна расстоянию

$$d_i = |c_i - S|, \quad (1.182a)$$

а в вычислительном аспекте более эффективен квадрат расстояния (1.182а), равный скалярному произведению $(c_i - S) \circ (c_i - S)$. Равноудалённые от наблюдателя точки образуют концентрические сферы с центрами в S .

Если наблюдатель удалён в бесконечность в направлении **вектора** S , то расстояния от него до всех точек c_i также бесконечны, а сортировку можно делать по относительным глубинам, отсчитываемым вдоль оси d , направленной навстречу S (рис. 1.181,б). Запишем проекцию радиус-вектора точки c_i на вектор S :

$$\text{pr}_S c_i = t_i S, \text{ где } t_i = c_i \circ S / |S|^2.$$

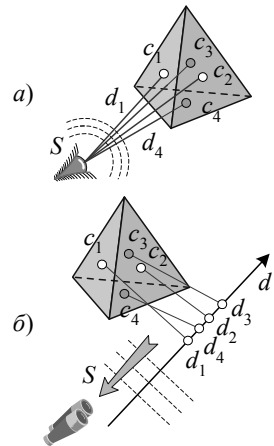


Рис. 1.181

При увеличении параметра t_i точка c_i приближается к наблюдателю. Следовательно, в качестве **эквивалента глубины**, который должен уменьшаться с ростом t_i , можно принять легко вычисляемое скалярное произведение

$$D_i = -c_i \circ S. \quad (1.182б)$$

Равноудалённые от наблюдателя точки лежат в параллельных плоскостях, ортогональных вектору S .

Алгоритм художника отличается высоким быстродействием, но, к сожалению, имеет серьёзный **недостаток: ячейки каркаса должны быть примерно одинакового и достаточно малого размера в сравнении с габаритами поверхности**. В определённом ракурсе разновеликие ячейки могут быть выведены на экран в неверном порядке, что иллюстрируется на рис. 1.182 изображениями боковой поверхности цилиндрической призмы с прямоугольными гранями. Ориентация призмы (а) выбрана так, что центр длинной грани c_1 расположен к наблюдателю чуть-чуть ближе, чем центр короткой грани c_2 . Вращение объекта (б) всего лишь на угол $\varphi_y = 2^\circ$ вокруг левого вертикального ребра расположило точку c_1 вдоль линии зрения немного дальше точки c_2 , из-за чего вторая грань была выведена на экран позже первой и неестественно её перекрыла. Этот дефект визуализации был присущ ранним версиям пакета MathCAD, но затем он был устранён, видимо, благодаря использованию другого, более качественного метода изображения поверхностей.

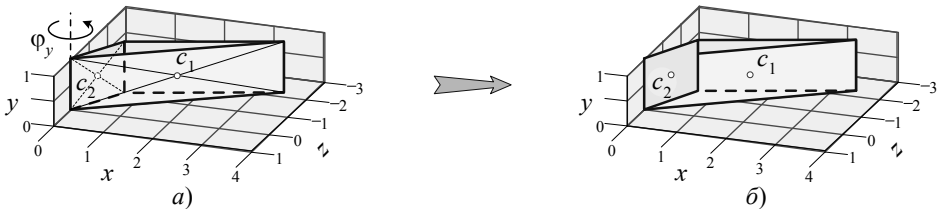


Рис. 1.182

Не самым худшим средством восстановления работоспособности алгоритма художника является ручное или автоматическое разбиение граней полиэдров на приблизительно равновеликие простейшие фигуры, например, треугольники. Для этого можно использовать разработанный в п. 1.4.5 алгоритм контролируемой триангуляции полигона *contri_poly* (см. рис. 1.96).

Но наиболее серьёзный **недостаток** алгоритма художника связан с **принципиальной** невозможностью установления приоритетов вывода протяжённых граней по глубинам их выборочных точек в следующих случаях (рис. 1.183):

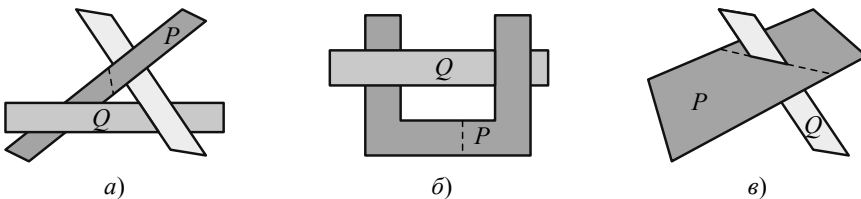


Рис. 1.183

- при циклическом экранировании как выпуклых (*a*), так и невыпуклых (*б*) граней друг другом;
- при «протыкании» одной грани другой (*в*), что также равносильно их взаимному экранированию.

В [50] описан метод **М. Ньюэла – Р. Ньюэла – Т. Санчи**, разрешающий данные конфликты формирования приоритетов. Исходными данными служат полигоны граней полиэдров, собранные в список $LP = \{G_1, \dots, G_\gamma\}$. Для уменьшения размера списка приблизительно вдвое рекомендуется включать в него только лицевые грани. Далее будем обозначать *два первых полигона* списка как $P \equiv G_1$ и $Q \equiv G_2$, а габаритные глубины их вершин как $P_{\min}, P_{\max}, Q_{\min}$ и $Q_{\max}$.

Правильный вывод граней на экран связан как с локальной сортировкой вершин каждого полигона G_i с получением *габаритной глубины* $[G_{i\min}, G_{i\max}]$, так и глобальной сортировкой полигонов по глубинам d_i , рассчитываемым по (1.182) для точек $c_i = G_{\max}$ — максимально удалённых вершин полигонов.

Для максимально быстрого обнаружения самой дальней грани, не экранирующей никакие другие, в алгоритме выполняется серия *тестов* как с самими полигонами P и Q , так и с их экранными проекциями P_3 и Q_3 . При положительном результате проведённого теста проекция P_3 целиком записывается в буфер кадра видеопамати, а полигон P исключается из списка LP , после чего серия тестов выполняется с начала с новым составом списка LP . По достижении пустого списка все грани из буфера кадра выводятся на экран.

Сущность **алгоритма Ньюэла – Ньюэла – Санчи** следующая.

Шаг 1. Сортировка граней $G_i \forall i = \overline{1, \gamma}$ *по убыванию* глубин максимальных габаритов $G_{i\max}$.

Шаг 2. Тест *неперекрывания* габаритных глубин полигонов P и Q (рис. 1.184,а):

$$P_{\min} \geq Q_{\max}.$$

Шаг 3. Тест *неперекрывания* габаритных оболочек проекций P_3 и Q_3 (см. рис. 1.27) с помощью функции $gab2_poly(P_3, Q_3)$. В её определении (1.50б) нужно строгие неравенства заменить на нестрогие.

Шаг 4. Тест расположения всего полигона $P = p_1 p_2 \dots p_n p_1$ с позиции наблюдателя *за плоскостью* полигона $Q = q_1 q_2 \dots q_m q_1$ (рис. 1.184,б):

$$((p_i - q_1) \circ N_Q)(H_Q \circ N_Q) \leq 0 \quad \forall i = \overline{1, n}, \quad (1.183a)$$

где $N_Q = \text{norm}(Q)$ — нормаль к плоскости полигона Q , а H_Q — вектор, проведённый от полигона Q к наблюдателю, равный S при дальнем расположении последнего и $S - q_1$ — при ближнем.

Шаг 5. Тест расположения всего полигона Q *перед плоскостью* полигона P относительно наблюдателя (рис. 1.184,в):

$$((q_i - p_1) \circ N_P)(H_P \circ N_P) \geq 0 \quad \forall i = \overline{1, m}, \quad (1.183б)$$

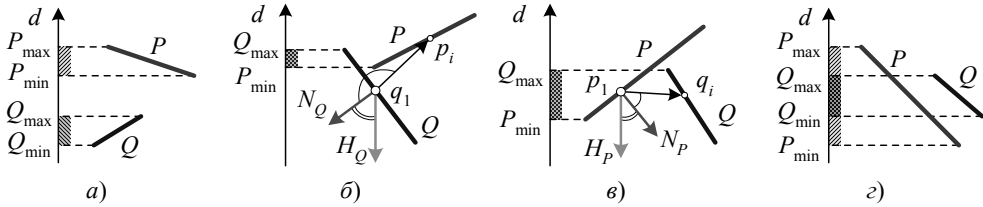


Рис. 1.184

где $N_P = \text{norm}(P)$ — нормаль к плоскости полигона P , а H_P — вектор, проведённый от полигона P к наблюдателю, равный S при дальнем расположении последнего и $S - p_1$ — при ближнем.

Шаг 6. Тест $\{P_3 \cap Q_3 = \emptyset\}$ **неперекрывтия** проекций P_3 и Q_3 , заключающийся в решении задачи пересечения полигонов $\text{mul2}(P_3, Q_3)$, рассмотренной в п. 1.4.2 (см. рис. 1.83, а).

Шаг 7. Перестановка полигонов P и Q :

- первая перестановка разрешает изображённый на рис. 1.184, б конфликт поглощения глубины заднего полигона Q глубиной ближнего полигона P путём повторения **шагов 3 ÷ 6** с обновлённым списком LP ;
- вторая перестановка устраняет циклическое перекрытие грани P путём её разрезания плоскостью грани Q (см. рис. 1.183). Полученные фрагменты $P_1 \div P_N$ добавляются в список LP , а полигон P из него удаляется, после чего алгоритм переходит на **шаг 1** с новым списком LP .

□

Операция разрезания на **шаге 7** нуждается в развёрнутом пояснении:

- сначала методом, изложенным в п. 1.5.2, находится линия $\{o, V\}$ пересечения плоскостей $\{p_1, N_P\}$ и $\{q_1, N_Q\}$ (см. рис. 1.104);
- для перевода операции из пространства на плоскость полигона P вычисляется матрица преобразования $C_K = MP2(p_1, N_P, O_3, z^0)$ и находится положение объектов $\tilde{P}' = \tilde{P}C_K$, $\tilde{o}' = \tilde{o}C_K$ и $\tilde{V}' = \tilde{V}C_K$ в новой системе координат $x'y'z'$;
- полигон P' разрезается прямой $\{o', V'\}$ на N фрагментов $P'_1 \div P'_N$ алгоритмом *cut_line*, изложенным в п. 1.4.4 (см. рис. 1.89);
- вычисляется положение полигонов $\tilde{P}_i = \tilde{P}'_i C_K^{-1} \quad \forall i = \overline{1, N}$ в исходной СК xuz

Начиная вывод граней с самых дальних от наблюдателя, алгоритм Ньюэла – Ньюэла – Санчи не требует предварительной отбраковки нелицевых граней, не удовлетворяющих тесту (1.181). Однако, эта несложная процедура сокращает список обрабатываемых граней приблизительно вдвое и **весьма желательна**. При условии, что все грани в списке LP лицевые с нормальными N_i , обращёнными к наблюдателю, получим более простые правила сортировки граней: вместо (1.183а) используются неравенства $(p_i - q_1) \circ N_Q \leq 0 \quad \forall i = \overline{1, n}$, а вместо (1.183б) — неравенства $(q_i - p_1) \circ N_P \geq 0 \quad \forall i = \overline{1, m}$.

□ Фигура с одностонно закрашенными полигональными гранями выглядит, особенно на близком расстоянии, как лоскутное одеяло со ступенчатым изменением освещённости на стыке граней. Это объясняется постоянным цветом закраски каждого пиксела проекции грани. Наиболее реалистичным является **точечное изображение** поверхности с индивидуальной закраской каждой её видимой точки. Эффект объёмности достигается благодаря игре цвета, света и тени в исполнении искусного программиста-художника.

Самым известным методом точечной визуализации произвольных поверхностей является впервые предложенный **Э. Кэтмулом** простой и достаточно эффективный **алгоритм z-буфера** [50, 65], ныне аппаратно реализованный во всех видеокартах. Сущность метода заключается в переборе пикселей, принадлежащих проекции поверхности, восстановлении положения соответствующих точек на самой поверхности и записи в видеопамять цветовых кодов **ближайших** к наблюдателю точек из всех, имеющих одинаковые экранные координаты. Название метода объясняется использованием z -координаты для вычисления удалённости точки от наблюдателя в случае прохождения проецирующих лучей параллельно оси z (см. рис. 1.151,а). В других методах проецирования дальность находится как-то иначе, но устоявшееся название метода, тем не менее, сохранилось.

Для реализации процесса сортировки точек по глубине сцены в памяти ЭВМ создаются две матрицы:

- **буфер глубины Z** , хранящий параметры удаления точек сцены, ближайших к наблюдателю и потому им видимых. Объём z -буфера равен $width \times height$ слов, где $width = right - left + 1$ и $height = down - up + 1$ — размеры окна вывода на экране дисплея, а разрядность слова N_z определяется числом 2^{N_z} уровней квантования диапазона дальностей точек;

- **буфер кадра F** , хранящий цвета точек окна вывода. Объём буфера кадра равен $width \times height$ слов, разрядность которых N_F определяется числом 2^{N_F} уровней квантования цветовых оттенков.

Алгоритм работы z -буфера поясняется блок-схемой на рис. 1.185,а. Сразу отметим, что **все вычисления ведутся в экранной системе координат**. Из всех полигонов поверхностей всех объектов сцены составляется список $LP = \{P_k\}$ с **произвольной** нумерацией элементов. Вначале выполняется **инициализация буферов** глубины и кадра фоновыми значениями дальности и цвета. Фон — это любая картинка в окне вывода в отсутствие объектов. Дальность фона задаётся числом, бóльшим дальности самой дальней точки сцены. Например, это может быть максимальное число разрядной сетки, условно говоря, ∞ .

На каждом кадре визуализации очередной полигон P_k выбирается из списка LP , проецируется на экран, отсекается окном вывода (рис. 1.185,б,в), после чего начинается сканирование точек экранной проекции P_k (рис. 1.185,г). Этот процесс называется **растеризацией**, т. е. разложением в **растр** — двухмерный массив пикселей, заполняющих контур проекции. Обработка пикселей раstra может выполняться в **произвольном** порядке, но обычно она происходит построч-

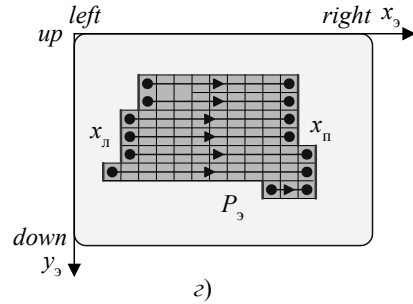
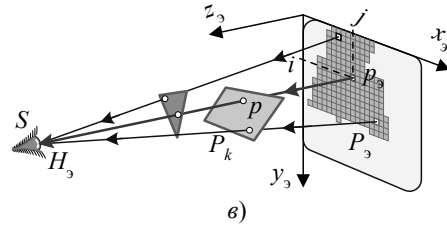
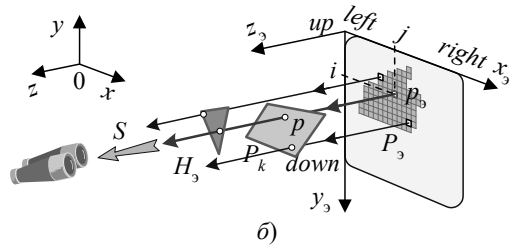
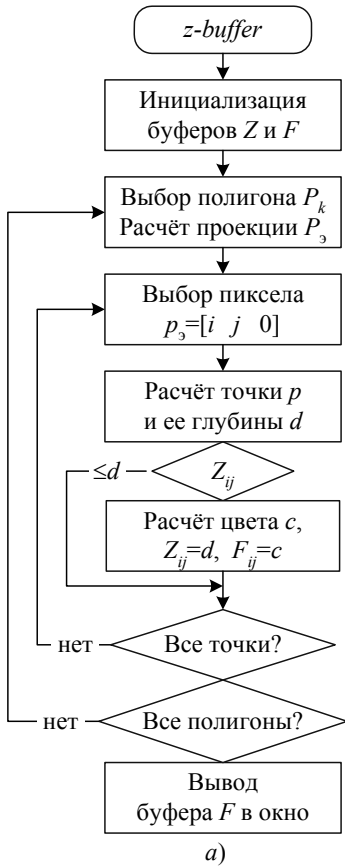


Рис. 1.185

но слева направо и сверху вниз. Многообразие растровых алгоритмов, составляющих отдельный раздел компьютерной графики, приведено в [6, 47, 50, 61 – 64 и др.], а один из них будет рассмотрен в п. 4.2.3.

Для каждого пиксела $p_3 = [i \ j \ 0]$ восстанавливается образ точки $p = [x \ y \ z]$ по модели поверхности, на которой она лежит. Геометрически это означает проведение из пиксела p_3 сканирующей прямой в направлении H_3 и расчёт её пересечения с поверхностью грани.

Вид вектора H_3 обусловлен типом наблюдателя:

- в случае дальнего наблюдателя (рис. 1.185,б) фиксированный вектор $H_3 = S$ соответствует заданной параллельной проекции;
- в сцене с ближним наблюдателем в точке S (рис. 1.185,в) вектор $H_3 = S - p_3$.

Метод расчёта пересечения прямой $r(\theta) = p_3 + H_3 \theta$ с объектом определяется моделью ограничивающей его поверхности:

- для неявной модели $f(p) = 0$ решается скалярное уравнение $f(p_3 + H_3 \theta) = 0$ относительно параметра θ , дающего точку $p = r(\theta)$. В частном случае плоскости $\{o, N\}$ с НФ $f(p) = (p - o) \circ N$ получим $p = \text{cross_line}(p_3, H_3, o, N)$;

- пересечение прямой с параметрической поверхностью $p(t, \tau)$ находится решением векторного уравнения $p_3 + H_3 \theta = p(t, \tau)$, из которого $p = r(\theta)$ при условии принадлежности параметров $\{t, \tau\}$ интервалам определения поверхности.

После вычисления по (1.182) дальности d найденной точки p проверяется состояние (i, j) -го элемента буфера глубины Z_{ij} . При $d < Z_{ij}$ точка p находится к наблюдателю ближе, чем другие ранее обработанные точки (либо фон) с такими же экранными координатами $[i \ j]$. Тогда в z -буфере сохраняется новое состояние $Z_{ij} = d$, а в пиксел буфера кадра F_{ij} записывается цветовой код c , вычисленный с помощью широко известных моделей освещённости и функций закраски [20, 50, 58, 61, 65]. Таким образом, из нескольких точек, «нанизанных» на (i, j) -ю сканирующую прямую, наблюдатель увидит лишь самую *ближнюю*. Если на прямой нет ни одной точки, то цвет пиксела останется фоновым.

По окончании обоих циклов содержимое буфера кадра копируется в экранное окно вывода сцены, начиная со строки up и столбца $left$.

Главные **преимущества** алгоритма z -буфера в сравнении с другими точечными методами визуализации [50] — простота и произвольный порядок обработки поверхностей, позволяющий экономить машинные ресурсы на специальную сортировку по глубине. Вычислительная трудоёмкость алгоритма линейно зависит от числа ячеек и площади экрана, покрываемой их проекциями.

Основными **недостатками** рассматриваемого алгоритма являются:

- большой объём памяти, занимаемой двумя буферами при высоком графическом разрешении и большой площади окна вывода. Также имеет тенденцию увеличение разрядностей кодирования цвета и глубины точек. С развитием элементной базы острота проблемы нехватки данных ресурсов снижается;
- необходимость предварительной растровой развёртки проекции поверхности, что в случае её криволинейности представляет собой непростую задачу. Например, не ортогональной проекцией поверхности сферы является эллипс общего вида, для которого достаточно не просто получить интервал изменения координаты y_3 и зависимости границ $x_{\text{л}}(y_3)$ и $x_{\text{п}}(y_3)$;
- сложность реализации свойства полупрозрачности из-за произвольного порядка вывода поверхностей на экран, тогда как для видимости других объектов сквозь полупрозрачную поверхность она должна выводиться последней.

Рассмотрим некоторые растровые методы визуализации, по возможности устраняющие недостатки базового алгоритма z -буфера [50].

□ Для уменьшения потребляемой памяти **А. Майерсом** предложено сократить высоту буферов кадра и глубины до *одной* строки экрана, т. е. сделать значение $height = 1$. В полученном **методе построчного сканирования с z -буфером** однострочная развёртка поверхности представляет собой экранную проекцию линии её **сечения плоскостью зрения** Π , проходящей через крайние точки сканирующей строки $[left \ i], [right \ i]$ и точку S расположения ближнего наблюдателя (рис. 1.186,а). Отметим, что эта плоскость почти всегда не ортогональна картинной плоскости. В случае же наблюдателя, удалённого в бесконечность по

вектору S , плоскость зрения проходит через третью точку $[left\ i]+S$ и, например, при $S \parallel z^0$, всегда ортогональна экрану (рис. 1.186,б).

Наиболее сложным является расчёт плоского сечения криволинейной поверхности, не аппроксимированной полигональными гранями (рис. 1.186,а). Здесь приходится формировать множество точек пересечения сканирующего луча с кривой линией (а может быть и не с одной) сечения, чтобы после сортировки по глубине занести в буфер кадра ближайшую к наблюдателю точку. Аналитические формулы решения данных задач существуют лишь для узкого класса неплоских поверхностей и кривых линий, в основном, квадратичного типа (см. п.п. 1.1.7, 1.5.9). В остальных случаях приходится прибегать к услугам медленных численных методов решения нелинейных уравнений.

Плоская же грань пересекается с плоскостью зрения по прямому отрезку, имеющему не более одной точки пересечения с лучом, но всё равно эти расчёты должны выполняться в пространстве (рис. 1.186,б). Поэтому эффективнее будет сначала найти экранную проекцию абриса поверхности или грани, а затем построочно решать на плоскости x_3y_3 задачу **внешнего отсечения** горизонтального отрезка $\{[left\ i].[right\ i]\}$ полученным полигоном (замкнутая кривая тоже является полигоном с близко расположенными вершинами). Несколько алгоритмов отсечения отрезка полигоном разработано в п. 1.4.1. Из них наибольший интерес представляет универсальный алгоритм *clip2_seg* (рис. 1.79,з).

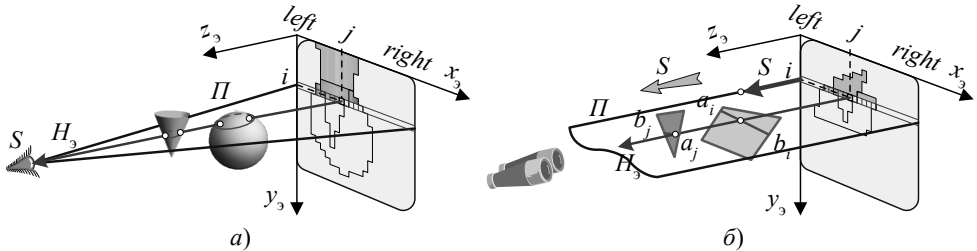


Рис. 1.186

В результате в сканирующей строке формируется список проекций отрезков $Ls = \{a'_1b'_1 \dots a'_nb'_n\}$, после чего начинается их попиксельная обработка с помощью Z - и F -буферов строки, предварительно инициализированных фоновыми значениями глубины и цвета. По завершении перебора содержимого списка Ls буфер F отображается в сканируемой строке.

□ Дальнейшее сокращение расчётов дальностей конкурирующих отрезков a_ib_i в плоскости зрения Π достигается в рамках **интервального алгоритма построеного сканирования**, впервые предложенного Д. Уоткинсом для точечной визуализации **полигональных** объектов. Суть оптимизации в разбиении сканирующей строки концами проекций $a'_ib'_i$ на интервалы с константной удалённостью граней от наблюдателя и быстром определении ближайшей грани в единственной средней точке каждого интервала (рис. 1.187,а). При пересечении отрезков a_ib_i и a_jb_j друг с другом в список границ интервалов включаются проекции точек пересечения c_{ij} .

Практически интервальное разбиение может быть реализовано с помощью **списка границ интервалов** B и **буфера интервалов**, совмещённого с буфером строки F ввиду их использования в разные моменты времени.

Шаг 1. Инициализируется нулевой буфер кадра $F_k=0 \quad \forall k=\overline{left, right}$ и граничный список $B=\{left, right\}$.

Шаг 2. Для каждого полигона G_i из списка граней $LP=\{G_1, \dots, G_\gamma\}$ находится отрезок $a_i b_i$ пересечения с плоскостью зрения Π . Если $a_i b_i \neq \emptyset$, то:

- рассчитывается экранная проекция отрезка $a_i' b_i'$ и выполняется её внешнее отсечение $a_i'' b_i''$ границами сканирующей строки $[left, right]$ (см. отрезок $a_1 b_1$);
- концы **непустой** проекции $a_i'' b_i'' \neq \emptyset$ **логически** (для исключения кратности, см. проекции $a_3'' = b_2''$) добавляются в список границ: $B = B \cup \overline{a_i'' b_i''}$;
- инкрементируется интервал буфера строки $F_k++ \quad \forall k=\overline{a_i'', b_i''}$.

Шаг 3. Рассчитываются точки c_{ij} пересечения пар отрезков $\{a_i b_i, a_j b_j\} \quad \forall i \neq j$, имеющих пересекающиеся проекции $a_i'' b_i'' \cap a_j'' b_j'' \neq \emptyset$ (см. пары отрезков $\{a_1 b_1, a_2 b_2\}$ и $\{a_1 b_1, a_3 b_3\}$). При **непустом** результате $c_{ij} \neq \emptyset$ проекция c_{ij}' **логически** (см. проекции $c_{23}' = b_2''$) добавляется в список границ: $B = B \cup c_{ij}'$.

Шаг 4. После сортировки $B = \text{sort}(B)$ буфер F оказывается разделённым на интервалы $\Delta_i = [\alpha_i, \beta_i]$, $i = \overline{1, N}$, с равными значениями $F_k = \text{const} \quad \forall k = \overline{\alpha_i, \beta_i}$.

Шаг 5. Анализируется число F_{m_i} в середине каждого интервала $m_i = \lceil 0.5(\alpha_i + \beta_i) \rceil \in \Delta_i$ и принимается решение о заполнении отрезка буфера строки $F_k \quad \forall k = \overline{\alpha_i, \beta_i}$:

- при $F_{m_i} = 0$ интервал Δ_i **пуст**, а в буфер заносится отрезок фона;
- если $F_{m_i} = 1$, то интервал Δ_i покрыт проекцией **одного** отрезка $a_j'' b_j'' \supset m_i$, а пиксели буфера строки в интервале $[\alpha_i, \beta_i]$ закрашиваются в цвета соответствующих точек j -й грани G_j ;
- в остальных случаях интервал Δ_i покрыт проекциями **нескольких** отрезков $a_j'' b_j'' \supset m_i$ с индексами j из множества $\Theta \subseteq [1, \gamma]$. Видимым на интервале будет от-

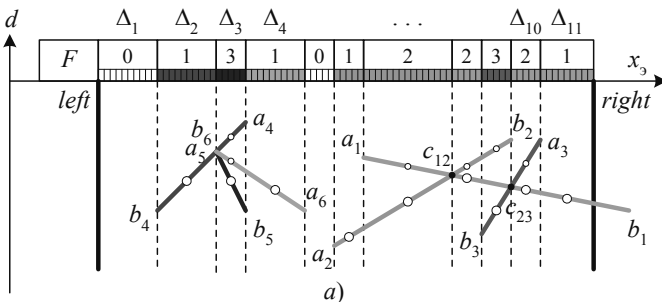


Рис. 1.187

резок $a_\omega b_\omega$ ($\omega \in \Theta$) с **минимальной** дальностью $d_\omega = \min_j \{d_j\}$, рассчитанной для точки m_i , а в буфер строки записываются цвета соответствующих точек ω -й грани G_ω .

Шаг 6. По окончании обработки всех N интервалов заполненный буфер строки выводится на экран. $\square$

Нетрудно видеть, что интервальный алгоритм построчного сканирования практически не нуждается в z -буфере: требуется всего лишь одна скалярная переменная d_ω для сортировки дальностей $d_j \forall j \in \Theta$. Алгоритм отличается высоким быстродействием в сравнении с другими растровыми методами и способен выполнять точечное реалистичное окрашивание частично экранированных плоских граней, освещённых близкими источниками света (рис. 1.187,б). Если же используется монотонная окраска граней, то скорость работы можно ещё больше повысить, закрашивая на **шаге 5** все пиксели интервала Δ_i одним цветом.

$\square$ Проблемой растровых методов визуализации произвольной поверхности является необходимость расчёта её **абриса** (экранной проекции внешнего контура) и сечений поверхности плоскостями зрения. Все эти вычисления нужны для того, чтобы сканировать только те пиксели экрана, куда попали проекции объектов сцены, а пропущенные пиксели по умолчанию заполнять цветом фона.

Расчёт точек пересечения сканирующих лучей с поверхностями, тем не менее, необходим для их сортировки по дальности. Так нельзя ли отказаться от действий, усложняющих визуализацию, и ограничиться лишь расчётом пересечений лучей с поверхностями? Рассмотрим три способа реализации этой идеи:

- во-первых, можно организовать развёртку прямоугольных проекций **габаритных оболочек** объектов (см. рис. 1.21) $C_k = [x'_{k\min}, x'_{k\max}] \times [y'_{k\min}, y'_{k\max}] \forall k = \overline{1, N}$, строящихся в экранной системе координат x_3, y_3, z_3 (рис. 1.188,а). Из каждого пиксела $(i, j) \in C_k$ выпускается луч в сторону наблюдателя и рассчитываются все точки его пересечения с k -й поверхностью. Запоминаются глубина и цвет точки, ближайшей к наблюдателю. Поскольку один и тот же пиксел может входить в несколько пересекающихся **поочерёдно** обрабатываемых прямоугольников C_k, C_m и т. д., то работа данного **метода сканирования габаритных оболочек** невозможна без z -буфера, в котором производится сортировка глубин проецирующихся в пиксел ближайших точек поверхностей. Пиксели, не входящие ни в одну из габаритных оболочек, остаются окрашенными в цвет фона, тогда как остальные пиксели принимают цвет ближайших точек поверхностей;

- обобщением предыдущего метода, эффективного лишь при небольшом числе объектов, является **метод сканирования габаритной оболочки сцены**, по которому сначала строится прямоугольная экранная оболочка V_c , охватывающая проекции всех объектов сцены, а затем из каждого пиксела $(i, j) \in C_c$ проводится сканирующий луч в поиске ближайшей из нанизанных на него точек поверхностей. Благодаря **однократности** просмотра пикселов теперь нет необходимости в z -буфере, что вдвое сокращает затраты памяти;

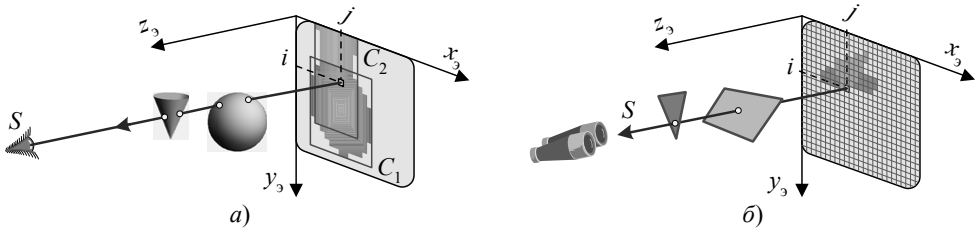


Рис. 1.188

- если габаритная оболочка сцены занимает большую часть экрана или даже полностью охватывает его, можно совсем отказаться от использования каких-либо оболочек для организации сканирования пикселей. Тогда мы приходим к простейшим вариантам **метода трассировки лучей** [35], в которых однократно выпускаются либо **прямые** сканирующие лучи из каждого пикселя экрана к наблюдателю (рис. 1.177,а, 1.188,б), либо **обратные** лучи от наблюдателя в каждый пиксел экрана (рис. 1.177,б). В отсутствие пересечения луча с какой бы то ни было поверхностью пиксел окрашивается в цвет фона, иначе — в цвет ближайшей к наблюдателю поверхности, проткнутой этим лучом. Повышение быстродействия этих вовсе не быстрых методов визуализации достигается сортировкой **непересекающихся** объектов по глубине сцены для того, чтобы после вывода первой пересекаемой лучом поверхности прекращать обработку других.

Помимо задачи простой визуализации 3d-объектов, т. е. удаления их невидимых элементов, методы трассировки лучей, особенно обратных, хорошо приспособлены для решения более сложных задач, чем геометрическое экранирование. Речь идёт об оптических эффектах простых и многократных отражений одних объектов на поверхностях других, взаимного затенения объектов при нескольких источниках света, изображения объектов, изготовленных из полупрозрачных материалов, с учётом эффекта преломления и т. д. [28, 35, 36, 38, 65].

Остаток главы посвятим разработке авторских методов удаления невидимых элементов графических объектов, основанных на геометрическом подходе. Будем полагать, что все анализируемые отрезки $a_i b_i$ и лицевые грани P_j собраны в двух общих списках $Ls = \{a_1 b_1, \dots, a_n b_n\}$ и $LP = \{P_1, \dots, P_m\}$. Введём также **признак близости наблюдателя** $nearH \in \{0, 1\}$, задаваемый равным 0 для дальнего и 1 для ближнего типа расположения этого субъекта сцены.

1.7.3. Удаление невидимых линий

Элементарный способ (называть его алгоритмом слишком амбициозно) изображения **выпуклого** каркасного тела с непрозрачными **плоскими** гранями, полностью экранирующими невидимые рёбра (см. рис. 1.176,б,в), заключается в выводе проекций контуров только **лицевых** граней (они же являются видимыми) на том основании, что **каждое ребро, окаймляющее видимую грань, также является видимым**. При этом рёбра между двумя лицевыми гранями рисуются дважды. Вовсе не факт, что дополнительные вычисления, необходимые для исключения двойного вывода рёбер, будут выгоднее, чем их отсутствие.

Если видимые рёбра некоторого k -го выпуклого полиэдра необходимы для дальнейшей работы, то из них нужно сформировать список Ls_k и удалить из него кратные элементы. Для этого просматриваются полигоны $P_i = p_{i1} \dots p_{in_i} p_{i1} \forall i \in \lambda_k$, где λ_k — список индексов граней k -го элемента в списке полигонов LP , и по алгоритму, приведённому на рис. 1.63,а, вычисляется **объединение** рёбер лицевых граней выпуклого полиэдра, не содержащее кратные отрезки:

$$Ls_k = \bigcup_{i \in \lambda_k} \bigcup_{j=1}^{n_i} p_{ij} p_{i,j+1}. \quad (1.184)$$

Рассмотрим общую задачу удаления отрезков, **произвольно** расположенных относительно плоских полигональных граней. Общность означает возможность **пересечения** отрезков с гранями и их **частичную** видимость вследствие экранирования (см. рис. 1.132,а).

Вначале разработаем алгоритм экранирования (*shielding*) отдельно взятого отрезка ab отдельно взятой полигональной **выпуклой** гранью $P = p_1 p_2 \dots p_n p_1$ с учётом их ориентации относительно друг друга и наблюдателя. Обозначим вектор отрезка $V = b - a$ и сформируем вектор наблюдателя $H = S - \text{near}H \cdot a$, принимающий при бесконечно удалённом наблюдателе с $\text{near}H = 0$ постоянное значение вектора S (рис. 1.189,а), а при $\text{near}H = 1$ значение вектора $S - a$, направленного в точку S расположения ближнего наблюдателя (рис. 1.189,б).

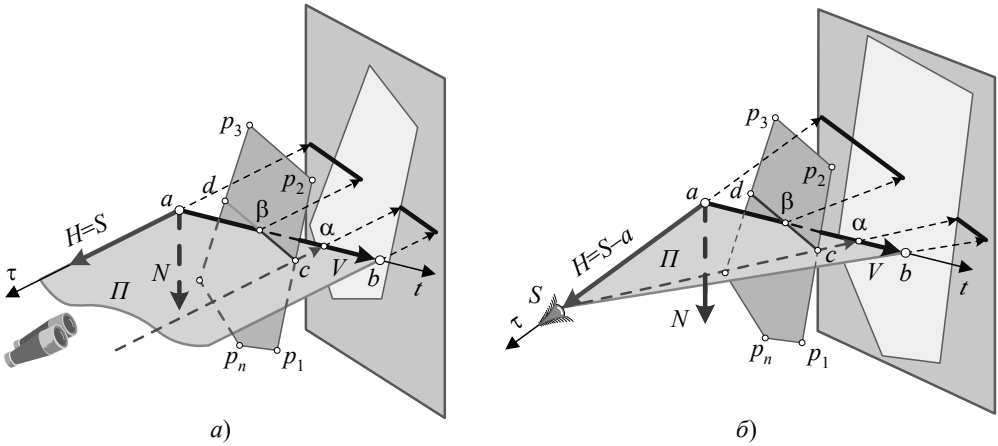


Рис. 1.189

Первое, что делает отрезок невидимым, — это его совпадение с лучом зрения, что быстро проверяется тестом $|V \times H| = 0$. Такой отрезок имеет экранную проекцию, вырожденную в точку, и должен быть исключён из анализа экранирования всеми гранями.

Второй особый случай касается совпадения плоскости грани с плоскостью зрения. Это вырождает её проекцию в отрезок нулевой площади, который не в состоянии экранировать никакой другой площадный объект. Если все грани объектов прошли тест видимости (1.181), то совпадающие с плоскостью зрения грани будут на этом этапе отбракованы.

Обратные лучи, проведённые от наблюдателя до точек не параллельного им отрезка ab , лежат в **плоскости зрения** $\Pi = \{a, N\}$ с нормалью $N = V \times H$. Экранирование отрезка гранью P возникает при выполнении двух условий:

- грань пересекается с плоскостью зрения по отрезку cd , который можно найти по алгоритму $sect_conv_hedr(P, G, a, N)$ сечения полиэдра **бесконечной** плоскостью (см. рис. 1.170), использующему список $G = \{G_1\}$ из одной грани $G_1 = \{1, \dots, n, 1\}$. Достоинство этого алгоритма — предварительное тестирование пересечения по (1.146) и возвращение пустого списка в его отсутствие. Это означает, что отрезок ab не экранируется полигоном P , или, другими словами, экранируется пустым отрезком;

- обратные лучи, проведённые от наблюдателя до точек отрезка ab (но не дальше!), по пути пересекают весь отрезок cd или хотя бы его часть.

Расчёт отрезков, экранирующих ab , проведём в следующем порядке.

 **Шаг 1.** Аналогично (1.43б) найдём параметры разложения векторов $c - a = Vt_c + H\tau_c$ и $d - a = Vt_d + H\tau_d$ в базисе $\{V, H\}$:

$$\begin{bmatrix} t_c & \tau_c \\ t_d & \tau_d \end{bmatrix} = \begin{bmatrix} (c-a) \circ V & (c-a) \circ H \\ (d-a) \circ H & (d-a) \circ H \end{bmatrix} \begin{bmatrix} V \circ V & V \circ H \\ V \circ H & H \circ H \end{bmatrix}^{-1}. \quad (1.185)$$

Шаг 2. Выполним внешнее **параметрическое** отсечение отрезка cd областью видимости $\tau \geq 0$, при необходимости смещая параметры задних точек c или d на границу $\tau = 0$:

- при $\{\tau_c \geq 0\} \wedge \{\tau_d \geq 0\}$ весь отрезок cd расположен от наблюдателя не дальше отрезка ab , поэтому параметры, вычисленные в (1.185), не изменяются;
- при $\{\tau_c > 0\} \wedge \{\tau_d < 0\}$ (рис. 1.189) ближняя часть отрезка cd примыкает к точке c , поэтому смещению подвергаются параметры точки d :

$$t_d = t_c + (t_d - t_c)\tau_c / (\tau_c - \tau_d), \quad \tau_d = 0; \quad (1.186a)$$

- при $\{\tau_c < 0\} \wedge \{\tau_d > 0\}$ смещаются параметры точки c :

$$t_c = t_d + (t_c - t_d)\tau_d / (\tau_d - \tau_c), \quad \tau_c = 0; \quad (1.186б)$$

- при $\{\tau_c \leq 0\} \wedge \{\tau_d < 0\} \vee \{\tau_c < 0\} \wedge \{\tau_d \leq 0\}$ весь отрезок cd отстоит от наблюдателя дальше, чем отрезок ab , т. е. экранирование отсутствует.

Шаг 3. В случае экранирования спроецируем ближнюю часть сечения грани плоскостью $\{a + Vt_c + H\tau_c, a + Vt_d + H\tau_d\}$ на прямую отрезка ab :

- при дальнем наблюдателе и параллельном проецировании получаем точки

$$\alpha = a + Vt_c, \quad \beta = a + Vt_d; \quad (1.187a)$$

- при ближнем наблюдателе и центральном проецировании из соотношений $(\alpha - a)/t_c = V/(1 - \tau_c)$ и $(\beta - a)/t_d = V/(1 - \tau_d)$ следует, что

$$\alpha = a + Vt_c / (1 - \tau_c), \quad \beta = a + Vt_d / (1 - \tau_d). \quad (1.187б)$$



Обобщая сказанное, построим на рис. 1.190 блок-схему алгоритма $conv_shield(ab, P, S, nearH)$ экранирования отрезка ab плоской полигональ-

ной **выпуклой** гранью $P=p_1p_2\dots p_np_1$. В нём наблюдатель задаётся объяснёнными выше параметрами S и $nearH \in \{0,1\}$. Алгоритм возвращает экранирующий отрезок $\alpha\beta \subseteq ab$ либо пустой результат в отсутствие экранирования.

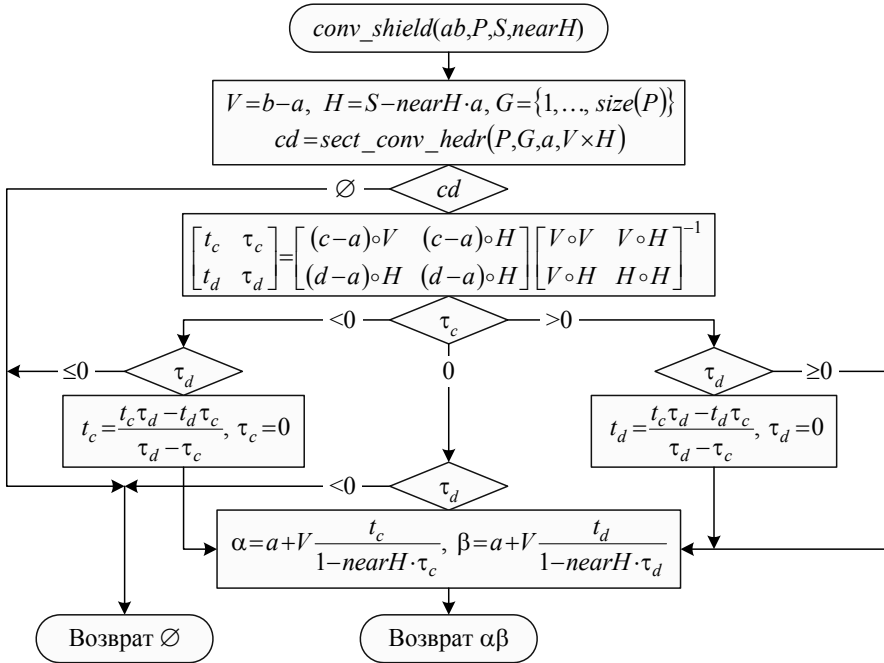


Рис. 1.190

Вспомня, что в нашей коллекции есть алгоритм *cross_poly* расчёта пространственного пересечения двух плоских полигонов (см. рис. 1.129,з), используем его для решения задачи экранирования отрезка ab произвольным, в том числе и **невыпуклым** полигоном $P=p_1p_2\dots p_np_1$ (рис. 1.191). Смысл идеи в том, что **все** отрезки, по которым P пересекается с **полигоном** (не плоскостью!) **видимости** Π , экранируют отрезок ab и не нуждаются в параметрическом отсече-

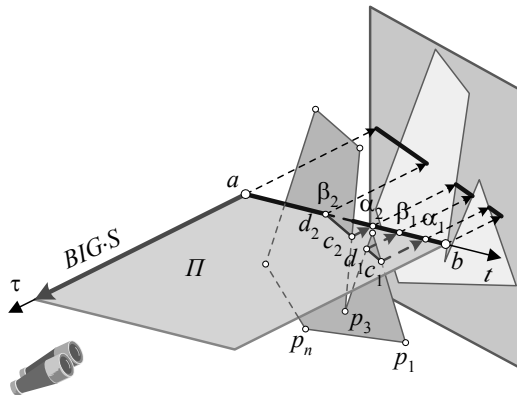


Рис. 1.191

нии по формулам (1.186). Достаточно для каждого отрезка $c_i d_i$ из списка Ls' , возвращённого функцией $cross_poly(P, \Pi, Ls', 1) = 1$, найти по (1.185) параметры концевых точек $\{t_{ci}, \tau_{ci}\}$ и $\{t_{di}, \tau_{di}\}$, спроецировать эти точки по (1.187) на прямую отрезка ab и сформировать конкатенацию полученных отрезков $\alpha_i \beta_i$.

Что касается способа задания полигона видимости, то для ближнего наблюдателя это треугольник $\Pi = abSa$, а для дальнего — полубесконечная полоса. Аналогично способу представления полуплоскости (см. п. 1.4.3) зададим большое число BIG , заведомо превышающее глубину сцены, и представим полосу видимости конечным полигоном $\Pi = \{a, b, b + BIG \cdot S, a + BIG \cdot S, a\}$.

Описанный метод реализован в виде алгоритма *shield*($ab, P, S, nearH$) экранирования отрезка ab плоской полигональной гранью $P = p_1 p_2 \dots p_n p_1$ с наблюдателем, задаваемым параметрами S и $nearH \in \{0, 1\}$ (рис. 1.192). Алгоритм возвращает список отрезков, экранирующих ab , либо пустой список в отсутствие экранирования.

Несмотря на простую блок-схему, алгоритм достаточно трудоёмок, т. к. основан на сложном алгоритме *cross_poly*. Поэтому в задачах визуализации объектов, все грани которых являются выпуклыми полигонами, рекомендуется использовать более простой и быстрый алгоритм *conv_shield*.

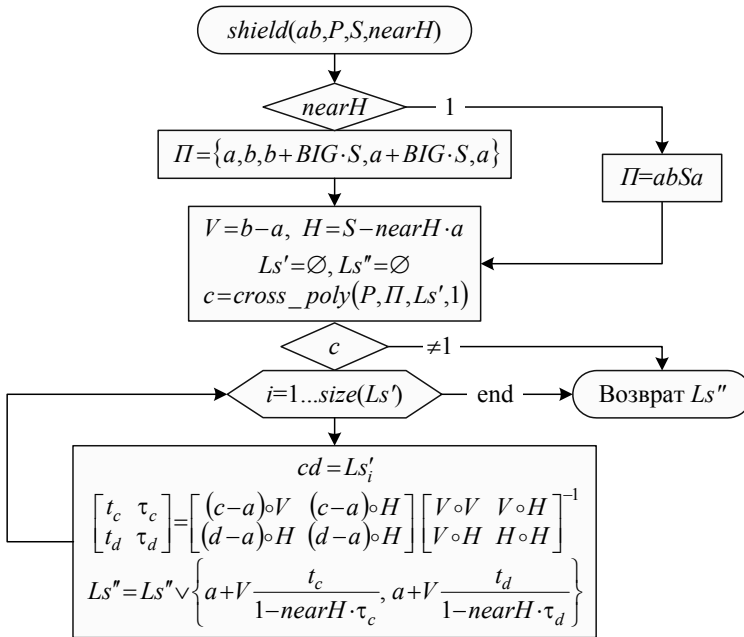


Рис. 1.192

Завершим тему разработкой процесса удаления отрезков списка $Ls = \{a_1 b_1, \dots, a_n b_n\}$, экранированных плоскими полигональными гранями, составляющими список $LP = \{P_1, \dots, P_m\}$. Если грани этого списка не проходят отбраковку, то читатель должен самостоятельно дополнить алгоритм расчётом их

нормалей $N_i = \text{norm}(P_i)$, скалярных произведений $\sigma_i = N_i \circ H$ и работать только с лицевыми гранями, у которых $\sigma_i > 0$.

Каркасное изображение объектов сцены создаётся как самостоятельными отрезками, так и рёбрами лицевых граней. Есть ещё отрезки взаимного пересечения граней, отсутствующие в исходных данных. Они рассчитываются по алгоритму *cross_poly*. Все эти отрезки объединяются в общий список Ls_0 . Для каждого отрезка Ls_{oi} , не совпадающего с лучом зрения, формируется список экранирующих отрезков Ls_{zi} , после чего исключение $Ls_{oi} - Ls_{zi}$ добавляется в список видимых отрезков Ls_B .

На основе сказанного на рис. 1.193 составлена блок-схема алгоритма *hide_seg(Ls, LP, S, nearH)* удаления отрезков списка Ls , экранируемых полигональными гранями списка LP , с наблюдателем, задаваемым параметрами S и $nearH \in \{0, 1\}$. Алгоритм возвращает список отрезков в границах их видимости наблюдателем в объектном пространстве.

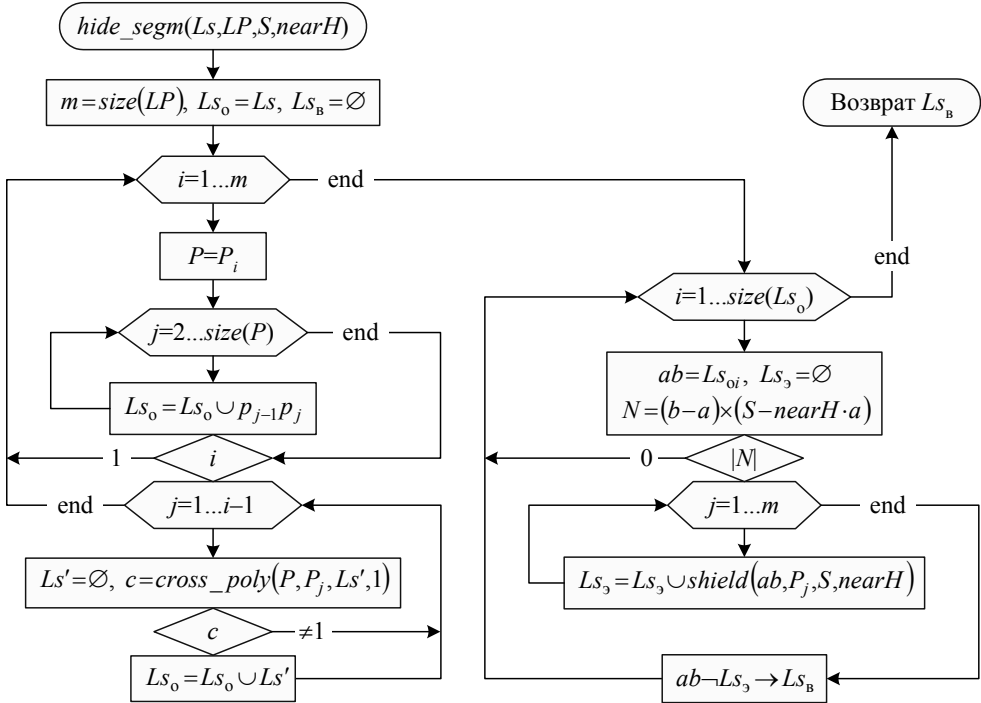


Рис. 1.193

1.7.4. Удаление невидимых граней

II простейший способ изображения *одинокого* и *выпуклого* сплошного полиэдра представляет собой метод Робертса (см. п. 1.7.2) и заключается в произвольном порядке вывода экранных проекций лицевых граней. Благодаря выпуклости объекта его грани целиком видимы либо невидимы. Все невидимые грани

являются нелицевыми и отбраковываются тестом (1.181). Смежные грани пересекаются только по общим рёбрам, а несмежные не пересекаются. Таким образом, у выпуклых полиэдров исключена частичная видимость граней и рёбер.

Рассмотрим общую задачу определения видимости плоских полигональных, **произвольно** расположенных граней. Она решается при изображении как отдельных невыпуклых тел, возможно, имеющих отверстия и самопересечения, так и совокупности пересекающихся тел. В отличие от выпуклой задачи здесь возможна **частичная** видимость граней вследствие их взаимного экранирования, в том числе и за счёт пересечения друг с другом (см. рис. 1.112, 1.132).

Сначала изучим эффект **локального** экранирования плоской грани Q другой плоской гранью P с учётом их ориентации относительно друг друга и наблюдателя, но без учёта остальных граней сцены. Экранирование аналогично оптическому эффекту тенеобразования и заключается в попадании части или всей поверхности Q в **теньевую зону** T , находящуюся за гранью P . В итоге наблюдатель видит лишь часть грани Q , находящуюся вне теневой зоны.

В случае дальнего наблюдателя область T представляет собой незамкнутую призму с основанием P (рис. 1.194,а), а при ближнем наблюдателе это будет не замкнутая задней гранью усечённая пирамида с передней гранью P (рис. 1.194,б). Задача экранирования эквивалентна **внутреннему отсечению** полигона Q областью T .

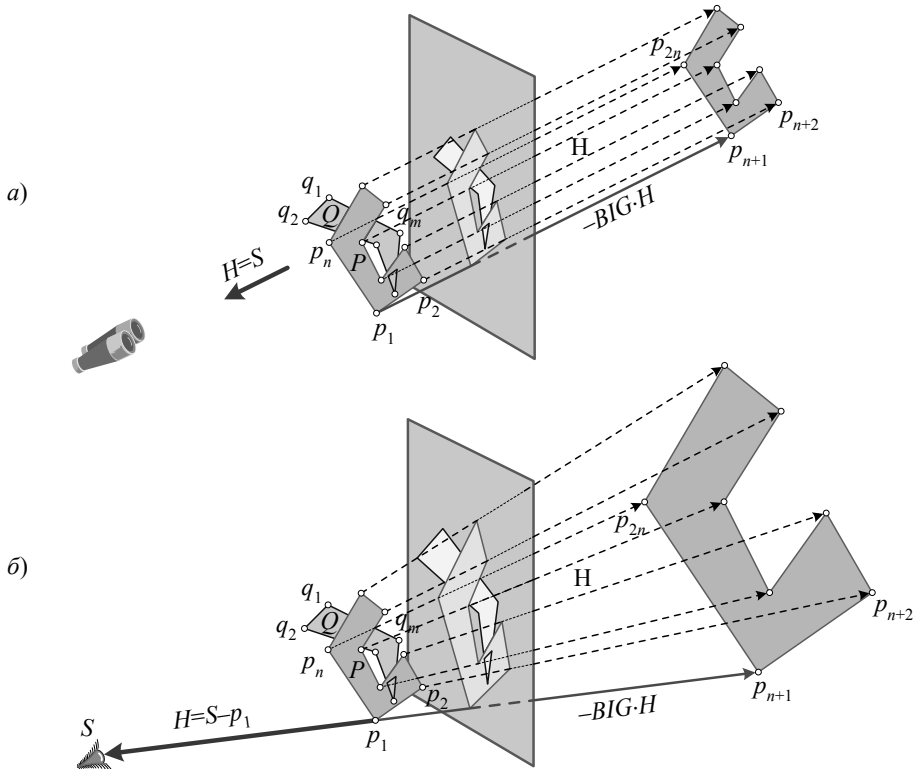


Рис. 1.194

Единственное, что мешает использовать для такого отсекаания алгоритм *inclip3_poly*, разработанный в п. 1.6.2, — это бесконечная протяжённость теневой зоны в глубину сцены. Аналогично ранее применённому методу представления неограниченных полупространств зададим достаточно большое число *BIG*, заведомо превышающее максимальную глубину сцены. Тогда полубесконечная теневая зона полигона $P=p_1p_2\dots p_np_1$ заменяется конечным полиэдром $H=\{O,G\}$ с параллельными передней и задней гранями. Координаты его $2n$ вершин, составляющих список $O=\{o_1,\dots,o_{2n}\}$, в зависимости от типа наблюдателя, задаваемого параметром $nearH\in\{0,1\}$, вычисляются следующим образом:

$$o_i = \begin{cases} p_i & \text{при } i \leq n; \\ p_{i-n} - BIG \cdot (S - nearH \cdot p_{i-n}) & \text{при } i > n. \end{cases} \quad (1.188a)$$

Грани полиэдра H опишем группой $G=\{G_1, G_2, G_3, \dots, G_{n+2}\}$ списков

$$\begin{cases} G_1 = \{1, \dots, n, 1\}, G_2 = \{2n, 2n-1, \dots, n+1, 2n\}, \\ G_{2+i} = \{i, n+i, n+1+\text{mod}(i, n), 1+\text{mod}(i, n), i\} \forall i = \overline{1, n}. \end{cases} \quad (1.188b)$$

Введём вспомогательную **функцию** *shadow*($P, S, nearH$) **формирования параметров полиэдра теневой зоны** грани $P=p_1p_2\dots p_np_1$ с наблюдателем, задаваемым параметрами S и $nearH\in\{0,1\}$. Функция возвращает рассчитанные по (1.188) списки вершин O и граней G .

Теперь, имея конечный полиэдр $H=\{O,G\}$, можно использовать его в качестве внутреннего отсекаателя грани Q , экранированной гранью P . После вычисления параметров теневого полиэдра $\{O,G\}=\text{shadow}(P,S,nearH)$ и отсекаания им полигона Q по алгоритму *inclip3_poly*(Q,O,G,U,F) получим список полигонов — видимых фрагментов исходных граней.

На рис. 1.195,а приведен **алгоритм** *hide_poly*($LP,S,nearH$) **удаления невидимых фрагментов полигональных граней** списка LP с точки зрения наблюдателя, задаваемого параметрами S и $nearH\in\{0,1\}$. Алгоритм возвращает список полигонов — видимых фрагментов исходных граней.

Алгоритм рассчитывает попарное экранирование m граней списка $LP=\{P_1,\dots,P_m\}$ путём внутреннего отсекаания граней $Q=P_i \forall i=\overline{1,m}$ теньевыми полиэдрами граней $P=P_j \forall j=\overline{1,m}, j \neq i$. В зависимости от размера и содержимого вспомогательного списка LP' , возвращённого алгоритмом *inclip3_poly*, возможны следующие варианты действий:

- если список LP' пустой, то грань Q полностью экранирована гранью P (рис. 1.195,б) и удаляется из списка LP ;
- если LP' содержит один элемент, то это либо полигон исходной полностью видимой грани Q (рис. 1.195,в), либо один видимый фрагмент частично экранированной грани Q (рис. 1.195,г). В первом случае ($LP'=Q$) алгоритм переходит к проверке экранирования полигона Q следующей гранью P , а во втором элементы списка LP' добавляются в список LP , а Q из него исключается;

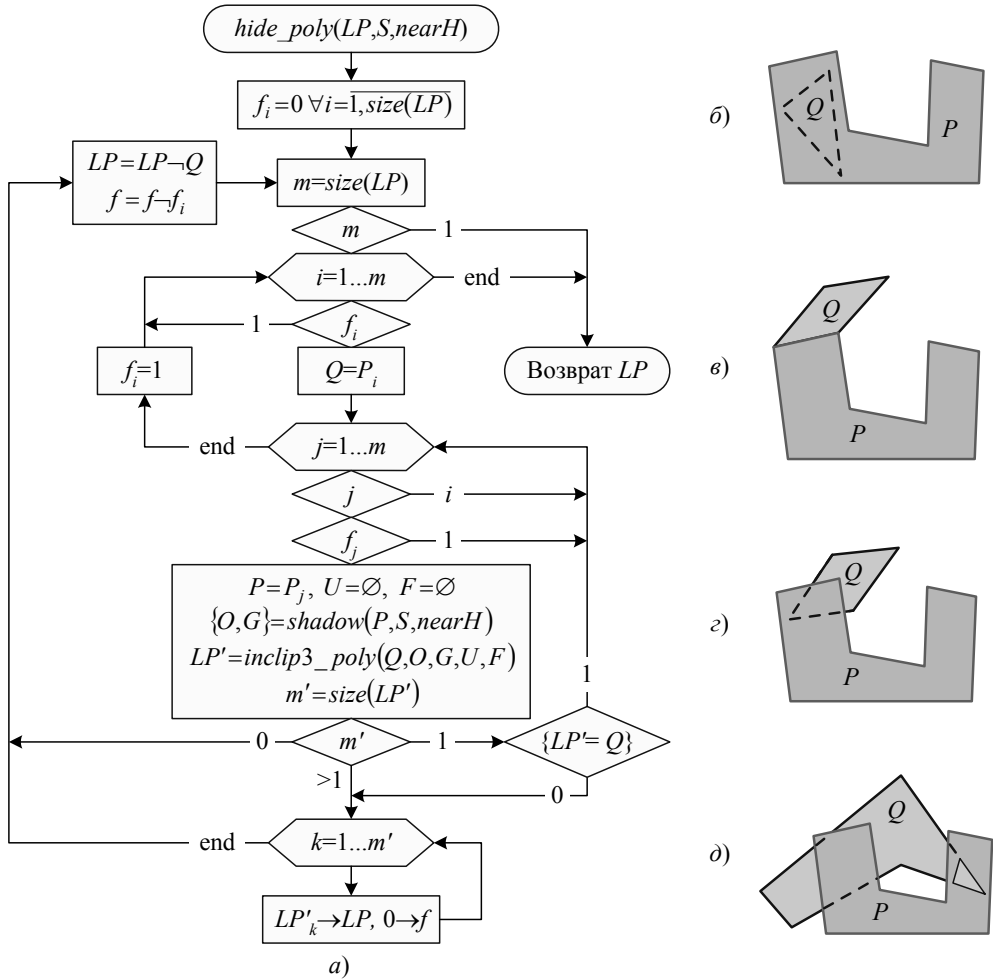


Рис. 1.195

- если в LP' более одного элемента (рис. 1.195, д), то все видимые фрагменты частично экранированной грани Q добавляются в список LP , а сама грань Q из него исключается.

При любом изменении размера или содержимого списка граней LP , выполненном в результате отсечения, алгоритм переходит на начало обработки обновлённого списка. Если перед очередным циклом обработки LP содержит единственную грань, то алгоритм заканчивает работу с возвратом этой грани.

По окончании обработки список LP содержит не экранирующие друг друга полигональные фрагменты видимых наблюдателем граней, которые алгоритм возвращает как результат своей работы. Далее эти полигоны можно проецировать на экран или использовать в последующих расчётах.

Рассмотренный алгоритм *hide_poly* содержит важный элемент оптимизации, позволяющий сократить число переборов пар $\{Q, P\}$ с помощью **флажков полной видимости граней**. Флажок i -й грани f_i является её неотделимым атри-

бутом, таким же, как число и координаты вершин, цвет, нормаль и другие параметры. Он участвует во всех списковых операциях вместе со своей гранью. Перед началом обработки граней все m флажков сбрасываются в нуль.

Грань $Q=P_i$, не экранированная ни одной гранью $P=P_j$, $j \neq i$, является полностью видимой наблюдателем, поэтому после перебора всех граней P флажок f_i устанавливается в единицу. Больше эта грань не будет участвовать в проверках экранирования и отсечения ни как объект Q , ни как экран P . Новые полигоны, добавляемые в LP , снабжаются своими флажками полной видимости, устанавливаемыми в нуль. Исключение грани Q из списка LP сопровождается исключением её флажка из вектора f . В конце обработки списка LP флажки всех его полигонов становятся равными единице.

В заключение хочется ещё раз подчеркнуть, что автор не ставил своей целью энциклопедический охват всех существующих алгоритмов удаления, с которыми можно познакомиться в большом количестве книг по компьютерной графике, а попытался проторить свою, пусть и неширокую, тропинку в беспредельном пространстве решений проблем визуализации сложных геометрических объектов.

На сегодняшний день наиболее эффективное решение графических задач обеспечивает конвейерная архитектура, реализованная как программными, так и аппаратными средствами [3, 65]. В будущем, совершенно очевидно, графические СБИС станут работать быстрее, степень их интеграции многократно возрастет, а стоимость уменьшится. Это является предпосылкой тотального наступления методов распараллеливания графических вычислений, обеспечивающих более высокое качество изображения, учёт большего числа источников света и глубину проработки разнообразных световых эффектов. Следовательно, пользователь такой графической системы получит возможность включать в отображаемую сцену больше объектов с самыми разнообразными оптическими свойствами, которые будут моделироваться так же, как они проявляются в природе, — быстро и параллельно.

Линии, применяемые в графических приложениях как самостоятельные геометрические объекты либо как вспомогательные конструктивные элементы поверхностей, могут быть описаны как целиком одной неявной либо параметрической моделью, так и линией, составленной из нескольких **сегментов**, соединённых друг с другом в концевых точках. Простейшими составными линиями являются незамкнутые полилинии $p_1 p_2 \dots p_n$ и замкнутые полигоны $p_1 p_2 \dots p_n p_1$ с сегментами в виде отрезков прямых с нулевой кривизной k , определённой по (1.10а) или (1.110а). С помощью таких отрезков изображается любая кривая линия $p(t)$, кусочно-линейная интерполяция которой строится на конечном множестве **узловых точек** $u_i = p(t_i)$, $i=1, \overline{n}$ (рис. 2.1,а).

Во временном аспекте вывод линии производится двумя методами:

- в **реальном времени** после вычисления точки u_i рисуется отрезок $u_{i-1} u_i$;
- **ретроспективно** полилиния выводится целиком после расчёта всех её узловых точек и формирования из них линейного списка точек $Lp = u_1 u_2 \dots u_n$.

Визуальная иллюзия гладкости кривой линии достигается оптимальным распределением узлов. С одной стороны, они не должны располагаться слишком близко друг к другу во избежание наложения на экране, а с другой плотность узловых точек не должна быть слишком малой, чтобы кривая не выглядела грубой ломаной линией. Некоторые аспекты достижения компромисса в этом важном вопросе обсуждаются как в этой, так и в последующих главах книги.

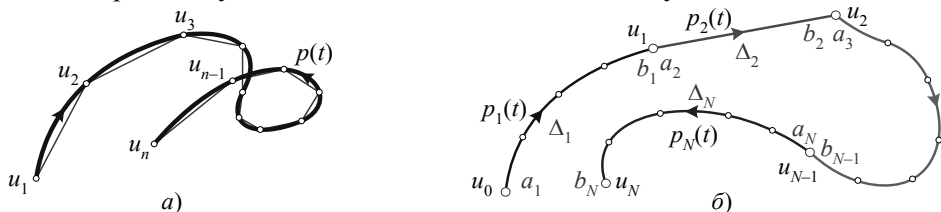


Рис. 2.1

В общем случае каждый из N сегментов составной линии может описываться индивидуальной векторной функцией $p_i(t)$ в интервале изменения параметра $t \in \Delta_i = [a_i, b_i]$, $i \in [1, N]$ (рис. 2.1,б). На практике применяются следующие **методы локализации параметра составной линии**:

- **локальный**, в котором параметр $t \in [a_i, b_i]$, называемый в [13, 20] **местным**, в каждом сегменте начинается с исходного значения $a_i = 0$;
- **сквозной**, когда параметр пробегает поочерёдно все интервалы $\Delta_i = [t_{i-1}, t_i]$ с начального значения t_0 до конечного t_N с непрерывным сопряжением $b_{i-1} = a_i$.

Важным аспектом конструирования составной линии является выбор **параметрических длин** её интервалов $d_i = b_i - a_i$. По **методу нормализованной**

параметризации все длины задаются равными $d_i=1$, а получаемая при этом составная линия называется **нормализованной** или **равномерной** [13].

В кинематической интерпретации длина вектора $V_i(t)$ скорости движения точки по сегменту $p_i(t)$ приблизительно обратно пропорциональна времени движения d_i . Стремясь к равномерному движению и к плавным, без рывков переходам точки с одного сегмента на другой, **следует задавать значения d_i пропорционально длинам дуг $\overrightarrow{u_{i-1}u_i}$** . Однако, пока линия не построена, длины её дуг неизвестны, поэтому приходится довольствоваться априорно известными данными о плотности расположения узловых точек — длинами **хорд $u_{i-1}u_i$** . Отсюда происходит **метод хордовой параметризации** [14, 15, 51] с выбором параметрических длин интервалов, равных расстояниям между узловыми точками

$$d_i = |u_i - u_{i-1}|. \quad (2.1a)$$

Самый же точный **метод дуговой параметризации** заключается в задании по (2.1a) начального значения $d_i^{(0)}$, построении начального приближения сегмента $p_i^{(0)}(t)$ и итеративном вычислении $k+1$ -го приближения его длины

$$d_i^{(k+1)} = \sum_{j=1}^n |u_{ij}^{(k)} - u_{i,j-1}^{(k)}|, \quad k=0,1,2,\dots \quad (2.1б)$$

как суммы длин отрезков между точками $u_{ij}^{(k)} = p_i^{(k)}(a_i + jd_i^{(k)}/n)$ его k -го приближения при достаточно большой частоте n равномерного квантования параметрического интервала $\Delta_i = [a_i, b_i]$. Вполне возможно, что несколько итераций быстро приведут к установившейся форме сегмента $p_i(t)$ и длине его дуги d_i .

Полагая теперь известными длины всех интервалов $d_1 \div d_N$ и обозначив во избежание путаницы разными символами локальный параметр t и сквозной параметр θ , рассмотрим **правила изменения типа локализации**:

- в **локальном** методе положение точки сегмента $p_i(t)$ задаётся номером сегмента $i \geq 1$ и значением параметра $t \in [0, d_i]$. Сквозной параметр точки $p(\theta)$ вычисляется по формуле

$$\theta = t + \{i > 1\} \sum_{k=1}^{i-1} d_k; \quad (2.2a)$$

- в **сквозном** методе положение точки на составной линии $p(\theta)$ задаётся значением $\theta \in [t_0, t_N]$. Локализация сегмента заключается в нахождении его номера $i \geq 1$, после чего вычисляется местный параметр t локальной модели $p_i(t)$:

$$i = \min\{j\}: \theta \leq \sum_{k=1}^j d_k \Rightarrow t = \theta - \{i > 1\} \sum_{k=1}^{i-1} d_k. \quad (2.2б)$$

В точках стыковки сегментов $p_i(t)$ и $p_{i+1}(t)$ могут складываться разнообразные **свойства сопряжения**:

• при $p_i(b_i) \neq p_{i+1}(a_{i+1})$ составная линия терпит **разрыв** (рис. 2.2,а), создающий трудности её изображения: промежуток между несовпадающими концами сегментов не должен ничем заполняться в отличие от неразрывного сопряжения, рисуемого «без отрыва пера от бумаги». При ретроспективном выводе разрывной полилинии в список её узлов Lp между точками разрыва вставляется **фиктивная точка** или особый символ, обнаружив который, функция рисования полилинии пропускала бы вывод примыкающих к ней отрезков;

• в точке соединения $p_i(b_i) = p_{i+1}(a_{i+1})$ составная линия **непрерывна**, а при дополнительном условии $p'_i(b_i) = p'_{i+1}(a_{i+1})$ она обладает свойством **гладкости** (рис. 2.2,б) — непрерывности векторной функции скорости движения

$$V_i(b_i) = V_{i+1}(a_{i+1}). \quad (2.3a)$$

Следует иметь в виду, что отсутствие гладкости не равносильно излому составной линии в точке сопряжения u_i : векторы $V_i(b_i)$ и $V_{i+1}(a_{i+1})$ могут быть параллельны, но иметь разные длины (рис. 2.2,в). Назовём более слабое условие

$$\overline{V}_i(b_i) = \overline{V}_{i+1}(a_{i+1}) \quad (2.3б)$$

нормированной (геометрической [65]) **гладкостью** составной линии, при которой излом в точке сопряжения отсутствует, но остаётся возможным скачкообразное изменение модуля скорости движения. Геометрической гладкостью обладают все сопряжения окружностей с прямыми и другими окружностями;

• условия непрерывности $k+1$ -го и гладкости k -го порядков сопряжения

$$p_i^{(m)}(b_i) = p_{i+1}^{(m)}(a_{i+1}) \quad \forall m = \overline{0, k} \quad (2.4a)$$

обобщают рассмотренные выше условия непрерывности и гладкости. Аналогично (2.3б) записываются условия **нормированной гладкости k -го порядка**:

$$p_i(b_i) = p_{i+1}(a_{i+1}), \quad \overline{p_i^{(m)}(b_i)} = \overline{p_{i+1}^{(m)}(a_{i+1})} \quad \forall m = \overline{1, k}. \quad (2.4б)$$

В случае $k=2$ сегменты непрерывно сопрягаются друг с другом в точке $u_i = p_i(b_i) = p_{i+1}(a_{i+1})$ с **ненулевыми** векторами скорости $V_i = p'_i(b_i) = p'_{i+1}(a_{i+1})$ и ускорения $A_i = p''_i(b_i) = p''_{i+1}(a_{i+1})$ (рис. 2.2,з). При $k>1$ невозможно сопряжение типа **точки перегиба** с гладкостью лишь первого порядка (рис. 2.2,б).

Завершив общее знакомство с составными линиями рассмотрением вопроса их визуализации полилиниями, аппроксимирующими каждый сегмент $p_i(t)$ между узловыми точками u_{i-1} и u_i . Прямолинейный сегмент с нулевой кривизной строится всего по двум узловым точкам, тогда как для изображения кривой

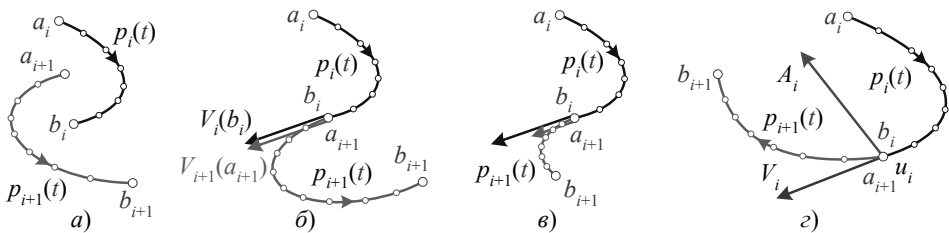


Рис. 2.2

линии требуются дополнительные точки — **подузлы**, вычисленные по модели $p_i(t)$ для выборочных значений параметра $t \in (a_i, b_i)$. В идеале плотность размещения вершин полилинии на дуге должна быть пропорциональна модулю её кривизны $|\kappa(t)|$ — примерно так, как показано на рис. 2.1,а.

Анализируя формулы вычисления кривизны (1.10а) и (1.110а), замечаем, что она обратно пропорциональна скорости движения $|V(t)|$: на сильно искривлённых участках кривой точка движется медленнее, а на слабо искривлённых — быстрее. Отсюда делаем *интуитивный* вывод о том, что **наилучшим является равномерное распределение отсчётов в параметрическом интервале сегмента** $[a_i, b_i]$. Задав примерно величину **параметрического кванта** h , найдём **целое** число квантов n_i , точное значение кванта Δt_i и отсчёты t_{ij} , после чего вычислим вершины u_{ij} полилинии, аппроксимирующей дугу $u_{i-1}u_i$:

$$n_i = \lceil d_i/h \rceil, \quad \Delta t_i = d_i/n_i, \quad t_{ij} = a_i + j\Delta t_i, \quad u_{ij} = p_i(t_{ij}) \quad \forall j = \overline{0, n_i}. \quad (2.5)$$

В зависимости от выбранного метода параметризации сегментов распределение подузлов u_{ij} будет следующим:

- в нормализованной составной линии значения $d_i=1$, $n_i = \lceil 1/h \rceil$ и $\Delta t_i = 1/n_i$ у всех сегментов одинаковы, как и распределение подузлов на дугах равной длины. При большом разбросе длин дуг сегментов точки u_{ij} будут размещены крайне неравномерно: на коротких дугах часто, а на длинных — редко;
- для равномерного распределения подузлов по всей составной линии независимо от межузловых расстояний нужно задавать параметрические длины методами хордовой параметризации (2.1). Тогда по (2.5) у всех сегментов значения квантов $\Delta t_i \approx h$ будут сопоставимы при любых межузловых расстояниях d_i .

2.1. ПОЛИНОМИАЛЬНЫЕ ЛИНИИ

В вычислительной математике разработано множество методов **приближения** — восполнения данных в интервалах между **узловыми** (выборочными) состояниями объектов [13 – 15, 20, 21, 26, 51, 56, 60, 63]. Они широко используются в компьютерной графике для конструирования неполностью определённых линий и поверхностей, создания анимаций, изменяющих форму объектов с течением времени, сглаживания окраски полигональных поверхностей, восстановления сжатых видеоданных по опорным кадрам и т. п. Суть **интерполяции** состоит в получении аналитической зависимости, заставляющей графический объект пройти **точно** через все заданные узловые состояния. Назначение **аппроксимации** иное: она призвана сгладить недостатки интерполяционного решения ценой допустимого увеличения узловых отклонений. Сочетание обоих свойств решения будем называть **приближением**.

Сложность процесса приближения и свойства созданного в нём объекта определяются используемой системой функций разложения, удачный выбор которых сочетает хорошее приближение к узлам и малые межузловые отклонения с

небольшим числом варьируемых параметров и простотой вычислений. Выбор степеней параметра t элементами **вектора базисных функций (ВБФ)**

$$T_m(t) = [1 \ t \ t^2 \ \dots \ t^m]^T \in \mathbb{R}^{(m+1) \times 1} \quad (2.6a)$$

означает приближение **полиномом** степени m . В дальнейшем нам потребуются две первые производные ВБФ:

$$T'_m(t) = [0 \ 1 \ 2t \ 3t^2 \ \dots \ mt^{m-1}]^T \in \mathbb{R}^{(m+1) \times 1}, \quad (2.6б)$$

$$T''_m(t) = [0 \ 0 \ 2 \ 6t \ \dots \ m(m-1)t^{m-2}]^T \in \mathbb{R}^{(m+1) \times 1}. \quad (2.6в)$$

Наиболее востребованы следующие **формы разложений полинома**:

- **по узловым точкам** $u_0 \div u_n$, составляющим вектор $U = [u_0 \ u_1 \ \dots \ u_n] \in V^{n+1}$:

$$p(t) = l_{m0}(t)u_0 + \dots + l_{mn}(t)u_n = \sum_{i=0}^n l_{mi}(t)u_i = UL_n(t) \quad \forall t \in [t_0, t_n], \quad (2.7a)$$

где $L_n(t) = [l_{m0}(t) \ l_{m1}(t) \ \dots \ l_{mn}(t)]^T$ — столбцовый $n+1$ -вектор **весовых функций (ВФ)** $l_{mi}(t)$, имеющих вид полиномов степени m и характеризующих вклад узлов u_i в суммарное значение $p(t)$. Абсолютные значения весов $|l_{mi}(t)|$ отражают качество полиномиального приближения как в узловых точках, так и между ними. Из (2.7a) следует, что для любого значения параметра t точка $p(t)$ принадлежит пространству, натянутому на узловые точки $u_0 \div u_n$. В частности, если все узлы лежат в одной точке $u_0 = u_1 = \dots = u_n$, то очевидное условие $p(t) = u_0 \quad \forall t$ означает, что **сумма всех весовых функций приближающего полинома равна единице** для любого допустимого значения параметра t :

$$\sum_{i=0}^n l_{mi}(t) = 1 \quad \forall t \in [t_0, t_n];$$

- **по базисным функциям** параметра в форме **Фергюсона** [27, 56]:

$$p(t) = s_0 + s_1 t + \dots + s_m t^m = \sum_{i=0}^m s_i t^i = ST_m(t) \quad \forall t \in [t_0, t_n], \quad (2.7б)$$

где $S = [s_0 \ s_1 \ \dots \ s_m] \in V^{m+1}$ — **вектор полиномиальных коэффициентов (ВПК)**, зависящих от расположения узловых точек. В случае неподвижности последних вектор S вычисляется один раз, что делает форму (2.7б) удобной в применении и компактной в хранении параметров полинома в памяти ЭВМ. Однако, после изменения любой узловой точки весь вектор S пересчитывается заново;

- **бинарное разложение** по узловым точкам и базисным функциям:

$$p(t) = \sum_{i=0}^n \sum_{j=0}^m h_{ij} u_i t^j = UH_{nm} T_m(t) \quad \forall t \in [t_0, t_n], \quad (2.7в)$$

где H_{nm} — числовая $(n+1) \times (m+1)$ -**матрица весовых коэффициентов (МВК)**, связанная с векторами $L_m(t)$ и S соотношениями

$$L_n(t) = H_{nm} T_m(t), \quad S = UH_{nm}. \quad (2.8)$$

В наиболее простом и распространённом методе локальной нормализованной параметризации матрица H_{nm} фиксирована и зависит лишь от числа, но не от расположения узловых точек. Благодаря этому полиномиальная модель (2.7в) выгодна в системах **интерактивного** проектирования приближающих линий и построенных на их основе графических объектов, поскольку она не требует пересчёта ВПК S после каждого перемещения узловых точек. Тем самым резко сокращается время перерисовки конфигурации объектов, что делает процесс их проектирования «живым», проходящим в реальном времени.

Нельзя не отметить то, что в разложениях (2.7) объекты U и S являются не обычными векторами со скалярными элементами, а **структурными** векторами, элементы которых — векторы с размерностью узловых точек $u_0 \div u_n$. Существуют ещё и структурные матрицы. Операции со структурными объектами выполняются по обычным правилам матричной алгебры, в результате которых также получают структурные объекты.

Для работы со структурными объектами следует продумать организацию их представления. Например, если принять не строчный, а **столбцовый формат узловых точек u_i и коэффициентов полинома s_i** , то объекты U и S могут быть организованы в виде обычных матриц размерами $2 \times (n+1)$ либо $3 \times (n+1)$ в зависимости от евклидовой размерности пространства узлов.

2.1.1. Интерполяция

Главное свойство **интерполяционного полинома (ИП)**, заставляющее параметрическую линию $p(t)$ пройти через все узловые точки $u_0 \div u_n$, имеет вид

$$p(t_i) = u_i \quad \forall i = \overline{0, n}. \quad (2.9)$$

Эта система уравнений лежит в основе получения весовых функций и коэффициентов во всех разложениях (2.7).

Простейшей и широко применяемой в разных задачах является **линейная интерполяция $\text{lin}(A, B, t)$** состояния некоторого графического объекта S между двумя его граничными состояниями A и B в **локальной** шкале **нормализованного** параметра t :

$$\boxed{C(t) = A + (B - A)t = A \cdot (1 - t) + B \cdot t} \quad \forall t \in [0, 1]. \quad (2.10a)$$

Данный метод образования различных линейных графических объектов уже был использован нами в (1.176), (1.34) и (1.138). Подставив в (2.10a) вместо t выражение $(t-a)/(b-a)$, получим формулу **ненормализованной** линейной интерполяции в **произвольной** шкале параметра t :

$$C(t) = A + (B - A) \frac{t-a}{b-a} = A \frac{b-t}{b-a} + B \frac{t-a}{b-a} \in [A, B] \quad \forall t \in [a, b]. \quad (2.10б)$$

В кинематическом аспекте обе модели (2.10) описывают прямолинейное движение точки $p(t) = s_0 + s_1 t$ из начального положения $A = s_0 + s_1 a$ в конечное положение $B = s_0 + s_1 b$ с **постоянной** скоростью $V = s_1 = (B - A)/(b - a)$ (рис. 2.3, а).

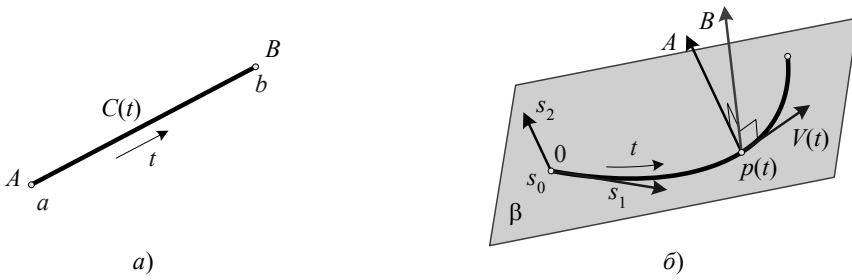


Рис. 2.3

Вторая степень параметрической функции $p(t)=s_0+s_1t+s_2t^2$ создаёт движение по **параболе** с линейно изменяющейся скоростью $V(t)=s_1+2s_2t$ и **постоянным** ускорением $A=2s_2$. Вся эта линия целиком лежит в **одной** соприкасающейся плоскости $\beta=\{s_0, B\}$ с **постоянной** бинормалью $B=V(t)\times A=2s_1\times s_2$ (рис. 2.3,б), т. к. согласно (1.110а) её кручение $\chi=0$ (см. рис. 1.105). В частном случае параллельности векторов $V(t)$ и A при $s_1\parallel s_2$ парабола вырождается в прямую линию равноускоренного движения с нулевой кривизной $\kappa=0$ и неопределённой соприкасающейся плоскостью. Свойство планарности параболической кривой означает **невозможность построения дважды гладкой составной линии из не компланарных параболических сегментов**.

И лишь начиная со степени $m=3$ полиномиальная линия (2.7б) способна иметь переменный вектор ускорения $A(t)$, ненулевое кручение $\chi\neq 0$ и быть не-компланарной, благодаря чему **становится возможным дважды гладкое сопряжение кубических сегментов составной линии** по скорости и ускорению. Вот почему такие линии (**сплайны**) получили широчайшее распространение.

Приступая к изучению методов синтеза интерполяционных полиномов, поставим задачу проведения **односегментной** линии $p(t)$ с **монотонной** параметризацией $t_0 < t_1 < \dots < t_n$ через узловые точки сетки $\{t_i, u_i\}$ в порядке их нумерации $i=\overline{0, n}$ (рис. 2.4). Производные линии в крайних узлах u_0 и u_n для начала ничем ограничивать не будем.

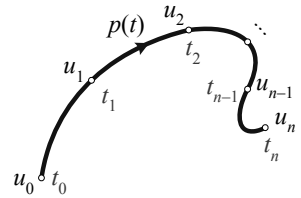


Рис. 2.4

К классическим **методам полиномиального приближения** относятся методы Лагранжа и наименьших квадратов. Напомним кратко их суть.

Метод Лагранжа

Интерполяционный полином Лагранжа определяется формулой прямого действия (2.7а) с полиномиальными весовыми функциями n -й степени:

$$l_{ni}(t) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{t-t_j}{t_i-t_j} = \frac{t-t_0}{t_i-t_0} \cdot \dots \cdot \frac{t-t_{i-1}}{t_i-t_{i-1}} \cdot \frac{t-t_{i+1}}{t_i-t_{i+1}} \cdot \dots \cdot \frac{t-t_n}{t_i-t_n}. \quad (2.11a)$$

Благодаря их **узловым свойствам**

$$l_{ni}(t_j) = \delta_i^j \quad \forall i, j \in [0, n] \quad (2.116)$$

вес точки u_i в значении $p(t_i)$ равен единице, а в других узловых значениях $p(t_j) \forall j \neq i$ — нулю. Это обеспечивает точное, как показано на рис. 2.4, прохождение линии $p(t)$ через все узловые точки $u_0 \div u_n$ согласно условиям (2.9).

В отсутствие ограничений на производные интерполяционного полинома в узловых точках его степень m равна числу межузловых интервалов n , а явная зависимость весовых функций от отсчётов параметра t_i означает критичность формы получаемого интерполанта к используемому методу параметризации — нормализованному с равными длинами всех интервалов $d_i=1$ либо хордовому с межузловыми расстояниями $d_i = |u_i - u_{i-1}|$. Только **при всех равных значениях $d_i=d$ оба метода дают идентичные линии**. Действительно, при $t_i=id$ и параметре $t=\tau d$, выраженном в относительных единицах длины d , из (2.116) следует независимость от неё всех функций $l_{ni}(t)$:

$$l_{ni}(\tau d) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{\tau d - jd}{id - jd} = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{\tau - j}{i - j} = l_{ni}(\tau).$$

Недостаток метода Лагранжа — повышенная сложность программирования и высокая, пропорциональная n^2 , трудоёмкость получения точки $p(t)$.

Метод наименьших квадратов

Интерполяционный МНК-полином имеет вид разложения (2.76) по степенным базисным функциям $\{t^0, t^1, \dots, t^n\}$ максимальной степени $m=n$. Он обеспечивает точное выполнение условий (2.9) и абсолютный (нулевой) минимум квадратичной (отсюда и название полинома) суммы

$$\mathfrak{Z} = \sum_{i=0}^n |p(t_i) - u_i|^2. \quad (2.12)$$

Зафиксировав точки $p(t) = ST_n(t)$ в узловых отсчётах параметра t_i , получим полностью определённую **базовую СЛАУ**

$$ST_n(t_i) = u_i \quad \forall i = \overline{0, n} \quad (2.13a)$$

с неизвестным структурным ВПК S , элементы которого $s_0 \div s_n$ имеют размерность узловых точек $u_0 \div u_n$. Система (2.13a) записывается в векторно-матричной форме (1.6a) как

$$S[T_n(t_0) \ T_n(t_1) \ \dots \ T_n(t_n)] = [u_0 \ u_1 \ \dots \ u_n] \quad \text{или} \quad SQ_n = U. \quad (2.13б)$$

Определив в общем случае прямоугольную **матрицу Вандермонда**

$$Q_{mn} = [T_m(t_0) \ T_m(t_1) \ \dots \ T_m(t_n)] = \begin{bmatrix} 1 & 1 & \dots & 1 \\ t_0 & t_1 & \dots & t_n \\ \vdots & \vdots & \ddots & \vdots \\ t_0^m & t_1^m & \dots & t_n^m \end{bmatrix} \in R^{(m+1) \times (n+1)} \quad (2.13в)$$

с элементами $q_{ij} = t_j^i \quad \forall i = \overline{0, m}, j = \overline{0, n}$, найдём решение (1.66) уравнения (2.13б):

$$S = UQ_n^{-1}, \text{ где } Q_n \equiv Q_{nm}. \quad (2.13г)$$

Сравнивая его с (2.8), получаем параметры двух других форм записи ИП:

$$H_n \equiv H_{nm} = Q_n^{-1}, \quad L_n(t) = H_n T_n(t). \quad (2.13д)$$

Условие существования решения — невырожденность квадратной матрицы Q_n , которая ввиду равенства **определителя Вандермонда**

$$|Q_n| = \prod_{i=0}^{n-1} \prod_{j=i+1}^n (t_j - t_i) \quad (2.14)$$

произведению разностей отсчётов параметра эквивалентна **попарному несовпадению отсчётов** $t_i \neq t_j \quad \forall i \neq j$, что автоматически удовлетворяется при монотонной параметризации $t_0 < t_1 < \dots < t_n$. Существенным фактором, осложняющим вычисление и использование обратной матрицы Q_n^{-1} , является сильное влияние погрешностей округления даже при невысоких степенях полинома $n \approx 7 \div 9$ или использовании хордовой параметризации с большим разбросом значений d_i .

Несмотря на разные нотации, **все три метода интерполяции (2.7) дают одно и то же решение** степени n , единственно возможное на сетке узлов $\{t_i, u_i\} \quad \forall i = \overline{0, n}$. С добавлением в сетку новых узлов растёт и степень полинома $m = n$, поскольку для определённости матричного уравнения (2.13б) должно быть столько же полиномиальных коэффициентов $s_0 \div s_m$, сколько и узлов $u_0 \div u_n$. Каждый новый дополнительный узел всё сильнее вскрывает **основной недостаток полиномиальной интерполяции** — возрастание **межузловых осцилляций** колебательного характера (рис. 2.5), которые можно трактовать как отклонения точек полинома $p(t)$ в интервалах (t_{i-1}, t_i) от кратчайших хорд $u_{i-1}u_i$.

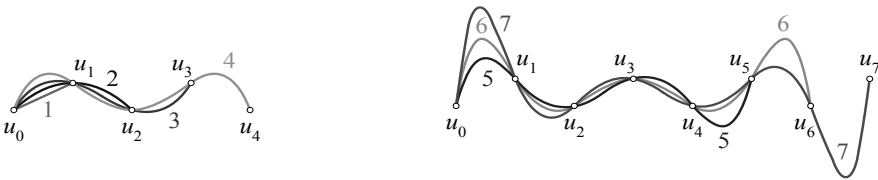


Рис. 2.5

Причину возникновения данного эффекта устанавливает анализ экстремумов весовых функций Лагранжа, определённых в (2.11). На рис. 2.6,а построено по одному для каждой степени $n \in \{1, 2, 3, 5, 7, 8, 9\}$ графику $l_{ni}(t) \quad \forall t \in [0, 1]$ с узлами $t_j = j/n$. Вид этих графиков и зависимости абсолютных максимумов $\delta(n) = \max_{t,j} |l_{ni}(t)|$, построенной на рис. 2.6,б, позволяют сделать следующие важные практические **выводы о приближающих свойствах интерполяционных полиномов высоких степеней**:

- максимальный размах межузловых осцилляций с увеличением степени полинома растёт более чем экспоненциально;

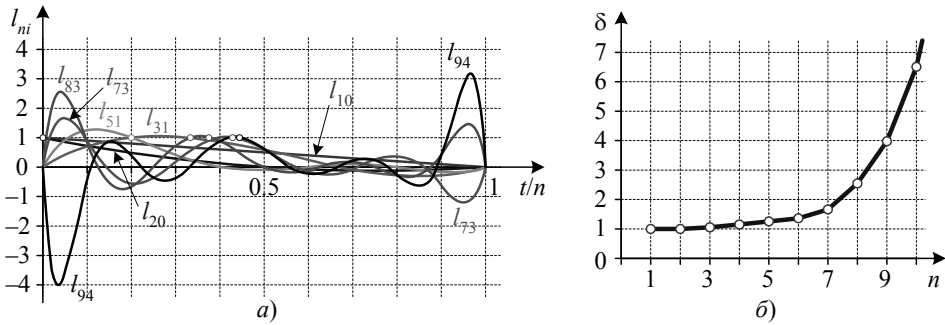


Рис. 2.6

• наиболее сильные межузловые отклонения происходят на крайних (первом и последнем) интервалах сетки;

• влияние узловых точек на межузловые отклонения не имеет локального характера и распространяется на удалённые интервалы. С конструкторской точки зрения это главный **недостаток** интерполяционного метода, приводящий к деформации линии на участках, далёких от перемещаемой узловой точки.

В совокупности перечисленные здесь свойства ограничивают на практике максимальные степени интерполяционных полиномов значениями $5 \div 7$.

Следует опровергнуть ошибочное мнение [51], что уменьшению межузловых осцилляций и повышению качества конструируемой линии способствует использование хордовой параметризации $d_i = |u_i - u_{i-1}|$ вместо нормализованной с равными параметрическими длинами $d_i = 1$. Моделирование на разнообразных сетках с большим разбросом значений d_i показывает (рис. 2.7, а), что нормализованная интерполяция (l) чаще даёт более привлекательные линии, чем хордовая (2). К тому же при высоких степенях n и длинах $d_i \gg 1$ определитель (2.14) достигает чрезвычайно больших значений и снижает точность вычислений. Однако нередки и обратные результаты моделирования, когда учёт межузловых расстояний даёт более гладкое решение (рис. 2.7, б).

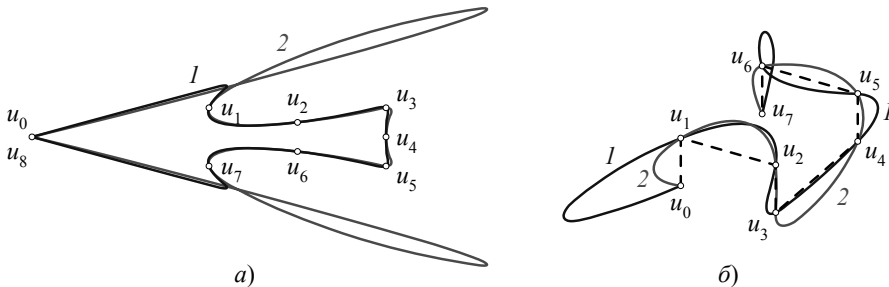


Рис. 2.7

В литературе по численным методам практически не затронута проблема **избыточной интерполяции**. Единственность решения (2.13г) касается исключительно случая $m=n$, тогда как множество полиномиальных линий степеней $m>n$, проходящих через n -интервальную узловую сетку, бесконечно! Действительно, если число неизвестных коэффициентов $m+1$ превышает число узлов

$n+1$, то в (2.13б) получим **недоопределённую** СЛАУ $SQ_{mn}=U$ с **бесконечным** множеством решений. Одно из них согласно (1.6г) имеет вид

$$S=UH_{nm}, \text{ где } H_{nm}=(Q_{mn}^T Q_{mn})^{-1} Q_{mn}^T, \quad (2.15)$$

и минимизирует сумму квадратов коэффициентов $S \circ S = |s_0|^2 + \dots + |s_m|^2$. Однако, уменьшение этой суммы с ростом степени полинома m вовсе не даёт автоматического улучшения качества интерполяционной линии в смысле снижения её межузловых осцилляций. В подтверждение сказанному на рис. 2.8,а построены интерполанты минимальной степени $m=n=6$ и избыточных степеней $m > n$.

Ещё более поучительны 3d-линии степеней $m=\{7,8,9\}$ на рис. 2.8,б. Сложно назвать их интерполяционными из-за очевидного несовпадения околоконечных точек $p(t_i)$ с узлами сетки u_i . Отклонения увеличиваются с ростом степени полинома и вызваны погрешностями обращения матрицы $Q_{mn}^T Q_{mn}$.

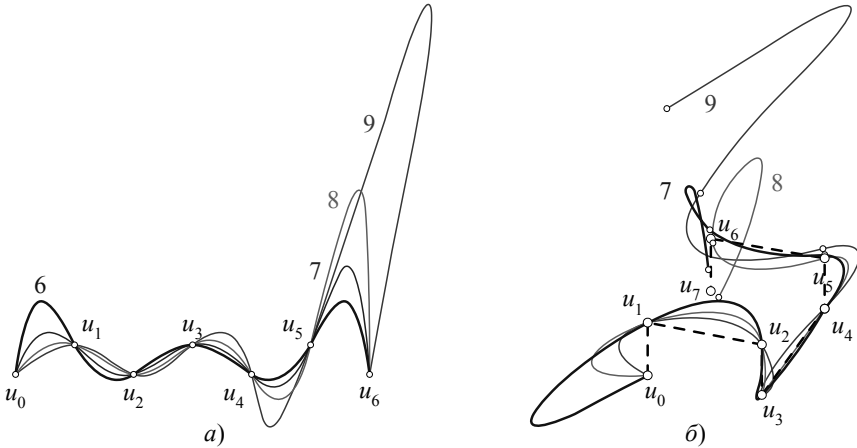


Рис. 2.8

Анализ построенных графиков позволяет сделать **вывод: следует избегать избыточных степеней интерполяционных полиномов!** Процесс конструирования интерполяционных линий должен быть интерактивным с выбором разных подходов, настроек параметров и наиболее приемлемого решения.

Для получения практического навыка найдём параметры нормализованных ($t_i=i$) интерполяционных полиномов трёх первых степеней:

- линейный ИП ($m=n=1$, $U=[u_0 \ u_1]$, $t \in [0,1]$):

$$Q_1 = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad H_1 = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad L_1(t) = \begin{bmatrix} 1-t \\ t \end{bmatrix}, \quad S = [u_0 \ u_1 - u_0]; \quad (2.16a)$$

- квадратичный ИП ($m=n=2$, $U=[u_0 \ u_1 \ u_2]$, $t \in [0,2]$):

$$Q_2 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}, \quad H_2 = \frac{1}{2} \begin{bmatrix} 2 & -3 & 1 \\ 0 & 4 & -2 \\ 0 & -1 & 1 \end{bmatrix}, \quad L_2(t) = \frac{1}{2} \begin{bmatrix} 2-3t+t^2 \\ 4t-2t^2 \\ -t+t^2 \end{bmatrix}, \quad (2.16b)$$

$$S = \begin{bmatrix} u_0 & \frac{-3u_0+4u_1-u_2}{2} & \frac{u_0-2u_1+u_2}{2} \end{bmatrix};$$

- кубический ИП ($m=n=3$, $U=[u_0 \ u_1 \ u_2 \ u_3]$, $t \in [0,3]$):

$$Q_3 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 4 & 9 \\ 0 & 1 & 8 & 27 \end{bmatrix}, \quad H_3 = \frac{1}{6} \begin{bmatrix} 6 & -11 & 6 & -1 \\ 0 & 18 & -15 & 3 \\ 0 & -9 & 12 & -3 \\ 0 & 2 & -3 & 1 \end{bmatrix},$$

$$L_3(t) = \begin{bmatrix} \frac{6-11t+6t^2-t^3}{6} & \frac{6t-5t^2+t^3}{2} & \frac{-3t+4t^2-t^3}{2} & \frac{2t-3t^2+t^3}{6} \end{bmatrix}^T, \quad (2.16\text{в})$$

$$S = \begin{bmatrix} u_0 & \frac{-11u_0+18u_1-9u_2+2u_3}{6} & \frac{2u_0-5u_1+4u_2-u_3}{2} & \frac{-u_0+3u_1-3u_2+u_3}{6} \end{bmatrix}.$$

По (2.16) на рис. 2.9 построены графики весовых функций $l_{n0}(t) \div l_{nn}(t)$, составляющих векторы $L_n(t)$, для полиномов степеней $n \in \{1, 2, 3\}$. Графики подтверждают наличие у ВФ узловых свойств (2.11б). Легко проверить, что у каждого полинома сумма весовых функций равна единице.

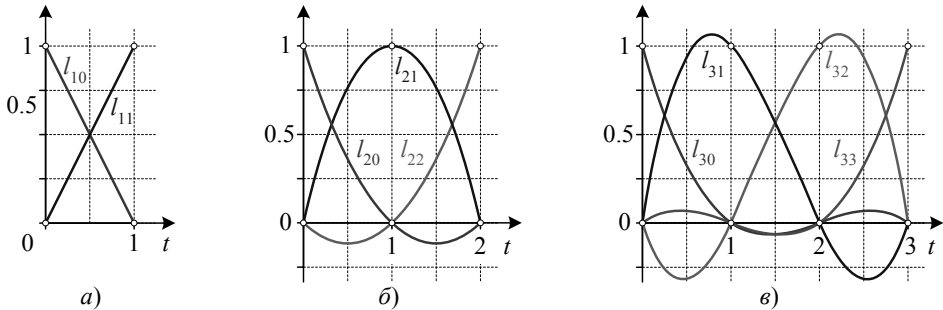


Рис. 2.9

Пример 2.1. Построить интерполяционный полином, проходящий через пять узловых точек $u_0=u_4=[1 \ 0]^T$, $u_1=[0 \ 1]^T$, $u_2=[-1 \ 0]^T$ и $u_3=[0 \ -1]^T$, лежащих на окружности единичного радиуса (рис. 2.10).

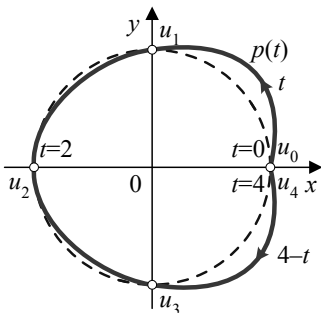


Рис. 2.10

Решение. Ввиду равенства длин хорд между смежными узлами будем строить нормализованный полином на четырехинтервальной параметрической сетке $t_i=i \ \forall i=\overline{0,4}$. Сформировав матрицу Вандермонда Q_4 , вычислим обратную МВК $H_4=Q_4^{-1}$ и ВПК $S=[u_0 \ u_1 \ u_2 \ u_3 \ u_4]H$:

$$Q_4 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 \\ 0 & 1 & 4 & 9 & 16 \\ 0 & 1 & 8 & 27 & 64 \\ 0 & 1 & 16 & 81 & 256 \end{bmatrix}, \quad H_4 = \frac{1}{24} \begin{bmatrix} 24 & -50 & 35 & -10 & 1 \\ 0 & 96 & -104 & 36 & -4 \\ 0 & -72 & 114 & -48 & 6 \\ 0 & 32 & -56 & 28 & -4 \\ 0 & -6 & 11 & -6 & 1 \end{bmatrix},$$

$$S = \frac{1}{6} \begin{bmatrix} 6 & 4 & -17 & 8 & -1 \\ 0 & 16 & -12 & 2 & 0 \end{bmatrix}.$$

Таким образом, векторная функция $p(t)=[x(t) \ y(t)]^T$, график которой точно проходит через заданные узловые точки, принимает следующий вид:

$$p(t) = \frac{1}{6} \begin{bmatrix} 6+4t-17t^2+8t^3-t^4 \\ 16t-12t^2+2t^3 \end{bmatrix} \quad \forall t \in [0, 4].$$

Эта линия максимально отклонена от единичной окружности на 17.9%. Налицо свойство осевой симметрии функции $p(t)$, подтверждаемое реверсом параметра:

$$p(4-t) = \frac{1}{6} \begin{bmatrix} 6+4t-17t^2+8t^3-t^4 \\ -16t+12t^2-2t^3 \end{bmatrix} = \begin{bmatrix} x(t) \\ -y(t) \end{bmatrix}.$$



Метод Эрмита

Графики на рис. 2.5–2.8 доказывают, что с ростом степени *свободного* полинома с коэффициентами (2.13г) либо (2.15) межузловые осцилляции наиболее сильно выражены на крайних интервалах $[t_0, t_1]$ и $[t_{n-1}, t_n]$. Чтобы уменьшить этот нежелательный эффект и заставить линию вести себя более предсказуемо, можно наложить на неё μ дополнительных условий, увеличивающих степень полинома до $m = n + \mu$. Этими условиями могут быть требования к производным функции $p(t)$ в некоторых или всех узлах интерполяции. Получаемое решение называется *интерполяционным полиномом Эрмита (ИПЭ)*.

Закрепим, например, *векторы краевых направлений* линии V_0 и V_n . Это означает $\mu=2$ и расширение системы (2.13а) с помощью вектора производных базисных функций (2.6б) двумя уравнениями $S_3 T'_m(t_0) = V_0$ и $S_3 T'_m(t_n) = V_n$:

$$S_3 [Q_{mn} \mid T'_m(t_0) \mid T'_m(t_n)] = [U \mid V_0 \mid V_n] \quad \text{или} \quad S_3 Q_3 = U_3. \quad (2.17a)$$

Решение этой полностью определённой при $m = n + 2$ системы — вектор коэффициентов $S_3 = [s_0 \ s_1 \ \dots \ s_m]$ — даётся формулой (1.6б)

$$S_3 = U_3 H_3^{-1}, \quad \text{где} \quad H_3 = Q_3^{-1}, \quad (2.17б)$$

после чего его можно использовать в (2.7б) для построения линии.

Аналогично (2.7а) формируется разложение полинома Эрмита по узловым элементам $\{u_0, \dots, u_n, V_0, V_n\}$

$$p(t) = \sum_{i=0}^n l_i^3(t) u_i + l_{n+1}^3(t) V_0 + l_{n+2}^3(t) V_n = U_3 L_3(t) \quad \forall t \in [t_0, t_n] \quad (2.18a)$$

с вычисляемым по (2.8) вектором весовых функций $L_3(t) = H_3 T_m(t)$, элементы которого есть полиномы степени m со следующими **узловыми свойствами**:

$$l_i^3(t_j) = \delta_i^j, \quad l'_{n+1}^3(t_j) = \delta_0^j, \quad l'_{n+2}^3(t_j) = \delta_n^j \quad \forall i, j \in [0, n]. \quad (2.18б)$$

В отличие от (2.11б) весовые функции векторов краевых направлений $l_{n+1}^3(t)$ и $l_{n+2}^3(t)$ обращаются в нуль во всех узлах сетки, а их производные равны единице в собственных крайних узлах.

Векторы V_0 и V_n можно зафиксировать разными способами:

- явно задать их координаты согласно желаемому поведению линии в начальной и конечной точках (рис. 2.11,а). Подобная задача встаёт, например, при

гладком заполнении разрыва между двумя линиями, у которых соединяемые концы имеют фиксированные направления. При этом могут существовать дополнительные точки, через которые должна пройти сопрягающая линия;

- задать направления выхода из узла u_0 в сторону точки u_1 и входа в узел u_n со стороны точки u_{n-1} (рис. 2.11,б):

$$V_0 = \gamma_0(u_1 - u_0), \quad V_n = \gamma_n(u_n - u_{n-1}). \quad (2.19)$$

Это притянет линию на крайних интервалах к хордам u_0u_1 и $u_{n-1}u_n$. Регулирование скоростей выхода и входа осуществляется с помощью **коэффициентов удлинения** γ_0 и γ_n . Уменьшение этих коэффициентов ослабляет притягивающий эффект. Однако, решение с коэффициентами $\gamma_0 = \gamma_n = 0$ (1 на рис. 2.11,в) не равносильно интерполяционной линии (2) со свободными концами, в которых векторы направлений $p'(t_0) = ST'_n(t_0)$ и $p'(t_n) = ST'_n(t_n)$ не будут нулевыми;

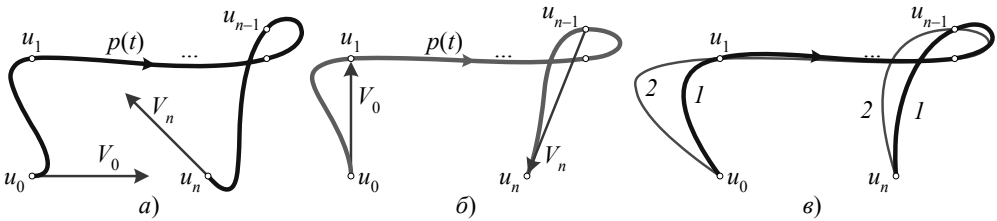


Рис. 2.11

- использовать формулы численного дифференцирования — **двухточечного**

$$V_0 = (u_1 - u_0)/d_1, \quad V_n = (u_n - u_{n-1})/d_n, \quad (2.20a)$$

совпадающего с (2.19) при $\gamma_0 = 1/d_1$ и $\gamma_n = 1/d_n$, или **трёхточечного**

$$\begin{cases} V_0 = \frac{-(2d_1 + d_2)d_2u_0 + (d_1 + d_2)^2u_1 - d_1^2u_2}{d_1(d_1 + d_2)d_2}, \\ V_n = \frac{d_n^2u_{n-2} - (d_{n-1} + d_n)^2u_{n-1} + d_{n-1}(d_{n-1} + 2d_n)u_n}{d_{n-1}(d_{n-1} + d_n)d_n}. \end{cases} \quad (2.20б)$$

В частном случае нормализованной параметризации ($d_i = 1 \quad \forall i = \overline{1, n}$) эти формулы приобретают простой вид [21, 60] — для двухточечной аппроксимации

$$V_0 = u_1 - u_0, \quad V_n = u_n - u_{n-1}, \quad (2.21a)$$

а для трёхточечной

$$V_0 = -1.5u_0 + 2u_1 - 0.5u_2, \quad V_n = 0.5u_{n-2} - 2u_{n-1} + 1.5u_n. \quad (2.21б)$$

На рис. 2.12 показаны эффекты фиксации краевых направлений линий Эрмита по двухточечной (а) и трёхточечной (б) схемам. Нормализованные линии показаны сплошными, а пунктирные построены хордовым методом.

Кроме (2.19) – (2.21) возможны и другие варианты расширения базовой системы (2.13а) дополнительными узловыми условиями:

- нулевая скорость $V_i = 0$ предотвращает сильный «разгон» в i -м узле и уменьшает осцилляции в смежных интервалах. Эффект обнуления краевых ско-

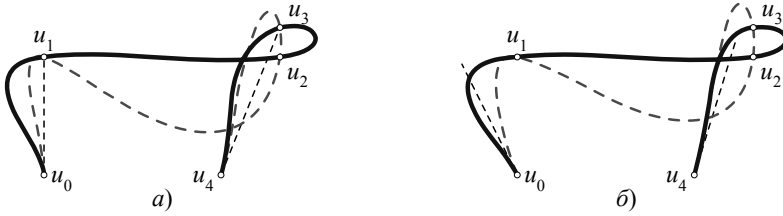


Рис. 2.12

ростей V_0 и V_n демонстрирует ИПЭ (1) на рис. 2.11, в, имеющий меньшие осцилляции в сравнении с ИП (2) со свободными концами;

• ненулевые векторы скоростей V_i при $0 < i < n$ заставляют полином пройти через **внутренние** узлы в желаемых направлениях и тем самым делают его форму более управляемой. В отсутствие явных требований к векторам направлений их можно задать трёхточечными аппроксимациями

$$V_i = \frac{-d_{i+1}^2 u_{i-1} + (d_{i+1}^2 - d_i^2) u_i + d_i^2 u_{i+1}}{d_i(d_i + d_{i+1})d_{i+1}} \quad \forall i = \overline{1, n-1}, \quad (2.22a)$$

принимающими в нормализованном варианте вид центральных разностей

$$V_i = 0.5(u_{i+1} - u_{i-1}) \quad \forall i = \overline{1, n-1}; \quad (2.22б)$$

• фиксация направлений линии во **всех** узловых точках приводит к системе $S_{\mathcal{Q}} \mathcal{Q} = U_{\mathcal{Q}}$, состоящей из $m = 2n + 1$ уравнений, с матрицами

$$\mathcal{Q} = [\mathcal{Q}_{mn} \mid T'_m(t_0) \ T'_m(t_1) \ \dots \ T'_m(t_n)], \quad U_{\mathcal{Q}} = [U \mid V_0 \ V_1 \ \dots \ V_n]. \quad (2.23)$$

Результат расчёта с матрицами (2.23) и аппроксимациями (2.21a) и (2.22б) иллюстрирует нормализованная линия Эрмита $m=15$ -й (!) степени, практически лучшей формы среди других возможных гладких решений (рис. 2.13, а);

• задание нулевого ускорения $A_i = 0$ создаёт в узле u_i нулевую кривизну и позволяет получить в нём точку перегиба интерполяционной линии. Уравнение фиксации желаемого ускорения A_i , записанное с помощью вектора вторых производных базисных функций (2.6в), имеет вид $S_{\mathcal{Q}} T''_m(t_i) = A_i$;

• у **циклической линии** помимо *необязательного* совпадения крайних узлов $u_0 = u_n$ должны быть равны векторы направлений $V_0 = V_n$, откуда следуют одно дополнительное уравнение $S_{\mathcal{Q}} T'_m(t_0) = S_{\mathcal{Q}} T'_m(t_n)$ при $m = n + 1$ и матрицы СЛАУ

$$\mathcal{Q}_{\text{ц}} = [\mathcal{Q}_{mn} \mid T'_m(t_n) - T'_m(t_0)], \quad U_{\text{ц}} = [U \mid 0]. \quad (2.24a)$$

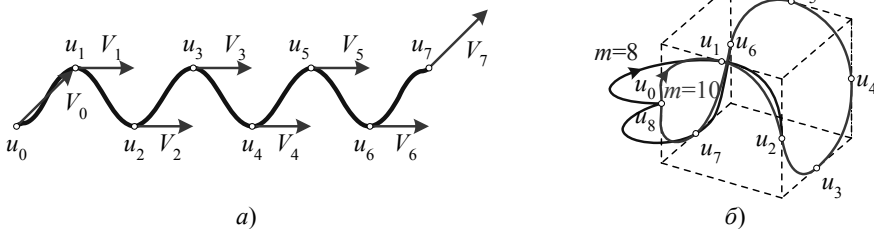


Рис. 2.13

Для получения дважды гладкого замыкания с непрерывным вектором ускорения $A_0=A_n$ система (2.24а) при $m=n+2$ расширяется вторым дополнительным уравнением $S_{\mathcal{D}}T_m''(t_0)=S_{\mathcal{D}}T_m''(t_n)$, в результате чего её матрицы приобретают вид

$$Q_{\Pi}=[Q_{mn} \mid T_m'(t_n)-T_m'(t_0) \mid T_m''(t_n)-T_m''(t_0)], \quad U_{\Pi}=[U \mid O \mid O]. \quad (2.24б)$$

Влияние условий двойной гладкости на получаемое решение иллюстрирует рис. 2.13,б, где построены графики двух пространственных замкнутых линий, проходящих через 9 узлов, причём $u_0=u_8$. Свободный полином степени $m=8$ имеет большие осцилляции и негладкое замыкание, чего нет у закреплённого ИПЭ степени $m=10$ с равными краевыми скоростями и ускорениями.

Обобщённый анализ графиков, построенных на рис. 2.11–2.13, позволяет сделать **вывод: закрепление узловых производных интерполяционных полиномов способствует, как правило, улучшению их качества.**

☞ Часто построение циклических линий становится невозможным из-за вырожденности матрицы Q_{Π} . Исследование этого свойства задачи полиномиальной интерполяции в книгах отсутствует. Рассмотрим суть проблемы на простом примере проведения кубической ($m=3$) линии $p(t)$ через три ($n=2$) узловые точки $\{u_0, u_1, u_2\}$ так, чтобы были равны её краевые производные $p'(t_0)$ и $p'(t_2)$.

Составим по (2.24а) матрицу Q_{Π} и вычислим её определитель как функцию параметрических длин $d_1=t_1-t_0>0$ и $d_2=t_2-t_1>0$:

$$Q_{\Pi}=[Q_{32} \mid T_3'(t_2)-T_3'(t_0)]=\begin{bmatrix} 1 & 1 & 1 & 0 \\ t_0 & t_1 & t_2 & 0 \\ t_0^2 & t_1^2 & t_2^2 & 2(t_2-t_0) \\ t_0^3 & t_1^3 & t_2^3 & 3(t_2^2-t_0^2) \end{bmatrix},$$

$$|Q_{\Pi}|=(t_1-t_0)(t_2-t_0)^2(t_2-t_1)(t_2-2t_1+t_0)=d_1d_2(d_1+d_2)^2(d_2-d_1). \quad (2.25а)$$

Отсюда следует, что матрица Q_{Π} вырождена, а **задача интерполяции с циклической гладкостью первого порядка неразрешима в двух случаях равенства параметрических длин $d_1=d_2$ [40, 41]:**

- при **нормализованной** параметризации ($d_1=d_2=1$) с **произвольным** расположением узловых точек;
- при **хордовой** параметризации с **равными** межузловыми расстояниями $|u_1-u_0|=|u_2-u_1|$.

Таким образом, двухинтервальная циклическая интерполяция с равными краевыми направлениями разрешима только методом **хордовой** параметризации и при **разных** длинах хорд $u_0=u_1$ и $u_1=u_2$, как показано на рис. 2.14,а. Этот вывод распространяется на все сетки с **чётным** числом интервалов n . При **нечётных** значениях n матрица Q_{Π} в (2.24а) всегда невырождена, а задача циклической интерполяции первого порядка гладкости разрешима как с нормализованной, так и с хордовой параметризацией. Например, при $n=3$ получаем определитель

$$|Q_{\Pi}|=d_1d_2d_3(d_1+d_2)(d_2+d_3)(d_1+d_2+d_3)^2(d_1^2+d_1d_2+d_2d_3+d_3^2), \quad (2.25б)$$

не равный нулю в обоих методах параметризации и при любом расположении узлов $u_0 \div u_3$, что доказывает гладко замкнутая линия на рис. 2.14,б.

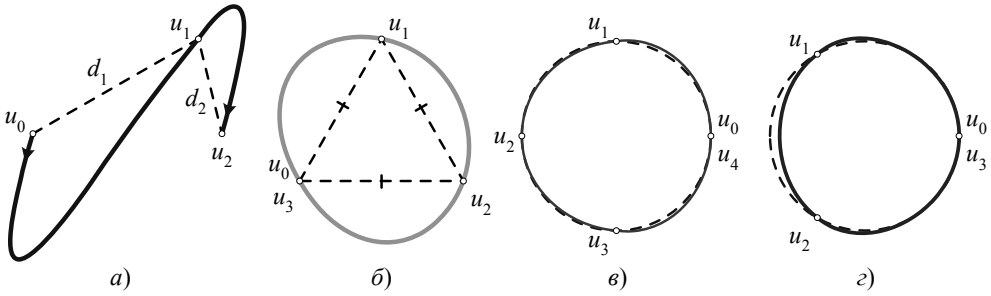


Рис. 2.14

Увеличение порядка циклической гладкости до второго делает, как показывает моделирование, матрицу (2.24б) невырожденной при любом расположении узлов и обоих методах параметризации. Например, при $n=2$ она принимает вид

$$Q_{\text{ц}} = [Q_{42} \quad T'_4(t_2) - T'_4(t_0) \quad T''_4(t_2) - T''_4(t_0)] = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ t_0 & t_1 & t_2 & 0 & 0 \\ t_0^2 & t_1^2 & t_2^2 & 2(t_2 - t_0) & 0 \\ t_0^3 & t_1^3 & t_2^3 & 3(t_2^2 - t_0^2) & 6(t_2 - t_0) \\ t_0^4 & t_1^4 & t_2^4 & 4(t_2^3 - t_0^3) & 12(t_2^2 - t_0^2) \end{bmatrix}$$

и имеет всегда ненулевой определитель

$$|Q_{\text{ц}}| = -12d_1^2 d_2^2 (d_1 + d_2)^3. \quad (2.25\text{в})$$

Таким образом, **при построении циклических линий рекомендуется всегда задавать равенство двух производных в конечных точках**. При совпадении последних линия автоматически станет дважды гладко замкнутой. Именно такая четырёхинтервальная полиномиальная линия 6-й степени с координатами

$$x(t) = \frac{(1-t)(3-t)(24+32t+8t^2-8t^3+t^4)}{72}, \quad y(t) = \frac{t(2-t)(4-t)(16+12t-3t^2)}{75}, \quad (2.26)$$

проходящая через точки $u_0 = u_4 = [1 \ 0]^T$, $u_1 = [0 \ 1]^T$, $u_2 = [-1 \ 0]^T$ и $u_3 = [0 \ -1]^T$, построена на рис. 2.14,в. Она прижата к единичной окружности значительно ближе (максимальное отклонение составляет 3.1%), чем интерполяционный полином со свободными концами (см. рис. 2.10). Для сравнения на рис. 2.14,г показан трёхинтервальный (узлы равномерно распределены по окружности) дважды гладкий ИПЭ, максимально отклонённый от окружности на 10.2%.

Для закрепления практического навыка получим элементы моделей нескольких нормализованных интерполяционных полиномов Эрмита на **двухузловой** ($n=1$) сетке $\{u_0, u_1\}$ при $t \in [0, 1]$ (вспомним, что по (2.16а) интерполянт со свободными концами представляет собой отрезок прямой $p(t) = u_0(1-t) + u_1 t$ с постоянной ненулевой производной $V(t) = u_1 - u_0$):

- квадратичный ИПЭ с фиксированным начальным направлением V_0 :

$$Q_{\mathfrak{D}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad H_{\mathfrak{D}} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 1 \\ 0 & 1 & -1 \end{bmatrix}, \quad L_{\mathfrak{D}}(t) = \begin{bmatrix} 1-t^2 \\ t^2 \\ t-t^2 \end{bmatrix}, \quad (2.27a)$$

$$U_{\mathfrak{D}} = [u_0 \ u_1 \ V_0], \quad S_{\mathfrak{D}} = [u_0 \ V_0 \ u_1 - u_0 - V_0];$$

- квадратичный ИПЭ с фиксированным конечным направлением V_1 :

$$Q_{\mathfrak{D}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \quad H_{\mathfrak{D}} = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}, \quad L_{\mathfrak{D}}(t) = \begin{bmatrix} 1-2t+t^2 \\ 2t-t^2 \\ -t+t^2 \end{bmatrix}, \quad (2.27б)$$

$$U_{\mathfrak{D}} = [u_0 \ u_1 \ V_1], \quad S_{\mathfrak{D}} = [u_0 \ 2(u_1 - u_0) - V_1 \ u_0 - u_1 + V_1];$$

- кубический ИПЭ с фиксированными векторами начальных направления V_0 и ускорения A_0 в узле u_0 :

$$Q_{\mathfrak{D}} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad H_{\mathfrak{D}} = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 0.5 & -0.5 \end{bmatrix}, \quad L_{\mathfrak{D}}(t) = \begin{bmatrix} 1-t^3 \\ t^3 \\ t-t^3 \\ 0.5(t^2-t^3) \end{bmatrix}, \quad (2.27в)$$

$$U_{\mathfrak{D}} = [u_0 \ u_1 \ V_0 \ A_0], \quad S_{\mathfrak{D}} = [u_0 \ V_0 \ 0.5A_0 \ u_1 - u_0 - V_0 - 0.5A_0];$$

- **кубический полином Эрмита** с фиксированными векторами краевых направлений V_0 и V_1 :

$$Q_{\mathfrak{D}} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 3 \end{bmatrix}, \quad H_{\mathfrak{D}} = \begin{bmatrix} 1 & 0 & -3 & 2 \\ 0 & 0 & 3 & -2 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & -1 & 1 \end{bmatrix}, \quad L_{\mathfrak{D}}(t) = \begin{bmatrix} 1-3t^2+2t^3 \\ 3t^2-2t^3 \\ t-2t^2+t^3 \\ -t^2+t^3 \end{bmatrix}, \quad (2.27г)$$

$$U_{\mathfrak{D}} = [u_0 \ u_1 \ V_0 \ V_1], \quad S_{\mathfrak{D}} = [u_0 \ V_0 \ 3(u_1 - u_0) - 2V_0 - V_1 \ 2(u_0 - u_1) + V_0 + V_1].$$

Его параметрическая модель

$$p(t) = U_{\mathfrak{D}} L_{\mathfrak{D}}(t) = (1-3t^2+2t^3)u_0 + (3t^2-2t^3)u_1 + (t-2t^2+t^3)V_0 + (-t^2+t^3)V_1 \quad (2.28)$$

находит широкое применение в конструкторской практике и различных графических редакторах благодаря простоте, небольшому объёму исходных данных (две точки и два вектора) и гибкости формы получаемых линий.

Пример 2.2. Построить полиномиальные модели двух линий, гладко сопрягающих отрезок ab с отрезками cd и ef (рис. 2.15). Найти оптимальные коэффициенты удлинения $\gamma \geq 0$, при которых получаемые линии наиболее близки к дугам $1/4$ и $1/2$ окружности.

Решение. Подставив в (2.27г) узловыe точки $u_0 = b = [1 \ 0]^T$, $u_1 = c = [0 \ 1]^T$ и векторы краевых направлений $V_0 = \gamma(b-a) = \gamma u^0$ и $V_1 = \gamma(d-c) = -\gamma x^0$ с общим коэффициентом удлинения $\gamma = \gamma_0 = \gamma_1$, запишем параметрическую модель линии, сопрягающей отрезки ab и cd на рис. 2.15,а, в виде

$$p(t, \gamma) = \begin{bmatrix} 1 & 0 & 0 & -\gamma \\ 0 & 1 & \gamma & 0 \end{bmatrix} L_{\mathfrak{D}}(t) = \begin{bmatrix} 1-3t^2+2t^3+\gamma t^2(1-t) \\ 3t^2-2t^3+\gamma t(1-t)^2 \end{bmatrix}, \quad 0 \leq t \leq 1. \quad (2.29a)$$

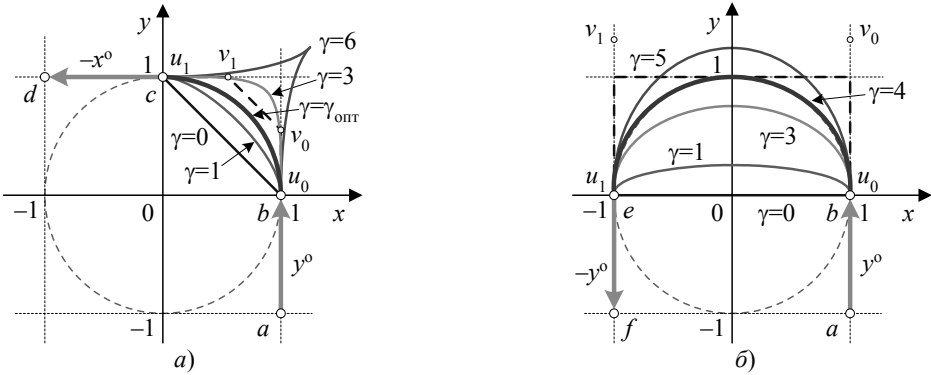


Рис. 2.15

Численный расчёт показал, что показатель близости полученной линии к дуге $1/4$ окружности $\mathfrak{Z}(\gamma) = \max_{t \in [0,1]} |p(t, \gamma) - 1|$ достигает минимума $2 \cdot 10^{-4} \approx 0.02\%$ при $\gamma_{\text{опт}} \approx 1.656$. Заметим, что условие прохождения линии (2.29а) через точку с координатами $x(0.5) = y(0.5) = \sqrt{2}/2$ даёт коэффициент $\gamma = 4(\sqrt{2} - 1) \approx 1.657$ и близкое к минимальному значение показателя $2.7 \cdot 10^{-4}$.

Для получения полиномиальной модели $1/2$ дуги окружности (рис. 2.15, б) подставим в (2.27г) узловые точки $u_0 = b = [1 \ 0]^T$, $u_1 = e = [-1 \ 0]^T$ и векторы $V_0 = \gamma y^0$, $V_1 = \gamma(f - e) = -\gamma y^0$:

$$p(t, \gamma) = \begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & \gamma & -\gamma \end{bmatrix} L_{\gamma}(t) = \begin{bmatrix} 1 - 6t^2 + 4t^3 \\ \gamma t(1-t) \end{bmatrix}, \quad 0 \leq t \leq 1. \quad (2.29б)$$

Показатель $\mathfrak{Z}(\gamma)$ достигает минимума $0.013 \approx 1.3\%$ при $\gamma_{\text{опт}} \approx 3.947$. Близкое к оптимальному значение $\gamma = 4$, полученное из условия $y(0.5) = 1$, даёт линию $p(t, 4)$, отклоняющуюся от дуги полуокружности не более, чем на $0.018 \approx 1.8\%$. $\square$

2.1.2. Аппроксимация и приближение по МНК

В п. 2.1.1 изучался метод построения ИП, доставляющего квадратичной сумме (2.12) абсолютный нулевой минимум при условии равенства числа интервалов сетки n степени полинома m . Отмечалось, что при $m \geq 7 \div 8$ резко возрастают межузловые осцилляции, ограничивающие практически оправданный размер узловой сетки. Вторым фактором, препятствующим широкому использованию интерполяции при неточных данных, выступает присущее ей свойство проводить решение через каждую узловую точку, что также создаёт сильные осцилляции при больших отклонениях узлов от незашумлённой траектории.

Рассмотрим ряд методов *сглаживания полиномиальных линий* [42].

$\square$ Один из методов основан на сознательном ограничении изгибаемости линии путём уменьшения степени полинома $m < n$. В расплату за это она теряет интерполяционные свойства $p(t_i) = u_i$ и превращается в *аппроксимационную*, проходящую лишь в *окрестности* вершин узловой сети (рис. 2.16). При-

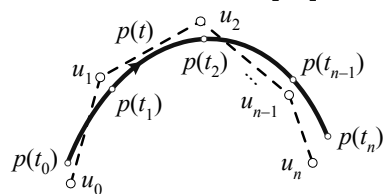


Рис. 2.16

мёним к построению полиномиальных линий оптимизационный подход, заключающийся в минимизации **функционала невязки** — суммы квадратов отклонений узловых точек полинома $p(t_i)$ от узловых точек сетки u_i :

$$\mathfrak{Z} = \sum_{i=0}^n |p(t_i) - u_i|^2 = (SQ_{mn} - U)(SQ_{mn} - U)^T \rightarrow \min, \quad (2.30a)$$

где Q_{mn} — матрица Вандермонда (2.13в). Ввиду **переопределённости** системы $SQ_{mn} = U$ её **приближённое**, минимизирующее критерий (2.30а) МНК-решение находится по (1.6в) как

$$S = UH_{nm}, \text{ где } H_{nm} = Q_{mn}^T (Q_{mn} Q_{mn}^T)^{-1}, \quad (2.30б)$$

а минимальное значение функционала невязки составляет

$$\mathfrak{Z}_{\min} = |UH_{nm}Q_{mn} - U|^2 = U(E_{n+1} - H_{nm}Q_{mn})U^T. \quad (2.30в)$$

Таким образом, при выборе степени приближающего полинома m меньшей числа интервалов сетки n получается ненулевая квадратичная сумма узловых отклонений, а **свободный приближающий полином не проходит точно ни через один узел**. Это расплата за уменьшение межузловых осцилляций, которыми грешат интерполяционные полиномы высоких степеней при большом числе узлов. Иллюстрацией сказанного служат построенные на рис. 2.17, а, б аппроксимационные полиномы степеней $m=1 \div 6$ на детерминированной (не зашумлённой) сетке с числом интервалов $n=7$.

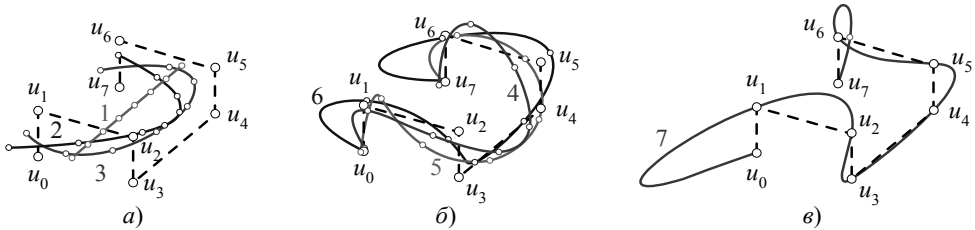


Рис. 2.17

В пределе $m=n$ решение (2.30б) сводится к (2.13г), сумма (2.30в) достигает абсолютного минимума $\mathfrak{Z}_{\min}=0$, а полиномиальная аппроксимация превращается в интерполяцию — точное прохождение приближающей линии через все узлы с максимальными осцилляциями (рис. 2.17, в). Аналогично (2.15) для устранения вычислительной неустойчивости расчёт МВК $Q_{nn}^T (Q_{nn} Q_{nn}^T)^{-1}$, **теоретически** равной Q_n^{-1} , следует заменить обращением матрицы Q_n .

□ Часто координаты вершин узловой сети зашумлены случайными помехами. Скажем, есть некоторая эталонная полиномиальная зависимость $p_3(t)$ степени m_3 и модель её измерения

$$u_i = p_3(t_i) + \xi_i \quad \forall i = \overline{0, n} \quad (2.31)$$

с векторами случайных смещений ξ_i , элементы которых есть РРСЧ (см. п. 1.1.6) $\text{rnd}(2\sigma) - \sigma \in (-\sigma, \sigma)$, где σ — полуширина центрированного интервала равно-

мерного распределения случайных чисел. В результате интерполяционная линия $p(t)$ степени $m=n$, стремясь пройти через все точки $u_0 \div u_n$, приобретает хаотические изгибы (рис. 2.18,а).

Аппроксимационный полином степени $m < n$ обладает замечательной способностью сглаживать осцилляции стохастического характера. По мере увеличения степени линия $p(t)$ проходит всё ближе и ближе к узлам u_i (рис. 2.18,б), а значение функционала (2.30в) уменьшается, достигая нуля при $m=n$, когда сглаживающий эффект исчезает. Наибольший спад зависимости $\mathfrak{Z}(m)$ при разных параметрах разброса σ стабильно наблюдается по достижении степени $m=m_3$ (рис. 2.18,в). Эту закономерность можно использовать для идентификации неизвестной степени полинома $p_3(t)$.

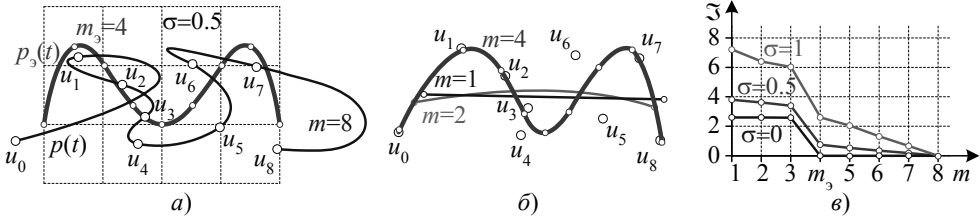


Рис. 2.18

□ Суммирование в (2.30а) квадратичных невязок в равных пропорциях приводит к получению полинома $p(t)$, равномерно приближенного ко всем узловым точкам независимо от погрешностей их измерения. Однако, во многих практических задачах узловые точки могут быть измерены с разными ошибками. Точные данные несут больше информации о неизвестной зависимости и вызывают больше доверия. Большие погрешности измерений приводят к чрезмерному «влиянию» интерполяционной линии, стремящейся пройти через каждый узел. Алгоритм аппроксимации должен давать решение, проходящее тем ближе к узловой точке, чем точнее она измерена.

Усовершенствуем метод (2.30) так, чтобы можно было регулировать близость линии (2.7б) к вершинам узловой сети с учётом ошибок их измерения. Для этого введём в квадратичный функционал невязки **весовые коэффициенты** $\varphi_i \geq 0$, входящие в диагональную матрицу $\Phi = \text{diag}[\varphi_1 \ \varphi_2 \ \dots \ \varphi_n]$ и обратно пропорциональные погрешностям измерения точек u_i :

$$\mathfrak{Z} = \sum_{i=0}^n \varphi_i |p(t_i) - u_i|^2 = (SQ_{mn} - U)\Phi(SQ_{mn} - U)^T \rightarrow \min_S. \quad (2.32a)$$

При задании коэффициента $\varphi_k=0$ вклад невязки $|p(t_k) - u_k|$ в общую сумму исчезает, что делает линию $p(t)$ полностью независимой от позиции точки u_k . Напротив, при $\varphi_k \rightarrow \infty$ зазор между точкой $p(t_k)$ и узлом u_k стремится к нулю.

Выполним минимизацию функционала (2.32а) **взвешенным методом наименьших квадратов (ВМНК)**. Из условия

$$d\mathfrak{Z}/dS = 2SQ_{mn}^T \Phi Q_{mn}^T - 2U\Phi Q_{mn}^T = O_{m+1}$$

вытекает решение задачи — вектор полиномиальных коэффициентов

$$S = UH_{nm}, \text{ где } H_{nm} = \Phi Q_{mn}^T (Q_{mn} \Phi Q_{mn}^T)^{-1}. \quad (2.32б)$$

Подставив его в (2.32а), получим минимальное значение функционала невязки

$$\mathfrak{I}_{\min} = U(E_{n+1} - H_{nm}Q_{mn})\Phi U^T. \quad (2.32в)$$

Очевидно, что задача (2.30а) и её решение (2.30б) представляют собой частный случай взвешенной задачи (2.32) с единичной матрицей Φ .

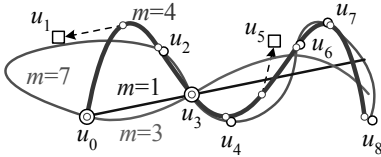


Рис. 2.19

Иллюстрацией описанного алгоритма полиномиальной аппроксимации служат линии разных степеней на рис. 2.19, построенные на эталонных узлах $p_3(t_i)$ из рис. 2.18,а, смещённых по (2.31) с разными максимальными отклонениями σ_i . Точки u_0 и u_3 , обозначенные

крупным символом «О», сгенерированы при малых отклонениях $\sigma=0.01$, точки выбросов u_1 и u_5 с большим значением $\sigma=1$ обозначены символом «□», а остальные узлы созданы при $\sigma=0.1$. Весовые коэффициенты функционала (2.32а) задавались равными $\varphi_i = \sigma_i^{-2}$.

□ При любом выборе **конечных** весовых коэффициентов $\varphi_0 \div \varphi_n$ линия $p(t)$, минимизирующая функционал невязки (2.32а), всегда будет аппроксимационной, поскольку для точного попадания в некоторый узел u_k нужно задать практически недостижимое значение коэффициента $\varphi_k = \infty$. К тому же с ростом отношения $\varphi_{\max}/\varphi_{\min}$ ухудшается обусловленность обращаемой в (2.32б) матрицы $Q_{mn}\Phi Q_{mn}^T$, что уменьшает вычислительную устойчивость взвешенного МНК.

В рамках МНК существует возможность совместить в **приближающей** линии как интерполяционные, так и аппроксимационные свойства, т. е. зафиксировать в некотором множестве узловых точек u_i желаемые состояния $p(t_i)$, направления $V(t_i)$ или ускорения $A(t_i)$, а вокруг остальных узлов u_j дать возможность линии вести себя более свободно. Например, при конструировании линий, составленных из сегментов невысоких порядков, нужно обеспечить непрерывное и гладкое сопряжение сегментов, что достигается закреплением их концов в определённых точках и направлениях.

Поставим задачу создания **приближающего полинома** $p(t)$ степени m на сетке узлов $\{t_i, u_i\} \forall i = \overline{0, n}$, обладающего следующими **свойствами** (рис. 2.20):

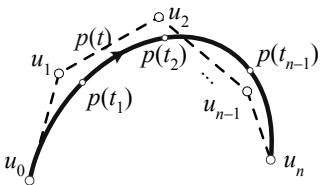


Рис. 2.20

- был закреплён, например, в крайних узлах u_0 и u_n , т. е. являлся **интерполяционным** решением системы из $v=2$ уравнений:

$$S[T_m(t_0) \ T_m(t_n)] = [u_0 \ u_n] \quad (2.33а)$$

или $SQ_n = U_n$.

Аналогично (2.17а) может быть задан и другой состав v условий фиксации узловых состояний

$p(t_i)=u_i$, скоростей $p'(t_i)=V_i$ и ускорений $p''(t_i)=A_i$ путём надлежащего формирования матрицы $Q_n \in \mathbb{R}^{(m+1) \times v}$ и вектора $U_n \in \mathbb{V}^v$. Например, система уравнений, фиксирующая концевые точки $p(t_0)=u_0$, $p(t_n)=u_n$ и векторы направлений $p'(t_0)=V_0$, $p'(t_n)=V_n$, имеет вид

$$S[T_m(t_0) \ T_m(t_n) \ T'_m(t_0) \ T'_m(t_n)] = [u_0 \ u_n \ V_0 \ V_n];$$

• проходил в окрестности внутренних узлов $u_1 \div u_{n-1}$, т. е. являлся **аппроксимационным** решением системы из $\mu = n-1$ приближённых уравнений:

$$S[T_m(t_1) \dots T_m(t_{n-1})] \approx [u_1 \dots u_{n-1}] \text{ или } SQ_a \approx U_a. \quad (2.33б)$$

Число μ и состав аппроксимируемых узлов могут быть изменены путём формирования соответствующих матрицы $Q_a \in \mathbb{R}^{(m+1) \times \mu}$ и вектора $U_a \in \mathbb{V}^\mu$.

В случае иного состава, чем в (2.33а,б), матрицы Q_n и Q_a должны быть **полноранговыми** и не иметь одинаковых столбцов. Другими словами, **условие** $ST_m(t_i)=u_i$ **должно входить лишь в одну из двух систем**, а узел u_i должен быть либо интерполяционным, либо аппроксимационным, что представляется вполне естественным требованием. В зависимости от соотношений между числом неизвестных коэффициентов $m+1$ и размерностями v и μ обе системы могут быть различных типов определённости, поэтому выбор из (1.6) подходящей формулы вычисления вектора S не однозначен.

Алгоритм решения общей **задачи условной аппроксимации** — минимизации квадратичного функционала невязок

$$\mathfrak{Z} = \sum_{i=1}^{n-1} |p(t_i) - u_i|^2 = (SQ_a - U_a)(SQ_a - U_a)^T \rightarrow \min_S \quad (2.33в)$$

в подпространстве решений системы $SQ_n=U_n$ — использует операцию **псевдообращения** матриц любых размерностей и рангов [10, 12, 24, 59]. Применительно к прямоугольной матрице $A \in \mathbb{R}^{n \times m}$ **полного ранга** $r = \min\{n, m\}$ псевдообратная матрица $A^+ \in \mathbb{R}^{m \times n}$ вычисляется следующим образом:

$$A^+ = \begin{cases} A^T (AA^T)^{-1} & \text{при } n < m; \\ A^{-1} & \text{при } n = m; \\ (A^T A)^{-1} A^T & \text{при } n > m. \end{cases}$$

Псевдообращение матрицы неполного ранга может быть выполнено методами ортогонального или сингулярного разложений. В частности, нулевая матрица $A = O_{n,m}$ имеет нулевую же псевдообратную матрицу $A^+ = O_{m,n}$.

С помощью операции псевдообращения матрицы решение системы линейных уравнений (1.6а) записывается в единой универсальной форме $p = BA^+$ и в случае полноранговой матрицы A превращается в BA^{-1} при $n=m$, $BA^T(AA^T)^{-1}$ при $n < m$ или $B(A^T A)^{-1}A^T$ при $n > m$.

Как показано в [24], решение линейной задачи наименьших квадратов с ограничениями-равенствами в принятых здесь обозначениях имеет вид

$$S = U_{\text{и}} Q_{\text{и}}^+ + (U_{\text{а}} - U_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}}) (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+. \quad (2.33\text{г})$$

Первое слагаемое $S_{\text{и}} = U_{\text{и}} Q_{\text{и}}^+$ представляет собой нормальное (минимальной нормы) псевдорешение СЛАУ $S Q_{\text{и}} = U_{\text{и}}$, а второе сдвигает вектор $S_{\text{и}}$ в точку минимума функционала (2.33в).

Путём изъятия из общей задачи одной из систем уравнений получим два частных случая задачи полиномиального приближения:

- в отсутствие аппроксимационной СЛАУ $S Q_{\text{а}} \approx U_{\text{а}}$ задачу **интерполяции** с решением $S = U_{\text{и}} (Q_{\text{и}}^T Q_{\text{и}})^{-1} Q_{\text{и}}^T$, которое при $U_{\text{и}} = U$, $m = n$ и $Q_{\text{и}} = Q_{nn}$ превращается в (2.13г);
- в отсутствие интерполяционной СЛАУ $S Q_{\text{и}} = U_{\text{и}}$ задачу **аппроксимации** с решением $S = U_{\text{а}} Q_{\text{а}}^T (Q_{\text{а}} Q_{\text{а}}^T)^{-1}$, переходящим в (2.30б) при $U_{\text{а}} = U$ и $Q_{\text{а}} = Q_{mn}$.

С помощью перестановок столбцов выражение (2.33г) вектора S через подмножества узлов $U_{\text{и}}$ из (2.33а) и $U_{\text{а}}$ из (2.33б) можно записать в типовом виде $S = UH$ с регулярным вектором узлов $U = [u_0 \ u_1 \ \dots \ u_{n-1} \ u_n]$:

$$S = [U_{\text{и}} \ U_{\text{а}}] \begin{bmatrix} Q_{\text{и}}^+ - Q_{\text{и}}^+ Q_{\text{а}} (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+ \\ (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+ \end{bmatrix} =$$

$$= U \underbrace{\begin{bmatrix} 1 & 0 & | & O_{1,n-1} \\ \hline O_{n-1,1} & O_{n-1,1} & | & E_{n-1} \\ 0 & 1 & | & O_{1,n-1} \end{bmatrix}}_H \begin{bmatrix} Q_{\text{и}}^+ - Q_{\text{и}}^+ Q_{\text{а}} (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+ \\ (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+ \end{bmatrix}. \quad (2.33\text{д})$$

Условие единственности решения — равенство ранга составной матрицы $[Q_{\text{и}} \ Q_{\text{а}}] \in R^{(m+1) \times (v+\mu)}$ числу $m+1$ коэффициентов приближающего полинома [24], которое (это **условие существования решения**) не должно быть меньшим числа интерполяционных уравнений:

$$v \leq m+1 \leq v+\mu. \quad (2.33\text{е})$$

Для матриц $Q_{\text{и}}$ и $Q_{\text{а}}$ эти условия сводятся к неравенству $1 \leq m \leq n$.

По алгоритму (2.33) на рис. 2.21 построены семейства приближающих полиномов разных степеней m с концами, зафиксированными в крайних узлах.

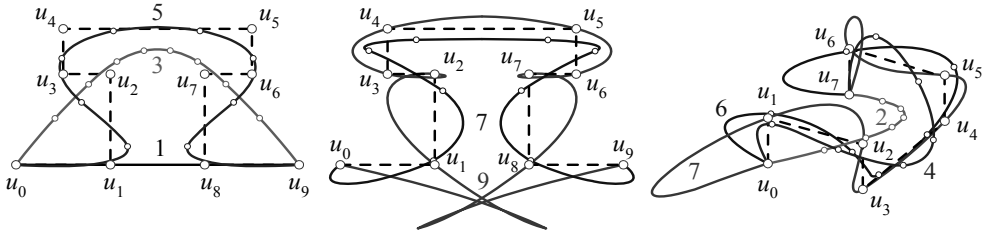


Рис. 2.21

2.1.3. Линии Безье

Переходим к изучению линий, совмещающих в себе как интерполяционные, так и аппроксимационные свойства относительно узловых точек. Это означает, что через одни точки линия проходит точно, а к другим приближается в некоторых окрестностях. Из всей последовательности вершин **характеристической полилинии** (ХП) $u_0 u_1 \dots u_n$, служащей основой построения приближающей линии, в качестве узлов интерполяции принимаются крайние вершины u_0 и u_n , тогда как остальные рассматриваются как аппроксимационные.

Кубические линии Безье

В сравнении с кубическим полиномом Эрмита (2.28) **интерактивное** конструирование линий, определённых положениями концов и краевыми направлениями, удобнее выполнять с помощью трёхзвенной ХП $u_0 v_0 v_1 u_1$, вершинами которой служат две **фиксирующие точки** $\{u_0, u_1\}$ и пара **управляющих точек** $\{v_0, v_1\}$. Они, как в (2.19), задают векторы начального $V_0 = \gamma_0(v_0 - u_0)$ и конечного $V_1 = \gamma_1(u_1 - v_1)$ направлений, длины которых регулируются коэффициентами $\gamma_0 \geq 0$ и $\gamma_1 \geq 0$ (рис. 2.22, а). Захватив курсором точку v_0 и перемещая её в плоскости экрана, можно интерактивно менять направление выхода линии из начального узла u_0 . Аналогично перемещением точки v_1 задаётся направление входа линии в конечный узел u_1 .

Подставив в (2.28) зависимости векторов $V_0 = \gamma_0(v_0 - u_0)$ и $V_1 = \gamma_1(u_1 - v_1)$ от вершин ХП, получим четырёхточечную модель кубической линии

$$p(t) = b_0(t)u_0 + b_1(t)v_0 + b_2(t)v_1 + b_3(t)u_1 \quad \forall t \in [0, 1] \quad (2.34a)$$

с весовыми функциями

$$\begin{aligned} b_0(t) &= 1 - \gamma_0 t + (2\gamma_0 - 3)t^2 + (2 - \gamma_0)t^3, & b_1(t) &= \gamma_0 t(1 - t)^2, \\ b_2(t) &= \gamma_1 t^2(1 - t), & b_3(t) &= (3 - \gamma_1)t^2 + (\gamma_1 - 2)t^3. \end{aligned} \quad (2.34b)$$

Зададимся вопросом: *какими следует задать коэффициенты γ_0 и γ_1 , чтобы достичь хорошей гибкости и предсказуемого поведения конструируемой линии?*

Из (2.34) следует, что при выборе $\gamma_0 = \gamma_1 = 0$ линия становится абсолютно лишённым гибкости отрезком **прямой** $p(t) = u_0 + t^2(3 - 2t)(u_1 - u_0)$. С другой стороны, при слишком больших краевых скоростях линия может очень далеко отклониться от ХП, иметь излом или даже петлю (рис. 2.22, б), что в большинстве проектов нежелательно. Конструктору хотелось бы иметь гладкую кривую, которая бы максимально заполняла пространство между вершинами характеристической полилинии, не выходя из него. Другими словами, **она должна принадлежать выпуклой оболочке массива точек** $\{u_0, v_0, v_1, u_1\}$ — четырёхугольнику минимальной площади в R^2 либо треугольной пирамиде минимального объёма в R^3 , плотно охватывающим четыре точки массива.

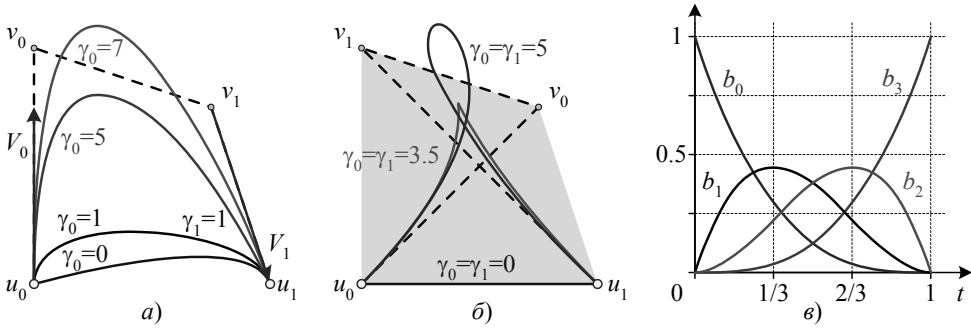


Рис. 2.22

По определению [48] **выпуклой оболочкой массива точек** $\{u_0, u_1, \dots, u_n\}$ называется линейное многообразие

$$p = \sum_{i=0}^n \alpha_i u_i, \text{ где } \alpha_i \geq 0 \quad \forall i = \overline{0, n}, \quad \sum_{i=0}^n \alpha_i = 1. \quad (2.35)$$

Легко убедиться, что сумма весовых функций (2.34б) равна единице. Поэтому условием принадлежности линии (2.34а) выпуклой оболочке массива точек $\{u_0, v_0, v_1, u_1\}$ является неотрицательность всех её ВФ в интервале $0 \leq t \leq 1$. Отсюда получаем следующие допустимые значения коэффициентов удлинения:

$$0 \leq \gamma_0 \leq \min_{t \in [0,1]} \left\{ \frac{1+2t}{t} \right\} = 3, \quad 0 \leq \gamma_1 \leq \min_{t \in [0,1]} \left\{ \frac{3-2t}{1-t} \right\} = 3.$$

Наибольшей полноты в границах выпуклой оболочки при любом расположении её вершин линия (2.34) достигает при $\gamma_0 = \gamma_1 = 3$. Она названа **линией Безье** в честь её патентообладателя — французского инженера и математика **Пьера Безье** (автор — акад. С. Н. Бернштейн). Параметрическая модель этой линии

$$p(t) = (1-t)^3 u_0 + 3t(1-t)^2 v_0 + 3t^2(1-t) v_1 + t^3 u_1 = U_B L_B(t) \quad \forall t \in [0,1] \quad (2.36a)$$

с узловым вектором $U_B = [u_0 \quad v_0 \quad v_1 \quad u_1]$ и весовыми функциями

$$b_0(t) = (1-t)^3, \quad b_1(t) = 3t(1-t)^2, \quad b_2(t) = 3t^2(1-t), \quad b_3(t) = t^3 \quad (2.36б)$$

(их графики построены на рис. 2.22,в) в большинстве источников приводится в готовом виде без связи с полиномом Эрмита и каких-либо обоснований выбора коэффициентов удлинения. Проведённый здесь анализ строго доказывает оптимальность данного решения. Параметры разложений (2.7) принимают вид

$$L_B(t) = \begin{bmatrix} 1-3t+3t^2-t^3 \\ 3t-6t^2+3t^3 \\ 3t^2-3t^3 \\ t^3 \end{bmatrix}, \quad H_B = \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (2.36в)$$

$$S_B = [u_0 \quad 3(v_0 - u_0) \quad 3(u_0 - 2v_0 + v_1) \quad u_1 - u_0 - 3(v_1 - v_0)].$$

Замечательным свойством кубических полиномиальных линий является их способность изгибаться в трёх измерениях в отличие от параболических линий, все точки которых лежат в одной плоскости (см. рис. 2.3,б). Непланарность по-

зволяет составлять из сегментов Безье пространственные линии, гладко сопряжённые как по направлению, так и по кривизне.

Для пояснения сказанного найдём производные векторной функции (2.36a)

$$\begin{aligned} V(t) &= p'(t) = -3(1-t)^2 u_0 + 3(1-4t+3t^2) v_0 + 3t(2-3t) v_1 + 3t^2 u_1, \\ A(t) &= V'(t) = 6(1-t) u_0 - 6(2-3t) v_0 + 6(1-3t) v_1 + 6t u_1, \end{aligned} \quad (2.37a)$$

определяющие векторы бинормали $B(t) = V(t) \times A(t)$ и нормали $N(t) = B(t) \times V(t)$, которые вместе с $V(t)$ составляют репер Френэ $\{\bar{V}, \bar{N}, \bar{B}\}$ (см. рис. 1.105) и по (1.110a) определяют кривизну $\kappa(t)$ и кручение $\chi(t)$. Обозначив для наглядности векторы рёбер ХП (*годографа* по [20]) как $G_0 = v_0 - u_0$, $G_1 = v_1 - v_0$ и $G_2 = u_1 - v_1$, запишем значения основных краевых векторов сегмента Безье:

$$\begin{aligned} V_0 &= 3G_0, \quad A_0 = 6(G_1 - G_0), \quad B_0 = 18(G_0 \times G_1), \quad N_0 = 54(G_0 \times G_1) \times G_0; \\ V_1 &= 3G_2, \quad A_1 = 6(G_2 - G_1), \quad B_1 = 18(G_1 \times G_2), \quad N_1 = 54(G_1 \times G_2) \times G_2. \end{aligned} \quad (2.37б)$$

Они объясняют влияние вершин ХП на **краевые свойства линий Безье**:

- краевые направления задаются двумя парами крайних вершин;
- краевые изгибы задаются двумя тройками крайних вершин;
- тройка векторов $\{V_i, A_i, N_i\}$ в концевой точке u_i , $i \in \{0, 1\}$, лежит в соприкасающейся плоскости $\beta_i = \{u_i, v_0, v_1\}$, определённой тремя крайними вершинами характеристической полилинии;
- вектор бинормали B_i ортогонален соприкасающейся плоскости β_i .

В зависимости от вида выпуклой оболочки точек $\{u_0, v_0, v_1, u_1\}$ возможны следующие формы и свойства линии Безье (рис. 2.23):

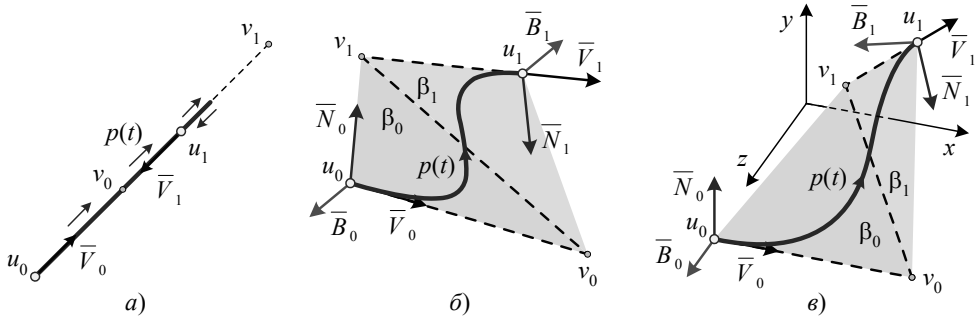


Рис. 2.23

• **коллинеарные** вершины (а) образуют одномерную оболочку в виде *отрезка*, внутри которого точка $p(t)$ движется *прямолинейно*, но с переменной скоростью $V(t)$, с параллельным вектором ускорения $A(t) \parallel V(t)$, нулевыми векторами $N(t) = O$, $B(t) = O$ и нулевыми кривизной $\kappa(t) = 0$ и кручением $\chi(t) = 0$. Если управляющие вершины v_0 или v_1 оказываются граничными точками оболочки, то линия Безье может иметь одну или две точки возврата;

• **компланарные** вершины (б) образуют *плоскую* четырёхугольную оболочку, совпадающую с обеими плоскостями β_0 и β_1 . Все бинормали $B(t)$ параллельны,

что даёт линии Безье нулевое кручение $\chi(t)=0$. При Z-образном расположении вершин ХП на линии $p(t)$ появляется точка перегиба из-за противоположных направлений векторов B_0 и B_1 ;

- в общем случае (в) вершины образуют **объёмную** тетраэдральную оболочку, гранями которой являются непараллельные соприкасающиеся плоскости β_0 и β_1 , из-за чего краевые бинормали B_0 и B_1 также не параллельны, а линия $p(t)$ имеет **ненулевые** кривизну $\kappa(t)$ и кручение $\chi(t)$.

Кубические линии Безье находят широкое применение в проектировании сложных составных линий, входя в них в качестве сегментов, сопрягаемых друг с другом с порядком гладкости не выше второго. Пример такой непрерывной линии, составленной из шести сегментов Безье, приведён на рис. 2.24. Нулевой порядок гладкости сопряжения позволяет создать изломы составной линии в точках соединения сегментов (все точки, кроме u_2). Для обеспечения первого (только по направлению) порядка гладкости сопряжения нужно расположить управляющие точки смежных сегментов на одной прямой линии по разные стороны от желаемой узловой точки (например, u_2). Для получения второго порядка гладкости тройки крайних вершин ХП смежных сегментов размещаются так, чтобы кривизна κ_1 предыдущего сегмента была равна кривизне κ_0 следующего сегмента, что не всегда выполнимо из-за малого числа вершин характеристических полилиний сегментов.

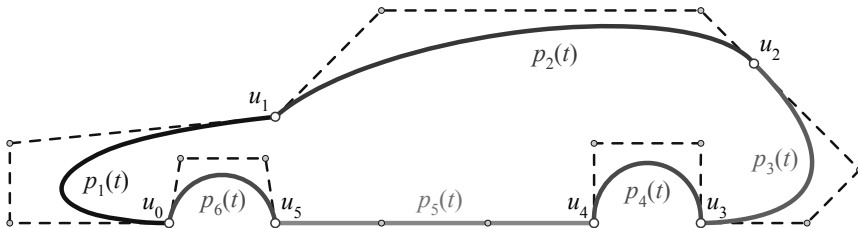


Рис. 2.24

Полиномиальные линии Безье

Обобщением кубической линии Безье (2.36а) на ХП $u_0 u_1 \dots u_n$ является нормализованный полином степени $n \geq 1$ (тоже носящий имя Безье)

$$p(t) = \sum_{i=0}^n b_{ni}(t) u_i = U B_n(t) = S_n T_n(t) = U H_n T_n(t) \quad \forall t \in [0, 1], \quad (2.38a)$$

весовые функции которого составляют **базис Бернштейна** [13, 20, 27, 50, 56]

$$b_{ni}(t) = C_n^i t^i (1-t)^{n-i} = \sum_{j=i}^n (-1)^{j-i} C_n^j C_j^i t^j \quad \forall i = \overline{0, n}, \quad (2.38б)$$

где $C_n^i = n! / (i!(n-i)!)$ — биномиальные коэффициенты при $0! = 1$. Параметры разложений (2.7) также обобщают (2.36в) на вектор вершин ХП $U = [u_0 \ u_1 \ \dots \ u_n]$:

$$H_n = \begin{bmatrix} h_{00} & h_{01} & \dots & h_{0n} \\ 0 & h_{11} & \dots & h_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & h_{nn} \end{bmatrix}, \quad L_n(t) = B_n(t), \quad S_n = [s_0 \ s_1 \ \dots \ s_n], \quad (2.38в)$$

$$\text{где } h_{ij} = (-1)^{j-i} C_n^j C_j^i \quad \forall 0 \leq j \leq n, \quad 0 \leq i \leq j, \quad s_j = \sum_{i=0}^j h_{ij} u_i \quad \forall 0 \leq j \leq n.$$

Весовые функции (2.38б) и построенная на их основе линия (2.38а) обладают следующими **свойствами**:

- неотрицательность и ограниченность:

$$0 \leq b_{ni}(t) \leq b_{ni}(i/n) = C_n^i \frac{i^i (n-i)^{n-i}}{n^n} \leq 1 \quad \forall t \in [0, 1], \quad i = \overline{0, n}. \quad (2.39а)$$

Максимальное влияние характеристической точки u_i на форму линии Безье оценивается числом $b_{ni}(i/n)$ и уменьшается с приближением номера точки i к среднему значению $[n/2]$, что ясно демонстрирует рис. 2.25,а;

- сумма всех весовых функций равна единице:

$$\sum_{i=0}^n b_{ni}(t) = \sum_{i=0}^n C_n^i (1-t)^{n-i} t^i = (1-t+t)^n = 1. \quad (2.39б)$$

Благодаря свойствам (2.39а,б), удовлетворяющим условиям (2.35), **линия Безье лежит в выпуклой оболочке массива точек** $\{u_0, u_1, \dots, u_n\}$;

- краевые значения ВФ равны $b_{ni}(0) = \delta_i^0$, $b_{ni}(1) = \delta_i^n \quad \forall i = \overline{0, n}$, благодаря чему **линия Безье фиксирована в крайних узлах ХП**:

$$p(0) = b_{n0}(0)u_0 = u_0, \quad p(1) = b_{nn}(1)u_n = u_n; \quad (2.39в)$$

- производная векторной функции (2.38а) определяется выражением

$$p'(t) = n \sum_{i=0}^{n-1} b_{n-1,i}(t)(u_{i+1} - u_i),$$

из которого находятся векторы краевых направлений линии Безье

$$V_0 = p'(0) = n(u_1 - u_0), \quad V_n = p'(1) = n(u_n - u_{n-1}), \quad (2.39г)$$

задаваемые крайними отрезками ХП u_0u_1 и $u_{n-1}u_n$;

- вторая производная векторной функции (2.38а)

$$p''(t) = n(n-1) \sum_{i=0}^{n-2} b_{n-2,i}(t)(u_{i+2} - 2u_{i+1} + u_i)$$

есть вектор ускорения $A(t)$, лежащий в соприкасающейся плоскости линии $p(t)$ (см. рис. 1.105). Краевые ускорения линии Безье определяются крайними тройками характеристических вершин $\{u_0, u_1, u_2\}$ и $\{u_n, u_{n-1}, u_{n-2}\}$:

$$A_0 = p''(0) = n(n-1)(u_0 - 2u_1 + u_2), \quad A_n = p''(1) = n(n-1)(u_n - 2u_{n-1} + u_{n-2}). \quad (2.39д)$$

Направления этих векторов совпадают с диагоналями параллелограммов $(u_0 - u_1) + (u_2 - u_1)$ и $(u_{n-2} - u_{n-1}) + (u_n - u_{n-1})$ (рис. 2.25,а);

- симметрией $b_{ni}(t) = b_{n,n-i}(1-t)$ относительно медианы параметра $t=0.5$:

$$C_n^i = C_n^{n-i} \Rightarrow C_n^i t^i (1-t)^{n-i} = C_n^{n-i} (1-t)^{n-i} t^i \equiv b_{n,n-i}(1-t). \quad (2.39\epsilon)$$

Иллюстрацией этого свойства служат графики $b_{70}(t) \div b_{77}(t)$ на рис. 2.25,б. Благодаря ему линия $p(t)$, построенная на ХП $u_0 u_1 \dots u_n$, проходит по линии $q(t) = p(1-t)$, созданной на реверсивной ХП $u_n u_{n-1} \dots u_0$.

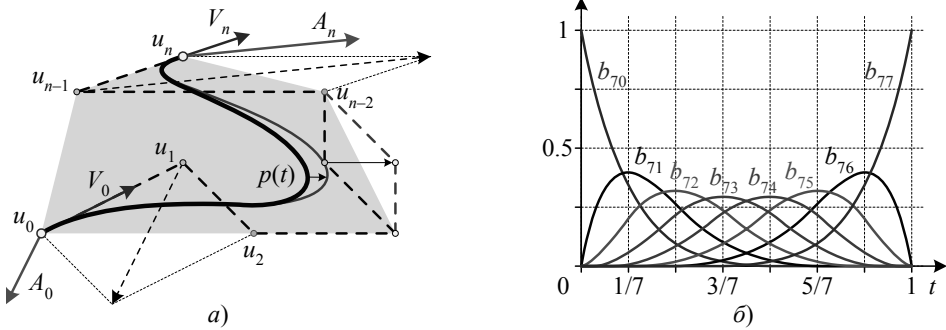


Рис. 2.25

Рассмотрим формы линий Безье некоторых степеней $n \neq 3$:

- при $n=1$ вектор $U = \{u_0, u_1\}$ включает только крайние узлы ХП, параметры трёх форм разложений (2.7) принимают вид

$$L_1(t) = \begin{bmatrix} 1-t \\ t \end{bmatrix}, \quad H_1 = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad S_1 = [u_0 \quad u_1 - u_0], \quad (2.40a)$$

а полином (2.38а) превращается в **отрезок** $p(t) = (1-t)u_0 + tu_1$ (рис. 2.26,а);

- при $n=2$ одна вершина u_1 двухзвенной ХП $U = \{u_0, u_1, u_2\}$ управляет обоими краевыми направлениями $V_0 = 2(u_1 - u_0)$ и $V_2 = 2(u_2 - u_1)$ **квадратичной** линии Безье $p(t) = (1-t)^2 u_0 + 2t(1-t)u_1 + t^2 u_2$ с параметрами разложений (2.7)

$$L_2(t) = \begin{bmatrix} (1-t)^2 \\ 2t(1-t) \\ t^2 \end{bmatrix}, \quad H_2 = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 2 & -2 \\ 0 & 0 & 1 \end{bmatrix}, \quad S_2 = [u_0 \quad 2(u_1 - u_0) \quad u_2 - 2u_1 + u_0]. \quad (2.40б)$$

На всем протяжении эта **плоская** (см. рис. 2.3,б) линия имеет **постоянные** соприкасающуюся плоскость $\beta = \{u_0, u_1, u_2\}$, вектор ускорения $A = 2(u_2 - 2u_1 + u_0)$ и вектор бинормали $B = (u_1 - u_0) \times (u_2 - u_1)$ (рис. 2.26,б);

- при $n=4$ в (2.38) получается линия Безье

$$p(t) = (1-t)^4 u_0 + 4t(1-t)^3 u_1 + 6t^2(1-t)^2 u_2 + 4t^3(1-t) u_3 + t^4 u_4$$

с параметрами разложений

$$L_4(t) = \begin{bmatrix} (1-t)^4 \\ 4t(1-t)^3 \\ 6t^2(1-t)^2 \\ 4t^3(1-t) \\ t^4 \end{bmatrix}, H_4 = \begin{bmatrix} 1 & -4 & 6 & -4 & 1 \\ 0 & 4 & -12 & 12 & -4 \\ 0 & 0 & 6 & -12 & 6 \\ 0 & 0 & 0 & 4 & -4 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, S_4 = \begin{bmatrix} u_0 \\ 4(u_1 - u_0) \\ 6(u_2 - 2u_1 + u_0) \\ 4(u_3 - 3u_2 + 3u_1 - u_0) \\ u_4 - 4u_3 + 6u_2 - 4u_1 + u_0 \end{bmatrix}^T. \quad (2.40в)$$

Каждая из вершин u_1 и u_3 четырёхзвенной ХП $U = \{u_0, u_1, u_2, u_3, u_4\}$ управляет своим краевым направлением $V_0 = 4(u_1 - u_0)$ и $V_4 = 4(u_4 - u_3)$, тогда как смещение точки u_2 , общей для соприкасающихся плоскостей $\beta_0 = \{u_0, u_1, u_2\}$ и $\beta_4 = \{u_2, u_3, u_4\}$, изменяет обе бинормали B_0, B_4 и векторы ускорения A_0, A_4 (рис. 2.26, в). Это обстоятельство связывает кривизны концов друг с другом и усложняет дважды гладкое сопряжение таких сегментов;

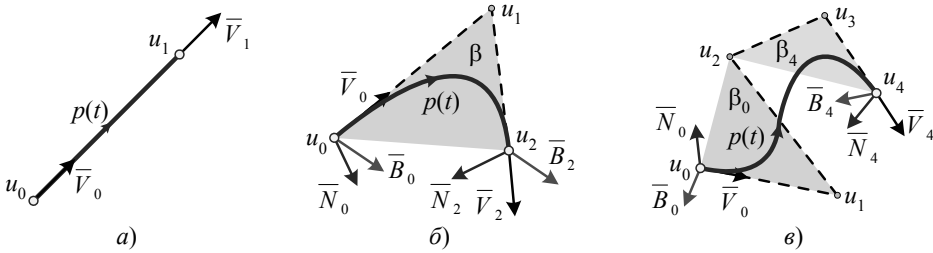


Рис. 2.26

• минимальное число характеристических точек, необходимых для построения линии Безье с независимым управлением краевыми направлениями и изгибами, равно шести. Полином $n=5$ -й степени

$$p(t) = (1-t)^5 u_0 + 5t(1-t)^4 u_1 + 10t^2(1-t)^3 u_2 + 10t^3(1-t)^2 u_3 + 5t^4(1-t) u_4 + t^5 u_5$$

имеет параметры разложений

$$L_5(t) = [(1-t)^5 \ 5t(1-t)^4 \ 10t^2(1-t)^3 \ 10t^3(1-t)^2 \ 5t^4(1-t) \ t^5]^T, \\ H_5 = \begin{bmatrix} 1 & -5 & 10 & -10 & 5 & -1 \\ 0 & 5 & -20 & 30 & -20 & 5 \\ 0 & 0 & 10 & -30 & 30 & -10 \\ 0 & 0 & 0 & 10 & -20 & 10 \\ 0 & 0 & 0 & 0 & 5 & -5 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, S_5 = \begin{bmatrix} u_0 \\ 5(u_1 - u_0) \\ 10(u_2 - 2u_1 + u_0) \\ 10(u_3 - 3u_2 + 3u_1 - u_0) \\ 5(u_4 - 4u_3 + 6u_2 - 4u_1 + u_0) \\ u_5 - 5u_4 + 10u_3 - 10u_2 + 5u_1 - u_0 \end{bmatrix}^T. \quad (2.40г)$$

По желаемым краевым свойствам линии Безье, задаваемым двумя наборами $\{u_0, V_0, A_0\}$ и $\{u_n, V_n, A_n\}$, на основе (2.39г,д) находятся управляющие вершины

$$u_1 = u_0 + n^{-1}V_0, \quad u_2 = u_0 + 2n^{-1}V_0 + n^{-1}(n-1)^{-1}A_0, \\ u_{n-1} = u_n - n^{-1}V_n, \quad u_{n-2} = u_n - 2n^{-1}V_n + n^{-1}(n-1)^{-1}A_n. \quad (2.41)$$

Расположение остальных внутренних точек $u_3 \div u_{n-3}$ при $n \geq 6$ может быть вычислено по краевым производным третьего и более высоких порядков.

Пример 2.3. Построить линии Безье небольших порядков, наиболее близкие к окружности единичного радиуса.

Решение. Для того чтобы линия, приближающая полную окружность, обладала дважды гладким замыканием и симметрией, её ХП $u_0 u_1 \dots u_n$ должна быть замкнутой в вершинах $u_0 = u_n$, иметь равные коллинеарные отрезки $u_0 u_1$, $u_{n-1} u_n$ и ортогональную им ось симметрии, проходящую через точку u_0 . Всё это возможно при числе рёбер и степени полинома $n \geq 4$. Ограничимся построением кругоподобных линий Безье двух наименьших порядков.

Установим крайние вершины ХП в точке $u_0 = u_n = [1 \ 0]^T$, тогда остальные вершины будут зеркально симметричны вдоль оси y : $u_1 = [1 \ y_1]^T$ и $u_{n-1} = [1 \ -y_1]^T$, $u_2 = [x_2 \ y_2]^T$ и $u_{n-2} = [x_2 \ -y_2]^T \dots$ Неизвестные координаты должны быть найдены из условий прохождения линии через опорные точки или оптимизационным методом приближения к окружности.

1. При $n=4$ нужно разместить три вершины u_1 , $u_2 = [x_2 \ 0]^T$ и $u_3 = [1 \ -y_1]^T$ (рис. 2.27,а). Абсцисса x_2 , управляющая горизонтальным габаритом линии, находится из условия прохождения середины линии $p(t) = [x(t) \ y(t)]^T$ через точку $[-1 \ 0]^T$:

$$p(0.5) = 2^{-4}(u_0 + 4u_1 + 6u_2 + 4u_3 + u_4) = [(5+3x_2)/8 \ 0]^T \Rightarrow x_2 = -13/3.$$

Ординату y_1 , управляющую вертикальным габаритом линии, найдём по опробованному в примере 2.2 минимаксному критерию $\mathfrak{Z}(y_1) = \max_{t \in [0,1]} |p(t, y_1) - 1| \rightarrow \min$. Численный расчёт показал, что минимум этого критерия радиальной близости линии Безье к единичной окружности, равный $0.046 \approx 4.6\%$, достигается при $y_1 \approx 2.546$. Весьма большое максимальное отклонение построенной по (2.38) оптимальной линии Безье от окружности объясняется малым количеством характеристических точек, управляющих её формой.

2. При $n=5$ выберем четыре вершины ХП $u_1 = [1 \ y_1]^T$, $u_2 = [x_2 \ y_2]^T$, $u_3 = [x_2 \ -y_2]^T$ и $u_4 = [1 \ -y_1]^T$ с тремя неизвестными координатами (рис. 2.27,б). Абсцисса x_2 по-прежнему влияет на горизонтальный габарит линии и находится из условия $p(0.5) = [-1 \ 0]^T$:

$$p(0.5) = 2^{-5}(u_0 + 5u_1 + 10u_2 + 10u_3 + 5u_4 + u_5) = [(3+5x_2)/8 \ 0]^T \Rightarrow x_2 = -11/5.$$

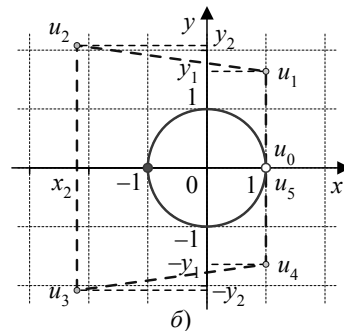
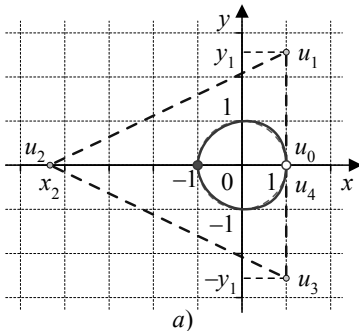


Рис. 2.27

Неизвестные ординаты y_1 и y_2 , управляющие вертикальным габаритом и формой линии, найдём по минимаксному критерию $\mathfrak{Z}(y_1, y_2) = \max_{t \in [0,1]} |p(t, y_1, y_2) - 1| \rightarrow \min$. Оптимизация в пространстве двух параметров дала минимум показателя $0.0014 \approx 0.14\%$ при $y_1 \approx 1.64$ и $y_2 \approx 2.08$. Существенно лучшая точность приближения к окружности объясняется большим числом характеристических вершин, управляющих формой линии Безье. $\square$

Редактирование линий Безье

Область применения линий Безье — системы автоматизированного проектирования разнообразных линий и поверхностей со сложными краями. Создание таких линий происходит в **интерактивном** режиме: сначала конструктор задаёт начальное расположение ХП и строит на ней линию (2.38), после чего в процессе **редактирования характеристической полилинии** добивается желаемой формы получаемого решения.

Редактирование ломаной линии означает проведение операций с её вершинами: перемещение имеющихся, добавление новых и удаление лишних. После каждой операции линия Безье перестраивается и заново оценивается. Если перенос характеристической вершины не вызывает затруднений (это можно сделать ручным вводом координат самой точки или вектора её переноса, либо захватом и перетаскиванием точки мышью), то операции добавления и удаления вершин полилинии изменяют её размер и требуют отдельного рассмотрения.

□ Цель **добавления** новой вершины в полилинию $u_0 u_1 \dots u_{n-1} u_n$ — повышение порядка и гибкости линии Безье. В результате в новой ХП $u'_0 u'_1 \dots u'_n u'_{n+1}$ перераспределяется влияние узлов на значения $p(t)$. Рассмотрим два способа увеличения порядка линии Безье, отличающиеся изменчивостью её формы.

1. По первому способу новая вершина u'_i вставляется между выбранными узлами u_{i-1} и u_i , например, в центр отрезка $u_{i-1} u_i$, **без изменения существующих вершин**. Таким образом, новая ХП определяется вершинами

$$u'_j = u_j \quad \forall j = \overline{0, i-1}, \quad u'_i = 0.5(u_{i-1} + u_i), \quad u'_j = u_{j-1} \quad \forall j = \overline{i+1, n+1}, \quad (2.42a)$$

а порождаемая ею линия Безье $n' = n+1$ -го порядка описывается моделью

$$p'(t) = \sum_{i=0}^{n'} b_{n'i}(t) u'_i \quad \forall t \in [0, 1]. \quad (2.42б)$$

Поскольку весовые функции $b_{n'i}(t)$ отличны от ВФ n -й степени (2.38б), а старая и новая ХП отличаются лишь одной вершиной u'_{i+1} , обладающей дополнительным притяжением, то новая и старая линии Безье не совпадают (рис. 2.28). Больше всего форма линии меняется вблизи новой вершины u'_i , будь она на среднем (а) либо крайнем (б) рёбрах ХП. Таким образом, **метод вставки новой вершины путём дробления ребра характеристической полилинии изменяет форму линии Безье** в зоне притяжения вставленной вершины.

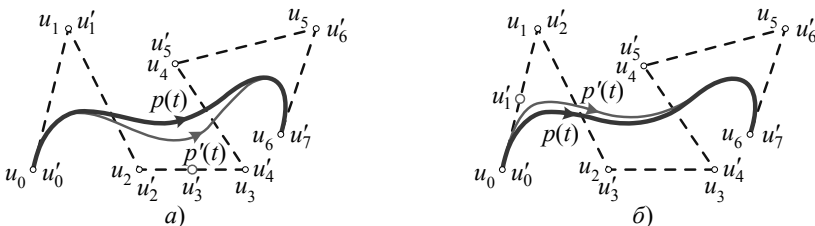


Рис. 2.28

2. В основе второго способа добавления в ХП новых вершин лежит условие **сохранения формы линии Безье** $p'(t)=p(t)$. Известный из [20] метод решения этой задачи при $n'=n+1$ основан на умножении суммы (2.38а) на тождество $(t+1-t)=1$, увеличивающее степень полинома на 1, и приведении слагаемых к виду, содержащему весовую функцию $n+1$ -й степени $C_{n+1}^i t^i (1-t)^{n+1-i}$. В результате не совсем простых преобразований получается следующий алгоритм инкрементирования размера характеристической полилинии:

$$u'_0 = u_0, \quad u'_i = \frac{i}{n+1} u_{i-1} + \left(1 - \frac{i}{n+1}\right) u_i \quad \forall i = \overline{1, n}, \quad u'_{n+1} = u_n. \quad (2.43)$$

В [51] этот же результат приведён без вывода. Но ни в одном из этих уважаемых источников не исследованы свойства полученной линии $p'(t)$.

Ключевым моментом предлагаемого альтернативного алгоритма [33] является тот факт, что **реальная степень полинома** $p'(t) = s'_0 + s'_1 t + \dots + s'_{n+1} t^{n+1}$, **повторяющего форму полинома** $p(t) = s_0 + s_1 t + \dots + s_n t^n$, **равна n** , а не $n+1$. Действительно, нулевая разность векторных полиномов $p'(t) - p(t)$, равная

$$(s'_0 - s_0) + (s'_1 - s_1)t + \dots + (s'_n - s_n)t^n + s'_{n+1}t^{n+1} = 0$$

на всем непрерывном интервале $t \in [0, 1]$, возможна лишь при $s'_i = s_i \quad \forall i = \overline{0, n}$ и $s'_{n+1} = 0$. Таким образом, связав векторы полиномиальных коэффициентов $S_n = UH_n$ и $S_{n+1} = U'H_{n+1}$ зависимостью $S_{n+1} = [S_n \quad 0]$ и вычислив по (2.38в) невырожденные МВК бинарных разложений H_n и H_{n+1} , получим вектор вершин новой ХП размера $n+1$:

$$U' = [UH_n \quad 0]H_{n+1}^{-1} = UH_n [E_{n+1} \quad 0_{n+1,1}]H_{n+1}^{-1} = UH_n R_{n,n+1}, \quad (2.44a)$$

где $R_{n,n+1} = E_{n+1,n+2}H_{n+1}^{-1}$ — верхний $(n+1) \times (n+2)$ -блок матрицы H_{n+1}^{-1} .

Простота полученного решения несравнима с трудоёмким в выводе и не элементарным в программировании циклическим алгоритмом (2.43). А естественное обобщение (2.44а) на ХП **произвольного** размера $n' > n$

$$U' = [UH_n \quad \underbrace{0 \dots 0}_{n'-n}]H_{n'}^{-1} = UH_n R_{nn'} = UH_n E_{n+1,n'+1}H_{n'}^{-1}, \quad (2.44б)$$

использующее верхний блок $R_{nn'} = E_{n+1,n'+1}H_{n'}^{-1}$ матрицы $H_{n'}^{-1}$ и способное увеличить порядок линии Безье сразу на $n' - n \geq 2$ единиц, вообще нигде, кроме [33], не рассматривалось, хотя можно получить аналогичный (2.43) алгоритм путём умножения (2.38а) на $(t+1-t)^{n'-n}$ и сборки слагаемых с $C_{n'}^i t^i (1-t)^{n'-i}$.

 **Пример 2.4.** Проиллюстрируем работу методов (2.44) увеличения порядка линии Безье без изменения её формы на трёх задачах.

1. Преобразуем по (2.44а) двухзвенную ХП $u_0 u_1 u_2$ в трёхзвенную $u'_0 u'_1 u'_2 u'_3$ (рис. 2.29,а) с помощью матриц H_2 из (2.40б) и $H_3 = H_B$ из (2.36в):

$$H_3^{-1} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1/3 & 2/3 & 1 \\ 0 & 0 & 1/3 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_{23} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1/3 & 2/3 & 1 \\ 0 & 0 & 1/3 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad H_2 R_{23} = \begin{bmatrix} 1 & 1/3 & 0 & 0 \\ 0 & 2/3 & 2/3 & 0 \\ 0 & 0 & 1/3 & 1 \end{bmatrix},$$

$$[u'_0 \ u'_1 \ u'_2 \ u'_3] = [u_0 \ u_1 \ u_2] H_2 R_{23},$$

$$u'_0 = u_0, \quad u'_1 = u_0/3 + 2u_1/3, \quad u'_2 = 2u_1/3 + u_2/3, \quad u'_3 = u_2.$$

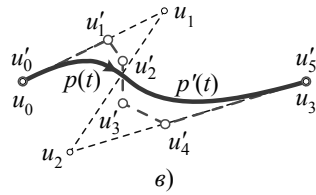
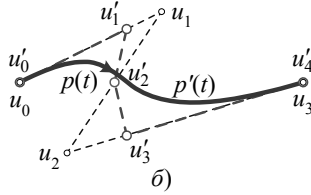
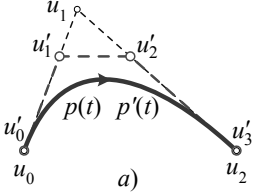


Рис. 2.29

2. Преобразование по (2.44а) трёхзвенной ХП $u_0 u_1 u_2 u_3$ в четырёхзвенную $u'_0 u'_1 u'_2 u'_3 u'_4$ (рис. 2.29,б) выполним с помощью матриц H_3 и H_4 из (2.40в):

$$H_4^{-1} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1/4 & 1/2 & 3/4 & 1 \\ 0 & 0 & 1/6 & 1/2 & 1 \\ 0 & 0 & 0 & 1/4 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_{34} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1/4 & 1/2 & 3/4 & 1 \\ 0 & 0 & 1/6 & 1/2 & 1 \\ 0 & 0 & 0 & 1/4 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad H_3 R_{34} = \begin{bmatrix} 1 & 1/4 & 0 & 0 & 0 \\ 0 & 3/4 & 1/2 & 0 & 0 \\ 0 & 0 & 1/2 & 3/4 & 0 \\ 0 & 0 & 0 & 1/4 & 1 \end{bmatrix},$$

$$[u'_0 \ u'_1 \ u'_2 \ u'_3 \ u'_4] = [u_0 \ u_1 \ u_2 \ u_3] H_3 R_{34},$$

$$u'_0 = u_0, \quad u'_1 = u_0/4 + 3u_1/4, \quad u'_2 = u_1/2 + u_2/2, \quad u'_3 = 3u_2/4 + u_3/4, \quad u'_4 = u_3.$$

Результаты решения обеих задач полностью совпадают с (2.43).

3. Создадим по (2.44б) пятизвенную ХП $u'_0 u'_1 u'_2 u'_3 u'_4 u'_5$ из трёхзвенной $u_0 u_1 u_2 u_3$ (рис. 2.29,в) с помощью матрицы H_5 из (2.40г):

$$H_5^{-1} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0.2 & 0.4 & 0.6 & 0.8 & 1 \\ 0 & 0 & 0.1 & 0.3 & 0.6 & 1 \\ 0 & 0 & 0 & 0.1 & 0.4 & 1 \\ 0 & 0 & 0 & 0 & 0.2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_{35} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0.2 & 0.4 & 0.6 & 0.8 & 1 \\ 0 & 0 & 0.1 & 0.3 & 0.6 & 1 \\ 0 & 0 & 0 & 0.1 & 0.4 & 1 \\ 0 & 0 & 0 & 0 & 0.2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad H_3 R_{35} = \begin{bmatrix} 1 & 0.4 & 0.1 & 0 & 0 & 0 \\ 0 & 0.6 & 0.6 & 0.3 & 0 & 0 \\ 0 & 0 & 0.3 & 0.6 & 0.6 & 0 \\ 0 & 0 & 0 & 0.1 & 0.4 & 1 \end{bmatrix},$$

$$[u'_0 \ u'_1 \ u'_2 \ u'_3 \ u'_4 \ u'_5] = [u_0 \ u_1 \ u_2 \ u_3] H_3 R_{35},$$

$$u'_0 = u_0, \quad u'_1 = \frac{4u_0 + 6u_1}{10}, \quad u'_2 = \frac{u_0 + 6u_1 + 3u_2}{10}, \quad u'_3 = \frac{3u_1 + 6u_2 + u_3}{10}, \quad u'_4 = \frac{6u_2 + 4u_3}{10}, \quad u'_5 = u_4.$$

Так вместо двух преобразований, последовательно выполняемых по (2.43), сразу создаётся шеститочечная характеристическая полилиния с независимым управлением её поведением в конечных точках по (2.41). □

□ Цель **удаления** вершины из полилинии $u_0 u_1 \dots u_{n-1} u_n$, состоящей из $n > 1$ отрезков, — уменьшение её избыточного размера и исключение точек, слабо влияющих на форму линии Безье. Наименьшим влиянием при чётном n обладает средняя точка u_m с номером $m = n/2$ (при нечётном n средних точек две — с номерами $\lfloor n/2 \rfloor$ и $\lceil n/2 \rceil$, см. рис. 2.25,б): её вес согласно (2.39а) не превышает значения $b_{nm}(1/2)$, которое при больших n стремится к нулю. Таким образом, центральные вершины многозвенных ХП можно удалять безболезненно для формы линии Безье, которая в результате этой операции всё же **изменяется**.

Возможность понижения порядка линии Безье *без изменения её формы* определяется **реальной** степенью полинома n' , которая должна быть *меньшей* формальной степени n , задаваемой числом звеньев ХП $u_0 u_1 \dots u_{n-1} u_n$ [33]. Примеры различия этих двух степеней дают совпадающие линии $p'(t)=p(t)$ на каждом из рис. 2.29. Разность формальной и реальной степеней полинома называется **дефектом степени** $d=n-n'$.

Дефект степени полинома равен числу нуль-векторных коэффициентов $s_{n'+1}=\dots=s_n=0$ **в конце вектора** $S_n=UH_n$. Если расположение характеристических точек $u_0 \div u_n$ таково, что дефект степени $d>0$, то **минимальная ХП** $u'_0 u'_1 \dots u'_{n'}$ состоит из $n'=n-d$ звеньев, а $n'+1$ её вершин находятся из уравнения $UH_n=[U'H_{n'} \ O_d]$ с помощью левого блока $L_{nn'}=H_n E_{n+1,n'+1}$ матрицы H_n :

$$U'=UL_{nn'}H_n^{-1}=UH_n E_{n+1,n'+1} H_n^{-1}. \quad (2.45)$$

Отметим полное совпадение этой формулы с (2.44б)!

Пример 2.5. Линия Безье пятого порядка построена на ХП с вершинами $u_0=[10 \ 0]^T$, $u_1=[22 \ 6]^T$, $u_2=[19 \ 12]^T$, $u_3=[11 \ 15]^T$, $u_4=[8 \ 12]^T$ и $u_5=[20 \ 0]^T$ (рис. 2.30). Найти реальную степень полинома и построить ХП минимального порядка.

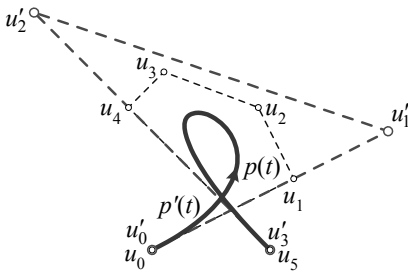


Рис. 2.30

Решение. С помощью матрицы H_5 из (2.40г) вычислим матрицу полиномиальных коэффициентов линии Безье:

$$S_5=UH_5=\begin{bmatrix} 10 & 60 & -150 & 100 & 0 & 0 \\ 0 & 30 & 0 & -30 & 0 & 0 \end{bmatrix}.$$

Два её последних нулевых столбца означают дефект степени полинома $d=2$ и реальную степень $n'=3$. По (2.45) находим характеристические точки **кубической** линии Безье, полностью совпадающей с исходной линией формально пятого порядка:

$$L_{53}=\begin{bmatrix} 1 & -5 & 10 & -10 \\ 0 & 5 & -20 & 30 \\ 0 & 0 & 10 & -30 \\ 0 & 0 & 0 & 10 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad L_{53}H_3^{-1}=\begin{bmatrix} 1 & -2/3 & 1 & -4 \\ 0 & 5/3 & -10/3 & 15 \\ 0 & 0 & 10/3 & -20 \\ 0 & 0 & 0 & 10 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

$$[u'_0 \ u'_1 \ u'_2 \ u'_3]=[u_0 \ u_1 \ u_2 \ u_3 \ u_4 \ u_5]L_{53}H_3^{-1},$$

$$u'_0=u_0, \ u'_1=\frac{-2}{3}u_0+\frac{5}{3}u_1=\begin{bmatrix} 30 \\ 10 \end{bmatrix}, \ u'_2=u_0-\frac{10}{3}u_1+\frac{10}{3}u_2=\begin{bmatrix} 0 \\ 20 \end{bmatrix}, \ u'_3=-4u_0+15u_1-20u_2+10u_3=\begin{bmatrix} 20 \\ 0 \end{bmatrix}=u_5.$$

□

Рациональные линии Безье

Расширение возможностей синтеза линий Безье предоставляет описание точек на плоскости $p=[x \ y] \in \mathbb{R}^2$ либо в пространстве $p=[x \ y \ z] \in \mathbb{R}^3$ (возвра-

щаемся обратно к строчной записи векторов) с помощью так называемых **однородных координат**, в которых точка $p \in R^n$ представляется $n+1$ -вектором

$${}^o p = [\omega p \ \omega] = \begin{cases} [X \ Y \ \omega] & \text{при } n=2, \\ [X \ Y \ Z \ \omega] & \text{при } n=3, \end{cases} \quad (2.46)$$

последний элемент которого $\omega \neq 0$, играющий роль масштабного множителя, связывает остальные элементы с декартовыми координатами точки соотношениями $X=\omega x$, $Y=\omega y$, $Z=\omega z$. Одному декартовому вектору p соответствует бесконечное число однородных векторов ${}^o p = \omega \tilde{p}$ с разными значениями ω .

Назовём операцию

$$x=X/\omega, \quad y=Y/\omega, \quad z=Z/\omega \quad (2.47a)$$

приведением однородного вектора к декартовой форме и впредь будем её записывать с помощью знака « $\sim$ » **эквивалентности векторов**

$$\tilde{p} \sim {}^o p \quad \text{или} \quad [p \ 1] \sim [\omega p \ \omega]. \quad (2.47б)$$

Эквивалентность векторов означает, что они имеют равные соответствующие декартовы координаты. Задание масштаба $\omega \neq 0$ позволяет без арифметических операций изменить сразу все декартовы координаты в $1/\omega$ раз.

Возможны два способа выполнения операций с однородными векторами:

- операции проводятся по одной, а полученные однородные векторы сразу переводятся по (2.47) в декартову форму;
- все операции выполняются непрерывно друг за другом по соответствующим формулам, после чего результат цепочки преобразований приводится к декартовой форме.

В основе построения рациональных линий лежит формальное правило по координатного сложения двух однородных векторов

$${}^o p_1 + {}^o p_2 = [\omega_1 p_1 + \omega_2 p_2 \ \omega_1 + \omega_2] \sim \left[\frac{\omega_1}{\omega_1 + \omega_2} p_1 + \frac{\omega_2}{\omega_1 + \omega_2} p_2 \ 1 \right].$$

Записав модель линии Безье (2.38) в однородном пространстве

$${}^o p(t) = \sum_{i=0}^n b_{ni}(t) {}^o u_i = {}^o U B_n(t) \quad \forall t \in [0,1], \quad (2.48a)$$

где каждой характеристической вершине u_i соответствует однородный вектор ${}^o u_i = [\omega_i u_i \ \omega_i] = \omega_i \tilde{u}_i$ с заданным **весом** (значимостью) ω_i , получим декартову форму **рациональной линии Безье** n -го порядка

$$p(t) = \sum_{i=0}^n b_{ni}(t) \omega_i u_i / \sum_{i=0}^n b_{ni}(t) \omega_i \quad \forall t \in [0,1]. \quad (2.48б)$$

Экономичнее вычислять точки $p(t)$ не по прямой формуле (2.48б) с двумя суммами и двукратным вычислением каждой функции Безье $b_{ni}(t)$, а по матричному варианту (2.48a) с помощью вектора однородных вершин ХП

$$\overset{\circ}{U} \equiv [\overset{\circ}{u}_0 \ \overset{\circ}{u}_1 \ \dots \ \overset{\circ}{u}_n] = [\tilde{u}_0 \ \tilde{u}_1 \ \dots \ \tilde{u}_n] \bullet [\omega_0 \ \omega_1 \ \dots \ \omega_n] = \tilde{U} \bullet \Omega,$$

получаемого декартовым произведением (поэлементным перемножением) вектора $\tilde{U}$ расширенных координат вершин на весовой вектор Ω . Далее однородный вектор $\overset{\circ}{p}(t)$ делится на произведение строчного вектора Ω на столбцовый вектор $B_n(t)$, равное знаменателю дроби в (2.48б):

$$p(t) = (\tilde{U} \bullet \Omega) B_n(t) / \Omega B_n(t) \quad \forall t \in [0, 1]. \quad (2.48в)$$

Рациональная линия Безье имеет следующие **граничные свойства** [20, 56]:

- фиксирована в крайних характеристических узлах:

$$p(0) = \frac{b_{n0}(0)\omega_0 u_0}{b_{n0}(0)\omega_0} = u_0, \quad p(1) = \frac{b_{nn}(1)\omega_n u_n}{b_{nn}(1)\omega_n} = u_n; \quad (2.49а)$$

- краевые направления задаются крайними отрезками ХП $u_0 u_1$ и $u_{n-1} u_n$:

$$V_0 = p'(0) = n \frac{\omega_1}{\omega_0} (u_1 - u_0), \quad V_n = p'(1) = n \frac{\omega_{n-1}}{\omega_n} (u_n - u_{n-1}); \quad (2.49б)$$

- краевые ускорения задают крайние тройки вершин $u_0 u_1 u_2$ и $u_{n-2} u_{n-1} u_n$:

$$A_0 = p''(0) = 2n \frac{\omega_1}{\omega_0} \left(n \frac{\omega_1}{\omega_0} - 1 \right) (u_0 - u_1) + n(n-1) \frac{\omega_2}{\omega_0} (u_2 - u_0), \quad (2.49в)$$

$$A_n = p''(1) = 2n \frac{\omega_{n-1}}{\omega_n} \left(n \frac{\omega_{n-1}}{\omega_n} - 1 \right) (u_n - u_{n-1}) + n(n-1) \frac{\omega_{n-2}}{\omega_n} (u_{n-2} - u_n).$$

Нетрудно проверить, что при равных весах $\omega_0 = \omega_1 = \omega_2$ и $\omega_{n-2} = \omega_{n-1} = \omega_n$ эти свойства совпадают с (2.39в-д).

При всех равных весах $\omega_i = \omega_0 \ \forall i = \overline{1, n}$ дробно-рациональная функция (2.48б) в силу свойства (2.39б) превращается в (2.38а). Следовательно, **полиномиальные линии Безье являются частным, а именно, равновесным случаем рациональных линий Безье**. Благодаря варьированию весами вершин ХП рациональные линии расширяют возможности полиномиальных, но и затрудняют процесс настройки исходных данных для получения желаемых свойств линии.

Форма рациональной линии зависит не от абсолютных весов, а от их отношений $\omega_0 : \omega_1 : \dots : \omega_n$, что, кстати, видно из (2.49б,в). Увеличение веса ω_k относительно других весов ведёт к приближению точек линии к характеристической вершине u_k . Для пояснения этого свойства сравним близость к u_k двух точек рациональных линий — эталонной $p(t)$ и тестовой $p_m(t)$, у которой вес k -й вершины ХП равен $m\omega_k$ при $m > 1$ (рис. 2.31,а). Обозначив дробь (2.48б) как $\mu(t)/\nu(t)$, запишем тестовую рациональную функцию в виде

$$p_m(t) = \frac{\mu(t) + (m-1)b_{nk}(t)\omega_k u_k}{\nu(t) + (m-1)b_{nk}(t)\omega_k} = \frac{p(t) + \gamma_k(t)u_k}{1 + \gamma_k(t)}, \quad \text{где } \gamma_k(t) = \frac{(m-1)b_{nk}(t)\omega_k}{\nu(t)}.$$

Очевидно, что $p_1(t) = p(t)$, а при $m \rightarrow \infty$ и $b_{nk}(t) \neq 0$ точка $p_\infty(t)$ сходится к вершине u_k . При $m > 1$ и $\gamma_k(t) > 0$ точка $p_m(t)$ лежит на отрезке между точками $p(t)$ и u_k , деля его в отношении $\gamma_k(t) : 1$: $p_m(t) - p(t) = \gamma_k(t)(u_k - p_m(t))$.

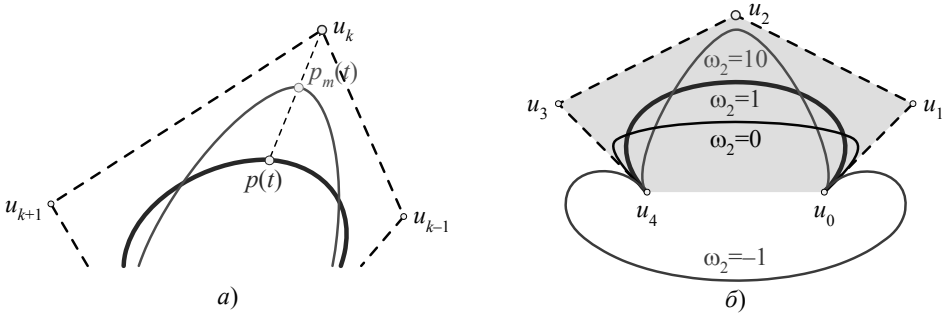


Рис. 2.31

Вес любой вершины ХП может быть положительным (тогда она «притягивает» к себе точки линии), отрицательным (тогда она их «отталкивает») и нулевым (тогда она не влияет на форму рациональной линии). Это свойство проиллюстрировано на рис. 2.31, б, где на одной и той же ХП построено четыре линии с разными весами вершины u_2 . Следствием отрицательности любого из весов является **возможность выхода рациональной линии Безье из выпуклой оболочки характеристической полилинии** (см. линию с весом $\omega_2 = -1$) из-за невыполнения всех условий (2.35), в которых некоторые из коэффициентов $\alpha_i = b_{ni}(t)\omega_i/v(t)$ могут стать отрицательными. **При неотрицательности всех весов $\omega_0 \div \omega_n$ все коэффициенты $\alpha_i \geq 0$ и тогда рациональная линия лежит в выпуклой оболочке вершин своей характеристической полилинии.**

К ХП рациональной линии n -го порядка применимы оба алгоритма редактирования — (2.44) добавления $n' - n > 0$ новых и (2.45) удаления $n - n' > 0$ лишних вершин. Эти операции можно выполнить по единой формуле

$$\overset{\circ}{U}' = \overset{\circ}{U} H_n E_{n+1, n'+1} H_{n'}^{-1}, \quad (2.50)$$

правильно работающей при любых соотношениях между старым n и новым n' порядками характеристических полилиний.

Самым замечательным свойством рациональных линий является их способность **точно** воспроизводить все квадратичные линии — дуги окружностей, эллипсов, гипербол и парабол. Этим данный класс линий отличается от полиномиальных линий Безье, моделирующих квадратики, отличные от параболы, лишь приближённо (см. рис. 2.27). На мысль о возможности дробно-рационального представления линий второго порядка наводят способы параметризации двух неявных канонических уравнений $f(x, y) = 0$ из табл. 1.1:

- эллипса с $f(x, y) = (x/a)^2 + (y/b)^2 - 1$ и функциями подстановки

$$x(t) = a(1-t^2)/(1+t^2), \quad y(t) = 2bt/(1+t^2);$$

- гиперболы с $f(x, y) = (x/a)^2 - (y/b)^2 - 1$ и функциями подстановки

$$x(t) = a(1+t^2)/(1-t^2), \quad y(t) = 2bt/(1-t^2).$$

Во многих источниках, например, [13, 20, 51, 56, 58, 63] утверждается, что дуги **всех** квадратичных линий могут быть описаны рациональными по-

линомами второго порядка на основе **трёхточечной** ХП $u_0u_1u_2$ с весами крайних вершин $\omega_0=\omega_1=1$ и переменным весом ω средней вершины u_1 :

$$p(t) = \frac{(1-t)^2 u_0 + 2t(1-t)\omega u_1 + t^2 u_2}{(1-t)^2 + 2t(1-t)\omega + t^2} \quad \forall t \in [0,1]. \quad (2.51a)$$

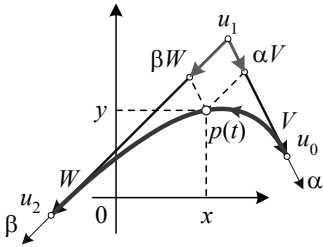


Рис. 2.32

В косоугольной системе координат $\{u_1, V, W\}$ с центром в точке u_1 и базисными векторами $V = u_0 - u_1$ и $W = u_2 - u_1$ произвольная точка с координатами $\{\alpha, \beta\}$ имеет разложение $p = u_1 + \alpha V + \beta W$ (рис. 2.32). Подставив в (2.51a) косоугольное представление вершин $u_0 = [1 \ 0]$, $u_1 = [0 \ 0]$ и $u_2 = [0 \ 1]$, получим координатные функции рациональной линии $p(t) = [\alpha(t) \ \beta(t)]$

$$\alpha(t) = \frac{(1-t)^2}{(1-t)^2 + 2t(1-t)\omega + t^2}, \quad \beta(t) = \frac{t^2}{(1-t)^2 + 2t(1-t)\omega + t^2} \quad \forall t \in [0,1], \quad (2.51б)$$

удовлетворяющие (проверьте!) неявному квадратичному уравнению

$$\alpha^2 + \beta^2 + 2(1-2\omega^2)\alpha\beta - 2\alpha - 2\beta + 1 = 0. \quad (2.51в)$$

Поскольку косоугольные координаты $\{\alpha, \beta\}$ связаны с декартовыми $\{x, y\}$ **линейным** преобразованием (1.43в)

$$[\alpha \ \beta] = (p - u_1) \begin{bmatrix} V \\ W \end{bmatrix}^{-1} = ([x \ y] - u_1) \begin{bmatrix} u_0 - u_1 \\ u_2 - u_1 \end{bmatrix}^{-1}, \quad (2.52)$$

то и в декартовой системе координат линия (2.51a) также будет квадратичной.

Вычислив по (1.58) инварианты и полуинвариант квадрики (2.51в) $I_1 = 2 > 0$, $I_2 = 4\omega^2(1-\omega^2)$, $I_3 = -4\omega^4 \leq 0$ и $I'_2 = 0$, классифицируем по табл. 1.1 тип этой линии в зависимости от весового коэффициента ω (рис. 2.33):

- $\omega = 0$ — исчезающее влияние вершины u_1 на форму линии $p(t)$ вырождает её в отрезок **прямой** u_0u_1 с переменной скоростью движения $p'(t)$;

- $0 < |\omega| < 1$ — при $I_2 \neq 0$ и $I_1 I_3 < 0$ по (2.51a) строятся дуги **эллипса**, лежащие внутри треугольника $\Delta = u_0u_1u_2$ при $0 < \omega < 1$ и вне его при $-1 < \omega < 0$. Две дуги с равномодульными весами $\pm\omega$ образуют полный эллипс;

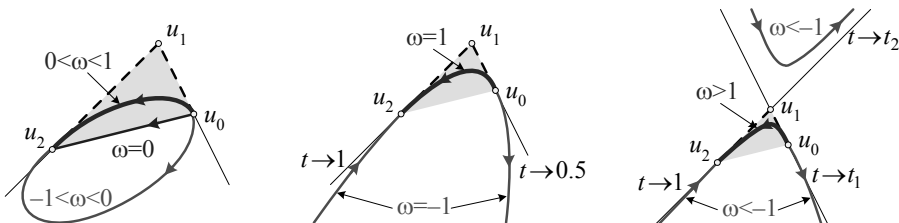


Рис. 2.33

• $|\omega|=1$ — инварианты $I_2=0$ и $I_3 \neq 0$ соответствуют **параболе**. При $\omega=1$ знаменатель дроби в (2.51а) $v(t)=1+2(\omega-1)t(1-t)$ становится равным единице, а линия превращается в полиномиальную дугу $(1-t)^2 u_0 + 2t(1-t)u_1 + t^2 u_2$ внутри треугольника Δ . При $\omega=-1$ значение знаменателя $v(0.5)=0$ разрывает параболу на две полубесконечные ветви вне Δ , соответствующие $t \in [0, 0.5]$ и $t \in (0.5, 1]$;

• $|\omega|>1$ — значения $I_2 < 0$ и $I_3 \neq 0$ соответствуют **гиперболе**. При $\omega>1$ дуга одной из её ветвей вписана в треугольник Δ . Немного сложнее всё обстоит при весе $\omega<-1$, обнуляющем знаменатель $v(t)$ при $t_{1,2}=0.5(1 \pm \sqrt{(\omega+1)/(\omega-1)}) \in (0, 1)$. В результате по (2.51а) строятся три неограниченные ветви гиперболы вне треугольника Δ в интервалах изменения параметра $t \in [0, t_1)$, $t \in (t_1, t_2)$ и $t \in (t_2, 1]$.

Рассмотрим задачу проведения рациональной линии Безье второго порядка через заданную точку $p_0 = [x_0 \ y_0]$ (рис. 2.34). Анализируя по рис. 2.33 области возможного расположения квадратичных линий и соответствующие им значения ω , обнаруживаем следующие закономерности:

- все линии сосредоточены в положительном и (гиперболы) отрицательном направлениях створа угла $\angle u_0 u_1 u_2$ с **однозначными** координатами α и β . В двух областях с $\alpha\beta < 0$ решений не существует;

- линиям внутри треугольника Δ соответствуют $\omega>0$, $\alpha>0$, $\beta>0$ и $\alpha+\beta<1$;

- на отрезке $u_0 u_2$ вес $\omega=0$, а косоугольные координаты удовлетворяют условиям $\alpha>0$, $\beta>0$ и $\alpha+\beta=1$;

- в положительной области створа угла, находящейся вне треугольника, имеют место неравенства $\omega<0$, $\alpha>0$, $\beta>0$ и $\alpha+\beta>1$;

- в отрицательной области створа работают неравенства $\omega<0$, $\alpha<0$ и $\beta<0$.

Исходя из декартовых координат точки x_0 и y_0 , вычислим по (2.52) её косоугольные координаты α_0 и β_0 . При $\alpha_0 \beta_0 > 0$ значение веса ω находится решением уравнения (2.51в) и с учётом проведённого выше анализа равно

$$\omega = \text{sgn}(\alpha_0)(1 - \alpha_0 - \beta_0) / 2\sqrt{\alpha_0 \beta_0}. \quad (2.53)$$

Теперь, руководствуясь рис. 2.33, можно строить рациональную линию (2.51а) в виде ограниченной дуги при $\omega>0$ или $-1<\omega<0$, отрезка при $\omega=0$, двух ветвей параболы при $\omega=-1$ либо трёх ветвей гиперболы при $\omega<-1$. Уходящие в бесконечность ветви параболы или гиперболы изображаются ограниченными дугами. При $\alpha_0 \beta_0 \leq 0$ нет ни одной квадратичной линии, проходящей через точку p_0 .

В компьютерной графике востребованы разнообразные **дуги окружностей**. В отличие от полиномиальных рациональные линии Безье могут воспроизвести их **точно**. Второй порядок этих линий, строящихся по (2.51а) на трёхузловой ХП $u_0 u_1 u_2$, позволяет создать дугу окружности с любым углом раствора, **кроме**

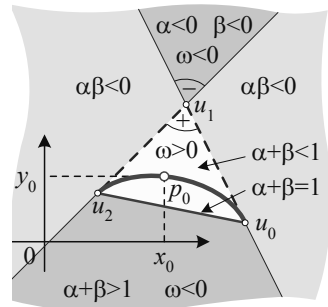


Рис. 2.34

180° и 360°. Исключения связаны с тем, что непараллельные отрезки u_1u_0 и u_1u_2 не могут быть касательными к половине или целой окружности.

Вписывание в угол $\angle u_0u_1u_2 = 2\gamma \in (0, \pi)$ **внутренней** дуги окружности с углом раствора $2\delta = \pi - 2\gamma \in (0, \pi)$ либо **внешней** дуги с углом $2\delta = \pi + 2\gamma \in (\pi, 2\pi)$ возможно лишь при **равенстве длин касательных сторон** $|u_0u_1| = |u_1u_2| = d$

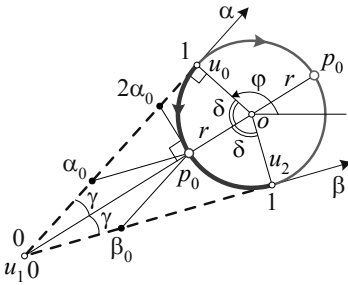


Рис. 2.35

(рис. 2.35), иначе по (2.51a) будут построены дуги эллипса, а не окружности. Определим косоугольные координаты $\alpha_0 = \beta_0$ **средней** точки дуги $p_0 = p(0.5)$, равные друг другу благодаря симметрии ХП и дуги относительно прямой, соединяющей точку u_1 и центр окружности o . Радиус окружности, несущей обе дуги, есть $r = d \cdot \operatorname{tg}(\gamma)$. Из равенств двух расстояний $|u_1o| = |u_1p_0| + r$ для внутренней и $|u_1o| = |u_1p_0| - r$ для внешней вписанных дуг составим уравнение

$$d/\cos(\gamma) = 2d\alpha_0 \cos(\gamma) \pm d \operatorname{tg}(\gamma), \quad (2.54a)$$

из которого находим координату α_0 , а по (2.53) весовой коэффициент ω :

$$\alpha_0 = \frac{1 \mp \sin(\gamma)}{2 \cos^2(\gamma)} \Rightarrow \omega = \frac{1 - 2\alpha_0}{2\alpha_0} = \pm \sin(\gamma) = \cos(\delta). \quad (2.54б)$$

В частности, дуги $1/4$ и $3/4$ окружности с углами раствора $2\delta = \pi/2$ и $2\delta = 3\pi/2$ строятся по (2.51a) на ортогональной равносторонней ХП $u_0u_1u_2$ с весами средней вершины $\omega = 1/\sqrt{2}$ и $\omega = -1/\sqrt{2}$ (рис. 2.36,a).

В (2.54a,б) у двойных знаков « $\pm$ » и « $\pm \mp$ » верхний соответствует внутренней, а нижний — внешней дугам окружности. Отметим, что формула $\omega = \cos(\delta)$ для угла раствора дуги $2\delta > \pi$ даёт правильное отрицательное значение веса.

Обратная задача нахождения характеристических точек $u_0 \div u_2$, соответствующих дуге окружности с центром в точке o , радиусом r , начальным углом φ и углом раствора $2\delta \notin \{\pm\pi, \pm 2\pi\}$, решается достаточно просто:

$$\begin{aligned} u_0 &= o + r[\cos(\varphi) \sin(\varphi)], \quad u_1 = o + r \sec(\delta)[\cos(\varphi + \delta) \sin(\varphi + \delta)], \\ u_2 &= o + r[\cos(\varphi + 2\delta) \sin(\varphi + 2\delta)]. \end{aligned} \quad (2.54в)$$

Рациональные линии, воспроизводящие дуги окружности с углами раствора 180° и 360° , должны иметь порядки $n > 2$ для того, чтобы их ХП с параллельными рёбрами u_0u_1 и u_nu_{n-1} были выпуклыми. Исходя из симметрии этих линий, зададим для получения **полуокружности** вершины ХП $u_0 = [1 \ 0]$, $u_1 = [1 \ y_1]$, $u_2 = [-1 \ y_1]$ и $u_3 = [-1 \ 0]$ с весами $\omega_0 = \omega_3 = 1$ и $\omega_1 = \omega_2$ (рис. 2.36,б), а для полной **окружности** — вершины $u_0 = u_5 = [1 \ 0]$, $u_1 = [1 \ y_1]$, $u_2 = [x_2 \ y_1]$, $u_3 = [x_2 \ -y_1]$ и $u_4 = [1 \ -y_1]$ с весами $\omega_0 = \omega_5 = 1$, $\omega_1 = \omega_4$ и $\omega_2 = \omega_3$ (рис. 2.36,в). Незвестные координаты y_1, x_2 и веса ω_1, ω_2 находятся из двух условий: прохождения

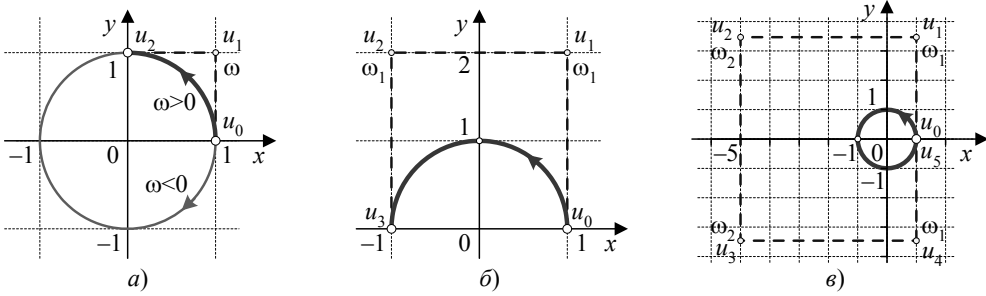


Рис. 2.36

медианных точек $p(0.5)$ через середины дуг полуокружности $[0\ 1]$ и полной окружности $[-1\ 0]$ и удовлетворения элементов дробно-рационального вектора $p(t)=[x(t)\ y(t)]$ неявному уравнению окружности $x^2(t)+y^2(t)=1$.

Читателю предоставляется возможность самостоятельно решить эти уравнения и получить рациональные модели (2.48б):

- полуокружности порядка $n=3$ с координатой $y_1=2$ и весом $\omega_1=1/3$;
- полной окружности порядка $n=5$ с координатами $y_1=2\sqrt{3}$, $x_2=-5$ и весами $\omega_1=0.2$ и $\omega_2=0.1$.

Рассмотрим практически не затронутый в источниках вопрос о **геометрических преобразованиях** рациональной линии и её характеристической полилинии. Влияние линейных преобразований на неявные функции линий и поверхностей изучалось ранее в п.п. 1.1.9, 1.5.11.

В (2.51а,б) представлены параметрические модели квадратичной линии $p(t)$ в разных координатах $\{x(t), y(t)\}$ и $\{\alpha(t), \beta(t)\}$, связанных уравнением (2.52):

$$[x\ y] = [\alpha\ \beta]A + B, \text{ где } A = \begin{bmatrix} u_0 - u_1 \\ u_2 - u_1 \end{bmatrix} \in \mathbb{R}^{2 \times 2}, \quad B = u_1 \in \mathbb{R}^2. \quad (2.55)$$

Матрица A отвечает за центрированные преобразования (масштабирование, сдвиг, вращение) системы координат $\alpha\beta$, а вектор B — за перенос её начала.

Геометрические преобразования могут выполняться также и с самим графическим объектом в неизменной СК. Допустим, некоторая рациональная линия $p(t)$ и её ХП с вершинами $u_0 \div u_n$ подвергаются масштабным, вращательным и переносным преобразованиям. Вся цепочка преобразований аккумулирована в матрице A и векторе B , по которым вычисляются конечные положения точек $u'_i = u_i A + B \quad \forall i = \overline{0, n}$ и построенная на них по (2.48б) рациональная линия

$$p'(t) = \frac{\sum_{i=0}^n b_{ni}(t) \omega_i u'_i}{\sum_{i=0}^n b_{ni}(t) \omega_i} = \frac{\sum_{i=0}^n b_{ni}(t) \omega_i u_i A + \sum_{i=0}^n b_{ni}(t) \omega_i B}{\sum_{i=0}^n b_{ni}(t) \omega_i} = p(t)A + B.$$

Отсюда следует, что характеристическая сеть U и её рациональная линия $p(t)$ преобразуются **одинаковыми** элементами $\{A, B\}$ с **неизменяемыми** весами характеристических вершин $\omega_0 \div \omega_n$.

Исследуем эти свойства на примерах. Возьмём **прямой** угол $x^0 O_2 y^0$ с единичными весами крайних вершин и вычисленным по (2.54б) весом средней вершины $\omega = \sin(45^\circ) = 1/\sqrt{2}$ (рис. 2.37,а). Построим на этой ХП по (2.51а) дугу $1/4$ **окружности** $o(t) = [\alpha(t) \beta(t)]$ с координатными функциями (2.51б), удовлетворяющими неявному уравнению $(\alpha-1)^2 + (\beta-1)^2 - 1 = 0$. Подвергнув эту линию масштабному искажению по оси x с коэффициентом m_x , получим дугу $1/4$ **эллипса** $e(t) = [x(t) y(t)]$ с координатами $x(t) = m_x \alpha(t)$ и $y(t) = \beta(t)$. Такая же дуга строится на масштабированной ХП $u_0 O_2 y^0$ с вершиной $u_0 = m_x x^0$:

$$x(t) = \frac{m_x (1-t)^2}{(1-t)^2 + \sqrt{2}t(1-t) + t^2}, \quad y(t) = \frac{t^2}{(1-t)^2 + \sqrt{2}t(1-t) + t^2}.$$

Поскольку вращение и перенос не изменяют форму графического объекта, то в **равные стороны произвольного прямого угла** $u_0 u_1 u_2$ **вписывается дуга окружности**, а при **разных длинах сторон** $|u_0 u_1| \neq |u_1 u_2|$ — дуга эллипса. Для построения этих линий можно использовать как готовую формулу (2.51а) с весом $\omega = 1/\sqrt{2}$, так и преобразование (2.55) дуги $o(t)$.

Изучим теперь другое преобразование (рис. 2.37,б) с матрицей

$$A = \begin{bmatrix} x^0 \\ u_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \cos(\gamma) & \sin(\gamma) \end{bmatrix}$$

и вектором $B = O_2$, переводящее вершину y^0 в точку $u_2 = [\cos(\gamma) \sin(\gamma)]$ с углом $\gamma \neq 90^\circ$, а дугу окружности $o(t)$ в линию $e(t) = o(t)A + B$ с координатами

$$x(t) = \frac{(1-t)^2 + \cos(\gamma)t^2}{(1-t)^2 + \sqrt{2}t(1-t) + t^2}, \quad y(t) = \frac{\sin(\gamma)t^2}{(1-t)^2 + \sqrt{2}t(1-t) + t^2}.$$

Она является дугой **эллипса**, т. к. построена хоть и на равносторонней ХП $x^0 O_2 u_2$, но с весом средней вершины $\omega = 1/\sqrt{2} \neq \sin(\gamma/2)$. В общем случае в **произвольный угол** $u_0 u_1 u_2$ с весом средней вершины $\omega = \sin(\gamma/2)$ по (2.51а) **вписывается дуга эллипса, трансформированная из дуги окружности, вписанной в такой же образом трансформируемую ХП с углом раствора γ** .

Используем данное свойство для решения одной задачи построения **пространственной рациональной линии** второго порядка (рис. 2.37,в). Вообразим двухгранный угол Γ , образованный движением полилинии $u_0 u_1 u_2$ вдоль век-

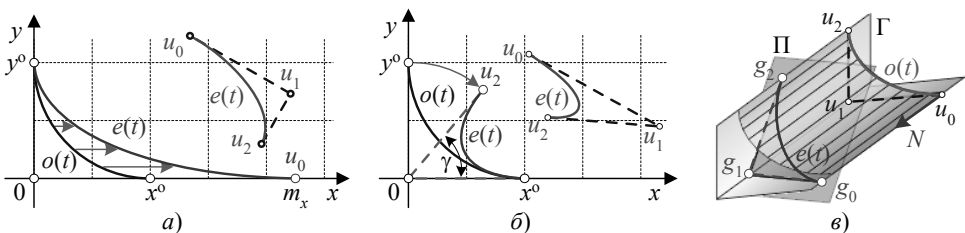


Рис. 2.37

тора N . Движение дуги $o(t)$, построенной по (2.51а) на ХП $u_0u_1u_2$ с весом ω , создаёт цилиндрический сегмент $c(t, \tau) = o(t) + N\tau$. Пусть некоторая плоскость Π пересекает поверхность Γ по полилинии $g_0g_1g_2$. Изученные закономерности означают, что дуга эллипса $e(t)$, построенная на ХП $g_0g_1g_2$ с тем же весом ω , является линией пересечения поверхностей Π и $c(t, \tau)$. Эта же линия получается цепочкой преобразований $e(t) = (o(t) - u_1)A_o^{-1}A_e + g_1$ с матрицами

$$A_o = \begin{bmatrix} u_0 - u_1 \\ u_2 - u_1 \\ N \end{bmatrix} \in \mathbb{R}^{3 \times 3}, \quad A_e = \begin{bmatrix} g_0 - g_1 \\ g_2 - g_1 \\ N \end{bmatrix} \in \mathbb{R}^{3 \times 3}. \quad (2.56)$$

При равных длинах векторов $V = u_0 - u_1$ и $W = u_2 - u_1$, ортогональных вектору $N = V \times W$, и задании веса $\omega = \sin(0.5 \arccos(\bar{V} \circ \bar{W}))$ движение дуги **окружности** $o(t)$ создаёт сегмент поверхности, **скругляющей** двугранный угол.

2.1.4. Конгруэнтные преобразования моделей линий

Многообразие рассмотренных моделей полиномиальных линий обладает как общими чертами — одинаковыми формами разложений (2.7) и объёмом исходных данных, определяющих полиномы **равных** степеней n , — так и различиями, важнейшими из которых являются [43]:

- **параметрические шкалы** — **единичная** $t \in [0, 1]$ у полиномиальной (2.38а) и рациональной (2.48) моделей линий Безье; **нормализованные** $t \in [0, n]$ у интерполяционного и $t \in [0, n-2]$ у эрмитова полиномов; **произвольная** $t \in [a, b]$;
- состав исходных данных — вектор узловых точек $U \in \mathbb{V}^{n+1}$ у интерполяционного полинома, векторы узлов $U \in \mathbb{V}^{n-1}$ и конечных направлений $\{V_0, V_n\}$ у полинома Эрмита, вектор $U \in \mathbb{V}^{n+1}$ вершин характеристической полилинии у линий Безье плюс вектор $\Omega \in \mathbb{R}^{n+1}$ весов этих вершин у рациональной линии.

Рассмотрим задачу **конгруэнтного преобразования**

$$p_n(t_1) = U_1 H_1 T_n(t_1) \Rightarrow p_n(t_2) = U_2 H_2 T_n(t_2) \quad (2.57)$$

одной модели линии в другую **с сохранением её формы** и степени n обоих полиномов. Ключ к преобразованию — приведение параметров t_1 и t_2 к единому параметру t , а обеих моделей — к единому ВБФ $T_n(t) = [1 \ t \ \dots \ t^n]^T$. Составим **невыврожденную** матрицу преобразования $T_n(\tau) = D_n(m, s)T_n(t)$ в шкалу параметра t из шкалы масштабированного и смещённого параметра $\tau = mt + s$

$$\begin{bmatrix} 1 \\ mt+s \\ (mt+s)^2 \\ \vdots \\ (mt+s)^n \end{bmatrix} = D_n(m, s) \begin{bmatrix} 1 \\ t \\ t^2 \\ \vdots \\ t^n \end{bmatrix} \Rightarrow D_n(m, s) = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ s & m & 0 & \dots & 0 \\ s^2 & 2ms & m^2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ s^n & C_n^1 m s^{n-1} & C_n^2 m^2 s^{n-2} & \dots & m^n \end{bmatrix} \quad (2.58a)$$

с элементами $d_{ij} = \text{if}(i \geq j: C_i^j m^j s^{i-j}, 0) \ \forall i, j \in [0, n]$ и следующими свойствами:

- матрица $D_n(1,0)=E_{n+1}$ — единичная;
- матрица обратного преобразования шкалы $t=(\tau-s)/m=\tau m^{-1}-sm^{-1}$ равна

$$D_n(m,s)^{-1}=D_n(m^{-1},-sm^{-1}); \quad (2.58б)$$

- цепочка преобразований шкал $t_1=m_1t+s_1$ и $t_2=m_2t_1+s_2=m_1m_2t+m_2s_1+s_2$ выполняется произведением матриц

$$D_n(m_2,s_2)D_n(m_1,s_1)=D_n(m_1m_2,m_2s_1+s_2). \quad (2.58в)$$

Подстановкой в (2.58а) нужных аргументов формируются матрицы M_1 и M_2 преобразований ВБФ $T_n(t_1)=M_1T_n(t)$ и $T_n(t_2)=M_2T_n(t)$ в единичную шкалу $t \in [0,1]$. Совмещая равношкальные модели $p_n(t)$, получим уравнение, решением которого является узловой вектор конгруэнтной модели:

$$U_1H_1M_1=U_2H_2M_2 \Rightarrow U_2=U_1H_1MH_2^{-1}, \text{ где } M=M_1M_2^{-1}. \quad (2.59)$$

Применим метод (2.59) в расчёте моделей полиномиальных линий.

1. В задаче конгруэнтного **преобразования параметрической шкалы** $t_1 \in [a_1, b_1] \Rightarrow t_2 \in [a_2, b_2]$ из соотношений $t_i=(b_i-a_i)t+a_i$ следуют матрицы $M_i=D_n(b_i-a_i, a_i) \quad \forall i=\overline{1,2}$. Тогда при **неизменном** векторе U в (2.59) получаем

$$H_2=H_1M_1M_2^{-1}=H_1M, \text{ где } M=D_n(b_1-a_1, a_1)D_n(b_2-a_2, a_2)^{-1}. \quad (2.60а)$$

К примеру, в (2.13) приведён алгоритм расчёта матрицы H_1 бинарного разложения интерполяционного полинома $p_n(t_1)=UH_1T_n(t_1)$ в нормализованной шкале $t_1=nt \in [0, n]$. Перейдя в единичную шкалу $t_2 \equiv t \in [0, 1]$ с помощью матрицы $M=D_n(n,0)D_n(1,0)^{-1}=D_n(n,0)$, получим модель $p_n(t)=UH_2T_n(t)$ с матрицей $H_2=H_1D_n(n,0)$. Возьмём нормализованную параболу

$$p_n(t_1)=(1-1.5t_1+0.5t_1^2)\mu_0+(2t_1-t_1^2)\mu_1+(-0.5t_1+0.5t_1^2)\mu_2 \quad \forall t_1 \in [0, 2],$$

проходящую через три точки $u_0=p_2(0)$, $u_1=p_2(1)$ и $u_2=p_2(2)$. Её матрица H_1 приведена в (2.166). Нормировка шкалы даёт следующие матрицу и модель параболы $p_2(t)$:

$$H_2=H_1D_2(2,0)=\begin{bmatrix} 1 & -1.5 & 0.5 \\ 0 & 2 & -1 \\ 0 & -0.5 & 0.5 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 4 \end{bmatrix}=\begin{bmatrix} 1 & -3 & 2 \\ 0 & 4 & -4 \\ 0 & -1 & 2 \end{bmatrix},$$

$$p_2(t)=(1-3t+2t^2)\mu_0+(4t-4t^2)\mu_1+(-t+2t^2)\mu_2 \quad \forall t \in [0, 1].$$

Проверка $p_2(0)=u_0$, $p_2(0.5)=u_1$ и $p_2(1)=u_2$ убеждает, что после нормировочно-го **сжатия** параметрической шкалы t_1 в два раза полином $p_2(t)$ остался интерполяционным с тем же графиком, что и у модели $p_2(t_1)$.

Может быть полезным и **растяжение** шкалы, например, для перевода единичной шкалы $t \in [0, 1]$ линии Безье (2.38а) в интервал $t_2=nt \in [0, n]$, пропорциональный её порядку n . Матрица конгруэнтного преобразования шкалы равна

$$M=D_n(1,0)D_n(n,0)^{-1}=D_n(n^{-1}, 0).$$

Исходя из вычисляемой по (2.38в) матрицы H_1 , получим по (2.60а) матрицу $H_2 = H_1 D_n(n^{-1}, 0)$ функции $p_n(t_2) = UH_2 T_n(t_2)$. Есть, скажем, парабола Безье $p_2(t) = (1-t)^2 u_0 + 2t(1-t)u_1 + t^2 u_2 \forall t \in [0, 1]$ с концевыми точками $u_0 = p_2(0)$, $u_2 = p_2(1)$ и матрицей H_1 , взятой из (2.40б). В растянутой шкале $t_2 = 2t \in [0, 2]$ матрица

$$H_2 = H_1 D_2(0.5, 0) = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 2 & -2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.25 \end{bmatrix} = \begin{bmatrix} 1 & -1 & 0.25 \\ 0 & 1 & -0.5 \\ 0 & 0 & 0.25 \end{bmatrix}$$

нормализованной параболы Безье $p_2(t_2) = (1-t_2+0.25t_2^2)u_0 + (t_2-0.5t_2^2)u_1 + 0.25t_2^2 u_2$ даёт те же концевые точки $p_2(0) = u_0$ и $p_2(2) = u_2$.

2. Теперь рассмотрим ряд задач **преобразования типа** полиномиальной линии. Эта операция полезна в редакторе линий для отображения одновременно двух полилиний U_1 и U_2 , связанных первым уравнением в (2.59). Известно, например, что интерполяционный полином высокой степени обладает большими межузловыми осцилляциями, далеко выходящими из области расположения вершин узловой сети (см. рис. 2.7). В то же время линия Безье целиком лежит в выпуклой оболочке вершин своей ХП (см. рис. 2.25,а). Вывод на экран области гарантированного расположения интерполяционной линии позволит сделать её интерактивное редактирование более осмысленным.

□ Переходы между **интерполяционной** моделью $p_n(t_n) = U_n H_n T_n(t_n)$ в шкале $t_n = nt \in [0, n]$ с вектором $U_n = [u_0^n \ u_1^n \ \dots \ u_n^n]$, матрицей $H_n = Q_n^{-1}$ из (2.13д) и **эрмитовой** моделью $p_n(t_\exists) = U_\exists H_\exists T_n(t_\exists)$ в другой шкале $t_\exists = (n-2)t \in [0, n-2]$ с вектором $U_\exists = [u_0^\exists \ \dots \ u_{n-2}^\exists \ V_0^\exists \ V_{n-2}^\exists]$, объединяющим $n-2$ узловые точки и два вектора концевых направлений, и матрицей $H_\exists = [Q_{n,n-2} \ T_n'(0) \ T_n'(n-2)]^{-1}$ из (2.17б) осуществляются с помощью матриц $M_n = D_n(n, 0)$ и $M_\exists = D_n(n-2, 0)$:

$$\begin{aligned} U_\exists &= U_n H_n M_{n\exists} H_\exists^{-1}, \text{ где } M_{n\exists} = M_n M_\exists^{-1} = D_n(n/(n-2), 0); \\ U_n &= U_\exists H_\exists M_{\exists n} H_n^{-1}, \text{ где } M_{\exists n} = M_\exists M_n^{-1} = D_n((n-2)/n, 0). \end{aligned} \quad (2.60б)$$

Оба преобразования работают при степени полинома $n \geq 3$. Возьмём, например, кубический интерполяционный полином на сетке $U_n = [u_0^n \ u_1^n \ u_2^n \ u_3^n]$ с матрицей H_n из (2.16в) и найдём с помощью матрицы $M_{n\exists} = D_3(3, 0)$ узловой вектор конгруэнтного эрмитова полинома с матрицей $H_\exists$ из (2.27г):

$$\begin{aligned} [u_0^\exists \ u_1^\exists \ V_0^\exists \ V_1^\exists] &= [u_0^n \ u_1^n \ u_2^n \ u_3^n] \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 4 & 9 \\ 0 & 1 & 8 & 27 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 9 & 0 \\ 0 & 0 & 0 & 27 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 3 \end{bmatrix} = \\ &= \begin{bmatrix} u_0^n & u_3^n & \frac{-11u_0^n + 18u_1^n - 9u_2^n + 2u_3^n}{2} & \frac{-2u_0^n + 9u_1^n - 18u_2^n + 11u_3^n}{2} \end{bmatrix}. \end{aligned}$$

Аналогично выполняется обратное преобразование $H_\exists \Rightarrow H_n$ (рис. 2.38,а):

$$\begin{aligned}
 \begin{bmatrix} u_0^{\text{н}} & u_1^{\text{н}} & u_2^{\text{н}} & u_3^{\text{н}} \end{bmatrix} &= \begin{bmatrix} u_0^{\text{э}} & u_1^{\text{э}} & V_0^{\text{э}} & V_1^{\text{э}} \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 3 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1/3 & 0 & 0 \\ 0 & 0 & 1/9 & 0 \\ 0 & 0 & 0 & 1/27 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 4 & 9 \\ 0 & 1 & 8 & 27 \end{bmatrix} = \\
 &= \begin{bmatrix} u_0^{\text{э}} & \frac{20u_0^{\text{э}} + 7u_1^{\text{э}} + 4V_0^{\text{э}} - 2V_1^{\text{э}}}{27} & \frac{7u_0^{\text{э}} + 20u_1^{\text{э}} + 2V_0^{\text{э}} - 4V_1^{\text{э}}}{27} & u_1^{\text{э}} \end{bmatrix}.
 \end{aligned}$$

Изображение векторов $V_0^{\text{э}}$ и $V_1^{\text{э}}$ (для компактности они укорочены в пять раз), дополняющее узловую сеть $U_{\text{н}}$, позволяет оценить концевые направления интерполяционной линии ещё до её построения.

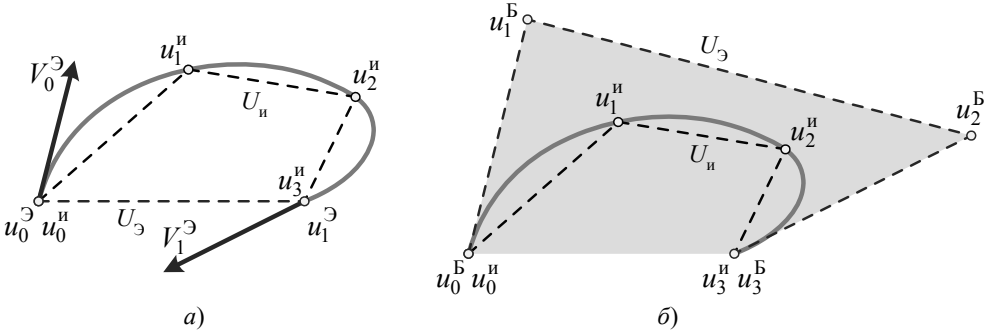


Рис. 2.38

□ Переходы между **интерполяционной** моделью $p_{\text{н}}(t_{\text{н}}) = U_{\text{н}} H_{\text{н}} T_{\text{н}}(t_{\text{н}}) \quad \forall t_{\text{н}} \in [0, n]$ и моделью **Безье** $p_{\text{б}}(t) = U_{\text{б}} H_{\text{б}} T_{\text{н}}(t) \quad \forall t \in [0, 1]$ с вектором $U_{\text{б}} = [u_0^{\text{б}} \ u_1^{\text{б}} \ \dots \ u_n^{\text{б}}]$ и матрицей $U_{\text{б}}$ из (2.38в) осуществляются с помощью матриц $M_{\text{нб}} = D_{\text{н}}(n, 0)$ и $M_{\text{бн}} = D_{\text{н}}(n, 0)^{-1} = D_{\text{н}}(n^{-1}, 0)$:

$$U_{\text{б}} = U_{\text{н}} H_{\text{н}} D_{\text{н}}(n, 0) H_{\text{б}}^{-1}, \quad U_{\text{н}} = U_{\text{б}} H_{\text{б}} D_{\text{н}}(n^{-1}, 0) H_{\text{н}}^{-1}. \quad (2.60\text{в})$$

Результат преобразования $H_{\text{н}} \Rightarrow H_{\text{б}}$ показан на рис. 2.38,б, где из интерполяционной сетки $u_0^{\text{н}} u_1^{\text{н}} u_2^{\text{н}} u_3^{\text{н}}$ получена ХП линии Безье $u_0^{\text{б}} u_1^{\text{б}} u_2^{\text{б}} u_3^{\text{б}}$ и построена её выпуклая оболочка:

$$\begin{aligned}
 \begin{bmatrix} u_0^{\text{б}} & u_1^{\text{б}} & u_2^{\text{б}} & u_3^{\text{б}} \end{bmatrix} &= \begin{bmatrix} u_0^{\text{н}} & u_1^{\text{н}} & u_2^{\text{н}} & u_3^{\text{н}} \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 4 & 9 \\ 0 & 1 & 8 & 27 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 9 & 0 \\ 0 & 0 & 0 & 27 \end{bmatrix} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 1 & 3 & -3 \\ 0 & 1 & 0 & 1 \end{bmatrix}^{-1} = \\
 &= \begin{bmatrix} u_0^{\text{н}} & \frac{-5u_0^{\text{н}} + 18u_1^{\text{н}} - 9u_2^{\text{н}} + 2u_3^{\text{н}}}{6} & \frac{2u_0^{\text{н}} - 9u_1^{\text{н}} + 18u_2^{\text{н}} - 5u_3^{\text{н}}}{6} & u_3^{\text{н}} \end{bmatrix}.
 \end{aligned}$$

📁 **Пример 2.6.** Проиллюстрируем обратное преобразование $H_{\text{б}} \Rightarrow H_{\text{н}}$ на примере вычисления узловой сети интерполяционной линии (рис. 2.39), совпадающей с линией Безье порядка $n=5$, аппроксимирующей единичную окружность с максимальной погрешностью

0.14% (см. пример 2.3). Стоит сравнить это отклонение с недопустимо большой погрешностью 17.9% приближения к окружности интерполянта, построенного на равномерной четырёхинтервальной сетке (см. рис. 2.10).

Сформируем по рис. 2.27,б вектор ХП Безье

$$U_B = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1.642 \end{bmatrix} \begin{bmatrix} -2.2 \\ 2.084 \end{bmatrix} \begin{bmatrix} -2.2 \\ -2.084 \end{bmatrix} \begin{bmatrix} 1 \\ -1.642 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

и рассчитаем по (2.60в) вектор узловых вершин интерполяционной сетки U_n с помощью матриц H_n из (2.13д) и H_B из (2.40г):

$$U_n = U_B \begin{bmatrix} 1 & -5 & 10 & -10 & 5 & -1 \\ 0 & 5 & -20 & 30 & -20 & 5 \\ 0 & 0 & 10 & -30 & 30 & -10 \\ 0 & 0 & 0 & 10 & -20 & 10 \\ 0 & 0 & 0 & 0 & 5 & -5 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 5^{-1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 5^{-2} & 0 & 0 & 0 \\ 0 & 0 & 0 & 5^{-3} & 0 & 0 \\ 0 & 0 & 0 & 0 & 5^{-4} & 0 \\ 0 & 0 & 0 & 0 & 0 & 5^{-5} \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 & 5 \\ 0 & 1 & 2^2 & 3^2 & 4^2 & 5^2 \\ 0 & 1 & 2^3 & 3^3 & 4^3 & 5^3 \\ 0 & 1 & 2^4 & 3^4 & 4^4 & 5^4 \\ 0 & 1 & 2^5 & 3^5 & 4^5 & 5^5 \end{bmatrix}^{-1} =$$

$$= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0.181 \\ 0.982 \end{bmatrix} \begin{bmatrix} -0.843 \\ 0.54 \end{bmatrix} \begin{bmatrix} -0.843 \\ -0.54 \end{bmatrix} \begin{bmatrix} 0.181 \\ -0.982 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Отсюда получаем вектор коэффициентов и параметрическую функцию очень точной и быстро вычисляемой **нетригонометрической** модели окружности пятого порядка:

$$S = U_n H_n = \begin{bmatrix} 1 & 0 & -1.28 & 0.512 & -0.0512 & 0 \\ 0 & 1.642 & -0.48 & -0.2728 & 0.10104 & -0.00808 \end{bmatrix},$$

$$p(t) = \begin{bmatrix} 1 - 1.28t^2 + 0.512t^3 - 0.0512t^4 \\ 1.642t - 0.48t^2 - 0.2728t^3 + 0.10104t^4 - 0.00808t^5 \end{bmatrix} \quad \forall t \in [0, 5]. \quad (2.61)$$

□

□ Переходы между любой **полиномиальной** и **рациональной** моделями возможны лишь в сторону последней. Сначала нужно рассчитать вектор U_B характеристической сети линии Безье n -го порядка, после чего сформировать вектор **единичных** весов вершин $\Omega = [1 \ 1 \ \dots \ 1]$. Совокупность этих векторов $\{U_B, \Omega\}$ составляет числовую основу построения рациональной линии Безье по (2.48в).

Необратимость преобразования рациональной модели с произвольными весами вершин $\omega_i \neq 1$ обусловлена невозможностью избавления в (2.57) от векторов базисных функций после их приведения в единичную шкалу. Только при всех единичных весах в знаменателе дроби (2.48б) сумма функций Бернштейна по свойству (2.39б) становится равной единице и независимой от параметра t . Но это уже лишь частный, полиномиальный случай рациональных линий Безье, преобразование типа которых рассмотрено выше.

3. Рассмотрим, наконец, задачу **преобразования порядка** полиномиальной линии, определяющего степень её полинома n и размер узлового вектора U . Изменение порядка возможно как в сторону повышения, так и понижения, но всегда **фактическая степень полинома остаётся неизменной** (см. п. 2.1.3). Понять это явление помогают два простых примера:

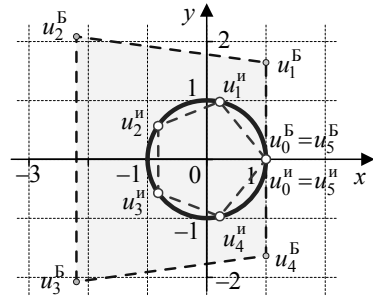


Рис. 2.39

• фактическая степень $n=1$ полинома $p_n(t)=1+2t$ может быть повышена до любой формальной степени $n'>n$, при которой линия $p_{n'}(t)=1+2t+0t^2+\dots+0t^{n'}$ задаётся $n'+1$ -й точками, лежащими на одной прямой;

• формальная степень $n\geq 2$ полинома $p_n(t)=3t+0t^2+\dots+0t^n$ понижается до фактической степени $n'=1$ линии $p_{n'}(t)=3t$, определяемой всего двумя точками.

Отсюда видно, что понижение формального порядка полиномиальной линии возможно лишь при особом расположении узловых точек, создающем хотя бы один нулевой старший коэффициент полинома. Порядок полиномиальной линии можно понизить на число последних нулевых коэффициентов, называемое дефектом степени полинома. Повышение же порядка возможно на любое число, но при этом дефект степени появляется либо увеличивается.

Всё сказанное эквивалентно тому, что преобразование $n\Rightarrow n'$ порядка ВБФ

$$T_n(t)=E_{n+1,n'+1}T_{n'}(t), \quad (2.62)$$

записанное с помощью $(n+1)\times(n'+1)$ -матрицы $E_{n+1,n'+1}$, при $n<n'$ (в сторону увеличения порядка) осуществимо всегда в силу равенства

$$\begin{bmatrix} 1 \\ \vdots \\ t^n \end{bmatrix} = \begin{bmatrix} 1 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & 0 & \dots & 0 \end{bmatrix} \begin{bmatrix} 1 \\ \vdots \\ t^n \\ \vdots \\ t^{n'} \end{bmatrix},$$

а при $n>n'$ (в сторону уменьшения порядка) лишь при фактической незначимости в полиноме $p_n(t)=UHT_n(t)$ степеней $t^{n'+1}\div t^n$, когда в векторе коэффициентов $S=UH=[s_0 \dots s_n \ 0 \dots 0]$ последние $n'-n>0$ элементов нулевые.

Подставив (2.62) в общего вида преобразование $p_n(t_1)=U_1H_1T_n(t_1)\Rightarrow p_{n'}(t_2)=U_2H_2T_{n'}(t_2)$, получим аналогичное (2.59) уравнение, решением которого является узловой вектор конгруэнтной модели n' -го порядка:

$$U_1H_1M_1E_{n+1,n'+1}=U_2H_2M_2\Rightarrow U_2=U_1H_1MH_2^{-1}, \text{ где } M=M_1E_{n+1,n'+1}M_2^{-1}. \quad (2.63)$$

Здесь матрицы $M_1=D_n(b_1-a_1, a_1)$ и $M_2=D_{n'}(b_2-a_2, a_2)$ переводят параметры $t_1\in[a_1, b_1]$ и $t_2\in[a_2, b_2]$ в единичную шкалу $t\in[0, 1]$. В (2.44б) и (2.45) выведены сходные формулы конгруэнтного изменения порядка нормированной линии Безье с матрицей $M=E_{n+1,n'+1}$, совпадающей с (2.63) при $M_1=E_{n+1}$ и $M_2=E_{n'+1}$.

Необходимость в изменении порядка модели полиномиальной линии возникает в процессе её редактирования. Увеличение порядка добавляет в узловую сеть новые вершины и повышает потенциальную гибкость, недостижимую при меньших порядках. Например, парабола, построенная на трёхузловой сети, не может иметь более одной выпуклости (см. рис. 2.29,а). Формальный инкремент порядка узловой сети позволяет строить на ней кубическую полиномиальную линию, способную иметь точку перегиба и уже два участка с разными направлениями выпуклостей (см. рис. 2.29,б). Дальнейшее увеличение порядка создаёт

ещё большие возможности редактирования и разнообразные формы кривой, достигаемые интерактивным перемещением новых узловых вершин.

Понижение порядка модели полиномиальной линии может быть полезным для оптимизации объёма памяти, достаточной для хранения данных, по которым линия может быть вновь построена без искажения формы. Сохранение избыточных узловых точек имеет смысл в незавершённом проектировании.

Пример 2.7. Преобразуем узловую сеть $u_0^u u_1^u u_2^u u_3^u$ нормализованного ИП порядка $n=3$ (рис. 2.40,а) в ХП $u_0^B u_1^B u_2^B u_3^B u_4^B u_5^B$ нормированной линии Бэзе порядка $n'=5$:

$$U_B = U_u H_u D_3(3,0) E_{4,6} H_B^{-1} =$$

$$= \begin{bmatrix} u_0^u & u_1^u & u_2^u & u_3^u \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 4 & 9 \\ 0 & 1 & 8 & 27 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 9 & 0 \\ 0 & 0 & 0 & 27 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -5 & 10 & -10 & 5 & -1 \\ 0 & 5 & -20 & 30 & -20 & 5 \\ 0 & 0 & 10 & -30 & 30 & -10 \\ 0 & 0 & 0 & 10 & -20 & 10 \\ 0 & 0 & 0 & 0 & 5 & -5 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}^{-1} =$$

$$= \begin{bmatrix} u_0^u & \frac{-u_0^u + 18u_1^u - 9u_2^u + 2u_3^u}{10} & \frac{-6u_0^u + 27u_1^u - u_3^u}{20} & \frac{-u_0^u + 27u_2^u - 6u_3^u}{20} & \frac{2u_0^u - 9u_1^u + 18u_2^u - u_3^u}{10} & u_3^u \end{bmatrix}.$$

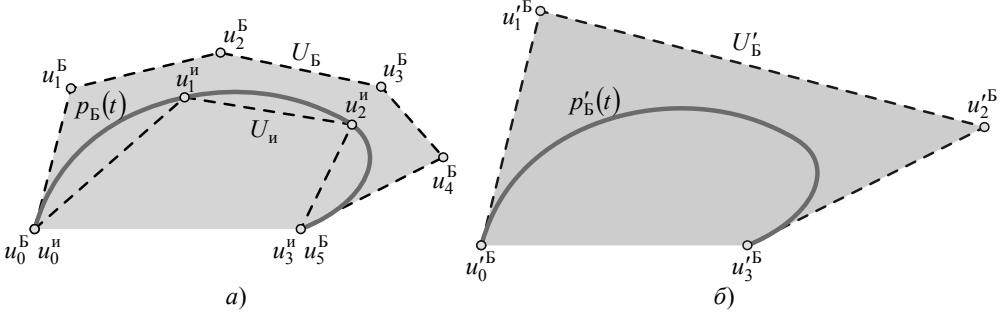


Рис. 2.40

У вектора коэффициентов полинома Бэзе $p_B(t) = U_B H_B T_5(t)$ на рис. 2.40,а

$$U_B H_B = \begin{bmatrix} u_0^u & \frac{-11u_0^u + 18u_1^u - 9u_2^u + 2u_3^u}{2} & \frac{18u_0^u - 45u_1^u + 36u_2^u - 9u_3^u}{2} & \frac{-9u_0^u + 27u_1^u - 27u_2^u + 9u_3^u}{2} & 0 & 0 \end{bmatrix}$$

дефект степени равен двум, поэтому формальная степень $n'=5$ может быть понижена до минимально возможного значения $n'-2=3$ с пересчётом ХП редуцированного полинома Бэзе $p'_B(t) = U'_B H'_B T_3(t)$ на рис. 2.40,б:

$$\begin{bmatrix} u_0^B & u_1^B & u_2^B & u_3^B \end{bmatrix} = U_B H_B E_{6,4} H_B^{-1} =$$

$$= \begin{bmatrix} u_0^B & u_1^B & u_2^B & u_3^B & u_4^B & u_5^B \end{bmatrix} \begin{bmatrix} 1 & -5 & 10 & -10 & 5 & -1 \\ 0 & 5 & -20 & 30 & -20 & 5 \\ 0 & 0 & 10 & -30 & 30 & -10 \\ 0 & 0 & 0 & 10 & -20 & 10 \\ 0 & 0 & 0 & 0 & 5 & -5 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} =$$

$$= \begin{bmatrix} u_0^u & \frac{-5u_0^u + 18u_1^u - 9u_2^u + 2u_3^u}{6} & \frac{2u_0^u - 9u_1^u + 18u_2^u - 5u_3^u}{6} & u_3^u \end{bmatrix}.$$



2.2. СОСТАВНЫЕ ЛИНИИ

Изучив общие принципы конструирования односегментных интерполяционных, аппроксимационных и приближающих полиномов $p(t)$ на конечной узловой сетке $\{t_i, u_i\}$, $i \in [0, n]$, перечислим основные **способы уменьшения межузловых осцилляций**:

- интерактивный выбор оптимального метода параметризации — нормализованной либо хордовой;
- фиксация желаемых узловых скоростей $V(t_i)$ и ускорений $A(t_i)$ интерполяционного полинома за счёт увеличения его степени до $m > n$ и усложнения расчёта коэффициентов $s_0 \div s_m$;
- уменьшение степени приближающего полинома до $m < n$ за счёт допустимого увеличения узловых невязок $|p(t_i) - u_i|$;
- использование линий Безье.

Когда число узловых точек измеряется десятками, применение перечисленных методов подавления осцилляций может не дать желаемого эффекта и приемлемой формы строящейся линии $p(t)$ из-за высокой степени её полинома и вычислительной неустойчивости расчёта его коэффициентов. Лучшим выходом из создавшегося положения становится конструирование этой линии как **составной**, сшитой из нескольких полиномиальных сегментов невысоких порядков (см. рис. 2.1, б). Для этого упорядоченное множество узлов разбивается на N **кластеров** — групп точек, и на каждом $i \in [1, N]$ -м кластере строится свой сегмент составной линии $p_i(t) = U_i L_{n_i}(t) = S_i T_{m_i}(t) = U_i H_i T_{m_i}(t)$.

Обозначим:

- n_i и m_i — число интервалов узловой сетки кластера и степень полинома;
- $\{d_{i1}, d_{i2}, \dots, d_{in_i}\}$ — список длин параметрических интервалов;
- $\{a_i \equiv t_{i0}, t_{i1}, \dots, t_{i, n_i-1}, t_{in_i} \equiv b_i\}$ — узловые отсчёты параметра $t_{ik} = t_{i, k-1} + d_{ik}$;
- P_{i-1} и P_i — концевые узловые точки кластера;
- $U_i = [P_{i-1} \equiv u_{i0} \ u_{i1} \ \dots \ u_{i, n_i-1} \ u_{in_i} \equiv P_i] \in V^{n_i+1}$ — вектор узловых точек;
- $H_i \in R^{(n_i+1) \times (m_i+1)}$ — матрица весовых коэффициентов разложения (2.7в);
- $L_{n_i}(t) \in R^{(n_i+1) \times 1}$ — вектор-столбец параметрических функций разложения (2.7а), связанный с MBK H_i соотношением (2.8) $L_{n_i}(t) = H_i T_{m_i}(t)$;

• $S_i = [s_{i0} \ s_{i1} \ \dots \ s_{i, m_i-1} \ s_{im_i}] \in V^{m_i+1}$ — вектор полиномиальных коэффициентов разложения (2.7б), связанный с матрицей H_i соотношением $S_i = U_i H_i$.

При выборе состава кластеров следует согласовать взаимное расположение узловых точек с изгибательными способностями полиномиальных линий. Хорошее качество приближения достигается при сходимости формы полилинии кластера $u_{i0} u_{i1} \dots u_{in_i}$ с реально достижимой формой полинома:

- при $m_i = 1$ сегмент представляет собой отрезок прямой линии;

- квадратичный полином степени $m_i=2$ хорошо приближает к V-образной схеме расположения узлов, лежащих **в одной плоскости**;
- кубический полином ($m_i=3$) пригоден для слежения за П- и N-образными узловыми схемами **не только на плоскости, но и в пространстве**;
- полином степени $m_i=4$ способен принять M-образную форму;
- полиномиальный сегмент степени $m_i \geq 3$ имеет до m_i-2 точек перегиба и способен как изгибаться, так и закручиваться.

С целью снижения трудоёмкости расчётов **рекомендуется использовать локальную нормализованную параметризацию** сегментов с фиксированными отсчётами $t_{ij}=j$ и параметрами $a_i=0$ и $b_i=n_i$, независимыми от межузловых расстояний $|u_{ij}-u_{i,j-1}|$. Это позволяет использовать готовые, как в (2.16), параметры полиномов разных степеней и резко повысить быстродействие и точность вычислительного процесса.

2.2.1. Сопряжение интерполяционных сегментов

Первый порядок **непрерывности** сопряжения автоматически обеспечивается применением интерполяционных алгоритмов (2.11), (2.13) и (2.17), а также алгоритма (2.33) построения каждого сегмента как приближающего полинома с закреплёнными концами. **Простейшей непрерывной составной линией является полилиния** (рис. 2.41,а) со следующими параметрами линейной модели i -го двухузлового сегмента (2.10б):

$$n_i=m_i=1, \quad U_i=[P_{i-1} \quad P_i],$$

$$Q_i=\begin{bmatrix} 1 & 1 \\ a_i & b_i \end{bmatrix}, \quad H_i=\frac{1}{b_i-a_i}\begin{bmatrix} b_i & -1 \\ -a_i & 1 \end{bmatrix},$$

$$L_i(t)=\frac{1}{b_i-a_i}\begin{bmatrix} b_i-t \\ t-a_i \end{bmatrix}, \quad S_i=\frac{1}{b_i-a_i}[P_{i-1}b_i-P_ia_i \quad P_i-P_{i-1}].$$

В локальном нормализованном варианте ($a_i=0$, $b_i=1$) эти параметры полностью совпадают с (2.16а).

Гораздо больший интерес представляет **гладкое** сопряжение сегментов составной линии. Сейчас мы ограничимся рассмотрением задачи обеспечения условий первого порядка гладкости (2.3), а к конструированию дважды гладкой составной линии обратимся позднее при изучении сплайновых линий. Равенство краевых направлений сегментов во внутренних точках сопряжения $P_1 \div P_{N-1}$ может быть достигнуто несколькими способами.

□ **Явным** заданием векторов $V_1 \div V_{N-1}$ (рис. 2.41,б) и нахождением по (2.17) или (2.33) векторов $S_1 \div S_N$ сегментов с закреплёнными концами. При этом уравнения первого и последнего сегментов включают по одному краевому условию $S_1 T'_{m_1}(b_1)=V_1$ и $S_N T'_{m_N}(a_N)=V_{N-1}$, а уравнения внутренних сегментов — по два условия $S_i T'_{m_i}(a_i)=V_{i-1}$ и $S_i T'_{m_i}(b_i)=V_i$. Впрочем, если зафиксировать оба

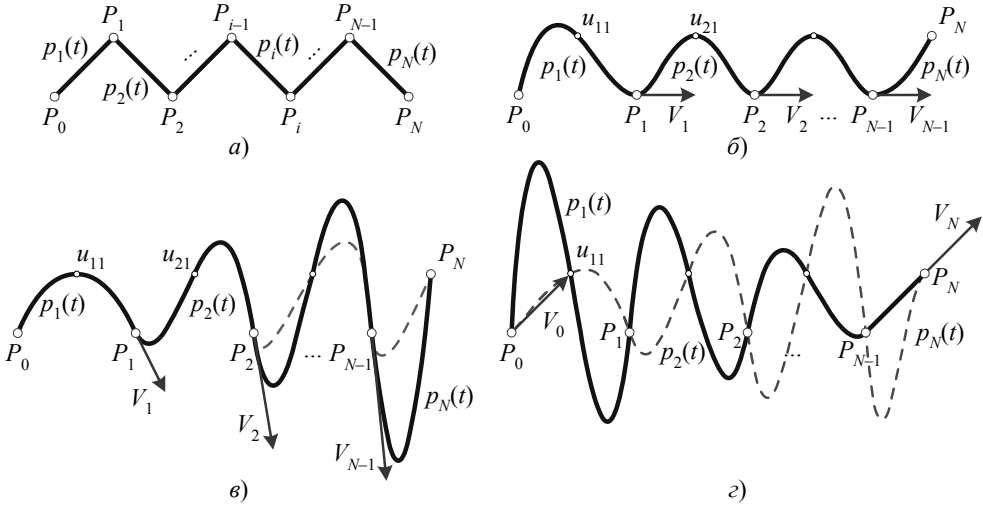


Рис. 2.41

краевые направления составной линии V_0 и V_N , то уравнения всех сегментов будут однотипными и проще программируемыми.

В отсутствие желаемых значений краевых направлений сегментов их можно аппроксимировать аналогично (2.20а) и (2.22а) выражениями

$$V_0 = (u_{11} - P_0) / d_{11}, \quad V_N = (P_N - u_{N,n_N-1}) / d_{Nn_N},$$

$$V_i = \frac{-d_{i+1,1}^2 u_{i,n_i-1} + (d_{i+1,1}^2 - d_{in_i}^2) P_i + d_{in_i}^2 u_{i+1,1}}{d_{in_i} (d_{in_i} + d_{i+1,1}) d_{i+1,1}}, \quad \forall i = \overline{1, N-1}, \quad (2.64a)$$

принимающими при нормализованной параметризации разностные формы

$$V_0 = u_{11} - P_0, \quad V_N = P_N - u_{N,n_N-1}, \quad V_i = (u_{i+1,1} - u_{i,n_i-1}) / 2 \quad \forall i = \overline{1, N-1}. \quad (2.64b)$$

□ **Поочерёдным** нахождением параметров сегментов $S_i = U_i H_i \quad \forall i = \overline{1, N}$ и использованием расчётного вектора конечного направления i -го сегмента $V_i = S_i T'_{m_i}(b_i)$ в качестве начального направления следующего $i+1$ -го сегмента (рис. 2.41, в). При этом первый сегмент строится по алгоритму (2.13) или (2.30) как полином со свободными краевыми направлениями либо аналогично (2.17) или (2.33) как полином Эрмита с заданным начальным направлением V_0 . Если некоторую точку P_i полагать лишь приближённым местом сопряжения, не совпадающим с узлом $u_{i+1,0}$, то расчётное конечное состояние i -го сегмента $S_i T'_{m_i}(b_i)$ следует задавать начальным состоянием следующего сегмента. Так будет обеспечена непрерывность сопряжения полиномиальных сегментов.

Из-за неконтролируемости конечных направлений сегментов последовательный метод сопряжения склонен к усилению осцилляций с ростом номера сегмента. Некоторому снижению скоростей «разгона» составной линии в точках сопряжения и уменьшению межузловых осцилляций может способствовать применение условий нормированной гладкости (2.3б), по которым начальные

направления сегментов задаются векторами $|V_0|\bar{V}_i$ длиной $|V_0|$, где $V_0 = S_1 T'_{m_1}(a_1)$. Эффект такой модификации алгоритма показан штрихом на рис. 2.41, в.

□ **Одновременным** вычислением сразу всех ВПК $S_1 \div S_N$, связанных **неявными условиями** непрерывного

$$S_i T_{m_i}(b_i) = S_{i+1} T_{m_{i+1}}(a_{i+1}) \quad \forall i = \overline{1, N-1} \quad (2.65a)$$

(только для сегментов с незакреплёнными концами!) и гладкого

$$S_i T'_{m_i}(b_i) = S_{i+1} T'_{m_{i+1}}(a_{i+1}) \quad \forall i = \overline{1, N-1} \quad (2.65б)$$

сопряжения сегментов. В отличие от явных методов сопряжения любое i -е неявное уравнение в (2.65) может быть связано лишь с одним из двух соседних сегментов $p_{i-1}(t)$ либо $p_i(t)$, увеличивая тем самым на единицу соответствующее число v_{i-1} или v_i интерполяционных уравнений сегмента. Зададим распределение неявных условий фиксации концов сегментов с помощью флажков $\phi_1 \div \phi_N$, устанавливаемых по следующему правилу: флажок ϕ_i равен числу неявных условий, отнесённых к **правому** концу i -го сегмента — точке P_i .

Определившись с числом v_i точных и задав число μ_i приближённых свойств i -го полиномиального сегмента, выбираем его порядок $m_i \geq 1$, ограниченный аналогично (2.33е) двусторонним неравенством

$$\max\{v_i - 1, 1\} \leq m_i \leq v_i + \mu_i - 1 \quad \forall i = \overline{1, N}. \quad (2.66a)$$

Размер составного ВПК $S = [S_1 \ S_2 \ \dots \ S_N] \in V^m$

$$m = \sum_{i=1}^N (m_i + 1) = \sum_{i=1}^N m_i + N$$

также ограничен **условиями существования и единственности решения**

$$v \leq m \leq v + \mu, \quad (2.66б)$$

в которых числа $v = \sum_{i=1}^N v_i$ и $\mu = \sum_{i=1}^N \mu_i$ определяют размерности матриц $Q_n \in R^{m \times v}$,

$Q_a \in R^{m \times \mu}$ и векторов $U_n \in V^v$, $U_a \in V^\mu$ интерполяционной $SQ_n = U_n$ и аппроксимационной $SQ_a \approx U_a$ систем уравнений, общее решение которых (2.33г) даёт искоемые коэффициенты полиномов всех сегментов составной линии.

При обращении всех правых неравенств (2.66) в равенства составная линия превращается в интерполяционную. Превышение степенью какого-либо полинома правой границы неравенства делает его интерполяционно избыточным с вытекающими отсюда *возможными* недостатками (см. рис. 2.8).

Изучим особенности применения неявного метода к решению частных задач полиномиального приближения [29].

□ При построении гладкой **интерполяционной составной линии** аппроксимационная СЛАН $SQ_a \approx U_a$ отсутствует, из (2.66) при $\mu_i = 0$ следует единственный

набор порядков сегментных полиномов $m_i = v_i \forall i = \overline{1, N}$, а при $m = v$ решение (2.33г) принимает вид $S = U_{\text{и}} Q_{\text{и}}^{-1}$.

Рассмотрим ряд **схем фиксации краевых направлений составной линии**:

- определяющее составную линию со **свободными направлениями концов**

число уравнений $v_{\text{св}} = \sum_{i=1}^N (n_i + 1) + N - 1$ складывается из суммарного числа узлов в N сегментах и $N - 1$ -го уравнения гладкого сопряжения (2.65б). С учётом свободно задаваемых флажков $\phi_i \in \{0, 1\} \forall i = \overline{1, N-1}$ и фиксированного флажка $\phi_N = 0$ условия (2.66а) выбора порядков сегментов записываются следующим образом:

$$m_1 = n_1 + \phi_1, \quad m_i = n_i + 1 - \phi_{i-1} + \phi_i \quad \forall i = \overline{2, N}. \quad (2.67a)$$

Отнеся, например, все неявные условия равенства направлений сегментов к их левым концам ($\phi_1 = \dots = \phi_{N-1} = 0$), получим один из возможных наборов степеней:

$$m_1 = n_1, \quad m_i = n_i + 1 \quad \forall i = \overline{2, N}.$$

Матрицы интерполяционной СЛАУ свободной линии имеют вид

$$Q_{\text{св}} = \left[\begin{array}{cccc|ccc} Q_{m_1 n_1} & 0 & \dots & 0 & T'_{m_1}(b_1) & \dots & 0 \\ 0 & Q_{m_2 n_2} & \dots & 0 & -T'_{m_2}(a_2) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 0 & \dots & T'_{m_{N-1}}(b_{N-1}) \\ 0 & 0 & \dots & Q_{m_N n_N} & 0 & \dots & -T'_{m_N}(a_N) \end{array} \right], \quad (2.67б)$$

$$U_{\text{св}} = [U_1 \ U_2 \ \dots \ U_N \ | \ 0 \ \dots \ 0].$$

Результат моделирования полученного варианта алгоритма (2.67) совпадает со сплошной линией на рис. 2.41,в. В противоположном варианте фиксации неявных условий на правых концах всех сегментов $\phi_1 = \dots = \phi_{N-1} = 1$ степени их полиномов задаются равными $m_i = n_i + 1 \quad \forall i = \overline{1, N-1}$ и $m_N = n_N$, что даёт составную линию, изображённую сплошным стилем на рис. 2.41,г;

- при фиксации **начального направления** составной линии V_0 число уравнений $v = v_{\text{св}} + 1$, степени полиномов сегментов определяются из условий (2.67а), кроме $m_1 = n_1 + 1 + \phi_1$, а матрицы СЛАУ эрмитовой составной линии имеют вид

$$Q_{\text{Э}} = \left[\begin{array}{c|c} & T'_{m_1}(a_1) \\ Q_{\text{св}} & \begin{matrix} 0 \\ \vdots \\ 0 \end{matrix} \end{array} \right], \quad U_{\text{Э}} = [U_{\text{св}} \ | \ V_0]; \quad (2.68a)$$

- формирование СЛАУ расчёта эрмитовой составной линии с фиксированным **конечным** направлением V_N оставляем читателю в качестве упражнения;

- явное закрепление **обоих краевых направлений** составной линии Эрмита V_0 и V_N фиксирует флажок $\phi_N = 1$, даёт число уравнений $v = v_{\text{св}} + 2$, степени полиномов сегментов $m_1 = n_1 + 1 + \phi_1$, $m_i = n_i + 1 - \phi_{i-1} + \phi_i \quad \forall i = \overline{2, N}$ и матрицы СЛАУ

$$Q_{\Xi} = \left[\begin{array}{c|cc} T'_{m_1}(a_1) & O & \\ \hline O & \vdots & \\ \vdots & O & \\ O & T'_{m_N}(b_N) & \end{array} \right], \quad U_{\Xi} = [U_{\text{св}} \mid V_0 \mid V_N]. \quad (2.68б)$$

Значения флажков $\phi_1 = \dots = \phi_{N-1} = 0$ дают набор степеней полиномов $m_i = n_i + 1 \quad \forall i = \overline{1, N-1}$ и $m_N = n_N + 2$. Составная линия построена штрихом на рис. 2.41,з;

• наконец, уравнивание краевых направлений с помощью неявного условия $S_1 T'_{m_1}(a_1) = S_N T'_{m_N}(b_N)$ при свободно задаваемых флажках $\phi_i \in \{0, 1\} \quad \forall i = \overline{1, N}$ даёт степени полиномов сегментов

$$m_1 = n_1 + 1 - \phi_N + \phi_1, \quad m_i = n_i + 1 - \phi_{i-1} + \phi_i \quad \forall i = \overline{2, N} \quad (2.69а)$$

и матрицы системы уравнений **циклической составной линии**

$$Q_{\Pi} = \left[\begin{array}{c|cc} -T'_{m_1}(a_1) & & \\ \hline O & & \\ T'_{m_N}(b_N) & & \end{array} \right], \quad U_{\Pi} = [U_{\text{св}} \mid O]. \quad (2.69б)$$

При выборе всех флажков $\phi_1 = \dots = \phi_N = 0$ либо $\phi_1 = \dots = \phi_N = 1$ в (2.69) получим регулярный набор степеней $m_i = n_i + 1 \quad \forall i = \overline{1, N}$.

Аналогично случаю односегментной циклической кривой (см. рис. 2.14) в задаче конструирования циклической составной линии возможно вырождение матрицы Q_{Π} и невычислимость вектора $S = U_{\Pi} Q_{\Pi}^{-1}$ [40]. Рассмотрим суть проблемы на примере проведения двухсегментной линии ($N=2$) с кубическими сегментами ($m_1 = m_2 = 3$) через узловую сеть, состоящую из двух двухинтервальных ($n_1 = n_2 = 2$) кластеров $\{P_0, u_{11}, P_1\}$ и $\{P_1, u_{21}, P_2\}$, так, чтобы были равны краевые производные $p'_1(t_{10})$ и $p'_2(t_{22})$.

Составим по (2.67б), (2.68б) матрицу Q_{Π} и выразим её определитель через параметрические длины

$$d_{11} = t_{11} - t_{10}, \quad d_{12} = t_{12} - t_{11}, \quad d_{21} = t_{21} - t_{20}, \quad d_{22} = t_{22} - t_{21} :$$

$$Q_{\Pi} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ t_{10} & t_{11} & t_{12} & 0 & 0 & 0 & 1 & -1 \\ t_{10}^2 & t_{11}^2 & t_{12}^2 & 0 & 0 & 0 & 2t_{12} & -2t_{10} \\ t_{10}^3 & t_{11}^3 & t_{12}^3 & 0 & 0 & 0 & 3t_{12}^2 & -3t_{10}^2 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & t_{20} & t_{21} & t_{22} & -1 & 1 \\ 0 & 0 & 0 & t_{20}^2 & t_{21}^2 & t_{22}^2 & -2t_{20} & 2t_{22} \\ 0 & 0 & 0 & t_{20}^3 & t_{21}^3 & t_{22}^3 & -3t_{20}^2 & 3t_{22}^2 \end{bmatrix},$$

$$|Q_{\Pi}| = d_{11} d_{12} d_{21} d_{22} (d_{11} + d_{12})^2 (d_{21} + d_{22})^2 (d_{11} d_{21} - d_{12} d_{22}). \quad (2.70)$$

Отсюда следует, что матрица Q_{Π} вырождена, а **задача интерполяции с циклической гладкостью первого порядка неразрешима в двух случаях:**

- при нормализованной параметризации ($d_{11}=d_{12}=d_{21}=d_{22}=1$) с произвольным расположением узловых точек;
- при хордовой параметризации с равными произведениями межузловых расстояний $d_{11}d_{21}=d_{12}d_{22}$, где $d_{11}=|u_{11}-P_0|$, $d_{12}=|P_1-u_{11}|$, $d_{21}=|u_{21}-P_1|$ и $d_{22}=|P_2-u_{21}|$.

Моделирование с разными исходными данными показало **неразрешимость задачи циклической нормализованной интерполяции при чётном числе интервалов узловой сети** $n=\sum n_i$ независимо от её разделения на кластеры. То же относится и к хордовой параметризации с симметричным расположением нечётного числа узлов, когда расстояния между ними удовлетворяют некоторым условиям неразрешимости. При **нечётных** же значениях n задача циклической составной интерполяции разрешима обоими методами параметризации.

Всё сказанное здесь иллюстрируют графики на рис. 2.42:

- гладко замкнутая двухсегментная линия (а) успешно построена на $n=4$ -интервальной узловой сети методом хордовой параметризации благодаря неравенству $d_{11}d_{21} \neq d_{12}d_{22}$, что было бы невозможным на равномерной узловой сети, показанной на рис. 2.14,в;
- трёхсегментная циклическая нормализованная составная линия (б) построена на $n=9$ -интервальной узловой сети, разбитой на три кластера с числами интервалов $n_1=n_2=n_3=3$;
- линия (в) построена методом хордовой параметризации на той же узловой сети, но с иным разбиением на кластеры $\{n_1, n_2, n_3\}=\{4, 1, 4\}$.

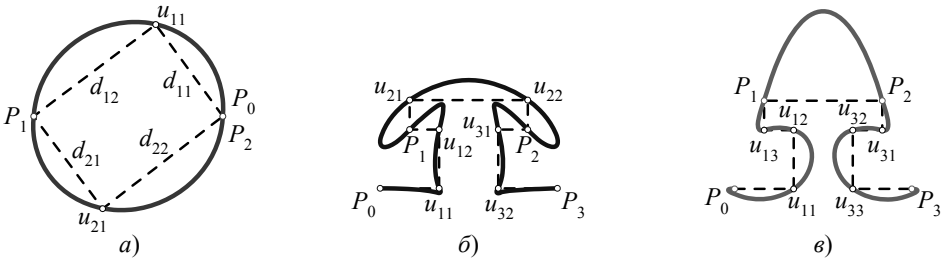


Рис. 2.42

□ Для построения гладкой N -сегментной **аппроксимационной составной линии** формируются две системы линейных уравнений, включающих:

- $v=2(N-1)$ **интерполяционных** условий (2.65) непрерывного и гладкого сопряжения сегментов:

$$SQ_{\text{и}}=U_{\text{и}}, \text{ где}$$

$$Q_{\text{и}} = \begin{bmatrix} T_{m_1}(b_1) & T'_{m_1}(b_1) & \dots & 0 & 0 \\ -T_{m_2}(a_2) & -T'_{m_2}(a_2) & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & T_{m_{N-1}}(b_{N-1}) & T'_{m_{N-1}}(b_{N-1}) \\ 0 & 0 & \dots & -T_{m_N}(a_N) & -T'_{m_N}(a_N) \end{bmatrix}, \quad U_{\text{и}} = [0 \ 0 \ \dots \ 0 \ 0]; \quad (2.71a)$$

- $\mu = \sum_{i=1}^N (n_i + 1)$ **аппроксимационных** условий приближения точек сегментов

$S_i T_{m_i}(t_{ij})$ к узлам $u_{ij} \forall i = \overline{1, N}, j = \overline{0, n_i}$:

$$SQ_a \approx U_a, \text{ где} \\ Q_a = \begin{bmatrix} Q_{m_1 n_1} & \cdots & O \\ \vdots & \ddots & \vdots \\ O & \cdots & Q_{m_N n_N} \end{bmatrix}, \quad U_a = [U_1 \dots U_N]. \quad (2.71б)$$

Благодаря нулевому составному вектору U_i общее решение (2.33г) упрощается до следующего вида:

$$S = U_a (Q_a - Q_i Q_i^+ Q_a)^+. \quad (2.71в)$$

При выборе числа сегментов N и степеней их полиномов $m_1 \div m_N$ следует руководствоваться высказанными ранее соображениями о расположении узлов в кластерах и вытекающими из (2.66а) при $\phi_i \in \{0, 1, 2\} \forall i = \overline{1, N-1}$ неравенствами:

$$\begin{cases} 1 \leq m_1 \leq n_1 + \phi_1, \quad 1 \leq m_N \leq n_N + 2 - \phi_{N-1}, \\ \max\{1 - \phi_{i-1} + \phi_i, 1\} \leq m_i \leq n_i + 2 - \phi_{i-1} + \phi_i \quad \forall i = \overline{2, N-1}. \end{cases} \quad (2.71г)$$

Из 3^{N-1} вариантов привязки неявных условий к концам сегментов можно остановиться на симметричном варианте с флажками $\phi_1 = \dots = \phi_{N-1} = 1$:

$$1 \leq m_1 \leq n_1 + 1, \quad 1 \leq m_i \leq n_i + 2 \quad \forall i = \overline{2, N-1}, \quad 1 \leq m_N \leq n_N + 1. \quad (2.71д)$$

По алгоритму (2.71) на сетке из трёх трёхинтервальных ($n_1 = n_2 = n_3 = 3$) сегментов построены гладкие составные линии со степенями полиномов сегментов $\{m_1, m_2, m_3\}$ (рис. 2.43). Три из этих линий $\{2, 2, 2\}$, $\{2, 3, 2\}$ и $\{3, 4, 3\}$ являются аппроксимационными, показывающими очевидное улучшение качества узлового приближения с ростом степеней полиномов. Порядки сегментов четвёртой составной линии $\{4, 5, 4\}$, проходящей через все узловые точки, удовлетворяют всем равенствам в (2.71д) и делают эту линию интерполяционной.

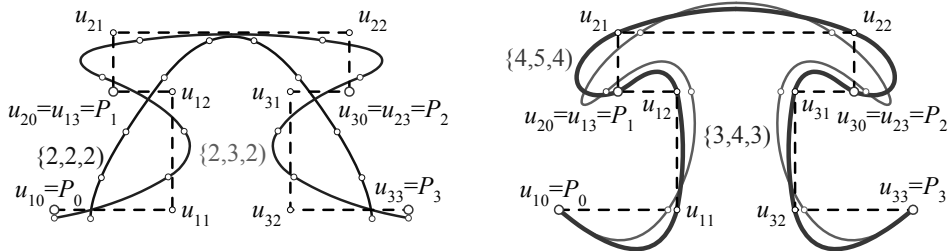


Рис. 2.43

□ В общем случае гладкая **приближающая составная линия** строится путём формирования интерполяционной $SQ_i = U_i$ и аппроксимационной $SQ_a \approx U_a$ систем уравнений с учётом всех желаемых явных и неявных условий приближения и

сопряжения и последующего расчёта составного вектора коэффициентов по формуле (2.33г):

$$S = U_{\text{и}} Q_{\text{и}}^+ + (U_{\text{а}} - U_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}}) (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+.$$

Для получения составной линии, все сегменты которой фиксированы в концевых точках $P_0 \div P_N$ и гладко сопряжены в точках $P_1 \div P_{N-1}$, а крайние сегменты имеют краевые направления V_0 и V_N , составляются матрицы

$$Q_{\text{и}} = \left[\begin{array}{ccccc|ccccc} T_{m_1}(a_1) & T_{m_1}(b_1) & \dots & 0 & 0 & T'_{m_1}(a_1) & T'_{m_1}(b_1) & \dots & 0 & 0 \\ 0 & 0 & \dots & 0 & 0 & 0 & -T'_{m_2}(a_2) & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots & T'_{m_{N-1}}(b_{N-1}) & 0 \\ 0 & 0 & \dots & T_{m_N}(a_N) & T_{m_N}(b_N) & 0 & 0 & \dots & -T'_{m_N}(a_N) & T'_{m_N}(b_N) \end{array} \right],$$

$$U_{\text{и}} = [P_0 \ P_1 \ P_1 \ P_2 \ \dots \ P_{N-1} \ P_N | V_0 \ 0 \ \dots \ 0 \ V_N],$$

$$Q_{\text{а}} = \left[\begin{array}{cccc|cccc} T_{m_1}(t_{11}) & \dots & T_{m_1}(t_{1,n_1-1}) & \dots & 0 & \dots & 0 & \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \\ 0 & \dots & 0 & \dots & T_{m_N}(t_{N1}) & \dots & T_{m_N}(t_{N,n_N-1}) & \end{array} \right], \quad (2.72a)$$

$$U_{\text{а}} = [u_{11} \ \dots \ u_{1,n_1-1} | \dots | u_{N1} \ \dots \ u_{N,n_N-1}].$$

В отсутствие желаемых значений краевых направлений сегментов их можно задать по (2.64а) векторами узловых разностей

$$V_0 = (u_{11} - P_0)/d_{11}, \quad V_N = (P_N - u_{N,n_N-1})/d_{Nn_N}. \quad (2.72б)$$

С учётом флажков $\phi_i \in \{0,1\} \forall i = \overline{1, N-1}$ условия выбора порядков сегментов $m_1 \div m_N$ задаются следующими неравенствами:

$$\begin{cases} 2 + \phi_1 \leq m_1 \leq n_1 + 1 + \phi_1, & 3 - \phi_{N-1} \leq m_N \leq n_N + 2 - \phi_{N-1}, \\ 2 - \phi_{i-1} + \phi_i \leq m_i \leq n_i + 1 - \phi_{i-1} + \phi_i & \forall i = \overline{2, N-1}. \end{cases} \quad (2.72в)$$

Из 2^{N-1} возможных вариантов привязки неявных условий (2.65б) к внутренним концам сегментов можно остановиться на варианте с флажками $\phi_1 = \dots = \phi_{N-1} = 0$

$$2 \leq m_i \leq n_i + 1 \quad \forall i = \overline{1, N-1}, \quad 3 \leq m_N \leq n_N + 2, \quad (2.72г)$$

а при нечётном числе сегментов $N = 2k + 1$ и симметричном расположении равноудалённых от краев узлов рекомендуется задавать порядки центрального и остальных сегментов из условий

$$3 \leq m_{k+1} \leq n_{k+1} + 2, \quad 2 \leq m_i = m_{2k+2-i} \leq n_i + 1 \quad \forall i \neq k + 1. \quad (2.72д)$$

Если выбрать степени полиномов всех сегментов на правых границах неравенств (2.72в–д), то составная линия будет воплощать одно из 2^{N-1} интерполяционных решений. Понижение степени хотя бы одного полинома делает линию сначала приближающей, а при значении m_j , меньшем левой границы j -го неравенства, перестают выполняться интерполяционные условия j -го сегмента и линия становится аппроксимационной.

По алгоритму (2.72), (2.64г) рассчитано и на рис. 2.44 построено несколько гладких приближающих (а) и интерполяционных (б) трёхсегментных ($N=3$) составных линий с тремя интервалами в каждом сегменте ($n_1=n_2=n_3=3$). Около каждой линии в фигурных скобках проставлены степени её сегментных полиномов $\{m_1, m_2, m_3\}$.

Система неравенств (2.72в) с двумя свободными переменными $\phi_1 \in \{0,1\}$ и $\phi_2 \in \{0,1\}$ принимает вид

$$2 + \phi_1 \leq m_1 \leq 4 + \phi_1, \quad 2 - \phi_1 + \phi_2 \leq m_2 \leq 4 - \phi_1 + \phi_2, \quad 3 - \phi_2 \leq m_3 \leq 5 - \phi_2 \quad (2.73)$$

и позволяет понять свойства полученных решений:

- линия $\{2,2,2\}$, составленная из параболических сегментов, не проходит ни через один узел и является аппроксимационной, поскольку при всех четырёх возможных значениях $\{\phi_1, \phi_2\}$ система неравенств (2.72в) несовместна в целых числах m_1 , m_2 и m_3 . Действительно, выбор значения $m_1=2$ на левой границе первого неравенства (2.73) означает, что $\phi_1=0$. Тогда из второго неравенства при $m_2=2$ следует $\phi_2=0$, что противоречит третьему неравенству при $m_3=2$;
- составная линия $\{2,3,2\}$ является приближающей с порядками сегментов на левых границах неравенств (2.73) при $\phi_1=0$ и $\phi_2=1$. Лучшее качество приближения на крайних сегментах имеет линия $\{3,3,3\}$ с большими на единицу степенями полиномов m_1 и m_3 ;
- у линии $\{3,5,3\}$ порядок среднего сегмента $m_2=5$ оказывается на правой границе второго неравенства (2.73) при $\phi_1=0$ и $\phi_2=1$, поэтому данный сегмент является интерполяционным и точно проходит через все свои узловые точки;
- все четыре набора степеней — $\{4,4,5\}$ при $\phi_1=\phi_2=0$, $\{4,5,4\}$ при $\phi_1=0$, $\phi_2=1$, $\{5,3,5\}$ при $\phi_1=1$, $\phi_2=0$ и $\{5,4,4\}$ при $\phi_1=\phi_2=1$ (эта линия симметрична первой и для ясности рисунка не показана) — совпадают с правыми границами неравенств (2.73) и делают эти составные линии интерполяционными, но с разным качеством межузлового приближения.

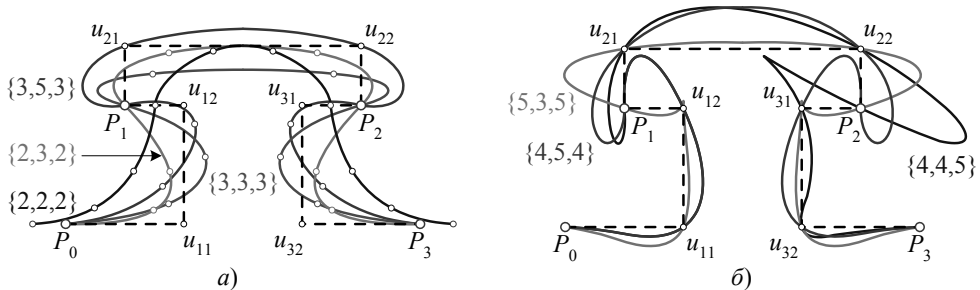


Рис. 2.44

Анализируя графики на рис. 2.41–2.44, убеждаемся, что процесс конструирования составных линий допускает множество возможных решений и потому должен носить интерактивный характер.

2.2.2. Сопряжение с перекрытием сегментов

Рассмотрим один интересный метод конструирования интерполяционной составной линии, не требующий специальных уравнений гладкого сопряжения сегментов (2.65). Суть метода состоит в частичном перекрытии соседних сегментов друг другом и линейной интерполяции сегментных моделей на перекрываемых интервалах [30].

Возьмём две гладкие линии — левую $l(\tau) \forall \tau \in [\tau_0, \tau_n]$ и правую $r(\theta) \forall \theta \in [\theta_0, \theta_m]$, интерполирующие узловые сетки $\{l_i, \tau_i\} \forall i = \overline{0, n}$ и $\{r_j, \theta_j\} \forall j = \overline{0, m}$ с $k+1$ -й парой совпадающих узлов $l_{n-k} = r_0, \dots, l_n = r_k$ (рис. 2.45, а). Важно, чтобы **обе линии были одинаково параметризованы**: с равными длинами перекрываемых интервалов $\tau_{n-k+l} - \tau_{n-k+l-1} = \theta_l - \theta_{l-1} \forall l = \overline{1, k}$ (отсюда следует, что $\tau_n - \tau_{n-k} = \theta_k - \theta_0$) и общим методом отсчёта параметров τ и θ — локальным с $\tau_0 = 0$ и $\theta_0 = 0$ либо сквозным с $\tau_{n-k} = \theta_0$.

Сформируем из перекрываемых интервалов новый сегмент и составим модели трёх не перекрывающихся линий, каждая из которых определена в собственной параметрической шкале:

- первой, совпадающей с линией $l(\tau)$ на её левых $n-k$ интервалах:

$$p_1(t) = l(t - t_{10} + \tau_0) \quad \forall t \in [t_{10}, t_{1, n-k}], \quad t_{1i} = t_{10} + \tau_i - \tau_0 \quad \forall i = \overline{0, n-k}; \quad (2.74a)$$

- второй как линейной интерполяции (2.10б) от линии $l(\tau)$ к линии $r(\theta)$ на k перекрываемых интервалах:

$$\begin{cases} p_2(t) = l(t - t_{20} + \tau_{n-k}) \frac{t_{2k} - t}{t_{2k} - t_{20}} + r(t - t_{20} + \theta_0) \frac{t - t_{20}}{t_{2k} - t_{20}} \\ \forall t \in [t_{20}, t_{2k}], \quad t_{2i} = t_{20} + \theta_i - \theta_0 \quad \forall i = \overline{0, k}; \end{cases} \quad (2.74б)$$

- третьей, совпадающей с линией $r(\theta)$ на её правых $m-k$ интервалах:

$$p_3(t) = r(t - t_{30} + \theta_k) \quad \forall t \in [t_{30}, t_{3, m-k}], \quad t_{3i} = t_{30} + \theta_{k+i} - \theta_k \quad \forall i = \overline{0, m-k}. \quad (2.74в)$$

Применение независимой параметризации позволяет получить желаемое описание сегментов (2.74) при любой параметризации линий $l(\tau)$ и $r(\theta)$. Докажем гладкость сопряжения сегментов полученной составной линии:

- на левом конце второго сегмента в точке $l_{n-k} = r_0$:

$$p'_1(t_{1, n-k}) = l'(\tau_{n-k}) \equiv l'_{n-k}, \quad p'_2(t_{20}) = \frac{l'_{n-k} \cdot (t_{2k} - t_{20}) - l_{n-k} \cdot r'_0 \cdot 0 + r_0}{t_{2k} - t_{20}} = l'_{n-k};$$

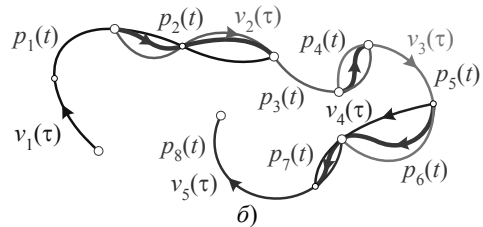
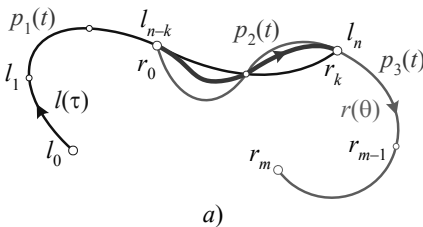


Рис. 2.45

- на правом конце второго сегмента в точке $l_n=r_k$:

$$p_2'(t_{2k}) = \frac{l_n' \cdot 0 - l_n + r_k' \cdot (t_{2k} - t_{20}) + r_k}{t_{2k} - t_{20}} = r_k', \quad p_3'(t_{30}) = r'(\theta_k) \equiv r_k'.$$

Двойное дифференцирование функций сегментов в точках их сопряжения, однако, не даёт равных ускорений:

$$p_1''(t_{1,n-k}) = l_{n-k}'' \neq p_2''(t_{20}) = l_{n-k}'' + \frac{2}{t_{2k} - t_{20}}(r_0' - l_{n-k}'),$$

$$p_2''(t_{2k}) = r_k'' + \frac{2}{t_{2k} - t_{20}}(r_k' - l_n') \neq p_3''(t_{30}) = r_k''.$$

Таким образом, линейная интерполяция перекрывающихся сегментов даёт **первый** порядок гладкости сопряжения среднего сегмента с крайними.

Пусть исходные сегменты $l(\tau)$ и $r(\theta)$, интерполирующие узловые сети $U_l = [l_0 \ l_1 \ \dots \ l_n]$ и $U_r = [r_0 \ r_1 \ \dots \ r_m]$ на параметрических шкалах $\Delta_\tau = \{\tau_0, \tau_1, \dots, \tau_n\}$ и $\Delta_\theta = \{\theta_0, \theta_1, \dots, \theta_m\}$, имеют полиномиальные модели (2.76)

$$l(\tau) = S_l T_n(\tau) \quad \forall \tau \in [\tau_0, \tau_n], \quad r(\theta) = S_r T_m(\theta) \quad \forall \theta \in [\theta_0, \theta_m]$$

с векторами коэффициентов $S_l = U_l Q_n^{-1}(\Delta_\tau)$ и $S_r = U_r Q_m^{-1}(\Delta_\theta)$, рассчитанными по (2.13г) с помощью матриц Вандермонда $Q_n(\Delta_\tau)$ и $Q_m(\Delta_\theta)$, сформированных по (2.13в). При едином векторе узловых точек $U = S Q_n(\Delta)$ из (2.13б) следует формула преобразования ВПК S из одной шкалы Δ_1 в другую шкалу Δ_2 :

$$S_1 Q_n(\Delta_1) = S_2 Q_n(\Delta_2) \Rightarrow S_2 = S_1 Q_n(\Delta_1) Q_n^{-1}(\Delta_2). \quad (2.75)$$

Составим на основе выражений (2.74) и (2.75) полиномиальные модели сегментов $p_1(t) \div p_3(t)$ с векторами коэффициентов $C_1 \div C_3$.

Первый сегмент в начале линии $l(\tau)$ описывается полиномом n -й степени

$$p_1(t) = S_l T_n(t - t_{10} + \tau_0) = C_1 T_n(t) \quad \forall t \in [t_{10}, t_{1,n-k}] \quad (2.76a)$$

с вектором коэффициентов

$$C_1 = S_l Q_n(\Delta_\tau) Q_n^{-1}(\Delta_\tau - \tau_0 + t_{10}) \in V^{n+1}. \quad (2.76b)$$

Поскольку линейная интерполяция инкрементирует степени смешиваемых полиномов, то модель второго сегмента порядка $\mu = \max\{n, m\} + 1$ имеет вид

$$p_2(t) = C_2 T_\mu(t) \quad \forall t \in [t_{20}, t_{2k}]. \quad (2.77a)$$

Для нахождения вектора коэффициентов $C_2 \in V^{\mu+1}$ составим линейную интерполяцию (2.10б) линий $l(\tau)$ и $r(\theta)$ на перекрываемых интервалах:

$$p_2(t) = L T_n(t) \frac{t_{2k} - t}{t_{2k} - t_{20}} + R T_m(t) \frac{t - t_{20}}{t_{2k} - t_{20}} \quad \forall t \in [t_{20}, t_{2k}]. \quad (2.77b)$$

Вспомогательные векторы коэффициентов $L \in V^{n+1}$ и $R \in V^{m+1}$ рассчитываются по (2.75) с учётом сдвига начал отсчётов параметров τ и θ к значению t_{20} :

$$L = S_l Q_n(\Delta_\tau) Q_n^{-1}(\Delta_\tau - \tau_{n-k} + t_{20}), \quad R = S_r Q_m(\Delta_\theta) Q_m^{-1}(\Delta_\theta - \theta_0 + t_{20}). \quad (2.77b)$$

Введём обозначения трёх операций расширения некоторого вектора коэффициентов полинома v -й степени $S \in V^{v+1}$ до степени μ :

- $\Rightarrow^{\mu-v} S = [S \ O_{\mu-v}] \in V^{\mu+1}$ — вправо $\mu-v$ нулевыми элементами;
- $\Leftarrow^{\mu-v-1} S = [O_1 \ S \ O_{\mu-v-1}] \in V^{\mu+1}$ — влево одним и вправо $\mu-v-1$ -м нулевыми

элементами. При $\mu=v+1$ правого расширения нет и $\Rightarrow^0 \Leftarrow^0 S = [O_1 \ S] \in V^{\mu+1}$.

Теперь интерполяция (2.77б) может быть сведена к виду (2.77а) с ВПК

$$C_2 = \frac{\Rightarrow^{\mu-n} L - \Leftarrow^{\mu-n-1} L + \Leftarrow^{\mu-m-1} R - \Rightarrow^{\mu-m} R}{t_{2k} - t_{20}}. \quad (2.77г)$$

Третий сегмент в конце линии $r(\theta)$ описывается полиномом m -й степени

$$p_3(t) = S_r T_m(t - t_{30} + \theta_k) = C_3 T_m(t) \quad \forall t \in [t_{30}, t_{3,m-k}] \quad (2.78а)$$

с вектором коэффициентов

$$C_3 = S_r Q_m(\Delta_\theta) Q_m^{-1}(\Delta_\theta - \theta_k + t_{30}) \in V^{m+1}. \quad (2.78б)$$

Настало время применить методику (2.76)–(2.78) к преобразованию составной линии с перекрывающимися сегментами (рис. 2.45,б). Исходными данными, необходимыми для расчёта параметров неперекрывающихся сегментов составной линии $p_1(t) \div p_M(t)$, являются:

- число исходных перекрывающихся сегментов N ;
- порядки n_i , векторы коэффициентов $S_i \in R^{n_i+1}$ и параметрические шкалы $\Delta_i = \{\tau_{i0}, \tau_{i1}, \dots, \tau_{in_i}\}$ сегментов, описываемых полиномиальными векторными функциями $v_i(\tau) = S_i T_{n_i}(\tau) \quad \forall i = \overline{1, N}$;

• **глубина перекрытия** каждой пары сегментов $\{v_i, v_{i+1}\}$ как число их общих интервалов $k_i \geq 1$;

• тип локализации проектируемых сегментов $p_1(t) \div p_M(t)$ зададим флажком $loc \in \{0, 1\}$, устанавливаемым в 0 при выборе сквозного либо в 1 при выборе локального метода отсчёта параметра t .

Составим алгоритм расчёта элементов гладкой составной линии с перекрывающимися сегментами, возвращающий:

- число M результирующих сегментов $p_1(t) \div p_M(t)$;
- список параметрических интервалов $D = \{D_1, \dots, D_M\}$;
- список векторов полиномиальных коэффициентов $C = \{C_1, \dots, C_M\}$.

Шаг 1. Инициализация переменных $M=0$, $b=0$ и элементов вектора глубин перекрытий $k_0 = k_N = 0$.

Шаг 2. Цикл $i = \overline{1, N}$ обработки сегментов исходной составной линии:

- если $i > 1$, то: // перекрывающиеся сегменты

$$\{a=(1-\text{loc})b, \quad b=a+\tau_{i,k_{i-1}}-\tau_{i0}, \quad D_{++M}=\{a,b\};$$

$$L=S_{i-1}Q_{n_{i-1}}(\Delta_{i-1})Q_{n_{i-1}}^{-1}(\Delta_{i-1}-\tau_{i-1,n_{i-1}-k_{i-1}}+a);$$

$$R=S_iQ_{n_i}(\Delta_i)Q_{n_i}^{-1}(\Delta_i-\tau_{i0}+a);$$

$$\mu=\max\{n_{i-1}, n_i\}+1, \quad C_M=\frac{\overset{\Rightarrow \mu-n_{i-1}}{b} \overset{1 \Leftrightarrow \mu-n_{i-1}-1}{L} - \overset{1 \Leftrightarrow \mu-n_i-1}{L} + \overset{\Rightarrow \mu-n_i}{R} - a \overset{\Rightarrow \mu-n_i}{R}}{b-a}; \}$$

- если $k_{i-1}+k_i < n_i$, то:

// не перекрытая часть сегмента

$$\{a=(1-\text{loc})b, \quad b=a+\tau_{i,n_i-k_i}-\tau_{i,k_{i-1}}, \quad D_{++M}=\{a,b\};$$

$$C_M=S_iQ_{n_i}(\Delta_i)Q_{n_i}^{-1}(\Delta_i-\tau_{i,k_i}+a); \}$$

Шаг 3. Возврат $\{M, D, C\}$.



По этому алгоритму на рис. 2.46 построены две гладкие составные линии. У первой исходно были заданы три сегмента с порядками $\{n_1=n_3=4, n_2=3\}$ и глубинами перекрытий $k_1=k_2=1$. Результирующая составная линия включила пять сегментов с порядками $\{n_1=n_5=4, n_2=n_4=5, n_3=3\}$. Для сравнения рядом рассчитана вторая линия на основе пяти сегментов с параметрами перекрытия $\{n_1=n_5=2, n_2=n_4=3, n_3=5\}$ и $\{k_1=k_4=1, k_2=k_3=2\}$. После интерполяции получено семь сегментов с порядками $\{n_1=n_7=2, n_2=n_6=4, n_3=n_5=6, n_4=5\}$. **Налицо уменьшение межузловых осцилляций составной линии при понижении порядков перекрывающихся сегментов.**

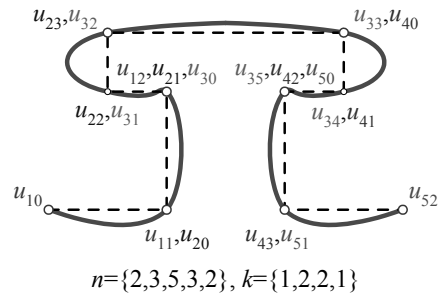
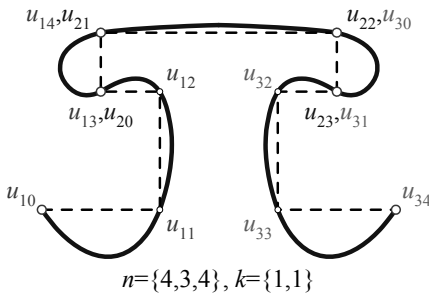


Рис. 2.46

Наилучшее качество приближения достигается при линейной интерполяции N **параболических** сегментов ($n_i=2 \quad \forall i=\overline{1, N}$) с **одноинтервальным** перекрытием ($k_i=1 \quad \forall i=\overline{1, N-1}$). Этот частный случай решённой выше общей задачи межсегментной интерполяции, к тому же лишь с локальной нормализованной параметризацией, впервые разработан **А. Оверхаузером** и описан в [51].

Получаемая составная линия имеет следующие **свойства** (рис. 2.47):

- каждый из $M=N+1$ сегментов $p_i(t) \quad \forall i=\overline{1, M}$ занимает по одному интервалу $[P_{i-1}, P_i]$ узловой сетки со сквозной нумерацией узлов $P_0 P_1 \dots P_N P_M$;
- крайние сегменты $p_1(t)$ и $p_M(t)$ представляют собой фрагменты квадратичных полиномов;

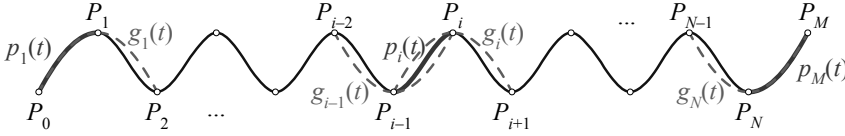


Рис. 2.47

- внутренние сегменты $p_2(t) \div p_N(t)$ являются кубическими полиномами;
- смежные сегменты сопряжены с **первым** порядком гладкости (2.3а).

Составим **прямые узловые модели** нормализованных ($t_{ij}=j$) локальных ($a=0$) одноинтервальных ($b=1$) сегментов искомой составной линии $p_1(t) \div p_M(t)$ на узловой сетке $P_0 P_1 \dots P_M$. Полагаем, что каждый вектор S_i коэффициентов исходного параболического сегмента

$$g_i(\tau) = s_{i0} + s_{i1}\tau + s_{i2}\tau^2 = S_i T_2(\tau) \quad \forall \tau \in [0, 2]$$

также вычислен методом локальной нормализованной параметризации:

$$S_i = [P_{i-1} \ P_i \ P_{i+1}] \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}^{-1} = \left[P_{i-1} \ \frac{-3P_{i-1} + 4P_i - P_{i+1}}{2} \ \frac{P_{i-1} - 2P_i + P_{i+1}}{2} \right].$$

Первый сегмент $p_1(t) = g_1(t)$ строится по (2.76а) в интервале $[P_0, P_1]$ по трём первым узлам $\{P_0, P_1, P_2\}$ и имеет по (2.76б) вектор коэффициентов

$$C_1 = S_1 = \left[P_0 \ \frac{-3P_0 + 4P_1 - P_2}{2} \ \frac{P_0 - 2P_1 + P_2}{2} \right], \quad (2.79a)$$

совпадающий с вектором коэффициентов из (2.16б), и параметрическую модель в виде полинома второй степени

$$p_1(t) = P_0 + \frac{-3P_0 + 4P_1 - P_2}{2}t + \frac{P_0 - 2P_1 + P_2}{2}t^2 \quad \forall t \in [0, 1]. \quad (2.79б)$$

Внутренние сегменты $p_i(t) = (1-t)g_{i-1}(t) + tg_i(t)$ строятся по (2.77б) в интервалах $[P_{i-1}, P_i] \quad \forall i = \overline{2, N}$ как полиномы степени $\mu=3$ с векторами коэффициентов вида (2.77г). Найдём по (2.77в) векторы $L \equiv [l_0 \ l_1 \ l_2]$ и $R \equiv [r_0 \ r_1 \ r_2]$:

$$L = [P_{i-2} \ P_{i-1} \ P_i] \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} = \left[P_{i-1} \ \frac{-P_{i-2} + P_i}{2} \ \frac{P_{i-2} - 2P_{i-1} + P_i}{2} \right],$$

$$R = [P_{i-1} \ P_i \ P_{i+1}] \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}^{-1} = S_i = \left[P_{i-1} \ \frac{-3P_{i-1} + 4P_i - P_{i+1}}{2} \ \frac{P_{i-1} - 2P_i + P_{i+1}}{2} \right].$$

Тогда по (2.77г) при $t_{20}=0$ и $t_{21}=1$ получим вектор коэффициентов

$$\begin{aligned} C_i &= \overset{\Rightarrow 1}{L} - \overset{1 \Leftarrow}{L} + \overset{1 \Leftarrow}{R} = [l_0 \ l_1 - l_0 + r_0 \ l_2 - l_1 + r_1 \ r_2 - l_2] = \\ &= \left[P_{i-1} \ \frac{-P_{i-2} + P_i}{2} \ \frac{2P_{i-2} - 5P_{i-1} + 4P_i - P_{i+1}}{2} \ \frac{-P_{i-2} + 3P_{i-1} - 3P_i + P_{i+1}}{2} \right] \end{aligned} \quad (2.79в)$$

и параметрическую модель в виде полинома третьей степени

$$p_i(t) = P_{i-1} + \frac{-P_{i-2} + P_i}{2}t + \frac{2P_{i-2} - 5P_{i-1} + 4P_i - P_{i+1}}{2}t^2 + \frac{-P_{i-2} + 3P_{i-1} - 3P_i + P_{i+1}}{2}t^3 \quad \forall t \in [0, 1], i = \overline{2, N}. \quad (2.79\text{г})$$

Последний сегмент $p_M(t) = g_N(t)$ строится по (2.78а) в интервале $[P_N, P_M]$ по трём узлам $\{P_{N-1}, P_N, P_M\}$, имеет по (2.78б) вектор коэффициентов

$$C_N = [P_{N-1} \ P_N \ P_M] \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} = \left[P_N \quad \frac{-P_{N-1} + P_M}{2} \quad \frac{P_{N-1} - 2P_N + P_M}{2} \right] \quad (2.79\text{д})$$

и параметрическую модель в виде полинома второй степени

$$p_N(t) = P_N + \frac{-P_{N-1} + P_M}{2}t + \frac{P_{N-1} - 2P_N + P_M}{2}t^2 \quad \forall t \in [0, 1]. \quad (2.79\text{е})$$

Зависимости (2.79б,г,е) можно использовать для быстрого поинтервального интерполирования составной линии прямо по узловым точкам без вычисления полиномиальных коэффициентов. Образцы построенных таким способом плоских и пространственной линий приведены на рис. 2.48. Они демонстрируют исключительно хорошее качество межузлового приближения, благодаря чему **метод перекрытия параболических сегментов можно считать одним из лучшим в практике построения гладких составных линий.**

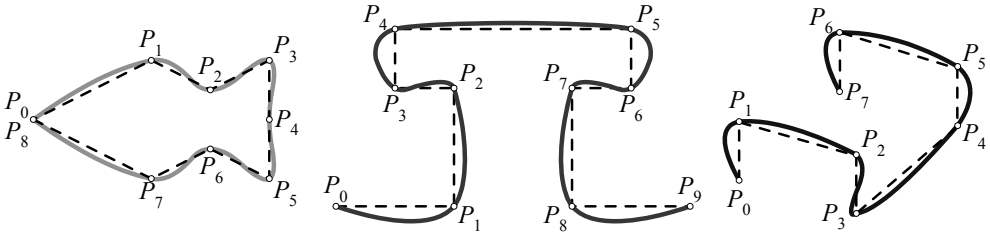


Рис. 2.48

2.2.3. Составные линии Безье

В п. 2.1.3 нами были рассмотрены методы получения и свойства односегментных линий Безье (2.36а) и (2.38). Соединение их друг с другом с нулевым порядком гладкости сопряжения (см. рис. 2.24) не представляет никакой сложности и применяется для создания изломов в точках сопряжения. Однако, чаще требуется получить гладкие и даже дважды гладкие сложные составные линии с большим числом узловых точек и интерактивным управлением формой путём перемещения небольшого числа управляющих точек. Связь околонечных вершин XII сегментов определена соотношениями (2.41), весьма затрудняющими обеспечение желаемого порядка гладкости сопряжения сегментов при независимом размещении управляющих точек.

Построение **кубических** сегментов **составной линии Безье** (рис. 2.49)

$$p_i(t) = (1-t)^3 P_{i-1} + 3t(1-t)^2 u_{i1} + 3t^2(1-t)u_{i2} + t^3 P_i \quad \forall t \in [0, 1], i = \overline{1, N} \quad (2.80\text{а})$$

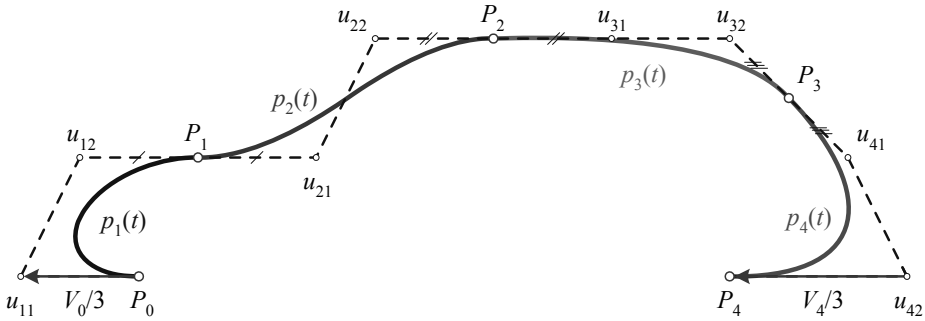


Рис. 2.49

производится на трёхзвенных ХП $P_{i-1}u_{i1}u_{i2}P_i$ в интервале **сквозного** параметра $\theta \in [0, N]$. Номер сегмента i и локальный параметр t вычисляются по формулам

$$i = \lceil \theta \rceil + \{\theta = 0\}, \quad t = \theta + 1 - i. \quad (2.80б)$$

Первый порядок гладкости сопряжения сегментов обеспечивается уравнением векторов узловых направлений

$$V_i(t) = -3(1-t)^2 P_{i-1} + 3(1-t)(1-3t)u_{i1} + 3t(2-3t)u_{i2} + 3t^2 P_i \quad (2.81a)$$

во внутренних узлах $P_1 \div P_{N-1}$. Из равенства $V_i(1) = V_{i+1}(0)$ получаем условие связи узловой точки P_i с управляющими точками u_{i2} и $u_{i+1,1}$:

$$P_i - u_{i2} = u_{i+1,1} - P_i. \quad (2.81б)$$

Отсюда следует, что **характеристический узел P_i должен лежать в середине отрезка между управляющими вершинами u_{i2} и $u_{i+1,1}$.**

На основе проведённого анализа сформулируем **методику и правила интерактивного конструирования составной линии Безье**:

- задаётся число кубических сегментов N ;
- первыми фиксируются концевые точки сегментов составной линии $P_0 \div P_N$;
- затем ставятся управляющие точки u_{11} и u_{N2} , задающие краевые направления линии $V_0 = 3(u_{11} - P_0)$ и $V_N = 3(P_N - u_{N2})$;
- остальные управляющие точки устанавливаются парами $\{u_{i2}, u_{i+1,1}\}$ вокруг внутренних узлов P_i — свободно задав одну, по (2.81б) находим место второй:

$$u_{i+1,1} = 2P_i - u_{i2} \Leftrightarrow u_{i2} = 2P_i - u_{i+1,1};$$

- при переносе узловой точки P_i на вектор F соседние управляющие точки переносятся на тот же вектор (рис. 2.50,а):

$$P_i \rightarrow P_i + F \Rightarrow u_{i2} \rightarrow u_{i2} + F, \quad u_{i+1,1} \rightarrow u_{i+1,1} + F;$$

- свободное смещение одной управляющей точки из $\{u_{i2}, u_{i+1,1}\}$ на вектор G при неподвижном узле P_i переносит вторую точку на вектор $-G$ (рис. 2.50,б):

$$u_{i2} \rightarrow u_{i2} + G \Rightarrow u_{i+1,1} \rightarrow u_{i+1,1} - G,$$

$$u_{i+1,1} \rightarrow u_{i+1,1} + G \Rightarrow u_{i2} \rightarrow u_{i2} - G.$$

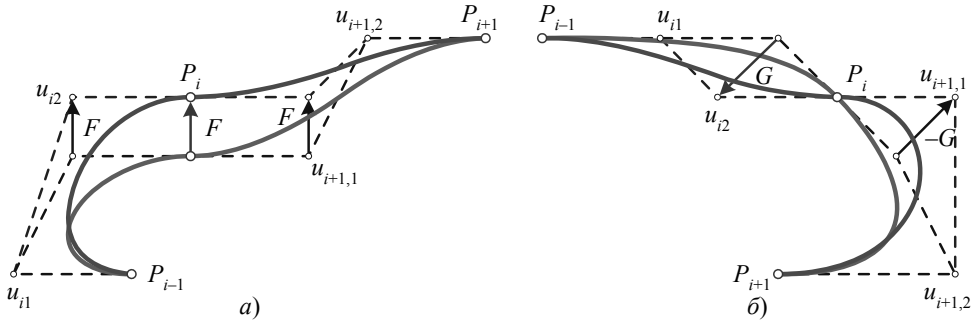


Рис. 2.50

Визуально оценивая форму линии, получающейся после каждого перемещения узловой или управляющей точки, конструктор добивается удовлетворяющего его решения.

Составные кубические линии Безье не обладают достаточной гибкостью формообразования и не обеспечивают второго порядка гладкости сопряжения сегментов — равенства узловых ускорений $A_i(1)=A_{i+1}(0)$ и непрерывности кривизны κ , определённой в (1.110а). Действительно, получив дифференцированием (2.81а) вектор ускорения

$$A_i(t)=6(1-t)P_{i-1}+6(3t-2)u_{i1}+6(1-3t)u_{i2}+6tP_i, \quad (2.81в)$$

убеждаемся в том, что ускорения в точке сопряжения смежных сегментов

$$A_i(1)=6(u_{i1}-2u_{i2}+P_i), \quad A_{i+1}(0)=6(P_i-2u_{i+1,1}+u_{i+1,2})$$

в общем случае различны из-за независимости точек u_{i1} и $u_{i+1,2}$.

Но не всегда разрывность кривизны сопряжения сегментов является недостатком: она позволяет создать в желаемом узле P_i **точку перегиба** составной линии путём размещения управляющих точек u_{i1} и $u_{i+1,2}$ по разные стороны от отрезка $u_{i2}u_{i+1,1}$. Примером такого решения служит узел P_1 на рис. 2.49.

Непрерывность кривизны сопряжения достигается в составной линии Безье с сегментами выше третьего порядка. Кроме двойной гладкости польза таких линий состоит в их повышенной гибкости за счёт введения в характеристические полилинии дополнительных контрольных вершин.

Запишем модель (2.38а) i -го сегмента Безье порядка $n_i \geq 1$ на кластере $U_i = u_{i0}u_{i1} \dots u_{i,n_i-1}u_{in_i}$ с концевыми точками $u_{i0} \equiv P_{i-1}$ и $u_{in_i} \equiv P_i$:

$$p_i(t) = \sum_{j=0}^{n_i} b_{n_i,j}(t) u_{ij} = U_i B_{n_i}(t), \quad \theta \in [0, N], \quad i = \lceil \theta \rceil + \{\theta=0\}, \quad t = \theta + 1 - i \in [0, 1]. \quad (2.82)$$

Базисные функции Бернштейна $b_{ni}(t)$ определены в (2.38б).

Первый порядок гладкости сопряжения сегментов $p_i(t)$ и $p_{i+1}(t)$ означает равенство векторов направлений $p'_i(1)=p'_{i+1}(0)$ и, согласно (2.39г), связывает положение управляющих вершин характеристической линии $\{u_{i,n_i-1}, u_{i+1,1}\}$, окружающего узел P_i , условием

$$V_i = n_i(P_i - u_{i,n_i-1}) = n_{i+1}(u_{i+1,1} - P_i) \quad (2.83a)$$

их **коллинеарности** и деления отрезка $u_{i,n_i-1}u_{i+1,1}$ точкой P_i обратно пропорционально степеням полиномов сегментов (рис. 2.51,а).

В точке **дважды гладкого сопряжения** сегментов $p_i(t)$ и $p_{i+1}(t)$ кроме направлений уравниваются еще и векторы вторых производных $p_i''(1) = p_{i+1}''(0)$. Используя выражения (2.39д), получаем второе ограничение на расположение двух пар вершин характеристической полилинии $\{u_{i,n_i-2}, u_{i,n_i-1}\}$ и $\{u_{i+1,1}, u_{i+1,2}\}$, окружающих узловую вершину P_i :

$$A_i = n_i(n_i - 1)(P_i - 2u_{i,n_i-1} + u_{i,n_i-2}) = n_{i+1}(n_{i+1} - 1)(P_i - 2u_{i+1,1} + u_{i+1,2}). \quad (2.83б)$$

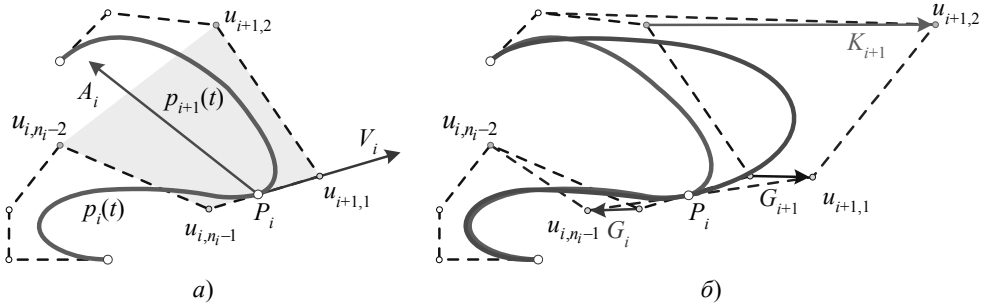


Рис. 2.51

Геометрически условия (2.83) означают **компланарность всех пяти точек** и **одностороннее** расположение вершин u_{i,n_i-2} и $u_{i+1,2}$, управляющих кривизной сопряжения, относительно отрезка $u_{i,n_i-1}u_{i+1,1}$. В результате **достижение двойной гладкости сопряжения сегментов в узле P_i делает невозможным создание в нём точек перегиба и кручения**. Это существенно ограничивает изгибающую способность и свободу расположения проектируемой линии.

Интерактивное конструирование составной линии Безье начинается с создания характеристической полилинии и назначения некоторых её вершин узлами интерполяции $P_0 \div P_N$ (это равносильно разбиению ХП на N кластеров и фиксации степеней полиномов сегментов $n_1 \div n_N$), после чего следуют перемещения вершин ХП и визуальная оценка формы получающейся линии. Любое смещение узловой или управляющей точки приводит к нарушению условий гладкости сопряжения (2.83), для восстановления которых нужно должным образом переместить связанные ими вершины ХП (рис. 2.51,б). Обозначим:

- F_i — вектор переноса узловой точки P_i ;
- G_i и G_{i+1} — векторы переноса точек u_{i,n_i-1} и $u_{i+1,1}$, управляющих направлением сегментов $p_i(t)$ и $p_{i+1}(t)$ в узле P_i ;
- K_i и K_{i+1} — векторы переноса точек u_{i,n_i-2} и $u_{i+1,2}$, управляющих кривизной сопряжения сегментов в узле P_i .

Расширение возможностей конструирования составных линий Безье достигается назначением каждому узлу P_i желаемого порядка $k_i \in [0, 1, 2]$ гладкости сопряжения с примыкающими к нему сегментами $p_i(t)$ и $p_{i+1}(t)$. Крайние узлы P_0 и P_N получают нулевые порядки сопряжения $k_0 = k_N = 0$. Во всех узлах действуют следующие **правила перемещения вершин характеристической полилинии**:

- в узле с **нулевым порядком гладкости сопряжения** $k_i = 0$ условия (2.83) не действуют, что позволяет создать в нём точку излома составной линии. Перемещения узла и окружающих его управляющих точек выполняются свободно;

- **первый порядок гладкости сопряжения** $k_i = 1$, возможный при порядках сопрягаемых сегментов $n_i \geq 2$ и $n_{i+1} \geq 2$, связывает векторы F_i , G_i и G_{i+1} вытекающим из (2.83а) условием

$$n_i(F_i - G_i) = n_{i+1}(G_{i+1} - F_i). \quad (2.84a)$$

Поскольку в интерактивном процессе движения вершин характеристической полилинии производятся поочерёдно, то:

- ◇ свободное смещение узла P_i вызывает такие же зависимые смещения соседних управляющих точек u_{i,n_i-1} и $u_{i+1,1}$ на векторы $G_i = F_i$ и $G_{i+1} = F_i$;

- ◇ свободный перенос любой из управляющих точек $\{u_{i,n_i-1}, u_{i+1,1}\}$ вынуждает зависимую точку двигаться в противоположном направлении, как показано на рис. 2.50,б. Два вектора переноса управляющих вершин связаны соотношением $n_i G_i = -n_{i+1} G_{i+1}$, позволяющим по одному вектору найти другой:

$$G_{i+1} = -\frac{n_i}{n_{i+1}} G_i, \quad G_i = -\frac{n_{i+1}}{n_i} G_{i+1};$$

- ◇ смещения точек $u_{i2} \div u_{i,n_i-2}$ кластера порядка $n_i \geq 4$ выполняются **свободно** и не влияют на расположение других точек;

- ◇ если порядок i -го сегмента равен двум (см. рис. 2.26,б), то смещение средней точки u_{i1} , управляющей обоими краевыми направлениями, приводит к зависимым смещениям управляющих точек $i-1$ -го и $i+1$ -го кластеров. Цепочки зависимых смещений обрываются по достижении сегментов со степенями полиномов выше второй;

- **второй порядок гладкости сопряжения** $k_i = 2$, реализуемый при порядках сегментов $n_i \geq 4$ и $n_{i+1} \geq 4$, связывает векторы переноса F_i , G_i , K_i , G_{i+1} и K_{i+1} вытекающим из (2.83б) условием

$$n_i(n_i - 1)(F_i - 2G_i + K_i) = n_{i+1}(n_{i+1} - 1)(F_i - 2G_{i+1} + K_{i+1}). \quad (2.84б)$$

Рассмотрим варианты свободных движений вершин характеристической полилинии и порождаемые ими в соответствии с (2.84) зависимые движения:

- ◇ смещение узла P_i влечёт такие же смещения $G_i = K_i = G_{i+1} = K_{i+1} = F_i$ четырёх управляющих точек $\{u_{i,n_i-2}, u_{i,n_i-1}, u_{i+1,1}, u_{i+1,2}\}$;

- ◇ векторы переноса управляющих точек u_{i,n_i-1} и $u_{i+1,1}$ связаны условием $n_i G_i = -n_{i+1} G_{i+1}$. Для обеспечения непрерывности кривизны одна из управ-

ляющих точек сопряжённого кластера $u_{i+1,2}$ либо u_{i,n_i-2} смещается на вектор K_{i+1} либо K_i (см. рис. 2.51,б) согласно соотношению

$$n_i(n_i-1)(K_i-2G_i)=n_{i+1}(n_{i+1}-1)(K_{i+1}-2G_{i+1}).$$

Например, если совершён перенос вершины u_{i,n_i-1} на вектор G_i в отсутствие смещения K_i , то

$$G_{i+1}=-\frac{n_i}{n_{i+1}}G_i, \quad K_{i+1}=-2\frac{n_i(n_i+n_{i+1}-2)}{n_{i+1}(n_{i+1}-1)}G_i.$$

Если же была смещена точка $u_{i+1,1}$ на вектор G_{i+1} при нулевом K_{i+1} , то

$$G_i=-\frac{n_{i+1}}{n_i}G_{i+1}, \quad K_i=-2\frac{n_{i+1}(n_i+n_{i+1}-2)}{n_i(n_i-1)}G_{i+1};$$

◇ векторы переноса управляющих точек u_{i,n_i-2} и $u_{i+1,2}$ связаны условием $n_i(n_i-1)K_i=n_{i+1}(n_{i+1}-1)K_{i+1}$, позволяющим по одному вектору найти другой:

$$K_{i+1}=\frac{n_i(n_i-1)}{n_{i+1}(n_{i+1}-1)}K_i, \quad K_i=\frac{n_{i+1}(n_{i+1}-1)}{n_i(n_i-1)}K_{i+1};$$

◇ смещения точек $u_{i3} \div u_{i,n_i-3}$ кластера порядка $n_i \geq 6$ выполняются *свободно* и не влияют на расположение других точек;

◇ если степень i -го сегмента равна четырём (см. рис. 2.26,в), то смещение средней точки u_{i1} , управляющей обеими краевыми кривизнами, приводит к зависимым смещениям управляющих точек $i-1$ -го и $i+1$ -го кластеров. Цепочки зависимых смещений обрываются по достижении сегментов со степенями полиномов выше четвёртой.

Анализируя рис. 2.51,б, отметим высокую чувствительность управляющих точек $\{u_{i,n_i-2}, u_{i+1,2}\}$ к малым смещениям точек $\{u_{i,n_i-1}, u_{i+1,1}\}$.

На рис. 2.52 построен экземпляр составной линии Безье, включающей шесть сегментов порядков $n_1=n_2=5$, $n_3=6$, $n_4=4$, $n_5=1$ и $n_6=3$. Порядки гладкости сопряжений сегментов $k_0=k_3=k_4=k_5=0$, $k_1=1$ и $k_2=2$ обозначены в кружках рядом с узловыми точками. Благодаря такому разнообразию выбора удалось создать в узле P_1 точку перегиба, а в узлах P_3 , P_4 и P_5 — точки излома составной линии. Это невозможно сделать при жёстком задании второго порядка гладкости сопряжений во всех внутренних узлах ХП.

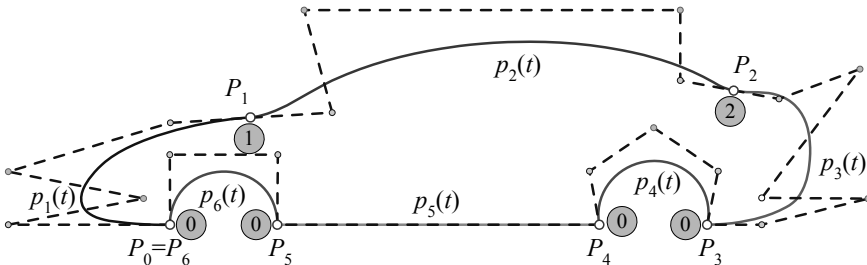


Рис. 2.52

Разработанная методика позволяет реализовать широкий спектр желаний конструктора в самых разных САПР в машиностроении, архитектуре, дизайне... Иллюстрацией сказанного служат составные линии на рис. 2.53, построенные с помощью программы, разработанной бакалавром Дарьей Арзамаскиной.

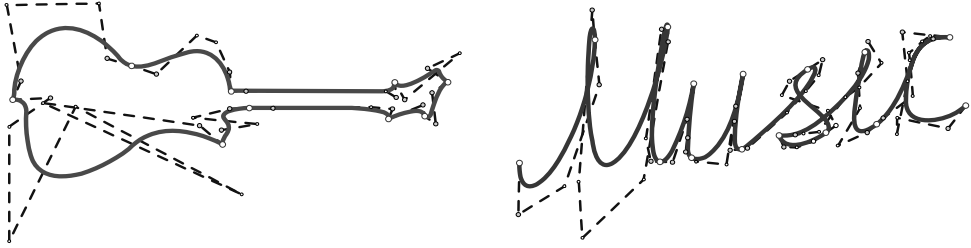


Рис. 2.53

В заключение изучим **три задачи изменения сегментов составной линии Безье**. Каждую из этих задач рассмотрим в двух вариантах: с произвольным и алгоритмическим изменением параметров сегмента [37].

Решение первой задачи отвечает на вопрос: *влияет ли изменение порядка сегмента $p_i(t)$, происходящее при добавлении или удалении вершин его кластера $P_{i-1}u_{i1}...u_{i,n_i-1}P_i$, на соседние сегменты $p_{i-1}(t)$ и $p_{i+1}(t)$* ? Если влияет, то программа построения составной линии должна уметь распространять изменения кластеров по «принципу домино» в обе стороны от редактируемого сегмента вплоть до самых крайних. В результате форма составной линии полностью изменится, что весьма нежелательно.

Допустим, порядок i -го кластера увеличивается до $n'_i = n_i + 1$. Если это происходит аналогично (2.42а) путём произвольной вставки новой вершины между существующими (например, u'_{i1} между $P_{i-1} \equiv u_{i0}$ и $u_{i1} \equiv u'_{i2}$), то из-за изменения длины касательного вектора (2.39г)

$$V'_i(0) = n'_i(u'_{i1} - P_{i-1}) = (n_i + 1)(u'_{i1} - P_{i-1}) \neq V_i(0) = n_i(u_{i1} - P_{i-1})$$

нарушается условие гладкости сопряжения $V'_i(0) = V_{i-1}(1)$ (в силе остаётся лишь нормированная гладкость), а из-за обнуления нового вектора ускорения $A'_i(0) = 0$ (три точки P_{i-1} , u'_{i1} и u'_{i2} коллинеарны) исчезает двойная гладкость сопряжения $A'_i(0) = A_{i-1}(1)$. Последнее условие непрерывности кривизны нарушается и при вставке новой вершины между точками u_{i1} и u_{i2} . Восстановление гладкости влечёт смещение вершин кластеров предшествующих сегментов и изменяет форму этой части составной линии (рис. 2.54,а).

Если же увеличение порядка сегмента делать **алгоритмически** по (2.44а), то в новом кластере $P_{i-1}u'_{i1}...u'_{i,n_i-1}u'_{in_i}P_i$ произойдет перераспределение всех внутренних вершин **без изменения формы сегмента** $p'_i(t) = p_i(t)$ (рис. 2.54,б). Вершины нового кластера устанавливаются по (2.43) в такие позиции

$$u'_{i1} = \frac{P_{i-1} + n_i u_{i1}}{n_i + 1}, \quad u'_{i2} = \frac{2u_{i1} + (n_i - 1)u_{i2}}{n_i + 1},$$

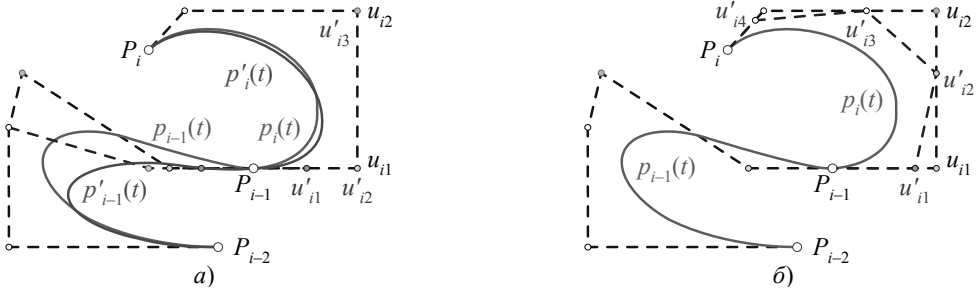


Рис. 2.54

что векторы конечных направлений

$$V'_i(0) = (n_i + 1)(u'_{i1} - P_{i-1}) = P_{i-1} + n_i u_{i1} - (n_i + 1)P_{i-1} = n_i(u_{i1} - P_{i-1}) = V_i(0)$$

и конечных ускорений

$$\begin{aligned} A'_i(0) &= (n_i + 1)n_i(u'_{i2} - 2u'_{i1} + P_{i-1}) = n_i(2u_{i1} + (n_i - 1)u_{i2} - 2(P_{i-1} + n_i u_{i1}) + (n_i + 1)P_{i-1}) = \\ &= n_i(n_i - 1)(u_{i2} - 2u_{i1} + P_{i-1}) = A_i(0) \end{aligned}$$

оказываются **не изменёнными**, как и условия дважды гладкого сопряжения нового сегмента $p'_i(t)$ с предыдущим сегментом $p_{i-1}(t)$.

Аналогичный анализ сопряжения правого конца сегмента $p_i(t)$ с началом следующего сегмента $p_{i+1}(t)$ приводит к выводу, что при алгоритмическом увеличении порядка сегмента сохраняются условия двойной гладкости сопряжения $V'_i(1) = V_{i+1}(0)$ и $A'_i(1) = A_{i+1}(0)$. Ввиду обратимости операций добавления (2.44) и удаления (2.45) можно утверждать, что алгоритмическое понижение порядка сегмента при положительном дефекте степени $n_i - n'_i > 0$ не нарушает гладкости его сопряжения с соседними сегментами.

Таким образом, **алгоритмическое изменение порядка сегмента составной линии носит локальный характер, не влияет на другие сегменты и условия его сопряжения со смежными сегментами.**

□ Целью **второй** задачи является повышение гибкости составной линии и увеличения числа её сегментов путём **разделения кластеров** на части. Пусть $U = u_0 u_1 \dots u_{n-1} u_n$ — разделяемый надвое кластер n -го порядка, а $B = b_0 b_1 \dots b_{v-1} b_v$ и $C = c_0 c_1 \dots c_{\mu-1} c_\mu$ — новые кластеры порядков v и μ с общей точкой $b_v = c_0$. Рассмотрим два способа деления и их свойства.

Ручной разрез кластера U по внутренней вершине u_k с номером $k \in [1, n-1]$ даёт новые кластеры $B = u_0 \dots u_{k-1} u_k$ и $C = u_k u_{k+1} \dots u_n$ порядков $v=k$ и $\mu=n-k$ (рис. 2.55,а). Поскольку линия Безье $p(t) = UH_n T_n(t)$ не проходит через точку u_k , а линии $p_B(t) = BH_v T_v(t)$ и $p_C(t) = CH_\mu T_\mu(t)$ соединяются в ней концами $p_B(1) = p_C(0)$, то форма составной линии после такого деления **изменяется**. Кроме того, из-за произвольности расположения вершин u_{k+1} и u_{k+2} новые кластеры **не удовлетворяют** довольно-таки жёстким условиям гладкого сопряжения (2.83), принимающим в выбранных обозначениях следующий вид:

$$v(u_k - u_{k-1}) = \mu(u_{k+1} - u_k),$$

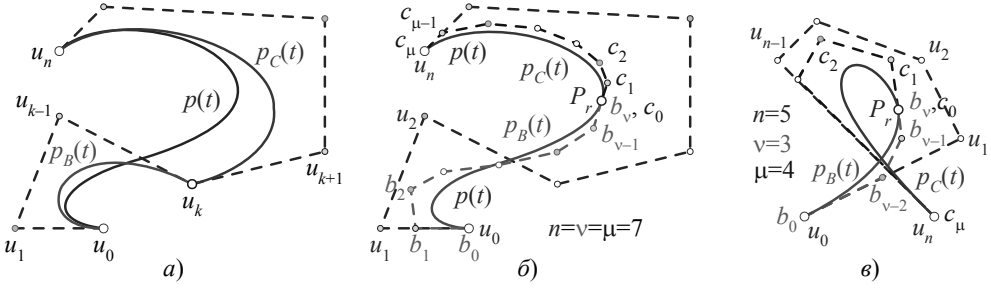


Рис. 2.55

$$v(v-1)(u_k - 2u_{k-1} + u_{k-2}) = \mu(\mu-1)(u_k - 2u_{k+1} + u_{k+2}).$$

В результате сопряжение новых сегментов происходит с **изломом**, т. е. обладает гладкостью (2.4а) **нулевого** порядка.

По **алгоритмическому** методу кластер U заменяется такими кластерами B и C , что составная линия $p_B(t) \cup p_C(t)$ **совпадает** с линией $p(t)$ (рис. 2.55, б, в). Назначим параметр $r \in (0, 1)$ и **точку разделения** $P_r = p(r)$, в которой соединяются концы новых кластеров и сегментов $b_v = c_0 = p_B(1) = p_C(0) = P_r$. Линии трёх сегментов представляют единый полином в разных комбинациях параметра t :

$$p_B(t) = p(rt), \quad p_C(t) = p((1-r)t + r) \quad \forall t \in [0, 1]. \quad (2.85a)$$

Подставив сюда модели сегментов Безье $p_B(t) = BH_v T_v(t)$, $p_C(t) = CH_{\mu} T_{\mu}(t)$ и $p(t) = UH_n T_n(t)$, сведём уравнения связи векторных полиномов к порядкам кластеров B и C по ранее применённой в (2.44б) и (2.45) формуле преобразования порядка ВБФ $T_n(t) = E_{n+1, n'+1} T_{n'}(t)$ (чуть ниже будут обоснованы условия её правомочности):

$$\begin{aligned} BH_v T_v(t) &= UH_n T_n(rt) = UH_n E_{n+1, v+1} T_v(rt), \\ CH_{\mu} T_{\mu}(t) &= UH_n T_n((1-r)t + r) = UH_n E_{n+1, \mu+1} T_{\mu}((1-r)t + r). \end{aligned} \quad (2.85б)$$

Осталось с помощью матрицы из (2.58а) выполнить преобразования ВБФ

$$T_v(rt) = D_v(r, 0) T_v(t), \quad T_{\mu}((1-r)t + r) = D_{\mu}(1-r, r) T_{\mu}(t)$$

из произвольных параметрических шкал $rt \in [0, r]$ и $(1-r)t + r \in [r, 1]$ в нормированную шкалу $t \in [0, 1]$, после чего уравнения (2.85б) становятся разрешимыми относительно векторов B и C :

$$\begin{aligned} BH_v &= UH_n E_{n+1, v+1} D_v(r, 0) \Rightarrow \boxed{B = UH_n E_{n+1, v+1} D_v(r, 0) H_v^{-1}}, \\ CH_{\mu} &= UH_n E_{n+1, \mu+1} D_{\mu}(1-r, r) \Rightarrow \boxed{C = UH_n E_{n+1, \mu+1} D_{\mu}(1-r, r) H_{\mu}^{-1}}. \end{aligned} \quad (2.86)$$

Обоснуем преобразования векторов базисных функций в (2.85б):

- при $v \geq n$ и $\mu \geq n$ замены вектора T_n векторами

$$E_{n+1, v+1} T_v = \begin{bmatrix} T_n \\ O_{v-n, 1} \end{bmatrix}, \quad E_{n+1, \mu+1} T_{\mu} = \begin{bmatrix} T_n \\ O_{\mu-n, 1} \end{bmatrix}$$

возможны **всегда**, но реальные степени (см. п. 2.1.3) полиномов $p_B(t)$ и $p_C(t)$ останутся равными n — степени разделяемого полинома $p(t)$;

- при выборе порядков $v < n$ или $\mu < n$ (рис. 2.55, в) замены T_n векторами

$$E_{n+1, v+1} T_v = \begin{bmatrix} T_v \\ O_{n-v, 1} \end{bmatrix}, \quad E_{n+1, \mu+1} T_\mu = \begin{bmatrix} T_\mu \\ O_{n-\mu, 1} \end{bmatrix}$$

равносильны лишь когда реальная степень полинома $p(t)$ не больше v или μ , а дефект степени $d \geq n - v$ или $d \geq n - \mu$.

Неравенство длин параллельных касательных векторов $v|b_v - b_{v-1}|$ и $\mu|c_1 - c_0|$, хорошо заметное на рис. 2.55, б, означает, что после разделения линии Безье новые сегменты обладают лишь **нормированной** (2.4б) гладкостью сопряжения друг с другом, а также с окружающими старыми сегментами. Моделирование выявило следующие зависимости векторов скорости и ускорения сегментов в точке деления:

$$(1-r)V_B(1) = rV_C(0), \quad (1-r)^2 A_B(1) = r^2 A_C(0). \quad (2.87)$$

Отсюда видно, что **только при выборе параметра разделения $r=0.5$ сопряжение новых сегментов становится полностью дважды гладким** по (2.4а).

Пример 2.8. Разделить на две части линию Безье из примера 2.5, построенную на характеристической полилинии с матрицей вершин $U = \begin{bmatrix} 10 & 22 & 19 & 11 & 8 & 20 \\ 0 & 6 & 12 & 15 & 12 & 0 \end{bmatrix}$ (рис. 2.56).

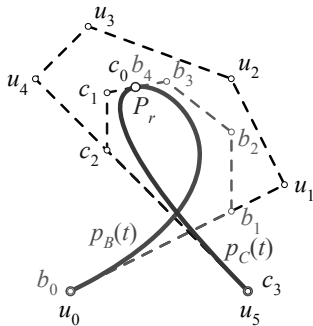


Рис. 2.56

Решение. Назначим коэффициент деления $r=0.6$ и с помощью матрицы H_5 из (2.40г) вычислим точку разделения линии Безье формальной степени $n=5$:

$$P_r = UH_5 T_5(r) = [13.6 \ 11.52].$$

По числу последних нулевых столбцов матрицы коэффициентов полинома $p(t) = S_5 T_5(t)$

$$S_5 = UH_5 = \begin{bmatrix} 10 & 60 & -150 & 100 & 0 & 0 \\ 0 & 30 & 0 & -30 & 0 & 0 \end{bmatrix}$$

определяем дефект степени полинома $d=2$ и его реальную степень $n-d=3$. Выберем степени новых полиномов $v=4$ и $\mu=3$. Обратные матрицы H_4^{-1} и H_3^{-1} даны в примере 2.4.

Сформировав по (2.58а) матрицы

$$D_4(0.6, 0) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0.6 & 0 & 0 & 0 \\ 0 & 0 & 0.36 & 0 & 0 \\ 0 & 0 & 0 & 0.216 & 0 \\ 0 & 0 & 0 & 0 & 0.1296 \end{bmatrix}, \quad D_3(0.4, 0.6) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.6 & 0.4 & 0 & 0 \\ 0.36 & 0.48 & 0.16 & 0 \\ 0.216 & 0.432 & 0.288 & 0.064 \end{bmatrix},$$

вычислим по (2.86) матрицы вершин кластеров разделённых сегментов

$$B = UH_5 E_{6,5} D_4(0.6, 0) H_4^{-1} = \begin{bmatrix} 10 & 19 & 19 & 15.4 & 13.6 \\ 0 & 4.5 & 9 & 11.88 & 11.52 \end{bmatrix}, \quad C = UH_5 E_{6,4} D_3(0.4, 0.6) H_3^{-1} = \begin{bmatrix} 13.6 & 16 & 22 & 10 \\ 11.52 & 12 & 6 & 0 \end{bmatrix}$$

и составим их полиномиальные модели:

$$p_B(t) = B H_4 T_4(t) = \begin{bmatrix} 10 + 36t - 54t^2 + 21.6t^3 \\ 18t - 6.48t^3 \end{bmatrix}, \quad p_C(t) = C H_3 T_3(t) = \begin{bmatrix} 13.6 - 4.8t + 4.8t^2 + 6.4t^3 \\ 11.52 - 0.96t - 8.64t^2 - 1.92t^3 \end{bmatrix}.$$

Анализируя пары векторов двух производных в точке сопряжения

$$V_B(1)=4(b_4-b_3)=[-7.2 \ -1.44] \text{ и } V_C(0)=3(c_1-c_0)=[-4.8 \ -0.96],$$

$$A_B(1)=12(b_4-2b_3+b_2)=[21.6 \ -38.88] \text{ и } A_C(0)=6(c_2-2c_1+c_0)=[9.6 \ -17.28],$$

убеждаемся в их параллельности, но различии длин в $r/(1-r)=1.5$ и $r^2/(1-r)^2=2.25$ раз. Это подтверждает, что при делении линии Безье с коэффициентом $r \neq 0.5$ полученные сегменты обладают лишь нормированной двойной гладкостью сопряжения. $\square$

$\square$ Цель *третьей* задачи — **объединение двух кластеров** $B=b_0b_1\dots b_v$ и $C=c_0c_1\dots c_\mu$ порядков v и μ с общими точками $b_v=c_0$ в кластер $U=u_0u_1\dots u_n$ n -го порядка. Рассмотрим два способа выполнения этой операции и их свойства.

Простое соединение полилиний B и C по их общей вершине $b_v=c_0$ в кластер $U=b_0\dots b_v c_1\dots c_\mu$ порядка $n=v+\mu$ даёт сегмент $p(t)=UH_nT_n(t)$, **не совпадающий** с линиями $p_B(t)=BH_vT_v(t)$ и $p_C(t)=CH_\mu T_\mu(t)$, т. к. точка u_v не является концевой вершиной кластера U (рис. 2.57,а). Благодаря вхождению цепочек $b_0b_1b_2$ и $c_{\mu-2}c_{\mu-1}c_\mu$ в объединение $U=B \cup C$ условия сопряжения линии $p(t)$ с окружающими её сегментами **сохраняются**.

По второму **алгоритмическому** способу кластеры B и C заменяются такой полилинией U , что построенный на ней сегмент Безье $p(t)$ точно совпадает с составной линией $p_B(t) \cup p_C(t)$ (см. рис. 2.55,б,в). По сути эта задача обратна рассмотренной выше задаче разделения кластера. Её решение может быть полезно при редактировании составных линий Безье для отмены операции разделения сегмента.

Если в задаче разделения параметр r задавался произвольно, то в обратной задаче объединения его нужно каким-то образом вычислить. Поскольку линии $p_B(t)$ и $p_C(t)$ сопряжены гладко, хотя бы нормированно, то в точке сопряжения векторы их направлений $V_B(1)=v(b_v-b_{v-1})$ и $V_C(0)=\mu(c_1-c_0)$ **коллинеарны**. Их длины связаны между собой первым соотношением в (2.87), из которого находится коэффициент разделения

$$r=1/\left(1+\frac{\mu|c_1-c_0|}{v|b_v-b_{v-1}|}\right). \quad (2.88a)$$

При выполнении условия (2.83а) равенства векторов $V_B(1)=V_C(0)$ в (2.88а) будет получено значение $r=0.5$. Любое другое число означает не полную, а лишь нормированную гладкость сопряжения линий $p_B(t)$ и $p_C(t)$.

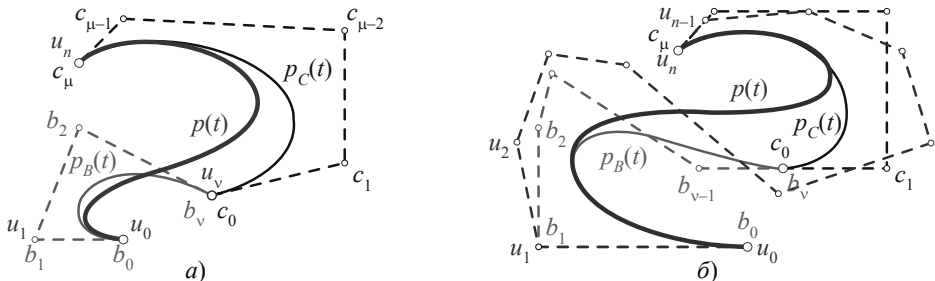


Рис. 2.57

Замена двух сегментов Безье одним возможна лишь в том случае, когда все эти линии представляют один и тот же полином $p(t)$. Именно из таких свойств (2.85а) и получены зависимости (2.86) между тремя кластерами U , B и C . Здесь же нужно проверить способность кластеров B и C быть основами разных частей единого сегмента $p_B(t) \cup p_C(t)$ без привлечения неизвестного пока кластера U . Приведа соотношения (2.85б) к его порядку n

$$BH_{\nu}E_{\nu+1,n+1}T_n(t)=UH_nD_n(r,0)T_n(t), \quad CH_{\mu}E_{\mu+1,n+1}T_n(t)=UH_nD_n(1-r,r)T_n(t),$$

получим, благодаря невырожденности матрицы (2.58а), **тест возможности объединения сегментов $p_B(t)$ и $p_C(t)$ в один сегмент $p(t)$:**

$$BH_{\nu}E_{\nu+1,n+1}D_n(r,0)^{-1}=CH_{\mu}E_{\mu+1,n+1}D_n(1-r,r)^{-1}. \quad (2.88б)$$

Из тех же соотношений находится вектор вершин объединяющего кластера:

$$U=BH_{\nu}E_{\nu+1,n+1}D_n(r,0)^{-1}H_n^{-1}. \quad (2.88в)$$

Если полилинии B и C не удовлетворяют условию (2.88б), то будет получен кластер линии Безье, не проходящей по линиям $p_B(t)$ и $p_C(t)$ (рис. 2.57,б).

В (2.88б,в) можно использовать свойство $D_n(m,s)^{-1}=D_n(m^{-1},-sm^{-1})$.

Совершённые выше подстановки $T_{\nu}(t)=E_{\nu+1,n+1}T_n(t)$ и $T_{\mu}(t)=E_{\mu+1,n+1}T_n(t)$ равносильны **всегда** при выборе порядка объединённого сегмента $n \geq \max\{\nu, \mu\}$. Наименьшее значение n , при котором задача объединения при условии (2.88б) имеет точное решение (2.88в), ограничено **наибольшей** реальной степенью полиномов $p_B(t)$ и $p_C(t)$.

Пример 2.9. Объединить **по возможности** сегменты $p_B(t)$ и $p_C(t)$ порядков $\nu=4$ и $\mu=3$, показанные на рис. 2.56, в одну линию Безье **минимального** порядка (рис. 2.58).

Решение. В примере 2.8 показано, что реальные степени дважды гладко сопряжённых сегментов $p_B(t)$ и $p_C(t)$ равны трём. Если возможность объединения этих сегментов существует, то минимальная степень полинома $p(t)$ также будет составлять $n=3$.

Используя найденное там же соотношение длин касательных векторов в точке сопряжения $|V_B(1)|/|V_C(0)|=1.5$, вычислим по (2.88а) параметр деления $r=(1+2/3)^{-1}=0.6$ и проведём тест (2.88б) с обратными матрицами $D_3^{-1}(3/5,0)=D_3(5/3,0)$ и $D_3^{-1}(2/5,3/5)=D_3(5/3,-1)$:

$$\begin{aligned} BH_4E_{5,4}D_3(5/3,0) &= CH_3D_3(5/3,-1) = \\ &= \begin{bmatrix} 10 & 60 & -150 & 100 \\ 0 & 30 & 0 & -30 \end{bmatrix}. \end{aligned}$$

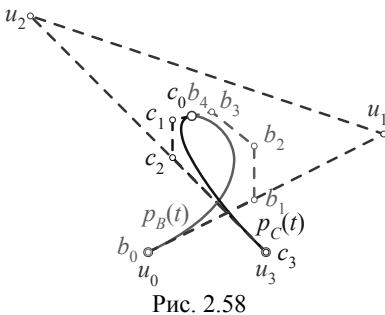
Таким образом, объединение сегментов **возможно**.

Вычислим по (2.88в) матрицу объединяющего кластера и составим модель его сегмента Безье:

$$U=BH_4E_{5,4}D_3(5/3,0)H_3^{-1}=\begin{bmatrix} 10 & 30 & 0 & 20 \\ 0 & 10 & 20 & 0 \end{bmatrix},$$

$$p(t)=UH_3T_3(t)=\begin{bmatrix} 10+60t-150t^2+100t^3 \\ 30t-30t^3 \end{bmatrix}.$$

Отметим, что матрица U точно совпадает с минимальной ХП $u'_0u'_1u'_2u'_3$ на рис. 2.30. $\square$



2.2.4. Рациональные составные линии Безье

Форма **рациональной составной линии Безье** определяется порядками сегментов $n_i \geq 1$, расположением вершин характеристических полилиний кластеров $U_i = [u_{i0} \ u_{i1} \ \dots \ u_{i,n_i}]$ и векторами их весов $\Omega_i = [1 \ \omega_{i1} \ \dots \ \omega_{i,n_i-1} \ 1] \ \forall i=1, N$. Модель i -го рационального сегмента объединяет модели (2.48б,в) и (2.82):

$$p_i(t) = \frac{\sum_{j=0}^{n_i} b_{n_i,j}(t) \omega_{ij} u_{ij}}{\sum_{j=0}^{n_i} b_{n_i,j}(t) \omega_{ij}} = \frac{(\tilde{U}_i \bullet \Omega_i) B_{n_i}(t)}{\Omega_i B_{n_i}(t)} \quad \forall t \in [0, 1]. \quad (2.89)$$

Составная линия строится в интервале **сквозного** параметра $\theta \in [0, N]$, по которому находятся номер сегмента $i = \lceil \theta \rceil + \{\theta = 0\}$ и локальный параметр $t = \theta + 1 - i$.

Граничные свойства рациональной линии Безье установлены в (2.49) и используются для обеспечения желаемых порядков непрерывности и гладкости сопряжения рациональных сегментов $p_i(t)$ и $p_{i+1}(t)$:

- концепция составной линии позволяет организовать **разрыв** между концевыми точками $u_{in_i} \neq u_{i+1,0}$, т. е. сопряжение с **нулевым** порядком непрерывности;
- для создания **излома** составной линии с **первым** порядком непрерывности сопряжения достаточно совпадения концевых вершин кластеров $u_{in_i} = u_{i+1,0} \equiv P_i$;
- сопряжение **первого** порядка гладкости обеспечивается уравниванием векторов концевых касательных $p'_i(1) = p'_{i+1}(0)$ в точке соединения кластеров:

$$V_i = n_i \omega_{i,n_i-1} (P_i - u_{i,n_i-1}) = n_{i+1} \omega_{i+1,1} (u_{i+1,1} - P_i); \quad (2.90a)$$

- сопряжение сегментов **второго** порядка гладкости обеспечивается уравниванием векторов их концевых ускорений $p''_i(1) = p''_{i+1}(0)$ в точке p_i :

$$\begin{aligned} A_i &= 2n_i \omega_{i,n_i-1} (n_i \omega_{i,n_i-1} - 1) (P_i - u_{i,n_i-1}) + n_i (n_i - 1) \omega_{i,n_i-2} (u_{i,n_i-2} - P_i) = \\ &= 2n_{i+1} \omega_{i+1,1} (n_{i+1} \omega_{i+1,1} - 1) (P_i - u_{i+1,1}) + n_{i+1} (n_{i+1} - 1) \omega_{i+1,2} (u_{i+1,2} - P_i). \end{aligned} \quad (2.90б)$$

Сравнив условия гладкого сопряжения (2.90) и (2.83), понимаем, что конструирование гладких рациональных составных линий более трудоёмко, как и подобные (2.84) соотношения синхронизации перемещения вершин кластеров.

Рассмотрим ряд задач построения рациональных составных линий с сегментами (2.89). По сути всё сводится к расчёту кластерных данных $\{U_i, \Omega_i\}$ и формированию из них списков $U = \{U_1, U_2, \dots, U_N\}$ и $\Omega = \{\Omega_1, \Omega_2, \dots, \Omega_N\}$.

□ Простейшая составная линия — **полилиния** $P_1 P_2 \dots P_n$ — строится из $N = n - 1 \geq 1$ рациональных сегментов с кластерами порядков $n_i = 1$ и векторами

$$U_i = [P_i \ P_{i+1}], \quad \Omega_i = [1 \ 1] \quad \forall i = 1, N. \quad (2.91)$$

Вектор **линейных** весовых функций Бернштейна имеет вид $B_i(t) = [1 - t \ t]^T$, при котором дробно-рациональная функция (2.89) со знаменателем $\Omega B_i(t) = 1$ превращается в полиномиальную $p_i(t) = (1 - t)P_i + tP_{i+1}$.

□ Рассмотрим задачу **рационального скругления угла** abc дугой окружности радиуса r (рис. 2.59), решённую в п. 1.2.3 тригонометрическим способом. Вычислив векторы сторон $V=a-b$ и $W=c-b$, половину угла раствора

$$\gamma = 0.5 \arccos(\bar{V} \circ \bar{W}) \in (0^\circ, 90^\circ) \quad (2.92a)$$

и отложив от точки b на сторонах угла отрезки длиной

$$d = \min\{r \operatorname{ctg}(\gamma), \rho|V|, \rho|W|\}, \quad (2.92б)$$

где $\rho \in [0, 1]$ — **коэффициент скругления** (см. рис. 1.57), получим концы дуги

$$g = b + d\bar{V}, \quad h = b + d\bar{W}. \quad (2.92в)$$

Сформируем **три** кластера со следующими векторами вершин и весов:

$$U_1 = [a \ g], \quad U_2 = [g \ b \ h], \quad U_3 = [h \ c], \quad \Omega_1 = \Omega_3 = [1 \ 1], \quad \Omega_2 = [1 \ \sin(\gamma) \ 1]. \quad (2.92г)$$

Вес угловой вершины $\omega_{21} = \sin(\gamma)$ назначен в соответствии с (2.54б) и рис. 2.35.

Крайние кластеры порядков $n_1=n_3=1$ создают прямолинейные концы угла и для их построения по (2.89) квант параметра Δt задаётся единичным. На среднем кластере порядка $n_2=2$ строится скругляющая дуга $p_2(t)$ с достаточно частыми отсчётами параметра $\Delta t \ll 1$.

В трёх случаях число кластеров может быть равно **двум**:

- при $\{\rho=0\} \vee \{r=0\}$ полилиния abc **не скругляется**, что обеспечивается кластерными векторами (2.91) $U_1=[a \ b]$, $U_2=[b \ c]$ и $\Omega_1=\Omega_2=[1 \ 1]$;
- при $\{\rho=1\} \wedge \{g=a\}$ **начало** дуги совпадает с точкой a , что обеспечивается кластерными векторами $U_1=[g \ b \ h]$, $\Omega_1=[1 \ \sin(\gamma) \ 1]$ и $U_2=[h \ c]$, $\Omega_2=[1 \ 1]$;
- при $\{\rho=1\} \wedge \{h=c\}$ **конец** дуги совпадает с точкой c , что обеспечивается кластерными векторами $U_1=[a \ g]$, $\Omega_1=[1 \ 1]$ и $U_2=[g \ b \ h]$, $\Omega_2=[1 \ \sin(\gamma) \ 1]$.

□ Задача **скругления углов полилинии** $P_1P_2\dots P_n$ решалась в п. 1.2.3 путём логического объединения концевых точек P_1 и P_n с тригонометрическими дугами окружностей, рассчитанными функцией $\arcs$. Разрывы между объединяемыми элементами заполнялись отрезками прямых (см. рис. 1.58,а).

В составной рациональной модели (2.89) i -й сегмент описывается рассчитанной зависимостью $p_i(t)$, по которой линия быстро строится с квантом параметра $\Delta t \leq 1$. Рациональными сегментами скруглённой полилинии являются дуги **окружностей** радиусов $r_1 \div r_n$ (концевые вершины P_1 и P_n углов не имеют,

поэтому радиусы их скругления $r_1=r_n=0$) и, возможно, **отрезков** между дугами (рис. 2.60). Чередование отрезков и дуг вовсе не обязательно:

- при задании $r_i=0$ дуга при вершине P_i отсутствует. Если все $r_1=r_2=\dots=r_n=0$, то составная линия включает только отрезки рёбер полилинии с кластерными векторами (2.91);

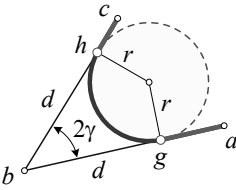


Рис. 2.59

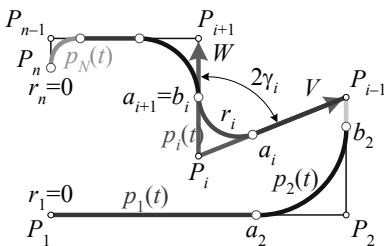


Рис. 2.60

- при больших радиусах r_2 или r_{n-1} крайние дуги могут полностью вписываться в концевые рёбра без образования отрезков;
- на коротких внутренних рёбрах дуги могут стыковаться без отрезков.

С учётом сказанного разработаем алгоритм $rat_line(P, R)$ рационального скругления углов полилинии $P=P_1P_2\dots P_n$ дугами окружностей радиусов $r_i \geq 0$, составляющих список $R=\{r_1, r_2, \dots, r_n\}$ с крайними значениями $r_1=r_n=0$.

Работа алгоритма начинается с инициализации пустых списков $U=\Omega=\emptyset$ и установки точки $e=P_1$. Затем при $n \geq 3$ выполняется цикл $i=2, n-1$ обработки примыкающих к вершине P_i рёбер P_iP_{i-1} и P_iP_{i+1} :

- вычисляются их векторы V и W , половина угла раствора γ_i , коэффициенты скругления α и β , точки $a \in P_iP_{i-1}$ и $b \in P_iP_{i+1}$:

$$\begin{aligned} V &= P_{i-1} - P_i, \quad W = P_{i+1} - P_i, \quad \gamma_i = 0.5 \arccos(\overline{V} \circ \overline{W}), \\ \alpha &= \text{if}(r_{i-1} : 0.5, 1), \quad \beta = \text{if}(r_{i+1} : 0.5, 1), \\ d &= \min\{r_i \text{ctg}(\gamma_i), \alpha|V|, \beta|W|\}, \quad a = P_i + d\overline{V}, \quad b = P_i + d\overline{W}; \end{aligned} \quad (2.93a)$$

- при несовпадении точек e и a в списки U и Ω добавляются кластерные векторы *отрезка* ea :

$$[e \ a] \rightarrow U, \quad [1 \ 1] \rightarrow \Omega; \quad (2.93б)$$

- при $r_i \neq 0$ в списки U и Ω добавляются кластерные векторы *дуги* $\overrightarrow{ab}$:

$$[a \ P_i \ b] \rightarrow U, \quad [1 \ \sin(\gamma_i) \ 1] \rightarrow \Omega; \quad (2.93в)$$

- запоминается конечная точка сегмента $e=b$.

В заключение при несовпадении точек e и P_n в списки U и Ω добавляются кластерные векторы *отрезка* eP_n :

$$[e \ P_n] \rightarrow U, \quad [1 \ 1] \rightarrow \Omega. \quad (2.93г)$$

Функция rat_line возвращает два списка кластерных векторов вершин $U=\{U_1, U_2, \dots, U_N\}$ и весов $\Omega=\{\Omega_1, \Omega_2, \dots, \Omega_N\}$, размер которых $N \geq 1$ равен числу полученных сегментов составной линии.

Непрерывное *сглаживание углов полилинии* $P_1P_2\dots P_n$ дугами эллипсов (рис. 2.61), заключёнными между точками

$$\begin{aligned} e_1 &= P_1, \quad e_{n-1} = P_n, \\ e_i &= 0.5(P_i + P_{i+1}) \quad \forall i = \overline{2, n-2}, \end{aligned} \quad (2.94a)$$

осуществляется путём расчёта углов

$$\gamma_i = 0.5 \arccos(\overline{P_{i-1} - P_i} \circ \overline{P_{i+1} - P_i}) \quad \forall i = \overline{2, n-1} \quad (2.94б)$$

и формирования кластерных векторов

$$U_i = [e_i \ P_{i+1} \ e_{i+1}], \quad \Omega_i = [1 \ \sin(\gamma_{i+1}) \ 1] \quad \forall i = \overline{1, n-2}. \quad (2.94в)$$

□ Отличие *скругления углов полигона* $P=P_1P_2\dots P_nP_1$ от полилинии $P_1P_2\dots P_nP_{n+1}$, пусть даже замкнутой с вершиной $P_{n+1}=P_1$, заключается в существовании угла $P_nP_1P_2$ и необходимости его скругления дугой окружности ра-

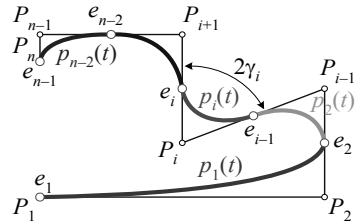


Рис. 2.61

диусом $r_1=0$ (рис. 2.62). Если все значения $r_1=r_2=\dots=r_n=0$, то составная линия включает только отрезки рёбер полигона с $N=n$ кластерными векторами (2.91). Разработаем **алгоритм** $rat_poly(P,R)$ **рационального скругления углов полигона** $P=P_1P_2\dots P_nP_1$ дугами окружностей радиусов $r_i \geq 0$, составляющих список $R=\{r_1, r_2, \dots, r_n\}$.

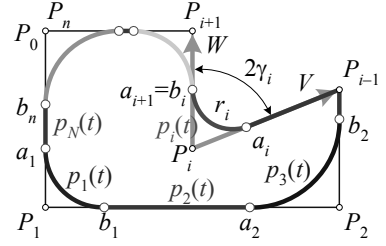


Рис. 2.62

Блок-схема алгоритма приведена на рис. 2.63. Процесс начинается с инициализации пустых списков $U=\Omega=\emptyset$, расширения полигона вершиной $P_0=P_n$ до $P=P_nP_1P_2\dots P_nP_1$, списка радиусов $r_0=r_n$ и $r_{n+1}=r_1$ до $R=\{r_n, r_1, r_2, \dots, r_n, r_1\}$. Затем в цикле $i=\overline{1,n}$ производится обработка рёбер P_iP_{i-1} и P_iP_{i+1} при вершине P_i :

- по (2.93а) вычисляются векторы рёбер V и W , половина угла раствора γ_i , коэффициенты скругления α и β , точки $a \in P_iP_{i-1}$ и $b \in P_iP_{i+1}$;
- при $i=1$ запоминается точка $a_1=a$. Для остальных $i>1$ при $a \neq e$ в списки U и Ω по (2.93б) добавляются кластерные векторы **отрезка** ea ;
- при $r_i \neq 0$ в U и Ω по (2.93в) добавляются кластерные векторы **дуги** $\overline{ab}$;
- запоминается конечная точка сегмента $e=b$.

По окончании цикла при несовпадении точек e и a_1 в списки U и Ω добавляются кластерные векторы **отрезка** ea_1 :

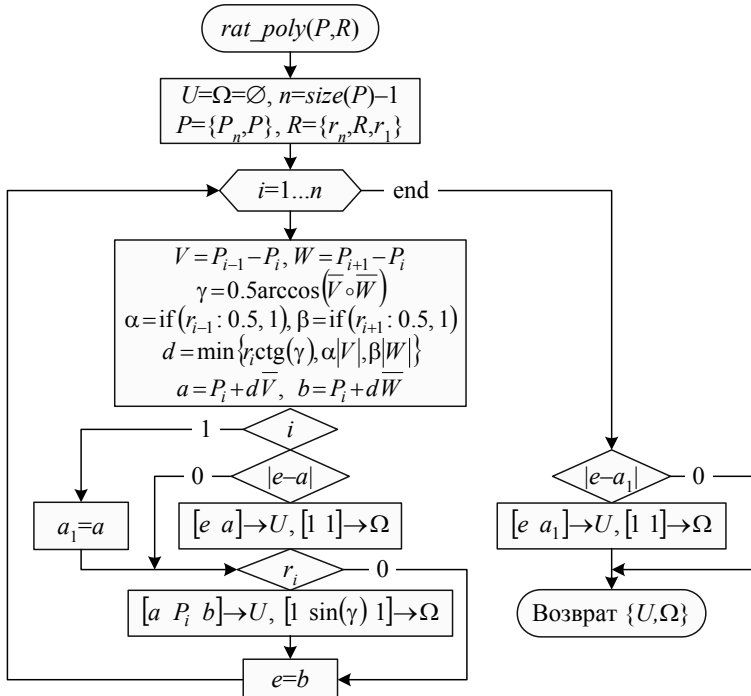


Рис. 2.63

$$[e \ a_1] \rightarrow U, \ [1 \ 1] \rightarrow \Omega. \quad (2.95)$$

Непрерывное **сглаживание углов** **полигона** $P_1 P_2 \dots P_n P_1$ дугами **эллипсов**, заключёнными между точками

$$e_i = 0.5(P_i + P_{i+1}) \quad \forall i = \overline{0, n}, \quad (2.96a)$$

где $P_0 = P_n$ и $P_{n+1} \equiv P_1$ (рис. 2.64), обеспечивается путём расчёта углов

$$\gamma_i = 0.5 \arccos(\overline{P_{i-1} - P_i} \circ \overline{P_{i+1} - P_i}) \quad (2.96б)$$

и формирования n пар кластерных векторов эллипсных дуг $\overline{e_{i-1} e_i}$:

$$U_i = [e_{i-1} \ P_i \ e_i], \ \Omega_i = [1 \ \sin(\gamma_i) \ 1] \quad \forall i = \overline{1, n}. \quad (2.96в)$$

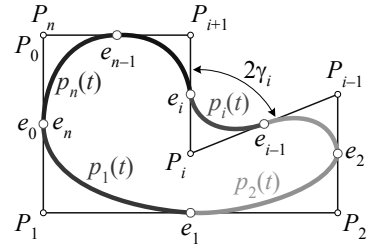


Рис. 2.64

□ Попытки полиномиального приближения **полной окружности** предпринимались нами с самого начала главы. К приближённым моделям, отличающимся близостью к идеальной единичной окружности $x^2(t) + y^2(t) = 1$, относятся очень далёкая от неё интерполяционная линия со свободными концами (см. рис. 2.10), циклическая эрмитова линия (2.26) с дважды гладким замыканием (см. рис. 2.14,в) и два варианта линий Безье (см. рис. 2.27). Точным совпадением с окружностью может похвастать односегментная рациональная модель (2.48) с не совсем простой ХП, показанной на рис. 2.36,в.

Минимальное число **неколлинеарных** характеристических вершин, на которых можно построить рациональную окружность, равно **трьём**! Из (2.54б) и рис. 2.35 следует, что на ХП $u_0 u_1 u_2 u_1 u_0$, разделённой по точке u_2 на два кластера второго порядка с векторами $U_1 = [u_0 \ u_1 \ u_2]$, $\Omega_1 = [1 \ \omega \ 1]$ и $U_2 = [u_2 \ u_1 \ u_0]$, $\Omega_2 = [1 \ -\omega \ 1]$, где $\omega = \sin(0.5 \arccos(\angle u_0 u_1 u_2))$, по (2.89) будет построена рациональная составная линия из двух дважды гладко сопряжённых дуг полной окружности. Наиболее простое расположение характеристических вершин, показанное на рис. 2.36,а, и соответствующее ему значение ω таковы:

$$u_0 = [1 \ 0], \ u_1 = [1 \ 1], \ u_2 = [0 \ 1], \ \omega = 1/\sqrt{2}.$$

Недостаток всех рациональных линий — **неравномерное распределение точек** $p_i(t_k)$ при равномерном квантовании параметра $t_k = k \cdot \Delta t$. Особенно сильно это проявляется на кластерах с отрицательными весами вершин, что подтверждает второй сегмент с весом $-\omega$ на рис. 2.65.

Одинаковое распределение точек на всех сегментах и симметричная ХП достигаются путём составления полной окружности из N равноугольных (угол $\delta = \pi/N$) рациональных дуг (рис. 2.66,а), построенных на кластерах второго порядка с векторами $U_i = [P_{i-1} \ u_{i1} \ P_i]$, $\Omega_i = [1 \ \cos(\delta) \ 1]$ и координатами вершин

$$P_i = [\cos(2i\delta) \ \sin(2i\delta)], \ u_{i1} = \sec(\delta) [\cos(2i\delta - \delta) \ \sin(2i\delta - \delta)]. \quad (2.97a)$$

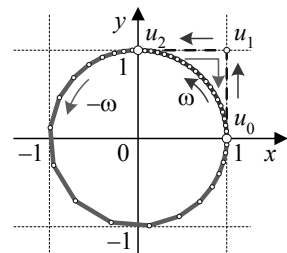


Рис. 2.65

Наиболее экономично окружность составляется из $N=3$ равных дуг с углами $\delta=60^\circ$, весами $\omega=\cos(\delta)=0.5$ и следующими вершинами ХП (рис. 2.66,б):

$$\begin{aligned} P_0=P_3=[1 \ 0], \quad u_{11}=[1 \ \sqrt{3}], \quad P_1=[-1/2 \ \sqrt{3}/2], \\ u_{21}=[-2 \ 0], \quad P_2=[-1/2 \ -\sqrt{3}/2], \quad u_{31}=[1 \ -\sqrt{3}]. \end{aligned} \quad (2.97б)$$

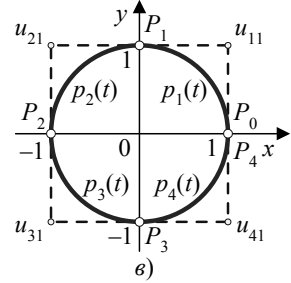
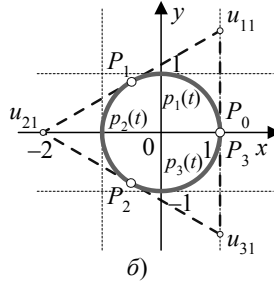
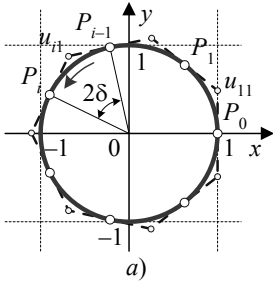


Рис. 2.66

Самый простой квадратный вид (рис. 2.66,в) ХП принимает на $N=4$ кластерах с углами $\delta=45^\circ$, весами $\omega=\cos(\delta)=1/\sqrt{2}$ и координатами вершин

$$\begin{aligned} P_0=P_4=[1 \ 0], \quad u_{11}=[1 \ 1], \quad P_1=[0 \ 1], \quad u_{21}=[-1 \ 1], \\ P_2=[-1 \ 0], \quad u_{31}=[-1 \ -1], \quad P_3=[0 \ -1], \quad u_{41}=[1 \ -1]. \end{aligned} \quad (2.97в)$$

Если же вершины ХП разместить на прямоугольнике в точках

$$\begin{aligned} P_0=P_4=[a \ 0], \quad u_{11}=[a \ b], \quad P_1=[0 \ b], \quad u_{21}=[-a \ b], \\ P_2=[-a \ 0], \quad u_{31}=[-a \ -b], \quad P_3=[0 \ -b], \quad u_{41}=[a \ -b], \end{aligned} \quad (2.97г)$$

то при $\omega=\cos(\delta)=1/\sqrt{2}$ по (2.89) строится составной **эллипс** (рис. 2.67,а).

Изменяя всего лишь веса внутренних вершин кластеров ХП (веса крайних вершин единичны), можно на одной и той же геометрической основе получить разнообразные составные рациональные линии. Примеры таких линий с равными весами $\omega_i \ \forall i=1,4$ показаны на рис. 2.67,б, а с разными значениями весов — на рис. 2.67,в. Тип квадратичного сегмента $p_i(t)$ определяется весом ω_{i1} согласно рис. 2.33: прямая $P_{i-1}P_i$ при $\omega_{i1}=0$, внутренние (в угле $\angle P_{i-1}u_{i1}P_i$) дуги эллипса ($0 < \omega_{i1} < 1$), параболы ($\omega_{i1}=1$), гиперболы ($\omega_{i1} > 1$) или внешняя (вне угла $\angle P_{i-1}u_{i1}P_i$) дуга эллипса при $-1 < \omega_{i1} < 0$. Значения весов $\omega_{i1} \leq -1$ задавать недопустимо из-за разрывности внешних дуг параболы и гиперболы.

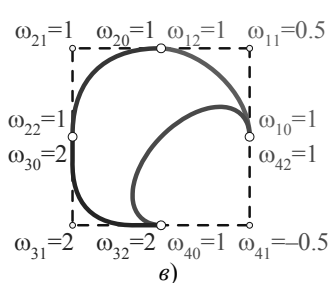
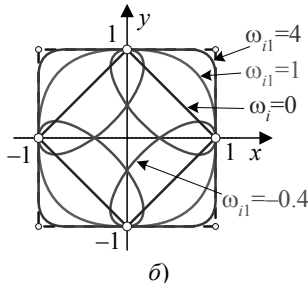
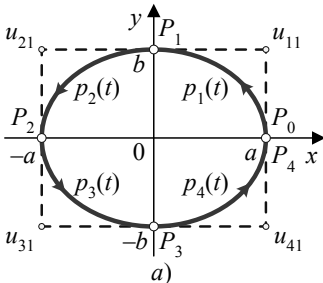


Рис. 2.67

2.3. СПЛАЙНОВЫЕ ЛИНИИ

Распространим изложенную методику полиномиального приближения на задачу дважды гладкого сопряжения сегментов составной линии, которому в (2.4а) соответствует значение $k=2$. Дополнительное свойство непрерывности второй производной кусочно-полиномиальной линии означает согласно (1.110а) непрерывность её кривизны $\kappa(t)$. Создание таких линий очень востребовано на практике. Например, поездка по железнодорожному пути, спроектированному как составная линия с дважды гладким сопряжением рельсов, ощущается комфортной благодаря отсутствию скачков центростремительного ускорения и как следствие — боковых толчков вагонов.

В обширной литературе по компьютерной графике [13, 14, 15, 20, 26, 27, 51, 56, 63, 65] кусочно-полиномиальные линии обобщённо называются «сплайновыми кривыми», хотя термин «сплайн» (*spline*) имеет отношение лишь к одной из этих линий — *кусочно-кубической*. Исторически он происходит от чертёжного инструмента в виде гибкой рейки, которую применяли для рисования гладких кривых, проходящих на плоскости через желаемые опорные точки. Фиксируя рейку в этих точках и давая ей свободно без изломов изогнуться, конструктор получал гладкую линию контура детали. Трёхмерная аналогия сплайна представляет собой упругий стальной или резиновый прут, протянутый с минимальными напряжениями изгибов через последовательность опорных точек.

В разд. 2.2 и [29] рассмотрены общие принципы формирования составных линий путём сопряжения N полиномиальных сегментов $p_i(t)$ с разными порядками m_i и размерами узловых кластеров n_i . В широком смысле их также можно назвать сплайнами, так что любители обобщений и терминологической эклектики вправе считать себя уже знакомыми с принципами построения сплайновых линий. Здесь же и в [32] более обстоятельно изучается сплайн в узком смысле как частный случай кусочно-*кубической* ($m_i=3$) линии, составленной из *одноинтервальных* ($n_i=1$) сегментов на узловой полилинии $P_0P_1\dots P_n$.

Переобозначим в (2.6) векторы базисных функций и их производных в

$$T(t) = [1 \ t \ t^2 \ t^3]^T, \quad T'(t) = [0 \ 1 \ 2t \ 3t^2]^T, \quad T''(t) = [0 \ 0 \ 2 \ 6t]^T \quad (2.98)$$

и запишем параметрическую модель i -го сегмента сплайна в виде

$$p_i(t) = s_{i0} + s_{i1}t + s_{i2}t^2 + s_{i3}t^3 = S_i T(t) \quad \forall t \in [a_i, b_i], \quad i = \overline{1, n}. \quad (2.99)$$

Напомним, что при локальной параметризации $a_i=0$, при сквозной $a_i=b_{i-1}$ ($b_0=0$), при нормализованной $b_i=a_i+1$, а при хордовой $b_i=a_i+|P_i-P_{i-1}|$.

2.3.1. Интерполяция

По законам физики устойчивое состояние механического сплайна, построенного на узловой полилинии $P_0P_1\dots P_n$ (рис. 2.68), достигается при минимуме потенциальной энергии изгиба, пропорциональной *функционалу кривизны*

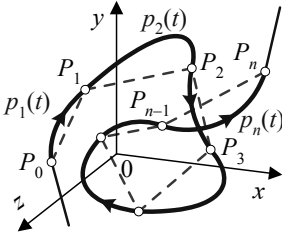


Рис. 2.68

$$\mathfrak{Z}(p) = \sum_{i=1}^n \int_{a_i}^{b_i} |p_i''(t)|^2 dt \rightarrow \min_p.$$

Как показано в [14, 15, 20, 51], решением задачи минимизации является составная интерполяционная линия (2.99) с условиями непрерывного

$$p_i(a_i) = P_{i-1}, \quad p_i(b_i) = P_i \quad \forall i = \overline{1, n} \quad (2.100a)$$

и дважды гладкого

$$p_i'(b_i) = p_{i+1}'(a_{i+1}), \quad p_i''(b_i) = p_{i+1}''(a_{i+1}) \quad \forall i = \overline{1, n-1} \quad (2.100б)$$

сопряжения с краевыми условиями

$$p_1''(a_1) = 0, \quad p_n''(b_n) = 0, \quad (2.100в)$$

означающими нулевую кривизну сплайна на свободных концах крайних сегментов, имеющих прямолинейные экстраполяции до точки P_0 и после точки P_n .

Число неизвестных векторных коэффициентов i -го сегмента, составляющих вектор $S_i = [s_{i0} \ s_{i1} \ s_{i2} \ s_{i3}]$, равно четырём. Общее число неизвестных параметров сплайновой линии равно $4n$. Столько же уравнений относительно векторов $S_1 \div S_n$ необходимо составить для однозначного решения задачи сплайн-интерполяции.

□ **Условия непрерывности** сплайна в узловых точках (2.100a) дают $2n$ **явных** уравнений

$$S_i T(a_i) = P_{i-1}, \quad S_i T(b_i) = P_i \quad \forall i = \overline{1, n}. \quad (2.101a)$$

□ **Условия двойной гладкости** сопряжения сегментов сплайна во внутренних узлах (2.100б) дают ещё $2(n-1)$ **неявных** уравнений

$$S_i T'(b_i) = S_{i+1} T'(a_{i+1}), \quad S_i T''(b_i) = S_{i+1} T''(a_{i+1}) \quad \forall i = \overline{1, n-1}. \quad (2.101б)$$

□ Оставшиеся $4n - 2n - 2(n-1) = 2$ уравнения соответствуют **краевым условиям**, налагаемым на производные сплайна в крайних узлах P_0 и P_n . Их вид варьируется в зависимости от специфики строящейся линии:

- **явные** уравнения **свободного сплайна** с нулевой кривизной концов (2.100в)

$$S_1 T''(a_1) = 0, \quad S_n T''(b_n) = 0; \quad (2.102a)$$

- в **эрмитовом сплайне** с закреплёнными направлениями концов $p_1'(a_1) = V_0$ и $p_n'(b_n) = V_n$ (см. рис. 2.11,а) возникают **явные** уравнения

$$S_1 T'(a_1) = V_0, \quad S_n T'(b_n) = V_n. \quad (2.102б)$$

Если нет известных векторов краевых направлений, то их можно аппроксимировать по (2.19) разностными векторами $V_0 = \gamma_0(P_1 - P_0)$ и $V_n = \gamma_n(P_n - P_{n-1})$, длины которых регулируются коэффициентами γ_0 и γ_n :

• **циклический сплайн** кроме совпадения (вовсе необязательного!) крайних узлов $P_0 = P_n$ удовлетворяет условиям дважды гладкого сопряжения концов $p_1'(a_1) = p_n'(b_n)$, $p_1''(a_1) = p_n''(b_n)$ с **неявными** уравнениями

$$S_1 T'(a_1) = S_n T'(b_n), \quad S_1 T''(a_1) = S_n T''(b_n); \quad (2.102\text{в})$$

• **ациклический сплайн** отличается от циклического противоположными направлениями векторов краевых скоростей и ускорений, что реализуется следующими **неявными** уравнениями:

$$S_1 T'(a_1) = -S_n T'(b_n), \quad S_1 T''(a_1) = -S_n T''(b_n). \quad (2.102\text{г})$$

Могут быть и смешанные краевые условия с соответствующими уравнениями в (2.102). Например, у сплайна со свободным концом в узле P_0 и заданным направлением V_n в узле P_n уравнения краевых условий примут вид

$$S_1 T''(a_1) = 0, \quad S_n T''(b_n) = V_n.$$

Сформируем из уравнений (2.101), (2.102) систему

$$\begin{cases} S_1 A_1 - S_2 B_2 = U_1, \\ \dots\dots\dots \\ S_{n-1} A_{n-1} - S_n B_n = U_{n-1}, \\ S_n C - S_1 D = U_n \end{cases} \quad (2.103\text{а})$$

со следующими матрицами $A_i \div A_n$, $B_i \div B_n$, C , D и $U_1 \div U_n$:

$$\begin{cases} A_i = [T(a_i) \ T(b_i) \ T'(b_i) \ T''(b_i)], \ B_i = [0 \ 0 \ T'(a_i) \ T''(a_i)], \\ C^{(a)} = [T(a_n) \ T(b_n) \ 0 \ T''(b_n)], \ D^{(a)} = [0 \ 0 \ -T''(a_1) \ 0], \\ C^{(б)} = [T(a_n) \ T(b_n) \ 0 \ T'(b_n)], \ D^{(б)} = [0 \ 0 \ -T'(a_1) \ 0], \\ C^{(в)} = C^{(r)} = A_n, \ D^{(в)} = B_1, \ D^{(r)} = -B_1, \\ U_i = [P_{i-1} \ P_i \ 0 \ 0], \ U_n^{(б)} = [P_{n-1} \ P_n \ V_0 \ V_n]. \end{cases} \quad (2.103\text{б})$$

Уравнения (2.101а) образуют первые два столбца всех матриц, уравнения (2.101б) — третий и четвёртый столбцы, кроме матриц C , D и $U_n^{(б)}$, которые заполняются согласно краевым условиям (2.102а – г). Надстрочные индексы ^(а), ^(б), ^(в) и ^(r) соответствуют вариантам (2.102) формирования матриц C , D и U_n .

При $n=1$ уравнение $S_1(C-D)=U_1$ имеет решение $S_1=U_1(C-D)^{-1}$.

Рассмотрим два подхода к решению системы уравнений (2.103) при $n>1$.

□ Сформировав из элементов (2.103б) блочные конструкции

$$Q = \begin{bmatrix} A_1 & 0 & \dots & 0 & -D \\ -B_2 & A_2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & A_{n-1} & 0 \\ 0 & 0 & \dots & -B_n & C \end{bmatrix} \in \mathbb{R}^{4n \times 4n}, \quad U = [U_1 \ U_2 \ \dots \ U_n], \quad (2.104\text{а})$$

получим уравнение с полностью определённой матрицей и решением (1.6б) относительно блочного вектора коэффициентов $S = [S_1 \ S_2 \ \dots \ S_n]$:

$$SQ = U \Rightarrow S = UQ^{-1}. \quad (2.104\text{б})$$

Особенности обращаемой матрицы Q :

• **большой размер** $4n \times 4n$, всё-таки меньший размера $12n \times 12n$, который получился бы при группировке скалярных неизвестных в виде поэлементного, а не блочного вектора S ;

- **сильная разреженность**, т. е. относительно большое содержание нулей, из-за чего при решении СЛАУ выполняется много холостых операций;

- **блочно-ленточная структура**, позволяющая применить некоторые оптимизирующие приёмы снижения трудоёмкости расчёта параметров сплайна. Один такой приём рассматривается далее.

□ Благодаря невырожденности матриц A_i (все их определители $|A_i|=2d_i^3 \neq 0$) трудоёмкий метод (2.104) ретроспективного расчёта неизвестных параметров сплайна $S_1 \div S_n$ с зависимым от n размером матрицы Q может быть заменён их **последовательным** нахождением при постоянном и небольшом (4×4) размере всех обрабатываемых матриц по следующему двухшаговому **рекуррентному алгоритму расчёта параметров кубического сплайна** [28, 32].

📁 **Шаг 1.** Получение векторов L_i и матриц M_i в выражениях $S_i = L_i + S_n M_i$:

$$L = O_4, \quad M = E_4, \quad \left\{ L = (U_i + L B_{i+1}) A_i^{-1}, \quad M = M B_{i+1} A_i^{-1} \quad \forall i = \overline{n-1, 1} \right\}. \quad (2.105a)$$

Шаг 2. Вычисление векторов коэффициентов сплайна:

$$S_n = (U_n + L D)(C - M D)^{-1}, \quad S_i = (U_i + S_{i+1} B_{i+1}) A_i^{-1} \quad \forall i = \overline{n-1, 1}. \quad (2.105b)$$

□

Алгоритм легко программируется и характеризуется малыми затратами памяти и **линейной** зависимостью времени расчёта параметров сплайна от размера узловой сетки в отличие от метода (2.104), трудоёмкость которого квадратично зависит от значения n .

Итак, здесь разработаны два варианта алгоритма сплайн-интерполяции, вычисляющего ВПК кубических полиномов сегментов напрямую по узловым точкам, в отличие от описанного в большинстве источников двухступенчатого метода, который сначала поэлементно рассчитывает векторы узловых направлений V_i , а лишь затем находит ВПК сплайна S_i .

В различных вариантах параметризации интервалов возможны упрощения матричных блоков (2.103б), ускоряющие расчёт параметров сплайна:

- при локальной параметризации все начальные параметры;
- при нормализованной параметризации все конечные параметры $b_i = a_i + 1$.

Таким образом, в простейшем варианте локальной нормализованной параметризации ($a_i=0$, $b_i=1$) матрицы A , B , C и D становятся **числовыми** и не зависят от индексов i . Это позволяет единожды вычислить матрицы

$$A^{-1} = \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -3 & 1 \\ 0 & -2 & 3 & -1 \\ 0 & 0.5 & -1 & 0.5 \end{bmatrix}, \quad B A^{-1} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & -2 & 3 & -1 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (2.106)$$

и применять их в (2.105) в интерактивном процессе конструирования сплайновой кривой с подвижными узлами интерполяции.

По алгоритму (2.105) рассчитаны и на рис. 2.69 построены три пары сплайновых линий с разными краевыми условиями (2.102):

- плоские сплайны (а) со свободными концами и неравномерными узлами отличаются друг от друга параметризацией: у составной линии 1 она нормализованная, а линия 2 построена с использованием хордовой параметризации;
- у трёхмерных нормализованных сплайнов (б) фиксированы векторы краевых направлений $V_0 = \gamma_0(P_1 - P_0)$ и $V_7 = P_7 - P_6$, заставляющие линии выходить из начального и входить в конечный узлы по касательным к отрезкам P_0P_1 и P_6P_7 . Линия 3 построена при $\gamma_0=1$, а сравниваемая с ней линия 4 — при коэффициенте $\gamma_0=10$, задающем на порядок бóльшую начальную скорость движения точки $p(t)$ с сильным разгоном, торможением и возвращением к узловой точке P_1 ;
- из двух замкнутых сплайновых линий (в) линия 5 является циклической с дважды гладким сопряжением концов в узле $P_0=P_8$ по условиям (2.102в), тогда как линия 6 построена при ациклических краевых условиях (2.102г).

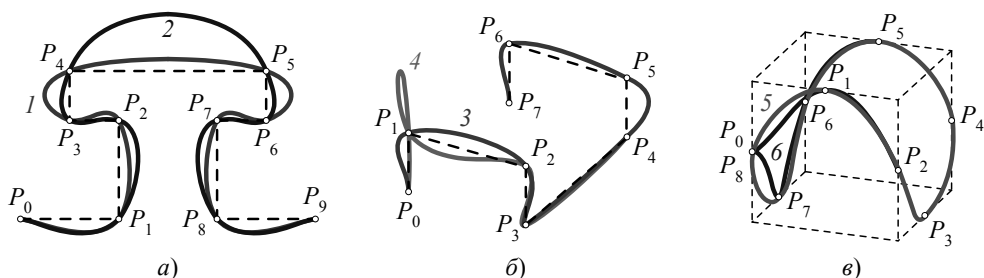


Рис. 2.69

Заметим, что возможен и отличный от (2.102) принцип задания **односторонних краевых условий** сплайна. Он порождает полностью **явные** схемы решения задачи сплайн-интерполяции. Например, если задать краевые условия

$$p'_1(a_1)=V_0, \quad p''_1(a_1)=A_0 \quad (2.107a)$$

с векторами направления V_0 и ускорения A_0 в **начальном** узле P_0 , то получится **последовательная** схема вычисления коэффициентов сегментных полиномов $S_1 \div S_n$ в цикле $i=1, n$:

- составляется уравнение i -го сегмента $S_i Q_i = U_i$ с элементами

$$Q_i = [T(a_i) \ T(b_i) \ T''(a_i) \ T''(b_i)], \quad U_i = [P_{i-1} \ P_i \ V_{i-1} \ A_{i-1}], \quad (2.107б)$$

решение которого даёт ВПК $S_i = U_i Q_i^{-1}$;

- вычисляются векторы конечных производных сегмента

$$V_i = S_i T'(b_i), \quad A_i = S_i T''(b_i), \quad (2.107в)$$

дважды гладко сопрягающие его со следующим сегментом $p_{i+1}(t)$.

Так же, только в обратном направлении $i=n, 1$, решается задача построения сплайновой линии при задании краевых условий в **конечном** узле P_n

$$p'_n(b_n)=V_n, \quad p''_n(b_n)=A_n:$$

- составляется уравнение i -го сегмента $S_i Q_i = U_i$ с элементами $Q_i = [T(a_i) \ T(b_i) \ T'(b_i) \ T''(b_i)]$, $U_i = [P_{i-1} \ P_i \ V_i \ A_i]$,

решение которого даёт ВПК $S_i = U_i Q_i^{-1}$;

- вычисляются векторы начальных производных сегмента $V_{i-1} = S_i T'(a_i)$ и $A_{i-1} = S_i T''(a_i)$, дважды гладко сопрягающие его с сегментом $p_{i-1}(t)$.

Достоинством явных схем является небольшой (4×4) размер обрабатываемых матриц Q_i , которые в локальном нормализованном варианте параметризации одинаковы для всех сегментов. Например, в первой из рассмотренных явных схем (2.107) матрицы Q_i и Q_i^{-1} равны матрицам Q_3 и H_3 из (2.27в).

Недостатки явных схем таятся в накоплении погрешностей последовательного расчёта векторов производных и возрастании осцилляций по мере продвижения от точки задания краевых условий до противоположного края (см. рис. 2.41,б). В неявной же схеме, соответствующей краевым условиям (2.102), разнесённым по разным концам, происходит двустороннее подавление осцилляций, благодаря чему поведение сплайновой линии более контролируемо.

Читатель может приобрести навык конструирования интерполяционных сплайнов решением следующих простых примеров.

Пример 2.10. Построить на трёх узлах $P_0 = [4 \ 0]^T$, $P_1 = [0 \ 3]^T$ и $P_2 = [0 \ 0]^T$ два *свободных* сплайна с локальной нормализованной $p_{\text{лн}}(t)$ и сквозной хордовой $p_{\text{сх}}(t)$ параметризациями, а также *эрмитов* сплайн $p_3(t, \gamma)$ с краевым направлениями $V_0(\gamma) = \gamma [1 \ 1]^T$ и $V_2 = [-2 \ 1]^T$. Проанализировать поведение последнего в зависимости от коэффициента γ .

Решение. Число сегментов всех трёх сплайнов $n=2$. Параметрические интервалы сегментов первого сплайна одинаковы $[a_1 \ b_1] = [a_2 \ b_2] = [0 \ 1]$, а матрицы в (2.103б) следующие:

$$A_1 = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 2 & 2 \\ 0 & 1 & 3 & 6 \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 6 \end{bmatrix}, \quad D = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad U_1 = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}, \quad U_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \end{bmatrix}.$$

Тогда матрица векторных коэффициентов сплайна находится по (2.104б) как

$$S = \begin{bmatrix} 4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 3 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 0 & 0 & 2 & 0 \\ 0 & 1 & 3 & 6 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -2 & 0 & 1 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 6 \end{bmatrix}^{-1} = \underbrace{\begin{bmatrix} 4 & -5 & 0 & 1 & 0 & -2 & 3 & -1 \\ 0 & 4.5 & 0 & -1.5 & 3 & 0 & -4.5 & 1.5 \end{bmatrix}}_{S_1 \quad S_2},$$

а полиномиальные выражения сегментов сплайна (2.99) принимают вид

$$p_{\text{лн}1}(t) = \begin{bmatrix} 4-5t+t^3 \\ 4.5t-1.5t^3 \end{bmatrix}, \quad p_{\text{лн}2}(t) = \begin{bmatrix} -2t+3t^2-t^3 \\ 3-4.5t^2+1.5t^3 \end{bmatrix} \quad \forall t \in [0, 1].$$

Непосредственной проверкой убеждаемся в прохождении сегментов сплайна через узловые точки $p_{\text{лн}1}(0) = [4 \ 0]^T$, $p_{\text{лн}1}(1) = p_{\text{лн}2}(0) = [0 \ 3]^T$, $p_{\text{лн}2}(1) = [0 \ 0]^T$ с дважды гладким сопряжением $p'_{\text{лн}1}(1) = p'_{\text{лн}2}(0) = [-2 \ 0]^T$, $p''_{\text{лн}1}(1) = p''_{\text{лн}2}(0) = [6 \ -9]^T$ и нулевыми краевыми изгибами $p''_{\text{лн}1}(0) = p''_{\text{лн}2}(1) = [0 \ 0]^T$. График первого сплайна построен на рис. 2.70,а.

Параметрические интервалы сегментов второго сплайна определяются длинами отрезков $d_1 = |P_1 - P_0| = 5$, $d_2 = |P_2 - P_1| = 3$ и составляют $[a_1 \ b_1] = [0 \ 5]$, $[a_2 \ b_2] = [5 \ 8]$. Тогда в (2.103б)

$$A_1 = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 5 & 1 & 0 \\ 0 & 25 & 10 & 2 \\ 0 & 125 & 75 & 30 \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 10 & 2 \\ 0 & 0 & 75 & 30 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 5 & 8 & 0 & 0 \\ 25 & 64 & 0 & 2 \\ 125 & 512 & 0 & 48 \end{bmatrix}$$

при тех же матрицах D , U_1 и U_2 . Применим для расчёта коэффициентов сплайна рекуррентный метод (2.105):

$$L = U_1 A_1^{-1} = \frac{1}{125} \begin{bmatrix} 500 & -300 & 60 & -4 \\ 0 & 225 & -45 & 3 \end{bmatrix}, \quad M = B_2 A_1^{-1} = \frac{1}{25} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & -50 & 15 & -1 \\ 0 & -375 & 100 & -5 \\ 0 & -1875 & 375 & 0 \end{bmatrix},$$

$$S_2 = (U_2 + LD)(C - MD) = \frac{1}{60} \begin{bmatrix} 440 & -183 & 24 & -1 \\ -400 & 306 & -48 & 2 \end{bmatrix}, \quad S_1 = L + S_2 M = \frac{1}{300} \begin{bmatrix} 1200 & -315 & 0 & 3 \\ 0 & 330 & 0 & -6 \end{bmatrix}.$$

Тогда полиномиальные выражения сегментов сплайна (2.99) примут вид

$$p_{cx1}(t) = \begin{bmatrix} 4 - 1.05t + 0.01t^3 \\ 1.1t - 0.02t^3 \end{bmatrix} \quad \forall t \in [0, 5], \quad p_{cx2}(t) = \begin{bmatrix} 7.333 - 3.05t + 0.4t^2 - 0.017t^3 \\ -6.667 + 5.1t - 0.8t^2 + 0.033t^3 \end{bmatrix} \quad \forall t \in [5, 8].$$

Читателю рекомендуется вычислительно проверить прохождение сегментов сплайна через все узловые точки с дважды гладким сопряжением и нулевыми краевыми изгибами. График второго сплайна построен на рис. 2.70,б. Заметно его небольшое отличие от нормализованного сплайна, вызванное разными длинами параметрических интервалов $d_1 \neq d_2$.

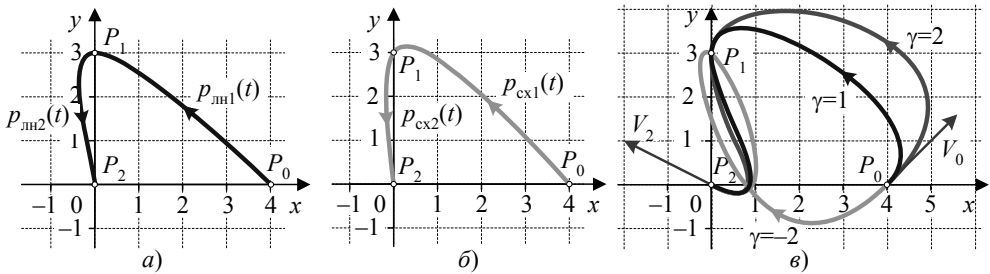


Рис. 2.70

Выберем для построения третьего, эрмитова сплайна локальную хордовую параметризацию на интервалах $[a_1 \ b_1] = [0 \ 5]$, $[a_2 \ b_2] = [0 \ 3]$. Сформируем соответствующие краевым условиям (2.102б) с векторами $V_0(\gamma) = \gamma[1 \ 1]^T$ и $V_2 = [-2 \ 1]^T$ матрицы системы (2.103):

$$A_1 = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 5 & 1 & 0 \\ 0 & 25 & 10 & 2 \\ 0 & 125 & 75 & 30 \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 9 & 0 & 6 \\ 0 & 27 & 0 & 27 \end{bmatrix}, \quad D = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

$$U_1 = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}, \quad U_2(\gamma) = \begin{bmatrix} 0 & 0 & \gamma & -2 \\ 3 & 0 & \gamma & 1 \end{bmatrix}.$$

Рассчитаем матрицы коэффициентов сплайна по рекуррентному алгоритму (2.105):

$$L = U_1 A_1^{-1} = \frac{1}{125} \begin{bmatrix} 500 & -300 & 60 & -4 \\ 0 & 225 & -45 & 3 \end{bmatrix}, \quad M = B_2 A_1^{-1} = \frac{1}{25} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & -50 & 15 & -1 \\ 0 & 125 & -50 & 5 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

$$S_2 = (U_2(\gamma) + LD)(C - MD)^{-1} = \begin{bmatrix} 0 & 0.175 - 0.1875\gamma & 0.55 + 0.125\gamma & -0.2028 - 0.0208\gamma \\ 3 & -0.9125 - 0.1875\gamma & -0.725 + 0.125\gamma & 0.2319 - 0.0208\gamma \end{bmatrix},$$

$$S_1 = L + S_2 M = \begin{bmatrix} 4 & \gamma & -0.515 - 0.3625\gamma & 0.071 + 0.0325\gamma \\ 0 & \gamma & 0.5425 - 0.3625\gamma & -0.0845 + 0.0325\gamma \end{bmatrix}.$$

В результате получаем две параметрические модели сегментов эрмитова сплайна:

$$P_{\Sigma 1}(t, \gamma) = \begin{bmatrix} 4 + \gamma t + (-0.515 - 0.3625\gamma)t^2 + (0.071 + 0.0325\gamma)t^3 \\ \gamma t + (0.5425 - 0.3625\gamma)t^2 + (-0.0845 + 0.0325\gamma)t^3 \end{bmatrix} \quad \forall t \in [0, 5],$$

$$P_{\Sigma 2}(t, \gamma) = \begin{bmatrix} (0.175 - 0.1875\gamma)t + (0.55 + 0.125\gamma)t^2 + (-0.2028 - 0.0208\gamma)t^3 \\ 3 + (-0.9125 - 0.1875\gamma)t + (-0.725 + 0.125\gamma)t^2 + (0.2319 - 0.0208\gamma)t^3 \end{bmatrix} \quad \forall t \in [0, 3].$$

На рис. 2.70, в построены графики этой линии для трёх значений параметра $\gamma \in \{1, 2, -2\}$, изменяющего вектор начального направления $V_0(\gamma)$ в узле P_0 . Очевидное отличие графиков друг от друга свидетельствует о сильной зависимости модели и формы сплайна как от модуля, так и направления краевого вектора $V_0(\gamma)$, который физически характеризует скорость начального движения точки. $\square$

Пример 2.11. Построить сплайновую интерполяцию окружности радиуса $r=1$ с центром в начале координат.

Решение. Зададим на плоскости xu пять узловых точек $P_0 = P_4 = [1 \ 0]^T$, $P_1 = [0 \ 1]^T$,

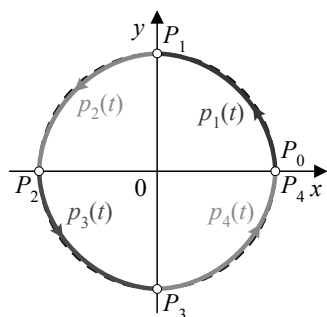


Рис. 2.71

$P_2 = [-1 \ 0]^T$ и $P_3 = [0 \ -1]^T$ (рис. 2.71). Ясно, что проектируемый четырёхсегментный сплайн должен быть **циклическим** с краевыми условиями (2.102в). Ввиду равенства длин хорд между соседними узлами будем строить локальный нормализованный сплайн ($a_i=0$, $b_i=1 \ \forall i=\overline{1,4}$). Используем рекуррентный алгоритм (2.105) с вычисляемыми по (2.103б) (вариант «в») матрицами

$$A=C = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 2 & 2 \\ 0 & 1 & 3 & 6 \end{bmatrix}, \quad B=D = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

готовыми матрицами (2.106) и следующими узловыми матрицами:

$$U_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad U_2 = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad U_3 = -U_1, \quad U_4 = -U_2.$$

На первом проходе (2.105а) находим матрицы

$$L = (U_1 + (U_2 + U_3 A^{-1} B) A^{-1} B) A^{-1} = \begin{bmatrix} 1 & 39 & -69 & 29 \\ 0 & -21 & 39 & -17 \end{bmatrix}, \quad M = (B A^{-1})^3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & -26 & 45 & -19 \\ 0 & 15 & -26 & 11 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

На втором проходе (2.105б) вычисляем коэффициенты полиномов циклического сплайна:

$$S_1 = -S_3 = \begin{bmatrix} 1 & 0 & -1.5 & 0.5 \\ 0 & 1.5 & 0 & -0.5 \end{bmatrix}, \quad S_2 = -S_4 = \begin{bmatrix} 0 & -1.5 & 0 & 0.5 \\ 1 & 0 & -1.5 & 0.5 \end{bmatrix}.$$

Таким образом, сегменты конструируемой сплайновой линии описываются следующими функциями в локальных интервалах изменения параметров $t \in [0, 1]$:

$$p_1(t) = \begin{bmatrix} 1 - 1.5t^2 + 0.5t^3 \\ 1.5t - 0.5t^3 \end{bmatrix}, \quad p_2(t) = \begin{bmatrix} -1.5t + 0.5t^3 \\ 1 - 1.5t^2 + 0.5t^3 \end{bmatrix},$$

$$p_3(t) = \begin{bmatrix} -1+1.5t^2-0.5t^3 \\ -1.5t+0.5t^3 \end{bmatrix} = -p_1(t), \quad p_4(t) = \begin{bmatrix} 1.5t-0.5t^3 \\ -1+1.5t^2-0.5t^3 \end{bmatrix} = -p_2(t).$$

По этим моделям на рис. 2.71 построены линии сегментов сплайна. Для сравнения там же штрихом показана окружность с тригонометрической моделью (1.64). Максимальное отклонение точек сплайна от неё составляет 2%. Такая малая погрешность позволяет использовать кубический сплайн для быстрого вывода как полной окружности, так и по отдельности её четырёх квадрантов, без использования медленно вычисляемых тригонометрических функций.

Альтернативное менее точное односегментное приближение к полной окружности в виде циклического интерполанта 6-го порядка с координатными функциями (2.26) отклонено от окружности максимум на 3.1% её радиуса (см. рис. 2.14,е). $\square$

Пример 2.12. Построить сплайновую модель плоской линии каплевидной формы.

Решение. Назначив узлами интерполяции четвёрку точек $P_0 = P_3 = [0 \ 4]^T$, $P_1 = [1 \ 0]^T$ и $P_2 = [-1 \ 0]^T$, определимся с типом проектируемого сплайна — он должен быть **ациклическим** с краевыми условиями (2.102г) в узлах P_0 и P_3 (рис. 2.72). Для упрощения ручных расчётов выберем локальную нормализованную параметризацию и рекуррентный алгоритм (2.105) с матрицами A , B и C из примера 2.11, матрицей $D = -B$ и узловыми матрицами

$$U_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 4 & 0 & 0 & 0 \end{bmatrix}, \quad U_2 = \begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad U_3 = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \end{bmatrix}.$$

Рассчитаем по (2.105) матрицы коэффициентов сплайна:

$$L = (U_1 + U_2 A^{-1} B) A^{-1} = \begin{bmatrix} 0 & 21 & -33 & 13 \\ 4 & -12 & 12 & -4 \end{bmatrix}, \quad M = (B A^{-1})^2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 7 & -12 & 5 \\ 0 & -4 & 7 & -3 \\ 0 & 0 & 0 & 0 \end{bmatrix},$$

$$S_3 = (U_3 + L D) (C - M D)^{-1} = \begin{bmatrix} -1 & -0.6 & 4.2 & -2.6 \\ 0 & 2.4 & 2.4 & -0.8 \end{bmatrix},$$

$$S_2 = (U_2 + S_3 B) A^{-1} = \begin{bmatrix} 1 & -0.6 & -4.2 & 2.8 \\ 0 & -2.4 & 2.4 & 0 \end{bmatrix}, \quad S_1 = (U_1 + S_2 B) A^{-1} = \begin{bmatrix} 0 & 0 & 3.6 & -2.6 \\ 4 & -4.8 & 0 & 0.8 \end{bmatrix}.$$

В результате получаем желаемую векторную модель сплайновой линии, которую для удобства запишем в интервале изменения **сквозного** параметра $t \in [0, 3]$:

$$p(t) = \begin{cases} \begin{bmatrix} 3.6t^2 - 2.6t^3 & 4 - 4.8t + 0.8t^3 \end{bmatrix}^T & \forall t \in [0, 1]; \\ \begin{bmatrix} 1 - 0.6(t-1) - 4.2(t-1)^2 + 2.8(t-1)^3 & -2.4(t-1) + 2.4(t-1)^2 \end{bmatrix}^T & \forall t \in [1, 2]; \\ \begin{bmatrix} -1 - 0.6(t-2) + 4.2(t-2)^2 - 2.6(t-2)^3 & 2.4(t-2) + 2.4(t-2)^2 - 0.8(t-2)^3 \end{bmatrix}^T & \forall t \in [2, 3]. \end{cases}$$

Для сравнения на рис. 2.72 пунктиром построен **циклический** сплайн с дважды гладким замыканием и краевыми условиями (2.102в). Расчёт его параметрической модели предоставляется читателю в качестве полезного упражнения. $\square$

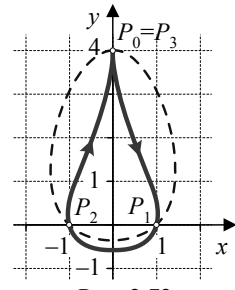


Рис. 2.72

Важной особенностью интерполяционного сплайна является **зависимость ВПК S_i и формы $p_i(t)$ каждого сегмента составной линии от всех возмущающих факторов**, составляющих вектор $F = [P_0 \ P_1 \ \dots \ P_n \ V_0 \ V_n]^T \in V^{m+1}$ (два последних фактора добавляются при эрмитовых краевых условиях (2.102б) и тогда $m = n + 2$, иначе их в векторе F нет и $m = n$). В этом проявляется свойство

глобальности носителя сплайна, в роли которого выступают векторы весовых функций сегментов $L_1(t) \div L_n(t)$, включающие функции Лагранжа и связанные с параметрами сегментов и вектором F аналогичными (2.7) соотношениями

$$p_i(t) = FL_i(t) = S_i T(t) = FH_i T(t). \quad (2.108)$$

Дальнее влияние $k \in [0, m]$ -го возмущающего фактора на форму не смежного с ним сегмента $p_i(t)$ и его ВПК $S_i = [s_{i0} \ s_{i1} \ s_{i2} \ s_{i3}]$ осуществляется через неявные условия сопряжения (2.101б). Численную оценку этого влияния согласно соотношению $S_i = FH_i$ содержит k -я строка $h_i^{(k)}$ весовой матрицы H_i . Векторы коэффициентов S_i рассчитываются по алгоритму (2.105), в котором

$$U_i = [\delta_{i-1}^k \ \delta_i^k \ 0 \ 0] \ \forall i = \overline{1, n-1}, \quad U_n = [\delta_{n-1}^k \ \delta_n^k \ \delta_{n+1}^k \ \delta_{n+2}^k] \ \forall k = \overline{0, m}.$$

Влияние k -го фактора на любую точку i -го сегмента определяет ВФ $l_i^{(k)}(t) = h_i^{(k)} T(t)$. Объединение этих ВФ по всем интервалам $i = \overline{1, n}$ даёт функцию $l_k(t)$ влияния k -го фактора на весь сплайн. Она обладает аналогичными (2.18б) узловыми свойствами $l_k(t_i) = \delta_i^k$, за исключением значений k , соответствующих крайним условиям V_0 и V_n , для которых все $l_k(t_i) = 0$ и $l'_{n+1}(t_i) = \delta_i^0$, $l'_{n+2}(t_i) = \delta_i^n$.

На рис. 2.73,а построено пять из девяти графиков функций влияния $l_0(t) \div l_4(t)$ **нормализованного** сплайна со свободными концами l , созданного на девятиузловой сетке $P_0 \div P_8$ (рис. 2.73,б). Благодаря симметрии расположения узлов остальные функции $l_5(t) \div l_8(t)$ также зеркально симметричны построенным от прямой $t=4$ и для ясности не приведены. Анализ графиков показывает, что влияние k -го возмущающего фактора на форму i -го сегмента убывает примерно в геометрической прогрессии по мере увеличения разности $|t_i - t_k|$, поэтому **реально интерполяционный сплайн имеет достаточно ярко выраженные локальные свойства**, означающие, что перемещение узловой точки влияет на форму лишь немногих сегментов, примыкающих к ней с каждой стороны. Эффект практической локальности сплайна хорошо виден на рис. 2.73,б, где узловые сетки сплайнов 2 и 3 отличаются от сетки сплайна 1 положением единственной узловой точки P_4 .

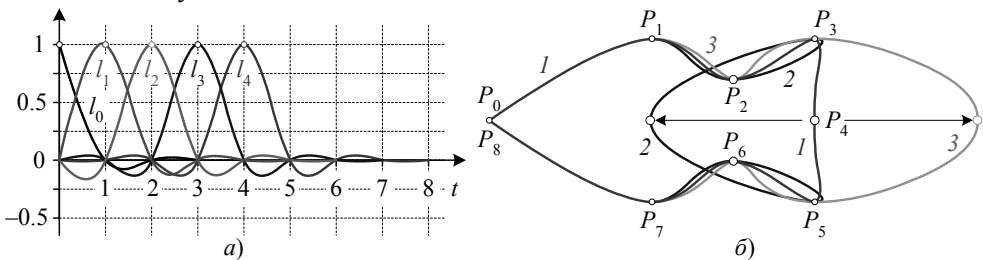


Рис. 2.73

Точно так же влияние крайних условий фактически распространяется лишь на пару ближайших к ним сегментов, в чём красноречиво убеждают сплайны, построенные на рис. 2.69,б,в.

Интересно проявляется свойство практической локальности носителя сплайна при малости длины некоторого интервала $d_i = t_i - t_{i-1}$ в условиях *хордовой* параметризации. Короткий сегмент $p_i(t)$ работает как барьер, препятствующий взаимному влиянию начальных сегментов $p_1(t) \div p_{i-1}(t)$ на конечные сегменты $p_{i+1}(t) \div p_n(t)$ [15]. Это свойство позволяет строить единую сплайновую линию (1 на рис. 2.74,а), имитирующую разрыв кривизны. Нужно окружить точку изгиба двумя близкими узлами интерполяции либо добавить новый узел (P'_2) рядом с узлом в этой точке (P_2), и тогда две части сплайна до короткого сегмента и после него будут практически независимы друг от друга. Построенные на рис. 2.74,б графики функций влияния узлов P_0, P_1, P_3 и P_4 наглядно демонстрируют барьерное свойство сегмента $P_2P'_2$, не пропускающего влияние окружающих сегментов сквозь короткий интервал $[t_2, t'_2]$. Без добавления близкого узла строится совсем другая линия 2 с непрерывной кривизной в точке P_2 , где хотелось бы иметь её разрыв. Данный приём непредсказуемо работает в условиях нормализованной параметризации, доказательством чего служит сплайновая линия 3, имеющая петлю на единичном интервале параметра $t'_2 - t_2 = 1$.

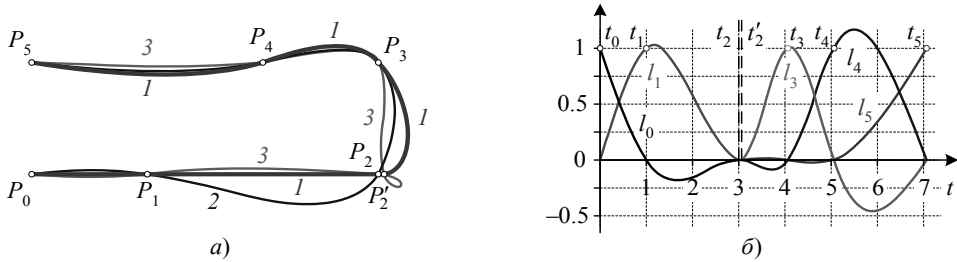


Рис. 2.74

Подведём итоги в форме нескольких **рекомендаций по настройке свойств алгоритма конструирования сплайновых составных линий**:

- выбор между методами локальной и сквозной параметризации теоретически не влияет на форму получаемой линии. Однако, локальная параметризация устойчивее к вычислительным погрешностям из-за меньшего разброса степеней t_i^j в матрицах (2.103б), поэтому её использование предпочтительнее. Сквозной же отсчёт параметра t благодаря непрерывности и монотонности оказывается удобнее в построении составной линии. Таким образом, **расчёт коэффициентов сплайна S_i целесообразно выполнять в локальных шкалах параметра $t \in [0, d_i]$, а отрисовку сплайна по (2.99) делать в сквозной параметризации с восстановлением локальных параметров сегментов по формулам (2.2б);**

- свойство нормализованности сплайна существенно влияет на его форму (см. рис. 2.69,а, 2.74,а), характер движения точки по линии, плотность распределения подузлов и трудоёмкость конструирования. Нормализация с равными длинами интервалов $d_i = 1$ проста в реализации, но при большом разбросе длин дуг сегментов приводит к неравномерному распределению подузлов (см. рис. 2.2,в). Хордовой параметризации с длинами интервалов $d_i = |P_i - P_{i-1}|$ свой-

ственно более равномерное распределение подузлов по разномасштабным сегментам, но она чуть сложнее в программировании. Визуальное качество получаемого решения оценивается весьма субъективно, поэтому многочисленные **рекомендации предпочтительного использования нормализованной параметризации не имеют под собой никаких законных оснований**;

- краевые условия, как видно из рис. 2.69,б,в, влияют в основном лишь на несколько крайних сегментов. Если нет специальных требований к поведению концов сплайна, то можно оставлять их по (2.102а) свободными. Фиксация краевых векторов V_0 и V_n заставляет линию согласно (2.102б) начинаться и заканчиваться в заданных направлениях, а скорости движения в концевых точках регулируются коэффициентами удлинения. Для получения замкнутых сплайновых линий с дважды гладким сопряжением концов должны использоваться циклические краевые условия (2.102в). Наконец, линии с противоположными краевыми направлениями и изгибами строятся на основе краевых условий (2.102г);

- из двух методов решения системы (2.103) относительно ВПК сплайна $S_1 \div S_n$ — ретроспективного (2.104) и рекуррентного (2.105) — следует предпочесть второй ввиду простоты его реализации, независимости затрат памяти и всего лишь линейной зависимости машинного времени от числа интервалов n ;

- интерполяционный сплайн, теоретически обладая свойством глобальности носителя, в реальности проявляет свойство локальности. Оно выражается в уменьшении влияния изменения узловой точки или краевого условия на форму сегмента по мере его удалённости от источника возмущения. В результате область влияния распространяется лишь на малое число сегментов, примыкающих к возмущающему фактору.

2.3.2. Аппроксимация и приближение

Часто в результате экспериментальных измерений положения узловых точек $P_0 \div P_n$ известны лишь приближённо с некоторой максимальной погрешностью $\delta \geq |P_i - v_i| \forall i=0, n$ от их истинных значений $v_0 \div v_n$. Проведение сплайновой линии точно через эти точки сделает её слишком осциллирующей, колеблющейся вокруг желаемой, но неизвестной линии (рис. 2.75). Разработаем **алгоритм построения аппроксимационного кубического сплайна**, проходящего в окрестности узловых точек $P_0 \div P_n$ с дважды гладким сопряжением сегментов и краевыми условиями (2.102) [32].

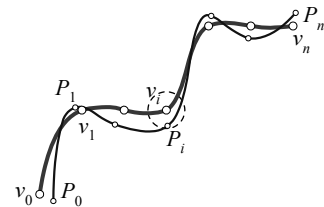


Рис. 2.75

Если в качестве минимизируемого критерия приближения взять аналогичную (2.30а) сумму квадратов невязок узловых точек сегментов

$$\mathfrak{Z} = \sum_{i=0}^n |p_i(t_i) - P_i|^2, \quad (2.109a)$$

то решением задачи синтеза дважды гладкой составной линии с кубическими сегментами будет интерполяционный сплайн со значением $\mathfrak{Z}=0$, поскольку в (2.109а) нет препятствий притяжению узловых то-

чек сплайна к заданным точкам $P_0 \div P_n$, как бы хаотично те ни располагались. Создадим **стимул сглаживания**, уменьшающий кривизну сегментов за счёт увеличения невязок, добавлением в (2.109а) квадратов узловых ускорений:

$$\mathfrak{Z} = \sum_{i=0}^n |p_i(t_i) - P_i|^2 + \sum_{i=0}^n \varphi_i^2 |p_i''(t_i)|^2 \rightarrow \min_S. \quad (2.109б)$$

Штрафные коэффициенты φ_i регулируют компромисс между стимулами приближения ($\varphi_i \rightarrow 0$) и сглаживания ($\varphi_i \rightarrow \infty$).

Следуя методике, изложенной в п. 2.1.2, составим две СЛАУ относительно составного ВПК $S = [S_1 \ S_2 \ \dots \ S_n]$, включающие:

• **интерполяционные** условия дважды гладкого сопряжения сегментов друг с другом и поведения на краях согласно желаемым краевым условиям (2.102):

$$SQ_{\text{и}} = U_{\text{и}}, \text{ где} \\ Q_{\text{и}} = \begin{bmatrix} A_1 & O & \dots & O & -D \\ -B_2 & A_2 & \dots & O & O \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ O & O & \dots & A_{n-1} & O \\ O & O & \dots & -B_n & C \end{bmatrix}, \quad U_{\text{и}} = [O \ O \ \dots \ O \ G], \quad (2.110а)$$

$$\begin{cases} A_i = [T(b_i) \ T'(b_i) \ T''(b_i)], \ B_i = [T(a_i) \ T'(a_i) \ T''(a_i)], \\ C^{(a)} = [O \ T''(b_n)], \ D^{(a)} = [-T''(a_1) \ O], \ G^{(a)} = [O \ O] \\ C^{(b)} = [O \ T'(b_n)], \ D^{(b)} = [-T'(a_1) \ O], \ G^{(b)} = [V_0 \ V_n] \\ C^{(b)} = C^{(r)} = A_n, \ D^{(b)} = B_1, \ D^{(r)} = -B_1, \ G^{(b)} = G^{(r)} = [O \ O \ O] \end{cases}$$

• **аппроксимационные** условия сегментного приближения $S_i T(a_i) \rightarrow P_{i-1}$, $S_i T(b_i) \rightarrow P_i$ и сглаживания $\varphi_i S_i T''(a_i) \rightarrow O$, $\varphi_i S_i T''(b_i) \rightarrow O \ \forall i = \overline{1, n}$:

$$SQ_{\text{а}} = U_{\text{а}}, \text{ где} \\ Q_{\text{а}} = \begin{bmatrix} F_1 & \dots & O \\ \vdots & \ddots & \vdots \\ O & \dots & F_n \end{bmatrix}, \quad U_{\text{а}} = [U_1 \ \dots \ U_n], \quad (2.110б)$$

$$F_i = [T(a_i) \ T(b_i) \ \varphi_i T''(a_i) \ \varphi_i T''(b_i)], \quad U_i = [P_{i-1} \ P_i \ O \ O].$$

В общем случае **решение задачи условной аппроксимации** имеет вид

$$S = U_{\text{и}} Q_{\text{и}}^+ + (U_{\text{а}} - U_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}}) (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+, \quad (2.110в)$$

а при отсутствии в краевых условиях фиксированных направлений V_0 или V_n благодаря нулевому вектору $U_{\text{и}}$ упрощается до

$$S = U_{\text{а}} (Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}})^+. \quad (2.110г)$$

Поскольку в (2.110а) матрица $Q_{\text{и}}$ полноранговая, то псевдообратная к ней матрица вычисляется по формуле $Q_{\text{и}}^+ = (Q_{\text{и}}^T Q_{\text{и}})^{-1} Q_{\text{и}}^T$, чего не скажешь о другой матрице $Q_{\text{а}} - Q_{\text{и}} Q_{\text{и}}^+ Q_{\text{а}}$, ранг которой может быть неполным, а псевдообращение

должно выполняться известными методами разложений, например, ортогонального либо сингулярного [24].

Влияние значений φ_i в критерии (2.109б) на форму сплайна следующее:

- при всех $\varphi_i \rightarrow 0$ сглаживающий стимул исчезает и сплайн превращается в интерполяционный;
- увеличение отдельного штрафа φ_i заставляет сегмент $p_i(t)$ быть менее изогнутым и оказывает спрямляющий эффект на смежные с ним сегменты;
- в пределе при всех $\varphi_i \rightarrow \infty$ сплайн вырождается в прямую составную линию с нулевой кривизной каждого сегмента.

Следуя принципам формирования матриц интерполяционной (2.110а) и аппроксимационной (2.110б) систем уравнений, можно построить **приближающие сплайны** с любыми желаемыми свойствами в узловых точках. Те уравнения состояния, скорости или ускорения, которые должны решиться точно, сводятся в интерполяционную систему $SQ_u = U_u$, а те, что допускают приближённое решение, — в аппроксимационную систему $SQ_a = U_a$. Например, фиксация концов приближающего сплайна в узловых точках P_0 и P_n приводит к следующим изменениям матриц систем (2.110а,б):

- к матрице C добавляется блок $[O \ T(b_n)]$;
- к матрице D добавляется блок $[-T(a_1) \ O]$;
- к вектору G добавляется узловой блок $[P_0 \ P_n]$;
- из матриц F_1 и F_n удаляются столбцы $T(a_1)$ и $T(b_n)$;
- из векторов U_1 и U_n удаляются элементы P_0 и P_n .

Если в ходе интерактивного конструирования потребуется изменить тип какого-либо уравнения, то соответствующие элементы исключаются из старых матриц и добавляются в новые. Состав элементов аппроксимационной системы можно и не изменять, а лишь варьировать элементы интерполяционной системы, поскольку решение последней не увеличивает значения критерия (2.109б).

Работа алгоритма сплайнового приближения иллюстрируется на рис. 2.76. Слева построены аппроксимационные, а справа — приближающие сплайны, концы которых зафиксированы в узлах P_0 и P_9 : линии 1 при всех $\varphi_i=0$, линии 2 при $\varphi_i=1$, линии 3 при $\varphi_i=10$ и линии 4 при $\varphi_i=50$. Формы линий полностью подтверждают высказанные выше соображения о роли штрафных коэффициентов, гибко управляющих стимулами приближения и сглаживания для достижения желаемых свойств сплайновой линии.

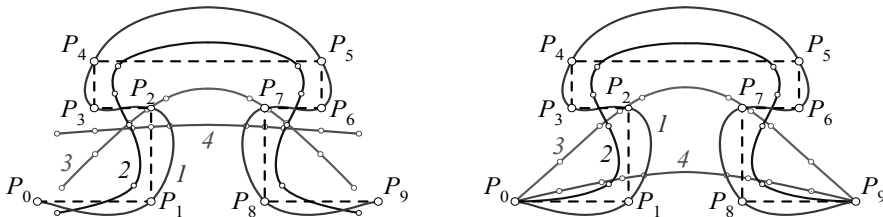


Рис. 2.76

2.3.3. Сплайны с конечными носителями

Кусочно-полиномиальное представление позволяет создать составную линию с *конечным носителем*, у которой влияние узловой точки P_i распространяется только на *конечное* число ближайших сегментов в отличие от кубического сплайна с глобальным носителем, в котором весовая функция каждого узла распределена по всему параметрическому интервалу, хотя и быстро затухает при удалении от соответствующего узлового параметра (см. рис. 2.73,а).

Свойство конечности носителя делает сплайн удобным инструментом интерактивного конструирования сложных линий, меняющего их форму лишь в ближней окрестности перемещаемого узла. Далее понятие «сплайн» будет расширено на кусочно-полиномиальные линии с сегментами *произвольной*, а не только третьей, степени.

Для лучшего понимания свойств конечного носителя выберем сквозную параметризацию n сегментов составной линии, согласно которой узловые отсчёты параметра t вычисляются как $t_i = t_{i-1} + d_i \quad \forall i = \overline{1, n}$, где параметрическая длина i -го интервала в нормализованном методе равна $d_i = 1$, а в хордовом $d_i = |P_i - P_{i-1}|$. Сплайны с равными значениями $d_i = d$ для всех интервалов называются *равномерными*, а с разными, соответственно, — *неравномерными*.

Линейный сплайн

Простейший конечный сплайн — полилиния $P_0 P_1 \dots P_n$ (см. рис. 2.41,а), i -й сегмент которой в виде отрезка $P_{i-1} P_i$ является линейной интерполяцией (2.10б) между узловыми точками P_{i-1} и P_i по параметру t и описывается полиномом

$$p_i(t) = P_{i-1} \frac{t_i - t}{d_i} + P_i \frac{t - t_{i-1}}{d_i} \quad \forall t \in [t_{i-1}, t_i], \quad i \in \overline{1, n}. \quad (2.111a)$$

Отсюда следуют параметры трёх аналогичных (2.7) разложений полинома сегмента $p_i(t) = U_i L_i(t) = S_i T_1(t) = U_i H_i T_1(t)$ (мы вернулись к исходному обозначению ВБФ произвольной степени (2.6а)) с узловым вектором $U_i = [P_{i-1} \ P_i]$:

$$L_i(t) = \frac{1}{d_i} \begin{bmatrix} t_i - t \\ t - t_{i-1} \end{bmatrix}, \quad S_i = \frac{1}{d_i} [P_{i-1} t_i - P_i t_{i-1} \quad P_i - P_{i-1}], \quad H_i = \frac{1}{d_i} \begin{bmatrix} t_i & -1 \\ -t_{i-1} & 1 \end{bmatrix}. \quad (2.111б)$$

Для вывода отрезка на экран задаётся оптимальное значение параметрического кванта Δt_i , при котором вычисленные по (2.111а) точки $p_i(t_{i-1} + j \cdot \Delta t_i)$ без пропусков и наложений засвечивают все пиксели отрезка с учётом дискретности их экранных координат. Во всех современных языках программирования есть функции рисования отрезков, работающие по растровому алгоритму **Брезенхэма** [1, 6, 47, 50, 51, 58, 61, 62, 65], качественно решающему эту задачу.

Составим из узловых точек $P_0 \div P_n$ вектор возмущающих факторов $F = [P_0 \ P_1 \ \dots \ P_n]$. Из (2.111а) и рис. 2.77 видно, что смещение какой-либо точки P_k приводит к изменению только примыкающих к ней сегментов:

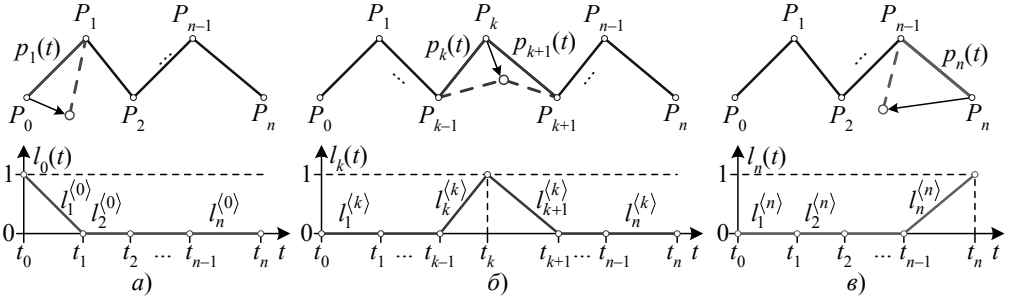


Рис. 2.77

- а) $p_1(t)$ при $k=0$;
 б) $p_k(t)$ и $p_{k+1}(t)$ при $0 < k < n$;
 в) $p_n(t)$ при $k=n$.

Влияние k -го возмущающего фактора на любую точку i -го сегмента в интервале параметра $t \in [t_{i-1}, t_i]$ определяет интервальная весовая функция

$$l_i^{(k)}(t) = \begin{cases} \frac{t_i - t}{t_i - t_{i-1}} & \text{при } k = i-1; \\ \frac{t - t_{k-1}}{t_i - t_{i-1}} & \text{при } k = i; \\ 0 & \text{при остальных } k. \end{cases} \quad (2.112a)$$

С помощью таких ВФ представление i -го сегмента (2.111a) выглядит как

$$p_i(t) = l_i^{(i-1)}(t)P_{i-1} + l_i^{(i)}(t)P_i \quad \forall t \in [t_{i-1}, t_i], \quad i \in \overline{1, n}. \quad (2.112б)$$

В конечности носителя линейного сплайна убеждают построенные под полилиниями графики факторных весовых функций

$$l_k(t) = l_1^{(k)}(t) \cup l_2^{(k)}(t) \cup \dots \cup l_n^{(k)}(t) \quad \forall k = \overline{0, n}, \quad (2.113a)$$

характеризующих влияние возмущающих факторов P_k на весь линейный сплайн. Операция " $\cup$ " обеспечивает на стыке сегментов верные узловые значения $l_k(t_k) = 1$, тогда как арифметическое суммирование интервальных ВФ даёт $l_k(t_k) = 2$. Правильный результат также можно получить ограничением сверху

$$l_k(t) = \min \left\{ \sum_{i=1}^n l_i^{(k)}(t), 1 \right\} \quad \forall k = \overline{0, n}. \quad (2.113б)$$

С помощью вектора факторных ВФ $L(t) = [l_0(t) \ l_1(t) \ \dots \ l_n(t)]^T$ полный сплайн представляется в форме (2.7a)

$$p(t) = \sum_k l_k(t)F_k = FL(t) \quad \forall t \in [t_0, t_n] \quad (2.113в)$$

без сегментации, в отличие от (2.112б), и без необходимости вычисления по

$$(2.2б) \text{ номера сегмента } i = \min \{j\}: t \leq \sum_{k=1}^j d_k.$$

Кубический сплайн с перекрытием параболических сегментов

В п. 2.2.2 были получены формулы (2.79б,г,е) прямого вычисления сегментов составной линии, сформированных линейной интерполяцией параболических линий, поинтервально перекрывающихся друг с другом (см. рис. 2.47). В результате внутренние сегменты получились кубическими, тогда как крайние без задания краевых условий остались параболическими полиномами.

Для получения третьих степеней полиномов крайних сегментов и превращения составной линии в кубический сплайн нужно, очевидно, добавить в каждой концевой точке P_0 и P_n по одному требованию к производным сегментов [44]. Задание векторов направлений V_0 и V_n явно либо разностным методом (2.19) присуще эрмитовым сплайнам. Длины векторов вторых производных параметрической линии A_0 и A_n управляют согласно (1.110а) кривизной на её концах. Задание нулевых значений $A_0=0$ и/или $A_n=0$ эквивалентно созданию сплайна с соответствующими свободными (спрямлёнными) концами.

Используя обозначения из п. 2.2.2, сконструируем на узловой полилинии $P_0P_1\dots P_n$ сегменты кубического сплайна как линейную интерполяцию (2.10а)

$$p_i(t) = (1-t)l_i(t) + tr_i(t) \quad \forall t \in [0, 1], \quad i = \overline{1, n} \quad (2.114а)$$

левого и правого параболических сегментов

$$l_i(t) = L_i T_2(t), \quad r_i(t) = R_i T_2(t). \quad (2.114б)$$

На тех интервалах, где эти сегменты не зависят от краевых условий (рис. 2.78,а), их ВПК вычисляются по (2.13) как

$$\begin{aligned} L_i &= [P_{i-2} \ P_{i-1} \ P_i] [T_2(-1) \ T_2(0) \ T_2(1)]^{-1} = [P_{i-2} \ P_{i-1} \ P_i] \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} = \\ &= \left[P_{i-1} \quad \frac{-P_{i-2} + P_i}{2} \quad \frac{P_{i-2} - 2P_{i-1} + P_i}{2} \right] \quad \forall i = \overline{2, n}, \end{aligned} \quad (2.114в)$$

$$\begin{aligned} R_i &= [P_{i-1} \ P_i \ P_{i+1}] [T_2(0) \ T_2(1) \ T_2(2)]^{-1} = [P_{i-1} \ P_i \ P_{i+1}] \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}^{-1} = \\ &= \left[P_{i-1} \quad \frac{-3P_{i-1} + 4P_i - P_{i+1}}{2} \quad \frac{P_{i-1} - 2P_i + P_{i+1}}{2} \right] \quad \forall i = \overline{1, n-1}. \end{aligned}$$

Тогда из (2.114а,б) следует узловая модель **внутреннего** сегмента $\forall i = \overline{2, n-1}$

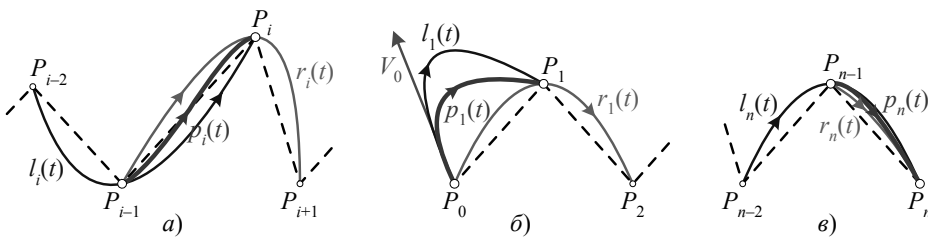


Рис. 2.78

$$\begin{aligned}
p_i(t) &= L_i(1-t)T_2(t) + R_i t T_2(t) = \\
&= \left[P_{i-1} \frac{-P_{i-2} + P_i}{2} \frac{P_{i-2} - 2P_{i-1} + P_i}{2} \right] \begin{bmatrix} 1-t \\ t-t^2 \\ t^2-t^3 \end{bmatrix} + \\
&+ \left[P_{i-1} \frac{-3P_{i-1} + 4P_i - P_{i+1}}{2} \frac{P_{i-1} - 2P_i + P_{i+1}}{2} \right] \begin{bmatrix} t \\ t^2 \\ t^3 \end{bmatrix} = \\
&= P_{i-1} + \frac{-P_{i-2} + P_i}{2} t + \frac{2P_{i-2} - 5P_{i-1} + 4P_i - P_{i+1}}{2} t^2 + \frac{-P_{i-2} + 3P_{i-1} - 3P_i + P_{i+1}}{2} t^3,
\end{aligned} \tag{2.114г}$$

удовлетворяющая условиям интерполяции $p_i(0) = P_{i-1}$, $p_i(1) = P_i$ и совпадающая с (2.79г). Точно такие же коэффициенты кубического полинома $p_i(t) = S_i T_3(t)$ при степенях параметра t получим по (2.79в):

$$\begin{aligned}
S_i &\stackrel{\Rightarrow 1}{=} L_i - L_i + R_i \stackrel{1 \leftarrow}{=} \\
&= \left[P_{i-1} \frac{-P_{i-2} + P_i}{2} \frac{P_{i-2} - 2P_{i-1} + P_i}{2} \text{ O} \right] - \left[\text{O} P_{i-1} \frac{-P_{i-2} + P_i}{2} \frac{P_{i-2} - 2P_{i-1} + P_i}{2} \right] + \\
&+ \left[\text{O} P_{i-1} \frac{-3P_{i-1} + 4P_i - P_{i+1}}{2} \frac{P_{i-1} - 2P_i + P_{i+1}}{2} \right] = \\
&= \left[P_{i-1} \frac{-P_{i-2} + P_i}{2} \frac{2P_{i-2} - 5P_{i-1} + 4P_i - P_{i+1}}{2} \frac{-P_{i-2} + 3P_{i-1} - 3P_i + P_{i+1}}{2} \right].
\end{aligned} \tag{2.114д}$$

В отличие от (2.114в,г) модели параболических $l_1(t)$, $r_n(t)$ и кубических $p_1(t)$, $p_n(t)$ сегментов зависят от краевых условий в точках P_0 и P_n . Например, для **первого** сегмента с начальным направлением V_0 (рис. 2.78,б) ВПК левой параболы $l_1(t)$ приведён в (2.27а), а правой $r_1(t)$ — в (2.114в):

$$\begin{aligned}
L_1 &= [V_0 \ P_0 \ P_1] \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}^{-1} = [P_0 \ V_0 \ -V_0 - P_0 + P_1], \\
R_1 &= [P_0 \ P_1 \ P_2] \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}^{-1} = \left[P_0 \ \frac{-3P_0 + 4P_1 - P_2}{2} \ \frac{P_0 - 2P_1 + P_2}{2} \right].
\end{aligned} \tag{2.115а}$$

Тогда из (2.114а,б) следует узловая модель первого эрмитова сегмента, удовлетворяющая условиям $p_1(0) = P_0$, $p_1(1) = P_1$ и $p_1'(0) = V_0$:

$$\begin{aligned}
p_1(t) &= L_1(1-t)T_2(t) + R_1 t T_2(t) = \\
&= [P_0 \ V_0 \ -V_0 - P_0 + P_1] \begin{bmatrix} 1-t \\ t-t^2 \\ t^2-t^3 \end{bmatrix} + \left[P_0 \ \frac{-3P_0 + 4P_1 - P_2}{2} \ \frac{P_0 - 2P_1 + P_2}{2} \right] \begin{bmatrix} t \\ t^2 \\ t^3 \end{bmatrix} = \\
&= P_0 + V_0 t + \frac{-4V_0 - 5P_0 + 6P_1 - P_2}{2} t^2 + \frac{2V_0 + 3P_0 - 4P_1 + P_2}{2} t^3.
\end{aligned} \tag{2.115б}$$

Что до краевого условия $p_1''(0) = A_0$, то оно, вопреки ожиданиям, не реализуется вычислением вектора L_1 по образу (2.115а) как

$$L_1 = [A_0 \ P_0 \ P_1] \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 2 & 0 & 1 \end{bmatrix}^{-1} = \left[P_0 \ -\frac{A_0}{2} - P_0 + P_1 \ \frac{A_0}{2} \right].$$

Подставив его в (2.115б) вместе с вектором R_1 из (2.115а), получим

$$\begin{aligned} p_1(t) &= L_1 \cdot (1-t)T_2(t) + R_1 t T_2(t) = \\ &= \left[P_0 \ -\frac{A_0}{2} - P_0 + P_1 \ \frac{A_0}{2} \right] \begin{bmatrix} 1-t \\ t-t^2 \\ t^2-t^3 \end{bmatrix} + \left[P_0 \ \frac{-3P_0+4P_1-P_2}{2} \ \frac{P_0-2P_1+P_2}{2} \right] \begin{bmatrix} t \\ t^2 \\ t^3 \end{bmatrix} = \\ &= P_0 + \frac{-A_0-2P_0+2P_1}{2}t + \frac{2A_0-P_0+2P_1-P_2}{2}t^2 + \frac{-2A_0+P_0-2P_1+P_2}{2}t^3 \end{aligned}$$

и ускорение $p_1''(0) = 2A_0 - P_0 + 2P_1 - P_2$. Причина его несовпадения с A_0 в том, что в отличие от (2.114а) вторая производная $p_i''(t)$ не является (докажите это самостоятельно!) линейной интерполяцией $(1-t) \cdot l_i''(t) + t \cdot r_i''(t)$, отсюда $p_i''(0) \neq l_i''(0)!$ Для достижения успеха нужно создать векторы

$$A = (A_0 + P_0 - 2P_1 + P_2)/2,$$

$$\begin{aligned} L_1 &= [A \ P_0 \ P_1] \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 2 & 0 & 1 \end{bmatrix}^{-1} = \left[P_0 \ -\frac{A}{2} - P_0 + P_1 \ \frac{A}{2} \right] = \\ &= \left[P_0 \ \frac{-A_0-5P_0+6P_1-P_2}{4} \ \frac{A_0+P_0-2P_1+P_2}{4} \right]. \end{aligned} \quad (2.115в)$$

Тогда вместе с вектором R_1 из (2.115а) аналогично (2.115б) получим

$$\begin{aligned} p_1(t) &= \left[P_0 \ \frac{-A_0-5P_0+6P_1-P_2}{4} \ \frac{A_0+P_0-2P_1+P_2}{4} \right] \begin{bmatrix} 1-t \\ t-t^2 \\ t^2-t^3 \end{bmatrix} + \\ &+ \left[P_0 \ \frac{-3P_0+4P_1-P_2}{2} \ \frac{P_0-2P_1+P_2}{2} \right] \begin{bmatrix} t \\ t^2 \\ t^3 \end{bmatrix} = \\ &= P_0 + \frac{-A_0-5P_0+6P_1-P_2}{4}t + \frac{A_0+P_0-2P_1+P_2}{2}t^2 + \frac{-A_0+P_0-2P_1+P_2}{4}t^3. \end{aligned} \quad (2.115г)$$

Проверяем: $p_1(0) = P_0$, $p_1(1) = P_1$ и $p_1''(0) = A_0$ — всё правильно! При желании создать в начале сплайна свободный конец задаётся нулевой вектор $A_0 = 0$.

Для **последнего** сегмента $p_n(t)$ с конечным направлением V_n ВПК левой параболы $l_n(t)$ получен в (2.114в), а правой $r_n(t)$ — в (2.276):

$$\begin{aligned} L_n &= [P_{n-2} \ P_{n-1} \ P_n] \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} = \left[P_{n-1} \ \frac{-P_{n-2}+P_n}{2} \ \frac{P_{n-2}-2P_{n-1}+P_n}{2} \right], \\ R_n &= [P_{n-1} \ P_n \ V_n] \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}^{-1} = [P_{n-1} \ -2P_{n-1}+2P_n-V_n \ P_{n-1}-P_n+V_n]. \end{aligned} \quad (2.116а)$$

Тогда из (2.114а,б) следует узловая модель конечного эрмитова сегмента:

$$\begin{aligned}
 p_n(t) &= \left[P_{n-1} \quad \frac{-P_{n-2}+P_n}{2} \quad \frac{P_{n-2}-2P_{n-1}+P_n}{2} \right] \begin{bmatrix} 1-t \\ t-t^2 \\ t^2-t^3 \end{bmatrix} + \\
 &+ \begin{bmatrix} P_{n-1} & -2P_{n-1}+2P_n-V_n & P_{n-1}-P_n+V_n \end{bmatrix} \begin{bmatrix} t \\ t^2 \\ t^3 \end{bmatrix} = \\
 &= P_{n-1} + \frac{-P_{n-2}+P_n}{2}t + (P_{n-2}-3P_{n-1}+2P_n-V_n)t^2 + \frac{-P_{n-2}+4P_{n-1}-3P_n+2V_n}{2}t^3.
 \end{aligned} \tag{2.116б}$$

Сами убедитесь, что $p_n(0)=P_{n-1}$, $p_n(1)=P_n$ и $p'_n(1)=V_n$.

И, наконец, для реализации краевого условия $p''_n(1)=A_n$ создадим векторы

$$\begin{aligned}
 A &= (P_{n-2}-2P_{n-1}+P_n+A_n)/2, \\
 R_n &= \begin{bmatrix} P_{n-1} & P_n & A \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 2 \end{bmatrix}^{-1} = \begin{bmatrix} P_{n-1} & -P_{n-1}+P_n-\frac{A}{2} & \frac{A}{2} \end{bmatrix} = \\
 &= \begin{bmatrix} P_{n-1} & \frac{-P_{n-2}-2P_{n-1}+3P_n-A_n}{4} & \frac{P_{n-2}-2P_{n-1}+P_n+A_n}{4} \end{bmatrix}.
 \end{aligned} \tag{2.116в}$$

Тогда вместе с вектором L_n из (2.116а) аналогично (2.116б) получим

$$\begin{aligned}
 p_n(t) &= \left[P_{n-1} \quad \frac{-P_{n-2}+P_n}{2} \quad \frac{P_{n-2}-2P_{n-1}+P_n}{2} \right] \begin{bmatrix} 1-t \\ t-t^2 \\ t^2-t^3 \end{bmatrix} + \\
 &+ \begin{bmatrix} P_{n-1} & \frac{-P_{n-2}-2P_{n-1}+3P_n-A_n}{4} & \frac{P_{n-2}-2P_{n-1}+P_n+A_n}{4} \end{bmatrix} \begin{bmatrix} t \\ t^2 \\ t^3 \end{bmatrix} = \\
 &= P_{n-1} + \frac{-P_{n-2}+P_n}{2}t + \frac{3P_{n-2}-6P_{n-1}+3P_n-A_n}{4}t^2 + \frac{-P_{n-2}+2P_{n-1}-P_n+A_n}{4}t^3.
 \end{aligned} \tag{2.116г}$$

Проверка $p_n(0)=P_{n-1}$, $p_n(1)=P_n$ и $p''_n(1)=A_n$ подтверждает правильность полученной модели последнего сегмента. При желании создать сплайн со свободным правым концом задаётся нулевой вектор $A_n=O$ (рис. 2.78,в).

Реализация остальных краевых условий, упомянутых в п. 2.3.1, в рассматриваемом сплайне возможна лишь путём явного задания равных либо противоположных векторов краевых направлений и ускорений:

- в циклическом сплайне $V_0=V_n$ и $A_0=A_n$;
- в ациклическом сплайне $V_0=-V_n$ и $A_0=-A_n$.

Полный нормализованный сплайн, фрагменты которого приведены на рис. 2.78, построен на рис. 2.79. Там же пунктирными линиями показано, что изменения узловой точки и краевых векторов распространяются на **конечное** число сегментов, а точки более дальних сегментов остаются неподвижными.

Составим из узловых точек $P_0 \div P_n$ и краевых векторов V_0 и A_n вектор из $n+3$ возмущающих факторов $F=[P_0 \ P_1 \ \dots \ P_n \ V_0 \ A_n]$. На рис. 2.79 видно, что

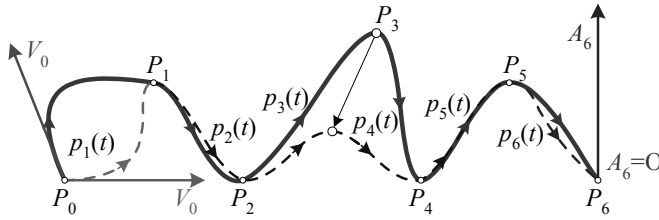


Рис. 2.79

смещение какой-либо узловой точки P_k с номером $k \in [0, n]$ влияет на форму не более чем двух сегментов, примыкающих к ней слева и справа, а изменения факторов $F_{n+1} \equiv V_0$ и $F_{n+2} \equiv A_n$ воздействуют только на крайние сегменты:

- согласно соотношению (2.115б) на **первом** ($i=1$) интервале $t \in [0, 1)$ активны возмущающие факторы $F_0 \div F_2$ и F_{n+1} , откуда же получаем интервальные ВФ:

$$l_1^{(k)}(t) = \begin{cases} 1 - 2.5t^2 + 1.5t^3 & \text{при } k=0; \\ 3t^2 - 2t^3 & \text{при } k=1; \\ -0.5t^2 + 0.5t^3 & \text{при } k=2; \\ t - 2t^2 + t^3 & \text{при } k=n+1; \\ 0 & \text{при остальных } k; \end{cases} \quad (2.117a)$$

- распределив зависимость (2.114г) по узловым точкам, сформируем интервальные ВФ влияния k -х возмущающих факторов на точки **внутренних** i -х интервалов, используя сквозной $t \in [i-1, i)$ и локальный $\tau = t - i + 1 \in [0, 1)$ параметры:

$$l_i^{(k)}(t) = \begin{cases} -0.5\tau + \tau^2 - 0.5\tau^3 & \text{при } k=i-2; \\ 1 - 2.5\tau^2 + 1.5\tau^3 & \text{при } k=i-1; \\ 0.5\tau + 2\tau^2 - 1.5\tau^3 & \text{при } k=i; \\ -0.5\tau^2 + 0.5\tau^3 & \text{при } k=i+1; \\ 0 & \text{при остальных } k; \end{cases} \quad (2.117б)$$

- наконец, на **последнем** ($i=n$) интервале $t \in [n-1, n]$ с локальным параметром $\tau = t - n + 1 \in [0, 1]$, судя по (2.116б), активны факторы $F_{n-2} \div F_n$ и F_{n+2} с интервальными весовыми функциями

$$l_n^{(k)}(t) = \begin{cases} -0.5\tau + \tau^2 - 0.5\tau^3 & \text{при } k=n-2; \\ 1 - 3\tau^2 + 2\tau^3 & \text{при } k=n-1; \\ 0.5\tau + 2\tau^2 - 1.5\tau^3 & \text{при } k=n; \\ -\tau^2 + \tau^3 & \text{при } k=n+2; \\ 0 & \text{при остальных } k. \end{cases} \quad (2.117в)$$

С помощью ВФ (2.117) представление i -го сегмента выглядит смесью

$$p_i(t) = \sum_k l_i^{(k)}(t) F_k \quad \forall t \in [i-1, i], \quad i \in \overline{1, n}. \quad (2.118a)$$

Интервальные ВФ с одинаковым значением k сшиваются в факторные ВФ

$$l_k(t) = \sum_{i=1}^n l_i^{(k)}(t) \quad \forall k = \overline{0, n+2}, \quad (2.118б)$$

характеризующие влияние возмущающих факторов F_k на весь сплайн. На рис. 2.80 построены 9 графиков факторных ВФ (без показа нулевых интервальных ВФ), соответствующих перекрывающемуся сплайну на рис. 2.79. С помощью вектора $L(t)=[l_0(t) \ l_1(t) \ \dots \ l_{n+2}(t)]^T$ полный сплайн представляется в виде

$$p(t)=\sum_{k=0}^{n+2} l_k(t) F_k = FL(t) \quad \forall t \in [0, n]. \quad (2.118\text{в})$$

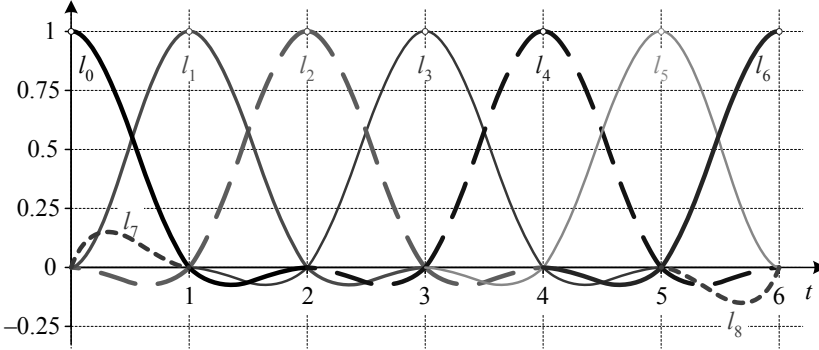


Рис. 2.80

Обобщим тему построением сегментов перекрывающегося сплайна $p_i(t)$ на интервалах $t \in [a_i, b_i]$ с $b_i = a_i + d_i$ и произвольно выбранной параметризацией — локальной ($a_i \neq 0$) либо сквозной ($b_i = a_{i+1}$), нормализованной ($d_i = 1$) либо хордовой ($d = |P_i - P_{i-1}|$), где d_i — длина i -го параметрического интервала.

Обобщением модели (2.114а) является линейная интерполяция (2.10б)

$$p_i(t) = l_i(t) \frac{b_i - t}{d_i} + r_i(t) \frac{t - a_i}{d_i} \quad \forall t \in [a_i, b_i], \quad i = \overline{1, n} \quad (2.119\text{а})$$

левой $l_i(t) = L_i T_2(t)$ и правой $r_i(t) = R_i T_2(t)$ парабол, у которых ВПК независимы от краевых условий (см. рис. 2.78,а) и находятся обобщением (2.114в) как

$$\begin{aligned} L_i &= [P_{i-2} \ P_{i-1} \ P_i] [T_2(a_i - d_{i-1}) \ T_2(a_i) \ T_2(b_i)]^{-1} \quad \forall i = \overline{2, n}, \\ R_i &= [P_{i-1} \ P_i \ P_{i+1}] [T_2(a_i) \ T_2(b_i) \ T_2(b_i + d_{i+1})]^{-1} \quad \forall i = \overline{1, n-1}. \end{aligned} \quad (2.119\text{б})$$

Глядя на (2.119а), запишем обобщающие (2.114д) ВПК кубических сегментов $p_i(t) = S_i T_3(t)$ как частный случай (2.77г):

$$S_i = \frac{\overset{\Rightarrow 1}{b_i} \overset{1 \Leftarrow}{L_i} - \overset{1 \Leftarrow}{L_i} + \overset{\Rightarrow 1}{R_i} - \overset{1 \Leftarrow}{a_i} \overset{\Rightarrow 1}{R_i}}{d_i} \quad \forall i = \overline{1, n}. \quad (2.119\text{в})$$

Модели параболических $l_1(t)$, $r_n(t)$ и кубических $p_1(t)$, $p_n(t)$ сегментов зависят от краевых условий в точках P_0 и P_n . Для **первого** эрмитова сегмента с заданным или вычисленным по (2.19) – (2.21) вектором начального направления V_0 (см. рис. 2.78,б) ВПК левой параболы находится обобщением (2.115а):

$$L_1 = [V_0 \ P_0 \ P_1] [T'_2(a_1) \ T_2(a_1) \ T_2(b_1)]^{-1}. \quad (2.120\text{а})$$

Что касается краевого условия $p_1''(a_1)=A_0$ с заданным ускорением A_0 , то оно реализуется аналогично (2.115в) расчётом вектора

$$L_1=[A \ P_0 \ P_1][T_2''(a_1) \ T_2(a_1) \ T_2(b_1)]^{-1} \quad (2.120б)$$

с таким вектором A , который создаёт в начале сплайна ускорение $F(A)=kA+B$, равное A_0 . Неизвестные скалярный коэффициент k и вектор B этой линейной функции находим по двум выборкам:

- задаём нулевой вектор $A=O$ и по (2.120б) находим вектор $L_1(O)$, а по (2.119в) вектор коэффициентов кубического сегмента $S_1(O)$ и его начальное ускорение $F(O)=S_1(O)T_3''(a_1)$, равное B ;

- задаём единичный вектор $A=x^0$ и по тем же формулам находим новые векторы $L_1(x^0)$, $S_1(x^0)$ и $F(x^0)=S_1(x^0)T_3''(a_1)=kx^0+B$, откуда получаем коэффициент $k=(F(x^0)-B)_x=F(x^0)_x-B_x$.

Теперь из условия $kA+B=A_0$ получаем искомое значение

$$A=\frac{A_0-B}{k}=\frac{A_0-F(O)}{F(x^0)_x-F(O)_x}. \quad (2.120в)$$

Для создания у сплайна свободного левого конца берётся вектор $A_0=O$.

У **последнего** эрмитова сегмента с заданным или вычисленным по (2.19)–(2.21) конечным направлением V_n ВПК правой параболы находится как

$$R_n=[P_{n-1} \ P_n \ V_n][T_2(a_n) \ T_2(b_n) \ T_2'(b_n)]^{-1}. \quad (2.121а)$$

Краевое условие $p_n''(b_n)=A_n$ с заданным ускорением A_n реализуется расчётом аналогичного (2.116в) вектора

$$R_n=[P_{n-1} \ P_n \ A][T_2(a_n) \ T_2(b_n) \ T_2''(b_n)]^{-1} \quad (2.121б)$$

с найденным аналогично (2.120в) вектором A , дающим конечное ускорение A_n :

$$A=\frac{A_n-F(O)}{F(x^0)_x-F(O)_x}. \quad (2.121в)$$

Здесь значение $F(A)$ от произвольного вектора A находится расчётом по (2.121б) и (2.119в) векторов R_n и $S_n(A)$, после чего $F(A)=S_n(A)T_3''(b_n)$. Чтобы получить сплайн со свободным правым концом (см. рис. 2.78,в), нужно задать нулевой вектор $A_n=O$.

Обсудим теперь вычислительную эффективность интерактивного построения сегментов сплайна $p_i(t)$ при вариации узловых точек и краевых условий:

- использование сложных, но зато прямых формул (2.114г), (2.115б,г) и (2.116б,г) оказалось возможным только после их достаточно трудоёмкого вывода и лишь в нормализованном локальном варианте параметризации, что ограничивает их универсальность;

- применение несложных интерполяционных формул (2.114а), (2.119а) либо параметрического разложения $p_i(t)=S_iT_3(t)$ требует пересчёта ВПК L_i , R_i и S_i

при каждом изменении любого возмущающего фактора. Для ускорения отрисовки сплайна можно ценой небольшого усложнения программы пересчитывать не все векторы коэффициентов, а благодаря конечности сплайна лишь зависящие от изменяемого фактора;

- наиболее сложными в выводе (и опять лишь в рамках нормализованной локальной параметризации) являются посегментное (2.118а) и общее (2.118б,в) узловые разложения, требующие тщательного программирования интервальных весовых функций (2.117).

Вспомнив комментарии к бинарному разложению (2.7в), разработаем метод расчёта числовых матриц H_i , с помощью которых интерактивный вывод сегментов по формуле $p_i(t) = U_i H_i T_3(t) \quad \forall i = \overline{1, n}$ происходит максимально быстро. К тому же при использовании нормализованной параметризации с равными значениями $d_i = 1 \quad \forall i = \overline{1, n}$ все эти MBK становятся независимыми от возмущающих факторов.

Дополним операции расширения ВПК нулевыми элементами новыми операциями двустороннего расширения матрицы $H \in \mathbb{R}^{3 \times 3}$ нулевыми строками и столбцами согласно направлениям надсимвольных стрелок:

$$\Downarrow \Rightarrow H = \begin{bmatrix} H & O_{3,1} \\ O_{1,3} & 0 \end{bmatrix}, \quad \Leftarrow \Downarrow H = \begin{bmatrix} O_{3,1} & H \\ 0 & O_{1,3} \end{bmatrix}, \quad \Uparrow \Rightarrow H = \begin{bmatrix} O_{1,3} & 0 \\ H & O_{3,1} \end{bmatrix}, \quad \Leftarrow \Uparrow H = \begin{bmatrix} 0 & O_{1,3} \\ O_{3,1} & H \end{bmatrix}. \quad (2.122)$$

Сформируем из возмущающих факторов (узловых точек $P_0 \div P_n$ и векторов конечных производных, например, V_0 и A_n) такой вектор

$$F = \left[V_0 \overbrace{P_0 \ P_1 \ P_2 \ P_3}^{U_1} \dots \overbrace{P_{n-2} \ P_{n-1} \ P_n \ A_n}^{U_{n-1}} \right],$$

чтобы по нему можно было перемещать слева направо четырёхэлементный

шаблон факторов i -го интервала $U_i = \underbrace{\begin{bmatrix} \bullet & \bullet & \bullet & \bullet \end{bmatrix}}_{U_{Ri}}^{U_{Li}}$, содержащий перекрывающиеся

трёхэлементные узловые векторы U_{Li} и U_{Ri} левой и правой парабол в (2.119а).

Записав векторы коэффициентов этих парабол как $L_i = U_{Li} H_{Li}$ и $R_i = U_{Ri} H_{Ri}$ с элементами

$$U_{Li} = [P_{i-2} \ P_{i-1} \ P_i], \quad H_{Li} = [T_2(a_i - d_{i-1}) \ T_2(a_i) \ T_2(b_i)]^{-1} \quad \forall i = \overline{2, n}, \quad (2.123a)$$

$$U_{Ri} = [P_{i-1} \ P_i \ P_{i+1}], \quad H_{Ri} = [T_2(a_i) \ T_2(b_i) \ T_2(b_i + d_{i+1})]^{-1} \quad \forall i = \overline{1, n-1},$$

получим бинарное разложение полинома i -го сегмента

$$\begin{aligned} p_i(t) &= U_{Li} H_{Li} \frac{b_i - t}{d_i} T_2(t) + U_{Ri} H_{Ri} \frac{t - a_i}{d_i} T_2(t) = \\ &= \frac{1}{d_i} U_i \left(\begin{bmatrix} b_i H_{Li} & O_{3,1} \\ O_{1,3} & 0 \end{bmatrix} - \begin{bmatrix} O_{3,1} & H_{Li} \\ 0 & O_{1,3} \end{bmatrix} + \begin{bmatrix} 0 & O_{1,3} \\ O_{3,1} & H_{Ri} \end{bmatrix} - \begin{bmatrix} O_{1,3} & 0 \\ a_i H_{Ri} & O_{3,1} \end{bmatrix} \right) T_3(t) \end{aligned} \quad (2.123б)$$

со сформированной с помощью операций (2.122) матрицей

$$H_i = d_i^{-1} \left(\overset{\Downarrow \Rightarrow}{b_i H_{Li}} - \overset{\Leftarrow \Downarrow}{H_{Li}} + \overset{\Leftarrow \Uparrow}{H_{Ri}} - \overset{\Uparrow \Rightarrow}{a_i H_{Ri}} \right) \quad \forall i = \overline{1, n}. \quad (2.123в)$$

В рамках локальной ($a_i=0$) нормализованной ($d_{i-1}=d_i=d_{i+1}=b_i=1$) параметризации с присутствующими в (2.114в) матрицами

$$H_{Li} = \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 0 & -0.5 & 0.5 \\ 1 & 0 & -1 \\ 0 & 0.5 & 0.5 \end{bmatrix}, \quad H_{Ri} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & -1.5 & 0.5 \\ 0 & 2 & -1 \\ 0 & -0.5 & 0.5 \end{bmatrix}$$

по (2.123в) получим матрицу

$$\begin{aligned} H_i &= \overset{\Downarrow \Rightarrow}{H_{Li}} - \overset{\Leftarrow \Downarrow}{H_{Li}} + \overset{\Leftarrow \Uparrow}{H_{Ri}} = \\ &= \begin{bmatrix} 0 & -0.5 & 0.5 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & -0.5 & 0.5 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 0.5 & 0.5 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & -1.5 & 0.5 \\ 0 & 0 & 2 & -1 \\ 0 & 0 & -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 0 & -0.5 & 1 & -0.5 \\ 1 & 0 & -2.5 & 1.5 \\ 0 & 0.5 & 2 & -1.5 \\ 0 & 0 & -0.5 & 0.5 \end{bmatrix}. \end{aligned}$$

Элементы моделей крайних сегментов, зависящих от краевых условий, имеют особый вид. Так, для **первого** сегмента с начальным направлением V_0 из (2.120а) следует

$$U_{L1} = [V_0 \ P_0 \ P_1], \quad H_{L1} = [T'_2(a_1) \ T_2(a_1) \ T_2(b_1)]^{-1}.$$

В случае локальной ($a_i=0$) нормализованной ($d_1=b_1=1$) параметризации с присутствующими в (2.115а) матрицами

$$H_{L1} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ 0 & 0 & 1 \end{bmatrix}, \quad H_{R1} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & -1.5 & 0.5 \\ 0 & 2 & -1 \\ 0 & -0.5 & 0.5 \end{bmatrix}$$

по (2.123в) получим матрицу

$$\begin{aligned} H_1 &= \overset{\Downarrow \Rightarrow}{H_{L1}} - \overset{\Leftarrow \Downarrow}{H_{L1}} + \overset{\Leftarrow \Uparrow}{H_{R1}} = \\ &= \begin{bmatrix} 0 & 1 & -1 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 1 & -1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & -1.5 & 0.5 \\ 0 & 0 & 2 & -1 \\ 0 & 0 & -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 0 & 1 & -2 & 1 \\ 1 & 0 & -2.5 & 1.5 \\ 0 & 0 & 3 & -2 \\ 0 & 0 & -0.5 & 0.5 \end{bmatrix}. \end{aligned}$$

Если же в начале сплайна задано ускорение A_0 , то вычислив по (2.120в) вектор A , формируем элементы бинарного разложения левой параболы

$$U_{L1} = [A \ P_0 \ P_1], \quad H_{L1} = [T''_2(a_1) \ T_2(a_1) \ T_2(b_1)]^{-1}.$$

Для **последнего** сегмента с конечным направлением V_n из (2.121б) следует

$$U_{Rn} = [P_{n-1} \ P_n \ V_n], \quad H_{Rn} = [T_2(a_n) \ T_2(b_n) \ T'_2(b_n)]^{-1}.$$

Если же в конце сплайна задано ускорение A_n , то, вычислив по (2.121в) вектор A , формируем элементы бинарного разложения правой параболы

$$U_{Rn} = [P_{n-1} \ P_n \ A], \quad H_{Rn} = [T_2(a_n) \ T_2(b_n) \ T''_2(b_n)]^{-1}.$$

В случае локальной ($a_n=0$) нормализованной ($d_n=b_n=1$) параметризации с матрицами из (2.116а,в)

$$H_{Ln} = \begin{bmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 0 & -0.5 & 0.5 \\ 1 & 0 & -1 \\ 0 & 0.5 & 0.5 \end{bmatrix}, \quad H_{Rn} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 2 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 0.5 \end{bmatrix}$$

по (2.123в) получим матрицу

$$H_n = \overset{\Downarrow \Rightarrow}{H_{Ln}} - \overset{\Leftarrow \Downarrow}{H_{Ln}} + \overset{\Leftarrow \Uparrow}{H_{Rn}} =$$

$$= \begin{bmatrix} 0 & -0.5 & 0.5 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & -0.5 & 0.5 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 0.5 & 0.5 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 0 & -0.5 & 1 & -0.5 \\ 1 & 0 & -2.5 & 1.5 \\ 0 & 0 & 3 & -2 \\ 0 & 0 & -0.5 & 0.5 \end{bmatrix}.$$

Работа алгоритма синтеза перекрывающегося сплайна иллюстрируется на рис. 2.81:

- плоский сплайн 1 со свободными концами построен методом хордовой параметризации на той же неравномерной сетке, что и обычный сплайн 2 (см. рис. 2.69, а), но демонстрирует более плотное прилегание к узловой сети;
- у трёхмерного нормализованного сплайна 3 векторы краевых направлений $V_0=10(P_1-P_0)$ и $V_7=P_7-P_6$ заданы такими же, как у обычного эрмитова сплайна 4 (см. рис. 2.69, б), но отклонения от узловой сетки также уменьшились;
- последний график 5 показывает богатые возможности свободного творчества, предоставляемые разработанным сплайном с конечным носителем.

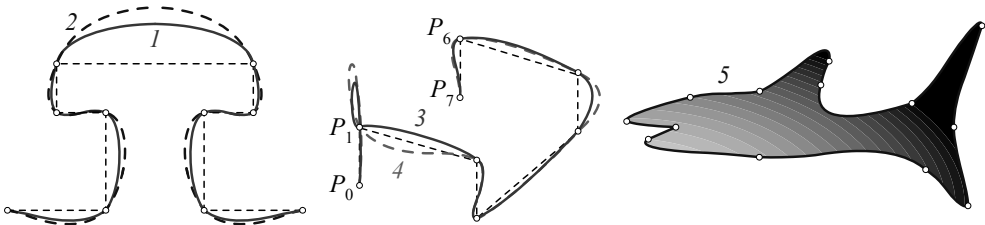


Рис. 2.81

Таким образом, сплайны с перекрытием параболических сегментов показывают лучшее качество интерполирования в сравнении с типовыми кубическими сплайнами при меньшей вычислительной сложности, характеризуемой размерами обрабатываемых матриц — 3×3 вместо $4n \times 4n$ в (2.104) и даже 4×4 в итеративном алгоритме (2.105).

Главным **недостатком** перекрывающегося сплайна является пониженная, но визуально незаметная, степень гладкости сопряжения его сегментов — первая вместо второй в (2.100б). Это означает скачки кривизны в точках сшивки сегментов. С другой стороны, разрывность кривизны сплайновой линии можно обратить на пользу для создания в стыках сегментов точек перегиба и расширения возможностей конструирования сложных линий.

ГЛАВА 3

ГЕОМЕТРИЧЕСКИЕ ПРЕОБРАЗОВАНИЯ

В этой главе рассматриваются вопросы описания движения геометрических объектов на плоскости и в пространстве, а также методы изображения трёхмерных объектов и сцен в различных ракурсах и проекциях. *Цель* изучения данных вопросов — овладение математическим языком синтеза объектов, описания динамики их движения, композиции из них сцен и проецирования на экран дисплея. Профессиональный художник интуитивно чувствует, как нужно изображать объёмный мир на плоском холсте. Реалистичностью экранного изображения компьютерная программа обязана математически строгому расчёту фаз движения объектов на основе закономерностей геометрических преобразований.

Геометрическое преобразование — это отображение $p' = f(p)$ точки $p \in R^n$ n -мерного пространства *образа* в точку $p' \in R^{n'}$ n' -мерного пространства *прообраза*. Геометрические преобразования делятся на *нелинейные* (например, образование тени на неплоской подложке или отражения в кривом зеркале) и *линейные*. Линейное преобразование точки описывается уравнением

$$p' = pA + B \quad (3.1)$$

с *матрицами преобразования* $A \in R^{n \times n'}$ и $B \in R^{n'}$, не зависящими от координат точки p . Из (3.1) очевидно, что матрица B , имеющая фактически структуру вектора, задаёт направление переноса точки в пространстве.

В зависимости от размерности пространств n , n' и свойств матрицы A линейные преобразования делятся (рис. 3.1) на *невырожденные (аффинные)* и *вырожденные (проективные)*. Присущие аффинному преобразованию (АП) свойства $n' = n$ и $\text{rang}[A] = n$ означают квадратность и невырожденность матрицы A . Вытекающее отсюда существование обратной матрицы A^{-1} позволяет по точке прообраза p' восстановить точку образа p :

$$p = (p' - B)A^{-1}. \quad (3.2)$$

В проективных преобразованиях $n' < n$ и не существует обратной матрицы A^{-1} , поэтому **однозначное восстановление образа по его проекции невозможно** из-за потери информации об одной или нескольких координатах.

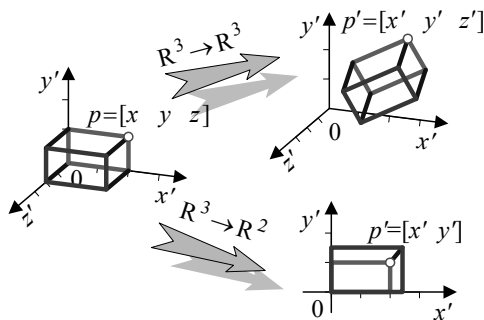


Рис. 3.1

3.1. ΑΦΟΡΟΥΜΕΝΕΣ ΠΡΟΩΡΑΖΟΜΕΝΕΣ

3.1.1. Основные понятия и методы преобразований

Аффинное (от англ. *affinity* — родство) **преобразование** обладает следующими свойствами:

- отображает n -мерный объект в n -мерный — точку в точку, линию в линию, поверхность в поверхность;
- сохраняет параллельность линий и плоскостей;
- сохраняет пропорции параллельных объектов — длин отрезков на параллельных прямых и площадей на параллельных плоскостях.

Этим свойствам удовлетворяет система n линейных уравнений пересчёта n координат **точки** $p=[p_1 \ p_2 \ \dots \ p_n]$ в такое же количество координат точки $p'=[p'_1 \ p'_2 \ \dots \ p'_n]$, записанная в координатной и векторно-матричной формах:

$$\begin{cases} p'_1 = p_1 a_{11} + p_2 a_{21} + \dots + p_n a_{n1} + b_1 \\ p'_2 = p_1 a_{12} + p_2 a_{22} + \dots + p_n a_{n2} + b_2 \\ \vdots \\ p'_n = p_1 a_{1n} + p_2 a_{2n} + \dots + p_n a_{nn} + b_n \end{cases} \Rightarrow \begin{aligned} & p' = pA + B, \text{ где} \\ & A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}, \\ & B = [b_1 \quad b_2 \quad \dots \quad b_n]. \end{aligned} \quad (3.3)$$

Матрица $A \in \mathbb{R}^{n \times n}$ определяет вращательное движение и изменения размеров объекта, тогда как вектор $B \in \mathbb{R}^{n'}$ отвечает за параллельное перемещение его точек. Далее для краткости будем говорить «матрицы преобразования A и B », не забывая о том, что B фактически является вектором. Также будем применять нижний индекс « $\cdot$ » в основном для нумерации разных точек объекта либо положений одной точки в разных состояниях движения, а координаты будем обозначать символами x, y, z в декартовой системе координат и общепринятыми символами в других системах.

Преобразование **вектора** $V = p_2 - p_1$, соединяющего точки p_1 и p_2 ,

$$V' = p'_2 - p'_1 = p_2 A + B - p_1 A - B = (p_2 - p_1)A = VA \quad (3.4)$$

показывает, что переносное движение с вектором B , одинаково действуя на оба конца вектора V , не влияет на его прообраз V' .

Обычно геометрическому преобразованию подвергается не одна, а множество точек объекта $p_1 \div p_m$, и совершается не одно, а цепочка преобразований. Программирование такого процесса связано с организацией вложенных циклов, перебирающих точки объекта и фазы движения. Рассмотрим **методы повышения эффективности программного кода** путём перевода циклов в операции умножения матриц. Примером такого перевода служит замена в (3.3) цикла по координатных вычислений двумя операциями с радиус-вектором p — умножением на матрицу A и сложением результата с вектором B .

□ Независимость коэффициентов матриц A и B от координат точек объекта позволяет рассчитать новые положения точек не последовательно, а параллельно **методом группового преобразования**

$$\begin{bmatrix} p'_1 \\ \dots \\ p'_m \end{bmatrix} = \begin{bmatrix} p_1 \\ \dots \\ p_m \end{bmatrix} A + \begin{bmatrix} B \\ \dots \\ B \end{bmatrix} \quad \text{или} \quad P' = PA + \begin{bmatrix} B \\ \dots \\ B \end{bmatrix}. \quad (3.5)$$

Предварительно нужно собрать аналогично (1.142) координаты m точек объекта в одну матрицу $P \in \mathbb{R}^{m \times n}$ и сформировать $m \times n$ -матрицу из векторов B . Строки матрицы P' содержат координаты преобразованных точек p'_i , которые соединяются друг с другом в том же порядке, как были соединены точки образа, например, вершины полигона на плоскости или полиэдра в пространстве.

□ **Цепочка аффинных преобразований точки**

$$\begin{array}{ccccccc} & A_1, B_1 & A_2, B_2 & & A_N, B_N & & \\ p_0 & \xrightarrow{\quad} & p_1 & \xrightarrow{\quad} & \dots & \xrightarrow{\quad} & p_N \\ & \underbrace{\hspace{10em}}_{A, B} & & & & & \uparrow \end{array}$$

с матрицами $\{A_i, B_i\}$ может быть осуществлена последовательными преобразованиями $p_i = p_{i-1} A_i + B_i$ с сохранением в памяти ЭВМ промежуточных состояний точки $p_1 \div p_{N-1}$. Если эти состояния не будут востребованы в дальнейших расчётах, то можно повысить эффективность алгоритма, найдя **матрицы сквозного преобразования** $p_N = p_0 A + B$ с помощью подстановок

$$\begin{aligned} p_1 &= p_0 A_1 + B_1, \quad p_2 = p_1 A_2 + B_2 = p_0 A_1 A_2 + B_1 A_2 + B_2, \\ p_3 &= p_2 A_3 + B_3 = p_0 A_1 A_2 A_3 + B_1 A_2 A_3 + B_2 A_3 + B_3, \dots, \end{aligned}$$

из которых по индукции получаем

$$\begin{aligned} A &= A_1 A_2 \dots A_N = \prod_{i=1}^N A_i, \\ B &= (\dots (B_1 A_2 + B_2) A_3 + \dots + B_{N-1}) A_N + B_N = \sum_{i=1}^{N-1} B_i \prod_{j=i+1}^N A_j + B_N. \end{aligned} \quad (3.6)$$

Этот алгоритм достаточно трудоёмок в расчёте вектора B .

Цепочка N преобразований **вектора** V_0 как и в (3.4) также не зависит от наличия в этих преобразованиях параллельных переносов:

$$V_N = V_0 \prod_{i=1}^N A_i = V_0 A. \quad (3.7)$$

□ Дальнейшее упрощение и унификация геометрических преобразований достигается, следуя [13, 20, 56], введением **расширенного пространства** (РП) $\mathbb{R}^{n+1}$, в котором $n+1$ -я координата равна фиксированному числу — единице у точки $\tilde{p} \equiv [p \ 1]$ и нулю у вектора $\tilde{V} \equiv [V \ 0]$. Координаты в расширенном пространстве часто называют **однородными**. Наиболее удачное, по мнению автора, объяснение этому спорному термину приведено в [64].

Преобразование (3.1) точки в РП принимает вид

$$[p' \ 1] = [p \ 1] \underbrace{\begin{bmatrix} A & O \\ B & 1 \end{bmatrix}}_{\substack{n \\ 1}} \}^n \quad \text{или} \quad \boxed{\tilde{p}' = \tilde{p} C} \quad (3.8)$$

с расширенной блочной матрицей преобразования

$$C = \begin{bmatrix} A & O_{n,1} \\ B & 1 \end{bmatrix} \in R^{(n+1) \times (n+1)}. \quad (3.9a)$$

Обратное преобразование точки, равносильное (3.2), записывается в РП как

$$[(p'-B)A^{-1} \ 1] = [p' \ 1] \begin{bmatrix} A^{-1} & O_{n,1} \\ -BA^{-1} & 1 \end{bmatrix},$$

откуда получаем уравнение и расширенную матрицу обратного АП:

$$\tilde{p} = \tilde{p}' C^{-1}, \quad C^{-1} = \begin{bmatrix} A^{-1} & O_{n,1} \\ -BA^{-1} & 1 \end{bmatrix}. \quad (3.9б)$$

Преобразование (3.4) **вектора** V записывается в РП аналогично (3.8):

$$[V' \ 0] = [V \ 0] \begin{bmatrix} A & O_{n,1} \\ B & 1 \end{bmatrix} \quad \text{или} \quad \tilde{V}' = \tilde{V} C. \quad (3.10)$$

Следовательно, одна и та же матрица C может использоваться для аффинного преобразования как отдельно взятой точки p , так и вектора $V = p_2 - p_1$ между двумя точками p_1 и p_2 . Для этого и определены различающиеся между собой расширенные формы графических элементов: точки $\tilde{p}$ и вектора $\tilde{V}$.

В отличие от (3.5) групповое преобразование m точек, собранных по (1.142) в расширенную матрицу $\tilde{P}$, выполняется за одно матричное умножение:

$$\begin{bmatrix} \tilde{p}'_1 \\ \dots \\ \tilde{p}'_m \end{bmatrix} = \begin{bmatrix} \tilde{p}_1 \\ \dots \\ \tilde{p}_m \end{bmatrix} C \quad \text{или} \quad \boxed{\tilde{P}' = \tilde{P} C}. \quad (3.11)$$

Цепочка N преобразований всех точек объекта в РП

$$\tilde{P}_0 \xrightarrow{C_1} \tilde{P}_1 \xrightarrow{C_2} \dots \xrightarrow{C_N} \tilde{P}_N$$

$\underbrace{\hspace{10em}}_C \quad \uparrow$

эквивалентна **одному** умножению $\tilde{P}_N = \tilde{P}_0 C$ с матрицей

$$C = \prod_{i=1}^N C_i. \quad (3.12)$$

В сравнении с алгоритмом (3.6) матрица преобразования в РП рассчитывается единообразно — **только перемножением матриц**. Унификация — основное **преимущество** преобразований в однородных координатах, позволяющее без долгих раздумий правильно записать и быстро вычислить матрицу многошаговой цепочки C как произведение матриц шаговых преобразований C_i **в порядке их выполнения**. Несмотря на то, что для этого требуется выход в пространство R^{n+1} , количество арифметических операций в общем итоге даже снижается, да и программирование (3.11) гораздо проще, чем (3.6).

К **недостаткам** преобразований в расширенном пространстве отнесём:

- избыточную размерность расширенных матриц и векторов;

- фиксированные элементы последнего столбца $[0 \dots 0 \ 1]^T$ матрицы C , лишь потребляющие машинные ресурсы;
- необходимость контроля и коррекции фиксированных элементов после матричных операций в связи с возможным накоплением погрешностей машинного округления.

Смещение произвольной точки p , преобразуемой по (3.1), равно

$$d(p) = p' - p = p(A - E_n) + B. \quad (3.13a)$$

Точку p_n , остающуюся в процессе АП **неподвижной**, можно найти, решая уравнение $d(p_n) = O_n$ или $p_n = p_n A + B$:

$$p_n = B(E_n - A)^{-1}. \quad (3.13б)$$

Условие существования единственной неподвижной точки p_n — невырожденность матрицы $E_n - A$. Точка, неподвижная при всех допустимых значениях матриц A и B , называется **центром преобразования**.

Важным свойством линейных преобразований является **равенство модулей определителей** $||A|| = ||C||$ **матриц аффинных преобразований изменению площади плоского и объёма пространственного объекта**, что позволяет вычислить эти важнейшие параметры геометрических фигур по известным их исходным значениям и матрицам преобразований.

Преобразование неявных моделей

Геометрические преобразования точек 2d-линий и 3d-поверхностей влекут за собой трансформацию их неявных уравнений $f(p) = 0$, в чём убеждают соотношения (1.174). Вспомним (см. п. 1.5.11) принцип строения функции $\phi(p')$: её аргумент является обратным преобразованием аргумента функции $f(p)$:

$$\phi(\tilde{p}') = f(\tilde{p}'C^{-1}). \quad (3.14)$$

Аффинное преобразование **прямой** в R^2 и **плоскости** в R^3 с НФ $f(\tilde{p}) = \tilde{p}F^T$ матрицей C изменяет их векторы F следующим образом:

$$\tilde{p}'F'^T = \tilde{p}'C^{-1}F^T \Rightarrow F'^T = C^{-1}F^T \Rightarrow \boxed{F' = F(C^{-1})^T = F(C^T)^{-1}}. \quad (3.15a)$$

Сдвинем, например, прямую $x + 2y - 2 = 0$ с нормалью $N = [1 \ 2]$ и вектором неявной формы $F = [1 \ 2 \ -2]$ влево на 2 единицы матрицей C , составленной по (3.9a) из блоков $A = E_2$ и $B = [-2 \ 0]$. По (3.15a) получаем следующий вектор НФ:

$$F' = [1 \ 2 \ -2] \begin{bmatrix} 1 & 0 & -2 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}^{-1} = [1 \ 2 \ -2] \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [1 \ 2 \ 0].$$

Полученная прямая $x + 2y = 0$ с нормалью $N' = N$ параллельна исходной прямой и проходит через начало координат, как показано на рис. 3.2,а.

Подстановка обратной матрицы C^{-1} из (3.9б) в (3.15а) даёт формулу вычисления нормали линейно преобразуемого объекта первого порядка:

$$[N' \ D'] = [N \ D] \begin{bmatrix} (A^{-1})^T & -(A^{-1})^T B^T \\ O_{1,n} & 1 \end{bmatrix} \Rightarrow \boxed{N' = N(A^{-1})^T}. \quad (3.15б)$$

Следует предостеречь от применения в расчёте преобразованной нормали метода (3.4): **вектор $N' = NA$ в общем случае не является нормалью к прямой или плоскости, преобразованными матрицей A .** Единственное **исключение** имеет место при **ортогональной** матрице A , удовлетворяющей условию $A^{-1} = A^T$. Со всем скоро станет известно, что ортогональными матрицами описываются лишь **элементарные** преобразования вращения и зеркального отражения.

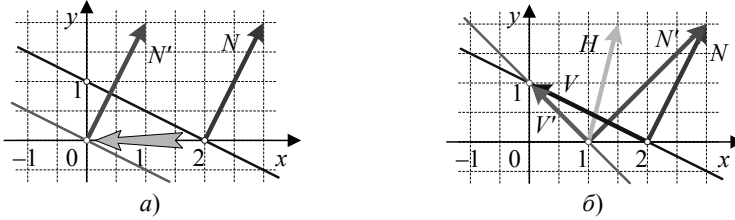


Рис. 3.2

К примеру, возьмём прямую $x+2y-2=0$ с нормалью $N=[1 \ 2]$, отсекающую на осях координат отрезки $h_x=2$ и $h_y=1$, и подвергнем её двойному сжатию вдоль оси x матрицей $A=\text{diag}[1/2 \ 1]$. По построению на рис. 3.2,б видно, что преобразованная прямая отсекает равные осевые отрезки $h'_x=h'_y=1$, а её нормаль имеет равные координаты $N'_x=N'_y$. По (3.15б) получается правильно — $N'=[1 \ 2] \cdot \text{diag}[2 \ 1]=[2 \ 2]$, тогда как формула преобразования (3.4) даёт вектор $H=[1 \ 2] \cdot \text{diag}[1/2 \ 1]=[1/2 \ 2]$, вовсе не ортогональный прямой $2x+2y-2=0$. Заметим, однако, что направляющий вектор прямой $V=\text{Lrot}(N)=[-2 \ 1]$ преобразуется по (3.4) верно: $V'=[-2 \ 1] \cdot \text{diag}[1/2 \ 1]=[-1 \ 1]$.

Таковыми же свойствами обладает преобразование нормали и направляющих векторов плоскости: правильными являются формулы $V'=VA$, $W'=WA$ и $N'=N(A^{-1})^T$, но неверна формула $N'=NA$.

Аналогично (3.15а) выводится формула преобразования матриц F неявной формы **квадратичных объектов** — линии (1.56а) и поверхности (1.158а):

$$\tilde{p}'F\tilde{p}'^T = \tilde{p}'C^{-1}F(C^{-1})^T\tilde{p}'^T \Rightarrow \boxed{F' = C^{-1}F(C^{-1})^T}. \quad (3.16)$$

Подставляя сюда блочные матрицы (3.9б) и (1.58б), получим блоки

$$F'_{11} = A^{-1}F_{11}(A^{-1})^T, \quad F'_{21} = F_{21}(A^{-1})^T - BA^{-1}F_{11}(A^{-1})^T$$

и формулу преобразования нормали преобразованного квадратичного объекта (1.57), совпадающую с (3.15б):

$$N' = 2(q'F'_{11} + F'_{21}) = 2((qA+B)A^{-1}F_{11} + F_{21} - BA^{-1}F_{11})(A^{-1})^T = N(A^{-1})^T.$$

К примеру, растянем сферу $f(x,y,z)=x^2+y^2+z^2-1=0$ с матрицей неявной формы $F=\text{diag}[1\ 1\ 1\ -1]$ в 2 раза по оси x матрицей $A=\text{diag}[2\ 1\ 1]$ и переместим полученный эллипсоид на вектор $B=[2\ 1\ 4]$ (рис. 3.3). Сформировав по (3.9б) обратную матрицу C^{-1} , получаем по (3.16) матрицу квадратичной формы и по (1.158) неявное уравнение эллипсоида:

$$F' = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -1 & -1 & -4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0.5 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -4 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.25 & 0 & 0 & -0.5 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -4 \\ -0.5 & -1 & -4 & 17 \end{bmatrix},$$

$$\phi(x,y,z)=0.25x^2+y^2+z^2-x-2y-8y+17=0.$$

Взяв на этой поверхности точку $q=[4\ 1\ 4]$ и подставив её координаты в полученную функцию, убеждаемся в её принадлежности эллипсоиду: $\phi(4,1,4)=0$.

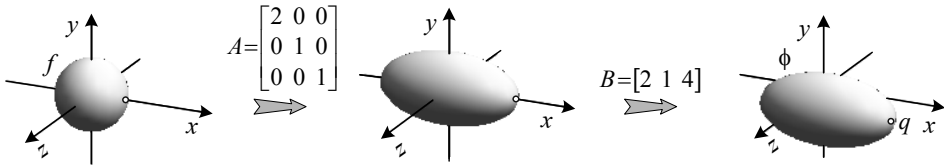


Рис. 3.3

Метод парных точек

Представляет интерес **обратная задача аффинного преобразования** — нахождение матриц $\{A,B\}$ и C по двум известным положениям объекта, связанным соотношениями (3.5) и (3.11), — начальному P и конечному P' . Прежде всего определим число уравнений m , необходимое и достаточное для разрешимости этих систем. Затем получим формулы вычисления матриц преобразования. И напоследок, выясним условия, которым должны удовлетворять выбранные точки объектов, чтобы обратная задача имела однозначное решение.

Матрицы преобразований содержат n^2+n неизвестных элементов $a_{11} \div a_{nn}$ и $b_1 \div b_n$. Их невозможно найти по координатам одной пары точек $\{p,p'\}$, связанным по (3.3) n скалярными уравнениями. Задача становится разрешимой, если взять $m=n+1$ -у пару точек образа и прообраза $\{p_1,p'_1\} \div \{p_m,p'_m\}$, сформировать из них пару **квадратных** матриц $\{\tilde{P},\tilde{P}'\}$ и получить полностью определённую систему (3.11). Её решения **методом парных точек (МПТ)** являются матрицы прямого и обратного преобразований:

$$\begin{aligned} \tilde{P}' = \tilde{P}C &\Rightarrow C = \tilde{P}^{-1}\tilde{P}' \equiv \begin{bmatrix} p_1 & 1 \\ \dots & \dots \\ p_{n+1} & 1 \end{bmatrix}^{-1} \begin{bmatrix} p'_1 & 1 \\ \dots & \dots \\ p'_{n+1} & 1 \end{bmatrix}, \\ C^{-1} = (\tilde{P}')^{-1}\tilde{P} &\equiv \begin{bmatrix} p'_1 & 1 \\ \dots & \dots \\ p'_{n+1} & 1 \end{bmatrix}^{-1} \begin{bmatrix} p_1 & 1 \\ \dots & \dots \\ p_{n+1} & 1 \end{bmatrix}. \end{aligned} \quad (3.17a)$$

Независимо от *синхронной* нумерации пар точек в матрицах P и P' структуры вычисленных по (3.17а) матриц будут соответствовать (3.9). Тогда верхний левый блок матрицы C есть матрица A , а нижний левый — вектор B .

Возможна альтернативная постановка обратной задачи, когда заданы две точки $\{o, o'\}$ и n пар $\{V_i, V'_i\}$ векторов $V_i = p_i - o$ и $V'_i = p'_i - o' \quad \forall i = \overline{1, n}$. Другими словами, начальное положение объекта задаётся точкой o и выходящими из неё в остальные точки векторами $V_1 \div V_n$, а конечное положение — точкой o' и выходящими из неё векторами $V'_1 \div V'_n$. Составим *невыврожденную* матрицу S , умножение которой на матрицу $\tilde{P}$ приводит к вычитанию последней строки $\tilde{o}$ из первых n строк $\tilde{p}_1 \div \tilde{p}_n$, тогда как сама строка $\tilde{o}$ не изменяется:

$$S = \begin{bmatrix} 1 & \dots & 0 & -1 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 1 & -1 \\ 0 & \dots & 0 & 1 \end{bmatrix} \Rightarrow S \begin{bmatrix} p_1 & 1 \\ \dots & \dots \\ p_n & 1 \\ o & 1 \end{bmatrix} = \begin{bmatrix} V_1 & 0 \\ \dots & \dots \\ V_n & 0 \\ o & 1 \end{bmatrix}.$$

Теперь матрицы прямого и обратного АП можно вычислить вместо (3.17а) по следующим альтернативным формулам:

$$C = \tilde{P}^{-1} S^{-1} S \tilde{P}' = (S \tilde{P})^{-1} (S \tilde{P}') = \begin{bmatrix} V_1 & 0 \\ \dots & \dots \\ V_n & 0 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} V'_1 & 0 \\ \dots & \dots \\ V'_n & 0 \\ o' & 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{V}_1 \\ \dots \\ \tilde{V}_n \\ \tilde{o} \end{bmatrix}^{-1} \begin{bmatrix} \tilde{V}'_1 \\ \dots \\ \tilde{V}'_n \\ \tilde{o}' \end{bmatrix}, \quad (3.17б)$$

$$C^{-1} = (\tilde{P}')^{-1} S^{-1} S \tilde{P} = (S \tilde{P}')^{-1} (S \tilde{P}) = \begin{bmatrix} V'_1 & 0 \\ \dots & \dots \\ V'_n & 0 \\ o' & 1 \end{bmatrix}^{-1} \begin{bmatrix} V_1 & 0 \\ \dots & \dots \\ V_n & 0 \\ o & 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{V}'_1 \\ \dots \\ \tilde{V}'_n \\ \tilde{o}' \end{bmatrix}^{-1} \begin{bmatrix} \tilde{V}_1 \\ \dots \\ \tilde{V}_n \\ \tilde{o} \end{bmatrix}.$$

Составив из векторов $V_1 \div V_n$ и $V'_1 \div V'_n$ координатные матрицы

$$\Omega = \begin{bmatrix} V_1 \\ \dots \\ V_n \end{bmatrix}, \quad \Omega' = \begin{bmatrix} V'_1 \\ \dots \\ V'_n \end{bmatrix},$$

получим, опираясь на (3.9), формулы расчёта матриц преобразования A и B , составляющих матрицу C из (3.17б):

$$C = \begin{bmatrix} \Omega & O \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} \Omega' & O \\ o' & 1 \end{bmatrix} = \begin{bmatrix} \Omega^{-1} & O \\ -o\Omega^{-1} & 1 \end{bmatrix} \begin{bmatrix} \Omega' & O \\ o' & 1 \end{bmatrix} = \begin{bmatrix} \Omega^{-1}\Omega' & O \\ o' - o\Omega^{-1}\Omega' & 1 \end{bmatrix}, \quad (3.17в)$$

$$A = \Omega^{-1}\Omega', \quad B = o' - oA.$$

В теории существуют понятия матриц перестановок, так что **можно произвольно, но синхронно, обменивать строки в матрицах $\{P, P'\}$ и $\{\Omega, \Omega'\}$.**

Условие существования решений, при которых обращаемые в (3.17) матрицы невырождены, состоит в *линейной независимости* расширенных векторов каждой из систем $\tilde{p}_1 \div \tilde{p}_{n+1}$ и $\tilde{p}'_1 \div \tilde{p}'_{n+1}$, что означает *равные n размерности пространств*, натянутых на точки $p_1 \div p_{n+1}$, $p'_1 \div p'_{n+1}$ и векторы $V_1 \div V_n$, $V'_1 \div V'_n$:

• при $n=2$ для вычисления матриц $C \in \mathbb{R}^{3 \times 3}$, $A \in \mathbb{R}^{2 \times 2}$ и $B \in \mathbb{R}^{1 \times 2}$ нужно выбрать в начальном и конечном состояниях фигуры на плоскости xu по две тройки **неколлинеарных** точек $\{p_1, p_2, p_3\}$, $\{p'_1, p'_2, p'_3\}$ либо по одной точке и два **неколлинеарных** вектора $\{o, V, W\}$, $\{o', V', W'\}$ (рис. 3.4,а):

$$C = \begin{bmatrix} p_1 & 1 \\ p_2 & 1 \\ p_3 & 1 \end{bmatrix}^{-1} \begin{bmatrix} p'_1 & 1 \\ p'_2 & 1 \\ p'_3 & 1 \end{bmatrix} = \begin{bmatrix} V & 0 \\ W & 0 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} V' & 0 \\ W' & 0 \\ o' & 1 \end{bmatrix}, \quad A = \begin{bmatrix} V \\ W \end{bmatrix}^{-1} \begin{bmatrix} V' \\ W' \end{bmatrix}, \quad B = o' - oA; \quad (3.18a)$$

• при $n=3$ матрица $C \in \mathbb{R}^{4 \times 4}$ преобразования в пространстве xuz находится по двум четвёркам **некомпланарных** точек $\{p_1 \div p_4\}$, $\{p'_1 \div p'_4\}$ либо по одной точке и трём **некомпланарным** векторам $\{o, V, W, U\}$, $\{o', V', W', U'\}$ (рис. 3.4,б):

$$C = \begin{bmatrix} p_1 & 1 \\ p_2 & 1 \\ p_3 & 1 \\ p_4 & 1 \end{bmatrix}^{-1} \begin{bmatrix} p'_1 & 1 \\ p'_2 & 1 \\ p'_3 & 1 \\ p'_4 & 1 \end{bmatrix} = \begin{bmatrix} V & 0 \\ W & 0 \\ U & 1 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} V' & 0 \\ W' & 0 \\ U' & 1 \\ o' & 1 \end{bmatrix}, \quad A = \begin{bmatrix} V \\ W \\ U \end{bmatrix}^{-1} \begin{bmatrix} V' \\ W' \\ U' \end{bmatrix}, \quad B = o' - oA. \quad (3.18б)$$

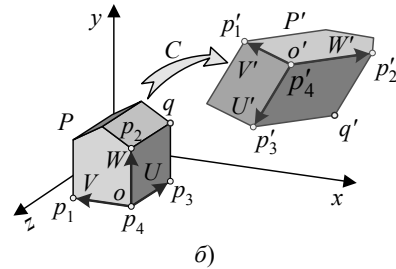
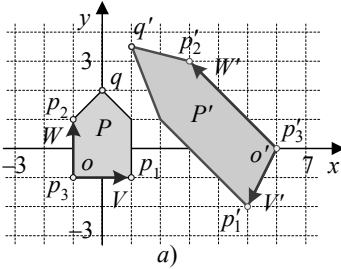


Рис. 3.4

С помощью вычисленных матриц по (3.3) либо (3.8) находится положение любой другой точки $q' \in P'$ по координатам соответствующей ей точки $q \in P$.

Пример 3.1. Рассчитать матрицы преобразования полигона P в полигон P' и найти положение точки q' (рис. 3.4,а).

Решение. Подставив координаты точек $\{p_1 \div p_3\}$ и $\{p'_1 \div p'_3\}$ в (3.17а), получаем матрицу

$$C = \begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & 1 \\ -1 & -1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 5 & -2 & 1 \\ 3 & 3 & 1 \\ 6 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -0.5 & -1 & 0 \\ -1.5 & 1.5 & 0 \\ 4 & 0.5 & 1 \end{bmatrix} \Rightarrow A = \begin{bmatrix} -0.5 & -1 \\ -1.5 & 1.5 \end{bmatrix}, \quad B = [4 \ 0.5].$$

То же выйдет при подстановке в (3.17б) координат точек $o = [-1 \ -1]$, $o' = [6 \ 0]$ и векторов $V = [2 \ 0]$, $V' = [-1 \ -2]$, $W = [0 \ 2]$ и $W' = [-3 \ 3]$:

$$C = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & -1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} -1 & -2 & 0 \\ -3 & 3 & 0 \\ 6 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -0.5 & -1 & 0 \\ -1.5 & 1.5 & 0 \\ 4 & 0.5 & 1 \end{bmatrix}.$$

Теперь можно найти прообраз точки $q = [0 \ 2]$: $q' = qA + B = [1 \ 3.5]$. $\square$

Оформим векторный 3d-вариант метода парных точек (3.18б) в виде **функции расчёта матрицы преобразования по четырём элементам объекта в начальном $\{o, V, W, U\}$ и конечном $\{o', V', W', U'\}$ состояниях.**

MP4($o, V, W, U, o', V', W', U'$)

$$\{\text{возврат матрицы } C = \begin{bmatrix} V & 0 \\ W & 0 \\ U & 0 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} V' & 0 \\ W' & 0 \\ U' & 0 \\ o' & 1 \end{bmatrix}; \quad (3.19)$$

} □

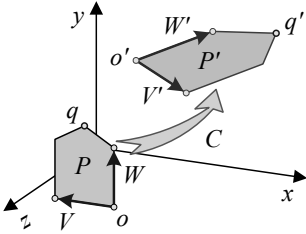


Рис. 3.5

Часто информация о начальном P и конечном P' состояниях объекта в пространстве бывает неполной. Например, в каждом из них задано по **три** неколлинеарных точки либо по одной точке и два непараллельных вектора. Тройки элементов $\{o, V, W\}$ и $\{o', V', W'\}$ определяют две плоскости состояний объектов, вне которых они не определены (рис. 3.5). Найти матрицу преобразования любой точки $q \in P$ в точку прообраза $q' \in P'$.

Для расчёта этой матрицы методом парных точек нужно иметь по четыре элемента в каждом состоянии объекта. Двух пар **неколлинеарных** векторов $\{V, W\}$ и $\{V', W'\}$ достаточно для восстановления третьих векторов $U = V \times W$ и $U' = V' \times W'$. Благодаря их перпендикулярности плоскостям $\{o, V, W\}$ и $\{o', V', W'\}$ формула (3.19) становится вычислимой. Таким образом, в копилку полезных инструментов программиста добавляется **функция расчёта матрицы преобразования по трём элементам объекта в начальном $\{o, V, W\}$ и конечном $\{o', V', W'\}$ состояниях.**

MP3(o, V, W, o', V', W')

$$\{\text{возврат матрицы } C = \begin{bmatrix} V & 0 \\ W & 0 \\ V \times W & 0 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} V' & 0 \\ W' & 0 \\ V' \times W' & 0 \\ o' & 1 \end{bmatrix}; \quad (3.20)$$

} □

В частных случаях начальное либо конечное состояния объекта задаются элементами системы координат — начальной точкой O_3 и ортами x^0 , y^0 и z^0 . Это упрощает вычисление матриц (3.19) и (3.20) благодаря превращению одной из перемножаемых матриц в **единичную**:

• перевод объекта из начального состояния, определённого четвёркой $\{O_3, x^0, y^0, z^0\}$, в конечное состояние $\{o, V, W, U\}$ осуществляется матрицами

$$C = MP4(O_3, x^0, y^0, z^0, o, V, W, U) = MP3(O_3, x^0, y^0, o, V, W) =$$

$$= \begin{bmatrix} V & 0 \\ W & 0 \\ U & 0 \\ o & 1 \end{bmatrix} = \begin{bmatrix} V & 0 \\ W & 0 \\ V \times W & 0 \\ o & 1 \end{bmatrix}; \quad (3.21a)$$

- перевод объекта из начального состояния $\{o, V, W, U\}$ в конечное состояние $\{O_3, x^o, y^o, z^o\}$ осуществляется обратными (3.21a) матрицами

$$C = MP4(o, V, W, U, O_3, x^o, y^o, z^o) = MP3(o, V, W, O_3, x^o, y^o) =$$

$$= \begin{bmatrix} V & 0 \\ W & 0 \\ U & 0 \\ o & 1 \end{bmatrix}^{-1} = \begin{bmatrix} V & 0 \\ W & 0 \\ V \times W & 0 \\ o & 1 \end{bmatrix}^{-1}. \quad (3.21б)$$

☞ Положение многих фигур, например, тел вращения, можно задать всего **двумя** элементами оси вращения — точкой o и вектором направления V . Благодаря осевой симметрии поверхность повернутого тела совпадает сама с собой. Информация о дополнительных векторах W и U становится неоднозначной, фактически излишней и вычислительно обременительной.

Для вычисления матрицы преобразования объекта из начального состояния $\{o, V\}$ в конечное состояние $\{o', V'\}$ восстановим необходимые методу парных точек две пары **ортогональных** векторов $\{N, B\}$ и $\{N', B'\}$ (рис. 3.6):

- начнем с создания вектора **нормали** N , ортогонального векторам V и V' . Если $V \# V'$ (векторы не параллельны), то $N = V \times V'$, в противном случае при $V \# z^o$ возьмём $N = V \times z^o$, а при $V \parallel z^o$ — $N = V \times x^o$;

- вычислив **коэффициент масштабирования** $m = |V'|/|V|$, отражающий соотношение линейных размеров объектов P' и P во всех направлениях, создадим второй вектор нормали $N' = mN$ длиной $m|N|$;

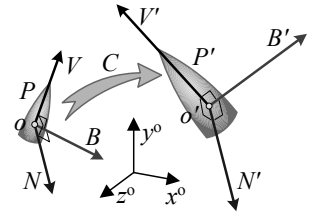


Рис. 3.6

- построим два вектора **бинормалей** — в начальном состоянии $B = V \times N$, а в конечном состоянии $B' = V' \times N$ длиной $|B'| = m|B|$.

Подставив четвёрки элементов $\{o, V, N, B\}$ и $\{o', V', N', B'\}$ в (3.18б), реализуем МПТ в виде **функции расчёта матрицы преобразования по двум элементам объекта в начальном $\{o, V\}$ и конечном $\{o', V'\}$ состояниях.**

☞ $MP2(o, V, o', V')$

$$\{N = V \times \text{if}(|V \times V'|:V', \text{if}(|V \times z^o|:z^o, x^o)), m = |V'|/|V|;$$

$$\text{возврат матрицы } C = \begin{bmatrix} V & 0 \\ N & 0 \\ V \times N & 0 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} V' & 0 \\ mN & 0 \\ V' \times N & 0 \\ o' & 1 \end{bmatrix}; \quad (3.22)$$

} ☐

В частных случаях, когда одна из точек $\{o, o'\}$ совпадает с началом координат O_3 , а соответствующий вектор из пары $\{V, V'\}$ — с каким-либо ортом x^o , y^o или z^o , ни одна из двух матриц в (3.22) не становится единичной, поэтому никаких упрощений в вычислении матрицы C не произойдёт.

Совсем не обременительное **условие существования матрицы преобразования** — ненулевые векторы V и V' . Благодаря ортогональности каждой тройки векторов $\{V, N, B\}$ и $\{V', N', B'\}$ обеспечивается наилучшая обусловленность обрабатываемой в (3.22) матрицы. Как будет показано далее, функция МР2 обладает свойством **экономичности** — она переводит объект из **начального состояния** $\{o, V\}$ в **конечное** $\{o', V'\}$ по **кратчайшим** направлениям переноса $o \rightarrow o'$ и угла вращения $V \rightarrow V'$. Кратчайшее направление вращения отличается от последовательности вращений вокруг осей координат так же, как один проход по гипотенузе треугольника отличается от двух проходов по его катетам.

Произвольность ориентации векторов N' и B' в двухэлементном МПТ оборачивается в некоторых задачах **недостатком** в виде неконтролируемого **кручения** преобразованного объекта вокруг оси V' . Позднее в п. 3.1.3 мы увидим визуальное проявление этого нежелательного эффекта.

Преобразование системы координат

Часто бывает удобнее и понятнее оставить объекты сцены неподвижными, а преобразования подвергать одну лишь **систему координат**. Это **пассивное** АП описывается матрицей $C_k = C^{-1}$, **обратной** к матрице C , выполняющей такое же активное преобразование объекта. Поясним этот **принцип инверсности** тремя примерами манипуляций с плоской декартовой системой координат x, y , в которой задана точка $p = [4 \ 3]$ (рис. 3.7):

- после переноса СК на вектор $B = [2 \ 1]$ точка обретает новые координаты $p' = [2 \ 2]$, которые получились бы при активном переносе по формуле $p'' = p - B$;
- удвоение масштаба по оси x и сохранение его по оси y вдвое уменьшает абсциссу $x' = x/2 = 2$ без изменения ординаты $y' = y = 3$. Такие же координаты получила бы точка p'' в результате активного преобразования двойного сжатия;
- поворот системы координат на угол $\varphi = \arctg(3/4)$ вокруг неподвижного центра делает координаты точки равными $p' = [5 \ 0]$. Такие же координаты получила бы точка p'' в результате активного преобразования вращения на угол $-\varphi$.

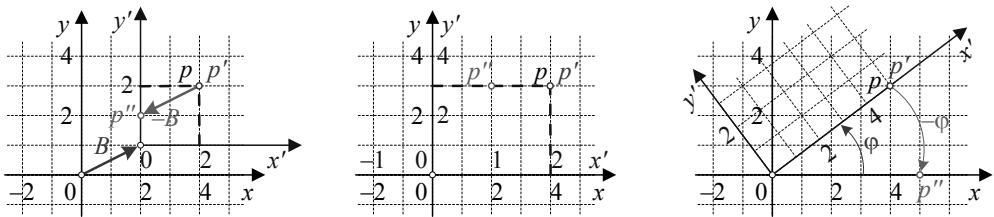


Рис. 3.7

Итак, если в исходной системе координат есть точки $p_1 \div p_n$, $p_{n+1} \equiv o$ и векторы $V_1 \div V_n$, составляющие матрицу Ω , а в изменённой СК эти же элементы заданы точками $p'_1 \div p'_n$, $p'_{n+1} \equiv o'$, векторами $V'_1 \div V'_n$ и матрицей Ω' , то матрицы преобразования системы координат находятся обращением формул (3.17):

$$C_K = \begin{bmatrix} \tilde{p}'_1 \\ \vdots \\ \tilde{p}'_n \\ \tilde{p}'_{n+1} \end{bmatrix}^{-1} \begin{bmatrix} \tilde{p}_1 \\ \vdots \\ \tilde{p}_n \\ \tilde{p}_{n+1} \end{bmatrix} = \begin{bmatrix} \tilde{V}'_1 \\ \vdots \\ \tilde{V}'_n \\ \tilde{o}' \end{bmatrix}^{-1} \begin{bmatrix} \tilde{V}_1 \\ \vdots \\ \tilde{V}_n \\ \tilde{o} \end{bmatrix}, \quad A_K = \Omega'^{-1} \Omega, \quad B_K = o - o' A_K. \quad (3.23a)$$

Расчёт упрощается, если исходное состояние СК $\{o, V_1, \dots, V_n\}$ представляют её собственные элементы — начальная точка O_n и орты $e_1 \div e_n$. Тогда правые матрицы в C_K и A_K становятся единичными, в результате чего

$$C_K = \begin{bmatrix} \tilde{V}'_1 \\ \vdots \\ \tilde{V}'_n \\ \tilde{o}' \end{bmatrix}^{-1}, \quad A_K = \Omega'^{-1}, \quad B_K = -o' A_K. \quad (3.23б)$$

Знание матриц A_K , B_K и C_K способствует вычислению координат точки p и вектора V в новой системе координат по формулам

$$p' = p A_K + B_K, \quad \tilde{p}' = \tilde{p} C_K, \quad V' = V A_K, \quad \tilde{V}' = \tilde{V} C_K. \quad (3.24)$$

Дополним метод парных точек шаблонами обращения к функциям $MP2 \div MP4$, аргументы которых по принципу инверсности следуют в порядке «куда $\rightarrow$ откуда», и некоторыми областями применения этих функций.

□ Перенос начала системы координат xuz в точку o и совмещение **одной** из перенесённых осей, допустим, z' , с вектором V (рис. 3.8) с неконтролируемой ориентацией осей x' и y' выполняется матрицей

$$C_K = MP2(O_3, z^0, o, V)^{-1} = MP2(o, V, O_3, z^0). \quad (3.25a)$$

В новой системе координат $x'y'z'$ длины всех ортов равны $m = |V|$, так что преобразованный объект $\tilde{p} C_K$ пропорционально масштабируется по всем направлениям в $1/m$ раз. Сохранение метрики системы координат достигается обращением к функции $MP2$ с нормированным вектором $\bar{V}$:

$$C_K = MP2(o, \bar{V}, O_3, z^0). \quad (3.25б)$$

Данный метод вычисления матрицы преобразования координат применяется там, где важно обеспечить строгую ориентацию лишь одной оси z' в направлении вектора V , а расположение других осей безразлично. Одна из таких задач — **вращение объекта вокруг произвольной прямой** $\{o, V\}$, временно ориентируемой вдоль оси z' для того, чтобы сделать операцию вращения элементарной. Вторая задача — **отражение точки от плоскости** $\{o, N\}$ с произвольной нормалью N , временно направляемой матрицей $C_K = MP2(o, N, O_3, z^0)$ по оси z' для выполнения операции элементарного отражения.

□ Преобразование системы координат из исходного состояния $\{O_3, x^0, y^0\}$ в конечное с центром o и **двумя** неколлинеарными векторами V и W , задающими направления новых осей x' и y' (рис. 3.9), осуществляется матрицей

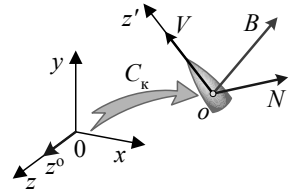


Рис. 3.8

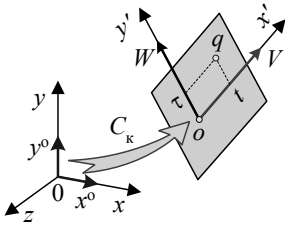


Рис. 3.9

$$C_k = MP3(o, V, W, O_3, x^0, y^0) = \begin{bmatrix} V & 0 \\ W & 0 \\ V \times W & 0 \\ o & 1 \end{bmatrix}^{-1}, \quad (3.26)$$

совпадающей со второй формой матрицы активного преобразования в (3.21б). Сохранение линейных и угловых размеров объектов в новой СК обеспечивается выбором **ортонормированных** векторов V и W , иначе матрица (3.26) осуществляет преобразование в **косоугольную** СК.

Данный метод пригоден для фиксации точки $q = [x \ y \ z]$ на параметрически определённой плоскости $o + Vt + W\tau$, направляющие векторы которой V и W задают оси косоугольной системы координат x' и y' , а точка o — её начало. Ранее (см. п. 1.1.4) изучался метод Грама, по которому координаты t и τ находились по (1.43б). С помощью матрицы C_k эти координаты вычисляются по формуле $[t \ \tau \ 0 \ 1] = [x \ y \ z \ 1]C_k$, после чего могут быть использованы в задачах локализации точки на произвольной плоскости.

□ Преобразование системы координат из исходного состояния $\{O_3, x^0, y^0, z^0\}$ в конечное с центром o и **тремя** некопланарными векторами V, W и U , задающими направления осей x', y' и z' (рис. 3.10,а), выполняется матрицами

$$C_k = MP4(o, V, W, U, O_3, x^0, y^0, z^0) = \begin{bmatrix} V & 0 \\ W & 0 \\ U & 0 \\ o & 1 \end{bmatrix}^{-1} \Rightarrow A_k = \begin{bmatrix} V \\ W \\ U \end{bmatrix}^{-1}, \quad B_k = -oA_k. \quad (3.27)$$

Отметим, что матрица C_k равна первой форме матрицы активного преобразования в (3.21б), а координаты точки $q' = qA_k + B_k = (q - o)A_k$ совпали с (1.150).

Отсутствие линейных и угловых искажений объектов в новой системе координат обеспечивается выбором **ортонормированных** векторов V, W и U . Если же длины и направления этих **некопланарных** векторов произвольны, то в косоугольной системе координат $x'y'z'$ линейные и угловые размеры преобразованного объекта $\tilde{P}' = \tilde{P}C_k$ будут искажены.

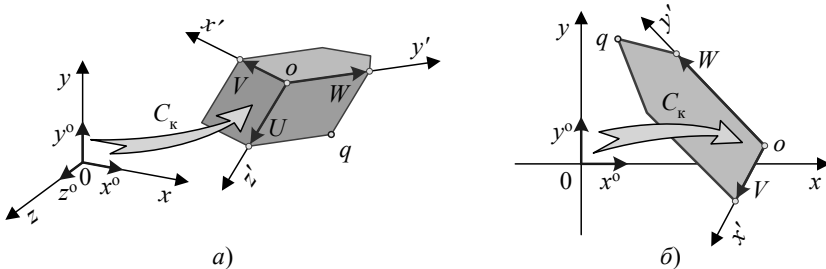


Рис. 3.10

Метод преобразования координат пригоден для фиксации точки в СК $x'y'z'$ с началом в точке o и векторами направлений осей V, W и U . Ранее эта задача

была решена в (1.150). Теперь с помощью матрицы C_k косоугольные координаты точки $q=[x \ y \ z]$ вычисляются по формуле $[x' \ y' \ z' \ 1]=[x \ y \ z \ 1]C_k$, после чего могут быть использованы в трёхмерных задачах локализации точки относительно полиэдров, рассмотренных в п. 1.5.6.

Среди выпуклых полиэдров особое место занимают треугольные пирамиды T и параллелепипеды Π , определённые любой вершиной o и тремя инцидентными ей рёбрами V , W и U (см. рис. 1.130,б,в). Объединением этих фигур (впрочем, достаточно только пирамид) можно получить полиэдры любой сложной формы. Поочерёдная проверка позволяет определить принадлежность точки составной фигуре по её вхождению в любую её часть. Тесты принадлежности точки q границе или внутренности объектов T и Π после преобразования координат $q'=[x' \ y' \ z']$ имеют вид, аналогичный (1.151):

$$q \subseteq \Pi : \{0 \leq x' \leq 1\} \wedge \{0 \leq y' \leq 1\} \wedge \{0 \leq z' \leq 1\}; \quad (3.28a)$$

$$q \subseteq T : \{x' \geq 0\} \wedge \{y' \geq 0\} \wedge \{z' \geq 0\} \wedge \{x' + y' + z' \leq 1\}. \quad (3.28б)$$

Двухмерное сужение метода (3.27) сводится к вычислению матриц перевода декартовой СК на **плоскости** xy в новое положение, задаваемое центром o и векторами V и W направлений осей x' и y' (рис. 3.10,б):

$$C_k = \begin{bmatrix} V & 0 \\ W & 0 \\ o & 1 \end{bmatrix}^{-1} \Rightarrow A_k = \begin{bmatrix} V \\ W \end{bmatrix}^{-1}, \quad B_k = -oA_k. \quad (3.29)$$

Новые координаты точки $q' = qA_k + B_k = (q - o)A_k$ точно совпали с (1.43в). В расширенном пространстве они же вычисляются по формуле $[x' \ y' \ 1] = [x \ y \ 1]C_k$, после чего могут быть использованы в 2d-задачах локализации (см. п. 1.1.4).

Среди выпуклых полигонов особое место занимают треугольники T и параллелограммы Π , определённые любой вершиной o и двумя инцидентными ей рёбрами V и W (рис. 3.11). Тесты принадлежности точки $q'=[x' \ y']$ границе или внутренности этих объектов имеют вид, аналогичный (1.44):

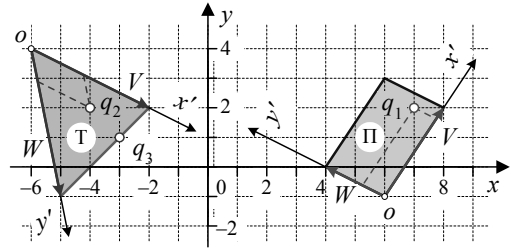


Рис. 3.11

$$q \subseteq \Pi : \{0 \leq x' \leq 1\} \wedge \{0 \leq y' \leq 1\}; \quad (3.30a)$$

$$q \subseteq T : \{x' \geq 0\} \wedge \{y' \geq 0\} \wedge \{x' + y' \leq 1\}. \quad (3.30б)$$

Например, для параллелограмма Π с элементами $o=[6 \ -1]$, $V=[2 \ 3]$ и $W=[-2 \ 1]$ по (3.29) получим следующие матрицы преобразования координат:

$$C_k = \begin{bmatrix} 2 & 3 & 0 \\ -2 & 1 & 0 \\ 6 & -1 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 0.125 & -0.375 & 0 \\ 0.25 & 0.25 & 0 \\ -0.5 & 2.5 & 1 \end{bmatrix}, \quad A_k = \begin{bmatrix} 0.125 & -0.375 \\ 0.25 & 0.25 \end{bmatrix}, \quad B_k = [-0.5 \ 2.5].$$

Точки $q_1=[7 \ 2]$, $q_2=[-4 \ 2]$ и $q_3=[-3 \ 1]$ имеют в СК $\{o, V, W\}$ координаты $q'_1=[0.875 \ 0.375]$, $q'_2=[-0.5 \ 4.5]$ и $q'_3=[-0.625 \ 3.875]$. По (3.30а) определяем, что точка q_1 лежит внутри параллелограмма, а точки q_2 и q_3 — вне его.

Те же точки в системе координат треугольника T с элементами $o=[-6 \ 4]$, $V=[4 \ -2]$, $W=[1 \ -5]$ и матрицей преобразования

$$C_k = \begin{bmatrix} 4 & -2 & 0 \\ 1 & -5 & 0 \\ -6 & 4 & 1 \end{bmatrix}^{-1} = \frac{1}{18} \begin{bmatrix} 5 & -2 & 0 \\ 1 & -4 & 0 \\ 26 & 4 & 1 \end{bmatrix}$$

имеют координаты $q'_1=[3.5 \ -1]$, $q'_2=[0.444 \ 0.222]$ и $q'_3=[0.667 \ 0.333]$. Из (3.30б) следует, что $q_1 \notin T$, $q_2 \in T$, а точка q_3 лежит на границе T .

3.1.2. Элементарные аффинные преобразования

Всё множество аффинных преобразований может быть сведено к следующим **элементарным преобразованиям** (ЭП).

Перенос

Параллельный **перенос** (*transfer*) всех точек объекта на один и тот же вектор B в неподвижной системе координат (преобразования I на рис. 3.12) описываются уравнением (3.1) с единичной матрицей A

$$p' = p + B \Rightarrow A = E_n$$

и **матрицами переноса** в прямом и обратном направлениях:

$$T(B) = \begin{bmatrix} E_n & O_{n,1} \\ B & 1 \end{bmatrix}, \quad T(-B) = \begin{bmatrix} E_n & O_{n,1} \\ -B & 1 \end{bmatrix} = T^{-1}(B). \quad (3.31)$$

Свойство $T^{-1}(B) = T(-B)$ позволяет избежать обращения матрицы переноса, заменив его подстановкой инвертированного вектора B .

В соответствии с (3.13) переносное смещение $d=B$ не зависит от положения точки, а неподвижная точка удовлетворяет уравнению $p_n O_{n,n} = B$. При ненулевом переносе $B \neq O_n$ она отсутствует, а при $B = O_n$ все точки объекта неподвижны. Определитель треугольной матрицы переноса $|T|=1$.

Масштабирование

Масштабирование (*scaling*) вдоль осей координат описывается координатными уравнениями $x' = m_x x$, $y' = m_y y$ и $z' = m_z z$ с коэффициентами m_x , m_y и m_z — диагональными элементами **матриц масштабирования**:

$$A = \text{diag}[\underbrace{m_x \ m_y \ m_z}_{n=3}], \quad M(m_x, m_y) = \begin{bmatrix} m_x & 0 \\ 0 & m_y \end{bmatrix}, \quad M(m_x, m_y, m_z) = \begin{bmatrix} m_x & 0 & 0 \\ 0 & m_y & 0 \\ 0 & 0 & m_z \end{bmatrix}. \quad (3.32)$$

При $m_\omega = -1$ точка отражается вдоль оси $\omega \in \{x, y, z\}$ от координатной плоскости, описываемой уравнением $\omega = 0$. Если **все** коэффициенты $m_x = m_y = m_z = -1$, то точка кососимметрично отражается от начала координат. На рис. 3.12 показаны операции:

- масштабирования 2 на плоскости матрицей $M(2, -1)$;
- отражения 3 на плоскости **вдоль** оси x матрицей $M(-1, 1)$;
- 3d-отражения 4 **вдоль** оси y (**от** плоскости xz) матрицей $M(1, -1, 1)$;
- отражения 5 в пространстве **от** оси z матрицей $M(-1, -1, 1)$;
- кососимметричного отражения 6 в пространстве **от** начала координат матрицей $M(-1, -1, -1)$.

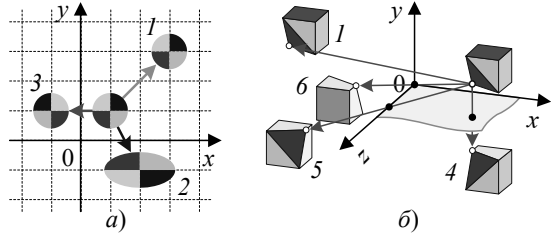


Рис. 3.12

Неподвижная точка, не подвергаясь масштабированию, удовлетворяет по (3.13а) векторному уравнению $p_n \cdot \text{diag}[1 - m_x \mid 1 - m_y \mid 1 - m_z] = O_n$. При $m_\omega = 1$ неподвижны все точки на оси ω . Центр масштабирования находится в начале координат, так как только там он остаётся неподвижным при всех $m_\omega \neq 1$.

Масштабирование с нечётным числом отрицательных коэффициентов изменяет топологические свойства отражённых объектов. В частности, направление обхода вершин полигона при таком масштабировании меняется на противоположное. В этом может убедиться каждый, наблюдая за своим отражением в зеркале: правша становится левшой, мужская одежда женской и т. п. Вторичное отражение во втором зеркале вновь обретает топологические свойства образа.

Возможность подобных эффектов необходимо предусматривать в алгоритмах, критичных к направлению обхода полигонов. Таковой является задача определения видимости плоской грани полиэдра по углу между векторами наблюдателя и нормали (1.145), направление которой критично к направлению обхода вершин полигона грани. Если направление обхода положительно и нормаль внешняя, то по условию (1.181) грань видима. Зеркальное отражение полигона этой грани имеет отрицательное направление обхода и внутреннюю нормаль, рассчитываемую по тем же формулам, что и нормаль самой грани. Таким образом, отражённая грань, реально видимая в зеркале, без учёта инверсии обхода будет оценена тестом (1.181) как невидимая и не отобразится на экране.

Определитель матрицы масштабирования равен произведению её диагональных элементов $|M| = m_x m_y m_z$. У матриц отражения с коэффициентами $m_\omega = \pm 1$ он равен $|M| = (-1)^k$, где k — число инвертируемых координат. Например, при отражении **вдоль одной** оси получим $|M| = -1$ и инвертирование направления обхода всех отражаемых граней без изменения их площадей.

Масштабирование с неравными коэффициентами m_ω изменяет углы между векторами, площади, объёмы и форму геометрических фигур. Неравномерное

растяжение или сжатие вдоль осей координат — простейший способ получения эллипса из окружности, эллипсоида из сферы, параллелепипеда из куба и т. п.

Сдвиг

Сдвиг (*shift*), называемый ещё **скосом** [65], вдоль направлений осей координат или их комбинации описывается следующими координатными уравнениями и **матрицами сдвига** на плоскости и в пространстве:

$$\begin{cases} x' = x + s_{yx}y \\ y' = y + s_{xy}x \end{cases} \Rightarrow S_2 = \begin{bmatrix} 1 & s_{xy} \\ s_{yx} & 1 \end{bmatrix}; \quad (3.33a)$$

$$\begin{cases} x' = x + s_{yx}y + s_{zx}z \\ y' = s_{xy}x + y + s_{zy}z \\ z' = s_{xz}x + s_{zy}y + z \end{cases} \Rightarrow S_3 = \begin{bmatrix} 1 & s_{xy} & s_{xz} \\ s_{yx} & 1 & s_{yz} \\ s_{zx} & s_{zy} & 1 \end{bmatrix}. \quad (3.33б)$$

Параметры сдвига задаются **внедиагональными** элементами матриц S_2 и S_3 . Единственный **коэффициент** $s_{ij} \neq 0$, **равный тангенсу угла между векторами** i^0 и $i^0 S$, задаёт смещение вдоль j -й оси, приходящееся на единицу i -й оси.

Сдвиг точки p можно рассматривать как её перенос на вектор $B(p)$, элементы которого хоть и линейно, но всё же зависят от её координат. Тем не менее, преобразования (3.33) являются аффинными, поскольку удовлетворяют признакам, перечисленным в начале п. 3.1.1.

Соответствующее (3.13a) сдвиговое смещение $d = p(S_n - E_n)$ зависит, однако, от положения точки p , что и отличает его от переносного смещения B . Неподвижная точка удовлетворяет векторному уравнению $p_n(S_n - E_n) = O_n$. Если в матрице S_n все недиагональные элементы ω -й строки нулевые, то при сдвиге неподвижны все точки на оси ω . Две нулевые строки матрицы $S_n - E_n$ с номерами i и j дают множество неподвижных точек на координатной плоскости $\omega_i \times \omega_j$. Центр сдвига при произвольных недиагональных элементах матрицы S_n находится в начале координат.

Важно отметить, что если матрица сдвигового преобразования содержит несколько ненулевых недиагональных элементов, то векторы смещения разных точек не параллельны, а сдвиг происходит **одновременно** вдоль нескольких осей СК, и его нельзя заменить цепочкой одноосевых сдвигов:

$$\begin{bmatrix} 1 & s_{xy} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ s_{yx} & 1 \end{bmatrix} = \begin{bmatrix} 1+s_{xy}s_{yx} & s_{xy} \\ s_{yx} & 1 \end{bmatrix} \neq \begin{bmatrix} 1 & s_{xy} \\ s_{yx} & 1 \end{bmatrix}.$$

В отличие от других элементарных АП, **преобразование сдвига может иметь вырожденную матрицу** с нулевым определителем, например,

$$S_2 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad S_2 = \begin{bmatrix} 1 & 0.5 \\ 2 & 1 \end{bmatrix}, \quad S_3 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad S_3 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & -1 & 1 \end{bmatrix}.$$

Размерность пространства, натянутого на точки сдвинутого объекта, равна рангу матрицы S_n . Следовательно, при $\text{rang}[S_n] < n$ преобразование сдвига не является аффинным: все точки на плоскости R^2 сдвигаются на одну прямую линию, а все точки в пространстве R^3 — на одну плоскость при $\text{rang}[S_3]=2$ или на одну прямую линию при $\text{rang}[S_3]=1$.

На рис. 3.13,а приведены примеры сдвига трёх объектов на плоскости со следующими матрицами преобразований и сдвиговыми смещениями:

- отрезка (1) матрицей $S_2 = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix}$ с параллельными векторами смещений $d = [-2y \ 0] \parallel x^0$ и множеством неподвижных точек $p_n = [x \ 0]$ на всей оси x ;
- квадрата (2) и круга (3) матрицей $S_2 = \begin{bmatrix} 1 & 1 \\ -2 & 1 \end{bmatrix}$ с непараллельными векторами смещений $d = [-2y \ x]$ и единственной неподвижной точкой $p_n = [0 \ 0]$.

На рис. 3.13,б показаны разные сдвиги объёмной фигуры:

- сдвиг 1 вдоль оси y треугольной матрицей $S_3 = \begin{bmatrix} 1 & -2 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ ранга 3 с параллельными векторами смещений $d = [0 \ -2x \ 0] \parallel y^0$ и множеством неподвижных точек на плоскости yz ;
- сдвиг 2 матрицей $S_3 = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & -1 & 1 \end{bmatrix}$ ранга 3 с векторами $d = [-y-z \ -z \ 0] \parallel yz$ и множеством неподвижных точек на оси x ;
- сдвиг 3 *вырожденной* матрицей $S_3 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$ неполного ранга 2, из-за чего все точки прообраза ложатся на плоскость $x-z=0$.

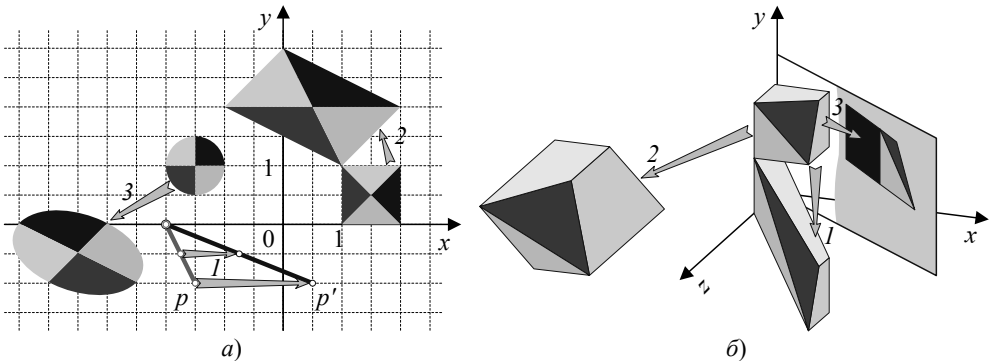


Рис. 3.13

Вращение

Вращение (*rotation*) произвольной точки $p=[x \ y] \in \mathbb{R}^2$ на плоскости xu вокруг начала координат на угол φ (рис. 3.14,а) описывается следующими координатными уравнениями:

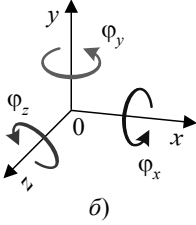
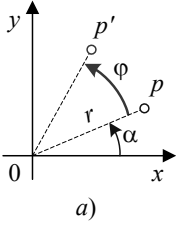


Рис. 3.14

$$\begin{cases} x' = r \cos(\alpha + \varphi) = x \cos(\varphi) - y \sin(\varphi), \\ y' = r \sin(\alpha + \varphi) = x \sin(\varphi) + y \cos(\varphi), \end{cases}$$

где $x = r \cos(\alpha)$ и $y = r \sin(\alpha)$. Отсюда получаем **матрицы вращения** в положительном (**против часовой стрелки**) и отрицательном (**по часовой стрелке**) направлениях:

$$R(\varphi) = \begin{bmatrix} \cos(\varphi) & \sin(\varphi) \\ -\sin(\varphi) & \cos(\varphi) \end{bmatrix}, \quad R(-\varphi) = \begin{bmatrix} \cos(\varphi) & -\sin(\varphi) \\ \sin(\varphi) & \cos(\varphi) \end{bmatrix} = R^T(\varphi) = R^{-1}(\varphi). \quad (3.34)$$

При вращении пространственной точки $p=[x \ y \ z] \in \mathbb{R}^3$ вокруг осей **правой** системы координат (положительные направления вращений показаны на рис. 3.14,б) матрицы вращений получим из соответствующих уравнений:

- $x' = x, \quad y' = y \cos(\varphi_x) - z \sin(\varphi_x), \quad z' = y \sin(\varphi_x) + z \cos(\varphi_x),$

$$R_x(\varphi_x) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\varphi_x) & \sin(\varphi_x) \\ 0 & -\sin(\varphi_x) & \cos(\varphi_x) \end{bmatrix}, \quad R_x(-\varphi_x) = R_x^{-1}(\varphi_x); \quad (3.35a)$$

- $x' = x \cos(\varphi_y) + z \sin(\varphi_y), \quad y' = y, \quad z' = -x \sin(\varphi_y) + z \cos(\varphi_y),$

$$R_y(\varphi_y) = \begin{bmatrix} \cos(\varphi_y) & 0 & \sin(\varphi_y) \\ 0 & 1 & 0 \\ \sin(\varphi_y) & 0 & \cos(\varphi_y) \end{bmatrix}, \quad R_y(-\varphi_y) = R_y^{-1}(\varphi_y); \quad (3.35б)$$

- $x' = x \cos(\varphi_z) - y \sin(\varphi_z), \quad y' = x \sin(\varphi_z) + y \cos(\varphi_z), \quad z' = z,$

$$R_z(\varphi_z) = \begin{bmatrix} \cos(\varphi_z) & \sin(\varphi_z) & 0 \\ -\sin(\varphi_z) & \cos(\varphi_z) & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_z(-\varphi_z) = R_z^{-1}(\varphi_z). \quad (3.35в)$$

Преобразование вращения не изменяет углы между жёстко связанными векторами, площади, объёмы и форму геометрических фигур, а также топологические свойства полигонов. Определители всех матриц вращения равны единице. Благодаря свойству **ортогональности** матриц $R^T R = R R^T = E_n$ при изменении знака угла вращения на противоположный обращение матриц вращения можно заменить их транспонированием: $R^{-1}(\varphi) = R^T(\varphi)$.

Векторы (3.13а) смещения вращательного движения $d = p(R_\omega(\varphi) - E_n)$ образуют цилиндрическое векторное поле, ось симметрии которого ω содержит множество неподвижных точек. Центр всех вращений на произвольные углы вокруг любых осей координат находится в начале системы координат.

Табличный расчёт тригонометрических функций

Весьма важным в практике применения матриц вращения является **быстрый** расчёт тригонометрических функций угла и наоборот — угла по значениям его тригонометрических функций. Актуальность этой проблемы объясняется присутствием вращения практически в каждом сложном преобразовании и высокой частотой выполнения этой операции, в том числе и во вложенных циклах. Использование встроенных в языки программирования функций $\cos$ и $\sin$, вычисляемых суммированием рядов Тейлора, конечно, заманчиво, но не практично из-за их невысокого быстродействия.

Ещё меньшая скорость сходимости у обратных тригонометрических функций, из которых во встроенной библиотеке обычно имеется только функция $\arctg$. Отсутствующие функции $\arcsin$ и $\arccos$ можно вычислить либо создав собственные варианты этих функций

$$\arcsin(x) = \text{if} \left(|x| < 1: \arctg \left(x / \sqrt{1-x^2} \right), \text{sgn}(x) \cdot \pi/2 \right), \quad (3.36a)$$

$$\arccos(x) = \text{if} \left(x: \arctg \left(\sqrt{1-x^2} / x \right) + \{x < 0\} \cdot \pi, \pi/2 \right), \quad (3.36б)$$

либо суммируя до сходимости по заданной погрешности ε соответствующие ряды Тейлора

$$\arcsin(x) = x + \frac{1}{2 \cdot 3} x^3 + \frac{3}{2 \cdot 4 \cdot 5} x^5 + \frac{3 \cdot 5}{2 \cdot 4 \cdot 6 \cdot 7} x^7 + \dots = x + \sum_{k=1}^{\infty} \frac{x^{2k+1}}{2k+1} \prod_{i=2}^{2k} i^{(-1)^{i-1}},$$

$$\arccos(x) = \pi/2 - \arcsin(x).$$

Оба ряда у границ области $|x| \leq 1$ сходятся очень медленно (более 4000 итераций) и с большой ошибкой (до 0.0086 рад или 0.5°) при пороге сложения $\varepsilon = 10^{-6}$.

В настоящее время наиболее быстрым считается **табличный метод** вычисления тригонометрических функций с кусочно-линейной интерполяцией между узлами сетки. Суть его в том, что при запуске программы **один раз** с высокой точностью вычисляется $N+1$ узловое значение наиболее востребованной в машинной графике монотонно убывающей функции $c(x) = \cos(x)$ в интервале $x \in [0, \pi/2]$ с шагом $\Delta x = \pi/2N$. Таким образом, в оперативной памяти ЭВМ создаётся $2 \times (N+1)$ -матрица (таблица) узлов сетки $x_i = i \cdot \Delta x$, $c_i = \cos(x_i) \quad \forall i = \overline{0, N}$.

□ При вычислении **тригонометрических функций** $\cos(x)$ и $\sin(x)$ произвольный аргумент x циклическим смещением $x = x - 2\pi \cdot \text{sgn}(x)$ приводится в интервал $-\pi < x \leq \pi$. Далее используются известные тождества, приводящие аргумент в интервал $[0, \pi/2]$:

- если $0 \leq x \leq \pi/2$, то $\cos(x) = c(x)$, $\sin(x) = c(\pi/2 - x)$; (3.37a)

- если $\pi/2 < x \leq \pi$, то $\cos(x) = -c(\pi - x)$, $\sin(x) = c(x - \pi/2)$; (3.37б)

- если $-\pi/2 \leq x < 0$, то $\cos(x) = c(-x)$, $\sin(x) = -c(\pi/2 + x)$; (3.37в)

- если $-\pi < x < -\pi/2$, то $\cos(x) = -c(\pi + x)$, $\sin(x) = -c(-\pi/2 - x)$. (3.37г)

Вычисляя значение $c(\xi)$ по аргументу $0 \leq \xi \leq \pi/2$, определим индекс $i = \lfloor \xi / \Delta x \rfloor$, при котором $x_i \leq \xi < x_{i+1}$ (рис. 3.15). Тогда приближённое линейно интерполированное значение функции равно

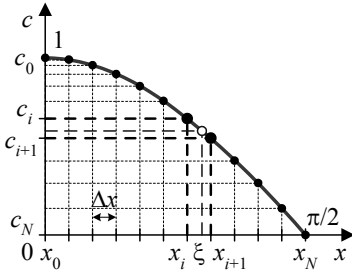


Рис. 3.15

$$c(\xi) \approx c_i + \frac{c_{i+1} - c_i}{\xi / \Delta x - i}.$$

Допустим, составлена таблица $\{x_i, c_i\}$ с крупным шагом $\Delta x = 0.01745$ рад (т. е. 1°) и задан аргумент $x = -14.2$ рад. Сместив его на два периода, получим $x = -14.2 + 4\pi = -1.6336$ рад или -93.6° . Вычислим по (3.37г) функции $\cos(x)$ и $\sin(x)$:

• $\xi = \pi + x = 1.508$ рад $\Rightarrow i = 86$, $c_{86} = 0.06976$, $c_{87} = 0.05234$, откуда получаем значение $-c(\xi) \approx -0.062789$, в то время как $\cos(x) = -0.062792$. Абсолютная и относительная погрешности интерполяции составили $2.25 \cdot 10^{-6}$ и $3.59 \cdot 10^{-5}$;

• $\xi = -\pi/2 - x = 0.06283$ рад $\Rightarrow i = 3$, $c_3 = 0.99863$, $c_4 = 0.99756$, откуда получаем $-c(\xi) \approx -0.99799$, тогда как $\sin(x) = -0.998027$. Абсолютная и относительная погрешности интерполяции составили $3.648 \cdot 10^{-5}$ и $3.655 \cdot 10^{-5}$.

□ Расчёт **обратных тригонометрических функций** $\arccos(a) \in [0, \pi]$ (рис. 3.16) и $\arcsin(a) \in [-\pi/2, \pi/2]$ (рис. 3.17) аргумента $-1 \leq a \leq 1$ начинается с поиска индекса i , для которого $c_{i+1} \leq |a| \leq c_i$. С помощью линейно-интерполяционной функции

$$L(a, i) = x_i + \frac{a - c_i}{c_{i+1} - c_i} \Delta x$$

получим приближённые значения

$$\arccos(a) \approx \begin{cases} L(a, i) & \text{при } a \geq 0, \\ \pi - L(-a, i) & \text{при } a < 0; \end{cases} \quad (3.38a)$$

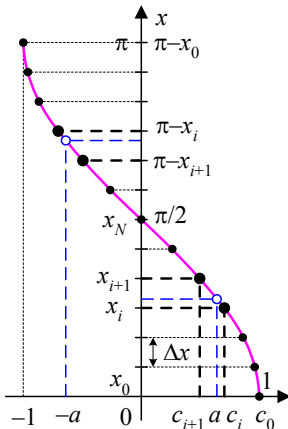


Рис. 3.16

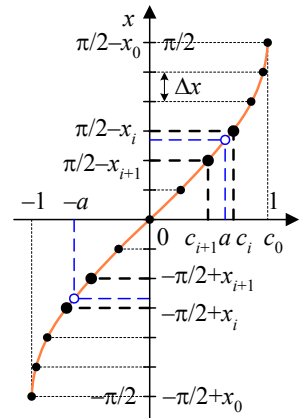


Рис. 3.17

$$\arcsin(a) \approx \begin{cases} \pi/2 - L(a, i) & \text{при } a \geq 0, \\ L(-a, i) - \pi/2 & \text{при } a < 0. \end{cases} \quad (3.38б)$$

Вычислим по набору $\{x_i, c_i\} \forall i = \overline{0, 90}$ функции $\arccos(a)$ и $\arcsin(a)$ для $a = -0.98$. Так как $c_{11} = 0.982$ и $c_{12} = 0.978$, то $i = 11$. Тогда по (3.38) получим

$$\arccos(a) \approx \pi - L(-a, i) = 2.941445 \text{ рад},$$

$$\arcsin(a) \approx L(-a, i) - \pi/2 = -1.370648 \text{ рад}.$$

Точные значения углов равны, соответственно, 2.941258 рад и -1.370461 рад. Даже на такой крупной сетке с шагом 1° максимальная абсолютная погрешность вычислений на границах интервала $-1 \leq a \leq 1$ составила $4.767 \cdot 10^{-4}$ рад или 0.03° .

Нелинейные элементарные преобразования

Если параметры матриц аффинных преобразований (3.31)–(3.35) сделать зависимыми от координат точки p , то преобразования станут **нелинейными**. Их ещё называют **преобразованиями деформации**, поскольку они неравномерно изменяют формы фигур в разных направлениях и плоскостях сечений.

Зависимый параметр матрицы преобразования может быть как функцией отдельно взятой координаты точки, например, $\phi(y)$, так и функциональной зависимостью нескольких координат, например, полярного радиуса $\phi(\sqrt{x^2 + z^2})$ или длины радиус-вектора $\phi(|p|)$. Соответствующее преобразование может обладать свойством симметрии относительно плоскости xz , оси y или начала СК.

Число способов деформации тел, в отличие от небольшого количества элементарных АП, неисчислимо, и каждый способ даёт свой неповторимый эффект. Проиллюстрируем действие лишь некоторых нелинейных преобразований на примере деформации параллелепипеда, изображённого на рис. 3.18, *а*. Для сравнения там же построена нейтральная плоскость xz , сечение которой объекта не деформируется ни одним из приведённых ниже преобразований. Отметим, что все сконструированные нами матрицы деформаций имеют только один зависимый параметр и только от одной координаты y — алгебраического расстояния от нейтральной плоскости до горизонтального сечения, в котором лежит точка. Присущие преобразованиям функциональные коэффициенты задаются пользователем при настройке деформации для конкретного объекта:

- рис. 3.18, *б* иллюстрирует **деформацию линейного уклона** левой и правой граней параллелепипеда навстречу друг другу матрицей $M(1 - \alpha y, 1, 1)$, которая масштабирует горизонтальные слои вдоль оси x по линейной зависимости от их ординат. Коэффициент $\alpha = \text{tg}(\varphi)$ управляет углом φ наклона левой и правой граней. Показанная фигура построена при $\alpha > 0$, но если нужно получить расширение объекта вверх, то следует задавать значение $\alpha < 0$. При деформации параллелепипеда матрицей $M(1 - \alpha_x y, 1, 1 - \alpha_z y)$ получается фигура, называемая **пьеDESTалом**, у которой углы наклонов боковых граней φ_x и φ_z регулируются коэффициентами α_x и α_z ;

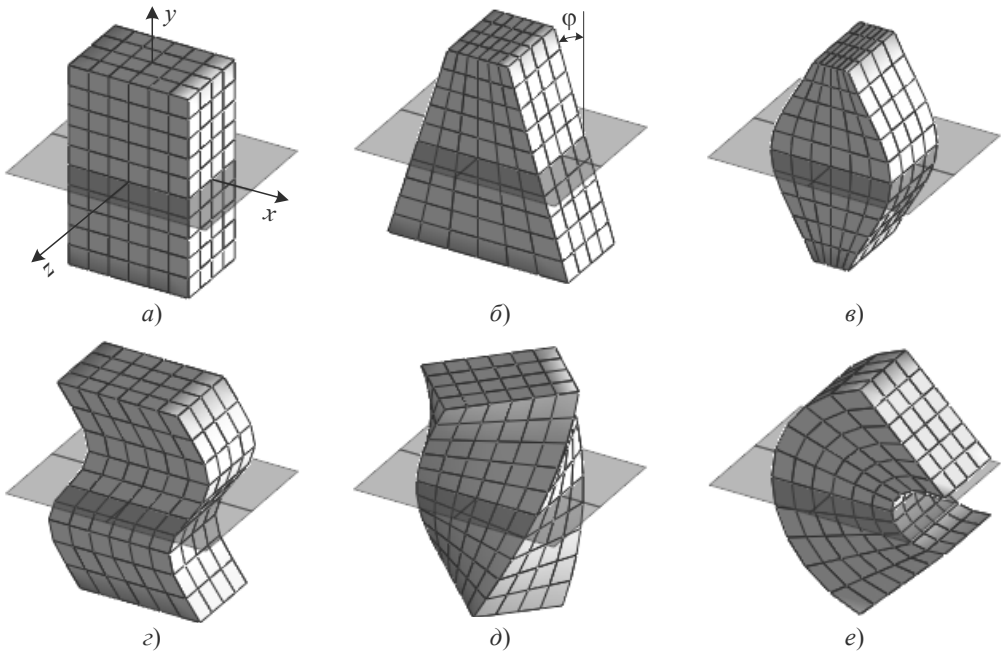


Рис. 3.18

- на рис. 3.18,б показана **деформация функционального уклона** граней параллелепипеда матрицей $M(\phi(y), 1, 1)$, масштабирующей горизонтальные слои вдоль оси x по зависимости $\phi(y) = \exp(\beta y^2)$ со свойствами $\phi(0) = 1$ и $\phi'(0) = 0$. Коэффициент $\beta < 0$ регулирует сужение, а $\beta > 0$ — расширение слоя от его высоты. Пьедестал с функциональными уклонами четырёх боковых граней изготавливается путём деформации параллелепипеда матрицей $M(\phi_x(y), 1, \phi_z(y))$;

- на рис. 3.18,г показана **деформация функционального сдвига** с матрицей $T(\phi(y)x^0)$, смещающей горизонтальные слои фигуры параллельно нейтральной плоскости вдоль оси x на расстояния $\phi(y) = A \sin(\omega y)$. При использовании линейной функции $\phi(y) = \beta y$ получается **линейный сдвиг** с матрицей (3.33б);

- рис. 3.18,д демонстрирует **деформацию кручения** (twisting) горизонтальных слоев тела вокруг оси y пропорционально их удалению от нейтральной плоскости с помощью матрицы $R_y(\gamma y)$. Коэффициент γ есть погонный угол кручения — приращение угла поворота слоя, приходящееся на единицу высоты;

- последняя представленная на рис. 3.18,е **деформация изгиба** (bending) выполняется вращением горизонтальных слоев тела вокруг оси z на угол, пропорциональный их удалению от нейтральной плоскости, с помощью матрицы $R_z(\delta y)$ с погонным углом изгиба δ .

Разумеется, в различных графических программах и системах автоматизированного проектирования применяются и другие нелинейные преобразования, которые здесь не довелось достойно представить. Тем не менее теперь читатель

ознакомлен с общими принципами деформирования плоских и объёмных фигур и сможет впредь самостоятельно разобраться в нехитрой механике инструментов получения сложных форм объектов из простых примитивов.

3.1.3. Сложные аффинные преобразования

Сложным (комбинированным) называется преобразование, реализующее цепочку из двух и более разнотипных элементарных преобразований. Матрица сложного преобразования находится по (3.12) перемножением матриц ЭП **в порядке их выполнения**. Если некоторая цепочка элементарных преобразований описывается матрицей C_1 вида (3.9а), включающей перенос на вектор B_1 , а затем делается перенос на B_2 , то матрица полного преобразования получается простым добавлением вектора B_2 к нижнему левому блоку матрицы C_1 :

$$C_1 T(B_2) = \begin{bmatrix} A_1 & O_{n,1} \\ B_1 & 1 \end{bmatrix} \begin{bmatrix} E_n & O_{n,1} \\ B_2 & 1 \end{bmatrix} = \begin{bmatrix} A_1 & O_{n,1} \\ B_1 + B_2 & 1 \end{bmatrix}.$$

Почти все аффинные преобразования не коммутативны, т. е. **результат цепочки преобразований зависит от порядка их выполнения**. Для цепочки из двух АП с матрицами C_1 и C_2 некоммутативность $C_1 C_2 \neq C_2 C_1$ соответствует одноимённому свойству операции перемножения матриц. Например, комбинации вращения объекта вокруг оси $\omega \in \{x, y, z\}$ на угол φ и переноса на вектор B имеют разные матрицы сложного АП:

$$\tilde{R}_\omega(\varphi) T(B) = \begin{bmatrix} R_\omega(\varphi) & O_{n,1} \\ B & 1 \end{bmatrix} \neq T(B) \tilde{R}_\omega(\varphi) = \begin{bmatrix} R_\omega(\varphi) & O_{n,1} \\ BR_\omega(\varphi) & 1 \end{bmatrix}.$$

Как исключение, **попарно коммутативны лишь преобразования перенос – перенос и масштабирование – масштабирование**:

$$T(B_1) T(B_2) = \begin{bmatrix} E_n & O_{n,1} \\ B_1 + B_2 & 1 \end{bmatrix} = T(B_2) T(B_1),$$

$$M(m_x, m_y, m_z) M(\mu_x, \mu_y, \mu_z) = \begin{bmatrix} m_x \mu_x & 0 & 0 \\ 0 & m_y \mu_y & 0 \\ 0 & 0 & m_z \mu_z \end{bmatrix} = M(\mu_x, \mu_y, \mu_z) M(m_x, m_y, m_z).$$

При выполнении преобразований **не теряйте из виду систему координат!** Например, вращение куба вокруг оси x , а затем вокруг оси y , **оставшейся неподвижной**, даёт матрицу $R_x R_y$ и конечное положение куба, показанное в конце цепочки $1 \rightarrow 2$ на рис. 3.19. Если же второе вращение куба сделать вокруг

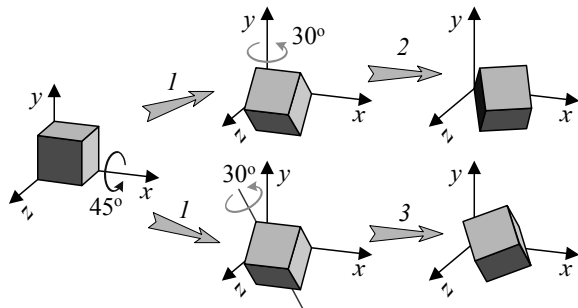


Рис. 3.19

отклонённого ребра, которое вначале было вертикальным (цепочка $1 \rightarrow 3$), то матрица полного преобразования с учётом вращения системы координат вокруг оси x , второго поворота куба и возврата СК в исходное состояние будет составлять $R_x R_x^{-1} R_y R_x = R_y R_x \neq R_x R_y$. Разные конечные положения наглядно доказывают, что **преобразования вращения вокруг разных осей не коммутативны**.

3.1.3.1. Методы расчёта матрицы сложного преобразования

Рассмотрим два принципиально различных метода расчёта матрицы сложного преобразования C в задачах с произвольным расположением осей вращения, прямых и плоскостей отражения.

1. Метод разложения (декомпозиции) сложного преобразования объекта на N элементарных преобразований, описываемых матрицами C_i **относительно координатных осей и плоскостей неподвижной СК**. Тогда по (3.12) получаем матрицу сложного преобразования $C = C_1 C_2 \dots C_N$.

Разложение процесса на составные части — отличительная черта «метода серого ящика», в котором форма (начальное и конечное состояния объекта) наполняется содержанием (промежуточными состояниями и способами их достижения), возможно, не единственным способом. Искусство программиста, достигаемое интуицией и опытом, заключается в оптимальном выборе последовательности элементарных преобразований и правильном расчёте их параметров.

Решим методом декомпозиции **задачу 2d-преобразования произвольного треугольника** $P = p_1 p_2 p_3$ с вершинами $p_i = [x_i \ y_i]$ в произвольный треугольник $P' = p'_1 p'_2 p'_3$ с вершинами $p'_i = [x'_i \ y'_i] \ \forall i = \overline{1,3}$ (рис. 3.20,а). Для определения числа N звеньев цепочки и их параметров запустим параллельно две ветви — прямую ветвь преобразования объекта P и обратную ветвь преобразования объекта P' до совпадения треугольников в точке соединения ветвей:

- **переносим** P и P' матрицами $C_1 = T(-p_1)$ и $C_N^{-1} = T(-p'_1)$ в положения $\tilde{P} = \tilde{P} C_1$ (б) и $\tilde{P}' = \tilde{P}' C_N^{-1}$ (в) с вершинами $p_1 = p'_1 = O_2$ в начале СК;

- **вращаем** P и P' матрицами $C_2 = \tilde{R}(-\varphi)$ и $C_{N-1}^{-1} = \tilde{R}(-\varphi')$ с углами $\varphi = \arg(p_2)$ и $\varphi' = \arg(p'_2)$ в положения $\tilde{P} = \tilde{P} C_2$ (з) и $\tilde{P}' = \tilde{P}' C_{N-1}^{-1}$ (д) с лежащими на правой части оси x вершинами $p_2 = [x_2 \ 0]$ и $p'_2 = [x'_2 \ 0]$;

- **масштабируем** треугольник P матрицей $C_3 = \text{diag}[m_x \ m_y \ 1]$ с коэффициентами $m_x = x'_2/x_2$ и $m_y = y'_3/y_3$, дающими в треугольниках $\tilde{P} = \tilde{P} C_3$ (е) и $\tilde{P}'$ (д) равные координаты $x_2 = x'_2$ и $y_3 = y'_3$;

- **сдвигаем** треугольник P вдоль оси x матрицей $C_4 = \begin{bmatrix} 1 & 0 & 0 \\ s_{yx} & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ с параметром $s_{yx} = (x'_3 - x_3)/y_3$, совмещающим треугольники $\tilde{P} = \tilde{P} C_4$ и $\tilde{P}'$ на рис. 3.20,д.

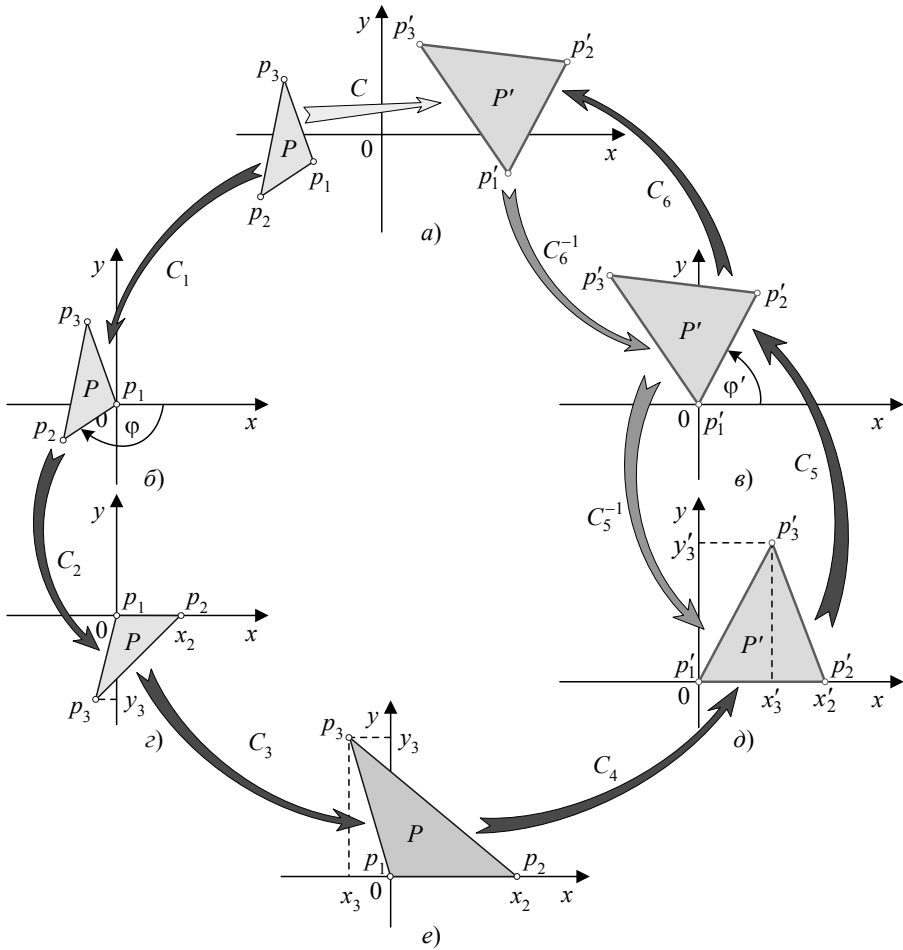


Рис. 3.20

Таким образом, при $N-1=5$ требуется не более $N=6$ элементарных преобразований, переводящих треугольник из любого начального состояния P в любое конечное состояние P' . Матрица сложного преобразования $P \rightarrow P'$ получается перемножением четырёх матриц ЭП прямой ветви на две инверсные матрицы ЭП обратной ветви:

$$C = T(-p_1) \tilde{R}(-\varphi) \tilde{M}(m_x, m_y) \tilde{S}_3(s_{yx}) \tilde{R}(\varphi') T(p'_1). \quad (3.39)$$

Отметим, что в частных конфигурациях треугольников некоторые матрицы ЭП могут быть единичными, а фактическое число ЭП может стать меньше шести:

- при расположении вершин p_1 и/или p'_1 в начале координат отсутствуют один или оба крайних переноса;
- при нулевых углах φ и/или φ' отсутствуют одно или оба вращения;
- при $m_x=m_y=1$ отсутствует масштабирование;
- если в положениях (д) и (е) получатся равные значения $x'_3=x_3$, то параметр $s_{yx}=0$ превращает матрицу сдвига в единичную.

Испытаем разработанный здесь метод декомпозиции на двух тройках точек $p_1 p_2 p_3$ и $p'_1 p'_2 p'_3$ на рис. 3.4,а. Матрицы шести ЭП принимают следующий вид:

$$C_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 1 & 1 \end{bmatrix}, C_2 = \begin{bmatrix} -0.707 & -0.707 & 0 \\ 0.707 & -0.707 & 0 \\ 0 & 0 & 1 \end{bmatrix}, C_3 = \begin{bmatrix} 1.904 & 0 & 0 \\ 0 & -1.182 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$C_4 = \begin{bmatrix} 1 & 0 & 0 \\ 0.722 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, C_5 = \begin{bmatrix} -0.371 & 0.928 & 0 \\ -0.928 & -0.371 & 0 \\ 0 & 0 & 1 \end{bmatrix}, C_6 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 5 & -2 & 1 \end{bmatrix},$$

а матрица сложного преобразования $C = C_1 C_2 C_3 C_4 C_5 C_6$ полностью совпадает с матрицей, вычисленной в примере 3.1.

Полезным приёмом, часто облегчающим выбор ЭП, является **временный** перевод системы координат матрицей C_k в такое положение $x'y'z'$, где оси вращения, прямые и плоскости отражения становятся **координатными**. В изменённой СК выполняются необходимые активные преобразования с общей матрицей C_a . Восстановление системы координат в исходное положение делает обратная матрица C_k^{-1} . Таким образом, сложное АП имеет матрицу

$$C = C_k C_a C_k^{-1}. \quad (3.40)$$

При расчёте матрицы C_k необходимо помнить об инвертировании знаков параметров элементарных преобразований системы координат — вектора переноса её начала и углов вращений вокруг осей СК.

2. Метод парных точек, формирующий матрицу преобразования в n -мерном пространстве по $n+1$ -й паре элементов образа и прообраза. Это принципиальное безразличие к способам перехода от начального состояния объекта к конечному — явный признак «метода чёрного ящика». Обращаемые в (3.17)–(3.23) матрицы будут невырождены при условии, что тройки точек в R^2 не коллинеарны, а четвёрки точек в R^3 — не компланарны. Наилучшая обусловленность обращения достигается при ортогональности систем векторов $V_1 \div V_n$ и $V'_1 \div V'_n$, построенных на точках образа и прообраза.

Читателю предлагается оценить эффективность МПТ на двух примерах. В первом примере 3.1 (см. рис. 3.4,а) матрица C рассчитана непосредственно по координатам заданных точек $\{p_1, p_2, p_3\}$ и $\{p'_1, p'_2, p'_3\}$ по единственной формуле (3.17а) за две матричные операции обращения и умножения! А теперь сравните трудоёмкость получения численно такой же матрицы сложного преобразования методом декомпозиции (3.39) в виде **правильно** составленного произведения матриц всех ЭП из п. 3.1.2 с **правильно** рассчитанными параметрами этих матриц — и вы почувствуете разницу методов!

 **Пример 3.2.** Рассчитать масштабный коэффициент μ_3 матрицу C_3 пропорционального отображения прямоугольного окна $\{x_{\min} \div x_{\max}, y_{\min} \div y_{\max}\}$ на плоскости xu МСК в центр окна $\{left \div right, up \div down\}$ на плоскости $x_3 y_3$ ЭСК (рис. 3.21).

Решение. Обозначим *коэффициенты прямоугольности* окон в МСК и ЭСК:

$$\rho = \frac{y_{\max} - y_{\min}}{x_{\max} - x_{\min}}, \quad \rho_3 = \frac{\text{down} - \text{up}}{\text{right} - \text{left}}.$$

У *пропорциональных* окон коэффициенты прямоугольности ρ и ρ_3 равны, а окно МСК отображается во всё окно ЭСК. В этом случае матрица C_3 наиболее просто находится *методом парных точек* (3.17а) при $n=2$. Хотя постановка данной задачи плоская, но

для использования её результата в пространственных преобразованиях выберем *четыре* пары точек в *трёхмерных* МСК и ЭСК, причём МСК-точки не должны быть компланарны:

$$\begin{aligned} p_1 &= [x_{\min} \ y_{\min} \ 0] \rightarrow p_{13} = [\text{left} \ \text{down} \ 0], \\ p_2 &= [x_{\min} \ y_{\max} \ 0] \rightarrow p_{23} = [\text{left} \ \text{up} \ 0], \\ p_3 &= [x_{\max} \ y_{\max} \ 0] \rightarrow p_{33} = [\text{right} \ \text{up} \ 0], \quad z^0 \rightarrow O_3. \end{aligned}$$

Тогда

$$C_3 = \begin{bmatrix} x_{\min} & y_{\min} & 0 & 1 \\ x_{\min} & y_{\max} & 0 & 1 \\ x_{\max} & y_{\max} & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} \text{left} & \text{down} & 0 & 1 \\ \text{left} & \text{up} & 0 & 1 \\ \text{right} & \text{up} & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mu_3 = \frac{\text{right} - \text{left}}{x_{\max} - x_{\min}} = \frac{\text{down} - \text{up}}{y_{\max} - y_{\min}}. \quad (3.41)$$

В случае *непропорциональных* окон получим матрицу C_3 *методом разложения*, свершив цепочку элементарных преобразований исходного окна, заданного в МСК.

Шаг 1. *Перенос* центра окна $p_{ц}$ в начало МСК матрицей

$$T_1 = T([-x_{ц} \ -y_{ц} \ 0]), \quad \text{где } x_{ц} = 0.5(x_{\min} + x_{\max}), \quad y_{ц} = 0.5(y_{\min} + y_{\max}). \quad (3.42a)$$

Шаг 2. Пропорциональное *масштабирование* окна по осям x и y , инвертирование направления оси y и формальное обнуление z -координаты матрицей

$$M = \text{diag}[\mu_3, -\mu_3, 0], \quad \text{где } \mu_3 = \begin{cases} (\text{right} - \text{left}) / (x_{\max} - x_{\min}) & \text{при } \rho_3 \geq \rho, \\ (\text{down} - \text{up}) / (y_{\max} - y_{\min}) & \text{при } \rho_3 < \rho. \end{cases} \quad (3.42б)$$

Шаг 3. *Перенос* центра окна в точку $p_{эц}$ в ЭСК матрицей

$$T_2 = T([x_{эц} \ y_{эц} \ 0]), \quad \text{где } x_{эц} = 0.5(\text{left} + \text{right}), \quad y_{эц} = 0.5(\text{up} + \text{down}). \quad (3.42в)$$

Матрица полного преобразования равна

$$C_3 = T_1 \tilde{M} T_2 = \begin{bmatrix} \mu_3 & 0 & 0 & 0 \\ 0 & -\mu_3 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ x_{эц} - \mu_3 x_{ц} & y_{эц} + \mu_3 y_{ц} & 0 & 1 \end{bmatrix}. \quad (3.42г)$$

Коэффициенты этой матрицы дают *прямые уравнения пересчёта координат точки из МСК в ЭСК* (координата z_3 вектора $\tilde{p}_3 = \tilde{p} C_3$ при этом получается нулевой):

$$x_3 = x_{эц} + \mu_3(x - x_{ц}), \quad y_3 = y_{эц} - \mu_3(y - y_{ц}). \quad (3.42д)$$

Описанный алгоритм гарантирует, что любое изображение в прямоугольном окне без искажений отобразится в любом желаемом окне дисплея, заполнив его максимально полностью и симметрично относительно центра.

Если при расчёте масштабного коэффициента μ_3 по (3.42б) заранее неизвестны границы окна МСК, целиком вмещающего сцену, то можно задать значение $\mu_3 = 40$ благодаря тому, что

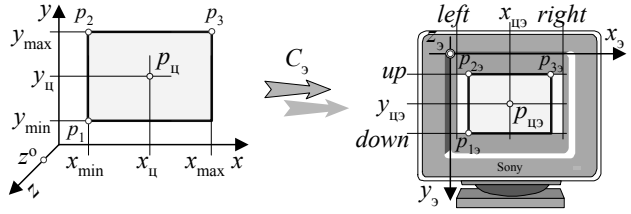


Рис. 3.21

один сантиметр экрана (40 пикселей) соответствует одной единице длины в пространстве МСК. Наконец, можно выбрать единицу длины в МСК равной размеру пиксела 0.25 мм. Тогда $\mu_0=1$, что сильно упрощает матрицу (3.42г) и координатные уравнения (3.42д):

$$C_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ x_{цз}-x_{ц} & y_{цз}+y_{ц} & 0 & 1 \end{bmatrix}, \quad x_3 = x_{цз} - x_{ц} + x, \quad y_3 = y_{цз} + y_{ц} - y. \quad (3.42e)$$

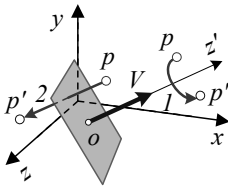


Рис. 3.22

Освоим методы вычисления матрицы сложного аффинного преобразования на примере распространённой **операции изменения системы координат**, состоящей в совмещении её начала с заданной точкой o , а какой-либо координатной оси, допустим, z — с направлением заданного вектора $V \neq O_3$ (рис. 3.22). Положение остальных осей в изменённой СК не регламентируется. Данное преобразование применяется в двух наиболее востребованных преобразованиях точки — вращения (1) вокруг произвольной прямой $\{o, V\}$ и отражения (2) от произвольной плоскости $\{o, V\}$ вдоль её нормали V .

Изменение системы координат происходит за два преобразования.

1. **Перенос** начала СК в точку o . Выполняется матрицей $T(-o)$.
2. **Поворот** оси z в направлении вектора V , выходящего из начала новой СК. Обозначим матрицу этого преобразования как $A_K(V)$.

Расширенная матрица полного преобразования СК равна

$$C_K(o, V) = T(-o) \tilde{A}_K(V). \quad (3.43)$$

Рассмотрим различные **методы вычисления матрицы** $A_K(V)$.

Метод координатных вращений

Алгоритм разложения, основанный на **методе координатных вращений (МКВ)**, рассматривается во многих учебниках [6, 51, 61, 62] и состоит в проецировании вектора V на координатную плоскость, например, xz , нахождении длины проекции $d = \sqrt{V_x^2 + V_z^2} \geq 0$ и **двух вращений системы координат** xyz вокруг её осей (рис. 3.23):

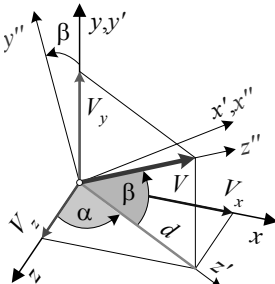


Рис. 3.23

- первое вращение вокруг оси y на **угол азимута** α :

$$\cos(\alpha) = V_z/d, \quad \sin(\alpha) = V_x/d. \quad (3.44a)$$

В результате СК принимает положение $x'y'z'$, в котором ось z' направлена по отрезку d ;

- второе вращение СК вокруг её **новой** оси x' на обратный **угол места** $-\beta$:

$$\cos(\beta) = d/|V|, \quad \sin(\beta) = V_y/|V|. \quad (3.44б)$$

После этого СК принимает желаемое положение $x''y''z''$ с осью $z'' \parallel V$.

Таким образом, **матрица $A_k(V)$ преобразования СК методом координатных вращений** с учётом инвертирования знаков углов приобретает вид

$$A_{\text{ккв}}(V) = R_y(-\alpha)R_x(\beta). \quad (3.44\text{в})$$

Всегда ли верно это решение? Очевидно, что **при $V_x=V_z=d=0$ угол α в (3.44а) не определён** из-за **несовместных** значений $\cos(\alpha)=\sin(\alpha)=0$, а **матрица $R_y(-\alpha)$ неверна** при любом итоге операции $0/0$. Последний же зависит от алгоритма микропрограммы деления числителя c на знаменатель $d=0$:

- если проверка числителя на значение $c=0$ не производится, то, как правило, все вычисления прекращаются с **сообщением об ошибке**;

- если перед делением обнаружено $c=0$, то $0/d=0$ для любых значений d . Именно так делятся числа в MathCAD [18]. Из (3.35б) видно, что матрица $R_y(-\alpha)$ с $\cos(\alpha)=0$ и $\sin(\alpha)=0$ совершает не вращение, а совсем другое преобразование $M(0,1,0)$ — проецирование любой точки пространства на ось y .

Следовательно, **случай $V \parallel y^0$ должен быть особым** случаем совмещения оси z с вектором V не за два, а за **одно вращение** СК вокруг оси x на угол -90° при $V \uparrow \uparrow y^0$ или $+90^\circ$ при $V \uparrow \downarrow y^0$. Матрицу такого вращения

$$A_{\text{ккв}}(V) = R_x(\text{sgn}(V_y) \cdot 90^\circ) \quad (3.44\text{г})$$

можно сделать частным случаем матрицы (3.44в), приняв $\alpha=0$ при $d=0$.

Однако, и при $V \# y^0$ метод (3.44в) имеет скрытый **недостаток** и **особенности** программирования, даже не обсуждаемые в перечисленных учебниках:

- легко совершить ошибку при подстановке в матрицы вращений (3.35) **неверных** аргументов $\alpha = \arccos(V_z/d)$ и $\beta = \arccos(d/|V|)$, вычисленных по (3.44а,б) в интервале $[0^\circ, 180^\circ]$, тогда как эти углы фактически изменяются в интервалах $\alpha \in (-180^\circ, 180^\circ]$ и $\beta \in [-90^\circ, 90^\circ]$. Аналогично нельзя вычислять угол первого вращения как $\alpha = \arcsin(V_x/d) \in [-90^\circ, 90^\circ]$. Правильным будет его расчёт с помощью определённой в (1.4) функции $\arg$ с 2d-аргументом $[V_z \ V_x]$. Итак:

$$\alpha = \text{if}(V_x^2 + V_z^2 : \arg([V_z \ V_x]), 0), \quad \beta = \arcsin(V_y/|V|); \quad (3.44\text{д})$$

- поскольку матрицам вращения нужны, вообще-то, не сами углы, а их косинусы и синусы, которые в (3.44а,б) явно выражены через элементы вектора V , то рекомендуется создать **модификации матриц вращения** (3.34) и (3.35) с двумя аргументами c и s — косинусом и синусом угла вращения:

$$\begin{aligned} R2(c,s) &= \begin{bmatrix} c & s \\ -s & c \end{bmatrix}, \quad R2_x(c,s) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & s \\ 0 & -s & c \end{bmatrix}, \\ R2_y(c,s) &= \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix}, \quad R2_z(c,s) = \begin{bmatrix} c & s & 0 \\ -s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned} \quad (3.45)$$

Эти матрицы рекомендуется использовать даже в том случае, когда исходно задан угол вращения φ : вычислив **один раз** значения косинуса $c = \cos(\varphi)$ и синуса $s = \sin(\varphi)$, использовать их аргументами функций (3.45) вместо **двукратного** вычисления в (3.34) и (3.35);

- подстановка косинусов и синусов углов α и β из (3.44а,б) в матрицы (3.45) и перемножение последних по (3.44в) с правилом сокращения $d/d=1 \quad \forall d$, в том числе и при $d=0$, даёт **явную форму матрицы преобразования**

$$A_{\text{ккв}}(V) = \begin{bmatrix} z/d & -xy/d & x \\ 0 & d & y \\ -x/d & -yz/d & z \end{bmatrix}, \text{ где } d = \sqrt{x^2 + z^2} \neq 0, \quad (3.46a)$$

а x, y и z — координаты **нормированного** вектора $\bar{V} = [x \ y \ z]$:

$$x = V_x / |V|, \quad y = V_y / |V|, \quad z = V_z / |V|; \quad (3.46b)$$

- при $x=y=z=0$ матрица (3.46а) не вычислима и превращается в правильную матрицу (3.44г) только путём **ручного** преодоления неопределённостей $0/0$ по **противоречивым** правилам $z/d=1$ и $x/d=0$.

Используем (3.46) в выводе матриц $A = A_{\text{ккв}}(V) A_a A_{\text{ккв}}^{-1}(V)$ двух самых востребованных в компьютерной графике сложных преобразований вращения и отражения, показанных на рис. 3.22.

1. **Матрица вращения** точки на угол φ вокруг **центрированной** (проходящей через начало СК) прямой в направлении произвольного вектора V , полученная при задании активной матрицы элементарного вращения $A_a = R_z(\varphi)$:

$$A_{\text{вр}} = \begin{bmatrix} x^2(1-c)+c & xy(1-c)+zs & xz(1-c)-ys \\ xy(1-c)-zs & y^2(1-c)+c & yz(1-c)+xs \\ xz(1-c)+ys & yz(1-c)-xs & z^2(1-c)+c \end{bmatrix} = (1-c)\bar{V}^T \bar{V} + cE_3 + s \begin{bmatrix} 0 & z & -y \\ -z & 0 & x \\ y & -x & 0 \end{bmatrix}, \quad (3.47a)$$

где $c = \cos(\varphi)$, $s = \sin(\varphi)$. В частных случаях $V \parallel x^0$, $V \parallel y^0$ и $V \parallel z^0$ получим, соответственно, матрицы вращений $R_x(\text{sgn}(x)\varphi)$, $R_y(\text{sgn}(y)\varphi)$ и $R_z(\text{sgn}(z)\varphi)$.

2. **Матрица отражения** точки от центрированной плоскости с произвольной нормалью V получена при задании активной матрицы элементарного отражения $A_a = M(1,1,-1)$:

$$A_{\text{отр}} = \begin{bmatrix} 1-2x^2 & -2xy & -2xz \\ -2xy & 1-2y^2 & -2yz \\ -2xz & -2yz & 1-2z^2 \end{bmatrix} = E_3 - 2\bar{V}^T \bar{V}. \quad (3.47b)$$

В частных случаях $V \parallel x^0$, $V \parallel y^0$ и $V \parallel z^0$ в (3.47б) получим, соответственно, матрицы отражений $M(-1,1,1)$, $M(1,-1,1)$ и $M(1,1,-1)$.

В (3.47) переменная d отсутствует благодаря **ручному** сокращению $d/d=1$. Напоминаем, что приведённые матрицы описывают лишь центрированные преобразования и не учитывают вспомогательных переносов СК в точку o и обратно. Расширенная матрица **группового** преобразования (3.11) имеет вид

$$C=T(-o)\tilde{A}_k(V)\tilde{A}_a\tilde{A}_k^{-1}(V)T(o). \quad (3.48a)$$

Ей соответствует сложное **точечное** преобразование в обычном пространстве

$$p'=(p-o)A_k(V)A_aA_k^{-1}(V)+o. \quad (3.48б)$$

В (3.48) $A_k^{-1}(V)=A_k^T(V)$ по свойству ортогональности матриц вращения.

Ручной аналитический вывод матриц сложных преобразований весьма громоздок, их конечные формы объёмны в записи и сложны в программировании. Автор призывает читателя не искушаться этим **порочным** методом разработки алгоритмов компьютерной графики по следующим причинам:

- в процессе сложных ручных выкладок легко наделать ошибок, примером чего являются некоторые неувязки при выводе матрицы (3.47a) в [61, с. 86];
- вместо элегантного однострочного перемножения матриц по (3.48a) поэлементное описание аналитически найденной матрицы C требует много места и большого внимания в процессе вывода. Этот путь ведет к разбуханию программного кода и является источником дополнительных ошибок и затрат времени на составление и отладку программы.

Расширенные матрицы некоторых сложных преобразований на **плоскости**, аналитически выведенные по (3.48a), приведены в табл. 3.1.

Таблица 3.1

Преобразование	Матрица преобразования
Масштабирование от точки o	$T(-o)\tilde{M}(m_x, m_y)T(o)=\begin{bmatrix} m_x & 0 & 0 \\ 0 & m_y & 0 \\ (1-m_x)o_x & (1-m_y)o_y & 1 \end{bmatrix}$
Отражение от точки o	$T(-o)\tilde{M}(-1, -1)T(o)=\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 2o_x & 2o_y & 1 \end{bmatrix}$
Отражение от прямой $\{o, V\}$ $\cos(\alpha)=\frac{V_x}{ V }, \sin(\alpha)=\frac{V_y}{ V }$	$\begin{aligned} T(-o)\tilde{R}(-\alpha)\tilde{M}(1, -1)\tilde{R}(\alpha)T(o)= \\ =\begin{bmatrix} 2\cos^2(\alpha)-1 & 2\cos(\alpha)\sin(\alpha) & 0 \\ 2\cos(\alpha)\sin(\alpha) & 2\sin^2(\alpha)-1 & 0 \\ X & Y & 1 \end{bmatrix}, \\ X=2\sin(\alpha)(o_x \sin(\alpha)-o_y \cos(\alpha)), \\ Y=-2\cos(\alpha)(o_x \sin(\alpha)-o_y \cos(\alpha)). \end{aligned}$
Отражение от прямой $y=ax+b$ $\alpha=\arctg(a), a <\infty$	$\begin{aligned} T(-by^o)\tilde{R}(-\alpha)\tilde{M}(1, -1)\tilde{R}(\alpha)T(by^o)= \\ =\begin{bmatrix} (1-a^2)/(1+a^2) & 2a/(1+a^2) & 0 \\ 2a/(1+a^2) & (a^2-1)/(1+a^2) & 0 \\ -2ab/(1+a^2) & 2b/(1+a^2) & 1 \end{bmatrix} \end{aligned}$
Отражение от прямой $x=a$	$T(-ax^o)\tilde{M}(-1, 1)T(ax^o)=\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 2a & 0 & 1 \end{bmatrix}$

Отражение от прямой $y=b$	$T(-by^0)\tilde{M}(1,-1)T(by^0)=\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 2b & 1 \end{bmatrix}$
Отражение от прямых $y=\pm x$	$\tilde{R}(\mp 45^\circ)\tilde{M}(1,-1)\tilde{R}(\pm 45^\circ)=\begin{bmatrix} 0 & \pm 1 & 0 \\ \pm 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Вращение вокруг точки o на угол φ	$T(-o)\tilde{R}(\varphi)T(o)=\begin{bmatrix} \cos(\varphi) & \sin(\varphi) & 0 \\ -\sin(\varphi) & \cos(\varphi) & 0 \\ X & Y & 1 \end{bmatrix},$ $X=o_x(1-\cos(\varphi))+o_y\sin(\varphi), Y=o_y(1-\cos(\varphi))-o_x\sin(\varphi)$
Сдвиг вдоль прямой $\{o,V\}$ $\cos(\alpha)=\frac{V_x}{ V }, \sin(\alpha)=\frac{V_y}{ V }$	$T(-o)\tilde{R}(-\alpha)\begin{bmatrix} 1 & 0 & 0 \\ s & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\tilde{R}(\alpha)T(o)=$ $=\begin{bmatrix} 1-s\sin(\alpha)\cos(\alpha) & -s\sin^2(\alpha) & 0 \\ s\cos^2(\alpha) & 1+s\sin(\alpha)\cos(\alpha) & 0 \\ s\cos(\alpha)(o_x\sin(\alpha)-o_y\cos(\alpha)) & s\sin(\alpha)(o_x\sin(\alpha)-o_y\cos(\alpha)) & 1 \end{bmatrix}$

Матрицы сложных преобразований в *пространстве* сведены в табл. 3.2.

Таблица 3.2

Преобразование	Матрица преобразования
Масштабирование от точки o	$T(-o)\tilde{M}(m_x, m_y, m_z)T(o)=\begin{bmatrix} m_x & 0 & 0 & 0 \\ 0 & m_y & 0 & 0 \\ 0 & 0 & m_z & 0 \\ (1-m_x)o_x & (1-m_y)o_y & (1-m_z)o_z & 1 \end{bmatrix}$
Отражение от точки o	$T(-o)\tilde{M}(-1,-1,-1)T(o)=\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 2o_x & 2o_y & 2o_z & 1 \end{bmatrix}$
Отражение от прямой $\{o,V\}$ $\cos(\alpha)=\frac{V_z}{d}, \sin(\alpha)=\frac{V_x}{d},$ $\cos(\beta)=\frac{d}{ V }, \sin(\beta)=\frac{V_y}{ V },$ $d=\sqrt{V_x^2+V_z^2}$	$V\#y^0:$ $T(-o)\tilde{R}_y(-\alpha)\tilde{R}_x(\beta)\tilde{M}(-1,-1,1)\tilde{R}_x(-\beta)\tilde{R}_y(\alpha)T(o)=$ $=\begin{bmatrix} 2\sin^2(\alpha)\cos^2(\beta)-1 & 2\sin(\alpha)\cos(\beta)\sin(\beta) & 2\cos(\alpha)\sin(\alpha)\cos^2(\beta) & 0 \\ 2\sin(\alpha)\cos(\beta)\sin(\beta) & 1-2\cos^2(\beta) & 2\cos(\alpha)\cos(\beta)\sin(\beta) & 0 \\ 2\cos(\alpha)\sin(\alpha)\cos^2(\beta) & 2\cos(\alpha)\cos(\beta)\sin(\beta) & 2\cos^2(\alpha)\cos^2(\beta)-1 & 0 \\ X & Y & Z & 1 \end{bmatrix}$ $X=2o_x(\sin^2(\beta)+\cos^2(\alpha)\cos^2(\beta))-2o_y\sin(\alpha)\sin(\beta)\cos(\beta)-$ $-2o_z\cos(\alpha)\sin(\alpha)\cos^2(\beta),$ $Y=2\cos(\beta)(-o_x\sin(\alpha)\sin(\beta)+o_y\cos(\beta)-o_z\cos(\alpha)\sin(\beta)),$ $Z=-2\cos(\alpha)\cos(\beta)(o_x\sin(\alpha)\cos(\beta)+o_y\sin(\beta))+2o_z(1-\cos^2(\alpha)\cos^2(\beta))$ $V\ x^0: T(-o)\tilde{M}(1,-1,-1)T(o)=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 2o_y & 2o_z & 1 \end{bmatrix}$

	$V\ y^0: T(-o)\tilde{M}(-1,1,-1)T(o)=\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 2o_x & 0 & 2o_z & 1 \end{bmatrix}$			
	$V\ z^0: T(-o)\tilde{M}(-1,-1,1)T(o)=\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 2o_x & 2o_y & 0 & 1 \end{bmatrix}$			
Отражение от плоскости $\{o,N\}$ $\cos(\alpha)=\frac{N_z}{v}, \sin(\alpha)=\frac{N_x}{v},$ $\cos(\beta)=\frac{d}{ N }, \sin(\beta)=\frac{N_y}{ N },$ $d=\sqrt{N_x^2+N_z^2}$	$N\#y^0: T(-o)\tilde{A}T(o), \text{ где}$ $A=R_y(-\alpha)R_x(\beta)M(1,1,-1)R_x(-\beta)R_y(\alpha)=E_3-2\frac{N^T N}{N \circ N}$			
	$N\ x^0: T(-o)\tilde{M}(-1,1,1)T(o)=\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 2o_x & 0 & 0 & 1 \end{bmatrix}$			
	$N\ y^0: T(-o)\tilde{M}(1,-1,1)T(o)=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 2o_y & 0 & 1 \end{bmatrix}$			
	$N\ z^0: T(-o)\tilde{M}(1,1,-1)T(o)=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 2o_z & 1 \end{bmatrix}$			
Вращение вокруг прямой $\{o,V\}$ на угол φ $\cos(\alpha)=\frac{V_z}{d}, \sin(\alpha)=\frac{V_x}{d},$ $\cos(\beta)=\frac{v}{ V }, \sin(\beta)=\frac{V_y}{ V },$ $v=\sqrt{V_x^2+V_z^2}, s_\omega=\text{sgn}(V_\omega),$ $x=\frac{V_x}{ V }, y=\frac{V_y}{ V }, z=\frac{V_z}{ V }$	$V\#y^0: T(-o)\tilde{A}T(o), \text{ где}$ $A=\tilde{R}_y(-\alpha)\tilde{R}_x(\beta)\tilde{R}_z(\varphi)\tilde{R}_x(-\beta)\tilde{R}_y(\alpha)=$ $=(1-\cos(\varphi))\frac{V^T V}{V \circ V}+\cos(\varphi)E_3+\sin(\varphi)\begin{bmatrix} 0 & z & -y \\ -z & 0 & x \\ y & -x & 0 \end{bmatrix}$			
	$V\ x^0: T(-o)\tilde{R}_x(s_x\varphi)T(o)=$ $=\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\varphi) & s_x \sin(\varphi) & 0 \\ 0 & -s_x \sin(\varphi) & \cos(\varphi) & 0 \\ 0 & o_y(1-\cos(\varphi))+s_x o_z \sin(\varphi) & o_z(1-\cos(\varphi))-s_x o_y \sin(\varphi) & 1 \end{bmatrix}$			
	$V\ y^0: T(-o)\tilde{R}_y(s_y\varphi)T(o)=$ $=\begin{bmatrix} \cos(\varphi) & 0 & -s_y \sin(\varphi) & 0 \\ 0 & 1 & 0 & 0 \\ s_y \sin(\varphi) & 0 & \cos(\varphi) & 0 \\ o_x(1-\cos(\varphi))-s_y o_z \sin(\varphi) & 0 & o_z(1-\cos(\varphi))+s_y o_x \sin(\varphi) & 1 \end{bmatrix}$			
	$V\ z^0: T(-o)\tilde{R}_z(s_z\varphi)T(o)=$ $=\begin{bmatrix} \cos(\varphi) & s_z \sin(\varphi) & 0 & 0 \\ -s_z \sin(\varphi) & \cos(\varphi) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ o_x(1-\cos(\varphi))+s_z o_y \sin(\varphi) & o_y(1-\cos(\varphi))-s_z o_x \sin(\varphi) & 0 & 1 \end{bmatrix}$			

Метод экономичного вращения

Поворот оси z до вектора V можно осуществить по **кратчайшему** углу $\beta = \angle(z^0, V)$ вокруг вектора $N = z^0 \times V = [-V_y, V_x, 0] \in xy$, нормального к плоскости вращения (рис. 3.24). Поскольку любой другой бóльший угол между векторами z^0 и V не лежит в плоскости $\{O_3, N\}$, описываемый ниже метод вращения назван автором «**экономичным**» (МЭВ).

Задача преобразования СК решается за **три** элементарных вращения:

- первое вокруг оси z на угол $\alpha = \angle(y^0, N)$ совмещает ось y' с вектором N ;
- второе вращение СК $x'y'z'$ вокруг её оси y' на угол β совмещает ось z'' с вектором V ;
- третье вращение СК $x''y''z''$ вокруг её оси z'' на угол $-\alpha$ ставит оси x''' и y''' в положения, соответствующие кратчайшему повороту исходной системы xuz вокруг вектора N на угол β . Без него поворот $xuz \rightarrow x''y''z''$ будет не кратчайшим, а произойдёт вокруг какой-то другой оси, поскольку невозможно за один

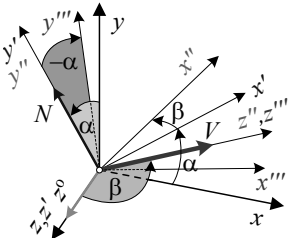


Рис. 3.24

раз повернуть СК xuz с осью y , отклонённой от вектора N на угол $\alpha \neq 0$, в положение $x''y''z''$ с осью y'' , совпадающей с его направлением.

В случае $V \parallel z^0$ первое и третье вращения не нужны, к тому же угол α становится невычислимым из-за вырожденности вектора нормали $N = [0 \ 0 \ 0]$. С учётом интервалов изменения углов $\alpha \in (-180^\circ, 180^\circ]$ и $\beta \in [0^\circ, 180^\circ]$ составим двухвариантную формулу расчёта **матрицы $A_k(V)$ преобразования системы координат методом экономичного вращения** в зависимости от значения $d = |N| = \sqrt{V_x^2 + V_y^2}$ — длины нормали N и проекции вектора V на плоскость xy :

$$\begin{cases} A_{\text{кэв}}(V) = R_z(-\alpha)R_y(-\beta)R_z(\alpha), \\ \alpha = \text{if}(d: \arg([V_x, V_y]), 0), \beta = \arccos(V_z/|V|). \end{cases} \quad (3.49a)$$

Вычислительно более эффективна формула расчёта матрицы $A_{\text{кэв}}(V)$, использующая двухаргументные матрицы вращений (3.45):

$$\begin{cases} A_{\text{кэв}}(V) = R2_z(c\alpha, -s\alpha)R2_y(c\beta, -s\beta)R2_z(c\alpha, s\alpha), \\ c\alpha = \text{if}(d: V_x/d, 1), s\alpha = \text{if}(d: V_y/d, 0), c\beta = V_z/|V|, s\beta = d/|V|. \end{cases} \quad (3.49б)$$

В сравнении с двухшаговым методом разложения (3.44) экономичный метод на одно преобразование сложнее, но оно обеспечивает **кратчайший** переход системы координат из начального положения xuz в конечное $x'''y'''z'''$.

Метод парных точек

Для получения 3×3 -матрицы $A_k(V)$ **методом парных точек** необходимо выбрать три пары векторов в исходном xuz и конечном $x'y'z'$ положениях систе-

мы координат. Зададим направление оси **кратчайшего** вращения СК нормалью к плоскости векторов z^0 и $V \# z^0$ либо ортом y^0 при $z^0 \parallel V$:

$$N = \text{if}(|z^0 \times V|; z^0 \times V, y^0). \quad (3.50a)$$

Построим по три **ортогональных и пропорциональных** вектора в исходной и конечной системах координат (рис. 3.25), совпадающих с СК экономического вращения $x^0y^0z^0$ и $x''y''z''$ на рис. 3.24:

- в исходной СК орт z^0 , нормаль $N \in xy$, отклонённую от оси y на угол α , и бинормаль

$$B = N \times z^0 = [N_y \ -N_x \ 0] \in xy; \quad (3.50б)$$

- в конечной СК вектор V длиной $m = |V|$, нормаль $N' = mN$, отклонённую от оси y' на такой же угол α , и бинормаль $B' = N' \times V$ длиной $m|B|$.

Подставив в матрицы Ω и Ω' из (3.23a) векторы образа $\{z^0, N, B\}$ и прообраза $\{V, N', B'\}$, получим формулу расчёта **матрицы $A_k(V)$ преобразования системы координат методом парных точек**:

$$A_{\text{кп}}(V) = \begin{bmatrix} V \\ |V|N \\ N \times V \end{bmatrix}^{-1} \begin{bmatrix} z^0 \\ N \\ N \times z^0 \end{bmatrix}. \quad (3.51a)$$

Внедрив её в (3.23a), получим матрицу (3.43) полного преобразования системы координат, включая перенос её начала в точку o :

$$C_{\text{кп}}(o, V) = \begin{bmatrix} V & 0 \\ |V|N & 0 \\ N \times V & 0 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} z^0 & 0 \\ N & 0 \\ N \times z^0 & 0 \\ O_3 & 1 \end{bmatrix} = MP2(o, V, O_3, z^0). \quad (3.51б)$$

Благодаря ортогональности троек векторов $z^0 \perp N \perp B$ и $V \perp N' \perp B'$ оси новой СК остаются ортогональными, но ввиду равной кратности длин соответствующих векторов в ней появляются **равномерные метрические искажения с единым коэффициентом** $m_x = m_y = m_z = 1/|V|$. Это, однако, не влияет на конечный результат преобразований (3.40) и (3.48), поскольку **искажения, вносимые матрицами (3.51), компенсируются их обратными матрицами**.

Если всё же требуется **равномасштабность** преобразования СК во всех направлениях, то нужно использовать функцию $MP2$ как в (3.25б) с **нормированным** вектором $\bar{V}$ и нормалью $N = \text{if}(|z^0 \times \bar{V}|; z^0 \times \bar{V}, y^0)$ вместо (3.50a):

$$C_{\text{кп}}(o, V) = MP2(o, \bar{V}, O_3, z^0) = \begin{bmatrix} \bar{V} & 0 \\ N & 0 \\ N \times \bar{V} & 0 \\ o & 1 \end{bmatrix}^{-1} \begin{bmatrix} z^0 & 0 \\ N & 0 \\ N \times z^0 & 0 \\ O_3 & 1 \end{bmatrix}. \quad (3.51в)$$

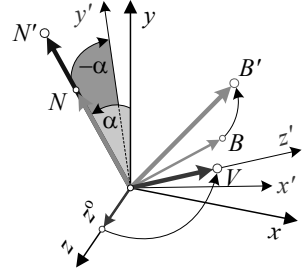


Рис. 3.25

Вспомнив обозначения (3.46б) и подставив в (3.51а) векторы $z^0 = [0 \ 0 \ 1]$, $\bar{V} = [x \ y \ z]$, $N = [-y \ x \ 0]$, $N \times z^0 = [x \ y \ 0]$ и $N \times \bar{V} = [xz \ yz \ -x^2 - y^2]$, получим **явный вид матрицы преобразования** системы координат **нормированным** МПТ:

$$A_{\text{кп}}(V) = \begin{bmatrix} (x^2 z + y^2)/d^2 & xy/d^2 & x \\ xy/d^2 & (y^2 z + x^2)/d^2 & y \\ -x & -y & z \end{bmatrix}, \text{ где } d^2 = x^2 + y^2. \quad (3.52)$$

Замечательно, что **точно такую же матрицу даёт метод экономичного вращения** (3.49б) с аргументами $c\alpha = x/d$, $s\alpha = y/d$, $c\beta = z$ и $s\beta = d$. Это доказывает упоминавшуюся выше **эквивалентность МЭВ и нормированного МПТ**.

Подстановка (3.52) в матрицы $A = A_{\text{к}}(V)A_{\text{а}}A_{\text{к}}^{-1}(V)$ сложных преобразований вращения с $A_{\text{а}} = R_z(\varphi)$ и отражения с $A_{\text{а}} = M(1, 1, -1)$ даёт матрицы $A_{\text{вр}}$ и $A_{\text{отр}}$, точно совпадающие с (3.47). Так проявляется эффект **компенсации искажений прямого преобразования системы координат противоположными искажениями её обратного преобразования**.

Пример 3.3. Сравнить методы расчёта матрицы A , вращающей тройку векторов $\{X, Y, Z\}$ из начального положения $\{x^0, y^0, z^0\}$ в конечное $\{y^0, z^0, x^0\}$ (рис. 3.26).

Решение. Из соображений симметрии ясно, что единый объект $\{X, Y, Z\}$ вращается на

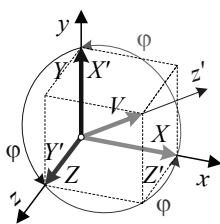


Рис. 3.26

угол $\varphi = 2\pi/3$ вокруг диагонали единичного куба с вектором направления $V = [1 \ 1 \ 1]$ длиной $|V| = \sqrt{3}$. Найдём три матрицы $A_{\text{к}}$ преобразования СК, ориентирующие её ось z вдоль вектора V :

- по методу координатных вращений независимо от значения $|V|$ получаем матрицу (3.46а):

$$A_{\text{ккв}} = \begin{bmatrix} 0.707 & -0.408 & 0.577 \\ 0 & 0.816 & 0.577 \\ -0.707 & -0.408 & 0.577 \end{bmatrix} \Rightarrow A_{\text{ккв}}^{-1} = A_{\text{ккв}}^T;$$

- по МЭВ и **нормированному** МПТ независимо от $|V|$ получаем матрицу (3.52):

$$A_{\text{кэв}} = \begin{bmatrix} 0.789 & -0.211 & 0.577 \\ -0.211 & 0.789 & 0.577 \\ -0.577 & -0.577 & 0.577 \end{bmatrix} \Rightarrow A_{\text{кэв}}^{-1} = A_{\text{кэв}}^T;$$

- в рамках МПТ с **ненормированным** вектором $V = [1 \ 1 \ 1]$ из (3.51а) следует матрица

$$A_{\text{кп}} = \begin{bmatrix} 0.455 & -0.122 & 0.333 \\ -0.122 & 0.455 & 0.333 \\ -0.333 & -0.333 & 0.333 \end{bmatrix} \Rightarrow \boxed{A_{\text{кп}}^{-1} \neq A_{\text{кп}}^T}.$$

уменьшающая в новой СК $x'y'z'$ длины всех векторов в $\sqrt{3}$ раз.

Таким образом, налицо различие матриц $A_{\text{к}}$, вычисленных разными методами. Тем не менее, матрица сложного вращения вокруг оси V одинакова для всех методов:

$$A = A_{\text{ккв}} R_z(\varphi) A_{\text{ккв}}^{-1} = A_{\text{кэв}} R_z(\varphi) A_{\text{кэв}}^{-1} = A_{\text{кп}} R_z(\varphi) A_{\text{кп}}^{-1} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

Уникальность метода парных точек в том, что последняя матрица сразу получается по (3.18б) либо как блок расширенной матрицы $MP4(O_3, x^0, y^0, z^0, O_3, y^0, z^0, x^0)$ без вспомогательного изменения системы координат и без угадывания вектора V и угла φ . $\square$

Сравнение рассмотренных методов сложных преобразований по критериям понятности, простоты вывода, краткости записи, вычислительной эффективности ставит вперёд всех, по мнению автора, **метод парных точек**. К сожалению, все методы страдают условностью — зависимостью от особых положений вектора V . Видимо, **единый алгоритм для произвольных направлений прямой $\{o, V\}$ отсутствует**, а те методы, что предлагаются в учебной литературе, — лишь частные случаи, не учитывающие особых ситуаций. Опыт курсового проектирования студентов под руководством автора показывает, что эти ситуации встречаются не так уж и редко. Внешне они проявляются во внезапном завершении программы из-за деления на ноль или в неконтролируемом осевом кручении объекта во время его движения в пространстве сцены.

☞ Гораздо проще, чем в трёхмерном пространстве, находятся **матрицы преобразования системы координат на плоскости**.

Допустим, нужно совместить начало СК с точкой o , а ось y направить по вектору V , ортогонально которому проходит правая нормаль $N = [V_y \ -V_x]$ (рис. 3.27). Такое преобразование бывает необходимо при расчёте отражения и масштабирования относительно произвольной прямой $\{o, V\}$.

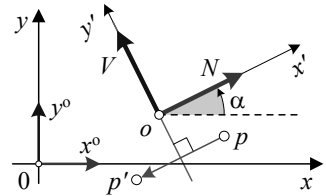


Рис. 3.27

Приведём без лишних рассуждений очевидные формулы расчёта матриц преобразования системы координат A_K , B_K и C_K :

- по **методу координатного вращения** на угол $\alpha = \arg(N)$

$$A_{\text{ККВ}}(V) = R(-\alpha), \quad B_{\text{ККВ}} = -oA_{\text{ККВ}}, \quad C_{\text{ККВ}}(o, V) = T(-o)\tilde{R}(-\alpha); \quad (3.53a)$$

- по **методу парных точек** благодаря ортогональности векторов V и N

$$A_{\text{КП}}(V) = \begin{bmatrix} N \\ V \end{bmatrix}^{-1} = \frac{1}{V \circ V} \begin{bmatrix} N \\ V \end{bmatrix}^T, \quad B_{\text{КП}} = -oA_{\text{КП}}, \quad C_{\text{КП}}(o, V) = \begin{bmatrix} N & 0 \\ V & 0 \\ o & 1 \end{bmatrix}^{-1}. \quad (3.53b)$$

Эти матрицы в отличие от (3.53a) равномерно масштабируют пространство плоскости с коэффициентами $m_x = m_y = 1/|V|$. Для исключения этого эффекта в формулах (3.53б) нужно использовать нормированные векторы $\bar{N}$ и $\bar{V}$.

☞ **Пример 3.4.** Построить на плоскости xu отражение треугольника с вершинами $p_1 = [0 \ 0]$, $p_2 = [0 \ 1]$ и $p_3 = [2 \ 0]$ от прямой линии, проходящей через две точки $p_4 = [3 \ 0]$ и $p_5 = [2 \ 2]$ (рис. 3.28).

Решение. Вычислим направляющий вектор и нормаль заданной прямой линии:

$$V = p_5 - p_4 = [-1 \ 2], \quad N = R \text{rot}(V) = [2 \ 1].$$

Последовательность действий для построения отражений p'_i исходных точек p_i ($i=1, 2, 3$) может быть, например, следующей.

Шаг 1. Получим по (3.53a) МКВ-матрицу сложного преобразования СК, состоящего из двух элементарных пре-

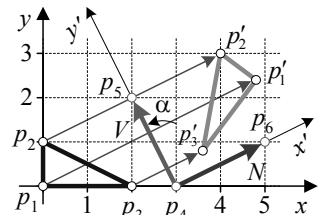


Рис. 3.28

образований: переноса начала в точку p_4 и вращения на угол α , у которого $\cos(\alpha)=V_y/|V|=\sqrt{4/5}$ и $\sin(\alpha)=-V_x/|V|=\sqrt{1/5}$:

$$C_{\text{ккв}}=T(-p_4)\tilde{R}(-\alpha)=\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -3 & 0 & 1 \end{bmatrix}\begin{bmatrix} \sqrt{4/5} & -\sqrt{1/5} & 0 \\ \sqrt{1/5} & \sqrt{4/5} & 0 \\ 0 & 0 & 1 \end{bmatrix}=\begin{bmatrix} \sqrt{4/5} & -\sqrt{1/5} & 0 \\ \sqrt{1/5} & \sqrt{4/5} & 0 \\ -\sqrt{36/5} & \sqrt{9/5} & 1 \end{bmatrix}.$$

Вторую матрицу преобразования СК получим методом парных точек (3.53б) по двум тройкам её элементов в исходном $\{x^0, y^0, O_2\}$ и конечном $\{N, V, p_4\}$ положениях:

$$C_{\text{кп}}=\begin{bmatrix} 2 & 1 & 0 \\ -1 & 2 & 0 \\ 3 & 0 & 1 \end{bmatrix}^{-1}=\begin{bmatrix} 0.4 & -0.2 & 0 \\ 0.2 & 0.4 & 0 \\ -1.2 & 0.6 & 1 \end{bmatrix}.$$

Читателя не должно смущать несовпадение полученных матриц. Как указывалось выше, матрица $C_{\text{кп}}$ включает иные, чем в матрице $C_{\text{ккв}}$, элементарные преобразования, которые, по сути дела, нас и не интересуют. Главная цель — получение матрицы преобразования СК — достигнута методом парных точек быстрее и проще.

Шаг 2. Отражение вдоль оси x' выполняется матрицей $M=\text{diag}[-1 \ 1]$.

Шаг 3. Восстановление исходной СК выполняется матрицами

$$C_{\text{ккв}}^{-1}=\begin{bmatrix} \sqrt{4/5} & \sqrt{1/5} & 0 \\ -\sqrt{1/5} & \sqrt{4/5} & 0 \\ 3 & 0 & 1 \end{bmatrix}, \quad C_{\text{кп}}^{-1}=\begin{bmatrix} 2 & 1 & 0 \\ -1 & 2 & 0 \\ 3 & 0 & 1 \end{bmatrix}.$$

Шаг 4. Матрица сложного преобразования

$$C=C_{\text{ккв}}\tilde{M}C_{\text{ккв}}^{-1}=C_{\text{кп}}\tilde{M}C_{\text{кп}}^{-1}=\begin{bmatrix} -0.6 & -0.8 & 0 \\ -0.8 & 0.6 & 0 \\ 4.8 & 2.4 & 1 \end{bmatrix}$$

одинакова в обоих методах расчёта матрицы изменения СК, что означает компенсацию масштабных искажений, внесённых матрицей $C_{\text{кп}}$, обратными искажениями матрицей $C_{\text{кп}}^{-1}$.

Шаг 5. Координаты отражённых точек рассчитаем групповым методом (3.11):

$$\begin{bmatrix} p'_1 & 1 \\ p'_2 & 1 \\ p'_3 & 1 \end{bmatrix}=\begin{bmatrix} p_1 & 1 \\ p_2 & 1 \\ p_3 & 1 \end{bmatrix}C=\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 2 & 0 & 1 \end{bmatrix}\begin{bmatrix} -0.6 & -0.8 & 0 \\ -0.8 & 0.6 & 0 \\ 4.8 & 2.4 & 1 \end{bmatrix}=\begin{bmatrix} 4.8 & 2.4 & 1 \\ 4 & 3 & 1 \\ 3.6 & 0.8 & 1 \end{bmatrix}.$$

□

Итак, нами изучено три метода расчёта матриц $A_{\text{к}}(V)$ и $C_{\text{к}}(o, V)$ **пассивно-го** преобразования **системы координат**, состоящего из совмещения оси z с направлением вектора V и переноса начала СК в точку o . По принципу инверсности обратные матрицы $A(V)=A_{\text{к}}^{-1}(V)$ и $C(o, V)=C_{\text{к}}^{-1}(o, V)$ выполняют **активное** преобразование **объекта**, ориентированного в исходном состоянии в начале СК вдоль оси z , в конечное, заданное точкой o и произвольным вектором V :

$$p'=pA(V)+o, \quad \tilde{p}'=\tilde{p}C(o, V). \quad (3.54)$$

Матрицы преобразования, состоящего из цепочки **вращений**, являются ортогональными и удовлетворяют условию $A_{\text{к}}^{-1}(V)=A_{\text{к}}^T(V)$, благодаря которому отпадает необходимость матричного инвертирования. С учётом этого свойства и обозначений (3.46б) получаем выражения матриц $A(V)$:

- по **МКВ** транспонируем матрицу (3.46а):

$$A_{\text{кв}}(V) = \begin{bmatrix} z/d & 0 & -x/d \\ -xy/d & d & -yz/d \\ x & y & z \end{bmatrix}, \text{ где } d = \sqrt{x^2 + z^2}; \quad (3.55a)$$

- по **МЭВ** и **нормированному МПТ** транспонируем матрицу (3.52):

$$A_{\text{эв}}(V) = \begin{bmatrix} (x^2z + y^2)/d^2 & xy/d^2 & -x \\ xy/d^2 & (y^2z + x^2)/d^2 & -y \\ x & y & z \end{bmatrix}, \text{ где } d^2 = x^2 + y^2; \quad (3.55б)$$

- по **МПТ** с вектором V произвольной длины инвертируем (3.51а):

$$A_{\text{п}}(V) = \begin{bmatrix} z^0 \\ N \\ N \times z^0 \end{bmatrix}^{-1} \begin{bmatrix} V \\ |V|N \\ N \times V \end{bmatrix}. \quad (3.55в)$$

На рис. 3.29 показано, как разные матрицы в (3.55) переводят объект (флажок единичной длины, вытянутый по оси z) из начального положения P_0 в веер конечных положений, направленных по векторам $V_1 \div V_7 \in xy$ с длинами $|V_i| = 1.5$. Последнее МПТ-преобразование, кроме изменения ориентации объекта, дополнительно равномерно масштабирует его во всех направлениях в $|V|$ раз.

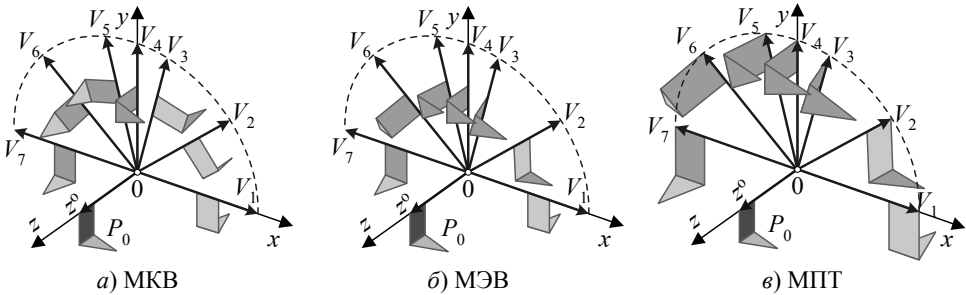


Рис. 3.29

3.1.3.2. Кинематический метод конструирования линий

Аффинные преобразования — основа **кинематического метода конструирования параметрических объектов** (линий и поверхностей), генерирующего состояния объекта на плоскости или в пространстве из заданных начальных состояний точки, отрезка, окружности как функции некоторого параметра, имеющего физический смысл времени движения. Следует ясно понимать, что конечным результатом преобразований являются не уравнения объекта, а его изображение на экране. Построение изображения эффективнее выполнять не по сложным аналитическим формулам, а непосредственно по векторной модели общего вида, задав желаемое начальное состояние объекта, необходимые матрицы преобразований и интервалы изменения параметров.

Окружность на плоскости

Для кинематического **построения** единичной окружности нужно задать:

- начальное состояние одной из её точек, например, $c(0)=[1 \ 0]$;
- возрастающие отсчёты параметра в интервале $t \in [0, 2\pi]$;
- векторную функцию (1.64) $c(t)=c(0)R(t)$, моделирующую вращение точки на единичном расстоянии от начала координат. Параметр t — угол между радиус-векторами $c(0)$ и $c(t)$ — имеет физический смысл времени движения.

При решении же задач **анализа** нужны координатные уравнения состояний объекта, получаемые обычно кропотливыми ручными выкладками либо с помощью возможностей какой-нибудь «умной» математической программы.

Рассмотрим основы кинематического конструирования. Линия $p(t)$ изображается на экране последовательностью узловых точек $p_i=p(t_i)$, соединённых отрезками прямых. Визуальная гладкость ломаной линии определяется плотностью размещения узлов по дуге кривой. **Оптимальная** плотность должна быть пропорциональна кривизне линии, при этом минимальное расстояние между соседними точками должно быть порядка одного пиксела. Для получения такого распределения необходим **анализ** скорости $V(t)=dp(t)/dt$ движения точки по линии. Например, вектор скорости движения по единичной окружности получается как векторным дифференцированием $V(t)=c(0)dR(t)/dt$, так и ручным покоординатным дифференцированием функции (1.64):

$$V(t)=\frac{d}{dt}\begin{bmatrix} 1 & 0 \end{bmatrix}\begin{bmatrix} \cos(t) & \sin(t) \\ -\sin(t) & \cos(t) \end{bmatrix}=\frac{d}{dt}\begin{bmatrix} \cos(t) & \sin(t) \end{bmatrix}=\begin{bmatrix} -\sin(t) & \cos(t) \end{bmatrix}. \quad (3.56)$$

Модуль векторной функции $|V(t)|=1$ означает, что точка движется по окружности с постоянной скоростью, поэтому оптимальным является **равномерное** распределение отсчётов параметра $t_i=i \cdot \Delta t \ \forall i=\overline{0, n}$ по интервалу $[0, 2\pi]$ с постоянным квантом $\Delta t=2\pi/n$. При таком выборе модели окружности соседние узловые точки c_i и $c_{i+1}=c_iR(\Delta t)$ равноотстоят друг от друга как по углу, так и по периметру: $\Delta t=\text{const}$, $|c_{i+1}-c_i|=2\sin(\Delta t/2)=\text{const}$.

Соединение точек отрезками прямых аппроксимирует окружность правильным n -угольником (рис. 3.30,а). С учётом масштабного множителя μ_3 из (3.42б) значений $n=2\pi\mu_3$ и $\Delta t=1/\mu_3$ вполне достаточно для того, чтобы расстояния между узловыми точками были равны одному пикселу, а многоугольник визуально не отличался от окружности. В действительности последнее произойдет даже при угловом шаге $\Delta t \approx 10/\mu_3$, равном $0.25 \text{ рад} \approx 14^\circ$ при $\mu_3=40$.

Возможны и **некинематические модели окружности**. Это, например, координатные функции, удовлетворяющие каноническому уравнению $x^2+y^2-1=0$, и составленный из их производных вектор скорости движения:

$$x(t)=\frac{1-t^2}{1+t^2}, \quad y(t)=\frac{2t}{1+t^2} \quad \forall t \in (-\infty, \infty) \Rightarrow V(t)=\frac{2}{(1+t^2)^2}\begin{bmatrix} -2t & 1-t^2 \end{bmatrix}. \quad (3.57)$$

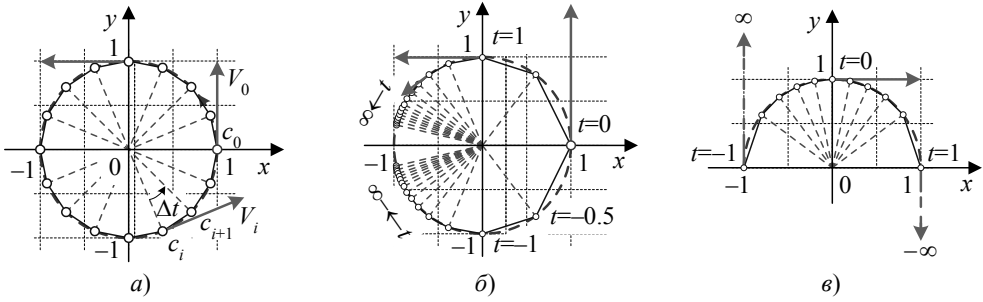


Рис. 3.30

Зависимость модуля вектора скорости $|V(t)|=2/(1+t^2)$ от параметра t означает, что при постоянном кванте Δt точки $[x(t_i) \ y(t_i)]$ распределены по окружности — кривой постоянной кривизны — неравномерно (рис. 3.30,б). Кроме того точка $[-1 \ 0]$, соответствующая значениям параметра $t=\pm\infty$, вычислительно недоступна. Следовательно, **дробно-рациональная модель (3.57) неудачна**, хотя и менее трудоёмка, чем тригонометрическая модель (1.64).

Ещё менее удачна другая некинематическая модель $x(t)=t$, $y(t)=\sqrt{1-t^2}$ $\forall t \in [-1, 1]$, полученная при неграмотном выводе явного уравнения окружности $y=\sqrt{1-x^2}$ из её канонической формы $x^2+y^2=1$. Во-первых, по этой модели строится не вся окружность, а лишь её верхняя половина. Нижняя половина — это другая явная зависимость $y(t)=-\sqrt{1-t^2}$. Во-вторых, вектор скорости $V(t)=(1 \ -t/\sqrt{1-t^2})$ не вычислим в крайних точках $[\pm 1 \ 0]$ при $t \rightarrow \pm 1$. В-третьих, непостоянство модуля скорости $|V(t)|=1/\sqrt{1-t^2}$ от параметра t также даёт неравномерное распределение точек $p(t_i)$ по периметру линии (рис. 3.30,в).

Проведённый сравнительный анализ трёх моделей одного и того же графического объекта показывает, насколько компетентно нужно подходить к выбору как уравнений модели, так и интервалов изменения её параметров для получения достоверного и качественного изображения объекта на экране дисплея.

Эллипс на плоскости

Параметрические уравнения **произвольного эллипса на плоскости** с центром в точке $o=[o_x \ o_y]$, полуосями a , b и углом α наклона первой полуоси к оси x получим за три преобразования (масштабирования, вращения и переноса) окружности $c(t)$ (рис. 3.31,а):

$$p(t)=c(t)\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}\begin{bmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha) & \cos(\alpha) \end{bmatrix}+o, \quad (3.58)$$

$$\begin{cases} x(t)=a\cos(\alpha)\cos(t)-b\sin(\alpha)\sin(t)+o_x, \\ y(t)=a\sin(\alpha)\cos(t)+b\cos(\alpha)\sin(t)+o_y. \end{cases}$$

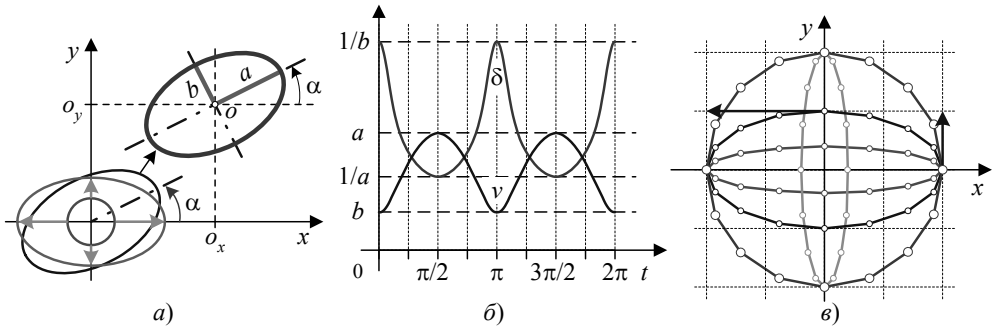


Рис. 3.31

Модуль $v(t)=|V(t)|=\sqrt{a^2 \sin^2(t)+b^2 \cos^2(t)}$ вектора скорости

$$V(t)=c'(t)M(a,b)R(\alpha)= \\ =[-a\cos(\alpha)\sin(t)-b\sin(\alpha)\cos(t) \quad -a\sin(\alpha)\sin(t)+b\cos(\alpha)\cos(t)]$$

движения точки по эллипсу зависит от параметра t (рис. 3.31,б): в точках апогея с максимальной кривизной скорость минимальна ($v=\min\{a,b\}$), в точках перигея с минимальной кривизной — максимальна ($v=\max\{a,b\}$). Следовательно, **при аппроксимации эллипса полигоном плотность распределения его вершин p_i по периметру должна быть переменна: максимальна на участках апогея и минимальна на участках перигея.**

Точное решение задачи распределения связано с расчётом длин дуг эллипсов в теории исчисления эллиптических интегралов. Простейшее решение — **равномерная** шкала отсчётов параметра $t_i=i\cdot\Delta t$ с постоянным квантом $\Delta t=2\pi/n$ — благодаря непропорциональному масштабированию окружности в (3.58) при $a\neq b$ фактически приводит к нужному результату (рис. 3.31,в): график зависимости $\delta(t)=1/v(t)=(a^2 \sin^2(t)+b^2 \cos^2(t))^{-1/2}$ плотности распределения узлов $p(t_i)$ по периметру эллипса, построенный на рис. 3.31,б, обратно коррелирует с графиком скорости $v(t)$. Оптимальные значения числа сторон полигона $n_3=\lceil 2\pi\mu_3\cdot\min\{a,b\}\rceil$ и величины кванта $\Delta t_3=2\pi/n_3$ дают расстояния между точками эллипса в один пиксел на участках с максимальной кривизной.

В частном случае при $a=b=r$ и $\alpha=0$ эллипс превращается в произвольную **окружность** с радиусом r и центром в точке o :

$$p(t)=rc(t)+o=[r\cos(t)+o_x \quad r\sin(t)+o_y]. \quad (3.59)$$

Наилучшее качество аппроксимации окружности правильным многоугольником достигается выбором оптимальных параметров $n_0=\lceil 2\pi\mu_3r\rceil$ и $\Delta t_0=2\pi/n_0$.

Окружность в пространстве

Получим разными методами модели **пространственной окружности**, лежащей в плоскости $\{o,N\}$ с радиусом r и центром в точке o . Ранее были найдены матрицы (3.55) активного преобразования объекта из начального положе-

ния $\{O_3, z^0\}$ в конечное $\{O, V\}$. Поэтому в качестве исходной возьмём окружность единичного радиуса с центром в начале координат и нормалью z^0 к плоскости её расположения xy :

$$c_{xy}(t) = [c(t) \ 0] = [\cos(t) \ \sin(t) \ 0] \in xy \quad \forall t \in [0, 2\pi]. \quad (3.60)$$

□ По двум **методам разложения** подвергнем окружность (3.60) трём активным преобразованиям:

- равномерному **масштабированию** до радиуса r ;
- совмещению нормали с вектором N с помощью **вращений**;
- **переносу** центра в точку o .

В соответствующей этим преобразованиям векторной модели

$$p(t) = rc_{xy}(t)A(N) + o \quad (3.61)$$

масштабирование матрицей $M(r, r, r) = rE_3$ заменено умножением на скаляр r , а матрица сложного преобразования $A(N)$ есть произведение матриц вращения:

- по **методу координатных вращений** (рис. 3.32, а) поворот $z^0 \rightarrow N$ выполняется двумя элементарными вращениями, обратными (3.44в), на углы (3.44д):

$$A_{\text{кв}}(N) = R_x(-\beta)R_y(\alpha), \quad (3.62a)$$

$$\alpha = \text{if}(N_x^2 + N_z^2 : \arg([N_z \ N_x]), 0), \quad \beta = \arcsin(N_y/|N|);$$

- по **методу экономичного вращения** (рис. 3.32, б) производится **кратчайший** поворот нормали $z^0 \rightarrow N$ вокруг вектора $V = z^0 \times N$ цепочкой трёх элементарных вращений, обратных (3.49а):

$$A_{\text{эв}}(N) = R_z(-\alpha)R_y(\beta)R_z(\alpha), \quad (3.62б)$$

$$\alpha = \text{if}(N_x^2 + N_y^2 : \arg([N_x \ N_y]), 0), \quad \beta = \arccos(N_z/|N|).$$

Сгенерированные по (3.60)–(3.62) окружности и окаймляющие их квадраты совпадают с точностью до кручения на угол α вокруг оси N , что наглядно показано на рис. 3.32, а, б разными положениями начальных точек $p(0)$ и углов квадратов. Поиск подходящей декомпозиции сложного преобразования требует развитого пространственного воображения и учёта особых случаев, что часто игнорируется и рано или поздно приводит к сбою работы программы.

Векторной модели (3.61) с вычисленными по (3.62) матрицами преобразований вполне достаточно для компьютерного **построения** окружности в произ-

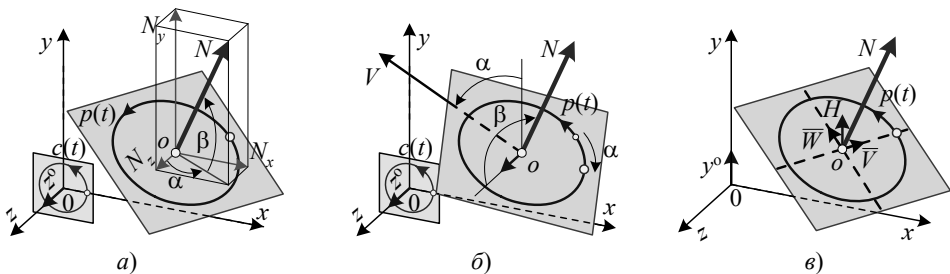


Рис. 3.32

вольно расположенной плоскости. В задачах **анализа** бывает нужна **аналитическая модель окружности** с параметрами $\{o, N, r\}$. Сложность её получения осознается в попытке ручного вывода координатных функций $x(t) \div z(t)$. Проиллюстрируем это на примере подстановки в (3.61) матрицы (3.55а), ранее выведенной **методом координатных вращений**:

- при $N \# y^0$ и $d = \sqrt{N_x^2 + N_z^2} \neq 0$ после упрощений получаем координатные функции окружности в произвольной плоскости:

$$\begin{cases} x(t) = o_x + r(N_z/d)\cos(t) - r(N_x N_y/d|N|)\sin(t), \\ y(t) = o_y + r(d/|N|)\sin(t), \\ z(t) = o_z - r(N_x/d)\cos(t) - r(N_y N_z/d|N|)\sin(t); \end{cases} \quad (3.63a)$$

- в особом случае $N \parallel y^0$ и $N_x = N_y = N_z = d = 0$ функции (3.63а) становятся невычислимыми. Преодоление неопределённостей $0/0$ достигается только ручным сокращением по **противоречивым** правилам $N_x/d = 0$ и $N_z/d = 1$:

$$x(t) = o_x + r\cos(t), \quad y(t) = o_y, \quad z(t) = o_z - \operatorname{sgn}(N_y)r\sin(t). \quad (3.63б)$$

Без принятия этих правил модель окружности в горизонтальной плоскости $\{o, y^0\}$ **не является частным случаем общей модели (3.63а)** и потому не имеет права на забвение при анализе и программировании.

□ По **методу парных точек** матрицы сложного преобразования в декартовой и расширенном пространствах

$$p(t) = c_{xy}(t)A_{\Pi}(N) + o, \quad \tilde{p}(t) = \tilde{c}_{xy}(t)C_{\Pi}(o, N) \quad (3.64a)$$

вычисляются на основе матрицы (3.55в) и обладают свойством равномерного масштабирования во всех направлениях в r раз:

$$A_{\Pi}(N) = \begin{bmatrix} z^0 \\ V \\ V \times z^0 \end{bmatrix}^{-1} \begin{bmatrix} r\bar{N} \\ rV \\ V \times r\bar{N} \end{bmatrix}, \quad C_{\Pi}(o, N) = \begin{bmatrix} z^0 & 0 \\ V & 0 \\ V \times z^0 & 0 \\ O_3 & 1 \end{bmatrix}^{-1} \begin{bmatrix} r\bar{N} & 0 \\ rV & 0 \\ V \times r\bar{N} & 0 \\ o & 1 \end{bmatrix}, \quad (3.64б)$$

где $V = \operatorname{if}\left(\left|z^0 \times \bar{N}\right|: z^0 \times \bar{N}, y^0\right)$.

Окружность (3.64а) полностью идентична той, что построена на рис. 3.32,б МЭВ-матрицей (3.62б). Применение МПТ, где задаются лишь начальное и конечное состояния объекта, позволяет исключить из процесса разработки алгоритмов сложных преобразований этапы выбора промежуточных ЭП, расчёта их матриц, учёта особых условий ориентации объекта относительно СК и другие тонкие моменты, непонимание которых может привести к нежелательным визуальным эффектам и программным сбоям.

□ Получим модель окружности **методом суперпозиции** колебаний точки вдоль двух **ортогональных** векторов V и W , **параллельных** плоскости $\{o, N\}$ (рис. 3.32,в). Найдём эти векторы способом, аналогичным (1.104):

- сначала подберём **произвольный** вектор H , не параллельный нормали N , например, один из координатных ортов $\omega^0 \# N$, где $\omega^0 \in \{x^0, y^0\}$:

$$H = \text{if}(|N \times y^0|; y^0, x^0);$$

- затем вычислим ортогональный нормали вектор $V = H \times N \perp N$;
- получим, наконец, второй ортогональный вектор $W = N \times V \perp N \perp V$.

Теперь составим комбинацию взаимно ортогональных гармонических колебаний в плоскости $\{o, N\}$ амплитудой r вокруг точки o :

$$p(t) = r \cos(t) \bar{V} + r \sin(t) \bar{W} + o = c_{xy}(t) A_c + o, \quad \tilde{p}(t) = \tilde{c}_{xy}(t) C_c. \quad (3.65a)$$

Матрицы преобразования окружности (3.60) методом суперпозиции имеют вид

$$A_c = r \begin{bmatrix} \bar{V} \\ \bar{W} \\ O_3 \end{bmatrix}, \quad C_c = \begin{bmatrix} r\bar{V} & 0 \\ r\bar{W} & 0 \\ O_3 & 0 \\ o & 1 \end{bmatrix}. \quad (3.65b)$$

С их помощью становится возможным преобразовывать любые другие объекты, например, перевести окаймляющий окружность квадрат из плоскости xu на плоскость $\{o, N\}$ с увеличением линейных размеров в r раз.

Возможны два варианта взаимного положения векторов N и y^0 :

- в общей ориентации $N \# y^0$, проверяемой условием $|N \times y^0| \neq 0$, примем

$$H = y^0, \text{ тогда } V = [N_z \ 0 \ -N_x] \text{ и } W = [-N_x N_y \ d^2 \ -N_y N_z], \text{ где } d = \sqrt{N_x^2 + N_z^2} \neq 0.$$

Подставив эти выражения в (3.65), получим аналогичные (3.63a) функции

$$\begin{cases} x(t) = o_x + r(N_z/d) \cos(t) - r(N_x N_y / d |N|) \sin(t), \\ y(t) = o_y + r(d/|N|) \sin(t), \\ z(t) = o_z - r(N_x/d) \cos(t) - r(N_y N_z / d |N|) \sin(t). \end{cases} \quad (3.66a)$$

Эта окружность и окаймляющий её квадрат (см. рис. 3.32,в) **точно** совпадают с окружностью $\{o, N, r\}$ и квадратом, построенными методом разложения на рис. 3.32,а;

- в частном случае $N \parallel y^0$, т. е. при $N = N_y y^0$, примем $H = x^0$, тогда, подставив в (3.65a) векторы $V = x^0 \times N_y y^0 = N_y z^0$ и $W = N_y y^0 \times N_y z^0 = N_y^2 x^0$, получим координатные функции окружности в горизонтальной плоскости:

$$x(t) = o_x + r \sin(t), \quad y(t) = o_y, \quad z(t) = o_z + \text{sgn}(N_y) r \cos(t). \quad (3.66b)$$

Сравнивая их с (3.63б), видим небольшое различие формул описания этих окружностей при полном геометрическом совпадении друг с другом.

Таким образом, **метод суперпозиции эквивалентен методу координатных вращений**, но немного проще его как в задачах анализа, так и в области практического применения.

Циклические линии

Циклические линии более общего класса, чем в (1.67), образуются суперпозицией вращательного движения точки по окружности **переменного** радиуса с переносным движением центра окружности по траектории $o(t)$ (рис. 3.33,а):

$$p(t) = r(t)c(\omega t)Q + o(t). \quad (3.67)$$

В этом выражении:

- $r(t)$ — закон изменения радиуса окружности от параметра t ;
- $c(t) = [\cos(t) \sin(t)]$ — параметрическая функция окружности на плоскости;
- ω — частота относительного вращения, **согласованная с переносным движением** и равная числу оборотов точки $c(\omega t)$ при изменении t на 2π ;
- Q — матрица управления начальным состоянием $p(0)$ и направлением вращения точки по окружности в соответствии с диаграммой на рис. 3.33, б.

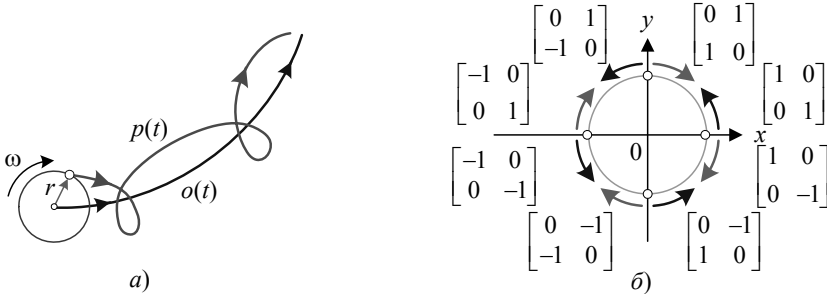


Рис. 3.33

В целом закон движения (3.67) расширяет возможности модели (1.67) за счёт выбора произвольной траектории $o(t)$. Получим с его помощью **модели некоторых циклических линий**.

□ **Циклоида** — это траектория движения фиксированной точки окружности радиуса r , катящейся **без скольжения** по прямой $y=0$ (рис. 3.34). Зададим в (3.67) частоту вращения $\omega=1$, а переносное движение функцией $o(t)=r[\omega t \ 1]$.

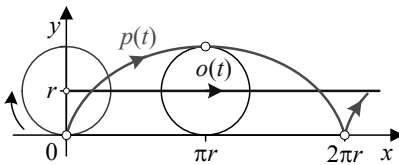


Рис. 3.34

При изменении параметра t на 2π центр катящейся окружности сместится вдоль оси x на расстояние $2\pi r\omega$ — длину ω периметров.

Выберем управляющую матрицу

$$Q = M(-1, 1)R(90^\circ) = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$$

такую, чтобы вращение точки $c(t)Q = [-\sin(\omega t) \ -\cos(\omega t)]$ происходило по часовой стрелке из начального положения $[0 \ -1]$. Подставив элементы движения в (3.67), получим векторное и координатные **уравнения циклоиды**

$$p(t) = r[t \ 1] - r[\sin(t) \ \cos(t)] \Rightarrow x(t) = r(t - \sin(t)), \quad y(t) = r(1 - \cos(t)).$$

Трёхмерным аналогом циклоиды является **циклоидная поверхность** — геометрическое место фиксированной точки сферы, произвольно катающейся по плоскости без проскальзывания и подпрыгиваний. Её параметрическую модель можно получить, вращая циклоиду $[p(t) \ 0]$ вокруг оси y на угол $\tau \in [0, 2\pi]$:

$$\begin{aligned} p(t, \tau) &= r[t - \sin(t) \ 1 - \cos(t) \ 0]R_y(\tau) = \\ &= r[(t - \sin(t))\cos(\tau) \ 1 - \cos(t) \ (\sin(t) - t)\sin(\tau)]. \end{aligned}$$

□ **Эпициклоида** представляет собой траекторию движения фиксированной точки окружности радиуса r , которая обкатывает без скольжения *снаружи* окружность радиуса r_0 . Выберем начальное положение внешней окружности сверху внутренней, а отслеживаемую точку выберем внизу внешней (рис. 3.35).

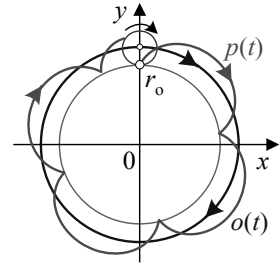


Рис. 3.35

Переносное движение центра внешней окружности происходит по круговой траектории $o(t) = (r_0 + r)[\sin(t) \cos(t)]$. При изменении параметра t на 2π он совершает один оборот вокруг внутренней окружности и $\omega = r_0/r + 1$ (почему на 1 больше отношения радиусов?) оборотов вокруг собственного центра. Матрица Q такая же, что у циклоиды.

Подставив элементы движения в общую модель (3.67), получим векторное и координатные **уравнения эпициклоиды**:

$$\begin{aligned} p(t) &= (r_0 + r)[\sin(t) \cos(t)] - r[\sin(\omega t) \cos(\omega t)], \\ \begin{cases} x(t) = (r_0 + r)\sin(t) - r\sin((r_0/r + 1)t), \\ y(t) = (r_0 + r)\cos(t) - r\cos((r_0/r + 1)t). \end{cases} \end{aligned}$$

□ **Гипоциклоида** — это траектория движения фиксированной точки окружности радиуса r в процессе её качения без скольжения *внутри* окружности радиуса r_0 . Выберем начальные положения внутренней окружности сверху внешней, а отслеживаемой точки вверху внутренней окружности (рис. 3.36).

Переносное движение катящейся окружности происходит по окружности радиуса $r_0 - r$:

$$o(t) = (r_0 - r)[\sin(t) \cos(t)].$$

При изменении t на 2π она совершает $\omega = r_0/r - 1$ (почему на 1 меньше, чем r_0/r ?) оборотов вокруг собственного центра. Управляющая матрица

$$Q = R(90^\circ) = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

создаёт вращение точки $c(t)Q = [-\sin(t) \cos(t)]$ против часовой стрелки из начального положения $[0 \ 1]$.

Подставляя найденные элементы движения в (3.67), получим векторное и координатные **уравнения гипоциклоиды**:

$$\begin{aligned} p(t) &= (r_0 - r)[\sin(t) \cos(t)] + r[-\sin(\omega t) \cos(\omega t)], \\ \begin{cases} x(t) = (r_0 - r)\sin(t) - r\sin((r_0/r - 1)t), \\ y(t) = (r_0 - r)\cos(t) + r\cos((r_0/r - 1)t). \end{cases} \end{aligned}$$

□ **Коническая винтовая линия** с углом раствора 2α и шагом h (рис. 3.37) образуется сложением равномерного переносного движения вверх по оси y и спираль-

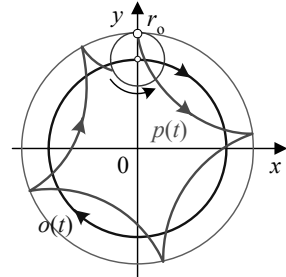


Рис. 3.36

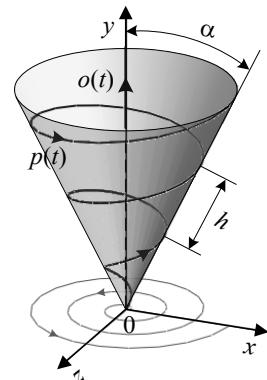


Рис. 3.37

ного движения в плоскости xz , полученного поворотом спирали Архимеда (см. рис. 1.43,а) на угол $\varphi_x = -90^\circ$:

$$p(t) = \frac{h \cos(\alpha)}{2\pi} [0 \ t \ 0] + \frac{h \sin(\alpha) t}{2\pi} [c(t) \ 0] R_x \left(-\frac{\pi}{2} \right) = \frac{ht}{2\pi} [\sin(\alpha) \cos(t) \ \cos(\alpha) \ -\sin(\alpha) \sin(t)].$$

□ Замкнутая **спиральная линия**, у которой n витков радиуса r равномерно навиты на поверхность тора с радиусом осевой окружности r_0 (рис. 3.38), образуется суперпозицией вращения точки $[r_0 + r \ 0 \ 0]$ с частотой $\omega = n$ по окружности

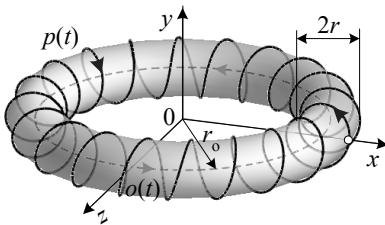


Рис. 3.38

сти $p_0(t) = r[c(nt) \ 0] + [r_0 \ 0 \ 0]$, смещённой в плоскости xy по оси x на величину r_0 , и одно-временного вращения этой **образующей** окружности с единичной частотой вокруг оси y :

$$p(t) = p_0(t) R_y(t) = \begin{bmatrix} r_0 + r \cos(nt) \cos(t) \\ r \sin(nt) \\ -(r_0 + r \cos(nt) \sin(t)) \end{bmatrix}^T.$$

Позже в гл. 4 мы применим кинематический метод для формирования параметрических поверхностей $p(t, \tau)$ типа конуса на рис. 3.37 и тора на рис. 3.38.

3.1.3.3. Кинематическая задача перемещения в пространстве

Продолжая изучение методов сложных преобразований, рассмотрим **кинематическую задачу перемещения в пространстве**. В компьютерной графике непрерывное движение объекта изображается последовательностью **состояний** (фаз, кадров) — элементов дискретного множества его положений во времени и пространстве. **Цель** кинематического анализа и синтеза — изучение и имитация реалистичного движения графических объектов, выявление в алгоритмах движения методических ошибок и непредвиденных эффектов.

Движущийся в R^n объект как система **связанных** точек имеет $2n$ степеней свободы: n координат центра масс и n параметров ориентации. Естественно, что наличие формул движения всех основных точек объекта полностью решает задачу кинематики и позволяет построить изображение объекта в любой момент времени. Однако, полная информация об объекте не всегда доступна и к тому же она избыточна. Согласно МПТ для расчёта состояния объекта достаточно знания координат его $n+1$ -й точки, чтобы по (3.17а) рассчитать матрицу его преобразования из предыдущего состояния и по групповой формуле (3.11) получить координаты остальных точек. Этот метод основан на **принципе пространственно-временной когерентности** — сохранении взаимной ориентации точек объекта в процессе его движения в пространстве и во времени.

В отсутствие полной информации об объекте логично полагать, что согласно **принципу минимального действия** он движется по **экономичной** траектории — **кратчайшим** направлениям переноса центра масс и поворота собственной оси. Без учёта осевого кручения объекта его состояние $P(o, V)$ задаётся центром (**базовой точкой**) o и вектором направления оси V . Возможно, что объ-

ект также подвергается **преобразованию подобия** — равномерному масштабированию во всех направлениях относительно своего центра. Управление размерами объекта удобно осуществлять изменением длины вектора V .

Рассмотрим **две схемы** кинематического расчёта перехода $P_i \rightarrow P_{i+1}$ объекта из предыдущего состояния P_i в последующее состояние P_{i+1} .

□ По **итеративной схеме** (рис. 3.39,а) состояния объекта

$$\tilde{P}_i = \tilde{P}_0 C_i \quad \forall i=1,2,\dots \quad (3.68a)$$

получаются из состояния $P_0(o_0, V_0)$ подобного объекту **примитива** — объекта единичных размеров, центр которого $o_0=O_3$ расположен в начале координат, а вектор V_0 ориентирован вдоль некоторой координатной оси.

В рамках этой схемы все геометрические объекты сцены рождены из примитивов, которые часто являются **виртуальными**, реально не существующими в сцене объектами. С примитивами выполняются нужные элементарные преобразования (масштабирования, вращения, сдвиги), после чего центр объекта переносится в требуемую точку пространства o_i . В следующее состояние P_{i+1} объект опять переносится из начала координат, будучи предварительно подготовлен необходимыми элементарными преобразованиями.

Положительными аспектами итеративной схемы являются:

- экономия на переносах $P_i \rightarrow P_0$ объекта в начало координат, которые требуются в одном из вариантов следующей, рекуррентной схемы;
- возможность использования меньшего числа примитивов, чем объектов, в сцене, содержащей группы подобных объектов.

Недостаток схемы заключается в неэквивалентности двух переходов $P_0 \rightarrow P_i$ и $P_0 \rightarrow P_{i+1}$ одному экономичному переходу $P_i \rightarrow P_{i+1}$. Визуально это проявляется как нежелательное кручение движущегося объекта вокруг своей оси. Итеративную схему рекомендуется использовать для организации движения тел вращения, изображение которых не зависит от осевого кручения.

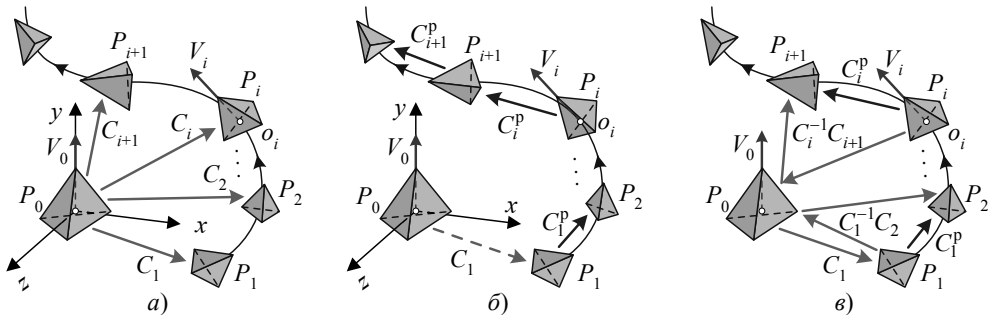


Рис. 3.39

□ По **рекуррентной схеме** (рис. 3.39,б) производится вычисление состояний

$$\tilde{P}_{i+1} = \tilde{P}_i C_i^p \quad \forall i=1,2,\dots, \quad (3.68б)$$

начиная с состояния P_1 , которое либо исходно задаётся элементами $\{o_1, V_1\}$, либо рассчитывается по формуле $\tilde{P}_1 = \tilde{P}_0 C_1$ из состояния примитива P_0 с матрицей C_1

из итеративной схемы. Линейные размеры объектов P_i , P_{i+1} и длины векторов V_i , V_{i+1} находятся в одинаковой пропорции.

Достоинство рекуррентной схемы заключается в возможности реализации экономичного движения и в некоторых случаях устранения осевого кручения. **Недостаток** схемы кроется в относительной сложности расчёта матриц преобразования. Её рекомендуется использовать в экономичном варианте при неполной информации об осевом кручении объекта вокруг вектора V .

Возможен другой вариант рекуррентной схемы (рис. 3.39,б), где один переход $P_i \rightarrow P_{i+1}$ заменён двумя переходами $P_i \rightarrow P_0 \rightarrow P_{i+1}$ через начальное состояние P_0 . Если состояние P_1 находить как $\tilde{P}_1 = \tilde{P}_0 C_1$, а в рекуррентной формуле $\tilde{P}_{i+1} = \tilde{P}_i C_i^p$ использовать матрицу $C_i^p = C_i^{-1} C_{i+1}$, то получим цепочку состояний

$$\tilde{P}_2 = \tilde{P}_1 C_1^{-1} C_2 = \tilde{P}_0 C_2, \quad \tilde{P}_3 = \tilde{P}_2 C_2^{-1} C_3 = \tilde{P}_0 C_3, \quad \dots, \quad \tilde{P}_{i+1} = \tilde{P}_0 C_i C_i^{-1} C_{i+1} = \tilde{P}_0 C_{i+1}, \quad \dots,$$

точно совпадающих с состояниями итеративной схемы (3.68а).

Переходим к разработке **кинематических алгоритмов** вычисления матриц преобразования состояний объекта с неполностью определёнными элементами движения. Итак, пусть даны два состояния объекта $\{o_1, V_1\}$ и $\{o_2, V_2\}$ с центрами в точках o_1 , o_2 и осевыми направлениями V_1 , V_2 . Относительное изменение размеров объекта зададим коэффициентом подобия $m = |V_2|/|V_1|$.

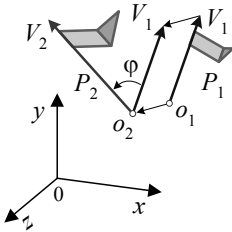


Рис. 3.40

Поставим задачу конструирования матриц преобразования $P_1 \rightarrow P_2$, включающего перенос центра объекта по прямой $o_1 \rightarrow o_2$ и поворот осевой линии в направлении $V_1 \rightarrow V_2$. Кратчайшим будет такое вращение вектора V_1 до вектора V_2 , которое происходит в плоскости $\{o_2, V_1, V_2\}$ и осуществляет одновременный поворот всех точек объекта, перенесённого из o_1 в o_2 , на один и тот же **минимальный** угол ϕ (рис. 3.40).

□ По методу **координатных вращений**, совершаемых **ортогональными** матрицами (3.44в,г), выполним переход из состояния $\{o_1, V_1\}$ в состояние $\{o_2, V_2\}$ за пять фаз с соответствующими матрицами фазовых преобразований.

Ф1: $\{o_1, V_1\} \rightarrow \{O_3, V_1\}$ — **перенос** объекта из точки o_1 в начало координат матрицей $T(-o_1)$.

Ф2: $\{O_3, V_1\} \rightarrow \{O_3, |V_1|z^0\}$ — **совмещение** вектора V_1 с осью z матрицей $A_{\text{ккв}}(V_1)$.

Ф3: $\{O_3, |V_1|z^0\} \rightarrow \{O_3, |V_2|z^0\}$ — **масштабирование** объекта в $m = |V_2|/|V_1|$ раз матрицей $M(m, m, m) = mE_3$.

Ф4: $\{O_3, |V_2|z^0\} \rightarrow \{O_3, V_2\}$ — **совмещение** вектора $|V_2|z^0$ с вектором V_2 матрицей $A_{\text{ккв}}^T(V_2)$.

Ф5: $\{O_3, V_2\} \rightarrow \{o_2, V_2\}$ — **перенос** объекта из начала координат в точку o_2 матрицей $T(o_2)$.

С учётом того, что фазы **Ф2** и **Ф4** включают до двух вращений, суммарное число элементарных преобразований достигает **семи**! Матрица сложного преобразования вычисляется по довольно-таки длинной формуле

$$C_{\text{кв}}(P_1, P_2) = T(-o_1) \tilde{A}_{\text{ккв}}(V_1) \tilde{M}(m, m, m) \tilde{A}_{\text{ккв}}^T(V_2) T(o_2), \quad (3.69)$$

в которой обращение матрицы $\tilde{A}_{\text{ккв}}(V_2)$ благодаря её ортогональности заменено транспонированием. На рис. 3.41 показаны выполняемые по описанному алгоритму фазы преобразования несимметричного объекта — двухцветного флажка, осевая ориентация которого определена вектором V_1 . Видно, что конечное состояние $\{o_2, V_2\}$ не совпадает с результатом экономичного движения, изображённым на рис. 3.40. Совпали лишь концы вектора V_2 . Остальные точки объекта могут быть закручены вокруг оси, при этом угол

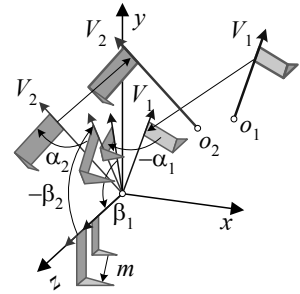


Рис. 3.41

кручения может достичь 180° при разности углов $|\alpha_1 - \alpha_2| = 180^\circ$. Подобный **резкий** переворот показан на рис. 3.29,а, где состояния объекта $P_1 \div P_n$ вычисляются по (3.68а) **итеративно** из начального состояния $P_0(O_3, z^0)$. Тот же эффект будет наблюдаться при моделировании движения объекта, не являющегося телом вращения, например, самолета: при проходе оси фюзеляжа через вертикаль произойдет его быстрый переворот вокруг оси на 180° , что, наверняка, вовсе не входило в планы программиста симуляции полёта.

В чём причина такого нежелательного эффекта? Очевидно, в том, что **сложное преобразование (3.69) не является экономичным**: четыре вращения объекта на углы $-\alpha_1, \beta_1, \alpha_2$ и $-\beta_2$ не происходят в одной плоскости и не дают в сумме кратчайшего поворота. Таким образом, **метод координатных вращений с промежуточной ориентацией объекта вдоль координатной оси, ортогональной плоскости углов α , не способен дать экономичного решения кинематической задачи**.

□ В **методе экономичного вращения** явно присутствует кратчайшее вращение объекта на угол $\varphi \geq 0$, заключённый между векторами V_1 и V_2 (см. рис. 3.40). Значение этого угла $\varphi = \arccos(\rho)$ определяется по скалярному произведению $\rho = \bar{V}_1 \circ \bar{V}_2$. Ось вращения ортогональна плоскости $\{o_2, V_1, V_2\}$ и направлена по нормали $N = V_1 \times V_2$. В случае параллельности векторов $V_1 \parallel V_2$, обнаруживаемой по значению $|\rho| = 1$, можно взять вектор $N = V_1 \times \omega^0$, ортогональный вектору V_1 и не параллельному ему орту $\omega^0 \# V_1$, где $\omega \in \{x, y, z\}$.

Составим фазы экономичного преобразования объекта (рис. 3.42) и соответствующие им матрицы.

Ф1: $\{o_1, V_1\} \rightarrow \{O_3, V_1\}$ — **перенос** объекта из точки o_1 в начало координат матрицей $T(-o_1)$.

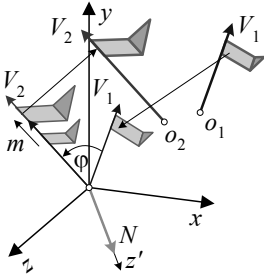


Рис. 3.42

Ф2: $\{x, y, z\} \rightarrow \{x', y', z'\}$ — **совмещение** оси z с вектором N **ортогональной** матрицей $A_{\text{кэв}}(N)$, включающей до трёх вращений.

Ф3: $\{O_3, V_1'\} \rightarrow \{O_3, V_2'\}$ — **вращение** объекта вокруг оси z' матрицей $R_z(\arccos(\rho))$, которую экономичнее получить по (3.45) как $R_{2z}(\rho, \sqrt{1-\rho^2})$.

Ф4: $\{O_3, V_2'\} \rightarrow \{O_3, mV_2'\}$ — **масштабирование** объекта в $m = |V_2|/|V_1|$ раз матрицей $M(m, m, m)$.

Ф5: $\{x', y', z'\} \rightarrow \{x, y, z\}$ — **восстановление** системы координат ортогональной матрицей $A_{\text{кэв}}^T(N)$, включающей до трёх вращений.

Ф6: $\{O_3, V_2\} \rightarrow \{o_2, V_2\}$ — **перенос** объекта из начала координат в точку o_2 матрицей $T(o_2)$.

В итоге матрица сложного преобразования, включающего до **десяти** (!) элементарных, вычисляется по следующему алгоритму:

$$\begin{cases} \rho = \bar{V}_1 \circ \bar{V}_2, \quad m = |V_2|/|V_1|, \quad N = V_1 \times \text{if}(|\rho| < 1: V_2, \text{if}(|V_1 \times z^0|: z^0, x^0)) \\ C_{\text{эв}}(P_1, P_2) = T(-o_1) \tilde{A}_{\text{кэв}}(N) \tilde{R}_{2z}(\rho, \sqrt{1-\rho^2}) \tilde{M}(m, m, m) \tilde{A}_{\text{кэв}}^T(N) T(o_2). \end{cases} \quad (3.70)$$

Ориентация флажков на рис. 3.29,б, экономично переведённых из состояния P_0 , показывает существование **плавного**, в отличие от МКВ, осевого кручения.

□ Неполная определённость состояний объекта и желание осуществить переход $P_1 \rightarrow P_2$ по кратчайшим направлениям переноса $o_1 \rightarrow o_2$ и вращения $V_1 \rightarrow V_2$

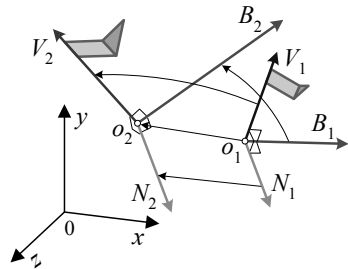


Рис. 3.43

(рис. 3.43) подталкивают реализовать его **методом парных точек**. Матрицу этого активного преобразования даёт определённая в (3.22) функция $MP2$ с двумя парами аргументов:

$$C_{\text{п}}(P_1, P_2) = MP2(o_1, V_1, o_2, V_2) = \begin{bmatrix} V_1 & 0 \\ N_1 & 0 \\ V_1 \times N_1 & 0 \\ o_1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} V_2 & 0 \\ N_2 & 0 \\ V_2 \times N_1 & 0 \\ o_2 & 1 \end{bmatrix}. \quad (3.71)$$

Аналитическая зависимость матрицы $C_{\text{п}}$ от исходных данных получается после выбора и подстановки в (3.18б) четырёх пар элементов исходного и конечного состояний объекта:

- в исходном состоянии известны точка o_1 и вектор V_1 . Добавим в этот набор векторы нормали $N_1 = V_1 \times V_2$ (если $V_1 \parallel V_2$, то при $V_1 \neq z^0$ возьмём $N_1 = V_1 \times z^0$, а при $V_1 \parallel z^0$ — $N = V_1 \times x^0$) и бинормали $B_1 = V_1 \times N_1$;

- в конечном состоянии даны точка o_2 и вектор V_2 . Добавим к ним векторы нормали $N_2 = mN_1$ и бинормали $B_2 = V_2 \times N_1$. Длины всех векторов конечного состояния отличаются от длин векторов начального состояния в $m = |V_2|/|V_1|$ раз.

Ранее отмечалась эквивалентность МЭВ и нормированного МПТ. Поскольку (3.70) включает операцию равномерного масштабирования объекта, приведённого в начало координат, матрицей $M(m, m, m)$, а в (3.71) она выполняется автоматически при $m=|V_2|/|V_1| \neq 1$, то можно утверждать — и это ниже подтвердится моделированием, — что **метод парных точек обеспечивает экономичный переход из состояния $P_1\{o_1, V_1\}$ в состояние $P_2\{o_2, V_2\}$ при существенно более простом алгоритме вычисления матрицы этого перехода.**

☞ Сравним две схемы (итеративную и рекуррентную) и три метода (координатных вращений, экономичного вращения и парных точек) в задаче движения правильного n -угольника, вписанного в единичную окружность, по цепочке состояний $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_m \rightarrow P_1$, центры которых o_i равномерно распределены с угловым шагом $\psi = 2\pi/m$ по **направляющей** окружности $o(t)$ радиуса r , а нормали полигонов V_i касательны к ней в точках o_i (рис. 3.44).

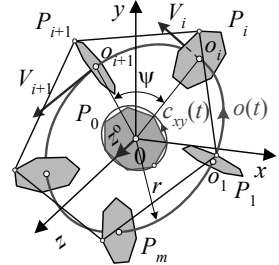


Рис. 3.44

Выберем в качестве примитива окружность (3.60) $c_{xy}(t) = [\cos(t) \ \sin(t) \ 0]^T$ с нормалью z^0 . Впишем в неё полигон $P_0 = p_{01}p_{02} \dots p_{0n}p_{01}$ с вершинами

$$p_{0k} = c_{xy}(2\pi(k-1)/n) \quad \forall k = \overline{1, n}. \quad (3.72a)$$

Получим направляющую линию масштабированием примитива до радиуса r и двумя координатными вращениями на углы β и α (см. рис. 3.32,а):

$$o(t) = r c_{xy}(t) A_o, \quad \text{где} \quad A_o = R_x(-\beta) R_y(\alpha). \quad (3.72б)$$

Лежащие на ней точки o_i , касательные в этих точках векторы V_i и состояния полигона P_i могут быть рассчитаны по следующим формулам:

$$\begin{aligned} o_i &= r c_{xy}(t_i) A_o, \quad V_i = c'_{xy}(t_i) A_o = r [-\sin(t_i) \ \cos(t_i) \ 0]^T A_o, \\ \tilde{P}_i &= \tilde{P}_0 \tilde{R}_x(-\pi/2) \tilde{A}_o T(o_i) \quad \forall t_i = (i-1)\psi, \quad i = \overline{1, m}. \end{aligned} \quad (3.72в)$$

Соединение смежных состояний вершин полигона отрезками $p_{ik}p_{i,k+1}$ даёт систему продольных, а контуры самих полигонов P_i — систему поперечных каркасных линий поверхности, названной в гл. 4 **трубчатой**. Формулы (3.72) составляют **аналитический** метод построения трубчатой поверхности с произвольной ориентацией плоскости направляющей окружности. Вид каркаса с параметрами $n=7$ и $m=36$ показан на двойном рис. 3.45,а: сверху для углов ориентации $\beta=\alpha=0$, а снизу — для $\beta=40^\circ$ и $\alpha=-30^\circ$. Примем эти поверхности, не имеющие осевого кручения, **эталонами** для сравнения с объектами, построенными по двум схемам тремя методами преобразований.

Моделирование кинематических схем (3.68) с применением матриц преобразований $C(P_1, P_2)$, полученных методами (3.69)–(3.71), дало следующие интересные результаты:

- по **итеративной** схеме (3.68а) все состояния $P_i\{o_i, V_i\}$ рассчитываются из начального состояния $P_0(O_3, z^0)$, построенного по (3.72а), с использованием

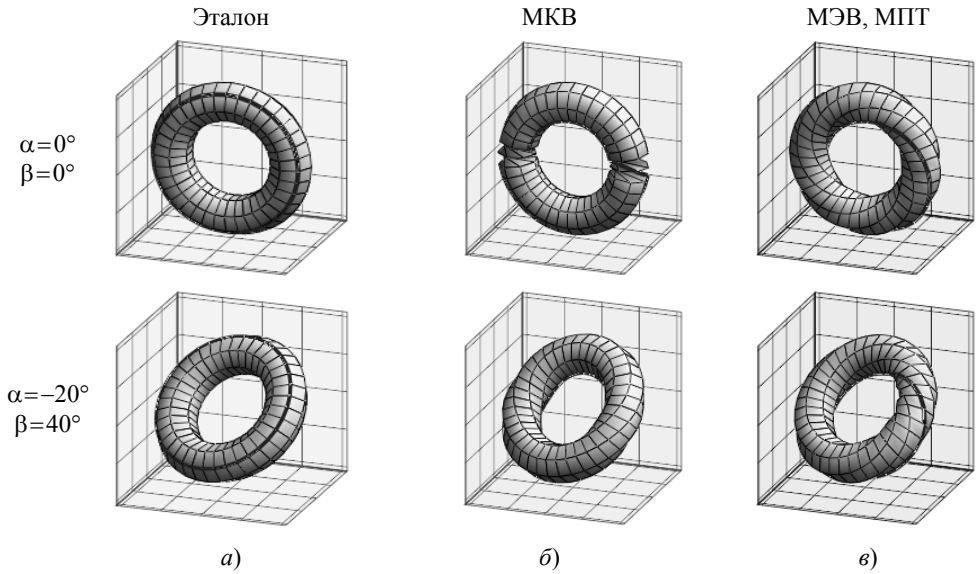


Рис. 3.45

матриц преобразования $C_i(P_0, P_i)$. По **методу координатных вращений** получаются поверхности с сильным осевым кручением (рис. 3.45, б). Максимум кручения поперечных сечений на 180° получается при угле $\beta=0$ на двух участках с вертикальной ориентацией касательного вектора $o'(t)$. Методы **экономического вращения** и **парных точек** показали, как и ранее, одинаковые решения, также подверженные осевому кручению на суммарный угол 360° , **равномерно** распределённый по всей направляющей траектории (рис. 3.45, в);

- по **рекуррентной** схеме (3.68б) состояния $P_{i+1}(o_{i+1}, V_{i+1})$ получаются из

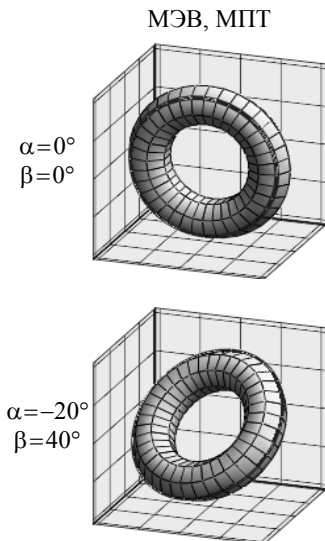


Рис. 3.46

$P_i(o_i, V_i)$ с помощью матриц $C_i^P(P_i, P_{i+1})$, а в расчёте первого состояния $\tilde{P}_1 = \tilde{P}_0 C_1$ используется матрица $C_1(P_0, P_1)$ одного из трёх сравниваемых методов. По **методу координатных вращений** получаются поверхности, точно совпадающие с решениями итеративной схемы на рис. 3.45, б, поскольку МКВ даже в рекуррентной схеме выполняется с помощью вспомогательного перевода объекта в начало координат (см. рис. 3.39, в). Методы **экономического вращения** и **парных точек** снова сгенерировали одинаковые поверхности, но с нулевым осевым кручением (рис. 3.46). Эти поверхности совпадают с эталонными **геометрически**, но не параметрически, в чём убеждает их сравнение по фиксированной продольной каркасной линии.

Таким образом, **наилучший результат кинематического моделирования достигается в ре-**

куррентной схеме (3.68б) применения метода парных точек с матрицами преобразований

$$C_1 = MP2(O_3, z^0, o_1, \bar{V}_1), \quad C_i^p = MP2(o_i, \bar{V}_i, o_{i+1}, \bar{V}_{i+1}). \quad (3.73)$$

Естественный вопрос: если существует аналитический метод кинематического расчёта (3.72), то зачем нужны другие алгоритмы, дающие нежелательные эффекты кручения? Ответ: в общем случае направляющая траектория $o(t)$ может формироваться интерактивно под управлением с клавиатуры. Протягивание вдоль неё объекта с помощью алгоритма, отличного от рекуррентного МПТ, неизбежно приведет к его осевому кручению, даже если направляющая линия является плоской с нулевым кручением $\chi(t)$, определённым в (1.110а).

Простой способ устранения осевого кручения объекта на **плоских** участках направляющей траектории заключается в уменьшении неопределённости задания его пространственной ориентации. Дополним, например, фиксацию двух элементов $\{o_i, \bar{V}_i\}$ i -го состояния полигона $P_i = p_{i1}p_{i2} \dots p_{in}p_{i1}$ требованием ориентации его первого радиуса $p_{i1} - o_i$ по нормали N к плоскости линии $o(t)$. В состоянии P_0 этому радиусу соответствует орт x^0 (рис. 3.47,а).

Постоянный вектор нормали можно вычислить как преобразованием орта z^0 матрицей A_o из (3.72б), так и по двум радиус-векторам на **центрированной** окружности $o(t)$:

$$N = z^0 A_o \quad \text{или} \quad N = o_1 \times o_2. \quad (3.74а)$$

Располагая двумя тройками элементов объекта в начальном $P_0(O_3, z^0, x^0)$ и текущем $P_i(o_i, \bar{V}_i, \bar{N})$ состояниях полигона, логично использовать более простую в программировании, чем (3.73), **итеративную** схему с матрицей преобразования, полученной с помощью функции $MP3$ из (3.20):

$$\tilde{P}_i = \tilde{P}_0 C_i, \quad \text{где} \quad C_i = MP3(O_3, z^0, x^0, o_i, \bar{V}_i, \bar{N}). \quad (3.74б)$$

Результат моделирования по (3.74) показан на рис. 3.47,б,в и коренным образом отличается от закрученных поверхностей на рис. 3.45,в, полученных итеративным МПТ с матрицами преобразований $C_i = MP2(O_3, z^0, o_i, \bar{V}_i)$. Таким образом, **итеративный алгоритм (3.74) составляет достойную конкуренцию рекуррентной схеме (3.73).**

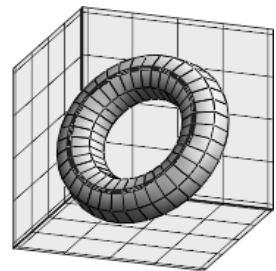
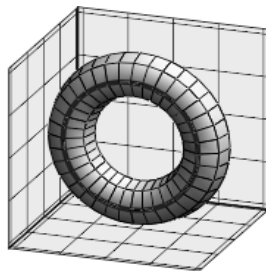
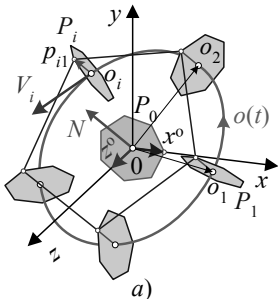


Рис. 3.47

3.2. ПРОЕКТИВНЫЕ ПРЕОБРАЗОВАНИЯ

Проецирование (от лат. *projectio* — выбрасывать вперед) означает отображение трёхмерного объекта на двухмерную **картинную плоскость** (КП). Получение проекции базируется на проведении лучей из центра проецирования S (**проектора**) через точки объекта до их пересечения с КП

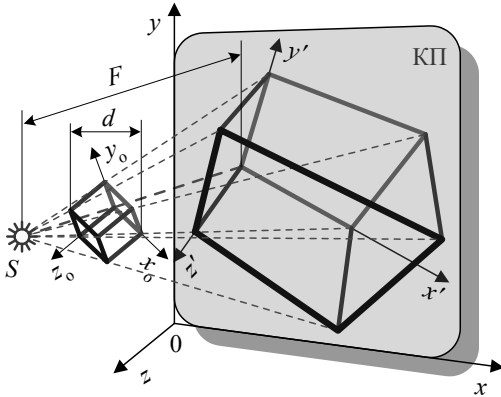


Рис. 3.48

(рис. 3.48). След, образуемый точками пересечения лучей с КП, составляет **проекцию** объекта. Если **фокусное расстояние** F от источника до картинной плоскости во много раз превосходит габариты объекта d , то пучок проецирующих лучей можно считать **параллельным**, а источник — **дальним**. Такая **идеализация** отображения объёмного мира на плоский экран облегчает построение проекций, за что приходится расплачиваться ухудшением реалистичности изображения.

За длительное время развития цивилизации человеком придумано множество способов изображения объёмных объектов на плоскости, более или менее точно передающих их геометрическую форму. Поскольку проецирование — не аффинное преобразование и в принципе необратимо, то любая проекция всегда содержит в себе ошибку (лучше сказать — неопределённость) восстановления координат объекта-образа. Назначение того или иного метода проецирования состоит в уменьшении неопределённости восприятия наиболее значимых свойств объекта за счёт менее значимых. Например, архитектурный чертёж фасада здания абсолютно точно передаёт геометрическую ширину и высоту как всего фасада, так и его деталей, за счёт полной неопределённости их глубины.

К числу важнейших свойств метода проецирования относится **достоверность восприятия** объекта наблюдателем по его проекции, т. е. узнаваемость объекта по его плоскому изображению. Если свойство достоверности восприятия объявляется доминирующим, то наилучшей может стать «живая» проекция с далеко не минимальной геометрической неопределённостью. Например, изометрическая проекция несёт искажённую информацию о размерах объекта во всех трёх измерениях, но лучше передаёт форму объекта в целом в сравнении с изображением объекта в «фас».

На рис. 3.49 приведена общепринятая **классификация проекций**, используемых для изображения трёхмерных объектов или сцен на картинной плоскости. С учётом вышесказанного следует признать, что **наилучшей проекции не существует**. Во-первых, критерий выбора проекции определяется её назначением — достижением геометрической точности или получением эстетического удовлетворения. Во-вторых, «на вкус и цвет товарищей нет»: автор данной книги, например, предпочитает диметрическую аксонометрическую проекцию (это

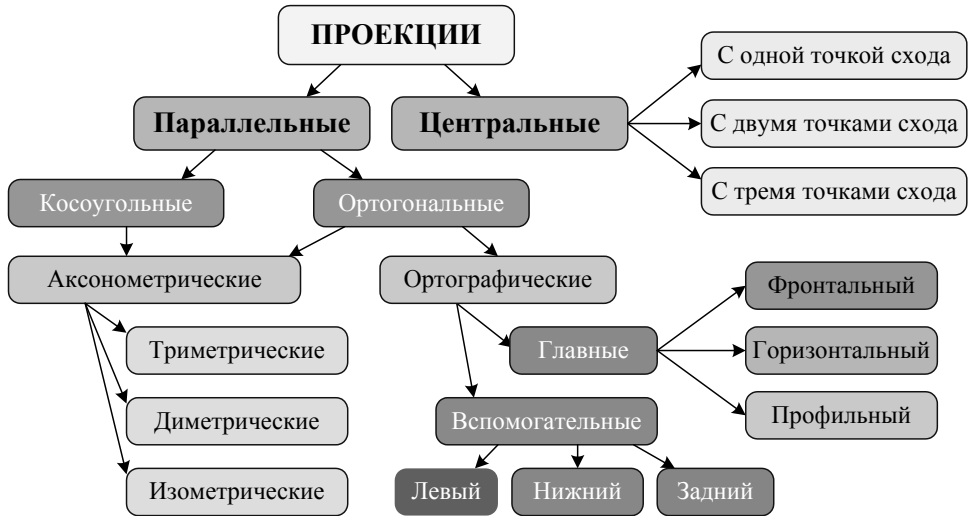


Рис. 3.49

видно по большинству рисунков), тогда как некоторые эстеты без ума от изометрии с её чересчур правильной и скучной симметрией.

Для лучшего восприятия ориентации объектов в пространстве мы иногда будем рисовать вместе с проекциями объектов ещё и проекции $x'y'z'$ осей СКС $x_0y_0z_0$ или ОСК $x_0y_0z_0$. Эти проекции создаются по тем же алгоритмам, которые используются для изображения объектов.

3.2.1. Ортографические проекции

Для получения **ортографических** проекций (**видов**) оси объекта ориентируются параллельно осям МСК xuz , вдоль которых идут пучки проецирующих лучей. После этого находятся точки пересечения лучей с координатными плоскостями, которые затем раскладываются на плоскости чертежа (рис. 3.50,а).

Так как проецирующие лучи перпендикулярны координатным плоскостям, то в каждом виде теряется информация о протяжённости вдоль оси координат, параллельной пучку лучей. **Отдельные виды необратимы.** Восстановление точек 3d-объекта возможно не менее чем по двум его видам, содержащим совместно все три координаты каждой точки.

С целью точной передачи размеров деталей в инженерной практике применяются **чертежи** в составе трёх **главных видов** (рис. 3.50,б):

- переднего или **фронтального** (*front*) на картинную плоскость **f** (ФКП), ле-

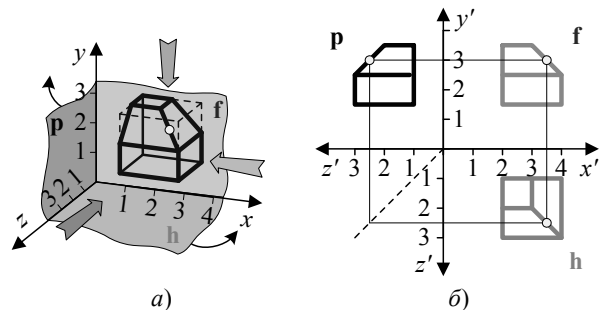


Рис. 3.50

жащую в координатной плоскости $xу$ МСК. Фронтальная проекция точки p имеет координаты $x'=x$, $y'=y$ и $z'=0$;

- верхнего или **горизонтального** (*horizon*) на КП **h** — плоскость xz МСК. Координаты горизонтальной проекции точки p равны $x'=x$, $y'=0$ и $z'=z$;
- правого или **профильного** (*profile*) на КП **p** — плоскость yz МСК. Координаты профильной проекции точки p равны $x'=0$, $y'=y$ и $z'=z$.

Если сплошной (не каркасный) объект имеет сложную форму, которую нельзя передать главными видами, то в чертежах дополнительно приводятся **вспомогательные виды: левый, нижний и задний**.

Матрицы ортогографического проецирования

$$O_f = \text{diag}[1 \ 1 \ 0], \quad O_h = \text{diag}[1 \ 0 \ 1], \quad O_p = \text{diag}[0 \ 1 \ 1] \quad (3.75)$$

вырождены, что подтверждает невозможность восстановления по (3.2) всех трёх координат точки только по одному её виду.

Область применения видов — машиностроительное черчение, архитектура и другие приложения, требующие точных измерений по чертежу линейных и угловых размеров благодаря параллельности измеряемых элементов плоскостям проецирования. Особое значение имеет фронтальное ортогональное проецирование с матрицей O_f как последняя фаза аксонометрического проецирования.

Создание иллюзии объёмности объекта с помощью только **одной** плоской проекции в параллельных лучах возможно двумя способами:

- во-первых, объект вместе с его ОСК подвергается преобразованиям вращения вокруг двух или трёх осей МСК, так что проецирующие лучи, оставаясь перпендикулярными к проективной плоскости, освещают объект сбоку. Так получаются **ортогональные аксонометрические проекции**;
- во-вторых, можно пустить проецирующий пучок под некоторым углом к КП, осветив тем самым неподвижный объект сбоку. Так образуются **косугольные проекции** и их оптический аналог — косые тени объектов на плоскости.

3.2.2. Аксонометрические проекции

Пусть первоначально ОСК совпадает с МСК. Выберем **фронтальную** КП проецирования $xу$ и следующую цепочку вращений объекта и его ОСК $x_0y_0z_0$: $\varphi_y \rightarrow \varphi_x \rightarrow \varphi_z$. Такая очередность поворотов даёт привычную вертикальную ориентацию проекции y' оси y_0 при произвольных углах первых двух вращений φ_y и φ_x . Лишь третье вращение ОСК на угол φ_z поворачивает проекцию этой оси на такой же угол. В результате получим ориентацию ОСК относительно ФКП, изображённую на рис. 3.51.

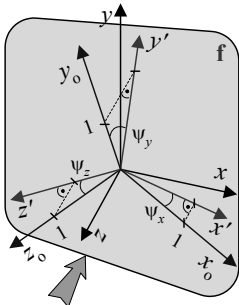


Рис. 3.51

Задача аксонометрии (от греч. *axo* — ось, *metreo* — измерять) состоит в расчёте матрицы проецирования A_f , вычислении **координат проекции точки**

$$\begin{bmatrix} x' & y' & 0 \end{bmatrix} = \begin{bmatrix} x & y & z \end{bmatrix} A_f, \quad (3.76)$$

осевых масштабов m_x , m_y , m_z и ориентации проекций осей ОСК x' , y' и z' на ФКП. Рассчитаем матрицу трёх вращений объекта в неподвижной МСК xyz :

$$A_{\text{акс}} = R_y(\varphi_y)R_x(\varphi_x)R_z(\varphi_z) = \begin{bmatrix} m_{xx} & m_{xy} & m_{xz} \\ m_{yx} & m_{yy} & m_{yz} \\ m_{zx} & m_{zy} & m_{zz} \end{bmatrix}. \quad (3.77a)$$

Её (i, j) -е элементы как j -е координаты i -х ортов ОСК

$$\begin{cases} m_{xx} = \cos(\varphi_y)\cos(\varphi_z) - \sin(\varphi_x)\sin(\varphi_y)\sin(\varphi_z), \\ m_{xy} = \cos(\varphi_y)\sin(\varphi_z) + \sin(\varphi_x)\sin(\varphi_y)\cos(\varphi_z), \quad m_{xz} = -\cos(\varphi_x)\sin(\varphi_y), \\ m_{yx} = -\cos(\varphi_x)\sin(\varphi_z), \quad m_{yy} = \cos(\varphi_x)\cos(\varphi_z), \quad m_{yz} = \sin(\varphi_x), \\ m_{zx} = \sin(\varphi_y)\cos(\varphi_z) + \sin(\varphi_x)\cos(\varphi_y)\sin(\varphi_z), \\ m_{zy} = \sin(\varphi_y)\sin(\varphi_z) - \sin(\varphi_x)\cos(\varphi_y)\cos(\varphi_z), \quad m_{zz} = \cos(\varphi_x)\cos(\varphi_y) \end{cases} \quad (3.77б)$$

связаны равенствами $m_{\omega x}^2 + m_{\omega y}^2 + m_{\omega z}^2 = 1 \quad \forall \omega \in \{x, y, z\}$. Получим соотношения для расчёта **углов ψ_x , ψ_y и ψ_z наклона осей ОСК к ФКП**:

$$\begin{cases} \sin(\psi_x) = m_{xz} = -\cos(\varphi_x)\sin(\varphi_y), \\ \sin(\psi_y) = m_{yz} = \sin(\varphi_x), \\ \sin(\psi_z) = m_{zz} = \cos(\varphi_x)\cos(\varphi_y), \end{cases} \quad (3.78)$$

из которых следуют независимость углов ψ_x , ψ_y и ψ_z от угла φ_z последнего в (3.77a) вращения и **условие связи углов**

$$\boxed{\sin^2(\psi_x) + \sin^2(\psi_y) + \sin^2(\psi_z) = 1}. \quad (3.79)$$

Оно означает, что **свободно могут быть выбраны только один или два угла наклона осей ОСК к КП**. Примеры:

- $\psi_x = 45^\circ, \psi_y = 30^\circ \Rightarrow \sin^2(\psi_z) = 0.25 \Rightarrow \psi_z = 30^\circ$;
- $\psi_x = 45^\circ, \psi_z = 45^\circ \Rightarrow \sin^2(\psi_y) = 0 \Rightarrow \psi_y = 0^\circ$;
- $\psi_x = 90^\circ \Rightarrow \psi_y = \psi_z = 0^\circ$.

Из (3.78) следует решение и **обратной задачи**: для получения желаемых углов ψ_x , ψ_y и ψ_z углы вращения ОСК следует выбирать из условий

$$\varphi_x = \psi_y, \quad \varphi_y = -\arcsin(\sin(\psi_x)/\cos(\psi_y)) = \arccos(\sin(\psi_z)/\cos(\psi_y)). \quad (3.80)$$

Масштабные коэффициенты осевых искажений

$$\begin{cases} m_x = \sqrt{m_{xx}^2 + m_{xy}^2} = \sqrt{1 - m_{xz}^2} = \sqrt{1 - \cos^2(\varphi_x)\sin^2(\varphi_y)} = \cos(\psi_x) \leq 1, \\ m_y = \sqrt{m_{yx}^2 + m_{yy}^2} = \sqrt{1 - m_{yz}^2} = |\cos(\varphi_x)| = \cos(\psi_y) \leq 1, \\ m_z = \sqrt{m_{zx}^2 + m_{zy}^2} = \sqrt{1 - m_{zz}^2} = \sqrt{1 - \cos^2(\varphi_x)\cos^2(\varphi_y)} = \cos(\psi_z) \leq 1 \end{cases} \quad (3.81)$$

есть длины отрезков — проекций ортов ОСК на КП. Из (3.79) и (3.81) следует **условие взаимосвязи масштабов**:

$$\boxed{m_x^2 + m_y^2 + m_z^2 = 2}. \quad (3.82)$$

Как и в (3.79), **свободно могут быть выбраны только один или два масштаба**. Если нужно обеспечить желаемые соотношения масштабов $m_x:m_y:m_z=k_x:k_y:k_z$, то, подставив $m_i=ck_i$ в (3.82), получим общий делитель c и масштабные коэффициенты m_x , m_y и m_z :

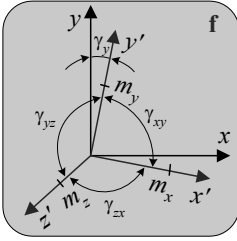


Рис. 3.52

$$c = \sqrt{\frac{2}{k_x^2 + k_y^2 + k_z^2}}, \quad m_i = k_i \sqrt{\frac{2}{k_x^2 + k_y^2 + k_z^2}}. \quad (3.83)$$

Ориентация проекций осей ОСК на ФКП может быть определена, например, углами γ_y , γ_{xy} , γ_{yz} и γ_{zx} (рис. 3.52), причём $\gamma_{xy} + \gamma_{yz} + \gamma_{zx} = 360^\circ$:

$$\cos(\gamma_y) = m_{yy}/m_y = \cos(\varphi_z) \Rightarrow \boxed{\gamma_y = \varphi_z}; \quad (3.84a)$$

$$\left\{ \begin{aligned} \cos(\gamma_{xy}) &= \frac{m_{xx}m_{yx} + m_{xy} + m_{yy}}{m_x m_y} = \frac{\sin(\varphi_x)\sin(\varphi_y)}{\sqrt{1 - \cos^2(\varphi_x)\sin^2(\varphi_y)}}, \\ \cos(\gamma_{yz}) &= \frac{m_{yx}m_{zx} + m_{yz} + m_{zy}}{m_y m_z} = \frac{-\sin(\varphi_x)\cos(\varphi_y)}{\sqrt{1 - \cos^2(\varphi_x)\cos^2(\varphi_y)}}, \\ \cos(\gamma_{zx}) &= \frac{m_{zx}m_{xx} + m_{zy} + m_{xy}}{m_z m_x} = \\ &= \frac{\cos^2(\varphi_x)\sin(\varphi_y)\cos(\varphi_y)}{\sqrt{1 - \cos^2(\varphi_x)\sin^2(\varphi_y)}\sqrt{1 - \cos^2(\varphi_x)\cos^2(\varphi_y)}}. \end{aligned} \right. \quad (3.84b)$$

Изобразим на рис. 3.53 ориентацию проекций осей x' , y' , z' и кубов в зависимости от знаков углов φ_x и φ_y . Из анализа (3.84) следуют **выводы**:

- углы первых двух вращений φ_y и φ_x определяют коэффициенты осевых искажений и углы между проекциями осей ОСК;
- только угол φ_z последнего поворота ОСК определяет угол γ_y вертикальной ориентации проекции ОСК на ФКП. Именно поэтому вращение ОСК вокруг оси z выбрано последним, чтобы и без него можно было получать разнообразные ракурсы изображения объекта с привычным **вертикальным** расположением проекции оси y' .

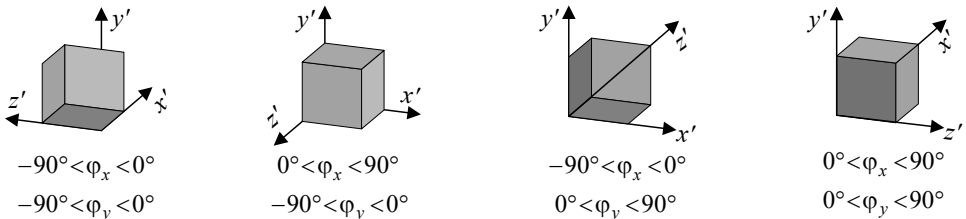


Рис. 3.53

Часто при построении аксонометрической проекции задаются масштабами m_x , m_y и m_z длин проекций ортов на осях объектных координат. Тогда остальные

параметры ортогональной аксонометрической проекции можно рассчитать по следующим из (3.81) формулам:

$$\begin{cases} \cos^2(\varphi_x) = m_y^2, \quad \cos^2(\varphi_y) = \frac{1-m_z^2}{m_y^2}, \\ \cos(\psi_x) = m_x, \quad \cos(\psi_y) = m_y, \quad \cos(\psi_z) = m_z, \\ \cos^2(\gamma_{xy}) = \frac{1-m_x^2}{1-m_y^2}, \quad \cos^2(\gamma_{yz}) = \frac{1-m_y^2}{1-m_z^2}, \quad \cos^2(\gamma_{zx}) = \frac{1-m_z^2}{1-m_x^2}. \end{cases} \quad (3.85)$$

Полная матрица аксонометрического ортогонального проецирования (3.76) с учётом последней операции ортографирования, дающей $z'=0$, равна

$$A_f = A_{\text{акс}} O_f = \begin{bmatrix} m_{xx} & m_{xy} & 0 \\ m_{yx} & m_{yy} & 0 \\ m_{zx} & m_{zy} & 0 \end{bmatrix}. \quad (3.86a)$$

Из (3.84a) следует, что для получения вертикальной ориентации проекции оси y' с углом $\gamma_y=0$ последний поворот сцены вокруг оси z не нужен. При $\varphi_z=0$ матрица аксонометрического преобразования получается равной $A_{\text{акс}} = R_y(\varphi_y)R_x(\varphi_x)$ и с учётом (3.77б) принимает вид

$$A_{\text{акс}} = \begin{bmatrix} \cos(\varphi_y) & \sin(\varphi_x)\sin(\varphi_y) & -\cos(\varphi_x)\sin(\varphi_y) \\ 0 & \cos(\varphi_x) & \sin(\varphi_x) \\ \sin(\varphi_y) & -\sin(\varphi_x)\cos(\varphi_y) & \cos(\varphi_x)\cos(\varphi_y) \end{bmatrix}. \quad (3.86б)$$

Во многих задачах (построение оптических эффектов, расчёт видимости поверхностей, их освещённости и т. д.) требуется знать **вектор S направления на источник проецирующих лучей**, выраженный **в координатах сцены**, в которых происходит движение всех её объектов. Например, при фронтальном ортографическом проецировании $S=[0 \ 0 \ 1]$, при горизонтальном — $S=[0 \ 1 \ 0]$, при профильном — $S=[1 \ 0 \ 0]$. Аксонометрическое преобразование сцены матрицей $A_{\text{акс}}$ изменяет его мировые координаты на $SA_{\text{акс}}$. Координаты вектора S в СКС находятся **обратным** преобразованием $S_c = SA_{\text{акс}}^{-1}$ с матрицей $A_{\text{акс}}^{-1} = A_{\text{акс}}^T$. Замена обращения матрицы её транспонированием обусловлена свойством ортогональности матриц элементарных вращений и упрощает формулу расчёта вектора источника в СКС:

$$S_c = SA_{\text{акс}}^T = \begin{bmatrix} s_x & s_y & s_z \end{bmatrix} \begin{bmatrix} m_{xx} & m_{yx} & m_{zx} \\ m_{xy} & m_{yy} & m_{zy} \\ m_{xz} & m_{yz} & m_{zz} \end{bmatrix}. \quad (3.87a)$$

В частном случае фронтальной аксонометрии с вектором $S=z^0$ и матрицей $A_{\text{акс}} = R_y(\varphi_y)R_x(\varphi_x)$ получим **нормированный** вектор

$$\bar{S}_c = \begin{bmatrix} m_{xz} & m_{yz} & m_{zz} \end{bmatrix} = \begin{bmatrix} -\cos(\varphi_x)\sin(\varphi_y) & \sin(\varphi_x) & \cos(\varphi_x)\cos(\varphi_y) \end{bmatrix}. \quad (3.87б)$$

АксонOMETрические проекции окружности и сферы

Ранее (см. п. 3.1.3.2) мы изучали кинематический метод получения параметрической модели $p(t)$ окружности радиуса r , лежащей в плоскости $\{o, N\}$, произвольно расположенной в пространстве. Положение точки на этой линии при любом значении параметра t вычисляется либо по векторным соотношениям (3.61), (3.64) или (3.65), либо по координатным зависимостям (3.63) или (3.66). Фронтальное аксонометрическое проецирование **окружности** ничем не отличается от проецирования других графических объектов и заключается в преобразовании с помощью матрицы A_f из (3.86) объектных координат выборочных точек в координаты их проекций на картинной плоскости $p'(t_i) = p(t_i)A_f$ и соединении точек проекции отрезками прямых.

Что представляет собой аксонометрическая проекция окружности на картинной плоскости? Исходя из способа получения линии $p(t) = c_{xy}(t)A(N) + o$ аффинным преобразованием окружности $c_{xy}(t) = [\cos(t) \sin(t) 0] \in xy$ матрицей $A(N)$ и переносом на вектор o , координатные функции её проекции имеют вид

$$x'(t) = a'_{11} \cos(t) + a'_{21} \sin(t) + o'_x, \quad y'(t) = a'_{12} \cos(t) + a'_{22} \sin(t) + o'_y, \quad (3.88)$$

где a'_{ij} — коэффициенты матрицы $A' = A(N)A_f$, а $o' = oA_f \equiv [o'_x \ o'_y \ 0]$. Сравнивая (3.88) с (3.58), заключаем, что **ортогональной аксонометрической проекцией окружности является эллипс** с полуосями $a \geq 0$, $b \geq 0$ и углом наклона $\alpha \in (-90^\circ, 90^\circ]$ (см. рис. 3.31,а), удовлетворяющими системе уравнений

$$\begin{cases} a'_{11} = a \cos(\alpha), & a'_{21} = -b \sin(\alpha) \\ a'_{12} = a \sin(\alpha), & a'_{22} = b \cos(\alpha) \end{cases} \Rightarrow \alpha = \arctg\left(\frac{a'_{12}}{a'_{11}}\right), \quad a = \frac{a'_{11}}{\cos(\alpha)}, \quad b = \frac{a'_{22}}{\cos(\alpha)}.$$

Наглядное представление об ориентации и длинах полуосей эллипса даёт изображённый на рис. 3.54,а процесс ортогонального проецирования окружности $\{o, N\}$ с нормалью N , наклонённой к картинной плоскости под углом ψ :

- малая полуось длиной $r \sin(\psi)$ параллельна проекции нормали $N' = NA_f$;
- большая полуось длиной r перпендикулярна вектору N' .

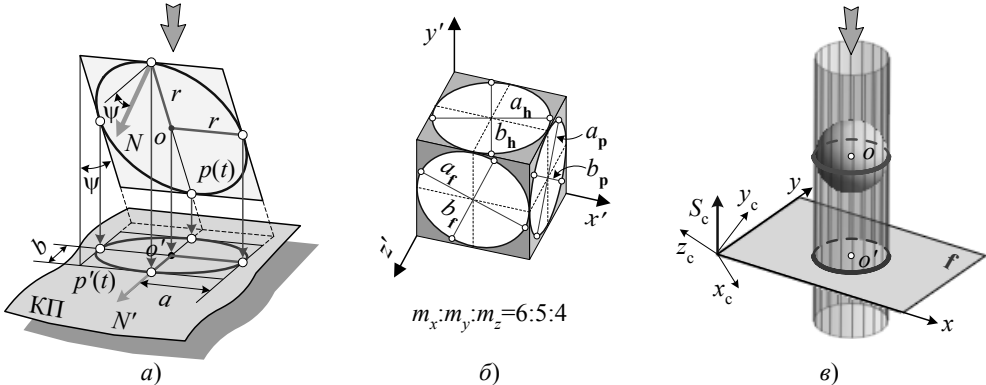


Рис. 3.54

В иллюстративных и учебных целях часто строят изображения окружностей, вписанных в грани модельного куба. Так как нормали граней куба параллельны осям ОСК $x_0y_0z_0$ и образуют с ФКП углы ψ_i , то по (3.85) получим следующие длины и ориентацию полуосей эллипсов, вписанных в проекции грани модельного куба (рис. 3.54,б):

$$\begin{cases} a_f = r \perp z', & a_h = r \perp y', & a_p = r \perp x', \\ b_f = r\sqrt{1-m_z^2} \parallel z', & b_h = r\sqrt{1-m_y^2} \parallel y', & b_p = r\sqrt{1-m_x^2} \parallel x'. \end{cases} \quad (3.89)$$

 **Пример 3.5.** Построить аксонометрическую проекцию куба со вписанными в его грани окружностями при соотношении масштабов $m_x:m_y:m_z=6:5:4$.

Решение. Вычислив по (3.83) $c=\sqrt{2/77} \approx 0.1612$, $m_x=6c \approx 0.967$, $m_y=5c \approx 0.806$ и $m_z=4c \approx 0.645$, определим по (3.84б) углы ориентации проекций осей ОСК $\gamma_{xy}=101.2^\circ$, $\gamma_{yz}=150.6^\circ$, $\gamma_{zx}=108.2^\circ$, а по (3.89) длины полуосей эллипсов $a_f=a_h=a_p=r$, $b_f=0.764r$, $b_h=0.592r$ и $b_p=0.255r$, являющихся фронтальными проекциями окружностей радиуса r на передней, верхней и правой гранях куба. Изображение объекта приведено на рис. 3.54,б. Для его построения по (3.84а), (3.85) и (3.86) рассчитаны следующие углы вращения и матрица проецирования: $\varphi_y=-18.4^\circ$, $\varphi_x=36.3^\circ$, $\varphi_z=0^\circ$, $\varphi_z=0^\circ$, $A_f = \begin{bmatrix} 0.949 & -0.187 & 0 \\ 0 & 0.806 & 0 \\ -0.316 & -0.562 & 0 \end{bmatrix}$. 

Что касается аксонометрической проекции **сферы** $\{o, r\}$, то ею благодаря ортогональности пучка проецирующих лучей всегда является **круг** радиуса r с центром в точке $o'=oA_f$ (рис. 3.54,в). Из бесконечного числа больших окружностей, образующихся сечением сферы плоскостями $\{o, N\}$ с различными направлениями нормалей N , лишь одна проецируется на картинную плоскость в круг, а не эллипс. Эта окружность создаёт **абрис** (контур, очертание) проекции сферы, а её нормаль направлена на источник проецирующих лучей S_c , определённый в (3.87). Ортогональная проекция окружности абриса строится по уравнениям (3.88), которые с учётом матриц аффинного преобразования $A(S_c)=rA_{\text{акс}}^{-1}$ и проецирования $A'=A(S_c)A_{\text{акс}}O_f=rO_f$ упрощаются до элементарного вида:

$$x'(t)=r\cos(t)+o'_x, \quad y'(t)=r\sin(t)+o'_y, \quad t \in [0, 2\pi].$$

Таким образом, мы изучили основы ортогональной аксонометрии — методики построения самой общей **триметрической** проекции, у которой все осевые масштабы m_x , m_y и m_z различны. Количество триметрических проекций бесконечно, **стандартная триметрия не регламентирована**.

Диметрические проекции

В **диметрических** проекциях два из трёх масштабов равны друг другу, что даёт три варианта выбора углов вращения объекта.

 Если $m_x=m_y \neq m_z$, то из (3.81) получим $\sin^2(\varphi_y)=\text{tg}^2(\varphi_x)$, откуда

$$\begin{aligned} |\varphi_x| &= |\arctg(\sin(\varphi_y))|, \quad m_x=m_y=|\cos(\varphi_x)|, \quad m_z=\sqrt{2}|\sin(\varphi_x)|, \\ \cos^2(\gamma_{yz}) &= \cos^2(\gamma_{zx}) = |\text{tg}(\varphi_x)/\text{tg}(2\varphi_x)|, \quad b_f=r\sqrt{|\cos(2\varphi_x)|}, \quad b_h=b_p=r|\sin(\varphi_x)|. \end{aligned}$$

Например, при $\varphi_y = -60^\circ$ получим $\operatorname{tg}^2(\varphi_x) = 0.75$. Из четырёх возможных углов выберем $\varphi_x = \operatorname{arctg}(\sqrt{3}/2) \approx 40.9^\circ$ и найдём параметры диметрической проекции куба на рис. 3.55,а: $m_x = m_y = 0.756$, $m_z = 0.926$, $\gamma_{yz} = \gamma_{zx} = 110.7^\circ$ и $\gamma_{xy} = 138.6^\circ$.

□ Если $m_y = m_z \neq m_x$, то из (3.81) получим $\cos^2(\varphi_y) = \operatorname{tg}^2(\varphi_x)$, откуда

$$|\varphi_x| = |\operatorname{arctg}(\cos(\varphi_y))|, \quad m_x = \sqrt{2}|\sin(\varphi_x)|, \quad m_y = m_z = |\cos(\varphi_x)|, \\ \cos^2(\gamma_{xy}) = \cos^2(\gamma_{zx}) = |\operatorname{tg}(\varphi_x)/\operatorname{tg}(2\varphi_x)|, \quad b_f = b_h = r|\sin(\varphi_x)|, \quad b_p = r\sqrt{|\cos(2\varphi_x)|}.$$

Выбрав, например, $\varphi_y = -30^\circ$, получим $\operatorname{tg}^2(\varphi_x) = 0.75$. Из четырёх возможных углов выберем $\varphi_x = \operatorname{arctg}(\sqrt{3}/2) \approx 40.9^\circ$ и определим параметры диметрической проекции на рис. 3.55,б: $m_x = 0.926$, $m_y = m_z = 0.756$, $\gamma_{xy} = \gamma_{zx} = 110.7^\circ$ и $\gamma_{yz} = 138.6^\circ$.

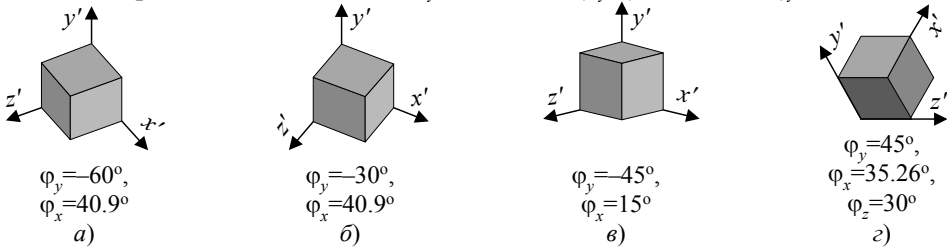


Рис. 3.55

□ Если $m_x = m_z \neq m_y$, то из (3.81) получим $\sin^2(\varphi_y) = \cos^2(\varphi_x)$, т. е. выбор углов вращения ОСК должен быть таков: $\varphi_y \in \{\pm 45^\circ, \pm 135^\circ\}$, φ_x — любой. Тогда параметры диметрической проекции имеют вид

$$m_x = m_z = \sqrt{3 - \cos(2\varphi_x)}/2, \quad m_y = |\cos(\varphi_x)|, \\ \cos^2(\gamma_{xy}) = \cos^2(\gamma_{yz}) = (1 - \cos(2\varphi_x))/(3 - \cos(2\varphi_x)), \\ b_f = b_p = r|\cos(\varphi_x)|/\sqrt{2}, \quad b_h = r|\sin(\varphi_x)|.$$

Выбрав, например, $\varphi_y = -45^\circ$ и $\varphi_x = 15^\circ$, получим параметры диметрической проекции на рис. 3.55,в: $m_x = m_z = 0.73$, $m_y = 0.966$, $\gamma_{xy} = \gamma_{yz} = 104.5^\circ$ и $\gamma_{xz} = 151^\circ$.

В **стандартной диметрической проекции** соотношение масштабов равно $m_x:m_y:m_z=2:2:1$. Тогда по (3.83) получим $m_x = m_y = 2\sqrt{2}/3 \approx 0.943$ и $m_z = \sqrt{2}/3 \approx 0.471$, а остальные параметры этой проекции (рис. 3.56) равны $\gamma_{yz} = \gamma_{zx} = 131.4^\circ$, $\gamma_{xy} = 97.2^\circ$,

$$b_f = r\sqrt{7}/3 \approx 0.882r \quad \text{и} \quad b_h = b_p = r/3 \approx 0.333r.$$

Для получения стандартной диметрии углы вращения должны быть по (3.85) равны

$$\boxed{\varphi_y = -\arccos(\sqrt{7/8}) \approx -20.7^\circ}, \quad \boxed{\varphi_x = \arccos(\sqrt{8/9}) \approx 19.47^\circ}$$

и $\varphi_z = 0^\circ$. Тогда матрица диметрического преобразования и удобный для запоминания вектор направления источника находятся по (3.86б) и (3.87б):

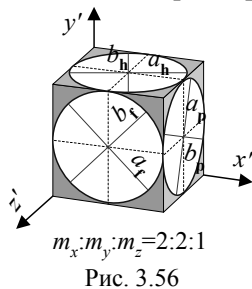


Рис. 3.56

$$A_{\text{дим}} = \begin{bmatrix} \sqrt{7/8} & -\sqrt{1/72} & 1/3 \\ 0 & \sqrt{8/9} & 1/3 \\ -\sqrt{1/8} & -\sqrt{7/72} & \sqrt{7/9} \end{bmatrix} \approx \begin{bmatrix} 0.935 & -0.118 & 0.333 \\ 0 & 0.943 & 0.333 \\ -0.354 & -0.312 & 0.882 \end{bmatrix}, \quad (3.90)$$

$$S_{\text{дим}} = [1 \ 1 \ \sqrt{7}] \approx [1 \ 1 \ 2.646], \quad |S_{\text{дим}}| = 3.$$

Изометрические проекции

При построении ортогональной **изометрической** проекции выбор углов вращения ОСК φ_x и φ_y связан условием $m_x = m_y = m_z$. Тогда из (3.83) получаем масштабы $m_i = \sqrt{2/3} \approx 0.816$, а из (3.85) и (3.89) следуют соотношения

$$\begin{aligned} \cos^2(\varphi_x) &= 2/3, \quad \cos^2(\varphi_y) = 1/2, \\ \cos^2(\gamma_{xy}) &= \cos^2(\gamma_{yz}) = \cos^2(\gamma_{zx}) = 1/4, \\ b_f &= b_h = b_p = r/\sqrt{3} \approx 0.577. \end{aligned}$$

Изометрическая проекция объекта может быть получена его вращением на углы $\varphi_y \in \{\pm 45^\circ, \pm 135^\circ\}$ и $\varphi_x \in \{\pm 35.26^\circ, \pm 144.74^\circ\}$. Например, изображение куба на рис. 3.55,з построено при углах $\varphi_y = 45^\circ$, $\varphi_x = 35.26^\circ$ и $\varphi_z = 30^\circ$.

Свойства изометрии: проекции осей ОСК ориентированы под углами, кратными 60° , все квадраты в координатных плоскостях ОСК выглядят ромбами, а проекции вписанных в них окружностей являются подобными эллипсами, отличающимися лишь ориентацией полуосей.

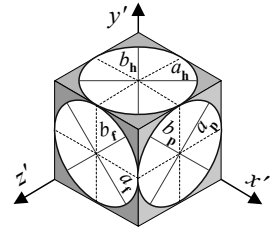
Стандартная изометрическая проекция (рис. 3.57) получается при углах вращения

$$\boxed{\varphi_y = -45^\circ}, \quad \boxed{\varphi_x = \arccos(\sqrt{2/3}) \approx 35.26^\circ}$$

и $\varphi_z = 30^\circ$, имеет симметрично ориентированные проекции осей ОСК с углами $\gamma_{xy} = \gamma_{yz} = \gamma_{zx} = 120^\circ$, а матрица преобразования (3.86б) и легко запоминаемый вектор источника (3.87б) равны

$$A_{\text{изо}} = \begin{bmatrix} \sqrt{1/2} & -\sqrt{1/6} & \sqrt{1/3} \\ 0 & \sqrt{2/3} & \sqrt{1/3} \\ -\sqrt{1/2} & -\sqrt{1/6} & \sqrt{1/3} \end{bmatrix} \approx \begin{bmatrix} 0.707 & -0.408 & 0.577 \\ 0 & 0.816 & 0.577 \\ -0.707 & -0.408 & 0.577 \end{bmatrix}, \quad (3.91)$$

$$S_{\text{изо}} = [1 \ 1 \ 1], \quad |S_{\text{изо}}| = \sqrt{3}.$$



$$m_x : m_y : m_z = 1 : 1 : 1$$

Рис. 3.57

3.2.3. Косоугольные проекции

Косоугольное проецирование отличается от ортогонального тем, что для объёмного видения объекта пучок параллельных лучей пускается не ортогонально КП. При этом сам объект и его ОСК остаются **неподвижными**. Соответственно $S_{\text{кос}} = S$. Косоугольное проецирование — основной инструмент расчёта тени объекта на плоскости и отражения объекта в плоском зеркале.

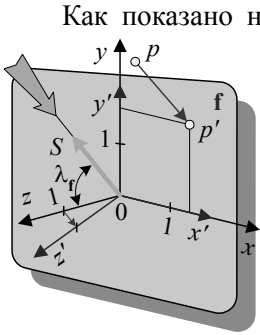


Рис. 3.58

Как показано на рис. 3.58, вектор направления на проецирующий пучок $S=[s_x \ s_y \ s_z]$ составляет с нормалью **фронтальной** КП **угол проецирования** λ_f с тангенсом $\operatorname{tg}(\lambda_f)=\sqrt{s_x^2+s_y^2}/s_z$. Параметрическая функция луча, проходящего через точку $p=[x \ y \ z]$ в направлении $-S$, есть $p-St$. Этот луч пересекает ФКП в точке $p'=[x' \ y' \ 0]$. Из условия $z'=z-s_z t_f=0$ получим параметр пересечения t_f и координаты проекции точки $p' \in f$:

$$t_f=z/s_z, \quad x'=x-zs_x/s_z, \quad y'=y-zs_y/s_z.$$

В векторной форме $p'=pK_f$ косоугольное проецирование на ФКП выполняется матрицей

$$K_f = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -c_x & -c_y & 0 \end{bmatrix}, \quad \text{где } c_x = \frac{s_x}{s_z}, \quad c_y = \frac{s_y}{s_z}. \quad (3.92)$$

Получим **коэффициенты осевых искажений**, проецируя орты ОСК на ФКП:

$$\begin{bmatrix} x^0 \\ y^0 \\ z^0 \end{bmatrix} K_f = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -c_x & -c_y & 0 \end{bmatrix} \Rightarrow m_x=m_y=1, \quad m_z=\sqrt{c_x^2+c_y^2}=\operatorname{tg}(\lambda_f). \quad (3.93)$$

Углы ориентации проекций осей ОСК на ФКП определяются условиями

$$\gamma_{xy}=90^\circ, \quad \operatorname{tg}(\gamma_{yz})=-c_x/c_y=-s_x/s_y, \quad \operatorname{tg}(\gamma_{zx})=-c_y/c_x=-s_y/s_x. \quad (3.94)$$

Косоугольные проекции окружности и сферы

Косоугольное проецирование пространственной **окружности** с помощью матрицы K заключается в преобразовании $p'(t)=p(t)K$ объектных координат $p(t)=c_{xy}(t)A(N)+o$ в координаты проекции, которая строится по уравнениям (3.88) с коэффициентами a'_{ij} матрицы $A'=AK$. Следовательно, **косоугольной проекцией окружности также является эллипс** (рис. 3.59,а).

В частном случае, когда плоскость окружности параллельна картинной плоскости, например фронтальной, все точки $p(t)$ имеют одинаковые координаты $z(t)=z_0$. Тогда изменение других координат $x'(t)=x(t)-c_x z_0$ и $y'(t)=y(t)-c_y z_0$ равносильно преобразованию параллельного переноса. Это означает, что **окружности в плоскостях, параллельных картинной плоскости, проецируются в окружности**, т. е. без искажений. В этом главное отличие и сильное преимущество косоугольного аксонометрического проецирования перед ортогональным, позволяющее передать на чертеже объём объекта и одновременно избежать построения эллипсов с наклонными полуосями.

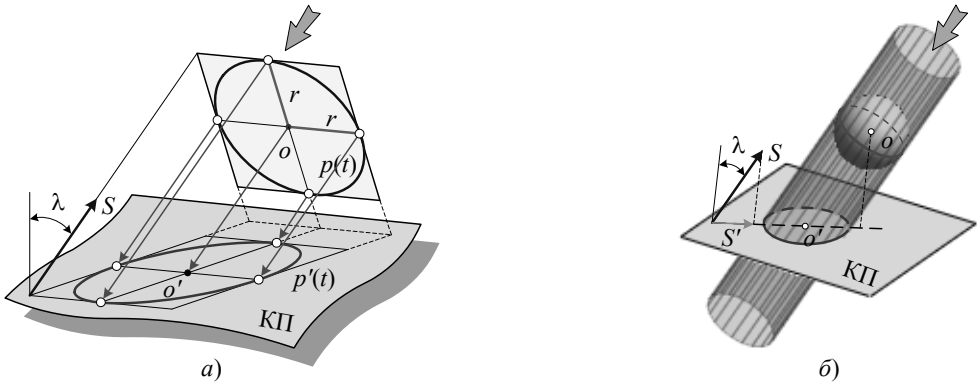


Рис. 3.59

Неортогональная проекция **сферы** $\{o, r\}$ образуется окружностью сечения круглого проективного цилиндра картинной плоскостью, наклонённой к оси цилиндра под углом λ . Следовательно, **косоугольной проекцией сферы является эллипс**, вытянутый вдоль ортогональной проекции $S' = SO_{\Gamma}$ вектора S (рис. 3.59, б). Центр эллипса находится в точке $o' = oK$, большая полуось длиной $a = r \sec(\lambda)$ параллельна вектору S' , а малая полуось длиной $b = r$ ортогональна ему. Окружность, создающая абрис сферы, имеет радиус r , лежит в плоскости $\{o, S\}$ и строится методами (3.61)–(3.64).

На рис. 3.60, а построена косоугольная **фронтальная** проекция модельного куба с вписанными в его грани окружностями. Параметры фронтального косоугольного проецирования вычисляются по (3.92)–(3.94). Приведём по аналогии формулы расчёта параметров других видов косоугольных проекций:

- **горизонтальной** (рис. 3.60, б):

$$K_h = \begin{bmatrix} 1 & 0 & 0 \\ -c_x & 0 & -c_z \\ 0 & 0 & 1 \end{bmatrix}, \text{ где } c_x = \frac{s_x}{s_y}, \quad c_z = \frac{s_z}{s_y}; \quad (3.95a)$$

$$m_x = m_z = 1, \quad m_y = \sqrt{c_x^2 + c_z^2} = \operatorname{tg}(\lambda_h); \quad (3.95б)$$

$$\gamma_{zx} = 90^\circ, \quad \operatorname{tg}(\gamma_{yz}) = -c_x/c_z = -s_x/s_z, \quad \operatorname{tg}(\gamma_{xy}) = -c_z/c_x = -s_z/s_x; \quad (3.95в)$$

- **профильной** (рис. 3.60, в):

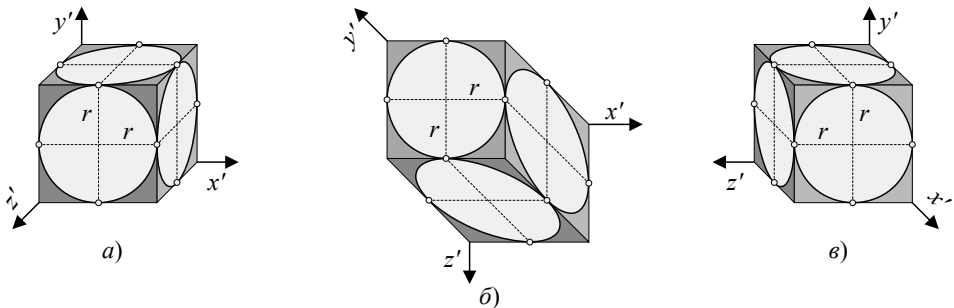


Рис. 3.60

$$K_p = \begin{bmatrix} 0 & -c_y & -c_z \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \text{ где } c_y = \frac{s_y}{s_x}, c_z = \frac{s_z}{s_x}; \quad (3.96a)$$

$$m_y = m_z = 1, m_x = \sqrt{c_y^2 + c_z^2} = \operatorname{tg}(\lambda_p); \quad (3.96б)$$

$$\gamma_{yz} = 90^\circ, \operatorname{tg}(\gamma_{zx}) = -c_y/c_z = -s_y/s_z, \operatorname{tg}(\gamma_{xy}) = -c_z/c_y = -s_z/s_y. \quad (3.96в)$$

Некоторые направления вектора S дают **стандартные косоугольные проекции**, применяемые в техническом черчении.

□ **Кабинетная проекция (косоугольная фронтальная диметрия)**, показанная на рис. 3.60,а, образуется при векторе направления источника $S = [1 \ 1 \ \sqrt{8}]$. Тогда по (3.92)–(3.94) определяем её параметры $m_x = m_y = 1, m_z = 0.5, \gamma_{xy} = 90^\circ, \gamma_{yz} = \gamma_{zx} = 135^\circ, \lambda_f = \operatorname{arctg}(0.5) \approx 26.6^\circ, c_x = c_y = 1/2\sqrt{2}$ и

$$K_{\text{каб}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1/2\sqrt{2} & -1/2\sqrt{2} & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0.354 & -0.354 & 0 \end{bmatrix}, S_{\text{каб}} = [1 \ 1 \ \sqrt{8}]. \quad (3.97)$$

Эта проекция наиболее наглядна и проста в построении. Часто кабинетной ошибочно называют косоугольную диметрию с вектором $S = [1 \ 1 \ 2]$, построенную на клеточной бумаге с соотношением масштабов $m_x : m_y : m_z = 1 : 1 : \sqrt{0.5}$, отличным от соотношения $1 : 1 : 0.5$ кабинетной проекции.

□ **Кавальерная** или военная проекция (**косоугольная фронтальная изометрия**), показанная на рис. 3.61, образуется при выборе вектора $S = [1 \ 1 \ \sqrt{2}]$. Тогда по (3.92)–(3.94) определяем параметры проекции $m_x = m_y = m_z = 1, \gamma_{xy} = 90^\circ, \gamma_{yz} = \gamma_{zx} = 135^\circ, \lambda_f = \operatorname{arctg}(1) = 45^\circ, c_x = c_y = 1/\sqrt{2}$ и

$$K_{\text{кав}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1/\sqrt{2} & -1/\sqrt{2} & 0 \end{bmatrix} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0.707 & -0.707 & 0 \end{bmatrix}, S_{\text{кав}} = [1 \ 1 \ \sqrt{2}]. \quad (3.98)$$

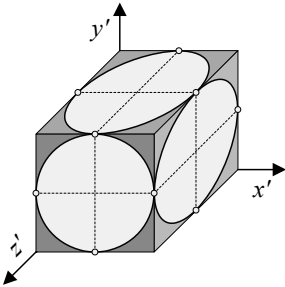


Рис. 3.61

В сравнении с кабинетной проекцией лучи в направлении $S_{\text{кав}}$ проходят положе, под большим углом λ_f к нормали картинной плоскости. Из-за этого кавальерная проекция объекта получается более вытянутой в z' -направлении и кажется неестественной. В то же время она удобна для измерения **неискажённых** длин отрезков, параллельных **всем** осям координат. Геометрическая точность и способность отображать объём объясняют использование в старину кавальерных проекций в фортификационном деле.

□ **Зенитная проекция (косоугольная горизонтальная изометрия)**, построенная на рис. 3.60,б, образуется при выборе вектора направления на проектор $S = [1 \ \sqrt{2} \ 1]$. Тогда по (3.92)–(3.94) находим её параметры

$$m_x=m_y=m_z=1, \gamma_{zx}=90^\circ, \gamma_{xy}=\gamma_{yz}=135^\circ, \lambda_h=45^\circ, c_x=c_z=1/\sqrt{2},$$

$$K_{\text{зен}} = \begin{bmatrix} 1 & 0 & 0 \\ -1/\sqrt{2} & 0 & -1/\sqrt{2} \\ 0 & 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 1 & 0 & 0 \\ -0.707 & 0 & -0.707 \\ 0 & 0 & 1 \end{bmatrix}, S_{\text{зен}} = \begin{bmatrix} 1 & \sqrt{2} & 1 \end{bmatrix}. \quad (3.99)$$

Данная проекция представляет наклонный горизонтальный вид на объект *сверху*. В аэрофотосъёмке используется **зенитная диметрия** — вид с высоты птичьего полета — с углом наблюдения $\lambda_h \approx 0^\circ$ и соотношениями масштабов $m_x:m_y:m_z=1:k_y:1$ и координат вектора наблюдателя $s_x^2+s_z^2=k_y^2 s_y^2$. Выбор малого коэффициента сжатия высоты $k_y=\text{tg}(\lambda_h) \ll 1$ позволяет изобразить на чертеже протяжённые ввысь объекты — горы, небоскрёбы, башни и т. д.

Если перед косоугольным проецированием подвергнуть объект вращению, то получим **косоугольную триметрию** — наиболее общий вид параллельного проецирования, в котором осевые масштабы связаны условием, обобщающим соотношение (3.82):

$$\boxed{m_x^2 + m_y^2 + m_z^2 = 2 + \text{tg}^2(\lambda)}. \quad (3.100)$$

3.2.4. Центральные (перспективные) проекции

Геометрически построение **центральной проекции (ЦП)** похоже на расчёт по (1.130а) тени точки q на плоскости $\{O, N\}$ от **ближнего** источника L при $U=q-L$ и условиях $U \circ N \neq 0$, $\theta > 0$. Решение является дробно-рациональной функцией декартовых координат точки $[x \ y \ z]$ и для получения матричной формы решения используются **однородные координаты** $[X \ Y \ Z \ \omega]$, определённые в (2.46). Приведение однородного вектора к декартовой форме по (2.47а)

$$x = \frac{X}{\omega}, \quad y = \frac{Y}{\omega}, \quad z = \frac{Z}{\omega} \quad (3.101a)$$

путём деления первых трёх однородных координат на четвёртую ω равносильно равномерному масштабированию. Определённая в (2.47б) эквивалентность множества однородных векторов одному расширенному вектору

$$[\omega x \ \omega y \ \omega z] \sim [x \ y \ z \ 1] \quad (3.101б)$$

означает равенство соответствующих декартовых координат при любых $\omega \neq 0$.

Получение центральной проекции

Рассмотрим **методику получения центральной проекции точки** $p=[x \ y \ z]$ на фронтальную картинную плоскость от **ближнего** проектора, находящегося в точке $S=[s_x \ s_y \ s_z]$ (рис. 3.62). Испущенный оттуда луч, проходящий через точку p в направлении вектора $V=p-S$, описывается параметрической функцией $S+(p-S)t \ \forall t \geq 0$. В отличие от параллельного проецирования центральные проекции имеются не у всех точек пространства, а лишь у тех, что

лежат от проектора по одну сторону с картинной плоскостью, т. е. удовлетворяют условиям $z < s_z$ при $s_z > 0$ либо $z > s_z$ при $s_z < 0$.

Координаты $p' = [x' \ y' \ 0]$ проекции точки p на ФКП находим из условия $z' = 0$ пересечения луча с плоскостью xy :

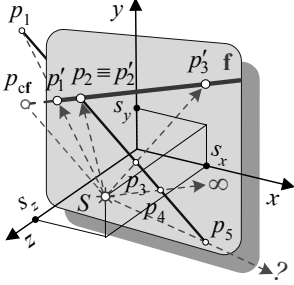


Рис. 3.62

$$z' = s_z + (z - s_z)t_f = 0 \Rightarrow t_f = \frac{s_z - z}{s_z - z} = \frac{1}{\omega}; \quad (3.102a)$$

$$\begin{cases} x' = s_x + (x - s_x)t_f = \frac{x - c_x z}{\omega}, \text{ где } c_x = \frac{s_x}{s_z}; \\ y' = s_y + (y - s_y)t_f = \frac{y - c_y z}{\omega}, \text{ где } c_y = \frac{s_y}{s_z}; \\ \omega = 1 - c_f z, \text{ где } c_f = \frac{1}{s_z}. \end{cases} \quad (3.102b)$$

Получаем уравнения проецирования в однородных координатах

$$\omega x' = x - c_x z, \quad \omega y' = y - c_y z, \quad \omega z' = 0, \quad \omega = 1 - c_f z \quad (3.103)$$

и эквивалентное декартово решение в координатной и векторной формах:

$$x' = \frac{x - c_x z}{1 - c_f z} = \frac{s_z x - s_x z}{s_z - z}, \quad y' = \frac{y - c_y z}{1 - c_f z} = \frac{s_z y - s_y z}{s_z - z}, \quad z' = 0; \quad (3.104a)$$

$$\tilde{p}' \sim \tilde{p} C_f, \text{ где } C_f = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -c_x & -c_y & 0 & -c_f \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (3.104b)$$

Это и есть **решение задачи центрального проецирования** точки на фронтальную плоскость, получаемое в два приема:

- расширенный вектор координат точки $\tilde{p} = [x \ y \ z \ 1]$ умножается на матрицу C_f для получения однородного вектора $\tilde{p} = [X \ Y \ Z \ \omega]$;

- однородный вектор приводится к декартовой форме $\tilde{p}' = [x' \ y' \ z' \ 1]$ делением (3.101a) на его четвёртую координату ω , которая не должна быть нулевой.

Тип полученного решения определяется значением ω . **Условие существования центральной проекции** $[t_f > 0]$ согласно (3.102a) равносильно условию $[\omega > 0]$. На рис. 3.62 ему удовлетворяют точки p_1 с $\omega_1 > 1$, p_2 с $\omega_2 = 1$ и p_3 с $0 < \omega_3 < 1$.

При $\omega \leq 0$ центральной проекции точки не существует. В частности, если луч $S \rightarrow p_4$ параллелен ФКП, то $z_4 = s_z$, $\omega_4 = 0$, а проекция p'_4 является **вырожденной**, т. е. удалённой в бесконечность: $x' = (x - s_x)/\omega \rightarrow \infty$ и $y' = (y - s_y)/\omega \rightarrow \infty$. Тем не менее **конечная** величина отношения $y'_4/x'_4 = (y_4 - s_y)/(x_4 - s_x)$ задаёт на ФКП направление на точку p'_4 . Наконец, если луч $S \rightarrow p_5$ после испускания удаляется от проективной плоскости, то на первом этапе получим $\omega_5 < 0$, а на втором p'_5 — **минимум** проекцию точки p_5 .

В связи с определяющим влиянием знака коэффициента ω на тип проекции точки назовём его **индикатором центрального проецирования**. Положительный индикатор сигнализирует о получении действительной, отрицательный — о мнимой, а нулевой — о вырожденной проекции точки.

При удалении проектора от картинной плоскости в бесконечность центральная проекция объекта вырождается в параллельную. Рассмотрим два варианта осуществления этого процесса:

- если проектор удаляется по **ортогональной** к КП прямой $S+z^0\lambda$, то при $\lambda \rightarrow \infty$ получим $c_x=c_y=c_f=0$, а в (3.104б) $\Pi_f = \tilde{O}_f$ становится матрицей ортогонального фронтального проецирования из (3.75);

- если проектор удаляется от КП по наклонной прямой $S\lambda$, то при $\lambda \rightarrow \infty$ получим $|c_x|=|s_x/s_z|<\infty$, $|c_y|=|s_y/s_z|<\infty$, $c_f=0$, а $\Pi_f = \tilde{K}_f$ превращается в расширенную матрицу косоугольного фронтального проецирования (3.92).

Очевидно сходство центрального проецирования точки q и расчёта её тени s на плоскости $\{o, N\}$ по (1.130а) (см. рис. 1.110): результат может быть действительным (при $\theta \geq 0$), мнимым (при $\theta < 0$) или вырожденным (при $\theta^{-1}=0$).

Есть и отличие этих процессов: точка, расположенная **за** плоскостью проецирования, имеет действительную проекцию, но не имеет действительной тени. Другим словами, **плоскость проецирования является прозрачной** (часто она вообще виртуальна, т. е. мысленно подразумевается, но в действительности не существует), а **плоскость тени реальная и непрозрачная**, иначе выходящие из источника лучи, обтекая объект, не создавали бы на ней освещённость вокруг тени — области подложки с пониженной или нулевой освещённостью.

Следует сказать, что матрица (3.104б) центрального проецирования на ФКП не единственна. Возьмём вместо $S+(p-S)t \ \forall t \geq 0$ другую модель проецирующего луча — $p+(p-S)t \ \forall t \geq -1$ — и, повторив рассуждения (3.102)–(3.103), получим следующие уравнения в однородных координатах:

$$\omega x' = s_z x - s_x z, \quad \omega y' = s_z y - s_y z, \quad \omega z' = 0, \quad \omega = s_z - z. \quad (3.105a)$$

Им соответствует матрица ЦП (сравните её с (3.104б))

$$\Pi_f = \begin{bmatrix} s_z & 0 & 0 & 0 \\ 0 & s_z & 0 & 0 \\ -s_x & -s_y & 0 & -1 \\ 0 & 0 & 0 & s_z \end{bmatrix}. \quad (3.105б)$$

То, что эквивалентное (3.105а) декартово решение

$$x' = \frac{s_z x - s_x z}{s_z - z}, \quad y' = \frac{s_z y - s_y z}{s_z - z}, \quad z' = 0 \quad (3.105в)$$

точно совпадает с (3.104а), доказывает **многообразие решений задачи центрального проецирования**, зависящих от выбора её элементов — проектора, картинной плоскости, модели проецирующего луча, уравнений в однородных координатах, матрицы проецирования.

Точки схода

Важнейшее **свойство центрального проецирования**, давшее ему второе название, — **перспективное** (от лат. *perspicio* — ясно вижу) **схождение проекций параллельных линий**. Изучим это интересное явление, благодаря которому плоские изображения тел выглядят на экране объёмно и реалистично.

Центральной проекцией отрезка прямой линии, например, p_1p_3 на рис. 3.62, также является отрезок $p'_1p'_3$ на прямой пересечения ФКП с плоскостью, проходящей через точки S , p_1 и p_3 . Подставим параметрическую модель прямой $p(t)=o+Vt$ в (3.104):

$$\begin{aligned} & [x(t) \ y(t) \ z(t) \ 1]U_f = \\ & = [o_x + V_x t - c_x(o_z + V_z t) \ o_y + V_y t - c_y(o_z + V_z t) \ 0 \ 1 - c_f(o_z + V_z t)] \sim \\ & \sim \left[\frac{o_x - c_x o_z + (V_x - c_x V_z)t}{1 - c_f o_z - c_f V_z t} \ \frac{o_y - c_y o_z + (V_y - c_y V_z)t}{1 - c_f o_z - c_f V_z t} \ 0 \ 1 \right] = [x'(t) \ y'(t) \ 0 \ 1]. \end{aligned}$$

В том, что полученные координаты проекции $x'(t)$ и $y'(t)$ описывают прямую линию, нетрудно убедиться, вычислив наклон этой линии:

$$\frac{dy'}{dx'} = \frac{dy'/dt}{dx'/dt} = \frac{(1 - c_f o_z)V_y - (c_y - c_f o_y)V_z}{(1 - c_f o_z)V_x - (c_x - c_f o_x)V_z} = \frac{(s_z - o_z)V_y - (s_y - o_y)V_z}{(s_z - o_z)V_x - (s_x - o_x)V_z} = \text{const} \ \forall t.$$

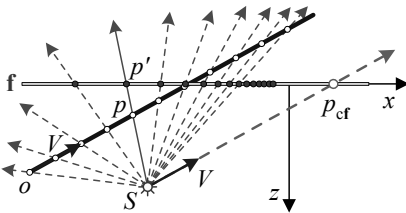


Рис. 3.63

Таким образом, прямая линия проецируется также в прямую линию, но с **неравномерной шкалой** (рис. 3.63, вид на ФКП сверху): при движении точки по прямой $p(t)$ с постоянной скоростью V возникает неравномерное движение её проекции $p'(t)$. Чем ближе направление проецирующего луча $p-S$ к вектору V , тем плотнее располагаются проекции точек, равномерно распределённых на прямой $o+Vt$.

Убедимся, что **предельное положение (точка схода) проекции прямой**

$$p_{cf} = \lim_{t \rightarrow \pm\infty} p'(t) = [s_x - (V_x/V_z)s_z \ s_y - (V_y/V_z)s_z \ 0] = S - (s_z/V_z)V \quad (3.106)$$

не зависит от расположения точки o , а определяется только координатами источника S и направляющего вектора V . Все параллельные прямые, **не параллельные плоскости проецирования**, имеют на перспективной проекции одну и ту же точку схода, получить которую достаточно просто (рис. 3.63): нужно провести из центра проецирования S прямую $S+Vt$ в направлении V до пересечения с ФКП при $t_f = -s_z/V_z$. Если эта прямая параллельна проективной плоскости ($V_z=0$), то точка схода удалена в бесконечность, т. е. отсутствует.

Из всего сказанного сделаем следующие **выводы**:

- центральные проекции пучка параллельных прямых сходятся в одну общую точку (3.106);
- **условие существования точки схода**: $V_z \neq 0$ — пучок параллельных прямых не должен быть параллелен плоскости проецирования;

• если $V_z=0$, то точка схода **вырождена**, т. е. удалена в бесконечность, а проекции параллельных прямых, параллельных картинной плоскости, параллельны и точки схода не имеют.

Аналогично (3.102)–(3.104) и (3.106) выводятся формулы для центрального проецирования на **горизонтальную**

$$C_h = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -c_x & 0 & -c_z & -c_h \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad c_x = \frac{s_x}{s_y}, \quad c_z = \frac{s_z}{s_y}, \quad c_h = \frac{1}{s_y}, \quad p_{ch} = S - \frac{s_y}{V_y} V \quad (3.107a)$$

и **профильную**

$$C_p = \begin{bmatrix} 0 & -c_y & -c_z & -c_p \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad c_y = \frac{s_y}{s_x}, \quad c_z = \frac{s_z}{s_x}, \quad c_p = \frac{1}{s_x}, \quad p_{cp} = S - \frac{s_x}{V_x} V \quad (3.107б)$$

картинные плоскости, позволяющие напрямую получить перспективные виды объекта с желаемых точек обзора.

Можно обойтись и одним фронтальным проецированием (3.104), поворачивая объект к проектору нужными гранями. Но если картинная поверхность состоит более чем из одной координатной плоскости, то лучше использовать совместно разные виды перспективных проекций для уменьшения числа геометрических преобразований и времени расчёта.

Центральные проекции **условно** классифицируются (см. рис. 3.49) по числу точек схода изображения **куба** или параллелепипеда — объектов, имеющих по три пучка параллельных рёбер. Например, проекции каркасных кубов на рис. 3.64 имеют одну (а), две (б) и три (в) точки схода по числу параллельных пучков, не параллельных проективной плоскости xy . Первый куб ориентирован своими гранями параллельно координатным плоскостям. Второй куб получен вращением первого вокруг его переднего левого ребра. Третий куб повернут относительно второго вокруг оси, параллельной оси x .

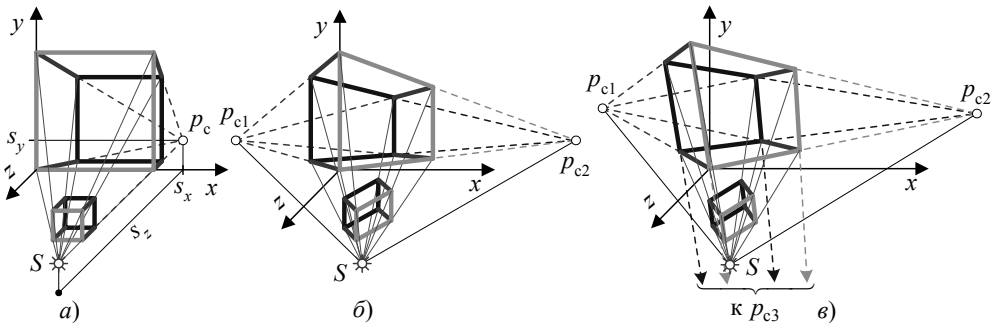


Рис. 3.64

Условность классификации ЦП по числу точек схода означает, что изображения объектов могут как вообще их не иметь, так и иметь более трёх таких точек. Примерами служат тетраэдр, у которого вообще нет параллельных рёбер, и октаэдр с шестью пучками таковых. Так что **строить центральную проекцию**

по заранее заданным точкам схода, как усердно навязывается многими учебниками по начертательной геометрии, принципиально неверно!

Центральная проекция отрезка

Существование и тип центральной проекции **отрезка** ab требует анализа значений индикаторов концевых точек $\omega_a = 1 - a_z/s_z$ и $\omega_b = 1 - b_z/s_z$ (табл. 3.3):

Таблица 3.3

$\begin{array}{c c} & b \\ \hline a & \end{array}$	$\omega_b \leq 0$	$\omega_b > 0$
$\omega_a \leq 0$	—	$b' \rightarrow 2b' - p_c$
$\omega_a > 0$	$a' \rightarrow 2a' - p_c$	$[a', b']$

• при $\{\omega_a > 0\} \wedge \{\omega_b > 0\}$ у отрезка ab есть **действительная** ЦП в виде **отрезка** $a'b'$;

• при $\{\omega_a \leq 0\} \wedge \{\omega_b \leq 0\}$ проекция отрезка отсутствует в силу **мнимости** проекций обоих его концов;

• в двух оставшихся случаях один из концов отрезка ab с направлением $V = b - a$ имеет действительную ЦП, а другой — мнимую.

В результате изображение отрезка имеет вид **луча**, начинающегося в точке действительной проекции (3.104) и уходящего в бесконечность через точку, лежащую **противоположно точке схода** $p_c = S - s_z V / V_z$, вычисляемой по (3.106):

◇ при $\{\omega_a > 0\} \wedge \{\omega_b \leq 0\}$ это луч $a' \rightarrow 2a' - p_c$, выходящий из точки $\tilde{a}' \sim \tilde{a}Ц_f$ в направлении $a' - p_c$ в бесконечность через точку $2a' - p_c$;

◇ при $\{\omega_a \leq 0\} \wedge \{\omega_b > 0\}$ это луч $b' \rightarrow 2b' - p_c$, выходящий из точки $\tilde{b}' \sim \tilde{b}Ц_f$ в направлении $b' - p_c$ в бесконечность через точку $2b' - p_c$.

В связи с возможностью получения проекции отрезка в виде полубесконечной прямой неизбежна задача её внешнего отсечения контуром ограниченного окна вывода сцены. Отсечению также должен подвергаться отрезок действительной проекции, целиком или частично выходящий за границы окна.

Функции отсечения отрезка $a'b'$ выпуклым и произвольным отсекателями O получены в п. 1.4.1 (рис. 1.78, 1.79). Для работы с лучом $a' \rightarrow b'$, начинающимся в точке a' и уходящим через точку b' в бесконечность, их нужно изменить так, чтобы в список L добавлялись только параметры $t > 0$. Рекомендуется ввести дополнительный аргумент $mod \in \{0, 1\}$ и совместить в модифицированных функциях $clip2_test(a'b', O, mod, Ls, >)$ и $clip2_segm(a'b', O, mod, Ls, >)$ оба варианта отсечения полигоном как отрезка (при $mod=0$), так и луча (при $mod=1$).

☞ **Пример 3.6.** Построить проекцию отрезка прямой ab на угол первого октанта, образованный пересечением координатных плоскостей xu и yz . Координаты концов отрезка $a = [0 \ 1 \ 6] \in yz$ и $b = [3 \ 2 \ 2] \in xu$, а положения проектора $S = [2 \ 1 \ 4]$ (рис. 3.65).

Решение. Рассчитаем по (3.104б) и (3.107б) матрицы центрального проецирования на фронтальную и профильную плоскости:

$$Ц_f = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -0.5 & -0.25 & 0 & -0.25 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad Ц_p = \begin{bmatrix} 0 & -0.5 & -2 & -0.5 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Вычислим однородные и декартовы координаты проекций концов отрезка на них:

$$\begin{bmatrix} a & 1 \\ b & 1 \end{bmatrix} U_f = \begin{bmatrix} 0 & 1 & 6 & 1 \\ 3 & 2 & 2 & 1 \end{bmatrix} U_f = \begin{bmatrix} -3 & -0.5 & 0 & -0.5 \\ 2 & 1.5 & 0 & 0.5 \end{bmatrix} \sim \begin{bmatrix} 6 & 1 & 0 & 1 \\ 4 & 3 & 0 & 1 \end{bmatrix} \equiv \begin{bmatrix} a'_f & 1 \\ b'_f & 1 \end{bmatrix},$$

$$\begin{bmatrix} a & 1 \\ b & 1 \end{bmatrix} U_p = \begin{bmatrix} 0 & 1 & 6 & 1 \\ 3 & 2 & 2 & 1 \end{bmatrix} U_p = \begin{bmatrix} 0 & 1 & 6 & 1 \\ 0 & 0.5 & -4 & -0.5 \end{bmatrix} \sim \begin{bmatrix} 0 & 1 & 6 & 1 \\ 0 & -1 & 8 & 1 \end{bmatrix} \equiv \begin{bmatrix} a'_p & 1 \\ b'_p & 1 \end{bmatrix}.$$

Из четырёх проекций только две $a'_p \equiv a$ и b'_f являются действительными. Проекция a'_f и b'_p — мнимые, лежащие на отрицательных продолжениях лучей $S \rightarrow a$ и $S \rightarrow b$, что подтверждается отрицательными индикаторами $\omega_{af} = -0.5 < 0$ и $\omega_{bp} = -0.5 < 0$.

Таким образом, у нас есть по одной точке проекции $a'_p \in yz$ и $b'_f \in xy$ на горизонтальной и фронтальной КП. Полная проекция отрезка ab на октант состоит из двух отрезков, для построения которых нужны ещё две точки — по одной на каждой плоскости проецирования.

Вычислив вектор направления отрезка $V = b - a = [3 \ 1 \ -4]$ и по (3.106) и (3.107б) точки схода на фронтальную и профильную плоскости $p_{cf} = [5 \ 2 \ 0]$ и $p_{cp} = [0 \ 1/3 \ 20/3]$ (их можно построить графически, проведя через точку S прямую, параллельную ab , до пересечения с плоскостями f и p), получим параметрические функции лучей

$$\begin{aligned} p'_f(t_f) &= b'_f + (b'_f - p_{cf})t_f = [4 \ 3 \ 0] + [-1 \ 1 \ 0]t_f \quad \forall t_f \geq 0, \\ p'_p(t_p) &= a'_p + (a'_p - p_{cp})t_p = [0 \ 1 \ 6] + [0 \ 2/3 \ -2/3]t_p \quad \forall t_p \geq 0. \end{aligned} \quad (3.108)$$

При $t_f = 4$ и $t_p = 9$ лучи пересекаются в точке $c = [0 \ 7 \ 0]$ на оси y , обеспечивая переход проекции отрезка с одной проективной плоскости на другую.

Используя мнимые проекции $b'_f \in yz$ и $a'_f \in xy$, можно записать альтернативные (3.108) параметрические функции лучей

$$\begin{aligned} p'_f(t_f) &= b'_f + (b'_f - a'_f)t_f = [4 \ 3 \ 0] + [-2 \ 2 \ 0]t_f \quad \forall t_f \geq 0, \\ p'_p(t_p) &= a'_p + (a'_p - b'_p)t_p = [0 \ 1 \ 6] + [0 \ 2 \ -2]t_p \quad \forall t_p \geq 0. \end{aligned}$$

пересекающихся при $t_f = 2$ и $t_p = 3$ в той же точке $c = [0 \ 7 \ 0]$.

Так или иначе, центральной проекцией отрезка ab является ломаная линия acb . Однако, если не проверять индикаторами концевые точки отрезка на мнимость проекций, будет получено неверное решение, состоящее из двух отрезков ab'_p и ba'_f . $\square$

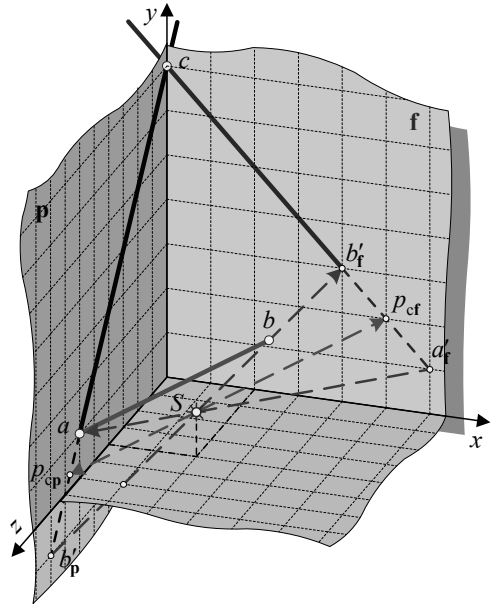


Рис. 3.65

Центральная проекция полигона

Принципиальное отличие центрального проецирования **полигона** $P = p_1 p_2 \dots p_n p_1$ от параллельного заключается в том, что если не все вершины p_i имеют действительные проекции с параметрами $\omega_i > 0$ из-за того, что находятся от картинной плоскости дальше проектора, как, например, точка p_3 на

рис. 3.66,а, то в результате образуется **незамкнутая** и **неограниченная** проекция P' (рис. 3.66,б), не являющаяся полигоном, т. е. замкнутой полилинией.

Применим для решения задачи отсечения объекта P' **выпуклым** полигональным окном O (а проекции в подавляющем большинстве случаев строятся именно в выпуклых, более того — в прямоугольных, окнах!) функцию *clip_line*, описанную в п. 1.4.3 (см. рис. 1.85,а). Поскольку она работает с **неограниченным** отсекателем — полуплоскостью, лежащей слева от отсекающей прямой, то по свойству коммутативности операции внешнего отсечения назовём отсекаемым объектом полигон O , а граничными линиями полуплоскостей — прямые, несущие действительные проекции рёбер полигона P .

В табл. 3.4 приведены все варианты выбора параметров отсекающей прямой $\{o, V\} \in \mathbf{f}$ в зависимости от вычисленных по (3.103) индикаторов ω_a и ω_b

Таблица 3.4

$\begin{matrix} a & b \end{matrix}$	$\omega_b \leq 0$	$\omega_b > 0$
$\omega_a \leq 0$	—	$\{b', p_c - b'\}$
$\omega_a > 0$	$\{a', a' - p_c\}$	$\{a', b' - a'\}$

концевых точек отрезка ab . По этой таблице выбираются аргументы $\{o, V\}$ функции *clip_line*(O, o, V), отсекающей полигон O линией действительной проекции ребра $p_i p_{i+1}$ **выпуклого** полигона P . Если ни у одной точки p_i нет действительной проекции, т. е. все значения $\omega_i \leq 0$, то проекция P' отсутствует.

Как узнать направление обхода фигуры P' , определяющее ориентацию отсекающих полуплоскостей относительно их граничных прямых? Ввиду возможной незамкнутости полилинии P' применение к ней теста *dir test* невозможно. К счастью, задача разрешима с помощью значения $d = nf3(p_1, p_2, p_3, S)$ НФ плоскости $\{p_1, p_2, p_3\}$ в точке $S = [s_x \ s_y \ s_z]$, определяющего взаимную ориентацию проектора, плоскости полигона и картинной плоскости:

- при $d=0$ проектор лежит в плоскости полигона P , а проекция P' вырождена в не полигональную фигуру нулевой площади (отрезок, луч или прямую);
- при $d>0$ точка S лежит с внешней стороны полигона и создаёт положительное направление движения по линиям проекции P' ;
- при $d<0$ проектор лежит с внутренней стороны плоскости полигона и создаёт отрицательное направление прохождения линий проекции P' .

Всё сказанное верно при $s_z > 0$, а если $s_z < 0$, то направления проекций рёбер полигона P инвертируются. Поэтому в аргументе $d \cdot V$ функции *clip_line* нужно использовать скорректированное значение $d = s_z \cdot nf3(p_1, p_2, p_3, S) \neq 0$.

Оформим эти идеи в **алгоритме** *clip_pro*(P, O, S) **отсечения центральной проекции выпуклого полигона** $P = p_1 \dots p_n p_1 \in \mathbf{R}^3$ **выпуклым окном** $O \in \mathbf{f}$, **возвращающем** часть проекции $P' \subset O$ внутри окна O либо пустое решение $\emptyset$.

Шаг 1. Находим значение $d = s_z \cdot nf3(p_1, p_2, p_3, S)$ произведения неявной функции плоскости $\{p_1, p_2, p_3\}$ в точке S на координату s_z . Если $d=0$, то проектор лежит в плоскости полигона P либо в КП, проекция полигона вырождена в прямую либо точку, а алгоритм **возвращает** пустое решение $O = \emptyset$.

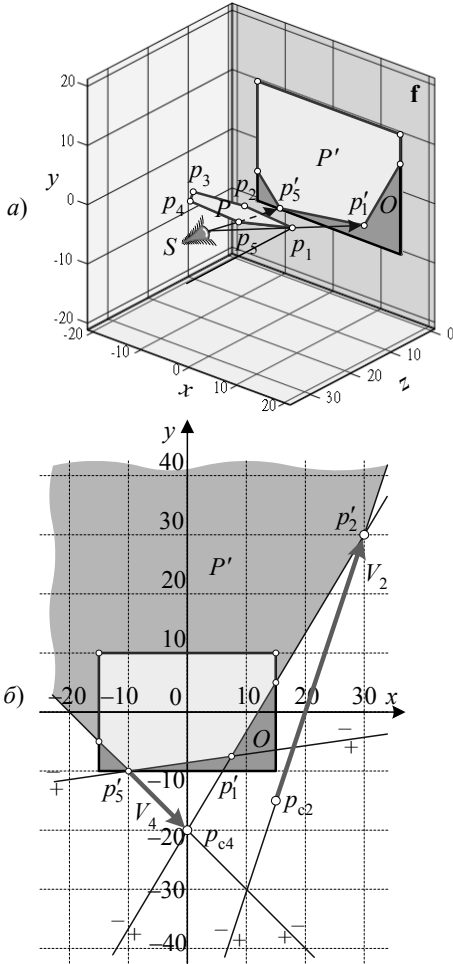


Рис. 3.66

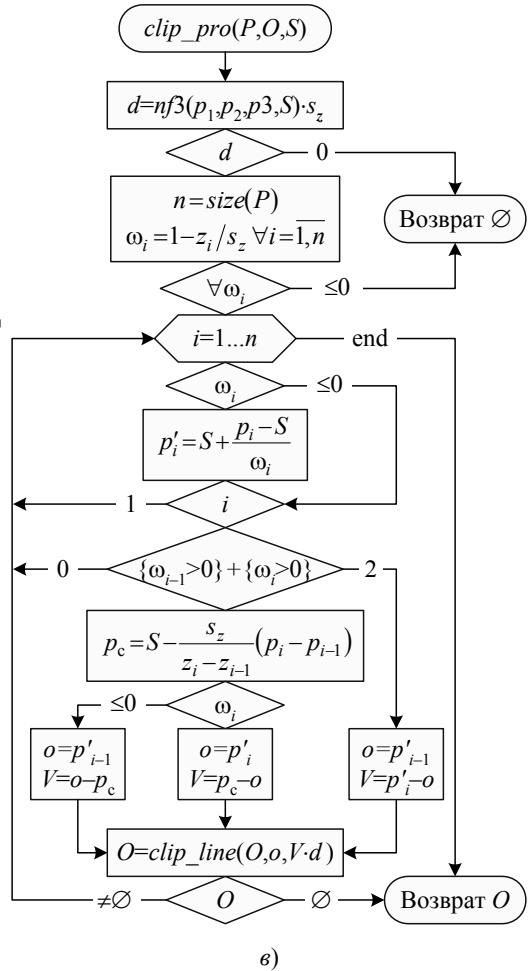
Шаг 2. По (3.103) вычисляем значения индикаторов $\omega_i = 1 - z_i/s_z$ в каждой вершине полигона $p_i = [x_i \ y_i \ z_i] \ \forall i = \overline{1, n+1}$. Если все $\omega_i \leq 0$, то нет ни одной действительной проекции вершины и **возвращается** пустое решение $O = \emptyset$.

Следующие **шаги 3÷5** выполняются в цикле $i = \overline{2, size(P)}$ для каждого отрезка $p_{i-1}p_i$.

Шаг 3. Находим действительные проекции концевых точек отрезка $p'_{i-1} = S + (p_{i-1} - S)/\omega_{i-1}$ при $\omega_{i-1} > 0$ и $p'_i = S + (p_i - S)/\omega_i$ при $\omega_i > 0$.

Шаг 4. При выполнении условия $\{\omega_{i-1} > 0\} \wedge \{\omega_i \leq 0\} \vee \{\omega_{i-1} \leq 0\} \wedge \{\omega_i > 0\}$ одна из точек $\{p_{i-1}, p_i\}$ имеет действительную, а другая — мнимую проекцию. По (3.106) вычисляем точку схода ребра $p_c = S - (p_i - p_{i-1})s_z/(z_i - z_{i-1})$.

Шаг 5. При $\{\omega_{i-1} > 0\} \vee \{\omega_i > 0\}$ выбираем по табл. 3.4 параметры отсекающей прямой — точку o и направление V , после чего производим отсечение по-



лигона O полуплоскостью: справа от граничной прямой $\{o, V\}$ при $d > 0$ и слева при $d < 0$. Получив пустой результат отсечения, алгоритм **возвращает** пустое решение $O = \emptyset$.

Шаг 6. По завершении цикла обхода рёбер **возвращается** полигон O .



Во избежание двойного расчёта проекций вершин полигона P и точек схода p_{ci} в алгоритме организован цикл обхода не рёбер $p_{i-1}p_i$ а вершин p_i .

Пример 3.7. Наблюдатель находится в точке $S = [0 \ 0 \ F]$ на фокусном расстоянии $F = 30 > 0$ от фронтальной плоскости проецирования. Построить перспективное изображение плоского пятиугольника P с вершинами $p_1 = [4 \ -4 \ 14]$, $p_2 = [4 \ 4 \ 26]$, $p_3 = [0 \ 8 \ 34]$, $p_4 = [-4 \ 4 \ 30]$ и $p_5 = [-4 \ -4 \ 18]$ внутри прямоугольного фронтального окна O с габаритами $-15 \leq x \leq 15$ и $-10 \leq y \leq 10$ (рис. 3.66,а).

Решение. Вычислим по (3.103) индикаторы пяти точек $\omega_i = 1 - z_i/F$:

$$\boxed{\omega_1 = 0.533 > 0}, \quad \boxed{\omega_2 = 0.133 > 0}, \quad \omega_3 = -0.133 < 0, \quad \omega_4 = 0, \quad \boxed{\omega_5 = 0.4 > 0}.$$

Анализируя их знаки, заключаем, что вершины полигона p_1 , p_2 и p_5 имеют действительные проекции $p'_1 = [7.5 \ -7.5 \ 0]$, $p'_2 = [30 \ 30 \ 0]$ и $p'_5 = [-10 \ -10 \ 0]$, проекция вершины p_3 мнимая, а точки p_4 вырождена. Следовательно, лишь рёбра p_1p_2 и p_5p_1 образуют на плоскости xy действительные проекции в виде отрезков $p'_1p'_2$ и $p'_5p'_1$. Проекция рёбер p_2p_3 и p_4p_5 , у которых $\omega_2\omega_3 < 0$ и $\{\omega_4 = 0\} \wedge \{\omega_5 > 0\}$, представляют собой лучи, направления которых V_2 и V_4 вычислим по табл. 3.4 с помощью точек схода (3.106):

$$p_{c2} = S - F(p_3 - p_2)/(z_3 - z_2) = [15 \ -15 \ 0] \Rightarrow V_2 = p'_2 - p_{c2} = [15 \ 45 \ 0];$$

$$p_{c4} = S - F(p_5 - p_4)/(z_5 - z_4) = [0 \ -20 \ 0] \Rightarrow V_4 = p_{c4} - p'_5 = [10 \ -10 \ 0].$$

Оставшееся ребро p_3p_4 , у которого $\{\omega_3 < 0\} \wedge \{\omega_4 = 0\}$, не проецируется на фронтальную плоскость ни в каком виде (рис. 3.66,б), т. к. находится сзади проектора.

Таким образом, центральной проекцией замкнутого полигона $p_1p_2p_3p_4p_5p_1$ на плоскость xy является фигура $p'_1p'_2 \rightarrow \infty \dots \infty \rightarrow p'_5p'_1$ с **незамкнутой** границей в виде двух отрезков $p'_1p'_2$, $p'_5p'_1$ и двух лучей $\{p'_2, V_2\}$, $\{p'_5, V_4\}$.

Удалённость части границы в бесконечность не позволяет считать эту фигуру полигоном и определить направление её обхода. Вычислив значение $nf3(p_1, p_2, p_3, S) = 256 > 0$, заключаем, что плоскость полигона лежит между проектором и картинной плоскостью, поэтому будем выполнять отсечение полигона O **справа** от граничных линий $\{o_i, V_i\}$.

Начинаем обход рёбер полигона P и отсечение окна O полуплоскостями линий, несущих проекции рёбер:

- первая же прямая $\{p'_1, p'_2 - p'_1\}$ отсекает от O правый нижний угол;
- вторая прямая $\{p'_2, V_2\}$ не изменяет окна, полностью лежащего слева от неё;
- третье ребро p_3p_4 не создаёт отсекающей полуплоскости;
- прямая $\{p'_5, V_4\}$ отсекает от окна левый нижний угол;
- наконец, последняя прямая $\{p'_5, p'_1 - p'_5\}$ отсекает от остатка окна нижний треугольник.

В итоге центральная проекция полигона P на ограниченное плоское окно O представляет собой шестиугольный полигон, занимающий большую часть окна.



Перспективные проекции окружности и сферы

Центральная проекция пространственной **окружности** $\{o, N, r\}$ вычисляется по общей формуле точечного преобразования $\tilde{p}'(t) \sim \tilde{p}(t) \Pi_{\Gamma}$ объектных координат $p(t) = c_{xy}(t)A(N) + o$. Аналитическое обоснование формы линии $p'(t)$, как это сделано выше для проекции прямой линии, достаточно сложно, даже если допустить, что у всех точек окружности индикаторы положительны: $\omega(t) > 0$. Однако, из геометрических соображений ясно, что эта линия является сечением картинной плоскостью эллиптического цилиндра с вершиной в точке S , образующие линии которого проходят через линию $p(t)$. Как доказано в п. 1.5.9, сечениями квадратичных поверхностей плоскостью являются **квадратичные линии**, из которых замкнут только эллипс. Следовательно, **замкнутой перспективной проекцией окружности является эллипс**, вписанный в общего вида четырёхугольник перспективной проекции квадрата, охватывающего окружность (рис. 3.67, а). Центр эллипса **не совпадает** с точкой $\tilde{o}' \sim \tilde{o} \Pi_{\Gamma}$.

В частном случае параллельности плоскости окружности и, например, ФКП, все точки $p(t)$ имеют одинаковые координаты $z(t) = z_0$. Тогда изменение других координат $x'(t) = \frac{x(t) - c_x z_0}{1 - c_f z_0}$ и $y'(t) = \frac{y(t) - c_y z_0}{1 - c_f z_0}$ равносильно переносу ортографической проекции окружности $[x(t) \ y(t) \ 0] \in \mathbf{f}$ на вектор $[-c_x z_0 \ -c_y z_0]$ и равномерному масштабированию с коэффициентами $m_x = m_y = (1 - c_f z_0)^{-1} > 0$. Таким образом, **окружности в плоскостях, параллельных картинной плоскости, проецируются в сдвинутые увеличенные окружности**. Здесь явно проявляется отличие **бокового** видения мира фотокамерой и глазом:

- в неподвижной камере изображение проецируется на плоскую поверхность плёнки или матрицы светочувствительных рецепторов с одинаковой чувствительностью как в центре кадра, так и на его краях, так что если плоскость окружности параллельна плоскости плёнки (матрицы), то на снимке она будет выглядеть как **окружность**;

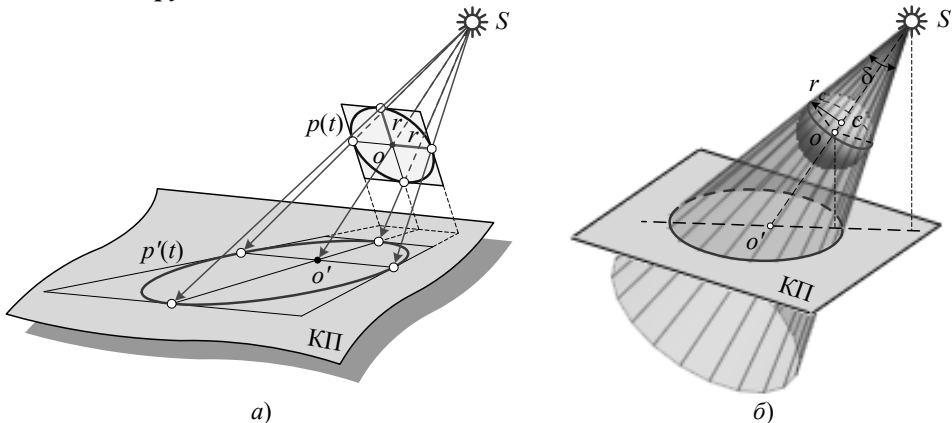


Рис. 3.67

• для большей чёткости изображения глаз инстинктивно поворачивается, иногда вместе с головой, в сторону объекта так, чтобы видеть его «в фас», а луч зрения был ортогонален центральному участку сетчатки. В результате неподвижная плоскость оказывается видимой глазом под косым углом, а окружность на ней — в виде *эллипса*. Если же усилием воли не поворачивать глаз в сторону объекта, то последний будет виден расплывчато из-за малой плотности и чувствительности рецепторов зоны периферийного зрения сетчатки.

При удалении окружности от картинной плоскости эллипс $p'(t)$ вытягивается и в момент достижения высшей точкой окружности высоты проектора S вырождается в *параболу*. Дальнейший подъём окружности превращает её проекцию в *гиперболу* вплоть до полного исчезновения, когда все точки $p(t)$ оказываются выше проектора и имеют отрицательные индикаторы $\omega(t) < 0$ как признак мнимости центральной проекции.

Проекция *сферы* $\{o, r\}$ на картинной плоскости образуется в сечении этой плоскостью круглого проективного конуса с углом раствора δ (рис. 3.67, б). **Замкнутой перспективной проекцией сферы является эллипс.** В отличие от параллельной проекции центр этого эллипса *не находится* в точке $\delta' \sim \delta \Pi_{\Gamma}$. При удалении сферы от картинной плоскости эллипс вытягивается вдоль ортогональной проекции луча $o-S$ и при достижении высшей точкой сферы высоты проектора S вырождается в неограниченную *параболу*. Дальнейший подъём сферы делает форму абриса *гиперболической* вплоть до полного исчезновения, когда проекции всех точек сферы становятся мнимыми.

Плоскость окружности $\{c, r_c\}$, создающей абрис сферы, *не проходит* через её центр o , а несколько смещена в сторону проектора S . Несложные геометрические построения, аналогичные рис. 1.53, дают следующие формулы расчёта угла δ , радиуса r_c и точки c в зависимости от расстояния $d = |S-o|$:

$$\delta = 2\arcsin(r/d), \quad r_c = r\sqrt{d^2 - r^2}/d, \quad c = o - r^2(S-o)/d^2. \quad (3.109)$$

Обратная перспектива

Самым парадоксальным видом центрального проецирования является так называемая «*обратная перспектива*». Построение изображений объёмных тел в этом ракурсе широко применялось в византийской и унаследовавшей её традиции древнерусской живописи. Яркими представителями этого необычного и противоречивого стиля являются знаменитая икона Андрея Рублева «Троица» XV в. (рис. 3.68, а) и миниатюра Дионисия из Евангелия XVI в. (рис. 3.68, б).

Визуально обратная перспектива отличается от прямой тем, что в ней из двух одинаковых предметов крупнее выглядит тот, что расположен от наблюдателя *дальше*, а мельче тот, что ближе. Это придаёт изображениям протяжённых в глубину фигур некоторую ирреальность, неестественную потусторонность, будто рукой художника водила некая высшая сила, находящаяся за пределами нашего мира. Сущность иконы как взгляда на наш мир «с той стороны» теологически объяснил русский ученый и богослов П. А. Флоренский. Психофизио-



Рис. 3.68

логическое обоснование обратной перспективы дано советским академиком Б. В. Раушенбахом в фундаментальном труде «Системы перспективы в изобразительном искусстве. Общая теория перспективы» (1986). Любопытному читателю рекомендуется научно-популярная книга [11], содержащая познавательный материал по истории взаимодействия и взаимообогащения математики и искусства (музыки, архитектуры, живописи) с античных времён до наших дней.

Рассмотрим геометрическую модель метода обратной перспективы и построение с помощью этой модели изображения объекта. Желаемый эффект достигается **отделением** активного процесса формирования проекции от пассивного процесса её созерцания. В соответствии с названием метода разместим наблюдателя и проектор S по **противоположные** стороны от картинной плоскости (рис. 3.69,а), а именно:

- направление на дальнего наблюдателя зададим ортом z^0 ;
- положение проектора $S = [0 \ 0 \ -F]$ на фокусном расстоянии F от ФКП зададим так, что минимальная аппликата объекта $z_{\min} > -F$. При этом условии объект целиком располагается между наблюдателем и проектором, так что вся его центральная проекция является действительной.

Таким образом, расчёт обратной перспективы точки $p = [x \ y \ z]$ выполняется по (3.104) с матрицей и координатами проекции

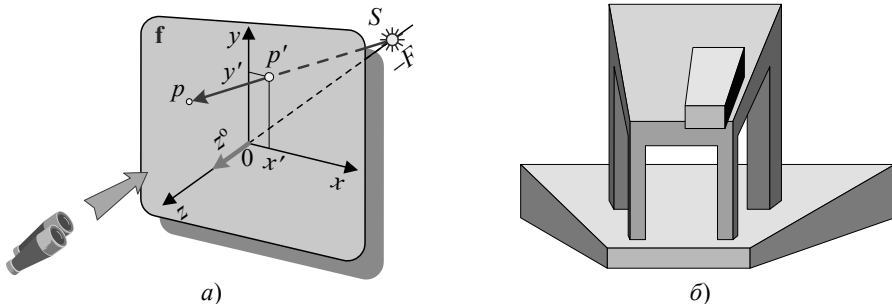


Рис. 3.69

$$C_f = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1/F \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad x' = \frac{Fx}{F+z}, \quad y' = \frac{Fy}{F+z}. \quad (3.110)$$

На рис. 3.69,б построена рассчитанная по (3.110) обратная перспективная проекция сцены, состоящей из квадратного подиума и стола с квадратной крышкой, на которой лежит, немного свесившись вперед, книга в форме параллелепипеда. Правильная визуализация граней непрозрачных тел достигнута путём вывода окрашенных полигонов их проекций в порядке приближения их центров к наблюдателю, находящемуся в направлении $[0 \ 0 \ 1]$. Каждая боковая невыпуклая грань поверхности стола заменялась объединением трёх прямоугольников одинакового цвета.

3.2.5. Проективные предискажения

Нами подробно изучены три механизма проецирования пространственных графических объектов на плоскую проективную поверхность — ортогональный, косоугольный и центральный, различающиеся направлением проецирующих лучей и удалённостью проектора. Претендующая на универсальность графическая программа визуализации 3d-сцен должна уметь изображать их во всевозможных ракурсах, рассчитывая в каждом случае необходимую проективную матрицу. Однако, в целях унификации можно произвести предварительное искажение (*предискажение*) объектов сцены и свести все виды плоского проецирования к одному простейшему — *ортографическому*, не требующему никаких вычислений и напрямую использующему две координаты точки $p = [x \ y \ z]$ для получения её фронтальной проекции $p' = [x \ y \ 0]$.

Другое важное преимущество ортогонального проецирования в сравнении с косоугольным и центральным состоит в простоте зоны видимости, которая имеет вид прямой прямоугольной призмы без оснований (см. рис. 1.151,а). Такая форма существенно упрощает решение важнейшей геометрической задачи визуализации — отсечения элементов сцены, не попавших в отображаемые пределы проективной плоскости. Всё определение попадания точки $p = [x \ y \ z]$ в зону видимости $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ сводится к простой проверке неравенств $x_{\min} \leq x \leq x_{\max}$ и $y_{\min} \leq y \leq y_{\max}$. Также упрощается расчёт пересечений произвольных прямых и плоскостей со стенками отсекающей области, описываемыми явными уравнениями $x = x_{\min}$, $x = x_{\max}$, $y = y_{\min}$ и $y = y_{\max}$.

Третий довод в пользу ортогонального проецирования — простота определения удалённости объектов сцены от проектора (наблюдателя), сводящегося к сравнению z -координат точек, имеющих одинаковые координаты x и y .

Всё сказанное ставит целью убедить читателя в том, что использование более сложных форм зоны видимости — косоугольной призмы либо пирамиды — ведёт к существенному увеличению трудоёмкости решения основных геометрических задач визуализации — отсечения и удаления.

□ Представим матрицу **косоугольного** фронтального проецирования (3.92) произведением матриц пространственного сдвига из (3.33б) и ортогографического проецирования из (3.75):

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -c_x & -c_y & 0 \end{bmatrix}}_{K_f} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -c_x & -c_y & 1 \end{bmatrix}}_{S_3} \cdot \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{O_f}. \quad (3.111)$$

Тогда операцию косоугольного проецирования точки $p' = pK_f$ можно заменить двумя преобразованиями (рис. 3.70,а):

• **сдвигом** на полученный в (3.13а) вектор $d = [-c_x z \ -c_y z \ 0] = -[s_x \ s_y \ 0]z/s_z$ в предыдущее состояние $p_n = pS_3 = p + d = [x' \ y' \ z]$ с координатами $x' = x - c_x z = x - z s_x / s_z$ и $y' = y - c_y z = y - z s_y / s_z$;

• **ортогографическим проецированием** точки p_n в точку $p' = p_n O_f = [x' \ y' \ 0]$.

В результате преобразования сдвига точки с разными знаками z -координат смещаются в противоположных направлениях, а точки, лежащие в проецивной плоскости, остаются в покое и проецируются сами в себя.

Преобразование сдвига $P_n = PS_3$, применённое к каждой точке полиэдра с матрицей вершин P , искажает форму объекта, сохраняя, однако, в силу аффинности, прямолинейность его рёбер, что позволяет строить ортогографическую проекцию полиэдра только по его вершинам (рис. 3.70,б). Между сдвинутыми объектами сцены также сохраняются отношения близости и видимости относительно бесконечно удалённого, теперь уже ортогонального, проектора. Это гарантирует правильное решение задачи удаления невидимых элементов.

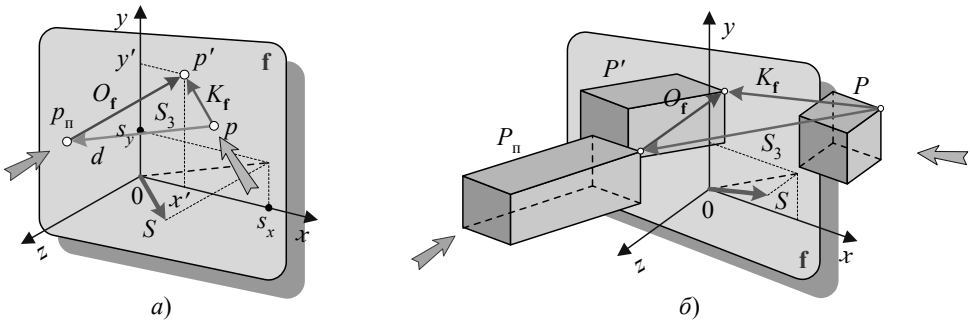


Рис. 3.70

□ Аналогично (3.111) представим матрицу **центрального** фронтального проецирования из (3.104б) произведением матриц сдвига $\tilde{S}_3$, перспективного масштабирования C_n и ортогографического проецирования $\tilde{O}_f$:

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -c_x & -c_y & 0 & -c_f \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathcal{U}_f} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -c_x & -c_y & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\tilde{S}_3} \cdot \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -c_f \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{C_n} \cdot \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\tilde{O}_f}. \quad (3.112)$$

Геометрически **перспективное предыскажение** точки $p = [x \ y \ z]$ и завершающее его проецирование проводятся по следующей схеме (рис. 3.71,а):

- **сдвиг** в направлении $d = [-c_x z \ -c_y z \ 0] = -[s_x \ s_y \ 0]z/s_z$ переводит точку в положение $p_c = pS_3 = p + d = [x_c \ y_c \ z]$ с координатами $x_c = x - c_x z$ и $y_c = y - c_y z$;

- **перспективное масштабирование** $\tilde{p}_c C_{\Pi} = [x_c \ y_c \ z \ \omega]$ с помощью индикатора декартова приведения $\omega = 1 - c_f z = (s_z - z)/s_z$, сдвигающего при $\omega \neq 0$ точку p_c вдоль её радиус-вектора в предыскажённое состояние $p_{\Pi} = \omega^{-1} p_c$ с координатами $x_{\Pi} = x_c/\omega$, $y_{\Pi} = y_c/\omega$ и $z_{\Pi} = z/\omega$;

- **ортографическое проецирование** точки p_{Π} на фронтальную плоскость в точку $p' = p_{\Pi} O_f = [x_{\Pi} \ y_{\Pi} \ 0] \equiv [x' \ y' \ 0]$.

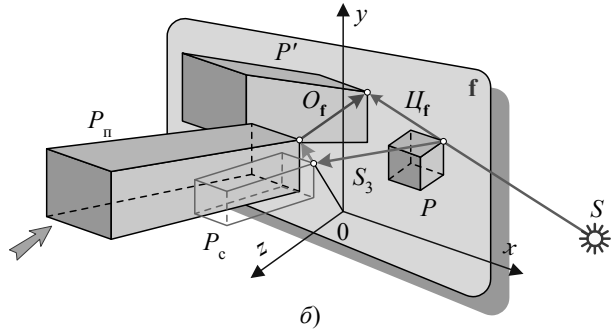
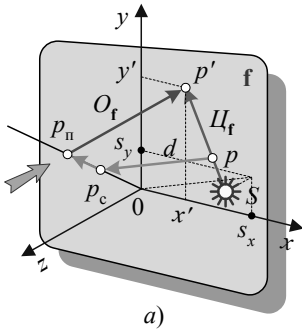


Рис. 3.71

Таким образом, перспективное предыскажение точки p выполняется по формуле $\tilde{p}_{\Pi} \sim \tilde{p} \tilde{S}_3 C_{\Pi}$ с **обязательным декартовым приведением по (3.101)**. В зависимости от значения индикатора ω возможно получение на радиус-векторе точки p_c или его продолжениях как действительной ($\omega > 0$), так и мнимой ($\omega < 0$) или вырожденной ($\omega = 0$) точки p_{Π} . В последнем случае при $z = s_z$ предыскажённая точка p_{Π} удалена в бесконечность.

Операция группового предыскажения $\tilde{P}_{\Pi} \sim \tilde{P} \tilde{S}_3 C_{\Pi}$, проведённая с матрицей точек объекта или группы объектов P , создаёт виртуальный объект (объекты) с матрицей точек P_{Π} , действительных только при всех индикаторах $\omega_i > 0$ (рис. 3.71,б). Однако, у объекта, пересекаемого воображаемой плоскостью $z = s_z > 0$, лишь часть точек p_i с координатами $z_i < s_z$ имеет действительные предыскажения $p_{\Pi i}$ и ортогональные проекции $p'_{\Pi i}$, тогда как оставшаяся часть объекта с мнимым предыскажением не должна проецироваться во избежание неверного изображения такого объекта на экране.

Сравнение (3.111) с (3.112) показывает, что **и для косоугольного, и для перспективного проецирования применима единая формула предыскажения** $\tilde{p}_{\Pi} \sim \tilde{p} \tilde{S}_3 C_{\Pi}$ с матрицей C_{Π} , в которой вместо $-c_f$ подставлено выражение $-nearH/s_z$, равное нулю при косоугольном и $-1/s_z$ при центральном проецировании, и признаком близости наблюдателя $nearH \in \{0, 1\}$ (см. п. 1.7.2).

3.3. ПРОЕКТИВНЫЕ АЛГОРИТМЫ СЛОЖНЫХ ПРЕОБРАЗОВАНИЙ

Проективные преобразования кроме своего непосредственного назначения — изображения сцены в желаемом ракурсе — могут применяться для быстрого построения:

- фигур и изображений на плоских и криволинейных поверхностях;
- сцен от первого лица, видимых подвижным наблюдателем;
- простейших оптических эффектов.

Основное преимущество проективных алгоритмов — возможность *группового преобразования точек* по (3.11), т. е. *параллельных* вычислений по одним и тем же формулам для всего множества точек. Такой подход к организации вычислительного процесса составляет серьезную конкуренцию алгоритмам, в которых проводятся *индивидуальные* лучи через каждую формообразующую точку каждого объекта. Процесс трассировки более трудоёмок в сравнении с групповым преобразованием и имеет преимущество лишь при малом числе точек и высокой сложности их преобразования.

Для выполнения групповых преобразований нужно сформировать *матрицу групповых координат* Q . Её строками являются *расширенные* формы:

- точек $\tilde{p}_i = [p_i \ 1]$ — центров объектов, концов отрезков, вершин полигонов;
- векторов $\tilde{V}_i = [V_i \ 0]$ — скоростей движений и направлений осей вращений

объектов. **Векторы нормалей плоскостей в матрицу Q не включаются!**

При групповой организации координат всех объектов сцены в одной матрице общее преобразование сцены матрицей C выполняется за *одно* умножение: $Q' = QC$. Если требуется индивидуально преобразовать m -й объект сцены матрицей $C^{<m>}$, то целесообразно сгруппировать строки вершин и векторов этого объекта в блок $Q^{<m>}$, тогда $Q'^{<m>} = Q^{<m>} C^{<m>}$.

Построение изображения 3d-объектов на плоском экране дисплея может быть выполнено двумя способами (рис. 3.72).

□ **Двухэтапное преобразование** координат точек:

- из МСК в СК ФКП с помощью C_{Π} — одной из проективных матриц $\tilde{O}_f$, $\tilde{A}_f$, $\tilde{K}_f$ или C_f из разд. 3.2:

$$Q_f \sim Q C_{\Pi}; \quad (3.113a)$$

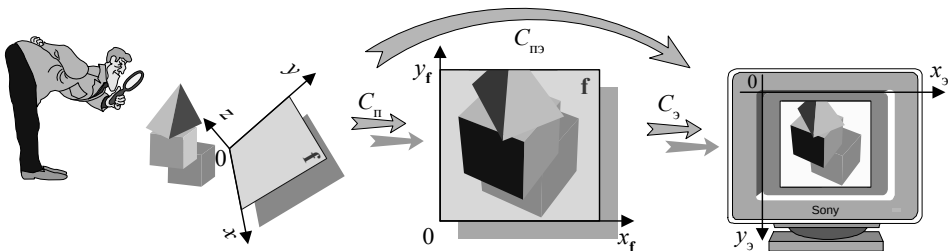


Рис. 3.72

- из СК КП в ЭСК с помощью матрицы C_3 из (3.41) или (3.42г,е):

$$Q_3 = Q_1 C_3. \quad (3.113б)$$

Двухэтапный метод применяется при построении изображения сложной сцены, состоящей из множества объектов. Между первым и вторым этапами решаются задачи *визуализации изображения*:

- *отсечения* графических элементов, не попавших на КП;
- *удаления* невидимых элементов, заслонённых другими элементами;
- *освещённости* видимых поверхностей;
- расчёта различных *оптических эффектов* — отражений, преломлений, теней и т. д.

При использовании центрального проецирования после первого этапа выполняется приведение координат точек к декартовой форме.

□ **Одноэтапное преобразование** координат точек из МСК сразу в ЭСК:

$$Q_3 \sim Q C_{пз}, \text{ где } C_{пз} = C_{пз} C_3. \quad (3.114)$$

Одноэтапный метод удобен для параллельного проецирования одиночных выпуклых объектов прямо на экран дисплея. Решение задачи отсечения выполняется быстрыми целочисленными алгоритмами в ЭСК.

Совмещение ЭСК с МСК превращает двухэтапное преобразование (3.113) в одноэтапное (3.114). Если при этом ось z_3 направлена к наблюдателю, то в *левой* ЭСК направления всех вращений будут *обратными* в сравнении с правой СК, что нужно учитывать при выборе параметров матриц вращений (3.35).

При выводе элементов изображения на экран сразу после их расчёта и при низком быстродействии ЭВМ наблюдается неприятный *эффект «мерцания»*, при котором последовательность вывода замедлена и видна в реальном времени. Устранение мерцания достигается *методом буферизации* [65] — разделения массивов памяти, используемых для формирования изображения и вывода его на экран. Если используемый программой графический режим имеет хотя бы две страницы видеопамати (*буфера кадра*), то одна из них выбирается пассивной видимой страницей, а другая — активной невидимой. После построения всего изображения на активной странице по командам прикладной программы выполняется переключение страниц.

В отсутствие второй видеостраницы в качестве буфера используется массив ОЗУ размером с окно вывода, откуда данные блочно и быстро копируются в видеопамать. Этот способ предпочтительнее, т. к. запись в ОЗУ выполняется быстрее, чем в видеопамать, но он требует от программиста создания собственной библиотеки растровых функций построения прямых, окружностей, полигонов и решения задач заливки, отсечения, вывода текста, решаемых встроенными библиотечными функциями любого развитого языка программирования.

3.3.1. Проецирование пространственных линий на плоскость

■ Применим проекционный подход к построению изображений графических объектов на различных поверхностях. Вначале рассмотрим некоторые *методы*

проецирования пространственных линий на плоскость, в том числе и на плоскость экрана. Они могут быть полезны программисту, создающему собственную библиотеку функций построения различных геометрических фигур.

Общий подход к проецированию **полностью детерминированной** линии $p(t) \in R^3$ из МСК на плоскость экрана заключается в расчёте по (3.114) матрицы проецирования $C_{пэ}$, выборе отсчётов параметра t_i , вычислении узловых точек $p_i = p(t_i)$, их экранных проекций $\tilde{p}_{эi} \sim \tilde{p}_i C_{пэ}$ и соединении соседних точек полилинией из отрезков $line(p_{эi}, p_{э,i+1})$.

Проецирование обладает полезным свойством: локальные участки линии $p(t)$, на которых вектор $V(t) = dp(t)/dt$ составляет с направлением проецирования малые углы, имеют на проекции $p_{э}(t)$ большую кривизну и сгущение узлов. Следовательно, **оптимальным является распределение узлов на кривой линии по следующему принципу: плотность точек p_i пропорциональна кривизне самой линии, а не её проекции**. Для изображения отрезка прямой достаточно проекций всего двух его конечных точек.

Проецирование окружности из пространства на плоскость эквивалентно аффинному сжатию плоской окружности в эллипс. Как показано на рис. 3.31,в, это преобразование автоматически сгущает точки $p_{эi}$, удалённые от центра эллипса, пропорционально кривизне этой линии, тогда как узлы p_i распределены по исходной окружности равномерно.

Рассмотрим **методы проецирования некоторых не полностью определённых линий**. Начнём с построения **правильной n -сегментной полилинии** $P = p_0 p_1 \dots p_n p_1$, вписанной в дугу окружности, с параметрами:

- o и N — центр окружности и нормаль к её плоскости $f\{o, N\}$;
- $p_0 \in f$ — начальная вершина полилинии, лежащая в плоскости f ;
- α рад — угол раствора дуги описанной окружности.

Наиболее рациональный путь решения задачи состоит в построении образа полилинии $P_o \in xy$ путём n -кратного вращения орта x^o вокруг оси z на угол $\psi = \alpha/n$, преобразовании образа в заданное положение P на плоскости f и, наконец, проецировании на картинную плоскость или экран (рис. 3.73).

Трёх элементов o, p_0 и N , связанных с плоскостью f , достаточно для расчёта матрицы C_0 преобразования из СК xyz в СК $x'y'z'$, возвращаемой определённой в (3.20) функцией МРЗ. Зададим две тройки её аргументов $\{O_3, x^o, z^o\}$ и $\{o, p_0 - o, \bar{N}\}$ так, чтобы

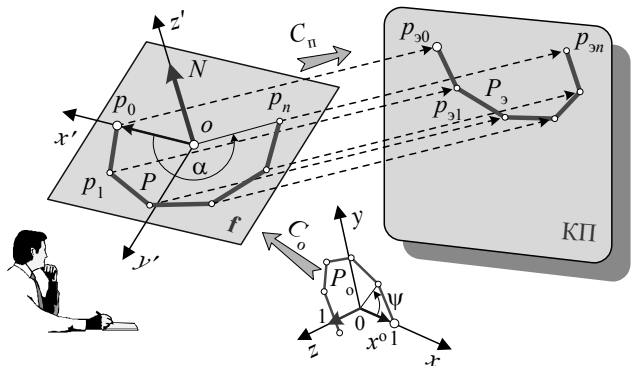


Рис. 3.73

образ точки p_0 был на конце x^0 , а коэффициенты масштабирования по осям x и y были равны радиусу описанной окружности $r = |p_0 - o|$. Отметим, что если третий элемент тройки прообраза задать ненормированным вектором N , то коэффициент масштабирования по оси y будет равен $r|N|$, а линия $P \in f$ будет вписана в дугу эллипса, а не окружности.

На основе сказанного составим **функцию проецирования дуги правильной полилинии** с матрицей проецирования $C_{\Pi} \in \{\tilde{A}_f, \tilde{K}_f, \Pi_f\}$.

```

❏ pro_arc( $o, N, p_0, \alpha, n, C_{\Pi}$ )
{  $R = R_z(\alpha/n)$ ; // матрица вращения на угол  $\psi$ 
   $C = MP3(O_3, x^0, z^0, o, p_0 - o, \bar{N})C_{\Pi}$ ; // матрица проецирования образа
   $p = x^0, \tilde{p}_3 \sim \tilde{p}C$ ; // образ первого узла и его проекция
   $\forall i = \overline{1, n}$  // цикл построения сегментов полилинии
  {  $q = p_i$ ; // запоминание проекции  $i$ -го узла
     $P = pR$ ; // расчёт образа  $i+1$ -го узла
     $\tilde{p}_3 \sim \tilde{p}C$ ; // проекция  $i+1$ -го узла
     $line(q, p_3)$ ; // проекция отрезка полилинии
  }
} ❏
```

Задание специальных значений аргументов функции *pro_arc* позволяет проецировать различные пространственные линии:

- при $\alpha = 2\pi$ получим проекцию **правильного n -угольника**;
- при достаточно большом значении n полилиния визуально совпадает с **дугой окружности**. Например, при $n = \lceil 180\alpha/\pi \rceil$ узлы полилинии будут равномерно распределены по дуге описанной окружности через один градус. Следуя рекомендациям, изложенным в комментариях к рис. 3.31, получим оптимальное значение $n = \lceil \alpha \mu_3 r \bar{N}_z \rceil$, где $r = |p_0 - o|$, а μ_3 — вычисленный по (3.42б) масштабный коэффициент перехода из ОСК в ЭСК;

- для проецирования **окружности** задаются угол раствора $\alpha = 2\pi$ и число сторон аппроксимирующего её правильного полигона, например, $n = 360$ или оптимальное значение $n = \lceil 2\pi \mu_3 r \bar{N}_z \rceil$. Проекция окружности представляет собой вписанный в эллипс n -угольник с плотностью вершин, пропорциональной кривизне эллипса, и минимальным расстоянием между ними в один пиксел.

В описанной выше функции *pro_arc* положение окружности в пространстве задаётся центром o и начальной точкой p_0 , лежащей в плоскости $\{o, N\}$ и определяющей радиус окружности $r = |p_0 - o|$. Часто исходными данными могут быть o, N и r , что делает построение окружности **неоднозначным**, т. к. при этом остаётся неизвестным расположение точки p_0 . В условиях неполной определённости состояний образа $\{O_3, z^0\}$ и прообраза $\{o, r\bar{N}\}$ вычисление матрицы C_o

активного преобразования образа из СК $xуz$ в СК $x'y'z'$ возможно методом парных точек с помощью функции $MP2$, определённой в (3.22). Взяв начало образа окружности на конце орта x^0 , получим точку $\tilde{p}_0 = \tilde{x}^0 C_o$, лежащую в плоскости $\{o, N\}$ на расстоянии r от точки o .

Реализуем всё сказанное в виде **функции проецирования окружности** с параметрами $\{o, N, r\}$ и матрицей проецирования C_Π .

```

☞ pro_circle( $o, N, r, C_\Pi$ )
{  $n = \lceil 2\pi\mu_z r \bar{N}_z \rceil$ ; // число вершин полигона
   $R = R_z(2\pi/n)$ ; // матрица вращения
   $C = MP2(O_3, z^0, o, r\bar{N})C_\Pi$ ; // матрица проецирования образа
   $p = x^0, \tilde{p}_3 \sim \tilde{p}C$ ; // образ начальной вершины и её проекция
   $\forall i = \overline{1, n}$  // цикл построения сторон полигона
  {  $q = p_i$ ; // запоминание проекции  $i-1$ -й вершины
     $P = pR$ ; // расчёт образа  $i$ -й вершины
     $\tilde{p}_i \sim \tilde{p}C$ ; // проекция  $i$ -й вершины
     $line(q, p_i)$ ; // проекция  $i$ -го ребра полигона
  }
} ☐

```

С помощью функции *pro_circle* были построены изображения окружностей, вписанных в грани кубов на рисунках в разд. 3.2.

☞ **Пример 3.8.** Построить диметрическую проекцию каркаса глобуса, состоящего из пяти параллелей с шагом по широте 30° , четырёх меридианов с шагом по долготе 45° и абриса. Радиус глобуса r , угол склонения вправо в плоскости $xу$ равен $\varphi_{\text{ск}} = 23.5^\circ$ (рис. 3.74).

Решение. Желаемое склонение глобуса с вертикальной осью достигается вращением его элементов матрицей

$$R_{\text{ск}} = R_z(-\varphi_{\text{ск}}) = \begin{bmatrix} 0.917 & -0.4 & 0 \\ 0.4 & 0.917 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Диметрические проекции параллелей с общими векторами нормалей $N_\Pi = y^0 R_{\text{ск}}$, разными центрами $o(\alpha_\Pi) = r \sin(\alpha_\Pi) y^0 R_{\text{ск}}$ и радиусами $r(\alpha_\Pi) = r \cos(\alpha_\Pi)$ строятся функцией

$$pro_circle(o(\alpha_\Pi), N_\Pi, r(\alpha_\Pi), \tilde{A}_{\text{дим}} C_3) \forall \alpha_\Pi \in \{0^\circ, \pm 30^\circ, \pm 60^\circ\}.$$

Проекции меридианов с разными нормальными $N(\alpha_m) = [\cos(\alpha_m) \ 0 \ \sin(\alpha_m)] R_{\text{ск}}$, нулевыми центрами и равными радиусами r строятся функцией

$$pro_circle(O_3, N(\alpha_m), r, \tilde{A}_{\text{дим}} C_3) \forall \alpha_m \in \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}.$$

Ортогональный фронтальный абрис глобуса является диметрической проекцией большой окружности в плоскости сечения $\{O_3, S\}$ с нормалью $S_{\text{дим}} = \begin{bmatrix} 1 & 1 & \sqrt{7} \end{bmatrix}$ из (3.90) и строится функцией $pro_circle(O_3, S_{\text{дим}}, r, \tilde{O}_f C_3)$. ☐

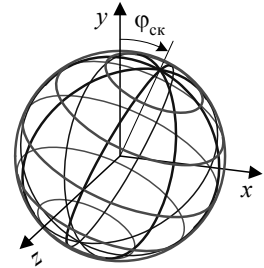


Рис. 3.74

3.3.2. Стереографические проекции

Во многих приложениях (компьютерные игры, тренажёры, авто-, авиа- и космические симуляторы, управление манипуляторами и т. п.) важно создать восприятие глубины сцены, максимально близкое к реальной действительности. Проецирование на плоскость из единственного центра равносильно съёмке неподвижной фотокамерой на плоскую плёнку или светочувствительную матрицу, дающей неполное ощущение глубины окружающего мира. Это, конечно, лучше параллельного проецирования, которое вообще не способно к передаче глубины из-за бесконечной удалённости наблюдателя.

Система **монокулярного зрения** человека преобразует двухмерную картинку на **неплоской**, почти сферической, поверхности сетчатки глаза в объёмную, используя следующие «подсказки» на подсознательном уровне [51]:

- **геометрическая перспектива** — сходжение в одну точку изображений параллельных прямых и уменьшение угловых размеров деталей объектов;
- **воздушная перспектива** — ослабление толщей атмосферы цветового контраста и яркости изображений;
- **параллакс движения** — ближние к глазу объекты смещаются на проекции больше, чем дальние объекты. Психофизиологически глубина объекта в сцене наиболее сильно ощущается при его движении в направлении поперёк зрению;
- **относительный размер** знакомых наблюдателю объектов;
- **экранирование** (перекрывание) дальних объектов ближними;
- **фокусирующая аккомодация** — зависимость напряжения фокусирующих мышц глаза от расстояния до объекта.

Система **бинокулярного зрения**, включающая два глаза и центр обработки зрительной информации, находящийся в коре головного мозга, обрабатывает дополнительные данные:

- **угол сведения** оптических осей глаз при рассматривании близких объектов;
- **несовпадение** изображений объекта на двух сетчатках, используемое мозгом для оценки направления и расстояния до объекта.

О попытках технической реализации стереоэффекта известно достаточно давно. Устройство для рассматривания стереоизображений — **стереоскоп** — имеет более чем 200-летнюю историю. Наброски оптических схем, свидетельствующие о проведении опытов по стереоскопии, найдены в работах великого Леонардо (рис. 3.75,а). Изобретателем стереоскопа считается Ч. Уитстон

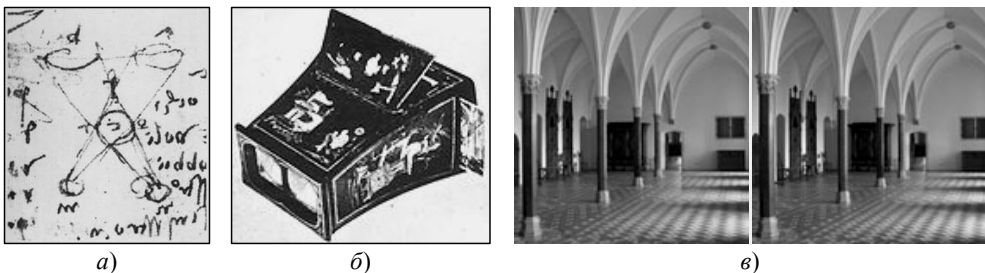


Рис. 3.75

(1837 г.), а дошедший до наших дней его линзовый вариант (рис. 3.75,б) создан Д. Брюстером в 1849 году. Для получения стереоэффекта создаётся сфотографированная сразу двумя камерами **стереопара** — два изображения, видимые отдельно каждым глазом (рис. 3.75,в). С тех пор ничего идейно нового в области стереоскопии не появилось — развивались лишь методы разделения стереопар и их компьютерного синтеза.

Программная имитация бинокулярного зрения заключается в разработке **алгоритма построения стереографической проекции** — двух ЦП на общую картинную плоскость от двух разнесённых в пространстве проекторов (рис. 3.76). Пусть расстояние между глазами — **оптическая база** — равно $2D$. Зрительная система фокусирует взгляд в определённой точке — **центре визуальной массы** объекта или группы объектов, находящемся от наблюдателя на фокусном расстоянии F .

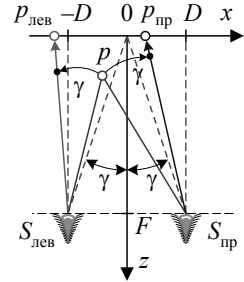


Рис. 3.76

Проведём через центр визуальной массы фронтальную картинную плоскость ортогонально направлению взгляда. Глаза наблюдателя (проекторы) находятся в точках $S_{лев} = [-D \ 0 \ F]$, $S_{пр} = [D \ 0 \ F]$ и фокусируются под **углом сведения** $\gamma = \arctg(D/F)$. Если $D \ll F$, то $\gamma \approx 0^\circ$ и оба глаза видят далёкий объект практически одинаково. При сравнимых расстояниях D и F изображения, формируемые в сетчатке каждого глаза, отличаются друг от друга, чем и достигается эффект объёмности при соединении мозгом двух картинок в одну. Наиболее комфортные для зрения условия и наилучший стереоэффект достигаются при $\tg(\gamma) \approx 0.1$, откуда $\gamma \approx 6^\circ$ и $F \approx 10D$. При среднестатистической базе $2D = 60$ мм получим $F = 30$ см — оптимальное гигиеническое расстояние для чтения, письма и точной ручной работы [22]. Такого же соотношения нужно придерживаться при создании стереофотографий объектов природы, архитектуры и т. п.

Стереопара состоит из двух центральных проекций объекта, рассчитанных отдельно для каждого глаза. Её рассматривание производится с **нулевым** углом сведения глаз, как бы глядя в даль. Преобразования произвольной точки объекта p в проекции левого и правого глаз состоят из операций вращения вокруг соответствующего глаза на углы $\phi_{лев} = \gamma$, $\phi_{пр} = -\gamma$ и центрального проецирования на ФКП, описываемого по (3.104б) матрицами

$$U_{лев} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ D/F & 0 & 0 & -1/F \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad U_{пр} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -D/F & 0 & 0 & -1/F \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Таким образом, создание стереопары $\{p_{лев}, p_{пр}\}$ точки p выполняется по следующим формулам:

$$\begin{aligned} \tilde{p}_{лев} &\sim \tilde{p} T(-S_{лев}) \tilde{R}_y(\gamma) T(S_{лев}) U_{лев}, \\ \tilde{p}_{пр} &\sim \tilde{p} T(-S_{пр}) \tilde{R}_y(-\gamma) T(S_{пр}) U_{пр}. \end{aligned} \quad (3.115)$$

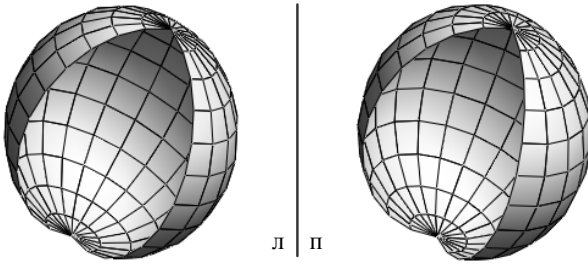


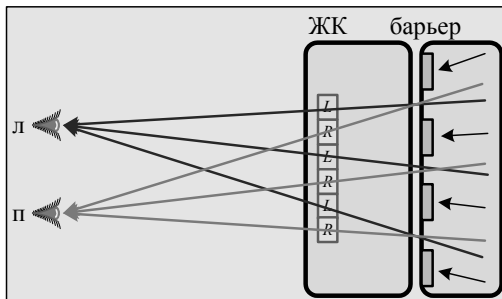
Рис. 3.77

На рис. 3.77 построено стереографическое изображение трёх четвертей глобуса радиуса $R=20$ мм при фокусном расстоянии $F=300$ мм и оптической базе $2D=60$ мм. Рассматривать его рекомендуется с расстояния 30 см, поместив между глазами непрозрачную перегородку на ли-

нии, разделяющей пространства левого и правого изображений. Максимально полное ощущение стереоэффекта достигается при рассматривании не статичных, а движущихся в пространстве сцены объектов.

В настоящее время происходит широкое внедрение идей стереоскопического зрения в технологию производства жидкокристаллических (ЖК) дисплеев и телевизионных панелей. Целая группа передовых восточных ИТ-компаний вышли на рынок со своими новыми моделями мониторов, ноутбуков и мобильных телефонов, оснащённых объёмными, точнее — стереоскопическими, дисплеями, используемыми без каких-либо вспомогательных средств.

Принцип действия одной из моделей ЖК-дисплея следующий (рис. 3.78,а). Каждый пиксел экрана разделён на два субпикселя — левый L и правый R , на которые выводятся **разные** изображения, рассчитанные для отдельных глаз аналогично (3.115). Если в обычном 2d-дисплее лучи из источника засветки проходят через каждый пиксел и попадают в оба глаза, то в стереоскопическом дисплее происходит разделение лучей левого и правого субпикселей с помощью **параллаксного барьера** на жидких кристаллах, работающего наподобие маски кинескопа цветного телевизора. Благодаря точно рассчитанному расположению непрозрачных барьерных зон левые и правые лучи фокусируются в тех областях предэкранного пространства, где обычно находятся глаза наблюдателя. В результате у последнего создаётся ощущение трёхмерности изображения (рис. 3.78,б). При желании непрозрачность жидкокристаллического барьера может быть отключена, и изображение станет плоским. То же произойдёт при отсутствии программы, создающей стереоэффект.



а)



б)

Рис. 3.78

Кроме рассмотренных способов **пространственного** разделения стереопар существуют и другие методы реализации этого процесса [6, 51]:

- **цветовое** — одно изображение (*анаглиф* от греч. *anaglyphos* — рельефный) несёт в себе два (рис. 3.79,а) — то, что для левого глаза, снято, например, в красных тонах, а для правого — в бирюзовых (эти цвета дополняют друг друга до белого). Цветоделение полноцветных фотографий и сведение их в один файл (рис. 3.79,б) можно сделать в графических программах типа Photoshop. Разделение левого и правого изображений происходит при просмотре анаглифа через цветные очки с соответствующими светофильтрами (рис. 3.79,в). В современных моделях очков цветные фильтры заменены поляризованными. Поляризационным очкам, используемым также в 3d-кинотеатрах, присуще снижение яркости изображения примерно на 50%. Анаглифный метод, основанный на аддитивном способе формирования цвета, хорошо подходит для демонстрации стереоизображений без потери разрешения экрана монитора. Полиграфическая же печать анаглифных изображений — весьма сложное дело, так как принцип формирования цвета в принтерах — субтрактивный, а однозначное соответствие между аддитивной и субтрактивной цветовыми моделями проблематично;

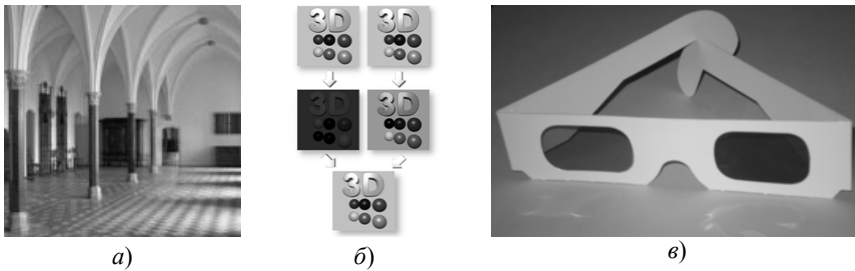


Рис. 3.79

- **пространственно-временное** — изображение для левого глаза строится в чётных строках экрана, для правого — в нечётных, а для просмотра используются электромеханические или жидкокристаллические (рис. 3.80,а) очки, перекрывающие соответствующий глаз на время прорисовки чётных и нечётных строк. Этот подход может быть реализован на дисплеях с чересстрочной развёрткой и позволяет выводить быстро меняющиеся динамические цветные полутоновые изображения, но с уменьшенным вдвое вертикальным разрешением;

- **временное** — для левого глаза используется одна страница видеопамяти, для правого — другая. За счёт эффекта инерции зрения в мозгу зрителя форми-

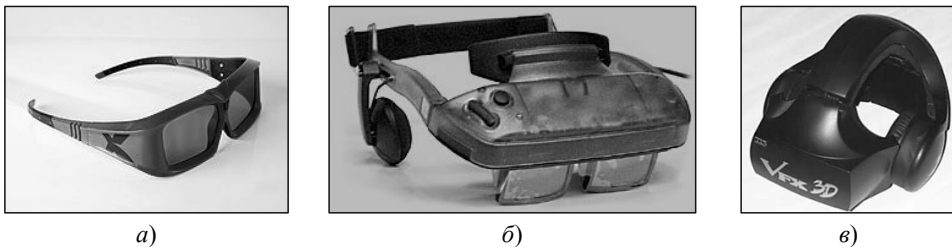


Рис. 3.80

руется цельное изображение. При этом требуется монитор с *двойной* частотой развёртки, чтобы для каждого глаза частота обновления изображения была обычной. Снижение яркости изображения у затворных устройств составляет примерно 80%. Для перекрытия глаз также используются очки с электронным переключением. Пространственное разрешение по строкам не теряется;

- наилучшие условия для восприятия стереоизображений созданы в *системах виртуальной реальности*, где в специальные очки (рис. 3.80,б) или шлем (рис. 3.80,в) вмонтирован бинокулярный монитор из двух мини-дисплеев. На них подаются картинки, рассчитанные отдельно для каждого глаза. Управляющая программа с помощью механических, инфракрасных или электромагнитных датчиков отслеживает повороты головы и глаз, что создаёт удивительный эффект «погружения в действительность» со стереозвуком и изображением, обрабатывающим изменение сцены в реальном времени в зависимости от движений наблюдателя. В перспективе создание систем с лазерной прорисовкой изображений непосредственно на сетчатке каждого глаза.

3.3.3. Картографические проекции

К типичным, широко распространённым и общеизвестным представителям сложных проективных преобразований относятся *картографические проекции*, применяемые для составления плоских географических карт поверхности Земли. Последняя, являясь приближённо эллипсоидом вращения с соотношением экваториального радиуса к полярному 1.00336:1 [55], не может быть развёрнута на плоскость без складок и растяжений. В связи с этим фактом здесь уместно процитировать стихотворение известного картографа А. В. Гедымина:



Изучая суть проекций,
Надо помнить положение:
С переходом сферы в плоскость
Неизбежны искажения.

Карты вовсе не безгрешны
И в пределах разных норм
Нарушают верность линий,
Площадей, углов и форм.

Искажения, возникающие на карте, являются расплатой за удобство, получаемое при пользовании плоским изображением. Характер искажений зависит от способа проецирования и от вида *картографической сетки* — системы меридианов и параллелей на карте, т. к. в неё, как в «прокрустово ложе», приходится вгонять изображения материков. В зависимости от неискажаемого параметра принято следующее деление карт [52]:

- *равноугольные* — проекции **Меркатора** (1512–1594), сохраняющие *угловые* соотношения и конфигурацию фигур за счёт искажения их площадей. В таких проекциях строятся навигационные карты, используемые в судовождении;

- *равновеликие* — проекции **Ламберта** (1728–1777), сохраняющие соотношения *площадей* фигур в разных частях карты за счёт искажений их формы, особенно сильных на краях карты. Такие проекции применяются для составления политических карт, где важно показать истинное соотношение размеров различных стран;

- **равнопромежуточные** (эквидистантные) с одинаковыми **расстояниями** между определёнными линиями (параллелями или меридианами) картографической сетки во всех частях карты;

- **произвольные** — проекции, которым свойственны в разной степени как угловые, так площадные и дистантные искажения.

Основными этапами картографической съёмки являются преобразования точек земной поверхности в систему координат проективной поверхности и в случае её непланарности развёртывание на плоскость. Идеализируя весьма сложную форму Земли сферой единичного радиуса, зададим положение точки на её поверхности общепринятыми географическими координатами:

- $\varphi^\circ \in (-180^\circ, 180^\circ]$ — долготой, измеряемой от Гринвичского меридиана;
- $\psi^\circ \in [-90^\circ, 90^\circ]$ — широтой, отсчитываемой от параллели экватора.

Механизм построения любой картографической проекции таков, что проектор и наблюдатель находятся по разные стороны от проективной поверхности. Проекция фактически является **обратной перспективой** (см. рис. 3.69,а), что хорошо заметно на многих картах. Важно отметить, что создание некоторых картографических проекций при **неподвижном** проекторе не всегда возможно. В этих случаях используются формулы преобразования координат, которые аналитически выводятся в рамках проективного метода с **подвижным проектором**, положение которого зависит от координат $\{\varphi, \psi\}$ [31].

Искажение какого-либо геометрического параметра образа характеризуется отношением к нему того же параметра прообраза на плоской карте. Возьмём в качестве образа на поверхности сферы прямоугольник с полудлинами сторон a по долготе и b по широте. Обозначим полудлины сторон спроецированного прообраза a' и b' . Эта фигура будет близка к прямоугольнику лишь при достаточно малых значениях $a \ll 1$ и $b \ll 1$. Определим **коэффициенты долготного** m_φ , **широтного** m_ψ , **углового** $m_\angle$ и **площадного** $m_\blacksquare$ **искажений**:

$$m_\varphi = \lim_{a \rightarrow 0} (a'/a), \quad m_\psi = \lim_{b \rightarrow 0} (b'/b), \quad m_\angle = m_\psi / m_\varphi, \quad m_\blacksquare = m_\varphi m_\psi. \quad (3.116)$$

На равноугольных картах должно быть $m_\angle = 1$, на равновеликих — $m_\blacksquare = 1$.

Возможность визуального оценивания искажений на построенных ниже картах дают проекции чёрных квадратов, размещённых в разных местах поверхности сферы. При аналитическом выводе коэффициентов искажений в различных картографических проекциях (для этого привлекался символический процессор MathCAD) благодаря малости значений a и b можно считать, что на широте ψ они имеют угловые размеры $a \sec(\psi)$ по долготе и b по широте.

В зависимости от формы и положения проективной поверхности наиболее распространены следующие **типы картографических проекций**.

Азимутальные проекции

Построение **азимутальной** карты заключается в проецировании точки $p(\varphi, \psi)$ на **плоскость**, касающуюся сферы в заданной точке $p_0(\varphi_0, \psi_0)$

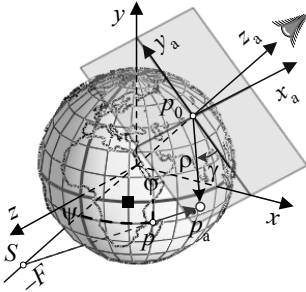


Рис. 3.81

(рис. 3.81). В этой точке карты искажения отсутствуют, а по мере радиального удаления возрастают.

В зависимости от широты точки p_0 азимутальные проекции делятся на **полярные** ($\psi_0 = \pm 90^\circ$), **экваториальные** ($\psi_0 = 0^\circ$) и **косые** ($0^\circ < |\psi_0| < 90^\circ$). Для уменьшения периферийных искажений проективную плоскость можно сделать секущей, минимаксно удалённой от точек проецируемой области сферы.

Фронтальная ориентация азимутальной плоскости достигается установкой начала СК $x_a y_a z_a$ в точке p_0 и направлением осей x_a и y_a по касательным к параллели ψ_0 и меридиану φ_0 , который на карте должен быть расположен вертикально. Такое сложное преобразование СК по принципу инверсности выполняется матрицей

$$C_a = \tilde{R}_y(-\varphi_0) \tilde{R}_x(\psi_0) T(-z^0). \quad (3.117a)$$

Поместив проектор в точку $S = [0 \ 0 \ -F]$ на оси z_a , проходящей через центр сферы — точку $[0 \ 0 \ -1]$ в СК $x_a y_a z_a$, рассчитаем координаты азимутальной проекции $p_a \equiv [x_a \ y_a \ 0]$ точки сферы $p(\varphi, \psi)$, кинематически получаемой вращениями точки z^0 вокруг мировых осей x и y на углы $-\psi$ и φ :

$$\begin{aligned} \tilde{p}_a &\sim \tilde{z}^0 \tilde{R}_x(-\psi) \tilde{R}_y(\varphi) C_a U_f, \\ x_a &= \frac{\sin(\varphi - \varphi_0) \cos(\psi)}{F - 1 + \sin(\psi) \sin(\psi_0) + \cos(\varphi - \varphi_0) \cos(\psi) \cos(\psi_0)} F, \\ y_a &= \frac{\sin(\psi) \cos(\psi_0) - \cos(\varphi - \varphi_0) \cos(\psi) \sin(\psi_0)}{F - 1 + \sin(\psi) \sin(\psi_0) + \cos(\varphi - \varphi_0) \cos(\psi) \cos(\psi_0)} F. \end{aligned} \quad (3.117б)$$

Центральная проекция (3.117б) действительна лишь при условии пересечения проективного луча с азимутальной плоскостью:

$$F - 1 + \sin(\psi) \sin(\psi_0) + \cos(\varphi - \varphi_0) \cos(\psi) \cos(\psi_0) > 0. \quad (3.117в)$$

У точки сферы, нарушающей это условие, центральная проекция становится мнимой, из-за чего эта точка на азимутальной карте не изображается.

Построение картографической проекции некоторой полилинии $p_1 p_2 \dots p_n$, изображающей, например, контур материка, русло реки или государственную границу и состоящей из достаточно коротких отрезков $p_i p_{i+1}$, наиболее просто и быстро выполняется путём вывода на экран только тех отрезков, у которых оба конца удовлетворяют условию (3.117в).

В зависимости от величины **постоянного** фокусного расстояния F картографические, в том числе азимутальные, проекции имеют особые названия:

- при $F=1$ проектор находится в центре сферы и создаёт **гномоническую** действительную проекцию неполной (без граничной окружности) полусферы, для которой характерно разрежение линий сетки при удалении от центра карты. При выборе $F>1$ становится возможным построение азимутальной карты поверхности, превышающей полусферу;

- проекция, соответствующая $F=2$, называется **стереографической**. Проецирующие лучи расходятся из диаметрально противоположной точки $S=-p_0$ и создают действительные проекции всех точек сферы, кроме, естественно, S ;

- проекция, построенная при $F>2$, называется **внешней**, т. к. проектор находится вне сферы. Поэтому все её точки имеют действительные проекции на касательную плоскость. Поскольку проецирующие лучи могут дважды пересечь сферу, то во избежание наложения на карте проекций ближних и дальних точек пересечения следует аккуратно ограничивать отображаемую область сферы;

- при $F \rightarrow \infty$ центральная проекция вырождается в **ортографическую**. Проецирующие лучи становятся параллельными и также, кроме касательных, дважды пересекают сферу. Отображаемая без наложений область составляет полу-сферу. Линии сетки на азимутальной карте сгущаются от центра к периферии.

При всех значениях фокусного расстояния искажения отсутствуют в точке касания проективной плоскости со сферой и существуют в остальных точках. Варьируя расположение точки касания, рассмотрим три **вида азимутальных картографических проекций**.

□ В **полярной проекции** плоскость касается сферы в полюсах при $\psi_0 = \pm 90^\circ$. Сетку карты составляют концентрические окружности параллелей и радиальные прямые меридианов. Коэффициенты долготных и широтных искажений, вычисленные по (3.116) при **постоянном** фокусном расстоянии F , равны

$$m_\varphi = \frac{F}{F-1+\sigma \sin(\psi)}, \quad m_\psi = \frac{1+\sigma(F-1)\sin(\psi)}{(F-1+\sigma \sin(\psi))^2} F, \quad (3.118)$$

где $\sigma = \sin(\psi_0) = \text{sgn}(\psi_0) \in \{-1, 1\}$.

Для их получения возьмём на меридиане φ_0 , имеющем вертикальную полярную проекцию, точку $p_1(\varphi_0, \psi)$, а на дуговых расстояниях от неё a и b ещё две точки $p_2(\varphi_0 + a \sec(\psi), \psi)$ и $p_3(\varphi_0, \psi + b)$. Найдём по (3.117б) их прообразы на проективной плоскости:

$$p'_1 = \frac{F \cos(\psi)}{F-1+\sigma \sin(\psi)} [0 \quad -\sigma], \quad p'_3 = \frac{F \cos(\psi+b)}{F-1+\sigma \sin(\psi+b)} [0 \quad -\sigma],$$

$$p'_2 = \frac{F \cos(\psi)}{F-1+\sigma \sin(\psi)} \left[\sin\left(\frac{a}{\cos(\psi)}\right) - \sigma \cos\left(\frac{a}{\cos(\psi)}\right) \right].$$

Определим долготное и широтное расстояния между этими точками

$$a' = x'_2 - x'_1 = \frac{F \cos(\psi)}{F-1+\sigma \sin(\psi)} \sin\left(\frac{a}{\cos(\psi)}\right),$$

$$b' = y'_3 - y'_1 = -\sigma \frac{F \cos(\psi+b)}{F-1+\sigma \sin(\psi+b)} + \sigma \frac{F \cos(\psi)}{F-1+\sigma \sin(\psi)}$$

и по правилам дифференцирования вычислим пределы в (3.116):

$$\lim_{a \rightarrow 0} \left(\frac{a'}{a} \right) = \frac{F}{F-1+\sigma \sin(\psi)} = m_\varphi,$$

$$\lim_{b \rightarrow 0} \left(\frac{b'}{b} \right) = -\sigma F \frac{d}{d\psi} \left(\frac{\cos(\psi)}{F-1+\sigma \sin(\psi)} \right) = \frac{1+\sigma(F-1)\sin(\psi)}{(F-1+\sigma \sin(\psi))^2} F = m_\psi.$$

Проекцию точки сферы на плоскость можно и часто удобнее получать не проективным методом (3.117), а **прямым расчётом** её полярных координат — модуля $\rho = |p_a|$ и угла γ , отсчитываемого от вертикали (см. рис. 3.81):

$$\rho(\psi, F) = \frac{F \cos(\psi)}{F - 1 + \sigma \sin(\psi)}, \quad \gamma = \sigma(\varphi - \varphi_0). \quad (3.119a)$$

Преобразование полярных координат в декартовы производится по формулам

$$p' = [0 \ -\sigma\rho \ 0]R_z(\gamma), \quad x' = \rho \sin(\gamma), \quad y' = -\rho \sigma \cos(\gamma). \quad (3.119б)$$

Функция $\rho(\psi, F)$ связана с коэффициентами искажений m_φ и m_ψ следующими соотношениями:

$$m_\varphi = \frac{\rho(\psi, F)}{\cos(\psi)}, \quad m_\psi = -\sigma \frac{d\rho(\psi, F)}{d\psi}. \quad (3.120a)$$

Использование здесь полной производной

$$\frac{d\rho(\psi, F)}{d\psi} = \frac{\partial \rho(\psi, F)}{\partial \psi} + \frac{\partial \rho(\psi, F)}{\partial F} \cdot \frac{dF(\psi)}{d\psi} \quad (3.120б)$$

приведёт в дальнейшем к аналитически неразрешимым нелинейным дифференциальным уравнениям. В связи с этим мы ограничимся частной производной

$$\frac{\partial \rho(\psi, F)}{\partial \psi} = -\sigma \frac{1 + \sigma(F-1)\sin(\psi)}{(F-1 + \sigma \sin(\psi))^2} F, \quad (3.120в)$$

подставив которую в (3.120a), получим коэффициент искажения m_ψ в (3.118).

Исследуем свойства и особенности расчёта некоторых азимутальных карт:

- **гномоническая** проекция с постоянным фокусом $F=1$ и параметрами

$$\rho(\psi) = \sigma \operatorname{ctg}(\psi), \quad m_\psi = \sin^{-2}(\psi), \quad m_\varphi = m_\angle = \sigma \sin^{-1}(\psi), \quad m_\blacksquare = \sigma \sin^{-3}(\psi)$$

по классификации искажений является произвольной. В ней сильно искажены углы и площади вдали от полюса, а проекция экватора вообще имеет бесконечный радиус. В этом убеждает рис. 3.82,а, где построена карта части северного полушария в широтном диапазоне $\psi \in [30^\circ, 90^\circ]$. Поэтому в гномонической проекции рекомендуется картографировать полярные области высоких широт $\psi \in [60^\circ, 90^\circ]$ либо $\psi \in [-90^\circ, -60^\circ]$;

- уравнивая в (3.118) коэффициенты m_φ и m_ψ , получаем фокусное расстояние $F=2$ и параметры полярной **стереографической** проекции:

$$\rho(\psi) = \frac{2 \cos(\psi)}{1 + \sigma \sin(\psi)}, \quad m_\varphi = m_\psi = \frac{2}{1 + \sigma \sin(\psi)}, \quad \boxed{m_\angle = 1}, \quad m_\blacksquare = \frac{4}{(1 + \sigma \sin(\psi))^2}.$$

На азимутальной **равноугольной** карте Меркатора, как видно из рис. 3.82,б, пропорции фигур на разных широтах сохраняются, а их площади увеличиваются от полюса до экватора в четыре раза;

- в пределе $F \rightarrow \infty$ из (3.119a) и (3.118) получаем свойства **ортографической** проекции, в которой отсутствуют долготные искажения:

$$\rho(\psi) = \cos(\psi), \quad \boxed{m_\varphi = 1}, \quad m_\psi = m_\angle = m_\blacksquare = \sigma \sin(\psi).$$

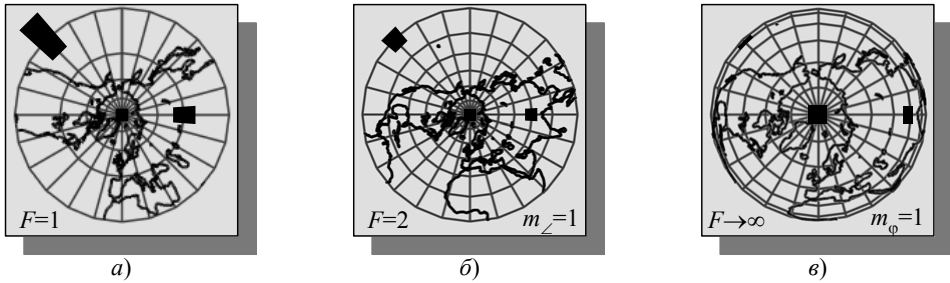


Рис. 3.82

Для неё характерно сгущение параллелей в экваториальной области (рис. 3.82,в) и способность отображения всей земной поверхности. Во избежание наложения проекций полушарий друг на друга следует ограничивать широтный диапазон либо северным ($\psi \in [0^\circ, 90^\circ]$), либо южным ($\psi \in [-90^\circ, 0^\circ]$) полушариями;

• приравняв в (3.120а) произведение коэффициентов m_ϕ и m_ψ к единице, получим дифференциальное уравнение $\rho(\psi) \cdot d\rho(\psi)/d\psi = -\sigma \cos(\psi)$ с начальным условием $\rho(\psi_0)=0$, решение которого $\rho(\psi) = \sqrt{2-2\sigma \sin(\psi)} \equiv s(\psi)$ подставляется в (3.119б) при прямом расчёте **равноплощадной** азимутальной карты Ламберта (рис. 3.83,а). Проективным методом (3.117) эта карта строится при **переменном** фокусном расстоянии

$$F(\psi, \rho) = \frac{\rho(\psi)(1 - \sigma \sin(\psi))}{\rho(\psi) - \cos(\psi)}. \quad (3.121)$$

Графики функций $s(\psi)$ и $F(\psi, s)$, соответствующих северному полушарию с $\sigma=1$, построены на рис. 3.83,б, а графики всех коэффициентов искажений — на рис. 3.83,в. Отклонение значений $m_\psi(\psi)$ от единицы на низких широтах $|\psi| < 45^\circ$ объясняется тем, что они рассчитаны по формуле (3.118), полученной из условия постоянства фокусного расстояния F без учёта в (3.120б) производной $dF(\psi)/d\psi$. Из-за этого и зависимость (3.121) даёт не точную, а лишь очень близкую к равноплощадной азимутальную карту.

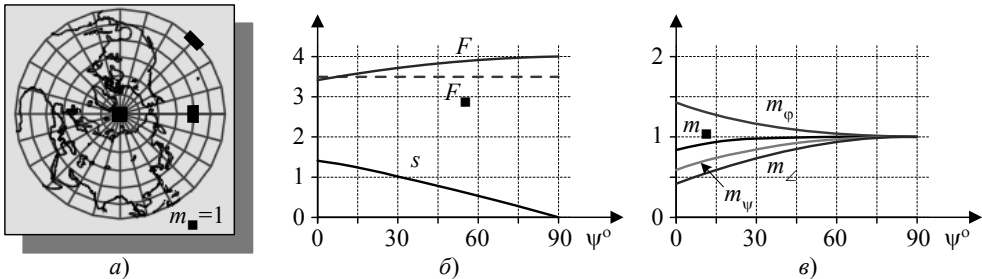


Рис. 3.83

Представляет интерес замена в (3.117) и (3.119) функции $F(\psi, \rho)$, зависящей от широты, **постоянным** значением фокусного расстояния $F_\square$, дающего на полярной карте распределение параллелей, наиболее близкое к равноплощадному. Введём среднеквадратичный критерий близости функций $r(\psi)$ и $\rho(\psi, F)$:

$$J(F, r, \rho) = \int_0^{\pi/2} (r(\psi) - \rho(\psi, F))^2 d\psi \rightarrow \min_F. \quad (3.122)$$

Найденное с помощью MathCAD оптимальное значение постоянного фокуса, доставляющего критерию $J(F, s, \rho)$ минимум $J_{\min} = 54 \cdot 10^{-6}$, равно $F_{\blacksquare} \approx 3.54$;

• приравнявая в (3.120а) коэффициент m_ψ к единице, получим линейное дифференциальное уравнение $d\rho(\psi)/d\psi = -\sigma$ с начальным условием $\rho(\psi_0) = 0$, решение которого $\rho(\psi) = \sigma(\psi_0 - \psi) = \pi/2 - \sigma\psi \equiv l(\psi)$ проективно реализуется при **переменном** фокусном расстоянии $F(\psi, l)$, определённом в (3.121). Оно даёт **равнопромежуточную** полярную проекцию с постоянным шагом параллелей. Зависимости $l(\psi)$ и $F(\psi, l)$ показаны на рис. 3.84,а, а графики всех коэффициентов искажений — на рис. 3.84,б. Отклонение графика $m_\psi(\psi)$ от единицы на низких широтах $|\psi| < 30^\circ$ также объясняется его расчётом при условии постоянства фокусного расстояния F .

Оптимальное по среднеквадратичному критерию $J(F, l, \rho)$ **постоянное** фокусное расстояние проектора, дающее сетку параллелей, наиболее близкую к равномерной, равно $F_\psi \approx 2.8$ при минимуме критерия $J_{\min} = 34 \cdot 10^{-6}$.

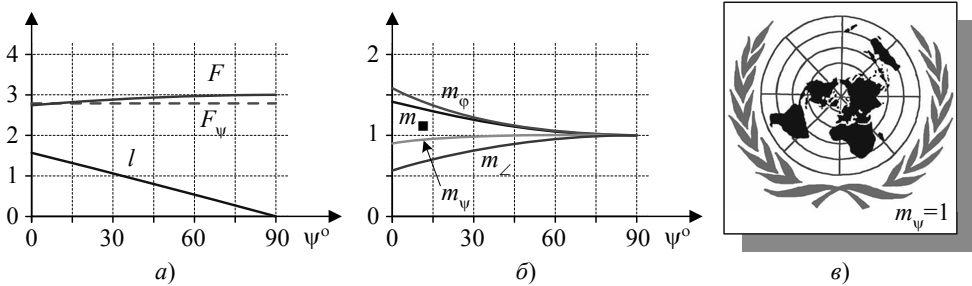


Рис. 3.84

Практичный прямой метод построения равнопромежуточных полярных карт заключается в распрямлении дуги φ -го меридиана на плоскость под углом $\varphi - \varphi_0$. При этом точка сферы на широте ψ оказывается от центра карты как раз на расстоянии длины дуги $\rho = \pi/2 - \text{sgn}(\psi_0)\psi$. Данный приём позволяет изобразить всю поверхность сферы внутри одной окружности радиуса π , что (без Антарктиды) реализовано в эмблеме ООН на рис. 3.84,в. На внешней окружности, представляющей Южный полюс, полная равнопромежуточная карта имеет максимальные угловые и площадные искажения.

□ Для получения **экваториальной** азимутальной проекции проективная плоскость $x_a y_a$ устанавливается параллельно оси вращения сферы и касается экватора $\psi_0 = 0^\circ$ на меридиане φ_0 , который на карте будет расположен вертикально. Дуги остальных меридианов сходятся на полюсах, а линия экватора проходит горизонтально. В **ортографической** проекции с присущими ей большими искажениями все параллели выглядят горизонтальными прямыми (рис. 3.85,а), а в

остальных центральных проекциях с $F < \infty$ — дугами эллипсов. В **стереографической равноугольной** проекции Меркатора принято строить карты полушарий, например, восточного при $\varphi_0 = 90^\circ$ (рис. 3.85, б). В этом ракурсе хорошо проявляется свойство обратной перспективы — уменьшение размеров элементов сферы при их приближении к наблюдателю.

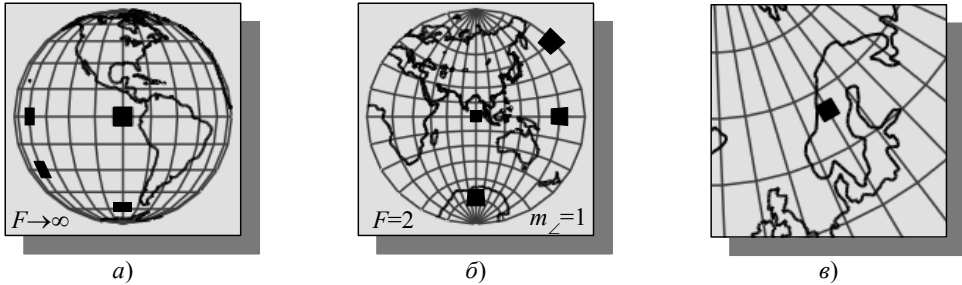


Рис. 3.85

□ Для получения **косой** азимутальной проекции касательная к сфере плоскость $x_a y_a$ устанавливается на произвольной широте $\psi_0 \notin \{0^\circ, \pm 90^\circ\}$ (см. рис. 3.81) и используется общий алгоритм проецирования (3.117). Данная проекция применяется для изображения как всего земного шара в наклонном положении, в котором дуги параллелей и меридианов наглядно подчёркивают его объёмность, так и локальной области земной поверхности с малыми искажениями, для чего параметрам φ_0 и ψ_0 присваиваются значения координат географического центра этой области.

Если требуется закрутить азимутальную карту на угол λ вокруг оси z_a , то вместо (3.117а) должна использоваться матрица преобразования

$$C_a = \tilde{R}_y(-\varphi_0) \tilde{R}_x(\psi_0) \tilde{R}_z(\lambda) T(-z^\circ).$$

По такому модифицированному алгоритму на рис. 3.85, в построена гномоническая карта Скандинавского полуострова с координатами центральной точки $\varphi_0 = 15^\circ$, $\psi_0 = 65^\circ$ (в ней помещён чёрный квадрат) и углом кручения $\lambda = 30^\circ$.

Цилиндрические проекции

Панорамная **цилиндрическая** проекция точек сферы строится на боковой поверхности **цилиндра**, касающегося сферы по большой окружности, а затем цилиндрическая поверхность, будучи линейчатой, развёртывается на плоскость. В зависимости от угла между осями цилиндра и сферы проекции делятся на:

- **продольные** с соосными цилиндром и сферой, касающимися по экватору. Параллели на карте горизонтальны, а вертикальные проекции меридианов равно отстоят друг от друга;
- **поперечные** с ортогональными осями цилиндра и сферы, касающимися по меридиану;
- **косые** с произвольным углом между осями цилиндра и сферы, касающихся по наклонной большой окружности.

Изучим свойства и особенности расчёта некоторых цилиндрических карт:

- подстановка значения $F=1$ в (3.123б) и (3.124) даёт зависимость $h(\psi)=\operatorname{tg}(\psi)$ и следующие коэффициенты искажений:

$$m_{\psi}=\sec^2(\psi), \quad m_{\angle}=\sec(\psi), \quad m_{\blacksquare}=\sec^3(\psi).$$

Эта **гномоническая** цилиндрическая проекция (рис. 3.87,а) сильно искажает на высоких широтах углы, площади и расстояния, т. е. является **произвольной**;

- уравнивая в (3.124) коэффициенты m_{φ} и m_{ψ} , получим дифференциальное уравнение $dh(\psi)/d\psi=\sec(\psi)$ с начальным условием $h(0)=0$. Его решение

$$h(\psi)=\ln(\sec(\psi)+\operatorname{tg}(\psi))\equiv\lambda(\psi)$$

есть нечётная **лямбда-функция** [21] широты ψ . **Равноугольная** цилиндрическая проекция Меркатора (рис. 3.87,б) с коэффициентами искажений

$$\boxed{m_{\angle}=1}, \quad m_{\varphi}=m_{\psi}=\sec(\psi), \quad m_{\blacksquare}=\sec^2(\psi)$$

строится при **переменном** фокусном расстоянии

$$F(\psi, h)=\frac{h(\psi)(1-\cos(\psi))}{h(\psi)-\sin(\psi)}. \quad (3.125)$$

В ней полюсы удалены в бесконечность, а полярные области отличаются большим искажением площадей. Оптимальное по среднеквадратичному критерию $J(F, \lambda, h)$, определённому в (3.122), **постоянное** значение фокусного расстояния с наименьшими угловыми искажениями равно $F_{\angle}\approx 1.29$ при $J_{\min}=0.077$;

- при $F\rightarrow\infty$ все проектирующие лучи горизонтальны, $h(\psi)=\sin(\psi)$, а коэффициенты искажений равны

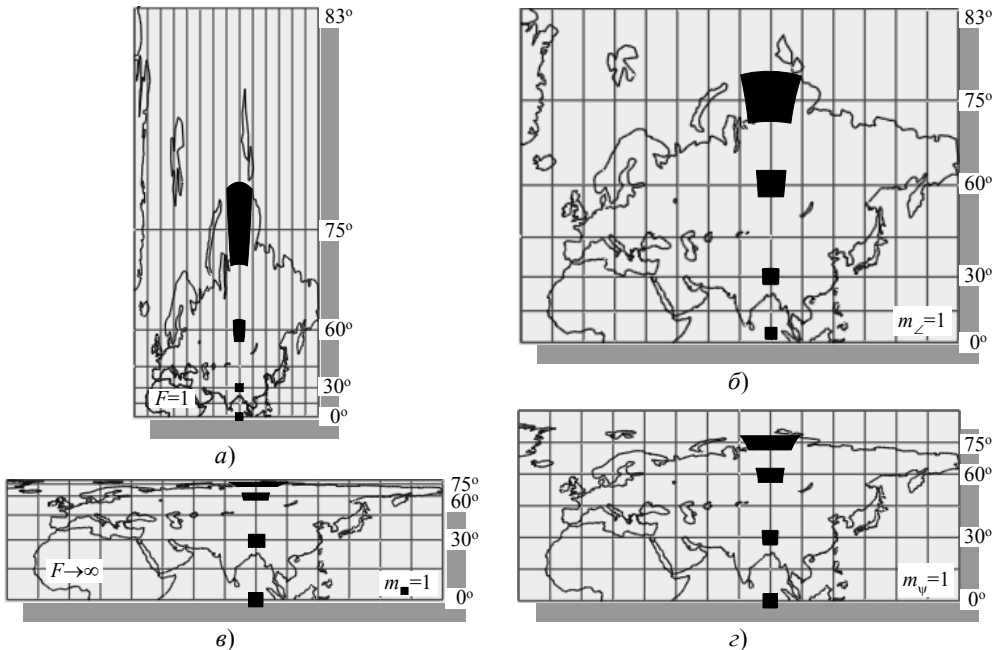


Рис. 3.87

$$m_{\varphi} = \sec(\psi), \quad m_{\psi} = \cos(\psi), \quad m_{\angle} = \cos^2(\psi), \quad \boxed{m_{\blacksquare} = 1}.$$

Эта **ортографическая** цилиндрическая проекция является **равноплощадной** (рис. 3.87,в). Её параллели сгущаются в направлении от экватора к полюсам;

- наконец, **равнопромежуточная** цилиндрическая проекция (рис. 3.87,г) с коэффициентами искажений

$$m_{\varphi} = \sec(\psi), \quad \boxed{m_{\psi} = 1}, \quad m_{\angle} = \cos(\psi), \quad m_{\blacksquare} = \sec(\psi)$$

получается применением линейного закона $h(\psi) = \psi \equiv l(\psi)$ распрямления меридиана на ось y' . Проективно она реализуется по (3.125) при **переменном** фокусном расстоянии $F(\psi, l)$. Оптимальное по среднеквадратичному критерию $J(F, l, h)$ **постоянное** значение фокуса $F_{\psi} \approx 2.8$ даёт наиболее равномерную сетку параллелей с минимальным значением критерия $J_{\min} = 34 \cdot 10^{-6}$.

□ В **косой** проекции ось цилиндра наклонена к точке сферы $p_0(\varphi_0, \psi_0)$ с широтой $\psi_0 \neq 90^\circ$. Произвольная точка сферы $p(\varphi, \psi)$ после преобразования матрицей $C_0 = \tilde{R}_y(-\varphi_0) \tilde{R}_x(\psi_0 - 90^\circ)$, переводящей точку p_0 на ось цилиндра, приобретает декартовы координаты $[x \ y \ z] = \tilde{z}^\circ \tilde{R}_x(-\psi) \tilde{R}_y(\varphi) C_0$. Вычислив по (1.36) новые угловые координаты точки $\varphi = \arg([z \ x])$ и $\psi = \arcsin(y)$, используем для построения косой цилиндрической проекции общий алгоритм (3.117).

Косая проекция используется для панорамного картографирования высокоширотных областей, включая полюсы, с существенно меньшими искажениями, чем в продольных проекциях. При задании широты $\psi_0 = 0^\circ$ сфера поворачивается на 90° , что позволяет создать **поперечную** цилиндрическую проекцию.

Конические проекции

Построение **конической** проекции точки $p(\varphi, \psi)$ осуществляется на боковую поверхность вертикально надетого на сферу **конуса** (рис. 3.88), после чего его линейчатая поверхность развёртывается на плоскость. Сетку конической карты составляют концентрические дуги параллелей и радиальные меридианы.

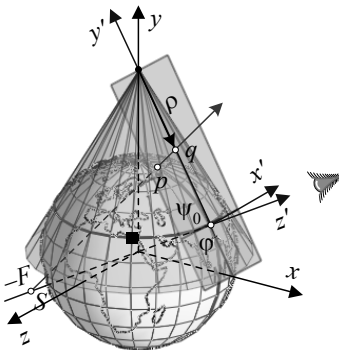


Рис. 3.88

Проективный конус касается сферы по параллели $\psi_0 \neq 0^\circ$. При $\psi_0 = 0^\circ$ конус вырождается в цилиндр и для построения карты должен использоваться алгоритм (3.123); задание $\psi_0 = \pm 90^\circ$ превращает конус в плоскость, а карту — в азимутальную полярную проекцию. На опорной параллели искажения отсутствуют, а при удалении от неё увеличиваются. Угол между проекциями двух меридианов φ_1 и φ_2 на конической развёртке равен $(\varphi_2 - \varphi_1) \sin(\psi_0)$. Измерив его, можно определить широту ψ_0 , на которой объекты изображаются без проективных искажений.

Коническая проекция применяется для картографирования протяжённых по долготе областей средних широт $20^\circ \leq |\psi^0| \leq 70^\circ$ (Россия, США, Канада, Средиземноморье), хотя и способна отобразить совместно объекты, находящиеся в разных полушариях. Привычная ориентация карты достигается заданием опорного меридиана φ_0 , который на развёртке должен располагаться вертикально.

Получим алгоритм проецирования точки p на коническую поверхность и её развёртывания на плоскость. Введём вспомогательную систему координат $x'y'z'$ с началом в точке $\{\varphi, \psi_0\}$, осями x' и y' , касательными к параллели ψ_0 и меридиану φ , и осью z' , внешне ортогональной поверхности конуса. Матрица такого преобразования системы координат равна

$$C_k(\varphi) = \tilde{R}_y(-\varphi) \tilde{R}_x(\psi_0) T(-z^0). \quad (3.126a)$$

Поместив проектор в точку $[0 \ 0 \ -F]$ на оси z' , получим по (3.110) матрицу U_f и центральную проекцию $\tilde{q} \sim \tilde{p} C_k(\varphi) U_f$, лежащую на оси y' . Развёртка точки q на плоскость $x_k y_k$, касающуюся конуса по меридиану φ_0 , выполняется переносом начала СК в вершину конуса на вектор $v = \text{ctg}(\psi_0) y^0$ и вращением точки q вокруг оси z' на угол $\gamma = (\varphi - \varphi_0) \sin(\psi_0)$. Таким образом, положение точки $p_k \equiv [x_k \ y_k \ 0]$ на конической развёртке находится следующим образом:

$$\begin{aligned} \tilde{p}_k &\sim \tilde{z}^0 \tilde{R}_x(\psi_0 - \psi) T(-z^0) U_f T(-v) \tilde{R}_z(\gamma), \\ x_k &= \left(\text{ctg}(\psi_0) - \frac{F \sin(\psi - \psi_0)}{F - 1 + \cos(\psi - \psi_0)} \right) \sin((\varphi - \varphi_0) \sin(\psi_0)), \\ y_k &= - \left(\text{ctg}(\psi_0) - \frac{F \sin(\psi - \psi_0)}{F - 1 + \cos(\psi - \psi_0)} \right) \cos((\varphi - \varphi_0) \sin(\psi_0)). \end{aligned} \quad (3.126б)$$

Полученная проекция действительна при условии

$$F - 1 + \cos(\psi - \psi_0) > 0. \quad (3.126в)$$

Положение точки p_k можно описать в полярной СК модулем ρ и углом γ , отсчитываемым от вертикальной проекции меридиана φ_0 :

$$\rho(\psi, F) = \text{ctg}(\psi_0) - \frac{F \sin(\psi - \psi_0)}{F - 1 + \cos(\psi - \psi_0)}, \quad \gamma = (\varphi - \varphi_0) \sin(\psi_0).$$

Полярные координаты преобразуются в декартовы по формулам (3.119б) с $\sigma=1$.

Коэффициенты долготных и широтных искажений, вычисленные по определениям (3.116) при **постоянном** фокусе, равны

$$\begin{aligned} m_\varphi &= \frac{\cos(\psi_0)}{\cos(\psi)} - \frac{F \sin(\psi - \psi_0) \sin(\psi_0)}{\cos(\psi) (F - 1 + \cos(\psi - \psi_0))} = \frac{\rho(\psi, F) \sin(\psi_0)}{\cos(\psi)}, \\ m_\psi &= \frac{1 + (F - 1) \cos(\psi - \psi_0)}{(F - 1 + \cos(\psi - \psi_0))^2} F = \frac{-d\rho(\psi, F)}{d\psi}. \end{aligned} \quad (3.127)$$

Исследуем влияние фокусного расстояния F на искажения конических картографических проекций и особенности расчёта некоторых из них:

• при $F=1$ получим зависимость $\rho(\psi)=\text{ctg}(\psi_0)-\text{tg}(\psi-\psi_0)$ и коэффициенты искажений $m_\psi=\sec^2(\psi-\psi_0)$, $m_\varphi=m_\angle=\sec(\psi-\psi_0)$, $m_\blacksquare=\sec^3(\psi-\psi_0)$, сильно отличные от единицы на широтах, далеких от ψ_0 . Следовательно, эта **гномоническая** коническая проекция является **произвольной**;

- из равенства $m_\varphi=m_\psi$ получим дифференциальное уравнение

$$\frac{d\rho(\psi)}{d\psi} = \frac{-\rho(\psi)\sin(\psi_0)}{\cos(\psi)}$$

с начальным условием $\rho(\psi_0)=\text{ctg}(\psi_0)$ и решением

$$\rho(\psi)=\text{ctg}(\psi_0)e^{\sin(\psi_0)(\lambda(\psi_0)-\lambda(\psi))}=\text{ctg}(\psi_0)\left(\frac{\sec(\psi_0)+\text{tg}(\psi_0)}{\sec(\psi)+\text{tg}(\psi)}\right)^{\sin(\psi_0)}\equiv r(\psi).$$

Равноугольная коническая проекция Меркатора (рис. 3.89,а) с коэффициентами искажений $[m_\angle=1]$, $m_\varphi=m_\psi=r(\psi)\sin(\psi_0)\sec(\psi)$ и $m_\blacksquare=r^2(\psi)\sin^2(\psi_0)\sec^2(\psi)$ проективно строится при **переменном** фокусном расстоянии $F(\psi,r)$, где

$$F(\psi,r)=\frac{(\text{ctg}(\psi_0)-\rho(\psi))(1-\cos(\psi-\psi_0))}{\text{ctg}(\psi_0)-\rho(\psi)-\sin(\psi-\psi_0)}; \quad (3.128)$$

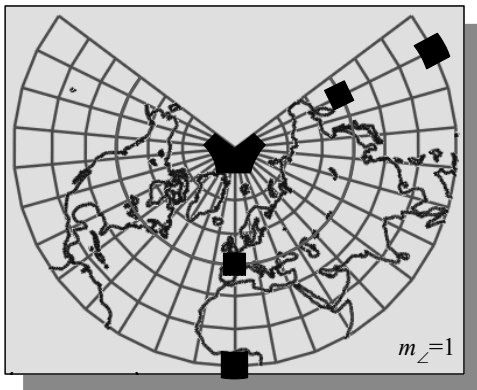
- приравнявая произведение коэффициентов m_φ и m_ψ из (3.127) к единице, получим дифференциальное уравнение

$$\rho(\psi)\frac{d\rho(\psi)}{d\psi}=\frac{-\cos(\psi)}{\sin(\psi_0)}$$

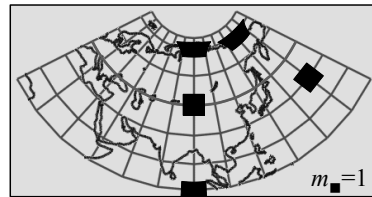
с начальным условием $\rho(\psi_0)=\text{ctg}(\psi_0)$ и решением

$$\rho(\psi)=\frac{\sqrt{1+\sin^2(\psi_0)-2\sin(\psi_0)\sin(\psi)}}{\sin(\psi_0)}\equiv s(\psi).$$

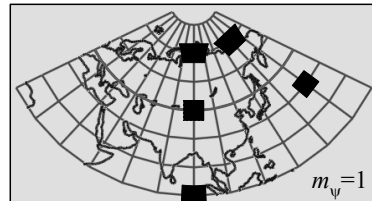
Равноплощадная коническая проекция Ламберта (рис. 3.89,б) проективно строится при **переменном** фокусном расстоянии $F(\psi,s)$, определённом в (3.128);



а)



б)



в)

Рис. 3.89

• наконец, **равнопромежуточная** коническая проекция с постоянным шагом параллелей (рис. 3.89,в) имеет коэффициенты искажений

$$m_{\psi}=1, \quad m_{\angle}=\frac{\cos(\psi)}{(\operatorname{ctg}(\psi_0)+\psi_0-\psi)\sin(\psi_0)}, \quad m_{\phi}=m_{\square}=\frac{(\operatorname{ctg}(\psi_0)+\psi_0-\psi)\sin(\psi_0)}{\cos(\psi)}$$

и строится либо по линейной формуле $\rho(\psi)=\operatorname{ctg}(\psi_0)+\psi_0-\psi \equiv l(\psi)$, либо проективным методом при **переменном** фокусном расстоянии $F(\psi, l)$.

□ В **косой** проекции ось конуса наклонена к точке сферы $p_0(\phi_0, \psi_0)$ с широтой $\psi_0 \neq 90^\circ$. Произвольная точка сферы $p(\phi, \psi)$ после преобразования матрицей $C_0 = \tilde{R}_y(-\phi_0)\tilde{R}_x(\psi_0 - 90^\circ)$, переводящей точку p_0 на ось конуса, приобретает декартовы координаты $[x \ y \ z \ 1] = \tilde{z}^0 \tilde{R}_x(-\psi) \tilde{R}_y(\phi) C_0$. Вычислив по (1.3б) новые угловые координаты точки $\phi = \arg([z \ x])$ и $\psi = \arcsin(y)$, используем для построения косой конической проекции общий алгоритм (3.126).

3.3.4. Построение сцены с подвижным наблюдателем

Далее рассмотрим методы построения сцены с **подвижным наблюдателем**, перемещающимся в трёхмерном пространстве по некоторой траектории. Это типовая задача, решаемая при разработке компьютерных игр, в которых объекты сцены изображаются в перспективной проекции «от первого лица», как их видит перемещающийся в пространстве наблюдатель. Положение объектов в МСК xyz описывается множеством точек p_i и векторов V_i , расширенные формы которых $\tilde{p}_i$ и $\tilde{V}_i$ составляют строки матрицы координат Q . Объекты могут быть как неподвижными относительно МСК (здания, рельеф местности, интерьер комнаты и др.), так и подвижными (персонажи, средства их передвижения, деревья, окна, двери, лифты и др.).

Для того чтобы составить модель сцены, представим, что перед глазами наблюдателя на фокусном расстоянии F находится прямоугольная рамка, через которую он рассматривает окружающий мир (рис. 3.90). Соотношения длин горизонтальной, вертикальной сторон рамки и фокусного расстояния $g:v:F$ определяют горизонтальный α и вертикальный β углы обзора:

$$\alpha = 2\operatorname{tg}\left(\frac{g}{2F}\right), \quad \beta = 2\operatorname{tg}\left(\frac{v}{2F}\right).$$

Среднестатистический глаз человека имеет углы обзора $\alpha \approx 150^\circ$ и $\beta \approx 110^\circ$ [25]. Отсюда следует **оптимальная пропорция** параметров рамки $g:v:F \approx 7.5:3:1$, при которой видимая область сцены заполняет всю рамку.

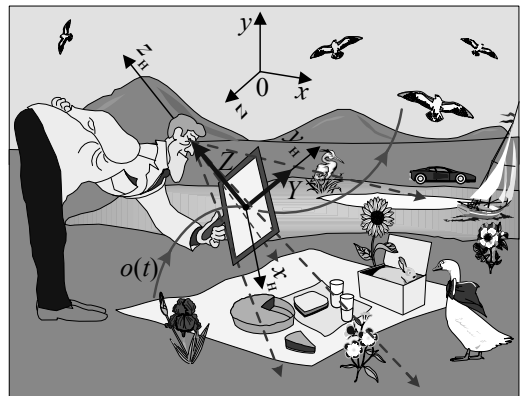


Рис. 3.90

Зададим положение рамки траекторий движения её центра $o(t)$, вектором нормали к её плоскости $Z(t)$, направленным к наблюдателю, и вектором $Y(t) \perp Z(t)$ ориентации вертикальной стороны. В центре рамки разместим начало подвижной **системы координат наблюдателя (СКН)** $x_n y_n z_n$ с ортами

$$y_n^0(t) = \bar{Y}(t), \quad z_n^0(t) = \bar{Z}(t), \quad x_n^0(t) = y_n^0(t) \times z_n^0(t). \quad (3.129)$$

С помощью аналогичной (3.26) матрицы преобразования из МСК в СКН

$$H(t) = MP3(o(t), \bar{Y}(t), \bar{Z}(t), O_3, y^0, z^0) \quad (3.130)$$

вычисляются координаты элементов сцены в СКН:

$$Q_n(t) = Q(t)H(t). \quad (3.131)$$

При выбранном расположении СКН наблюдатель находится на оси z_n в точке $S(t) = o(t) + F\bar{Z}(t)$, имеющей в СКН **постоянные** координаты $S_n = [0 \ 0 \ F]$, что упрощает уравнения (3.102б) центрального проецирования точки $[x_n(t) \ y_n(t) \ z_n(t)]$:

$$x'_n(t) = \frac{Fx_n(t)}{F - z_n(t)}, \quad y'_n(t) = \frac{Fy_n(t)}{F - z_n(t)}. \quad (3.132)$$

Проекция точки **действительна** при $z_n < F$, в противном случае точка находится сзади наблюдателя и через рамку не видна. Кроме того, невидима для наблюдателя и точка, чья действительная проекция отсечена прямоугольником рамки и не удовлетворяет **условиям видимости**

$$\{-g/2 \leq x'_n(t) \leq g/2\} \wedge \{-v/2 \leq y'_n(t) \leq v/2\}. \quad (3.133)$$

Рассмотрим подробнее особенности формирования в **реальном времени** параметров положения рамки в пространстве. При разработке программы, отображающей сцену глазами подвижного наблюдателя, определённым командам, подаваемым с клавиатуры или мышью, назначаются соответствующие движения наблюдателя и связанной с ним рамки. Например, возможна следующая **система клавишных команд управления**:

-  или  — смещение вдоль оси z_n на расстояние $-d_z$ или d_z ;
-  или  — смещение вдоль оси y_n на расстояние d_y или $-d_y$;
-  или  — смещение вдоль оси x_n на расстояние $-d_x$ или d_x ;
-  или  — поворот вокруг оси y_n на угол φ_y или $-\varphi_y$;
-  или  — поворот вокруг оси x_n на угол $-\varphi_x$ или φ_x ;
-  или  — поворот вокруг оси z_n на угол φ_z или $-\varphi_z$;
- отсутствие команды означает остановку в текущем положении.

Для бóльшего правдоподобия оси вращений **виртуальной** рамки должны проходить не через её центр, а через точку $S(t)$ расположения **действительных** глаз наблюдателя. Оси вращений при этом параллельны осям x_n , y_n и z_n . В самом общем виде движение рамки описывается в СКН универсальной матрицей преобразования

$$C_n = T(-S_n) \tilde{R}_x(\pm\varphi_x) \tilde{R}_y(\pm\varphi_y) \tilde{R}_z(\pm\varphi_z) T(S_n) T([\pm d_x \ \pm d_y \ \pm d_z]) \quad (3.134)$$

с соответствующими каждой команде значениями и знаками углов вращений и координатных смещений. Во избежание бесполезных умножений на единичные матрицы можно каждой конкретной команде сопоставить более простую, чем (3.134), матрицу АП. Например, в приведённой выше системе команд клавише $\boxed{\uparrow}$ соответствует матрица $C_n = T(-d_z z^0)$, а команда $\boxed{\leftarrow}$ обрабатывается матрицей $C_n = T(-S_n) \tilde{R}_y(\varphi_y) T(S_n)$.

Важная проблема при разработке программ с подвижными элементами — **обнаружение и обработка столкновений** объектов сцены с наблюдателем и друг с другом. Столкновение наблюдателя с каким-либо объектом можно обнаружить рассмотренными в п. 1.5.6 тестами локализации точки $S(t)$ внутри этого объекта. Столкновение объектов обнаруживается тестом *coll3_test* (см. рис. 1.132) по пересечению (а) либо касанию (б) любого ребра одного объекта с любой гранью другого.

Поведение наблюдателя и объектов при столкновении определяется их упругими свойствами. Например, при «встрече» со стеной наблюдатель должен остановиться либо упруго от неё отскочить. Остановка означает отмену команды управления, приведшей к столкновению. Упругий отскок по законам отражения требует расчёта соответствующей команды. Аналогично обрабатываются столкновения объектов друг с другом. Ниже приводятся **алгоритмы с остановкой** при столкновении наблюдателя с объектами сцены.

Пусть:

- t_k , где $k=0, 1, 2, \dots$, — вырабатываемые по сигналам таймера ЭВМ **моменты опроса** устройств — источников команд управления;
- o_0 , Z_0 и Y_0 — параметры начального положения рамки, дающие матрицу (3.130) пересчёта координат из МСК в СКН:

$$H_0 = MP3(o_0, \bar{Y}_0, \bar{Z}_0, O_3, y^0, z^0);$$

- Q_0 — матрица начальных координат элементов сцены.

Циклический **алгоритм расчёта перспективного изображения динамической сцены** в зависимости от команд управления можно реализовать **объективным** и **субъективным** методами, отличающимися тем, какая СК принимается неподвижной (см. рис. 3.90). В обоих методах на k -м шаге движения вычисляются матрицы C_{nk} и $C_k^{<m>} \forall m=\overline{1, M}$ собственных движений рамки в СКН и M объектов в МСК (у неподвижных элементов сцены матрицы движений единичные). Матрицы H_k и H_{k+1} на соседних шагах моделирования связаны соотношением $H_{k+1} = H_k C_{nk}^{-1}$.

В объективно **неподвижной МСК** хуэ движения наблюдателя и объектов рассчитываются независимо друг от друга. Проецирование выполняется на подвижную картинную плоскость, поэтому для **визуализации** сцены (**отсечения** проекций объектов контуром рамки, **удаления** невидимых элементов, **окрашивания** видимых поверхностей) требуется преобразование координат всех объектов из МСК в СКН. На k -м шаге алгоритма моделирования вычисляются:

- по (3.131) матрица координат объектов в СКН $Q_{nk}=Q_k H_k$, центральные проекции объектов и визуализация сцены на экране дисплея;
- по (3.134) матрица C_{nk} и матрица движения наблюдателя в МСК

$$C_k = H_k C_{nk} H_k^{-1}; \quad (3.135a)$$

- прогноз параметров рамки и наблюдателя на $k+1$ -м шаге:

$$\tilde{o}_{k+1} = \tilde{o}_k C_k, \quad \tilde{Z}_{k+1} = \tilde{Z}_k C_k, \quad S_{k+1} = o_{k+1} + F \tilde{Z}_{k+1}, \quad H_{k+1} = H_k C_{nk}^{-1}; \quad (3.135b)$$

- прогноз координат объектов сцены на $k+1$ -м шаге:

$$Q_{k+1}^{<m>} = Q_k^{<m>} C_k^{<m>} \quad \forall m = \overline{1, M}; \quad (3.135b)$$

- если точка S_{k+1} принадлежит хотя бы одному из $m \in [1, M]$ объектов, то выполняется остановка:

$$o_{k+1} = o_k, \quad Z_{k+1} = Z_k, \quad Q_{k+1}^{<m>} = Q_k^{<m>}, \quad H_{k+1} = H_k. \quad (3.135r)$$

Субъективно (с точки зрения наблюдателя) полагая **неподвижной СКН** $x_n y_n z_n$, мы должны перед циклом моделирования пересчитать начальные координаты всех объектов сцены из МСК в СКН по формуле группового преобразования $Q_{n,0} = Q_0 H_0$. Движение объекта в СКН есть суперпозиция двух движений: собственного и относительного, **обратного** движению рамки. Проецирование сцены выполняется на **неподвижную** фронтальную плоскость рамки. На k -м шаге алгоритма моделирования вычисляются:

- центральные проекции объектов и визуализация сцены на экране дисплея;
- по (3.134) матрица C_{nk} и матрицы собственных движений объектов в СКН:

$$C_{nk}^{<m>} = H_k^{-1} C_k^{<m>} H_k \quad \forall m = \overline{1, M}; \quad (3.136a)$$

- прогноз координат объектов сцены на $k+1$ -м шаге:

$$Q_{n,k+1}^{<m>} = Q_{nk}^{<m>} C_{nk}^{<m>} C_{nk}^{-1} \quad \forall m = \overline{1, M}, \quad H_{k+1} = H_k C_{nk}^{-1}; \quad (3.136b)$$

- если точка $S_n = [0 \ 0 \ F]$ принадлежит хотя бы одному из $m \in [1, M]$ объектов с координатами $Q_{n,k+1}^{<m>}$, то выполняется остановка:

$$Q_{n,k+1}^{<m>} = Q_{nk}^{<m>}, \quad H_{k+1} = H_k. \quad (3.136b)$$

Сравнительный анализ двух методов построения сцены с подвижным наблюдателем свидетельствует о несколько **меньшей трудоёмкости субъективного подхода**, в котором все расчёты ведутся в системе координат, жёстко связанной с наблюдателем. С учётом направленности её оси z_n навстречу наблюдателю и выраженности всех координат сцены относительно картинной плоскости субъективный метод идеален для реализации алгоритмов визуализации как на программном, так и на аппаратном уровнях.

Поверхность (*surface, side*) — это традиционное название двумерного многообразия $\mathfrak{X}$ в трёхмерном пространстве R^3 , в точках которого происходит *качественное* изменение геометрических, механических и оптических свойств контактирующих сред. Благодаря этим свойствам становится возможным физическое взаимодействие объектов друг с другом и с окружающим светом, что, собственно, обуславливает их ограниченность и видимость.

Как математический объект, описываемый неявным уравнением $f(p)=0$, поверхность разделяет подпространства с противоположными знаками функции $f(p)$, использование которой для поиска точек $p \in \mathfrak{X}$ часто затруднено отсутствием явного решения неявного уравнения относительно любой из координат.

Параметрическое описание поверхности векторной функцией $p(t, \tau)$ двух переменных t и τ , имеющих определённый геометрический смысл, позволяет явно и просто найти местоположение точки p на объекте $\mathfrak{X}$, представляющем собой *оболочку* наподобие бесконечно тонкой плёнки (см. рис. 1.142, 3.77). Благодаря отличию механических и оптических свойств оболочки (шероховатость, не полная прозрачность, избирательная отражательность) от свойств окружающей среды несветящаяся поверхность становится видимой в отражённых ею лучах. Важным свойством незамкнутой поверхности-оболочки является видимость у неё (естественно, в разных местах) обеих сторон.

Иная концепция поверхности (буквально — того, что лежит поверху) полагает этот геометрический объект краем *сплошного* тела, граничащим с внешней средой. Такая поверхность замкнута и, будучи непрозрачной, всегда видна односторонне (см. рис. 1.159, 3.17), даже будучи разрезанной, когда в плоскости разреза появляются новые грани (см. рис. 1.171).

Модели поверхностей играют важную роль в конструировании, производстве и изображении объёмных фигур. Приведём лишь несколько примеров востребованности математических моделей поверхностей:

- автоматизированное проектирование деталей, корпусов и сборных конструкций для широкого круга изделий машиностроения;
- в механообработке — разработка программ точной обработки поверхностей для станков с числовым управлением;
- в лёгкой промышленности — дизайн мебели, посуды, обуви, одежды;
- в архитектурном и ландшафтном проектировании зданий, дорог, рельефов;
- визуализация экспериментальных данных о границах раздела сред в естественных науках — медицине (рентгеновская, ультразвуковая и магниторезонансная съёмка, компьютерная томография), геологии и локации (построение рельефа поверхности Земли, морского дна, космических объектов).

4.1. КЛАССИФИКАЦИЯ ПОВЕРХНОСТЕЙ

Из всех аспектов изучения поверхностей наибольший интерес, на наш взгляд, представляют две темы:

- методы **конструирования** математических моделей поверхностей. Целью составления модели является достоверное воспроизведение формы реального объекта в рамках допустимых погрешностей;
- способы реалистичного **изображения** поверхностей на плоском носителе, к которому относится и экран дисплея.

В соответствии с этими категориями на рис. 4.1 представлена укрупнённая **схема классификации поверхностей**, раскрытие которой составляет содержание данной главы.



Рис. 4.1

Кратко расшифруем условную **конструктивную классификацию** математических моделей поверхностей (подробное изучение моделей производится в остальных разделах главы):

- **кинематические** поверхности создаются в результате движения в пространстве одних линий, называемых **образующими**, вдоль других линий, называемых **направляющими**. У некоторых кинематических поверхностей вместо направляющих линий могут быть оси вращения;
- **полиномиальные** поверхности описываются векторными функциями двух параметров с разнообразными свойствами близости к множеству опорных данных — узловых точек, векторов и линий;
- **составные** поверхности образуются путём соединения **сегментов**, обладающих индивидуальными математическими моделями, с соблюдением желаемых свойств непрерывности, гладкости сопряжения и краевых условий;
- **воксельные** поверхности (**облака точек**) формируются как негладкие или разрывные оболочки в виде тонкого слоя плотно упакованных мельчайших объёмных пикселей — **вокселей** (*VOLume piXEL*).

Классификация кинематических поверхностей показана на рис. 4.2:

- в первой группе собраны **аффинные** модели поверхностей, полученных элементарными геометрическими преобразованиями образующих линий, реализующими разные **виды движения** — перенос, вращение, масштабирование;
- вторую группу **линейчатых** составляют поверхности, образованные движением **прямых** образующих линий. Разнообразие линейчатых поверхностей обусловлено их возможностью развёртываться на плоскость и способами движения образующей прямой по направляющим линиям;
- в третьей группе **нелинейных** собрано многообразие поверхностей, заметаемых движением кривых образующих линий либо описываемых нелинейными неполиномиальными моделями.

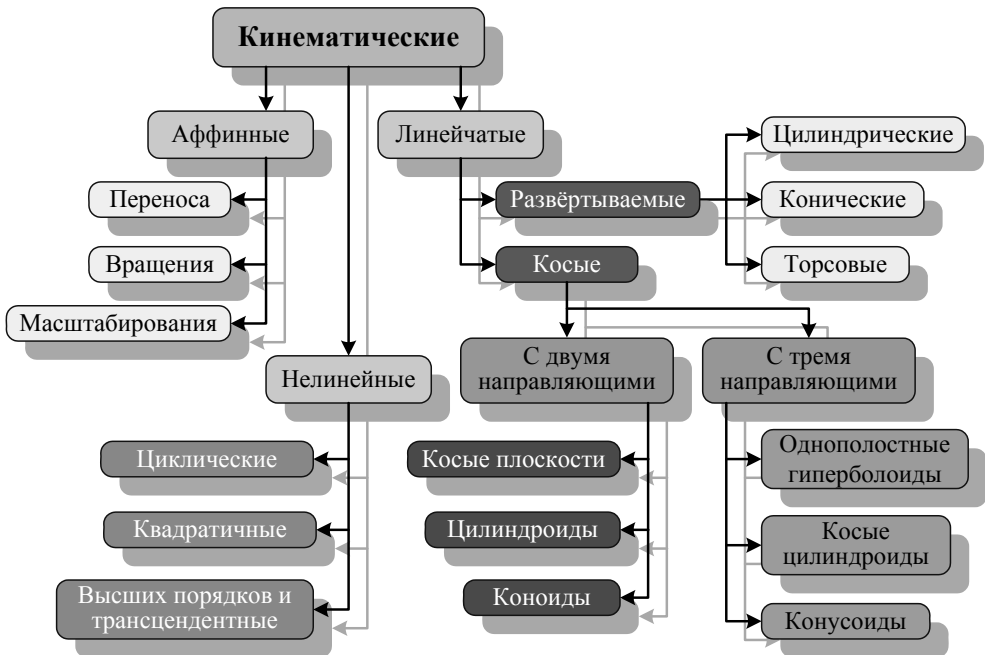


Рис. 4.2

Классификация полиномиальных поверхностей дана на рис. 4.3:

- **интерполяционные** поверхности удовлетворяют системе граничных, а в общем случае и внутренних, элементов характеристической сети, которыми могут быть узловые точки, векторы и линии;
- **аппроксимационные** поверхности проходят в некоторой окрестности заданных узловых элементов и строятся обычным либо взвешенным методами наименьших квадратов с целью минимизации межузловых осцилляций;
- **поверхности Безье** сочетают интерполяционные свойства в угловых точках характеристической сети и приближающие свойства в остальных узлах;
- **основные** поверхности строятся на основе одного либо двух наборов опорных линий, составляющих **базис** конструкции, методами интерполяции, Эрмита либо Безье.

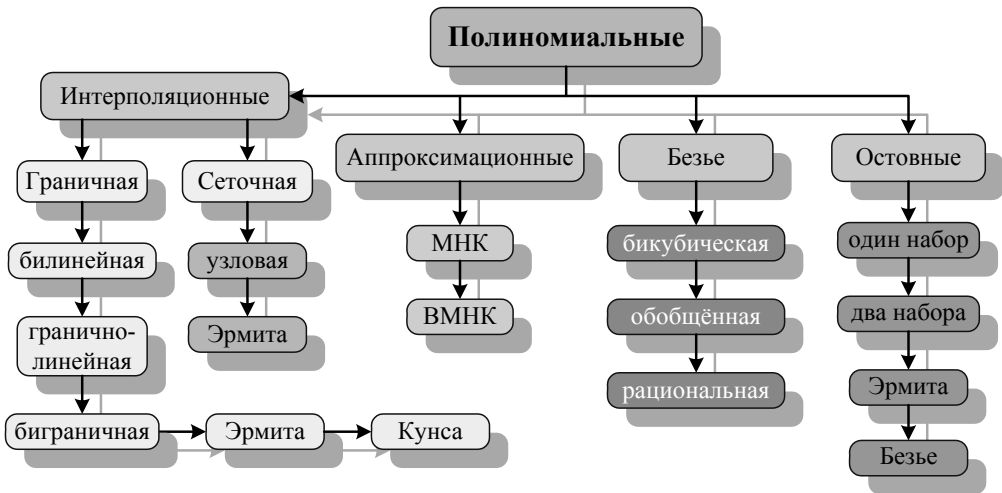


Рис. 4.3

Классификация составных поверхностей, представленная на рис. 4.4, осуществляется по следующим факторам:

- по типу *моделей сегментов* из отмеченных на рис. 4.3;
- по *схемам сопряжения*, отличающимся составом и порядком учёта условий гладкости сшивки сегментов;
- *краевыми условиями* в граничных узлах крайних сегментов.

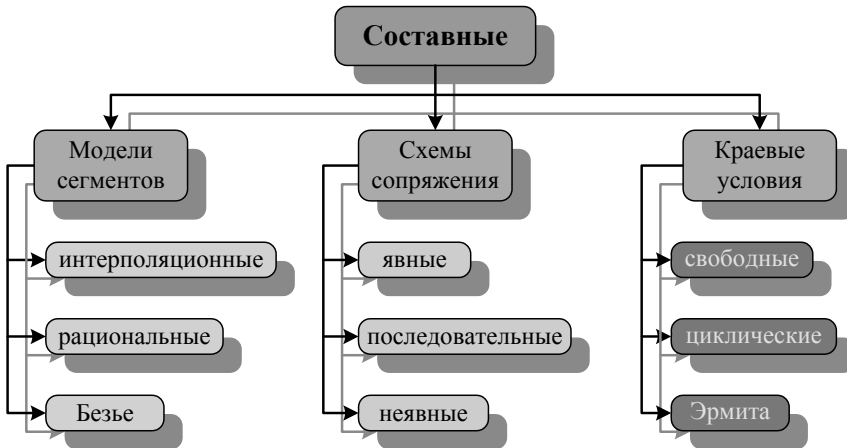


Рис. 4.4

Нелишне напомнить приверженцам разных устоев, что приведённые схемы классификаций поверхностей составлены на основании личного мнения автора так, как он видит логическую взаимосвязь отдельных факторов.

Логично начать углублённое изучение поверхностей со способов их изображения, чтобы потом можно было иллюстрировать конструктивные методы построения моделей в наглядной графической форме.

4.2. МЕТОДЫ ИЗОБРАЖЕНИЯ ПОВЕРХНОСТЕЙ

Изображение является мощным графическим средством зрительного восприятия геометрической информации. Далеко не каждый человек может построить в своём воображении объёмную фигуру по её словесному описанию либо уравнениям математической модели. Основная задача компьютерной графики, собственно, и состоит в преобразовании геометрических моделей в наглядные визуальные формы, реалистично отображающие строение объектов и их взаимное расположение в пространстве.

В задаче изображения поверхности возникает множество вопросов:

- как выбрать удачный ракурс, раскрывающий в плоском изображении особенности её строения?
- какими изобразительными средствами можно достоверно передать форму поверхности?
- как закрасить её изображение, наиболее точно подражая игре света и тени?

Окраска *криволинейной* поверхности должна выполняться *поточечно*, т. к. интенсивность света, отражённого от каждой её точки p в направлении наблюдателя, заданном вектором H , зависит от ряда факторов [20, 36, 65]:

- *собственного цвета* поверхности, заданного RGB-вектором $e = [e_r \ e_g \ e_b]$;
- ориентации вектора нормали $N(p)$, вычисляемого по (1.93), (1.94), (1.144);
- *фонового освещения* интенсивности $I_\phi = [I_{\phi r} \ I_{\phi g} \ I_{\phi b}]$ с коэффициентом фонового отражения $k_\phi \in [0, 1]$. Диффузное отражение рассеянного освещения по **закону Ламберта** пропорционально косинусу *угла видимости* $\psi = \angle(N, H)$, равному $\overline{H \circ N}$, а для повышения реалистичности градиентной закраски используется степенная зависимость $(\overline{H \circ N})^v$ с показателем $v \leq 1$;

• *направленного освещения* от n_i источников интенсивностей I_i , $i \in [1, n_i]$, чьи лучи приходят из точек L_i в точку p в направлениях $V_i = p - L_i$ под *углами падения* $\alpha_i = \angle(N, -V_i)$. Отражение направленных лучей бывает:

◇ *диффузным*, пропорциональным $(\overline{H \circ N})^v$, с уменьшением освещённости также по закону Ламберта в $-\overline{V_i \circ N}$ раз с коэффициентом $k_{\text{нд}} \in [0, 1]$;

◇ *зеркальным*, пропорциональным косинусам *углов бокового отражения* $\beta_i = \angle(M_i, H)$, где $M_i = V_i - 2(V_i \circ \overline{N})\overline{N}$, возведённым в степень $\mu \gg 1$, обостряющую резкость зеркальных бликов, с учётом коэффициента $k_{\text{нз}} \in [0, 1]$;

- коэффициента $\gamma(d) = 1 / (1 + k_1 d + k_2 d^2)$, уменьшающим интенсивность луча *точечного* источника в зависимости от пройденного им расстояния d .

С учётом перечисленных факторов видимый цвет точки поверхности формируется по формуле, называемой *функцией закраски Фонга (ФЗФ)*:

$$c = k_\phi |\overline{H \circ N}|^v e \bullet I_\phi + \sum_{\substack{\forall i: (H \circ N)(\overline{V_i \circ N}) < 0}} (k_{\text{нд}} |\overline{V_i \circ N}| |\overline{H \circ N}|^v e \bullet I_i + k_{\text{нз}} (\overline{M_i \circ H})^\mu I_i) \gamma(|V_i|). \quad (4.1)$$

4.2.1. Выбор проекции изображения

Выбор проекции изображения объекта, кратко называемой **ракурсом**, осуществляется из арсенала методов параллельного либо центрального проецирования, подробно рассмотренных в разд. 3.2. Как правило, конструируемая поверхность ориентируется своими осями симметрии параллельно координатным осям, поэтому её **ортографическая** проекция на координатную плоскость может выглядеть невыразительно (рис. 4.5,а). Для большей наглядности рекомендуется повернуть поверхность в пространстве так, чтобы были видны характерные детали её рельефа, а выпуклости как можно меньше заслоняли впадины. Такой ракурс достигается углами вертикального разворота φ_y и горизонтального наклона φ_x в интервале $20^\circ - 60^\circ$ (**триметрическая** проекция). Большинство иллюстраций в этой книге выполнено в стандартных проекциях, хорошо передающих глубину объекта в пространстве: **диметрической** с углами $\varphi_y \approx -20^\circ$, $\varphi_x \approx 20^\circ$ (рис. 4.5,б) и **изометрической** с углами $\varphi_y = -45^\circ$, $\varphi_x \approx 35^\circ$ (рис. 4.5,в). Углы наклона φ_x , близкие к 0° и 90° , делают изображение плоским, со скрытыми важными деталями рельефа (рис. 4.5,г).

Наиболее правдоподобно выглядит **центральная** (перспективная) проекция объекта, предварительно повернутого в пространстве на диметрические либо изометрические углы. Это более сложное, чем параллельное, проецирование содержит дополнительную операцию (3.101) приведения однородных координат точек к декартовой форме. Близость объекта к наблюдателю, собственно, и создаёт эффект перспективы (см. рис. 3.64). Особенно он заметен на расстоянии, сравнимом с размерами объекта. При удалении наблюдателя от сцены все проецирующие лучи становятся практически параллельными. Из-за этого размеры

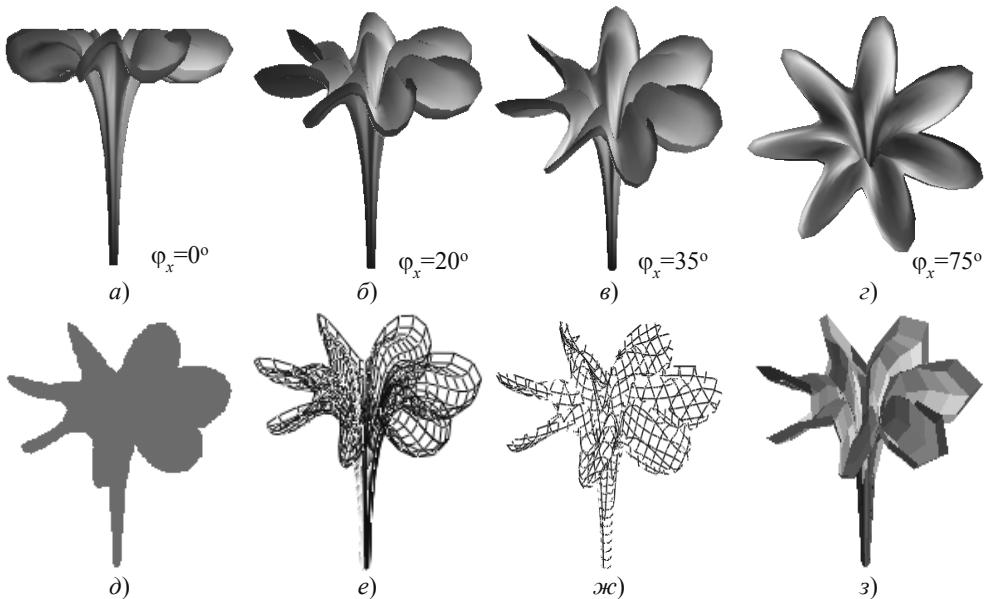


Рис. 4.5

проекций не зависят от приближения или удаления объектов (см. рис. 3.59), а смысл и эффект центрального проецирования теряется.

Таким образом, перспективная проекция рекомендуется для изображения протяжённых в глубину сцены статичных объектов и динамичных сцен, в которых наблюдатель и объекты перемещаются по глубине взгляда.

Во многих приложениях и компьютерных играх применяется показ сцены не в какой-то фиксированной проекции, предполагающей статичное положение наблюдателя, а в свободном ракурсе с подвижным наблюдателем, способным изменять как своё положение в пространстве, так и направление взгляда (см. п. 3.3.4). Программирование таких приложений требует хорошего понимания взаимосвязи различных систем координат, преобразований из одной СК в другую и свободного владения методами изменения ракурса с помощью разных устройств ввода.

Переходим к изучению *методов изображения поверхностей*. Не принимая всерьёз совершенно не информативное решение (рис. 4.5,д) выводить поверхность монохромно закрашенным пятном абриса (это годится лишь для плоских полигонов и построения теней), выделим три наиболее распространённых в компьютерной графике способа отображения рельефа не плоских поверхностей: *каркасный* (см. рис. 1.158, 3.74, 4.5,е,ж), *полигональный* (см. рис. 1.112, 1.114, 1.120, 4.5,з) и *точечный* (см. рис. 1.103,а, 1.144, 3.37, 3.38, 4.5,а – з). Также применяют комбинирование этих методов: например, для лучшей передачи формы поверхности каркасную сеть или рёбра полиэдров выводят поверх её точечного изображения (см. рис. 1.142, 3.17, 3.45 – 3.47, 3.77).

4.2.2. Каркасные сети

Многие прикладные программы 3d-графики для ускорения изображения сложной поверхности рисуют её на экране не полностью, а эскизно, ограничиваясь небольшим числом точек и линий, способных реалистично передать характер искривления поверхности в пространстве. Совокупность таких условных либо реально существующих линий составляет *каркасную сеть*, охватывающую поверхность наподобие проволочной сетки (*wireframe*). Минимальные контуры каркасной сети образуют *ячейки* (*cell*), граничащие друг с другом по *рёбрам* (*edge*), соединяющимся в *вершинах* (*vertex*).

В зависимости от модели поверхности строятся соответствующие каркасные линии, выводимые на экран дисплея в желаемом ракурсе.

□ Каркас *параметрической* поверхности $p(t, \tau)$ образуется системой *параметрических линий*, условно называемых *продольными* $p(t, \tau_j)$ и *поперечными* $p(t_i, \tau)$. Алгоритм построения каркасной поверхности очень прост.

📁 **Шаг 1.** Задаются целые числа n и m , определяющие размер *индексной сетки* $\{i, j\}$ целочисленных переменных $i=0, \overline{n}$, $j=0, \overline{m}$ (рис. 4.6,а).

Шаг 2. Выполняется квантование независимых параметров $t=t_0, t_1, \dots, t_n$ и $\tau=\tau_0, \tau_1, \dots, \tau_m$, составляющих прямоугольную *параметрическую сетку* $\{t_i, \tau_j\}$.

Часто ограничиваются *равномерной* сеткой с постоянными квантами параметров $\Delta t = t_{i+1} - t_i = \text{const} \forall i$ и $\Delta \tau = \tau_{j+1} - \tau_j = \text{const} \forall j$ (рис. 4.6,б).

Шаг 3. Вычисляются узловые точки *каркасной сетки* $u_{ij} = p(t_i, \tau_j) \in R^3$, которые соединяются отрезками прямых линий $u_{ij}u_{i+1,j}$ и $u_{ij}u_{i,j+1}$. Так создаются пучки *каркасных полилиний*: *продольных* $\{u_{0j}u_{1j} \dots u_{nj}, \forall j = \overline{0, m}\}$ и *поперечных* $\{u_{i0}u_{i1} \dots u_{im}, \forall i = \overline{0, n}\}$ (рис. 4.6,в).

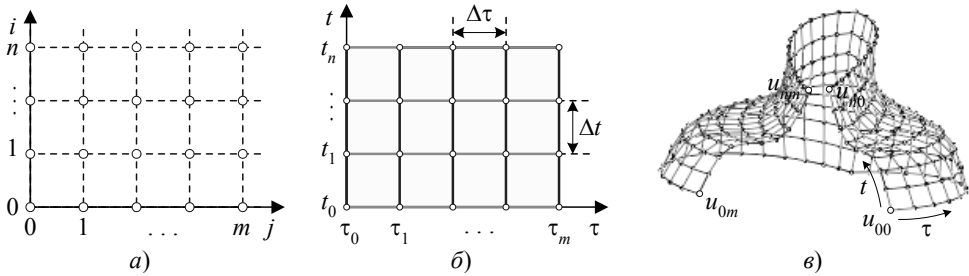


Рис. 4.6

Ячейки каркасных поверхностей бывают:

- прозрачными и непрозрачными;
- объёмными и плоскими. Последние будем называть *гранями*;
- с прямолинейными и криволинейными рёбрами.

Для показа *прозрачного* каркаса достаточно вывести на экран все его линии в произвольном порядке. **Недостаток** такой визуализации поверхности заключается в неоднозначности восприятия её положения в пространстве из-за полной видимости всех каркасных линий независимо от их удаления от наблюдателя (см. рис. 1.176,а). Для большей реалистичности можно дальние рёбра рисовать тоньше ближних либо пунктирно (см. рис. 1.114), но лучше совсем не выводить на экран всё, что экранировано от наблюдателя более близкими бесцветными ячейками (см. рис. 1.176,б,в).

Изображение *непрозрачного* каркаса составляют только ячейки, видимые наблюдателем. Для их определения необходимо выполнить процедуру *удаления* невидимых графических элементов или их фрагментов, экранированных другими элементами (см. разд. 1.7). Видимые фрагменты криволинейных ячеек выводятся на экран как точечные поверхности, а фрагменты плоских граней для ускорения вывода могут изображаться как полигональные.

Важный момент построения каркасных моделей — разумный выбор частоты квантования параметров, определяющей размеры ячеек и плотность наложения каркасных линий. Слишком плотно расположенные каркасные линии сливаются друг с другом и скрывают форму и цвет поверхности, как в центре и на стебле цветка на рис. 4.5,е. Наоборот, малое число крупных ячеек делает изображение криволинейной поверхности грубым и угловатым (см. рис. 4.5,з).

Компромиссным решением является создание в k раз более частой *субсетки*, показанной штрихом на рис. 4.7,а. Каркасные полилинии строятся с обыч-

ным шагом одного параметра и k -кратным шагом выборки вершин в направлении другого параметра:

$$p(t_i, \tau_j \forall j = \overline{0, 1, \dots, km}) \quad \forall i = \overline{0, k, \dots, kn}; \quad p(t_i \forall i = \overline{0, 1, \dots, kn}, \tau_j) \quad \forall j = \overline{0, k, \dots, km}.$$

Теперь ячейки будут выглядеть крупными, объёмными и с криволинейными границами (рис. 4.7,б). Реальные же, в k раз более мелкие, ячейки субсетки у прозрачного каркаса не выводятся, а у непрозрачного выводятся без окантовки, что позволяет лучше передать форму криволинейной поверхности (рис. 4.7,в). Субсетку легко отменить, задав коэффициент дробления $k=1$.

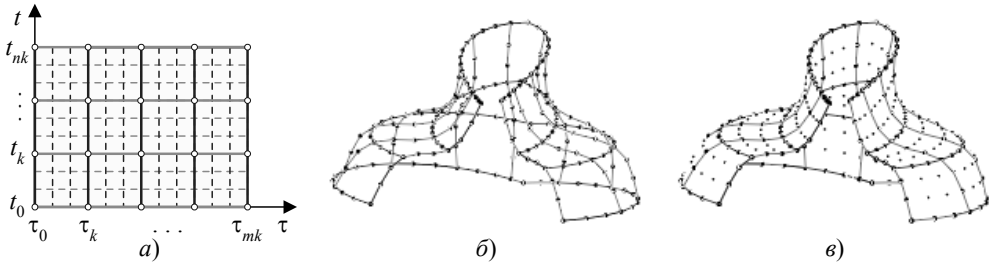


Рис. 4.7

□ Каркас **явной** поверхности $f(x, y, z) = 0$ образуется системой N **линий уровня (изолиний)** при фиксированных значениях $x = x_{yj}$, $y = y_{yj}$ или $z = z_{yk}$ (рис. 4.5,ж). Порядок расчёта изолиний, например, высот $y = y_{yj}$ следующий.

📁 **Шаг 1.** Задаются N значений уровней высоты $y_{yj} \quad \forall j = \overline{1, N}$.

Шаг 2. Рассчитываются сечения поверхности горизонтальными плоскостями, параллельными плоскости xz , путём N -кратного решения нелинейного уравнения $f(x, y_{yj}, z) = 0$ для $j \in [1, N]$. Если же поверхность допускает явное описание $x(y, z)$, то изолиния y_{yj} становится аналитической однопараметрической функцией $p(z) = [x(y_{yj}, z) \quad y_{yj} \quad z]$. В случае явного описания $z(x, y)$ изолинией является параметрическая функция $p(x) = [x \quad y_{yj} \quad z(x, y_{yj})]$. □

Изолинии широко применяются в метеорологии и картографии для построения плоских карт линий равных температур или высот некоторого рельефа (см. рис. 4.8,а). На численное решение этой задачи (рис. 4.8,б) ориентирован эффективный **метод маркированных квадратов** [65], суть которого такова.

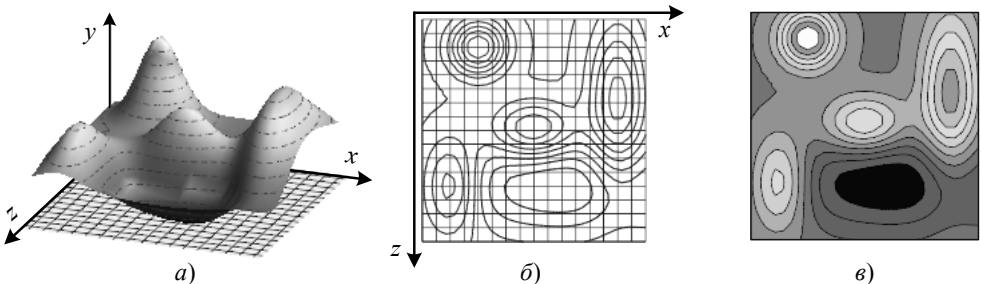


Рис. 4.8

Шаг 1. Задаётся набор N высот $y_{yj} \forall j=\overline{1, N}$, для каждой из которых будет строиться сечение поверхности $y=f(x, z)$ плоскостью $y=y_{yj}$.

Шаг 2. На плоскости xz создаётся прямоугольная $m \times n$ -сетка, в узлах которой $\{x_i, z_k\}$ рассчитываются высоты $y_{ki}=f(x_i, z_k) \forall i=\overline{1, n}, k=\overline{1, m}$.

Шаг 3. Изолинии j -го уровня состоят из отрезков, по которым каждая (k, i) -я ячейка поверхности с высотами вершин $\{y_{ki}, y_{k+1, i}, y_{k, i+1}, y_{k+1, i+1}\}$ пересекается с плоскостью $y=y_{yj}$. Обозначив вершины ячейки с высотами $y_{ki} \geq y_{yj}$ белым цветом, а с высотами $y_{ki} < y_{yj}$ — чёрным, проведём отрезки между рёбрами ячейки с разноцветными вершинами. Возможны 16 вариантов маркировки вершин ячейки и проведения в ней отрезков изолинии (рис. 4.9):

- в двух вариантах (а) все вершины ячейки окрашены одним цветом (белым либо чёрным), вследствие чего её пересечение с плоскостью уровня и отрезки пересечения *отсутствуют*;
- в восьми вариантах (б) цвет одной вершины отличен от цвета остальных вершин, что даёт *один* отрезок между рёбрами, инцидентными этой вершине;
- в четырёх вариантах (в) проводится *один* отрезок между рёбрами, соединяющими две разноцветные пары одноцветных смежных вершин;
- в двух вариантах с противоположными вершинами одного цвета существует неоднозначность проведения отрезков. Для её преодоления вычислим высоту центра ячейки $y_{ki\pi} = f((x_i + x_{i+1})/2, (z_k + z_{k+1})/2)$. Тогда *два* отрезка при $y_{ki\pi} \geq y_{yj}$ проводятся по шаблону (з), а при $y_{ki\pi} < y_{yj}$ — по шаблону (д).

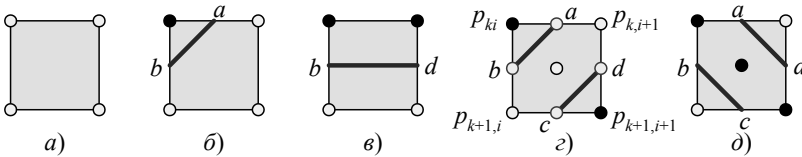


Рис. 4.9

При достаточно малых размерах ячеек концы отрезков изолиний можно фиксировать в серединах рёбер:

$$a = \frac{p_{ki} + p_{k, i+1}}{2}, \quad b = \frac{p_{ki} + p_{k+1, i}}{2}, \quad c = \frac{p_{k+1, i} + p_{k+1, i+1}}{2}, \quad d = \frac{p_{k, i+1} + p_{k+1, i+1}}{2}.$$

Точность построения изолиний на крупной сетке можно повысить путём установки концов отрезков методом *дихотомии* — деления рёбер ячейки пропорционально отклонениям высот их концевых вершин от текущего уровня y_{yj} :

$$a = \text{lin} \left(p_{ki}, p_{k, i+1}, \frac{y_{yj} - y_{ki}}{y_{k, i+1} - y_{ki}} \right), \quad b = \text{lin} \left(p_{ki}, p_{k+1, i}, \frac{y_{yj} - y_{ki}}{y_{k+1, i} - y_{ki}} \right),$$

$$c = \text{lin} \left(p_{k+1, i}, p_{k+1, i+1}, \frac{y_{yj} - y_{k+1, i}}{y_{k+1, i+1} - y_{k+1, i}} \right), \quad d = \text{lin} \left(p_{k, i+1}, p_{k+1, i+1}, \frac{y_{yj} - y_{k, i+1}}{y_{k+1, i+1} - y_{k, i+1}} \right).$$



Для построения карты с закраской межуровневых областей (см. рис. 4.8,в) нужно из полученных отрезков каждого уровня собрать замкнутые контуры (это можно сделать с помощью алгоритма типа *cont* (см. рис. 1.71)) и вывести их с заливкой желаемыми тонами в порядке возрастания уровней, реализуя метод художника. Среди наиболее популярных **цветовых палитр** отметим:

- **топографическую**, в которой цвета распределяются по высоте как на географической карте — глубины ниже уровня моря закрашиваются фиолетовым, синим и голубым, равнины — оттенками зелёного, возвышенности — жёлтого, горы — коричневого, а снежные шапки — светло-серым и белым цветами;
- **радужную**, цвета которой распределяются в желаемом, например, в вертикальном направлении по цветам радуги — от фиолетового до красного.

4.2.3. Полигональные сети

Непрозрачный каркас с однотонно закрашенными ячейками (см. рис. 4.7,в) не даёт того полного представления о форме криволинейной поверхности, которое создаётся методом точечной закраски с удачно поставленным освещением даже без вывода каркасных линий (рис. 4.10,а). Однако, этот самый реалистичный метод визуализации требует и самых больших вычислительных ресурсов.

Определённым компромиссом между скоростью и качеством вывода непрозрачной криволинейной поверхности является её приближение так называемой **полигональной сетью** (*polygonal mesh*) (**ПС**), составленной из ячеек четырёхугольной или треугольной формы (рис. 4.10,б,в), визуализируемых методами, рассмотренными в п. 1.7.2. В зависимости от упорядоченности ячеек полигональные сети делятся на две группы:

- к **регулярным** (структурированным) отнесём ПС, в которых смежные узлы и ячейки пронумерованы **последовательно** в линейном либо матричном порядках. Образцом регулярной сети служит аппроксимация параметрической поверхности $p(t, \tau)$ системой ячеек $U_{ij} = \{u_{i-1,j-1}u_{i-1,j}u_{i,j}u_{i,j-1}\}$ с узловыми вершинами $u_{ij} = p(t_i, \tau_j)$, лежащими на поверхности (рис. 4.10,б). При визуализации ячеек методом *z*-буфера (см. рис. 1.185) организуется построчная ($i = \overline{1, n}$) выборка клеток индексной и параметрической сеток (см. рис. 4.6) с инкрементным ($j = \overline{1, m}$) наращиванием номера клетки в строке. Такая регулярность выборки устраняет двойной расчёт вершин смежных ячеек и ускоряет визуализацию;
- в **нерегулярных** (неструктурированных) сетях последовательная нумерация **всех** смежных элементов невозможна (рис. 4.10,в), поэтому в целом она является **свободной**. Такими ПС аппроксимируются сложные поверхности с произ-

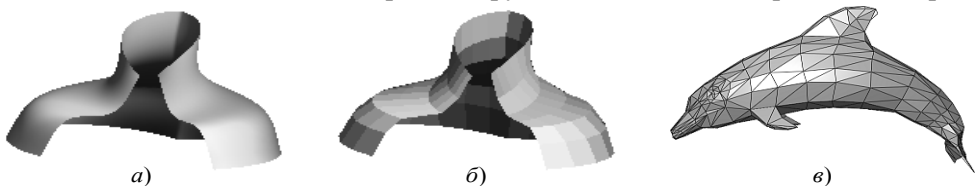


Рис. 4.10

вольными границами, узостями, отверстиями, разрывами и прочими особенностями. Примером нерегулярности служит модель полиэдра $H = \{P, G\}$, определённая списком $P = \{p_1, p_2, \dots, p_{n_p}\}$ произвольно пронумерованных n_p вершин и группой списков $G = \{G_1, G_2, \dots, G_{n_g}\}$ произвольно пронумерованных n_g граней (см. п. 1.5.4). Из пяти платоновых тел лишь октаэдр (см. рис. 1.114,б) может быть задан регулярной сетью размером 2×4 ячеек. Неупорядоченность элементов свободной сети замедляет их визуализацию из-за необходимости выборки либо расчёта всех вершин каждой изображаемой ячейки. Подробнее различные методы организации полигональных сетей будут рассмотрены в гл. 5 при изучении каркасных моделей трёхмерных объектов.

Вывод поверхности, определённой неявной функцией $f(x, y, z) = 0$, затруднён из-за несформированности на ней полигональной сети. Однако, если существует явная и **однозначная** зависимость какой-нибудь координаты через две другие, например, $y = \psi(x, z)$ на прямоугольном плане $\{x, z\}$, то, задав координаты x и z , можно вычислить высоты узловых точек $y_{ij} = \psi(x_i, z_j)$ и выводить на

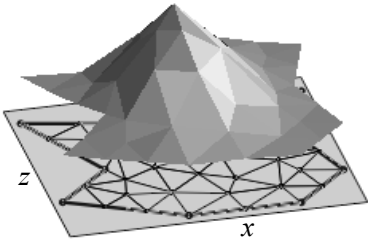


Рис. 4.11

экран ячейки U_{ij} . Ситуация усложняется, когда граница области плана на плоскости xz не прямоугольна. Тогда можно сделать на ней описанную в п. 1.4.5 **контролируемую триангуляцию** с желаемыми размерами треугольников, после чего на полученной нерегулярной сетке построить треугольные грани поверхности и вывести их на экран методом z -буфера (рис. 4.11).

Из всех полигонов наиболее просто выводятся грани-**треугольники**, что сделало фактически стандартом графических систем операцию **триангуляции** поверхностей — представление их полигональными сетями с треугольными ячейками. Перечислим некоторые факторы в пользу треугольных граней:

- предельная простота задания каждого полигона всего тремя вершинами;
- односторонняя видимость плоскости треугольника и невозможность её самоэкранирования в любом ракурсе;
- унификация алгоритмов обработки всех ячеек;
- простота алгоритма растеризации экранной проекции треугольника;
- простота теста (1.44б) локализации точки в алгоритме расчёта пересечения графических объектов с треугольниками.

Главным недостатком треугольных сетей является их нерегулярность и сложность матричной организации ячеек. Лишь некоторые треугольные структуры способны быть регулярными с вытекающими отсюда возможностями экономии памяти и ускорения визуализации. К ним относятся:

- **ленточная** (*strip*) сеть, в которой грани последовательно соединены друг с другом общими рёбрами в одну непрерывную ленту (рис. 4.12,а). Модель ленточной сети из n треугольников представляет собой линейно упорядоченный список точек $u_0 u_1 u_2 \dots u_n u_{n+1}$. Первые три точки задают первый треугольник

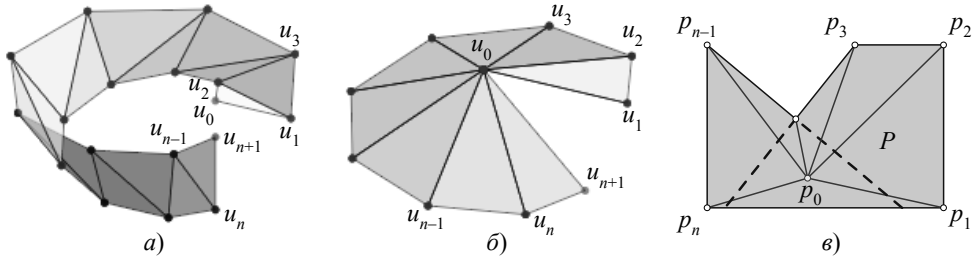


Рис. 4.12

$u_0u_1u_2u_0$, а для формирования i -го треугольника $u_{i-1}u_iu_{i+1}$ с номером $i > 1$ достаточно считать из памяти только одну дополнительную точку u_{i+1} , а две другие вершины берутся из предыдущего треугольника $u_{i-2}u_{i-1}u_i$;

- **веерная** (*fan*) треугольная сеть, все грани которой имеют общую вершину u_0 и последовательно соединены друг с другом общими рёбрами (рис. 4.12,б). Модель веерной сети из n треугольников экономно представляется линейно упорядоченным списком точек $u_0u_1u_2...u_nu_{n+1}$: первые три задают первый треугольник $u_0u_1u_2u_0$, а для формирования i -го треугольника $u_0u_iu_{i+1}$ с номером $i > 1$ из памяти считывается только одна дополнительная точка u_{i+1} , тогда как две другие вершины берутся из предыдущего треугольника $u_0u_{i-1}u_i$. Любой **выпуклый** полигон $P=p_1p_2...p_np_1$ как **площадный** графический объект можно задать веерной сетью $p_1p_2...p_n$ с общей точкой p_1 (см. рис. 1.31). Невыпуклый **звёздный** полигон также представляется веером $p_0p_1p_2...p_np_1$ с выбором общей точки в геометрическом центре его ядра $p_0 = \text{mean}(\text{kern}(P))$ (рис. 4.12,в).

Четырёхугольные ячейки в отличие от треугольных являются плоскими лишь при условии компланарности их вершин (1.112а), что часто автоматически обеспечивается при равномерном квантовании параметров функций $p(t, \tau)$ поверхностей вращения и других симметричных поверхностей с равномерным расположением узловых точек в продольном и поперечном параметрических направлениях (см. рис. 1.142). Если же какая-нибудь вершина ячейки не лежит в плоскости трёх других вершин, то работа с такой поверхностью, называемой **косой плоскостью**, упирается в следующие проблемы:

- некомпланарность ячейки порождает множество моделей заполнения её межрёберного пространства. В результате получаются разные решения геометрических задач пересечения ячейки с прямой, лучом, отрезком и с другими более сложными объектами, а отсюда — проблема выбора из этих решений;
- можно составить четырёхугольную ячейку P из нескольких **плоских** треугольников — двух, соединённых диагональю ячейки, либо четырёх с общей вершиной в средней точке $p_0 = \text{mean}(P)$. На рис. 4.13,а показан принятый в MathCAD метод триангуляции регулярной ПС, иллюстрируемый действием опции *Alternate Mesh* (вкладка *Build* (*Appearance*) в панели форматирования 3d-графика), закрашивающей только первый из двух треугольников четырёхугольной ячейки (рис. 4.13,б). Вывод же обоих некомпланарных треугольников

ячейки производится с *усреднённой* закраской и скрывтием линии сгиба (рис. 4.13, в). Если бы каждый треугольник закрашивался с учётом ориентации его плоскости, то получилось бы изображение ячейки, показанное на рис. 4.13, г. И даже плоские четырёхугольные ячейки всё равно делятся на два компланарных, одинаково закрашенных, треугольника;

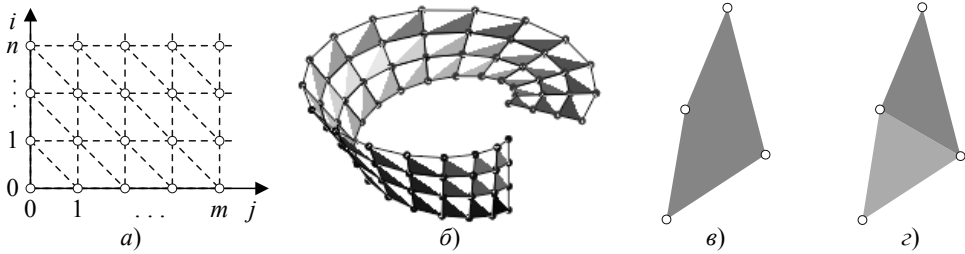


Рис. 4.13

- существует и *нелинейная* модель интерполяции фрагмента кривой плоскости между четырьмя узловыми точками (см. п. 4.4.1), пересечения которой с другими объектами рассчитываются в общем случае численно.

Плоская закрашка

С небольшими потерями качества вывод мелких ячеек вполне допустим с помощью *плоской* (монотонной, однородной) *закраски* (*flat shading*).

В отличие от криволинейной ячейки, *плоская* грань отражает фоновое освещение I_ϕ и свет *дальнего* источника всеми своими точками одинаково и в *параллельных* проекциях может закрашиваться монотонно с помощью быстрых функций заливки полигонов. Если же закрашка производится нормализованными методами, то благодаря *однократному* вычислению вектора нормали N и цвета c скорость вывода грани также будет максимально высокой.

Приступая к плоской закрашке проекции ячейки $u_1u_2u_3$, определимся с цветом c_π , которым заливаются все её пиксели. Возможны следующие варианты:

- цвет заливки задаётся равным собственному цвету e либо отражению фонового света $k_\phi e \bullet I_\phi$ независимо от расположения ячейки и источников освещения. В результате цвет поверхности получается абсолютно плоским с никак не переданной глубиной формы (см. рис. 4.5, д);

- более реалистичный цвет заливки ячейки, зависимый не только от её собственного цвета, но и от освещения и ориентации, даёт ФЗФ (4.1), использующая вектор нормали к поверхности ячейки N_π . В способах его вычисления также возможны варианты:

◇ если вся ячейка принадлежит изображаемой поверхности, т. е. является гранью или частью полигональной грани полиэдра, то по трём её первым вершинам находится *точная* нормаль $N_\pi = (u_2 - u_1) \times (u_3 - u_1)$;

◇ если ячейка лишь приближает гладкую криволинейную поверхность $p(t, \tau)$, опираясь на неё своими вершинами $u_1 \div u_3$, то по (1.93), (1.94) вычисляются *точные*, в общем случае непараллельные, *вершинные нормали*

$N_1 \div N_3$ (рис. 4.14), по которым находится *усреднённая* нормаль ячейки $N_{\text{я}} = \overline{N}_1 + \overline{N}_2 + \overline{N}_3$. Все вершинные нормали должны быть ненулевыми и однонаправленными, например внешними, к поверхности векторами. Перед суммированием их, ввиду разной длины, нужно *нормировать* во избежание наклона вектора $N_{\text{я}}$ в сторону более длинных слагаемых.

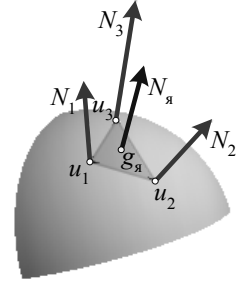


Рис. 4.14

В качестве точки приложения вектора $N_{\text{я}}$ можно взять геометрический центр ячейки $g_{\text{я}} = (u_1 + u_2 + u_3)/3$. Вычислим по (4.1) цвет $c_{\text{я}}$ и в ходе растеризации закрасим им каждый пиксел проекции треугольника. Сети с плоской закраской ячеек показаны на рис. 4.10, б, в, 4.11. Им присуща разрывность цвета на границах ячеек, а также психофизический *эффект полос Маха*, ощущаемый некоторыми людьми как кажущееся обострение цветового контраста в окрестности рёбер смежных ячеек [50, 65].

Сглаженная закрапка

Желание устранить цветовые скачки, присущие плоскому методу визуализации кусочно-плоских полигональных сетей, привело к разработке ряда алгоритмов создания *сглаженной закрапки* (*smoothed shading*), обеспечивающей непрерывность цветового сопряжения на границах смежных ячеек и плавное изменение цвета пикселей внутри каждой ячейки.

Часто геометрия грани задаётся по минимуму — списком её вершин. В отсутствие иных данных рационально считать рёбра ячейки прямолинейными отрезками. Заполнение межграничного пространства некоторого объекта составляет суть задачи *интерполяции*. Простейшая линейная интерполяция (2.10) *равномерно* изменяет состояние объекта между граничными состояниями A и B по зависимостям $\text{lin}(A, B, t)$ в нормированном $t \in [0, 1]$ и $\text{lin}(A, B, (t-a)/(b-a))$ в произвольном $t \in [a, b]$ параметрических интервалах. Под состояниями объекта может пониматься всё что угодно: например, в п. 2.1.1, ими были геометрические координаты точки, а здесь нас интересуют цвета закрапки пикселей.

Существуют и нелинейные методы интерполяции, например, кубическими полиномами Эрмита (2.28) и Безье (2.36), способными обеспечить *гладкое* сопряжение сегментов составного объекта за счёт дополнительных требований к градиентам цветов на их границах. Но поскольку цветовое сглаживание проекции ячейки совмещено с её построчно равномерной растеризацией, то далее будем рассматривать в основном линейные методы цветового сглаживания.

Поскольку ячейка ПС представляет собой фрагмент поверхности, т. е. графический объект с двумя степенями свободы, то для растеризации проекции ячейки применим метод *билинейной интерполяции*. Простой принцип его работы проиллюстрирован примером закрапки регулярного прямоугольника с *целочисленными* габаритами $\{x_{\text{л}}, x_{\text{п}}\}$, $\{y_{\text{в}}, y_{\text{н}}\}$ и нормированными цветами вершинных пикселей $c_{\text{вл}} = [1 \ 0 \ 0]$, $c_{\text{нл}} = [0 \ 0 \ 1]$, $c_{\text{вп}} = [1 \ 1 \ 0]$ и $c_{\text{нп}} = [0 \ 1 \ 1]$ (рис. 4.15):

с координатами $l_1^y = r_1^y = y_v$, $l_N^y = r_M^y = y_n$ и $\min_i \{l_i^x\} < \max_j \{r_j^x\}$. Для этого циклическим сдвигом $P = LCShift(P)$ подведём вершину p_1 к высоте y_v , затем найдём номер $k \in [2, n]$ самой нижней вершины p_k и направление обхода полигона $d = dir_test(P)$. При $d=1$ (обходе **по часовой стрелке** в **левой** экранной СК) составим полилинии $R = p_1 p_2 \dots p_k$ и $L = p_1 p_n \dots p_k$, а при $d=-1$ — полилинии $L = p_1 p_2 \dots p_k$ и $R = p_1 p_n \dots p_k$. Синхронно с контуром полигона список F разделяется на две функциональные цепочки $\Lambda = \lambda_1 \lambda_2 \dots \lambda_N$ и $P = p_1 p_2 \dots p_M$.

Шаг 2. Инициализируем начальные значения счётчиков звеньев левой и правой цепочек $v = \mu = 1$.

Шаг 3. В цикле перебора ординат растровых строк $y = \overline{y_v, y_n}$:

- инкрементируем счётчики на **горизонтальных** звеньях граничных цепочек:

$$v = v + \{l_v^y = l_{v+1}^y\}, \quad \mu = \mu + \{r_\mu^y = r_{\mu+1}^y\}; \quad (4.3a)$$

- вычисляем **целочисленные** линейно интерполированные по (2.10б) абсциссы крайних пикселей $p_l = [x_l \ y]$, $p_n = [x_n \ y]$ и в них функции f_l и f_n :

$$x_l = [\text{lin}(l_v^x, l_{v+1}^x, t_l)], \quad f_l = \text{lin}(\lambda_v, \lambda_{v+1}, t_l), \quad \text{где } t_l = (y - l_v^y) / (l_{v+1}^y - l_v^y), \quad (4.3б)$$

$$x_n = [\text{lin}(r_\mu^x, r_{\mu+1}^x, t_n)], \quad f_n = \text{lin}(\rho_\mu, \rho_{\mu+1}, t_n), \quad \text{где } t_n = (y - r_\mu^y) / (r_{\mu+1}^y - r_\mu^y); \quad (4.3в)$$

- в цикле $x = x_l, x_n$ перебора пикселей y -й растровой строки формируем интерполированное значение f_{yx} и цвет пикселя $Q_{yx} \in O$, **лежащего в окне O** :

$$f_{yx} = \text{lin}(f_l, f_n, \tau), \quad O_{yx} = \text{col}(x, y, f_{yx}), \quad \text{где } \tau = (x - x_l) / (x_n - x_l); \quad (4.3г)$$

- инкрементируем счётчики звеньев цепочек по достижении растровой строкой их нижних концов:

$$v = v + \{y = l_{v+1}^y\}, \quad \mu = \mu + \{y = r_{\mu+1}^y\}. \quad (4.3д)$$

Шаг 4. Возврат матрицы O . $\square$

Растреризация ячейки хорошо совмещается с её отсечением окном вывода. Например, если окно имеет форму регулярного **прямоугольника** с габаритными координатами *left*, *right*, *up* и *down* (см. рис. 3.21), то в (4.3г) рассчитываются и сохраняются цвета только тех пикселей $p_s \in O$, чьи координаты удовлетворяют неравенствам $left \leq x \leq right$ и $up \leq y \leq down$. Локализация пикселя в окне произвольной формы может быть выполнена подходящим тестом из п. 1.1.4.

Рассмотренная процедура растреризации *rastr* выполняется в конечной фазе двух популярных алгоритмов сглаживания закрашки полигональных сетей, аппаратно реализованных в современных видеокартах. Эти методы отличаются составом списка F и устройством цветовой функции $col(x, y, f)$:

- в **методе Гурё (Henri Gouraud)** билинейной интерполяции подвергаются **цветам вершин** ячейки, составляющие список $F = c_1 c_2 \dots c_n c_1$, а $col(x, y, f) = f$. Поэтому в (4.3г) сразу получается цвет закрашки пикселя $O_{yx} = c_{yx}$;

• в методе Фонга (Bui Tuong Phong) интерполируются **вершинные нормали** ячейки, собранные в список $F=N_1N_2...N_nN_1$. Вычисленная в (4.3г) нормаль $N_{yx}=f_{yx}$ преобразуется в цвет пиксела $O_{yx}=col(x,y,N_{yx})$ с помощью ФЗФ (4.1). Необходимая для её работы точка $p \in U_{ij}$ лежит в пересечении плоскости ячейки U_{ij} с прямой $p_3+H_3\theta$, проходящей через экранную проекцию $p_3=[x \ y \ 0]$ в направлении вектора $H_3=S-nearH \cdot p_3$:

$$p=cross_line(p_3, H_3, mean(U_{ij}), norm(U_{ij})). \quad (4.4)$$

В начальной фазе обоих алгоритмов находятся вершинные нормали N_i , по которым в алгоритме Гуро рассчитываются вершинные цвета $c_i=col(N_i)$. Все нормали должны быть ненулевыми векторами, внешне направленными к поверхностям всех ячеек. В способах их вычисления возможны варианты:

• если выводятся грани полиэдра либо полигональной сети, то каждая вершина может принадлежать не только данной, но и смежным граням, и тогда не существует единственной вершинной нормали. В таком случае можно вычислить её **усреднённое** направление путём сложения внешних нормалей m граней с общей вершиной в точке u_i (рис. 4.17):

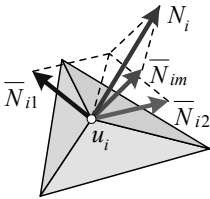


Рис. 4.17

$$N_i = \sum_{k=1}^m \overline{N}_{ik}. \quad (4.5)$$

Поскольку суммируемые векторы могут быть разной длины, то их нужно **нормировать** во избежание перекоса вектора (4.5) в сторону более длинных слагаемых. Для ускорения поиска граней, соединённых в вершине, в модель полиэдра нужно включить соответствующие списки связности;

• если вершина ячейки u_i принадлежит поверхности с неявной $f(p)=0$ либо параметрической $p(t,\tau)$ моделью, то по (1.93), (1.94) вычисляется **точная** нормаль N_i (см. рис. 4.14).

Испытаем изученные методы визуализации полигональных сетей на объекте, представляющем собой виток тороидной поверхности (см. рис. 3.38)

$$p(t,\tau)=[r(t)\cos(\tau)+R \ r(t)\sin(\tau) \ 0]R_y(t) \quad \forall t,\tau \in [0,2\pi],$$

образованной вращением окружности с центром в точке Rx^0 и **переменным** радиусом $r(t)=r_0-r_1t$ вокруг оси y на угол t . По этой модели путём квантования (1.173) параметров $t_i=2\pi i/n$ и $\tau_j=2\pi j/m$ при $n=18$ и $m=9$ создана полигональная сеть с узловыми вершинами $u_{ij}=p(t_i,\tau_j)$ и плоскими ячейками U_{ij} . В узлах сети вычислены нормали $N_{ij}=(\partial p/\partial t)_{u_{ij}} \times (\partial p/\partial \tau)_{u_{ij}}$. Оранжевая поверхность ячеек отражает светло-голубое фоновое освещение $I_\phi=[0.9 \ 1 \ 1]$ и белый свет точечного источника L , расположенного справа – сверху – сзади. В расчёте цвета видимой точки используем модель Фонга (4.1) с коэффициентами $k_\phi=1$, $k_{нд}=0.3$, $k_{нз}=0.7$ и показателями $v=0.7$, $\mu=50$. Визуализацию ячеек произведём в перспек-

тивной проекции (см. п. 3.2.4) по алгоритму художника (см. п. 1.7.2) в порядке уменьшения расстояний (1.182а) от центров ячеек $g_{ij} = \text{mean}(U_{ij})$ до конечно удалённого наблюдателя.

На рис. 4.18,а показано, как гранёно выглядит сеть, у которой каждая (i,j) -я ячейка залита плоским цветом c_{ij} , вычисленным в средней точке g_{ij} с усреднённой нормалью N_{ij} . Эффект перехода к закраске по Гуро показан на рис. 4.18,б, где кроме цветового сглаживания появилось слабое подобие бликов на тех ячейках, в чьих вершинах направленный свет отражается в близкую *окрестность* наблюдателя. Наибольшую *визуальную* плавность закраски кусочно-плоской поверхности показал метод Фонга (рис. 4.18,в). Благодаря вычислению нормали и освещённости в каждой точке каждой грани стали хорошо видны зеркальные блики там, где свет источника отражается *точно* к наблюдателю.

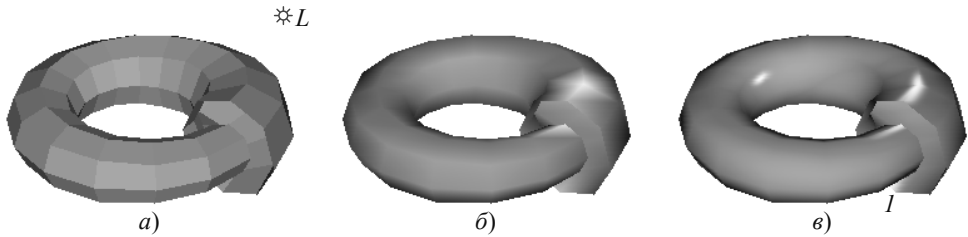


Рис. 4.18

Поскольку ФЗФ суммирует освещённость одной грани без учёта расположения других, то создаваемая ею закраска является *локальной*, что иллюстрирует ложный блик *I* на внутренней стороне поверхности.

Полигональная сеть минимальных размеров $n=m=1$ превращается в один плоский полигон U с единым вектором нормали $N = \text{norm}(U)$ во всех точках, в том числе и в вершинах, где невозможно усреднение (4.5) из-за отсутствия смежных граней. Вследствие параллельности вершинных нормалей возникают следующие *эффекты закраски протяжённого полигона*:

- в сцене с *дальними* наблюдателем S и источником L во всех точках грани имеют место *одинаковые* углы видимости ψ , падения α и бокового отражения β . В результате закраска полигона методами и Гуро, и Фонга выглядит одинаково и плоско (рис. 4.19,а). Высокая яркость отражения объясняется игнорированием затухания дальнего света на пройденном им бесконечном пути;
- *приближение* точечного источника к центру полигона увеличивает вершинные углы α и β , что делает закраску по Гуро парадоксально *темнее* и визуально плоской (рис. 4.19,б), тогда как по алгоритму Фонга создаётся чёткое изображение блика (рис. 4.19,в). Для получения более-менее похожей закраски методом Гуро (рис. 4.19,г) пришлось заменить протяжённую грань полигональной сетью из 8×8 мелких ячеек, что, естественно, увеличило время расчёта;
- если задать точку расположения источника L так, чтобы зеркальное отражение с углом $\beta \approx 0$ происходило вблизи одной из вершин полигона, то сглаживанием по Фонгу в ней создаётся блик (рис. 4.19,д), тогда как по Гуро получается хорошо заметная градиентная, но безбликовая закраска (рис. 4.19,е).

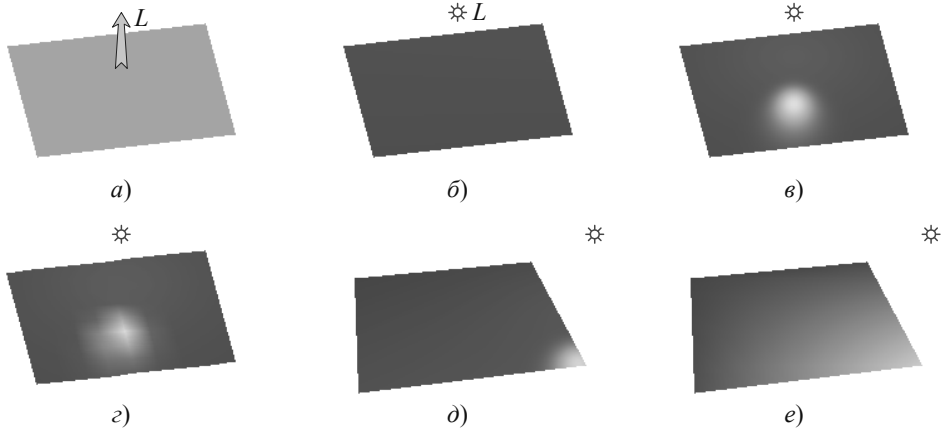


Рис. 4.19

Аналогичные эффекты цветового сглаживания возникают при визуализации **полиэдров** и не замкнутых многогранников, состоящих из плоских протяжённых полигонов, каждый из которых обладает собственным цветом. В отличие от полигональной сети, узлы которой лежат на некоторой гладкой поверхности, в вершинах полигонов, во-первых, не имеет смысла вычислять усреднённую нормаль (4.5), не соответствующую никакой аппроксимируемой поверхности, а во-вторых, не определены собственные цвета вершин полиэдра с разноцветными гранями и вычислять их аналогичным усреднением цветов смежных граней также бессмысленно.

Таким образом, **визуализацию i -й грани полиэдра следует выполнять с постоянными вершинными векторами цвета e_i и нормали N_i** , а цветовое сглаживание граней в сценах ближними источниками света или наблюдателем будет происходить за счёт разных вершинных углов в функции закраски Фонга. Именно так на рис. 4.20 строились изображения вогнутого трёхгранного разноцветного угла, освещённого помимо фонового света I_ϕ ещё и двумя точечными источниками L_1 и L_2 . Плоская закрашка граней (а) рассчитана по освещённостям их средних точек. Закраска по Фонгу (б) позволила получить световые блики и диффузные ореолы на всех гранях угла, тогда как методом Гуро (в) построилась визуально сглаженная безбликовая закрашка только левой грани, в одной из вершин которой зеркально отразился первый источник.

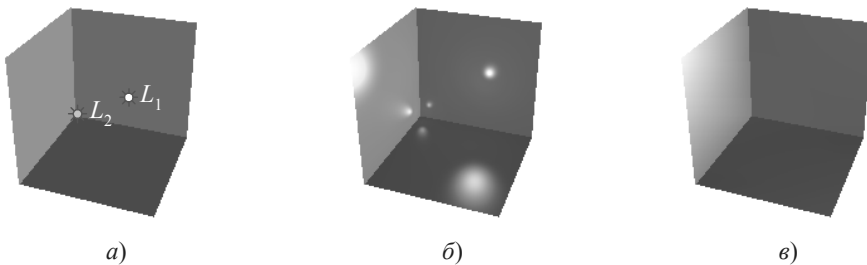


Рис. 4.20

Использование нормализуемой ФЗФ (4.1) позволяет создать *иллюзию* более сложного, чем в натуре, строения поверхности путём специального изменения векторов её нормалей в разных точках их приложения, в результате чего меняются углы ψ , α и β , определяющие видимый цвет поверхности. Можно, например, *регулярно* изменять наклон нормали путём гармонического покачивания вдоль одного (рис. 4.21,а) либо двух (рис. 4.21,б) параметрических направлений для имитации колец и клеток. А замена либо смещение расчётной нормали *случайным* вектором имитирует поверхностную шероховатость с хаотичным расположением неровностей (*шум Перлина*) и присущим ей реалистичным размытием бликов (рис. 4.21,в). Также возможно совмещение обоих методов модуляции векторов нормалей.

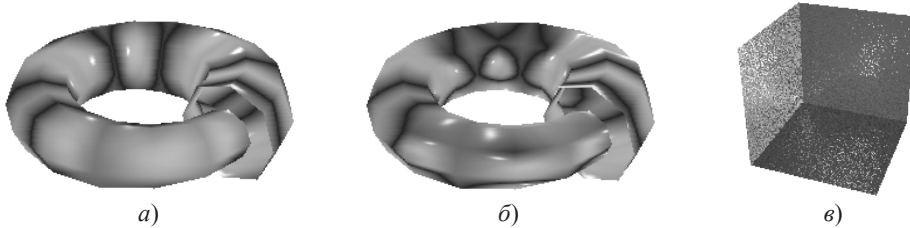


Рис. 4.21

4.2.4. Текстуры

Исчерпаемые возможности визуализации поверхности даёт **метод текстурирования** — наложения на неё *текстуры* (*texture* — ткань), представляющей собой массив цветных точек, образующих в совокупности единое изображение. Подобно тому, как с помощью разных покрытий гладких стен и полов здания создаётся впечатление рельефного фасада, каменной кладки, отделки керамической плиткой или паркетной доской (рис. 4.22), так и унылую монотонную поверхность можно в ходе попиксельной растеризации украсить любым узором с сохранением всех световых эффектов (теней, бликов) и без усложнения её геометрической модели. Кроме изобразительных возможностей текстурирование позволяет наглядно представить физические свойства объектов в приложениях научной визуализации. К примеру, цветовое кодирование температуры точек объекта позволяет увидеть протекание процессов теплопереноса.



Рис. 4.22

Минимальная единица текстуры носит название «*тексел*» (*TEXTure ELement*). Разница между текселом и пикселем проявляется при несовпадении формы или размеров текстуры и площадки, на которую она натягивается. При рас-

тяжении текстуры один тексел может покрыть несколько пикселей экрана, делая изображение крупнозернистым. Наоборот, при вписывании большой текстуры в малую площадь неизбежно сжатие и попадание нескольких текселов в один экранный пиксел, что ведёт к потере отдельных элементов изображения.

Специалистами компьютерной графики разработано множество методов **наложения текстуры** (*texture mapping*) — нанесения готового двумерного изображения (карты) на поверхность трёхмерного объекта или на её экранную проекцию либо закрашивания пикселей по определённому алгоритму без использования физического носителя изображения. Соответственно этим способам создания текстурных карт их можно разделить на **файловые** и **процедурные**.

В конечном итоге текстура изображается на экране, и простейший способ текстурирования заключается в прямом переносе текселов в экранные пиксели. Однако, реальным носителем текстуры является трёхмерная поверхность и если эта поверхность не плоская либо изображается не в ортографической проекции, то неизбежны геометрические искажения как её самой (см. разд. 3.2), так и наложенного на неё рисунка. Особенно сильными и, главное, нелинейными являются перспективные искажения, поэтому для достижения максимальной достоверности восприятия **изображение поверхности и наложение на неё текстуры должны выполняться по одинаковым геометрическим законам**.

Важное место среди текстур занимают **тайлы** (*tile* — плитка) или **бесшовные** текстуры (рис. 4.23). Их отличительная особенность — непрерывность рисунка в направлениях возможного размножения, позволяющая замостить небольшой текстурой большую площадь поверхности с неразрывным повторением узора. Для обеспечения желаемой ориентации и точной стыковки тайловые изображения при укладке могут подвергаться отражениям и вращениям.

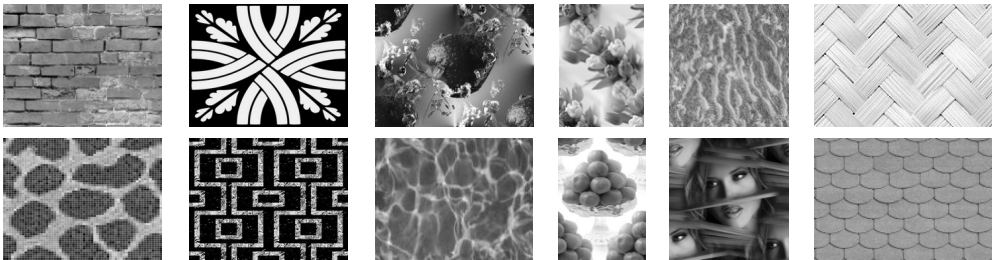


Рис. 4.23

4.2.4.1. Файловые текстуры

Источником файловой текстуры служит растровое изображение, хранящееся во внешней памяти компьютера в каком-либо графическом формате (bmp, jpg, gif, png и т. д.). Это может быть фотография реального объекта либо изображение, созданное в растровом редакторе либо программно сгенерированное и сохранённое в файле. Применение форматов с высокой степенью сжатия позволяет компактно хранить большое количество разнообразных текстур, при желании организованных в форме тематической библиотеки (дерево, камни, керамика, ткани, обои, природа и т. п.).

Работа с текстурой начинается с загрузки хранящего её файла и размещения считанных данных в оперативной памяти в виде прямоугольной матрицы текселей Q . Цветовыми атрибутами её элементов $q_{vu} = [u \ v]$ $\forall v=0, N$, $u=0, M$ являются векторы $\chi_{vu} = [r_{vu} \ g_{vu} \ b_{vu}]$. Обычно растр файлового изображения начинается с верхней строки, поэтому поместим начало **левой системы координат текстуры (СКТ)** в левый верхний угол, а её оси u и v направим по верхней и левой сторонам (рис. 4.24,а). Задача текстурирования заключается в установлении соответствия между экранными $\{x, y\}$ и текстурными $\{u, v\}$ координатами и распределении цветов текселей по экранным пикселям.

Весьма редко, а именно, лишь при совпадении размеров и ориентации текстуры и её экранного прообраза возможно прямое копирование «тексел в пиксел». В общем же случае реалистичный не ортографический вид текстурированного экранного четырёхугольника $S \in R^2$ формируется цепочкой трёх геометрических преобразований: наложения двумерного изображения $Q \in R^2$ из СКТ на пространственный параллелограмм $P \in R^3$ (рис. 4.24,б), проецирования P на картинную плоскость в мировой СК (рис. 4.24,в) и пересчёта проекции P' в область S экранной СК x_3y_3 (рис. 4.24,г). У каждого из этих преобразований есть своя матрица, а произведение матриц $C_{T3} = C_T C_P C_3$ позволяет найти расширенные координаты пикселя $[x \ y \ 0 \ 1] = [u \ v \ 0 \ 1] C_{T3}$ при параллельном и $[x \ y \ 0 \ 1] \sim [u \ v \ 0 \ 1] C_{T3}$ при центральном проецировании на втором этапе.

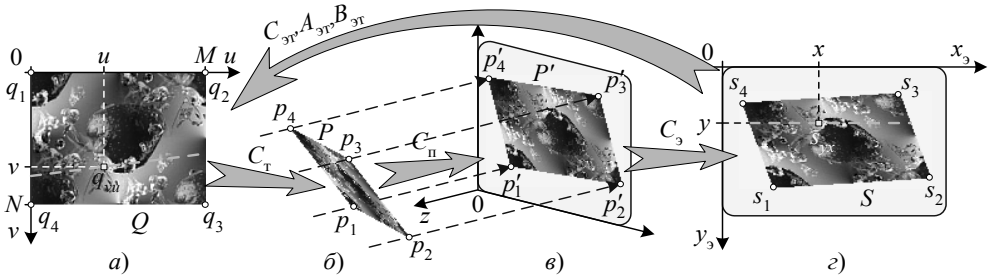


Рис. 4.24

В действительности же, однако, строчной растеризации подвергаются пикселы области S , поэтому актуальны преобразования в обратном направлении. Но из-за неполного ранга матрицы C_{T3} , обусловленного вырожденностью матрицы проецирования C_P (см. разд. 3.2), обратной матрицы C_{T3}^{-1} не существует. Тем не менее, далее будет показано, как **можно сделать преобразование экранных координат $\{x, y\}$ в тексельные $\{u, v\}$ без выхода в третье измерение.**

Аффинное текстурирование

В случае **параллельного** проецирования, **сохраняющего параллельность и пропорции параллельных объектов** (см. п. 3.1.1), текстурирование является **аффинным**. Например, текстурный **прямоугольник** $Q = q_1 q_2 q_3 q_4$ с вершинами $q_1 = [0 \ 0]$, $q_2 = [M \ 0]$, $q_3 = [M \ N]$ и $q_4 = [0 \ N]$ на экране примет форму **параллеле-**

лограмма $S=s_1s_2s_3s_4s_1$ с параллельными и равноотстоящими проекциями текстельных строк и столбцов (см. рис. 4.24, з).

Двухмерное аффинное преобразование экранных координат в текстурные

$$[u \ v]=[x \ y]A_{\text{эт}}+B_{\text{эт}} \quad (4.6a)$$

выполняется матрицей $A_{\text{эт}}$ и вектором $B_{\text{эт}}$, вычисляемыми методом парных точек по (3.18a) при $o=s_1$, $V=s_2-s_1$, $W=s_3-s_2$ и $o'=q_1$, $V'=q_2-q_1$, $W'=q_3-q_2$:

$$A_{\text{эт}}=\begin{bmatrix} s_2-s_1 \\ s_3-s_2 \end{bmatrix}^{-1} \begin{bmatrix} q_2-q_1 \\ q_3-q_2 \end{bmatrix}, \quad B_{\text{эт}}=q_1-s_1A_{\text{эт}}. \quad (4.6б)$$

По (4.6) текстурируется любой экранный полигон S , начиная с треугольника. Для этого на текстуре и полигоне задаются три пары точек $\{q_i, s_i\} \ \forall i=\overline{1,3}$, причём **точки s_1, s_2 и s_3 не должны быть коллинеарными** во избежание вырожденности матрицы, обращаемой в (4.6б).

Перспективное текстурирование

Если на втором этапе процесса, показанного на рис. 4.24, матрица $C_{\text{п}}$ аналогична матрице (3.104б) **центрального** проецирования, **не сохраняющего параллельность и пропорции параллельных линий**, то происходит **перспективное текстурирование**. Матрица $C_{\text{тз}}=C_{\text{т}}C_{\text{п}}C_{\text{з}}$ преобразования текстурного прямоугольника Q в экранный **четырёхугольник** S общего вида имеет, как и матрица $C_{\text{з}}$ в примере 3.2, нулевой третий столбец. В результате уравнение всей цепочки преобразований в расширенном пространстве принимает вид

$$[u \ v \ 0 \ 1] \begin{bmatrix} c_{11} & c_{12} & 0 & c_{14} \\ c_{21} & c_{22} & 0 & c_{24} \\ c_{31} & c_{32} & 0 & c_{34} \\ c_{41} & c_{42} & 0 & 1 \end{bmatrix} = [\omega x \ \omega y \ 0 \ \omega].$$

Запишем его в виде системы двух скалярных уравнений

$$\begin{cases} c_{11}u+c_{21}v+c_{41}=(c_{14}u+c_{24}v+1)x, \\ c_{12}u+c_{22}v+c_{42}=(c_{14}u+c_{24}v+1)y, \end{cases}$$

подвергаемой дальнейшим преобразованиям в двух направлениях:

- при известных коэффициентах c_{ij} она приводится к виду

$$\begin{cases} (c_{11}-c_{14}x)u+(c_{21}-c_{24}x)v=x-c_{41}, \\ (c_{12}-c_{14}y)u+(c_{22}-c_{24}y)v=y-c_{42} \end{cases}$$

с решением относительно текстельных координат

$$[u \ v]=[x-c_{41} \ y-c_{42}] \begin{bmatrix} c_{11}-c_{14}x & c_{12}-c_{14}y \\ c_{21}-c_{24}x & c_{22}-c_{24}y \end{bmatrix}^{-1}; \quad (4.7a)$$

- если **восемь** коэффициентов системы неизвестны, то их можно найти решением системы **восемью** уравнений

$$\begin{cases} (c_{11}-c_{14}x_i)u_i+(c_{21}-c_{24}x_i)v_i+c_{41}=x_i \\ (c_{12}-c_{14}y_i)u_i+(c_{22}-c_{24}y_i)v_i+c_{42}=y_i \end{cases} \quad \forall i \in [1,4],$$

составленной на основе координат **четырёх** разных пар $\{q_1, s_1\} \div \{q_4, s_4\}$ текселов $q_i = [u_i \ v_i]$ и пикселей $s_i = [x_i \ y_i]$:

$$[c_{11} \ c_{21} \ c_{41} \ c_{12} \ c_{22} \ c_{42} \ c_{14} \ c_{24}] = [x_1 \ y_1 \ x_2 \ y_2 \ x_3 \ y_3 \ x_4 \ y_4] \cdot \begin{bmatrix} u_1 & 0 & u_2 & 0 & u_3 & 0 & u_4 & 0 \\ v_1 & 0 & v_2 & 0 & v_3 & 0 & v_4 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & u_1 & 0 & u_2 & 0 & u_3 & 0 & u_4 \\ 0 & v_1 & 0 & v_2 & 0 & v_3 & 0 & v_4 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ -u_1x_1 & -u_1y_1 & -u_2x_2 & -u_2y_2 & -u_3x_3 & -u_3y_3 & -u_4x_4 & -u_4y_4 \\ -v_1x_1 & -v_1y_1 & -v_2x_2 & -v_2y_2 & -v_3x_3 & -v_3y_3 & -v_4x_4 & -v_4y_4 \end{bmatrix}^{-1}. \quad (4.76)$$

В отличие от аффинного преобразования (4.6а) для реализации преобразования (4.7а) из плоскости экрана на плоскость текстуры нужна четвёртая пара точек, определяющая глубину перспективного искажения.

Работа алгоритма перспективного текстурирования показана на рис. 4.25. Он правильно накладывает прямоугольное изображение не только на прямоугольники, параллельные картинной плоскости (когда перспективные искажения отсутствуют), — как близкие к наблюдателю (а), так и удалённые (б), но и на не подобные, произвольно ориентированные в пространстве (в), и на строчно непрерывные полигоны (г), и на треугольники (д). Заметим, что для наложения текстурного треугольника $q_1q_2q_3q_1$ на экранный треугольник $s_1s_2s_3s_1$ в (4.7) не хватает четвёртой пары точек $\{q_4, s_4\}$. Недостача восполняется просто: первую точку этой пары $q_4 = [u_4 \ v_4]$ зададим на текстуре произвольно, например, в центре треугольника $q_4 = \text{mean}(Q)$, а вторую точку $s_4 = [x_4 \ y_4]$ вычислим так же, как вершины треугольника $s_1 \div s_3$: $[x_4 \ y_4 \ 0 \ 1] = [u_4 \ v_4 \ 0 \ 1]C_{\text{тэ}}$.

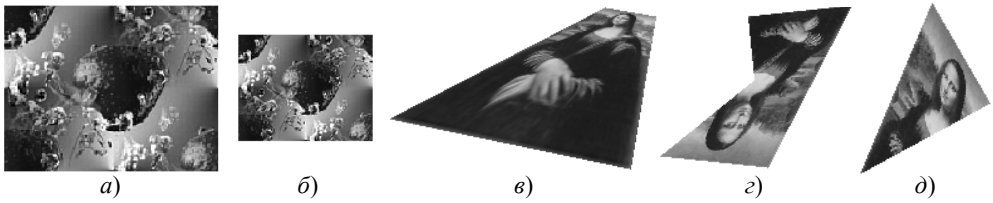


Рис. 4.25

Параметрическое текстурирование

Часто требуется наложить прямоугольную текстуру на **криволинейную** поверхность, описываемую параметрической моделью $p(t, \tau) \forall t \in [t_0, t_n], \tau \in [\tau_0, \tau_m]$. Тогда в схеме, построенной на рис. 4.24, между текстурой и экраном появляется новый посредник — параметрическая плоскость $t\tau$ с прямоугольной областью $[t_0, t_n] \times [\tau_0, \tau_m]$, а первый этап заменяется двумя — аффинным преобразованием $\{u, v\} \rightarrow \{t, \tau\}$ тексельных координат в параметрические и нелинейным преобразо-

ванием $\{t, \tau\} \rightarrow p(t, \tau)$ параметров в точку поверхности. В усечённом виде (без трёхмерных посредников) схема текстурирования показана на рис. 4.26.

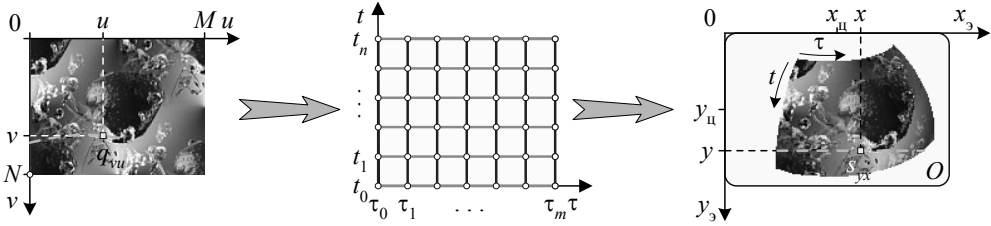


Рис. 4.26

Желаемое расположение текстуры на поверхности обеспечивается заданием точки $p(t_0, \tau_0)$, соответствующей начальным координатам текстуры $u=v=0$, и согласованием направлений изменения параметров $\{u, v\}$ и $\{t, \tau\}$.

Все известные автору печатные и сетевые источники, обучающие текстурированию параметрических поверхностей, ограничиваются линейным преобразованием $\{t, \tau\} \rightarrow \{u, v\}$ параметрических координат в текстурные по формулам

$$v = ((t - t_0) / (t_n - t_0))N, \quad u = ((\tau - \tau_0) / (\tau_m - \tau_0))M. \quad (4.8a)$$

Неужели апологеты данного решения всерьёз считают, что для наложения текстуры **без пропусков и повторов** пикселей достаточно задать (как?) отсчёты параметров $t \in \{t_0, t_1, \dots, t_n\}$ и $\tau \in \{\tau_0, \tau_1, \dots, \tau_m\}$, найти для каждой пары $\{t, \tau\}$ координаты тексела $\{u, v\}$, точку $p(t, \tau)$, пиксел проекции $\{x_3, y_3\}$ и закрасить его цветом χ_{vu} ? В действительности операция текстурирования встроена в процесс растеризации проекции поверхности и направлена **от пиксела к текселу**.

Таким образом, формулы (4.8a) описывают лишь последний (третий) этап параметрического текстурирования. Необходимо восполнить начальные этапы получения параметров точки $\{t, \tau\}$ из её пиксельных координат $\{x, y\}$.

Во-первых, решается обратная (3.42д) задача вычисления координат проекции $p' = [x' \ y' \ 0]$ точки p на картинной плоскости МСК по её экранным координатам $\{x, y\}$ (см. пример 3.2):

$$x' = (x - x_u) / \mu_3 + x'_u, \quad y' = (y - y_u) / \mu_3 + y'_u. \quad (4.8б)$$

Во-вторых, решением векторного **нелинейного** уравнения

$$p(t, \tau) = p' + (S - \text{near}H \cdot p')\theta \quad (4.9)$$

находится пересечение поверхности $p(t, \tau)$ и прямой $p' + H\theta$ с направляющим вектором $H' = S - \text{near}H \cdot p'$. В задаче текстурирования интересна не сама точка пересечения p (хотя она тоже нужна для вычисления в ней вектора нормали $N(p)$ и освещённости), а её параметры $\{t, \tau\}$. Из множества возможных решений $\{t, \tau, \theta\}$ отбирается набор параметров ближайшей к наблюдателю точки с максимальным значением θ и $t_0 \leq t \leq t_n$, $\tau_0 \leq \tau \leq \tau_m$.

И, наконец, в-третьих, по (4.8a) находятся тексельные координаты $\{u, v\}$ и по ним из текстурной карты извлекается цвет тексела χ_{vu} .

В завершение визуализации пиксела с помощью ФЗФ (4.1) рассчитывается видимый цвет в точке (4.9) при $e=\chi_{vi}$.

Работа алгоритма параметрического текстурирования проиллюстрирована на рис. 4.27. Визуализация поверхностей выполнялась методом прямой трассировки лучей с закраской по (4.1) с фоновым освещением и одним точечным источником света. Использование в ФЗФ модулей скалярных произведений $|\overline{H} \circ \overline{N}|$ и $|\overline{V_i} \circ \overline{N}|$ обеспечивает закраску обеих сторон поверхности при любой ориентации вектора нормали так, что у замкнутой поверхности (а) текстурируется видимая сторона, а у не замкнутой (б) — все видимые точки обеих сторон.

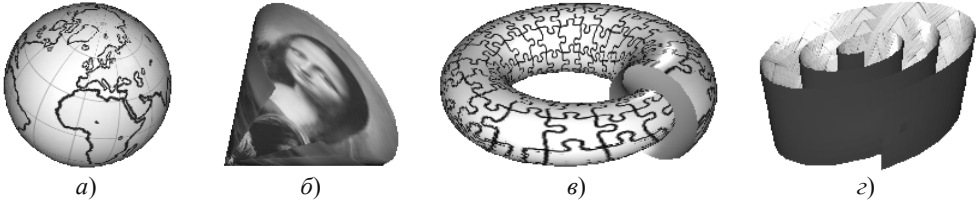


Рис. 4.27

Наложение текстуры можно делать не на обе, а лишь на одну сторону поверхности, например, нормальную, и тогда её рисунок будет виден в точках, удовлетворяющих условию $H \circ N > 0$, а точки противоположной стороны, в которых $H \circ N < 0$, могут быть окрашены в собственный цвет поверхности (в, г).

Текстурирование полигональных сетей

Попиксельное текстурирование параметрических поверхностей очень трудоёмко из-за максимально частого решения нелинейных уравнений (4.9), сортировки и расчёта освещённости точек. Для ускорения визуализации криволинейная поверхность аппроксимируется **полигональной сетью** (см. п. 4.2.3) с вершинами $u_{ij} = p(t_i, \tau_j)$, вершинными нормальными N_{ij} и ячейками U_{ij} , соответствующими текстурным ячейкам Q_{ij} (далее индексы « ij » будут опущены). Для каждого полигона U рассчитываются геометрический центр $g_u = \text{mean}(U)$, нормаль $N_u = \text{norm}(U)$ и экранная проекция S . Поскольку преобразование прямоугольника Q в четырёхугольник S общего вида **не аффинно при любом типе проецирования**, то текстурирование полигона S следует производить **перспективным** методом по формуле (4.7а), предварительно рассчитав по (4.7б) входящие в неё коэффициенты.

В алгоритме растеризации и сглаживания $\text{rastr}(S, F, \text{col}, O)$ каждый пиксел экранного полигона $s \in S$ закрашивается цветом, возвращённым функцией $\text{col}(f)$, аргумент которой f является билинейной интерполяцией элементов списка F . В отличие от монохромной поверхности, передающей в функцию закрашки (4.1) лишь один собственный цвет $c=e$, цвет пиксела s определяет, помимо различных источников освещения, тот из множества текстурных цветов χ_{vi} , который соответствует текстурным координатам $\{u, v\}$, полученным по (4.6) либо (4.7) из экранных координат пиксела $\{x, y\}$.

И вот при такой сложной взаимосвязи данных, участвующих в текстурировании, нужно умудриться реализовать описанные в п. 4.2.3 методы цветового сглаживания (плоского, по Гуро или по Фонгу) и при этом не смазать рисунок текстуры и учесть освещённость поверхности. Важно отметить, что термин «цветовое сглаживание» применительно к текстуре означает сглаживание **освещённости** поверхности, а не переносимого на неё рисунка. Один из способов разделения процессов текстурирования и сглаживания предложен в [2] и заключается в том, что сглаживание освещённости производится на **белой** поверхности, т. е. без учёта текстуры, а полученный цвет освещения c_o используется как модулятор цвета текстуры χ_{vi} в расчёте цвета пиксела $c = c_o \bullet \chi_{vi}$.

Рассмотрим организацию списка F и цветовой функции $col(f)$ в разных методах цветового сглаживания. В трёх рядах изображений на рис. 4.28 показана работа каждого метода на полигональных сетях, аппроксимирующих гладкие поверхности из рис. 4.27. Преднамеренное задание крупных ячеек ПС позволяет оценить и сравнить качество сглаживания с работой наилучшего, но самого медленного, алгоритма текстурирования методом трассировки лучей:

- **плоское наложение текстуры** (верхний ряд) на экранный четырёхугольник S выполняется со списком $F = c_{я}c_{я}c_{я}c_{я}$, составленным из **одинаковых** вершинных элементов, что и даёт константный результат интерполяции. Усреднённую освещённость ячейки $c_{я}$ вычислим по (4.1) в точке $p = g_{я}$ белого цвета $c = [1 \ 1 \ 1]$ с нормалью $N = N_{я}$. Цветовая функция $col(c) = c \bullet \chi_{vi}$ возвращает модулированный освещённостью $c_{я}$ тексельный цвет χ_{vi} точки

$$p = cross_line(p', H', g_{я}, N_{я}) \quad (4.10)$$

пересечения плоскости ячейки $\{g_{я}, N_{я}\}$ с прямой, выходящей из точки p' , найденной в (4.8б), в направлении вектора наблюдателя $H' = S - nearH \cdot p'$;

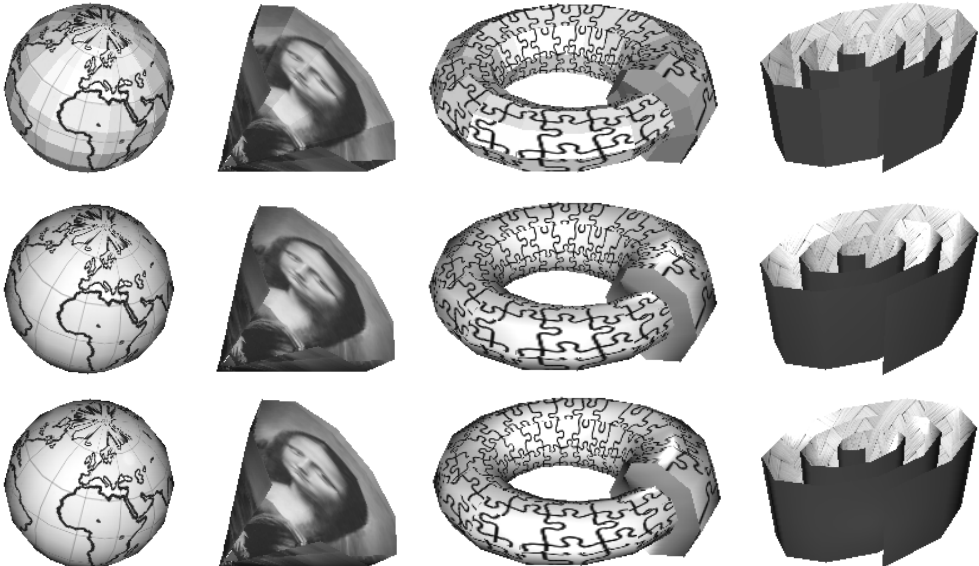


Рис. 4.28

- для **сглаживания текстуры по Гуро** (средний ряд) составляется список $F=c_1c_2c_3c_4c_1$ освещённостей вершин полигона $U=u_1u_2u_3u_4u_1$, вычисленных по (4.1) при $c=[1 \ 1 \ 1]$ в каждой точке $p=u_k$ с нормалью $N=N_k \ \forall k=\overline{1,4}$. Цветовая функция $col(c)=c \bullet \chi_{vi}$ возвращает тексельный цвет χ_{vi} точки (4.10), модулированный интерполированной в (4.3г) освещённостью c ;

- **сглаживание текстуры по Фонгу** (нижний ряд) производится на основе списка $F=N_1N_2N_3N_4N_1$, сформированного из вершинных нормалей полигона $U=u_1u_2u_3u_4u_1$. Цветовая функция $col(N)=c_o \bullet \chi_{vi}$, аргументом которой является интерполированный в (4.3г) вектор нормали N в вычисленной по (4.10) точке p , возвращает тексельный цвет χ_{vi} , модулированный освещённостью c_o , рассчитанной по (4.1) при $c=[1 \ 1 \ 1]$.

Анализируя изученные методы текстурного сглаживания и построенные ими изображения на рис. 4.28, делаем следующие вполне ожидаемые выводы:

- самый быстрый (с минимальным числом обращений к ФЗФ) метод плоского текстурирования обладает наихудшим качеством сглаживания (его попросту нет!) с отчётливо видимыми и равномерно освещёнными ячейками ПС;
- лучшим, практически идеальным качеством сглаживания обладает наиболее затратный по числу расчётов освещённости метод Фонга. О том, что перед нами не гладкая, а кусочно-плоская фигура, напоминает только её угловатый абрис. На сглаженных по Фонгу поверхностях также лучше и глаже всего выглядят диффузные и зеркальные отражения разнообразных источников света;
- среднее качество и трудоёмкость сглаживания текстур показал метод Гуро. Его недостаток — плохое качество световых бликов, которые становятся размытыми и слабо видимыми лишь при отражении лучей точно в вершинах ПС. Если же для повышения качества резко уменьшить размеры ячеек, то, во-первых, преимущества сглаживания по Гуро перед методом Фонга исчезнут, а во-вторых, обрабатываемые в (4.6) либо (4.7) матрицы могут стать вырожденными из-за слияния двух или более вершин экранного полигона в один пиксел.

Периодическое текстурирование

Итак, мы рассмотрели четыре метода наложения текстурного изображения целиком на всю поверхность. Однако, если пропорции текстуры и подложки отличаются друг от друга, то неизбежно возникнут масштабные искажения, из-за которых привычные образы выглядят растянуто (рис. 4.29,а) либо сжато (рис. 4.29,б). Для уменьшения искажений, а также для покрытия небольшими текстурами больших площадей, организуем периодическое повторение рисунка в одном или обоих направлениях. Если к тому же использовать тайловые текстуры (см. рис. 4.23), то получим бесшовные покрытия с гладкими стыками.

Зададим два **коэффициента размножения** $k_u > 0$ и $k_v > 0$, не обязательно, кстати, целые и большие единицы числа. Коэффициенты, меньшие единицы, применяются для покрытия полигона частью текстуры (рис. 4.29,в). Не целые значения коэффициентов позволяют уложить текстуру **без искажений** столько

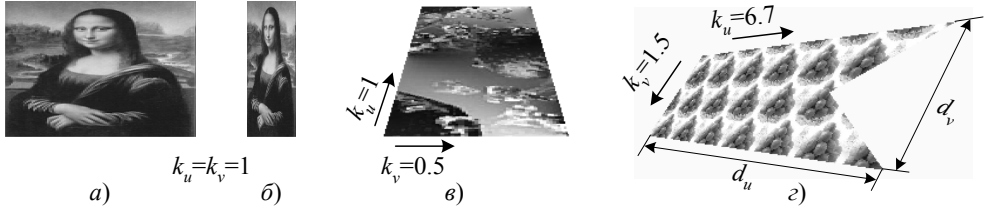


Рис. 4.29

раз, сколько её помещается в характерных пиксельных длинах полигона в направлениях размножения d_u и d_v (рис. 4.29,з). Для минимизации масштабных искажений коэффициенты k_u и k_v подбираются из следующей пропорции:

$$\frac{d_u}{d_v} \approx \frac{k_u M}{k_v N} \Rightarrow \frac{k_u}{k_v} \approx \frac{d_u N}{d_v M}. \quad (4.11a)$$

Эффект **тайлинга** достигается умножением координат u и v , рассчитанных в (4.6а), (4.7а) и (4.8а), на коэффициенты k_u и k_v , взятием полученных произведений по модулям M и N , после чего окончательным приведением результатов в интервалы $[0, M]$ и $[0, N]$:

$$u = \max\{0, \min\{M, \text{mod}(k_u u, M)\}\}, \quad v = \max\{0, \min\{N, \text{mod}(k_v v, N)\}\}. \quad (4.11б)$$

Данные текстельные координаты $\{u, v\}$, будучи индексами цветового вектора $\chi_{\text{мс}}$, обязаны быть целыми числами. Это непреложное требование нарушается в операциях деления и из-за вычислительных погрешностей. Округление вещественного числа $c \in \{u, v\}$ можно сделать любой из трёх операций $c = \lfloor c \rfloor$, $c = \lceil c \rceil$ либо $c = [c]$. Фотографические текстуры, наложенные с использованием этих простейших методов округления, выглядят практически одинаково. Искажения в виде смещений и пропусков отдельных пикселей заметны лишь на некоторых почти горизонтальных (параллельных развёртке) участках линий пиксельной толщины, часто встречающихся в процедурных текстурах (рис. 4.30).

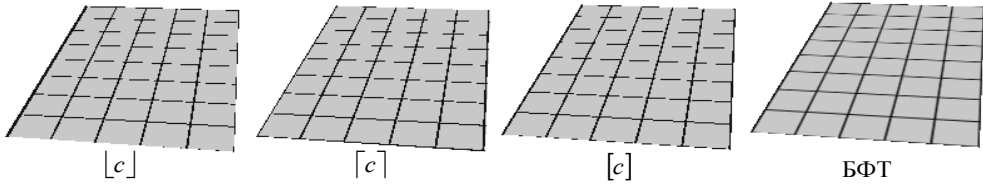


Рис. 4.30

Причина искажений в том, что при горизонтальной растеризации экранного полигона S сканирующая строка практически всегда не параллельна сторонам текстурного полигона Q (см. рис. 4.24), и по округлённым координатам $\{u, v\}$ выбирается один из соседних текселов с цветом, отличным от истинного цвета тонкой линии. Для устранения искажений можно формировать цвет пиксела в виде смеси цветов текселов, накрываемых им в результате обратного преобразования «экран $\rightarrow$ текстура». В зависимости от масштабного фактора преобразования имеют место две крайности (рис. 4.31):

- при сжатии большой текстуры в малую экранную область один пиксел захватывает много текселов (*а*), быстрая локализация которых за крайне ограниченное время текстурирования достаточно проблематична;
- при растяжении небольшой текстуры на обширную экранную область пиксел может оказаться целиком внутри одного тексела (*б*) и цветового смешивания не потребуется.

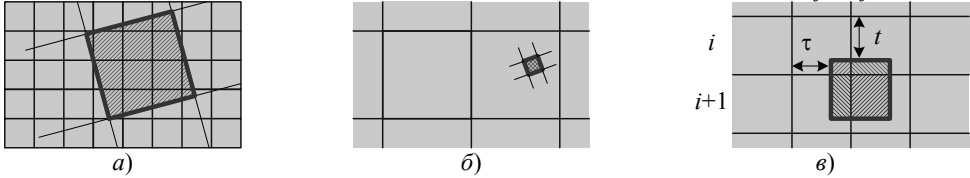


Рис. 4.31

Простейшая модель цветового смешивания — равновесная, а самая трудоемкая включает задание весов пропорционально площадям текселов, покрываемым преобразованным пикселом. С учётом нерегулярного расположения последнего на текстуре пропорциональный метод практически нецелесообразен.

Таким образом, ввиду сложностей, встающих на пути точного учёта числа и площадей текселов, вносящих вклад в цвет пиксела, обычно ограничиваются упрощенной схемой цветового смешивания (рис. 4.31, *в*), предполагающей:

- одинаковые размеры квадратов, представляющих текселы и пикселы;
- регулярную ориентацию преобразованного пиксела на текстуре.

На основе этих идеализаций в технологии наложения текстур разработан **метод билинейной фильтрации текстур (БФТ)**, использующий для закраски пиксела s_j цвет не только тексела q_{vu} , но и цвета трёх соседних текселов в тех случаях, когда текстурные координаты пиксела (4.11б) не являются целыми числами. Выделив из координат $v=i+t$ и $u=j+\tau$ целые и дробные части

$$i=\lfloor v \rfloor, \quad t=v-i \in [0,1), \quad j=\lfloor u \rfloor, \quad \tau=u-j \in [0,1), \quad (4.12a)$$

составим подобно (4.2) цвет (x,y) -го пиксела как сумму цветов четырёх текселов с весами, пропорциональными площадям их **прямоугольных** сегментов:

$$c = \text{lin}(\text{lin}(\chi_{i,j}, \chi_{i+1,j}, t), \text{lin}(\chi_{i,j+1}, \chi_{i+1,j+1}, \tau), \tau). \quad (4.12б)$$

Во избежание ошибок индексации в (4.12б) функция линейной интерполяции, обычно определяемая как $\text{lin}(a,b,t)=(1-t)a+tb$, должна при $t=0$ возвращать объект a **без обращения к объекту b** и определяться следующим образом:

$$\text{lin}(a,b,t) = \text{if}(t:(1-t)a+tb, a). \quad (4.12в)$$

Очевидное качество метода БФТ (4.12) в сравнении с тремя методами простого округления демонстрирует рис. 4.30. Отметим, однако, что если в текстурном рисунке нет тонких и длинных отрезков прямых, то отличия текстурирования разными методами округления практически незаметны, в чём можно лично убедиться на рис. 4.32.

Подводя промежуточный итог теме текстурирования, отметим два основных метода создания и использования растровых текстур:

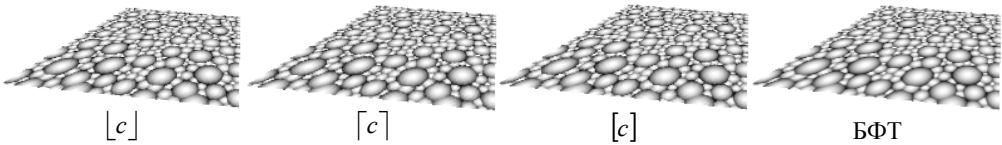


Рис. 4.32

- первый метод основан на использовании реальных фотоизображений с последующим ретушированием и небольшими корректировками;
- второй метод предполагает создание текстуры полностью с нуля при помощи графических программ.

Эти методы, как и их комбинация, обладают одним большим недостатком — невозможностью быстрого итерирования вариантов текстуры. В обоих случаях текстуры лишены даже минимальной параметризации картинки и как следствие — возможности быстрого изменения путём редактирования этих параметров. В результате даже минимальные изменения в одной текстуре влекут за собой необходимость внесения соответствующих изменений во всех связанных текстурах. Максимум, что можно сделать быстро, — это поменять яркость, контрастность и цвет тех или иных участков.

Таким образом, создание растровых текстур сводится к рутинной работе: множеству ручных операций инструментами графической программы, бесконечному перебору фототекстур в поисках подходящей, ретушированию и правке нежелательных элементов, созданию бесшовных границ изображений и т. д.

Всю эту нудную работу можно избежать, грамотно применяя там, где это возможно, процедурную генерацию текстур.

4.2.4.2. Процедурные текстуры

Процедурное генерирование текстур — это метод создания графического изображения с помощью специальной процедуры (формулы, алгоритма), в ходе которой оно выводится на экран, а на диске хранится не файл текстуры, а короткий код исполняемой программы [66]. Кроме образца билинейной цветовой интерполяции (см. рис. 4.15) и семи образцов процедурных текстур (см. рис. 4.23) на рис. 4.33 представлено ещё два ряда алгоритмически построенных монохромных изображений, имитирующих разные материалы и природные явления. Ввиду неисчерпаемости фантазии математиков и программистов число созданных ими процедурных текстур не поддаётся никакому учёту как и полное освещение методов их создания.

Часто в алгоритмы регулярного моделирования добавляются случайные возмущения формы и цвета фигур, что ещё больше разнообразит возможности текстурирования. Для создания компьютерного шума используются специальные алгоритмы, называемые *генераторами случайных чисел* (ГСЧ). Не секрет, что ГСЧ, запускаемый с «затравочного» числа (*seed* — семя, зерно), даёт серию не истинно случайных, а так называемых *псевдослучайных чисел*, многократно повторяющихся при запуске с одного и того же «семени».

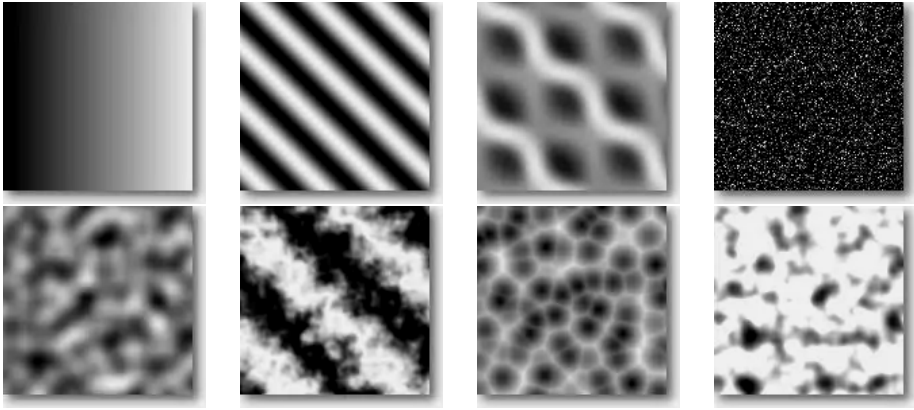


Рис. 4.33

Пусть $rf(\lambda)$ — функция генератора **центрированных** (с нулевым средним) случайных чисел s , распределённых по некоторому закону с **параметром разброса** λ . Не центрированные случайные числа со **средним значением** μ генерируются по формуле $\mu + rf(\lambda)$. Чаще всего используются генераторы с двумя наиболее популярными функциями плотности распределения вероятности [21]:

- **равномерный** ГСЧ $urf(\lambda) = rnd(2\lambda) - \lambda$ с постоянной плотностью вероятности $p(s) = 1/2\lambda$ в интервале $s \in (-\lambda, \lambda)$ и нулевой вне его (рис. 4.34,а);
- **нормальный** ГСЧ $nrf(\sigma)$ с гауссовской плотностью $p(s) = e^{-s^2/2\sigma^2} / \sigma\sqrt{2\pi}$ (рис. 4.34,б) и **среднеквадратичным отклонением** σ . Доля нормальных случайных чисел, сгенерированных в интервале $s \in [-3\sigma, 3\sigma]$, составляет 99.73%.

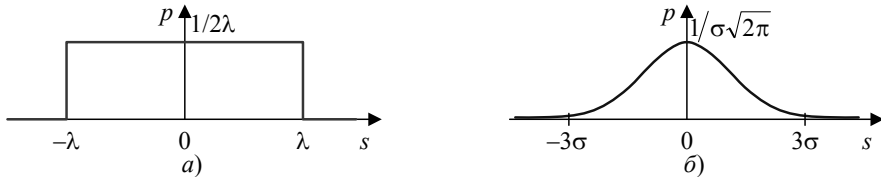


Рис. 4.34

Практически всегда текстура любого типа имеет вид прямоугольника (**карты**) размерами $(N+1) \times (M+1)$ текселов (см. рис. 4.24,а). В отличие от файловой текстуры Q процедурная текстура F заполнена значениями некоторой функции $F_{vu} \forall v=0, N, u=0, M$. Перед выводом карты на экран она подвергается преобразованиям, имеющим следующие цели:

- приведение карты к виду, удобному для её дальнейших преобразований;
- намеренное искажение для достижения желаемых визуальных эффектов;
- конвертация значений карты в используемую цветовую палитру дисплея.

Операции с картами

Хорошо, когда экстремальные значения функции карты равны 0 и 1 (такие **нормализованные** карты будем обозначать как H). Это желание реализуется по-

тексельной операцией **нормализации** $H = \text{nor}(F)$ карты F с произвольными, но **неодинаковыми** значениями F_{vu} , с элементами матричной функции

$$\text{nor}(F)_{vu} = \frac{F_{vu} - \min(F)}{\max(F) - \min(F)} \in [0, 1] \quad \forall v = \overline{0, N}, u = \overline{0, M}. \quad (4.13a)$$

Нормализация обеспечивает максимальное вписывание всех значений карты H в единичный интервал $[0, 1]$ с $\min(H) = 0$ и $\max(H) = 1$.

Нормализация минимизирует вычислительные затраты, а то и обеспечивает саму возможность выполнения других операций с картами:

- $k_v H$ — **масштабирование** карты H , позволяющее при $k_m < 1$ уменьшить яркость её изображения, а также преобразовать все её значения в оттенки серого с цветовыми матрицами $R = G = B = 255H$ палитры «TrueColor». Для получения разноцветной текстуры нужно сгенерировать три карты H_r, H_g, H_b и выводить на экран смесь цветовых матриц $R = 255H_r, G = 255H_g$ и $B = 255H_b$;

- $F + k_c$ — **смещение** карты F , позволяющее совместно с масштабированием повысить яркость (*brightness*) изображения нормализованной карты в k_a раз:

$$\text{bright}(H, k_a)_{vu} = (H_{vu} + k_a - 1) / k_a \quad \forall v = \overline{0, N}, u = \overline{0, M}; \quad (4.13б)$$

- в различных графических программах имеется великое множество **нелинейных фильтров**, потексельно, аналогично фильтрам (4.13а,б), обрабатывающих изображения по определённым алгоритмам. Из-за очевидного факта, что «нельзя объять необъятное», ограничимся лишь несколькими фильтрами:

◇ **гамма-коррекция**

$$\text{gamma}(H, p)_{vu} = H_{vu}^p \quad \forall v = \overline{0, N}, u = \overline{0, M} \quad (4.13в)$$

изменяет в нормализованной карте соотношение между тёмными и светлыми участками согласно графику степенной функции на интервале $[0, 1]$. При степени $p < 1$ яркость тёмных участков растёт сильнее, чем светлых, а при $p > 1$ яркость светлых участков снижается больше, чем тёмных;

◇ фильтр «**Штамп**» с порогом f_1 заменяет в карте F все значения $F_{vu} \geq f_1$ на 1, а все остальные — на 0, делая карту нормализованной:

$$\text{stamp}(F, f_1)_{vu} = \{F_{vu} \geq f_1\} \quad \forall v = \overline{0, N}, u = \overline{0, M}; \quad (4.13г)$$

◇ фильтр **ограничения** (*clamping*) с порогами f_0 и f_1 заменяет в карте F все значения $F_{vu} < f_0$ на f_0 , а все значения $F_{vu} > f_1$ на f_1 :

$$\text{clamp}(F, f_0, f_1)_{vu} = \min(\max(F_{vu}, f_0), f_1) \quad \forall v = \overline{0, N}, u = \overline{0, M}; \quad (4.13д)$$

◇ **абсолютный** фильтр заменяет в карте F все имеющиеся отрицательные значения на противоположные:

$$\text{abs}(F)_{vu} = |F_{vu}| \quad \forall v = \overline{0, N}, u = \overline{0, M}; \quad (4.13е)$$

◇ периодический **модульный** фильтр заменяет все значения карты, лежащие вне диапазона $[0, 1)$, на значения

$$\text{modul}(F)_{vu} = \text{mod}(\text{mod}(F_{vu}, 1) + 1, 1) \in [0, 1) \quad \forall v = \overline{0, N}, u = \overline{0, M}; \quad (4.13ж)$$

- **цветовое смешивание** (*mixing*) нормализованной карты H , создающее векторный цвет (v, u) -го тексела $\chi_{vu} \in \mathbb{R}^3$ как линейную интерполяцию от цвета c_0 до цвета c_1 по параметру $H_{vu} \in [0, 1]$:

$$\text{mix}2(c_0, c_1, H)_{vu} = \text{lin}(c_0, c_1, H_{vu}) \quad \forall v = \overline{0, N}, u = \overline{0, M}. \quad (4.14a)$$

Применение в (4.14a) рассматриваемых ниже нелинейных функций интерполяции $L_3(H_{vu})$ и $L_5(H_{vu})$ обостряет контраст цветового перехода. Более красочные эффекты даёт трёхцветное смешивание

$$\text{mix}3(c_0, c_1, c_2, H)_{vu} = (1-t)c_0 + 2t(1-t)c_1 + t^2c_2 \quad \forall v = \overline{0, N}, u = \overline{0, M}, t = H_{vu}, \quad (4.14б)$$

устроенное аналогично квадратичной линии Безье (2.40б). Благодаря свойству любой линии Безье лежать в выпуклой оболочке узловых точек (см. рис. 2.25,а) трёхцветная смесь всегда будет находиться в цветовом RGB-пространстве между точками c_0 , c_1 и c_2 . Отметим, что квадратичная интерполяция таким свойством не обладает (см. рис. 2.5) из-за локальной отрицательности весовых функций $l_{20}(t)$ и $l_{22}(t)$ (см. рис. 2.9,б), определённых в (2.16б). На основе полиномиальной модели линий Безье (2.38) можно смешивать любое количество цветов;

- **фрактализация** (*fraction* — дробить) — создание карты $F^{<vn \times \mu m>}$ с теми же размерами $\{N, M\}$, но на дробной решётке с кратным числом рядов vn и/или колонок μm (вот здесь-то и возможны решётки с нецелочисленными длиной и высотой клетки!). Подражая музыкальной октаве, в которой отношение частот верхнего и нижнего звуков равно двум, процедурную карту с частотами $v = \mu = 2^k$ также называют *k-й октавой*. Ничто не мешает использовать **гармоники** с разными и не равными степеням 2 частотами v и μ , что позволяет создать **анизотропные** текстуры с разными свойствами по горизонтали и вертикали;

- **смешивание текстур** с разными частотами позволяет добавить к пространственно протяжённым деталям изображения текстуры $H^{<n \times m>}$ более мелкие детали гармоник $H^{<vn \times \mu m>}$. В формуле фрактального смешивания

$$C = k_0 H^{<n \times m>} + k_1 H^{<v_1 n \times \mu_1 m>} + k_2 H^{<v_2 n \times \mu_2 m>} + \dots \quad (4.14в)$$

сумма коэффициентов $k_0, k_1, k_2 \dots$ должна быть равна 1 с целью удержания значений карты C в диапазоне $[0, 1]$. Если задавать веса в порядке убывания $k_0 \geq k_1 \geq k_2 \geq \dots$, то низкочастотные гармоники (крупные детали текстуры) будут иметь относительно большую яркость, чем высокочастотные (мелкие детали).

Далее рассмотрим некоторые **категории и виды процедурных текстур**.

Регулярные текстуры

В графических приложениях, связанных с автоматизированным проектированием в машиностроении, архитектуре, дизайне интерьеров и т. п., требуются текстуры, содержащие регулярные фигуры (полоски, клетки, правильные многоугольники), закономерно расположенные на её площади. Такие текстуры трудно получить фотографированием нарисованных картинок из-за неизбежных

перспективных искажений и необходимости ретуширования. К тому же файлы цветных текстур большой площади требуют много машинной памяти и времени чтения с диска. Рациональнее генерировать текселы регулярных текстур по несложным алгоритмам, гарантируя точное воспроизведение свойств изображения, желаемую периодичность повторения элементов и возможность тайлинга.

□ Простой пример регулярного генератора — создание изображения прямоугольного покрытого сеткой листа (см. рис. 4.30), в котором (v, u) -й тексел во вложенном цикле $v \in \overline{0, N}$, $u \in \overline{0, M}$ закрашивается при $\text{mod}(v, d) \wedge \text{mod}(u, d)$ в цвет фона, например, золотистый $[1 \ 0.8 \ 0]$, а иначе в цвет сетки, например синий $[0 \ 0 \ 1]$ (рис. 4.35, а). При N и M , кратных шагу d , весь периметр листа окаймлён линиями сетки (рис. 4.35, б), вплоть до одной клетки при $N=M=d$.

□ Чуть сложнее генерируется диагонально-клетчатая (со стороны клетки d) n -цветная текстура (рис. 4.35, в), у которой (v, u) -й тексел закрашивается в циклах $v \in \overline{0, N}$ и $u \in \overline{0, M}$ в один из цветов палитры $C = \{c_1, \dots, c_n\}$:

- находятся клеточные координаты $i_k = \lfloor v/d \rfloor$, $j_k = \lfloor u/d \rfloor$;
- вычисляется индекс цвета тексела $i_{\text{ц}} = \text{mod}(i_k + j_k, n) + 1$;
- (v, u) -й тексел закрашивается цветом $\chi_{vu} = c_{i_{\text{ц}}}$.

При значениях $N+1$ и $M+1$, кратных размеру клетки d , получится текстура с целыми клетками, а при $N=M=nd-1$ генерируется **тайловая** текстура (рис. 4.35, з), пригодная для бесшовного размножения. Если же заменить формулу вычисления индекса цвета на $i_{\text{ц}} = \text{mod}(|i_k - j_k + n|, n) + 1$, то получим другую ориентацию диагоналей клеток, изображённую на рис. 4.35, д.

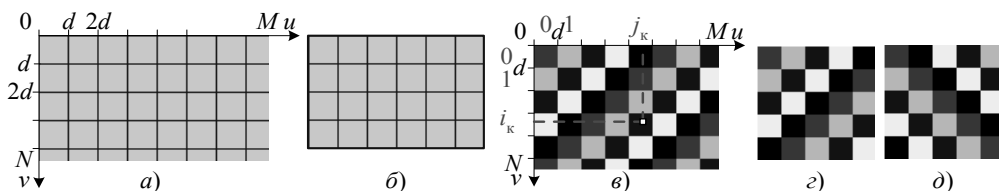


Рис. 4.35

□ Рассмотрим третью процедуру синтеза текстуры, имитирующей **кирпичную кладку** со следующими параметрами (рис. 4.36, а):

- d_k и h_k — длина и высота одного кирпича, а $d_{\text{ш}}$ — толщина швов, например, $d_k=50$, $h_k=20$ и $d_{\text{ш}}=6$;
- n и m — число секций по вертикали и горизонтали, например, $n=4$ и $m=2$. Секция длиной $d_c = d_k + d_{\text{ш}} = 56$ и высотой $h_c = h_k + d_{\text{ш}} = 26$ включает кирпич и два шва справа и снизу;
- c_k и $c_{\text{ш}}$ — векторы цветов кирпича и шва, например, $c_k = [160 \ 100 \ 80]$ и $c_{\text{ш}} = [178 \ 178 \ 178]$ в палитре «TrueColor»;
- нечётные ряды смещены на половину длины секции $0.5d_c = 28$.

Вычислив габариты текстуры $N = nh_c - 1 = 103$ и $M = md_c - 1 = 111$, во вложенном цикле $v \in \overline{0, N}$, $u \in \overline{0, M}$ формируем цвет (v, u) -го тексела χ_{vu} :

- находим $b_v = \text{mod}(\lfloor v/h_c \rfloor, 2) \in \{0, 1\}$ — бит чётности ряда (0 — чётное число);
- находим $b_v = \{0 \leq \text{mod}(v, h_c) < h_k\}$ и $b_u = \{0 \leq \text{mod}(u + 0.5b_v d_c, d_c) < d_k\}$ — биты принадлежности координат тексела прямоугольнику кирпича;
- вычисляем цвет тексела $\chi_{vu} = \text{if}(b_v b_u : c_k, c_{\text{ш}})$.

Минимальный размер тайла будет при $n=2$ и $m=1$ (рис. 4.36, б).

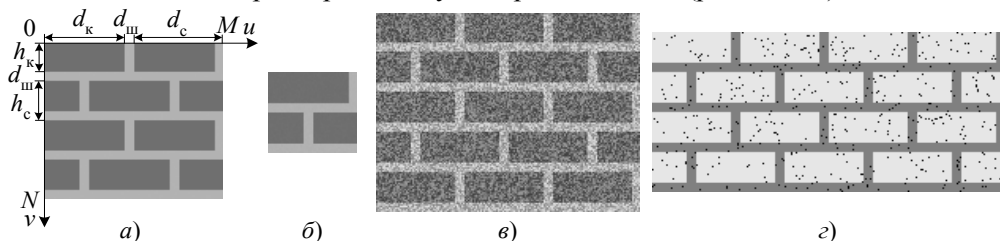


Рис. 4.36

Полученные изображения кирпичной кладки отдают унылым однообразием и малой похожестью на поверхности реальных кирпичей и цементных швов, которые всегда имеют неровности цвета, вкрапления мелких посторонних частиц и прочие неоднородности. Для моделирования случайных эффектов вполне подходят следующие незамысловатые решения:

- неравномерность окраски симитируем прибавлением к элементам детерминированного цветового вектора χ_{vu} случайных равномерно распределённых чисел $\text{urf}(\lambda) \in [-\lambda, \lambda]$. Результат при $\lambda=50$ виден на рис. 4.36, в. Во избежание эффекта цветового переполнения компоненты смещённого цветового вектора нужно удерживать в интервале, принятом для кодирования цвета, например, $[0, 255]$. При $\lambda=0$ цвет тексела не изменяется;

- для имитации случайных вкраплений чёрных точек с вероятностью $0 \leq p \leq 1$ введём функцию $\text{ver}(p) = \{\text{rnd}(1) \geq p\}$, возвращающую 0 с вероятностью p . Тогда цвет $\chi_{vu} \cdot \text{ver}(p)$ с заданной вероятностью p будет чёрным, а с вероятностью $1-p$ не изменится. Этот эффект проиллюстрирован на рис. 4.36, г при $p=0.02$ и цветах $c_k = [230 \ 230 \ 230]$ и $c_{\text{ш}} = [128 \ 128 \ 128]$.

Возможна реализация сразу обоих эффектов. Читателю предлагается самостоятельно смоделировать **неровные края** кирпичей и швов между ними.

Рассмотрим наиболее распространённые методы создания случайных текстур (**процедурных шумов**) с управляемой стохастичностью.

Клеточный шум

(Псевдо-) случайное распределение координат и цветов текселов, конечно, имеет право на существование, чему пример правая верхняя текстура на рис. 4.33, имитирующая звёздное небо неизвестной планеты. На этом интерес к абсолютному хаосу практически исчерпан, а более завлекательной становится разработка **генераторов клеточного шума (ГКШ)**, в котором сквозь хаотичность в большей (см. рис. 4.36, в, г) или в меньшей (нижний ряд на рис. 4.33) степени проглядываются геометрические и цветовые закономерности.

Работа ГКШ происходит на **решётке**, разбивающей текстурное поле на **клетки**, уложенные в n рядов и m колонок длиной и высотой

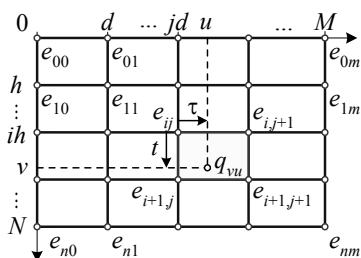


Рис. 4.37

$$d = M/m, \quad h = N/n \quad (4.15a)$$

текселов (рис. 4.37). Целочисленные клеточные координаты (i, j) , соответствующие тексельным координатам (v, u) , находятся округлением

$$i = \lfloor v/h \rfloor - \{v=N\}, \quad j = \lfloor u/d \rfloor - \{u=M\}. \quad (4.15б)$$

В узлах решётки задаются либо вычисляются узловые элементы $e_{00} \div e_{nm}$ — числа или векторы, используемые для расчёта значений

шумовой функции во всех текселах. Благодаря интерполяции создаётся необходимая коррелированность **клеточного шума** (*lattice noise*) между узлами, приводящая к образованию на текстуре причудливых фигур.

Очень простой способ сделать генерируемую текстуру **тайловой** — это уравнять узловые элементы верхнего ряда с нижним и левой колонки с правой:

$$e_{n0} = e_{00}, \quad e_{n1} = e_{01}, \dots, \quad e_{nm} = e_{0m}; \quad e_{0m} = e_{00}, \quad e_{1m} = e_{10}, \dots, \quad e_{nm} = e_{n0}. \quad (4.15в)$$

Поэтому все показанные ниже текстуры созданы тайловыми, способными к бесшовному размножению во всех направлениях. Для устранения назойливой периодичности мультитекстуры, собранной из одинаковых плиток, можно создавать **мультивариантные** тайловые текстуры с одинаковыми граничными узловыми элементами (4.15в), но с разными внутренними элементами $e_{11} \div e_{n-1,m-1}$.

□ Одномерная нормализованная интерполяция между узловыми элементами A и B производится по формуле

$$C(t) = A \cdot (1 - L(t)) + B \cdot L(t) = \text{lin}(A, B, L(t)) \in [A, B] \quad \forall t \in [0, 1]$$

с **быстро вычисляемой весовой функцией** $L(t)$, обеспечивающей:

- при **обязательном** условии $L(t) \in [0, 1]$ расположение элемента C в выпуклой оболочке (2.35) элементов A и B ;
- при **обязательном** условии $L(0) = 0$ и $L(1) = 1$ непрерывность сопряжения смежных клеток шумовой функции;
- при **желательных** условиях $L'(1) = L'(0) = 0$ и $L''(1) = -L''(0)$ первый и второй порядки гладкости сопряжения смежных клеток.

На рис. 4.38 построены графики и примеры градиентной закраски от чёрного цвета до белого для некоторых полиномиальных ВФ:

- линейной ВФ $L_1(t) = t$, при которой $C(t)$ превращается в (2.10а);
- нелинейных ВФ $L_3(t) = 3t^2 - 2t^3$ и $L_5(t) = 10t^3 - 15t^4 + 6t^5$, создающих двойную гладкость сопряжения и более контрастные изображения шума за счёт большего градиента $L'(0.5)$.

Возможно использование неполиномиальной (косинусной) весовой функции $L_{\cos}(t) = 0.5(1 - \cos(\pi t))$, отличающейся от $L_3(t)$ максимум на 0.01, но вычислительно более трудоёмкой. Поэтому её следует признать **неэффективной**.

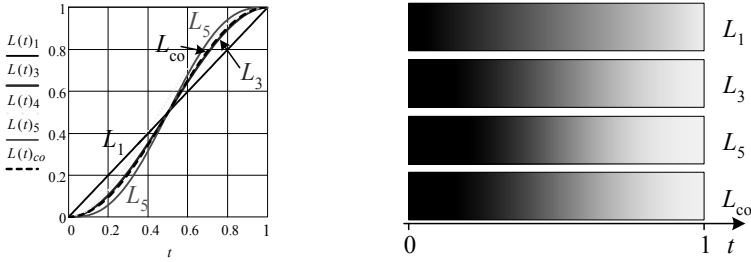


Рис. 4.38

Двухмерная (**бипараметрическая**) нормализованная интерполяция между угловыми элементами клетки $e_{ij} \div e_{i+1,j+1}$ (см. рис. 4.37) по параметрам

$$t = v/h - i \in [0, 1], \quad \tau = u/d - j \in [0, 1] \quad (4.15\Gamma)$$

выполняется по формуле

$$F_{vu} = \text{lin}(\text{lin}(e_{ij}, e_{i+1,j}, L(t)), \text{lin}(e_{i,j+1}, e_{i+1,j+1}, L(\tau)), L(\tau)) \quad \forall v = \overline{0, N}, u = \overline{0, M}, \quad (4.15\Delta)$$

которая при использовании двух линейных ВФ $L_1(t)$ и $L_1(\tau)$ превращается в метод **билинейной** интерполяции (4.2). Важно, что благодаря перебору в (4.15д) текселов во всей карте, а не в отдельных клетках, текстура будет закрашена без наложений и пропусков, а длина и высота клетки в (4.15а) **могут не быть степенями 2 и даже целочисленными!** Это особенно ценно при выборе размеров решётки произвольными целыми числами $n \leq N$ и $m \leq M$.

Результаты бипараметрической интерполяции четырёх угловых цветов с помощью ВФ L_1 , L_3 и L_5 показаны на рис. 4.39, а оценку эстетичности каждой закрашки оставляем на вкус читателя.



Рис. 4.39

Итак, в (4.15) дан алгоритм создания клеточного шума. Особенность того или иного ГКШ заключается в выборе (расчёте) узловых значений решётки e_{ij} .

□ В простейшем клеточном **генераторе численного шума** в узлах решётки формируются случайные числа $e_{ij} = \text{rnd}(1) \in [0, 1]$. Все значения шумовой функции F_{vu} лежат в единичном интервале, но из-за ничтожной вероятности генерирования функцией $\text{rnd}(1)$ чисел 0 и 1 максимально в нём не растянуты. Для повышения контраста рекомендуется нормализовать карту F по (4.13а).

Покажем на рис. 4.40 работу генератора при исходных данных $N=120$, $M=150$, $n=4$, $m=5$ и размерами клеток $d=h=30$ текселов:

а) на случайных узловых элементах тайловой решётки

$$E = \begin{bmatrix} 0.737 & 0.797 & 0.148 & 0.318 & 0.01 & 0.737 \\ 0.259 & 0.647 & 0.366 & 0.696 & 0.184 & 0.259 \\ 0.562 & 0.695 & 0.595 & 0.096 & 0.044 & 0.562 \\ 0.736 & 0.793 & 0.627 & 0.525 & 0.103 & 0.736 \\ 0.737 & 0.797 & 0.148 & 0.318 & 0.01 & 0.737 \end{bmatrix} \quad (4.16)$$

сгенерирована карта $F^{<n \times m>}$ с экстремальными значениями $\min(F) \approx 0.01$ и $\max(F) \approx 0.797$. Изображение получено по (4.14a) цветовым смешиванием $\text{mix2}(c_{\text{TK}}, c_{\text{CG}}, F)$ от тёмно-красного цвета $c_{\text{TK}} = [128 \ 0 \ 0]$ до светло-голубого $c_{\text{CG}} = [128 \ 255 \ 255]$;

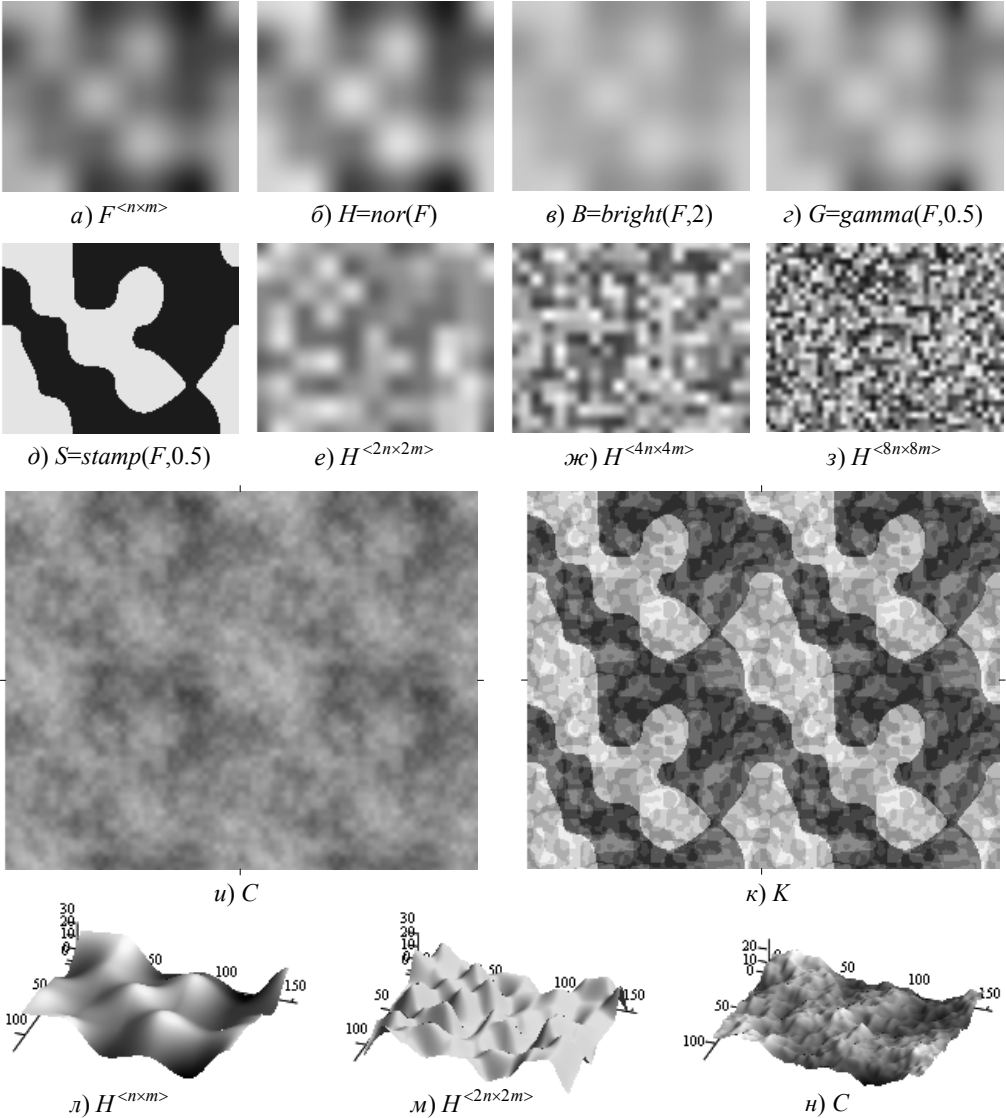


Рис. 4.40

б) на той же решётке по (4.13а) создана нормализованная карта $H = \text{nor}(F)$ с $\min(H)=0$, $\max(H)=1$ и хорошо заметным повышением контраста;

в) на той же решётке по (4.13б) сгенерирована карта $B = \text{bright}(F, 2)$ с $\min(B)=0.505$, $\max(B)=0.899$ и вдвое большей яркостью, чем у карты F ;

г) по (4.13в) произведена гамма-коррекция $G = \text{gamma}(F, 0.5)$ с $\min(G)=0.1$, $\max(G)=0.893$ и бóльшим контрастом, чем у карты B ;

д) на той же решётке по (4.13г) построен штамп $S^{<n \times m>} = \text{stamp}(F^{<n \times m>}, 0.5)$ с пороговым уровнем $f_1=0.5$;

е) изображение карты первой октавы $H^{<2n \times 2m>}$ получено по (4.14а) линейным смешиванием $\text{mix2}(c_{\text{тз}}, c_{\text{ж}}, H^{<8 \times 10>})$ от тёмно-зелёного цвета $c_{\text{тз}} = [0 \ 128 \ 0]$ до жёлтого $c_{\text{ж}} = [255 \ 255 \ 0]$;

ж) карта второй октавы $H^{<4n \times 4m>}$ скомбинирована линейным смешиванием $\text{mix2}(c_{\text{б}}, c_{\text{к}}, H^{<16 \times 20>})$ от белого $c_{\text{б}} = [255 \ 255 \ 255]$ до красного $c_{\text{к}} = [255 \ 0 \ 0]$;

з) карта третьей октавы $H^{<8n \times 8m>}$ получена линейным смешиванием $\text{mix2}(c_{\text{с}}, c_{\text{б}}, H^{<32 \times 40>})$ от синего $c_{\text{с}} = [0 \ 0 \ 255]$ до белого;

и) разноцветный шум $C = 0.5H^{<n \times m>} + 0.2H^{<2n \times 2m>} + 0.2H^{<4n \times 4m>} + 0.1H^{<8n \times 8m>}$ получен покомпонентным смешиванием по (4.14в) четырёх октавных карт и бесшовно размножен в два ряда и две колонки;

к) разноцветный камуфляж $K = 0.5S^{<n \times m>} + 0.2S^{<2n \times 2m>} + 0.2S^{<4n \times 4m>} + 0.1S^{<8n \times 8m>}$ получен покомпонентным смешиванием четырёх октавных штампов и бесшовно размножен в два ряда и две колонки.

Процедурный шум, и не только численный, имеет полезное применение как *генератор 3d-ландшафта* [53], в котором значения шумовой функции рассматриваются не как цвета текселов плоского изображения, а как *карта высот* рельефа на плане $v \times u$ по которой строится полигональная сеть. На рис. 4.40, л–н показаны поверхности карт $H^{<n \times m>}$ (рис. 4.40, б), $H^{<2n \times 2m>}$ (рис. 4.40, е) и одной секции тайловой текстуры на рис. 4.40, и. Для получения хорошей выпуклости рельефа значения карт увеличены в 30 раз.

Достаточно подробно изученный и проиллюстрированный генератор численного шума работает с узловыми значениями решётки e_{ij} , лежащими в единичном интервале $[0, 1]$. Из сопоставления значений матрицы E в (4.16) с яркостью участков текстур на рис. 4.40, а–г видно, что значениям $e_{ij} \approx 1$ соответствуют светлые оттенки, а малым значениям $e_{ij} \approx 0$ — тёмные. Это весьма напоминает процесс сглаживания закраски полигональных сетей методом Гурó (см. п. 4.2.3), где в узлах каркаса рассчитываются освещённости, пропорциональные косинусам углов видимости $\cos(\psi_{ij}) \in [0, 1]$. Поэтому вполне логично назвать этот генератор *шумом Гурó*.

□ Другую группу ГКШ представляют *градиентные шумы*, для генерации которых в узлах решётки $q_{ih,jd}$ задаются *векторы градиентов* $N_{ij} \in \mathbb{R}^3$. При условии *неотрицательности* z -координат $(N_{ij})_z \geq 0$ их можно согласно (1.93а)

интерпретировать как нормали к некоторой поверхности, которую наблюдатель видит окрашенной по закону Ламберта. Предлагаются два способа формирования градиентных векторов:

- с **углами**, равномерно распределёнными в верхней единичной **полусфере**

$$N_{ij} = z^0 R_x(\text{rnd}(\pi/2)) R_z(\text{rnd}(2\pi)) \quad \forall i=\overline{0,n}, j=\overline{0,m}; \quad (4.17a)$$

- с **координатами**, равномерно распределёнными в верхнем **полукубе**

$$N_{ij} = [\text{urf}(1) \text{urf}(1) \text{rnd}(1)] \quad \forall i=\overline{0,n}, j=\overline{0,m}. \quad (4.17б)$$

В отличие от шума Гуро интерполяции подвергаются не освещённости $\cos(\psi_{ij})$, а некоторые другие, связанные с градиентами, случайные числа, чем и достигается разнообразие **генераторов градиентного шума**.

В простейшем градиентном генераторе **шума Фонга**, напоминающем одноимённый метод сглаживания (см. п. 4.2.3), интерполируемыми в (4.15д) элементами e_{ij} являются случайные **нормированные** векторы нормалей N_{ij} , а значением шумовой функции принимается z -координата $(F_{vu})_z \in [0, 1]$, равная косинусу угла видимости при векторе наблюдателя $H = z^0$.

При выборе метода генерирования нормалей (4.17а,б) получаются текстуры (рис. 4.41,а,б), схожие своей размытостью с текстурой на рис. 4.40,и. Обе они, как и разноцветный штамп на рис. 4.41,в, построены как 2×2 -тайловые четырёхоктавные смеси (4.14в) на основе 0-октавной решётки с параметрами $N=80$, $M=100$, $n=3$ и $m=5$.

Возможны ещё два способа генерирования градиентных векторов:

- с **углами**, равномерно распределёнными в **полной** единичной сфере

$$N_{ij} = z^0 R_x(\text{rnd}(\pi)) R_z(\text{rnd}(2\pi)) \quad \forall i=\overline{0,n}, j=\overline{0,m}; \quad (4.17в)$$

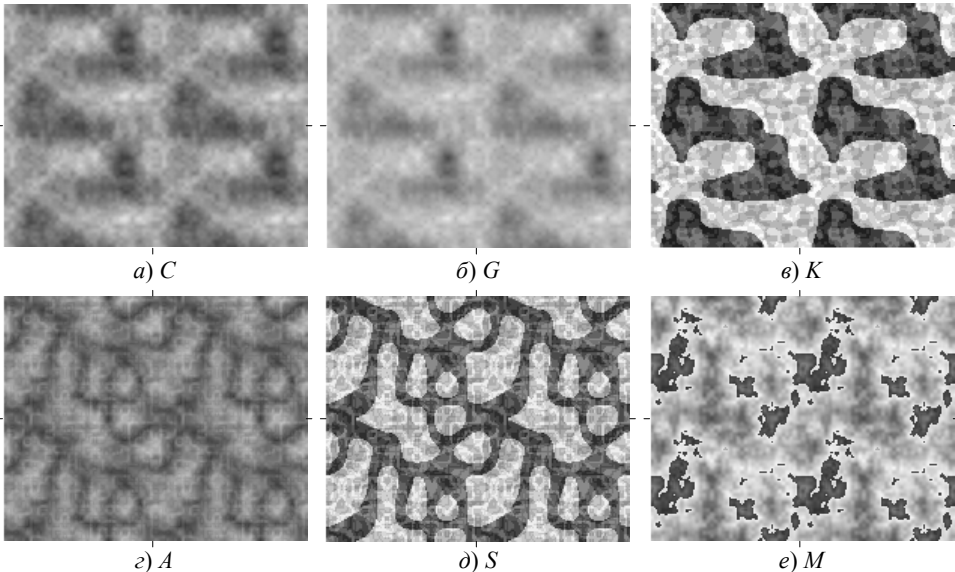


Рис. 4.41

- с **координатами**, равномерно распределёнными в **полном** кубе

$$N_{ij} = \overline{urf(1) \ urf(1) \ urf(1)} \quad \forall i = \overline{0, n}, \ j = \overline{0, m}. \quad (4.17r)$$

Эти векторы уже некорректно полагать нормальными к шумовой поверхности, по которым можно рассчитать освещённость в узлах решётки, т. к. для узлов с $(N_{ij})_z < 0$ получаются $\cos(\psi_{ij}) < 0$ и по (1.181) эти точки будут **не освещёнными**.

Если же отойти от концепции освещённости и сгенерировать градиентные векторы по (4.17в) или (4.17г), то получатся шумовые функции с элементами $(F_{vu})_z \in [-1, 1]$. Такие ненормализованные карты нужно обязательно подвергнуть преобразованию, вгоняющему все их значения в интервал $[0, 1]$. Это могут быть нормализация (4.13а) $H = nor(F)$, смещение и масштабирование $H = 0.5(F+1)$, формально совпадающее с двойным повышением яркости $bright(F, 2)$ из (4.13б), ограничение (4.13д) $clamp(F, 0, 1)$, абсолютная (4.15д) либо модульная (4.15е) фильтрации. Результаты этих преобразований, естественно, разные.

Наглядное представление о ненормализованном шуме, обработанном нелинейными фильтрами, дают тайловые текстуры $A = abs(F)$ (рис. 4.41,з) и $S = stamp(A, 0.3)$ (рис. 4.41,д). А применение модульной фильтрации (4.15е) к смещённой текстуре $F+1 \in [0, 2]$ даёт текстуру $M = modul(F+1)$ (рис. 4.41,е), в которой там, где зелёная компонента цвета превышала уровень 1, произошло её смещение на -1 , из-за чего цвета этих текселов стали оттенками розового.

□ Наиболее известным градиентным шумом является **шум Перлина** (*Perlin noise*), созданный Кеном Перлином в 1983 г. и нашедший широкое применение для имитации облаков, волн, огня, природных поверхностей и т. п. В 1997 г. за вклад своей работы по процедурным текстурам в индустрию кино Перлин был удостоен премии «Оскар» [66].

Работа генератора происходит **в плоскости решётки** и начинается с формирования градиентных векторов $N_{ij} \in R^2$ любым из двух способов, аналогичных 3d-версиям в (4.17в,г):

- с **углами**, равномерно распределёнными по единичной окружности

$$N_{ij} = x^0 R(rnd(2\pi)) \quad \forall i = \overline{0, n}, \ j = \overline{0, m}; \quad (4.18a)$$

- с **координатами**, равномерно распределёнными в единичном квадрате

$$N_{ij} = \overline{urf(1) \ urf(1)} \quad \forall i = \overline{0, n}, \ j = \overline{0, m}. \quad (4.18б)$$

Далее для каждого тексела q_{vu} определяются (рис. 4.42):

- по (4.15б) клеточные координаты (i, j) ;
- по (4.15г) клеточные параметры точки $p_\tau = [\tau \ t]$;
- из углов клетки в точку p_τ проводятся векторы

$$\begin{aligned} V_{00} &= [\tau \ t], & V_{01} &= [\tau-1 \ t], \\ V_{10} &= [\tau \ t-1], & V_{11} &= [\tau-1 \ t-1]; \end{aligned} \quad (4.18в)$$

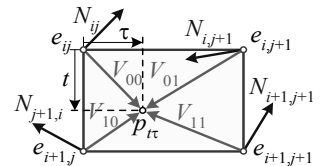


Рис. 4.42

- скалярные произведения градиентных и угловых векторов

$$e_{ij}=N_{ij}\circ V_{00}, \quad e_{i,j+1}=N_{i,j+1}\circ V_{01}, \quad e_{i+1,j}=N_{i+1,j}\circ V_{10}, \quad e_{i+1,j+1}=N_{i+1,j+1}\circ V_{11}; \quad (4.18\text{г})$$

- эти числа используются в (4.15д) для вычисления значения шума P_{vu} .

Шум Перлина обладает следующими **свойствами**:

- благодаря нормализованной параметризации (4.15г) все клетки решётки параметрически квадратны при любых геометрических размерах (4.15а);
- во всех узлах решётки значения карты P равны нулю ввиду обнуления соответствующих векторов (4.18в). Это объясняет появление в Перлин-текстурах горизонтально и вертикально ориентированных прямоугольных фигур;
- на одноклеточной тайловой решётке с $n=m=1$ и равными узловыми векторами $N_{00}=N_{01}=N_{10}=N_{11}$ значения карты кососимметричны: $P_{\tau\tau}=-P_{1-\tau,1-\tau}$;
- все значения карты гарантированно лежат в интервале $[-\sqrt{0.5}, \sqrt{0.5}]$ (реально в гораздо более узком), поэтому она нуждается в нормализации.

Возможности шума Перлина проиллюстрированы на рис. 4.43, где показано несколько его тайловых реализаций с размерами карты $N=120$ и $M=150$, с

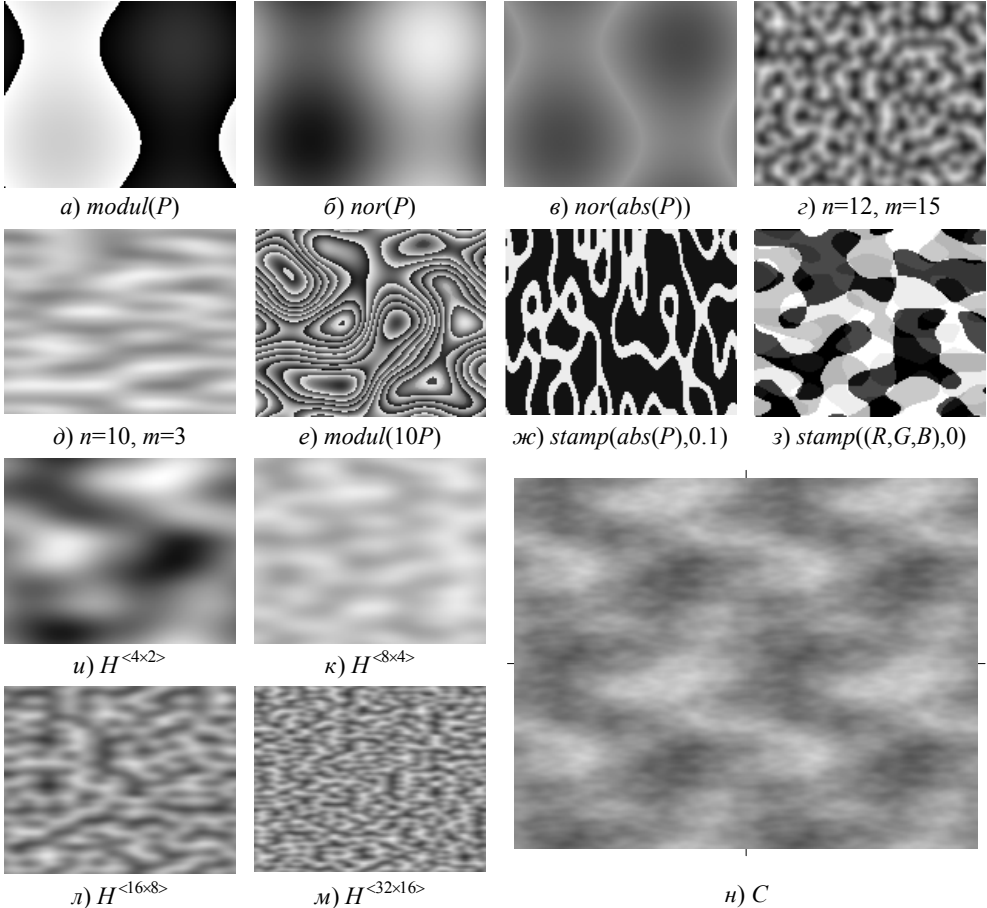


Рис. 4.43

разными размерами решётки $n \times m$, разными фильтрами (4.13) и цветовыми/фрактальными смесями (4.14):

а) карта $P \in [-0.197, 0.197]$ создана на одноклеточной решётке и автоматически промодулирована видеоадаптером так, что отрицательные значения $P_{vi} < 0$ заменились на положительные $P_{vi} + 1$;

б) та же карта P нормализована по (4.13а);

в) та же карта преобразована абсолютным фильтром (4.13е) и нормализована;

г) нормализованный Перлин-шум на решётке из 12×15 квадратных клеток;

д) волноподобный шум построен на решётке из 10×3 горизонтально вытянутых клеток;

е) шум построен на решётке 3×3 , усилен в 10 раз и модулирован по (4.13ж);

жс) шум, построенный на решётке 3×10 , абсолютизирован по (4.13е) и отштампован по (4.13г) с порогом $f_1 = 0.1$, влияющим на толщину светлых линий;

з) на решётке из 6×5 клеток созданы и соединены в конечное изображение независимые цветовые Перлин-карты $stamp(R, 0)$, $stamp(G, 0)$ и $stamp(B, 0)$;

и) – м) карта $H^{<4 \times 2>}$ и три октавные карты $H^{<8 \times 4>}$, $H^{<16 \times 8>}$ и $H^{<32 \times 16>}$;

н) фрактальная смесь $C = 0.5H^{<4 \times 2>} + 0.2H^{<8 \times 4>} + 0.2H^{<16 \times 8>} + 0.1H^{<32 \times 16>}$, размноженная в два ряда и две колонки.

Как было сказано выше, можно разнообразить восприятие размноженного бесшовного изображения, если во всех «плитках» граничные узловые элементы решётки делать одинаковыми, а внутренние — разными [45]. На рис. 4.44 показаны периодическая 2×3 -укладка (а) из шести одинаковых плиток и мультивариантная тайловая текстура (б), в которой все плитки отличаются друг от друга внутри, но совпадают по границам сшивки. Левое изображение создано смешиванием по (4.14а) двух цветов (от тёмно-коричневого до жёлтого), а правое — по (4.14б) трёх цветов (от коричневого через красный до жёлтого).

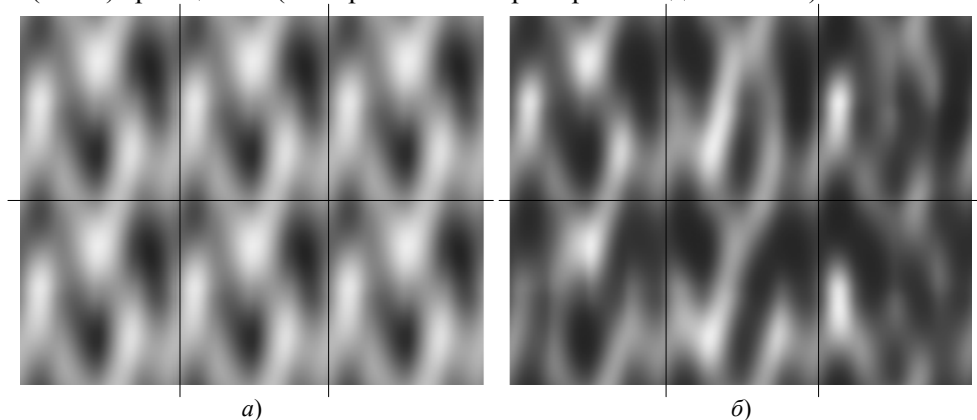


Рис. 4.44

Метод мультивариантного тайлинга предназначен для бесшовного **последовательного** покрытия больших площадей с меньшими затратами памяти в сравнении с ресурсоёмким одноразовым генерированием огромной текстуры.

Ячеистый шум

Сотообразный или **ячеистый шум** (*Cellular noise*), изобретённый Стивеном Уорли (**Steven Worley**) в 1996 г., представляет собой изображение, созданное путём генерации **опорных точек** в пространстве карты W , вычисления мер расстояний между ними и текселями и их преобразования в цвета текселей.

Каждый из трёх этапов работы генератора шума Уорли может быть реализован множеством вариантов, что объясняет большое разнообразие текстур и их широкое распространение в графике, дизайне, искусстве и т. д. Рассмотрим некоторые возможные варианты исходных данных и методов их обработки.

 **Этап 1.** Количество $n_T \geq 1$ опорных точек $p_k \in W \ \forall k = \overline{1, n_T}$ может быть в зависимости от заданного **флага случайности** $f_c \in \{0, 1\}$ либо задано фиксированным числом $n_T = N_T$ при $f_c = 0$, либо сгенерировано как случайное целое число $n_T = \lceil \text{rnd}(N_T) \rceil \in [1, N_T]$ при $f_c = 1$. Применение быстрого равномерного ГСЧ рационального генератора с более сложным распределением Пуассона, предлагаемым автором шума в [66].

Расположение опорных точек в пространстве карты также в зависимости от f_c может быть либо задано вручную, либо сгенерировано случайно. Для более равномерного распределения случайных точек по площади карты можно создать условную прямоугольную $n \times m$ -решётку (рис. 4.45,а) и в каждой (i_k, j_k) -й клетке размерами $h \times d$ генерировать в соответствии с f_c заданное число N_T либо случайное целое число $\lceil \text{rnd}(N_T) \rceil \in [1, N_T]$ точек

$$p_{++n_T} = [i_k h - \text{rnd}(h) \quad j_k d - \text{rnd}(d)] \quad \forall i_k = \overline{1, n}, j_k = \overline{1, m}, h = N/n, d = M/m, \quad (4.19)$$

накапливая при этом общее число опорных точек n_T , начиная с $n_T = 0$. Без использования решётки метод (4.19) работает при $n = m = 1$.

Этап 2. Во вложенных циклах $v = \overline{0, N}$, $u = \overline{0, M}$ для каждого элемента текстуры $q_{vu} = [u \ v]$ в цикле $k = \overline{0, n_T}$ вычисляются **нормы расстояний** l_k до каждой опорной точки $p_k = [x_k \ y_k]$ в соответствии с выбранным методом определения норм векторов $q_{vu} - p_k$. Если генерируемая текстура должна быть тайловой, то модули координат $x = |v - x_k|$ и $y = |u - y_k|$, бóльшие половин соответствующих размеров карты, корректируются так, будто точка p_k размножена в соседние плитки и стала к текселу ближе (рис. 4.45,б):

- если $2x > M$, то $x = M - x$;
- если $2y > N$, то $y = N - y$.

Обычно мерой длины служит **евклидова норма** $L_2 = \sqrt{x^2 + y^2}$, но интересны и другие метрики $L_s = (x^s + y^s)^{1/s}$ [10], например, **манхэттенская норма** $L_1 = x + y$ (рис. 4.45,в), **∞ -норма** $L_\infty = \max\{x, y\}$ (рис. 4.45,г) либо любая собственная норма $L(x, y) \geq 0$, дающая привлекательные рисунки текстуры. На текстурах рис. 4.45 построены красные эквинормальные линии L_2 , L_1 и L_∞ .

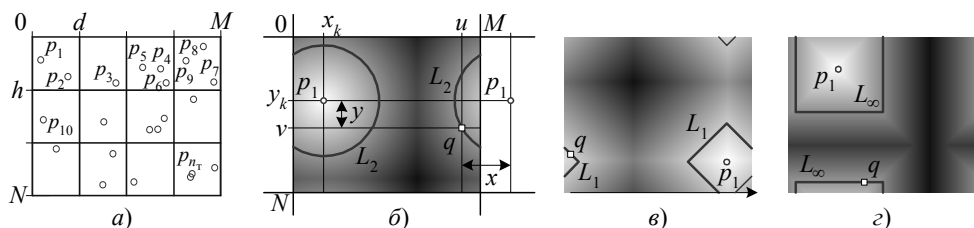


Рис. 4.45

В расширенном варианте генератора шума Уорли список длин $\{l_1, l_2, \dots, l_{n_t}\}$ дополняется номерами опорных точек $1 \div n_t$ и цветовыми векторами $c_1 \div c_{n_t}$, заданными в зависимости от флага f_c вручную либо сгенерированными случайно как $c_k = [\text{rnd}(1) \text{ rnd}(1) \text{ rnd}(1)]$. Результат внутреннего цикла оформим матрицей

$$P_{vu} = \begin{bmatrix} l_1 & l_2 & \dots & l_{n_t} \\ 1 & 2 & \dots & n_t \\ c_1 & c_2 & \dots & c_{n_t} \end{bmatrix} \begin{matrix} l \\ k \\ c \end{matrix}, \quad (4.20a)$$

у которой верхняя строка содержит меры расстояний от (v, u) -го тексела до всех опорных точек. После **сортировки** столбцов матрицы (4.20a) по возрастанию элементов строки l в первом столбце полученной матрицы

$$\underset{\uparrow l}{\text{sort}}(P_{vu}) = \begin{bmatrix} l_{vu1} & l_{vu2} & \dots & l_{vun_t} \\ k_{vu1} & k_{vu2} & \dots & k_{vun_t} \\ c_{vu1} & c_{vu2} & \dots & c_{vun_t} \end{bmatrix} \quad (4.20б)$$

будут собраны расстояние до ближайшей опорной точки, номер этой точки и присвоенный ей цвет. Второй столбец будет содержать расстояние, номер и цвет второй по близости точки и т. д. Из элементов матриц (4.20б), соответствующих желаемому типу карты, компонуется матричные шумовые функции первого F_1 , второго F_2 , третьего F_3 и т. д. порядков близости:

- **дистантные** карты, составленные из длин $\{l_{vu1}\}, \{l_{vu2}\}, \{l_{vu3}\}, \dots$, являются цвето-непрерывными, а между собой монотонно неубывающими $F_1 \leq F_2 \leq F_3 \leq \dots$;

- **ID-карты** I , составленные из идентификационных номеров опорных точек $\{k_{vu1}\}, \{k_{vu2}\}, \{k_{vu3}\}, \dots$, дают цвето-разрывные изображения с плоской закраской областей, меняющейся в соответствии с порядком нумерации опорных точек;

- **мозаичные** карты C , составленные из цветовых векторов опорных точек $\{c_{vu1}\}, \{c_{vu2}\}, \{c_{vu3}\}, \dots$, дают цвето-разрывные изображения с плоской закраской областей, меняющейся согласно цветам опорных точек.

Этап 3. Поскольку числа, составляющие дистантную/ID карту, сравнимы с её габаритами/числом опорных точек, то перед цветовым смешиванием по (4.14a,б) (текстуры на рис. 4.45 получены линейной интерполяцией норм от белого цвета до синего) она должна быть обязательно нормализована по (4.13a). Большое разнообразие ячеистых текстур даёт функциональное преобразование элементов карты, а также фрактальное смешивание (4.14в) карт, созданных с разными размерами решётки n и m .

Изображение мозаичной карты производится прямым выводом на экран составляющих её RGB-векторов, конвертированных в используемую цветовую палитру.



На рис. 4.46 представлена галерея тайловых ячеистых текстур:

- в первом ряду показаны дистантные $F_1 \div F_3$, разностная $F_2 - F_1$ и функциональная $\sin(F_1)$ карты, построенные по трём опорным точкам с евклидовой нормой L_2 , нормализованные и интерполированные от жёлтого до синего;
- для сравнения в той же цветовой гамме на основе 22 случайных опорных точек созданы текстуры второго ряда;
- в конце первого столбца изображены ID- и мозаичная карты I_1 и C_1 , построенные на тех же 22 опорных точках. Плоские цвета полигонов ID-карты изменяются от синего до жёлтого цвета в порядке увеличения номеров опорных точек в направлении слева направо и сверху вниз (см. рис. 4.45,а), тогда как в мозаичной текстуре цвета полигонов случайные;
- в конце галереи представлены две тайловые 2×2 сборки: левая создана с нормой L_∞ , а правая — фрактальным смешиванием трёх октавных решёток.

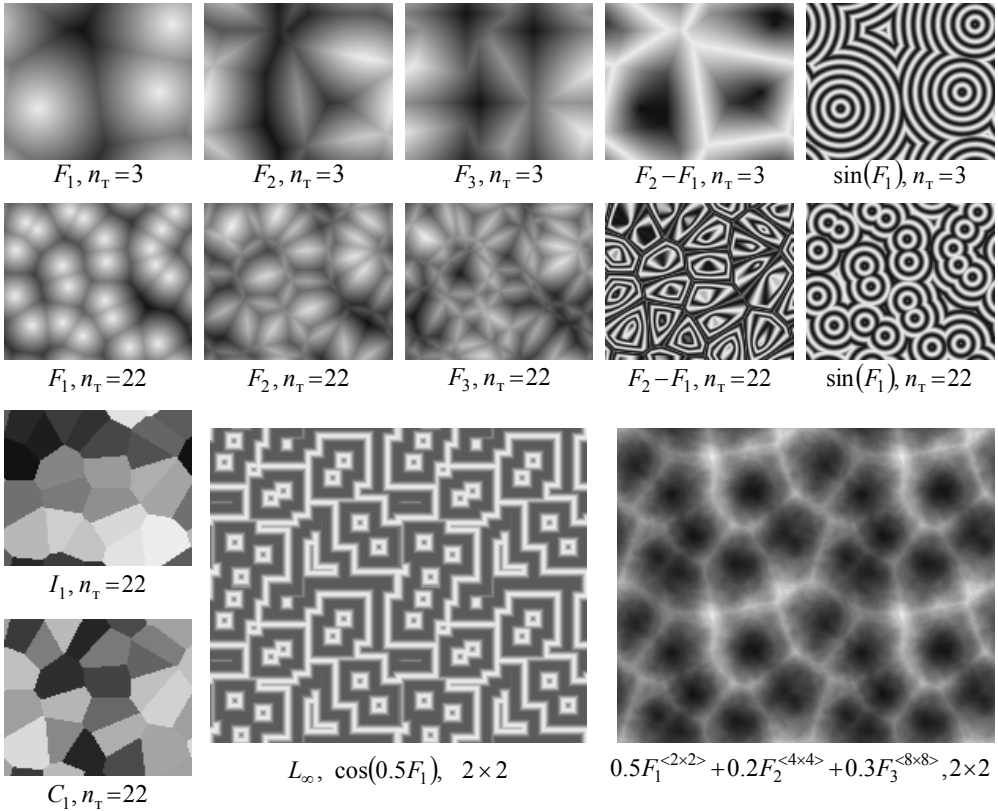


Рис. 4.46

4.3. КИНЕМАТИЧЕСКИЕ ПОВЕРХНОСТИ

Кинематические поверхности формируются непрерывным движением в пространстве некоторой линии по определённой траектории. Параметрическая модель кинематической поверхности имеет вид

$$p(t, \tau) = p_o(\tau)A(t, \tau) + p_n(t) \quad (4.21a)$$

или в расширенном пространстве

$$\tilde{p}(t, \tau) = \tilde{p}_o(\tau)C(t, \tau), \quad \text{где } C(t, \tau) = \begin{bmatrix} A(t, \tau) & O_{3,1} \\ p_n(t) & 1 \end{bmatrix}. \quad (4.21б)$$

Математическая модель (4.21) включает следующие **кинематические элементы** (рис. 4.47):

- $p_o(\tau)$ — исходное состояние **образующей линии** с центром (базовой точкой) в начале СК;
- $p_n(t)$ — **направляющую линию**, по которой движется центр образующей линии;
- $A(t, \tau)$ — **матрицу преобразования** линии $p_o(\tau)$, согласованного с траекторией $p_n(t)$;
- **интервалы изменения параметров** $t \in [t_n, t_k]$ и $\tau \in [\tau_n, \tau_k]$ от начальных $\{t_n, \tau_n\}$ до конечных $\{t_k, \tau_k\}$ значений.

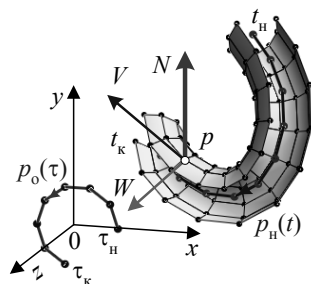


Рис. 4.47

Фиксация центра образующей линии в точках $p_n(t_i)$ даёт **поперечные** каркасные линии $p(t_i, \tau)$. Фиксированные точки $p_o(\tau_j)$ при движении образуют систему **продольных** каркасных линий $p(t, \tau_j)$. Вовсе не обязательно, что в точках взаимного пересечения продольные и поперечные линии ортогональны, хотя многие симметричные поверхности этим свойством обладают. При некоторых значениях параметров каркасные линии могут вырождаться в особые точки, как, например, параллели глобуса на его полюсах. Но это лишь **параметрическая** вырожденность, тогда как топологически все точки сферы одинаковы.

Важнейшее значение при конструировании поверхностей имеет правильный выбор кинематических элементов. Для любой пары параметров $\{t, \tau\}$ векторная функция $p(t, \tau)$ должна быть:

• **вычислимой** — это обеспечивает доступ к каждой точке поверхности. Примером поверхности, не вычислимой на двух особых линиях, служит четырёхполостная модель гиперболического параболоида (1.167б). Вычислимая единая модель той же поверхности, построенная на рис. 1.142,д, задаётся параметрической функцией (1.167а);

• **однозначной** — это важно при расчёте пересечения периодических поверхностей с другими геометрическими объектами (отрезками, лучами, плоскостями и т. д.). Уж на что проста кинематическая модель сферы

$$p(t, \tau) = [\sin(\tau) \cos(\tau) 0]R_y(t) = [\sin(\tau)\cos(t) \cos(\tau) -\sin(\tau)\sin(t)] \quad (4.22)$$

как поверхности, образованной вращением окружности $\sin(\tau)x^0 + \cos(\tau)y^0 \in xy$ вокруг оси y , так и для неё часто задают интервалы параметров $t \in [0, 2\pi]$ и $\tau \in [0, 2\pi]$, исходя только из периода 2π тригонометрических функций $\sin$ и $\cos$ [21, 51]. В результате произвольная точка такой поверхности приобретает как минимум двойное параметрическое описание $\{t, \tau\}$ и $\{t \pm \pi, 2\pi - \tau\}$. Например, точке экватора $[0 \ 0 \ 1]$ соответствуют две пары параметров $\{t = 3\pi/2, \tau = \pi/2\}$ и $\{t = \pi/2, \tau = 3\pi/2\}$. А точки на меридиане склейки имеют по три пары параметрических координат: $\{0, \tau\}$, $\{2\pi, \tau\}$ и $\{\pi, 2\pi - \tau\}$.

Гарантией правильного выбора интервалов изменения параметров функции $p(t, \tau)$ является **учёт кинематики формирования поверхности**, а именно, прохождение образующей линии не более раза по одной и той же её части. Например, сферу (4.22) с однозначным описанием не граничных точек можно получить вращением полной окружности ($\tau \in [0, 2\pi]$) на 180° ($t \in [0, \pi]$) либо вращением полуокружности ($\tau \in [0, \pi]$) на 360° ($t \in [0, 2\pi]$). Второй вариант предпочтительнее, т. к. он даёт на поверхности одну линию склейки вместо двух;

• **дифференцируемой** — это необходимо в алгоритмах визуализации, вычисляющих векторы нормали в точках поверхности для оценки их видимости и расчёта освещённости. В каждой точке $p(t, \tau)$ должны существовать направляющие векторы касательной плоскости

$$\begin{cases} V(t, \tau) = \frac{\partial p(t, \tau)}{\partial t} = p_o(\tau) \frac{\partial A(t, \tau)}{\partial t} + \frac{dp_n(t)}{dt}, \\ W(t, \tau) = \frac{\partial p(t, \tau)}{\partial \tau} = \frac{dp_o(\tau)}{d\tau} A(t, \tau) + p_o(\tau) \frac{\partial A(t, \tau)}{\partial \tau} \end{cases} \quad (4.23)$$

и образующий с ними правую тройку вектор нормали

$$N(t, \tau) = V(t, \tau) \times W(t, \tau). \quad (4.24a)$$

Для того чтобы нормаль была ненулевой и вычислимой во всех точках поверхности, **векторы $V(t, \tau)$ и $W(t, \tau)$ не должны быть нулевыми или параллельными**. Это принципиальное условие работоспособности многих алгоритмов удаления, например, Робертса, и моделей закрашивания, например, (4.1).

Неудачная модель эллипсоида с невычислимыми по (4.23) векторами $V(t, \tau)$ на полюсах ($t = \pm 1$) приведена в (1.163б). В то же время не следует идеализировать модель (1.163а), поскольку её направляющие векторы

$$\begin{aligned} V(t, \tau) &= [a \cos(t) \sin(\tau) \ -b \sin(t) \ c \cos(t) \cos(\tau)], \\ W(t, \tau) &= [a \sin(t) \cos(\tau) \ 0 \ -c \sin(t) \sin(\tau)] \end{aligned}$$

дают по (4.24а) всюду вычисляемый вектор нормали

$$\begin{aligned} N(t, \tau) &= [bc \sin^2(t) \sin(\tau) \ ac \sin(t) \cos(t) \ absin^2(t) \cos(\tau)] = \\ &= \sin(t) p(t, \tau) M(a^{-1}bc, ab^{-1}c, abc^{-1}), \end{aligned}$$

длина которого на полюсах эллипсоида ($t=0$ и $t=\pi$) обращается в ноль. В остальных точках поверхности нормали имеют переменную длину (рис. 4.48,а). В алгоритмах визуализации важно направление вектора нормали, а не его длина.

Например, в (4.1) используется вектор $\overline{N}$ единичной длины. Поэтому вместо (4.24а) лучше сразу вычислять нормированную нормаль

$$N(t, \tau) = \overline{V(t, \tau) \times W(t, \tau)}, \quad (4.24б)$$

а при нормировании нулевого вектора в **особых точках** — полюсах поверхностей вращения — следует отдельно задавать правильное направление единичного вектора нормали, как, например, на рис. 4.48,б: $N(0, \tau) = y^0$ и $N(\pi, \tau) = -y^0$.

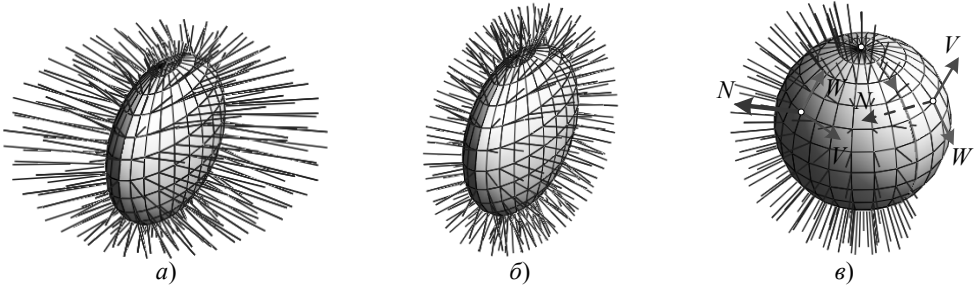


Рис. 4.48

На любом локальном участке поверхность является **двухсторонней**, причём стороны могут различаться собственным цветом и освещённостью. В случае непрозрачной **замкнутой** поверхности бессмысленно тратить машинные ресурсы на визуализацию её заведомо невидимой внутренней стороны. У незамкнутых или полупрозрачных поверхностей деление сторон на внешнюю и внутреннюю по принципу видимости удалённым наблюдателем условно. Назовем **внешней** ту сторону поверхности, которая обращена к вектору нормали (4.24). Алгоритмы визуализации, использующие нормаль, отображают именно внешнюю сторону. Противоположная сторона поверхности является **внутренней**, и для её визуализации используется вектор антинормали, равный $-N(t, \tau)$.

Неудачный выбор кинематических элементов модели (4.21) может привести к неожиданному эффекту **инверсии внешней стороны**. Например, в модели сферы (4.22) с интервалами изменения параметров $t \in [0, \pi]$ и $\tau \in [0, 2\pi]$ половина поверхности при $0 \leq \tau < \pi$ обращена нормалью (4.24б) внутрь, а другая половина — наружу (рис. 4.48,в). Причина этого явления — в инверсии знака ординаты вектора $W(t, \tau)$ при переходе параметра τ через значение π , в результате чего изменилось на противоположное и направление нормали $N(t, \tau)$.

Таким образом, синтез кинематической модели поверхности требует корректного выбора всех её элементов. Расчёт точек $p(t, \tau)$ производится непосредственно в векторной форме (4.21). Тем не менее, иногда будут приводиться и координатные выражения $x(t, \tau)$, $y(t, \tau)$ и $z(t, \tau)$ для тех читателей, которые почему-то вдруг не любят векторы, матрицы и операции с ними.

4.3.1. Поверхности элементарных преобразований

В зависимости от **характера движения образующей линии** $p_0(\tau)$ поверхность можно получить её вращением, переносом вдоль направляющей линии

$p_n(t)$ или комбинированием различных аффинных преобразований. Простейшая кинематическая поверхность **вращения** (поворота [51]) вокруг некоторой координатной оси $\omega \in \{x, y, z\}$ описывается моделями

$$p(t, \tau) = p_o(\tau)A(t), \quad \tilde{p}(t, \tau) = \tilde{p}_o(\tau)C(t) \quad (4.25a)$$

с матрицами преобразований $A(t) = R_\omega(t)$, $C(t) = \tilde{R}_\omega(t)$ и нулевой направляющей линией $p_n(t) = O_3$.

Плоскости, содержащие ось вращения, пересекают поверхность по **меридианам**. **Главный меридиан** лежит в плоскости, параллельной картинной плоскости, и ортографически проецируется на неё без искажений. Плоскости, ортогональные оси вращения, пересекают поверхность по **параллелям**. Параллель с наибольшим диаметром называется **экватором**, с наименьшим — **горлом**, с нулевым — **полюсом** (рис. 4.49, а).

Для получения поверхности **переноса** (заметания [51], экструзии [17])

$$p(t, \tau) = p_o(\tau) + p_n(t) \quad (4.25б)$$

образующая линия движется вдоль направляющей без вращения и масштабирования ($A = E_3$) (рис. 4.49, б). Во избежание получения вырожденных в линию участков поверхности линии $p_n(t)$ и $p_o(\tau)$ нигде не должны быть параллельны, т. е. $|dp_n(t)/dt \times dp_o(\tau)/d\tau| \neq 0$. В вырожденных точках параллельные направляющие векторы $V(t) = p'_n(t)$ и $W(\tau) = p'_o(\tau)$ дают по (4.24а) нулевую нормаль, что неприемлемо для работы алгоритмов визуализации поверхности.

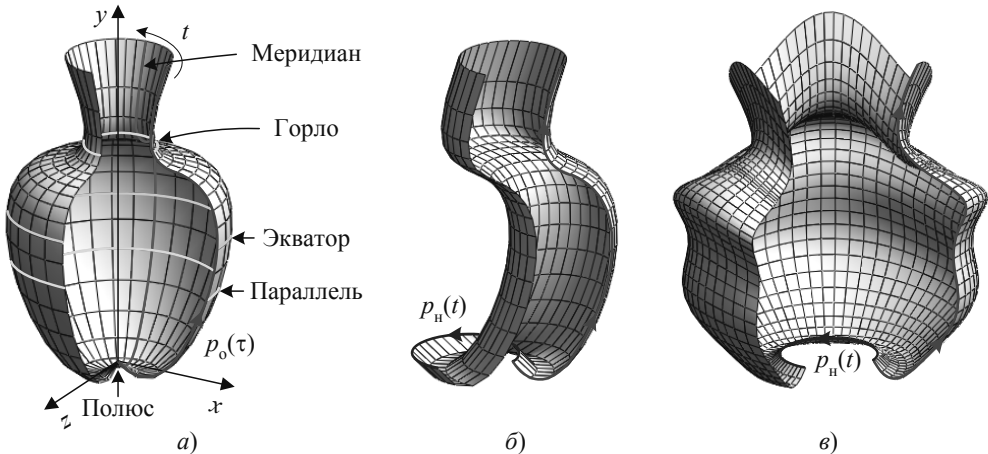


Рис. 4.49

Многие поверхности могут быть получены как вращением, так и переносом образующей. К примеру, боковая поверхность ориентированного вдоль оси x кругового цилиндра с радиусом оснований r и длиной d создаётся вращением образующей прямой $p_o(\tau) = [\tau \ r \ 0]$ вокруг оси x на угол 360° (рис. 4.50, а):

$$p(t, \tau) = p_o(\tau)R_x(t) = [\tau \ r \cos(t) \ r \sin(t)] \quad \forall t \in [0, 2\pi], \tau \in [0, d]. \quad (4.26a)$$

При этом вычисленные по (4.24а) нормальные векторы

$$N(t, \tau) = \begin{vmatrix} x^0 & y^0 & z^0 \\ 0 & -r \sin(t) & r \cos(t) \\ 1 & 0 & 0 \end{vmatrix} = [0 \ r \cos(t) \ r \sin(t)]$$

во всех точках поверхности направлены во внешнее подпространство цилиндра.

Та же поверхность получится протягиванием центра образующей окружности $p_o(\tau) = [0 \ r \cos(\tau) \ r \sin(\tau)] \in yz$ по направляющей линии $p_n(t) = [t \ 0 \ 0]$ на расстояние d (рис. 4.50,б):

$$p(t, \tau) = [t \ r \cos(\tau) \ r \sin(\tau)] \quad \forall t \in [0, d], \quad \tau \in [0, 2\pi]. \quad (4.26б)$$

Во всех её точках нормали $N(t, \tau) = [0 \ -r \cos(\tau) \ -r \sin(\tau)]$ направлены внутрь цилиндра. Обе модели (4.26) идентичны с точностью до перестановки t и τ .

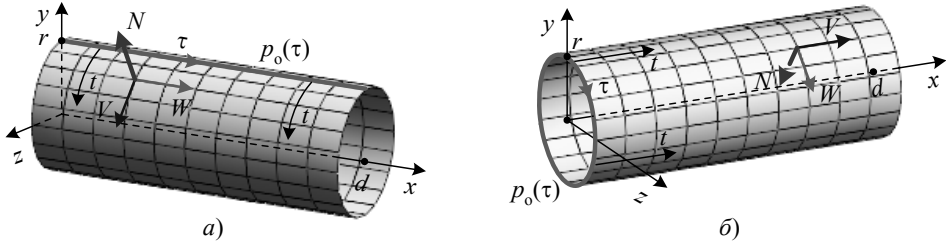


Рис. 4.50

Наибольшее разнообразие поверхностей достигается **комбинированием** преобразований образующей линии в процессе её продвижения вдоль направляющей линии. Так, масштабирование, зависящее от параметров t и τ , позволяет создать на поверхности различные волны (см. рис. 4.49,б), лепестки (см. рис. 4.5) и другие интересные формы. Перенос масштабируемой образующей окружности ортогонально направляющему вектору $dp_n(t)/dt$ лежит в основе построения разнообразных трубопроводов переменного сечения.

Последовательность аффинных преобразований отражена согласно (3.12) в структуре матрицы $C(t, \tau)$. К примеру, поверхность на рис. 4.49,б формируется в цепочке трёх преобразований: масштабирования образующей линии $p_o(\tau)$ матрицей $M(t, \tau)$, вращения вокруг оси y матрицей $R_y(t)$ и переноса на направляющую траекторию $p_n(t)$. В итоге получаем матрицу комбинированного преобразования $C(t, \tau) = \tilde{M}(t, \tau) \tilde{R}_y(t) T(p_n(t))$.

Рассмотрим несколько полезных в практике конструирования скалярных функций $m(\theta)$, модулирующих все или некоторые координаты образующей линии по параметру $\theta = t$ и/или $\theta = \tau$. Богатые возможности их использования на месте диагональных элементов матрицы $M(t, \tau)$ лишь в ограниченной мере иллюстрируются кинематическими поверхностями, построенными на рис. 4.51:

- с помощью функций

$$m(\theta) = 1 \pm a \sin(k\theta), \quad m(\theta) = 1 \pm a \cos(k\theta) \quad (4.27a)$$

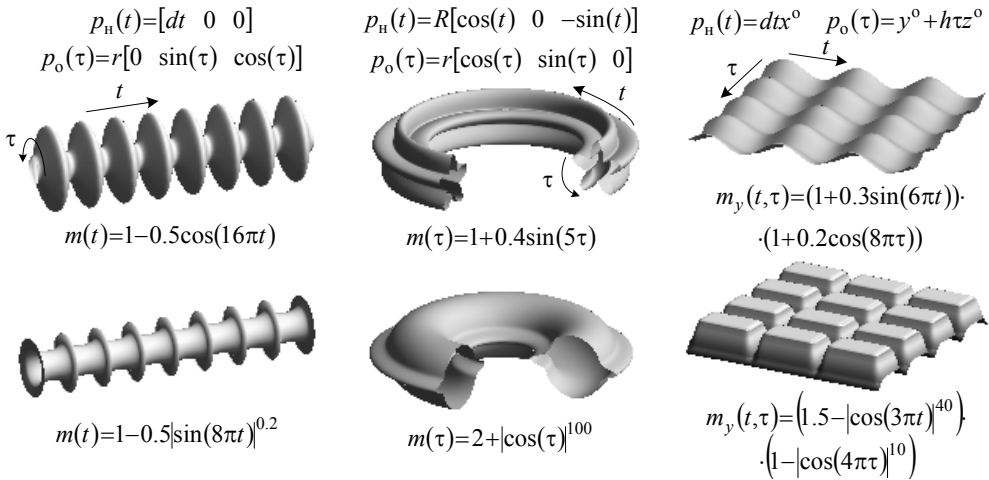


Рис. 4.51

на поверхности создаются *волны* с относительной амплитудой a , повторяющиеся в направлении параметра θ с периодом $2\pi/k$;

- масштабирующие функции

$$m(\theta) = b \pm a |\sin(k\theta)|^n, \quad m(\theta) = b \pm a |\cos(k\theta)|^n \quad (4.276)$$

формируют *полуволны* с амплитудой a от уровня b с периодом π/k параметра θ . Степенной показатель n управляет формой полуволн. При $0 < n \ll 1$ или $n \gg 10$ ровная поверхность разделяется на участки узкими рёбрами или желобами. Особенно ценным является свойство функции $m(t) = |\sin(\pi t/d)|^n$ создавать при $n \leq 0.1$ гладкие замыкания торцов цилиндрической поверхности $p(t, \tau) = [t \quad rm(t)\cos(\tau) \quad rm(t)\sin(\tau)]$ (рис. 4.52). Это освобождает от необходимости строить составную поверхность из трёх гладко сопрягающихся сегментов — бокового цилиндра и двух торцевых крышек. Важно только иметь в виду, что чем меньше выбирается значение показателя $n > 0$, тем на большее число квантов должен быть раздроблен интервал изменения параметра $t \in [0, d]$, чтобы каркас смог охватить крутые торцы модулированной поверхности.

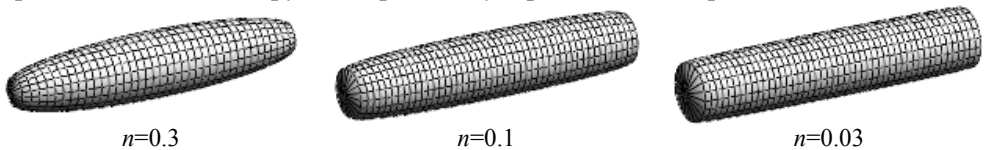


Рис. 4.52

Рассмотрим важную аналитическую задачу **восстановления кинематических элементов** поверхности вращения (4.25а) по её координатным функциям $x(t, \tau)$, $y(t, \tau)$ и $z(t, \tau)$. Восстановление образующей линии $p_o(\tau)$, координатной оси вращения $\omega \in \{x, y, z\}$ и интервалов изменения параметров t и τ необходимо для понимания механизма формирования поверхности и расположения на ней системы каркасных линий. Выполнив преобразование вращения

$$[x(t, \tau) \ y(t, \tau) \ z(t, \tau)] = [x_0(\tau) \ y_0(\tau) \ z_0(\tau)] R_\omega(t),$$

сведём его результаты в табл. 4.1.

Таблица 4.1

Ось	$x(t, \tau)$	$y(t, \tau)$	$z(t, \tau)$
$\omega=x$	$x_0(\tau)$	$y_0(\tau)\cos(t) - z_0(\tau)\sin(t)$	$y_0(\tau)\sin(t) + z_0(\tau)\cos(t)$
$\omega=y$	$x_0(\tau)\cos(t) + z_0(\tau)\sin(t)$	$y_0(\tau)$	$-x_0(\tau)\sin(t) + z_0(\tau)\cos(t)$
$\omega=z$	$x_0(\tau)\cos(t) - y_0(\tau)\sin(t)$	$x_0(\tau)\sin(t) + y_0(\tau)\cos(t)$	$z_0(\tau)$

Анализ координатных функций параметрической поверхности начинается с поиска в них координаты, зависящей лишь от одного параметра τ . Она даёт имя оси ω , вокруг которой вращается образующая, и первую координатную функцию $\omega_0(\tau)$. Анализируя оставшиеся функции ω -й строки табл. 4.1, определяем вторую и третью координаты в векторной функции $p_0(\tau)$. Если образующую задать в какой-либо координатной плоскости, то вторая координата будет содержать $\cos(t)$, а третья — $\sin(t)$. После восстановления вида образующей задаём интервалы изменения параметров t и τ с учётом соображений об однозначности поверхности и направлении её нормали, высказанных выше.

Возможно, что согласование координат функции $p(t, \tau)$ с табл. 4.1 потребует обмена в ней параметров $t \leftrightarrow \tau$, после чего образующая станет функцией $p_0(t)$ параметра t , а матрица вращения — функцией $R_\omega(\tau)$ параметра τ . При этом направление вектора нормали $N(\tau, t)$ изменится на противоположное.

Пример 4.1. Восстановить кинематическую модель верхней полости кругового конуса (см. рис. 1.142, з) как поверхности **вращения**, заданной векторной функцией (1.166) с параметрами $a=1$, b и $c=1$.

Решение. Сопоставив координатные функции $x(t, \tau)=t\sin(\tau)$, $y(t, \tau)=bt$ и $z(t, \tau)=t\cos(\tau)$ со второй строкой табл. 4.1, заключаем, что образующая линия $p_0(t)$ с элементами $y_0(t)=bt$, $x_0(t)=0$ и $z_0(t)=t$ вращается вокруг оси y с помощью матрицы $A(\tau)=R_y(\tau)$.

Итак, поверхность конуса формируется следующим образом (рис. 4.53): прямая линия $p_0(t)=[0 \ bt \ t] \in yz$ вращается вокруг оси y в положительном направлении на угол 360° . Для получения верхней полости конуса высотой h следует задать интервалы изменения параметров $t \in [0, h/b]$ и $\tau \in [0, 2\pi]$.

Построив в любой точке поверхности векторы

$$V(t, \tau) = \frac{dp_0(t)}{dt} R_y(\tau) = [0 \ b \ 1] \begin{bmatrix} \cos(\tau) & 0 & -\sin(\tau) \\ 0 & 1 & 0 \\ \sin(\tau) & 0 & \cos(\tau) \end{bmatrix} = [\sin(\tau) \ b \ \cos(\tau)],$$

$$W(t, \tau) = p_0(t) \frac{dR_y(\tau)}{d\tau} = [0 \ bt \ t] \begin{bmatrix} -\sin(\tau) & 0 & -\cos(\tau) \\ 0 & 0 & 0 \\ \cos(\tau) & 0 & -\sin(\tau) \end{bmatrix} = [t\cos(\tau) \ 0 \ -t\sin(\tau)],$$

касательные к продольной и поперечной каркасным линиям, пересекающимся в этой точке, получим вектор нормали

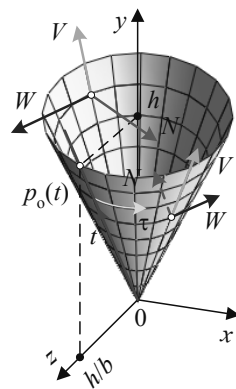


Рис. 4.53

$N(t, \tau) = [-bt \sin(t) \ t - bt \cos(\tau)]$ переменной длины $t\sqrt{b^2+1}$, направленный внутрь конуса. Нормировка и **ручное** сокращение на t даёт существующий во всех точках внутренней поверхности вектор нормали

$$\bar{N}_{\text{вну}}(t, \tau) = \frac{1}{\sqrt{b^2+1}} [-b \sin(\tau) \ 1 - b \cos(\tau)].$$

Нормированная нормаль внешней поверхности конуса имеет вид

$$\bar{N}_{\text{вне}}(t, \tau) = -\bar{N}_{\text{вну}}(t, \tau) = \frac{1}{\sqrt{b^2+1}} [b \sin(\tau) \ -1 \ b \cos(\tau)].$$



Пример 4.2. Построить внешнюю половину трёх четвертей **эллиптического тора** с полуосями образующего эллипса a, b и радиусом направляющей окружности r .

Решение. Выберем образующую и направляющую окружности

$$p_o(\tau) = y^0 R_z(-\tau) = [\sin(\tau) \ \cos(\tau) \ 0] \in xy, \quad p_n(t) = r x^0 R_y(t) = r [\cos(t) \ 0 \ -\sin(t)] \in xz,$$

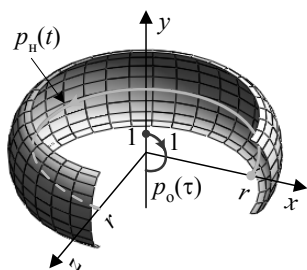


Рис. 4.54

а также интервалы изменения параметров $t \in [0, 3\pi/2]$ и $\tau \in [0, \pi]$, обеспечивающие формирование нужной части тора (рис. 4.54). Масштабируя линию $p_o(\tau)$ по осям x и y матрицей $M = \text{diag}[a \ b \ 0]$ и вращая её вокруг оси y , получим матрицу преобразования

$$A(t) = M R_y(t) = \begin{bmatrix} a \cos(t) & 0 & -a \sin(t) \\ 0 & b & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Подставив найденные кинематические элементы в (4.21а), получим модель поверхности эллиптического тора

$$p(t, \tau) = [(a \sin(\tau) + r) \cos(t) \ b \cos(\tau) \ -(a \sin(\tau) + r) \sin(t)] \quad \forall t \in [0, 3\pi/2], \tau \in [0, \pi]. \quad (4.28)$$

Возможна и другая последовательность преобразований образующей $p_o(\tau)$: сначала она масштабируется матрицей M , затем переносится вдоль оси x на расстояние r , и, наконец, вращается вокруг оси y . Матрица сложного преобразования $C(t) = \tilde{M} T(r x^0) \tilde{R}_y(t)$ даёт модель $\tilde{p}(t, \tau) = \tilde{p}_o(\tau) C(t)$, в точности совпадающую с (4.28). $\square$

Пример 4.3. Полоска бумаги длиной $l = 2\pi r$ и шириной $2h$ сворачивается в кольцо и перед склеиванием один из её концов перекручивается на n полуоборотов, как показано на рис. 4.55,а. Получить модель образующей таким способом **ленты Мёбиуса**.

Решение. Зададим образующий отрезок прямой $p_o(\tau) = h \tau y^0 \in y \quad \forall \tau \in [-1, 1]$ и последовательность таких его преобразований, чтобы за один оборот по направляющей траектории $p_n(t) = r [\cos(t) \ 0 \ -\sin(t)] \in xz \quad \forall t \in [0, 2\pi]$ он повернулся вокруг своего центра на угол $n\pi$:

- первое преобразование — вращение вокруг оси z на угол $\varphi = -n\pi/2$;

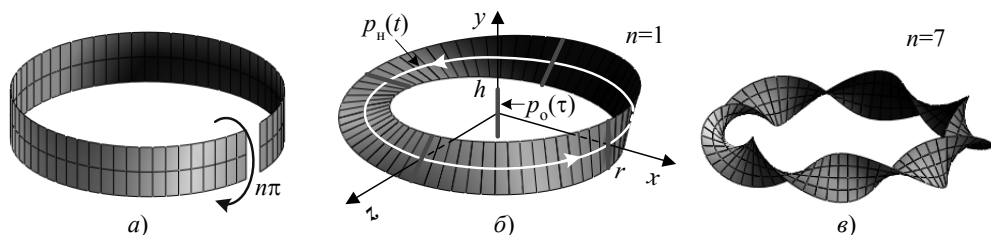


Рис. 4.55

- второе преобразование — вращение вокруг оси y на угол t ;
- третье преобразование — перенос в точку $p_n(t)$.

Подставив в (4.21а) матрицу $A(t)=R_z(\varphi)R_y(t)$, получим модель ленты Мёбиуса:

$$p(t, \tau) = [(r - h\tau \sin(\varphi)) \cos(t) \quad h\tau \cos(\varphi) \quad (h\tau \sin(\varphi) - r) \sin(t)] \quad \forall t \in [0, 2\pi], \tau \in [-1, 1].$$

Есть и другая последовательность преобразований образующего отрезка $p_o(\tau)$: сначала он вращается матрицей $R_z(\varphi)$, затем переносится вдоль оси x на расстояние r и, наконец, вращается матрицей $R_y(t)$. Матрица полного АП $C(t) = \tilde{R}_z(\varphi)T(rx^o)\tilde{R}_y(t)$ даёт модель $\tilde{p}(t, \tau) = \tilde{p}_o(\tau)C(t)$, координатно совпадающую с полученной выше.

При *нечётном* значении n построенная поверхность будет *односторонней* в том смысле, что можно перейти с одной её стороны на другую без проколов и разрывов. Топологически эти две стороны являются единственной стороной поверхности Мёбиуса (рис. 4.55, б). При чётном n поверхность является двусторонней. Задание больших значений n позволяет создать ленту-серпантин (рис. 4.55, в). $\square$

Пример 4.4. Построить модель объекта «шапочка с помпоном» (рис. 4.56, а).

Решение. Сформируем направляющую линию, по которой будет двигаться центр образующей фигуры. Её первый участок (ось шапочки) есть четверть окружности с радиусом r_1 ; второй (ось помпона) — отрезок прямой длиной $2r_2$ (рис. 4.56, б):

$$p_n(t) = \begin{cases} r_1 [\cos(t) \quad \sin(t) \quad 0] & \text{при } 0 \leq t \leq \pi/2, \\ [-r_2(1 + \cos(2t)) \quad r_1 \quad 0] & \text{при } \pi/2 < t \leq \pi. \end{cases}$$

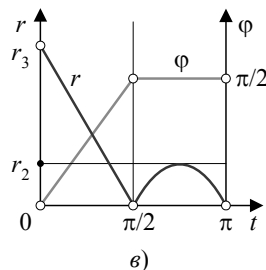
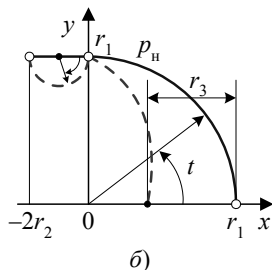
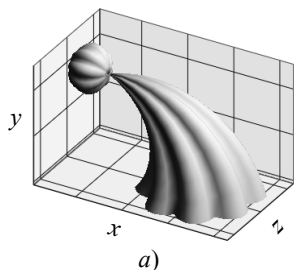


Рис. 4.56

Выбрав образующую окружность $p_o(\tau) = [\cos(\tau) \quad 0 \quad \sin(\tau)] \in xz$, изменим её форму и размер следующими параметрическими коэффициентами:

- функцией масштабирования $m(\tau) = 1 + 0.25|\sin(4\tau)|$, создающей на поверхности восемь продольных выпуклых полуволн;
- функцией радиуса

$$r(t) = \begin{cases} r_3(1 - 2t/\pi) & \text{при } 0 \leq t \leq \pi/2, \\ -r_2 \sin(2t) & \text{при } \pi/2 < t \leq \pi, \end{cases}$$

создающей на первом участке коническое от r_3 до 0, а на втором — сферическое от 0 до r_2 и опять до 0 изменения радиуса поперечного сечения поверхности (рис. 4.56, в).

Далее линия $m(\tau)r(t)p_o(\tau)$ вращается вокруг оси z на угол

$$\varphi(t) = \begin{cases} t & \text{при } 0 \leq t \leq \pi/2, \\ \pi/2 & \text{при } \pi/2 < t \leq \pi \end{cases}$$

(рис. 4.56, в), а её центр переносится в точку $p_n(t)$. Совокупность окружности $p_o(\tau)$, матрицы преобразования $A(t, \tau) = m(\tau)r(t)R_z(\varphi(t))$, линии $p_n(t)$ и интервалов изменения параметров

$t \in [0, \pi]$ и $\tau \in [0, 2\pi]$ составляет модель искомой составной поверхности, которая благодаря своему рельефу выглядит на рис. 4.56,а вполне реалистично и без каркасных линий. $\square$

4.3.2. Линейчатые поверхности

По типу образующей линии кинематические поверхности делятся на линейчатые и нелинейные (см. классификацию на рис. 4.2). **Линейчатые** поверхности формируются движением **прямой** образующей линии, поэтому имеют нулевую гауссову кривизну K , определённую в (1.95).

$\square$ **Цилиндрическая** поверхность получается переносом образующей прямой по направляющей линии $p_h(t)$ параллельно **постоянному** вектору W (рис. 4.57):

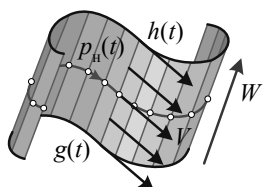


Рис. 4.57

$$p(t, \tau) = p_h(t) + W\tau. \quad (4.29)$$

Условие невырожденности цилиндрической поверхности в линию (оно же есть условие существования нормали $N(t) = V(t) \times W(t)$) — непараллельность вектора W вектору $V(t) = p'_h(t)$, касательному к направляющей линии.

$\square$ **Коническая** поверхность образуется движением прямой, проходящей через точку o и направляющую линию $p_h(t)$. Одна из двух моделей такой поверхности, отличающихся направлением **переменного** вектора $W(t)$, устроена согласно рис. 4.58:

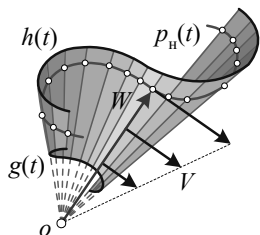


Рис. 4.58

$$p(t, \tau) = o + W(t)\tau, \quad W(t) = p_h(t) - o. \quad (4.30)$$

Вторая модель конуса $p(t, \tau) = p_h(t) + W(t)\tau$ имеет вектор $W(t) = o - p_h(t)$ противоположного направления.

Условие невырожденности конической поверхности в линию — непараллельность векторов $V(t, \tau) = p'_h(t)\tau$ и $W(t)$.

$\square$ **Торсовая** поверхность образуется движением прямой по касательной к направляющей линии $p_h(t)$ (рис. 4.59):

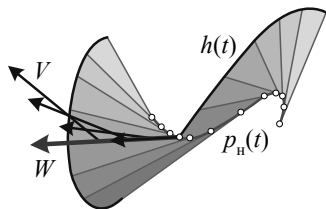


Рис. 4.59

$$p(t, \tau) = p_h(t) + (dp_h(t)/dt)\tau. \quad (4.31)$$

Условие невырожденности торсовой поверхности в линию — непараллельность векторов $W(t) = p'_h(t)$ и $V(t, \tau) = W(t) + p''_h(t)\tau$. На направляющей линии с $\tau=0$ эти векторы равны, следовательно, нормаль к поверхности вырождена в нулевой вектор. Если $p_h(t)$ — прямая линия с определённой

в (1.110а) кривизной $\kappa=0$, то $p''_h(t) = O_3$, $V(t) = W(t)$, а торсовая поверхность вырождается в прямую линию. Если $p_h(t)$ — плоская линия, то векторы V и W с ней компланарны, нормаль имеет постоянное направление, а поверхность тора является плоскостью, хотя и не с самой простой кинематической моделью.

Указание в моделях (4.29)–(4.31) интервалов изменения параметров делает линейчатую поверхность *ограниченной* линиями $g(t)$ и $h(t)$. Например, при $\tau \in [\tau_0, \tau_m]$ получим граничные линии

$$g(t) = p(t, \tau_0), \quad h(t) = p(t, \tau_m). \quad (4.32a)$$

Границы линейчатой поверхности могут измениться из-за её рассеечения другими поверхностями. В этом случае границы интервала изменения параметра τ становятся зависимыми от фиксированного параметра t . Например, в точке пересечения образующей прямой $q(t) + U(t)\tau$ (у рассмотренных выше линейчатых поверхностей точки $q(t)$ и векторы $U(t)$ индивидуально зависят от значения t) с плоскостью $\{o, N\}$ параметр τ рассчитывается по (1.130a):

$$\tau(t) = ((o - q(t)) \circ N) / (U(t) \circ N). \quad (4.32б)$$

При условии $\tau_0 < \tau(t) < \tau_m$ плоскость рассекает образующую прямую между граничными линиями $g(t)$ и $h(t)$ на два отрезка с $\tau \in [\tau_0, \tau(t)]$ и $\tau \in [\tau(t), \tau_m]$, так что остаётся выбрать, какой из них останется в усечённой поверхности.

Развёртки линейчатых поверхностей

Развёртываемые линейчатые поверхности могут быть разложены на плоскость без разрывов, складок и искажений длин линий и площадей фигур. Рассмотрим задачу построения *развёртки линейчатой поверхности* — плоской замкнутой фигуры, которую можно вырезать из листа гибкого *нерастяжимого* материала (бумаги, целлулоида, жести и т. п.) и свернуть в исходную форму.

Условием развёртываемости поверхности $p(t, \tau) = p_n(t) + W(t)\tau$ является *компланарность* образующих линий $p(t, \tau)$ и $p(t + dt, \tau) \approx p(t, \tau) + p'_t(t, \tau)dt$ при $dt \rightarrow 0$ [56]. Пересечение скрещивающихся прямых (см. рис. 1.109) будет при ортогональности векторов $p_n(t + dt) - p_n(t)$ и $W(t) \times W(t + dt)$, откуда

$$p'_n(t) \circ (W(t) \times W'(t)) = 0. \quad (4.33)$$

Рассмотрим сначала **аналитические методы развёртывания**, основанные на *точном* преобразовании декартовых координат точки $p = [x \ y \ z]$ в криволинейные координаты системы, соответствующей типу поверхности, и последующем распрямлении дуг окружностей на плоскость.

□ Положение точки на поверхности круглого прямого **цилиндра** радиуса ρ описывается в *цилиндрической* системе координат ординатой y и углом азимута φ , отсчитываемым, например, от оси z декартовой СК (см. рис. 1.1,б). Не аффинное преобразование (1.3a) декартовых координат точки $p = [x \ y \ z]$ в цилиндрические координаты $p_{\text{ц}} = [\rho \ y \ \varphi]$ выполняют *нелинейные* функции

$$\rho(p) = \sqrt{x^2 + z^2}, \quad \varphi(p) = \arg([-z \ -x]) + \pi \quad (4.34a)$$

Функция $\arg$ вычисления угла направления вектора на плоскости определена в (1.4). Заметим, что для обеспечения непрерывности развёртки угол азимута

должен быть расположен в интервале $\varphi \in (0, 2\pi]$, а не в интервале $(-\pi, \pi]$, который получился бы при использовании в (4.34а) функции $\varphi(p) = \arg([z \ x])$.

Декартовы координаты точки $x = \rho \sin(\varphi)$ и $z = \rho \cos(\varphi)$ получаются из цилиндрических координат за одно преобразование вращения:

$$[x \ y \ z] = [0 \ y \ \rho] R_y(\varphi). \quad (4.34б)$$

Допустим, поверхность круглого прямого цилиндра (4.29) с кинематическими элементами $p_n(t) = \rho[\sin(t) \ 0 \ \cos(t)]$ и $W = y^0$ ограничена линиями $g(t)$ и $h(t) \ \forall t \in [t_0, t_n]$, а отрезки $g_0 h_0$ и $g_n h_n$ лежат на меридианах. Радиус цилиндра ρ и начальный меридиан φ_0 могут быть найдены по (4.34а) для точки h_0 .

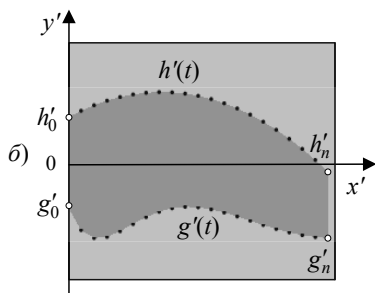
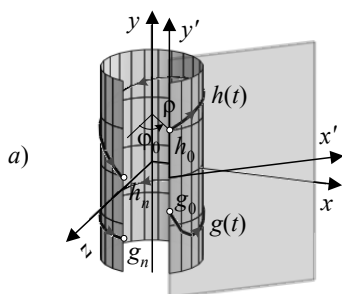


Рис. 4.60

Расположим плоскость развёртывания $x'y'$ касательно к цилиндру по отрезку $g_0 h_0 \subset y'$ (рис. 4.60,а). Так как длина дуги параллели радиуса ρ между меридианами φ_0 и φ равна $\rho \cdot (\varphi - \varphi_0)$, то точка $p = [x \ y \ z]$, лежащая на меридиане φ , имеет на развёртке координаты $x' = \rho \cdot (\varphi - \varphi_0)$ и $y' = y$. Для того чтобы цилиндрическая развёртка была неразрывной вдоль оси x' , перед её вычислением повернём цилиндр вокруг оси на угол $-\varphi_0$, после чего отрезок $g_0 h_0$ будет иметь угол $\varphi = 0$, а остальные точки поверхности — углы азимута в интервале $(0, 2\pi]$.

Объединяя все эти преобразования, получим следующий **алгоритм развёртывания точки $p = [x \ y \ z]$ поверхности круглого прямого цилиндра на плоскость $x'y'$** :

$$p' = [\rho(h_0) \cdot \varphi(p R_y(-\varphi(h_0))) \ y]. \quad (4.35)$$

Вычислив по этому алгоритму развёртки точек граничных линий g'_i и h'_i , сформируем замкнутый контур $g'_0 g'_1 \dots g'_n h'_n \dots h'_1 h'_0 g'_0$ — развёртку ограниченной цилиндрической поверхности на плоскость (рис. 4.60,б).

□ Зададим положение точки на поверхности круглого прямого **конуса** с углом раствора $\psi \in (0, \pi)$ в **сферической** системе координат расстоянием r до вершины конуса и углом азимута в интервале $\varphi \in [0, 2\pi]$, отличным от рис. 1.1,б. Альтернативное (1.3б) преобразование декартовых координат точки $p = [x \ y \ z]$ в сферические $p_k = [r \ \varphi \ \psi]$ устанавливается **нелинейными** функциями

$$r(p) = |p|, \quad \varphi(p) = \arg([-z \ -x]) + \pi, \quad \psi(p) = \arccos(y/|p|). \quad (4.36а)$$

Декартовы координаты точки $x = r \sin(\varphi) \sin(\psi)$, $y = r \cos(\varphi) \sin(\psi)$ и $z = r \cos(\varphi) \cos(\psi)$ можно получить из сферических за два преобразования вращения:

$$[x \ y \ z] = [0 \ r \ 0] R_x(\psi) R_y(\varphi). \quad (4.36б)$$

Допустим, боковая поверхность круглого прямого конуса (4.30) с кинематическими элементами $p_0 = O_3$ и $p_n(t) = [a \sin(t) \ b \ \text{acos}(t)] \ \forall t \in [t_0, t_n]$ ограничена линиями $g(t)$ и $h(t)$, а отрезки $g_0 h_0$ и $g_n h_n$ лежат на меридианах. На основе точки h_0 (или g_0 при $h_0 = O_3$) вычислим по (4.36а) угол раствора конуса ψ и по (4.34а) начальный меридиан φ_0 .

Поставим плоскость развёртывания $x'y'$ в касательное положение к конусу по отрезку $g_0 h_0 \subset y'$ (рис. 4.61,а). Дуга параллели между меридианами φ_0 и φ имеет длину $r \sin(\psi)(\varphi - \varphi_0)$ и развёртывается на плоскость $x'y'$ в виде дуги с радиусом r и углом $\gamma = -(\varphi - \varphi_0) \sin(\psi)$, отсчитываемым от оси y' (рис. 4.61,б). Для того чтобы развёртка была непрерывной в направлении вращения γ , перед её вычислением повернём конус вокруг оси y на угол $-\varphi_0$, после чего отрезок $g_0 h_0$ будет иметь угол $\varphi = 0$, а остальные точки поверхности — углы азимута в интервале $(0, 2\pi]$.

Объединяя данные преобразования, получим следующий алгоритм развёртывания точки p круглого прямого конуса на плоскость $x'y'$:

$$\begin{cases} \varphi_0 = \varphi(g_0), \psi = \psi(g_0), \\ \gamma = -\sin(\psi) \varphi(p R_y(-\varphi_0)), \\ p' = [0 \ |p|] R(\gamma). \end{cases} \quad (4.37)$$

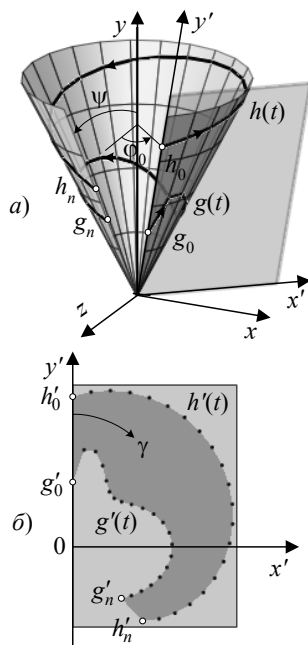


Рис. 4.61

Вычислив по этому алгоритму развёртки узловых точек граничных линий g'_i и h'_i , сформируем замкнутый контур $g'_0 g'_1 \dots g'_n h'_n \dots h'_0 h'_0 g'_0$ — развёртку ограниченной конической поверхности на плоскость.

□ Точное построение развёртки **произвольной** линейчатой поверхности — не круглых или не прямых цилиндра и конуса, а также тора, — сопряжено с трудоёмким исчислением длин дуг кривых линий в криволинейных системах координат. Рассмотрим понятный даже ребёнку **приближённый метод развёртывания**, основанный на отрезании от поверхности узких **треугольных** полосок и плотном укладывании их друг к другу на плоскости развёртки. Благодаря планарности вершин треугольника он перемещается из пространства на плоскость без искажений, а точность развёртывания повышается при увеличении числа полосок с одновременным уменьшением их ширины.

После квантования параметра t гладкие линии $g(t)$ и $h(t)$ заменяются ломаными с вершинами $g_i = g(t_i)$ и $h_i = h(t_i)$, а поверхность — цепочкой из n четырёхугольных полосок $g_{i-1} g_i h_i h_{i-1} g_{i-1}$, $i \in [1, n]$. Разделим каждую полоску диагональю $g_i h_{i-1}$ на два треугольника $g_{i-1} h_{i-1} g_i g_{i-1}$ и $g_i h_{i-1} h_i g_i$ таких, что у каждо-

го третья вершина лежит справа от первой стороны (рис. 4.62,а). Требуется построить плоский полигон $g'_0g'_1\dots g'_nh'_n\dots h'_1h'_0g'_0$, состоящий из треугольников $g'_{i-1}h'_{i-1}g'_ig'_{i-1}$, $g'_ih'_{i-1}h'_ig'_i \forall i=\overline{1,n}$, равных треугольникам-образам на развёртываемой поверхности. Начало развёртки на плоскости $x'y'$ зададим отрезком $g'_0h'_0$ длиной $|g_0-h_0|$, вытянутым по оси y' (рис. 4.62,б).

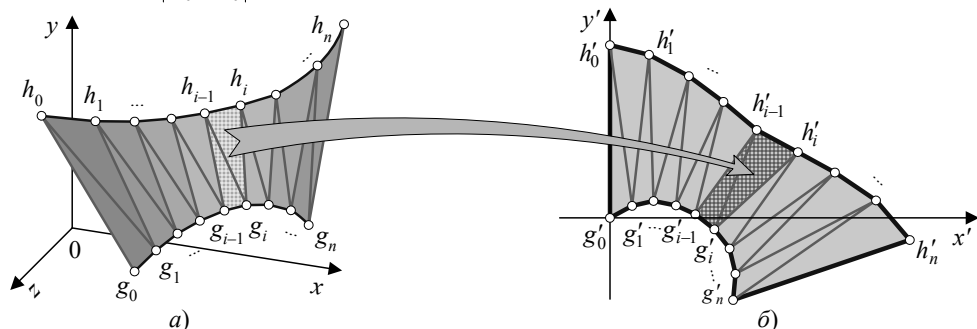


Рис. 4.62

Укладка треугольника $g_{i-1}h_{i-1}g_i$ на плоскость $x'y'$ методом геометрических преобразований состоит в расчёте нормали $N_1=(g_i-g_{i-1})\times(h_{i-1}-g_{i-1})$, матрицы $C_1=MP3(g_{i-1},h_{i-1}-g_{i-1},\overline{N}_1,g'_{i-1},h'_{i-1}-g'_{i-1},z^0)$ и точки $\tilde{g}'_i=\tilde{g}'_iC_1$. Аналогично для треугольника $g_ih_{i-1}h_i$ находятся нормаль $N_2=(h_i-g_i)\times(h_{i-1}-g_i)$, матрица $C_2=MP3(g_i,h_{i-1}-g_i,\overline{N}_2,g'_i,h'_{i-1}-g'_i,z^0)$ и точка $\tilde{h}'_i=\tilde{h}'_iC_2$.

Лучший же, по мнению автора, **безматричный** способ укладки использует определённую в (1.69а) функцию *triang*, аргументами которой являются вершины первой стороны треугольника и длины двух других сторон, а возвращает она третью **правую** вершину. Получим точку g'_1 как правую вершину треугольника с первой стороной $g'_0h'_0$ и длинами других сторон $|g_1-g_0|$ и $|g_1-h_0|$, затем точку h'_1 как правую вершину треугольника с первой стороной $g'_1h'_0$ и длинами других сторон $|h_1-g_1|$ и $|h_1-h_0|$. Получилась развёртка первой четырёхугольной полоски с правой граничной линией $g'_1h'_1$. Повторив данную процедуру ещё $n-1$ раз, получим вершины искомого полигона развёртки, которые сводятся в замкнутый контур в порядке следования вершин $g'_0g'_1\dots g'_nh'_n\dots h'_1h'_0g'_0$.

Оформим сказанное в виде следующего **алгоритма приближённого развёртывания линейчатой ограниченной поверхности**.

📁 **Шаг 1.** Вычисляются концы g_0 и h_0 начального положения образующего отрезка и соответствующие им точки развёртки $g'_0=O_2$ и $h'_0=|h_0-g_0|y^0$.

Шаг 2. В цикле $i=\overline{1,n}$ вычисляются концы образующего отрезка g_i , h_i и их развёртки:

$$g'_i=triang(g'_0,h'_0,|g_0-g_1|,|h_0-g_1|), \quad h'_i=triang(g'_1,h'_0,|g_1-h_1|,|h_0-h_1|).$$

Шаг 3. Вывод полигона развёртки $g'_0g'_1\dots g'_nh'_n\dots h'_1h'_0g'_0$. □

При вырождении какой-либо ограничивающей линии поверхности в точку (например, вершины конуса) нужно рассчитывать только невырожденные вершины полигона развёртки.

Пример 4.5. Получить модель и построить развёртку боковой поверхности усечённого эллиптического конуса (рис. 4.63,а), заданного точкой $o=[2 \ 4 \ 2]$, направляющим эллипсом нижнего основания в плоскости xz с полуосями $a=1$, $b=2$ и секущей плоскостью верхнего основания, проходящей через точку $p_n=[0 \ 2 \ 0]$ ортогонально оси эллипса.

Решение. Составим модель эллипса нижнего основания конуса так, чтобы начало развёртки приходилось на образующую линию между точками o и $[0 \ 0 \ b]$:

$$g(t)=[a\sin(t) \ 0 \ b\cos(t)] \quad \forall t \in [0, 2\pi].$$

Найдём левую границу интервала параметра $\tau \in [\tau_h(t), 1]$ и модель верхней границы $h(t)$ решением задачи (1.130а) пересечения образующей линии $o+(g(t)-o)\tau$ поверхности (4.30) с плоскостью, описываемой неявным уравнением $N_n \circ (p-p_n)=0$ с нормалью $N_n=o$:

$$o \circ (o+(g(t)-o)\tau) = o \circ p_n,$$

откуда получаем $\tau_h(t) = \frac{o \circ (p_n - o)}{o \circ (g(t) - o)} = \frac{8}{12 - \sin(t) - 2\cos(t)}$ и $h(t) = o + (g(t) - o)\tau_h(t)$.

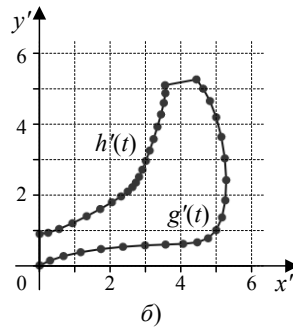
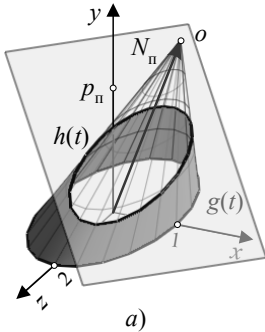


Рис. 4.63

Таким образом, боковая поверхность усечённого конуса описывается следующей векторной функцией с зависимым интервалом параметра τ :

$$p(t, \tau) = [1 + (0.5\sin(t) - 1)\tau \quad 2(1 - \tau) \quad 1 + (\cos(t) - 1)\tau] \quad \forall t \in [0, 2\pi], \tau \in \left[\frac{8}{12 - \sin(t) - 2\cos(t)}, 1 \right].$$

Развёртка усечённого конуса построена на рис. 4.63,б. $\square$

Пример 4.6. Получить модель и построить развёртку торсовой поверхности, образованной движением отрезка длиной $l=3$ по касательной к первому витку цилиндрической винтовой линии с радиусом $r=1$ и шагом $d=1.5$ (рис. 4.64,а).

Решение. Направляющая винтовая линия описывается параметрической функцией

$$p_n(t) = [r\sin(t) \quad dt/2\pi \quad r\cos(t)] \quad \forall t \in [0, 2\pi].$$

Необходимый в модели (4.31) вектор производной $p'_n(t) = [r\cos(t) \quad d/2\pi \quad -r\sin(t)]$ имеет независимую от параметра t длину $|p'_n(t)| = \sqrt{r^2 + \delta^2}$, где $\delta = d/2\pi$. Это даёт постоянные границы интервала изменения параметра τ в модели торсовой поверхности:

$$\begin{cases} p(t, \tau) = [r(\sin(t) + \tau\cos(t)) \quad \delta(t + \tau) \quad r(\cos(t) - \tau\sin(t))], \\ \forall t \in [0, 2\pi], \tau \in [0, \tau_h], \tau_h = l / \sqrt{r^2 + \delta^2} = 2.92. \end{cases}$$

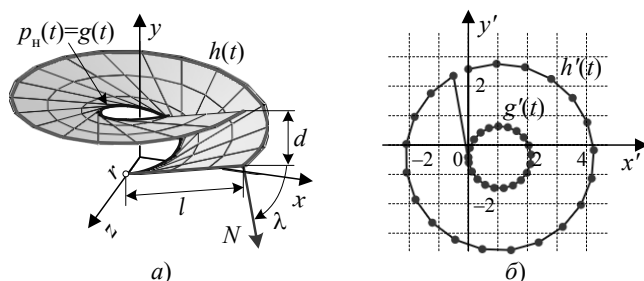


Рис. 4.64

Эта поверхность имеет постоянный угол λ наклона нормали к горизонтальной плоскости во всех точках:

$$N(t, \tau) = p'_t \times p'_\tau = r\tau [\delta \cos(t) - r - \delta \sin(t)],$$

$$\sin(\lambda) = N_y / |N| = -r / \sqrt{r^2 + \delta^2} = -0.97 \Rightarrow \lambda = -76.6^\circ.$$

Свойство равнаклонности торсовой поверхности используется при сооружении поворотов на скоростных автодорогах, велотреках, водостоках и т. п.

Приняв $g(t) = p_h(t)$ и $h(t) = p(t, \tau_h)$, строим развёртку торсовой поверхности на рис. 4.32, б. Вырезав её по всему контуру и подняв за левый верхний угол на высоту $d=1.5$, получим исходную торсовую поверхность в натуре. $\square$

Косые линейчатые поверхности

Согласно классификации, приведённой на рис. 4.2, **косые** (неразвёртываемые) линейчатые поверхности образуются при движении прямой по двум или трём направляющим линиям. В случае **двух направляющих** образующая прямая должна удовлетворять одному дополнительному условию. Например, она может быть ориентирована под определённым углом к некоторой плоскости Π .

В зависимости от вида направляющих косые поверхности делятся на:

- **цилиндровиды**, у которых обе направляющие являются кривыми линиями;
- **коноиды** с одной кривой и одной прямой направляющими;
- **косые плоскости** с двумя прямыми направляющими.

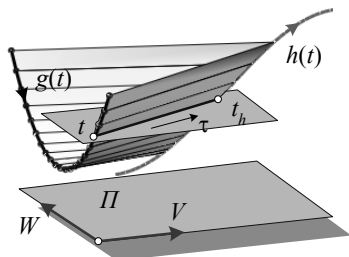


Рис. 4.65

Алгоритм получения модели косой поверхности с двумя направляющими $g(t)$, $h(t)$ и образующими, параллельными плоскости $\Pi(V, W)$ (рис. 4.65), следующий.

Шаг 1. Фиксируем параметр t и точку на линии $g(t)$.

Шаг 2. Через точку $g(t)$ проводим плоскость $g(t) + Vt_\Pi + W\tau_\Pi$, параллельную плоскости Π .

Шаг 3. Находим параметр t_h и точку $h(t_h)$ пересечения этой плоскости со второй направляющей:

$$g(t) + Vt_{II} + W\tau_{II} = h(t_h) \Rightarrow t_h(t) \Rightarrow h(t_h(t)). \quad (4.38)$$

Шаг 4. Составляем модель косо́й поверхности:

$$p(t, \tau) = \text{lin}(g(t), h(t_h(t)), \tau) = g(t)(1 - \tau) + h(t_h(t))\tau. \quad (4.39)$$



Требования к направляющим линиям $g(t)$ и $h(t)$:

- плоские направляющие не должны быть параллельны плоскости Π , иначе уравнение (4.38) либо не имеет решения, либо, при компланарности направляющих, имеет бесконечное множество точек пересечения;

- направляющие линии не должны пересекаться, иначе при $h(t_h(t)) = g(t)$ функция $p(t, \tau)$ в (4.39) не будет иметь двух степеней свободы.

Пример 4.7. Получить модель **цилиндриоида**, образованного движением концов отрезка с постоянным фазовым сдвигом φ по двум направляющим окружностям радиусами r_1 и r_2 , расположенным соосно в горизонтальных параллельных плоскостях на расстоянии d (рис. 4.66, а).

Решение. Составим модели направляющих окружностей с учётом фазового сдвига вращения концов образующего отрезка:

$$g(t) = [r_1 \cos(t) \ 0 \ r_1 \sin(t)], \quad h(t_k) = [r_2 \cos(t_k) \ d \ r_2 \sin(t_k)], \quad t_k = t + \varphi.$$

Подставив эти выражения в (4.39), получим координатные параметрические функции поверхности, заключённой между направляющими линиями:

$$\begin{cases} x(t, \tau) = r_1 \cos(t)(1 - \tau) + r_2 \cos(t + \varphi)\tau, \\ y(t, \tau) = d\tau, \\ z(t, \tau) = r_1 \sin(t)(1 - \tau) + r_2 \sin(t + \varphi)\tau \end{cases} \forall t \in [0, 2\pi], \tau \in [0, 1].$$

Условие постоянного фазового сдвига равносильно условию **постоянства угла** λ между образующими прямыми и плоскостью xz :

$$\sin(\lambda) = W_y / |W| = d / \sqrt{d^2 + r_1^2 + r_2^2 - 2r_1 r_2 \cos(\varphi)} = \text{const} \forall t,$$

где $W = p'_\tau(t, \tau) = h(t) - g(t)$. Например, при $\varphi \neq 0$ и $\varphi \neq \pi$ изучаемая линейчатая поверхность одновременно является квадратичной поверхностью **однополостного гиперболоида** (см. рис. 1.142, б). При $\varphi = 0$ получим боковую поверхность **усечённого конуса** (рис. 4.66, б) с углом наклона образующих прямых λ таким, что $\sin(\lambda) = d / \sqrt{d^2 + (r_1 - r_2)^2}$. При $r_1 = r_2$ это поверхность **цилиндра**. Наконец, при $\varphi = \pi$ все образующие прямые имеют общую точку $[0 \ d r_1 / (r_1 + r_2) \ 0]$ и образуют поверхность **двуполостного конуса** (см. рис. 1.142, в).

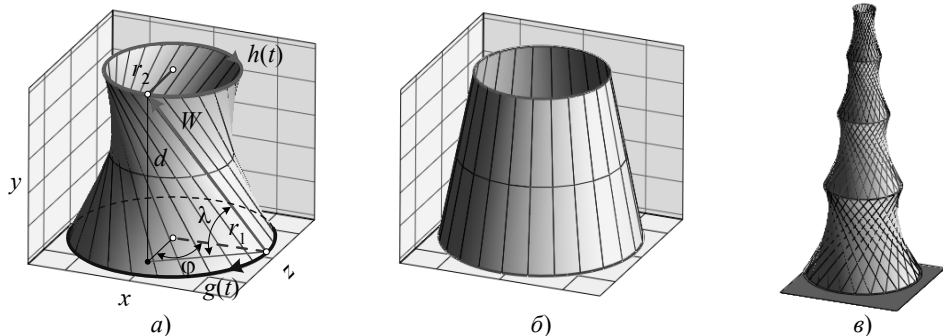


Рис. 4.66

Метод формирования криволинейной поверхности из отрезков прямых имеет практическое применение при создании прочных, но лёгких, ажурных и красивых конструкций башен, телевышек, опор и т. п. На рис. 4.66,в построена четырёхступенчатая модель известной в архитектуре «башни Шухова». Вторая и остальные ступени строятся преобразованиями масштабирования и переноса первой ступени так, чтобы радиус верхней окружности предыдущей ступени совпадал с радиусом нижней окружности следующей ступени. □

✚ **Пример 4.8.** Получить модель поверхности *винтового коноида*, образованной движением горизонтального отрезка прямой по оси y и N виткам цилиндрической винтовой линии с радиусом r и шагом d (рис. 4.67).

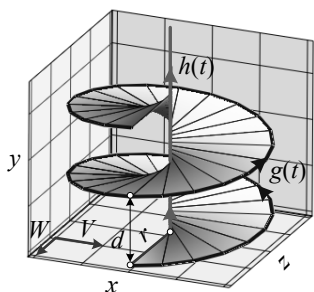


Рис. 4.67

Решение. Составим модели направляющих линий

$$g(t)=[r\sin(t) \ \delta t \ r\cos(t)], \quad h(t)=[0 \ t \ 0],$$

где $\delta=d/2\pi$, а также направляющие векторы горизонтальной плоскости $V=x^0$ и $W=z^0$. Подставив эти исходные данные в (4.38), получим векторное уравнение

$$[r\sin(t)+t_n \ \delta t \ r\cos(t)+\tau_n]=[0 \ t_h \ 0],$$

из которого следует соотношение параметров $t_h(t)=\delta t$ и параметрическая модель винтового коноида (4.39):

$$p(t,\tau)=[r(1-\tau)\sin(t) \ dt/2\pi \ r(1-\tau)\cos(t)] \ \forall t \in [0, 2\pi N], \ \tau \in [0, 1].$$

Данная поверхность находит практическое применение в конструкциях различных винтовых движителей, перемещающих материалы (снег, песок, вязкие жидкости, воду, мясо) и несущие их транспортные средства (снегоходы) вдоль оси винта. □

✚ **Пример 4.9.** Получить модель поверхности *косой плоскости*, образованной скольжением прямой образующей по скрещивающимся прямым направляющим линиям: первая линия проходит через точки $A=[0 \ 0 \ 1]$ и $B=[-1 \ 1 \ 0]$, вторая — через точки $C=[0 \ 0 \ -1]$ и $D=[1 \ 1 \ 0]$. Во время движения образующая должна быть параллельна плоскости с направляющими векторами $V=[0 \ 1 \ 0]$ и $W=[-1 \ 0 \ 1]$ (рис. 4.68,а).

Решение. Подставив в (4.38) линейные модели направляющих прямых

$$g(t)=A(1-t)+Bt=[-t \ t \ 1-t], \quad h(t)=C(1-t)+Dt=[t \ t \ 1-t], \quad (4.40)$$

получим векторное уравнение

$$[-t-\tau_{\Pi} \ t+\tau_{\Pi} \ 1-t+\tau_{\Pi}]=[t_h \ t_h \ t_h-1], \quad (4.41)$$

из которого следуют соотношение параметров $t_h(t)=1-t$ и модель косой плоскости (4.39):

$$p(t,\tau)=[\tau-t \ t+\tau-2t\tau \ 1-t-\tau]. \quad (4.42)$$

Уточним характер движения образующей прямой по направляющим отрезкам AB и CD . Подставив в (4.42) значения $\tau=0$ и $\tau=1$, получим с учётом (4.40) $p(t,0)=g(t)$ и $p(t,1)=h(t)=[1-t \ 1-t \ -t]=h(1-t)$. Это означает, что образующая движется по направлению прямой $g(t)$ и против направления прямой $h(t)$. В частности, при $t=0$ она проходит через точки A и D , а при $t=1$ — через точки B и C , оставаясь всё время параллельной плоскости $\{V, W\}$.

Выразив из координатных функций поверхности $x(t,\tau)=\tau-t$, $y(t,\tau)=t+\tau-2t\tau$ и $z(t,\tau)=1-t-\tau$ параметры $t=(1-x-z)/2$ и $\tau=(1+x-z)/2$, получим неявное уравнение косой плоскости $x^2-z^2-2y+1=0$ — поверхности второго порядка со следующими матрицей квадратичной формы (1.158б) и инвариантами $I_1 \div I_4$:

$$F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix}, \quad I_1 = 0, \quad I_2 = \begin{vmatrix} 1 & 0 \\ 0 & 0 \end{vmatrix} + \begin{vmatrix} 0 & 0 \\ 0 & -1 \end{vmatrix} + \begin{vmatrix} -1 & 0 \\ 0 & 1 \end{vmatrix} = -1, \quad I_3 = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{vmatrix} = 0, \quad I_4 = 1 > 0.$$

По классификации квадратичных поверхностей (см. табл. 1.3) это **гиперболический параболоид** со следующими сечениями координатными плоскостями (рис. 4.68, б):

- $x=0 \Rightarrow y = (1-z^2)/2$ — парабола (1) в плоскости yz ;
- $y=0 \Rightarrow x^2 = z^2 - 1$ — две ветви гиперболы (2) в плоскости xz ;
- $z=0 \Rightarrow y = (x^2+1)/2$ — парабола (3) в плоскости xy .

Что касается названия поверхности, то оно объясняется просто. Возьмём **плоский** квадратный лист резины $ABCD$ за противоположные края и повернём их в разные стороны, воспроизводя преобразование кручения (см. рис. 3.18, д). Скрученная из плоскости фигура получила название «косая плоскость».

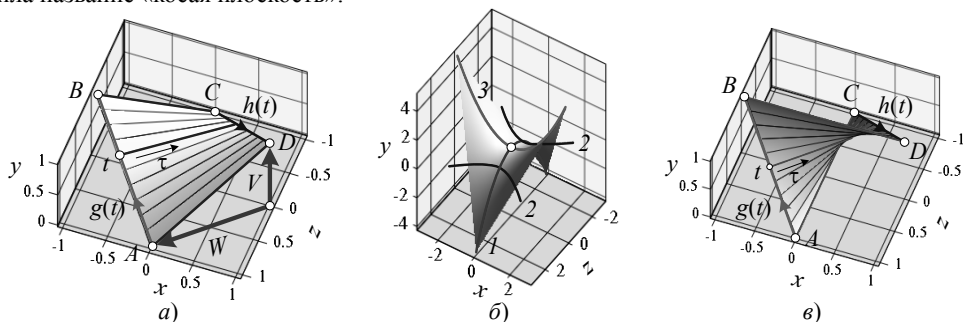


Рис. 4.68

Поверхность (4.42) получена по (4.39) как **линейная интерполяция** между направляющими $g(t)$ и $h(1-t)$. Однако если её строить без условия параллельности образующей линии с плоскостью $\{V, W\}$, то при образующих прямых (4.40) получим косую плоскость

$$p(t, \tau) = g(t)(1-\tau) + h(t)\tau = [t(2\tau-1) \quad t(t-1)(2\tau-1)]. \quad (4.43)$$

Это тоже гиперболический параболоид, имеющий в отличие от (4.42) другую ориентацию в пространстве. Как видно из рис. 4.68, в, при построении поверхности (4.43) образующая всё равно остаётся всегда параллельной, но теперь уже к плоскости xz . $\square$

В зависимости от вида **трёх направляющих**, которые не должны друг с другом попарно пересекаться, среди косых поверхностей выделяются:

- **однополостные гиперboloиды**, у которых все направляющие — прямые;
- **косые цилиндры** с двумя прямыми и третьей кривой направляющими;
- **конусоиды**, имеющие две кривые направляющие и одну прямую.

Алгоритм получения модели $p(t, \tau)$ **косой поверхности с тремя направляющими** $g(t)$, $h(t)$ и $u(t)$ (рис. 4.69) имеет следующий вид.

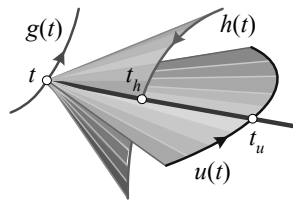


Рис. 4.69

Шаг 1. Фиксируем параметр t и точку $g(t)$.

Шаг 2. По (4.30) строим конусы на направляющих линиях $h(t)$ и $u(t)$ с общей вершиной $u(t)$:

$$h(t_h, \tau_h) = g(t) + (h(t_h) - g(t))\tau_h, \quad u(t_u, \tau_u) = g(t) + (u(t_u) - g(t))\tau_u.$$

Шаг 3. Находим прямую пересечения этих конусов, которая является образующей косої поверхности:

$$(h(t_h) - g(t))\tau_h = (u(t_u) - g(t))\tau_u \Rightarrow kh(t_h) - u(t_u) = (k-1)g(t), \text{ где } k = \tau_h/\tau_u. \quad (4.44)$$

Решая векторное уравнение (4.44) относительно переменных $k(t)$, $t_h(t)$ и $t_u(t)$, получим модель искомой поверхности в любой из двух форм:

$$p(t, \tau) = g(t)(1-\tau) + h(t_h(t))\tau \quad \text{или} \quad p(t, \tau) = g(t)(1-\tau) + u(t_u(t))\tau. \quad (4.45)$$



Пример 4.10. Классифицировать и построить модель поверхности, образованной движением прямой линии по трём скрещивающимся прямым направляющим $g(t) = [1 \ t \ 0]$, $h(t) = [0 \ 1 \ t]$ и $u(t) = [t \ 0 \ 1]$ (рис. 4.70, а).

Решение. Уравнение (4.44) принимает вид $[-t_u \ k \ kt_h - 1] = [k-1 \ (k-1)t \ 0]$, откуда получаем значения $k = t/(t-1)$, $t_u = 1-k = 1/(1-t)$ и параметрические функции поверхности

$$x(t, \tau) = 1 + t\tau/(1-t), \quad y(t, \tau) = t(1-\tau), \quad z(t, \tau) = \tau. \quad (4.46)$$

Исключив параметры $t = y/(1-z)$ и $\tau = z$, получим неявное координатное уравнение $xy + xz + yz - x - y - z + 1 = 0$. Оно представляет поверхность второго порядка со следующими матрицей (1.158б) и инвариантами (1.160):

$$F = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 \\ -1 & -1 & -1 & 2 \end{bmatrix}, \quad I_1 = 0, \quad I_2 = -3, \quad I_3 = 2 \neq 0, \quad I_4 = 1 > 0. \quad (4.47)$$

По классификации, приведённой в табл. 1.3, мы получили **однополостный гиперболоид**.

Анализируя координатную функцию $x(t, \tau)$ в (4.46), приходим к выводу, что она имеет разрыв при $t=1$. Следовательно, построение непрерывной поверхности по модели (4.46) возможно в два приёма. На рис. 4.70, а построена лишь часть поверхности в интервале изменения параметра $t \in [-10, 0.99]$. Другая часть при $t \in [1.01, 12]$ изображена на рис. 4.70, б. В обоих случаях интервал изменения второго параметра $\tau \in [-5, 5]$. Каждое из этих двух изображений в диметрической проекции мало похоже на однополостный гиперболоид из-за неудачного сочетания его ориентации в пространстве с диметрическим ракурсом изображения.

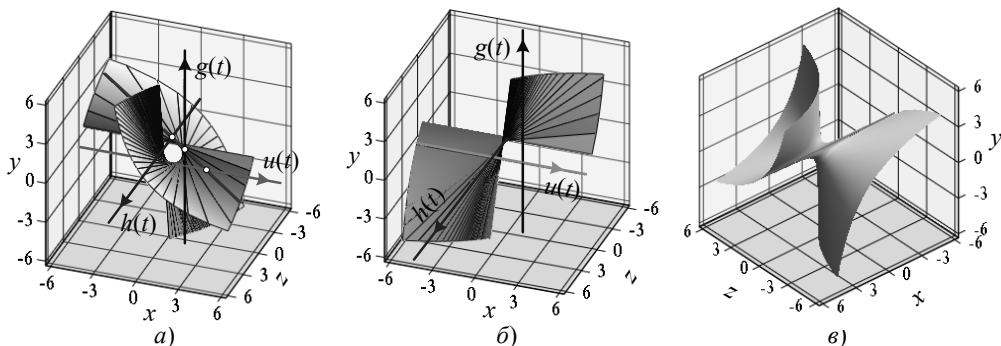


Рис. 4.70

Совместное изображение двух поверхностей в произвольной триметрической проекции (рис. 4.70,в) убеждает в том, что это именно однополостный гиперболоид, аналогичный изображённому на рис. 1.142,б. На его поверхности есть две линии разрыва, незаметные благодаря специально подобранному ракурсу. Одна из них соответствует неопределённости модели (4.46) при $t=1$, другая получается из-за конечных пределов изменения параметра t вместо теоретически бесконечных пределов. $\square$

Пример 4.11. Получить модель поверхности усечённого **конусоида**, основаниями которого являются окружность радиуса r и квадрат $a \times a$, а высота равна d (рис. 4.71).

Решение. В силу симметрии объекта построим модель поверхности его передней четверти, образованной движением прямой по трём направляющим:

- четверти дуги окружности

$$g(t) = [r \cos(t) \ 0 \ r \sin(t)] \quad \forall t \in [0, \pi/2];$$

- отрезку прямой линии между точками $[q \ d \ 0]$ и $[0 \ d \ q]$ при $q = a/\sqrt{2}$:

$$h(t) = [q(1-t) \ d \ qt] \quad \forall t \in [0, 1];$$

- прямой $u(t) = [0 \ t \ 0] \parallel y^0$.

Составив на основе этих линий уравнение (4.44)

$$[kq(1-t_h) \ kd-t_u \ kqt_h] = [(k-1)r \cos(t) \ 0 \ (k-1)r \sin(t)],$$

определим зависимость $t_h(t) = \sin(t)/(\sin(t) + \cos(t))$ па-

раметра верхней направляющей от параметра нижней и модель (4.45) четверти поверхности:

$$p_1(t, \tau) = \left[r(1-\tau) \cos(t) + \frac{a \cos(t)}{2 \sin(t+\pi/4)} \tau \quad d\tau \quad r(1-\tau) \sin(t) + \frac{a \sin(t)}{2 \sin(t+\pi/4)} \tau \right].$$

Поверхности остальных трёх четвертей конусоида получаются с помощью вращения поверхности $p_1(t, \tau)$ вокруг оси y на углы, кратные 90° :

$$p_i(t, \tau) = p_1(t, \tau) R_y(\pi i / 2) \quad \forall i = \overline{2, 4}, \quad t \in [0, \pi/2], \quad \tau \in [0, 1].$$

$\square$

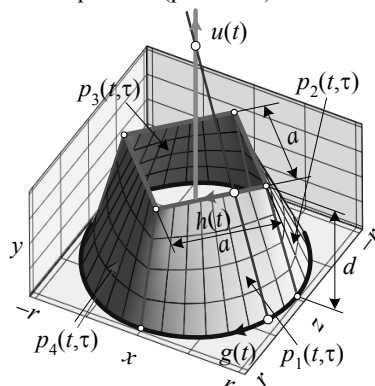


Рис. 4.71

4.3.3. Нелинейные поверхности

Нелинейная поверхность конструируется по общим формулам (4.21) с помощью **непрямой** образующей $p_0(\tau)$, которая при движении вдоль направляющей линии $p_n(t)$ изменяется матрицей $A(t, \tau)$. На эту матрицу возлагается основная задача преобразования исходного примитива $p_0(\tau)$, центр которого располагается обычно в начале координат, к виду, готовому к последнему преобразованию — переносу центра в точку $p_n(t)$.

Чаще всего матрица $A(t, \tau)$ формируется как произведение матриц элементарных преобразований, ответственных за последовательные движения образующей линии — вращения и масштабирования относительно координатных осей. Зависимость этих матриц от параметра t определяет размер и ориентацию образующей в различных местах направляющей линии, тогда как влияние параметра τ проявляется в изменении исходной формы самого примитива $p_0(\tau)$.

Раскроем секреты получения трёх поверхностей на рис. 4.49:

- все они построены на основе полиномиальной образующей линии

$$\begin{cases} p_o(\tau)=[x_o(\tau) \ y_o(\tau) \ 0] \forall \tau \in [0, 4], \\ x_o(\tau)=5.25\tau+1.21\tau^2-1.75\tau^3+0.3\tau^4, \\ y_o(\tau)=-6.5\tau+14.751\tau^2-6\tau^3+0.75\tau^4; \end{cases}$$

• поверхность (а) образована **вращением** (4.25а) линии $p_o(\tau)$ вокруг оси y на угол 270° . Таким образом, в модели (4.21) нужно задать $A(t, \tau)=R_y(t)$, $p_n(t)=O_3$ и интервал изменения параметра $t \in [0, 1.5\pi]$;

• поверхность (б) получена **переносом** (4.25б) линии $p_o(\tau)$ по направляющей окружности $p_n(t)=[3 \ 0 \ 0]R_y(t) \ \forall t \in [0, 1.5\pi]$;

• **комбинированная** поверхность (в) формируется масштабированием образующей линии матрицей $M(t, \tau)=\text{diag}[1+0.1\cos(\omega_\tau \tau) \ 1+0.1\cos(\omega_t t) \ 1]$, создающей на ней волнение с частотой $\omega_\tau=6$, и вращением вокруг оси y , по ходу которого образующая линия колебательно масштабируется ещё и по вертикали с частотой $\omega_t=3$. В результате в модели (4.21а) матрица $A(t, \tau)=M(t, \tau)R_y(t)$ и направляющая линия $p_n(t)=[3 \ 0 \ 0]R_y(t)$.

Циклические поверхности

Циклическая поверхность создаётся таким движением **окружности** или её дуги, что плоскость образующей ортогональна к направляющей линии $p_n(t)$ (см. рис. 4.47), т. е. нормаль плоскости касательна к направляющему вектору

$$V_n(t)=dp_n(t)/dt. \quad (4.48)$$

Получаемая таким способом кинематическая циклическая поверхность называется **каналовой**. Толщина канала определяется функцией изменения радиуса образующей окружности $r(t)$, входящей сомножителем в состав матрицы $A(t, \tau)$. Поверхность канала постоянного радиуса r называется **трубчатой**.

Рассмотрим **вычислительные аспекты построения каналовой поверхности**. Прежде всего необходимо задать кинематические элементы модели (4.21). В целях унификации в большинстве случаев будем использовать в качестве **образующей линии** окружность единичного радиуса с нормалью z^0 и центром в начале координат:

$$p_o(\tau)=\cos(\tau)x^0+\sin(\tau)y^0=[\cos(\tau) \ \sin(\tau) \ 0] \in xy. \quad (4.49a)$$

Однако, для исключения дополнительных вращений на углы $\pm 90^\circ$ иногда более целесообразным может оказаться выбор окружностей

$$p_o(\tau)=\cos(\tau)y^0+\sin(\tau)z^0=[0 \ \cos(\tau) \ \sin(\tau)] \in yz \quad (4.49б)$$

с нормалью x^0 или

$$p_o(\tau)=\cos(\tau)z^0+\sin(\tau)x^0=[\sin(\tau) \ 0 \ \cos(\tau)] \in xz \quad (4.49в)$$

с нормалью y^0 . Задание интервала изменения параметра τ , имеющего геометрический смысл угла дуги окружности, определяет угловой охват поперечного сечения каналовой поверхности. При $\tau \in [0, 2\pi]$ получим замкнутый канал. Выбор меньшего интервала позволяет построить разрез поверхности вдоль меридианов и увидеть её изнутри (см. рис. 3.77, 4.47, 4.49, 4.54).

Важным элементом модели каналовой поверхности является скалярная функция $r(t)$ изменения *радиуса окружности*, образующей поперечные кармашки линии при её движении вдоль направляющей траектории. Во многих отношениях предпочтительнее иметь аналитическое представление этой зависимости конкретной формулой. Его преимущества — это точность, компактность записи, простота вычисления при любых допустимых значениях параметра, возможность точного дифференцирования, необходимого в (4.23) и (4.24) для вычисления вектора нормали к поверхности $N(t, \tau)$.

К сожалению, аналитическое представление возможно лишь в самых простых случаях. Например, у трубчатой поверхности (1) на рис. 4.72 радиус $r(t) = r_0$ постоянный, канал конусного типа (2) получен при линейно изменяющемся радиусе $r(t) = r_0 - r_1 t$, а более сложный гофрированный канал может быть сформирован движением окружности переменного радиуса $r(t) = r_0 + r_1 \sin(\omega t)$ (3) или $r(t) = r_0 + r_1 |\sin(\omega t)|$ (4).



Рис. 4.72

Аналитическая форма функции $r(t)$ иногда бывает чрезмерно сложной в записи, вычислении и дифференцировании. Выгоднее становится задание $n+1$ -й узловой точки $\{t_i, r_i\} \forall i = \overline{0, n}$ и восстановление зависимости $r(t)$ в классе функций, сочетающем хорошее приближение к узлам и малые межузловые осцилляции с небольшим числом варьируемых параметров и простотой вычислений. К таким функциям относятся экспоненциальные, гармонические и полиномиальные. При большом числе узловых точек переходят к многосегментным составным линиям, из которых наиболее популярны составные линии и сплайны. Методы их конструирования рассмотрены в разд. 2.2, 2.3.

Сходные проблемы встают при выборе другого важного элемента кинематической модели (4.21) в целом и каналовой поверхности в частности — *направляющей линии* $p_n(t)$, по которой движется центр образующей линии. Траектория этого движения должна быть *гладкой*, т. е. непрерывно дифференцируемой по параметру t , иначе в точках излома либо разрыва вектор $V_n(t)$ не будет однозначно вычислим по (4.48).

Второе интуитивно понятное требование к направляющей линии состоит в том, чтобы при каждом значении параметра t радиус её кривизны, обратно про-

порциональный кривизне κ из (1.110а), был бы больше радиуса образующей окружности:

$$\rho_n(t) = \frac{|V_n(t)|^3}{|V_n(t) \times V'_n(t)|} > r(t). \quad (4.50)$$

В противном случае поперечные образующие дуги с вогнутой стороны направляющей линии будут совершать попятное движение (см. 2d-аналогию на рис. 1.60,в), а поверхность $p(t, \tau)$ окажется не гладкой, а смятой.

Аналитическое задание векторной функции $p_n(t) \in \mathbb{R}^3$ позволяет точно рассчитывать не только траекторию движения центра образующей окружности, но по (4.48) и вектор направления $V_n(t)$, необходимый для построения поперечных сечений каналовой поверхности (см. рис. 3.45) или, например, для решения кинематической задачи перемещения тела в пространстве (см. рис. 3.39). Однако, часто выражения координатных функций $x_n(t) \div z_n(t)$, обеспечивающих желаемое прохождение направляющей линии в пространстве, слишком громоздки либо их получение превращается в непосильную задачу. Тогда целесообразно использовать численные методы приближения векторной функции $p_n(t)$ на конечном множестве узловых точек $u_i = p_n(t_i) \quad \forall i = \overline{0, n}$, рассмотренные в гл. 2.

В кратком повторе суть полиномиального приближения состоит в представлении векторной функции произведением

$$p_n(t) = ST_m(t) \quad (4.51a)$$

векторов полиномиальных коэффициентов $S = [s_0 \ s_1 \ \dots \ s_m]$ и базисных функций m -й степени (2.6а). Аналитическое дифференцирование полиномиальной модели даёт формулу вычисления направляющего вектора

$$V_{ni}(t) = ST'_m(t) \quad (4.51б)$$

с помощью вектора производных базисных функций $T'_m(t)$, определённого в (2.6б). В отсутствие выражения $p_n(t)$ аппроксимация векторов узловых направлений выполняется по трёхточечным схемам (2.20б) и (2.22а), принимающим на равномерной сетке с постоянным шагом $d = t_{i+1} - t_i$ следующий вид:

$$V_{n0} = \frac{-3u_0 + 4u_1 - u_2}{2d}, \quad V_{ni} = \frac{u_{i+1} - u_{i-1}}{2d} \quad \forall i = \overline{1, n-1}, \quad V_{nn} = \frac{3u_n - 4u_{n-1} + u_{n-2}}{2d}. \quad (4.52)$$

Наибольшую сложность при моделировании каналовой поверхности по (4.21) представляет выбор **матриц преобразований** $A(t)$ или $C(t)$, обеспечивающих ортогональность плоскости образующей окружности вектору $V_n(t)$, касательному к траектории движения центра окружности $p_n(t)$. Конечно, если направляющая линия полностью аналитически определена и состоит из простейших сегментов (отрезков прямых или дуг окружностей с известными радиусами и координатами центров), то можно и без вычисления касательного вектора правильно сориентировать образующую окружность $p_o(\tau)$ в начале координат матрицей $A(t)$, изменить её радиус функцией $r(t)$ и перенести центр в точку $p_n(t)$.

Именно такой способ построения кинематических поверхностей был реализован в примерах 4.2–4.4.

Здесь читателю рекомендуется вспомнить подходы (итеративный и рекуррентный на рис. 3.39) и методы (два варианта разложения и парных точек на рис. 3.41–3.43) решения кинематической задачи, изученные в п. 3.1.3.3. Анализируя представленные там алгоритмы и результаты их моделирования (см. рис. 3.45–3.47), заключаем, что **расчёт матриц аффинных преобразований следует выполнять по рекуррентным экономичным алгоритмам**, из которых предпочтительнее алгоритм (3.73), основанный на **методе парных точек**.

Обобщая сказанное, сформулируем **рекуррентный алгоритм построения сечений каналовой поверхности** $p(t_i, \tau) \forall i=0, n$ при заданной в (4.49а) образующей окружности $p_0(\tau) \in xy$, функции изменения радиуса $r(t)$ и **произвольной** направляющей линии $p_n(t)$ с касательным вектором $V_n(t) = p'_n(t)$:

- начальное состояние $p(t_0, \tau)$ находится экономичным преобразованием примитива $p_0(\tau)$:

$$C_0 = MP2(O_3, z^0, u_0, r_0 \bar{V}_{n0}) \Rightarrow \tilde{p}(t_0, \tau) = \tilde{p}_0(\tau) C_0; \quad (4.53a)$$

- следующее состояние $p(t_{i+1}, \tau)$ достигается экономичным преобразованием предыдущего состояния $p(t_i, \tau)$:

$$C_i = MP2(u_i, r_i \bar{V}_{ni}, u_{i+1}, r_{i+1} \bar{V}_{n,i+1}) \Rightarrow \tilde{p}(t_{i+1}, \tau) = \tilde{p}(t_i, \tau) C_i. \quad (4.53б)$$

Часто на кинематических поверхностях наблюдается **кручение** продольных каркасных линий, распределённое по длине канала с разной степенью равномерности (см. рис. 3.45). Скоро мы увидим, что кручение может появиться даже при использовании алгоритма (4.53а,б), если направляющая линия $p_n(t)$ не плоская, с ненулевым кручением χ из (1.110а). Для компенсации этого нежелательного эффекта, особо заметного на стыке концов замкнутого канала, можно на i -м шаге алгоритма «подкручивать» образующую окружность вокруг нормали к её плоскости на определённый угол γ_i . Используемая в (4.53б) матрица сложного преобразования с подкручиванием на угол γ_i вычисляется через вспомогательное преобразование координат матрицей $C_k = MP2(u_i, \bar{V}_{ni}, O_3, z^0)$:

$$C_i = C_k \tilde{R}_z(\gamma_i) C_k^{-1} MP2(u_i, r_i \bar{V}_{ni}, u_{i+1}, r_{i+1} \bar{V}_{n,i+1}). \quad (4.53в)$$

При равномерном закручивании за n шагов на общий угол $|\theta| < \pi$ можно принять угол шагового кручения равным $\gamma = \theta/n$.

Работоспособность алгоритма (4.53) иллюстрируется построенным на рис. 4.73,а каркасом трубчатой поверхности с радиусом трубки $r=0.3$ и следующим **аналитическим** описанием направляющей траектории как гладкой составной линии со сквозной параметризацией:

$$p_n(t) = \begin{cases} [-1 \sin(t) \cos(t)] \forall 0 \leq t < \pi, \\ [\cos(t) \sin(t) -1] \forall \pi \leq t < 2\pi, \\ [1 \sin(t) -\cos(t)] \forall 2\pi \leq t < 3\pi, \\ [-\cos(t) \sin(t) 1] \forall 3\pi \leq t \leq 4\pi; \end{cases} \quad V_n(t) = \begin{cases} [0 \cos(t) \sin(t)] \forall 0 \leq t < \pi, \\ [-\sin(t) \cos(t) 0] \forall \pi \leq t < 2\pi, \\ [0 \cos(t) \sin(t)] \forall 2\pi \leq t < 3\pi, \\ [\sin(t) \cos(t) 0] \forall 3\pi \leq t \leq 4\pi. \end{cases}$$

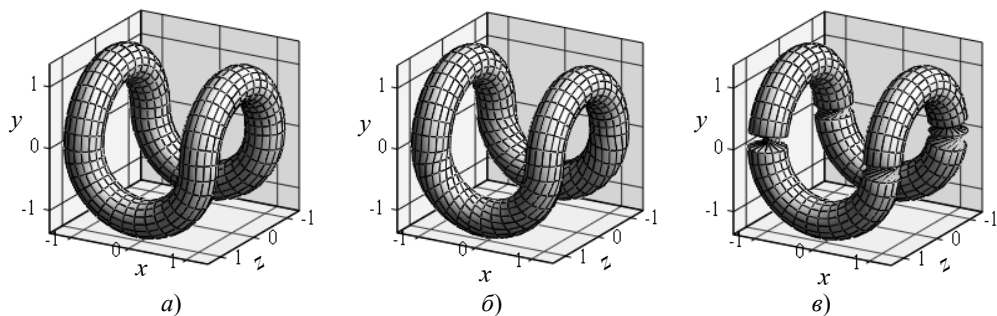


Рис. 4.73

Осевая линия трубки $p_n(t)$ интересна тем, что на ней есть восемь точек, где вектор $V_n(t)$ становится параллельным одной из координатных осей, в окрестностях которых возможно скачкообразное кручение поверхности, построенной по неэкономичному алгоритму. Очевидно, что МПТ-алгоритм (4.53) строит каналовую поверхность правильно и без подкручивания сечений по (4.53в).

Итеративный алгоритм, основанный на методе парных точек

$$C(t) = MP2(O_3, z^0, p_n(t), r(t)\bar{V}_n(t)) \Rightarrow \tilde{p}(t, \tau) = \tilde{p}_0(\tau)C(t), \quad (4.54)$$

позволяет получить сечение каналовой поверхности при любом значении параметра t экономичным преобразованием примитива $p_0(\tau)$ из начала координат в точку $p_n(t)$. Однако переходы между соседними сечениями не являются экономичными, что подтверждается подкручиванием продольных линий поверхности, изображённой на рис. 4.73, б. Отметим также, что эффект кручения наблюдается на плоских сегментах направляющей линии. Всё это делает алгоритм (4.54) не самым лучшим инструментом построения каналовых поверхностей.

Наконец, **худший** алгоритм построения каналовой поверхности основан на методе координатных вращений (3.69) с матрицей $A(t)$, вычисляемой по (3.44):

$$A(t) = A_{\text{ККВ}}^{-1}(V_n(t)) \Rightarrow p(t, \tau) = r(t)p_0(\tau)A(t) + p_n(t). \quad (4.55)$$

Методические ошибки алгоритма очевидны на рис. 4.73, в. Перекручивание соседних сечений на угол до 180° происходит на тех участках, где направляющая траектория проходит через **вертикальное** направление $V_n(t) \parallel y^0$. При этом вычисляемый по (3.44д) угол α скачком изменяется с $\pm 90^\circ$ на $\mp 90^\circ$ при неизменном угле $\beta \approx \pm 90^\circ$. Матрица сложного АП $A_{\text{ККВ}}^{-1}(V_n) = R_x(-\beta)R_y(\alpha)$, обратная к матрице (3.44в), выполняет вращение почти горизонтальной окружности $p_0(\tau)R_x(-\beta)$ вокруг оси y на угол α последним — отсюда становится понятной причина перекручивания поверхности на вертикальных участках. Тем не менее, мы будем использовать алгоритм (4.55) для получения **аналитических** моделей каналовых поверхностей, одна из которых рассматривается ниже.

Пример 4.12. Из проволоки длиной $l=100$ и радиусом $r=1$ изготовлена пружина путём навивки с шагом $h=6$ на боковую поверхность цилиндра радиусом $r_n=3$ (рис. 4.74, а). Затем пружина была поставлена вертикально на плоскость и под действием собственного веса

и упругости сжалась так, что два нижних витка соприкоснулись (рис. 4.74,б). Построить модели и изображения свободной и сжатой пружины.

Решение. Выберем образующую окружность $p_o(\tau)=[0 \sin(\tau) \cos(\tau)] \in yz$, а направляющую линию **свободной** пружины в виде цилиндрической винтовой линии с равномерным шагом витков h и их числом $n_b = l / \sqrt{4\pi^2(r_{ц}+r)^2 + h^2} \approx 3.87$:

$$p_{св}(t) = [(r_{ц}+r)\sin(t) \ k_{св}t \ (r_{ц}+r)\cos(t)] \forall t \in [0, 2\pi n_b]. \quad (4.56)$$

Благодаря постоянному коэффициенту $k_{св} = h/2\pi = 0.955$ наклона ординаты $y_{св}(t) = k_{св}t$ (график 1 на рис. 4.74,в) касательный к направляющей линии свободной пружины вектор

$$V_{св}(t) = [(r_{ц}+r)\cos(t) \ k_{св} \ -(r_{ц}+r)\sin(t)] \quad (4.57)$$

всюду наклонен к плоскости xz под постоянным углом $\beta_{св} = \arctg(k_{св}/(r_{ц}+r)) = 13.43^\circ$.

Построение трубчатой поверхности свободной пружины, изображённой на рис. 4.74,а, выполнено по алгоритму (4.53).

Так как направление винтовой линии (4.56) нигде не приближается к вертикальному, то получение аналитической модели поверхности свободной пружины возможно по формуле (4.55). Подставив в неё выбранные выше кинематические элементы $p_o(\tau)$, $p_{св}(t)$, матрицу преобразования образующей $A_{св}(t) = R_z(\beta_{св})R_y(t)$ и выполнив необходимые аналитические выкладки, получим координатные функции поверхности:

$$\begin{cases} x(t, \tau) = (r_{ц} + r + r\cos(\tau))\sin(t) - \frac{hn_b r}{l} \cos(t)\sin(\tau), \\ y(t, \tau) = \frac{2\pi n_b (r_{ц} + r)r}{l} \sin(t) + \frac{ht}{2\pi}, \\ z(t, \tau) = (r_{ц} + r + r\cos(\tau))\cos(t) + \frac{hn_b r}{l} \sin(t)\sin(\tau) \\ \forall t \in [0, 2\pi n_b], \tau \in [0, 2\pi]. \end{cases} \quad (4.58)$$

У **сжатой** пружины (рис. 4.74,б) каждый виток нагружен весом лежащих выше него витков. Не углубляясь в дебри теоретической механики, будем считать, что **направляющая линия** $p_{сж}(t)$ имеет ординату $y_{сж}(t)$ с **линейно возрастающим коэффициентом наклона** $k_{сж}(t) = at + b$ (график 2 на рис. 4.74,в):

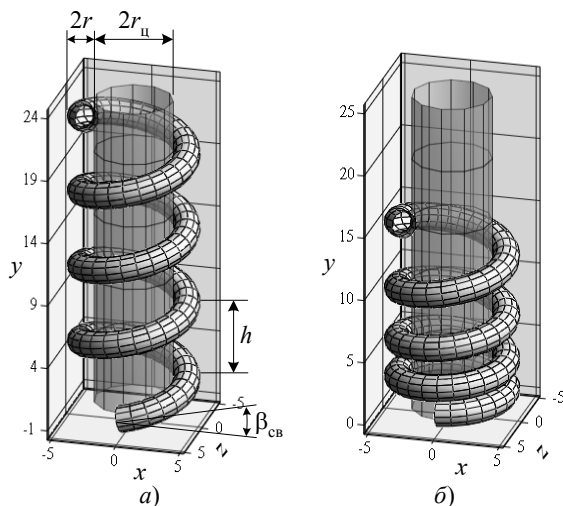
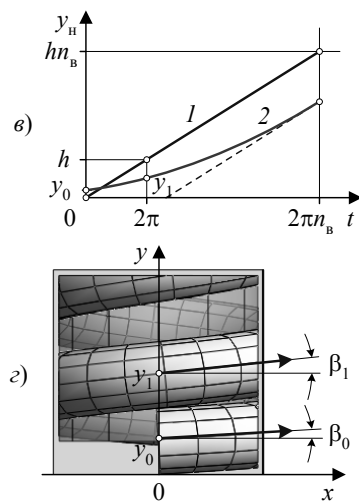


Рис. 4.74



$$p_{\text{сж}}(t) = [(r_{\text{ц}} + r) \sin(t) \quad 0.5at^2 + bt + c \quad (r_{\text{ц}} + r) \cos(t)]. \quad (4.59)$$

Касательный вектор $V_{\text{сж}}(t)$ также увеличивает угол своего наклона $\beta_{\text{сж}}(t)$ с ростом t :

$$V_{\text{сж}}(t) = [(r_{\text{ц}} + r) \cos(t) \quad at + b \quad -(r_{\text{ц}} + r) \sin(t)], \quad \beta_{\text{сж}}(t) = \arctg((at + b)/(r_{\text{ц}} + r)). \quad (4.60)$$

Подставив в (4.55) кинематические элементы $p_o(\tau)$, $p_{\text{сж}}(t)$ и $A_{\text{сж}}(t) = R_z(\beta_{\text{сж}}(t))R_y(t)$, получим координатные функции поверхности сжатой пружины:

$$\begin{cases} x_{\text{сж}}(t, \tau) = (r_{\text{ц}} + r + r \cos(\tau)) \sin(t) - \frac{r(at + b)}{\sqrt{(r_{\text{ц}} + r)^2 + (at + b)^2}} \cos(t) \sin(\tau), \\ y_{\text{сж}}(t, \tau) = \frac{r(r_{\text{ц}} + r)}{\sqrt{(r_{\text{ц}} + r)^2 + (at + b)^2}} \sin(\tau) + 0.5at^2 + bt + c, \\ z_{\text{сж}}(t, \tau) = (r_{\text{ц}} + r + r \cos(\tau)) \cos(t) + \frac{r(at + b)}{\sqrt{(r_{\text{ц}} + r)^2 + (at + b)^2}} \sin(t) \sin(\tau) \\ \forall t \in [0, 2\pi n_{\text{сж}}], \quad \tau \in [0, 2\pi] \end{cases} \quad (4.61)$$

Для практического построения этой поверхности необходимо определить неизвестные величины a , b , c и $n_{\text{сж}}$. Не учитывая достаточно сложной зависимости числа витков пружины от степени её сжатия (в действительности при сжатии оно немного увеличивается) и полагая $n_{\text{сж}} = n_{\text{в}}$, запишем исходные требования к сжатой пружине в виде трёх соотношений относительно неизвестных a , b и c :

- верхний конец сжатой пружины не нагружен, следовательно, он имеет значение наклона $k_{\text{сж}}(2\pi n_{\text{в}})$ как у свободной пружины $k_{\text{св}}$:

$$2\pi n_{\text{в}} a + b = h/2\pi; \quad (4.62)$$

- начальная ордината $y_0 = y_{\text{сж}}(0)$ с учётом опоры сжатой пружины о плоскость xz и наклона её нижнего торца на угол $\beta_0 = \beta_{\text{сж}}(0)$ равна $r \cos(\beta_0)$ (рис. 4.74, *а*);

$$c = r(r_{\text{ц}} + r) / \sqrt{(r_{\text{ц}} + r)^2 + b^2}; \quad (4.63)$$

- ордината направляющей $y_1 = y_{\text{сж}}(2\pi)$ в конце первого витка, касающегося начала второго с углом наклона $\beta_1 = \beta_{\text{сж}}(2\pi) > \beta_0$, равна $y_0 + r/\cos(\beta_0) + r/\cos(\beta_1)$:

$$4\pi^2 a + 2\pi b = r \left(\sqrt{(r_{\text{ц}} + r)^2 + b^2} + \sqrt{(r_{\text{ц}} + r)^2 + (2\pi a + b)^2} \right) / (r_{\text{ц}} + r). \quad (4.64)$$

Численное решение системы уравнений (4.62)–(4.64) при $l=100$, $r=1$, $r_{\text{ц}}=3$ и $h=6$ дало значения коэффициентов $a=0.03$, $b=0.224$ и $c=0.992$, подстановка которых в (4.59) и (4.57) позволила построить изображение сжатой пружины, приведённое на рис. 4.74, *б*. $\square$

Пример 4.13. Построить поверхность топологического объекта, называемого «*бутылкой Клейна*» (рис. 4.75, *а*). Эта объёмная фигура интересна тем, что её корпус плавно переходит в ручку, войдя в корпус у горлышка, внутри соединяется с ним, образуя замкнутую *одностороннюю* поверхность (рис. 4.75, *б*). Удалив овальный фрагмент корпуса в месте входа в него ручки, можно показывать фокус: налитая вода при наклоне бутылки не выливается, а скапливается внутри у горлышка.

Решение. Построение весьма замысловатой поверхности бутылки Клейна — хорошая иллюстрация богатых возможностей, гибкости, но и капризной непредсказуемости метода полиномиальной интерполяции (2.76), (2.13). Именно отсутствие *простых* аналитических выражений координат направляющей линии $p_{\text{н}}(t)$ и функции изменения радиуса $r(t)$ вынуждает строить их как интерполяционные линии.

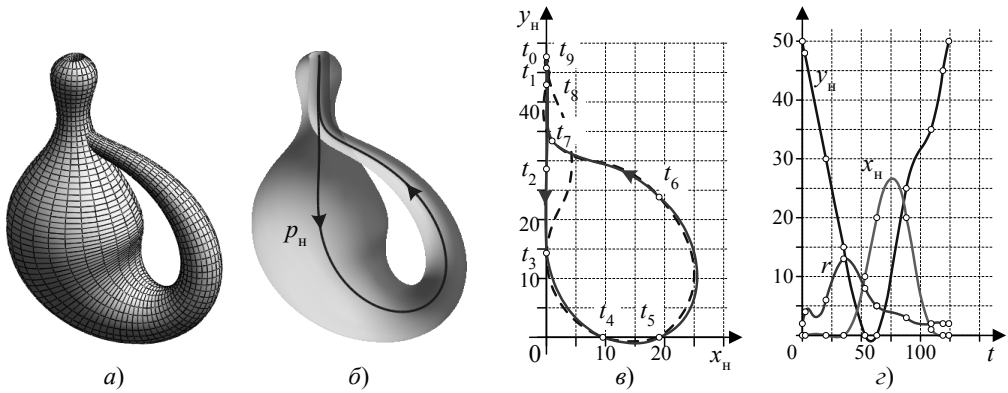


Рис. 4.75

Расположим на плоскости x_n десять узловых точек $p_{ni} = [x_{ni} \ y_{ni} \ 0] \ \forall i = \overline{0,9}$ направляющей $p_n(t)$ в наиболее характерных местах изменения направления и толщины канала (рис. 4.75,в). В каждом узле p_{ni} зададим радиус r_i образующей окружности, переносимой ортогонально направляющей линии. Для получения равномерной плотности поперечных каркасных линий поверхности зададим по (2.1а) длины параметрических интервалов d_i пропорционально **округлённым** до целых чисел межузловым расстояниям $|p_{ni} - p_{n,i-1}|$. Все необходимые для интерполяции данные сведены в табл. 4.2.

Таблица 4.2

i	0	1	2	3	4	5	6	7	8	9
t_i	0	2	20	35	53	63	88	109	119	124
x_{ni}	0	0	0	0	10	20	20	1	0	0
y_{ni}	50	48	30	15	0	0	25	35	45	50
r_i	2	4	6	13	8	5	3	2	2	2

Ручное вычисление по (2.13г) векторов коэффициентов интерполяционных полиномов девятой степени S_x , S_y и S_r является непреодолимо сложным, в чём убеждает значение (2.14) определителя матрицы Вандермонда $|Q| = 2 \cdot 10^{73}$. Полученные с помощью MathCAD интерполлянты $x_n(t)$, $y_n(t)$ и $r(t)$ (рис. 4.75,г) использовались для построения каналовой поверхности бутылки Клейна по алгоритму (4.53).

Решение задачи интерполяции очень чувствительно к выбору количества и расположения узловых точек, причём рост числа узлов увеличивает степень полинома и межузловые осцилляции. Похожий эффект наблюдается и при недостаточном числе узлов. Например, исключив как будто бы лишний узел p_{n2} , лежащий на прямой $p_{n1}p_{n3}$, получим направляющую линию с сильной осцилляцией, изображённую штрихом на рис. 4.75,в. □

Пример 4.14. Построить трубчатую поверхность топологического объекта, известного как «узел Эшера» (рис. 4.76,а).

Решение. Согласно (4.21) для построения каналовой поверхности необходимо иметь модель её направляющей линии $p_n(t)$. Ввиду отсутствия аналитических выражений для координат $x_n(t) \div z_n(t)$ построим эту **гладко замкнутую** линию методом сплайн-интерполяции. Зададим координаты семи узловых точек, через которые должна пройти направляющая линия, с помощью варьируемых коэффициентов $a=5$, $b=1$ и $c=3$ (рис. 4.76,б):

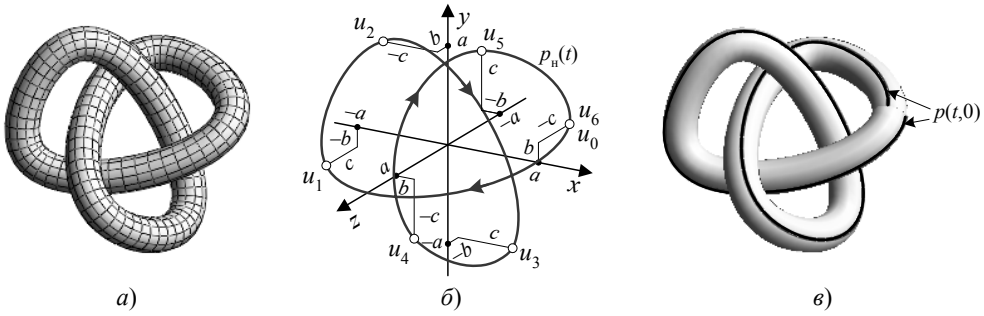


Рис. 4.76

$$u_0=u_6=[a \ b \ -c], \ u_1=[-a \ -b \ c], \ u_2=[-c \ a \ b], \ u_3=[c \ -a \ -b], \ u_4=[b \ -c \ a], \ u_5=[-b \ c \ -a].$$

Приняв локальный хордовый метод параметризации с длинами сегментных интервалов

$$d_1=d_3=d_5=2\sqrt{a^2+b^2+c^2} \approx 11.83, \quad d_2=d_4=d_6=\sqrt{(a-c)^2+(a+b)^2+(b-c)^2} \approx 6.63$$

(общая длина хорд $d_1+d_2+\dots+d_6 \approx 55.38$) и рассчитав по алгоритму (2.105) при краевых условиях (2.100б) матрицы

$$S_1 = \begin{bmatrix} 5 & 0.235 & -0.218 & 0.011 \\ 1 & -0.534 & -0.02 & 0.004 \\ -3 & 0.696 & 0.008 & -0.002 \end{bmatrix}, \quad S_2 = \begin{bmatrix} -5 & -0.431 & 0.161 & -0.008 \\ -1 & 0.799 & 0.133 & -0.018 \\ 3 & 0.029 & -0.065 & 0.002 \end{bmatrix},$$

$$S_3 = \begin{bmatrix} -3 & 0.696 & 0.008 & -0.002 \\ 5 & 0.235 & -0.218 & 0.011 \\ 1 & -0.534 & -0.02 & 0.004 \end{bmatrix}, \quad S_4 = \begin{bmatrix} 3 & 0.029 & -0.065 & 0.002 \\ -5 & -0.431 & 0.161 & -0.008 \\ -1 & 0.799 & 0.133 & -0.018 \end{bmatrix},$$

$$S_5 = \begin{bmatrix} 1 & -0.534 & -0.02 & 0.004 \\ -3 & 0.696 & 0.008 & -0.002 \\ 5 & 0.235 & -0.218 & 0.011 \end{bmatrix}, \quad S_6 = \begin{bmatrix} -1 & 0.799 & 0.133 & -0.018 \\ 3 & 0.029 & -0.065 & 0.002 \\ -5 & -0.431 & 0.161 & -0.008 \end{bmatrix}$$

полиномиальных коэффициентов шести сегментов направляющей линии $p_{ni}(t) = S_i T_3(t)$ $\forall t \in [0, d_i]$, $i = \overline{1, 6}$, построим поверхность трубки радиусом $r=1$ сначала по усечённому алгоритму (4.53а,б) без отрисовки каркасных линий (рис. 4.76,в). Специальное выделение продольной каркасной линии $p(t, 0)$ убеждает, что трубка имеет осевое кручение, приводящее к **параметрической** незамкнутости её поверхности.

Вычислив по (1.124) угол $\theta \approx 81.9^\circ$ между радиальными векторами $p_1(0, 0) - p_{n1}(0)$ и $p_6(d_6, 0) - p_{n6}(d_6)$, добавим по (4.53в) подкручивание каждого из 96 поперечных сечений на угол $\gamma = \theta/96 \approx 0.85^\circ$. В результате трубчатая поверхность становится замкнутой не только визуально, но и параметрически, как показано на рис. 4.76,а. $\square$

Квадратичные поверхности

Квадратичные поверхности описываются **алгебраическими** неявными уравнениями $f(p)=0$ типа (1.158б). Параметрические формы поверхностей второго порядка уже изучались нами в п. 1.5.10, однако там шла речь об их получении не путём перемещения линий в пространстве, а подбором подходящих функций подстановки $x(t, \tau) \div z(t, \tau)$, удовлетворяющих уравнению $f(x, y, z)=0$. Но именно кинематический метод с целенаправленным выбором элементов движения позволяет получить поверхность с желаемым расположением каркас-

ных линий, нужной ориентацией нормалей и корректными интервалами изменений параметров. При наличии кинематических элементов $p_n(t)$ и $p_o(\tau)$ построение касательных векторов $V(t, \tau)$, $W(t, \tau)$ и нормали $N(t, \tau)$ выполняется буквально «на пальцах», а не путём трудоёмких вычислений по (1.93б) и (1.94).

Изучая линейчатые поверхности, мы убедились, что некоторые квадратичные объекты — цилиндры, конусы, однополостные гиперболоиды и гиперболические параболоиды — могут быть сформированы движением прямой линии. Поверхности с **осевой симметрией** можно строить либо вращением специально выбранной образующей линии вокруг оси симметрии, либо переносом вдоль неё окружности переменного радиуса. Составим, например, двумя способами модель $p(t, \tau)$ кругового эллипсоида $(x/a)^2 + y^2 + z^2 - 1 = 0$:

- **вращением** половины эллипса

$$p_o(\tau) = [\cos(\tau) \sin(\tau) 0] \in xy \quad \forall \tau \in [0, \pi]$$

вокруг оси x матрицей $A(t) = R_x(t) \quad \forall t \in [0, 2\pi]$ при $p_n(t) = O$ (рис. 4.77, а). Тогда по табл. 4.1 получим

$$p(t, \tau) = [\cos(\tau) \cos(t) \sin(\tau) \sin(t) \sin(\tau)]. \quad (4.65)$$

Проведя в любой точке поверхности касательные векторы V и W в направлении роста параметров t и τ , построим нормаль $N = V \times W$, которая для модели (4.65) является внутренней;

- **переносом** образующей окружности

$$p_o(\tau) = [0 \cos(\tau) \sin(\tau)] \in yz \quad \forall \tau \in [0, 2\pi]$$

вдоль направляющей прямой

$$p_n(t) = [\cos(t) 0 0] \parallel x^o \quad \forall t \in [0, \pi]$$

с одновременным её **масштабированием** матрицей $A(t) = \sin(t)E_3$ (рис. 4.77, б). Подставляя эти кинематические элементы в (4.21), получим модель эллипсоида

$$p(t, \tau) = [\cos(t) \sin(t) \cos(\tau) \sin(\tau) \sin(\tau)], \quad (4.66)$$

формально совпадающую с (4.65) с точностью до имён параметров. Модель (4.66) имеет, судя по расположению векторов V и W , внешние нормали.

Второй способ ближе к универсальному кинематическому методу формирования произвольной квадратичной поверхности сложным аффинным преобразованием образующей **окружности** единичного радиуса. Процесс синтеза целесообразно разделить на два этапа.

Этап 1. Строится модель **нормированной канонической поверхности** с коэффициентами $a=b=c=1$ в координатных функциях (1.163)–(1.171):

$$p^k(t, \tau) = r(t)p_o(\tau) + p_n(t). \quad (4.67a)$$

Составим **возможные варианты кинематических элементов** нормированных квадратичных поверхностей (4.67a), изображённых на рис. 1.142: функ-

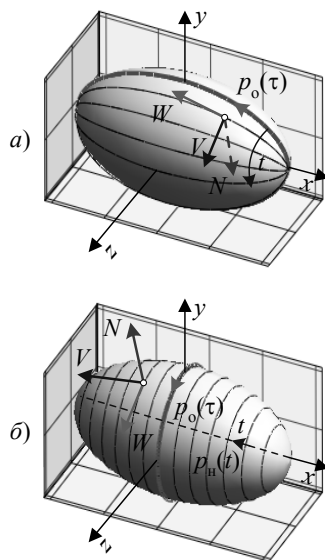


Рис. 4.77

ции изменения радиуса $r(t)$ окружности $p_o(\tau)$ в процессе переноса её центра по направляющей прямой $p_n(t)$. По умолчанию выбрана образующая окружность (4.49в) единичного радиуса в плоскости xz

$$p_o(\tau) = [\sin(\tau) \ 0 \ \cos(\tau)] \forall \tau \in [0, 2\pi] \quad (4.67б)$$

(лишь для построения гиперболического и параболического цилиндров используются другие образующие линии):

- у **действительной сферы** (1.163а)

$$r(t) = \sin(t), \ p_n(t) = [0 \ \cos(t) \ 0] \forall t \in [0, \pi]; \quad (4.68а)$$

- у **однополостного гиперboloида** (1.164)

$$r(t) = \text{ch}(t), \ p_n(t) = [0 \ \text{sh}(t) \ 0] \forall t \in (-\infty, \infty); \quad (4.68б)$$

• кинематическая модель **двуполостного гиперboloида** (1.165) используется со знаками (+) для верхней и (–) для нижней полостей:

$$r(t) = \text{sh}(t), \ p_n(t) = [0 \ \pm \text{ch}(t) \ 0] \forall t \in [0, \infty); \quad (4.68в)$$

- у **круглого конуса** (1.166)

$$r(t) = t, \ p_n(t) = [0 \ t \ 0] \forall t \in (-\infty, \infty); \quad (4.68г)$$

• кинематической модели **гиперболического параболоида** (1.167а) удовлетворяют следующие элементы:

$$r(t) = t, \ p_n(t, \tau) = [0 \ -t^2 \cos(2\tau) \ 0] \forall t \in [0, \infty), \ \tau \in [0, 2\pi]; \quad (4.68д)$$

- у **круглого параболоида** (1.168)

$$r(t) = t, \ p_n(t) = [0 \ t^2 \ 0] \forall t \in [0, \infty); \quad (4.68е)$$

- у **круглого цилиндра** (1.169)

$$r(t) = 1, \ p_n(t) = [0 \ t \ 0] \forall t \in (-\infty, \infty); \quad (4.68ж)$$

• кинематическая модель **гиперболического цилиндра** (1.170) содержит отличную от предыдущих вариантов образующую линию, состоящую из двух ветвей гиперболы в плоскости xz :

$$p_o(\tau) = [\pm \text{ch}(\tau) \ 0 \ \text{sh}(\tau)], \ r(t) = 1, \ p_n(t) = [0 \ t \ 0] \forall t, \tau \in (-\infty, \infty); \quad (4.68з)$$

• для построения **параболического цилиндра** (1.171) используется образующая линия в виде параболы в плоскости xz :

$$p_o(\tau) = [\tau \ 0 \ \tau^2], \ r(t) = 1, \ p_n(t) = [0 \ t \ 0] \forall t, \tau \in (-\infty, \infty). \quad (4.68и)$$

Этап 2. Построение **произвольной квадратичной поверхности** выполняется последовательностью аффинных преобразований канонической модели (4.67) с кинематическими элементами (4.68), соответствующими типу поверхности. Сложное преобразование с матрицей C

$$\tilde{p}(t, \tau) = \tilde{p}^k(t, \tau)C \quad (4.69)$$

устанавливает между матрицами F^k канонической и F произвольной неявных форм (1.158б) соотношения, аналогичные (3.16):

$$F^k = C F C^T, \quad F = C^{-1} F^k (C^{-1})^T. \quad (4.70)$$

Матрицы квадратичных форм аффинно связанных квадратичных объектов являются **конгруэнтными**. Зная матрицу F , всегда можно по (1.158) восстановить неявное уравнение поверхности $f(x, y, z) = 0$.

Пример 4.15. Получить кинематическую модель однополостного гиперboloида (см. рис. 1.142, б), каноническая форма которого подвергается масштабированию с коэффициентами $a=c=1/\sqrt{2}$, $b=0.5$, вращениям на углы $\psi = \arctg(\sqrt{2}) \approx 54.7^\circ$, $\varphi = 45^\circ$ (см. рис. 1.1, б) и переносу центра из начала координат в точку $o = [0.5 \ 0.5 \ 0.5]$.

Решение. Подставив (4.67б) и (4.68б) в (4.67а), получим координатные функции канонической модели однополостного гиперboloида: $x^k(t, \tau) = \text{ch}(t) \sin(\tau)$, $y^k(t, \tau) = \text{sh}(t)$ и $z^k(t, \tau) = \text{ch}(t) \cos(\tau)$. Вычислив матрицу сложного преобразования

$$C = \tilde{M}(a, b, c) \tilde{R}_x(\psi) \tilde{R}_y(\varphi) T(o) = \begin{bmatrix} 1/2 & 0 & -1/2 & 0 \\ 1/2\sqrt{3} & 1/2\sqrt{3} & 1/2\sqrt{3} & 0 \\ 1/2\sqrt{3} & -1/\sqrt{3} & 1/2\sqrt{3} & 0 \\ 1/2 & 1/2 & 1/2 & 1 \end{bmatrix}, \quad (4.71)$$

получим по (4.69) координатные функции преобразованной поверхности (рис. 4.78):

$$\begin{aligned} x(t, \tau) &= x^k(t, \tau)/2 + (1/2\sqrt{3})y^k(t, \tau) + (1/2\sqrt{3})z^k(t, \tau) + 1/2 = \\ &= (1 + \text{ch}(t) \sin(\tau))/2 + (1/2\sqrt{3})(\text{sh}(t) + \text{ch}(t) \cos(\tau)), \\ y(t, \tau) &= (1/2\sqrt{3})y^k(t, \tau) - (1/\sqrt{3})z^k(t, \tau) + 1/2 = \\ &= 1/2 + (1/2\sqrt{3})(\text{sh}(t) - 2\text{ch}(t) \cos(\tau)), \\ z(t, \tau) &= -x^k(t, \tau)/2 + (1/2\sqrt{3})y^k(t, \tau) + (1/2\sqrt{3})z^k(t, \tau) + 1/2 = \\ &= (1 - \text{ch}(t) \sin(\tau))/2 + (1/2\sqrt{3})(\text{sh}(t) + \text{ch}(t) \cos(\tau)). \end{aligned}$$

Сформировав по каноническому уравнению однополостного гиперboloида (см. табл. 1.3 при $a=b=c=1$) матрицу $F^k = \text{diag}[1 \ -1 \ 1 \ -1]$, получим по (4.70) матрицу неявной формы преобразованной поверхности:

$$F = \begin{bmatrix} 0 & -2 & -2 & 2 \\ -2 & 0 & -2 & 2 \\ -2 & -2 & 0 & 2 \\ 2 & 2 & 2 & -4 \end{bmatrix}. \quad (4.72)$$

Она с точностью до некритичного множителя, равного -2 , эквивалентна матрице (4.47) из примера 4.10. $\square$

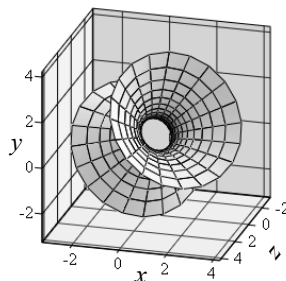


Рис. 4.78

Рассмотрим теперь обратную задачу параметризации квадратичного объекта — получения его параметрической модели $p(t) \in \mathbb{R}^2$ на плоскости либо $p(t, \tau) \in \mathbb{R}^3$ в пространстве по произвольному, т. е. не каноническому, неявному уравнению $f(p) \equiv \tilde{p} F \tilde{p}^T = 0$ с симметричной матрицей $F \in \mathbb{R}^{\mu \times \mu}$, где $\mu = n+1$, $n \in \{2, 3\}$. В самом общем случае квадрика принимает вид, аналогичный (1.58а):

$$f(p) = \sum_{i=1}^n \sum_{j=1}^n f_{ij} p_i p_j + 2 \sum_{i=1}^n f_{i\mu} p_i + f_{\mu\mu}. \quad (4.73)$$

Пусть C — неизвестная пока матрица преобразования (4.69) канонического объекта в заданный объект. Определив с помощью табл. 1.1, 1.3, 1.4 тип объекта

по инвариантам и полуинвариантам его матрицы F , сформируем каноническую матрицу F^K и соответствующее ей при $a=b=c=1$ параметрическое описание линии $p^K(t) \in \mathbb{R}^2$ типа (1.65) либо поверхности $p^K(t, \tau) \in \mathbb{R}^3$ из (1.163)–(1.171). Матрицы квадратичных форм канонического и заданного объектов связаны с матрицей преобразования соотношениями (4.70).

Помимо аффинных преобразований существует ещё один фактор, изменяющий коэффициенты неявного уравнения квадратичного объекта и его матрицу F . Это множитель $k_3 \neq 0$, используемый для записи неявного уравнения объекта $f(p)=0$ в более удобном эквивалентном виде:

$$k_3 f(p) = \tilde{p} F_3 \tilde{p}^T = 0 \Rightarrow F_3 = k_3 F. \quad (4.74)$$

Например, уравнение эллипса на плоскости $(x/2)^2 + (y/3)^2 - 1 = 0$, матрица которого $F = \text{diag}[1/4 \ 1/9 \ -1]$ имеет два дробных диагональных элемента, можно домножить на коэффициент $k_3 = 36$ и записать в виде $9x^2 + 4y^2 - 36 = 0$ с целочисленной матрицей $F_3 = \text{diag}[9 \ 4 \ -36]$. Сравнение матриц F одной и той же поверхности в (4.47) и (4.72) также доказывает возможность существования ненулевого множителя k_3 в уравнениях связи матриц канонического и заданного квадратичных объектов:

$$k_3 F^K = C F C^T, \quad F = k_3 C^{-1} F^K (C^{-1})^T. \quad (4.75)$$

Важно отметить, что не являющееся аффинным преобразование (4.74) сохраняет геометрическое место точек квадрики и в общем случае не может быть заменено масштабированием координат. Например, если в уравнении (4.73) есть слагаемые $f_{11}x^2$ и $2f_{1\mu}x$, то умножение на k_3 преобразует их в $f_{11}k_3x^2$ и $2f_{1\mu}k_3x$, тогда как замена координаты x на $m_x x$ даёт $f_{11}m_x^2x^2$ и $2f_{1\mu}m_x x$. Понятно, что при $m_x \neq 1$ соотношения $k_3 = m_x^2$ и $k_3 = m_x$ не совместны.

Для решения задачи приведения квадрики к канонической форме в ортонормированном базисе адаптируем известный по [8, 9, 12, 21, 51, 54, 59] **метод Лагранжа**, называемый ещё **методом диагонализации**. Ортогональность базиса канонической модели необходима для того, чтобы матрица C не содержала преобразования сдвига (см. п. 3.1.2), нарушающего симметричность изображения объекта при конечных интервалах изменения параметров t и τ .

Ключ к решению задачи диагонализации квадратичной формы — во взаимозависимости собственных чисел матриц F_{11} и F_{11}^K — блоков матриц F и F^K из (1.1586). Из вышеприведённых источников известно, что каждая действительная **симметричная** $n \times n$ -матрица $M = M^T$ ранга $r \leq n$ имеет диагональное разложение $M = X \Lambda X^T$, в котором:

- X — матрица **ортонормированных** собственных векторов-столбцов, обладающая свойствами $\text{rang}[X] = n$ и $X^{-1} = X^T$;
- $\Lambda = \text{diag}[\lambda_1 \ \dots \ \lambda_n]$ — диагональная матрица собственных чисел ранга r .

Следует отметить, что канонические матрицы всех квадратичных объектов, перечисленных в табл. 1.1, 1.3, 1.4, имеют диагональные блоки $F_{11}^K = \Lambda^K$ с единичными матрицами собственных векторов $X^K = E_n$.

По **закону инерции квадратичных форм** любое невырожденное преобразование не изменяет ранг матрицы формы $r = n_+ + n_-$, её **сигнатуру** $\sigma = n_+ - n_-$ (разность между числом n_+ положительных и n_- отрицательных собственных значений) и **дефект** $n - r$ (число нулевых собственных значений). Умножение матрицы M на число $k \neq 0$ изменяет её собственные значения λ_i на $k\lambda_i$, сигнатуру σ на $\text{sgn}(k)\sigma$ и не изменяет матрицу X . Следовательно, знак числа k_s в (4.75) можно определить сравнением знаков сигнатур σ и σ^K матриц F_{11} и F_{11}^K .

Матрица преобразования (4.69) находится в N -шаговом процессе последовательных преобразований матрицы квадратичной формы от исходного значения F до конечной канонической матрицы F^K

$$F \equiv F^{(0)} \xrightarrow{X_1} F^{(1)} \xrightarrow{X_2} F^{(2)} \xrightarrow{X_3} \dots \xrightarrow{X_N} F^{(N)} \equiv F^K$$

как произведение обратных матриц шаговых преобразований X_i :

$$C = \left(\prod_{i=1}^N X_i \right)^{-1} = \prod_{i=N}^1 X_i^{-1} = X_N^{-1} \dots X_2^{-1} X_1^{-1}.$$

С учётом (4.70) будем после вычисления очередной матрицы X формировать матрицы F и C по следующим рекуррентным формулам:

$$F = X^{-1} F (X^{-1})^T, \quad C = X^{-1} C. \quad (4.76)$$

Алгоритм расчёта матрицы приведения произвольной квадратки $\{F\}$ к канонической форме $\{F^K\}$ работает следующим образом.

 **Шаг 1.** Вычисляем матрицу X собственных векторов блока F_{11} матрицы F , после чего находим

$$C = \tilde{X}^T, \quad F = C F \tilde{X}. \quad (4.77a)$$

В результате блок F_{11} становится диагональным, т. е.

$$F = \left[\begin{array}{cc|c} f_{11} & 0 & f_{13} \\ 0 & f_{22} & f_{23} \\ \hline f_{13} & f_{23} & f_{33} \end{array} \right] \quad \text{либо} \quad F = \left[\begin{array}{ccc|c} f_{11} & 0 & 0 & f_{14} \\ 0 & f_{22} & 0 & f_{24} \\ 0 & 0 & f_{33} & f_{34} \\ \hline f_{14} & f_{24} & f_{34} & f_{44} \end{array} \right],$$

а в (4.73) двойная сумма превращается в сумму квадратов $\sum_{i=1}^n f_{ii} p_i^2$, т. е. исчезают смешанные произведения $2f_{ij} p_i p_j \quad \forall i \neq j$.

Шаг 2. Находим сигнатуры матриц F_{11} и F_{11}^K :

$$\sigma = \sum_{i=1}^n (\{f_{ii} > 0\} - \{f_{ii} < 0\}), \quad \sigma^K = \sum_{i=1}^n (\{f_{11}^K > 0\} - \{f_{11}^K < 0\}). \quad (4.77b)$$

Если $\sigma \neq \sigma^k$, то полагаем $F = -F$. В результате матрицы F_{11} и F_{11}^k приобретают одинаковую структуру собственных значений $\{n_+, n_-, r\}$, а неизвестный коэффициент k_3 будет положительным.

Шаг 3. Формируем матрицу переноса $X = T(\Delta p)$ с элементами

$$\chi_{\mu i} \equiv \Delta p_i = \text{if}(f_{ii} : f_{i\mu} / f_{ii}, 0) \forall i = \overline{1, n}, \mu = n+1 \quad (4.77\text{в})$$

и обновляем по (4.76) матрицы F и C . В результате обнуляются элементы $f_{i\mu} \equiv f_{\mu i}$, соответствующие ненулевым значениям $f_{ii} \neq 0$, а в (4.73) у каждой суммы $f_{ii} p_i^2 + 2f_{i\mu} p_i$ пропадает второе слагаемое.

Шаг 4. В блоке F_{11} ищем диагональный элемент $f_{mm} = 0$ с минимальным индексом $m \in [1, n]$. Если такой элемент найден, то, сформировав матрицу $X = E_{\mu, \mu}$, заменяем отдельные элементы её m -го столбца на

$$\chi_{im} = -f_{i\mu}, \quad \chi_{\mu m} = -f_{\mu\mu} \quad \forall i \geq m: \{f_{ii} = 0\} \wedge \{f_{i\mu} \neq 0\}. \quad (4.77\text{г})$$

После обновления матриц F и C по (4.76) линейная часть квадратики $2 \sum_{i \geq m} f_{i\mu} p_i + f_{\mu\mu}$ с отсутствующими членами $f_{ii} p_i^2$ заменяется на $-p_m$.

Данный шаг выполняется только для следующих **нецентральных** квадратичных объектов: парабола (см. табл. 1.1), гиперболический и эллиптический параболоиды (см. табл. 1.3) и параболический цилиндр (см. табл. 1.4).

Шаг 5. При $f_{\mu\mu} f_{\mu\mu}^k \neq 0$ рассчитаем коэффициент k_3 и обновлённую матрицу квадратичной формы F :

$$k_3 = f_{\mu\mu} / f_{\mu\mu}^k \Rightarrow F = k_3^{-1} F. \quad (4.77\text{д})$$

Шаг 6. Инициализируем нулевую матрицу $H = O_{n,n}$ и вычисляем её отдельные элементы по формуле

$$h_{ij} = \begin{cases} \sqrt{f_{ii}^k / f_{jj}} & \text{при } \{f_{jj} \neq 0\} \wedge \{f_{ii}^k f_{jj} > 0\}, \\ 1 & \text{при } \{f_{ii}^k = 0\} \wedge \{f_{jj} = 0\} \wedge \{f_{i\mu}^k = f_{j\mu}\} \\ \forall i = \overline{1, n}, j = \overline{1, n}. \end{cases} \quad (4.77\text{е})$$

В результате получаем **невыврожденную** матрицу масштабирования и перестановок H , преобразующую матрицу F к каноническому виду $F^k = \tilde{H} F \tilde{H}^T$.

Шаг 7. Возврат матрицы $\tilde{H}C$. $\square$

Пример 4.16. Получить параметрические модели и построить график неявной функции $(536 - 6x + 9y)(x - 3y + 14) - (x - y - 76)(3y - 50) = 0$.

Решение. Раскрывая скобки, получим квадратичную форму и её матрицу

$$f(x, y) = -6x^2 - 24y^2 + 24xy + 502x - 1304y + 3704, \quad F = \left[\begin{array}{cc|c} -6 & 12 & 251 \\ 12 & -24 & -652 \\ \hline 251 & -652 & 3704 \end{array} \right]$$

с инвариантами $I_1 = -30$, $I_2 = 0$, $I_3 = 135000$. По табл. 1.1 определяем тип канонической линии — **парабола** $x^2 - y = 0$ с матрицей квадратичной формы и параметрической моделью

$$F^k = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -0.5 \\ 0 & -0.5 & 0 \end{bmatrix}, \quad p^k(t) = \begin{bmatrix} t & t^2 \end{bmatrix} \forall t \in (-\infty, \infty).$$

Матрица собственных векторов блока $F_{11} = \begin{bmatrix} -6 & 12 \\ 12 & -24 \end{bmatrix}$ равна $X = \begin{bmatrix} 0.894 & -0.447 \\ 0.447 & 0.894 \end{bmatrix}$. Тогда по (4.77а) получаем

$$C = \begin{bmatrix} 0.894 & 0.447 & 0 \\ -0.447 & 0.894 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad F = \begin{bmatrix} 0 & 0 & -67.082 \\ 0 & -30 & -695.417 \\ -67.082 & -695.417 & 3704 \end{bmatrix}.$$

Блоки F_{11} и F_{11}^k имеют собственные числа $\lambda \in \{0, -30\}$, $\lambda^k \in \{1, 0\}$ и по (4.77б) сигнатуры $\sigma = -1$, $\sigma^k = 1 \neq \sigma$. Изменив знак матрицы F (теперь у неё $\lambda \in \{0, 30\}$, $\sigma = 1$), формируем по (4.77в) матрицу переноса $X = T([0 \ 23.181])$, а по (4.76) обновляем матрицы C и F :

$$C = \begin{bmatrix} 0.894 & 0.447 & 0 \\ -0.447 & 0.894 & 0 \\ 10.367 & -20.733 & 1 \end{bmatrix}, \quad F = \begin{bmatrix} 0 & 0 & 67.082 \\ 0 & 30 & 0 \\ 67.082 & 0 & -19\,824 \end{bmatrix}.$$

Поскольку в матрице F_{11} имеется один нулевой диагональный элемент при $m=1$, то по (4.77г) строим матрицу X , обновляющую по (4.76) матрицы C и F :

$$X = \begin{bmatrix} -134.164 & 0 & 0 \\ 0 & 1 & 0 \\ 19\,824 & 0 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} -0.0067 & -0.0033 & 0 \\ -0.447 & 0.894 & 0 \\ 142.528 & 45.347 & 1 \end{bmatrix}, \quad F = \begin{bmatrix} 0 & 0 & -0.5 \\ 0 & 30 & 0 \\ -0.5 & 0 & 0 \end{bmatrix}.$$

Сравнивая матрицы F и F^k , по (4.77е) формируем матрицу масштабирования и перестановки H , с помощью которой получаем окончательные матрицы C и F :

$$\tilde{H} = \begin{bmatrix} 0 & 0.183 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} -0.0816 & 0.1633 & 0 \\ -0.0067 & -0.0033 & 0 \\ 142.528 & 45.347 & 1 \end{bmatrix}, \quad F = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -0.5 \\ 0 & -0.5 & 0 \end{bmatrix}. \quad (4.78)$$

Параметрические функции, по которым строится график искомой параболы (рис. 4.79), имеют следующий вид:

$$\tilde{p}(t) = \tilde{p}^k(t)C = \begin{bmatrix} t & t^2 & 1 \end{bmatrix} \begin{bmatrix} -0.0816 & 0.1633 & 0 \\ -0.0067 & -0.0033 & 0 \\ 142.528 & 45.347 & 1 \end{bmatrix},$$

$$\begin{cases} x(t) = 142.528 - 0.0816t - 0.0067t^2, \\ y(t) = 45.347 + 0.163t - 0.0033t^2. \end{cases}$$

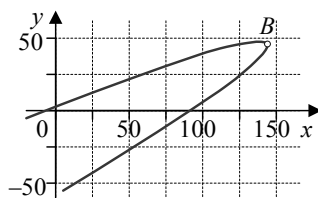


Рис. 4.79

Нетрудно установить параметры элементарных преобразований, которыми каноническая парабола превращается в данную. Допустим, первым выполняется масштабирование линии $p^k(t)$ матрицей (3.32) с коэффициентами m_x и m_y , вторым — вращение матрицей (3.34) на угол φ , а третьим — перенос на вектор $B = [142.528 \ 45.347]$. Сравнивая блок A матрицы C из (4.78) с произведением матриц первых двух преобразований

$$M(m_x, m_y)R(\varphi) = \begin{bmatrix} m_x \cos(\varphi) & m_x \sin(\varphi) \\ -m_y \sin(\varphi) & m_y \cos(\varphi) \end{bmatrix} = \begin{bmatrix} -0.0816 & 0.1633 \\ -0.0067 & -0.0033 \end{bmatrix},$$

получим значения $m_x = 0.183$, $m_y = 0.0075$ и $\varphi = 116.57^\circ$. $\square$

Пример 4.17. Построить параметрическую модель и изображение поверхности, заданной матрицей квадратичной формы (4.47) из примера 4.10.

Решение. Поскольку ранее было определено, что заданная матрица

$$F = \left[\begin{array}{ccc|c} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & -0 & -1 \\ \hline -1 & -1 & -1 & 2 \end{array} \right]$$

представляет **однополостный гиперболоид**, то по табл. 1.3 и (4.67), (4.68б) составляем

$$F^K = \left[\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{array} \right], \quad \begin{cases} p^K(t, \tau) = [\text{ch}(t)\sin(\tau) \text{ sh}(t) \text{ ch}(t)\cos(\tau)] \\ \forall t \in (-\infty, \infty), \tau \in [0, 2\pi]. \end{cases}$$

$$\text{Матрица собственных векторов блока } F_{11} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \text{ равна } X = \begin{bmatrix} 0.638 & 0.509 & -0.577 \\ -0.76 & 0.298 & -0.577 \\ 0.122 & -0.807 & -0.577 \end{bmatrix}.$$

$$\text{Тогда по (4.77а) получаем } C = \begin{bmatrix} 0.638 & -0.76 & 0.122 & 0 \\ 0.509 & 0.298 & -0.807 & 0 \\ -0.577 & -0.577 & -0.577 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad F = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 2 & 1.732 \\ 0 & 0 & 1.732 & 2 \end{bmatrix}.$$

Блоки F_{11} и F_{11}^K имеют собственные числа $\lambda \in \{-1, -1, 2\}$, $\lambda^K \in \{1, -1, 1\}$ и по (4.77б) сигнатуры равны $\sigma = -1$ и $\sigma^K = 1 \neq \sigma$. Изменив знак матрицы F , формируем по (4.77в) матрицу переноса $X = T([0 \ 0 \ 0.866])$, а по (4.76) обновляем матрицы C и F :

$$C = \begin{bmatrix} 0.638 & -0.76 & 0.122 & 0 \\ 0.509 & 0.298 & -0.807 & 0 \\ -0.577 & -0.577 & -0.577 & 0 \\ 0.5 & 0.5 & 0.5 & 1 \end{bmatrix}, \quad F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & -0.5 \end{bmatrix}.$$

Поскольку в матрице F_{11} отсутствуют нулевые диагональные элементы, то, пропуская **шаг 4**, переходим к вычислению по (4.77д) коэффициента $k_3 = 0.5$ и обновлённой матрицы $F = \text{diag}[2 \ 2 \ -4 \ -1]$. Сравнивая её с F^K , по (4.77е) формируем матрицу масштабирования и перестановок H , с помощью которой получаем окончательные матрицы C и $F = F^K$:

$$\tilde{H} = \begin{bmatrix} 0.707 & 0 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0.707 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} 0.451 & -0.537 & 0.086 & 0 \\ -0.289 & -0.298 & -0.289 & 0 \\ 0.36 & 0.211 & -0.571 & 0 \\ 0.5 & 0.5 & 0.5 & 1 \end{bmatrix}, \quad F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}. \quad (4.79)$$

Таким образом, параметрические зависимости, по которым строится искомая поверхность однополостного гиперболоида (см. рис. 4.78), приобретают следующий вид:

$$\tilde{p}(t, \tau) = \tilde{p}^K(t, \tau) C = [\text{ch}(t)\sin(\tau) \text{ sh}(t) \text{ ch}(t)\cos(\tau) \ 1] \begin{bmatrix} 0.451 & -0.537 & 0.086 & 0 \\ -0.289 & -0.289 & -0.289 & 0 \\ 0.36 & 0.211 & -0.571 & 0 \\ 0.5 & 0.5 & 0.5 & 1 \end{bmatrix},$$

$$\begin{cases} x(t, \tau) = 0.5 + 0.451 \text{ch}(t) \sin(\tau) - 0.289 \text{sh}(t) + 0.36 \text{ch}(t) \cos(\tau), \\ y(t, \tau) = 0.5 - 0.537 \text{ch}(t) \sin(\tau) - 0.289 \text{sh}(t) + 0.211 \text{ch}(t) \cos(\tau), \\ z(t, \tau) = 0.5 + 0.086 \text{ch}(t) \sin(\tau) - 0.289 \text{sh}(t) - 0.571 \text{ch}(t) \cos(\tau). \end{cases}$$

Отличие полученной матрицы C и параметрической модели $p(t, \tau)$ от (4.71) при полном совпадении геометрических мест точек поверхностей объясняется бесконечным числом цепочек элементарных преобразований канонической поверхности в заданную. Наличие в матрице C из (4.79) ненулевого элемента $C_{12} = -0.537$ свидетельствует, что она таит в себе дополнительное преобразование в сравнении с аналогичной матрицей в (4.71). Например, это может быть осевое вращение канонической поверхности вокруг оси y до масштабирования, не затрагивающее НФ $f(x, y, z)$, но изменяющее параметрическую модель $p(t, \tau)$. $\square$

Поверхности высших порядков и трансцендентные поверхности

Поверхности высших порядков описываются **алгебраическими** неявными уравнениями $f(p)=0$ или параметрическими функциями $p(t, \tau)$ со степенями координат или параметров выше второй. Высокие степени полиномов координатных функций могут быть лишь формальным признаком высокого порядка поверхности. Например, векторная функция

$$p(t, \tau) = [t^2 + \tau^3 \quad t^4 + \tau - 1 \quad t^4 + \tau + 1]$$

четвёртой степени относительно параметра t и третьей относительно τ описывает **плоскость** — поверхность первого порядка. Действительно, вычислив по (1.93б) и (1.94) направляющие векторы и нормаль

$$V = [2t \ 4t^3 \ 4t^3], \quad W = [3\tau^2 \ 1 \ 1] \Rightarrow N = V \times W = 2t(6t^2\tau^2 - 1)[0 \ 1 \ -1],$$

обнаруживаем постоянное направление последней для всех значений t и τ , что является доказательством того, что поверхность является плоскостью.

Трансцендентные поверхности имеют **трансцендентные** координатные функции в неявных уравнениях $f(p)=0$ или описаниях кинематических элементов — образующих и направляющих линий. Раскроем, например, секреты создания поверхности цветка, изображённого в разных ракурсах на рис. 4.5:

- образующая линия задаётся в виде горизонтальной окружности

$$p_o(\tau) = [\cos(\tau) \ 0 \ \sin(\tau)] \forall \tau \in [0, 2\pi];$$

- вертикальная направляющая линия описывается зависимостью

$$p_n(t) = [0 \ h(t) \ 0] \forall t \in [0, 8],$$

в которой функция $h(t) = 24.6t - 4.92t^2 + 0.318t^3$ задаёт неравномерное движение центра образующей по оси y так, что при $0 \leq t \leq 2.5$ большая скорость подъёма формирует длинный стебель, а при $2.5 \leq t \leq 8$ совершается одно колебательное движение, создающее выпукло-вогнутый рельеф лепестков в радиальном направлении. Такая зависимость $h(t)$ получена методом полиномиальной интерполяции (см. п. 2.1.1) при следующих параметрах узлов $\{t_i, h_i\}$, $i = \overline{0, 3}$:

$$t_0=0, \ t_1=1, \ t_2=4, \ t_3=8, \ h_0=0, \ h_1=20, \ h_2=40, \ h_3=40;$$

- изменение радиуса образующей окружности в процессе движения по формуле $r(t) = 1 + 0.41t + 0.635t^2 - 0.044t^3$ формирует узкую трубку стебля и резкое расширение поверхности в верхней области лепестков. Интерполяционный полином $r(t)$ имеет узловые значения $r_0=1, r_1=2, r_2=10$ и $r_3=20$;

- трансцендентная функция $\varphi(t, \tau) = 1 + 0.75(1 - e^{-0.1t})\cos(7\tau)$ формирует семь лепестков цветка путём модуляции радиуса $r(t)$ колебаниями с экспоненциально увеличивающейся амплитудой;

- трансцендентная функция $\psi(\tau) = 1 - 0.1\sin(7\tau)$ формирует изгиб лепестков путём модуляции высоты точек поверхности $h(t)$.

Взаимодействие всех рассмотренных элементов составляет желаемую кинематическую модель поверхности цветка: $p(t, \tau) = r(t)\varphi(t, \tau)p_o(\tau) + \psi(\tau)p_n(t)$.

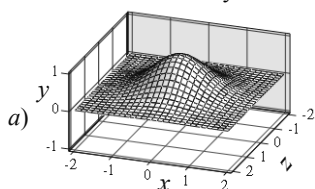
Наибольшую трудность представляет изображение **неявных** поверхностей $f(x,y,z)=0$, не допускающих параметризации либо не приводимых к явным формам. В п. 4.2.2 в общих чертах описан метод построения **линий уровня** — сечений поверхности плоскостями, параллельными координатным плоскостям.

Изображение сложной неявной поверхности облегчается, если:

- уравнение $f(x,y,z)=0$ разрешимо относительно некоторой координаты, например, $y=\psi(x,z)$. Параметризация $p(x,z)=[x \ \psi(x,z) \ z]$ позволяет построить каркас из линий уровня $p(x_i, z)$ и $p(x, z_j)$;
- сложная функция $f(x,y,z)$ факторизуема, т. е. представима произведением функций $f_i(x,y,z)$. Тогда поверхность $f(x,y,z)=0$ является объединением всех поверхностей $f_i(x,y,z)=0$, которые могут строиться независимо друг от друга.

 **Пример 4.18.** Построим следующие поверхности.

1. Аналог гауссовской функции плотности вероятности двух переменных



$\psi(x,z)=e^{-(x^2+z^2)}$ как параметрическую поверхность $p(x,z)=[x \ \psi(x,z) \ z]$ (рис. 4.80,а) и на её основе «холмистый рельеф» как сумму масштабируемых по осям координат гауссовских поверхностей:

$$y(x,z)=\sum_{i=1}^N h_i \psi(\gamma_{xi}(x-c_{xi}), \gamma_{zi}(z-c_{zi})). \quad (4.80)$$

Параметры i -го холма (впадины) задаются:

- h_i — высотой (глубиной при $h_i < 0$);
- c_{xi} и c_{zi} — координатами вершины;
- γ_{xi} и γ_{zi} — коэффициентами крутизны склонов в направлении осей x и z .

Рельеф на рис. 4.80,б построен суммированием четырёх холмов и одной впадины с векторами параметров

$$v_i=[h_i \ c_{xi} \ c_{zi} \ \gamma_{xi} \ \gamma_{zi}], \ i=\overline{1,5}:$$

$$v_1=[1.5 \ -1 \ -1.5 \ 2 \ 2], \ v_2=[1 \ 0 \ 0 \ 1.4 \ 2], \ v_3=[1.2 \ 1.5 \ -0.5 \ 2 \ 0.7],$$

$$v_4=[1 \ -1.5 \ 1 \ 2 \ 1.2], \ v_5=[-0.7 \ 0.5 \ 1 \ 0.5 \ 1.2].$$

2. Поверхность четвёртого порядка, описываемую неявным уравнением

$$f(x,y,z)=36x^4-16y^4+36z^4-140x^2y^2+153x^2z^2-20y^2z^2-216x^3+864xy^2-864xz^2+144x^2-1168y^2+1116z^2+864x-1152=0.$$

Выполним факторизацию функции $f(x,y,z)=f_1(x,y,z) \cdot f_2(x,y,z)$:

$$f_1(x,y,z)=x^2-4y^2+4z^2-4, \ f_2(x,y,z)=36x^2+4y^2+9z^2-216x+288.$$

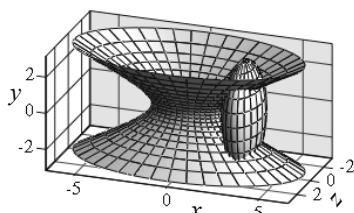


Рис. 4.81

Следовательно, искомая поверхность является объединением двух квадратичных поверхностей (рис. 4.81):

- $f_1(x,y,z)=4((x/2)^2-y^2+z^2-1)=0$ — однополостного гиперболоида с полуосями $a=2, b=c=1$;

- $f_2(x,y,z)=36((x-3)^2+(y/3)^2+(z/2)^2-1)=0$ — эллипсоида с полуосями $a=1, b=3, c=2$ и центром в точке $[3 \ 0 \ 0]$.



4.4. ПОЛИНОМИАЛЬНЫЕ ПОВЕРХНОСТИ

В процессе проектирования сложного изделия не всегда имеется полное аналитическое описание его поверхности, удовлетворяющее желаемым конструктивным и эстетическим требованиям. Чаще бывает известен лишь ограниченный объём данных, определяющих поверхность: отдельные опорные (узловые) точки и линии, направления касательных векторов и плоскостей в некоторых точках и т. п. Целью задачи построения приближающей поверхности является составление геометрической модели с двумя степенями свободы, непрерывно заполняющей промежутки между узловыми данными.

Неопределённость поверхности в межузловом пространстве и различный объём и характер исходных данных обуславливают многообразие методов приближения, отличающихся выбором базисных функций и формулами расчёта параметров разложений. Развивая методы однопараметрических приближений, изложенные в разд. 2.1, сосредоточим основное внимание на **полиномиальных** бипараметрических моделях поверхностей $p(t, \tau)$, использующих векторы базисных функций $T_n(t)$, $T_m(\tau)$ и их производные, определённые в (2.6).

Важным нежелательным свойством полиномиальной интерполяции является резкое усиление межузловых осцилляций с ростом числа узловых точек и степеней полиномов (см. рис. 2.5, 2.7, 2.8). Для уменьшения осцилляций приходится составлять сложную поверхность из **сегментов** (ячеек, клеток [27], порций [56], кусков [65], патчей [49]) невысоких порядков с индивидуальной полиномиальной моделью каждого сегмента. В качестве примеров таких составных поверхностей можно привести автомобильные кузова, фюзеляжи и крылья самолетов, корпуса кораблей, лопасти турбин и вентиляторов, скульптуры, фигуры людей и животных, оболочки с отверстиями и другие поверхности, отдельные части которых представляются в кусочном виде, подобно лоскутному одеялу. Сегментация также применяется при создании нерегулярных поверхностей с нарушениями непрерывности (разрывами) или гладкости (изломами).

Как правило, сегмент, определённый в параметрическом прямоугольнике $t \times \tau$ (см. рис. 4.6, б) имеет в объектном пространстве форму криволинейного и, в общем случае, неплоского **четырёхугольника** с двумя семействами изопараметрических каркасных линий $p(t_i, \tau)$ и $p(t, \tau_j)$, **трансверсальных** (сопряжённых) друг другу. Лишь при вырождении граничной линии в точку объектный сегмент становится треугольным, сохраняя топологические свойства четырёхугольника, в том числе и возможную непланарность.

Число методов полиномиальных приближений поверхностей велико и каждый из них работает с определённым составом исходных данных. Минимальный состав включает узловые элементы (точки и векторы), расположенные только на границах сегмента. Дополнительные требования к поверхности задаются межграничными узловыми элементами. Вовсе не факт, что увеличение объёма исходных данных повышает качество (субъективный фактор!) получаемого результата.

4.4.1. Интерполяция граничных элементов

Руководствуясь принципом «от простого к сложному», вначале рассмотрим алгоритмы конструирования сегментов поверхностей, заданных только граничными узловыми элементами — точками, линиями и угловыми векторами.

Билинейная интерполяция

Одной из простейших является **билинейная поверхность сегмента**, заданная **четырьмя узловыми точками** u_{00} , u_{01} , u_{10} и u_{11} (рис. 4.82). Такой минимальный объём исходных данных встречается, например, в машиностроительных САПР, измеряющих специальными щупами координаты отдельных точек поверхности сложной детали, по которым создаются кусочные модели сегментов.

Если к поверхности сегмента не предъявляются никаких особых требований, кроме прохождения через заданные узлы, то наиболее простым будет вычисление её произвольной точки как линейной интерполяции между противоположными прямыми $u_{00}u_{10}$ и $u_{01}u_{11}$. Зададим в параметрической плоскости единичный квадрат, четыре вершины которого $[t \ \tau]$ с $t \in \{0, 1\}$ и $\tau \in \{0, 1\}$ соответствуют узловым точкам объектного пространства u_{it} . Линейными интерполяциями пар граничных точек являются отрезки прямых

$$u_{t0}(t) = \text{lin}(u_{00}, u_{10}, t) = (1-t)u_{00} + tu_{10}, \quad u_{t1}(t) = \text{lin}(u_{01}, u_{11}, t) = (1-t)u_{01} + tu_{11}.$$

Зафиксировав значение первого параметра в интервале $0 \leq t \leq 1$, построим по (2.10а) отрезок $\text{lin}(u_{t0}(t), u_{t1}(t), \tau)$ линейной интерполяции граничных точек $u_{t0}(t)$ и $u_{t1}(t)$ по второму непрерывному параметру $0 \leq \tau \leq 1$:

$$p(t, \tau) = (1-\tau)u_{t0}(t) + \tau u_{t1}(t) = (1-t)(1-\tau)u_{00} + (1-t)\tau u_{10} + (1-t)\tau u_{01} + t\tau u_{11}. \quad (4.81a)$$

То же получится линейной интерполяцией $\text{lin}(u_{0\tau}(\tau), u_{1\tau}(\tau), t)$ граничных линий $u_{0\tau}(\tau) = \text{lin}(u_{00}, u_{01}, \tau)$ и $u_{1\tau}(\tau) = \text{lin}(u_{10}, u_{11}, \tau) \quad \forall \tau \in [0, 1]$ по параметру $t \in [0, 1]$:

$$p(t, \tau) = (1-t)u_{0\tau}(\tau) + t u_{1\tau}(\tau) = (1-t)(1-\tau)u_{00} + (1-t)\tau u_{01} + t(1-\tau)u_{10} + t\tau u_{11}.$$

Параметрическая модель (4.81a) может быть компактно записана в двух матричных формах

$$p(t, \tau) = \begin{bmatrix} 1-t & t \end{bmatrix} \begin{bmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{bmatrix} \begin{bmatrix} 1-\tau \\ \tau \end{bmatrix} = L_1^T(t) U L_1(\tau) = T_1^T(t) H^T U H T_1(\tau) \quad (4.81б)$$

разложений по векторным весовым функциям $L_1(t)$ и $L_1(\tau)$, устроенным по (2.16а), и базисным степенным функциям $T_1(t) = [1 \ t]^T$ и $T_1(\tau) = [1 \ \tau]^T$, определённым в (2.6а). В обеих формах используется **узловая матрица** $U = \{u_{ij}\} \quad \forall i, j \in \{0, 1\}$, а во второй — матрица H из (2.16а). Благодаря простоте и универсальности матричные формы широко применяются для описания разнообраз-

ных кусочных и составных поверхностей с использованием соответствующих векторов и матриц.

По классификации на рис. 4.2 билинейная поверхность является **косой плоскостью**, а согласно примеру 4.9, где изучалось движение прямой образующей по двум скрещивающимся прямым направляющим, она же является сегментом одной из квадратичных поверхностей — **гиперболического параболоида** (см. рис. 1.142, д). При некомпланарных отрезках $u_{00}u_{10}$ и $u_{01}u_{11}$ эта поверхность не плоская. С учётом того, что $L_1^T(t)=[-1 \ 1]$ и $L_1'(\tau)=[-1 \ 1]^T$, её направляющие векторы в произвольной точке $p(t, \tau)$ вычисляются по (1.93б):

$$V(t, \tau) = L_1^T(t)UL_1(\tau) = (1-\tau)(u_{10}-u_{00}) + \tau(u_{11}-u_{01}),$$

$$W(t, \tau) = L_1^T(t)UL_1'(\tau) = (1-t)(u_{01}-u_{00}) + t(u_{11}-u_{10}).$$

Тогда по определению (1.94) получаем билинейную интерполяцию нормали

$$N(t, \tau) = (1-t)(1-\tau)N_{00} + t(1-\tau)N_{10} + (1-t)\tau N_{01} + t\tau N_{11}, \quad (4.82a)$$

вычисляемую так же, как и точка $p(t, \tau)$, на основе нормалей в углах сегмента

$$N_{00} = (u_{10}-u_{00}) \times (u_{01}-u_{00}), \quad N_{01} = (u_{11}-u_{01}) \times (u_{01}-u_{00}),$$

$$N_{10} = (u_{10}-u_{00}) \times (u_{11}-u_{10}), \quad N_{11} = (u_{11}-u_{01}) \times (u_{11}-u_{10}).$$

При компланарности четвёрки узлов $u_{00} \div u_{11}$ все векторы $N_{00} \div N_{11}$ параллельны, а билинейная поверхность превращается в обычную плоскость с параллельными нормальными $N(t, \tau)$ во всех точках $p(t, \tau)$.

Также очевидно плоскую форму принимает билинейная поверхность, заданная **трем**я узлами a , b и c (рис. 4.83). Её просто получить из четырёхузловой модели (4.81) путём слияния двух смежных узлов в одной точке. Например, при $u_{00}=a$, $u_{10}=b$ и $u_{01}=u_{11}=c$ получим двухпараметрическую модель плоского **треугольного сегмента**

$$p(t, \tau) = (1-t)(1-\tau)a + t(1-\tau)b + \tau c$$

с направляющими векторами

$$V(t, \tau) = (1-\tau)(b-a), \quad W(t, \tau) = t(a-b) + c-a$$

и нормалью **постоянного** направления

$$N(t, \tau) = -t(1-\tau)(b-a) \times (b-a) + (1-\tau)(b-a) \times (c-a) = (1-\tau)N_a,$$

что и доказывает **планарность** поверхности, образованной трёхузловой билинейной интерполяцией.

Изображение криволинейной поверхности (4.81) в разных ракурсах может выглядеть как односторонним (см. рис. 4.68, а), так и двусторонним (см. рис. 4.68, в). Поэтому оно должно создаваться с помощью функций закраски, способных отображать обе стороны поверхности, например, по (4.1).

Почти всем методам закраски поверхностей, кроме использования фиксированного цвета $k_{\Phi}e \bullet I_{\Phi}$, нужен вектор нормали $N(t, \tau)$. Благодаря изменчивости его направления в разных точках $p(t, \tau)$ относительно источников света и на-

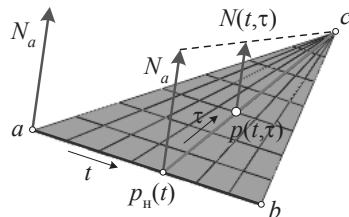


Рис. 4.83

блюдателя поверхность выглядит рельефно и не однотонно. Лишь полиэдральный объект, ограниченный плоскими гранями, кажется наблюдателю сшитым из монохромных кусков (см. рис. 4.5,з). Интерполяционный подход позволяет ускорить визуализацию криволинейной поверхности и решить **задачу цветового сглаживания** — устранения скачков цвета на межсегментных границах.

Наиболее популярны два метода билинейного сглаживания цвета точки $p(t, \tau)$, использующие вычисленные каким-либо образом векторы нормалей $N_{00} \div N_{11}$ в углах четырехугольной ячейки $u_{00} \div u_{11}$:

- по **методу Гурб** сначала по (4.1) вычисляются вершинные цвета $c_{00} \div c_{11}$, после чего аналогично (4.81) производится билинейная интерполяция цвета:

$$c(t, \tau) = (1-t)(1-\tau)c_{00} + t(1-\tau)c_{10} + (1-t)\tau c_{01} + t\tau c_{11} = L_1^T(t)CL_1(\tau); \quad (4.82б)$$

- по **методу Фонга** сначала по (4.82а) находится интерполированная нормаль $N(t, \tau)$, для которой функция закраски возвращает RGB-вектор цвета $c(t, \tau)$.

Интерполяция по Фонгу хотя и более трудоёмка из-за многократного вызова функции закраски, но даёт лучшее, более рельефное качество цветового сглаживания. Метод Гурб, напротив, менее ярок и рельефен, особенно при визуализации ячеек с почти параллельными нормальными в угловых точках.

Гранично-линейная интерполяция

Далее рассмотрим задачу построения **гранично-линейной поверхности сегмента**, заданной **двумя граничными линиями** $g(t)$ и $h(t)$ (рис. 4.84), общий параметр которых изменяется в **едином** и не обязательно единичном интервале

$t \in [a, b]$. Здесь требуется большой объём исходных данных: аналитические модели граничных линий или возможность их получения методами полиномиальной или сплайновой интерполяции, описанными в разд. 2.1, 2.3.

Поскольку поведение сегмента между граничными линиями не регламентировано, то для вычисления его внутренней точки примем линейную интерполяцию по параметру $\tau \in [0, 1]$ между граничными точками $g(t)$ и $h(t)$ с равными значениями параметра t :

$$p(t, \tau) = (1-\tau)g(t) + \tau h(t) = [g(t) \ h(t)]L_1(\tau). \quad (4.83а)$$

По классификации на рис. 4.2 это модель линейчатой поверхности с двумя направляющими, аналогичная (4.39), в которой в качестве дополнительного условия выступает общий параметр t согласованного движения отрезка образующей по граничным линиям. Каркасные линии $p(t_i, \tau)$ являются отрезками прямых, а линии $p(t, \tau_j)$ образуют систему **полиморфных линий**, гладко переходящих из начальной формы $p(t, 0) = g(t)$ в конечную форму $p(t, 1) = h(t)$.

При вырождении одной из граничных линий в точку $g(t) = g$ либо $h(t) = h$ получим модель **треугольного сегмента** с одной криволинейной границей:

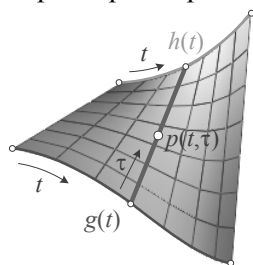


Рис. 4.84

$$p(t, \tau) = (1 - \tau)g + \tau h(t) \quad \text{либо} \quad p(t, \tau) = (1 - \tau)g(t) + \tau h. \quad (4.83б)$$

Если граничные линии заданы в разных параметрических интервалах, то их нужно привести в единую, например, **единичную** шкалу $t \in [0, 1]$. Допустим, некоторый граничный участок интерполируемого сегмента находится на линии $f(\theta)$ в **произвольном** интервале параметра $\theta \in [a, b]$ с любыми числами $a \neq b$. Взаимные преобразования произвольной $f(\theta)$ и нормированной $\phi(t)$ моделей линий производятся по следующим формулам:

$$\phi(t) = f(a + (b - a)t), \quad f(\theta) = \phi((\theta - a)/(b - a)) \quad \forall t \in [0, 1], \theta \in [a, b]. \quad (4.84)$$

Пример 4.19. Разработать гранично-линейную модель поверхности шторы шириной s и высотой d , подвешенной на прямолинейной струне с числом складок n и их нижней амплитудой a (см. рис. 4.85).

Решение. Задав нижнюю $g(t)$ и верхнюю $h(t)$ направляющие линии в единичной шкале $t \in [0, 1]$ как

$$g(t) = [st \ 0 \ a \sin(2\pi n t)], \quad h(t) = [st \ d \ 0],$$

по (4.83а) получим параметрическую модель шторы

$$p(t, \tau) = [st \ d\tau \ a \sin(2\pi n \tau)(1 - \tau)], \quad t \in [0, 1], \tau \in [0, 1].$$

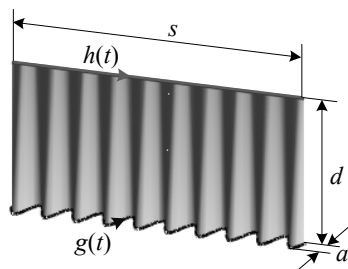


Рис. 4.85

Граничная интерполяция

Следующим линейно-интерполированным объектом является **граничная поверхность сегмента**, называемая в многочисленных источниках **линейной поверхностью Кунса** [20, 26, 27, 51, 56]. Она задаётся **четырьмя граничными линиями** $g(t)$, $h(t)$, $q(\tau)$ и $r(\tau)$, образующими объёмный четырёхугольник с вершинами u_{00} , u_{10} , u_{01} и u_{11} (рис. 4.86). Все граничные линии сегмента должны иметь параметрическое описание в одинаковых, например, единичных интервалах $t \in [0, 1]$ и $\tau \in [0, 1]$, иначе их нужно подвергнуть нормировке по (4.84).

Выберем следующие **краевые условия** фиксации параметров t и τ :

- $t=0$ на линии $q(\tau)$ с концевыми точками $q(0)=u_{00}$ и $q(1)=u_{01}$;
- $t=1$ на линии $r(\tau)$ с концами $r(0)=u_{10}$ и $r(1)=u_{11}$;
- $\tau=0$ на линии $g(t)$ с концевыми точками $g(0)=u_{00}$ и $g(1)=u_{10}$;
- $\tau=1$ на линии $h(t)$ с концами $h(0)=u_{01}$ и $h(1)=u_{11}$.

В кинематическом аспекте гранично определённая поверхность создаётся движением образующей линии переменной формы по двум направляющим. Её изопараметрические линии образуют две системы полиморфных линий:

- линии $p(t_i, \tau)$ переходят от $q(\tau)$ к $r(\tau)$, опираясь на точки $g(t_i)$ и $h(t_i)$;
- линии $p(t, \tau_j)$ движутся от $g(t)$ к $h(t)$, опираясь на точки $q(\tau_j)$ и $r(\tau_j)$.

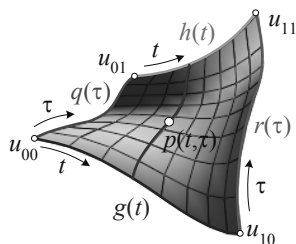


Рис. 4.86

Если для обеспечения двойного полиморфизма составить сумму векторных функций (4.83а) между парами граничных линий $\{g(t), h(t)\}$ и $\{q(\tau), r(\tau)\}$, то в ней будет содержаться билинейная поверхность (4.81а). Поэтому корректная модель гранично определённого сегмента Кунса исключает это излишество:

$$p(t, \tau) = (1-\tau)g(t) + \tau h(t) + (1-t)q(\tau) + tr(\tau) - \\ - (1-t)(1-\tau)u_{00} - t(1-\tau)u_{10} - (1-t)\tau u_{01} - t\tau u_{11}. \quad (4.85а)$$

Это разложение также можно записать в матричной форме:

$$p(t, \tau) = \begin{bmatrix} 1-t & t & 1 \end{bmatrix} \begin{bmatrix} -u_{00} & -u_{01} & q(\tau) \\ -u_{10} & -u_{11} & r(\tau) \\ g(t) & h(t) & 0 \end{bmatrix} \begin{bmatrix} 1-\tau \\ \tau \\ 1 \end{bmatrix} \quad \forall t, \tau \in [0, 1]. \quad (4.85б)$$

Убедимся в правильности интерполяции в узловых точках и на границах:

$$p(0, 0) = g(0) + q(0) - u_{00} = u_{00}, \\ p(t, 0) = g(t) + (1-t)q(0) + tr(0) - (1-t)u_{00} - tu_{10} = g(t).$$

При вырождении одной из граничных линий, например, $h(t)$, в точку $u_{01} = u_{11} = h$ получим модель **криволинейного треугольного сегмента**:

$$p(t, \tau) = (1-\tau)g(t) + (1-t)q(\tau) + tr(\tau) - (1-t)(1-\tau)u_{00} - t(1-\tau)u_{10} = \\ = \begin{bmatrix} 1-t & t & 1 \end{bmatrix} \begin{bmatrix} -u_{00} & -h & q(\tau) \\ -u_{10} & -h & r(\tau) \\ g(t) & h & 0 \end{bmatrix} \begin{bmatrix} 1-\tau \\ \tau \\ 1 \end{bmatrix}. \quad (4.85в)$$

Такие сегменты образуются у полюсов поверхностей вращения, при скруглении трёхгранных углов и вообще в задачах с не плоскими треугольниками.

Если граничные линии определены в произвольных параметрических интервалах $t \in [a, b]$ и $\tau \in [\alpha, \beta]$, то можно обойтись без их нормировки по (4.84), применив по каждому параметру линейную интерполяцию (2.10б):

$$p(t, \tau) = \begin{bmatrix} \frac{b-t}{b-a} & \frac{t-a}{b-a} & 1 \end{bmatrix} \begin{bmatrix} -u_{00} & -u_{01} & q(\tau) \\ -u_{10} & -u_{11} & r(\tau) \\ g(t) & h(t) & 0 \end{bmatrix} \begin{bmatrix} (\beta-\tau)/(\beta-\alpha) \\ (\tau-\alpha)/(\beta-\alpha) \\ 1 \end{bmatrix}. \quad (4.85г)$$

Обобщением линейной интерполяции граничных элементов является использование в (4.85) произвольных весовых функций l_0 и l_1 аргументов t и τ :

$$p(t, \tau) = \begin{bmatrix} l_0(t) & l_1(t) & 1 \end{bmatrix} \begin{bmatrix} -u_{00} & -u_{01} & q(\tau) \\ -u_{10} & -u_{11} & r(\tau) \\ g(t) & h(t) & 0 \end{bmatrix} \begin{bmatrix} l_0(\tau) \\ l_1(\tau) \\ 1 \end{bmatrix} \quad \forall t \in [t_0, t_1], \tau \in [\tau_0, \tau_1]. \quad (4.86а)$$

Они должны создать у поверхности сегмента, во-первых, граничные значения $p(t_i, \tau_j) = u_{ij} \quad \forall i, j \in \{0, 1\}$, $p(t, \tau_0) = g(t)$, $p(t, \tau_1) = h(t)$, $p(t_0, \tau) = q(\tau)$ и $p(t_1, \tau) = r(\tau)$; во-вторых, непрерывность и гладкость хотя бы первого порядка; в-третьих, отсутствие складок и принадлежность выпуклой оболочке граничных линий. Всем этим требованиям удовлетворяют монотонные весовые функции со следующими свойствами:

$$l_0(\theta_0) = l_1(\theta_1) = 1, \quad l_0(\theta_1) = l_1(\theta_0) = 0, \quad l_0(\theta) + l_1(\theta) = 1 \quad \forall \theta \in \{t, \tau\}. \quad (4.86б)$$

Пример 4.20. Построить линейную поверхность Кунса, приближающую первый октант ($x, y, z \geq 0$) сферы единичного радиуса (рис. 4.87).

Решение. Выберем граничные линии октанта в интервалах $t \in [0, 1]$, $\tau \in [0, 1]$:

- нижнюю $g(t) = [\sin(\pi t/2) \ 0 \ \cos(\pi t/2)]$ на экваторе;
- верхнюю $h(t) = [0 \ 1 \ 0]$ в точке полюса;
- левую $q(\tau) = [0 \ \sin(\pi \tau/2) \ \cos(\pi \tau/2)]$ на меридиане;
- правую $r(\tau) = [\cos(\pi \tau/2) \ \sin(\pi \tau/2) \ 0]$ на меридиане.

Подставив эти выражения в (4.85в), получим координатные функции кусочной поверхности $p(t, \tau) = [x(t, \tau) \ y(t, \tau) \ z(t, \tau)]$:

$$x(t, \tau) = t \cos(\pi \tau/2) + (1-t)(\sin(\pi t/2) - t).$$

$$y(t, \tau) = \sin(\pi \tau/2),$$

$$z(t, \tau) = (1-t)\cos(\pi \tau/2) + (1-t)(\cos(\pi t/2) + t - 1).$$

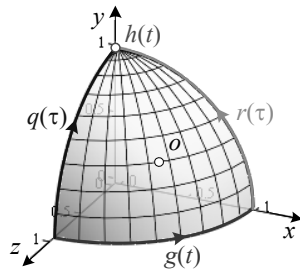


Рис. 4.87

Поскольку $x^2(t, \tau) + y^2(t, \tau) + z^2(t, \tau) \neq 1$, то эта поверхность не является частью единичной сферы. Наибольшее отклонение $\max \|p(t, \tau) - 1\| \approx 0.05$ имеет место в точке $o = p(1/2, 1/3)$. $\square$

Пример 4.21. Составить модель поверхности, заполняющей криволинейный четырёхугольник между линиями, по которым гиперболический параболоид $y = x^2 - z^2$ пересекается с плоскостями $x = \pm 1$ и $z = \pm 1$ (рис. 4.88, а).

Решение. Получим точки пересечения квадратичной поверхности $x^2 - y - z^2 = 0$ с парами смежных плоскостей $u_{00} = [-1 \ 0 \ 1]$, $u_{10} = [1 \ 0 \ 1]$, $u_{01} = [-1 \ 0 \ -1]$, $u_{11} = [1 \ 0 \ -1]$ и модели линий пересечения поверхности с плоскостями между узловыми точками:

- при $x = \pm 1$ это две параболы $y = 1 - z^2$, описываемые параметрическими моделями $r(\tau) = [1 \ 1 - \tau^2 \ -\tau]$ и $q(\tau) = [-1 \ 1 - \tau^2 \ -\tau]$ в интервале $\tau \in [-1, 1]$, откуда $\alpha = -1$, $\beta = 1$;
- при $z = \pm 1$ линиями пересечения являются параболы $y = x^2 - 1$ с параметрическими моделями $g(t) = [t \ t^2 - 1 \ 1]$ и $h(t) = [t \ t^2 - 1 \ -1]$ при $t \in [-1, 1]$, откуда $a = -1$, $b = 1$.

Подставив полученные данные в (4.85г), после упрощений получаем параметрическую модель линейного сегмента Кунса $p(t, \tau) = [t \ t^2 - \tau^2 \ -\tau]$ с координатными функциями $x = t$, $y = t^2 - \tau^2$ и $z = -\tau$, удовлетворяющими неявному уравнению гиперболического параболоида $x^2 - y - z^2 = 0$. График этой поверхности построен на рис. 4.88, б. $\square$

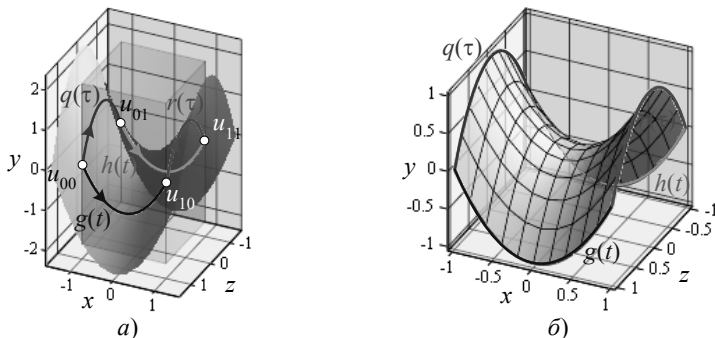


Рис. 4.88

Сложная поверхность, состоящая из множества сегментов, конструируется как объединение кусочных поверхностей, сшитых друг с другом по граничным

линиям. Общий **недостаток** методов кусочной интерполяции — негладкость составной поверхности в точках и на линиях стыковки сегментов, поскольку их модели строятся индивидуально без учёта требований к гладкости сопряжения.

Испытаем алгоритмы (4.81), (4.83), (4.85) на примере построения составной поверхности четырёхкорневого зуба. Зададим в пространстве семнадцать узловых точек, образующих 2×8 -каркас из шестнадцати ячеек (рис. 4.89,а):

- выглядящая угловатой поверхность на рис. 4.89,б составлена из **билинейных сегментов** — восьми треугольников и восьми косых четырёхугольников с прямолинейными границами и очевидной негладкостью сопряжения;
- продольные граничные линии сегментов **гранично-линейной поверхности**, построенной на рис. 4.89,в, рассчитаны как параболы трёхузловой полиномиальной интерполяции и создают гладкость сопряжения сегментов в направлении первого параметра. Прямолинейность граничных отрезков в направлении второго параметра делает порядок гладкости сопряжения нулевым;
- наиболее эстетично выглядит изображение с **граничными поверхностями** сегментов (рис. 4.89,г), у которых все граничные линии являются фрагментами сквозных интерполяционных полиномов, благодаря чему создаётся гладкость сопряжения в направлениях обоих параметров.

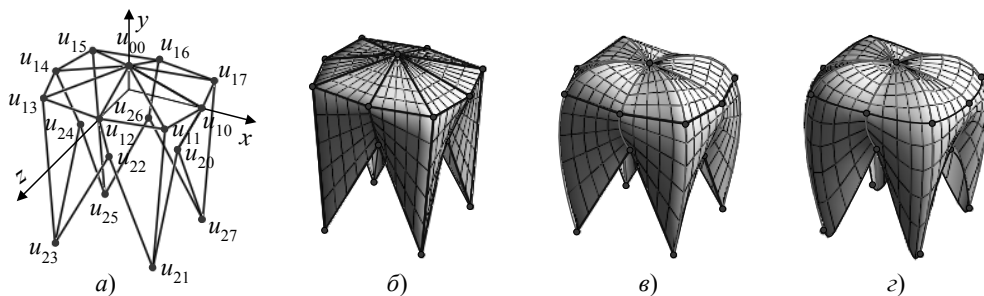


Рис. 4.89

Эрмитова граничная интерполяция

Добавим к четырём угловым точкам u_{00} , u_{10} , u_{01} и u_{11} дополнительные граничные элементы, определяющие поверхность сегмента $p(t, \tau)$ в единичных параметрических интервалах $t \in [0, 1]$ и $\tau \in [0, 1]$, — четыре пары векторов $V_{ij} = p'_t(i, j)$ и $W_{ij} = p'_\tau(i, j)$, задающих желаемые направления краёв поверхности в её углах u_{ij} , где $i, j \in \{0, 1\}$ (рис. 4.90,а). Традиционно объекты, создаваемые на основе производных их формы, называют эрмитовыми. Рассмотрим два способа построения эрмитовой поверхности сегмента.

1. Поскольку на каждом краю ячейки задано четыре граничных элемента (по две точки и два вектора первых параметрических производных поверхности в этих точках), то по ним можно восстановить четыре граничных линии:

- $g(t) = [u_{00} \ u_{10} \ V_{00} \ V_{10}] L_3(t)$ между точками $u_{00} = g(0)$ и $u_{10} = g(1)$ с касательными векторами $V_{00} = g'(0)$ и $V_{10} = g'(1)$;

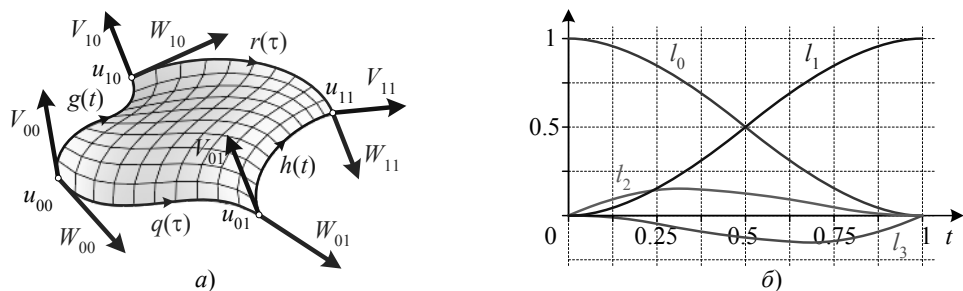


Рис. 4.90

- $h(t)=[u_{01} \ u_{11} \ V_{01} \ V_{11}]L_3(t)$ между точками $u_{01}=h(0)$ и $u_{11}=h(1)$ с касательными векторами $V_{01}=h'(0)$ и $V_{11}=h'(1)$;
- $q(\tau)=[u_{00} \ u_{01} \ W_{00} \ W_{01}]L_3(\tau)$ между точками $u_{00}=q(0)$ и $u_{01}=q(1)$ с касательными векторами $W_{00}=q'(0)$ и $W_{10}=q'(1)$;
- $r(\tau)=[u_{10} \ u_{11} \ W_{10} \ W_{11}]L_3(\tau)$ между точками $u_{10}=r(0)$ и $u_{11}=r(1)$ с касательными векторами $W_{10}=r'(0)$ и $W_{11}=r'(1)$.

Каждая из этих линий однозначно описывается кубическим полиномом Эрмита (2.28) с приведённым в (2.27г) вектором $L_3(t)$ (аналогично устроен и вектор $L_3(\tau)$), элементами которого служат весовые функции

$$l_0(t)=1-3t^2+2t^3, \quad l_1(t)=3t^2-2t^3, \quad l_2(t)=t-2t^2+t^3, \quad l_3(t)=-t^2+t^3 \quad (4.87a)$$

с графиками на рис. 4.90,б. Используем первые две ВФ, удовлетворяющие условиям (4.86б), для создания модели сегмента (4.86а). Выбор кубических функций смещения позволяет согласовать их с кубическим типом граничных линий сегмента и оптимизировать плотность изолиний каркасной сетки.

После подстановки моделей граничных линий и приведения подобных членов модель сегмента (4.86а) представляется взвешенной суммой двенадцати граничных элементов u_{ij} , V_{ij} и $W_{ij} \forall i, j \in \{0, 1\}$. Точки u_{ij} входят в сумму с весами $l_i(t)l_j(\tau)$, а векторы V_{ij} и W_{ij} — с весами $l_{i+2}(t)l_j(\tau)$ и $l_i(t)l_{j+2}(\tau)$. В матричной форме **бикубический сегмент Эрмита** компактно представляется в виде следующего разложения по граничным элементам:

$$p(t, \tau) = [l_0(t) \ l_1(t) \ l_2(t) \ l_3(t)] \begin{bmatrix} u_{00} & u_{01} & W_{00} & W_{01} \\ u_{10} & u_{11} & W_{10} & W_{11} \\ V_{00} & V_{01} & O & O \\ V_{10} & V_{11} & O & O \end{bmatrix} \begin{bmatrix} l_0(\tau) \\ l_1(\tau) \\ l_2(\tau) \\ l_3(\tau) \end{bmatrix} = \\ = L_3^T(t) U_3 L_3(\tau) \forall t, \tau \in [0, 1]. \quad (4.87б)$$

2. Эрмитова поверхность сегмента может быть создана и без восстановления его граничных линий. Поскольку в каждом параметрическом направлении края поверхности определены четырьмя граничными элементами, то полиномиальная модель сегмента может быть описана бикубическим полиномом

$$p(t, \tau) = \sum_{i=0}^3 \sum_{j=0}^3 s_{ij} t^i \tau^j = T_3^T(t) S T_3(\tau) \quad \forall t, \tau \in [0, 1] \quad (4.88a)$$

в виде разложения по базисным функциям, составляющим по (2.6а) векторы $T_3^T(t)=[1 \ t \ t^2 \ t^3]$ и $T_3(\tau)=[1 \ \tau \ \tau^2 \ \tau^3]^T$. Совокупность **векторных** коэффициентов $s_{ij} \in \mathbb{R}^3$ образует матрицу $S \in \mathbb{V}^{4 \times 4}$, подлежащую определению.

Поочерёдно дифференцируя (4.88а) по параметрам t и τ , получим формулы вычисления касательных векторов бикубической поверхности

$$p'_t(t, \tau) = T_3'^T(t) S T_3(\tau), \quad p'_\tau(t, \tau) = T_3^T(t) S T_3'(\tau), \quad (4.88б)$$

использующие векторы производных базисных функций $T_3'(t)=[0 \ 1 \ 2t \ 3t^2]$ и $T_3'(\tau)=[0 \ 1 \ 2\tau \ 3\tau^2]$, определённые в (2.6б).

Составим из условий фиксации угловых значений функций (4.88) систему уравнений, необходимых для вычисления матрицы S :

$$T_3^T(i) S T_3(j) = u_{ij}, \quad T_3'^T(i) S T_3(j) = V_{ij}, \quad T_3^T(i) S T_3'(j) = W_{ij} \quad \forall i, j = \overline{0, 1}. \quad (4.89а)$$

Все эти уравнения успешно укладываются в матричную форму

$$\begin{bmatrix} T_3^T(0) \\ T_3^T(1) \\ T_3'^T(0) \\ T_3'^T(1) \end{bmatrix} S \underbrace{\begin{bmatrix} T_3(0) & T_3(1) & T_3'(0) & T_3'(1) \end{bmatrix}}_{Q_3} = \begin{bmatrix} u_{00} & u_{01} & W_{00} & W_{01} \\ u_{10} & u_{11} & W_{10} & W_{11} \\ V_{00} & V_{01} & O & O \\ V_{10} & V_{11} & O & O \end{bmatrix} \text{ или } Q_3^T S Q_3 = U_3. \quad (4.89б)$$

Численный вид матрицы Q_3 приведен в (2.27г). В силу её невырожденности (матрица $H_3 = Q_3^{-1}$ дана там же) матричное решение системы имеет вид

$$S = H_3^T U_3 H_3. \quad (4.89в)$$

Благодаря аналогичной (2.8) зависимости $L_3(t) = H_3 T_3(t)$ две модели бикубического полинома Эрмита (4.87б) и (4.88а) эквивалентны. В интерактивном проектировании удобнее, однако, первая модель, позволяющая немедленно изобразить поверхность сегмента после любого изменения граничного элемента без пересчёта по (4.89в) матрицы её коэффициентов.

Пример 4.22. Построить бикубическую эрмитову поверхность, заданную узловыми точками $u_{00}=[-1 \ 0 \ -1]$, $u_{10}=[-1 \ 0 \ 1]$, $u_{01}=[1 \ 0 \ -1]$, $u_{11}=[1 \ 0 \ 1]$ и касательными векторами $V_{00}=V_{01}=W_{00}=W_{10}=\gamma[0 \ 1 \ 0]$, $V_{10}=V_{11}=W_{01}=W_{11}=\gamma[0 \ -1 \ 0]$ с переменным коэффициентом удлинения γ (рис. 4.91,а).

Решение. Подставив исходные данные в (4.87б), получим очень простые координатные функции бикубической эрмитовой поверхности $p(t, \tau)=[x(t, \tau) \ y(t, \tau) \ z(t, \tau)]$:

$$x(t, \tau) = -1 + 6t^2 - 4\tau^3, \quad y(t, \tau) = \gamma(t - t^2 + \tau - \tau^2), \quad z(t, \tau) = -1 + 6t^2 - 4t^3, \quad t \in [0, 1], \quad \tau \in [0, 1].$$

Видно, что коэффициент удлинения γ воздействует только на ординаты точек поверхности так, что при $\gamma=0$ сегмент становится плоским квадратом с неравномерной (бикубической) сеткой каркасных линий (рис. 4.91,а). Увеличение значения γ поднимает поверхность сегмента по параболической зависимости от краёв к центру (рис. 4.91,б).

Граничные линии сегмента $g(t) = p(t, 0)$, $h(t) = p(t, 1)$, $q(\tau) = p(0, \tau)$ и $r(\tau) = p(1, \tau)$ описываются моделями типа (2.29б), так что при выборе коэффициента удлинения $\gamma=4$ они принимают форму, наиболее близкую к половине дуги окружности (см. рис. 2.15,б). Соответст-

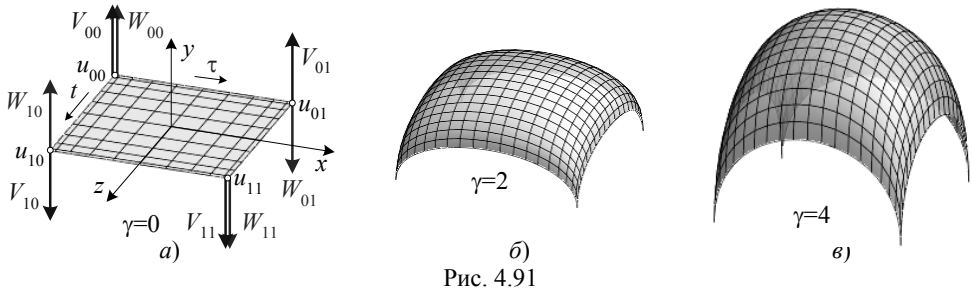


Рис. 4.91

вующая поверхность построена на рис. 4.91, в и вполне может представлять практический интерес дизайнерам различных архитектурных форм. $\square$

Бикубические поверхности Кунса

Побочным эффектом эрмитовой интерполяции (4.88) является получение в углах сегмента нулевых векторов смешанной производной $p''_{\tau}(i, j) = O_3$. Виною тому нулевые блоки узловой матрицы U_3 в (4.89б). Вычисляемый по формуле

$$p''_{\tau}(t, \tau) = T_3''(t) S T_3''(\tau) \quad (4.90a)$$

вектор кручения [51, 56] характеризует искривлённость изолиний бикубической поверхности в точке $p(t, \tau)$. Векторы угловых кручений

$$K_{ij} = T_3'^T(i) S T_3'(j) \quad \forall i, j \in \{0, 1\} \quad (4.90б)$$

управляют изгибами поверхности сегмента в точках u_{ij} . В линейных приращениях параметров поверхность $\varepsilon_t \times \varepsilon_\tau$ -окрестности (i, j) -й угловой точки

$$p(i + \varepsilon_t, j + \varepsilon_\tau) \approx u_{ij} + \varepsilon_t V_{ij} + \varepsilon_\tau W_{ij} + \varepsilon_t \varepsilon_\tau K_{ij} \quad (4.91)$$

отклоняется от касательной плоскости $\{u_{ij}, V_{ij}, W_{ij}\}$ в сторону вектора $\varepsilon_t \varepsilon_\tau K_{ij}$. В отсутствие кручения окрестная поверхность совпадает с касательной плоскостью.

Добавив к исходным данным эрмитовой интерполяции четыре вектора $K_{00} \div K_{11}$, получаем задачу синтеза поверхности **бикубического сегмента Кунса** (рис. 4.92), конструируемой аналогично (4.87)–(4.89), но с узловой матрицей U_K , в которой нулевые блоки заменены новыми граничными элементами:

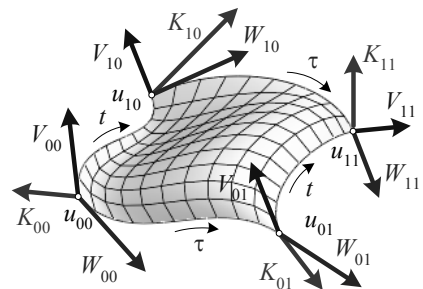


Рис. 4.92

$$p(t, \tau) = [l_0(t) \ l_1(t) \ l_2(t) \ l_3(t)] \underbrace{\begin{bmatrix} u_{00} & u_{01} & W_{00} & W_{01} \\ u_{10} & u_{11} & W_{10} & W_{11} \\ V_{00} & V_{01} & K_{00} & K_{01} \\ V_{10} & V_{11} & K_{10} & K_{11} \end{bmatrix}}_{U_K} \begin{bmatrix} l_0(\tau) \\ l_1(\tau) \\ l_2(\tau) \\ l_3(\tau) \end{bmatrix} = \quad (4.92)$$

$$= L_3^T(t) U_K L_3(\tau) = T_3^T(t) H_3^T U_K H_3 T_3(\tau) \quad \forall t, \tau \in [0, 1]$$

Произведение $H_{\mathcal{O}}^T U_K H_{\mathcal{O}}$ составляет матрицу S коэффициентов разложения.

Проанализируем **влияние угловых кручений на форму сегментов**, построенных по (4.92) на квадратной сетке с **компланарными** узлами $u_{00} \div u_{11}$ и разностной аппроксимацией векторов угловых направлений

$$V_{ij} = \gamma(u_{10} - u_{00}), \quad W_{ij} = \lambda(u_{11} - u_{10}) \quad \forall i, j \in \{0, 1\}$$

с равными коэффициентами удлинения $\gamma = \lambda = 0.5$ (рис. 4.93):

- у поверхности (а) угловые кручения отсутствуют и благодаря компланарности узлов она также получилась плоской. Попутно отметим **неравномерность изолиний эрмитовой поверхности** при задании коэффициентов удлинения $\gamma \neq 1$ или $\lambda \neq 1$;
- задание ненулевого кручения лишь в одном узле u_{ij} искривляет примыкающую к нему $\varepsilon_i \times \varepsilon_j$ -окрестность сегмента в сторону вектора $\varepsilon_i \varepsilon_j K_{ij}$. Для сравнения величин изгиба построены два сегмента (б) и (в) с длинами векторов $|K_{11}|$, отличающимися в 4 раза;
- фиксация кручений во всех четырёх узлах позволяет создать кусочную поверхность (з) весьма сложной формы. Разнообразие получаемых решений достигается интерактивной настройкой длин и направлений векторов $K_{00} \div K_{11}$.

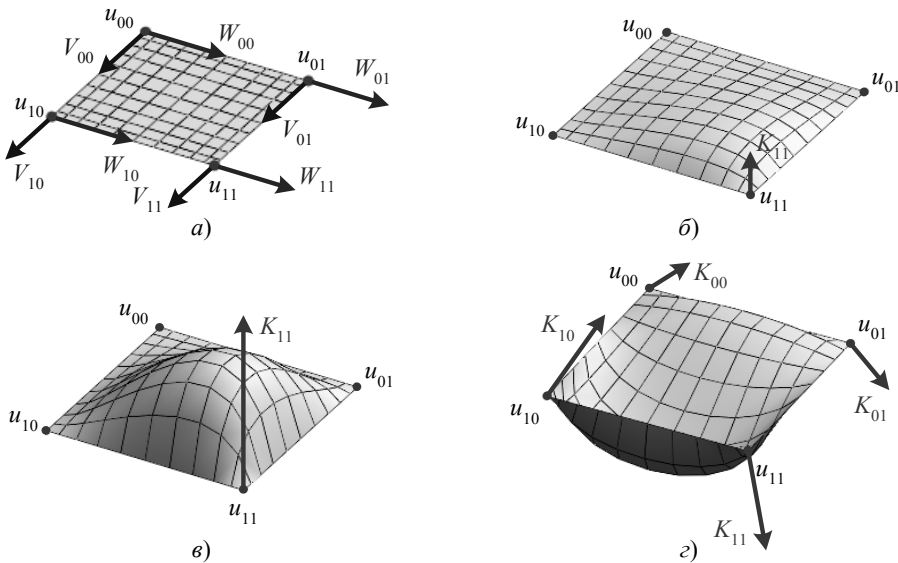


Рис. 4.93

Таким образом, бикубическая модель Кунса обладает в отличие от модели Эрмита дополнительным мощным инструментом формообразования в виде векторов угловых кручений. Серьёзным **недостатком** методов синтеза интерполяционных поверхностей с фиксированными краями является сложность подготовки исходных данных векторного типа — узловых направлений V_{ij} , W_{ij} и особенно кручений K_{ij} , геометрический смысл которых не столь очевиден, как у векторов касательных. Кроме того, загромождение графика поверхности допол-

нительными изображениями векторов лишь ухудшает качество её восприятия, а разнокалиберность графических элементов не позволяет показать векторы кручений и сегмент поверхности в едином масштабе. По этой причине все векторы K_{ij} на рис. 4.93 показаны условно в двадцать раз короче реальных.

4.4.2. Интерполяция узловых элементов

Распространим теорию однопараметрической интерполяции, описанную в п. 2.1.1, на задачу конструирования *полиномиальной поверхности сегмента* $p(t, \tau)$, заданной системой узловых элементов, расположенных не только на границе сегмента, как в п. 4.4.1, но и в межграничном пространстве.

Интерполяция узловой сети

Первой рассмотрим задачу построения поверхности, проходящей через систему точек $\{u_{ij}\} \forall i=\overline{0, n}, j=\overline{0, m}$, соединённых в двухмерную *узловую сеть* (рис. 4.94). Очевидно, что четырёхугольная ячейка на рис. 4.82 является частным случаем узловой сети при $n=m=1$. Соответственно, модель билинейной поверхности (4.81) полностью вписывается в решение общей задачи интерполяции.

Составим из вершин сети узловую матрицу $U \in V^{(n+1) \times (m+1)}$. Программно она организуется трёхмерным массивом с координатными слоями U_x, U_y и U_z . Как и в (2.7), возможны разные формы разложения двухпараметрического полинома:

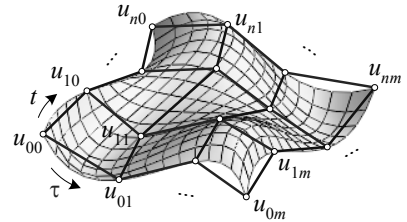


Рис. 4.94

- *по вершинам узловой сети* в форме Лагранжа

$$p(t, \tau) = \sum_{i=0}^n \sum_{j=0}^m \Lambda_{ij}(t, \tau) u_{ij} = L_n^T(t) U L_m(\tau) \quad \forall t \in [t_0, t_n], \tau \in [\tau_0, \tau_m] \quad (4.93a)$$

с векторами $L_n^T(t) = [l_{n0}(t) \dots l_{nn}(t)]$ и $L_m(\tau) = [l_{m0}(\tau) \dots l_{mm}(\tau)]^T$ весовых функций Лагранжа, декартовы произведения которых $\Lambda_{ij}(t, \tau) = l_{ni}(t) l_{mj}(\tau)$ в силу свойств (2.11б) характеризуют вклад узлов u_{ij} в суммарное значение $p(t, \tau)$;

- *по базисным функциям параметров* в форме Фергюсона [27, 56]

$$p(t, \tau) = \sum_{i=0}^n \sum_{j=0}^m s_{ij} t^i \tau^j = T_n^T(t) S T_m(\tau) \quad \forall t \in [t_0, t_n], \tau \in [\tau_0, \tau_m] \quad (4.93б)$$

с векторами базисных функций $T_n(t)$ и $T_m(\tau)$, составленными по определению (2.6а). Ядром разложения (4.93б) служит *матрица* $S \in V^{(n+1) \times (m+1)}$ *полиномиальных коэффициентов* (МПК) s_{ij} , размерность которых равна трём. Эта матрица может быть организована аналогично узловой матрице U в виде трёхмерной структуры с координатными слоями S_x, S_y и S_z ;

- **бинарное** разложение по вершинам узловой сети и базисным функциям

$$p(t, \tau) = T_n^T(t) H_n^T U H_m T_m(\tau) \quad \forall t \in [t_0, t_n], \tau \in [\tau_0, \tau_m] \quad (4.93b)$$

с числовыми матрицами $H_n \in \mathbb{R}^{(n+1) \times (n+1)}$ и $H_m \in \mathbb{R}^{(m+1) \times (m+1)}$, связанными с элементами других разложений соотношениями (2.8):

$$L_n(t) = H_n T_n(t), \quad L_m(\tau) = H_m T_m(\tau), \quad S = H_n^T U H_m. \quad (4.94)$$

Поверхность ограничена четырьмя линиями $p(t, \tau_0)$, $p(t, \tau_m)$, $p(t_0, \tau)$ и $p(t_n, \tau)$. В отличие от любой внутренней точки $p(t, \tau)$, зависящей от всех узлов, положение граничной точки определяется лишь их частью. К примеру, линия

$$p(t_0, \tau) = [1 \ 0 \ \dots \ 0] U L_m(\tau) = [u_{00} \ u_{01} \ \dots \ u_{0m}] L_m(\tau) \quad (4.95)$$

определена граничными узлами $u_{00} \div u_{0m}$. Сравнивая полученное выражение с (2.7a), делаем **вывод: каждая граничная линия является интерполяционной линией, построенной на соответствующих ей граничных узлах.**

Приступая к интерполяции, составим из условий прохождения поверхности со свободными краями через узловые точки $p(t_i, \tau_j) = u_{ij}$ систему уравнений

$$T_n^T(t_i) S T_m(\tau_j) = u_{ij} \quad \forall i = \overline{0, n}, \quad j = \overline{0, m} \quad (4.96a)$$

и запишем её в матричной форме

$$\begin{bmatrix} T_n^T(t_0) \\ \vdots \\ T_n^T(t_n) \end{bmatrix} \begin{bmatrix} s_{00} & \dots & s_{0m} \\ \vdots & \ddots & \vdots \\ s_{n0} & \dots & s_{nm} \end{bmatrix} \begin{bmatrix} T_m(\tau_0) & \dots & T_m(\tau_m) \end{bmatrix} = \begin{bmatrix} u_{00} & \dots & u_{0m} \\ \vdots & \ddots & \vdots \\ u_{n0} & \dots & u_{nm} \end{bmatrix}. \quad (4.96b)$$

Сформировав на параметрической сетке $\{t_0, t_1, \dots, t_n\} \times \{\tau_0, \tau_1, \dots, \tau_m\}$ матрицы Вандермонда свободного варианта задачи

$$\begin{aligned} Q_{\text{св}} &= [T_v(t_0) \ T_v(t_1) \ \dots \ T_v(t_n)] \in \mathbb{R}^{(v+1) \times (n+1)}, \\ \Theta_{\text{св}} &= [T_\mu(\tau_0) \ T_\mu(\tau_1) \ \dots \ T_\mu(\tau_m)] \in \mathbb{R}^{(\mu+1) \times (m+1)}, \end{aligned} \quad (4.96b)$$

квадратные при $v=n$ и $\mu=m$, перепишем систему (4.96b) в виде

$$Q^T S \Theta = U. \quad (4.96г)$$

Благодаря невырожденности матриц Q и Θ её решение находится как

$$S = H^T U H, \quad \text{где } H = Q^{-1}, \quad H = \Theta^{-1}, \quad (4.97a)$$

а вычислительно реализуется индивидуально для каждого слоя матрицы S :

$$S_x = H^T U_x H, \quad S_y = H^T U_y H, \quad S_z = H^T U_z H. \quad (4.97б)$$

Необходимые для расчёта вектора нормали $N(t, \tau) = V(t, \tau) \times W(t, \tau)$ направляющие векторы поверхности (4.93б) находятся по определению (1.93б):

$$V(t, \tau) = p'_t(t, \tau) = T_n^{rT}(t) S T_m(\tau), \quad W(t, \tau) = p'_\tau(t, \tau) = T_n^T(t) S T'_m(\tau). \quad (4.98)$$

На форму интерполяционной поверхности влияют значения обоих узловых параметров $t_0 \div t_n$ и $\tau_0 \div \tau_m$. Их выбор может быть как произвольным, так и регулярным, основанным на следующих рекомендациях:

• **локальная** параметризация сегмента с начальными значениями $t_0=\tau_0=0$ обладает большей вычислительной устойчивостью обращения матриц Вандермонда в сравнении со **сквозной** параметризацией. Последняя же обеспечивает непрерывность изменения параметров t и τ между смежными сегментами поверхности, составленной из множества полиномиальных кусков;

• **нормализованная** параметризация с длинами интервалов $d_i=\delta_j=1$ и отсчётами $t_i=t_{i-1}+1$ и $\tau_j=\tau_{j-1}+1$, не зависящими от расстояний между соседними узловыми точками, рекомендуется к использованию на сетях с примерно **равноотстоящими** друг от друга соседними узлами. При этом распределение каркасных линий субсетки по сегменту также будет близко к равномерному;

• **хордовая** параметризация применяется на произвольных узловых сетях $\{u_{ij}\}$ для получения более равномерного распределения линий субсетки, чем то, что даёт нормализованная параметризация. В отличие от линии, единственный параметр которой квантуется по формуле $t_i=t_{i-1}+|u_i-u_{i-1}|$, интерполяционная поверхность строится на узловой сетке с двумя пучками продольных $\{u_{0j}u_{1j}\dots u_{nj} \forall j=\overline{0,m}\}$ и поперечных $\{u_{i0}u_{i1}\dots u_{im} \forall i=\overline{0,n}\}$ полилиний, отстоящих друг от друга на **разных** расстояниях. Поэтому параметрические длины $d_i=t_i-t_{i-1}$ и $\delta_j=\tau_j-\tau_{j-1}$ определяются всей совокупностью межузловых расстояний $|u_{ij}-u_{i-1,j}|$ и $|u_{ij}-u_{i,j-1}|$, например, их средними значениями

$$d_i = \text{mean}(|u_{ij}-u_{i-1,j}| \forall j=\overline{0,m}), \quad \delta_j = \text{mean}(|u_{ij}-u_{i,j-1}| \forall i=\overline{0,n}). \quad (4.99)$$

Каркасное изображение интерполяционной поверхности посредством одной лишь узловой сети, составленной из полигональных ячеек с далеко отстоящими вершинами и прямолинейными рёбрами, весьма приблизительно отражает её форму в межузловом пространстве (см. рис. 4.94). На это способна субсетка из более мелких ячеек, созданная путём дробления интервалов изменения параметров t и τ на $N=k_t n$ и $M=k_\tau m$ квантов $\Delta t=(t_n-t_0)/N$ и $\Delta \tau=(\tau_m-\tau_0)/M$, где $k_t \geq 1$ и $k_\tau \geq 1$ — целочисленные коэффициенты дробления. В результате получаем формулы построения двух пучков полилиний субсетки:

$$\{p(t_0+i\Delta t, \tau_j) \forall i=\overline{0,N}\} \forall j=\overline{0,M}, \quad \{p(t_i, \tau_0+j\Delta \tau) \forall j=\overline{0,M}\} \forall i=\overline{0,N}.$$

В задаче интерполирования на **неравномерной** узловой сети субсетка характеризуется следующими **свойствами**, иллюстрируемыми на рис. 4.95:

• в рамках **нормализованной** параметризации (*а*), не учитывающей расстояний между узлами интерполяции, субсетка также является весьма неравномерной, поскольку одно и то же число полилиний пучка распределяется по разновеликим пространственным интервалам;

• по методу **хордовой** параметризации (*б*) субсетка получается более равномерной, но её линии могут и не проходить через все узловые точки.

Сравнение двух поверхностей, построенных на одной и той же **неравномерной** узловой сети, воочию убеждает в **существенной зависимости вида получаемого решения от выбора метода параметризации**.



Рис. 4.95

Главным **недостатком** полиномиальной интерполяции является возрастание межузловых осцилляций поверхности (4.93) с ростом числа узлов и степеней полинома, что подтверждает нормализованное решение на рис. 4.96,а. Каркасные линии $p(t_i, \tau)$ ведут себя как полиномы степени $m=7$ аналогично линии того же порядка на рис. 2.5. Закономерно то, что наибольший размах межузловых осцилляций приходится на крайние интервалы параметра τ . Во втором же пучке линии $p(t, \tau_j)$ малого порядка $n=2$ фактически являются неосциллирующими прямыми благодаря коллинеарности узловых точек $u_{0j} \div u_{2j}$.

Другой недостаток полиномиальной 3d-интерполяции — **глобальная чувствительность** формы всей поверхности $p(t, \tau)$ к изменению отдельной узловой точки u_{ij} , характеризуемая в (4.93а) весовой функцией $\Lambda_{ij}(t, \tau)$. Благодаря её узловому свойству $\Lambda_{ij}(t_v, \tau_\mu) = \delta_i^v \delta_j^\mu$ вес узла u_{ij} в точке $p(t_i, \tau_j)$ равен 1, а в остальных точках $p(t_v, \tau_\mu) = 0$. В межузловом пространстве поверхность $\Lambda_{ij}(t, \tau)$ совершает знакопеременные колебания. Зависимость максимумов осцилляций от степеней полинома показана на рис. 2.6,б. Иллюстрацией описанных свойств служит построенная на рис. 4.96,б весовая функция $\Lambda_{03}(t, \tau)$, отклоняющая даже далекие от узла u_{03} точки ячеек на краях поверхности.

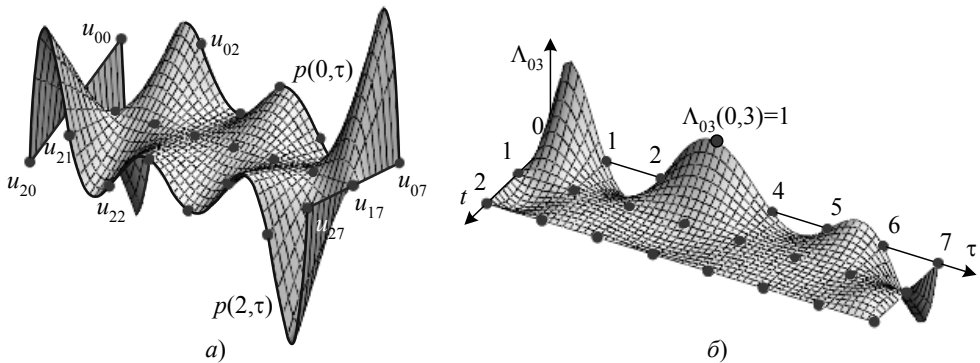


Рис. 4.96

Посильные возможности алгоритма полиномиальной интерполяции в задаче моделирования поверхностей различной сложности показаны на рис. 4.97, где построены точечные изображения фигур, похожих на ложку (а), тапку (б) и двухместный снегокат (в).

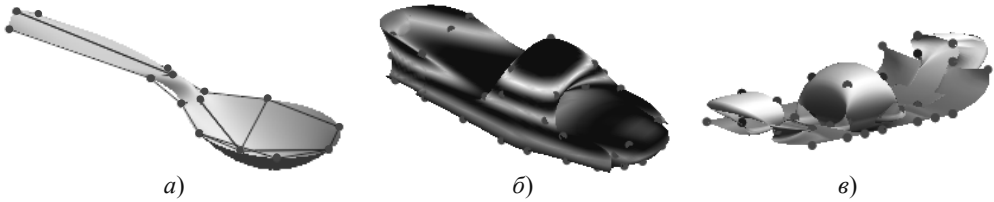


Рис. 4.97

Пример 4.23. Построить нормализованную полиномиальную поверхность на сетке из девяти узловых точек поверхности гиперболического параболоида $x^2 - y - z^2 = 0$:

$$\begin{aligned} u_{00} &= [-1 \ 0 \ 1], & u_{01} &= [-1 \ 1 \ 0], & u_{02} &= [-1 \ 0 \ -1], \\ u_{10} &= [0 \ -1 \ 1], & u_{11} &= [0 \ 0 \ 0], & u_{12} &= [0 \ -1 \ -1], \\ u_{20} &= [1 \ 0 \ 1], & u_{21} &= [1 \ 1 \ 0], & u_{22} &= [1 \ 0 \ -1]. \end{aligned}$$

Решение. Задав узловые отсчёты параметров $t = \{0, 1, 2\}$, $\tau = \{0, 1, 2\}$ и сформировав матрицы Вандермонда (4.96в) и координатные узловые матрицы

$$Q = \Theta = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 4 \end{bmatrix}, \quad U_x = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad U_y = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}, \quad U_z = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix},$$

вычислим по (4.97б) координатные МПК полинома (4.93б) с учётом того, что обратные матрицы Вандермонда $Q^{-1} = \Theta^{-1}$ равны матрице H из (2.16б):

$$S_x = \begin{bmatrix} -1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad S_y = \begin{bmatrix} 0 & 2 & -1 \\ -2 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad S_z = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

В результате получаем следующие координатные функции сегмента интерполяционной поверхности в параметрических интервалах $t \in [0, 2]$, $\tau \in [0, 2]$:

$$x(t, \tau) = -1 + t, \quad y(t, \tau) = -2t + t^2 + 2\tau - \tau^2, \quad z(t, \tau) = 1 - \tau.$$

Прямой подстановкой легко проверить, что они удовлетворяют неявному уравнению гиперболического параболоида $x^2(t, \tau) - y(t, \tau) - z^2(t, \tau) = 0$. На рис. 4.98 построен её график с каркасными линиями субсетки, проведёнными через кванты параметров $\Delta t = \Delta \tau = 0.2$. □

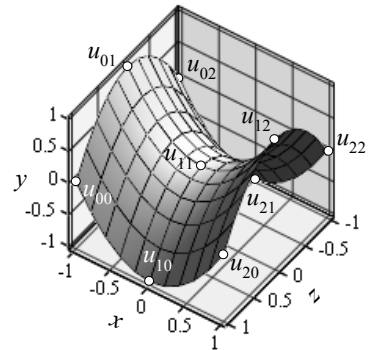


Рис. 4.98

Эрмитова узловая интерполяция

Уменьшению межузловых осцилляций интерполяционной поверхности способствует фиксация её **трансверсальных** (сопряжённых границам) краевых направлений четырьмя пучками векторов $V_{00} \div V_{0m}$, $V_{n0} \div V_{nm}$, $W_{00} \div W_{n0}$ и $W_{0m} \div W_{nm}$ (рис. 4.99), задающих изгиб поверхности на краях, где осцилляции наиболее сильны. Фиксацию можно выполнить следующими способами (см. рис. 2.11):

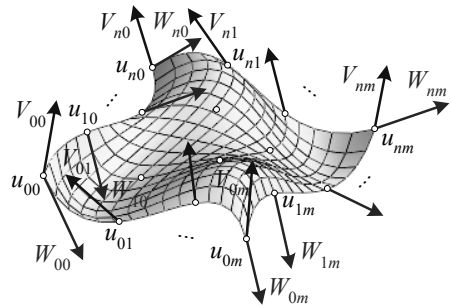


Рис. 4.99

• явно задать координаты этих векторов согласно желаемым наклонам краёв поверхности. Подобная задача возникает, например, при конструировании гладкого перехода между поверхностями с фиксированными направлениями соединяемых краёв. Также могут существовать дополнительные точки, через которые должна пройти сопрягающая поверхность;

- направить краевые векторы в сторону смежных трансверсальных узлов:

$$\begin{aligned} V_{0j} &= \gamma_0(u_{1j} - u_{0j}), \quad V_{nj} = \gamma_n(u_{nj} - u_{n-1,j}) \quad \forall j = \overline{0, m}, \\ W_{i0} &= \lambda_0(u_{i1} - u_{i0}), \quad W_{im} = \lambda_m(u_{im} - u_{i,m-1}) \quad \forall i = \overline{0, n}. \end{aligned} \quad (4.100a)$$

Регулирование длин краевых векторов осуществляется с помощью **коэффициентов удлинения** $\gamma_0, \gamma_n, \lambda_0$ и λ_m , каждый из которых ответственен за свой край сегмента. Уменьшение коэффициентов ослабляет прижимающий эффект. Однако, решение с нулевыми коэффициентами вовсе не равносильно поверхности со свободными краями, у которой краевые направления не нулевые;

- использовать формулы численного дифференцирования — двухточечного

$$\begin{aligned} V_{0j} &= (u_{1j} - u_{0j})/d_1, \quad V_{nj} = (u_{nj} - u_{n-1,j})/d_n \quad \forall j = \overline{0, m}, \\ W_{i0} &= (u_{i1} - u_{i0})/\delta_1, \quad W_{im} = (u_{im} - u_{i,m-1})/\delta_m \quad \forall i = \overline{0, n}, \end{aligned} \quad (4.100б)$$

совпадающего с (4.100a) при выборе $\gamma_0 = 1/d_1$, $\gamma_n = 1/d_n$, $\lambda_0 = 1/\delta_1$ и $\lambda_m = 1/\delta_m$, либо трёхточечного на основе (2.20б).

По два дополнительных ограничения в каждом параметрическом направлении увеличивают порядки **интерполяционной поверхности Эрмита** до $v = n + 2$ и $\mu = m + 2$, в результате чего она вместо (4.93б,в) принимает вид

$$p(t, \tau) = T_v^T(t) S T_\mu(\tau) = T_v^T(t) H_\mathfrak{Z}^T U_\mathfrak{Z} H_\mathfrak{Z} T_\mu(\tau) \quad \forall t \in [t_0, t_n], \tau \in [\tau_0, \tau_m] \quad (4.101a)$$

с трёхслойной $(v+1) \times (\mu+1)$ -матрицей коэффициентов S . Векторы касательных к этой поверхности находятся аналогично (4.98):

$$V(t, \tau) = T_v^T(t) S T_\mu(\tau), \quad W(t, \tau) = T_v^T(t) S T'_\mu(\tau). \quad (4.101б)$$

Обновив систему уравнений (4.96a) до

$$T_v^T(t_i) S T_\mu(\tau_j) = u_{ij} \quad \forall i = \overline{0, n}, \quad j = \overline{0, m} \quad (4.102a)$$

и дополнив её уравнениями фиксации векторов краевых касательных

$$\begin{aligned} T_v^T(t_i) S T_\mu(\tau_j) &= V_{ij} \quad \forall j = \overline{0, m}, \quad i \in \{0, n\}, \\ T_v^T(t_i) S T'_\mu(\tau_j) &= W_{ij} \quad \forall i = \overline{0, n}, \quad j \in \{0, m\}, \end{aligned} \quad (4.102б)$$

составим эрмитову СЛАУ (4.96г), использующую матрицы $Q_{св}$ и $\Theta_{св}$ из (4.96в):

$$\begin{aligned} Q_\mathfrak{Z} &= [Q_{св} \mid T'_v(t_0) \quad T'_v(t_n)], \quad \Theta_\mathfrak{Z} = [\Theta_{св} \mid T'_\mu(\tau_0) \quad T'_\mu(\tau_m)], \\ U_\mathfrak{Z} &= \left[\begin{array}{cc|cc} & & W_{00} & W_{0m} \\ & U & \vdots & \vdots \\ & & W_{n0} & W_{nm} \\ \hline V_{00} & \dots & V_{0m} & \text{O} & \text{O} \\ V_{n0} & \dots & V_{nm} & \text{O} & \text{O} \end{array} \right] \Rightarrow \begin{aligned} S &= H_\mathfrak{Z}^T U_\mathfrak{Z} H_\mathfrak{Z}, \text{ где} \\ H_\mathfrak{Z} &= Q_\mathfrak{Z}^{-1}, \quad H_\mathfrak{Z} = \Theta_\mathfrak{Z}^{-1}. \end{aligned} \end{aligned} \quad (4.102в)$$

Поскольку матрицы Q_Σ и Θ_Σ невырождены, то это решение всегда существует.

Полезное свойство интерполяционного полинома Эрмита уменьшать межузловые осцилляции показано на рис. 4.100 графиком поверхности (4.101a), построенной на той же узловой сети, что и на рис. 4.96,а, но с дополнительной фиксацией краевых векторов методом двухточечной разностной аппроксимации (4.100б).

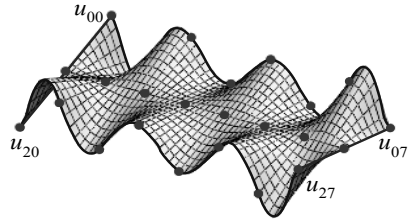


Рис. 4.100

Возможны и другие способы подавления межузловых осцилляций путём фиксации касательных векторов в различных узловых точках. Линейные аналоги этих методов рассмотрены в п. 2.1.1, а разработка поверхностных вариантов предоставляется читателю в качестве полезной практики.

Многие поверхности с топологией полузамкнутого цилиндра или замкнутого тора могут быть получены из поверхности со свободными краями путём соединения одной или двух пар её противоположных границ. Например, сшивка краёв $u_{00} \div u_{0m}$ и $u_{n0} \div u_{nm}$ узловой сети на рис. 4.94 даст поверхность цилиндрического типа, которая после соединения краёв $u_{00} \div u_{n0}$ и $u_{0m} \div u_{nm}$ превратится в поверхность тороидального типа.

Помимо физического совмещения противоположных граничных узлов

$$u_{0j}=u_{nj} \quad \forall j=\overline{1,m}, \quad u_{i0}=u_{im} \quad \forall i=\overline{1,n},$$

обеспечиваемого без помощи каких-либо уравнений относительно МПК S , построение гладкой **циклической поверхности** требует формирования дополнительных к (4.102а) уравнений и соответствующего расширения матриц свободной системы (4.96в) до размеров $Q \in \mathbb{R}^{(v+1) \times (v+1)}$, $\Theta \in \mathbb{R}^{(\mu+1) \times (\mu+1)}$ и $U \in \mathbb{V}^{(v+1) \times (\mu+1)}$:

- при гладком смыкании краёв $u_{00} \div u_{0m}$ и $u_{n0} \div u_{nm}$ уравниваются векторы их краевых касательных в t -направлении $V(t_0, \tau_j) = V(t_n, \tau_j)$, откуда с учётом соотношений (4.101б) возникают неявные уравнения

$$T_v'^T(t_0)ST_\mu(\tau_j) = T_v'^T(t_n)ST_\mu(\tau_j) \quad \forall j=\overline{0,m}$$

и следующие расширенные матрицы t -циклического замыкания и степени полинома по параметрам t и τ :

$$Q_{\text{ц}} = [Q_{\text{св}} | T_v'(t_n) - T_v'(t_0)], \quad U_{\text{ц}} = \begin{bmatrix} U \\ 0 \end{bmatrix}, \quad v=n+1, \quad \mu=m; \quad (4.103a)$$

- для гладкого соединения краёв $u_{00} \div u_{n0}$ и $u_{0m} \div u_{nm}$ уравниваются векторы их краевых касательных в τ -направлении $W(t_i, \tau_0) = W(t_i, \tau_m)$, в результате чего появляются неявные уравнения $T_v^T(t_i)ST_\mu'(\tau_0) = T_v^T(t_i)ST_\mu'(\tau_m) \quad \forall i=\overline{0,n}$ и следующие расширенные матрицы τ -циклического замыкания и степени полинома по параметрам t и τ :

$$\Theta_{\text{ц}} = [\Theta_{\text{св}} | T_\mu'(\tau_m) - T_\mu'(\tau_0)], \quad U_{\text{ц}} = [U | 0], \quad v=n, \quad \mu=m+1; \quad (4.103б)$$

• и, наконец, гладкая сшивка двух пар краёв поверхности осуществляется согласно неявным уравнениям

$$\begin{aligned} T_v'^T(t_0)ST_\mu(\tau_j) &= T_v'^T(t_n)ST_\mu(\tau_j) \quad \forall j = \overline{0, m}, \\ T_v^T(t_i)ST_\mu'(\tau_0) &= T_v^T(t_i)ST_\mu'(\tau_m) \quad \forall i = \overline{0, n} \end{aligned}$$

с помощью следующих матриц и степеней полинома:

$$Q_\Pi = Q_\Pi, \quad \Theta_\Pi = \Theta_\Pi, \quad U_\Pi = \begin{bmatrix} U & O \\ O & O \end{bmatrix}, \quad v = n+1, \quad \mu = m+1. \quad (4.103в)$$

Построение гладкой циклической поверхности часто становится невозможным из-за вырожденности матриц Q_Π и/или Θ_Π . Суть данного явления объясняет анализ определителей (2.25) в зависимости от выбора метода параметризации (нормализованной или хордовой), равномерности узловой сети и чётности её размеров n и m . Выводы, сделанные в п. 2.1.1, полностью применимы к задаче поверхностной интерполяции ввиду идентичности матриц Q_Π в (2.24а) и (4.103а). Нелишне ещё раз вспомнить **два случая разрешимости задачи полиномиальной интерполяции с циклической гладкостью первого порядка**, относящиеся к направлениям замыкания t и/или τ :

- **нечётный** размер узловой сети в замыкаемом направлении;
- при чётном размере это **неравномерность** узловой сети и использование **хордовой** параметризации. Напротив, симметрия и равномерность расположения узловых точек в направлении чётного числа интервалов либо выбор нормализованной параметризации делают создание гладко замкнутой интерполяционной поверхности невозможным ввиду вырожденности или очень плохой обусловленности матриц Q и/или Θ .

Три попытки решения этой незаурядной задачи показаны на рис. 4.101. Более-менее осмысленные формы поверхностей получены тщательным подбором расположения узлов интерполяции с учётом перечисленных условий разрешимости. Особенно это касается поверхности, созданной методом **хордовой** параметризации на **несимметричной** сетке с чётными размерами $n=2$ и $m=4$.

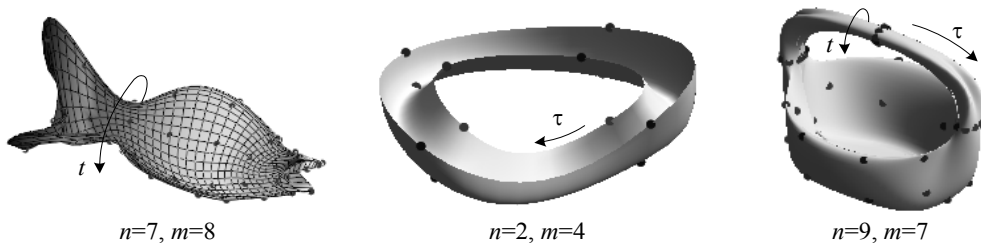


Рис. 4.101

Кардинальным средством разрешимости задачи циклической интерполяции является выбор **второго порядка гладкости замыкания** путём добавления в систему новых уравнений $p_t''(t_0, \tau_j) = p_t''(t_n, \tau_j)$ и $p_\tau''(t_i, \tau_0) = p_\tau''(t_i, \tau_m)$, расширяющих матрицы и степени системы следующим образом:

$$U_{\alpha} = \begin{bmatrix} U & O_{1,2} \\ O_{2,1} & O_{2,2} \end{bmatrix}; \quad \begin{aligned} Q_{\alpha} &= [Q_{\alpha v} \mid T'_{\alpha}(t_n) - T'_{\alpha}(t_0) \mid T''_{\alpha}(t_n) - T''_{\alpha}(t_0)], \quad v=n+2; \\ \Theta_{\alpha} &= [\Theta_{\alpha \mu} \mid T'_{\alpha}(\tau_m) - T'_{\alpha}(\tau_0) \mid T''_{\alpha}(\tau_m) - T''_{\alpha}(\tau_0)], \quad \mu=m+2. \end{aligned} \quad (4.103\Gamma)$$

Как показано в (2.25в), такие матрицы Q_{α} и Θ_{α} всегда невырождены и дают вычислительно устойчивое решение системы (4.96г). Доказательством служит дважды гладкая циклическая поверхность (рис. 4.102), построенная на **симметричной** сети с **чётными** размерами $n=2$ и $m=4$ методом **нормализованной** параметризации.

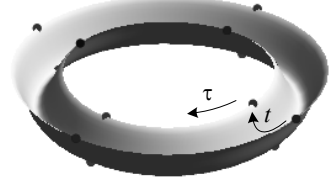


Рис. 4.102

Таким образом, **при создании гладких циклических поверхностей рекомендуется всегда соблюдать равенство двух трансверсальных производных на линиях сшивки**. Цена этого решения — увеличение степени полинома в замыкаемом направлении, а выигрыш — вычислительная устойчивость алгоритма интерполяции и предсказуемая форма получаемой поверхности.

4.4.3. Аппроксимация и приближение по МНК

Один из методов сглаживания осцилляций полиномиальной поверхности

$$p(t, \tau) = \sum_{i=0}^v \sum_{j=0}^{\mu} s_{ij} t^i \tau^j = T_v^T(t) S T_{\mu}(\tau) \quad \forall t \in [t_0, t_n], \quad \tau \in [\tau_0, \tau_m] \quad (4.104)$$

заключается в сознательном ограничении её изгибаемости путём уменьшения степеней полинома $v \in [1, n]$ и $\mu \in [1, m]$ в одном или обоих параметрических направлениях t и τ . В расплату за это поверхность (4.104) теряет интерполяционные свойства $p(t_i, \tau_j) = u_{ij}$ и превращается в **аппроксимационную**, проходящую лишь в окрестности **всех** вершин узловой сети $U \in V^{(v+1) \times (\mu+1)}$. Применим к расчёту её МПК $S \in V^{(v+1) \times (\mu+1)}$ оптимизационный подход, заключающийся в минимизации **функционала невязки** — суммы квадратов отклонений узловых точек полинома $p(t_i, \tau_j)$ от узлов сетки u_{ij} [42]:

$$\mathfrak{I} = \sum_{i=0}^n \sum_{j=0}^m |p(t_i, \tau_j) - u_{ij}|^2 \rightarrow \min_S. \quad (4.105a)$$

Сформируем по (4.96в) на узловых отсчётах параметров $\{t_0, t_1, \dots, t_n\}$ и $\{\tau_0, \tau_1, \dots, \tau_m\}$ полноранговые матрицы Вандермонда

$$\begin{aligned} Q_{vn} &= [T_v(t_0) \ T_v(t_1) \ \dots \ T_v(t_n)] \in \mathbb{R}^{(v+1) \times (n+1)}, \\ \Theta_{\mu m} &= [T_{\mu}(\tau_0) \ T_{\mu}(\tau_1) \ \dots \ T_{\mu}(\tau_m)] \in \mathbb{R}^{(\mu+1) \times (m+1)}. \end{aligned} \quad (4.105б)$$

При $v < n$ и $\mu < m$ эти матрицы не квадратны, а система (4.96г) переопределена и не имеет точного матричного решения (4.97а). По аналогии с (2.30б) запишем **приближённое МНК-решение**

$$S = H_{vn}^T U H_{\mu m}, \text{ где } H_{vn} = Q_{vn}^T (Q_{vn} Q_{vn}^T)^{-1}, \quad H_{\mu m} = \Theta_{\mu m}^T (\Theta_{\mu m} \Theta_{\mu m}^T)^{-1}, \quad (4.105\text{в})$$

доставляющее функционалу (4.105а) минимум

$$\mathfrak{I}_{\min} = \text{tr}((Q_{vn}^T S \Theta_{\mu m} - U)(Q_{vn}^T S \Theta_{\mu m} - U)^T). \quad (4.105\text{г})$$

В предельном случае $v=n$ и $\mu=m$ матрицы Q_{vn} и $\Theta_{\mu m}$ становятся квадратными, аппроксимационное решение (4.105в) превращается в интерполяционное (4.97а), а функционал невязки достигает абсолютного минимума $\mathfrak{I}_{\min}=0$.

Пример 4.24. Методом трёхмерной аппроксимации построить полиномиальную модель поверхности, сглаживающей набор точек с зашумлёнными координатами.

Решение. Возьмём, например, *квадратичную* поверхность гиперболического параболоида $x^2 - 4y - z^2 = 0$ с эталонной параметрической моделью $p_3(t, \tau) = [t(t^2 - \tau^2)/4, \tau]$ в параметрических интервалах $t \in [-3, 3]$, $\tau \in [-2, 2]$. Сформируем узловые отсчёты параметров $t_i = i - 3 \quad \forall i = \overline{0, 6}$, $\tau_j = j - 2 \quad \forall j = \overline{0, 4}$ и сгенерируем вершины узловой сети с координатами

$$p_{ij} = p_3(t_i, \tau_j) + [rnd(2\sigma) - \sigma, rnd(2\sigma) - \sigma, rnd(2\sigma) - \sigma], \quad (4.106)$$

где $\sigma = 0.3$ — полуширина центрированного интервала равномерного распределения случайных чисел $rnd(2\sigma) - \sigma \in (-\sigma, \sigma)$ (см. п. 1.1.6).

Построим на одной и той же зашумлённой узловой сети набор полиномиальных поверхностей (4.104) для всех сочетаний степеней полинома $v \in \{1, 2, 6\}$ и $\mu \in \{1, 2, 4\}$ (рис. 4.103). Как и полагается, билинейная модель с $v=\mu=1$ представляет собой косую плоскость, поверхности с $v=1$ имеют каркасные прямые в t -направлении, с $\mu=1$ — в τ -направлении, степени $v=2$ и $\mu=2$ создают параболические изгибы соответствующих каркасных линий.

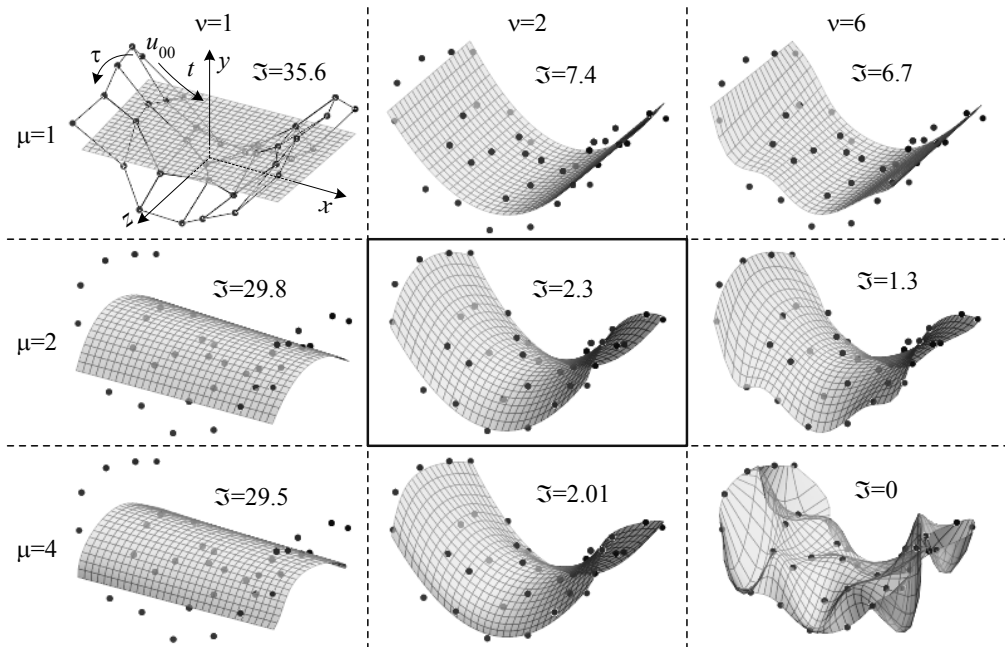


Рис. 4.103

Анализируя форму поверхностей, замечаем, что аппроксимация со степенями $v=\mu=2$ наиболее гладко отражает общие тенденции расположения узловых точек (параболические выгнутости вверх вдоль оси x и вниз вдоль оси z) и в то же время малочувствительна к их случайным смещениям. К тому же функционал невязки на этой паре $\{v, \mu\}$ претерпевает наиболее резкое уменьшение до $\mathfrak{Z} \approx 2.3$ в сравнении с меньшими степенями.

Дальнейшее увеличение степеней аппроксимационного полинома вплоть до интерполяционных значений $v=6$ и $\mu=4$ изгибает поверхность в локальных местах, приближая их к зашумлённым узловым точкам и теряя общие закономерности их расположения.

По совокупности признаков заключаем, что **эталонная поверхность описывается би-квадратичной полиномиальной моделью** (4.104) со степенями $v_3=\mu_3=2$. Численным расчётом была получена следующая матрица векторных коэффициентов этой модели, соответствующей интервалам параметров $t \in [-3, 3]$ и $\tau \in [-2, 2]$:

$$S = \begin{bmatrix} [-0.028 & 0.052 & -0.082] \\ [1.018 & 0.044 & -0.028] \\ [-0.002 & 0.26 & 0.002] \end{bmatrix} \begin{bmatrix} [0.052 & 0.066 & 0.988] \\ [0.018 & 0.011 & -0.018] \\ [0.008 & 0.079 & -0.008] \end{bmatrix} \begin{bmatrix} [0.053 & -0.246 & -0.023] \\ [0.012 & 0.017 & -0.012] \\ [-0.003 & 0.003 & 0.003] \end{bmatrix}.$$



В системах обработки данных вершины узловой сети u_{ij} часто бывают измерены с разными погрешностями. Точно измеренные точки достоверно представляют неизвестную функцию двух переменных, тогда как сильно зашумлённые весьма от неё далеки. В результате суммирование в (4.105а) квадратов невязок с одинаковыми весами приводит к получению полиномиальной зависимости $p(t, \tau)$, равномерно приближенной ко всем узловым точкам независимо от погрешностей их измерения. Хотелось бы, однако, чтобы восстанавливаемая поверхность была тем ближе к узловой точке, чем та точнее измерена, а влияние на поверхность грубых выбросов было минимальным.

Усовершенствуем метод (4.105) так, чтобы можно было регулировать близость поверхности (4.104) к вершинам узловой сети без изменения степеней полиномов $v \leq n$ и $\mu \leq m$. Для этого введём в квадратичный функционал невязки **весовые коэффициенты** $\varphi_{ij} \geq 0$, составляющие матрицу $\Phi \in \mathbb{R}^{(n+1) \times (m+1)}$ и обратно пропорциональные дисперсиям погрешностей измерения точек u_{ij} :

$$\mathfrak{Z} = \sum_{i=0}^n \sum_{j=0}^m \varphi_{ij} |p(t_i, \tau_j) - u_{ij}|^2 \rightarrow \min_S. \quad (4.107a)$$

При задании некоторого коэффициента $\varphi_{kl}=0$ вклад невязки $|p(t_k, \tau_l) - u_{kl}|$ в общую сумму исчезает, что делает поверхность полностью независимой от расположения точки u_{kl} .

Выполним минимизацию функционала (4.107а) **взвешенным методом наименьших квадратов (ВМК)**. Из-за произвольности весовых коэффициентов φ_{ij} представление взвешенной суммы квадратов невязок с помощью матриц Φ , S , Q_{vn} и $\Theta_{\mu m}$ отсутствует (при всех $\varphi_{ij}=1$ функционал (4.105а) может быть записан в виде следа матричного произведения $\text{tr}((Q^T S \Theta - U)(Q^T S \Theta - U)^T)$), но существует в поэлементной форме

$$\Im = \sum_{i,j} \Phi_{ij} \left| \sum_{k,l} s_{kl} t_i^k \tau_j^l - u_{ij} \right|^2 \quad \forall i=\overline{0,n}, j=\overline{0,m}, k=\overline{0,v}, l=\overline{0,\mu}. \quad (4.107б)$$

Из условий $\partial \Im / \partial s_{kl} = 0 \quad \forall k, l$ составим систему из $N = (v+1)(\mu+1)$ линейных уравнений относительно такого же числа элементов матрицы S :

$$\sum_{\kappa, \lambda} s_{\kappa\lambda} \underbrace{\sum_{i,j} \Phi_{ij} t_i^{k+\kappa} \tau_j^{l+\lambda}}_{a_{kl\kappa\lambda}} = \underbrace{\sum_{i,j} \Phi_{ij} t_i^k \tau_j^l u_{ij}}_{b_{kl}} \quad \forall \kappa=\overline{0,v}, \lambda=\overline{0,\mu}. \quad (4.107в)$$

Для решения этой СЛАУ развернём построчно прямоугольную матрицу $S \in V^{(v+1) \times (\mu+1)}$ в $(v+1)(\mu+1)$ -вектор

$$\vec{S} = [s_{00} \dots s_{0\mu} | s_{10} \dots s_{1\mu} | \dots | s_{v0} \dots s_{v\mu}] \in V^N. \quad (4.108а)$$

Используя принятые в (4.107в) обозначения индексированных коэффициентов $a_{kl\kappa\lambda}$ и b_{kl} , получим векторно-матричную СЛАУ N -го порядка:

$$\vec{S} A = B, \text{ где}$$

$$A = \begin{bmatrix} A_{00} & \dots & A_{0v} \\ \vdots & \ddots & \vdots \\ A_{v0} & \dots & A_{vv} \end{bmatrix} \in M^{(v+1) \times (v+1)}, \quad A_{k\kappa} = \begin{bmatrix} a_{k0\kappa 0} & \dots & a_{k\mu\kappa 0} \\ \vdots & \ddots & \vdots \\ a_{k0\kappa \mu} & \dots & a_{k\mu\kappa \mu} \end{bmatrix} \in R^{(\mu+1) \times (\mu+1)}, \quad (4.108б)$$

$$B = [b_{00} \dots b_{0\mu} | b_{10} \dots b_{1\mu} | \dots | b_{v0} \dots b_{v\mu}] \in R^N.$$

Решением этой системы, полностью определённой при ненулевой весовой матрице Φ , является вектор полиномиальных коэффициентов

$$\vec{S} = B A^{-1}, \quad (4.108в)$$

который по (4.108а) сворачивается в матрицу S , после чего по (4.104) строится аппроксимационная поверхность.

Иллюстрацией описанного алгоритма взвешенной МНК-аппроксимации служат поверхности на рис. 4.104, построенные на эталонной узловой сети из примера 4.24, вершины которой смещены по (4.106) с разными максимальными отклонениями равномерного распределения σ_{ij} . Точки u_{00} , u_{06} , u_{23} и u_{46} , обозначенные символом «○», сгенерированы при малых отклонениях $\sigma=0.05$, точка выброса u_{40} с большим значением $\sigma=5$ обозначена как «□», а остальные точки u_{ij} соответствуют $\sigma=0.5$. Весовые коэффициенты функционала (4.107а) задавались равными $\Phi_{ij} = \sigma_{ij}^{-2}$. Анализ поверхностей предоставляется читателю.

4.4.4. Поверхности Безье

Переходим к изучению полиномиальных поверхностей, сочетающих в себе как интерполяционные, так и аппроксимационные свойства относительно конечного множества узловых точек. Это означает, что через одни точки проектируемая поверхность проходит точно, а к другим, своего рода центрам притяжения, приближается лишь в некоторых окрестностях. В данном разделе из всей

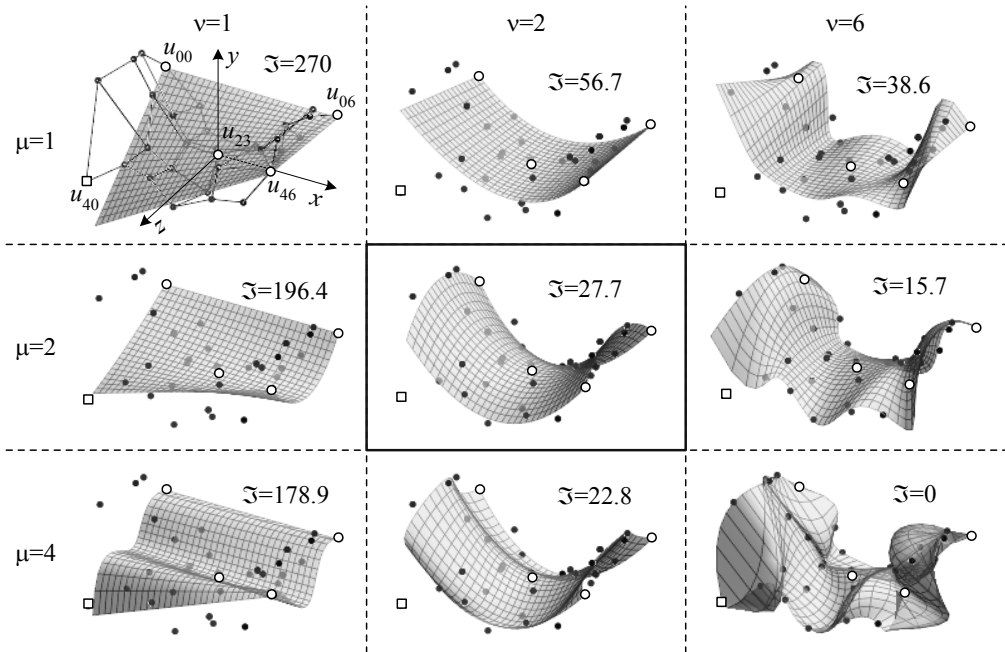


Рис. 4.104

совокупности вершин *характеристической полигональной сети* (ХПС) $\{u_{ij}\}$, $i=\overline{0,n}$, $j=\overline{0,m}$ — базы построения приближающей поверхности — в качестве узлов интерполяции принимаются угловые вершины u_{00} , u_{n0} , u_{0m} и u_{nm} , тогда как остальные рассматриваются как аппроксимационные.

Бикубические поверхности Безье

В интерактивном конструировании поверхностей удобнее манипулировать не точками и векторами, а одними лишь точками, перемещая их в пространстве с помощью мыши или клавиатуры и сразу оценивая получаемое решение. Распространим метод преобразования модели кубической линии Эрмита (2.28) в модель линии Безье (2.36а) на 3d-задачу построения сегмента приближающей поверхности на базе ХПС, вершины которой организованы по матричной схеме 4×4 и группируются по назначению следующим образом (рис. 4.105):

- **угловые** точки $u_{00} \div u_{11}$ являются узлами интерполяции, через которые поверхность проходит в обязательном порядке;

- **боковые** вершины $v_{00} \div v_{11}$ и $w_{00} \div w_{11}$ управляют направлениями векторов угловых касательных $V_{00} \div V_{11}$ и $W_{00} \div W_{11}$;

- **внутренние** точки $k_{00} \div k_{11}$ влияют на направления векторов угловых кручений $K_{00} \div K_{11}$.

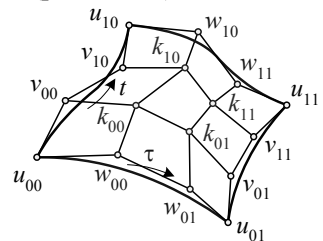


Рис. 4.105

Параметрическая модель **бикубического сегмента Безье** записывается в аналогичной (4.92) нормированной форме

$$p(t, \tau) = L_B^T(t) U_B L_B(\tau) = T_3^T(t) H_B^T U_B H_B T_3(\tau) \quad \forall t, \tau \in [0, 1] \quad (4.109)$$

с элементами разложений из (2.36в)

$$L_B(t) = \begin{bmatrix} 1-3t+3t^2-t^3 \\ 3t-6t^2+3t^3 \\ 3t^2-3t^3 \\ t^3 \end{bmatrix}, \quad H_B = \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad U_B = \begin{bmatrix} u_{00} & w_{00} & w_{01} & u_{01} \\ v_{00} & k_{00} & k_{01} & v_{01} \\ v_{10} & k_{10} & k_{11} & v_{11} \\ u_{10} & w_{10} & w_{11} & u_{11} \end{bmatrix}.$$

Исходя из краевых значений вектора весовых функций и его производной

$$L_B(0) = [1 \ 0 \ 0 \ 0]^T, \quad L_B(1) = [0 \ 0 \ 0 \ 1]^T, \quad L'_B(0) = [-3 \ 3 \ 0 \ 0]^T, \quad L'_B(1) = [0 \ 0 \ -3 \ 3]^T,$$

вычислим по (4.89а) и (4.90б) векторы угловых касательных и кручений

$$V_{ij} = L_B^{\pi}(i) U_B L_B(j), \quad W_{ij} = L_B^T(i) U_B L'_B(j), \quad K_{ij} = L_B^{\pi}(i) U_B L'_B(j) \quad \forall i, j \in \{0, 1\},$$

сведя результаты в табл. 4.3.

Таблица 4.3

$V_{00} = 3(v_{00} - u_{00})$ $W_{00} = 3(w_{00} - u_{00})$ $K_{00} = 9(u_{00} - v_{00} - w_{00} + k_{00})$	$V_{01} = 3(v_{01} - u_{01})$ $W_{01} = 3(u_{01} - w_{01})$ $K_{01} = -9(u_{01} - v_{01} - w_{01} + k_{01})$
$V_{10} = 3(u_{10} - v_{10})$ $W_{10} = 3(w_{10} - u_{10})$ $K_{10} = -9(u_{10} - v_{10} - w_{10} + k_{10})$	$V_{11} = 3(u_{11} - v_{11})$ $W_{11} = 3(u_{11} - w_{11})$ $K_{11} = 9(u_{11} - v_{11} - w_{11} + k_{11})$

Каждая граничная линия сегмента Безье определяется четвёркой соответствующих ей граничных вершин ХПС, например,

$$p(t, 0) = L_B^T(t) U_B L_B(0) = (1-t)^3 u_{00} + 3t(1-t)^2 v_{00} + 3t^2(1-t) v_{10} + t^3 u_{10}. \quad (4.110)$$

Сравнивая это выражение с (2.36а), заключаем, что **все граничные линии поверхности (4.109) являются линиями Безье, построенными на соответствующих граничных полилиниях характеристической сети.**

Взаимные переходы между бикубической моделью Кунса с матрицами U_K , $H_{\mathfrak{Z}}$ и моделью Безье с матрицами U_B , H_B осуществляются аналогичным (4.70) **методом конгруэнтных преобразований** [43]

$$U_B = M^T U_K M, \quad U_K = (M^{-1})^T U_B M^{-1}$$

с прямой и обратной матрицами

$$M = H_{\mathfrak{Z}} H_B^{-1} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1/3 & 0 & 0 \\ 0 & 0 & -1/3 & 0 \end{bmatrix}, \quad M^{-1} = \begin{bmatrix} 1 & 0 & -3 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & -3 \\ 0 & 1 & 0 & 3 \end{bmatrix}.$$

Отсюда получаем формулы нахождения вершин характеристической сети сегмента Безье по угловым точкам, векторам касательных и кручений бикубического сегмента Кунса:

$$\begin{cases} v_{00}=u_{00}+V_{00}/3, w_{00}=u_{00}+W_{00}/3, w_{01}=u_{01}-W_{01}/3, v_{01}=u_{01}+V_{01}/3, \\ k_{00}=u_{00}+V_{00}/3+W_{00}/3+K_{00}/9, k_{01}=u_{01}+V_{01}/3-W_{01}/3-K_{01}/9, \\ k_{10}=u_{10}-V_{10}/3+W_{10}/3-K_{10}/9, k_{11}=u_{11}-V_{11}/3-W_{11}/3+K_{11}/9, \\ v_{10}=u_{10}-V_{10}/3, w_{10}=u_{10}+W_{10}/3, w_{11}=u_{11}-W_{11}/3, v_{11}=u_{11}-V_{11}/3. \end{cases} \quad (4.111)$$

Анализируя изложенный материал, приходим к следующим **выводам**:

- конструктор поверхности Безье может управлять её формой и векторами угловых касательных и кручений путём интерактивного перемещения в пространстве одних лишь вершин характеристической сетки;
- все векторы угловых касательных сегмента Безье направлены по рёбрам ХПС, примыкающим к угловым вершинам;
- все коэффициенты удлинения касательных векторов имеют значения $\gamma_{ij}=\lambda_{ij}=3 \ \forall i, j \in \{0,1\}$. По аналогии с одноимённой линией это означает, что **поверхность сегмента Безье наиболее плотно заполняет выпуклую оболочку вершин характеристической сети**;
- каждый вектор углового кручения определяется тремя векторами, выходящими из соответствующей угловой точки ХПС в три смежные с ней вершины:

$$K_{00}=9(k_{00}-u_{00})-9(v_{00}-u_{00})-9(w_{00}-u_{00}),$$

$$K_{01}=9(v_{01}-u_{01})+9(w_{01}-u_{01})-9(k_{01}-u_{01}),$$

$$K_{10}=9(v_{10}-u_{10})+9(w_{10}-u_{10})-9(k_{10}-u_{10}),$$

$$K_{11}=9(k_{11}-u_{11})-9(v_{11}-u_{11})-9(w_{11}-u_{11}).$$

Отсюда следует, что **угловая ячейка характеристической сети, имеющая форму плоского параллелограмма, обладает нулевым кручением**;

- каждая граничная линия сегмента Безье определяется только четырьмя граничными вершинами соответствующего края характеристической сети и сама является линией Безье.

Приведём примеры построения сегментов Безье, подтверждающие сделанные выводы. Поверхность на рис. 4.106,а создана на характеристической сетке U_b , вершины которой рассчитаны по формулам (4.111) с использованием векторов угловых касательных и кручений бикубического сегмента Кунса, показанного на рис. 4.93,з. Хорошо видно, что для получения заметного изгиба поверхности при компланарных векторах касательных приходится размещать управляющие точки $k_{00} \div k_{11}$ на значительном расстоянии от плоскости узлов.

Несмотря на небольшой размер характеристической сетки и ограниченную гибкость кубического полинома, модель (4.109) способна создать вполне сложную, сильно изогнутую поверхность, один из экземпляров которой показан на рис. 4.106,б. Конструирование подобной фигуры как бикубического полинома Кунса (4.92) весьма проблематично из-за неочевидности выбора векторов угловых кручений, тогда как манипулирование вершинами характеристической сети

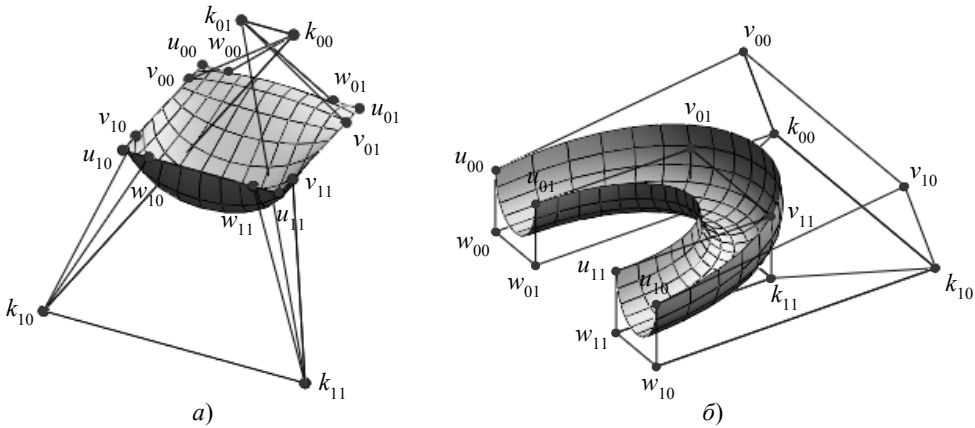


Рис. 4.106

с гарантией того, что получаемая в результате поверхность Бэзе всегда будет находиться внутри выпуклой оболочки ХПС, не представляет особых затруднений. Это объясняет чрезвычайную популярность этого метода в системах компьютерного проектирования.

Пример 4.25. Построить сегменты Бэзе, приближающие $1/8$, $1/4$ и $1/2$ поверхности сферы единичного радиуса (рис. 4.107).

Решение. Вся сложность решения поставленных задач состоит в конструировании таких характеристических сетей, которые по (4.109) дадут поверхности, максимально близкие к желаемым частям сферы. Наиболее точно должны быть построены границы сегментов, по которым они смогут стыковаться с круглыми краями других поверхностей.

Таким образом, необходимо обеспечить максимальную близость граничных линий проектируемых сегментов к дугам $1/4$ и $1/2$ окружности. Построение таких эрмитовых линий (см. рис. 2.15) рассмотрено в примере 2.2. Каждая из них является также линией Бэзе и определяется тремя отрезками характеристической полилинии. Длины крайних отрезков, задающих краевые направления дуг, должны быть в три раза меньше оптимальных коэффициентов удлинения $\gamma_{\text{опт}} \approx 1.656$ и $\gamma_{\text{опт}} \approx 3.947$, поскольку в модели линии Бэзе (2.36) коэффициент удлинения равен трём.

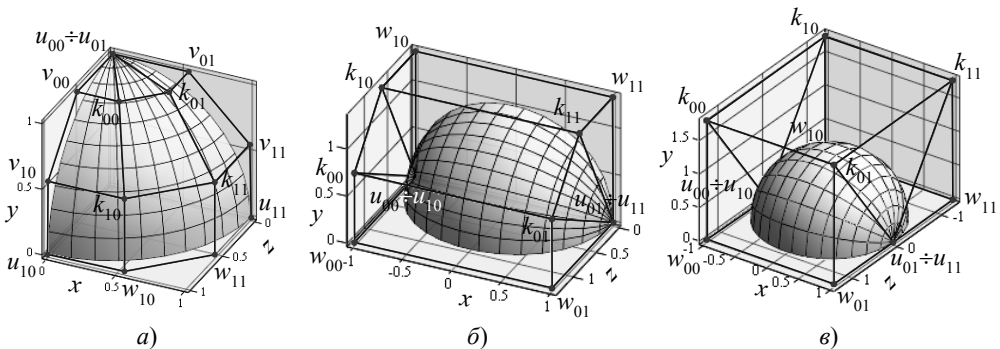


Рис. 4.107

С учётом сказанного введём числа $\alpha \approx 0.552$, $\beta \approx 1.316$ и создадим по одной из возможных матриц вершин ХПС, на основе которых по (4.109) строятся:

- поверхность Безье, наиболее близкая к октанту сферы (рис. 4.107,а)

$$U_B = \begin{bmatrix} [0 & 1 & 0] & [0 & 1 & 0] & [0 & 1 & 0] & [0 & 1 & 0] \\ [0 & 1 & \alpha] & [\alpha^2 & 1 & \alpha] & [\alpha & 1 & \alpha^2] & [\alpha & 1 & 0] \\ [0 & \alpha & 1] & [\alpha & \alpha & 1] & [1 & \alpha & \alpha] & [1 & \alpha & 0] \\ [0 & 0 & 1] & [\alpha & 0 & 1] & [1 & 0 & \alpha] & [1 & 0 & 0] \end{bmatrix};$$

- поверхность Безье, наиболее близкая к квадранту сферы (рис. 4.107,б)

$$U_B = \begin{bmatrix} [-1 & 0 & 0] & [-1 & 0 & \beta] & [1 & 0 & \beta] & [1 & 0 & 0] \\ [-1 & 0 & 0] & [-1 & \alpha\beta & \beta] & [1 & \alpha\beta & \beta] & [1 & 0 & 0] \\ [-1 & 0 & 0] & [-1 & \beta & \alpha\beta] & [1 & \beta & \alpha\beta] & [1 & 0 & 0] \\ [-1 & 0 & 0] & [-1 & \beta & 0] & [1 & \beta & 0] & [1 & 0 & 0] \end{bmatrix};$$

- поверхность Безье, наиболее близкая к полусфере (рис. 4.107,в)

$$U_B = \begin{bmatrix} [-1 & 0 & 0] & [-1 & 0 & \beta] & [1 & 0 & \beta] & [1 & 0 & 0] \\ [-1 & 0 & 0] & [-1 & \beta^2 & \beta] & [1 & \beta^2 & \beta] & [1 & 0 & 0] \\ [-1 & 0 & 0] & [-1 & \beta^2 & -\beta] & [1 & \beta^2 & -\beta] & [1 & 0 & 0] \\ [-1 & 0 & 0] & [-1 & 0 & -\beta] & [1 & 0 & -\beta] & [1 & 0 & 0] \end{bmatrix}.$$

Аналитические модели построенных поверхностей имеют среднюю степень сложности и не столь интересны для анализа, чтобы здесь их приводить. $\square$

Полиномиальные поверхности Безье

Аналогично тому, как кубическая линия Безье (2.36) обобщается на полиномиальную линию Безье (2.38), так и бикубическая поверхность Безье (4.109) может быть обобщена на характеристическую сеть $\{u_{ij}\}$ размера $(n+1) \times (m+1)$ вершин и описана аналогично (4.93а)

$$p(t, \tau) = \sum_{i=0}^n \sum_{j=0}^m B_{ij}(t, \tau) u_{ij} = B_n^T(t) U B_m(\tau) \quad \forall t, \tau \in [0, 1] \quad (4.112a)$$

с векторами весовых функций Бернштейна, определённых в (2.38б):

$$B_n(t) = [b_{n0}(t) \dots b_{nn}(t)]^T, \quad B_m(\tau) = [b_{m0}(\tau) \dots b_{mm}(\tau)]^T.$$

Ядром разложения (4.112а) является матрица вершин ХПС $U = \{u_{ij}\}$. Вклад точки u_{ij} в суммарное значение $p(t, \tau)$ характеризуется неотрицательной двухпараметрической весовой функцией

$$B_{ij}(t, \tau) = b_{ni}(t) b_{mj}(\tau) = C_n^i C_m^j t^i (1-t)^{n-i} \tau^j (1-\tau)^{m-j} \geq 0.$$

Оставшиеся в (4.93б,в) формы разложения сегмента

$$p(t, \tau) = T_n^T(t) S T_m(\tau) = T_n^T(t) H_n^T U H_m T_m(\tau), \quad (4.112б)$$

связанные с формой Лагранжа соотношениями (4.94)

$$B_n(t) = H_n T_n(t), \quad B_m(\tau) = H_m T_m(\tau), \quad S = H_n^T U H_m,$$

реализуются после расчёта по (2.38в) верхних треугольных матриц H_n и H_m с элементами $h_{ij} = (-1)^{j-i} C_k^j C_j^i \quad \forall 0 \leq i \leq j \leq k, \quad k \in \{n, m\}$. Вспомнив (2.40), приведём матрицы $H_1 \div H_5$ (исключая матрицу $H_3 = H_B$ из (4.109)) и H_6 , готовые к использованию при степенях полинома Безье $n \leq 6$ и $m \leq 6$:

$$H_1 = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad H_2 = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 2 & -2 \\ 0 & 0 & 1 \end{bmatrix}, \quad H_4 = \begin{bmatrix} 1 & -4 & 6 & -4 & 1 \\ 0 & 4 & -12 & 12 & -4 \\ 0 & 0 & 6 & -12 & 6 \\ 0 & 0 & 0 & 4 & -4 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$

$$H_5 = \begin{bmatrix} 1 & -5 & 10 & -10 & 5 & -1 \\ 0 & 5 & -20 & 30 & -20 & 5 \\ 0 & 0 & 10 & -30 & 30 & -10 \\ 0 & 0 & 0 & 10 & -20 & 10 \\ 0 & 0 & 0 & 0 & 5 & -5 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad H_6 = \begin{bmatrix} 1 & -6 & 15 & -20 & 15 & -6 & 1 \\ 0 & 6 & -30 & 60 & -60 & 30 & -6 \\ 0 & 0 & 15 & -60 & 90 & -60 & 15 \\ 0 & 0 & 0 & 20 & -60 & 60 & -20 \\ 0 & 0 & 0 & 0 & 15 & -30 & 15 \\ 0 & 0 & 0 & 0 & 0 & 6 & -6 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

По аналогии с линией (2.38) назовём модель (4.112) при **произвольных** значениях $n \geq 1$ и $m \geq 1$ **полиномиальной поверхностью Безье** в отличие от её частного случая — поверхности (4.109), строящейся на характеристической сетке фиксированного размера 4×4 вершин. Наследуя свойства линейного сегмента (2.39), поверхность Безье обладает в свою очередь следующими **свойствами**:

- степень полинома $p(t, \tau)$ в каждом параметрическом направлении на единицу меньше числа узлов ХПС;
- лежит в выпуклой оболочке массива точек $\{u_{00} \div u_{nm}\}$ благодаря удовлетворению всеми весовыми функциями $B_{ij}(t, \tau)$ свойств (2.35) неотрицательности и единичной суммы;
- благодаря значениям $B^T(0) = [1 \ 0 \ \dots \ 0]$ и $B^T(1) = [0 \ 0 \ \dots \ 1]$ фиксирована в угловых вершинах ХПС $p(0,0) = u_{00}$, $p(1,0) = u_{n0}$, $p(0,1) = u_{0m}$ и $p(1,1) = u_{nm}$;
- ограничена линиями $p(t,0)$, $p(t,1)$, $p(0,\tau)$ и $p(1,\tau)$, каждая из которых определена только граничными вершинами соответствующего края характеристической сетки и является полиномиальной линией Безье (2.38);
- векторы угловых касательных $V_{i,n,j,m} = B_n'^T(i)UB_m(j)$ и $W_{i,n,j,m} = B_n^T(i)UB'_m(j)$, где $i, j \in \{0,1\}$, направлены по крайним рёбрам четырёх угловых ячеек ХПС, а векторы угловых кручений $K_{i,n,j,m} = B_n'^T(i)UB'_m(j)$ определены четырьмя вершинами этих ячеек в соответствии с табл. 4.4.

Таблица 4.4

$V_{00} = n(u_{10} - u_{00})$ $W_{00} = m(u_{01} - u_{00})$ $K_{00} = nm(u_{00} - u_{10} - u_{01} + u_{11})$	$V_{0m} = n(u_{1m} - u_{0m})$ $W_{0m} = m(u_{0m} - u_{0,m-1})$ $K_{0m} = -nm(u_{0m} - u_{1m} - u_{0,m-1} + u_{1,m-1})$
$V_{n0} = n(u_{n0} - u_{n-1,0})$ $W_{n0} = m(u_{n1} - u_{n0})$ $K_{n0} = -nm(u_{n0} - u_{n-1,0} - u_{n1} + u_{n-1,1})$	$V_{nm} = n(u_{nm} - u_{n-1,m})$ $W_{nm} = m(u_{nm} - u_{n,m-1})$ $K_{nm} = nm(u_{nm} - u_{n-1,m} - u_{n,m-1} + u_{n-1,m-1})$

Из неё видно отличие зависимостей длин векторов от размерностей ХПС: касательные векторы удлинняются пропорционально лишь одному порядку, тогда как длины векторов кручений пропорциональны произведению порядков сети. Это объясняет отмеченную ранее разномасштабность геометрических размеров поверхности Кунса и векторов её первых и смешанных производных, делающую процесс её интерактивного конструирования неудобным. Манипулирование одними вершинами ХПС, учитывая свойства поверхностей Безье целиком уместаться в их выпуклой оболочке, эргономичней и наглядней.

Из таблицы также следует, что **угловая ячейка характеристической сети в форме плоского параллелограмма обладает нулевым угловым кручением**. Например, если точки $u_{00} \div u_{11}$ таковы, что $u_{10} - u_{00} = u_{11} - u_{01}$, то ячейка имеет вид параллелограмма, а нулевая разность векторов её рёбер даёт нулевой вектор кручения K_{00} . Если же у угловой ячейки форма произвольного плоского четырёхугольника, то ненулевой вектор углового кручения будет компланарен её плоскости. Согласно (4.91) поверхность в окрестности угловой точки будет совпадать с плоскостью ячейки и иметь квадратично изогнутые t - и τ -изолинии.

Алгоритм полиномиального приближения (4.112) даёт более разнообразные и гибкие поверхности, чем бикубический сегмент Безье, благодаря свободе выбора размеров ХПС, задаваемых произвольными числами n и m . Это иллюстрируют фигуры на рис. 4.108, а, б построенные на характеристических сетях с узловыми размерами 10×4 и 7×3 . Налицо ещё одно важное **свойство полиномиального приближения**: уменьшение влияния вершин ХПС на форму по-

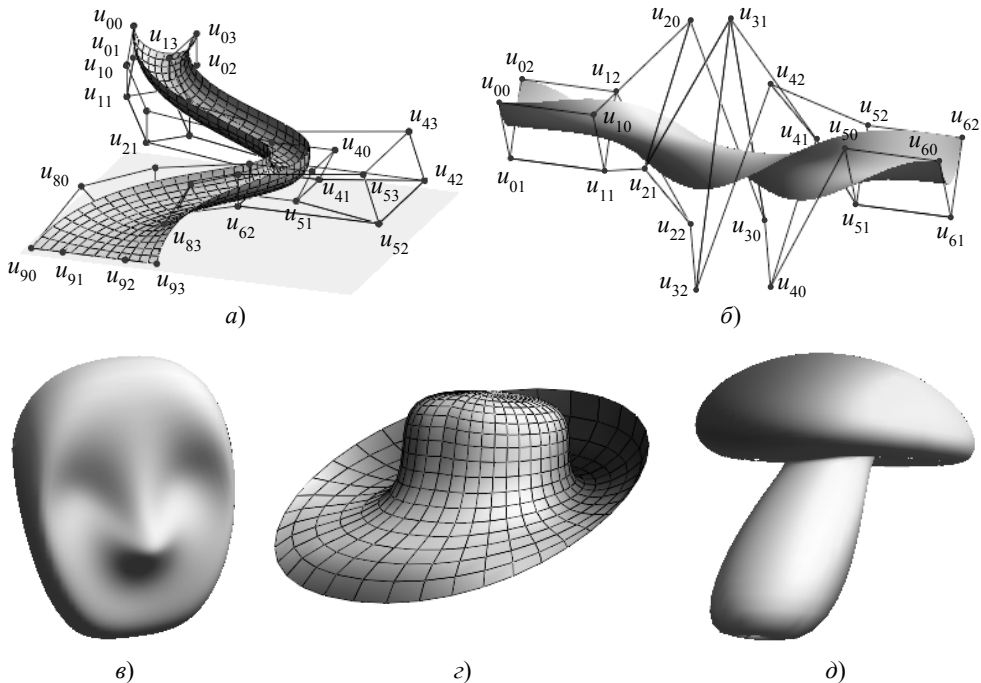


Рис. 4.108

верхности по мере их удаления от краёв (см. линейный аналог на рис. 2.25,а), из-за чего приходится увеличивать амплитуду отклонений внутренних вершин сети для получения желаемых изгибов в средней части поверхности.

Богатые возможности интерактивного проектирования скульптурных форм демонстрируют изображения лицевой маски, шляпы и гриба на рис. 4.108, в–д. Для полноты восприятия объектов их характеристические сетки не показаны.

Редактирование поверхностей Безье

Область применения поверхностей Безье — системы автоматизированного проектирования разнообразных изделий сложной формы. Их создание происходит в **интерактивном** режиме: сначала конструктор задаёт начальное расположение вершин ХПС и строит на ней поверхность (4.112), после чего в режиме **редактирования характеристической сети** добивается желаемой формы получаемого решения. Редактирование ХПС означает проведение операций с её вершинами: **перемещение** имеющихся, **добавление** новых и **удаление** лишних. После каждой операции поверхность перестраивается и оценивается заново.

Приступая к перемещению характеристической вершины, которое можно выполнить либо ручным вводом её новых координат или вектора переноса, либо захватом и перетаскиванием точки в пространстве с помощью мыши и клавиатуры, следует помнить об уменьшении влияния узла u_{ij} на поверхность Безье по мере приближения индексов i и j к средним значениям $[n/2]$ и $[m/2]$. Из-за этого приходится далеко сдвигать срединные узлы для получения заметного изменения поверхности. Иллюстрацией сказанного служит рис. 4.109, где показаны изменения поверхности (а) при равных перемещениях двух вершин ХПС, по-разному удалённых от её центра u_{13} . Смещение граничной вершины u_{10} гораздо сильнее изменяет поверхность (б) в сравнении с малой деформацией поверхности (в), вызванной таким же переносом срединной вершины u_{13} .

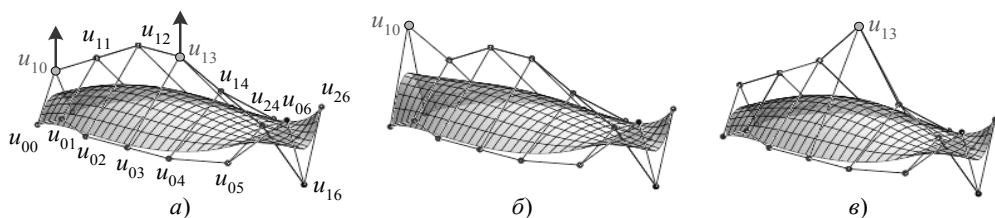


Рис. 4.109

Операции добавления и удаления характеристических вершин изменяют размеры сети $U = \{u_{ij}\}$ и степени $\{n, m\}$ бипараметрического полинома $p(t, \tau)$. В отличие от линии Безье в ХПС одноимённой поверхности можно добавлять или удалять не отдельные вершины, а только целые полилинии $u_{0k}u_{1k} \dots u_{n-1,k}u_{nk}$ и/или $u_{k0}u_{k1} \dots u_{k,m-1}u_{km}$ в параметрических направлениях t и/или τ . В результате в новой сети $U' = \{u'_{ij}\}$ с узловыми размерами $(n'+1) \times (m'+1)$ перераспределяется влияние вершин на значения функции

$$p'(t, \tau) = \sum_{i=0}^{n'} \sum_{j=0}^{m'} b_{n'i}(t) b_{m'j}(\tau) u'_{ij} = B_{n'}^T(t) U' B_{m'}(\tau) \quad \forall t, \tau \in [0, 1].$$

Рассмотрим два способа изменения порядков характеристической сети, отличающиеся изменчивостью формы поверхности Безье [33].

□ В **ручном** режиме добавление новых или удаление лишних полилиний ХПС приводит к перенумерации вершин и перестройке поверхности Безье, поскольку при новых степенях $n \neq n'$ и $m \neq m'$ её определяют уже другие весовые функции Бернштейна $b_{n'i}(t)$ и $b_{m'j}(\tau)$. В результате **ручное редактирование размеров характеристической полигональной сети меняет форму поверхности Безье**. На рис. 4.110 показано, как это выглядит для поверхности из рис. 4.108,б, если (а) добавить в её ХПС новую полилинию $u'_{60}u'_{61}u'_{62}$ и (б) удалить среднюю полилинию $u_{30}u_{31}u_{32}$.

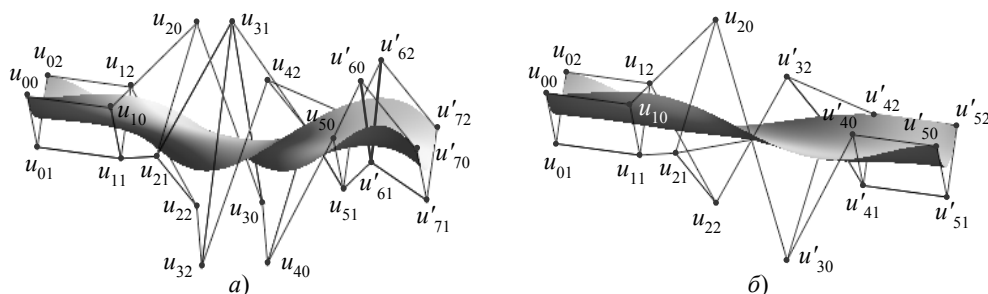


Рис. 4.110

□ В интерактивном проектировании более удобно **алгоритмическое** изменение размеров ХПС, при котором происходит перераспределение её вершин с **сохранением формы** поверхности Безье. Это позволяет быстро получить новую характеристическую сеть желаемых размеров и продолжить её дальнейшее редактирование, плавно и предсказуемо конструируя требуемую поверхность.

Новая конфигурация ХПС находится из условия совпадения новой и старой моделей $p'(t, \tau) = p(t, \tau)$, записанных в форме (4.112б):

$$T_{n'}^T(t) H_{n'}^T U' H_{m'} T_{m'}(\tau) = T_n^T(t) H_n^T U H_m T_m(\tau) \quad \forall t, \tau \in [0, 1].$$

Этому уравнению удовлетворяет узловое решение

$$U' = (H_n E_{n+1, n'+1} H_{n'}^{-1})^T U (H_m E_{m+1, m'+1} H_m^{-1}), \quad (4.113)$$

обобщающее линейные аналоги (2.44б) и (2.45) сразу на обе задачи — как на добавление характеристических полилиний, так и на их удаление. По (4.113) может быть создана ХПС с увеличенным размером в одном параметрическом направлении и с уменьшенным — в другом, например, с $n' > n$ и $m' < m$. Более того, данный метод даёт решения восьми задач изменения размеров ХПС во всевозможных сочетаниях с $n' < n$, $n' = n$, $n' > n$ и $m' < m$, $m' = m$, $m' > m$.

Возможность изменения порядков полиномов n_i и m_j определяется структурой матриц $S = H_n^T U H_m$ и $S' = H_{n'}^T U' H_{m'}$:

• **увеличение** порядков на $d=n'-n>0$ и/или $\delta=m'-m>0$ возможно **всегда**. При этом матрица S' , вычисленная на новой сетке U' , принимает одну из форм

$$\begin{bmatrix} S \\ O_{d,m+1} \end{bmatrix}, [S' | O_{n+1,\delta}], \left[\begin{array}{c|c} S & O_{n+1,\delta} \\ \hline O_{d,m+1} & O_{d,\delta} \end{array} \right] \quad (4.114a)$$

и содержит d нулевых строк и/или δ нулевых столбцов. Это означает, что реальные порядки биполиномиальной функции $p'(t,\tau)$ остались равными n и m ;

• **уменьшение** порядков максимум на $d=n-n'>0$ и/или $\delta=m-m'>0$ возможно **не всегда**, а лишь при условии, что реальные порядки поверхности $p(t,\tau)$, зависящие от расположения вершин ХПС U , меньше формальных, т. е. $n'=n-d$ и/или $m'=m-\delta$. Дефекты степеней d и δ определяются числом **последних** нулевых строк и столбцов в матрице S , принимающей одну из следующих форм:

$$\begin{bmatrix} S' \\ O_{d,m+1} \end{bmatrix}, [S' | O_{n+1,\delta}], \left[\begin{array}{c|c} S' & O_{n'+1,\delta} \\ \hline O_{d,m'+1} & O_{d,\delta} \end{array} \right]. \quad (4.114б)$$

При нулевом дефекте понижение соответствующего порядка невозможно.

Работа алгоритма изменения размеров ХПС проиллюстрирована на рис. 4.111. Исходная поверхность (а) порядков $n=4$ и $m=3$ построена на сетке из 5×4 узлов и имеет матрицу S первого типа в (4.114б), содержащую одну нижнюю нулевую строку. Значит, дефект степени $d=1$, а реальный t -порядок полинома $p(t,\tau)$ составляет $n'=n-d=3$. Это подтверждает поверхность (б), построенная на сетке U' узловых размеров 4×4 . Третья поверхность (в) демонстрирует возможность одновременного понижения t -порядка до реального $n'=3$ и повышения τ -порядка до формального $m'=6$.

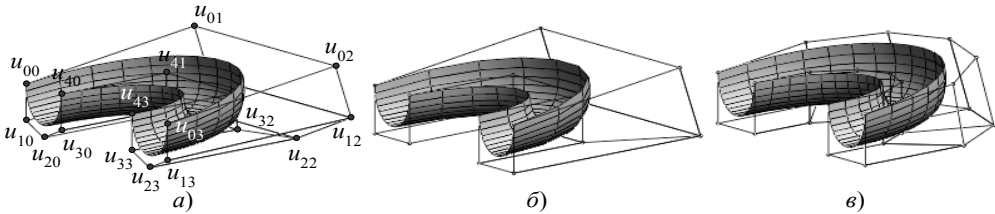


Рис. 4.111

Рациональные поверхности Безье

Существенное расширение возможностей конструирования полиномиальных поверхностей Безье (4.112) достигается наделением вершин характеристической полигональной сети $\{u_{ij}\}=U$ весами $\{\omega_{ij}\}=\Omega$ и описанием поверхности в пространстве однородных координат (2.46). В результате получаются модели **рациональной поверхности Безье** в однородных и декартовых координатах

$${}^o p(t,\tau) = \sum_{i=0}^n \sum_{j=0}^m b_{ni}(t) b_{mj}(\tau) {}^o u_{ij} = B_n^T(t) {}^o U B_m(\tau), \quad (4.115)$$

$$p(t, \tau) = \frac{\sum_{i=0}^n \sum_{j=0}^m b_{ni}(t) b_{mj}(\tau) \omega_{ij} u_{ij}}{\sum_{i=0}^n \sum_{j=0}^m b_{ni}(t) b_{mj}(\tau) \omega_{ij}} = \frac{B_n^T(t) (U \bullet \Omega) B_m(\tau)}{B_n^T(t) \Omega B_m(\tau)} \quad \forall t, \tau \in [0, 1]$$

с функциями Бернштейна, определёнными в (2.38б). Эти модели сочетают основные свойства 2d-аналога (2.48) и полиномиальной поверхности Безье. Так, форма рациональной поверхности определяется не самими весами, но их соотношениями, а **полиномиальная поверхность Безье является частным равновесным** (все $\omega_{ij}=1$) **случаем рациональной**. При *всех* неотрицательных весах $\omega_{ij} \geq 0$ рациональная поверхность всегда лежит в выпуклой полиэдральной оболочке ХПС (рис. 4.112,а). Это свойство теряется, если какой-либо вес ω_{kl} задан отрицательным для имитации отталкивания поверхности от точки u_{kl} (рис. 4.112,б). От полиномиальных поверхностей Безье рациональным передана **зависимость граничных линий** $p(t, 0)$, $p(t, 1)$, $p(0, \tau)$ и $p(1, \tau)$ **только от вершин и весов соответствующего края ХПС**.

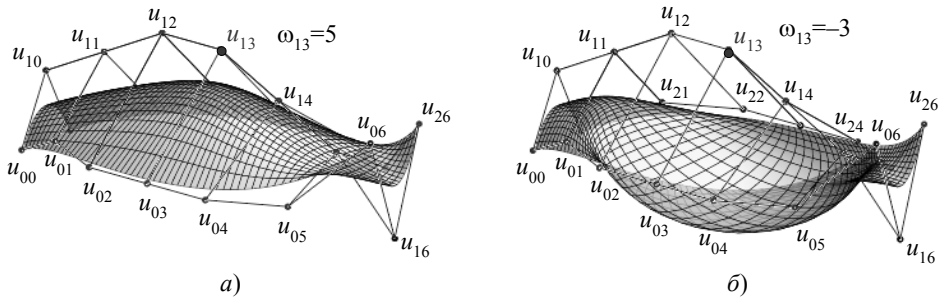


Рис. 4.112

Возможность изменения формы поверхности одной лишь вариацией весов **при неподвижной характеристической сети** замечательно отличает рациональные модели Безье от полиномиальных. К примеру, поместим граничные вершины 3×3 -сетки в середины рёбер и углах плоского квадрата, а вершину u_{11} — на перпендикуляре к его центру (рис. 4.113). Назначим угловым вершинам сетки фиксированные веса $\omega_{00} = \omega_{20} = \omega_{02} = \omega_{22} = 1$. Целенаправленным выбором весов остальных вершин можно получить следующие поверхности:

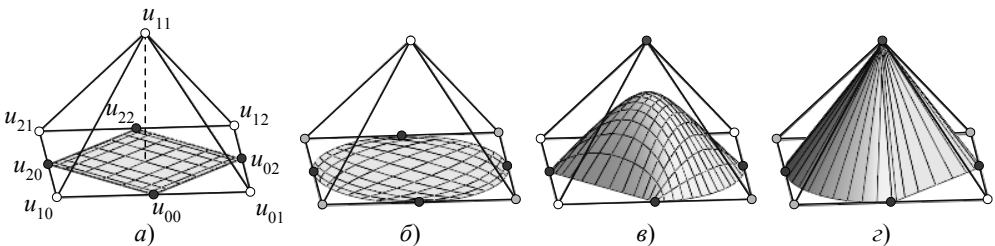


Рис. 4.113

- плоского **квадрата** (a) при нулевых весах $\omega_{10}=\omega_{12}=\omega_{01}=\omega_{21}=\omega_{11}=0$;
- плоского **круга** (b) при весах $\omega_{10}=\omega_{12}=\omega_{01}=\omega_{21}=1/\sqrt{2}$, соответствующих углу $\delta=\pi/4$ в (2.54б), и нулевом весе $\omega_{11}=0$;
- выпуклость (c) при $\omega_{11}>0$ или вогнутость при $\omega_{11}<0$ с плоской граничной линией, форма которой определяется весами ω_{10} , ω_{12} , ω_{01} и ω_{21} ;
- сколь угодно близкую к конусу или пирамиде (z) при $\omega_{11}\rightarrow\infty$.

К характеристической сети рациональной поверхности полностью применим алгоритм (4.113) изменения размеров и автоматической переконфигурации в новую сеть узловых размеров $(n'+1)\times(m'+1)$, принимающий в пространстве однородных координат следующий вид:

$$\dot{U}'=(H_n E_{n+1,n'+1} H_n^{-1})^T \dot{U} (H_m E_{m+1,m'+1} H_m^{-1}).$$

Здесь кроме новых вершин u'_{ij} дополнительно формируются их новые веса ω'_{ij} . Форма поверхности $p'(t, \tau)$, построенной на сетке U' с матрицей весов Ω' , точно совпадает с поверхностью $p(t, \tau)$.

В рациональной модели (4.115) каждый узел u_{ij} имеет индивидуальный вес ω_{ij} , что позволяет создавать несимметричные поверхности даже на симметричной ХПС (см. рис. 4.113, в, з). Возможен менее универсальный метод **декартова произведения** рациональных линий $p(t_i, \tau) \times p(t, \tau_j)$ и их параметрических весов $\omega_{it} \times \omega_{\tau j}$, когда в (4.115) вес (i, j) -й вершины равен $\omega_{it} \omega_{\tau j}$ — произведению весов ω_{it} и $\omega_{\tau j}$ в t - и τ -направлениях. При этом полный набор $(n+1)(m+1)$ весов определяется двумя независимыми наборами $\{\omega_{t0}, \omega_{t1}, \dots, \omega_{tm}\}$ и $\{\omega_{\tau0}, \omega_{\tau1}, \dots, \omega_{\tau m}\}$. В тех задачах, где выбор весов характеристических вершин обусловлен геометрией их расположения, метод декартова произведения оказывается работоспособным лишь на особых, геометрически **симметричных** ХПС.

Наиболее привлекательным свойством рациональных поверхностей, наследуемым от рациональных линий (см. п. 2.1.3), является их **способность точно воспроизводить сегменты квадратичных поверхностей**, которые можно использовать для скругления углов полиэдров и сопряжения поверхностей. Для получения требуемого типа и желаемой формы рационального сегмента нужно не только правильно расположить вершины характеристической сети, но и присвоить им верные веса, что представляет собой весьма непростую задачу. Здесь на помощь могут прийти свойства симметрии конструируемой поверхности, позволяющие назначить симметричным вершинам равные веса и тем самым уменьшить неопределённость выбора.

Аналогично тому, как квадратичные линии (кроме полу- и полных эллипсов) описываются рациональными полиномами второго порядка (2.51а) на основе трёхузловой характеристической полилинии, так и большинство квадратичных и тороидальных (кроме полу- и полностью замкнутых в любом из параметрических направлений) **симметричных** сегментов может быть создано на девятиузловой ХПС $\{u_{ij}\} \quad \forall i=\overline{0,2}, j=\overline{0,2}$ с матрицами вершин и весов

$$U = \begin{bmatrix} u_{00} & u_{01} & u_{02} \\ u_{10} & u_{11} & u_{12} \\ u_{20} & u_{21} & u_{22} \end{bmatrix}, \quad \Omega = \begin{bmatrix} 1 \\ \omega_t \\ 1 \end{bmatrix} [1 \ \omega_\tau \ 1] = \begin{bmatrix} 1 & \omega_\tau & 1 \\ \omega_t & \omega_t \omega_\tau & \omega_t \\ 1 & \omega_\tau & 1 \end{bmatrix}. \quad (4.116)$$

Подобие краевых полилиний $\{u_{00}u_{01}u_{02}, u_{20}u_{21}u_{22}\}$, $\{u_{00}u_{10}u_{20}, u_{02}u_{12}u_{22}\}$ и равенство весов $\omega_{10}=\omega_{12}=\omega_t$, $\omega_{01}=\omega_{21}=\omega_\tau$ обеспечивает подобие пар граничных линий сегмента $\{p(0,\tau), p(1,\tau)\}$ и $\{p(t,0), p(t,1)\}$. Положительные веса ω_t и ω_τ создают вогнутость, а отрицательные — выпуклость граничных линий и всей поверхности в соответствующем параметрическом направлении (рис. 4.114). Модули весов определяют тип квадратичной граничной линии (см. комментарии к рис. 2.33). У цилиндрического (ж) и конического (з) сегментов три тройки вершин ХПС в прямолинейном направлении должны быть **коллинеарными**, а веса средних вершин — **нулевыми**. Поэтому можно понизить порядок сегмента в этом направлении путём изъятия «невесомых» вершин из ХПС.

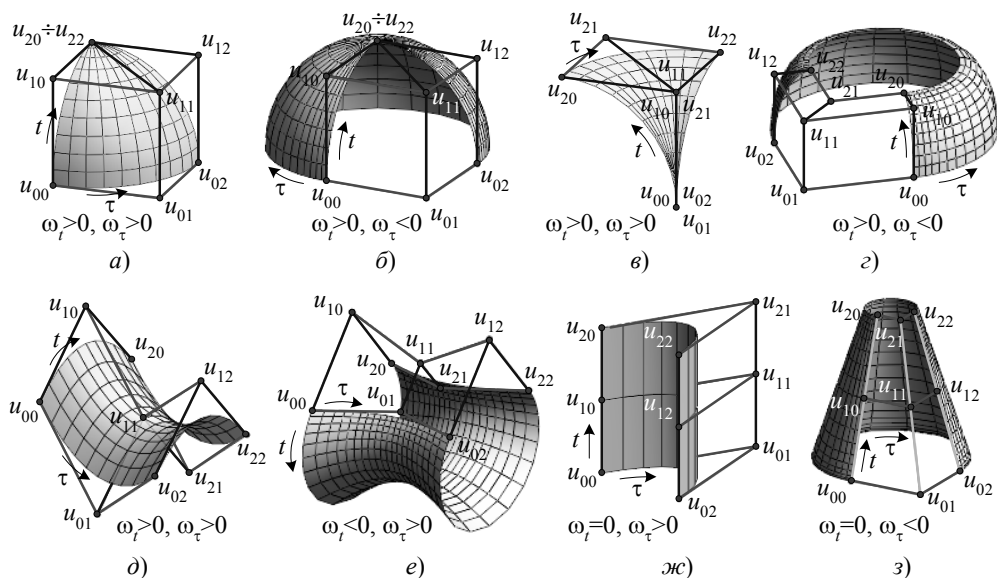


Рис. 4.114

Рассмотрим задачу создания характеристической сети, на которой по (4.115) строится **рациональный симметричный сегмент сферы** единичного радиуса в интервалах углов по долготе $\varphi \in [\varphi_0 - \delta_\varphi, \varphi_0 + \delta_\varphi]$ и широте $\psi \in [\psi_0 - \delta_\psi, \psi_0 + \delta_\psi]$ с углами раствора $2\delta_\varphi \notin \{\pi, 2\pi\}$ и $2\delta_\psi < \pi$ (рис. 4.115,а). Поскольку граничные дуги сегмента не являются полу- либо полными окружностями, то для их моделирования рациональными линиями достаточно размеров характеристической сети $n=m=2$. Зададим девять вершин ХПС $u_{ij} = [x_{ij} \ y_{ij} \ z_{ij}]$ $\forall i=\overline{0,2}, j=\overline{0,2}$ из следующих соображений (рис. 4.115,б):

- четыре угловые вершины расположим на поверхности сферы согласно их сферическим координатам:

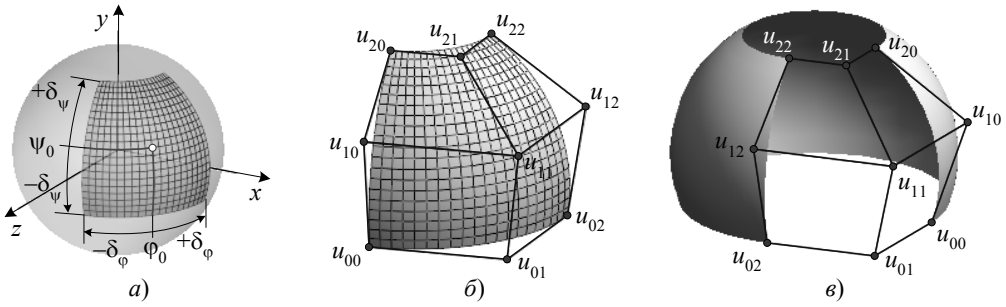


Рис. 4.115

$$\begin{aligned} u_{00} &= z^0 R_x(-\psi_0 + \delta_\psi) R_y(\varphi_0 - \delta_\varphi), \quad u_{02} = z^0 R_x(-\psi_0 + \delta_\psi) R_y(\varphi_0 + \delta_\varphi), \\ u_{20} &= z^0 R_x(-\psi_0 - \delta_\psi) R_y(\varphi_0 - \delta_\varphi), \quad u_{22} = z^0 R_x(-\psi_0 - \delta_\psi) R_y(\varphi_0 + \delta_\varphi); \end{aligned} \quad (4.117a)$$

• две боковые вершины u_{10} и u_{12} лежат в пересечении угловых касательных к меридианам $\varphi_0 - \delta_\varphi$ и $\varphi_0 + \delta_\varphi$ на расстоянии $1/\cos(\delta_\psi)$ от центра сферы:

$$u_{10} = \sec(\delta_\psi) z^0 R_x(-\psi_0) R_y(\varphi_0 - \delta_\varphi), \quad u_{12} = \sec(\delta_\psi) z^0 R_x(-\psi_0) R_y(\varphi_0 + \delta_\varphi); \quad (4.117б)$$

• две боковые вершины u_{01} и u_{21} лежат в точках пересечения горизонтальных угловых касательных к параллелям $\psi_0 - \delta_\psi$ и $\psi_0 + \delta_\psi$:

$$u_{01} = \sec(\delta_\varphi)(u_{00} - y_{00}y^0)R_y(\delta_\varphi) + y_{00}y^0, \quad u_{21} = \sec(\delta_\varphi)(u_{20} - y_{20}y^0)R_y(\delta_\varphi) + y_{20}y^0; \quad (4.117в)$$

• центральная вершина лежит на пересечении горизонтальных перпендикуляров к плоскостям меридианов $\varphi_0 - \delta_\varphi$ и $\varphi_0 + \delta_\varphi$ в точках u_{10} и u_{12} :

$$u_{11} = \sec(\delta_\varphi)(u_{10} - y_{10}y^0)R_y(\delta_\varphi) + y_{10}y^0. \quad (4.117г)$$

Ввиду актуальности не абсолютных весов вершин ХПС, а их соотношений, зададим веса угловых вершин единичными, боковых вершин по (2.54б), а вес центральной вершины ввиду симметричности сегмента по (4.116):

$$\begin{aligned} \omega_{00} &= \omega_{02} = \omega_{20} = \omega_{22} = 1, \quad \omega_{01} = \omega_{21} = \cos(\delta_\varphi), \\ \omega_{10} &= \omega_{12} = \cos(\delta_\psi), \quad \omega_{11} = \cos(\delta_\varphi) \cos(\delta_\psi). \end{aligned} \quad (4.117д)$$

Если угол раствора $2\delta_\varphi \in (\pi, 2\pi)$, то веса $\omega_{01} \div \omega_{21}$ отрицательны и «выталкивают» точки сегмента из выпуклой оболочки вершин ХПС (рис. 4.115, в).

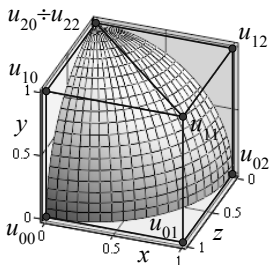


Рис. 4.116

Поверхность сегмента (4.115) с вершинами и весами ХПС, определёнными в (4.117), **точно** удовлетворяет (проверено символическим процессором MathCAD!) неявному уравнению сферы единичного радиуса $p(t, \tau) \circ p(t, \tau) = 1 \quad \forall t, \tau$. В частности, задав угловые параметры **первого октанта сферы** $\varphi_0 = \psi_0 = \pi/4$ и $\delta_\varphi = \delta_\psi = \pi/4$ (рис. 4.116), то по (4.116) получим иско-
мые матрицы характеристических вершин и весов:

$$U = \begin{bmatrix} [0 & 0 & 1] \\ [0 & 1 & 1] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [1 & 0 & 1] \\ [1 & 1 & 1] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [1 & 0 & 0] \\ [1 & 1 & 0] \\ [0 & 1 & 0] \end{bmatrix}, \quad \Omega = \begin{bmatrix} 1 & 1/\sqrt{2} & 1 \\ 1/\sqrt{2} & 1/2 & 1/\sqrt{2} \\ 1 & 1/\sqrt{2} & 1 \end{bmatrix}. \quad (4.118)$$

Для построения рациональных сегментов сферы с углами раствора $2\delta_\varphi \in \{\pi, 2\pi\}$ и/или $2\delta_\psi = \pi$ мало размеров ХПС $n=2$ и/или $m=2$. В обсуждении рис. 2.36,б,в получены расположения характеристических вершин и их веса, при которых линии (2.48б) становятся полу- и полной окружностями. Аналогичными должны быть границы ХПС в направлениях граничных линий сегмента.

Пример 4.26. Построить рациональные поверхности Безье, моделирующие $1/4$, $1/2$ и целую часть сферы единичного радиуса.

Решение. Основываясь на конфигурации и весах характеристических вершин, необходимых для построения половины и полной окружности, составим матрицы вершин U и весов Ω для следующих рациональных поверхностей:

- правой ближней четверти сферы (рис. 4.117,а)

$$U = \begin{bmatrix} [0 & -1 & 0] \\ [0 & -1 & 2] \\ [0 & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [2 & -1 & 2] \\ [2 & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [2 & -1 & 0] \\ [2 & 1 & 0] \\ [0 & 1 & 0] \end{bmatrix}, \quad \Omega = \begin{bmatrix} 1 & 1/\sqrt{2} & 1 \\ 1/3 & 1/3\sqrt{2} & 1/3 \\ 1/3 & 1/3\sqrt{2} & 1/3 \\ 1 & 1/\sqrt{2} & 1 \end{bmatrix};$$

- правой полусферы (рис. 4.117,б)

$$U = \begin{bmatrix} [0 & -1 & 0] \\ [0 & -1 & 2] \\ [0 & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [4 & -1 & 2] \\ [4 & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [4 & -1 & -2] \\ [4 & 1 & -2] \\ [0 & 1 & -2] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [0 & -1 & -2] \\ [0 & 1 & -2] \\ [0 & 1 & 0] \end{bmatrix}, \quad \Omega = \begin{bmatrix} 1 & 1/3 & 1/3 & 1 \\ 1/3 & 1/9 & 1/9 & 1/3 \\ 1/3 & 1/9 & 1/9 & 1/3 \\ 1 & 1/3 & 1/3 & 1 \end{bmatrix};$$

- полной сферы (рис. 4.117,в):

$$U = \begin{bmatrix} [0 & -1 & 0] \\ [0 & -1 & 2] \\ [0 & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [4\sqrt{3} & -1 & 2] \\ [4\sqrt{3} & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [4\sqrt{3} & -1 & -10] \\ [4\sqrt{3} & 1 & -10] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [-4\sqrt{3} & -1 & -10] \\ [-4\sqrt{3} & 1 & -10] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [-4\sqrt{3} & -1 & 2] \\ [-4\sqrt{3} & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix} \begin{bmatrix} [0 & -1 & 0] \\ [0 & -1 & 2] \\ [0 & 1 & 2] \\ [0 & 1 & 0] \end{bmatrix},$$

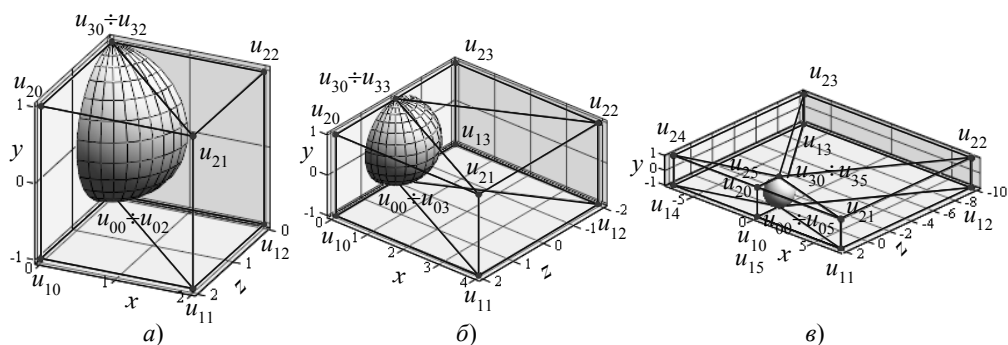


Рис. 4.117

$$\Omega = \begin{bmatrix} 1 & 1/5 & 1/10 & 1/10 & 1/5 & 1 \\ 1/3 & 1/15 & 1/30 & 1/30 & 1/15 & 1/3 \\ 1/3 & 1/15 & 1/30 & 1/30 & 1/15 & 1/3 \\ 1 & 1/5 & 1/10 & 1/10 & 1/5 & 1 \end{bmatrix}.$$



Квадратичные сегменты широко применяются в конструкторской практике как **поверхности скругления** углов при рёбрах и вершинах полиэдров. Примером служит цилиндрическая поверхность, вписанная в двугранный угол на рис. 2.37, в. Обозначив радиус скругления как r , а вес вершины прямого угла как $\omega_{\perp} = \sin(45^\circ) = 1/\sqrt{2}$, составим матрицы узлов и весов ХПС, необходимые для построения сегментов по (4.115), а также альтернативные модели поверхностей скругления, основанные на методах граничной интерполяции (см. п. 4.4.1).

Начнём с задачи **цилиндрического скругления** двугранного угла Γ с торцами $a_0a_1a_2$, $b_0b_1b_2$ и общим ребром граней a_1b_1 (рис. 4.118). Его угол раствора 2γ лежит в **ортогональном** сечении двугранника плоскостью Π и находится по (1.136) как угол между нормальными векторами граней N_1 и N_2 , вычисляемыми с помощью вспомогательных нормированных векторов V , V_1 и V_2 :

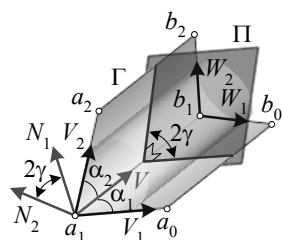


Рис. 4.118

$$\begin{aligned} V &= \overline{b_1 - a_1}, \quad V_1 = \overline{a_0 - a_1}, \quad V_2 = \overline{a_2 - a_1}, \quad N_1 = \overline{V_1 \times V}, \\ N_2 &= \overline{V_2 \times V}, \quad \gamma = 0.5 \arccos(N_1 \circ N_2). \end{aligned} \quad (4.119a)$$

Для задания вершин ХПС нам потребуются векторы

$$W_1 = \overline{b_0 - b_1}, \quad W_2 = \overline{b_2 - b_1}. \quad (4.119b)$$

Главный итог исследования, проведённого в конце п. 2.1.3, применительно к задаче скругления двугранного угла заключается в том, что **рациональные дуги всех поперечных сечений цилиндрического сегмента строятся на трёх-узловых ХПС с одинаковым весом всех средних вершин** $\omega = \sin(\gamma)$, где γ — половина угла **ортогонального** сечения, вычисляемая по (4.119a). Поэтому и ввиду линейности цилиндрической поверхности в направлении вектора V её характеристические матрицы имеют, в отличие от (4.116), размеры 2×3 :

$$U = \begin{bmatrix} u_{00} & u_{01} & u_{02} \\ u_{10} & u_{11} & u_{12} \end{bmatrix}, \quad \Omega = \begin{bmatrix} 1 & \omega & 1 \\ 1 & \omega & 1 \end{bmatrix}, \quad \omega = \sin(\gamma) = \sqrt{(1 - N_1 \circ N_2)/2}. \quad (4.119b)$$

С ними рациональная модель (4.115) принимает вид $p(t, \tau) = (1-t)g(\tau) + th(\tau)$ гранично-линейной интерполяции (4.83a) с рациональными границами (2.51a)

$$g(\tau) = \frac{(1-\tau)^2 u_{00} + 2\tau(1-\tau)\omega u_{01} + \tau^2 u_{02}}{(1-\tau)^2 + 2\tau(1-\tau)\omega + \tau^2}, \quad h(\tau) = \frac{(1-\tau)^2 u_{10} + 2\tau(1-\tau)\omega u_{11} + \tau^2 u_{12}}{(1-\tau)^2 + 2\tau(1-\tau)\omega + \tau^2}.$$

Составим формулы вычисления характеристических вершин в (4.119b):

- в случае **прямого** ($2\gamma = 90^\circ$, $\omega = 1/\sqrt{2}$) двугранного угла с **ортогональными** торцами вершины ХПС выбираются в следующих точках (рис. 4.119a):

$$\begin{aligned} u_{00} &= a_1 + rV_1, \quad u_{01} = a_1, \quad u_{02} = a_1 + rV_2, \\ u_{10} &= b_1 + rW_1, \quad u_{11} = b_1, \quad u_{12} = b_1 + rW_2. \end{aligned} \quad (4.120a)$$

При этом отрезки $u_{00}u_{10}$ и $u_{02}u_{12}$ отстоят от ребра a_1b_1 на расстоянии r , а торцами скругляющей поверхности являются дуги **окружностей** $g(\tau)$ и $h(\tau)$;

- если торцы граней ортогональны, а угол $2\gamma \neq 90^\circ$ (рис. 4.119,б), то угловые вершины отстоят от средних вершин на расстоянии $d = r \operatorname{ctg}(\gamma)$ (см. рис. 2.35):

$$\begin{aligned} u_{00} &= a_1 + dV_1, \quad u_{01} = a_1, \quad u_{02} = a_1 + dV_2, \\ u_{10} &= b_1 + dW_1, \quad u_{11} = b_1, \quad u_{12} = b_1 + dW_2. \end{aligned} \quad (4.120б)$$

Торцами скругляющей поверхности также являются дуги **окружностей**;

- более сложен расчёт вершин ХПС **произвольного** двугранного угла с **произвольными** торцами $a_0a_1a_2$ и $b_0b_1b_2$ (рис. 4.119,в). Вычислив углы между отрезком a_1b_1 и торцевыми отрезками a_1a_0 , a_1a_2 и b_1b_0 , b_1b_2 , расположим характеристические вершины, исходя из условий параллельности отрезков $u_{00}u_{10} \parallel u_{02}u_{12}$ и их удаления от ребра a_1b_1 на равные расстояния d :

$$\begin{aligned} \alpha_i &= \arccos(V \circ V_i), \quad \beta_i = \arccos(W \circ W_i) \quad \forall i = \overline{1,2}, \quad d = r \operatorname{ctg}(\gamma), \\ u_{00} &= a_1 + V_1 d / \sin(\alpha_1), \quad u_{01} = a_1, \quad u_{02} = a_1 + V_2 d / \sin(\alpha_2), \\ u_{10} &= b_1 + W_1 d / \sin(\beta_1), \quad u_{11} = b_1, \quad u_{12} = b_1 + W_2 d / \sin(\beta_2). \end{aligned} \quad (4.120в)$$

Торцами поверхности скругления $g(\tau)$ и $h(\tau)$ при неортогональности плоскостей сечения сегмента круглого цилиндра являются дуги **эллипсов**;

- изложенная методика просто обобщается на **общую задачу скругления двугранного угла** рациональной **конической** поверхностью с произвольными торцевыми радиусами скругления r_1 и r_2 (рис. 4.119,з):

$$\begin{aligned} d_i &= r_i \operatorname{ctg}(\gamma), \quad \alpha_i = \arccos(V \circ V_i), \quad \beta_i = \arccos(W \circ W_i) \quad \forall i = \overline{1,2}, \\ u_{00} &= a_1 + V_1 d_1 / \sin(\alpha_1), \quad u_{01} = a_1, \quad u_{02} = a_1 + V_2 d_1 / \sin(\alpha_2), \\ u_{10} &= b_1 + W_1 d_2 / \sin(\beta_1), \quad u_{11} = b_1, \quad u_{12} = b_1 + W_2 d_2 / \sin(\beta_2). \end{aligned} \quad (4.120г)$$

Если один из радиусов нулевой, например, $r_1 = 0$, то три вершины ХПС $u_{00} \div u_{02}$ сливаются в точке a_1 , а рациональная поверхность (4.115) упрощается до гранично-линейной конической модели (4.83б) $p(t, \tau) = (1-t)a_1 + th(\tau)$.

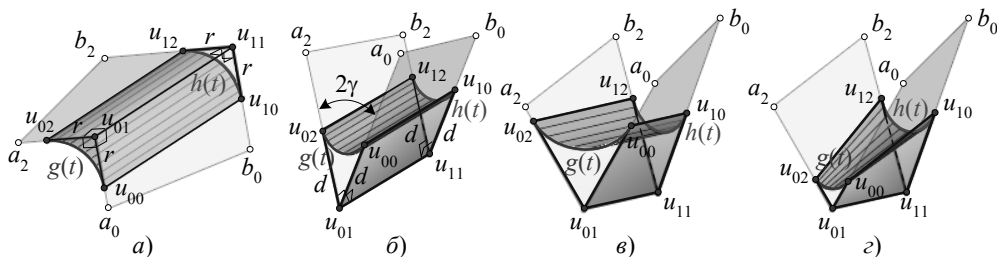


Рис. 4.119

□ Далее рассмотрим два типа **сферического скругления** трёхгранного угла Т в октанте пересечения трёх плоскостей, ограниченном вершиной a_0 и тремя рёбрами a_0a_1 , a_0a_2 и a_0a_3 . Скругляющий сегмент лежит на **сфере**, касающейся угла в трёх точках b_1 , b_2 и b_3 (рис. 4.120). В первой задаче касание происходит по **рёбрам** угла Т ($a, б$), а во второй — по его **граням** ($в, г$). На рисунках ($a, в$) показано вписывание сферы в трёхгранный угол с **прямыми**, а на ($б, г$) — с **произвольными** плоскими углами $\angle a_1a_0a_2$, $\angle a_2a_0a_3$ и $\angle a_3a_0a_1$. Требуется составить характеристические матрицы вершин U и весов Ω , используемые в (4.115) для расчёта поверхности скругления $p(t, \tau)$.

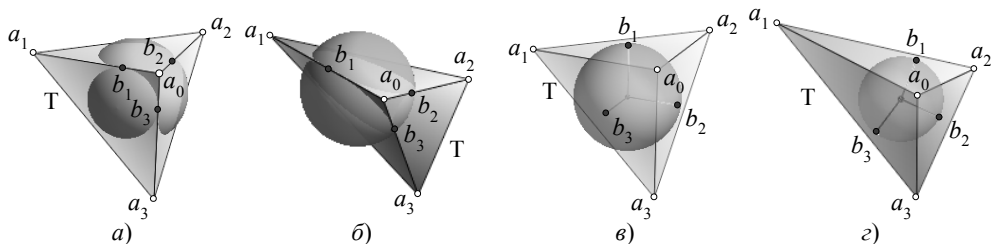


Рис. 4.120

Начнём построение **первого типа скругления** с расчёта нормированных векторов рёбер угла, смотрящих из точки a_0 в вершины $a_1 \div a_3$ (рис. 4.121, а):

$$V_1 = \overline{a_1 - a_0}, \quad V_2 = \overline{a_2 - a_0}, \quad V_3 = \overline{a_3 - a_0}. \quad (4.121a)$$

Затем найдём направление V прямой, соединяющей точку a_0 с центром сферы o , равноудалённым от рёбер a_0a_1 , a_0a_2 и a_0a_3 на расстояние r . Из равенства прямоугольных треугольников a_0ob_i и косинусов углов $\angle VV_i \quad \forall i = \overline{1,3}$ следуют уравнения $V \circ (V_2 - V_1) = 0$ и $V \circ (V_3 - V_1) = 0$ ортогональности вектора V векторам $V_2 - V_1$ и $V_3 - V_1$. Поэтому его можно получить как векторное произведение

$$V = \overline{(V_2 - V_1) \times (V_3 - V_1)}. \quad (4.121б)$$

Он направлен во внешнее пространство трёхгранного угла, если тройка векторов $\{V_1, V_2, V_3\}$ — правая, и во внутреннее, если левая. С помощью расстояния $d = |a_0b_i| = r |\operatorname{ctg}(\angle VV_i)|$ найдём точки b_i касания сферы с рёбрами:

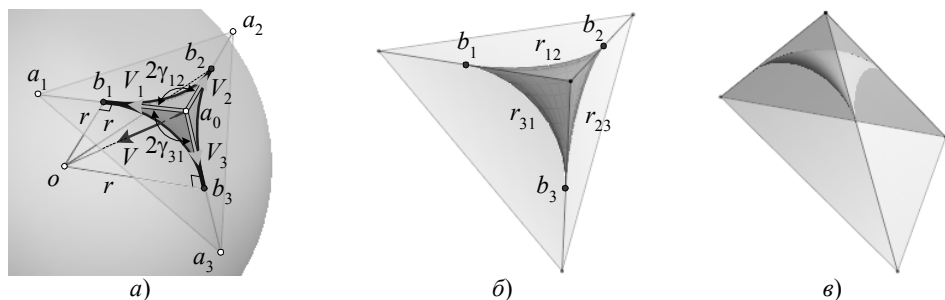


Рис. 4.121

$$c=|V \circ V_1| \Rightarrow d=rc/\sqrt{1-c^2} \Rightarrow b_i=a_0+dV_i \quad \forall i=\overline{1,3}. \quad (4.121\text{в})$$

Граничными линиями скругляющего сегмента являются три дуги окружностей $\overline{b_1b_2}$, $\overline{b_2b_3}$ и $\overline{b_3b_1}$ в сечениях сферы гранями угла Т (см. рис. 4.120,а,б). Составим матрицы вершин и весов ХПС так, чтобы эти дуги строились с весами, соответствующими по (2.54б) плоским углам γ_{ij} :

$$U=\begin{bmatrix} b_1 & a_0 & b_2 \\ a_0 & a_0 & a_0 \\ b_3 & b_3 & b_3 \end{bmatrix}, \quad \Omega=\begin{bmatrix} 1 & \omega_{01} & 1 \\ \omega_{10} & \omega_{11} & \omega_{12} \\ 1 & \omega_{21} & 1 \end{bmatrix}, \quad \text{где } \gamma_{ij}=0.5\arccos(V_i \circ V_j), \quad (4.121\text{г})$$

$$\omega_{10}=\sin(\gamma_{31}), \quad \omega_{12}=\sin(\gamma_{23}), \quad \omega_{01}=\omega_{21}=\sin(\gamma_{12}), \quad \omega_{11}=0.5(\omega_{10}+\omega_{12})\omega_{01}.$$

В отличие от (4.11б) возможны разные веса $\omega_{10} \neq \omega_{12}$ из-за неравенства углов $\gamma_{31} \neq \gamma_{23}$. Радиусы дуг $\overline{b_1b_j}$ находятся как $r_{ij}=d\text{tg}(\gamma_{ij})$:

$$r_{12}=d\omega_{01}/\sqrt{1-\omega_{01}^2}, \quad r_{23}=d\omega_{12}/\sqrt{1-\omega_{12}^2}, \quad r_{13}=d\omega_{10}/\sqrt{1-\omega_{10}^2}. \quad (4.121\text{д})$$

Работа алгоритма проиллюстрирована на рис. 4.121 примерами скругления трёхгранных углов с прямыми (б) и произвольными (в) плоскими углами.

По **второму типу скругления** сфера вдвигается в трёхгранный угол Т до касания его граней в точках $b_1 \div b_3$, равноудалённых от центра сферы o на расстоянии r (рис. 4.122,а). Для нахождения направления V из вершины a_0 в точку o вычислим нормированные векторы рёбер и нормали граней:

$$V_1=\overline{a_1-a_0}, \quad V_2=\overline{a_2-a_0}, \quad V_3=\overline{a_3-a_0}, \quad N_1=\overline{V_1 \times V_2}, \quad N_2=\overline{V_2 \times V_3}, \quad N_3=\overline{V_3 \times V_1}. \quad (4.122\text{а})$$

Из равенства прямоугольных треугольников a_0ob_i и косинусов углов $\angle VN_i$ следуют уравнения $V \circ (N_2 - N_1) = 0$ и $V \circ (N_3 - N_1) = 0$ ортогональности вектора V векторам $N_2 - N_1$ и $N_3 - N_1$. Для того чтобы он был нормирован и направлен **внутрь** телесного угла $\angle V_1V_2V_3$, рассчитаем его как векторное произведение с **инверсией** при левой ориентации тройки векторов $\{V_1, V_2, V_3\}$:

$$V=\text{sgn}(V_1 \times V_2 \circ V_3) \overline{(N_2 - N_1) \times (N_3 - N_1)}. \quad (4.122\text{б})$$

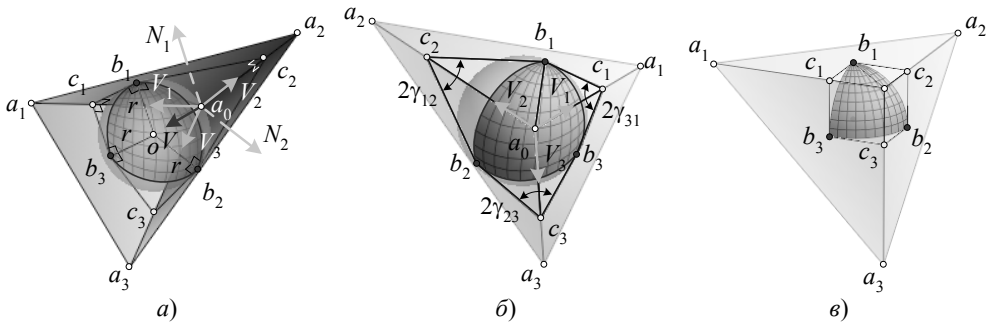


Рис. 4.122

С помощью расстояния $|a_0o| = r / \cos(\angle VN_1)$ найдём точки o и $b_i = o - \text{pr}_{N_i}(o - a_0)$:

$$o = a_0 + rV / |V \circ N_1| \Rightarrow b_i = o - ((o - a_0) \circ N_i)N_i \quad \forall i = \overline{1, 3}. \quad (4.122\text{в})$$

Примем граничными линиями скругляющего сегмента три дуги $\overline{b_1b_2}$, $\overline{b_2b_3}$ и $\overline{b_3b_1}$ в сечениях сферы плоскостями, ортогональными двугранным углом T . Найдём точки $c_i = a_0 + \text{pr}_{V_i}(b_i - a_0)$ пересечения этих плоскостей с рёбрами a_0a_i :

$$c_i = a_0 + ((b_i - a_0) \circ V_i)V_i \quad \forall i = \overline{1, 3}. \quad (4.122\text{г})$$

Составим матрицы характеристических вершин и весов так, чтобы граничные дуги окружностей строились на граничных полилиниях с весами, соответствующими по (2.54б) углам $\gamma_{ij} = 0.5 \arccos(-N_i \circ N_j)$ (рис. 4.122,б):

$$U = \begin{bmatrix} b_1 & b_1 & b_1 \\ c_1 & a_0 & c_2 \\ b_3 & c_3 & b_2 \end{bmatrix}, \quad \Omega = \begin{bmatrix} 1 & \omega_{01} & 1 \\ \omega_{10} & \omega_{11} & \omega_{12} \\ 1 & \omega_{21} & 1 \end{bmatrix}, \quad \text{где } \omega_{10} = \sin(\gamma_{31}), \quad (4.122\text{д})$$

$$\omega_{12} = \sin(\gamma_{12}), \quad \omega_{01} = \omega_{21} = \sin(\gamma_{23}), \quad \omega_{11} = 0.5(\omega_{10} + \omega_{12})\omega_{01}.$$

В этом типе скругления также возможны разные веса $\omega_{10} \neq \omega_{12}$ при $\gamma_{31} \neq \gamma_{12}$.

Работа алгоритма проиллюстрирована на рис. 4.122 примерами скругления трёхгранных углов с произвольными (б) и прямыми (в) двугранными углами.

□ Завершим тему построением **тороидального скругления** выпуклого вогнутого трёхгранного угла с **плоской** невыпуклой гранью $a_0a_1b_1c_1c_0b_0a_0$, торцами $a_0a_1a_2$, $c_0c_1c_2$ и изломом по линии $b_0b_1b_2$ (рис. 4.123,а). Ограничимся конфигурацией с **равными** углами $2\gamma_{12} = 2\gamma_{31}$ между верхней и нижними боковыми гранями, но **произвольным** углом $2\gamma_{23}$ между последними.

Как известно (см. рис. 1.150, 4.54), кривизну тора задают два радиуса — окружности поперечного сечения $r > 0$ и вращения её центра R . Отверстие тора имеет радиус просвета $R - r$. При $R \leq r$ отверстие отсутствует. Ввиду планарности верхней грани тор касается её по окружности радиуса R .

В процессе построения ХПС скругляющего сегмента нам потребуются нормированные векторы рёбер и нормалей к граням угла

$$V_1 = \overline{a_1 - b_1}, \quad V_2 = \overline{c_1 - b_1}, \quad V_3 = \overline{b_2 - b_1}, \quad N_1 = \overline{V_1 \times V_2}, \quad N_2 = \overline{V_1 \times V_3}, \quad N_3 = \overline{V_3 \times V_2}, \quad (4.123\text{а})$$

а также полууглы между рёбрами и гранями

$$\alpha_{ij} = 0.5 \arccos(V_i \circ V_j), \quad \gamma_{ij} = 0.5 \arccos(-N_i \circ N_j). \quad (4.123\text{б})$$

Вычислив расстояния между вершинами ХПС

$$\begin{aligned} d &= |u_{00}u_{10}| = |u_{10}u_{20}| = |u_{02}u_{12}| = |u_{12}u_{22}| = r \text{ctg}(\gamma_{12}), \\ h_1 &= |u_{01}u_{11}| = d / \sin(\alpha_{12}), \quad h_2 = |u_{11}u_{21}| = d / \sin(\gamma_{23}), \\ g_1 &= |u_{00}u_{01}| = |u_{01}u_{02}| = R \text{ctg}(\alpha_{12}), \quad g_2 = |u_{10}u_{11}| = |u_{11}u_{12}| = g_1(1 - d/R), \\ g_3 &= |u_{20}u_{21}| = |u_{21}u_{22}| = g_2 - d \text{ctg}(2\alpha_{31}), \end{aligned} \quad (4.123\text{в})$$

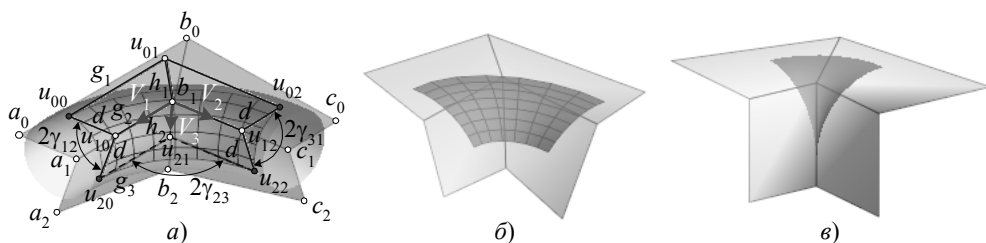


Рис. 4.123

начнём её построение с вершины $u_{11}=b_1$. Установим вершину u_{01} на расстоянии h_1 от точки u_{11} в обратном направлении $-(V_1+V_2)$ биссектрисы угла $2\alpha_{12}$. Отложив от неё расстояния g_1 в направлениях векторов V_1 и V_2 , получим вершины u_{00} и u_{02} . Отложив от b_1 в этих же направлениях расстояния g_2 , получим точки u_{10} и u_{12} , а в направлении V_3 расстояние h_2 — вершину u_{21} . От неё вершины u_{20} и u_{22} устанавливаются на расстоянии g_3 в направлениях V_1 и V_2 .

Веса неугловых вершин ХПС задаются по (2.54б) согласно углам полуохвата граничных дуг $\overline{u_{00}u_{20}}$, $\overline{u_{02}u_{22}}$, $\overline{u_{00}u_{02}}$ и $\overline{u_{20}u_{22}}$. При углах $\alpha_{12} \neq \gamma_{23}$ получатся разные веса $\omega_{01} \neq \omega_{21}$, усредняемые при расчёте веса ω_{11} .

В результате получаем следующие характеристические матрицы скругляющего тороидального сегмента:

$$U = \begin{bmatrix} b_1 - h_1 \overline{V_1 + V_2} + g_1 V_1 & b_1 - h_1 \overline{V_1 + V_2} & b_1 - h_1 \overline{V_1 + V_2} + g_1 V_2 \\ b_1 + g_2 V_1 & b_1 & b_1 + g_2 V_2 \\ b_1 + h_2 V_3 + g_3 V_1 & b_1 + h_2 V_3 & b_1 + h_2 V_3 + g_3 V_2 \end{bmatrix}, \quad (4.123\text{г})$$

$$\Omega = \begin{bmatrix} 1 & \sin(\alpha_{12}) & 1 \\ \sin(\gamma_{12}) & 0.5 \sin(\gamma_{12})(\sin(\alpha_{12}) + \sin(\gamma_{23})) & \sin(\gamma_{12}) \\ 1 & \sin(\gamma_{23}) & 1 \end{bmatrix}.$$

Работа алгоритма иллюстрируется на рис. 4.123 двумя примерами тороидального скругления трёхгранного угла: с произвольными гранями при $R > r$ (б) и ортогональными гранями при $R = r$ (в). В составной поверхности тороидальный сегмент сопрягает цилиндрические скругления рёбер $b_1 a_1$, $b_1 c_1$ и $b_1 b_2$.

4.4.5. Конгруэнтные преобразования моделей поверхностей

Расширим область применения **метода конгруэнтных преобразований**, подробно рассмотренного в п. 2.1.4, на полиномиальные модели поверхностей (4.93в), (4.101а) и (4.112б), обобщённо представляемые бинарным разложением

$$p_{nm}(t, \tau) = T_n^T(t) H^T U H T_m(\tau) \quad \forall t \in [a, b], \quad \tau \in [\alpha, \beta].$$

Задача преобразования модели $p_{nm}(t_1, \tau_1) \Rightarrow p_{n'm'}(t_2, \tau_2)$ с **сохранением формы поверхности** решается [43] приведением параметров $\{t_1, \tau_1\}$ и $\{t_2, \tau_2\}$ в единичные шкалы $t, \tau \in [0, 1]$ путём преобразований ВБФ $T_n(t_1) = M_1 T_n(t)$, $T_n(t_2) = M_2 T_n(t)$, $T_m(\tau_1) = M_1 T_m(\tau)$ и $T_m(\tau_2) = M_2 T_m(\tau)$ с помощью матриц

$$\begin{aligned} M_1 &= D_n(b_1 - a_1, a_1), \quad M_2 = D_{n'}(b_2 - a_2, a_2), \\ M_1 &= D_m(\beta_1 - \alpha_1, \alpha_1), \quad M_2 = D_{m'}(\beta_2 - \alpha_2, \alpha_2), \end{aligned} \quad (4.124a)$$

а по (2.62) — к единым порядкам $\{n', m'\}$. Совмещая равношкальные и равнопорядковые модели $p_{n'm'}(t, \tau)$, получим аналогичное (2.63) уравнение

$$E_{n'+1, n+1} M_1^T H_1^T U_1 H_1 M_1 E_{m+1, m'+1} = M_2^T H_2^T U_2 H_2 M_2, \quad (4.124б)$$

в решениях которого далее будут использованы матрицы

$$\begin{aligned} M &= M_1 E_{n+1, n'+1} M_2^{-1} = D_n(b_1 - a_1, a_1) E_{n+1, n'+1} D_{n'}(b_2 - a_2, a_2)^{-1}, \\ M &= M_1 E_{m+1, m'+1} M_2^{-1} = D_m(\beta_1 - \alpha_1, \alpha_1) E_{m+1, m'+1} D_{m'}(\beta_2 - \alpha_2, \alpha_2)^{-1}. \end{aligned} \quad (4.124в)$$

□ В задаче **преобразования параметрических шкал** $t_1 \in [a_1, b_1] \Rightarrow t_2 \in [a_2, b_2]$ и $\tau_1 \in [\alpha_1, \beta_1] \Rightarrow \tau_2 \in [\alpha_2, \beta_2]$ при **равных** матрицах $U_1 = U_2$ и порядках $n = n'$, $m = m'$ из (4.124а,б) следуют аналогичные (2.60а) формулы пересчёта матриц бинарного разложения:

$$H_2 = H_1 M_1 M_2^{-1}, \quad H_2 = H_1 M_1 M_2^{-1}. \quad (4.125a)$$

Переведём, например, нормализованную модель гиперболического параболоида из примера 4.23 $p_{22}(t_1, \tau_1) = [t_1 - 1 \quad t_1^2 - 2t_1 + 2\tau_1 - \tau_1^2 \quad 1 - \tau_1] \forall t_1, \tau_1 \in [0, 2]$ с матрицами $H_1 = H_1$ из (2.16б) в произвольные симметричные шкалы $t_2, \tau_2 \in [-1, 1]$:

$$\begin{aligned} M_1 &= M_1 = D_2(2, 0) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 4 \end{bmatrix}, \quad M_2 = M_2 = D_2(2, -1) = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 2 & 0 \\ 1 & -4 & 4 \end{bmatrix}, \\ H_2 &= H_2 = \begin{bmatrix} 1 & -1.5 & 0.5 \\ 0 & 2 & -1 \\ 0 & -0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -1 & 2 & 0 \\ 1 & -4 & 4 \end{bmatrix}^{-1} = \begin{bmatrix} 0 & -0.5 & 0.5 \\ 1 & 0 & -1 \\ 0 & 0.5 & 0.5 \end{bmatrix}. \end{aligned}$$

В результате конгруэнтная модель принимает вид, полученный в примере 4.21 для линейной поверхности Кунса:

$$p_{22}(t_2, \tau_2) = \begin{bmatrix} 1 & t_2 & t_2^2 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ -0.5 & 0 & 0.5 \\ 0.5 & -1 & 0.5 \end{bmatrix} \begin{bmatrix} u_{00} & u_{01} & u_{02} \\ u_{10} & u_{11} & u_{12} \\ u_{20} & u_{21} & u_{22} \end{bmatrix} \begin{bmatrix} 0 & -0.5 & 0.5 \\ 1 & 0 & -1 \\ 0 & 0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ \tau_2 \\ \tau_2^2 \end{bmatrix} = \begin{bmatrix} t_2 & t_2^2 - \tau_2^2 & -\tau_2 \end{bmatrix}.$$

□ В задаче **преобразования типа поверхности** (у каждого типа свои параметрические шкалы, узловая матрица U и метод вычисления матриц H и H) из уравнения (4.124б) с **разными** матрицами U_1 и U_2 , но одинаковыми порядками $n = n'$ и $m = m'$, получаем решение а

$$U_2 = (H_1 M_1 M_2^{-1} H_2^{-1})^T U_1 (H_1 M_1 M_2^{-1} H_2^{-1}) = (H_1 M H_2^{-1})^T U_1 (H_1 M H_2^{-1}). \quad (4.125б)$$

Заметим, что в (4.113) выведена сходная формула конгруэнтного изменения **порядков** поверхности Безье с матрицами $M = E_{n+1, n'+1}$ и $M = E_{m+1, m'+1}$.

Конгруэнтное преобразование одной полиномиальной модели в другую равнопорядковую модель производится подстановкой в (4.125б) матриц $\{U_1, H_1, N_1, M_1, M_1\}$ и $\{U_2, H_2, N_2, M_2, M_2\}$, соответствующих типам поверхностей и их шкалам:

- **интерполяционная** модель (4.93в) с узловой матрицей U из (4.96б), матрицами H и N из (4.97а) и матрицами $M = D_n(t_n - t_0, t_0)$, $M = D_m(\tau_m - \tau_0, \tau_0)$;

- **эрмитова** модель (4.101а) с матрицами U_Σ , H и N из (4.102в) и матрицами $M = D_n(t_{n-2} - t_0, t_0)$, $M = D_m(\tau_{m-2} - \tau_0, \tau_0)$;

- модель **Кунса**, отличающаяся от эрмитовой лишь матрицей U_K , содержащей аналогично (4.92) в нижнем правом блоке векторы угловых кручений K_{00} , $K_{n-2,0}$, $K_{0,m-2}$ и $K_{n-2,m-2}$;

- четыре **циклические** модели, отличающиеся от эрмитовой приведёнными в (4.103) матрицами $U_\square$, $Q_\square = H_\square^{-1}$, $\Theta_\square = N_\square^{-1}$ и разностями порядков полинома и узловой матрицы U , определяющими вид матриц M и M . Например, у t -циклической модели с элементами (4.103а) будут параметрические шкалы $t \in [t_0, t_{n-1}]$, $\tau \in [\tau_0, \tau_m]$ и матрицы $M = D_n(t_{n-1} - t_0, t_0)$, $M = D_m(\tau_m - \tau_0, \tau_0)$;

- модель **Безье** (4.112б) с ХПС U_B , матрицами H и N из (2.38в) и единичными матрицами $M = D_n(1,0) = E_{n+1}$, $M = D_m(1,0) = E_{m+1}$.

Результаты четырёх преобразований показаны на рис. 4.124, где построены характеристические сети Безье (а)–(в) для интерполяционной (см. рис. 4.97,а), эрмитовой (см. рис. 4.91,в) и циклической (см. рис. 4.102) поверхностей, а также интерполяционная сеть (г) для поверхности Безье, изображённой на рис. 4.108,в.

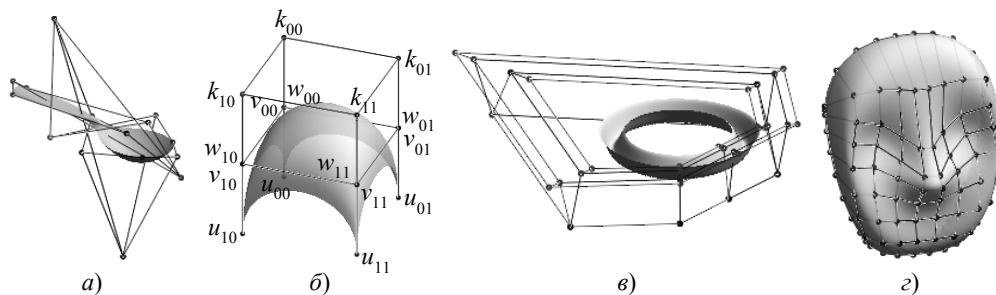


Рис. 4.124

□ Ну и, наконец, общая задача конгруэнтного преобразования модели полиномиальной поверхности $p_{nm}(t_1, \tau_1) \Rightarrow p_{n'm'}(t_2, \tau_2)$, включая пересчёт узловой матрицы $U_1 \Rightarrow U_2$ и **преобразования порядков** $n \Rightarrow n'$ и $m \Rightarrow m'$, имеет похожее на (4.125б) решение с матрицами M и M из (4.124в):

$$U_2 = (H_1 M_1 E_{n+1, n'+1} M_2^{-1} H_2^{-1})^T U_1 (H_1 M_1 E_{m+1, m'+1} M_2^{-1} H_2^{-1}) = (H_1 M H_2^{-1})^T U_1 (H_1 M H_2^{-1}).$$

Изменение каждого порядка n и/или m возможно как в сторону повышения, так и понижения, но так или иначе всегда **фактические степени полинома ос-**

таются неизменными. Увеличение одного или обоих порядков модели добавляет в узловую сеть новые вершины и повышает потенциальную изгибаемость поверхности, недостижимую при меньших порядках. После перемещения дополнительных вершин фактические степени полиномов увеличиваются, а дефекты степеней уменьшаются либо исчезают совсем. Понижение порядков полиномиальной модели может быть полезным для оптимизации объёма данных, достаточного для хранения и воспроизведения поверхности.

Работа алгоритма изменения порядков характеристической сети поверхности Безье проиллюстрирована на рис. 4.111:

- исходная поверхность (а) порядков $n=4$ и $m=3$ построена на сетке размеров 5×4 узлов и имеет в матрице коэффициентов $S = H^T U N$ одну нижнюю нулевую строку, как в первой матрице (4.114б). Это означает, что степень n имеет дефект $d=1$, а фактический t -порядок полинома $p_{4,3}(t, \tau)$ составляет $n' = n - 1 = 3$;
- возможность уменьшения формального порядка $n=4$ до фактического $n'=3$ подтверждает конгруэнтная поверхность (б), построенная на сетке минимальных узловых размеров 4×4 . Матрица коэффициентов такой поверхности не имеет нулевых последних ни строк, ни столбцов, и никакой из порядков не может быть понижен;
- третья конгруэнтная поверхность (в) демонстрирует возможность одновременного понижения у поверхности (а) формального t -порядка $n=4$ до фактического значения $n'=3$ и повышения τ -порядка $m=3$ до формального $m'=6$ с дефектом степени $m' - m = 3$.

4.4.6. Остовные поверхности

В машиностроительной практике конструирования корпусов различных плавающих и летающих изделий с давних времён существует метод проведения поверхности через систему *остовных линий*, которые чаще всего задают *поперечные* и/или *продольные* сечения корпуса. Термины «продольный» и «поперечный» условны и предполагают наличие у изделия осевой линии либо направления движения и разные габариты вдоль и поперёк этой оси.

До наступления компьютерной эпохи изготовление корпуса происходило

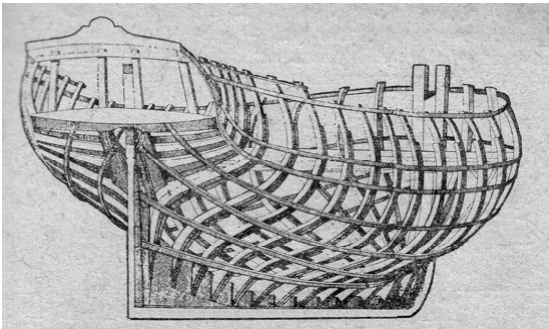


Рис. 4.125

так: по вручную созданным чертежам в натуральную величину изготавливался **набор** силовых элементов корпуса — шпангоутов и стрингеров (рис. 4.125), который затем обшивался изогнутыми листами корпусного материала. В современных САПР конструирование ведётся по математическим моделям остовных линий, из которых создаётся модель обтягивающей их **остовой**

поверхности. Она позволяет построить реалистичное изображение поверхности на экране дисплея и выполнить операции, необходимые для её изготовления в реальном производстве.

Введём обозначения, используемые в различных моделях остоновых поверхностей $p(t, \tau)$:

- $n \geq 1$ и $m \geq 1$ — количества промежутков между поперечными $c_i(\tau)$ и продольными $a_j(t)$ линиями, пронумерованными индексами $i = \overline{0, n}$ и $j = \overline{0, m}$;
- $\{t_0, t_1, \dots, t_n\}$ — список отсчётов параметра $t \in [t_0, t_n]$, фиксирующих поперечные линии $c_i(\tau)$, составляющие набор $C(\tau) = [c_0(\tau) \ c_1(\tau) \ \dots \ c_n(\tau)]$;
- $\{\tau_0, \tau_1, \dots, \tau_m\}$ — список отсчётов параметра $\tau \in [\tau_0, \tau_m]$, фиксирующих продольные линии $a_j(t)$, составляющие набор $A(t) = [a_0(t) \ a_1(t) \ \dots \ a_m(t)]$.

Все линии одного набора должны иметь единые параметризацию (единичную, нормализованную либо хордовую) и интервал изменения параметра. Если не так, то следует унифицировать её по (4.84). Методы получения остоновых линий могут быть самыми разными — от прямого задания координатных функций до применения алгоритмов интерполяции и приближения. При использовании обоих наборов $c_0(\tau) \div c_n(\tau)$ и $a_0(t) \div a_m(t)$ входящие в них линии должны удовлетворять достаточно жёстким условиям взаимного пересечения в общих узловых точках:

$$u_{ij} = c_i(\tau_j) = a_j(t_i) \quad \forall i = \overline{0, n}, \quad j = \overline{0, m}. \quad (4.126)$$

Справедлив вопрос: зачем нужен ещё один метод интерполяции, если можно вычислить точки пересечения остоновых линий u_{ij} и на полученной узловой сети строить поверхность по (4.93). Ответ, однако, прост: границы созданной этим способом поверхности совпадают с заданными крайними остоновыми линиями $c_0(\tau)$, $c_n(\tau)$, $a_0(t)$ и $a_m(t)$ лишь в конечном множестве $2(n+m+1)$ точек, а не на всем их протяжении. И тогда, например, при конструировании непрерывного соединения двух поверхностей с присущими им краями произойдёт нестыковка и разрыв составной поверхности на линиях сопряжения.

Интерполяция одного набора остоновых линий

Рассмотрим задачу построения поверхности на одном наборе остоновых линий, например, на поперечных $c_0(\tau) \div c_n(\tau)$ (рис. 4.126). При $n=1$ получаем частный случай гранично-линейной интерполяции (см. п. 4.4.1), приводящей к модели линейчатой поверхности (4.83а) с двумя направляющими $c_0(\tau) \equiv g(\tau)$ и $c_1(\tau) \equiv h(\tau)$ (см. рис. 4.84).

Формально заменив в (2.7) точки u_i на параметрические линии $c_i(\tau)$, видим идентичность методов синтеза ин-

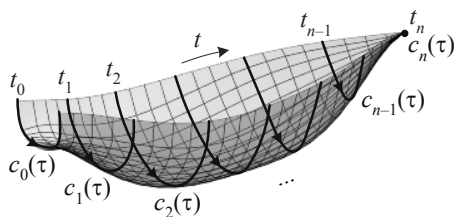


Рис. 4.126

терполяционных линии $p(t)$ и поверхности $p(t, \tau)$. Поэтому сходны с (2.7) и формы бипараметрических разложений:

- по **остовным линиям** в форме Лагранжа:

$$p(t, \tau) = \sum_{i=0}^n c_i(\tau) l_{ni}(t) = C(\tau) L_n(t), \quad (4.127a)$$

где $L_n(t) = [l_{n0}(t) \ l_{n1}(t) \ \dots \ l_{nn}(t)]^T$ — столбцовый **вектор весовых функций** $l_{ni}(t)$, определённых в (2.11) полиномами степени n и характеризующих вклад линий $c_i(\tau)$ в суммарное значение $p(t, \tau)$;

- по **базисным функциям** параметра t в форме Фергюсона:

$$p(t, \tau) = \sum_{i=0}^n s_i(\tau) t^i = S_c(\tau) T_n(t), \quad (4.127б)$$

где $S_c(\tau) = [s_0(\tau) \ s_1(\tau) \ \dots \ s_n(\tau)]$ — **вектор параметрических функций** $s_i(\tau)$, зависящих от расположения поперечных остовных линий. При изменении любой из них весь вектор $S_c(\tau)$ пересчитывается заново;

- **бинарное разложение** по остовным линиям и базисным функциям:

$$p(t, \tau) = \sum_{i=0}^n \sum_{j=0}^n h_{ij} c_i(\tau) t^j = C(\tau) H_n T_n(t), \quad (4.127в)$$

где $H_n = Q_n^{-1}$ — числовая **матрица весовых коэффициентов**, обратная к матрице Вандермонда $Q_n = [T_n(t_0) \ T_n(t_1) \ \dots \ T_n(t_n)]$, составленной по (2.13в). С её помощью по (2.8) находятся параметрические векторы $L_n(t) = H_n T_n(t)$ и $S_c(\tau) = C(\tau) H_n$, используемые в (4.127а,б) для расчёта точек $p(t, \tau)$.

Векторы параметрических производных модели (4.127а)

$$V(t, \tau) = C(\tau) L'_n(t), \quad W(t, \tau) = C'(\tau) L_n(t) \quad (4.127г)$$

касаются поверхности, соответственно, в продольном и поперечном направлениях. Поскольку столбец $L_n(t_i)$ содержит лишь одну единицу в i -м элементе, то векторы $W_i(\tau) = C'(\tau) L_n(t_i) = c'_i(\tau)$ касательны к остовным линиям $c_i(\tau)$.

Аналогично (4.127) устроены модели поверхности, затягивающей продольный набор остовных линий $a_0(t) \div a_m(t)$:

$$\begin{aligned} p(t, \tau) &= A(t) L_m(\tau) = S_a(t) T_m(\tau) = A(t) H_m T_m(\tau), \\ V(t, \tau) &= A'(t) L_m(\tau), \quad W(t, \tau) = A(t) L'_m(\tau), \end{aligned} \quad (4.128)$$

где $H_m = [T_m(\tau_0) \ T_m(\tau_1) \ \dots \ T_m(\tau_m)]^{-1}$ — матрица весовых коэффициентов, связывающая параметрические векторы $L_m(\tau) = H_m T_m(\tau)$ и $S_a(t) = A(t) H_m$.

 **Пример 4.27.** Сконструировать поверхность храмового купола-луковки, построенного на квадратной крыше.

Решение. Зададим набор поперечных остовных линий в горизонтальных сечениях поверхности купола, пронумерованных в направлении снизу вверх (рис. 4.127):

• нижний остовный квадрат с вершинами в точках $p_1=[10\ 0\ 0]$, $p_2=[0\ 0\ 10]$, $p_3=[-10\ 0\ 0]$ и $p_4=[0\ 0\ -10]$. Составим **параметрическую модель четырёхугольника** $p_1p_2p_3p_4p_1$ как замкнутую полилинию со сквозной параметризацией $\tau \in [0,4]$ и линейными моделями отрезков в виде (2.106):

$$c_0(\tau) = \begin{cases} (1-\tau)p_1 + \tau p_2 & \forall 0 \leq \tau < 1; \\ (2-\tau)p_2 + (\tau-1)p_3 & \forall 1 \leq \tau < 2; \\ (3-\tau)p_3 + (\tau-2)p_4 & \forall 2 \leq \tau < 3; \\ (4-\tau)p_4 + (\tau-3)p_1 & \forall 3 \leq \tau \leq 4; \end{cases} \quad (4.129)$$

- пять остовных окружностей

$$c_i(\tau) = [r_i \cos(\pi\tau/2) \ y_i \ r_i \sin(\pi\tau/2)] \quad \forall i=1,5$$

с радиусами $r_1=3$, $r_2=3$, $r_3=5$, $r_4=4$, $r_5=1$ и высотами $y_1=3$, $y_2=5$, $y_3=7$, $y_4=10$, $y_5=14$. Интервал изменения параметра $\tau \in [0,4]$ выбран как у $c_0(\tau)$;

- верхняя остовная точка $c_6=[0\ 18\ 0]$.

Из шести остовных линий и точки был составлен набор $C(\tau)=[c_0(\tau) \dots c_5(\tau) \ c_6]$, выбрана нормализованная параметризация $t_j=j \ \forall j=\overline{0,6}$, по (2.13в) составлена матрица Вандермонда Q_6 с элементами $q_{ij}=j^i$, вычислена обратная матрица $H_6=Q_6^{-1}$ и по (4.127в) построена поверхность купола $p(t, \tau)$, $t \in [0,6]$, $\tau \in [0,4]$, показанная на рис. 4.127. $\square$

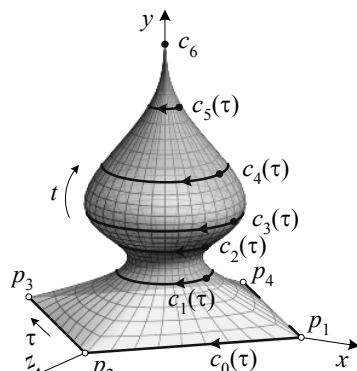


Рис. 4.127

Интерполяция двух наборов остовных линий

Более сложным является затягивание поверхностью двух наборов остовных линий — продольных $a_0(t) \div a_m(t)$ и поперечных $c_0(\tau) \div c_n(\tau)$ (рис. 4.128). При $n=1$ и $m=1$ получается частный случай граничной интерполяции (см. рис. 4.86) с разными вариантами модели линейной поверхности Кунса, приведёнными в (4.85).

Сложность задачи кроется в обеспечении пересечения каждой линии одного набора с каждой линией другого набора в общих узловых точках (4.126). Можно, конечно, воссоздать остовные линии путём интерполяции вершин узловой сети $U=\{u_{ij}\}$, через кото-

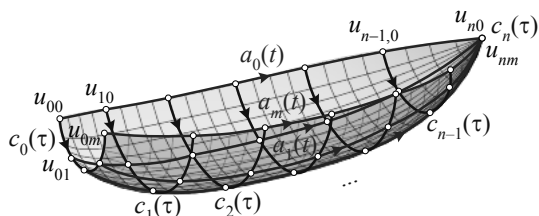


Рис. 4.128

рые они проходят — линии $a_j(t)$ по точкам $\{u_{0j} \div u_{nj}\}$, а линии $c_i(\tau)$ по точкам $\{u_{i0} \div u_{im}\}$, — а затем строить на них остовную поверхность. Однако, её модель (см. ниже) оказывается сложнее интерполяционных моделей (4.93), напрямую использующих узловую матрицу U . Кроме того, интерполяционные линии могут иметь нежелательные межузловые осцилляции, особенно сильные на крайних интервалах. Вот почему **остовные линии $a_j(t)$ и $c_i(\tau)$ должны быть зада-**

ны точными аналитическими зависимостями, возможно и не полиномиальными, но обеспечивающими их взаимное пересечение в общих узлах u_{ij} .

Повторяя ход рассуждений, приведших к модели линейной поверхности Кунса (4.85а), получим её полиномиальное обобщение, называемое **поверхностью Гордона** [13, 20, 56], в следующих формах:

$$p(t, \tau) = \sum_{i=0}^n l_{ni}(t) c_i(\tau) + \sum_{j=0}^m l_{mj}(\tau) a_j(t) - \sum_{i=0}^n \sum_{j=0}^m l_{ni}(t) l_{mj}(\tau) u_{ij} =$$

$$= C(\tau) L_n(t) + A(t) L_m(\tau) - L_n^T(t) U L_m(\tau) = \begin{bmatrix} L_n^T(t) & 1 \end{bmatrix} \begin{bmatrix} -U & C^T(\tau) \\ A(t) & O \end{bmatrix} \begin{bmatrix} L_m(\tau) \\ 1 \end{bmatrix}, \quad (4.130a)$$

где $L_n(\tau) = [l_{n0}(\tau) \dots l_{nn}(\tau)]^T$ и $L_m(\tau) = [l_{m0}(\tau) \dots l_{mm}(\tau)]^T$ — векторы весовых функций Лагранжа (2.11а), задающих вклад остовных линий и точек их пересечения в суммарное значение $p(t, \tau)$. А если вычислить матрицы $H_n = Q_n^{-1}$ и $H_m = Q_m^{-1}$, обратные матрицам Вандермонда (4.96в), и с их помощью два параметрических ВПК $S_c(\tau) = C(\tau) H_n$, $S_a(t) = A(t) H_m$ и числовую МПК $S_u = H_n^T U H_m$, то можно составить ещё две формы описания поверхности Гордона:

$$p(t, \tau) = S_c(\tau) T_n(t) + S_a(t) T_m(\tau) - T_n^T(t) S_u T_m(\tau) = \begin{bmatrix} T_n^T(t) & 1 \end{bmatrix} \begin{bmatrix} -S_u & S_c^T(\tau) \\ S_a(t) & O \end{bmatrix} \begin{bmatrix} T_m(\tau) \\ 1 \end{bmatrix}. \quad (4.130б)$$

Направляющие векторы этой поверхности вычисляются по формулам

$$V(t, \tau) = C(\tau) L'_n(t) + A'(t) L_m(\tau) - L_n^T(t) U L_m(\tau),$$

$$W(t, \tau) = C'(\tau) L_n(t) + A(t) L'_m(\tau) - L_n^T(t) U L'_m(\tau), \quad (4.131a)$$

при этом векторы

$$V_j(t) = C(\tau_j) L'_n(t) + A'(t) L_m(\tau_j) - L_n^T(t) C^T(\tau_j) = a'_j(t) \quad \forall j = \overline{0, m},$$

$$W_i(\tau) = C'(\tau) L_n(t_i) + A(t_i) L'_m(\tau) - L_m^T(\tau) A^T(t_i) = c'_i(\tau) \quad \forall i = \overline{0, n} \quad (4.131б)$$

касательны, соответственно, к продольным и поперечным остовным линиям.

С помощью двух наборов остовных линий можно более целенаправленно формировать профиль поверхности в сравнении с тем, что даёт интерполяция одного набора. Например, каждая продольная каркасная линия поверхности на рис. 4.127 проходит через семь точек на линиях $c_0(\tau) \div c_6(\tau)$ и по законам интерполяции изгибается между ними как полином шестой степени, придающий куполу характерный абрис. Небольшие изменения радиусов и вертикальные смещения центров остовных окружностей сильно трансформируют продольный профиль поверхности вплоть до её самопересечения. Так что достижение желаемой формы полиномиальной поверхности лишь по одному набору остовных линий может оказаться весьма сложной задачей.

 **Пример 4.28.** Построить поверхность купола из примера 4.27, у которой продольные линии, выходящие из вершин остовного квадрата $c_0(\tau)$, имели форму полилиний $u_{0j} u_{1j} \dots u_{5j} c_6$, где $u_{ij} = c_i(j)$, $i = \overline{0, 5}$, $j = \overline{0, 4}$ (рис. 4.129).

Решение. Составим на основе нелокального варианта интерполяционной модели отрезка (2.106) аналогичные (4.129) **параметрические модели полилиний** $a_j(t)$ в интервале изменения параметра $t \in [0, 6]$:

$$a_j(t) = (i+1-t)u_{ij} + (t-i)u_{i+1,j} \quad \forall j = \overline{0,4},$$

где $i = \lfloor t \rfloor - \{t=6\}$ — номер отрезка полилинии, получаемый округлением параметра $0 \leq t < 6$ до ближайшего меньшего целого и равный пяти при $t=6$. Приняв эти остовные линии за продольные и взяв поперечные линии $c_j(\tau)$ из примера 4.27, подставим их вместе с узловыми точками u_{ij} в (4.130а). В результате получается желаемая поверхность, изображённая на рис. 4.129.

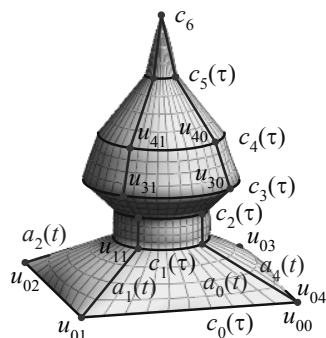


Рис. 4.129

Эрмитова остовная поверхность

Наиболее трудоёмким как в подготовке исходных данных, так и в решении, является конструирование остовой поверхности с желаемыми наклонами краёв в трансверсальных направлениях к граничным линиям. Таким дополнительным требованиям должна удовлетворять, например, **поверхность перехода**, гладко сопряжённая с краями двух соединяемых поверхностей и проходящая, возможно, ещё через некоторые остовные линии. Построение таких поверхностей актуально во многих задачах машиностроения, архитектуры, дизайна и т. д.

Пусть, например, заданы **поперечные** линии $c_0(\tau) \div c_n(\tau)$ и аналитические зависимости векторов $V_0(\tau) = p'_t(t, \tau)|_{t_0}$ и $V_n(\tau) = p'_t(t, \tau)|_{t_n}$, касающихся краёв остовой поверхности $c_0(\tau)$ и $c_n(\tau)$ в трансверсальных (в данном случае — **продольных**) направлениях (рис. 4.130). Пара дополнительных функциональных ограничений в t -направлении увеличивает порядок полиномиальной модели $p(t, \tau)$ до $v = n + 2$, тогда как в τ -направлении степень полинома остается равной m .

Формально заменив в (2.18а) точки u_i на линии $c_i(\tau)$, а векторы V_0 и V_n на векторные функции $V_0(\tau)$ и $V_n(\tau)$, замечаем сходство двух эрмитово интерполированных объектов — линии $p(t)$ и поверхности $p(t, \tau)$. Сходны и формы параметрических разложений

$$p(t, \tau) = C_{\mathfrak{Z}}(\tau) L_{\mathfrak{Z}}(t) = C_{\mathfrak{Z}}(\tau) H_{\mathfrak{Z}} T_v(t) = S_c(\tau) T_v(t) \quad (4.132a)$$

с матрицей $H_{\mathfrak{Z}} = [Q_{\text{св}} | T'_v(t_0) \ T'_v(t_n)]^{-1}$, в вычислении которой используется матрица $Q_{\text{св}}$ из (4.96в), и параметрическими векторами узловых элементов и параметрических коэффициентов

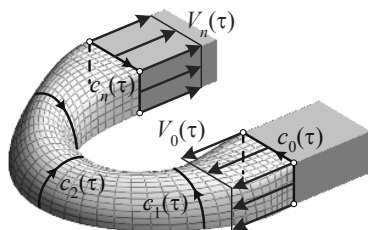


Рис. 4.130

$$C_{\mathfrak{Z}}(\tau)=[C(\tau)|V_0(\tau) V_n(\tau)], \quad S_c(\tau)=C_{\mathfrak{Z}}(\tau)H_{\mathfrak{Z}}. \quad (4.132б)$$

Аналогично устроены модели поверхности, затягивающей **продольный** набор остовных линий $a_0(t) \div a_m(t)$ с краевыми касательными векторами $W_0(t)$, $W_m(t)$ и степенью полинома в τ -направлении $\mu = m + 2$:

$$p(t, \tau) = A_{\mathfrak{Z}}(t)L_{\mathfrak{Z}}(\tau) = A_{\mathfrak{Z}}(t)H_{\mathfrak{Z}}T_{\mu}(\tau) = S_a(t)T_{\mu}(\tau) \quad (4.133а)$$

с матрицей $H_{\mathfrak{Z}} = [\Theta_{\text{св}} | T'_{\mu}(\tau_0) \ T'_{\mu}(\tau_m)]^{-1}$, в расчёте которой используется матрица $\Theta_{\text{св}}$ из (4.96в), и параметрическими векторами

$$A_{\mathfrak{Z}}(t) = [A(t) | W_0(t) \ W_m(t)], \quad S_a(t) = A_{\mathfrak{Z}}(t)H_{\mathfrak{Z}}. \quad (4.133б)$$

При построении эрмитовой поверхности, проходящей через **два набора**

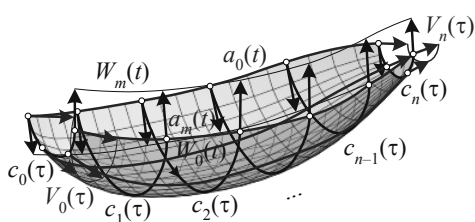


Рис. 4.131

остовных линий $a_0(t) \div a_m(t)$ и $c_0(\tau) \div c_n(\tau)$ с функциональными векторами краевых направлений $V_0(\tau)$, $V_n(\tau)$, $W_0(t)$ и $W_m(t)$ (рис. 4.131) нужно удовлетворить жёстким требованиям равенства выборок трансверсальных направлений с касательными векторами на концах остовных линий:

$$V_0(\tau_j) = a'_j(t_0), \quad V_n(\tau_j) = a'_j(t_n) \quad \forall j = \overline{0, m},$$

$$W_0(t_i) = c'_i(\tau_0), \quad W_m(t_i) = c'_i(\tau_m) \quad \forall i = \overline{0, n}.$$

Если это выполнено, то составляются аналогичные (4.130) модели двухнаборной остовой поверхности

$$\begin{aligned} p(t, \tau) &= C_{\mathfrak{Z}}(\tau)L_{\mathfrak{Z}v}(t) + A_{\mathfrak{Z}}(t)L_{\mathfrak{Z}\mu}(\tau) - L_{\mathfrak{Z}v}^T(t)U_{\mathfrak{Z}}L_{\mathfrak{Z}\mu}(\tau) = \\ &= S_c(\tau)T_v(t) + S_a(t)T_{\mu}(\tau) - T_n^T(t)S_uT_{\mu}(\tau) = \\ &= C_{\mathfrak{Z}}(\tau)H_{\mathfrak{Z}}T_v(t) + A_{\mathfrak{Z}}(t)H_{\mathfrak{Z}}T_{\mu}(\tau) - T_n^T(t)H^{\mathfrak{Z}T}U_{\mathfrak{Z}}H_{\mathfrak{Z}}T_{\mu}(\tau) \end{aligned} \quad (4.134а)$$

с матрицами $H_{\mathfrak{Z}} = [Q_{\text{св}} | T'_v(t_0) \ T'_v(t_n)]^{-1}$, $H_{\mathfrak{Z}} = [\Theta_{\text{св}} | T'_{\mu}(\tau_0) \ T'_{\mu}(\tau_m)]^{-1}$, сходной с приведённой в (4.102в) узловой эрмитовой матрицей

$$U_{\mathfrak{Z}} = \left[\begin{array}{ccc|cc} u_{00} & \dots & u_{0m} & W_0(t_0) & W_m(t_0) \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ u_{n0} & \dots & u_{nm} & W_0(t_n) & W_m(t_n) \\ \hline V_0(\tau_0) & \dots & V_0(\tau_m) & O & O \\ V_n(\tau_0) & \dots & V_n(\tau_m) & O & O \end{array} \right] \quad (4.134б)$$

и векторами $A_{\mathfrak{Z}}(t)$, $C_{\mathfrak{Z}}(\tau)$, $S_a(t)$, и $S_c(\tau)$, определёнными в (4.132б) и (4.133б).

Пример 4.29. Построить поверхность купола (см. рис. 4.127), гладко сопряжённую с вертикальными стенами здания (рис. 4.132).

Решение. Добавим к набору поперечных остовных линий $c_0(\tau) \div c_6(\tau)$, определённых в примере 4.27, направленные вертикально вверх касательные векторы $V_0(\tau)=[0 \ 10 \ 0]$ нижнего и $V_6=[0 \ 1 \ 0]$ верхнего краёв строящейся поверхности. Длины векторов подобраны равными 10 и 1 с учётом средних расстояний между нижней и верхней парами узловых элементов $\{c_0(\tau), c_1(\tau)\}$ и $\{c_5(\tau), c_6(\tau)\}$.

Расширив вектор $C(\tau)$ до $C_3(\tau)=[C(\tau) \ V_0(\tau) \ V_6]$ и вычислив матрицу $H_3=[Q_{86} \ T_8'(0) \ T_8'(6)]^{-1}$, составляем параметрическую модель эрмитовой остовной поверхности купола $p(t, \tau)=C_3(\tau)H_3T_8(t)$, $t \in [0, 6]$, $\tau \in [0, 4]$, изображение которой построено на рис. 4.132. $\square$

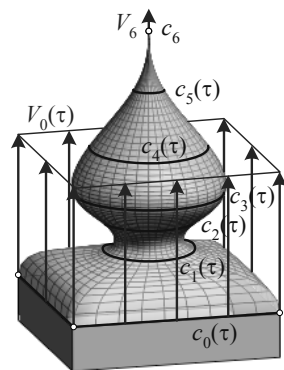


Рис. 4.132

Остовные поверхности Бэзе

Распространим метод получения остовных интерполяционных поверхностей из интерполяционных линий, состоящий в замене вершин узловой сети параметрическими остовными линиями, на вывод моделей **остовных поверхностей Бэзе**. Второй способ их получения заключается в замене векторных функций Лагранжа функциями Бернштейна, а обратных матриц Вандермонда — матрицами H_n и H_m с элементами $h_{ij}=(-1)^{j-i}C_k^jC_j^i \ \forall 0 \leq i \leq j \leq k, \ k \in \{n, m\}$.

$\square$ Симбиоз (2.38) с (4.127) даёт модели **одноостовой** поверхности Бэзе, построенной на **поперечном** наборе $C(\tau)=[c_0(\tau) \ c_1(\tau) \ \dots \ c_n(\tau)]$:

$$p(t, \tau)=C(\tau)B_n(t)=S_c(\tau)T_n(t)=C(\tau)H_nT_n(t) \ \forall t \in [0, 1], \ \tau \in [\tau_0, \tau_m]. \quad (4.135a)$$

С двух сторон этот сегмент (рис. 4.133) ограничен остовными линиями $\{c_0(\tau), c_n(\tau)\}$, а с двух других сторон — линиями Бэзе (2.38a), построенными на базе ХП $c_0(\tau_0) \dots c_n(\tau_0)$ и $c_0(\tau_m) \dots c_n(\tau_m)$. Две остовные линии $c_1(\tau)$ и $c_{n-1}(\tau)$ управляют функциональными векторами продольных краевых направлений

$$V_0(\tau)=n(c_1(\tau)-c_0(\tau)), \quad V_n(\tau)=n(c_n(\tau)-c_{n-1}(\tau)).$$

Аналогично (4.135a) на **продольном** наборе $A(t)=[a_0(t) \ a_1(t) \ \dots \ a_m(t)]$ строится одноостовная поверхность Бэзе

$$p(t, \tau)=A(t)B_m(\tau)=S_a(t)T_m(\tau)=A(t)H_mT_m(\tau) \ \forall t \in [t_0, t_n], \ \tau \in [0, 1], \quad (4.135b)$$

ограниченная остовными линиями $\{a_0(t), a_m(t)\}$ и линиями Бэзе на базе ХП $a_0(t_0) \dots a_m(t_0)$ и $a_0(t_n) \dots a_m(t_n)$. Две остовные линии $\{a_1(t), a_{m-1}(t)\}$ управляют функциональными векторами поперечных краевых направлений

$$W_0(t)=m(a_1(t)-a_0(t)), \quad W_m(t)=m(a_m(t)-a_{m-1}(t)).$$

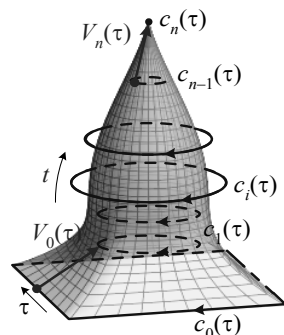


Рис. 4.133

□ Отталкиваясь от (4.130), построим модель **двухостовой** поверхности Безье на продольном и поперечном наборах $A(t)$ и $C(\tau)$ (рис. 4.134):

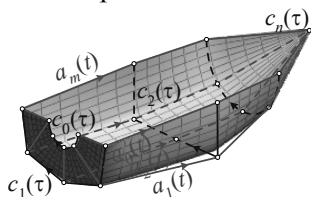


Рис. 4.134

- заменим ВФ Лагранжа $L_n(t)$ и $L_m(\tau)$ векторами ВФ Бернштейна $B_n(t)$ и $B_m(\tau)$;
- нормируем по (4.84) интервалы изменения параметров $t \in [t_0, t_n]$ и $\tau \in [\tau_0, \tau_m]$;
- из точек (4.126) пересечения остовных линий $c_i(\tau)$ и $a_j(t)$ составим матрицу $U = \{u_{ij}\}$.

Результат этих операций представлен следующими моделями:

$$\begin{aligned} p(t, \tau) &= [B_n^T(t) \ 1] \begin{bmatrix} -U & C^T(\tau_c) \\ A(t_a) & O \end{bmatrix} \begin{bmatrix} B_m(\tau) \\ 1 \end{bmatrix} = \\ &= [T_n^T(t) \ 1] \begin{bmatrix} -H_n^T U H_m & H_n^T C^T(\tau_c) \\ A(t_a) H_m & O \end{bmatrix} \begin{bmatrix} T_m(\tau) \\ 1 \end{bmatrix}, \end{aligned} \quad (4.136)$$

где $t, \tau \in [0, 1]$, $t_a = t_0 + (t_n - t_0)t$, $\tau_c = \tau_0 + (\tau_m - \tau_0)\tau$.

Поверхность (4.136) со всех сторон ограничена крайними остовными линиями $\{a_0(t_a), a_m(t_a)\}$ и $\{c_0(\tau_c), c_n(\tau_c)\}$, а линии $\{a_1(t_a), a_{m-1}(t_a)\}$ и $\{c_1(\tau_c), c_{n-1}(\tau_c)\}$ управляют крайними направлениями поверхности $\{V_0(\tau), V_n(\tau)\}$ и $\{W_0(t), W_m(t)\}$.

□ Наделив остовные линии $c_i(\tau)$ и $a_j(t)$ весами ω_i и ϖ_j (веса крайних линий каждого набора задаются единичными), сформируем из однородных форм $\overset{\circ}{c}_i(\tau) = \omega_i \tilde{c}_i(\tau)$ и $\overset{\circ}{a}_j(t) = \varpi_j \tilde{a}_j(t)$ наборы векторов $\overset{\circ}{C}(\tau)$ и $\overset{\circ}{A}(t)$, входящих по аналогии с (4.135) в модели **рациональных одноостовных** поверхностей Безье:

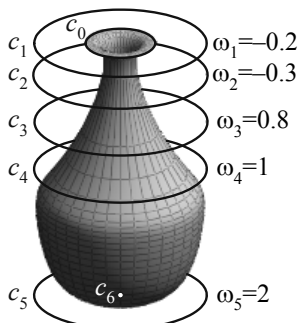


Рис. 4.135

- на поперечном наборе (рис. 4.135)

$$\begin{aligned} \overset{\circ}{p}(t, \tau) &= \overset{\circ}{C}(\tau) B_n(t) = \overset{\circ}{C}(\tau) H_n T_n(t) \\ \forall t &\in [0, 1], \tau \in [\tau_0, \tau_m]; \end{aligned} \quad (4.137a)$$

- на продольном наборе

$$\begin{aligned} \overset{\circ}{p}(t, \tau) &= \overset{\circ}{A}(t) B_m(\tau) = \overset{\circ}{A}(t) H_m T_m(\tau) \\ \forall t &\in [t_0, t_n], \tau \in [0, 1]. \end{aligned} \quad (4.137b)$$

Делением каждого однородного вектора из (4.137) на его четвертую координату получим расширенный вектор декартовых координат $\tilde{p}(t, \tau)$.

Составив матрицу $\overset{\circ}{U} = \{\overset{\circ}{u}_{ij}\}$ однородных форм точек (4.126) пересечения остовных линий $c_i(\tau)$ и $a_j(t)$, скомбинируем из (4.130a) и (4.136) модель **рациональной двухостовой** поверхности Безье:

$$\begin{aligned} \overset{\circ}{p}(t, \tau) &= [B_n^T(t) \ 1] \begin{bmatrix} -\overset{\circ}{U} & \overset{\circ}{C}^T(\tau_c) \\ \overset{\circ}{A}(t_a) & O \end{bmatrix} \begin{bmatrix} B_m(\tau) \\ 1 \end{bmatrix} = [T_n^T(t) \ 1] \begin{bmatrix} -H_n^T \overset{\circ}{U} H_m & H_n^T \overset{\circ}{C}^T(\tau_c) \\ \overset{\circ}{A}(t_a) H_m & O \end{bmatrix} \begin{bmatrix} T_m(\tau) \\ 1 \end{bmatrix}, \end{aligned} \quad (4.138)$$

где $t, \tau \in [0, 1]$, $t_a = t_0 + (t_n - t_0)t$, $\tau_c = \tau_0 + (\tau_m - \tau_0)\tau$.

4.5. СОСТАВНЫЕ ПОВЕРХНОСТИ

Главный недостаток поверхностей, сложенных из независимых кусочно-определённых сегментов, — нулевой порядок гладкости их сопряжения: разрывы производных в узловых точках и на граничных линиях. Помимо эстетического дискомфорта и травм изломы в виде острых вершин и рёбер создают повышенное сопротивление быстрому движению. Даже просто гладкое сопряжение создаёт на границах сегментов турбулентные завихрения и неустойчивое движение. Эти и подобные им многочисленные примеры вполне убедительно доказывают важность изучения методов конструирования составных поверхностей с гладким и дважды гладким сопряжением сегментов.

В области конструирования гладких составных поверхностей разработано множество интерполяционных и приближающих алгоритмов [15, 20, 27, 51, 56, 58, 65]. Естественно, всё это разнообразие невозможно полностью осветить в рамках ограниченного объёма книги, не претендующей на роль энциклопедии. Здесь будет уделено внимание нескольким методам гладкого сопряжения кусочных поверхностей с хорошо зарекомендовавшими себя линейными аналогами.

Конструирование составной поверхности начинается с *кластеризации* матрично упорядоченных узловых точек — разделения их на $N \times M$ групп узлов, служащих основой сегментов (рис. 4.136,а). Смысл такого разделения — в создании небольших (не более пятих-шестых) степеней сегментных полиномов во избежание появления сильных межузловых осцилляций (см. рис. 4.96). Математически задача гладкого сопряжения сводится к составлению уравнений связи параметров смежных сегментов, ограничивающих свободу расположения вершин узловых сеток. Условия сопряжения совместно с условиями узловой интерполяции (приближения) определяют систему уравнений, решение которой даёт полиномиальные коэффициенты сегментов составной поверхности.

Введём необходимые в дальнейшем обозначения параметров (i, j) -го сегмента поверхности $p_{ij}(t, \tau)$, где $i \in [1, N]$, $j \in [1, M]$ (рис. 4.136,б):

- n_i и m_j — количества ячеек узловой сетки с размерами $(n_i+1) \times (m_j+1)$ в параметрических направлениях t и τ ;
- $\{d_{i1}, d_{i2}, \dots, d_{in_i}\}$ и $\{\delta_{j1}, \delta_{j2}, \dots, \delta_{jm_j}\}$ — списки длин параметрических интервалов в направлениях t и τ ;

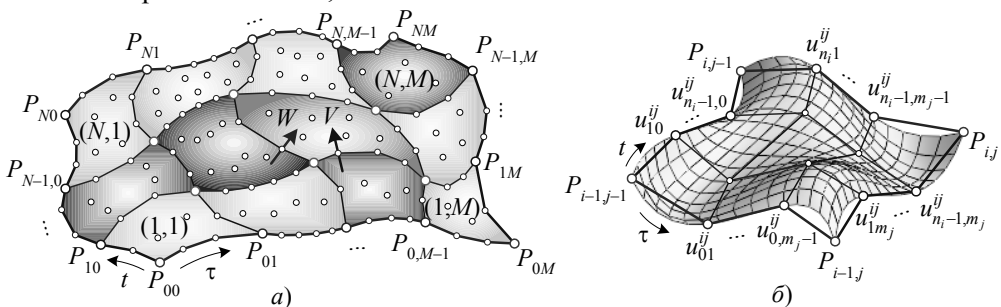


Рис. 4.136

• $\{a_i \equiv t_{i0}, t_{i1}, \dots, t_{i, n_i-1}, t_{in_i} \equiv b_i\}$ и $\{\alpha_j \equiv \tau_{j0}, \tau_{j1}, \dots, \tau_{j, m_j-1}, \tau_{jm_j} \equiv \beta_j\}$ — списки узловых отсчётов параметров $t_{ik} = t_{i, k-1} + d_{ik}$ и $\tau_{jl} = \tau_{j, l-1} + \delta_{jl}$;

$$U_{ij} = \begin{bmatrix} P_{i-1, j-1} \equiv u_{00}^{ij} & u_{01}^{ij} & \dots & u_{0, m_j-1}^{ij} & u_{0m_j}^{ij} \equiv P_{i-1, j} \\ u_{10}^{ij} & u_{11}^{ij} & \dots & u_{1, m_j-1}^{ij} & u_{1m_j}^{ij} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ u_{n_i-1, 0}^{ij} & u_{n_i-1, 1}^{ij} & \dots & u_{n_i-1, m_j-1}^{ij} & u_{n_i-1, m_j}^{ij} \\ P_{i, j-1} \equiv u_{n_i 0}^{ij} & u_{n_i 1}^{ij} & \dots & u_{n_i, m_j-1}^{ij} & u_{n_i m_j}^{ij} \equiv P_{ij} \end{bmatrix} \in V^{(n_i+1) \times (m_j+1)} \text{ — трёхслой-}$$

ная (по координатам x, y, z) матрица узлов сегментной сетки;

- v_i и μ_j — степени полинома сегмента по параметрам t и τ ;
- $S_{ij} \in V^{(v_i+1) \times (\mu_j+1)}$ — трёхслойная МПК разложения в форме Фергюсона

$$p_{ij}(t, \tau) = T_{v_i}^T(t) S_{ij} T_{\mu_j}(\tau) \quad \forall t \in [a_i, b_i], \quad \tau \in [\alpha_j, \beta_j], \quad i = \overline{1, N}, \quad j = \overline{1, M}, \quad (4.139)$$

с помощью которой вычисляются векторы касательных

$$V_{ij}(t, \tau) = T_{v_i}'^T(t) S_{ij} T_{\mu_j}(\tau), \quad W_{ij}(t, \tau) = T_{v_i}^T(t) S_{ij}' T_{\mu_j}'(\tau) \quad (4.140a)$$

и кручения

$$K_{ij}(t, \tau) = T_{v_i}'^T(t) S_{ij} T_{\mu_j}'(\tau). \quad (4.140b)$$

Вектор базисных функций $T_m(t)$ и его производные определены в (2.6).

Совокупность МПК сегментов составляет блочную матрицу

$$S = \begin{bmatrix} S_{11} & \dots & S_{1M} \\ \vdots & \ddots & \vdots \\ S_{N1} & \dots & S_{NM} \end{bmatrix} \in M^{v \times \mu}$$

с суммарными размерами $v = \sum_{i=1}^N v_i + N$ и $\mu = \sum_{j=1}^M \mu_j + M$.

4.5.1. Сопряжение интерполяционных сегментов

Переходим к рассмотрению вопросов получения желаемых свойств составной поверхности в точках сшивки её сегментов. **Первый порядок непрерывности** сопряжения автоматически обеспечивается применением интерполяционных и приближающих алгоритмов, разработанных в разд. 4.4, благодаря аналогичным (4.95) зависимостям граничных линий сегмента $p_{ij}(t, \alpha_j)$, $p_{ij}(t, \beta_j)$, $p_{ij}(a_i, \tau)$ и $p_{ij}(b_i, \tau)$ только от граничных узловых точек при равных векторах направлений в общих углах смежных сегментов. **Простейшая непрерывная составная поверхность сшивается из билинейных кусков** (см. рис. 4.89, б), описываемых локальными нормализованными моделями (4.81б)

$$p_{ij}(t, \tau) = L_1^T(t) U_{ij} L_1(\tau) = T_1^T(t) S_{ij} T_1(\tau) \quad \forall t, \tau \in [0, 1] \quad (4.141)$$

с сегментными параметрами $n_i=v_i=m_j=\mu_j=1$, $U_{ij} = \begin{bmatrix} P_{i-1,j-1} & P_{i-1,j} \\ P_{i,j-1} & P_{i,j} \end{bmatrix} \in V^{2 \times 2}$ и МПК

$$S_{ij} = \begin{bmatrix} P_{i-1,j-1} & P_{i-1,j} - P_{i-1,j-1} \\ P_{i,j-1} - P_{i-1,j-1} & P_{i,j-1} - P_{i-1,j-1} - P_{i-1,j} + P_{i,j} \end{bmatrix} \in V^{2 \times 2}.$$

Коэффициенты модели общего вида (4.139) получаются подстановкой в (4.141) вместо t и τ выражений $(t-a_i)/(b_i-a_i)$ и $(\tau-\alpha_j)/(\beta_j-\alpha_j)$. Оставим это занятие читателям, стремящимся доводить любое дело до последней точки.

Гораздо больший интерес представляет *гладкое* сопряжение сегментов составной поверхности. Сейчас мы ограничимся рассмотрением задачи обеспечения аналогичных (2.3а) **условий первого порядка гладкости**

$$\begin{cases} V_{i-1,j}(b_{i-1}, \tau_{jl}) = V_{ij}(a_i, \tau_{jl}) \quad \forall i = \overline{2, N}, j = \overline{1, M}, l = \overline{0, m_j}, \\ W_{i,j-1}(t_{ik}, \beta_{j-1}) = W_{ij}(t_{ik}, \alpha_j) \quad \forall i = \overline{1, N}, j = \overline{2, M}, k = \overline{0, n_i}, \end{cases} \quad (4.142)$$

а дважды гладкое сопряжение используется в сплайновых поверхностях.

Как и в линейном случае (см. п. 2.2.1), могут быть организованы разные схемы соединения кусочных сегментов друг с другом:

- **явное** задание векторов направлений, трансверсальных к граничным линиям сегментов — наиболее трудоёмкий способ формообразования, часто осложняющийся неочевидностью выбора этих векторов. Однако, он позволяет достичь желаемых свойств сопряжения индивидуально на каждой границе каждого сегмента, подобно составным линиям с разными порядками гладкости сопряжения (см. рис. 2.52, 2.53). Сегменты с явно заданными направлениями границ строятся независимо друг от друга как поверхности Эрмита (4.101) и Безье (4.109), (4.112). Для получения гладкого сопряжения необходимо позаботиться о создании равных наклонов смежных сегментов поперёк их общей границы;

- **последовательное** присоединение новых сегментов к границам строящейся составной поверхности с учётом желаемых свойств сопряжения. Очередность сшивки сегментов может быть как произвольной, так и регулярной, например, по принципу строчной развёртки:

$$\begin{array}{ccccccc} (1,1) & \rightarrow & (1,2) & \rightarrow \dots \rightarrow & (1,M-1) & \rightarrow & (1,M) \\ \downarrow & & \downarrow & & \downarrow & & \downarrow \\ (2,1) & \rightarrow & (2,2) & \rightarrow \dots \rightarrow & (2,M-1) & \rightarrow & (2,M) \\ \downarrow & & \downarrow & & \downarrow & & \downarrow \\ \dots & & \dots & & \dots & & \dots \\ \downarrow & & \downarrow & & \downarrow & & \downarrow \\ (N,1) & \rightarrow & (N,2) & \rightarrow \dots \rightarrow & (N,M-1) & \rightarrow & (N,M) \end{array}$$

Для получения гладкой стыковки поперечные наклоны краёв присоединяемого сегмента уравниваются с граничными направлениями уже установленных сегментов, что чревато возрастанием межузловых осцилляций по мере удаления от сегмента, установленного первым (см. линейный аналог на рис. 2.41, в);

- **одновременным** вычислением всех МПК, связанных системой *неявных* уравнений непрерывного и гладкого сопряжения. Данный метод отличается более равномерным распределением межузловых осцилляций по сегментам, чем в последовательной схеме, но и высокой размерностью решаемой СЛАУ.

□ Начнём с задачи гладкого сопряжения полиномиальных кусочных поверхностей (4.139) со **свободными краевыми направлениями** граничных сегментов $V_{lj}(a_l, \tau_{jl}), V_{Nj}(b_N, \tau_{jl}) \quad \forall j \in \overline{1, M}, l = \overline{0, m_j}$ и $W_{il}(t_{ik}, \alpha_l), W_{iM}(t_{ik}, \beta_M) \quad \forall i \in \overline{1, N}, k = \overline{0, n_i}$. Запишем уравнения, необходимые для вычисления матриц $S_{11} \div S_{NM}$:

• условия прохождения поверхности (i, j) -го сегмента через вершины узловой сетки с матрицей U_{ij} дают аналогичное (4.96г) матричное уравнение

$$\begin{aligned} Q_i^T S_{ij} \Theta_j &= U_{ij} \quad \forall i = \overline{1, N}, j = \overline{1, M}, \text{ где} \\ Q_i &= [T_{v_i}(a_i) \ T_{v_i}(t_{i1}) \ \dots \ T_{v_i}(t_{i, n_i-1}) \ T_{v_i}(b_i)] \in \mathbb{R}^{(v_i+1) \times (n_i+1)}, \\ \Theta_j &= [T_{\mu_j}(\alpha_j) \ T_{\mu_j}(\tau_{j1}) \ \dots \ T_{\mu_j}(\tau_{j, m_j-1}) \ T_{\mu_j}(\beta_j)] \in \mathbb{R}^{(\mu_j+1) \times (m_j+1)}; \end{aligned} \quad (4.143a)$$

• условия (4.142) гладкого сопряжения сегментов в узловых точках внутренних граничных линий превращаются с учётом (4.140а) в неявные уравнения

$$\begin{cases} T_{v_{i-1}}'^T(b_{i-1}) S_{i-1, j} T_{\mu_j}(\tau_{jl}) = T_{v_i}^T(a_i) S_{ij} T_{\mu_j}(\tau_{jl}) \quad \forall i = \overline{2, N}, j = \overline{1, M}, l = \overline{0, m_j}, \\ T_{v_i}^T(t_{ik}) S_{i, j-1} T_{\mu_{j-1}}'(\beta_{j-1}) = T_{v_i}^T(t_{ik}) S_{ij} T_{\mu_j}'(\alpha_j) \quad \forall i = \overline{1, N}, j = \overline{2, M}, k = \overline{0, n_i}. \end{cases} \quad (4.143б)$$

В совокупности интерполяционные уравнения (4.143) сводятся в одно матричное уравнение

$$Q^T S \Theta = U \quad (4.144a)$$

с аналогичными (2.67б) матрицами свободного варианта

$$\begin{aligned} Q_{cb} &= \left[\begin{array}{cccc|ccc} Q_1 & 0 & \dots & 0 & T_{v_1}'(b_1) & \dots & 0 \\ 0 & Q_2 & \dots & 0 & -T_{v_2}'(a_2) & \dots & \vdots \\ \vdots & \vdots & \ddots & \vdots & 0 & \ddots & 0 \\ 0 & 0 & \dots & 0 & \vdots & \dots & T_{v_{N-1}}'(b_{N-1}) \\ 0 & 0 & \dots & Q_N & 0 & \dots & -T_{v_N}'(a_N) \end{array} \right] \in \mathbb{R}^{v \times v}, \\ \Theta_{cb} &= \left[\begin{array}{cccc|ccc} \Theta_1 & 0 & \dots & 0 & T_{\mu_1}'(\beta_1) & \dots & 0 \\ 0 & \Theta_2 & \dots & 0 & -T_{\mu_2}'(\alpha_2) & \dots & \vdots \\ \vdots & \vdots & \ddots & \vdots & 0 & \ddots & 0 \\ 0 & 0 & \dots & 0 & \vdots & \dots & T_{\mu_{M-1}}'(\beta_{M-1}) \\ 0 & 0 & \dots & \Theta_M & 0 & \dots & -T_{\mu_M}'(\alpha_M) \end{array} \right] \in \mathbb{R}^{\mu \times \mu}, \\ U_{cb} &= \left[\begin{array}{ccc|ccc} U_{11} & \dots & U_{1M} & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ U_{N1} & \dots & U_{NM} & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & \dots & 0 \end{array} \right] \in \mathbb{R}^{v \times \mu} \end{aligned} \quad (4.144б)$$

и решением (4.97):

$$S = (Q^{-1})^T U \Theta^{-1}. \quad (4.144в)$$

При выборе степеней полиномов сегментов следует определиться, к каким смежным сегментам будут отнесены неявные условия сопряжения (4.143б). Из $2^{(N-1)(M-1)}$ вариантов есть NM регулярных решений, заключающихся в задании по одной из степеней v_k и μ_l равными числам интервалов n_k и m_l , а остальных степеней v_i и μ_j на единицу большими чисел n_i и m_j :

$$\begin{cases} v_k = n_k, v_i = n_i + 1 \quad \forall k \in [1, N], i = \overline{1, N}, i \neq k; \\ \mu_l = m_l, \mu_j = m_j + 1 \quad \forall l \in [1, M], j = \overline{1, M}, j \neq l. \end{cases} \quad (4.144\Gamma)$$

Если проектируемая составная поверхность обладает пространственной симметрией в параметрических направлениях t и/или τ , то **целесообразно разбивать узловую сеть на нечётные числа кластеров N и/или M с симметричным распределением степеней v_i и/или μ_j** . Примером такого решения служит изображённая на рис. 4.137,а узловая сетка размерами 6×9 вершин, разделённая на $N=M=3$ кластерных полос, границы которых выделены утолщёнными полилиниями. Отметим, что из-за слияния вершин в точках P_{00} , P_{10} , P_{01} и P_{11} четыре угловые ячейки узловой сети вырождены в треугольники.

Подберём подходящие размеры и порядки сегментов:

$$\begin{aligned} \{n_1, n_2, n_3\} = \{1, 3, 1\} &\Rightarrow \{v_1, v_2, v_3\} = \{2, 3, 2\}; \\ \{m_1, m_2, m_3\} = \{2, 4, 2\} &\Rightarrow \{\mu_1, \mu_2, \mu_3\} = \{3, 4, 3\}. \end{aligned}$$

Соответствующая составная поверхность, построенная методом хордовой параметризации, показана на рис. 4.137,б. Моделирование убеждает, что форма получаемого по (4.144) решения весьма чувствительна к перемещению узловых точек и изменению степеней полиномов сегментов. Это доказывает поверхность, построенная при $\{n_1, n_2, n_3\} = \{2, 1, 2\}$ и $\{m_1, m_2, m_3\} = \{3, 2, 3\}$ (рис. 4.137,в).

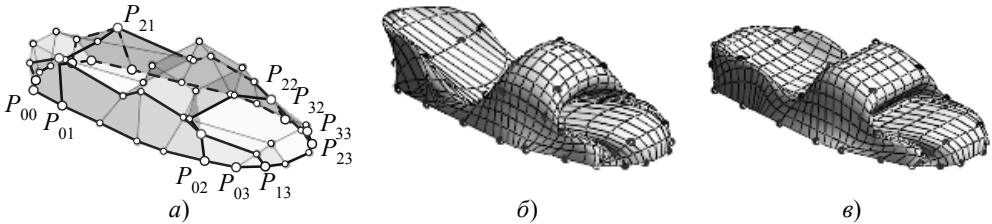


Рис. 4.137

Побочным решением задачи (4.144) являются равенства

$$\begin{aligned} T_{v_{i-1}}^{rT}(b_{i-1})S_{i-1,j-1}T'_{\mu_{j-1}}(\beta_{j-1}) - T_{v_i}^{rT}(a_i)S_{i,j-1}T'_{\mu_{j-1}}(\beta_{j-1}) = \\ = T_{v_{i-1}}^{rT}(b_{i-1})S_{i-1,j}T'_{\mu_j}(\alpha_j) - T_{v_i}^{rT}(a_i)S_{ij}T'_{\mu_j}(\alpha_j) \quad \forall i = \overline{2, N}, j = \overline{2, M}, \end{aligned}$$

означающие с учётом (4.140б) соотношения $K_{n_{i-1}m_{j-1}}^{i-1,j-1} + K_{00}^{ij} = K_{0m_{j-1}}^{i,j-1} + K_{n_{i-1}0}^{i-1,j}$ между векторами угловых кручений четырёх смежных сегментов в их общей узловой точке $P_{i-1,j-1}$. Таким образом, в отличие от (4.102в) **нулевые блоки в нижнем правом углу узловой матрицы вовсе не означают получение нулевых векторов угловых кручений сегментов**. Для создания во всех углах сегментов P_{ij} желаемых, в том числе и нулевых, кручений K_{ij} должны выполняться условия

$$\begin{cases} T_{v_i}^{\prime T}(a_i)S_{ij}T_{\mu_j}'(\alpha_j)=K_{i-1,j-1}, & T_{v_i}^{\prime T}(a_i)S_{ij}T_{\mu_j}'(\beta_j)=K_{i-1,j}, \\ T_{v_i}^{\prime T}(b_i)S_{ij}T_{\mu_j}'(\alpha_j)=K_{i,j-1}, & T_{v_i}^{\prime T}(b_i)S_{ij}T_{\mu_j}'(\beta_j)=K_{ij}, \end{cases}$$

матрицы (4.144б) расширены до вида

$$\begin{aligned} Q_K &= \left[\begin{array}{c|ccc} Q_{CB} & T'_{v_1}(a_1) & T'_{v_1}(b_1) & \dots & O & O \\ & \vdots & \vdots & \ddots & \vdots & \vdots \\ & O & O & \dots & T'_{v_N}(a_N) & T'_{v_N}(b_N) \end{array} \right], \\ \Theta_K &= \left[\begin{array}{c|ccc} \Theta_{CB} & T'_{\mu_1}(\alpha_1) & T'_{\mu_1}(\beta_1) & \dots & O & O \\ & \vdots & \vdots & \ddots & \vdots & \vdots \\ & O & O & \dots & T'_{\mu_M}(\alpha_M) & T'_{\mu_M}(\beta_M) \end{array} \right], \\ U_K &= \left[\begin{array}{c|ccc} U_{CB} & O & \dots & O \\ & \vdots & \ddots & \vdots \\ & O & \dots & O \\ \hline O & \dots & O & K_{11} & \dots & K_{1M} \\ & \vdots & \vdots & \vdots & \ddots & \vdots \\ O & \dots & O & K_{N1} & \dots & K_{NM} \end{array} \right], \text{ где } K_{ij} = \begin{bmatrix} K_{i-1,j-1} & K_{i-1,j} \\ K_{i,j-1} & K_{ij} \end{bmatrix}, \end{aligned} \quad (4.145)$$

а все степени полиномов сегментов (4.144г) увеличены на два.

□ Многие составные поверхности с топологией полузамкнутого цилиндра или замкнутого тора могут быть получены путём гладкого сопряжения одной или двух пар противоположных границ поверхности. Например, сшивка краёв $P_{00} \div P_{0M}$ и $P_{N0} \div P_{NM}$ узловой сети на рис. 4.136,а даёт объект цилиндрического типа, который после соединения краёв $P_{00} \div P_{N0}$ и $P_{0M} \div P_{NM}$ превращается в торообразную поверхность.

Помимо физического совмещения противоположных граничных узлов

$$u_{0l}^{1j} = u_{n_N l}^{Nj} \quad \forall j = \overline{1, M}, \quad l = \overline{0, m_j} \quad \text{и} \quad u_{k0}^{i1} = u_{km_M}^{iM} \quad \forall i = \overline{1, N}, \quad k = \overline{0, n_i},$$

которое обеспечивается конструктором без использования каких-либо новых уравнений относительно МПК S , построение гладкой **циклической составной поверхности** требует формирования дополнительных к (4.143) уравнений и соответствующего расширения системы (4.144):

- при гладком замыкании краёв $P_{00} \div P_{0M}$ и $P_{N0} \div P_{NM}$ уравниваются векторы трансверсальных касательных в t -направлении $V_{1j}(a_1, \tau_{jl}) = V_{Nj}(b_N, \tau_{jl})$, откуда с учётом (4.140а) возникают неявные уравнения

$$T_{v_1}^{\prime T}(a_1)S_{1j}T_{\mu_j}'(\tau_{jl}) = T_{v_N}^{\prime T}(b_N)S_{Nj}T_{\mu_j}'(\tau_{jl}) \quad \forall j = \overline{1, M}, \quad l = \overline{0, m_j}, \quad (4.146a)$$

расширенные матрицы t -циклического варианта

$$Q_{\Pi} = \left[\begin{array}{c|c} Q_{CB} & \begin{bmatrix} -T'_{v_1}(a_1) \\ O \\ T'_{v_N}(b_N) \end{bmatrix} \end{array} \right], \quad U_{\Pi} = \begin{bmatrix} U_{CB} \\ O \end{bmatrix} \quad (4.146б)$$

и регулярный набор степеней полиномов по параметру t :

$$v_i = n_i + 1 \quad \forall i = \overline{1, N}; \quad (4.146в)$$

• для гладкого замыкания краёв $P_{00} \div P_{N0}$ и $P_{0M} \div P_{NM}$ уравниваются векторы трансверсальных касательных в τ -направлении $W_{i1}(t_{ik}, \alpha_1) = W_{iM}(t_{ik}, \beta_M)$, в результате чего появляются неявные уравнения

$$T_{v_i}^T(t_{ik})S_{i1}T'_{\mu_1}(\alpha_1) = T_{v_i}^T(t_{ik})S_{iM}T'_{\mu_M}(\beta_M) \quad \forall i=1, N, \quad k=0, n_i, \quad (4.147a)$$

расширенные матрицы τ -циклического варианта

$$\Theta_{\tau\tau} = \begin{bmatrix} \Theta_{\text{св}} & \begin{matrix} -T'_{\mu_1}(\alpha_1) \\ \text{O} \\ T'_{\mu_M}(\beta_M) \end{matrix} \end{bmatrix}, \quad U_{\tau\tau} = [U_{\text{св}} | \text{O}] \quad (4.147б)$$

и регулярный набор степеней полиномов по параметру τ :

$$\mu_j = m_j + 1 \quad \forall j=1, \overline{M}; \quad (4.147в)$$

• наконец, при двойном $t \times \tau$ -циклическом замыкании свободной составной поверхности создаются вышеприведённые матрицы $Q_{\tau} = Q_{\tau\tau}$, $\Theta_{\tau} = \Theta_{\tau\tau}$ и двусторонне расширенная узловая матрица $U_{\tau} = \begin{bmatrix} U_{\text{св}} & | & \text{O} \\ \hline \text{O} & & \text{O} \end{bmatrix}$. Степени полиномов всех сегментов задаются по (4.146в) и (4.147в).

В результате получаем матричное уравнение (4.144а) с решением (4.144в) и полиномиальной моделью сегментов (4.139).

☞ Построение циклической составной поверхности может стать **невозможным** из-за вырожденности матриц Q_{τ} и/или Θ_{τ} ввиду обнуления их определителей, зависящих подобно (2.70) от параметрических интервалов d_i и δ_j . Выводы, сделанные в п. 2.2.1 для составных циклических линий, полностью применимы к циклическим поверхностям благодаря сходному устройству матриц в (2.69б), (4.146б) и (4.147б). Вспомним **случай неразрешимости задачи при чётных** размерах узловой сети $n = \sum n_i$ или $m = \sum m_i$ для каждого из параметрических направлений t или τ , по которому происходит циклическое замыкание **первого** порядка гладкости:

- нормализованная параметризация на любых узловых сетках;
- хордовая параметризация на сетках с равномерным или каким-либо симметричным расположением узлов интерполяции, при котором определитель матрицы Q_{τ} и/или Θ_{τ} обращается в нуль.

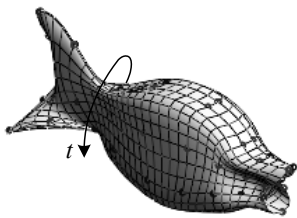
Следует понимать, что даже при выполнении условий разрешимости построение циклических составных поверхностей требует тщательного подбора узловых точек, малое перемещение которых существенно изменяет форму получаемого решения. Возникающие из-за этого на поверхности сильные межузловые осцилляции вызваны большим числом обусловленности (отношения максимального сингулярного числа к минимальному) обращаемой в (4.144в) циклической матрицы Q_{τ} и/или Θ_{τ} . **Эффективным средством повышения устойчивости задачи является увеличение порядка циклической гладкости до второго** путём использования вдоль направлений замыкания матриц

$$Q_{\Pi} = \begin{bmatrix} Q_{\text{св}} & \begin{bmatrix} -T'_{v_1}(a_1) & -T''_{v_1}(a_1) \\ 0 & 0 \\ T'_{v_N}(b_N) & T''_{v_N}(b_N) \end{bmatrix} \\ \begin{bmatrix} T'_{v_N}(b_N) & T''_{v_N}(b_N) \end{bmatrix} & \end{bmatrix}, \quad \Theta_{\Pi} = \begin{bmatrix} \Theta_{\text{св}} & \begin{bmatrix} -T'_{\mu_1}(\alpha_1) & -T''_{\mu_1}(\alpha_1) \\ 0 & 0 \\ T'_{\mu_M}(\beta_M) & T''_{\mu_M}(\beta_M) \end{bmatrix} \\ \begin{bmatrix} T'_{\mu_M}(\beta_M) & T''_{\mu_M}(\beta_M) \end{bmatrix} & \end{bmatrix}. \quad (4.148)$$

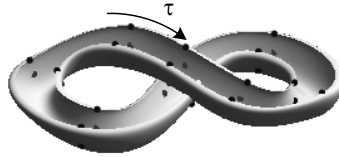
Их определители аналогично (2.25в) всегда ненулевые.

В результате проведённого анализа методов разрешимости задачи циклической интерполяции построены фигуры на рис. 4.138:

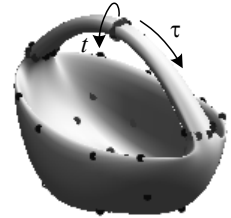
- t -циклическая поверхность рыбоподобного вида с размерами узловой сетки $m=7$, $m=8$ и порядками сегментов $\{v_1, v_2, v_3\} = (4, 2, 4)$, $\{\mu_1, \mu_2, \mu_3\} = (3, 4, 3)$;
- $t \times \tau^2$ -циклическая поверхность ∞ -образного жёлоба с размерами $n=3$, $m=8$ (чётное число!) и порядками $\{v_1, v_2, v_3\} = (2, 1, 2)$, $\{\mu_1, \mu_2, \mu_3\} = (3, 6, 3)$. К сожалению, первый порядок цикличности не в состоянии устранить осцилляции получаемой поверхности. Ситуацию спасло дважды гладкое замыкание в направлении параметра τ с хорошо обусловленной матрицей Θ_{Π} из (4.148);
- $t^2 \times \tau^2$ -циклическая поверхность на узловой сетке с размерами $n=9$ и $m=7$ с порядками сегментов $\{v_1, v_2, v_3, v_4, v_5\} = (2, 3, 5, 3, 2)$ и $\{\mu_1, \mu_2, \mu_3\} = (2, 7, 2)$.



$$\begin{aligned} \{n_1, n_2, n_3\} &= \{3, 1, 3\} \\ \{m_1, m_2, m_3\} &= \{2, 4, 2\} \end{aligned}$$



$$\begin{aligned} \{n_1, n_2, n_3\} &= \{1, 1, 1\} \\ \{m_1, m_2, m_3\} &= \{2, 4, 2\} \end{aligned}$$



$$\begin{aligned} \{n_1, n_2, n_3, n_4, n_5\} &= \{1, 2, 3, 2, 1\} \\ \{m_1, m_2, m_3\} &= \{1, 5, 1\} \end{aligned}$$

Рис. 4.138

□ Уменьшению межузловых осцилляций составной поверхности способствует фиксация наклонов её границ четырьмя пучками векторов (рис. 4.139)

$$\begin{cases} V_{1j}(a_1, \tau_{jl}) = V_{0l}^{1j}, \quad V_{Nj}(b_N, \tau_{jl}) = V_{n_N l}^{Nj} \quad \forall j = \overline{1, M}, l = \overline{0, m_j}, \\ W_{i1}(t_{ik}, \alpha_1) = W_{k0}^{i1}, \quad W_{iM}(t_{ik}, \beta_M) = W_{km_M}^{iM} \quad \forall i = \overline{1, N}, k = \overline{0, n_i}, \end{cases} \quad (4.149a)$$

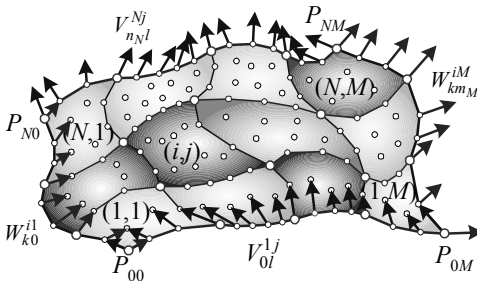


Рис. 4.139

ограничивающих свободу изгибания поверхности на краях, где осцилляции наиболее сильны. Даже при небольшом числе сегментов и невысоких степенях их полиномов количество краевых векторов слишком велико для ручного определения. В этом случае, равно как и в отсутствие явных данных, целесообразно задавать векторы краевых касательных автоматическим методом разностной аппроксимации

$$\left\{ \begin{array}{l} V_{0l}^{1j} = \gamma_0 \frac{u_{1l}^{1j} - u_{0l}^{1j}}{d_{11}}, \quad V_{n_N l}^{Nj} = \gamma_N \frac{u_{n_N l}^{Nj} - u_{n_N - 1, l}^{Nj}}{d_{N n_N}} \quad \forall j = \overline{1, M}, l = \overline{0, m_j}, \\ W_{k0}^{i1} = \lambda_0 \frac{u_{k1}^{i1} - u_{k0}^{i1}}{\delta_{11}}, \quad W_{km_M}^{iM} = \lambda_M \frac{u_{km_M}^{iM} - u_{k, m_M - 1}^{iM}}{\delta_{M m_M}} \quad \forall i = \overline{1, N}, k = \overline{0, n_i} \end{array} \right. \quad (4.149б)$$

с возможностью регулировки их длин едиными по каждому краю коэффициентами γ_0 , γ_N , λ_0 и λ_M . Задание их нулевыми позволяет получить **ненапряжённую** составную поверхность с нулевыми векторами краевых направлений. Следует отличать её от свободной поверхности, краевые направления которой формируются по соотношениям (4.140а) с матрицами S_{ij} , полученными в (4.144в).

Добавление новых условий (4.149а) ведет к расширению свободной системы (4.143) уравнениями **эрмитовой составной поверхности**

$$\left\{ \begin{array}{l} T_{v_1}^{\tau} (a_1) S_{1j} T_{\mu_j} (\tau_{jl}) = V_{0l}^{1j}, \quad T_{v_N}^{\tau} (b_N) S_{Nj} T_{\mu_j} (\tau_{jl}) = V_{n_N l}^{Nj} \quad \forall j = \overline{1, M}, l = \overline{0, m_j}, \\ T_{v_i}^{\tau} (t_{ik}) S_{i1} T_{\mu_1} (\alpha_1) = W_{k0}^{i1}, \quad T_{v_i}^{\tau} (t_{ik}) S_{iM} T_{\mu_M} (\beta_M) = W_{km_M}^{iM} \quad \forall i = \overline{1, N}, k = \overline{0, n_i} \end{array} \right.$$

и получению матричного уравнения (4.144а) с подобными (2.68б) элементами

$$Q_{\Xi} = \left[\begin{array}{c|cc} T'_{v_1}(a_1) & O & \\ \vdots & \vdots & \\ O & O & \\ \vdots & \vdots & \\ O & T'_{v_N}(b_N) & \end{array} \right], \quad \Theta_{\Xi} = \left[\begin{array}{c|cc} T'_{\mu_1}(\alpha_1) & O & \\ \vdots & \vdots & \\ O & O & \\ \vdots & \vdots & \\ O & T'_{\mu_M}(\beta_M) & \end{array} \right],$$

$$U_{\Xi} = \left[\begin{array}{cccccc|cc} & & & & & & W_{00}^{11} & W_{00}^{N1} \\ & & & & & & \vdots & \vdots \\ & & & & & & W_{n_1 0}^{11} & W_{n_N 0}^{N1} \\ & & & & & & \vdots & \vdots \\ & & & & & & W_{0m_M}^{1M} & W_{0m_M}^{NM} \\ & & & & & & \vdots & \vdots \\ & & & & & & W_{n_1 m_M}^{1M} & W_{n_N m_M}^{NM} \\ & & & & & & O & O \\ & & & & & & \vdots & \vdots \\ & & & & & & O & O \\ \hline V_{00}^{11} & \dots & V_{0m_1}^{11} & \dots & V_{00}^{1M} & \dots & V_{0m_M}^{1M} & O & \dots & O & O & O \\ V_{n_N 0}^{N1} & \dots & V_{n_N m_1}^{N1} & \dots & V_{n_N 0}^{NM} & \dots & V_{n_N m_M}^{NM} & O & \dots & O & O & O \end{array} \right] \quad (4.150а)$$

и решением (4.102в). Из всех вариантов выбора степеней полиномов сегментов есть NM регулярных решений, заключающихся в задании по одной из степеней v_k и μ_l равными $n_k + 2$ и $m_l + 2$, а остальных степеней v_i и μ_j на 1 меньшими:

$$\begin{aligned} v_k &= n_k + 2, \quad v_i = n_i + 1 \quad \forall k \in [1, N], \quad i = \overline{1, N}, \quad i \neq k; \\ \mu_l &= m_l + 2, \quad \mu_j = m_j + 1 \quad \forall l \in [1, M], \quad j = \overline{1, M}, \quad j \neq l. \end{aligned} \quad (4.150б)$$

Как и в (4.102в), нулевые векторы в нижнем правом блоке матрицы U_{Ξ} обеспечивают нулевые векторы кручения $K_{00} = K_{0M} = K_{N0} = K_{NM} = O_3$ в четырёх углах

составной поверхности. Для задания желаемых значений этих векторов указанный блок должен иметь вид $\begin{bmatrix} K_{00} & K_{0M} \\ K_{N0} & K_{NM} \end{bmatrix}$.

Эффект фиксации векторов краевых направлений в задаче эрмитовой 3d-интерполяции показан на рис. 4.140,а, где на сетке с интервальными размерами $n=4$ и $m=5$ построена составная поверхность с размерами сегментов $\{n_1, n_2, n_3\}=\{1, 2, 1\}$, $\{m_1, m_2, m_3\}=\{2, 1, 2\}$, их порядками $\{v_1, v_2, v_3\}=\{2, 4, 2\}$, $\{\mu_1, \mu_2, \mu_3\}=(3, 3, 3)$ и всеми коэффициентами удлинения в (4.149б) $\gamma=\lambda=1$. Для сравнения на рис. 4.140,б показано ненапряжённое решение с коэффициентами удлинения $\gamma=\lambda=0$. Очевидно отличие эрмитовых решений от свободной поверхности (рис. 4.140,в) с хорошо заметными межузловыми осцилляциями.

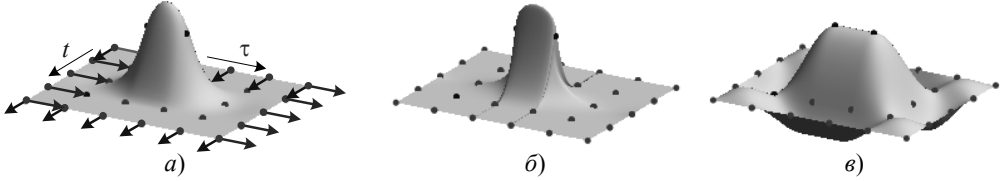


Рис. 4.140

Рассмотренному здесь методу построения интерполяционной составной поверхности Эрмита требуется весьма большой объём исходных данных, он весьма трудоёмок в программировании и отличается повышенными размерами обрабатываемых матриц. Наконец, в процессе интерактивного проектирования рабочее пространство загромождается множеством векторов, заслоняющих конструируемую поверхность от взгляда дизайнера, что совсем не хорошо. Вот чем приходится платить за высокое качество получаемых решений!

4.5.2. Составные поверхности Безье

Более удобной для интерактивного конструирования сложных составных поверхностей является бикубическая *составная поверхность Безье*, чей (i, j) -й сегмент описывается параметрической моделью (4.109):

$$p_{ij}(t, \tau) = L_B^T(t) U_{ij}^B L_B(\tau) = T_3^T(t) H_B^T U_{ij}^B H_B T_3(\tau) \quad \forall t, \tau \in [0, 1]. \quad (4.151)$$

Основой составной поверхности Безье является характеристическая полигональная сеть, разбитая на $N \times M$ кластеров. Вершины (i, j) -го кластера, составляющие узловую матрицу U_{ij}^B , подразделяются (см. рис. 4.105) на **угловые** интерполяционные точки $u_{00}^{ij} \div u_{11}^{ij}$, **боковые** вершины $v_{00}^{ij} \div v_{11}^{ij}$ и $w_{00}^{ij} \div w_{11}^{ij}$ управления векторами угловых касательных (см. табл. 4.3)

$$\begin{aligned} V_{00}^{ij} &= 3(v_{00}^{ij} - u_{00}^{ij}), & V_{01}^{ij} &= 3(u_{01}^{ij} - v_{01}^{ij}), & V_{10}^{ij} &= 3(u_{10}^{ij} - v_{10}^{ij}), & V_{11}^{ij} &= 3(u_{11}^{ij} - v_{11}^{ij}), \\ W_{00}^{ij} &= 3(w_{00}^{ij} - u_{00}^{ij}), & W_{01}^{ij} &= 3(u_{01}^{ij} - w_{01}^{ij}), & W_{10}^{ij} &= 3(w_{10}^{ij} - v_{10}^{ij}), & W_{11}^{ij} &= 3(u_{11}^{ij} - w_{11}^{ij}) \end{aligned}$$

и **внутренние** точки $k_{00}^{ij} \div k_{11}^{ij}$ влияния на векторы угловых кручений $K_{00}^{ij} \div K_{11}^{ij}$:

$$K_{00}^{ij} = 9(u_{00}^{ij} - v_{00}^{ij} - w_{00}^{ij} + k_{00}^{ij}), \quad K_{01}^{ij} = -9(u_{01}^{ij} - v_{01}^{ij} - w_{01}^{ij} + k_{01}^{ij}),$$

$$K_{10}^{ij} = -9(u_{10}^{ij} - v_{10}^{ij} - w_{10}^{ij} + k_{10}^{ij}), \quad K_{11}^{ij} = 9(u_{11}^{ij} - v_{11}^{ij} - w_{11}^{ij} + k_{11}^{ij}).$$

Рассмотрим **условия сопряжения** сегмента $p_{ij}(t, \tau)$ с соседними сегментами $p_{i+1,j}(t, \tau)$ и $p_{i,j+1}(t, \tau)$ (рис. 4.141):

- как показано в (4.110), каждая из четырёх граничных линий сегмента определяется четырьмя вершинами соответствующей границы кластера ХПС. Поэтому **непрерывность** и **продольная гладкость** сопряжения, означающие совпадение граничных линий $p_{ij}(1, \tau) = p_{i+1,j}(0, \tau)$ и $p_{ij}(t, 1) = p_{i,j+1}(t, 0) \quad \forall t, \tau \in [0, 1]$, достигаются попарным совмещением четырёх вершин смежных границ кластеров:

$$\begin{cases} u_{10}^{ij} = u_{00}^{i+1,j}, w_{10}^{ij} = w_{00}^{i+1,j}, w_{11}^{ij} = w_{01}^{i+1,j}, u_{11}^{ij} = u_{01}^{i+1,j}, \\ u_{01}^{ij} = u_{00}^{i,j+1}, v_{01}^{ij} = v_{00}^{i,j+1}, v_{11}^{ij} = v_{10}^{i,j+1}, u_{11}^{ij} = u_{10}^{i,j+1}; \end{cases} \quad (4.152a)$$

- условиями **трансверсальной гладкости** сопряжения трёх сегментов являются равенства касательных векторов $V_{ij}(1, \tau) = V_{i+1,j}(0, \tau)$ и $W_{ij}(t, 1) = W_{i,j+1}(t, 0)$ на всём протяжении их граничных линий $\forall t, \tau \in [0, 1]$. Они приводят к уравнениям

$$[0 \ 0 \ -3 \ 3]U_{ij}^B = [-3 \ 3 \ 0 \ 0]U_{i+1,j}^B, \quad U_{ij}^B[0 \ 0 \ -3 \ 3]^T = U_{i,j+1}^B[-3 \ 3 \ 0 \ 0]^T,$$

связывающим вершины (i, j) -го и $(i+1, j)$ -го кластеров соотношениями

$$\begin{cases} u_{10}^{ij} - v_{10}^{ij} = v_{00}^{i+1,j} - u_{00}^{i+1,j}, w_{10}^{ij} - k_{10}^{ij} = k_{00}^{i+1,j} - w_{00}^{i+1,j}, \\ w_{11}^{ij} - k_{11}^{ij} = k_{01}^{i,j+1} - w_{01}^{i,j+1}, u_{11}^{ij} - v_{11}^{ij} = v_{01}^{i+1,j} - u_{01}^{i+1,j}, \end{cases} \quad (4.152б)$$

а вершины (i, j) -го и $(i, j+1)$ -го кластеров соотношениями

$$\begin{cases} u_{01}^{ij} - w_{01}^{ij} = w_{00}^{i,j+1} - u_{00}^{i,j+1}, v_{01}^{ij} - k_{01}^{ij} = k_{00}^{i,j+1} - v_{00}^{i,j+1}, \\ v_{11}^{ij} - k_{11}^{ij} = k_{10}^{i,j+1} - v_{10}^{i,j+1}, u_{11}^{ij} - w_{11}^{ij} = w_{10}^{i,j+1} - u_{10}^{i,j+1}. \end{cases} \quad (4.152в)$$

С учётом совпадающих вершин (4.152a) это означает не только коллинеарность восьми троек вершин сопрягаемых кластеров, но и равенство восьми пар расстояний между ними, как показано на рис. 4.141.

Условия (4.152б,в) автоматически приводят к гладкому сопряжению сегментов по векторам угловых кручений $K_{10}^{ij} = K_{00}^{i+1,j}$, $K_{11}^{ij} = K_{01}^{i+1,j} = K_{10}^{i,j+1}$ и $K_{01}^{ij} = K_{00}^{i,j+1}$. Докажем это на примере первого равенства:

$$K_{10}^{ij} = 9(v_{10}^{ij} - u_{10}^{ij}) + 9(u_{10}^{ij} - k_{10}^{ij}) = 9(u_{00}^{i+1,j} - v_{00}^{i+1,j}) + 9(k_{00}^{i+1,j} - w_{00}^{i+1,j}) = K_{00}^{i+1,j}.$$

Таким образом, отпадает необходимость вывода условий гладкости сопряжения сегментов по угловым кручениям, что, несомненно, радует.

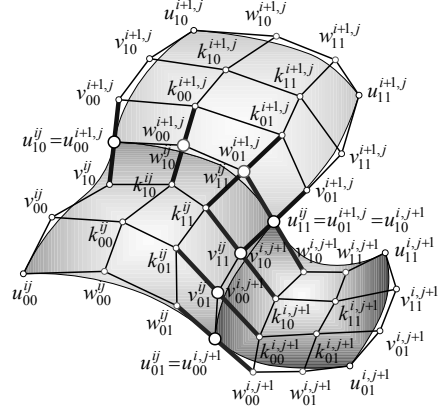


Рис. 4.141

Совокупность соотношений (4.152) сильно ограничивает свободу конструктора в интерактивной модификации характеристической сети гладкой составной поверхности и, соответственно, формы последней. Сложности начинаются с начального задания взаимозависимых вершин ХПС. При **последовательном** создании кластеров, например, **методом строчной развёртки** (рис. 4.142) в кластере (1,1) свободно устанавливаются все 16 вершин, предопределяя тем самым положение восьми вершин кластера (1,2). Таким образом, в $(1, 2 \div M)$ -х кластерах свободно задаётся

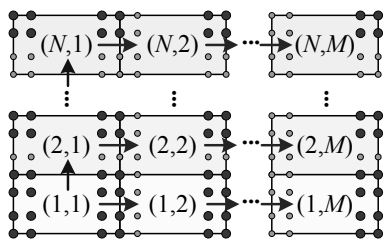


Рис. 4.142

только по 8 вершин. В остальных последовательно устанавливаемых по строкам $(2 \div N, 1)$ -х кластерах свобода выбора распространяется на 8, а в $(2 \div N, 2 \div M)$ -х кластерах — на 4 вершины.

Другой подход заключается в **одновременной** установке сразу всех узлов $N \times M$ кластеров ХПС во взаимно перпендикулярных направлениях одной, например, горизонтальной, **плоскости** на расстояниях, удовлетворяющих условиям (4.152), т. е. буквально так, как показано на рис. 4.142. Затем начинается процесс целенаправленного интерактивного перемещения вершин ХПС в пространстве с соблюдением всех условий их взаимосвязи аналогично методике, изложенной в п. 2.2.3 (см. рис. 2.51).

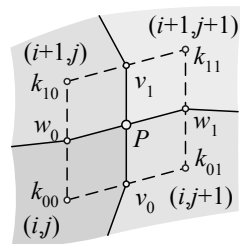


Рис. 4.143

Для наглядности обозначим узловую точку на стыке четырёх сегментов как P , а окружающие её управляющие точки как $v_0, v_1, w_0, w_1, k_{00}, k_{10}, k_{01}$ и k_{11} (рис. 4.143). Тогда условия непрерывности и гладкости сопряжения сегментов (4.152) запишутся в следующем виде:

$$\begin{aligned} v_0 + v_1 &= 2P, & w_0 + w_1 &= 2P, \\ k_{00} + k_{01} &= 2v_0, & k_{10} + k_{11} &= 2v_1, \\ k_{00} + k_{10} &= 2w_0, & k_{01} + k_{11} &= 2w_1. \end{aligned} \quad (4.153)$$

Предлагаются следующие **правила интерактивного конструирования составной поверхности Безье**:

- при переносе узловой вершины P на вектор F так же переносятся четыре окружающие её управляющие точки v_0, v_1, w_0 и w_1 ;
- смещение управляющей вершины v_m , где $m \in \{0, 1\}$, на вектор G переносит точки k_{m0} и k_{m1} на вектор G , а точки $v_{1-m}, k_{1-m,0}$ и $k_{1-m,1}$ на вектор $-G$;
- перенос управляющей вершины w_m на вектор G сдвигает точки k_{0m} и k_{1m} на вектор G , а точки $w_{1-m}, k_{0,1-m}$ и $k_{1,1-m}$ на вектор $-G$;
- перенос управляющей точки k_{mn} , где $m, n \in \{0, 1\}$, на вектор G ведет к смещению точки $k_{1-m,1-n}$ на вектор G , а точек $k_{1-m,n}$ и $k_{m,1-n}$ на вектор $-G$.

Данные правила соответствуют положению узловой точки P на стыке четырёх сегментов, когда её окружают восемь вершин ХПС. В двух других поло-

жениях этого узла (на стыке двух сегментов или в углах характеристической сети) смещение несуществующих управляющих точек игнорируется.

После каждого перемещения какой-либо вершины ХПС конструктор оценивает результирующее изменение формы составной поверхности и принимает решение о дальнейших манипуляциях либо о завершении её проектирования.

Примеры моделирования составных бикубических поверхностей Безье по описанной методике показаны на рис. 4.144.

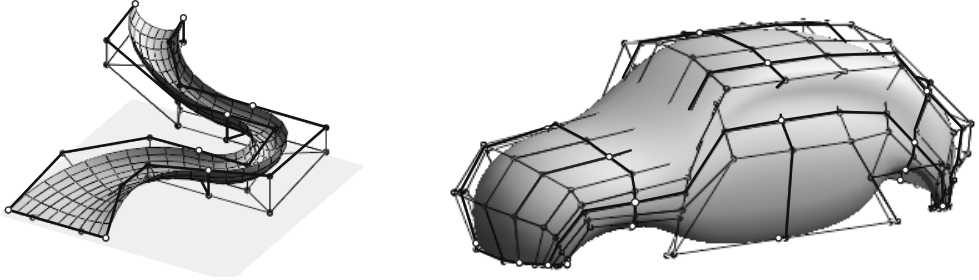


Рис. 4.144

Расширение модели бикубического сегмента (4.151) на **произвольные** порядки $n_i \geq 1$ и $m_j \geq 1$ задаётся биполиномиальной векторной функцией

$$p_{ij}(t, \tau) = B_{n_i}^T(t) U_{ij} B_{m_j}(\tau) = T_{n_i}^T(t) H_{n_i}^T U_{ij} H_{m_j} T_{m_j}(\tau) \quad \forall t \in [0, 1], \tau \in [0, 1]. \quad (4.154)$$

Соединение этого сегмента с соседними $p_{i+1,j}(t, \tau)$ и $p_{i,j+1}(t, \tau)$ связывает вершины ХПС ограничениями, скомбинированными из (2.83) и (4.152) с использованием индексов $k = \overline{0, n_i}$ и $l = \overline{0, m_j}$, на интервалах параметров $t, \tau \in [0, 1]$:

- условиями **непрерывности** и **продольной гладкости** сопряжения, означающими совпадение граничных линий $p_{ij}(1, \tau) = p_{i+1,j}(0, \tau)$ и $p_{ij}(t, 1) = p_{i,j+1}(t, 0)$, достигаемое попарным совмещением всех вершин общих границ кластеров:

$$\begin{cases} u_{n_i l}^{ij} = u_{0 l}^{i+1, j} \quad \forall i = \overline{1, N-1}, j = \overline{1, M}, \\ u_{k m_j}^{ij} = u_{k 0}^{i, j+1} \quad \forall i = \overline{1, N}, j = \overline{1, M-1}; \end{cases} \quad (4.155a)$$

- условиями **трансверсальной гладкости** сопряжения сегментов — равенствами касательных векторов $V_{ij}(1, \tau) = V_{i+1,j}(0, \tau)$ и $W_{ij}(t, 1) = W_{i,j+1}(t, 0)$ на всём протяжении их граничных линий:

$$\begin{cases} n_i (u_{n_i l}^{ij} - u_{n_i-1, l}^{ij}) = n_{i+1} (u_{0 l}^{i+1, j} - u_{0 l}^{i+1, j}) \quad \forall i = \overline{1, N-1}, j = \overline{1, M}, \\ m_j (u_{k m_j}^{ij} - u_{k, m_j-1}^{ij}) = m_{j+1} (u_{k 1}^{i, j+1} - u_{k 0}^{i, j+1}) \quad \forall i = \overline{1, N}, j = \overline{1, M-1}. \end{cases} \quad (4.155b)$$

С учётом совпадающих вершин (4.155a) это означает не только коллинеарность $(n_i+1)(m_j+1)$ троек вершин в каждой тройке сопрягаемых кластеров, но и равенство того же числа пар расстояний между ними, как показано на рис. 4.141;

- условиями **двойной гладкости** сопряжения сегментов, т. е. равенствами векторов ускорений $V'_{ij}(1, \tau) = V'_{i+1,j}(0, \tau)$ и $W'_{ij}(t, 1) = W'_{i,j+1}(t, 0)$ на всём протяжении их граничных линий:

$$\left\{ \begin{array}{l} n_i(n_i-1)(u_{n_i-1,i}^{ij}-2u_{n_i-2,i}^{ij}+u_{n_i-3,i}^{ij})= \\ =n_{i+1}(n_{i+1}-1)(u_{2l}^{i+1,j}-2u_{1l}^{i+1,j}+u_{0l}^{i+1,j}) \forall i=\overline{1,N-1}, j=\overline{1,M}, \\ m_j(m_j-1)(u_{km_j}^{ij}-2u_{k,m_j-1}^{ij}+u_{k,m_j-2}^{ij})= \\ =m_{j+1}(m_{j+1}-1)(u_{k2}^{i,j+1}-2u_{k1}^{i,j+1}+u_{k0}^{i,j+1}) \forall i=\overline{1,N}, j=\overline{1,M-1}. \end{array} \right. \quad (4.155\text{в})$$

Данные условия нужно использовать при выводе аналогичных (2.84) и (4.153) соотношений и правил перемещения узловых вершин в режиме редактирования составной поверхности Безье. Особенно напряжённые ограничения на конфигурацию ХПС накладывают условия (4.155в), так что, скорее всего, придётся ограничиться достижением простой гладкости (4.155б), а при необходимости получения излома по некоторой граничной линии сегмента достаточно удовлетворить лишь условиям (4.155а).

В заключение рассмотрим **три задачи алгоритмического изменения характеристического кластера** $U_{ij} \in V^{(n_i+1) \times (m_j+1)}$ **составной поверхности Безье** [37], линейные аналоги которых решены в п. 2.2.3. Ввиду матричной организации характеристической сетки следует иметь в виду:

- если преобразование кластера U_{ij} изменяет его порядок n_i , то такому же изменению порядка должны быть подвергнуты все остальные кластеры, входящие в полосу с индексами $\{i, j=\overline{0,M}\}$;
- если преобразование кластера U_{ij} изменяет его порядок m_j , то такому же изменению порядка должны быть подвергнуты все остальные кластеры, входящие в полосу с индексами $\{i=\overline{0,N}, j\}$.

□ В первой задаче **изменения порядков сегмента** $p_{ij}(t)$ требуется построить такой кластер $U'_{ij} \in V^{(n'_i+1) \times (m'_j+1)}$, чтобы поверхность $p'_{ij}(t, \tau)$ совпала с исходной. Матрица вершин этого кластера находится по формуле, сходной с (4.113):

$$U'_{ij} = (H_{n_i} E_{n_i+1, n'_i+1} H_{n_i}^{-1})^T U_{ij} (H_{m_j} E_{m_j+1, m'_j+1} H_{m_j}^{-1}). \quad (4.156\text{а})$$

Если какой-либо порядок кластера не менять, то соответствующая скобка в (4.156а) превратится в единичную матрицу. Это упрощает расчёт кластера:

$$n'_i \neq n_i, \quad m'_j = m_j \Rightarrow U'_{ij} = (H_{n_i} E_{n_i+1, n'_i+1} H_{n_i}^{-1})^T U_{ij}; \quad (4.156\text{б})$$

$$n'_i = n_i, \quad m'_j \neq m_j \Rightarrow U'_{ij} = U_{ij} (H_{m_j} E_{m_j+1, m'_j+1} H_{m_j}^{-1}). \quad (4.156\text{в})$$

Возможность изменения порядков n_i и m_j определяется структурой матриц коэффициентов $S_{ij} = H_{n_i}^T U_{ij} H_{m_j}$ и $S'_{ij} = H_{n'_i}^T U'_{ij} H_{m'_j}$:

- **увеличение** порядков кластера на $d_i = n'_i - n_i > 0$ и/или $\delta_j = m'_j - m_j > 0$ возможно **всегда**. При этом матрица S'_{ij} принимает одну из следующих форм

$$\left[\begin{array}{c} S_{ij} \\ \hline O_{d_i, m_j+1} \end{array} \right], \quad [S_{ij} | O_{n_i+1, \delta_j}], \quad \left[\begin{array}{c|c} S_{ij} & O_{n_i+1, \delta_j} \\ \hline O_{d_i, m_j+1} & O_{d_i, \delta_j} \end{array} \right] \quad (4.157\text{а})$$

и содержит d_i нулевых строк и/или δ_j нулевых столбцов, а реальные степени би-полиномиальной функции $p'_{ij}(t, \tau)$ остаются равными n_i и m_j ;

• **уменьшение** порядков максимум на $d_i = n_i - n'_i > 0$ и/или $\delta_j = m_j - m'_j > 0$ возможно **не всегда**, а лишь при условии, что реальные степени поверхности $p_{ij}(t, \tau)$, зависящие от расположения вершин кластера U_{ij} , меньше формальных, т. е. $n'_i = n_i - d_i$ и/или $m'_j = m_j - \delta_j$. Дефекты степеней d_i и δ_j равны числам нулевых строк и столбцов в матрице S_{ij} , принимающей одну из следующих форм:

$$\left[\begin{array}{c|c} S'_{ij} & \\ \hline 0 & \end{array} \right]_{d_i, m_j+1}, \left[S'_{ij} \middle| \begin{array}{c} 0 \\ \hline 0 \end{array} \right]_{n_i+1, \delta_j}, \left[\begin{array}{c|c} S'_{ij} & 0 \\ \hline 0 & 0 \end{array} \right]_{d_i, m'_j+1}^{n'_i+1, \delta_j}. \quad (4.157б)$$

При нулевом дефекте понижение соответствующего порядка невозможно.

На рис. 4.145 показаны фазы редактирования трёхсегментной поверхности с порядками кластеров $n_1=n_2=n_3=3$, $m_1=m_2=3$, $m_3=2$. После повышения τ -порядка второго сегмента до $m'_2=6$ у кластера U'_{12} появились дополнительные вершины, перемещением которых удалось изменить форму сегмента с сохранением **нормированной** гладкости сопряжения со смежными сегментами.

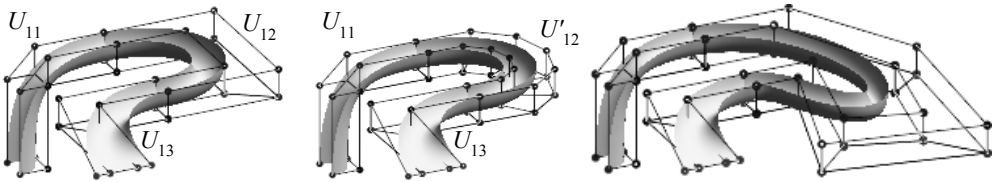


Рис. 4.145

□ Целью **второй** задачи является увеличение числа сегментов и повышение гибкости составной поверхности путём **разделения кластеров**. Положим в основу разделения кластера $U \in V^{(n+1) \times (m+1)}$, на котором по (4.112) построен сегмент $p(t, \tau)$, методику разделения линии Безье, описанную в п. 2.2.3.

Назначим параметры $r, \rho \in (0, 1)$ и **точку разделения** $P_{rp} = p(r, \rho)$, в которой новые кластеры и сегменты соединяются углами. Деление надвое каждого параметрического интервала $t, \tau \in [0, 1]$ даёт сегменты $p_{ij}(t, \tau) \forall i, j \in \{1, 2\}$, представляющие единый полином от разных комбинаций параметров $\{t, \tau\}$ и совместно покрывающие исходный сегмент $p(t, \tau)$:

$$\begin{aligned} p_{11}(t, \tau) &= p(rt, \rho\tau), & p_{12}(t, \tau) &= p(rt, (1-\rho)\tau + \rho), \\ p_{21}(t, \tau) &= p((1-r)t + r, \rho\tau), & p_{22}(t, \tau) &= p((1-r)t + r, (1-\rho)\tau + \rho). \end{aligned} \quad (4.158a)$$

Подставив сюда модели сегментов $p_{ij}(t, \tau) = T_{n_i}^T(t) H_{n_i}^T U_{ij} H_{m_j} T_{m_j}(\tau)$ и $p(t, \tau) = T_n^T(t) H_n^T U H_m T_m(\tau)$, сведём уравнения связи узловых матриц к порядкам кластеров $U_{ij} \in V^{(n_i+1) \times (m_j+1)}$.

$$\begin{cases} T_{n_1}^T(t)H_{n_1}^T U_{11}H_{m_1}T_{m_1}(\tau)=T_{n_1}^T(rt)E_{n_1+1,n+1}H_n^T UH_mE_{m+1,m_1+1}T_{m_1}(\rho\tau), \\ T_{n_1}^T(t)H_{n_1}^T U_{12}H_{m_2}T_{m_2}(\tau)=T_{n_1}^T(rt)E_{n_1+1,n+1}H_n^T UH_mE_{m+1,m_2+1}T_{m_2}((1-\rho)\tau+\rho), \\ T_{n_2}^T(t)H_{n_2}^T U_{21}H_{m_1}T_{m_1}(\tau)=T_{n_2}^T((1-r)t+r)E_{n_2+1,n+1}H_n^T UH_mE_{m+1,m_1+1}T_{m_1}(\rho\tau), \\ T_{n_2}^T(t)H_{n_2}^T U_{22}H_{m_2}T_{m_2}(\tau)= \\ =T_{n_2}^T((1-r)t+r)E_{n_2+1,n+1}H_n^T UH_mE_{m+1,m_2+1}T_{m_2}((1-\rho)\tau+\rho). \end{cases} \quad (4.158б)$$

Четырежды применив преобразование (2.58а), получим формулы вычисления матриц кластеров разделённых сегментов:

$$\begin{cases} U_{11}=(H_n E_{n+1,n_1+1} D_{n_1}(r,0) H_{n_1}^{-1})^T U (H_m E_{m+1,m_1+1} D_{m_1}(\rho,0) H_{m_1}^{-1}), \\ U_{12}=(H_n E_{n+1,n_1+1} D_{n_1}(r,0) H_{n_1}^{-1})^T U (H_m E_{m+1,m_2+1} D_{m_2}(1-\rho,\rho) H_{m_2}^{-1}), \\ U_{21}=(H_n E_{n+1,n_2+1} D_{n_2}(1-r,r) H_{n_2}^{-1})^T U (H_m E_{m+1,m_1+1} D_{m_1}(\rho,0) H_{m_1}^{-1}), \\ U_{22}=(H_n E_{n+1,n_2+1} D_{n_2}(1-r,r) H_{n_2}^{-1})^T U (H_m E_{m+1,m_2+1} D_{m_2}(1-\rho,\rho) H_{m_2}^{-1}). \end{cases} \quad (4.158в)$$

При разделении кластера U лишь по одному параметрическому направлению его заменяют только два новых кластера:

$$t: \begin{cases} U_{11}=(H_n E_{n+1,n_1+1} D_{n_1}(r,0) H_{n_1}^{-1})^T U, \\ U_{21}=(H_n E_{n+1,n_2+1} D_{n_2}(1-r,r) H_{n_2}^{-1})^T U \end{cases} \quad (4.158г)$$

либо

$$\tau: \begin{cases} U_{11}=U (H_m E_{m+1,m_1+1} D_{m_1}(\rho,0) H_{m_1}^{-1}), \\ U_{12}=U (H_m E_{m+1,m_2+1} D_{m_2}(1-\rho,\rho) H_{m_2}^{-1}). \end{cases} \quad (4.158д)$$

Возможность выбора новых порядков $n_i \neq n$ и $m_j \neq m$ определяется структурой МПК $S=H_n^T UH_m$ и $S_{ij}=H_{n_i}^T U_{ij} H_{m_j}$. Выбор значений $n_i > n$ и $m_j > m$ возможен *всегда*, но реальные степени сегментов останутся равными n и m , а структуры матриц S_{ij} аналогичны (4.157а). Для выбора $n_i < n$ или $m_j < m$ полином $p(t, \tau)$ должен иметь дефекты степеней $d_i \geq n - n_i$ или $\delta_j \geq m - m_j$, а структуры матрицы S должны быть аналогичными (4.157б).

Как и в (2.87) **двойная гладкость сопряжения новых сегментов друг с другом получится только при $r=\rho=0.5$** — делении сегмента средней точкой.

Возьмём для испытаний разработанного алгоритма односегментную поверхность, показанную на рис. 4.108,в без своей характеристической сети размерами 10×10 узлов. Полное изображение с ХПС U на рис. 4.146,а подтверждает сделанный в п. 4.4.4 вывод о большом удалении околоцентральных узлов сетки от поверхности Безье при высоких степенях полинома (4.112). Подобрал параметры $r=0.4$ и $\rho=0.5$, при которых точка деления P_r находится на острие «носа», получим по (4.158в) четыре кластера $U_{11} \div U_{22}$, довольно-таки плотно охватывающие составную поверхность в отличие от ХПС U (рис. 4.146,б).

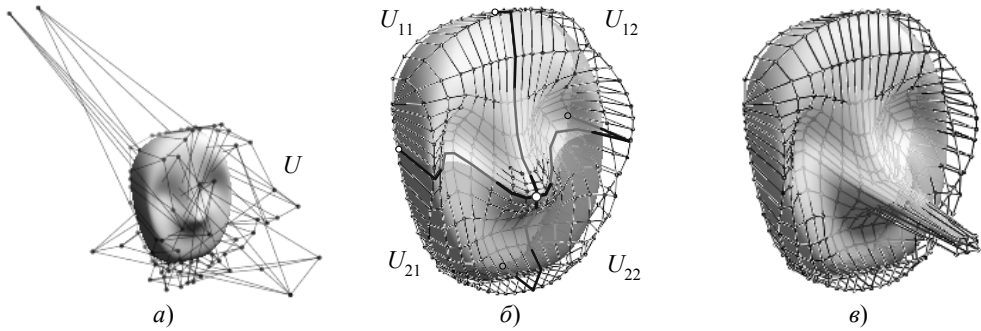


Рис. 4.146

Теперь, с превращением односегментной поверхности в составную, появились более мощные возможности её редактирования. Одна из них — перемещение точки деления и окружающих её восьми узловых точек — реализована на рис. 4.146,в, и такой остроты «носа» нельзя достичь без разделения кластера.

Пример 4.30. Разделить на четыре сегмента *минимальных* степеней поверхность Безье (рис. 4.147,а), у которой вершины ХПС порядков $n=m=3$ составляют матрицу

$$U = \begin{bmatrix} [8 & 0 & 0] & [8 & 0 & -12] & [-8 & 0 & -12] & [-8 & 0 & 0] \\ [6 & -4 & 0] & [6 & -4 & -9] & [-6 & -4 & -9] & [-6 & -4 & 0] \\ [4 & -4 & 0] & [4 & -4 & -6] & [-4 & -4 & -6] & [-4 & -4 & 0] \\ [2 & 0 & 0] & [2 & 0 & -3] & [-2 & 0 & -3] & [-2 & 0 & 0] \end{bmatrix}.$$

Решение. Для определения минимально возможных степеней разделённых сегментов вычислим МПК исходной поверхности Безье:

$$S = H_3^T U H_3 = \begin{bmatrix} [8 & 0 & 0] & [0 & 0 & -36] & [-48 & 0 & 36] & [32 & 0 & 0] \\ [-6 & -12 & 0] & [0 & 0 & 27] & [36 & 0 & -27] & [-24 & 0 & 0] \\ [0 & 12 & 0] & [0 & 0 & 0] & [0 & 0 & 0] & [0 & 0 & 0] \\ [0 & 0 & 0] & [0 & 0 & 0] & [0 & 0 & 0] & [0 & 0 & 0] \end{bmatrix}.$$

В ней есть одна последняя нулевая строка, значит, в t -направлении поверхность *параболическая* с дефектом степени $d=1$, в τ -направлении — кубическая, а минимальные степени новых сегментов равны $n_1=n_2=n-d=2$ и $m_1=m_2=m=3$.

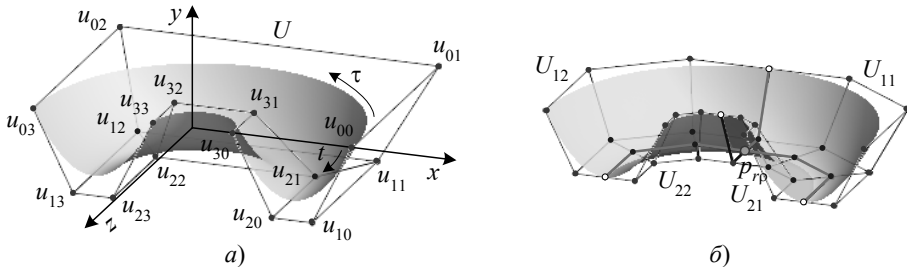


Рис. 4.147

Задав параметры разделения $r=\rho=0.5$ и вычислив по (4.158в) матрицы кластеров $U_{11} \div U_{22}$, убеждаемся, что построенные на них поверхности Безье (4.151) точно совпадают с исходной поверхностью (рис. 4.147,б). □

□ Цель *третьей* задачи — *объединение кластеров* в один для упрощения структуры составной поверхности без изменения её формы и, возможно,

уменьшения числа вершин ХПС. Эта задача по сути обратна к задаче разделения кластера. Следует понимать, что она разрешима при жёстких ограничениях на расположение вершин объединяемых кластеров.

По аналогии с (2.88) составим алгоритм замены четырёх матрично сопряжённых кластеров $U_{ij} \in V^{(n_i+1) \times (m_j+1)}$ с $i, j \in \{1, 2\}$ одним кластером $U \in V^{(n+1) \times (m+1)}$, поверхность которого $p(t, \tau)$ делится точкой

$$P_{rp} = p_{11}(1, 1) = p_{12}(1, 0) = p_{21}(0, 1) = p_{22}(0, 0)$$

на четыре поверхности Безье $p_{ij}(t, \tau)$, построенные на кластерах U_{ij} . Используем для вычисления параметров r и ρ точку P_{rp} и четыре окружающие её граничные вершины кластеров, обозначенные для простоты как на рис. 4.143:

$$r = \left(1 + \frac{n_2 |P_{rp} - v_1|}{n_1 |P_{rp} - v_0|} \right)^{-1}, \quad \rho = \left(1 + \frac{m_2 |P_{rp} - w_1|}{m_1 |P_{rp} - w_0|} \right)^{-1}. \quad (4.159a)$$

Возможность алгоритмического объединения четырёх сегментов сильно ограничена таким расположением вершин кластеров $U_{11} \div U_{22}$, что построенные на них **поверхности Безье** $p_{ij}(t, \tau)$ **представляют единый векторный полином** от разных комбинаций параметров $t, \tau \in [0, 1]$. Приведя уравнения (4.158a) к порядкам n и m , получим соотношения

$$\begin{cases} E_{n+1, n_1+1} H_{n_1}^T U_{11} H_{m_1} E_{m_1+1, m+1} = D_n^T(r, 0) H_n^T U H_m D_m(\rho, 0), \\ E_{n+1, n_1+1} H_{n_1}^T U_{12} H_{m_2} E_{m_2+1, m+1} = D_n^T(r, 0) H_n^T U H_m D_m(1-\rho, \rho), \\ E_{n+1, n_2+1} H_{n_2}^T U_{21} H_{m_1} E_{m_1+1, m+1} = D_n^T(1-r, r) H_n^T U H_m D_m(\rho, 0), \\ E_{n+1, n_2+1} H_{n_2}^T U_{22} H_{m_2} E_{m_2+1, m+1} = D_n^T(1-r, r) H_n^T U H_m D_m(1-\rho, \rho). \end{cases}$$

Отсюда вытекает **тест возможности объединения сегментов** $p_{11}(t, \tau), p_{12}(t, \tau), p_{21}(t, \tau)$ и $p_{22}(t, \tau)$ **в один сегмент** $p(t, \tau)$ в виде системы матричных уравнений

$$\begin{cases} U_{11} H_{m_1} E_{m_1+1, m+1} D_m^{-1}(\rho, 0) = U_{12} H_{m_2} E_{m_2+1, m+1} D_m^{-1}(1-\rho, \rho), \\ U_{11}^T H_{n_1} E_{n_1+1, n+1} D_n^{-1}(r, 0) = U_{21}^T H_{n_2} E_{n_2+1, n+1} D_n^{-1}(1-r, r), \\ U_{21} H_{m_1} E_{m_1+1, m+1} D_m^{-1}(\rho, 0) = U_{22} H_{m_2} E_{m_2+1, m+1} D_m^{-1}(1-\rho, \rho), \end{cases} \quad (4.159б)$$

а также формула вычисления матрицы вершин объединяющего кластера по матрице ХПС сегмента $p_{11}(t, \tau)$:

$$U = \left(H_{n_1} E_{n_1+1, n+1} D_n^{-1}(r, 0) H_n^{-1} \right)^T U_{11} \left(H_{m_1} E_{m_1+1, m+1} D_m^{-1}(\rho, 0) H_m^{-1} \right). \quad (4.159в)$$

При объединении сегментов лишь по одному параметрическому направлению (t либо τ) в (4.159) берутся соответствующие параметр разделения (r либо ρ) и пара тестируемых кластеров ($\{U_{11}, U_{21}\}$ либо $\{U_{11}, U_{12}\}$).

Подстановки $T_{n_i}(t) = E_{n_i+1, n+1} T_n(t)$ и $T_{m_j}(\tau) = E_{m_j+1, m+1} T_m(\tau)$, сделанные при выводе (4.159б, в), равносильны **всегда** при выборе порядков объединённого сегмента $n \geq \max\{n_1, n_2\}$ и $m \geq \max\{m_1, m_2\}$. Наименьшие значения n и m , при ко-

торых задача объединения при условиях (4.159б) имеет точное решение (4.159в), ограничены **наибольшими** реальными степенями полиномов $p_{ij}(t, \tau)$.

Испытаем алгоритм на объединении четырёх сегментов Безье с кластерами $U_{11} \div U_{22}$ порядков $n_1=n_2=2$ и $m_1=m_2=3$ (см. рис. 4.147,б) в поверхность **минимальных** степеней $n=2$ и $m=3$ (рис. 4.148), определяемых максимальными степенями объединяемых сегментов.

По точке $P_{rp}=u_{00}^{22}$ и четырём граничным вершинам кластеров $v_0=u_{13}^{11}$, $v_1=u_{13}^{21}$,

$w_0=u_{32}^{11}$ и $w_1=u_{31}^{12}$ по (4.159а) находим параметры $r=\rho=0.5$. Выполнение теста (4.159б) позволяет найти по (4.159в) матрицу вершин ХПС

$$U = \begin{bmatrix} [8 & 0 & 0] & [8 & 0 & -12] & [-8 & 0 & -12] & [-8 & 0 & 0] \\ [5 & -6 & 0] & [5 & -6 & -7.5] & [-5 & -6 & -7.5] & [-5 & -6 & 0] \\ [2 & 0 & 0] & [2 & 0 & -3] & [-2 & 0 & -3] & [-2 & 0 & 0] \end{bmatrix}.$$

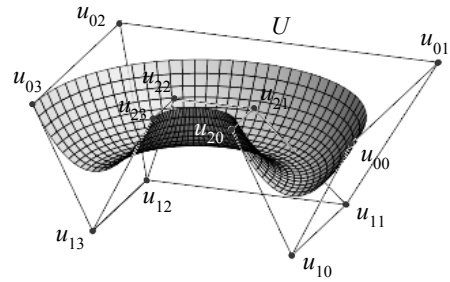


Рис. 4.148

4.5.3. Рациональные составные поверхности Безье

Форма **рациональной составной поверхности Безье** определяется порядками кластеров $n_i \geq 1$ и $m_j \geq 1$, расположением вершин в кластерах U_{ij} и весовыми матрицами $\Omega_{ij} \forall i=\overline{1, N}, j=\overline{1, M}$. Модель $\{i, j\}$ -го рационального сегмента объединяет модели (4.115) и (4.154):

$$p_{ij}(t, \tau) = \frac{B_{n_i}^T(t)(U_{ij} \bullet \Omega_{ij})B_{m_j}(\tau)}{B_{n_i}^T(t)\Omega_{ij}B_{m_j}(\tau)} \quad \forall t, \tau \in [0, 1]. \quad (4.160a)$$

Ввиду актуальности не весов, а их отношений, далее примем веса угловых вершин кластеров $\omega_{00}^{ij} = \omega_{n_i, 0}^{ij} = \omega_{0, m_j}^{ij} = \omega_{n_i, m_j}^{ij} = 1$. Свойства и возможности рациональных сегментов Безье изучены в п. 4.4.4, а условия их гладкого сопряжения аналогичны (4.155) в рациональной модификации линейного аналога (2.90).

Мощные возможности формообразования составной поверхности манипулированием лишь весами вершин ХПС сегментов показаны на рис. 4.149. Назначив отмеченным вершинам указанные в овалах веса (у остальных вершин веса единичные), удалось существенно изменить форму поверхности автомобильного кузова, построенной на рис. 4.144.

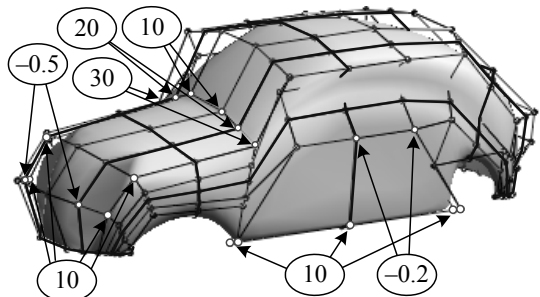


Рис. 4.149

Конструирование составной рациональной поверхности сводится к заданию кластерных матриц вершин и весов с матричной

$$U_{ij} = \begin{bmatrix} P_{i-1,j-1} & u_{01}^{ij} & \dots & u_{0,m_j-1}^{ij} & P_{i-1,j} \\ u_{10}^{ij} & u_{11}^{ij} & \dots & u_{1,m_j-1}^{ij} & u_{1,m_j}^{ij} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ u_{n_i-1,0}^{ij} & u_{n_i-1,1}^{ij} & \dots & u_{n_i-1,m_j-1}^{ij} & u_{n_i-1,m_j}^{ij} \\ P_{i,j-1} & u_{n_i,1}^{ij} & \dots & u_{n_i,m_j-1}^{ij} & P_{ij} \end{bmatrix}, \quad \Omega_{ij} = \begin{bmatrix} 1 & \omega_{01}^{ij} & \dots & \omega_{0,m_j-1}^{ij} & 1 \\ \omega_{10}^{ij} & \omega_{11}^{ij} & \dots & \omega_{1,m_j-1}^{ij} & \omega_{1,m_j}^{ij} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \omega_{n_i-1,0}^{ij} & \omega_{n_i-1,1}^{ij} & \dots & \omega_{n_i-1,m_j-1}^{ij} & \omega_{n_i-1,m_j}^{ij} \\ 1 & \omega_{n_i,1}^{ij} & \dots & \omega_{n_i,m_j-1}^{ij} & 1 \end{bmatrix}$$

либо векторной

$$U_i = \begin{bmatrix} u_{00}^i & u_{01}^i & \dots & u_{0,m_i-1}^i & u_{0,m_i}^i \\ u_{10}^i & u_{11}^i & \dots & u_{1,m_i-1}^i & u_{1,m_i}^i \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ u_{n_i-1,0}^i & u_{n_i-1,1}^i & \dots & u_{n_i-1,m_i-1}^i & u_{n_i-1,m_i}^i \\ u_{n_i,0}^i & u_{n_i,1}^i & \dots & u_{n_i,m_i-1}^i & u_{n_i,m_i}^i \end{bmatrix}, \quad \Omega_i = \begin{bmatrix} 1 & \omega_{01}^i & \dots & \omega_{0,m_i-1}^i & 1 \\ \omega_{10}^i & \omega_{11}^i & \dots & \omega_{1,m_i-1}^i & \omega_{1,m_i}^i \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \omega_{n_i-1,0}^i & \omega_{n_i-1,1}^i & \dots & \omega_{n_i-1,m_i-1}^i & \omega_{n_i-1,m_i}^i \\ 1 & \omega_{n_i,1}^i & \dots & \omega_{n_i,m_i-1}^i & 1 \end{bmatrix}$$

индексацией кластеров. Второй способ не связывает конструирование сложной поверхности возможностью матричной организации кластеров и нумерует их индексами $i \in \{1, N\}$ в **произвольном** порядке. Кластерные матрицы $\{U_i, \Omega_i\}$ составляют **линейные** списки $U = \{U_1, U_2, \dots, U_N\}$, $\Omega = \{\Omega_1, \Omega_2, \dots, \Omega_N\}$ и используются для построения поверхности i -го сегмента по формуле

$$p_i(t, \tau) = \frac{B_{n_i}^T(t)(U_i \bullet \Omega_i)B_{m_i}(\tau)}{B_{n_i}^T(t)\Omega_i B_{m_i}(\tau)} \quad \forall t, \tau \in [0, 1]. \quad (4.160б)$$

Примером списковой структуры служит модель полиэдра с произвольной нумерацией граней (см. п. 1.5.4).

Рассмотрим ряд задач конструирования составных поверхностей с рациональными сегментами Безье. Линейные аналоги некоторых задач изучались ранее в п. 2.2.4.

□ Простейшей составной поверхностью является **полигональная грань** — часть плоскости, ограниченная замкнутой полилинией $P = P_1 P_2 \dots P_n P_1$. Если этот полигон **выпуклый**, то его поверхность можно покрыть $N = n$ треугольниками $sP_i P_{i+1} c$, где $i \in [1, n]$ и $P_{n+1} \equiv P_1$, с общей вершиной в точке c , находящейся **внутри** полигона, например, в его центре $c = \text{mean}(P)$.

В более общем случае **звёздного** полигона P (см. п. 1.1.6), всегда имеющего выпуклое **ядро** K , возьмём общую точку в центре ядра $c = \text{mean}(K)$, откуда на прямую видны все вершины $P_i \div P_n$ (рис. 4.150,а). Поскольку функция *kern_poly* (её блок-схема приведена на рис. 1.86,д) работает в плоскости **двух** координат, то для получения ядра плоского 3d-полигона нужно найти по (3.25а) матрицу C_K преобразования системы координат $xuz \rightarrow x'y'z^0$, перевести полигон во фронтальную плоскость $x'y'$, определённую точкой P_1 и нормалью $\text{norm}(P)$, вычислить полигон ядра K' и вернуть его в исходную систему координат:

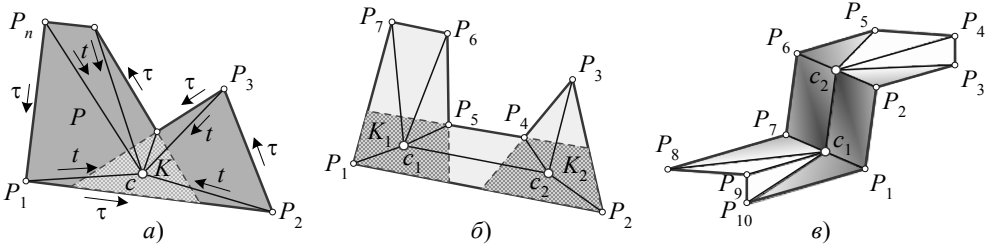


Рис. 4.150

$$C_K = MP2(P_1, \text{norm}(P), O_3, z^0), \quad \tilde{P}' = \tilde{P}C_K, \quad (4.161a)$$

$$K' = \text{kern_poly}(P'E_{3,2})E_{2,3}, \quad \tilde{K} = \tilde{K}'C_K^{-1}, \quad c = \text{mean}(K).$$

В модели $N=n$ -сегментной поверхности звёздного полигона с порядками сегментов $n_1=m_1=\dots=m_n=1$ используются кластерные матрицы

$$U_i = \begin{bmatrix} P_i & P_{i+1} \\ c & c \end{bmatrix}, \quad \Omega_i = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \forall i \in \overline{1, n}. \quad (4.161b)$$

Ввиду прямолинейности границ треугольных сегментов их построение по (4.160б) возможно с квантами параметров $\Delta t = \Delta \tau = 1$.

Для построения составной поверхности **произвольного** плоского полигона, не имеющего единого ядра, нужно найти центры всех ядер, из которых видны в совокупности все вершины $P_1 \div P_n$, и составить ленту из n трёх- или четырёх-угольных сегментов, покрывающих полигон. К примеру, в изображённом на рис. 4.150,в полигоне с двумя ядрами K_1 и K_2 берутся две точки $c_1 \in K_1$ и $c_2 \in K_2$, входящие в кластерные матрицы вершин сегментов:

$$U_1 = \begin{bmatrix} P_1 & P_2 \\ c_1 & c_2 \end{bmatrix}, \quad U_2 = \begin{bmatrix} P_2 & P_3 \\ c_2 & c_2 \end{bmatrix}, \quad U_3 = \begin{bmatrix} P_3 & P_4 \\ c_2 & c_2 \end{bmatrix}, \quad U_4 = \begin{bmatrix} P_4 & P_5 \\ c_2 & c_1 \end{bmatrix}, \quad (4.162)$$

$$U_5 = \begin{bmatrix} P_5 & P_6 \\ c_1 & c_1 \end{bmatrix}, \quad U_6 = \begin{bmatrix} P_6 & P_7 \\ c_1 & c_1 \end{bmatrix}, \quad U_7 = \begin{bmatrix} P_7 & P_1 \\ c_1 & c_1 \end{bmatrix}.$$

Метод внутренней точки применим и для построения поверхности **неплоского**, точнее, **мультипланарного полигона**, вершины которого лежат в нескольких плоскостях, пересекающихся по m линиям сгибов. Достаточно на этих линиях выбрать точки $c_1 \div c_m$, из которых все вершины полигона видны в его плоскостях, сформировать подобно (4.162) кластерные матрицы вершин и строить по ним поверхности сегментов (рис. 4.150,в).

□ Задача **скругления углов полигональной грани** $P = P_1 P_2 \dots P_n P_1$ рациональными дугами окружностей радиусов $r_i \geq 0 \quad \forall i \in [1, n]$ (рис. 4.151,а) решается по алгоритму *rat_poly* скругления углов 2d-полигона (см. блок-схему на рис. 2.63), дополненному расчётом по (4.161а) ядра полигона K с центром $c = \text{mean}(K) \in P$ и расширением кластерных векторов вершин $[e \ a]$, $[a \ P_i \ b]$, $[e \ a_1]$ и весов $[1 \ 1]$, $[1 \ \sin(\gamma_i) \ 1]$ до матриц

$$\begin{bmatrix} e & a \\ c & c \end{bmatrix}, \quad \begin{bmatrix} a & P_i & b \\ c & c & c \end{bmatrix}, \quad \begin{bmatrix} e & a_1 \\ c & c \end{bmatrix} \quad \text{и} \quad \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & \sin(\gamma_i) & 1 \\ 1 & 1 & 1 \end{bmatrix}. \quad (4.163a)$$

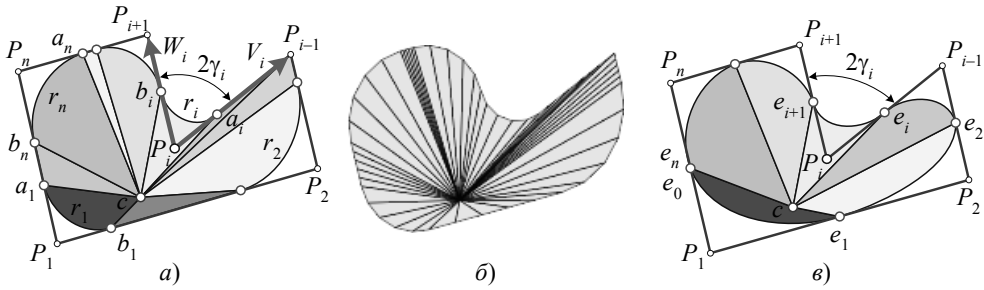


Рис. 4.151

Ввиду криволинейности τ -границных дуг окружностей построение поверхностей сегментов по (4.160б) должно выполняться с достаточно малым квантом параметра $\Delta\tau \ll 1$ при $\Delta t = 1$, как показано на рис. 4.151,б.

Непрерывное **сглаживание углов полигональной грани** $P_1 \dots P_n P_1$ дугами **эллипсов**, заключёнными между точками $e_i = 0.5(P_i + P_{i+1}) \quad \forall i = 0, n$, где $P_0 = P_n$ и $P_{n+1} = P_1$ (рис. 4.151,в), производится на n кластерах второго порядка с матрицами, расширяющими кластерные векторы (2.96в) аналогично (4.161б):

$$U_i = \begin{bmatrix} e_{i-1} & P_i & e_i \\ c & c & c \end{bmatrix}, \quad \Omega_i = \begin{bmatrix} 1 & \sin(\gamma_i) & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \forall i = \overline{1, n}. \quad (4.163б)$$

□ Применим метод создания поверхности грани P из сегментов с общей компланарной точкой $c \in P$ для конструирования составной **конической поверхности** (см. рис. 4.58), образованной движением отрезка между фиксированной точкой $c \notin P$, лежащей **вне** плоскости грани, и составной направляющей линией $p_n(\tau)$. Последняя строится (см. п. 2.2.4) на полилинии $P = P_1 P_2 \dots P_n$ либо полигоне $P = P_1 P_2 \dots P_n P_1$, углы которых могут быть скруглены алгоритмами *rat_line* и *rat_poly* дугами окружностей радиусов $r_i \geq 0$ (см. рис. 2.60, 2.62) либо непрерывно сглажены дугами эллипсов (см. рис. 2.61, 2.64).

Все шесть видов составной конической поверхности на рис. 4.152, построены по (4.160б) на кластерах, матрицы которых $\{U_i, \Omega_i\}$ получены расширением векторов вершин и весов из (2.91)–(2.96) вторыми строками аналогично (4.163). В случае полигонального основания коническая поверхность становится замкнутой. Если направляющей линией является полигон P , то получается боковая поверхность **пирамиды**, острые рёбра которой скругляются заданием радиусов скругления $r_i > 0$. Для получения всюду изогнутой составной поверхности **конуса** углы полигона основания нужно сгладить дугами эллипсов.

Напомним, что плоская грань основания замкнутой конической поверхности строится на кластерных матрицах вершин U_i с точкой $c \in \text{kern}(P)$.

□ Другим представителем класса линейчатых является **цилиндрическая поверхность**, формируемая движением отрезка образующей прямой по направляющей линии первого основания $p_n(\tau)$ параллельно фиксированному вектору W (см. рис. 4.57). При длине отрезка, равной $|W|$, его второй конец перемещает-

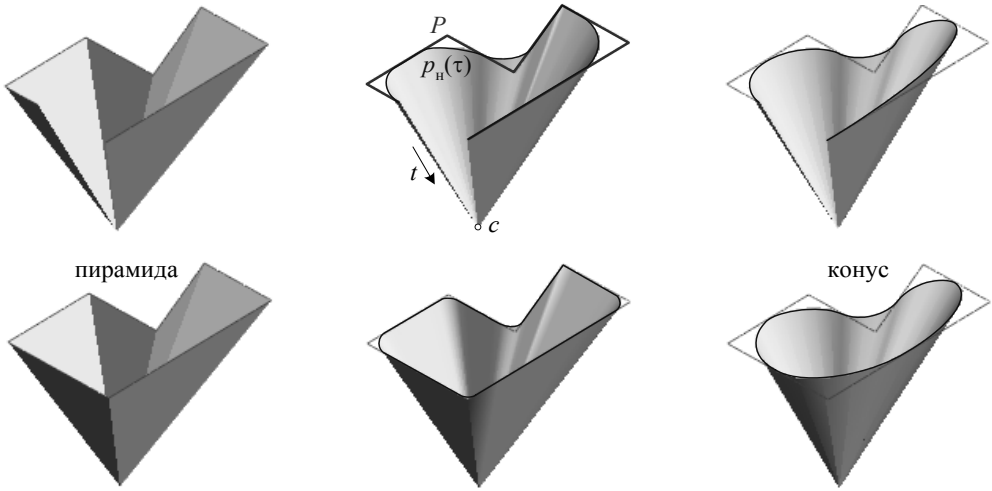


Рис. 4.152

ся по линии второго основания $q_n(\tau) = p_n(\tau) + W$. Направляющая $p_n(\tau)$ строится на полилинии $P = P_1 P_2 \dots P_n$ либо полигоне $P = P_1 P_2 \dots P_n P_1$, углы которых скругляются дугами окружностей радиусов $r_i \geq 0$ либо сглаживаются дугами эллипсов. Вершины параллельного объекта Q лежат в точках $Q_i = P_i + W$.

Все шесть видов составной цилиндрической поверхности, показанных на рис. 4.153, построены по (4.160, б) на кластерах, у которых, в отличие от (4.163), вторые строки матриц вершин U_i содержат точки полигона Q

$$\begin{bmatrix} e & a \\ e+W & a+W \end{bmatrix}, \begin{bmatrix} a & P_i & b \\ a+W & P_i+W & b+W \end{bmatrix}, \begin{bmatrix} e & a_1 \\ e+W & a_1+W \end{bmatrix}, \quad (4.164a)$$

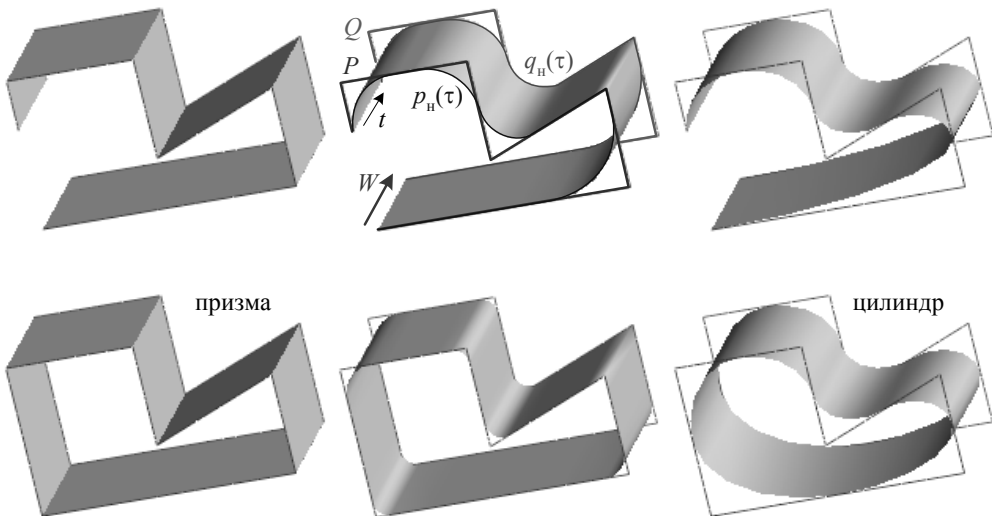


Рис. 4.153

а в матрицах весов Ω_i вторые строки совпадают с первыми:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 \sin(\gamma_i) & 1 \\ 1 \sin(\gamma_i) & 1 \end{bmatrix}. \quad (4.164б)$$

В случае полигональной основы цилиндрическая поверхность является замкнутой. Если направляющей линией $p_n(\tau)$ является полигон, то получается боковая поверхность **призмы**, острые рёбра которой можно скруглить заданием радиусов $r_i > 0$ скругления углов полигонов оснований P и Q . Для получения всюду изогнутой составной поверхности **цилиндра** углы полигонов оснований нужно сглаживать дугами эллипсов.

□ Принцип формирования кластеров составных конических и цилиндрических поверхностей легко переносится в задачу кластеризации произвольной рациональной **линейчатой поверхности с двумя** направляющими линиями $p_n(\tau)$ и $q_n(\tau)$, построенными на полилиниях либо полигонах P и Q . Эти характеристические объекты и их кластерные списки $\{U^P, \Omega^P\}$ и $\{U^Q, \Omega^Q\}$, рассчитанные по (2.91)–(2.96), должны обладать равными числом вершин n , числом кластеров N и размерами одноиндексных кластеров $\{U_i^P, U_i^Q\}$ и $\{\Omega_i^P, \Omega_i^Q\}$.

Обобщая (4.163) и (4.164), сформируем кластеры составной линейчатой поверхности путём соединения кластеров направляющих линий:

$$U_i = \begin{bmatrix} U_i^P \\ U_i^Q \end{bmatrix}, \quad \Omega_i = \begin{bmatrix} \Omega_i^P \\ \Omega_i^Q \end{bmatrix} \quad \forall i = \overline{1, N}. \quad (4.165а)$$

На рис. 4.154 построено по две линейчатые поверхности на основе полилиний и полигонов с разными вариантами скругления боковых рёбер дугами окружностей и сглаживания их дугами эллипсов.

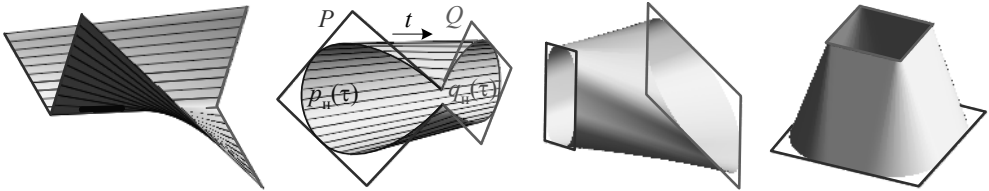


Рис. 4.154

Дальнейшее обобщение метода (4.165а) касается синтеза **полилинейчатой поверхности**, образованной движением ломаной линии по $M+1$ направляющим линиям $p_{n0}(\tau) \div p_{nM}(\tau)$, построенным на полилиниях (полигонах) $P_0 \div P_M$. Эти характеристические объекты должны иметь равные число вершин n , число кластеров N и размеры линейных кластеров $U_i^{P_j}$ и $\Omega_i^{P_j}$. Кластеры составной полилинейчатой поверхности образуют матричную структуру

$$U_{ji} = \begin{bmatrix} U_i^{P_{j-1}} \\ U_i^{P_j} \end{bmatrix}, \quad \Omega_{ji} = \begin{bmatrix} \Omega_i^{P_{j-1}} \\ \Omega_i^{P_j} \end{bmatrix} \quad \forall i = \overline{1, N}, \quad j = \overline{1, M}. \quad (4.165б)$$

На рис. 4.155 построено две полилинейчатые поверхности на основе трёх полилиний и четырёх полигонов со скруглением и сглаживанием боковых рёбер.

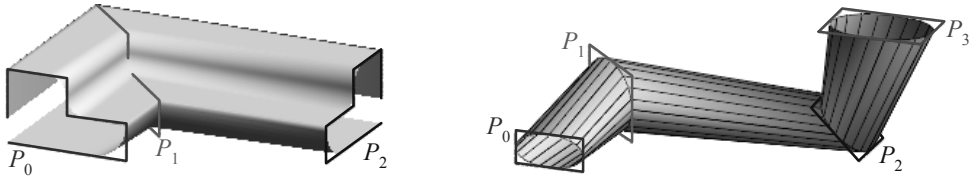


Рис. 4.155

□ Теперь построим составные поверхности, содержащие в одном либо обоих параметрических направлениях половину или полную окружность. Односегментные реализации этих линий, показанные на рис. 2.36,б,в, отличаются весьма «рыхлыми» характеристическими полилиниями, узловые точки которых далеко отстоят от порождённых ими рациональных решений третьего и пятого порядков. Наиболее простой и плотной ХП обладает рациональная линия Безье второго порядка, представляющая четверть окружности (см. рис. 2.36,а).

Аналогично обстоит дело и с рациональными односегментными фрагментами *сферы*, имеющими один или два угла раствора в 180° или 360° и сложные характеристические матрицы вершин и весов (см. рис. 4.117). Наиболее простой и плотной ХПС $U \equiv U_1$ обладает однооктантный сегмент (см. рис. 4.116) с кластерными матрицами (4.118). Размножая его в одном или двух параметрических направлениях путём вращения вокруг объектных осей и отражения от координатных плоскостей, можно составлять четверть-, полу- и полносферические поверхности (рис. 4.156).

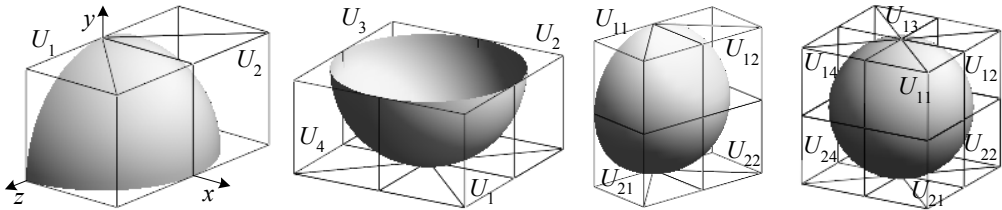


Рис. 4.156

Составление фрагментов поверхности *тора* с углами охвата определяющих его окружностей радиусов r и R в 180° или 360° осуществляется с помощью ХПС двух базовых кластеров с матрицами вершин

$$\begin{bmatrix} [0 \ r \ R] & [R \ r \ R] & [R \ r \ 0] \\ [0 \ r \ R \pm r] & [R \pm r \ r \ R \pm r] & [R \pm r \ r \ 0] \\ [0 \ 0 \ R \pm r] & [R \pm r \ 0 \ R \pm r] & [R \pm r \ 0 \ 0] \end{bmatrix}.$$

В символах « $\pm$ » знак «+» относится к координатам вершин кластера U_{11} внешнего сегмента тора, а «-» — к кластеру V_{11} внутреннего сегмента. Другие кластеры U_{ij} и V_{ij} получаются размножением в одном или двух параметрических направлениях с помощью вращения на кратный 90° угол вокруг объектной оси y и отражения от плоскости xz (рис. 4.157). Полный тор состоит из восьми внешних и восьми внутренних сегментов.

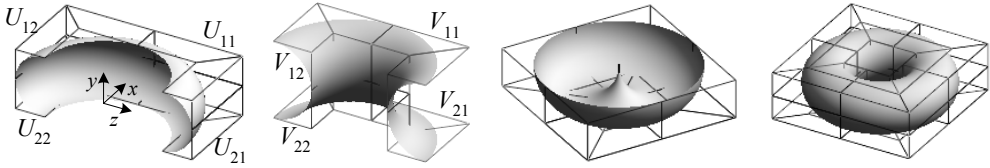


Рис. 4.157

□ В заключение рассмотрим три примера конструирования составных поверхностей полиэдров со скруглением их углов и рёбер цилиндрическими, сферическими и тороидальными сегментами, построение которых рассмотрено в п. 4.4.4. Ввиду невозможности матричной организации сегментов полиэдра их кластеры $\{U_i, \Omega_i\}$ нумеруются в произвольном порядке, составляют линейные списки вершин $U = \{U_1, U_2, \dots, U_N\}$, весов $\Omega = \{\Omega_1, \Omega_2, \dots, \Omega_N\}$ и служат основой построения поверхностей по (4.160б).

У первого объекта — полиэдра со скошенной гранью (рис. 4.158,а) — скруглению подвергся один из **выпуклых** трёхгранных углов путём вписывания в него сферы заданного радиуса с касанием по рёбрам (см. рис. 4.120,а,б). Сферический сегмент ① построен на ХПС с кластерными матрицами, рассчитанными по (4.121а–г). Составные сегменты ②, ③ и ④ представляют собой плоские полигоны. В каждом из них один из углов скруглён дугой окружности (см. рис. 4.151,а). Радиусы скругления углов рассчитаны по (4.121д). Кластерные матрицы сегментов, составляющих скруглённые полигоны, имеют вид (4.163а).

Трёхгранный **выпуклый** угол второго объекта (рис. 4.158,б) скруглён сегментом ① сферы, касающейся его граней (см. рис. 4.120,в,г). Кластерные матрицы этого сегмента рассчитаны по (4.122). Рёбра угла скруглены цилиндрическими (см. рис. 4.119) сегментами ②, ③ и ④, протяжёнными от краёв сферического сегмента до торцевых граней полиэдра. Вершины их ХПС рассчитаны по (4.120в). Оставшиеся плоские фрагменты граней ⑤, ⑥ и ⑦ строятся как полигоны без скругления углов (см. рис. 4.150) на кластерных матрицах, созданных по формулам (4.161).

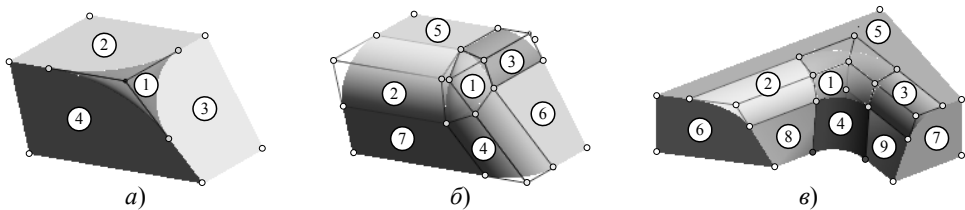


Рис. 4.158

Третий объект, изображённый на рис. 4.158,в, представляет собой полиэдр с **невыпуклым** трёхгранным углом, скругление которого (см. рис. 4.123) производится тороидальным сегментом ① с радиусами $R > r > 0$. Кластерные матрицы этого сегмента рассчитаны по алгоритму (4.123). Рёбра угла скруглены цилиндрическими сегментами ②, ③ и ④, протяжёнными от краёв тороидального сегмента до торцевых граней полиэдра. Вершины их ХПС рассчитаны по (4.120в).

Составные сегменты ⑤, ⑥ и ⑦ представляют собой плоские полигоны с одним скруглённым углом. Кластерные матрицы сегментов, составляющих скруглённый полигон, строятся по (4.163а). Оставшиеся плоские фрагменты граней ⑧ и ⑨ строятся как полигоны с не скруглёнными углами на кластерных матрицах, созданных по (4.161).

4.5.4. Эквидистантные поверхности

Поверхность, все точки которой отстоят от некоторой *базовой поверхности* (БП) на заданном расстоянии r , называется *эквидистантной* (ЭДП). Равноудалённые поверхности широко используются во многих конструкторских задачах создания вокруг 3d-объекта оболочки равномерной толщины, играющей роль защитной зоны, изоляционного слоя, припуска на механическую обработку, поверхности движения центра обрабатывающего инструмента и т. п.

□ Изучим построение ЭДП вокруг *незамкнутой* гладкой БП, описываемой дифференцируемой моделью $p(t, \tau)$ в конечных интервалах изменения параметров $t \in [t_n, t_k]$, $\tau \in [\tau_n, \tau_k]$ с вектором нормали $N(t, \tau) = V(t, \tau) \times W(t, \tau)$, вычисляемым по (1.94) с помощью направляющих векторов $V(t, \tau) = p'_t(t, \tau)$ и $W(t, \tau) = p'_\tau(t, \tau)$. Отложим ортогонально поверхности в обе стороны от точки p расстояния r (рис. 4.159):

$$\begin{aligned} e(t, \tau) &= p(t, \tau) + r\bar{N}(t, \tau), \\ q(t, \tau) &= p(t, \tau) - r\bar{N}(t, \tau) = 2p(t, \tau) - e(t, \tau). \end{aligned} \quad (4.166)$$

Назовем *внешней* ЭДП $e(t, \tau)$, расположенную с нормальной стороны БП, а противоположную поверхность $q(t, \tau)$ — *внутренней*.

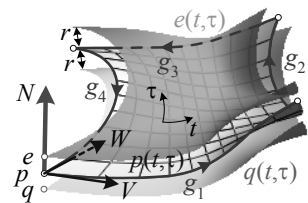


Рис. 4.159

Кроме этих поверхностей, условно параллельных БП и приближённо повторяющих её форму, полная ЭДП (рис. 4.160,а) включает ещё два типа граничных эквидистантных сегментов, описываемых моделями с независимыми от t и τ параметрами u и v :

- сегменты *цилиндрического* типа $c_1(u, v) \div c_4(u, v)$, гладко соединяющие края обеих ЭДП полукруглыми трубчатыми сегментами (рис. 4.160,б);
- сегменты *сферического* типа $s_1(u, v) \div s_4(u, v)$ (рис. 4.160,в), гладко соединяющие торцы трубчатых сегментов в углах базовой поверхности.

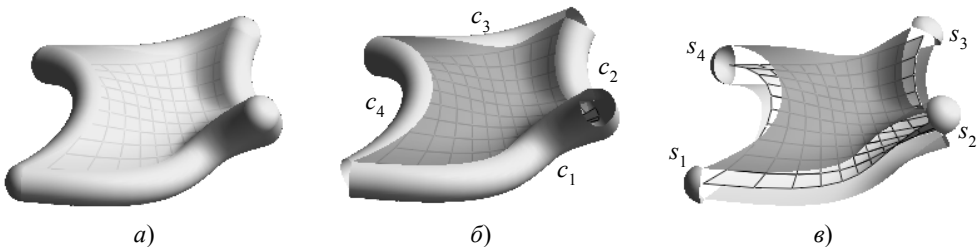


Рис. 4.160

Разработаем алгоритмы построения граничных эквидистантных сегментов. Ввиду произвольного, возможно, не полиномиального и не квадратичного описания граничных линий БП, рациональный подход к конструированию граничных ЭДП неприемлем. Эти поверхности с успехом могут быть созданы **кинематическим методом**, изложенным в разд. 4.3.

Трубчатая поверхность сегмента $c(u, v)$ формируется движением образующей **полуокружности** $o(v) = g(u) + N \cos(v) + B \sin(v) \quad \forall v \in [0, \pi]$, у которой центр скользят по граничной линии $g(u)$, плоскость ортогональна касательному вектору $V(u) = g'(u)$, концы опираются на эквидистантные линии $h(u)$ и $d(u)$, а выпуклость радиуса r обращена наружу (рис. 4.161). Все эти кинематические требования реализуются с помощью векторов нормали $N(u)$ и бинормали $B(u)$ длиной r , составляющих с $V(u)$ правую тройку $\{V, N, B\}$:

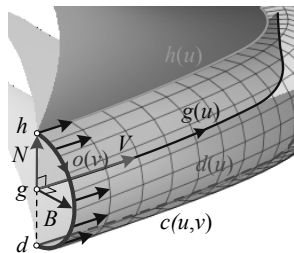


Рис. 4.161

$$\begin{aligned} N(u) &= h(u) - g(u), \quad B(u) = \bar{V}(u) \times N(u), \\ c(u, v) &= g(u) + N(u) \cos(v) + B(u) \sin(v) \quad \forall v \in [0, \pi]. \end{aligned} \quad (4.167)$$

Для того чтобы выпуклости всех сегментов $c_1 \div c_4$ были обращены наружу «триплета» $\{e, p, q\}$, организуем их построение по (4.167) в процессе обхода периметра БП в следующем порядке (см. рис. 4.159):

- пройдя по линии $g_1(u) = p(u, \tau_h)$ в **прямом** направлении параметра $u \in [t_h, t_k]$, получим поверхность $c_1(u, v)$;
- пройдя по линии $g_2(u) = p(t_k, u)$ в **прямом** направлении $u \in [\tau_h, \tau_k]$, получим поверхность $c_2(u, v)$;
- пройдя по линии $g_3(u) = p(u, \tau_k)$ в **обратном** направлении $u \in [t_k, t_h]$, получим поверхность $c_3(u, v)$;
- завершая проход по линии $g_4(u) = p(t_h, u)$ в **обратном** направлении параметра $u \in [\tau_k, \tau_h]$, получим поверхность $c_4(u, v)$.

Сферическая поверхность сегмента $s(u, v)$ создаётся вращением **полуокружности** $o(v) = g + N \cos(v) + B \sin(v)$ вокруг вектора нормали N длиной r , соединяющего угловые вершины БП и внешней ЭДП (рис. 4.162). Полагая базовые касательные векторы V и W **выходящими** из вершины g , получим начальную бинормаль B длиной r и угол поворота $\beta = \angle(-W, V)$:

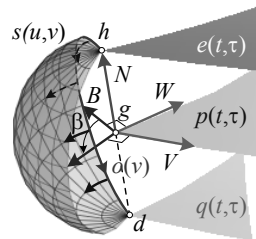


Рис. 4.162

$$\begin{aligned} N &= h - g, \quad B = N \times \bar{W}, \quad \beta = \arccos(-\bar{W}, \bar{V}), \\ s(u, v) &= g + N \cos(v) + B A_{\text{вп}}(\bar{N}, \beta u) \sin(v) \quad (4.168a) \\ &\quad \forall u \in [0, 1], \quad v \in [0, \pi]. \end{aligned}$$

Матрица вращения на угол ϕ вокруг **центрированной** прямой $\{O_3, N\}$ с **нормированным** ($|N|=1$) вектором направления N аналогична (3.47а):

$$A_{\text{вр}}(N, \varphi) = (1 - \cos(\varphi))N^T N + \cos(\varphi)E_3 + \sin(\varphi) \begin{bmatrix} 0 & N_z & -N_y \\ -N_z & 0 & N_x \\ N_y & -N_x & 0 \end{bmatrix}. \quad (4.168б)$$

Наружная выпуклость всех сегментов $s_1 \div s_4$ достигается правильным заданием направлений векторов V и W во всех углах БП (см. рис. 4.159):

- сегмент $s_1(u, v)$ создаётся на векторах $V = p'_t(t_h, \tau_h)$ и $W = p'_\tau(t_h, \tau_h)$;
- сегмент $s_2(u, v)$ создаётся при $V = p'_\tau(t_k, \tau_k)$ и $W = -p'_t(t_k, \tau_k)$;
- выпуклость сегмента $s_3(u, v)$ обеспечивается направляющими векторами $V = -p'_t(t_k, \tau_k)$ и $W = -p'_\tau(t_k, \tau_k)$;
- сегмент $s_4(u, v)$ будет выпуклым при $V = -p'_\tau(t_h, \tau_h)$ и $W = p'_t(t_h, \tau_h)$.

□ Следующий объект эквидистантного окружения — плоский произвольный **полигон** $P = p_1 p_2 \dots p_n p_1$. Его полная ЭДП (рис. 4.163, а) состоит из двух сдвинутых в обе стороны на расстояние r полигональных сегментов E и Q (рис. 4.163, б), соединённых по краям полукруглой трубчатой поверхностью (рис. 4.163, в), состоящей из цилиндрических $c(u, v)$ и сферических $s(u, v)$ сегментов. Аналогично устройству внешнего эквидистантного контура 2d-полигона (см. рис. 1.61, б) сферические сегменты существуют лишь при выпуклых углах базового полигона и гладко сопрягаются с цилиндрическими, а при невыпуклых углах цилиндрические сегменты сопрягаются непосредственно и не гладко.

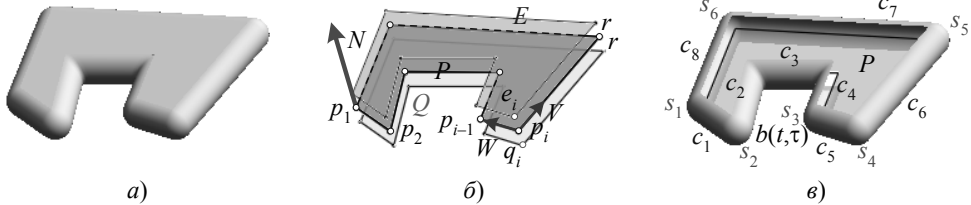


Рис. 4.163

Вершины эквидистантных полигонов $E = e_1 e_2 \dots e_n e_1$ и $Q = q_1 q_2 \dots q_n q_1$ находятся аналогично (4.166) путём ортогонального откладывания от каждой базовой вершины p_i расстояний r по разные стороны от плоскости полигона:

$$e_i = p_i + r\bar{N}, \quad q_i = p_i - r\bar{N} = 2p_i - e_i \quad \forall i = \overline{1, n}. \quad (4.169a)$$

Вектор нормали вычисляется по (1.144) как $N = \text{norm}(P)$. Для того чтобы общий обход полигона P с нормальной стороны выглядел положительным, его **первые три вершины должны составлять выпуклый положительный угол** $\angle p_1 p_2 p_3$. Составные поверхности полигонов E и Q как площадных объектов (см. рис. 4.150, а, б) строятся по алгоритму (4.161).

С целью гладкого сопряжения краёв полигонов E и Q разработаем алгоритм построения трубчатой поверхности $b(t, \tau)$, заметаемой дугой **полукруглости**, у которой центр скользит по периметру полигона P , а плоскость совершает следующее сложное движение:

- на линейных участках ортогональна направляющим рёбрам $p_i p_{i+1}$;
- в выпуклых углах p_v вращается вокруг оси $q_v e_v$ между ортогональными торцами сопрягаемых цилиндрических участков;
- в невыпуклых углах p_n осуществляет *негладкий* переход между цилиндрическими участками через *эллиптическую* сопрягающую дугу.

Для реализации сказанного нужно, в первую очередь, уметь классифицировать вершины 3d-полигона по признаку выпуклости. В аналогичной 2d-задаче это делается сравнением глобального направления обхода полигона функцией $dir_test(P)$, определённой в (1.48) либо (1.55), с локальным направлением угла при вершине p_i с помощью функции $nf2(p_{i-1}, p_i, p_{i+1})$. В R^3 понятий «слева» и «справа» нет даже у прямой на плоскости, имеющей две стороны наблюдения. Вот почему выше требовалось **положительное направление обхода полигона P по его нормальной стороне** (рис. 4.163б). Если это так, то знак смешанного произведения $\sigma = N \circ (V \times W)$ векторов нормали N и сторон угла V и W сигнализирует (см. рис. 1.127, а, б), что при $\sigma > 0$ угол $\angle p_{i-1} p_i p_{i+1}$ выпуклый, а при $\sigma < 0$ — невыпуклый (избыточные полигоны с развёрнутыми углами должны быть минимизированы). Таким образом, в каждом выпуклом углу между цилиндрическими сегментами нужно вставить сферический сегмент.

Отсюда вытекает задача формирования $M = 2n_v + n_n$ -сегментной составной поверхности $b(t, \tau)$, включающей n_v сферических и $n_v + n_n$ цилиндрических сегментов. Для построения k -го сегмента нужно знать его порядок $d_k \in \{1, 2\}$ (линейный либо квадратичный) в направлении параметра t , положение центра образующей дуги o_k и начальное положение вектора бинормали B_k . Расширив полигон вершиной $p_0 \equiv p_n$ до полилинии $P = p_n p_1 p_2 \dots p_n p_1$ и инициализировав счётчик сегментов $k=0$, сформируем списки d , o и B в цикле $i = \overline{1, n}$:

- вычисляем нормированные векторы сторон $V = \overline{p_{i+1} - p_i}$ и $W = \overline{p_{i-1} - p_i}$, угол между ними $\gamma = \arccos(V \circ W)$ и признак выпуклости угла $\sigma = N \circ (V \times W)$;
- если $\sigma > 0$, то:
 - ◊ добавляем данные квадратичного сегмента $d_{++k} = 2$, $o_k \equiv p_i$, $B_k = r \bar{N} \times W$;
 - ◊ добавляем данные линейного сегмента $d_{++k} = 1$, $o_k \equiv p_i$, $B_k = V \times r \bar{N}$;
- если $\sigma < 0$, то добавляем данные линейного сегмента $d_{++k} = 1$, $o_k \equiv p_i$, $B_k = r \sin^{-1}(0.5\gamma) \overline{V + W}$, начальной образующей которого является дуга эллипса, лежащая в биссекторной плоскости невыпуклого угла $\{p_i, V, W\}$.

По окончании цикла замыкаем списки o и B конечными значениями $o_{k+1} = o_1$ и $B_{k+1} = B_1$. В итоге счётчик сегментов накопил значение $k=M$. К примеру, поверхность на рис. 4.163в состоит из $M=14$ сегментов со следующими списками порядков и центров образующих дуг:

$$d = \{2, 1, 2, 1, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1\},$$

$$o = \{p_1, p_1, p_2, p_2, p_3, p_4, p_5, p_5, p_6, p_6, p_7, p_7, p_8, p_8, p_1\}.$$

Построение составной трубчатой поверхности $b(t, \tau)$ в параметрических интервалах $t \in [0, M]$ и $\tau \in [0, \pi]$ выполняется следующим образом:

- по текущему значению t вычисляем номер сегмента $k = \lceil t \rceil + \{t=0\}$ и локальный параметр $\theta = t + 1 - k \in [0, 1]$;

- если $d_k=1$, то линейчатая поверхность **цилиндрического** сегмента формируется движением дуги $o(\theta) + r\bar{N}\cos(\tau) + B(\theta)\sin(\tau)$ с линейной интерполяцией её центра $o(\theta)$ и вектора бинормали $B(\theta)$ по параметру θ (рис. 4.164,а):

$$b(\theta, \tau) = (1-\theta)o_k + \theta o_{k+1} + r\bar{N}\cos(\tau) + ((1-\theta)B_k + \theta B_{k+1})\sin(\tau); \quad (4.169б)$$

- при $d_k=2$ квадратичная поверхность **сферического** сегмента формируется вращением дуги $o_k + r\bar{N}\cos(\tau) + B_k\sin(\tau)$ с помощью матрицы (4.168б) от начального положения бинормали B_k до конечного положения B_{k+1} (рис. 4.164,б):

$$b(\theta, \tau) = o_k + r\bar{N}\cos(\tau) + B_k A_{\text{вп}}(\bar{N}, \arccos(\bar{B}_k, \bar{B}_{k+1})\theta)\sin(\tau). \quad (4.169в)$$

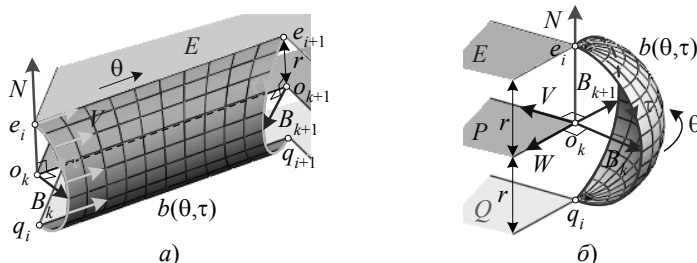


Рис. 4.164

По достижении конечного значения параметра $t=M$ поверхность оказывается замкнутой в этом параметрическом направлении.

□ В заключение остановимся на задаче построения эквидистантной оболочки произвольного **базового полиэдра** (рис. 4.165,а), модель которого $\{P, G\}$ включает список вершин $P = \{p_1, p_2, \dots, p_n\}$ и группу списков граней $G = \{G_1, G_2, \dots, G_m\}$ (см. п. 1.5.4). Процесс создания ЭДП аналогичен внешнему эквидистантному окружению полигона (см. рис. 1.61,б) и состоит из двух этапов:

- формирования **внешнего** полиэдра $\{E, G\}$ с вершинами $E = \{e_1, e_2, \dots, e_n\}$, находящимися в точках пересечения плоскостей, равноудалённых от граней базового полиэдра на расстояние r (рис. 4.165,б);

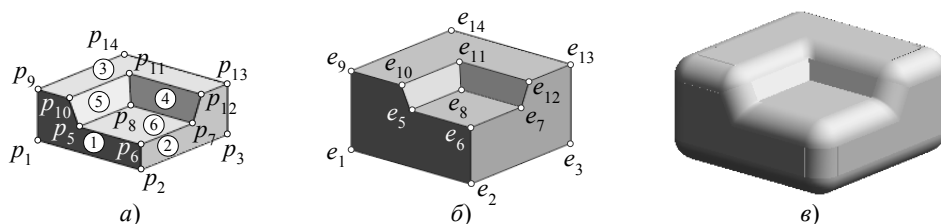


Рис. 4.165

• скругления **выпуклых** двугранных углов при рёбрах и трёхгранных углов при вершинах полиэдра $\{E, G\}$ сегментами второго порядка с радиусом скругления r (рис. 4.165, в). Более чем трёхгранные углы квадратичному скруглению не поддаются, поскольку четыре и более попарно не параллельных плоскостей в одной точке в общем случае не пересекаются. Это замечание, кстати, относится и к задачам из п. 4.4.4, в которых ищутся центры вписанных сферы и тора, равноотстоящие от плоскостей граней углов (см. рис. 4.122, 4.123).

Итак, полагаем, что у базового полиэдра **все углы при вершинах** $p_1 \div p_n$ **являются трёхгранными**, а все m граней имеют **внешние** нормали $N_1 \div N_m$. Эти векторы могут быть заданы таковыми при составлении модели полиэдра либо вычислены по (1.145) как $N_j = \text{norm}(\text{side}(P, G, j)) \quad \forall j = \overline{1, m}$ при соблюдении правил составления списков граней (см. п. 1.5.4).

На **первом этапе** для каждой вершины p_i ищутся номера $\{k_1, k_2, k_3\}$ трёх инцидентных ей граней. Просматривая все списки $G_j = \{g_{j1}, g_{j2}, \dots, g_{jm_j}, g_{j1}\} \quad \forall j = \overline{1, m}$, запоминаем номера тех, в которых есть число i . Затем строим равноудалённые от найденных граней внешние плоскости $\{o_{k_1}, N_{k_1}\} \div \{o_{k_3}, N_{k_3}\}$ и по (1.140б) вычисляем точку их пересечения e_i (рис. 4.166, а):

$$o_{k_l} = p_{g_{k_l1}} + r \bar{N}_{k_l} \quad \forall l = \overline{1, 3},$$

$$e_i = [o_{k_1} \circ N_{k_1} \quad o_{k_2} \circ N_{k_2} \quad o_{k_3} \circ N_{k_3}] [N_{k_1}^T \quad N_{k_2}^T \quad N_{k_3}^T]^{-1} \quad \forall i \in \overline{1, n}. \quad (4.170)$$

В результате получаем матрицу E вершин полиэдра, грани которого параллельны одноимённым граням базового полиэдра и отстоят от них на расстоянии r .

На **втором этапе** по алгоритмам, описанным в п. 4.4.4, скругляются **выпуклые** углы полиэдра $\{E, G\}$:

- трёхгранные углы при вершинах e_i — сферическими сегментами (см. рис. 4.122) $s_i(u, v)$, торцы которых ортогональны рёбрам, инцидентным вершинам углов (рис. 4.166, б). Кластерные матрицы сегментов рассчитаны по (4.122);
- двухгранные углы при рёбрах $e_i e_j$ — цилиндрическими сегментами (см. рис. 4.119) $c_{i,j}(u, v)$, торцы которых заключены между граничными дугами соседних сегментов (рис. 4.166, в). Вершины их ХПС рассчитываются по (4.120в).

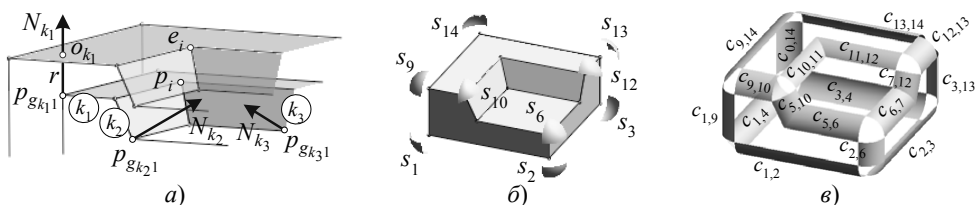


Рис. 4.166

Оставшиеся плоские фрагменты граней строятся как не скруглённые полигоны (см. рис. 4.150) на кластерных матрицах, созданных по формулам (4.161).

4.6. СПЛАЙНОВЫЕ ПОВЕРХНОСТИ

Физически сплайновая интерполяционная поверхность представляет собой упругую неразрывную оболочку, проходящую с минимальной кривизной через множество $(n+1)(m+1)$ матрично упорядоченных узловых точек

$$P_{ij} = [x_{ij} \ y_{ij} \ z_{ij}] \quad \forall i = \overline{0, n}, \ j = \overline{0, m}.$$

Параметрическая модель (i, j) -го сплайнового сегмента, заполняющего **одну** четырёхугольную (при совпадении соседних узлов — треугольную) ячейку узловой сети (рис. 4.167) описывается бикубическим полиномом

$$p_{ij}(t, \tau) = T^T(t) S_{ij} T(\tau) \in \mathbb{R}^3 \quad \forall t \in [\alpha_i, \beta_i], \ \tau \in [\alpha_j, \beta_j], \ i = \overline{1, n}, \ j = \overline{1, m}, \quad (4.171)$$

где $T(t)$ и $T(\tau)$ — векторы базисных функций кубических полиномов, определённые в (2.98), а $S_{ij} \in V^{4 \times 4}$ — матрица векторных коэффициентов с трёхслойной структурой — по одному 4×4 -слою на каждую координату x, y и z .

Вспомним основные методы параметризации — выбора границ параметрических интервалов $a_i, b_i, \alpha_j, \beta_j$ и их длин $d_i = b_i - a_i$ и $\delta_j = \beta_j - \alpha_j$:

- начальные значения интервалов при **локальной** параметризации задаются нулевыми $a_i = \alpha_j = 0$, а при **сквозной** равными $a_i = b_{i-1}$ и $\alpha_j = \beta_{j-1}$;
- длины интервалов при **нормализованной** параметризации задаются единичными $d_i = \delta_j = 1$, а методом **хордовой** параметризации они получаются как в (4.99) усреднением длин рёбер ячеек в каждом параметрическом направлении:

$$d_i = \text{mean}(|P_{ij} - P_{i-1,j}| \mid \forall j = \overline{0, m}), \quad \delta_j = \text{mean}(|P_{ij} - P_{i,j-1}| \mid \forall i = \overline{0, n}).$$

4.6.1. Бикубические сплайны

В отличие от рассмотренных в п. 4.5.1 полиномиальных составных поверхностей **сплайны имеют второй порядок гладкости сопряжения сегментов** — непрерывность функции (4.171) и двух её производных по параметрам t и τ в общих вершинах узловой сети. На основе этих свойств формируются условия решения задачи поверхностной сплайн-интерполяции (её линейный аналог рассмотрен в п. 2.3.1 и [28, 32]) и составляется система уравнений (4.96г), после чего по (4.97) вычисляются МПК $S_{11} \div S_{nm}$ и по (4.171) производится построение поверхности сегментов.

□ **Условия непрерывности** поверхности сегментов в узлах интерполяции:

$$\begin{cases} p_{ij}(a_i, \alpha_j) = P_{i-1,j-1}, \ p_{ij}(a_i, \beta_j) = P_{i-1,j}, \\ p_{ij}(b_i, \alpha_j) = P_{i,j-1}, \ p_{ij}(b_i, \beta_j) = P_{i,j} \quad \forall i = \overline{1, n}, \ j = \overline{1, m}. \end{cases} \quad (4.172)$$

□ **Условия двойной гладкости** сопряжения сегментов по параметру t

$$V_{ij}(b_i, \tau) = V_{i+1,j}(a_{i+1}, \tau), \quad V'_{ij}(b_i, \tau) = V'_{i+1,j}(a_{i+1}, \tau) \quad \forall i = \overline{1, n-1}, \ j = \overline{1, m}, \ \tau \in \{\alpha_j, \beta_j\} \quad (4.173a)$$

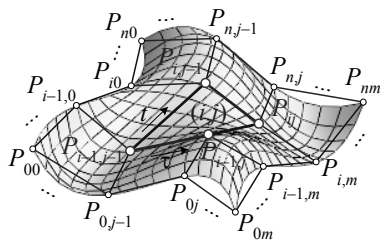


Рис. 4.167

и параметру τ

$$W_{ij}(t, \beta_j) = W_{i,j+1}(t, \alpha_{j+1}), \quad W'_{ij}(t, \beta_j) = W'_{i,j+1}(t, \alpha_{j+1}) \quad \forall i = \overline{1, n}, \quad j = \overline{1, m-1}, \quad t \in \{a_i, b_i\} \quad (4.173б)$$

с векторами двух частных производных по двум параметрам

$$V_{ij}(t, \tau) = T'^T(t) S_{ij} T(\tau), \quad W_{ij}(t, \tau) = T^T(t) S_{ij} T'(\tau),$$

$$V'_{ij}(t, \tau) = T''^T(t) S_{ij} T(\tau), \quad W'_{ij}(t, \tau) = T'^T(t) S_{ij} T''(\tau).$$

□ **Краевые условия**, налагаемые на производные сплайна в граничных узлах P_{i0} , P_{im} , P_{0j} и P_{nj} , варьируются в зависимости от специфики решаемой задачи:

- **свободные границы** дают условия нулевой кривизны краёв поверхности:

$$\begin{cases} V'_{1j}(a_1, \tau) = V'_{nj}(b_n, \tau) = W'_{i1}(t, \alpha_1) = W'_{im}(t, \beta_m) = O_3 \\ \forall i = \overline{1, n}, \quad j = \overline{1, m}, \quad t \in \{a_i, b_i\}, \quad \tau \in \{\alpha_j, \beta_j\}; \end{cases} \quad (4.174а)$$

- **закреплённые векторы краевых направлений** (рис. 4.139 при $n_i = m_j = 1$)

$$\begin{cases} V_{1j}(a_1, \alpha_j) = V_{0,j-1}, \quad V_{1j}(a_1, \beta_j) = V_{0j}, \quad V_{nj}(b_n, \alpha_j) = V_{n,j-1}, \quad V_{nj}(b_n, \beta_j) = V_{nj}, \\ W_{i1}(a_i, \alpha_1) = W_{i-1,0}, \quad W_{i1}(b_i, \alpha_1) = W_{i0}, \quad W_{im}(a_i, \beta_m) = W_{i-1,m}, \quad W_{im}(b_i, \beta_m) = W_{im} \\ \forall i = \overline{1, n}, \quad j = \overline{1, m} \end{cases} \quad (4.174б)$$

соответствуют задаче эрмитовой сплайн-интерполяции с присущим ей большим объёмом исходных данных. В отсутствие желаемых векторов краевых направлений они аппроксимируются аналогично (4.149б) разностными векторами

$$\begin{cases} V_{0j} = \gamma_0 (P_{1j} - P_{0j}) / d_1, \quad V_{nj} = \gamma_n (P_{nj} - P_{n-1,j}) / d_n \quad \forall j = \overline{0, m}, \\ W_{i0} = \lambda_0 (P_{i1} - P_{i0}) / \delta_1, \quad W_{im} = \lambda_m (P_{im} - P_{i,m-1}) / \delta_m \quad \forall i = \overline{0, n} \end{cases}$$

с возможностью регулировки их длин коэффициентами γ_0 , γ_n , λ_0 и λ_m ;

- **циклический сплайн** кроме необязательного совпадения узлов $P_{0j} = P_{nj}$ и $P_{i0} = P_{im}$ должен удовлетворять условиям дважды гладкого сопряжения краев:

$$\begin{cases} V_{1j}(a_1, \tau) = V_{nj}(b_n, \tau), \quad V'_{1j}(a_1, \tau) = V'_{nj}(b_n, \tau) \quad \forall j = \overline{1, m}, \quad \tau \in \{\alpha_j, \beta_j\}, \\ W'_{i1}(t, \alpha_1) = W'_{im}(t, \beta_m), \quad W''_{i1}(t, \alpha_1) = W''_{im}(t, \beta_m) \quad \forall i = \overline{1, n}, \quad t \in \{a_i, b_i\}; \end{cases} \quad (4.174в)$$

- **ациклический сплайн** отличается от циклического противоположными направлениями векторов краевых скоростей и ускорений:

$$\begin{cases} V_{1j}(a_1, \tau) = -V_{nj}(b_n, \tau), \quad V'_{1j}(a_1, \tau) = -V'_{nj}(b_n, \tau) \quad \forall j = \overline{1, m}, \quad \tau \in \{\alpha_j, \beta_j\}, \\ W'_{i1}(t, \alpha_1) = -W'_{im}(t, \beta_m), \quad W''_{i1}(t, \alpha_1) = -W''_{im}(t, \beta_m) \quad \forall i = \overline{1, n}, \quad t \in \{a_i, b_i\}. \end{cases} \quad (4.174г)$$

Как показано в п. 4.5.1 и на рис. 4.138, **циклическая гладкость второго порядка делает интерполяционную задачу разрешимой обоими методами параметризации и при любом расположении узловых точек.**

Могут быть и отличные от (4.174) комбинации краевых условий. Например, возможна сплайновая поверхность цилиндрического типа с циклическим сопряжением в узлах $P_{i0} = P_{im}$ и свободными краями $P_{00} \div P_{0m}$ и $P_{n0} \div P_{nm}$.

Совокупность уравнений (4.172)–(4.174) составляет ретроспективно решаемую систему алгебраических уравнений

$$Q^T S \Theta = U \Rightarrow S = (Q^{-1})^T U \Theta^{-1} \quad (4.175)$$

с аналогичными (2.104а) блочными матрицами

$$Q = \begin{bmatrix} A_1 & O & \dots & O & -D \\ -B_2 & A_2 & \dots & O & O \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ O & O & \dots & A_{n-1} & O \\ O & O & \dots & -B_n & C \end{bmatrix}, \quad \Theta = \begin{bmatrix} A_1 & O & \dots & O & -\Delta \\ -B_2 & A_2 & \dots & O & O \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ O & O & \dots & A_{m-1} & O \\ O & O & \dots & -B_m & X \end{bmatrix},$$

$$S = \begin{bmatrix} S_{11} & \dots & S_{1m} \\ \vdots & \ddots & \vdots \\ S_{n1} & \dots & S_{nm} \end{bmatrix}, \quad U = \begin{bmatrix} U_{11} & \dots & U_{1m} \\ \vdots & \ddots & \vdots \\ U_{n1} & \dots & U_{nm} \end{bmatrix},$$

блоки которых представляют собой матрицы

$$\begin{cases} A_i = [T(a_i) \ T(b_i) \ T'(b_i) \ T''(b_i)], \ B_i = [O \ O \ T'(a_i) \ T''(a_i)], \\ C^{(a)} = [T(a_n) \ T(b_n) \ O \ T''(b_n)], \ D^{(a)} = [O \ O \ -T''(a_1) \ O], \\ C^{(6)} = [T(a_n) \ T(b_n) \ O \ T'(b_n)], \ D^{(6)} = [O \ O \ -T'(a_1) \ O], \\ C^{(b)} = C^{(r)} = A_n, \ D^{(b)} = B_1, \ D^{(r)} = -B_1; \end{cases}$$

$$\begin{cases} A_j = [T(\alpha_j) \ T(\beta_j) \ T'(\beta_j) \ T''(\beta_j)], \ B_j = [O \ O \ T'(\alpha_j) \ T''(\alpha_j)], \\ X^{(a)} = [T(\alpha_m) \ T(\beta_m) \ O \ T''(\beta_m)], \ \Delta^{(a)} = [O \ O \ -T''(\alpha_1) \ O], \\ X^{(6)} = [T(\alpha_m) \ T(\beta_m) \ O \ T'(\beta_m)], \ \Delta^{(6)} = [O \ O \ -T'(\alpha_1) \ O], \\ X^{(b)} = X^{(r)} = A_m, \ \Delta^{(b)} = B_1, \ \Delta^{(r)} = -B_1; \end{cases}$$

$$U_{ij} = \begin{bmatrix} P_{i-1,j-1} & P_{i-1,j} & O & O \\ P_{i,j-1} & P_{ij} & O & O \\ O & O & O & O \\ O & O & O & O \end{bmatrix}, \quad U_{im}^{(6)} = \begin{bmatrix} P_{i-1,m-1} & P_{i-1,m} & W_{i-1,0} & W_{i-1,m} \\ P_{i,m-1} & P_{im} & W_{i0} & W_{im} \\ O & O & O & O \\ O & O & O & O \end{bmatrix},$$

$$U_{nj}^{(6)} = \begin{bmatrix} P_{n-1,j-1} & P_{n-1,j} & O & O \\ P_{n,j-1} & P_{nj} & O & O \\ V_{0,j-1} & V_{0j} & O & O \\ V_{n,j-1} & V_{nj} & O & O \end{bmatrix}, \quad U_{nm}^{(6)} = \begin{bmatrix} P_{n-1,m-1} & P_{n-1,m} & W_{n-1,0} & W_{n-1,m} \\ P_{n,m-1} & P_{nm} & W_{n0} & W_{nm} \\ V_{0,m-1} & V_{0m} & O & O \\ V_{n,m-1} & V_{nm} & O & O \end{bmatrix}.$$

Согласованные с (4.174) четыре варианта устройства блоков C и D матрицы Q определяют краевые условия в узлах $P_{00} \div P_{0m}$ и $P_{n0} \div P_{nm}$, а блоков X и Δ матрицы Θ — в узлах $P_{00} \div P_{n0}$ и $P_{0m} \div P_{nm}$. В (4.175) расчёт матриц параметрических коэффициентов бикубического сплайна S ведётся отдельно для каждого координатного слоя трёхмерных матриц U и S , как показано в (4.97б).

Алгоритм сплайнового интерполирования (4.171)–(4.175) даёт более гладкие и эстетически привлекательные решения в сравнении с методами, имеющими порядок гладкости сопряжения сегментов не выше первого. В этом убеждает построенная на 3×9 -узловой сетке (см. рис. 4.89,а) модель зуба на рис. 4.168,а, краевыми условиями которой являются цикличность на вертикальной t -границе $u_{i0} = u_{i8}$ и свободные горизонтальные τ -границы u_{0j} и u_{2j} . На том же рисунке показано еще три примера синтеза 3d-сплайнов на тех же узловых сетках, что и у объектов на рис. 4.138. Несвободными краевыми условиями конструирования

этих фигур выбраны разностная аппроксимация векторов краевых направлений V_{0j} и V_{7j} у поверхности (б), циклические замыкания в одном τ -направлении поверхности (в) и обоих t - и τ -направлениях объекта (г).

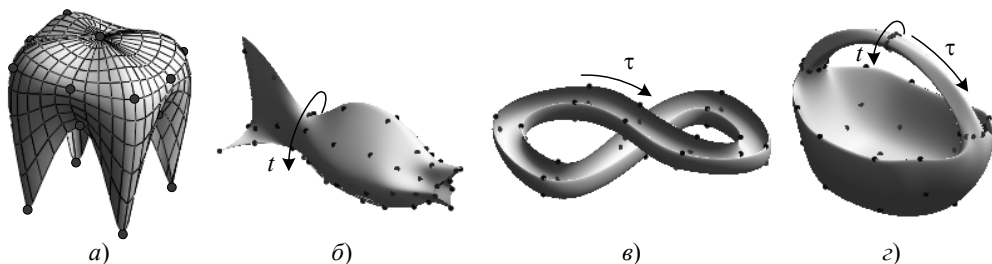


Рис. 4.168

Сплайновое описание поверхностей, придя на смену полигональному, широко применяется в промышленных САПР для конструирования корпусов автомобилей, судов, самолетов, в медицине (компьютерная томография), картографии (реконструкция рельефа местности по небольшому объёму геодезических измерений), компьютерных играх (изображение трёхмерных объектов, в том числе животных и людей) благодаря возможности получения аналитической формулы $p(t, \tau)$ гладкой оболочки по её отдельным узловым точкам P_{ij} . Имея модель поверхности, мы получаем возможности доступа к любой её точке, геометрического преобразования, изображения в желаемой проекции, расчёта пересечений с другими графическими объектами, изготовления натурной модели, программирования станка с числовым управлением для её изготовления или обработки и т. д.

Пример 4.31. Построить сплайновую поверхность со свободными краями и нормализованной сквозной параметризацией $t, \tau \in [0, 2]$ по девяти точкам

$$\begin{aligned} P_{00} &= [-1 \ 0 \ 1], \quad P_{01} = [-1 \ 1 \ 0], \quad P_{02} = [-1 \ 0 \ -1], \\ P_{10} &= [0 \ -1 \ 1], \quad P_{11} = [0 \ 0 \ 0], \quad P_{12} = [0 \ -1 \ -1], \\ P_{20} &= [1 \ 0 \ 1], \quad P_{21} = [1 \ 1 \ 0], \quad P_{22} = [1 \ 0 \ -1]. \end{aligned}$$

Поверхность гиперболического параболоида, несущая эти точки, построена методами как граничной (см. пример 4.21), так и узловой (см. пример 4.23) интерполяций, имеющих в сквозных шкалах $t, \tau \in [0, 2]$ одинаковый вид $p(t, \tau) = [t-1 \ t^2-2t-\tau^2+2\tau \ 1-\tau]$.

Решение. В рамках сквозной нормализованной параметризации матрицы $Q = \Theta$ и координатные слои матриц U и $S = (Q^{-1})^T U \Theta^{-1}$ принимают в (4.175) следующий вид:

$$Q = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 2 & 0 & 0 & 2 & 0 \\ 0 & 1 & 3 & 6 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 2 & 0 & 0 \\ 0 & 0 & -2 & -2 & 1 & 4 & 0 & 2 \\ 0 & 0 & -3 & -6 & 1 & 8 & 0 & 12 \end{bmatrix}, \quad U_x = \begin{bmatrix} -1 & -1 & 0 & 0 & -1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix},$$

$$U_y = \left[\begin{array}{ccc|ccc} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right], \quad U_z = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right],$$

$$S_x = \left[\begin{array}{ccc|ccc} -1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right], \quad S_y = \left[\begin{array}{ccc|ccc} 0 & 1.5 & 0 & -0.5 & -1 & 4.5 & -3 & 0.5 \\ -1.5 & 0 & 0 & 0 & -1.5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.5 & 0 & 0 & 0 & 0.5 & 0 & 0 & 0 \\ 1 & 1.5 & 0 & -0.5 & 0 & 4.5 & -3 & 0.5 \\ -4.5 & 0 & 0 & 0 & -4.5 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 3 & 0 & 0 & 0 \\ -0.5 & 0 & 0 & 0 & -0.5 & 0 & 0 & 0 \end{array} \right], \quad S_z = \left[\begin{array}{ccc|ccc} 1 & -1 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right].$$

Подставив матрицы коэффициентов в модель бикубического сплайна (4.171), получим параметрические модели поверхностей четырёх сегментов (рис. 4.169)

$$\begin{aligned} p_{11}(t, \tau) &= [t-1 \quad -1.5t+0.5t^3+1.5\tau-0.5\tau^3 \quad 1-\tau], \quad t \in [0, 1], \quad \tau \in [0, 1], \\ p_{12}(t, \tau) &= [t-1 \quad -1-1.5t+0.5t^3+4.5\tau-3\tau^2+0.5\tau^3 \quad 1-\tau], \quad t \in [0, 1], \quad \tau \in [1, 2], \\ p_{21}(t, \tau) &= [t-1 \quad 1-4.5t+3t^2-0.5t^3+1.5\tau-0.5\tau^3 \quad 1-\tau], \quad t \in [1, 2], \quad \tau \in [0, 1], \\ p_{22}(t, \tau) &= [t-1 \quad -4.5t+3t^2-0.5t^3+4.5\tau-3\tau^2+0.5\tau^3 \quad 1-\tau], \quad t \in [1, 2], \quad \tau \in [1, 2], \end{aligned}$$

каждая из которых определена в индивидуальных интервалах изменения параметров t и τ . Очевидно, что координатные функции этих сегментов **не удовлетворяют** квадратичному уравнению гиперболического параболоида $x_{ij}^2(t, \tau) - y_{ij}(t, \tau) - z_{ij}^2(t, \tau) = 0$ из-за линейности функций $x_{ij}(t, \tau) = t-1$, $z_{ij}(t, \tau) = 1-\tau$ и кубичности всех $y_{ij}(t, \tau)$.

Это означает, что сплайновая поверхность является лишь интерполяционным приближением этой квадратики, хоть и достаточно близким ей по форме (ср. с рис. 4.88, б, 4.98). Для подтверждения этого факта вычислим при $t=0.5$ и $\tau=1$ ординату сплайна $y_{11}=5/16$, тогда как при $x=-0.5$ и $z=0$ ордината $y=x^2-z^2=1/4 \neq 5/16$. $\square$

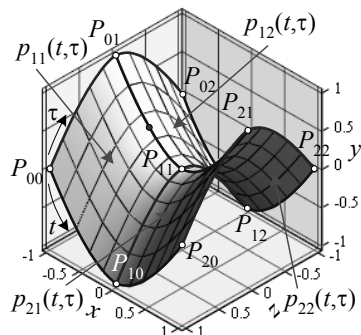


Рис. 4.169

Пример 4.32. Допустим, нужно создать достоверное изображение некоторого рельефа по измерениям высот y_{ij} множества точек на плане $\{x_i, z_j\}$. В качестве источника координат возьмём модель поверхности (4.80). Наложим на план местности (карту линий равных высот) **регулярную** сетку с размерами ячеек $\Delta x = \Delta z = 0.5$ и координатами узлов $x_i = -2 + 0.5i \quad \forall i = \overline{0, 8}$, $z_j = -2 + 0.5j \quad \forall j = \overline{0, 8}$, после чего построим на этой 81-узловой сетке нормализованную сплайновую поверхность со свободными границами (рис. 4.170, а). Налицо практически полная идентичность этого изображения с образцом (см. рис. 4.80, б) благодаря выбору частой сетки и совпадению её узлов с вершинами и впадиной рельефа.

Однако, вдвое более редкое размещение 25 узловых точек с координатами $x_i = -2 + i \quad \forall i = \overline{0, 4}$, $z_j = -2 + j \quad \forall j = \overline{0, 4}$ даёт сплайновую поверхность (рис. 4.170, б), существенно отличающуюся от образца. Это объясняется выбором слишком редкого пространственного

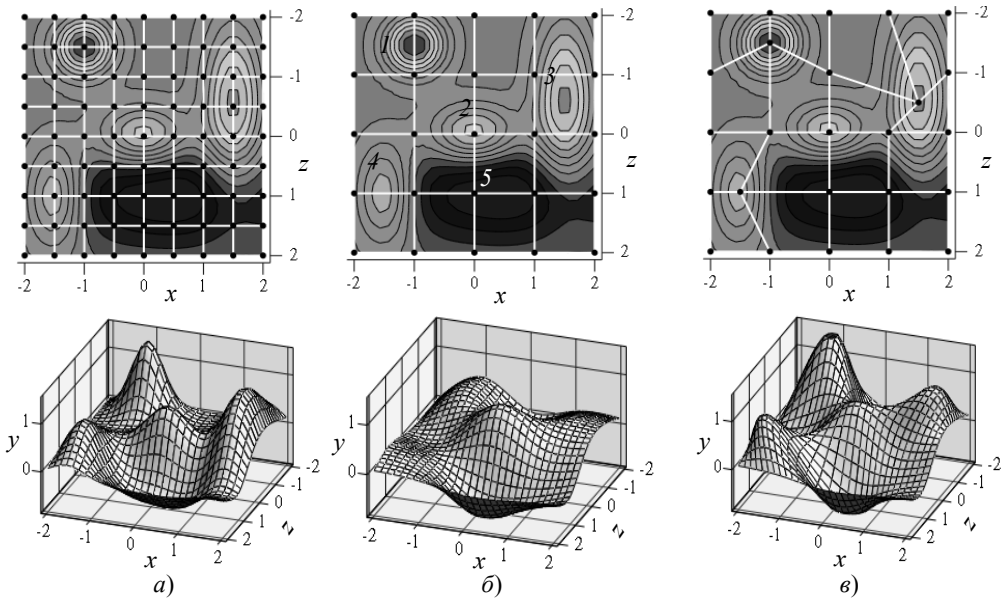


Рис. 4.170

квантования $\Delta x=1$ и $\Delta z=1$, при котором почти все пиковые точки поверхности (кроме обозначенной цифрой 2) не совпали с узлами интерполяции и оказались «незамеченными» алгоритмом построения сплайна.

Целенаправленным смещением всего лишь трёх узлов удалось на *нерегулярной* 25-узловой сетке получить сплайновую поверхность (рис. 4.170,в), весьма близкую к регулярной 81-узловой поверхности. Однако, данный искусственный приём минимизации числа узлов интерполяции не отменяет изложенного ниже правила. $\square$

Вывод (пространственный аналог теоремы Шеннона – Котельникова): при построении интерполяционной поверхности выбор интервалов квантования в пространстве ограничен сверху минимальными размерами элементов интерполируемого объекта. Для получения достоверного изображения рельефа поверхности расстояния между вершинами узловой сети должны быть хотя бы **вдвое** меньше размера минимально различимого элемента рельефа.

4.6.2. Сплайны с перекрытием параболических сегментов

Распространим метод перекрытия параболических сегментов, разработанный в п. 2.3.3 для кубических сплайновых линий [44], на задачу синтеза бикубических сплайновых поверхностей, для построения которых задаются узловая сеть $\{P_{ij}\}$ с индексами вершин $i=0, n$, $j=0, m$ и краевые условия в виде векторов производных в граничных узлах. В отличие от узловой сети обычного сплайна (см. рис. 4.167), размеры которой вполне могут задаваться значениями $n=1$ и/или $m=1$, для создания сплайновой поверхности методом перекрытия сегментов её размер должен быть задан числами $n \geq 2$ и $m \geq 2$.

Расширим модель линейного перекрытия (2.114а) на локально-нормализованный вариант модели поверхности (i, j) -го сегмента

$$p_{ij}(t, \tau) = [1-t \quad t] \begin{bmatrix} g_{00}(t, \tau) & g_{01}(t, \tau) \\ g_{10}(t, \tau) & g_{11}(t, \tau) \end{bmatrix} \begin{bmatrix} 1-\tau \\ \tau \end{bmatrix} \quad \forall t, \tau \in [0, 1], \quad i = \overline{1, n}, \quad j = \overline{1, m}, \quad (4.176a)$$

в которой, как схематично показано на рис. 4.171, билинейно интерполируются четыре параболических сегмента. Интерполяционные модели этих сегментов строятся по (4.93б) с матрицами (4.97а) как

$$g_{v\mu}(t, \tau) = T_2^T(t) (Q_v^{-1})^T U_{v\mu} \Theta_\mu^{-1} T_2(\tau) \quad \forall v, \mu \in \{0, 1\}. \quad (4.176б)$$

Матрицы этих моделей вычисляются в зависимости от места сегмента p_{ij} в узловой сети. Для **внутренних** сегментов с индексами $i \in [2, n-1]$ и $j \in [2, m-1]$

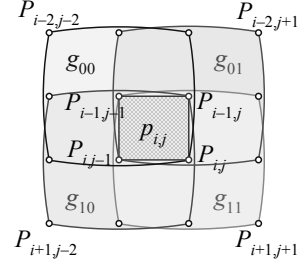


Рис. 4.171

$$\begin{aligned} \Theta_0 &= [T_2(-1) \quad T_2(0) \quad T_2(1)], \quad \Theta_1 = [T_2(0) \quad T_2(1) \quad T_2(2)], \\ Q_0 &= [T_2(-1) \quad T_2(0) \quad T_2(1)], \quad U_{00} = \begin{bmatrix} P_{i-2,j-2} & P_{i-2,j-1} & P_{i-2,j} \\ P_{i-1,j-2} & P_{i-1,j-1} & P_{i-1,j} \\ P_{i,j-2} & P_{i,j-1} & P_{i,j} \end{bmatrix}, \quad U_{01} = \begin{bmatrix} P_{i-2,j-1} & P_{i-2,j} & P_{i-2,j+1} \\ P_{i-1,j-1} & P_{i-1,j} & P_{i-1,j+1} \\ P_{i,j-1} & P_{i,j} & P_{i,j+1} \end{bmatrix}, \\ Q_1 &= [T_2(0) \quad T_2(1) \quad T_2(2)], \quad U_{10} = \begin{bmatrix} P_{i-1,j-2} & P_{i-1,j-1} & P_{i-1,j} \\ P_{i,j-2} & P_{i,j-1} & P_{i,j} \\ P_{i+1,j-2} & P_{i+1,j-1} & P_{i+1,j} \end{bmatrix}, \quad U_{11} = \begin{bmatrix} P_{i-1,j-1} & P_{i-1,j} & P_{i-1,j+1} \\ P_{i,j-1} & P_{i,j} & P_{i,j+1} \\ P_{i+1,j-1} & P_{i+1,j} & P_{i+1,j+1} \end{bmatrix}. \end{aligned} \quad (4.177a)$$

Для **граничных** сегментов с векторами краевых направлений (4.174б):

- верхнего левого с индексами $i=j=1$:

$$\begin{aligned} \Theta_0 &= [T_2'(0) \quad T_2(0) \quad T_2(1)], \quad \Theta_1 = [T_2(0) \quad T_2(1) \quad T_2(2)], \\ Q_0 &= [T_2'(0) \quad T_2(0) \quad T_2(1)], \quad U_{00} = \begin{bmatrix} O & V_{00} & V_{01} \\ W_{00} & P_{00} & P_{01} \\ W_{10} & P_{10} & P_{11} \end{bmatrix}, \quad U_{01} = \begin{bmatrix} V_{00} & V_{01} & V_{02} \\ P_{00} & P_{01} & P_{02} \\ P_{10} & P_{11} & P_{12} \end{bmatrix}, \\ Q_1 &= [T_2(0) \quad T_2(1) \quad T_2(2)], \quad U_{10} = \begin{bmatrix} W_{00} & P_{00} & P_{01} \\ W_{10} & P_{10} & P_{11} \\ W_{20} & P_{20} & P_{21} \end{bmatrix}, \quad U_{11} = \begin{bmatrix} P_{00} & P_{01} & P_{02} \\ P_{10} & P_{11} & P_{12} \\ P_{20} & P_{21} & P_{22} \end{bmatrix}; \end{aligned} \quad (4.177б)$$

- верхних внутренних с индексами $i=1$ и $j \in [2, m-1]$:

$$\begin{aligned} \Theta_0 &= [T_2(-1) \quad T_2(0) \quad T_2(1)], \quad \Theta_1 = [T_2(0) \quad T_2(1) \quad T_2(2)], \\ Q_0 &= [T_2'(0) \quad T_2(0) \quad T_2(1)], \quad U_{00} = \begin{bmatrix} V_{0,j-2} & V_{0,j-1} & V_{0,j} \\ P_{0,j-2} & P_{0,j-1} & P_{0,j} \\ P_{1,j-2} & P_{1,j-1} & P_{1,j} \end{bmatrix}, \quad U_{01} = \begin{bmatrix} V_{0,j-1} & V_{0,j} & V_{0,j+1} \\ P_{0,j-1} & P_{0,j} & P_{0,j+1} \\ P_{1,j-1} & P_{1,j} & P_{1,j+1} \end{bmatrix}, \\ Q_1 &= [T_2(0) \quad T_2(1) \quad T_2(2)], \quad U_{10} = \begin{bmatrix} P_{0,j-2} & P_{0,j-1} & P_{0,j} \\ P_{1,j-2} & P_{1,j-1} & P_{1,j} \\ P_{2,j-2} & P_{2,j-1} & P_{2,j} \end{bmatrix}, \quad U_{11} = \begin{bmatrix} P_{0,j-1} & P_{0,j} & P_{0,j+1} \\ P_{1,j-1} & P_{1,j} & P_{1,j+1} \\ P_{2,j-1} & P_{2,j} & P_{2,j+1} \end{bmatrix}; \end{aligned} \quad (4.177в)$$

- верхнего правого с индексами $i=1$ и $j=m$:

$$\begin{aligned} \Theta_0 &= [T_2(-1) \quad T_2(0) \quad T_2(1)], \quad \Theta_1 = [T_2(0) \quad T_2(1) \quad T_2'(1)], \\ Q_0 &= [T_2'(0) \quad T_2(0) \quad T_2(1)], \quad U_{00} = \begin{bmatrix} V_{0,m-2} & V_{0,m-1} & V_{0m} \\ P_{0,m-2} & P_{0,m-1} & P_{0m} \\ P_{1,m-2} & P_{1,m-1} & P_{1m} \end{bmatrix}, \quad U_{01} = \begin{bmatrix} V_{0,m-1} & V_{0m} & O \\ P_{0,m-1} & P_{0m} & W_{0m} \\ P_{1,m-1} & P_{1m} & W_{1m} \end{bmatrix}, \end{aligned} \quad (4.177г)$$

$$Q_1=[T_2(0) \ T_2(1) \ T_2(2)], \ U_{10}=\begin{bmatrix} P_{0,m-2} & P_{0,m-1} & P_{0m} \\ P_{1,m-2} & P_{1,m-1} & P_{1m} \\ P_{2,m-2} & P_{2,m-1} & P_{2m} \end{bmatrix}, \ U_{11}=\begin{bmatrix} P_{0,m-1} & P_{0m} & W_{0m} \\ P_{1,m-1} & P_{1m} & W_{1m} \\ P_{2,m-1} & P_{2m} & W_{2m} \end{bmatrix};$$

- левых внутренних с индексами $i \in [2, n-1]$ и $j=1$:

$$\begin{aligned} \Theta_0 &= [T_2'(0) \ T_2(0) \ T_2(1)], \ \Theta_1 = [T_2(0) \ T_2(1) \ T_2(2)], \\ Q_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ U_{00} = \begin{bmatrix} W_{i-2,0} & P_{i-2,0} & P_{i-2,1} \\ W_{i-1,0} & P_{i-1,0} & P_{i-1,1} \\ W_{i0} & P_{i0} & P_{i1} \end{bmatrix}, \ U_{01} = \begin{bmatrix} P_{i-2,0} & P_{i-2,1} & P_{i-2,2} \\ P_{i-1,0} & P_{i-1,1} & P_{i-1,2} \\ P_{i0} & P_{i1} & P_{i2} \end{bmatrix}, \\ Q_1 &= [T_2(0) \ T_2(1) \ T_2(2)], \ U_{10} = \begin{bmatrix} W_{i-1,0} & P_{i-1,0} & P_{i-1,1} \\ W_{i0} & P_{i0} & P_{i1} \\ W_{i+1,0} & P_{i+1,0} & P_{i+1,1} \end{bmatrix}, \ U_{11} = \begin{bmatrix} P_{i-1,0} & P_{i-1,1} & P_{i-1,2} \\ P_{i0} & P_{i1} & P_{i2} \\ P_{i+1,0} & P_{i+1,1} & P_{i+1,2} \end{bmatrix}; \end{aligned} \quad (4.177д)$$

- правых внутренних с индексами $i \in [2, n-1]$ и $j=m$:

$$\begin{aligned} \Theta_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ \Theta_1 = [T_2(0) \ T_2(1) \ T_2'(1)], \\ Q_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ U_{00} = \begin{bmatrix} P_{i-2,m-2} & P_{i-2,m-1} & P_{i-2,m} \\ P_{i-1,m-2} & P_{i-1,m-1} & P_{i-1,m} \\ P_{i,m-2} & P_{i,m-1} & P_{im} \end{bmatrix}, \ U_{01} = \begin{bmatrix} P_{i-2,m-1} & P_{i-2,m} & W_{i-2,m} \\ P_{i-1,m-1} & P_{i-1,m} & W_{i-1,m} \\ P_{i,m-1} & P_{im} & W_{im} \end{bmatrix}, \\ Q_1 &= [T_2(0) \ T_2(1) \ T_2(2)], \ U_{10} = \begin{bmatrix} P_{i-1,m-2} & P_{i-1,m-1} & P_{i-1,m} \\ P_{i,m-2} & P_{i,m-1} & P_{im} \\ P_{i+1,m} & P_{i+1,m-1} & P_{i+1,m} \end{bmatrix}, \ U_{11} = \begin{bmatrix} P_{i-1,m-1} & P_{i-1,m} & W_{i-1,m} \\ P_{i,m-1} & P_{i,m} & W_{i,m} \\ P_{i+1,m-1} & P_{i+1,m} & W_{i+1,m} \end{bmatrix}; \end{aligned} \quad (4.177е)$$

- нижнего левого с индексами $i=n$ и $j=1$:

$$\begin{aligned} \Theta_0 &= [T_2'(0) \ T_2(0) \ T_2(1)], \ \Theta_1 = [T_2(0) \ T_2(1) \ T_2(2)], \\ Q_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ U_{00} = \begin{bmatrix} W_{n-2,0} & P_{n-2,0} & P_{n-2,1} \\ W_{n-1,0} & P_{n-1,0} & P_{n-1,1} \\ W_{n0} & P_{n0} & P_{n1} \end{bmatrix}, \ U_{01} = \begin{bmatrix} P_{n-2,0} & P_{n-2,1} & P_{n-2,2} \\ P_{n-1,0} & P_{n-1,1} & P_{n-1,2} \\ P_{n0} & P_{n1} & P_{n2} \end{bmatrix}, \\ Q_1 &= [T_2(0) \ T_2(1) \ T_2'(1)], \ U_{10} = \begin{bmatrix} W_{n-1,0} & P_{n-1,0} & P_{n-1,1} \\ W_{n0} & P_{n0} & P_{n1} \\ O & V_{n0} & V_{n1} \end{bmatrix}, \ U_{11} = \begin{bmatrix} P_{n-1,0} & P_{n-1,1} & P_{n-1,2} \\ P_{n0} & P_{n1} & P_{n2} \\ V_{n0} & V_{n1} & V_{n2} \end{bmatrix}; \end{aligned} \quad (4.177ж)$$

- нижних внутренних с индексами $i=n$ и $j \in [2, m-1]$:

$$\begin{aligned} \Theta_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ \Theta_1 = [T_2(0) \ T_2(1) \ T_2(2)], \\ Q_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ U_{00} = \begin{bmatrix} P_{n-2,j-2} & P_{n-2,j-1} & P_{n-2,j} \\ P_{n-1,j-2} & P_{n-1,j-1} & P_{n-1,j} \\ P_{n,j-2} & P_{n,j-1} & P_{nj} \end{bmatrix}, \ U_{01} = \begin{bmatrix} P_{n-2,j-1} & P_{n-2,j} & P_{n-2,j+1} \\ P_{n-1,j-1} & P_{n-1,j} & P_{n-1,j+1} \\ P_{n,j-1} & P_{nj} & P_{n,j+1} \end{bmatrix}, \\ Q_1 &= [T_2(0) \ T_2(1) \ T_2'(1)], \ U_{10} = \begin{bmatrix} P_{n-1,j-2} & P_{n-1,j-1} & P_{n-1,j} \\ P_{n,j-2} & P_{n,j-1} & P_{nj} \\ V_{n,j-2} & V_{n,j-1} & V_{nj} \end{bmatrix}, \ U_{11} = \begin{bmatrix} P_{n-1,j-1} & P_{n-1,j} & P_{n-1,j+1} \\ P_{n,j-1} & P_{nj} & P_{n,j+1} \\ V_{n,j-1} & V_{nj} & V_{n,j+1} \end{bmatrix}; \end{aligned} \quad (4.177з)$$

- нижнего правого с индексами $i=n$ и $j=m$:

$$\begin{aligned} \Theta_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ \Theta_1 = [T_2(0) \ T_2(1) \ T_2'(1)], \\ Q_0 &= [T_2(-1) \ T_2(0) \ T_2(1)], \ U_{00} = \begin{bmatrix} P_{n-2,m-2} & P_{n-2,m-1} & P_{n-2,m} \\ P_{n-1,m-2} & P_{n-1,m-1} & P_{n-1,m} \\ P_{n,m-2} & P_{n,m-1} & P_{nm} \end{bmatrix}, \ U_{01} = \begin{bmatrix} P_{n-2,m-1} & P_{n-2,m} & W_{n-2,m} \\ P_{n-1,m-1} & P_{n-1,m} & W_{n-1,m} \\ P_{n,m-1} & P_{nm} & W_{nm} \end{bmatrix}, \end{aligned} \quad (4.177и)$$

$$Q_1 = [T_2(0) \ T_2(1) \ T_2'(1)], \quad U_{10} = \begin{bmatrix} P_{n-1,m-2} & P_{n-1,m-1} & P_{n-1,m} \\ P_{n,m-2} & P_{n,m-1} & P_{nm} \\ V_{n,m-2} & V_{n,m-1} & V_{nm} \end{bmatrix}, \quad U_{11} = \begin{bmatrix} P_{n-1,m-1} & P_{n-1,m} & W_{n-1,m} \\ P_{n,m-1} & P_{nm} & W_{nm} \\ V_{n,m-1} & V_{nm} & O \end{bmatrix}.$$

Реализация особых краевых направлений в сплайне с перекрытиями возможна лишь путём явного задания равных либо противоположных векторов краевых направлений $\forall i=\overline{0,n}$ и $j=\overline{0,m}$:

- в циклическом сплайне $V_{0j}=V_{nj}$ и $W_{i0}=W_{im}$;
- в ациклическом сплайне $V_{0j}=-V_{nj}$ и $W_{i0}=-W_{im}$.

Второй тип краевых условий — свободные края с нулевыми значениями векторов вторых производных $p''(0,\tau)$, $p''(n,\tau)$, $p''(t,0)$ и $p''(t,m)$ — реализуется по (4.176) с матрицами (4.177), в которых нужно заменить векторные функции T_2' на функции T_2'' с теми же аргументами, а векторы краевых направлений V и W на аналогичные (2.115в) и (2.116в) векторы краевых ускорений:

- V_{0j} на $0.5(P_{0j}-2P_{1j}+P_{2j})$, а V_{nj} на $0.5(P_{n-2,j}-2P_{n-1,j}+P_{nj}) \ \forall j=\overline{0,m}$;
- W_{i0} на $0.5(P_{i0}-2P_{i1}+P_{i2})$, а W_{im} на $0.5(P_{i,m-2}-2P_{i,m-1}+P_{im}) \ \forall i=\overline{0,n}$.

На рис. 4.172 приведены примеры построения сплайновых поверхностей с перекрытием параболических сегментов, демонстрирующие не худшее, а чаще лучшее качество интерполяции в сравнении с поверхностями, показанными на рис. 4.169 (при этом параметрические модели сегментов точно совпали с моделями из примера 4.31), 4.168 и 4.97,б. Благодаря отсутствию операций обращения матриц высокой размерности, необходимых в (4.175), построение кубических сплайнов с перекрытиями происходит гораздо быстрее.

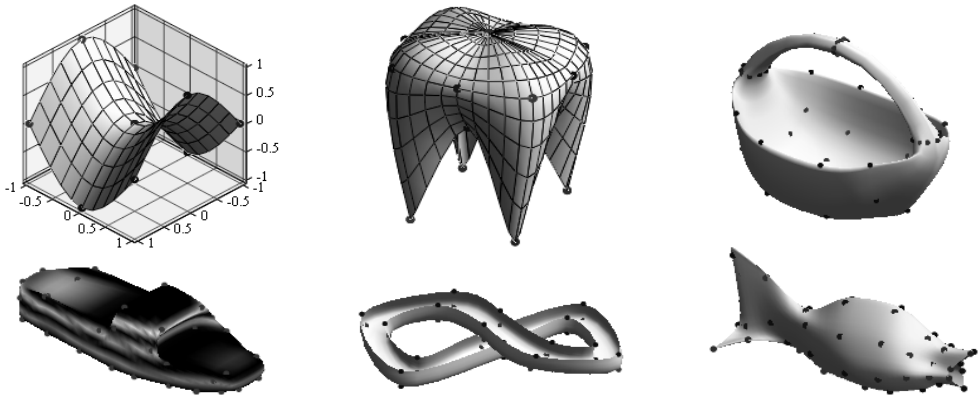


Рис. 4.172

Главным **недостатком** изученного здесь вида сплайна является пониженная, но визуально незаметная степень гладкости сопряжения его сегментов — первая вместо второй, свойственной алгоритму, описанному в п. 4.6.1. Это приводит к скачкам кривизны поверхности в точках сшивки её сегментов. С другой стороны, разрывность кривизны сплайновой поверхности можно обратить на пользу для создания в стыках сегментов линий перегиба и расширения возможностей конструирования сложных поверхностей.

4.7. ДИСКРЕТНЫЕ ПОВЕРХНОСТИ

Традиционная визуализация сложных поверхностей, основанная на представлении их системой плоских полигонов (треугольников), вплотную подошла к пределу своего развития. Современные видеоадаптеры достигли скорости вывода миллиарда треугольников в секунду. В чистом виде полигональные модели криволинейных поверхностей имеют ряд недостатков, с которыми приходится бороться на программном и аппаратном уровнях визуализации:

- окраска пикселей плоской ячейки в постоянный цвет, зависящий по (4.1) от ориентации нормали, создаёт явно различимые границы между полигонами (см. рис. 4.10,б), особенно хорошо заметные при приближении наблюдателя к поверхности. Наиболее распространённым «методом грубой силы» решения данной проблемы является уменьшение размеров ячеек, сопровождаемое увеличением их числа. Хорошего сглаживания окраски на границах крупных полигонов и создания иллюзии кривизны плоской грани можно добиться путём билинейной интерполяции либо вектора нормали по (4.82а) в методе Фонга, либо на прямую цвета закраски по (4.82б) в методе Гуро (см. п. 4.2.3);

- абрис проявляет на контрастном фоне структуру ломаной линии (см. рис. 4.18). Этот дефект частично устраняется аппроксимацией поверхности бóльшим числом полигонов. Наиболее эффективна адаптивная аппроксимация, изменяющая размер и число ячеек в зависимости от расстояния до наблюдателя;

- увеличение количества аппроксимирующих поверхность полигонов приводит к возрастанию (обычно квадратичному) сложности анализа их видимости и взаимного экранирования. С учётом достижения практически предельно возможной частоты системной шины компьютера лучшим средством повышения скорости визуализации является распараллеливание вычислений;

- множество мелких граней полигональных поверхностей усложняет расчёт их взаимных пересечений и структуру результата булевых операций;

- полигональная 3d-графика не является трёхмерной в полном смысле этого слова, ибо у реальных объектов не существует абсолютно гладких и плоских граней. Благодаря молекулярному строению тел их поверхности при любом разрешении имеют шероховатую структуру с множеством случайно ориентированных микрограней, трудно поддающихся полигональной аппроксимации;

- невозможность полигональной визуализации большого класса аморфных систем слабосвязанных частиц: потока жидкости, клубов дыма, облаков, языков пламени, снегопада, почвенного покрова и других природных явлений.

Границы таких объектов — *дискретные* (графические) поверхности, не гладкие ни в каком своём фрагменте, имеющие разрывы, просветы и другие нарушения непрерывности, — описываются конечным множеством принадлежащих им *точек*. Помимо вышеперечисленных к ним относятся поверхности раздела физических сред, полученные в результате объёмного сканирования: рентгеновские снимки, компьютерные томограммы и ультразвуковые снимки внутренних органов, данные эхолотации морского дна и подводных объектов, радиолокации воздушных и космических целей и т. д. и т. п.

Цель изучения графических поверхностей заключается в умении строить их разноразмерные изображения и выполнять анимацию таких объектов. Весьма ценной в медицинской практике является возможность создания 3d-атласов внутренних органов и тренажёров, с помощью которых будущие врачи смогут приобрести знания и наглядный анатомический опыт.

В современной терминологии минимальные элементы (атомы) трёхмерного пространства называются **вокселями** (*VOlume piXEL*) — объёмными пикселями. Воксельная технология разбивает пространство на дискретные неделимые 3d-элементы, имеющие в отличие от математических точек **ненулевой объём**. Исходная информация о дискретной поверхности, состоящей из n_v вокселей, задаётся **массивом координат** в виде линейного списка $P = \{p_1, \dots, p_{n_v}\}$ либо как $n \times m \times 3$ -матрица. При преобразовании параметрической поверхности $p(t, \tau)$ в дискретную можно расположить воксели в узлах сетки $p(t_i, \tau_j)$.

Для создания впечатления о целостности воксельной поверхности её элементы должны отстоять друг от друга на оптимальном расстоянии. Оптимальная **частота пространственного квантования** вокселей должна быть такой, чтобы их изображения плотно заполняли пиксели экрана без проколов и наложений (рис. 4.173,а). Например, если размер пиксела равен 0.25 мм, а масштаб изображения 1/1000, то оптимальный интервал между вокселями, имеющими на экране размер пиксела, составляет 25 см. Такими же должны быть и диаметры вокселей, чтобы они размещались в пространстве вплотную друг к другу.

Редкое расположение вокселей характерно скорее для системы независимых точек (рис. 4.173,б), чем для упорядоченной структуры. Один из способов заполнения пустоты между редкими вокселями — увеличение их диаметра с небольшим перекрытием друг друга (рис. 4.173,в). При большой частоте выборки резко возрастают машинные ресурсы для хранения и обработки всех точек. Однако, это не улучшает детализации изображения, т. к. дополнительные точки не могут быть помещены между пикселями экрана. Избыточная частота квантования может быть полезна в том случае, если в дальнейшем потребуется увеличить графическое разрешение экрана или масштаб изображения (рис. 4.173,г).

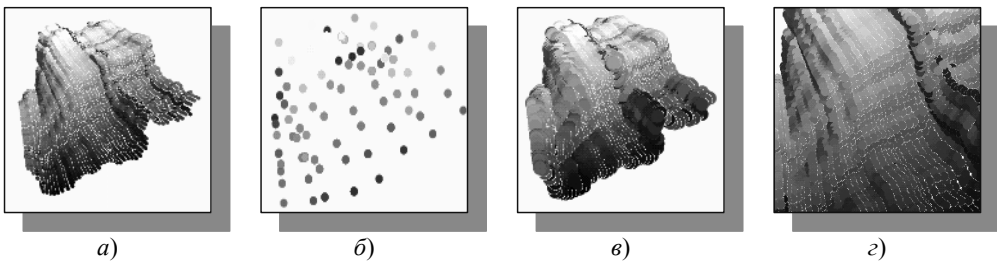


Рис. 4.173

Рассмотрим особенности **визуализации воксельных поверхностей**. В начальном приближении воксел можно изобразить маленьким **кубиком**, у которого видно не более трёх граней, закрашиваемых по (4.1). Более естественно воксел выглядит **шариком** определённого радиуса r_v (рис. 4.174). Равномерная за-

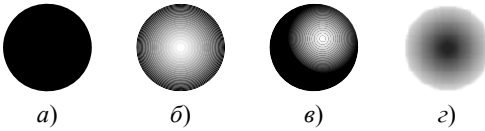


Рис. 4.174

ливка круга (а) применяется тогда, когда диаметр его проекции на экране составляет единицы пикселей. Для придания крупному изображению воксела иллюзии объёмности используется градиентная закраска круга —

как центральная (б), так и смещённая к источнику освещения (в). Эффект бархатистой поверхности создаётся изображением воксела в виде аэрозольного пятна с радиально уменьшающейся плотностью точек (г).

Многоцветье воксельной поверхности может быть достигнуто изменением цвета закраски воксела в зависимости от некоторого направления сортировки:

- в топографической или радужной палитрах — вдоль выбранного направления, например, по высоте как на рис. 4.173;
- для создания воздушной перспективы — по удалённости центра воксела p_i от наблюдателя S , вычисляемой как $d_i = |p_i - S|$ при ближнем или $d_i = -p_i \circ S$ при дальнем наблюдателе (см. рис. 1.181);
- комбинирование этих способов: например, выбор цвета воксела по высоте, а уменьшение яркости цвета — по удалению воксела от наблюдателя.

Реалистичное окрашивание воксела по (4.1) требует знания нормали к поверхности в месте его расположения. В связи с неаналитичностью и недифференцируемостью дискретной поверхности аппроксимируем её в окрестности изображаемого воксела p_i полиэдром, для чего найдём ближайшие к p_i некопланарные воксели p_j, p_k и p_l (рис. 4.175). Вектор нормали N_i вычислим усреднением, а ещё проще — аналогично (4.5) суммированием трёх нормированных нормалей к треугольным граням $p_i p_j p_k, p_i p_k p_l$ и $p_i p_l p_j$:

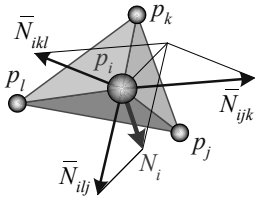


Рис. 4.175

ренцируемостью дискретной поверхности аппроксимируем её в окрестности изображаемого воксела p_i полиэдром, для чего найдём ближайшие к p_i некопланарные воксели p_j, p_k и p_l (рис. 4.175). Вектор нормали N_i вычислим усреднением, а ещё проще — аналогично (4.5) суммированием трёх нормированных нормалей к треугольным граням $p_i p_j p_k, p_i p_k p_l$ и $p_i p_l p_j$:

$$N_i = \bar{N}_{ijk} + \bar{N}_{ikl} + \bar{N}_{ijl}, \text{ где } \bar{N}_{ijk} = (p_j - p_i) \times (p_k - p_i), \quad (4.178)$$

$$N_{ikl} = (p_k - p_i) \times (p_l - p_i), \quad N_{ijl} = (p_l - p_i) \times (p_j - p_i).$$

Поиск вокселей, ближайших к p_i , осуществляется методом сортировки скалярных произведений $(p_j - p_i) \circ (p_j - p_i) \quad \forall j = \overline{1, n_v}, j \neq i$. Полный перебор требует очень больших вычислительных затрат. Рациональнее изображать дискретную поверхность методом, подобным алгоритму z-буфера (см. рис. 1.185):

- определяются экранные проекции всех вокселей

$$\tilde{p}_{i3} = [x_{i3} \ y_{i3} \ 0 \ 1] \sim [x_i \ y_i \ z_i \ 1] C_{п3}, \quad (4.179)$$

где $C_{п3}$ — матрица (3.114) преобразования пространственных координат точки в её экранные координаты;

- из каждой группы точек с равными **округлёнными** координатами $\{x_3, y_3\}$ отбирается **одна** видимая наблюдателем S ближайшая точка p_6 с **минимальным** удалением $d_6 = |p_6 - S|$ от ближнего или $d_6 = -p_6 \circ S$ от дальнего наблюдателя;

• по (4.178) вычисляется нормаль N_6 , а по (4.1) — цвет окраски ближайшего к наблюдателю воксела, который можно принять равным $c = \left| \overline{H} \circ \overline{N}_6 \right| c_6 \bullet I_\phi$.

Богатые возможности в изображении дискретных поверхностей даёт использование вокселей с изменяемой прозрачностью. Введение **коэффициента непрозрачности** (*opacity*) $0 \leq \alpha \leq 1$, одной из ипостаси **альфа-канала** [65], позволяет видеть одни поверхности сквозь другие, а заданием $\alpha=0$ быстро отключать видимость отдельного воксела или группы вокселей без их физического удаления, т. е. сделать их прозрачными. Одновременная видимость нескольких поверхностей в одной области экрана означает, что перекрываемые ими пиксели должны нести информацию о цвете и степени прозрачности поверхностей. С учётом очередности вывода поверхностей на экран определим **два способа наложения** пиксела **источника** цвета c_n с коэффициентом непрозрачности α_n на пиксел **приёмника** с цветом c_n :

- **детерминированное** смешивание цветов

$$c = \text{lin}(c_n, c_n, \alpha_n) = (1 - \alpha_n) c_n + \alpha_n c_n. \quad (4.180a)$$

При $0 < \alpha_n < 1$ все пиксели воксельного круга изменяют свою окраску;

- **стохастический** выбор цвета пиксела, равного c_n с вероятностью α_n либо c_n с вероятностью $1 - \alpha_n$:

$$c = \text{lin}(c_n, c_n, \beta) = (1 - \beta) c_n + \beta c_n, \text{ где } \beta = \{ \text{rnd}(1) \leq \alpha_n \} \in \{0, 1\}. \quad (4.180б)$$

Здесь нет иных цветов, кроме c_n и c_n , а эффект полупрозрачности создаётся в мозгу наблюдателя благодаря пространственному усреднению сигналов рецепторов глаза и малому размеру пикселей. Однако, при $\alpha_n \approx 0$ воксел изображается слишком малым числом пикселей, с трудом передающим его форму.

Работоспособность приведённых методов цветосмешения проиллюстрирована на рис. 4.176, где на светло-сером фоне построены изображения трёх равномерно окрашенных кругов: дальнего (1), среднего (2) и ближнего (3). На изображении (а) круги полностью непрозрачны, т. е. $\alpha_1 = \alpha_2 = \alpha_3 = 1$. На (б) эти же круги с коэффициентами непрозрачности $\alpha_1 = 0.3$, $\alpha_2 = 0.7$ и $\alpha_3 = 0.5$ построены способом (4.180а), а на (в) — по (4.180б). Эффект полной прозрачности красного воксела с коэффициентом $\alpha_2 = 0$ виден на рис. 4.176,г. Сравнение рисунков показывает, что метод детерминированного смешивания цветов даёт изображение полупрозрачных объектов более реалистичное, сплошное, с чёткими границами, а стохастическая цветовая смесь лучше подходит для изображения вокселей в виде «пушистых» шариков (см. рис. 4.174,г).

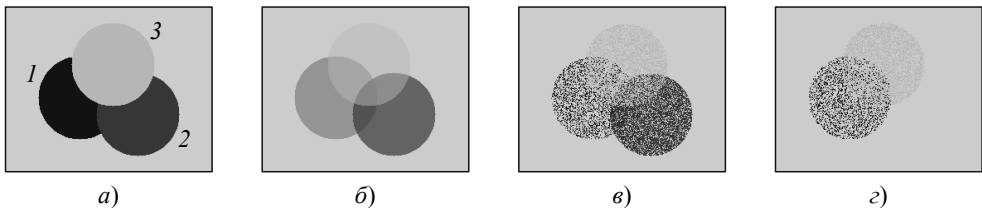


Рис. 4.176

Переходим к технологии изображения сферического вокселя радиуса r_b с центром в точке $p_i = [x_i \ y_i \ z_i]$ и собственным цветом c_b в каждом из четырёх видов на рис. 4.174. В первую очередь по (4.179) вычисляется проекция его центра $p_{i3} = [x_{i3} \ y_{i3} \ 0]$. Далее необходимо определиться с радиусом r круга, изображающего воксел на экране. Эта величина может быть:

- задана произвольно, независимо от радиуса вокселя r_b и его расположения;
- при дальнем наблюдателе, расположенном в направлении вектора S , по законам параллельного проецирования полагаем $r = r_b$;
- при ближнем наблюдателе, находящемся в точке S :

$$r = r_b |p_{i3} - S| / |p_i - S|. \quad (4.181)$$

Закраска площади проекции вокселя выполняется построчным перебором $x = x_l(y), x_p(y)$ пикселей круга (x, y) от левой до правой целочисленных границ $x_l(y) = \left\lceil x_{i3} - \sqrt{r^2 - (y - y_{i3})^2} \right\rceil$ и $x_p(y) = \left\lfloor x_{i3} + \sqrt{r^2 - (y - y_{i3})^2} \right\rfloor$ в цикле перебора ординаты $y = \left\lceil y_{i3} - r \right\rceil, \left\lfloor y_{i3} + r \right\rfloor$. Цвет закраски пикселя-источника $c_{и}$, используемый в (4.180), выбирается в зависимости от желаемого вида вокселя:

- **равномерное окрашивание** получается при $c_{и} = c_b$ без учёта освещения поверхности (рис. 4.176) либо при $c_{и} = c_b \bullet I_{\Phi}$ с учётом фонового освещения I_{Φ} ;
- **центральная градиентная закраска** (рис. 4.177,а) имитирует диффузное отражение фонового освещения от сферы:

$$c_{и}(x, y) = (c_b \bullet I_{\Phi} / r) \sqrt{r^2 - (x - x_{i3})^2 + (y - y_{i3})^2}; \quad (4.182a)$$

- **смещённая градиентная закраска** (рис. 4.177,б) моделирует диффузное отражение косого направленного освещения. Во избежание сложных оптических расчётов, неоправданных из-за малого размера вокселя, целесообразно просто сдвинуть центральную градиентную заливку (4.182а) в желаемом направлении, например, на вектор Δ и рассчитывать цвет пикселя (x, y) по аналогичной формуле с вектором интенсивности направленного освещения I_n :

$$c_{и}(x, y) = (c_b \bullet I_n / r) \sqrt{r^2 - (x - x_{i3} - \Delta_x)^2 + (y - y_{i3} - \Delta_y)^2}. \quad (4.182б)$$

При этом светлые блики на изображениях всех вокселей располагаются одинаково независимо от реального размещения источника света и наблюдателя в пространстве сцены. Данный метод изображения дискретной поверхности страдает отсутствием теней вокселей друг на друга от направленного источника, но, как говорится, «хотеть не вредно» и «всего в меру»;

- **аэрозольная закраска** (см. рис. 4.174,з) реализуется соединением стохастического метода выбора цвета пикселя (4.180б) с неравномерной радиальной плотностью пикселей цвета $c_{и}$. Зададим желаемую **функцию радиальной плотности** $\mu(d) \in [0, 1] \forall d \in [0, r]$, максимальную при $d=0$ и минимальную (не обязательно нулевую) при $d=r$. Скажем, подходящими функциями являются

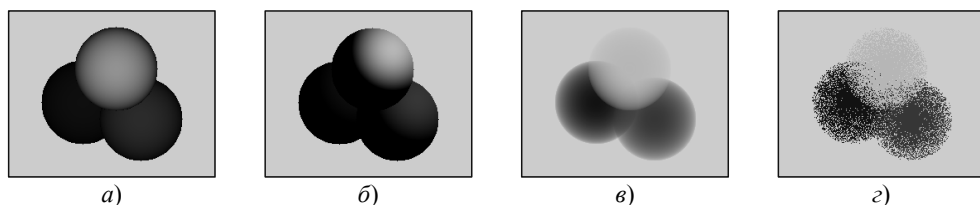


Рис. 4.177

$$\mu_{\text{ш}}(d) = \sqrt{1 - d^2/r^2}, \quad \mu_{\text{л}}(d) = 1 - d/r, \quad \mu_{\text{г}}(d) = \mu_r^{d^2/r^2}.$$

Зависимость $\mu_{\text{ш}}(d)$ соответствует модели вокселя как пушистого **шарика** и описывает его относительную толщину вдоль луча, проходящего от центра на расстоянии d . Функция $\mu_{\text{л}}(d)$ задаёт **линейное** уменьшение плотности шарика, а $\mu_{\text{г}}(d)$ моделирует **гауссово** распределение плотности аэрозольного распыления от единицы в центре круга до желаемого краевого значения $\mu_r(r) = \mu_r < 1$.

Рассматривая $\mu(d)$ как вероятность присвоения пикселу-источнику (x, y) , находящемуся от центра круга на расстоянии $d = \sqrt{(x - x_{i_0})^2 + (y - y_{i_0})^2}$, цвета вокселя $c_{\text{в}}$, получим два аналогичных (4.180) варианта окраски этого пиксела:

- детерминированное смешение цветов вокселя и приёмника (рис. 4.177, в)

$$c_{\text{и}} = \text{lin}(c_{\text{п}}, c_{\text{в}}, \mu(d)) = (1 - \mu(d))c_{\text{п}} + \mu(d)c_{\text{в}}; \quad (4.183a)$$

- стохастический выбор цвета, равного $c_{\text{в}}$ с вероятностью $\mu(d)$ либо $c_{\text{п}}$ с вероятностью $1 - \mu(d)$ (рис. 4.177, г)

$$c_{\text{и}} = \text{lin}(c_{\text{п}}, c_{\text{в}}, \lambda(d)), \quad \text{где } \lambda(d) = \{ \text{rnd}(1) \leq \mu(d) \} \in \{0, 1\}. \quad (4.183б)$$

В заключение отметим некоторые **преимущества** техники воксельного моделирования и визуализации поверхностей в сравнении с полигональным:

- адекватность изображения большого числа объектов дискретного строения со сложными геометрическими и оптическими свойствами;
- снятие ограничений на сложность поверхности, определяемой лишь числом вокселей и их расположением, а не уравнением, формой, непрерывностью и кривизной, что характеризует параметрическую поверхность;
- лёгкость выполнения булевых операций, заключающаяся в физическом удалении либо изменении прозрачности соответствующих вокселей по результатам проверки их принадлежности поверхностям-операндам;
- простота преобразования одной поверхности в другую («морфинг») путём покадрового перемещения вокселей со старых позиций в новые.

Недостатками дискретных моделей являются:

- большой объём машинной памяти, необходимой для хранения вокселей;
- сложности подбора геометрического разрешения и размера вокселей для создания впечатления неразрывности поверхности;
- трудоёмкость преобразования воксельной модели в полигональную, связанного с большим объёмом поиска вокселей, ближайших друг к другу.

В настоящей главе рассматриваются методы моделирования сложных пространственных объектов и систем объектов, составляющих конструкции. *Объект* есть часть пространства, заключённая между некоторыми *границами*. Границы играют важную роль в моделировании объектов компьютерной графики. Отметим лишь некоторые аспекты роли граничных элементов в компьютерной графике:

- благодаря непрозрачности граничных поверхностей освещённого объекта становится видна его форма;
- отражательные свойства граничных поверхностей обуславливают цветовое восприятие объекта наблюдателем;
- взаимное расположение объектов определяется специальными тестами ориентации их граничных элементов;
- в алгоритмах трассировки находятся пересечения световых лучей с границами объектов;
- в задачах пересечения полезно окружать сложные объекты виртуальными габаритными оболочками простых форм типа сферы или параллелепипеда. При небольшом числе граничных поверхностей оболочек резко уменьшается объём перебора пересекаемых элементов и сокращается время расчёта.

Изображение 3d-объектов и конструкций является мощным графическим средством зрительного восприятия содержащейся в них геометрической информации. Далеко не каждый человек может построить в своем воображении объёмную фигуру по её словесному описанию и даже по чертежам отдельных видов (см. рис. 3.50,б). Проектируя изделие с помощью кульмана или заменяющей его программы, конструктор отдаёт основное время рутинной, лишённой творчества работе по выбору геометрических примитивов (отрезков и дуг) и их расположению на чертеже, а не осмыслению конструкции в целом. Особенно наглядно недостатки видовой визуализации проявляются при ограниченности объекта гладко сопряжёнными криволинейными поверхностями без явно выраженных рёбер и вершин, когда кроме абриса на видах и изображать-то нечего.

Гораздо более информативен показ пространственных объектов в аксонометрических или перспективной проекциях (см. разд. 3.2), повышению реалистичности которых способствует окраска поверхностей по (4.1), зависящая от их ориентации и освещения. Наиболее выгодный для обзора ракурс достигается воздействием на объекты преобразований переноса и вращений. Всё это, разумеется, имеет смысл только в том случае, когда модели объектов содержат графические элементы, способные отразить не только внешние очертания фигур, но и детали рельефа их поверхностей и внутреннего строения. Изучением методов описания трёхмерных объектов мы и займёмся в этой главе.

5.1. МЕТОДЫ ТРЁХМЕРНОГО МОДЕЛИРОВАНИЯ

А лишь малая часть пространственных объектов ограничена какой-то одной замкнутой поверхностью. Для описания строения большинства объёмных тел приходится использовать фрагменты нескольких граничных поверхностей, стыкующихся друг с другом по некоторым граничным линиям. Физические свойства объекта (масса, центр тяжести, моменты и оси инерции) определяются свойствами материала, заполняющего его внутренность.

К граничным элементам объекта относятся [13, 17, 20, 47, 50]:

- **ячейки** — ограниченные поверхности, отделяющие внутреннее пространство объекта от внешнего мира. Плоские ячейки будем называть **гранями**;
- **рёбра** — линии соединения ячеек. Грани пересекаются по отрезкам прямых линий. Последовательность рёбер, ограничивающих ячейку, образует **цикл**;
- **вершины** — точки стыковки рёбер.

На рис. 5.1 приведена **классификация моделей** пространственных объектов, используемых в современных системах автоматизированного проектирования. Наибольшее распространение получили три основных типа моделей [17]: каркасные, граничные и сплошные. Каждая из них характеризуется своей сложностью и точностью описания реальных трёхмерных тел.

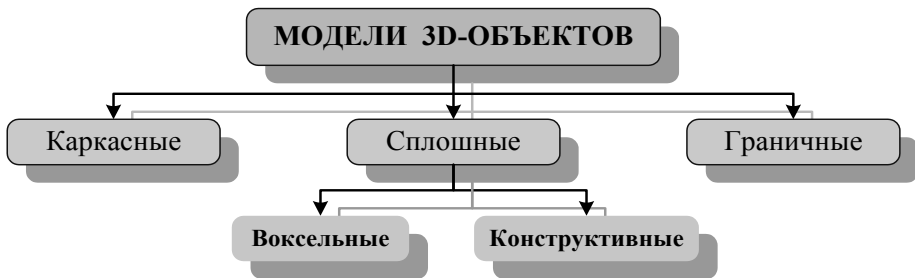


Рис. 5.1

❑ **Каркасная** модель в виде охватывающей объект проволоочной сетки (*wire-frame*) является простейшим средством передачи формы трёхмерного объекта. Элементами каркаса могут быть отрезки прямых и кривых линий, полилинии и, наконец, составные линии пересечения поверхностей, ограничивающих объект. Поверхности между каркасными линиями не определены, но могут быть воссозданы методами полиномиальной интерполяции, изученными в разд. 4.4.

Проволоочные модели применяются для быстрого, не требующего детальной визуализации, **эскизного** изображения объектов и их габаритных оболочек. Построение каркаса является начальным этапом трёхмерного моделирования, позволяющего показать форму изделия в общих чертах. Для получения более реалистичных изображений используются продвинутые средства визуализации — скрытие невидимых линий и окрашивание видимых поверхностей, что приближает каркасные объекты к объектам следующего класса.

❑ **Граничная** (*boundary*) (синоним — **поверхностная**) модель напоминает пустую коробку и обеспечивает лучшее восприятие объекта, чем каркасная мо-

дель. Она включает перечень всех элементов, составляющих его границы — тонкие стенки, под которыми находится пустое пространство. Оболочка, ограничивающая тело, имеет площадь и определённые механико-оптические свойства, делающие возможной её реалистичную визуализацию.

Граничное моделирование информативнее каркасного: оно позволяет детально проработать форму ячеек и показать их взаимное расположение благодаря скрытию невидимых элементов. Наличие моделей поверхностей позволяет обнаружить пересечение объектов друг с другом. Каркасное изображение гранично определённой модели строится достаточно легко, т. к. описание рёбер явно присутствует в структуре данных. Определённую трудность представляет вычисление инерционно-массовых характеристик пустотелых объектов и выполнение с ними булевых операций, при которых появляются новые линии и поверхности, отсутствующие в граничных моделях исходных тел.

Область применения поверхностного моделирования — конструирование сложных составных поверхностей, не подвергаемых разрезам и сечениям.

□ **Сплошная** (***solid***) (синонимы — объёмная, монолитная, твёрдотельная) модель не только хранит полную информацию о внешних граничных элементах объекта, но и позволяет описать его внутреннее устройство, которое становится видным при разрезании. Благодаря монолитности сплошная модель содержит физические характеристики материала, из которого изготовлен 3d-объект, что позволяет рассчитать его объём и массо-инерционные характеристики.

Описание сплошных тел подразумевает их заполнение **вокселями** — мельчайшими 3d-элементами (кубиками, шариками) — либо **конструирование** сложной формы как композиции из небольшого числа простых объёмных примитивов (куб, сфера, цилиндр, конус, тор), с которыми выполняются необходимые аффинные преобразования на этапе сборки. Последовательность конструирования задаётся древовидной структурой, терминальными вершинами которой являются примитивы, нетерминальными — операторы над примитивами, а ветвями — матрицы преобразований промежуточных форм.

Твёрдотельное моделирование лежит в основе конструирования изделий машиностроения. Оно позволяет получать необходимые проекции пространственного объекта, разрезы и сечения, необходимые для оформления рабочей документации, а также эффективно решать компоновочные задачи. Сплошные модели органично приспособлены к работе в системах логического конструирования сложных форм. Алгоритмы выполнения булевых операций (пересечения, объединения и исключения) с 3d-объектами являются составной частью данного способа моделирования, области применения которого — различные САПР сложных монолитных конструкций и сборочных изделий, системы архитектурного и других видов проектирования.

5.2. КАРКАСНЫЕ МОДЕЛИ

Нзначение каркасной модели 3d-объекта — отразить форму его граней с помощью рёбер и других линий. В зависимости от свойств граней

выбираются и методы построения каркасного изображения. В случае *плоской* грани вполне достаточен вывод её рёбер (рис. 5.2,а). Заметим, что условие компланарности вершин $m=2$ в (1.1126) — только необходимое условие планарности всех точек грани (см. рис. 4.93). Следовательно, информация о свойствах граничных поверхностей объекта должна быть априорно известной.

Адекватное восприятие *объёмной* ячейки невозможно без дополнительных каркасных линий, проведённых в междрёберном пространстве. Особенно это важно в случае планарности вершин, когда информация о внутренних точках поверхности становится принципиально необходимой: без них любые методы интерполяции дают плоские решения. В подтверждение сказанного на рис. 5.2,б показан объект, имеющий неплоскую ячейку с такими же планарными вершинами, что у объекта на рис. 5.2,а. Очевидно, что без промежуточных каркасных линий реальная форма объёмной грани не воспринимается.

Для построения каркаса криволинейной поверхности с известными граничными элементами можно использовать алгоритмы граничной интерполяции, а при задании дополнительных узловых точек и касательных векторов внутри ячейки и на её рёбрах могут быть полезны методы полиномиальной или сплайновой интерполяции (см. разд. 4.4, 4.6).

Каркасное изображение объекта, построенное без учёта перспективы, выглядит неоднозначно ввиду субъективности восприятия его формы и ориентации в пространстве разными наблюдателями: один видит параллелепипед на рис. 5.2,а как бы сверху и справа, как показано на рис. 5.2,в, а другому наблюдателю ближними кажутся передняя, левая и нижняя грани (рис. 5.2,г).

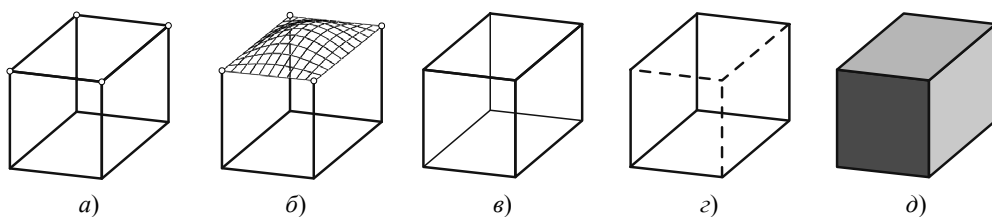


Рис. 5.2

Большей реалистичности и однозначности восприятия можно достичь несколькими способами, заимствованными из методов визуализации непрозрачных полигональных сетей, описанных в п. 1.7.2:

- **утолщением** рёбер видимых граней (рис. 5.2,в);
- выводом невидимых рёбер **штриховым стилем** (рис. 5.2,г);
- полным **удалением** невидимых рёбер путём блокировки их вывода на экран либо закраски видимых граней (рис. 5.2,д).

Реализация этих решений методами Робертса и приоритетного списка основана на определении видимости граней объекта наблюдателем по правилу (1.181). В первых двух вариантах различения видимых и невидимых рёбер контуры лицевых граней, чьи **внешние** нормали обращены в сторону наблюдателя, рисуются сплошными линиями желаемой толщины, а контуры нелицевых — тонкими сплошными либо штриховыми линиями. При этом сплошные линии

закрывают просветы в штриховых линиях абриса, который в результате наложения выглядит сплошным. Реалистичное изображение полигонального объекта с непрозрачными ячейками достигается выводом на экран окрашенных и, возможно, оконтуренных полигонов только лицевых граней. тогда как вывод нелицевых граней блокируется.

5.2.1. Модели полиэдров

Рассмотрим подробнее, чем в п. 1.5.4, вопросы создания и применения различных моделей полиэдра, состоящего из n_v вершин, n_p рёбер и n_f граней. Напоминаем, что эти количественные параметры полиэдра связаны формулой Эйлера (1.141а): $n_v + n_f = 2 + n_p$. Полиэдры с глухими и сквозными отверстиями подчиняются, соответственно, соотношениям (1.141б) и (1.141в). Сравнительный анализ моделей проведём на примере полиэдра Н с параметрами $n_v=9$, $n_f=7$ и $n_p=14$, показанного на рис. 5.3,а.

□ Простейшим описанием топологии полиэдра, достаточным для построения его каркасного изображения, является список $Ls = \{a_1 b_1, \dots, a_{n_p} b_{n_p}\}$, составляющий *модель «рёбра»*. Элемент этого списка — отрезок i -го ребра ($i=1, n_p$), соединяющего смежные вершины a_i и b_i в *произвольном* направлении. Согласно *произвольной* нумерации рёбер анализируемого полиэдра Н их список таков:

$$Ls = \{p_1 p_2, p_2 p_3, p_3 p_4, p_1 p_4, p_4 p_5, p_5 p_6, p_6 p_7, \\ p_3 p_7, p_1 p_5, p_6 p_8, p_1 p_8, p_7 p_9, p_2 p_9, p_8 p_9\}. \quad (5.1a)$$

Построение каркаса производится в цикле перебора элементов списка Ls и рисования на экране дисплея *проекций* $a_i b_i$ отрезков между точками a_i и b_i с *отрывом пера* в конце каждого отрезка: $ab = Ls_i, \text{ line}(a_i, b_i) \forall i=1, \text{size}(Ls)$. При этом каждое ребро рисуется на экране только один раз, что делает данный способ построения каркаса самым быстрым из всех возможных.

Недостаток модели «рёбра» в том, что каждая вершина «прописана» в списке Ls многократно (минимум трижды) и нерегулярно. Из-за этого ручное задание топологии даже неподвижного полиэдра чревато ошибками ввода одинаковых чисел в разные места списка, что ведёт к нестыковке рёбер. Если же полиэдр как-то движется в пространстве сцены, то из-за нерегулярного положения вершин в списке рёбер приходится обновлять их по (3.1) поочерёдно и индивидуально, что увеличивает объём текста программы и усложняет её отладку.

□ Устранение множественности описания вершин полиэдра достигается с помощью специального *координатного списка* $P = \{p_1, p_2, \dots, p_{n_v}\}$, в котором каждая вершина имеет свой уникальный номер (индекс). Благодаря однородности элементов список вершин может быть организован по (1.142) в виде *координатных матриц* $P \in \mathbb{R}^{n_v \times 3}$ либо $\tilde{P} \in \mathbb{R}^{n_v \times 4}$, строки которых хранят декартовы либо однородные координаты точек p_i .

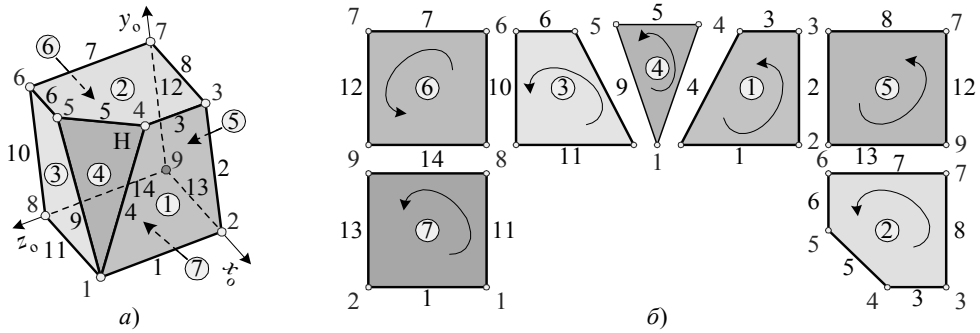


Рис. 5.3

Положения вершин полиэдра отсчитываются от начала системы координат сцены $x_c y_c z_c$ (абсолютный метод) либо от некоторой базовой точки — начала объектной системы координат $x_0 y_0 z_0$ (относительный метод). Ею может служить центр симметрии полиэдра или любая из его вершин. Составим, например, координатную матрицу полиэдра на рис. 5.3,а с базовой точкой в вершине p_9 :

$$P = \begin{bmatrix} 2 & 2 & 2 & 2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 2 & 2 & 2 & 2 & 0 & 0 \\ 2 & 0 & 0 & 1 & 2 & 2 & 0 & 2 & 0 \end{bmatrix}^T. \quad (5.16)$$

Относительные координаты предпочтительнее абсолютных. Например, координаты вершин параллелепипеда, повернутого вокруг некоторых, пусть даже координатных осей на известные углы, не столь очевидны, тогда как задать регулярное центрированное положение куба единичных размеров с гранями, параллельными координатным плоскостям, очень просто. Такого рода стандартные фигуры (призмы, пирамиды, сферы, цилиндры, конусы) называются **примитивами**. Превращение примитива P_0 в подобный ему полиэдр P с желаемыми размерами и ориентацией в пространстве сцены (см. рис. 3.39) производится с помощью геометрических преобразований по формулам (3.68).

Благодаря однозначной нумерации вершин в координатном списке порядок их соединения рёбрами определён **индексным** списком $R = \{R_1, R_2, \dots, R_{n_p}\}$, организованным, например, в виде $n_p \times 2$ -матрицы **рёбер** R , строки которой хранят пары индексов вершин, соединённых ребром: $R_i = [r_{i1} \ r_{i2}] \ \forall i = \overline{1, n_p}$. Например, топология объекта на рис. 5.3,а описывается матрицей рёбер

$$R = \begin{bmatrix} 1 & 2 & 3 & 1 & 4 & 5 & 6 & 3 & 1 & 6 & 1 & 7 & 2 & 8 \\ 2 & 3 & 4 & 4 & 5 & 6 & 7 & 7 & 5 & 8 & 8 & 9 & 9 & 9 \end{bmatrix}^T. \quad (5.1b)$$

В список индексов R_i можно добавить **атрибуты ребра**: цвет, толщину, стиль (сплошной, штриховой, штрихпунктирный) и другие свойства линии. Совокупность матриц $\{P, R\}$ составляет **модель «вершины + рёбра»**.

Построение каркаса полиэдра производится в цикле перебора строк матрицы рёбер R_i , выбора из них индексов $v=r_{i1}$ и $\mu=r_{i2}$, выбора из матрицы P вершин полиэдра p_v, p_μ и вывода на экран проекций отрезков $p_v p_\mu$:

$$[v \mu] = R_i, \text{ line}(p_{v3}, p_{\mu3}) \quad \forall i = \overline{1, \text{size}(R)}.$$

Общим **недостатком** моделей, описывающих топологию полиэдра списком рёбер, является отсутствие прямой информации о порядке их соединения в каждой грани. Это затрудняет поиск невидимых рёбер, экранированных от наблюдателя непрозрачными гранями и рисуемых особым стилем (см. рис. 5.2, в, з) либо вообще не включаемых в изображение (см. рис. 5.2, д).

□ Прямое описание граней полиэдра содержится в списке $LP = \{P_1, \dots, P_n\}$, составляющем **модель «граней»**. Элемент списка P_i представляет собой полигон i -й грани $P_i = p_{i1} p_{i2} \dots p_{in_i} p_{i1}$, вершины которого перечислены в порядке их соединения рёбрами. Направления обхода вершин в полигонах задаются:

- **произвольными**, если они не влияют на результат визуализации полиэдра. Это имеет место, во-первых, при построении каркаса с прозрачными ячейками сплошным стилем рисования всех рёбер (см. рис. 5.2, а); во-вторых, при изображении непрозрачных полиэдров (см. рис. 5.2, д) алгоритмами, не использующими векторы нормалей к плоскостям граней, например, художника или z -буфера (см. п. 1.7.2); в-третьих, при **ручном** задании **внешних** нормалей для алгоритмов, требующих такого направления, например, Робертса;

- **положительными** (против часовой стрелки относительно внешнего наблюдателя) для обеспечения внешней ориентации всех векторов нормалей, вычисляемых по (1.144) как $N_i = \text{norm}(P_i)$. Такая ориентация нужна для определения видимости граней наблюдателем по правилу (1.181) и принятия решения о способах отображения видимых и невидимых элементов полиэдра.

В целях унификации **рекомендуется всегда задавать положительное направление обхода граней полиэдра независимо от способа и алгоритма его изображения**. Руководствуясь этим пожеланием, составим полигональную модель семигранника H , показанного на рис. 5.3, а:

$$LP = \{p_1 p_2 p_3 p_4 p_1, p_3 p_7 p_6 p_5 p_4 p_3, p_1 p_5 p_6 p_8 p_1, p_1 p_4 p_5 p_1, p_2 p_9 p_7 p_3 p_2, p_6 p_7 p_9 p_8 p_6, p_1 p_8 p_9 p_2 p_1\}. \quad (5.1\Gamma)$$

В структуру, содержащую список P_i , можно добавить **атрибуты грани**: цвет, толщину и стиль линии контура, цвет заливки, признак видимости, вектор нормали и другие статические и динамические параметры, необходимые для её реалистичного изображения.

Построение каркаса полиэдра с прозрачными гранями производится в цикле перебора полигонов P_i и выводе их контуров на экран желаемым стилем. При этом каждое ребро, входящее в полигоны двух смежных граней, рисуется на экране **дважды**, что может стать причиной **визуального** изменения его стиля, отличного от сплошного. Поясним этот неочевидный эффект на примере стыковки (для наглядности с небольшими зазорами) трёх штриховых контуров (рис. 5.4, а). Штрих-линия выводится путём чередования отрезков штриха и просвета, возможно, разных длин. Ребро, по которому стыкуются две грани с одинаковыми направлениями обхода, всегда проходится дважды в **разных** направ-

лениях. В результате может произойти полное наложение штрихов прямого хода на просветы обратного хода, из-за чего ребро будет выглядеть сплошным. Для устранения данного дефекта программисту придётся найти либо самому создать растровую функцию вывода произвольного штрихового отрезка, у которого штрихи всегда расположены по краям, чтобы создавалось одинаковое изображение отрезка независимо от направления его рисования на экране.

В рамках полигональной модели полиэдра легко изобразить его невидимые рёбра стилем, отличным от стиля вывода видимых рёбер (см. рис. 5.2, в, г). Достаточно для каждого полигона P_i вычислить нормаль $N_i = \text{norm}(P_i)$, определить по (1.181) его видимость и рисовать контур видимого полигона сплошным утолщённым стилем, а невидимого — сплошным тонким либо штриховым. Блокировка вывода контура невидимой грани позволяет создать реалистичное изображение каркаса с непрозрачными гранями (см. рис. 5.2, д).

Полигональная модель полиэдра унаследовала у модели «рёбра» **недостаток** множественности описания вершин, влекущий более тяжкие последствия:

- если некоторая вершина пронумерована первой в $m \geq 3$ -х смежных гранях, то она должна быть одинаково задана в $2m \geq 6$ позициях списка LP , что слишком избыточно потребляет память ЭВМ;
- неверное клонирование какой-либо вершины делает содержащую её грань неплюской (рис. 5.4, б), из-за чего она может стать даже невидимой;
- нестыковка вершин смежных граней в одной точке создаёт в этой части полиэдра «дыру», за которой может быть не видно ни одной задней грани, которые по Робертсу обнаруживаются как нелицевые и не рисуются (рис. 5.4, в).

□ Наиболее экономичной и удобной в эксплуатации является **модель «вершины + грани»** $\{P, G\}$, включающая список (матрицу) вершин полиэдра P и **группу списков граней** $G = \{G_1, G_2, \dots, G_{n_g}\}$ (см. п. 1.5.4). Элемент этой группы представляет собой список $G_i = \{g_{i1}, \dots, g_{in_i}, g_{i1}\}$, хранящий индексы вершин i -й грани в порядке их обхода по замкнутому контуру. Он может быть дополнен атрибутами грани, перечисленными в описании модели «грани». Вопрос выбора направления обхода обсуждён там же.

Заменив в (5.1г) обозначения вершин p_i их индексами i , получим следующие списки граней полиэдра H , показанных на рис. 5.3, б разложенными на плоскость нормалью к наблюдателю:

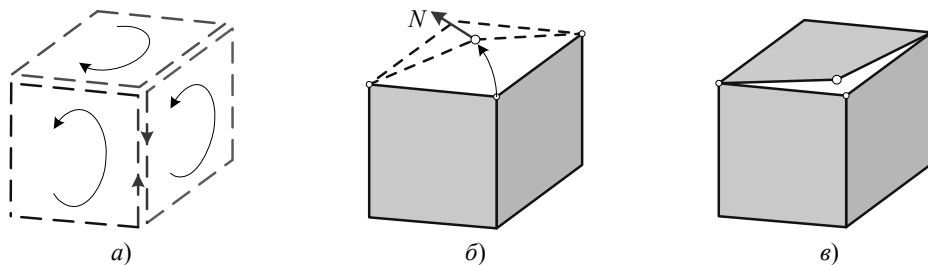


Рис. 5.4

$$G_1 = \{1, 2, 3, 4, 1\}, G_2 = \{3, 7, 6, 5, 4, 3\}, G_3 = \{1, 5, 6, 8, 1\}, G_4 = \{1, 4, 5, 1\}, \\ G_5 = \{2, 9, 7, 3, 2\}, G_6 = \{6, 7, 9, 8, 6\}, G_7 = \{1, 8, 9, 2, 1\}. \quad (5.1д)$$

Построение полиэдра производится в цикле $i = \overline{1, n_r}$ формирования полигонов граней $P_i = \text{side}(P, G, i)$, расчёта нормалей $N_i = \text{norm}(P_i)$ и принятия решения о выводе и стиле рисования каждой грани. Стоит обратить внимание на некоторые дополнительные аспекты создания моделей полиэдров:

- обход **невыпуклой** грани против часовой стрелки должен начинаться с такой вершины, чтобы угол между первыми тремя вершинами был **выпуклым**, т. е. загибался влево. Например, топологию грани, заштрихованной на рис. 5.5,а, можно описать списком $\{1, 2, 10, 9, 6, 5, 1\}$, но никак не $\{10, 9, 6, 5, 1, 2, 10\}$;
- всё сказанное о важности выбора направления нумерации вершин граней следует учитывать лишь при расчёте нормали к её плоскости по первым трём вершинам. Если же вектор N_i задаётся внешним вручную, то порядок обхода вершин в списке грани G_i может быть произволен;
- для описания полигона с отверстием введём **фиктивное ребро** $\overline{ij}$, связывающее внешний и внутренний контуры полигона между i -й и j -й вершинами (рис. 5.5,б). Отразим это добавлением в список грани символа Φ между индексами вершин, соединённых фиктивным ребром: $\{...i, \Phi, j, ..., j, \Phi, i, ...\}$ [1]. Этот символ означает, что **при сканировании списка ребро $\overline{ij}$ не должно изображаться**. Направление обхода полигона должно быть таким, чтобы он оставался всё время **слева**. Например, топологию передней грани на рис. 5.5,в можно определить списком $\{1, 2, 3, 4, \Phi, 12, 11, \Phi, 19, 18, 17, 19, \Phi, 11, 10, 9, 12, \Phi, 4, 1\}$.

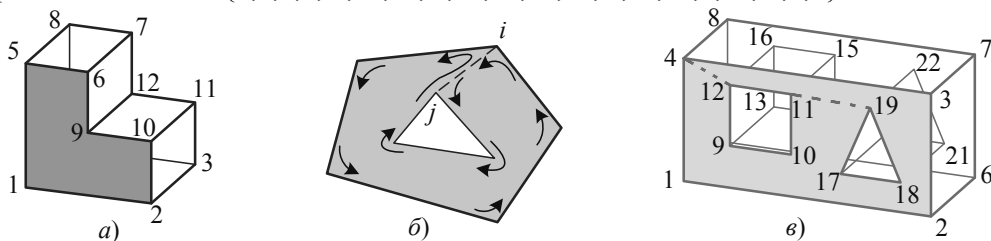


Рис. 5.5

Если для объектов с числом граней $n_r < 10$ составление моделей в силах выполнить вручную, то более многогранные модели должны строиться исключительно **алгоритмически**. Серьёзному программисту рекомендуется создать библиотеку моделей разных объектов и алгоритмов их формирования.

5.2.2. Преобразования моделей полиэдров

Возможно и избыточное кодирование топологии полиэдра одновременно матрицей рёбер R и группой списков граней G , позволяющее использовать в конкретной задаче наиболее подходящую модель. Однако, при необходимости всегда можно перейти от одной модели описания полиэдра к другой. В связи с этим рассмотрим задачи **взаимного преобразования моделей полиэдра**.

□ При составлении группы списков G по матрице R используется заложенная в ней информация о вершинах полиэдра, связанных рёбрами. Совсем не тривиальная, как может вначале казаться, операция определения числа n_r и составления списков $G_i \forall i=\overline{1, n_r}$ заключается в стыковке отрезков R_i совпадающими концами в замкнутые контуры с учётом следующих факторов:

- просмотр строк матрицы R должен выполняться многократно до тех пор, пока все рёбра не окажутся соединёнными в замкнутые полигоны граней, аналогично тому, как это делается в алгоритме *conts* (см. рис. 1.69);
- каждое ребро **замкнутого** полиэдра входит в состав **двух** граней, поэтому необходимы счётчики использования рёбер $c_i \in \{0, 1, 2\} \forall i=\overline{1, n_r}$. Вначале все они обнуляются, а при использовании i -го ребра счётчик c_i инкрементируется. По достижении значения $c_i=2$ ребро R_i блокируется. Двукратное использование всех рёбер завершает формирование списков граней полиэдра;
- начало списка новой грани $g=\{r_{i1}, r_{i2}\}$ формируется из обоих элементов очередного списка R_i , для которого $c_i < 2$;
- каждый последующий $n+1$ -й элемент списка грани $k \equiv r_{i,3-j} \in R_i$ выбирается

из **условий стыковки**

$$i, j: g_n=r_{ij}, c_i < 2, j \in \{1, 2\}; \quad (5.2)$$

- третий элемент k должен быть таким, чтобы расширенный список $\{g_1, g_2, k\}$ не совпадал ни с одной из троек подряд идущих чисел в списках уже найденных граней $G_\mu \forall \mu=\overline{1, m}$. Это является гарантией того, что новая грань G_{m+1} не будет повторять предыдущие грани $G_1 \div G_m$;

• каждый следующий, допущенный по условиям (5.2) элемент списка грани k должен выбираться из условия компланарности точки p_k с точками $p_{g_1} \div p_{g_n}$. По (1.112б) ранг матрицы M , сформированной из векторов $p_{g_i} - p_k \forall i=\overline{1, n}$, должен быть меньше трёх, что равносильно нулевому определителю 3×3 -матрицы $M^T M$. Отметим, что при расчёте ранга и определителя выполняются арифметические операции с вещественными координатами вершин, тогда как все остальные вычисления являются целочисленными;

- по условию $g_n=g_1$ замыкания грани $g=\{g_1, g_2, \dots, g_n\}$ сформированный список g добавляется в группу списков G , а значение n сбрасывается в ноль;

• если требуется положительное направление обхода всех граней, нужно скорректировать полученную группу списков G функцией $norm_test(P, G)$ (см. рис. 1.135), обеспечивающей **внешнюю** ориентацию нормалей всех граней.

С учётом всего сказанного на рис. 5.6 построен **алгоритм $RtoG(R, P)$ преобразования модели «вершины + рёбра» в модель «вершины + грани»**, собирающий группу списков граней G из строк матрицы рёбер R . Размер этой группы в соответствии с формулой Эйлера (1.141а) равен $size(R) - size(P) + 2$.

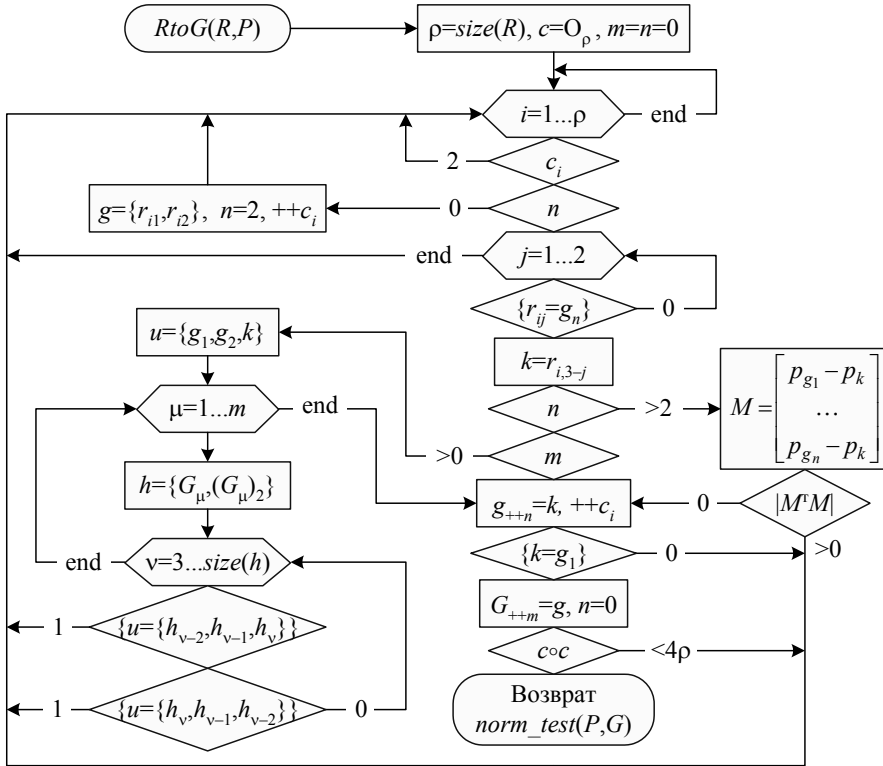


Рис. 5.6

Запустим процесс формирования списков граней полиэдра (см. рис. 5.3,а) с матрицами вершин P из (5.1б) и рёбер R из (5.1в). Проинициализируем счётчики $c_i=0 \forall i=\overline{1,14}$ и начнём просмотр строк R_i , для которых $c_i < 2$:

- элементы первой строки $R_1 = [1 \ 2]$ начинают список $g = \{1, 2\}$, $c_1 = 1$;
- первый элемент второй строки $R_2 = [2 \ 3]$ совпадает с концом списка g , поэтому $g = \{1, 2, 3\}$, $c_2 = 1$;
- аналогично выполняем ещё два сцепления с проверкой компланарности получаемой грани: $g = \{1, 2, 3, 4, 1\}$, $c_3 = 1$, $c_4 = 1$. Поскольку конец списка g совпал с его началом, то запоминаем список первой грани $G_1 = \{1, 2, 3, 4, 1\}$;
- из элементов пятой строки $R_5 = [4 \ 5]$ начинаем новый список: $g = \{4, 5\}$, $c_5 = 1$. Далее с ним компланарно сцепляются строки R_6 , R_7 и R_8 : $g = \{4, 5, 6, 7, 3\}$, $c_6 = 1$, $c_7 = 1$, $c_8 = 1$. В оставшихся строках $R_9 \div R_{14}$ нет элемента, равного $g_5 = 3$;
- на втором проходе находим строку R_2 с элементом $r_{22} = 3$, но другой её элемент $r_{21} = 2$ не годится для продолжения грани, т. к. точка p_2 не лежит в плоскости полилинии $p_4 p_5 p_6 p_7 p_3$. Следующая строка $R_3 = [3 \ 4]$ удовлетворяет условию компланарности, поэтому получаем $g = \{4, 5, 6, 7, 3, 4\}$ и $c_3 = 2$. Больше строка R_3 в проверках участвовать не будет. Поскольку конец списка g совпал с его началом, то формирование второй грани $G_2 = \{4, 5, 6, 7, 3, 4\}$ закончено;

• далее читатель может самостоятельно получить остальные пять граней $G_3 = \{1, 4, 5, 1\}$, $G_4 = \{6, 8, 1, 5, 6\}$, $G_5 = \{6, 7, 9, 8, 6\}$, $G_6 = \{1, 8, 9, 2, 1\}$ и $G_7 = \{2, 3, 7, 9, 2\}$.

Сравнив полученные списки с (5.1д), убеждаемся в их идентичности с точностью до нумерации и последовательности составляющих их индексов. Последняя операция, выполняемая функцией *norm_test*, реверсирует направления обхода граней $G_2 = \{4, 3, 7, 6, 5, 4\}$ и $G_7 = \{2, 9, 7, 3, 2\}$, делая их положительными.

□ Формирование матрицы рёбер R по номерам вершин, хранящимся в списках группы G , существенно проще предыдущей задачи. Оно заключается в переборе каждого списка G_i , описывающего n_i -угольную грань, анализе в нём всех смежных пар индексов $\{g_{j-1}, g_j\} \forall j=2, n_i+1$ и последовательном накоплении в матрице R строк $[g_{j-1} \ g_j]$, не совпадающих с уже накопленными ранее строками как в прямом, так и обратном $[g_j \ g_{j-1}]$ направлениях.

На рис. 5.7 приведён алгоритм $GtoR(G)$ преобразования модели «вершины + грани» в модель «вершины + рёбра», т. е. получения матрицы рёбер R из группы списков граней G . Алгоритм не использует матрицу вершин P и является полностью целочисленным. Размер списка рёбер, возвращаемого алгоритмом, согласно формуле Эйлера (1.141а) равен $size(G) + size(P) - 2$.

Составим для примера матрицу рёбер по спискам граней (5.1д):

• список первой грани $G_1 = \{1, 2, 3, 4, 1\}$ даёт первые четыре ребра $R_1 = [1 \ 2]$, $R_2 = [2 \ 3]$, $R_3 = [3 \ 4]$ и $R_4 = [4 \ 1]$;

• второй список $G_2 = \{3, 7, 6, 5, 4, 3\}$ даёт ещё четыре ребра $R_5 = [3 \ 7]$, $R_6 = [7 \ 6]$, $R_7 = [6 \ 5]$ и $R_8 = [5 \ 4]$, тогда как отрезок $u = [4 \ 3]$ совпадает с R_3 и не нумеруется;

• третий список $G_3 = \{1, 5, 6, 8, 1\}$ добавляет ещё три новых ребра $R_9 = [1 \ 5]$, $R_{10} = [6 \ 8]$ и $R_{11} = [8 \ 1]$;

• оставшиеся списки $G_4 \div G_7$ добавляют последнюю тройку новых рёбер $R_{12} = [2 \ 9]$, $R_{13} = [9 \ 7]$ и $R_{14} = [9 \ 8]$.

Налицо совпадение индексов полученных рёбер со строками матрицы (5.1в) с точностью до перестановки элементов, что доказывает правильность работы алгоритма.

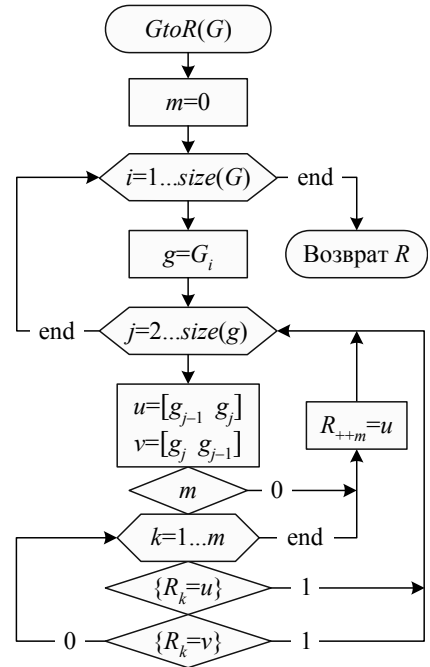


Рис. 5.7

□ Если преобразование модели «вершины + грани» $H = \{P, G\}$ в полигональную модель $LP = \{P_1, \dots, P_{n_r}\}$ просто выполняется с помощью функции *side* как

$$P_i = side(P, G, i) \quad \forall i = \overline{1, n_r},$$

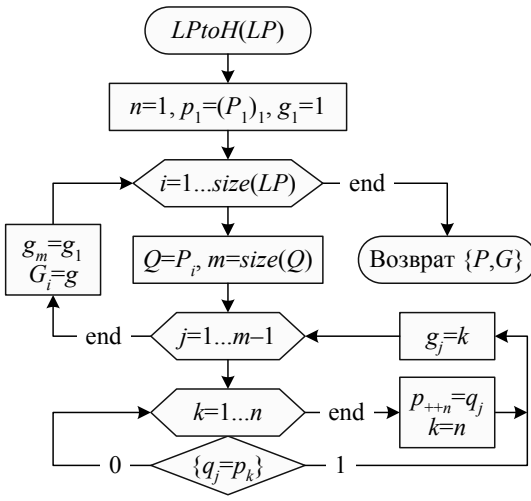


Рис. 5.8

то обратный переход $LP \rightarrow \{P, G\}$ делает алгоритм $LPtoH(LP)$ преобразования полигональной модели полиэдра в модель «вершины + грани», блок-схема которого построена на рис. 5.8. В нём производится перебор граней $P_i \equiv Q = q_1 q_2 \dots q_{n_i} q_1 \quad \forall i = \overline{1, n_t}$ и добавление отсутствующих точек $q_j \notin P$ при $j = \overline{1, n_i}$ в координатную матрицу P с одновременным формированием списков граней G_i из индексов вершин, составляющих полигоны P_i .

5.3. ГРАНИЧНЫЕ МОДЕЛИ

Граничная модель представляет объёмное тело системой элементов, описывающих его границы (поверхности, края поверхностей и указатели пересечения поверхностей), и топологической информации о соединении элементов друг с другом.

Структура геометрических данных граничной модели включает:

- векторы неявного описания плоскостей F_1 и матрицы квадратичных поверхностей F_2 , с помощью которых наиболее просто производится локализация точки p относительно этих поверхностей по знаку неявных функций $f(p) = \tilde{p} \circ F_1$ и $f(p) = \tilde{p} F_2 \tilde{p}^T$. Аффинное преобразование с матрицей C даёт векторы плоскостей (3.15а) и матрицы квадратичных поверхностей (3.16):

$$F'_1 = F_1 (C^{-1})^T, \quad F'_2 = C^{-1} F_2 (C^{-1})^T; \quad (5.3)$$

- элементы неявного $f(p)=0$ или параметрического $p(t, \tau)$ описаний используемых поверхностей высших порядков и трансцендентных поверхностей;
- интервалы изменения параметров t, τ и данные об их квантовании, позволяющие построить каркасную сетку по узловым точкам $p(t_i, \tau_j)$;
- параметрические функции $l(t)$ и интервалы изменения их параметров, задающие линии пересечения граничных поверхностей;
- координатные матрицы P и топологические массивы R и G , описывающие взаимосвязь граничных элементов полиэдров друг с другом.

Составим, например, граничное описание замкнутого с обоих торцов цилиндра единичной высоты с основаниями единичного радиуса (рис. 5.9). Оно включает в себя следующие модели:

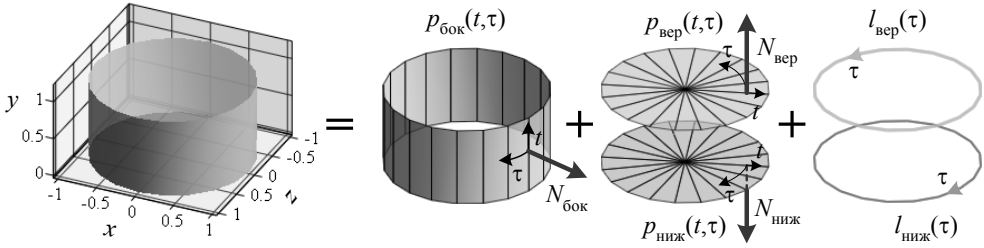


Рис. 5.9

• боковой поверхности цилиндра единичного радиуса и с внешней нормалью в составе матрицы НФ $F_{\text{бок}} = \text{diag}[1 \ 0 \ 1 \ -1]$ и векторной ПФ

$$p_{\text{бок}}(t, \tau) = [\cos(\tau) \ t \ \sin(\tau)], \quad (5.4a)$$

полученной коррекцией модели (1.169) с параметрами $a=b=c=1$, у которой нормаль направлена внутрь цилиндра;

• нижней торцевой плоскости с внешней нормалью $N = -y^0$ в составе вектора неявной формы и ПФ круга единичного радиуса в плоскости $y=0$:

$$F_{\text{ниж}} = [0 \ -1 \ 0 \ 0], \quad p_{\text{ниж}}(t, \tau) = [t \cos(\tau) \ 0 \ t \sin(\tau)]; \quad (5.4б)$$

• верхней торцевой плоскости с внешней нормалью $N = y^0$ в составе вектора неявной формы и ПФ круга единичного радиуса в плоскости $y=1$:

$$F_{\text{верх}} = [0 \ 1 \ 0 \ -1], \quad p_{\text{верх}}(t, \tau) = [t \cos(\tau) \ 1 \ -t \sin(\tau)]; \quad (5.4в)$$

• векторных параметрических моделей линий пересечения боковой поверхности с торцевыми плоскостями:

$$\begin{aligned} l_{\text{ниж}}(\tau) &= p_{\text{бок}}(0, \tau) = p_{\text{ниж}}(1, \tau) = [\cos(\tau) \ 0 \ \sin(\tau)], \\ l_{\text{верх}}(\tau) &= p_{\text{бок}}(1, -\tau) = p_{\text{верх}}(1, \tau) = [\cos(\tau) \ 1 \ -\sin(\tau)]; \end{aligned} \quad (5.4г)$$

• интервалы изменения параметров $t \in [0, 1]$ и $\tau \in [0, 2\pi]$ в функциях (5.4а–г), частоты квантования $n=1$, $m=20$ и узловые значения параметров

$$t_i = i/n \quad \forall i = \overline{0, n}, \quad \tau_j = 2\pi j/m \quad \forall j = \overline{0, m}. \quad (5.4д)$$

Кроме геометрической информации об объекте его полное граничное описание может включать в себя физические параметры поверхностей (материал, цвет, коэффициент прозрачности, диаграммы направленности диффузного и зеркального отражений и т. п.), позволяющие сделать их изображение на экране реалистичным.

При моделировании объекта границами структура данных строится одновременно с процессом создания модели. Благодаря явному присутствию в этой структуре моделей поверхностей и линий их пересечения легко решается задача каркасного изображения объекта. В отличие от каркасной модели с плоскими гранями граничное описание позволяет изображать нелинейную поверхность каркасом с криволинейными ячейками.

Основной проблемой граничного описания является выбор возможных форм задания поверхностей. Так как замкнутые объекты в большинстве случаев непрозрачны, то для их правильной визуализации требуется, как уже не раз отмечалось, **внешняя ориентация нормалей** во всех граничных точках. Автоматическое выполнение этого требования основано на **методе пробной точки**, описанном в п. 1.5.6 применительно к анализу граней полиэдра (см. рис. 1.134). Коррекция моделей гладкой поверхности методом пробной точки осуществляется следующим образом:

- в **неявной** форме описания поверхности уравнением $\pm f(p)=0$ конкретная функция $+f(p)$ или $-f(p)$ выбирается по правилу локализации **внешней** точки $q_{\text{вне}}$ в той части пространства, где $f(q_{\text{вне}})>0$, либо **внутренней** точки $q_{\text{вну}}$ там, где $f(q_{\text{вну}})<0$:

$$f^+(p)=\text{sgn}(f(q_{\text{вне}}))f(p)=-\text{sgn}(f(q_{\text{вну}}))f(p). \quad (5.5a)$$

В частности, матрицы поверхностей первого и второго порядков корректируются так:

$$\begin{cases} F_1^+ = \text{sgn}(\tilde{q}_{\text{вне}} \circ F_1) F_1 = -\text{sgn}(\tilde{q}_{\text{вну}} \circ F_1) F_1; \\ F_2^+ = \text{sgn}(\tilde{q}_{\text{вне}} F_2 \tilde{q}_{\text{вне}}^T) F_2 = -\text{sgn}(\tilde{q}_{\text{вну}} F_2 \tilde{q}_{\text{вну}}^T) F_2. \end{cases} \quad (5.5б)$$

Это означает инверсию знаков всех элементов матрицы F , если в заведомо внешней точке получено значение $f(q_{\text{вне}})<0$, а в заведомо внутренней точке — значение $f(q_{\text{вну}})>0$;

- в **параметрической** форме $p(t, \tau)$ ориентация нормали $N=V \times W$ определяется направляющими векторами $V=p'_t$ и $W=p'_\tau$. Обмен параметров $t \leftrightarrow \tau$ в функции $p(t, \tau)$ меняет направление нормали на противоположное. То же происходит при расчёте нормали по формуле. Из двух возможных параметрических форм выбирается та, в которой угол между нормалью в граничной точке $p_{\text{гр}}$, и вектором $q_{\text{п}} - p_{\text{гр}}$, направленным из неё в пробную точку $q_{\text{п}}$, **острый** при выборе **внешней** точки $q_{\text{п}} \equiv q_{\text{вне}}$ и **тупой** при выборе **внутренней** точки $q_{\text{п}} \equiv q_{\text{вну}}$:

$$\{(q_{\text{вне}} - p_{\text{гр}}) \circ N > 0\} \vee \{(q_{\text{вну}} - p_{\text{гр}}) \circ N < 0\}, \quad (5.6)$$

где направляющие векторы $V=p'_t|_{p_{\text{гр}}}$, $W=p'_\tau|_{p_{\text{гр}}}$ и нормаль $N=V \times W$ вычислены в граничной точке исходной поверхности $p(t, \tau)$.

Условие (5.6) выполняется в **любой** внутренней точке **выпуклого** объекта и **близкой** к $p_{\text{гр}}$ внутренней точке **невыпуклого** объекта. Если оно не выполняется, то нужно либо заменить исходную модель $p(t, \tau)$ на инверсную $p(\tau, t)$, либо вычислять нормаль по формуле $N=W \times V$.

Требование близости пробной $q_{\text{п}}$ и граничной $p_{\text{гр}}$ точек является принципиальным. Построим, например, нормаль к замкнутой поверхности эллиптического тора с моделью (4.28)

$$\begin{cases} p(t, \tau) = [(a \sin(\tau) + r) \cos(t) \ b \cos(\tau) - (a \sin(\tau) + r) \sin(t)] \\ \forall t, \tau \in [0, 2\pi], \ r > a, \ r > b \end{cases} \quad (5.7)$$

в точке $p_{\text{гр}} = [0 \ 0 \ -(r-a)]$, соответствующей параметрам $t_{\text{гр}} = \pi/2$ и $\tau_{\text{гр}} = 3\pi/2$:

$$V = [-(r + a \sin(\tau)) \sin(t) \ 0 \ -(r + a \sin(\tau)) \cos(t)]_{t_{\text{гр}}, \tau_{\text{гр}}} = [-(r-a) \ 0 \ 0],$$

$$W = [a \cos(t) \cos(\tau) - b \sin(\tau) \ -a \sin(t) \cos(\tau)]_{t_{\text{гр}}, \tau_{\text{гр}}} = [0 \ b \ 0],$$

$$N = \begin{vmatrix} x^0 & y^0 & z^0 \\ -(r-a) & 0 & 0 \\ 0 & b & 0 \end{vmatrix} = [0 \ 0 \ -b(r-a)].$$

Она направлена внутрь тора. Выбор **близкой** к $p_{\text{гр}}$ **внутренней** точки $q_{\text{вну}} = [0 \ 0 \ -r]$ подтверждает эту направленность: $(q_{\text{вну}} - p_{\text{гр}}) \circ N = ab(r-a) > 0$.

Ввиду невыполнения условия (5.6) корректируем параметрическую модель (5.7) путём обмена параметров $t \leftrightarrow \tau$:

$$p(t, \tau) = [(a \sin(t) + r) \cos(\tau) \ b \cos(t) - (a \sin(t) + r) \sin(\tau)] \forall t, \tau \in [0, 2\pi]. \quad (5.8)$$

Теперь нормаль $N = [0 \ 0 \ b(r-a)]$ является **внешней** к поверхности тора (рис. 5.10):

$$(q_{\text{вну}} - p_{\text{гр}}) \circ N = -ab(r-a) < 0.$$

Однако, если выбрать **не близкую** к $p_{\text{гр}}$, но тоже внутреннюю точку $q'_{\text{вну}} = [0 \ 0 \ r]$, то тест (5.6) даёт ложный ответ:

$$(q'_{\text{вну}} - p_{\text{гр}}) \circ N = (2r+a)(r-a)b > 0.$$

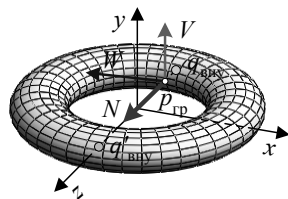


Рис. 5.10

Многие объекты не являются замкнутыми, из-за чего часть их внутренней поверхности видна одновременно с внешней (см. рис. 1.142, 3.77, 4.5, 4.10, 4.49, 4.114 и др.). Для визуализации внутренних поверхностей **они должны быть описаны как внешние**. Другими словами, каждая тонкая оболочка, претендующая на двухстороннюю видимость, рассматривается как две граничные поверхности $p(t, \tau)$ и $p(\tau, t)$ с противоположно ориентированными внешними нормальными. Если какая-либо внутренняя поверхность не видна ни в каких ракурсах, то нет необходимости включать её описание в граничную модель.

5.4. СЛОЖНЫЕ МОДЕЛИ

Сплошная модель включает в объект как граничные, так и внутренние точки. Это позволяет производить **декомпозицию** сложного объекта на простые составляющие — **примитивы** (кубы, сферы, торы, пирамиды, призмы и т. д.) — и строить разнообразные **композиции** из простых примитивов.

Процесс композиции заключается в выполнении аффинных преобразований примитивов для создания нужных форм в нужном месте пространства и булевых операций с этими формами для сборки единого объекта. Это напоминает строительный конструктор, позволяющий возводить из типовых блоков разно-

образные сооружения. За счёт небольшого числа примитивов и всего четырёх булевых операций конструктивный метод описания сложных объектов позволяет уменьшить объём памяти и трудоёмкость процесса построения сцены.

Обратная операция **декомпозиции** состоит в изучении строения сложного объекта, выявлении его структурной организации (минимального набора примитивов и способов их преобразования и соединения) и параметрического описания (матриц преобразований примитивов). Благодаря малому числу примитивов и матриц их преобразований математическое описание сложного объекта становится компактным и удобным для моделирования.

5.4.1. Воксельные модели

Моделирование сплошных объектов с помощью **вокселей** (см. разд. 4.7) позволяет создавать тела с нерегулярной структурой, полостями, разрывами и негладкими границами, что обычно затруднительно при полигональном представлении (рис. 5.11,а). Бинарная структура данных, описывающая воксельный объект, представляет собой трёхмерную $n \times m \times l$ -матрицу битов состояний ячеек (0 — пустая, 1 — занятая) и, например, при $n=m=l=512$ её объём превышает 16 Мбайт. Зная размеры объектной области $\{d_x, d_y, d_z\}$, нетрудно получить вектор координат центра (i, k, j) -го вокселя в объектной системе координат в виде

$$[(i-0.5)d_x/n \ (k-0.5)d_y/m \ (j-0.5)d_z/l] \ \forall i \in [1, n], k \in [1, m], j \in [1, l].$$

Нулевое состояние ячейки воксельного параллелепипеда можно интерпретировать как её полную прозрачность. При построении разрезов или сечений сплошных тел можно либо удалить группу отрезанных вокселей из их общего списка, либо установить их полную прозрачность с помощью альфа-канала вокселя $\alpha=0$ (об управлении прозрачностью пиксела см. рис. 4.17б), после чего будут видны только непрозрачные элементы объёма (рис. 5.11,б). Прозрачные воксели даже не нужно исключать из полного описания объекта, чтобы можно было быстро восстановить его прежний вид.

Более совершенна воксельная модель, несущая информацию не только о координатах, но и о цвете каждого вокселя и его альфа-канале $\alpha \in [0, 1]$. Цветовая дифференциация вокселей позволяет наглядно отобразить в пространстве различные функциональные системы организма. Дальнейшее развитие модели связано с сегментацией элементов, позволяющей включать один и тот же воксел в

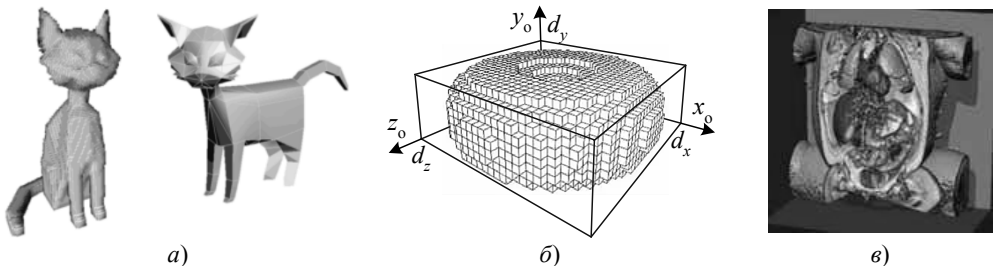


Рис. 5.11

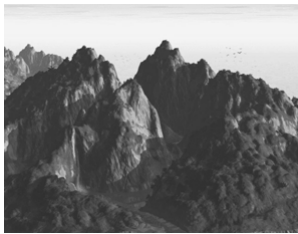
разные системы объекта и по-разному отображать его в зависимости от выбора пользователя (рис. 5.11,в). Наконец, в «интеллектуальных» объёмных моделях атрибуты воксела включают расширенный набор свойств, таких как имена на различных языках, указатели на дополнительную информацию и т. п.

Воксельная технология получает всё более широкое распространение в сложнейших современных информационных комплексах объёмного конструирования и визуализации:

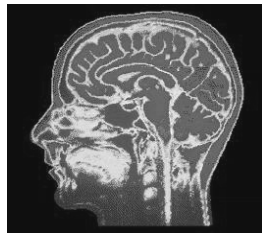
- для генерирования виртуальных ландшафтов (рис. 5.12,а) в различных компьютерных приложениях: от игровых программ до авиасимуляторов и тренажёрных комплексов, позволяющих пилотам приобрести правильные и устойчивые навыки управления летательными аппаратами в реальном времени и природных условиях, близких к условиям театра возможных боевых действий;

- в системах медицинской интроскопии (технологии получения изображений внутренних органов человека) с помощью томографических сканеров получают снимки срезов тела пациента, характеризующих особенности его анатомии и физиологии (рис. 5.12,б). Для некоторых клинических задач, подобных хирургическому планированию, необходимо иметь трёхмерное изображение организма в разных ракурсах, чтобы знать точное положение больного органа, всесторонне осмотреть пути доступа к нему и осуществить минимально травмирующее воздействие. Другим потенциальным применением, уменьшающим риск врачебного вмешательства, является планирование радиотерапии: необходимо сфокусировать энергию так, чтобы облучался только больной орган и не затрагивались здоровые. Объёмная визуализация органа-мишени и моделирование воздействия доз радиации позволяют проводить реалистичные репетиции лечебной процедуры и выбор оптимального решения;

- при создании интерактивных мультимедийных энциклопедий, таких, например, как 3d-атлас анатомии человека (рис. 5.12,в), обучающий будущих врачей выполнять на экране компьютера разнообразные оперативные действия (рассечение, удаление и соединение органов) без единой капли живой крови;



а)



б)



в)

Рис. 5.12

- в автоматизированных системах естественнонаучных исследований воксельные модели помогают химикам видеть строение сложнейших молекул, астрономам — динамику развития звёзд и галактик, физикам — моделировать процессы микромира и электроники, метеорологам — исследовать атмосферные явления и природные катастрофы, конструкторам-механикам и технологам — проектировать новые изделия, процессы их обработки и т. д.;



Рис. 5.13

- уже появились опытные и недалёк час, когда они станут широко распространёнными в повседневной жизни, экземпляры 3d-мониторов, создающих полноценные объёмные изображения, а не проекции объёмных тел на плоский экран. Разрешающая способность устройства, показанного на рис. 5.13, составляет почти 100 млн. вокселей разной формы: 198 «ломтиков», содержащих по 768×768 вокселей. Угол обзора по горизонтали составляет 360° , по вертикали — 270° . В дисплее применяется набор оптических компонент (параболических зеркал и расщепителей луча), с помощью

которых свет фокусируется в пространстве, образуя объёмное распределение цвета. Изображения внутри объёмных дисплеев плавают, как рыбы в аквариуме. На них можно посмотреть с любой стороны, а в комплекте со специальными тактильными перчатками, создающими давление на подушечки пальцев, потрогать руками. Спектр применения таких чудо-устройств по-голливудски неисчерпаем: от виртуальных 3d-музеев мировых археологических раритетов до объёмного Интернета с 3d-поисковыми системами и WEB-магазинами, от анализа и синтеза молекулярных структур новых лекарств до мониторинга воздушной обстановки вокруг аэропорта, от простого детского конструктора до мастерской скульптора, от компьютерных игр до полного погружения в виртуальное пространство размером с комнату без всяких специальных очков и шлемов!

Сжатие объёмных данных

Воксельный метод моделирования прост, но требует большого объёма памяти для описания крупных объектов с разрешением, достаточным для отображения мелких деталей. Без применения специальных методов оптимизации рост разрешения в N раз увеличивает объём памяти, необходимой для хранения одного однобитного атрибута, в N^3 раз. В сравнении с воксельной поверхностью, имеющей один тонкий слой «вещества» (см. рис. 4.173), объёмный объект состоит из множества воксельных слоев и, что досадно, при достаточно плотной упаковке большинство внутренних ячеек практически никогда не будет видно, занимая, тем не менее, уйму машинной памяти.

Рассмотрим некоторые методы сокращения машинных ресурсов, необходимых для описания воксельных объектов, и организацию доступа к отдельному вокселу из получаемых сжатых структур данных.

□ Исключение из структуры, описывающей воксельный объект, внутренних ячеек выполняется одним из самых простых **методом группового кодирования RLE (Run-Length Encoding)**, широко применяемым для архивации файлов и изображений [5]. Например, для сжатого описания в объектной системе координат x_0, y_0, z_0 некоторого воксельного ландшафта, построенного из **неразрывных** вертикальных «столбиков» (рис. 5.14,а), в памяти организуется **карта высот** размерами $n \times l$. В каждой ячейке матрицы плана Y достаточно хранить лишь ми-

нимальную $k_{ij}^{\min}$ и максимальную $k_{ij}^{\max}$ ординаты столбика вокселей, расположенных над (i, j) -й клеткой (рис. 5.14, б). Это равносильно затратам памяти всего на два слоя — нижний и верхний — плюс на палитру (зависимость цвета вокселя от высоты), например, радужную или топографическую (см. п. 4.2.2). Если же нижний слой никогда не бывает виден либо все его воксели расположены в одной горизонтальной плоскости, то достаточно хранить лишь ординаты верхних вокселей k_{ij} . С ростом разрешения сцены в N раз объём памяти, необходимой для кодирования объекта, увеличивается в $N^2 < N^3$ раз.

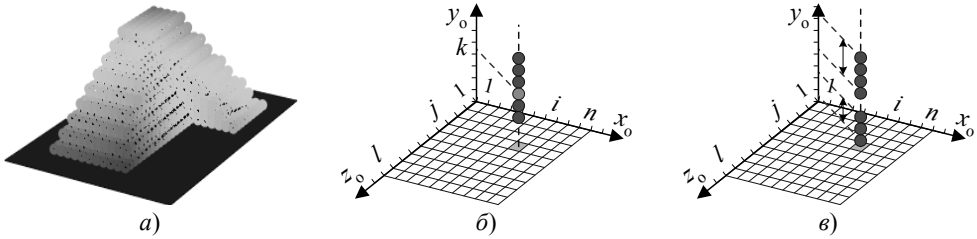


Рис. 5.14

Выборка (i, j, k) -го вокселя из матрицы Y заключается в чтении элемента $Y_{ij} = \{k_{ij}^{\min}, k_{ij}^{\max}\}$ и проверке условия $k_{ij}^{\min} \leq k \leq k_{ij}^{\max}$. Если оно выполняется, то воксел принадлежит объекту и добавляется в список визуализации.

Таким образом, метод группового кодирования экономит машинную память примерно в $N/2$ раз и обладает высокой скоростью выборки вокселей и вывода их на экран, сокращая число обращений к массиву данных. Если в воксельном объекте есть пустоты или отверстия, то в местах разрыва столбиков воксели можно задать прозрачными либо хранить по (i, j) -м координатам габаритные высоты нескольких столбиков (рис. 5.14, в).

□ Более эффективное уменьшение избыточности объёмных данных достигается использованием **пирамидально-ячеечной структуры** объекта, начинающей декомпозицию его параллелепипедной оболочки на восемь ячеек-октантов (рис. 5.15, а) и рекурсивно разбивающей частично заполненные ячейки на восемь подобных подъячеек вдвое меньшего размера до тех пор, пока не будут достигнуты полное заполнение объёма либо предельное пространственное разрешение. При этом обычно самые мелкие ячейки группируются на границе объекта, а его внутренность занята небольшим числом крупных ячеек.

Подобное структурирование объёмных тел называется **методом октодерев** аналогично тому, как разбиение плоского прямоугольного изображения при сжатии выполняется методом квадродерева. Каждая вершина октантного дерева имеет бинарный код занятости ячейки (1 — ячейка занята целиком) и восемь ветвей — указателей на ячейки-потомки. Терминальная вершина, у которой ветви отсутствуют (все указатели пустые), является листом, а если её код нулевой, то соответствующая ячейка пуста.

На рис. 5.15, б построено октодерево декомпозиции объекта, изображённого на рис. 5.15, в. Воксели наименьшего размера достигаются за три этапа раз-

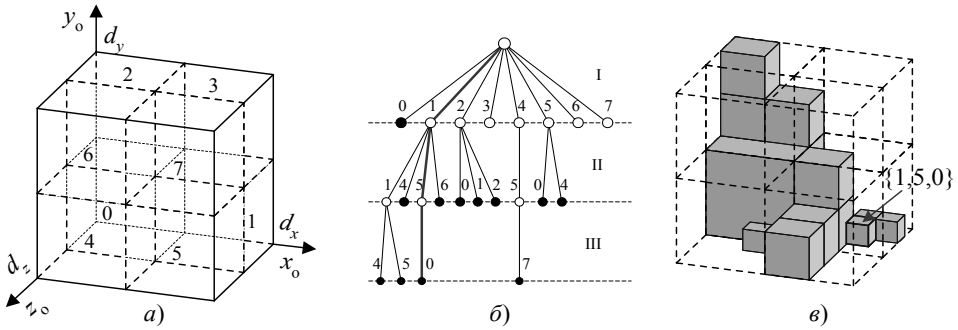


Рис. 5.15

биения. Чёрные кружки означают целиком занятые ячейки, а белые — частично занятые либо свободные. Для большей наглядности, начиная со второго уровня разбиения, построены только ветви, ведущие к непустым ячейкам. Общее количество листьев октодерева равно двенадцати (требуется хранить 12 трёхбайтовых чисел, т. е. 288 бит), тогда как для несжатого хранения топологии этого объекта нужен массив в $8^3=512$ бит, а сжатие методом вертикального группового кодирования потребует массив объёмом $2 \cdot 8^2=128$ байт или 1024 бит.

Платой за существенную экономию памяти, достигаемую при использовании октантного дерева, являются затраты машинного времени на его составление и декодирование, а также усложнение визуализации объекта, состоящего из разнокалиберных вокселей.

Кодирование занятой ячейки заключается в составлении k -звенной цепочки ветвей $\{n_1, n_2, \dots, n_k\}$, ведущих к ней от корня дерева. Проход по ветви i -го уровня означает двойное уменьшение размеров параллелепипеда и перенос его вдоль осей объектной системы координат в соответствии с номером ветви n_i . Выборка вокселя k -го уровня дробления осуществляется умножением координат вершин исходного параллелепипеда с размерами $\{d_x, d_y, d_z\}$ на матрицу аффинного преобразования

$$C = \tilde{M} (2^{-k}, 2^{-k}, 2^{-k}) T \left(\sum_{i=1}^k 2^{-i} \left[\{n_i \in \{1,3,5,7\}\} d_x \quad \{n_i \in \{2,3,6,7\}\} d_y \quad \{n_i \in \{n_i \geq 4\}\} d_z \right] \right).$$

Например, воксел, отмеченный на рис. 5.15, в, кодируется трёхзвенной цепочкой $\{1,5,0\}$, означающей, что на первом уровне разбиения он находится в 1-м октанте, на втором уровне — в 5-м, а на третьем уровне — в 0-м и формируется уменьшением исходного параллелепипеда в $2^3=8$ раз и переносом полученного полиэдра на вектор $[3d_x/4 \ 0 \ d_z/4]$.

Визуализация воксельных объектов

Рассмотрим теперь *особенности визуализации воксельных объектов*. Дело в том, что в отличие от «обычных» полигональных или точечных поверхностей, выводимых растровыми методами попиксельно, но не обязательно пообъектно (см. п. 1.7.2, разд. 4.2), изображение вокселя строится по выбранному

шаблону (куб или шар) **целиком** в порядке приближения к наблюдателю, аналогично методу художника. Это требует предварительной сортировки вокселей по глубине сцены (см. рис. 1.181) по (1.182а) для ближнего и по (1.182б) для дальнего наблюдателя. Ввиду огромного числа вокселей категорически приветствуется применение подходящих методов ускорения процесса перебора.

У каждого шестигранного вокселя-кубика изображаются не более трёх лицевых граней (см. рис. 3.53, 3.55, 3.60), видимых наблюдателем с вектором направления H и отбираемых по правилу $N_i \circ H > 0$. Дополнительный трудоёмкий анализ смежности позволяет исключить из визуализации заведомо невидимые лицевые грани, касающиеся нелицевых граней соседних кубиков.

Использование сферических вокселей более просто в визуализации (см. рис. 4.176, 4.177), но сложнее в достижении плотного заполнения пространства объекта, т. к. приходится решать сложную геометрическую задачу размещения сфер в 3d-области с криволинейными границами. Но уж если имеется множество плотно упакованных шариков радиуса r_b с центрами в точках $p_i = [x_i \ y_i \ z_i]$, то процедура их вывода на экран может выглядеть следующим образом:

- по (1.182) вычисляются удалённости d_i точек p_i от наблюдателя, а массив вокселей P сортируется по убыванию элементов вектора d : $P = \underset{\downarrow d}{sort}(P)$;
- по (4.179) находятся экранные проекции всех вокселей $\tilde{P}_3 \sim \tilde{P}C_{пз}$;
- в порядке от $d_{\max}$ до $d_{\min}$ на экран выводятся проекции вокселей в виде кругов с центрами p_{i3} и найденными по (4.181) радиусами $r_{i3} = r_b |p_{i3} - S| / |p_i - S|$.

Выбор метода визуализации вокселя на экране осуществляется из шаблонов, предложенных на рис. 4.174, и соответствующих алгоритмов (4.182), (4.183) с учётом прозрачности вокселей. Правильность экранирования вокселей друг другом гарантируется их отсортированностью по глубине, выполненной в начале визуализации. Дело в том, что результат наложения по (4.180а) двух пикселей с цветами и альфа-каналами $\{c_1, \alpha_1\}$, $\{c_2, \alpha_2\}$ на цвет фона c_Φ зависит от очерёдности пикселей в списке визуализации:

- если сначала выводится первый (дальний) полупрозрачный пиксел, а после него второй (ближний), то результирующий цвет пикселя равен

$$c_{12} = \alpha_2 c_2 + (1 - \alpha_2)(\alpha_1 c_1 + (1 - \alpha_1)c_\Phi);$$

- если вначале выводится ближний пиксел, а за ним дальний, то получается совсем другой цвет пикселя

$$c_{21} = \alpha_1 c_1 + (1 - \alpha_1)(\alpha_2 c_2 + (1 - \alpha_2)c_\Phi) \neq c_{12}.$$

В заключение покажем возможности воксельного метода на примерах моделирования таких сложных и непредсказуемых по форме и цвету объектов, как облака (рис. 5.16). Алгоритмы и программа динамической симуляции облаков разработаны магистрантом Нижегородского государственного технического университета Максимом Захаровым под руководством автора книги.

Пространство симуляции облака в форме эллипсоида разбивается на воксели, состояние которых описывают три битовые переменные:

- влажность h — отражает наличие в вокселе водяного пара;
- облачность c — отражает наличие в вокселе облака;
- фактор активации a — фиксирует превращение пара в воду.

Симуляция облака заключается в применении на каждом шаге времени t_s логических формул изменения состояний вокселей:

$$h_{ijk}(t_{s+1}) = h_{ijk}(t_s) \wedge \overline{a_{ijk}(t_s)}, \quad c_{ijk}(t_{s+1}) = c_{ijk}(t_s) \vee a_{ijk}(t_s),$$

$$a_{ijk}(t_{s+1}) = \overline{a_{ijk}(t_s)} \wedge h_{ijk}(t_s) \wedge f_{ijk}(t_s),$$

где i, j, k — координаты вокселя, а $f_{ijk}(t_s)$ — булева функция, значение которой зависит от значений параметров a соседних вокселей. Манипуляция с этой функцией позволяет задавать направление роста облаков. Например, для имитации роста облака вверх и по горизонтали может использоваться функция

$$f_{ijk}(t) = a_{i+1,j,k}(t) \vee a_{i,j+1,k}(t) \vee a_{i,j,k+1}(t) \vee a_{i-1,j,k}(t) \vee a_{i,j-1,k}(t) \vee a_{i,j,k-1}(t) \vee a_{i-2,j,k}(t) \vee a_{i+2,j,k}(t) \vee a_{i,j-2,k}(t) \vee a_{i,j,k-2}(t).$$

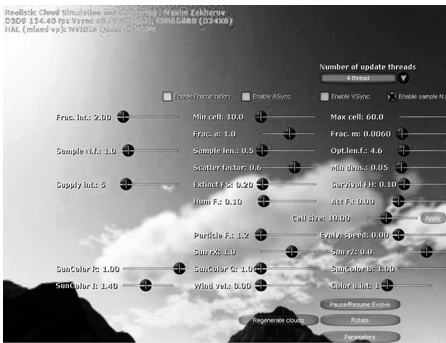


Рис. 5.16

В программе реализована возможность моделирования движения облаков под действием ветра. Визуализация облака производится методом воксельного рендеринга, состоящего из двух шагов. На первом шаге вычисляется интенсивность света, достигающего центра вокселя, и рассчитываются тени облаков друг на друга и окружающий воксельный ландшафт. На втором шаге генерируется само изображение облака в очередности от дальних вокселей к ближним.

Подробное изложение алгоритмов моделирования облаков приведено в [16] и в последней части уникальной магистерской диссертации М. М. Захарова.

5.4.2. Конструктивные модели

Конструктивное моделирование или *конструктивная твёрдотельная геометрия* (CSG — *Constructive Solid Geometry*) представляет объекты комбинацией простых плоских и объёмных примитивов, являющихся «строительными блоками» создаваемых конструкций. Простота примитивов означает относительную математическую простоту и удобство конструирования из них произвольных тел. **Плоский** примитив есть часть плоскости, ограниченная замкнутой кривой или полигоном. **Объёмный** примитив есть часть пространства, ограни-

ченная функционально определёнными поверхностями первого и второго порядков, Эрмита, Безье и т. п. Выбор минимальных порядков граничных поверхностей обусловлен необходимостью аналитического решения задачи пересечения с ними световых лучей.

Каждый примитив имеет граничное описание (см. разд. 5.3). Исходное положение примитивов целесообразно выбирать в начале координат, т. е. описывать их геометрию в собственных объектных координатах. Многообразие объёмных конструкций достигается аффинными преобразованиями примитивов и логическими операциями с ними. В целом конструктивная геометрия является основным способом описания трёхмерных тел в современных САПР.

☞ Рассмотрим **правила построения комбинаций примитивов**. Множество примитивов $\Pi_1, \Pi_2, \dots$, всё пространство I и пустое пространство $\emptyset$ образуют булеву алгебру со следующими **логическими операциями** (символ « $\bullet$ » означает точку).

□ **Отрицание (дополнение)** до I примитива Π :

$$\bullet \in \neg\Pi, \text{ если } \bullet \notin \Pi. \quad (5.9a)$$

Выполнение этой операции означает:

- изменение направлений нормалей N_i ко всем граничным поверхностям Π на противоположные $-N_i$;
- матрицы F_i плоскостей и квадратичных поверхностей заменяются на $-F_i$;
- направления обхода полигональных граней заменяются на инверсные;
- параметрические функции $p(t, \tau)$ заменяются на функции $p(\tau, t)$.

□ **Объединение (сложение)** примитивов Π_i и Π_j :

$$\bullet \in \Pi_i \cup \Pi_j, \text{ если } \{\bullet \in \Pi_i\} \vee \{\bullet \in \Pi_j\}. \quad (5.9б)$$

Выполнение этой операции заключается в расчёте всех пересечений поверхностей примитивов Π_i и Π_j . Поверхность объекта $\Pi_i \cup \Pi_j$ формируется из **внешних** частей поверхности Π_i относительно Π_j , **внешних** частей Π_j относительно Π_i и **общих** частей Π_i и Π_j .

□ **Пересечение (произведение)** примитивов Π_i и Π_j :

$$\bullet \in \Pi_i \cap \Pi_j, \text{ если } \{\bullet \in \Pi_i\} \wedge \{\bullet \in \Pi_j\}. \quad (5.9в)$$

Поверхность объекта $\Pi_i \cap \Pi_j$ формируется из **внутренних** частей поверхности объекта Π_i относительно объекта Π_j , **внутренних** частей Π_j относительно Π_i и **общих** частей Π_i и Π_j .

□ **Исключение (вычитание)** примитива Π_j из примитива Π_i :

$$\bullet \in \Pi_i - \Pi_j, \text{ если } \{\bullet \in \Pi_i\} \wedge \{\bullet \notin \Pi_j\}. \quad (5.9г)$$

Поверхность $\Pi_i - \Pi_j$ формируется из **внешних** частей поверхности Π_i относительно Π_j , **внутренних** частей Π_j относительно Π_i и **общих** частей Π_i и Π_j .

Логические операции подчинены следующим **законам булевой алгебры**:

- законам **отрицания**

$$\neg I = \emptyset, \neg \emptyset = I, \neg(\neg P) = P; \quad (5.10a)$$

- **коммутативным** законам

$$P_i \cup P_j = P_j \cup P_i, \quad P_i \cap P_j = P_j \cap P_i; \quad (5.10б)$$

- **ассоциативным** законам

$$\begin{aligned} (P_i \cup P_j) \cup P_k &= P_i \cup (P_j \cup P_k), \\ (P_i \cap P_j) \cap P_k &= P_i \cap (P_j \cap P_k); \end{aligned} \quad (5.10в)$$

- **дистрибутивным** законам

$$\begin{aligned} (P_i \cup P_j) \cap P_k &= P_i \cap P_k \cup P_j \cap P_k, \\ P_i \cap P_j \cup P_k &= (P_i \cup P_k) \cap (P_j \cup P_k); \end{aligned} \quad (5.10г)$$

- законам **двойственности де Моргана**

$$\begin{aligned} \neg(P_i \cup P_j) &= (\neg P_i) \cap (\neg P_j), \\ \neg(P_i \cap P_j) &= (\neg P_i) \cup (\neg P_j); \end{aligned} \quad (5.10д)$$

- законам **идемпотентности**

$$P \cup P = P, \quad P \cap P = P; \quad (5.10е)$$

- законам **поглощения**

$$(P_i \cup P_j) \cap P_i = P_i, \quad P_i \cap P_j \cup P_i = P_i. \quad (5.10ж)$$

Последовательность логических операций, выполняемых с примитивами в процессе конструирования объекта, называется его **булевым описанием**. Любое булево выражение, описывающее объект O , можно представить в виде объединения 2^N пересечений N объектов o_i — примитивов и их отрицаний:

$$O = \bigcup_{k=1}^{2^N} \left(\bigcap_{i=1}^N (o_i C_i) \right), \quad (5.11)$$

где $o_i C_i$ означает преобразование объекта o_i матрицей C_i . Данная **каноническая форма** булева описания может быть минимизирована известными в булевой алгебре методами карт Карно, диаграмм Эйлера–Венна и т. д. [21].

Полная сплошная модель объекта включает его минимальное булево описание совместно с данными о типе каждого примитива с его граничной моделью, оптическими свойствами каждой поверхности и матрицами аффинных преобразований на каждом шаге конструирования сложного объекта или сцены.

Процесс композиции можно изобразить в виде **бинарного дерева** с правилом обхода «от листьев к корню». Терминальные вершины (листья) означают исходные примитивы, узлы — логические операции, ветви — аффинные преобразования, корень — объект. Нелишне будет ещё раз повторить здесь формулы (5.3) аффинных преобразований векторов плоскостей и матриц квадратичных поверхностей:

$$F' = F(C^{-1})^T, \quad F' = C^{-1}F(C^{-1})^T.$$

Правила логического конструирования позволяют формальными методами рассчитать положение точки относительно *замкнутого* объекта. Оно определяется троичной **функцией принадлежности** $\varphi(p, \Pi)$ точки p примитиву Π :

$$\varphi(p, \Pi) = \begin{cases} -1, & \text{если } p \text{ внутри } \Pi; \\ 0, & \text{если } p \text{ на поверхности } \Pi; \\ 1, & \text{если } p \text{ вне } \Pi. \end{cases} \quad (5.12)$$

Пусть **выпуклый** примитив Π ограничен N поверхностями $f_i(p)=0$ $\forall i=\overline{1, N}$, нормали которых внешне ориентированы по правилам (5.5), (5.6). Функция принадлежности точки p примитиву Π устроена аналогично тесту *conv3_test*, приведённому в (1.149):

$$\varphi(p, \Pi) = \begin{cases} -1, & \text{если } f_i(p) < 0 \quad \forall i=\overline{1, N}; \\ 1, & \text{если } \exists i \in [1, N]: f_i(p) > 0; \\ 0, & \text{если не } \pm 1. \end{cases} \quad (5.13)$$

Точка лежит **внутри** выпуклого примитива, если в ней все значения $f_i(p) < 0$. Точка лежит **вне** примитива, если хотя бы одна из функций $f_i(p) > 0$. Точка принадлежит поверхности выпуклого примитива, если хотя бы одна из функций в этой точке обращается в нуль, а остальные неположительные.

Требование выпуклости примитива **принципиально**. Для убедительности вычислим по (5.13) функцию принадлежности точки, находящейся под гранью $\{5, 6, 7, 8, 5\}$ объекта на рис. 5.5,а: $\varphi(p, \Pi) = 1$, т. к. имеется грань $\{9, 10, 11, 12, 9\}$, относительно которой эта **внутренняя** точка расположена **снаружи**.

Правило (5.13) совместно с логической формулой (5.11) позволяет формально вычислить ориентацию точки относительно сложного объекта, полученного композицией выпуклых примитивов. Решение этой задачи актуально для его визуализации. В методе трассировки луч пересекает множество поверхностей разных примитивов. Из всех точек пересечения отбираются те, что принадлежат **границам объекта**, а из них — ближайшая к источнику света. Таким образом, важно уметь вычислять расположение точки относительно комбинированного объекта. В табл. 5.1 приведены функции принадлежности точки p объектам A, B и их логическим комбинациям:

$$\begin{aligned} \varphi(p, \neg A) &= -\varphi(p, A), \quad \varphi(p, A \cap B) = \max\{\varphi(p, A), \varphi(p, B)\}, \\ \varphi(p, A \cup B) &= \min\{\varphi(p, A), \varphi(p, B)\}, \quad \varphi(p, A - B) = \max\{\varphi(p, A), -\varphi(p, B)\}. \end{aligned} \quad (5.14)$$

Отметим, что при расчёте функции принадлежности точки логической комбинации примитивов **объект O не обязан быть выпуклым**. Главное, чтобы расчёт по (5.13) выполнялся для выпуклых примитивов o_i .

Особенность троичной логики $-0=0$ накладывает ограничение на расположение примитивов с совпадающими границами. Во избежание

Таблица 5.1

$\varphi(A)$	$\varphi(B)$	$\varphi(\neg A)$	$\varphi(A \cap B)$	$\varphi(A \cup B)$	$\varphi(A - B)$
-1	-1	1	-1	-1	1
	0	1	0	-1	0
	1	1	1	-1	-1
0	-1	0	0	-1	1
	0	0	0	0	0
	1	0	1	0	0
1	-1	-1	1	-1	1
	0	-1	1	0	1
	1	-1	1	1	1

ошибки расчёта положения точки на общей границе двух объектов, когда $\varphi(p, A) = \varphi(p, B) = 0$, должны быть **запрещены** совпадения границ в двух операциях (рис. 5.17), выделенных в табл. 5.1:

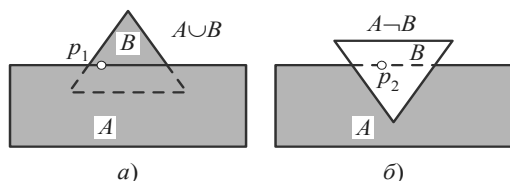


Рис. 5.17

• **внешнее касание** границ при **объединении** объектов (а). По табл. 5.1 определяем граничное значение функции принадлежности $\varphi(p_1, A \cup B) = 0$, хотя в действительности точка p_1 лежит **внутри** объекта $A \cup B$;

• **внутреннее касание** границ при операции **исключения** объектов (б). По табл. 5.1 имеем $\varphi(p_2, A - B) = 0$, хотя точка p_2 лежит **вне** объекта $A - B$.

Для исключения запрещённых совпадений границ между объектами A и B можно изменить один из примитивов, например, B , так, чтобы было $\varphi(p_i, B) = -1$. Теперь по табл. 5.1 функции принадлежности $\varphi(p_1, A \cup B) = -1$ и $\varphi(p_2, A - B) = 1$ определяются верно.

Пример 5.1. Составить описание объекта, изображённого на рис. 5.18,а, и вычислить принадлежность объекту точек $q_1 = [1 \ 3 \ 3]$, $q_2 = [2 \ 3 \ 3]$ и $q_3 = [3 \ 3 \ 3]$.

Решение. Выбрав примитивы Π_1 (куб на рис. 5.18,б) и Π_2 (призму на рис. 5.18,в), составим бинарное дерево композиции объекта (рис. 5.18,г). Булево описание объекта имеет следующий вид: $O = (\Pi_1 C_1) \cap (\Pi_2 C_2) - (\Pi_1 C_3)$.

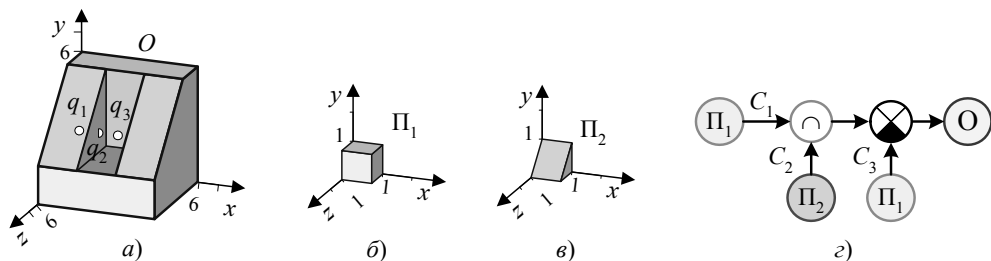


Рис. 5.18

Сформируем массивы неявного описания плоскостей объектов Π_1 , Π_2 и матрицы аффинных преобразований $C_1 \div C_3$:

$$\Pi_1 \equiv \begin{bmatrix} 1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & 0 & -1 \\ -1 & -1 & -1 & 0 & 0 & 0 \end{bmatrix}, \quad \Pi_2 \equiv \begin{bmatrix} 0 & 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 & 0 \\ -1 & -1 & 0 & 0 & 0 & 0 \end{bmatrix},$$

$$C_1 = \begin{bmatrix} 6 & 0 & 0 & 0 \\ 0 & 6 & 0 & 0 \\ 0 & 0 & 6 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad C_2 = \begin{bmatrix} 8 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 \\ 0 & 0 & 8 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad C_3 = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 2 & 2 & 2 & 1 \end{bmatrix}. \quad (5.15)$$

Матрица C_3 выбрана так, чтобы внешние грани объектов $\Pi_1 C_1$ и $\Pi_1 C_3$ не совпадали. Обходя дерево от листьев к корню, формируем массив матриц плоскостей объекта:

$$\begin{aligned} \varphi_{11} \wedge \varphi_{12} \neg \varphi_{13} &= (-1) \neg 1 = -1, \\ \Pi_1 C_1 &\equiv \begin{bmatrix} 1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & 0 & -1 \\ -6 & -6 & -6 & 0 & 0 & 0 \end{bmatrix}, \quad \Pi_2 C_2 \equiv \begin{bmatrix} 0 & 1 & -1 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ -8 & -8 & 0 & 0 & 0 \end{bmatrix}, \\ \Pi_1 C_3 &\equiv \begin{bmatrix} 1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & 0 & -1 \\ -4 & -7 & -7 & 2 & 2 & 0 \end{bmatrix}, \quad (\Pi_1 C_1) \cap (\Pi_2 C_2) \equiv \begin{bmatrix} 1 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & -1 & 1 \\ -6 & -6 & -6 & 0 & 0 & 0 & -8 \end{bmatrix}, \\ O &\equiv \begin{bmatrix} 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & -1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & -1 & 1 & 1 & 0 & 0 & 0 & 1 \\ -6 & -6 & -6 & 0 & 0 & 0 & -8 & -8 & 4 & -2 & -2 & -2 \end{bmatrix}. \end{aligned}$$

Вычислим по (5.13) функции принадлежности точек $q_1 \div q_3$ примитивам $\Pi_1 C_1$, $\Pi_2 C_2$ и $\Pi_1 C_3$, а по табл. 5.1 — всему объекту O :

- $\varphi_{11} = \varphi(q_1, \Pi_1 C_1) = -1$, $\varphi_{12} = \varphi(q_1, \Pi_2 C_2) = -1$, $\varphi_{13} = \varphi(q_1, \Pi_1 C_3) = 1$: $\varphi_{11} \wedge \varphi_{12} \neg \varphi_{13} = (-1) \neg 1 = -1$, из чего делаем вывод, что точка q_1 находится внутри объекта O ;
- $\varphi_{21} = \varphi(q_2, \Pi_1 C_1) = -1$, $\varphi_{22} = \varphi(q_2, \Pi_2 C_2) = -1$, $\varphi_{23} = \varphi(q_2, \Pi_1 C_3) = 0$: $\varphi_{21} \wedge \varphi_{22} \neg \varphi_{23} = (-1) \neg 0 = 0$, откуда следует, что точка q_2 лежит на поверхности объекта O ;
- $\varphi_{31} = \varphi(q_3, \Pi_1 C_1) = -1$, $\varphi_{32} = \varphi(q_3, \Pi_2 C_2) = -1$, $\varphi_{33} = \varphi(q_3, \Pi_1 C_3) = -1$: $\varphi_{31} \wedge \varphi_{32} \neg \varphi_{33} = (-1) \neg (-1) = 1$, что означает расположение точки q_3 вне объекта O . $\square$

В заключение проиллюстрируем возможности конструктивного метода моделирования примерами создания сложных объектов — фонаря, лавочки, шахматного коня и макета Нижегородского кремля (рис. 5.19). Изображения построены с помощью программы **3dLoCon** (трёхмерный логический конструктор), разработанной магистрантом Нижегородского государственного технического университета Дмитрием Васиным под руководством автора книги.

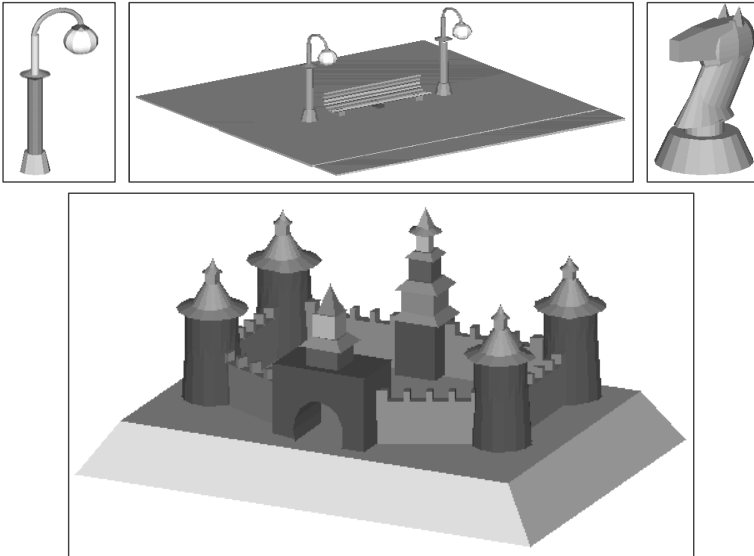


Рис. 5.19

5.5. ОПТИМИЗАЦИЯ ВИЗУАЛИЗАЦИИ

Есть два принципиальных подхода к уменьшению времени, затрачиваемого на визуализацию сцены в целом и удаление её невидимых элементов в частности как наиболее сложной задачи визуализации. Во-первых, это максимально возможное сокращение вычислений; во-вторых, распараллеливание вычислительного процесса. Не вдаваясь в детали реализации второго метода с помощью многопроцессорных и транспьютерных систем [17], а также конвейерной организации вычислений [3, 65], рассмотрим некоторые приёмы ускорения процесса вывода изображений за счёт сокращения вычислений.

5.5.1. Выбор мировой системы координат

Весьма эффективным средством упрощения векторных и матричных вычислений является удачный выбор мировой системы координат x_wz_w , с которой связаны положения наблюдателя и проективной плоскости. Как показано в п. 3.3.4, такой системой является система координат наблюдателя $x_ny_nz_n$. В СКН, играющей роль мировой, основные геометрические элементы и соотношения приобретают следующие наиболее простые формы.

□ Плоскость проецирования совпадает с плоскостью x_n . На ней организуется регулярное *симметричное* окно-отсекатель с координатами центра $x_{ц}=y_{ц}=0$ и габаритными размерами $-g \leq x \leq g$ и $-v \leq y \leq v$. Через это окно наблюдатель смотрит на окружающий его мир (см. рис. 3.90). Габариты окна g и v либо задаются априорно, либо определяются габаритами $left \leq x_3 \leq right$ и $up \leq y_3 \leq down$ окна вывода (см. рис. 3.21) и масштабным множителем μ_3 :

$$g = (right - left) / 2\mu_3, \quad v = (down - up) / 2\mu_3. \quad (5.16)$$

Соотношения (3.42д) между проекцией $p' = [x' \ y' \ 0]$ точки на картинной плоскости и её экранной проекцией $p_3 = [x_3 \ y_3 \ 0]$ принимают следующий вид:

$$x_3 = 0.5(left + right) + \mu_3 x', \quad y_3 = 0.5(up + down) - \mu_3 y'; \quad (5.17a)$$

$$x' = (x_3 - 0.5(left + right)) / \mu_3, \quad y' = (0.5(up + down) - y_3) / \mu_3. \quad (5.17б)$$

□ В сцене с *дальним наблюдателем* его вектор $S = [0 \ 0 \ 1]$ направлен по оси z , матрица проецирования равна матрице O_f из (3.75), ортографическая проекция точки $p = [x \ y \ z]$ есть $p' = [x' \ y' \ z'] = [x \ y \ 0]$, а экранные координаты проекции рассчитываются по (5.17а) при $x' = x$ и $y' = y$.

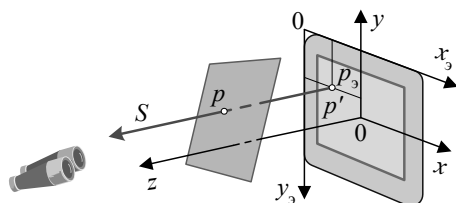


Рис. 5.20

Во многих алгоритмах удаления, например, методом z -буфера (см. п. 1.7.2), требуется восстановить положение точки p по её проекции p_3 (рис. 5.20). Благодаря простоте ортографического проецирования две координаты точки равны координатам проекции (5.17б):

$$x=x', \quad y=y'. \quad (5.18)$$

Для восстановления координаты z нужна некоторая дополнительная информация о поверхности, которой принадлежит точка p .

В случае **неявного** описания поверхности нелинейным уравнением $f(x, y, z)=0$ находится его аналитическое или численное решение $z=\varphi(x, y)$. При **параметрическом** описании поверхности векторной функцией $p(t, \tau)=[x(t, \tau) \ y(t, \tau) \ z(t, \tau)]$ составляется система двух нелинейных уравнений

$$x(t, \tau)=x', \quad y(t, \tau)=y', \quad (5.19)$$

аналитическое или численное решение которой относительно параметров t и τ определяет искомую координату $z(t, \tau)$.

В частном случае **плоскости** , не параллельной оси z , получим несколько вариантов расчёта z -координаты точки $p=[x \ y \ z]$ в зависимости от способа описания плоскости в п. 1.5.1:

- при **неявном** описании вектором $[A \ B \ C \ D]$ с коэффициентом $C \neq 0$ (плоскость не параллельна оси z) из уравнения $Ax+By+Cz+D=0$ получим

$$z=-(Ax+By+D)/C; \quad (5.20a)$$

- при **нормальном** описании плоскости точкой $o=[o_x \ o_y \ o_z]$ и вектором нормали $N=[N_x \ N_y \ N_z]$, не ортогональным оси z , из (5.20a) и соотношений $A=N_x, B=N_y, C=N_z, D=-N \circ o = -N_x o_x - N_y o_y - N_z o_z$ получим

$$z=o_z - (N_x(x-o_x) + N_y(y-o_y))/N_z; \quad (5.20б)$$

- при **параметрическом** описании плоскости точкой $o=[o_x \ o_y \ o_z]$ и непараллельными векторами $V=[V_x \ V_y \ V_z]$ и $W=[W_x \ W_y \ W_z]$, плоскость которых не параллельна оси z , путём подстановки координат нормали (1.94)

$$N_x = \begin{vmatrix} V_y & V_z \\ W_y & W_z \end{vmatrix}, \quad N_y = \begin{vmatrix} V_z & V_x \\ W_z & W_x \end{vmatrix}, \quad N_z = \begin{vmatrix} V_x & V_y \\ W_x & W_y \end{vmatrix}$$

в (5.20б) получим

$$z=o_z + [x-o_x \ y-o_y] \begin{bmatrix} V_x & V_y \\ W_x & W_y \end{bmatrix}^{-1} \begin{bmatrix} V_z \\ W_z \end{bmatrix}. \quad (5.20в)$$

В СКН с вектором наблюдателя $S=z^0$ сортировка глубин точек p_i по убыванию скалярных произведений $d_i=-p_i \circ S$ из (1.182б) (см. рис. 1.181,б) превращается в сортировку аппликат этих точек z_i . Аналогичное упрощение происходит при нахождении лицевых граней полиэдра с помощью теста $N_i \circ S > 0$ из (1.181), который в СКН становится простым анализом знаков аппликат **внешних** нормалей N_{z_i} . **Грань является лицевой при $N_{z_i} > 0$.**

□ **Ближний наблюдатель** находится в точке $S=[0 \ 0 \ F]$ оси z на фокусном расстоянии $0 < F < \infty$ от картинной плоскости (рис. 5.21). Центральное проектирование выполняется по (3.104) с матрицей

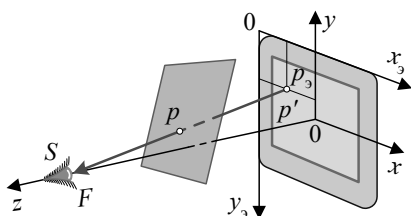


Рис. 5.21

$$C_f = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1/F \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

либо по аналогичным (3.132) координатным формулам

$$x' = Fx/(F-z), \quad y' = Fy/(F-z). \quad (5.21)$$

Экранные координаты проекции точки вычисляются по (5.17а).

Восстановление координат $\{x, y, z\}$ точки p по её экранным координатам $\{x_3, y_3\}$ начинается с вычисления по (5.17б) значений x' и y' . При *неявном* описании $f(x, y, z) = 0$ поверхности, пересекаемой лучом зрения в точке p , координаты последней находятся решением системы двух линейных уравнений, полученных из (5.21), и нелинейного уравнения поверхности:

$$\begin{cases} Fx + x'z = x'F \\ Fy + y'z = y'F \\ f(x, y, z) = 0 \end{cases} \quad \text{или} \quad f\left(\frac{x'(F-z)}{F}, \frac{y'(F-z)}{F}, z\right) = 0. \quad (5.22)$$

При *параметрическом* описании поверхности функциями $x(t, \tau)$, $y(t, \tau)$ и $z(t, \tau)$ координаты лежащей на ней точки находятся решением системы двух нелинейных в общем случае уравнений

$$Fx(t, \tau) + x'z(t, \tau) = x'F, \quad Fy(t, \tau) + y'z(t, \tau) = y'F \quad (5.23)$$

относительно параметров $\{t, \tau\}$ и подстановкой их в функцию $z(t, \tau)$.

В частном случае плоской поверхности получим несколько вариантов расчёта координат точки p в зависимости от способа описания *плоскости*:

- при *неявном* описании вектором $[A \ B \ C \ D]$:

$$\begin{cases} Fx + x'z = x'F, \\ Fy + y'z = y'F, \\ Ax + By + Cz + D = 0 \end{cases} \Rightarrow [x \ y \ z] = [x'F \ y'F \ -D] \begin{bmatrix} F & 0 & A \\ 0 & F & B \\ x' & y' & C \end{bmatrix}^{-1}. \quad (5.24a)$$

Без применения матричных операций после ручных преобразований выражение (5.24a) принимает вид

$$x = -\frac{CF + D}{Ax' + By' - CF}x', \quad y = -\frac{CF + D}{Ax' + By' - CF}y', \quad z = \frac{Ax' + By' + D}{Ax' + By' - CF}F; \quad (5.24б)$$

- при *нормальном* описании плоскости $\{o, N\}$ применяем формулы (5.24) с коэффициентами $A = N_x$, $B = N_y$, $C = N_z$ и $D = -o \circ N$;

- при *параметрическом* описании плоскости $\{o, V, W\}$ применяем формулы (5.24) с $N = V \times W$, $A = N_x$, $B = N_y$, $C = N_z$ и $D = -o \circ N$.

Также представляет интерес решение (1.130а) задачи пересечения луча $p' + Vt$, проведённого из точки $p' = [x' \ y' \ 0]$ в направлении наблюдателя

$$V = S - p' = [-x' \ -y' \ F],$$

с плоскостью $\{o, N\}$:

$$p = p' + \frac{(o - p') \circ N}{V \circ N} V = \frac{(S - o) \circ N}{(S - p') \circ N} p' + \frac{(o - p') \circ N}{(S - p') \circ N} S. \quad (5.25)$$

В СКН с наблюдателем в точке S сортировка точек p_i по уменьшению их дальностей d_i превращается в сортировку по уменьшению длин векторов

$$S - p_i = \frac{(S - o) \circ N}{(S - p'_i) \circ N} (S - p'_i). \quad (5.26)$$

Основной **недостаток** выбора СКН в качестве рабочей системы — необходимость пересчёта в неё координат всех объектов и матриц преобразований, выполняемых обычно в СК сцены. Подробное обсуждение особенностей работы в этой субъективно неподвижной системе координат приведено в п. 3.3.4.

5.5.2. Отбраковка нелицевых граней и нормировка векторов

Процесс создания реалистичного изображения сцены можно представить как **графический конвейер**, на который различные объекты ставят свои элементы — точки, отрезки, полигоны — для последующей обработки. Пройдя через этапы геометрических преобразований в пространстве, проецирования на картинную плоскость и отсечения окном вывода, элементы поступают на этап удаления невидимых элементов, где определяется их взаимное экранирование и видимость наблюдателем. В результате из видимых фрагментов формируется растр изображения в желаемой цветовой палитре — цветной, чёрно-белой, в градациях серого и т. п. Одновременно с расчётом растра выполняется цветовой расчёт (**тонирование** по [65]), после чего готовая картинка из буфера кадра выводится на экран.

Сразу после размещения объектов сцены в пространстве становится возможным определить, какие графические элементы **не будут** видны наблюдателю целиком по причине их полного экранирования другими элементами. Эти элементы целесообразно немедленно **отбраковать** (снять с конвейера), чтобы не тратить машинные ресурсы на их дальнейшую бесполезную обработку. Отбраковка касается также граней, являющихся **нелицевыми**, т. е. не обращёнными «лицом» к наблюдателю.

У задачи поиска и отбраковки нелицевых граней есть два аспекта: алгоритмический и геометрический. **Алгоритмический аспект** имеет следующий смысл: если используемый алгоритм удаления не основан на изображении только лицевых граней (примером является алгоритм художника, описанный в п. 1.7.2), то отпадает сама необходимость их обнаружения, не говоря уж об удалении из конвейерного списка. Таким образом, не для всех алгоритмов удаления отбраковка нелицевых граней является необходимой, хотя, несомненно, вреда она не принесёт, сократив размер списка обрабатываемых граней.

Геометрический аспект основан на свойстве любой поверхности как границы раздела пространства иметь две стороны. Не вникая в строгое топологическое определение стороны поверхности, используем обиходное понятие сто-

рон листа как двумерных сущностей, у которых переход с одной на другую выполняется путём прокола листа. Какую из двух сторон назвать внешней (наружной), а какую внутренней? Может ли внутренняя сторона быть лицевой?

Ответы на эти вопросы зависят от вида модели объекта, его замкнутости и позиции наблюдателя. Внешней **условно** назовём сторону поверхности **замкнутого** объекта, видимую наблюдателем из неограниченного подпространства (рис. 5.22,а). У объекта, описываемого **сплошной** 3d-моделью, внутренней стороны поверхности вообще нет из-за отсутствия внутренней полости. При разрезании сплошного тела с помощью булевой операции исключения функцией *inclip3_hedr* его поверхность изменяется, но всегда остаётся внешней (рис. 5.22,б). Таким образом, лицевые участки поверхности сплошного тела могут располагаться только на её наружной стороне, а описания внутренних поверхностей в моделях данного класса отсутствуют.

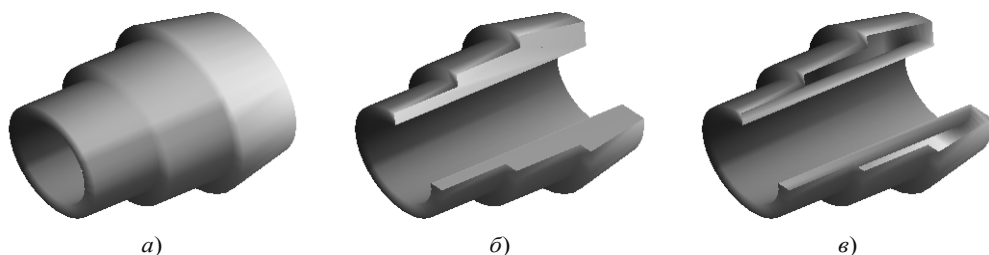


Рис. 5.22

Внутренняя сторона поверхности замкнутого **граничного** объекта существует, может иметь цвет, отличающийся от цвета наружной стороны, но остаётся невидимой, пока поверхность непрозрачна. Установление ненулевого процента прозрачности делает обе стороны видимыми (см. рис. 3.38, 3.86, 5.9). При разрезании граничного объекта он становится незамкнутым, что эквивалентно замкнутому оставшемуся объекту с полностью прозрачными гранями сечения (рис. 5.22,в). Так или иначе, лицевые участки замкнутой граничной поверхности могут располагаться на обеих её сторонах, а **модель полупрозрачного объекта должна содержать двустороннее описание граничной поверхности**.

Определения внешней и внутренней сторон поверхности **незамкнутого** объекта вообще бессмысленны, т.к. обе стороны благодаря кривизне могут быть видны одновременно и никакой из них нельзя отдать предпочтение (см. рис. 1.142, 3.77, 4.49, 5.22,в). Таким образом, **модель незамкнутой поверхности всегда должна быть двусторонней** и на каждой её стороне могут быть одновременно как лицевые, так и нелицевые участки.

Признаком лицевой ориентации i -й грани или касательной плоскости к выпуклой поверхности является **острый** угол ψ_i между векторами **внешней** нормали N_i и наблюдателя H_i (см. рис. 1.179), а самым простым её тестом (1.181) — значение $N_i \circ H_i > 0$. Более того, как говорилось в п. 5.5.1, при выборе системы координат с вектором дальнего наблюдателя $H_i = z^0$ отбраковка превращается в простой анализ знаков аппликат внешних нормалей N_{zi} .

Внешняя ориентация нормалей 3d-объекта обеспечивается:

- **вручную** на этапе создания модели объекта в виде параметрической функции поверхности $p(t, \tau)$ с **правой** тройкой векторов $\{V(t, \tau), W(t, \tau), N(t, \tau)\}$ (см. рис. 1.98) или списков граней $\{G_i\}$ с **положительными** направлениями обхода вершин с точки зрения наружного наблюдателя (см. грани 1, 3, 5 на рис. 1.113);

- **алгоритмически** путём тестирования и, при необходимости, коррекции моделей поверхностей с внутренними нормальными. Применительно к полиэдру это реализуется функцией *norm_test* (см. рис. 1.135), а проверка и коррекция неявной либо параметрической моделей поверхности осуществляется методом пробной точки по формулам (5.5) и (5.6).

Что означает термин «двустороннее описание» поверхности или грани? Для простоты обычно пренебрегают толщиной листа, разделяющего пространство. Таким образом, точки, составляющие бесконечно тонкую поверхность на её противоположных сторонах, совпадают. Лишь нормали N к внешней и $-N$ к внутренней сторонам имеют противоположные направления (рис. 5.23). Следовательно, модели противоположных сторон могут иметь общее параметрическое $p(t, \tau)$ или полигональное $P = p_1 p_2 \dots p_n p_1$ описания, но разные нормали, либо общий вектор нормали и разные знаки «+» и «-», с которыми он должен применяться каждой из сторон.

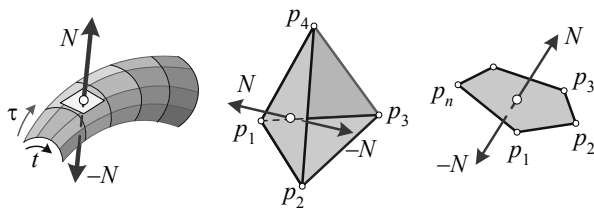


Рис. 5.23

В заключение необходимо отметить, что небольшое ускорение визуализации может быть достигнуто с помощью **предварительной нормировки** $\bar{V} = V/|V|$ всех векторов V (имя V — обобщающее, вместо него может быть любое другое обозначение вектора), используемых более одного раза. Сокращение времени достигается за счёт:

- исключения многократных вычислений длины вектора $|V|$ как квадратного корня $\sqrt{V \circ V}$ — достаточно медленной и **приближённой** операции, выполняемой с помощью разложения в степенной ряд;
- отсутствия повторений выполнения арифметической операции деления на $|V|$, т. к. у нормированного вектора $|\bar{V}| = 1$.

Перечислим несколько типовых случаев нормировки векторов:

- расчёт расстояний между различными графическими элементами;
- трассировка лучей и расчёт оптических эффектов (отражений и преломлений) на произвольных поверхностях, в частности, на плоскостях [28, 35, 36, 38];
- вычисление по (1.124) угла между векторами V и W либо косинуса угла $\cos(\angle VW) = \bar{V} \circ \bar{W}$. Этот расчёт необходим, например, для нахождения лицевых граней и закраски поверхности по (4.1) на основе закона диффузного отражения Ламберта. В связи с этим рекомендуется сразу после задания или вычисления

нормировать векторы наблюдателя и нормалей ко всем граням полиэдров и во всех необходимых точках криволинейных поверхностей.

5.5.3. Построение охватывающих оболочек

Метод оболочек получил широкое распространение в системах машинной графики благодаря резкому сокращению трудоёмкости решения задач трассировки лучей и визуализации. Суть метода в том, что сложный 3d-объект с большим числом разнообразных (полигональных или криволинейных, плоских или объёмных, выпуклых или невыпуклых) ячеек окружается (вручную или автоматически) виртуальной **выпуклой оболочкой** существенно более простой формы, чем объект. **Назначение оболочки:**

- локализовать часть пространства, содержащего объект;
- заменить выполнение вычислительных операций со сложной поверхностью объекта операциями с простой оболочкой.

Модели оболочек добавляются к списку моделей объектов сцены либо составляют отдельный список оболочек. Оболочки обновляются после каждого геометрического преобразования, изменяющего их параметры.

Трудоёмкость метода оболочек пропорциональна суммарному числу граней оболочек сцены $N_T = N_o n_{го}$, где $n_{го}$ — среднее число граней оболочки одного объекта. Уменьшение трудоёмкости обработки оболочки в сравнении с объектом оценивается **коэффициентом простоты оболочки** $\eta_o = n_{го} / n_r < 1$, где n_r — среднее число граней объекта.

Обозначим некоторые задачи компьютерной графики, где применение охватывающих оболочек наиболее эффективно для экономии машинных ресурсов:

- практически все алгоритмы, использующие **трассировку лучей**. Метод трассировки лежит в основе некоторых алгоритмов удаления. Расчёт пересечения лучей с объектами сцены занимает от 75% до 95% всех вычислений [61], причём большая доля расчётов при малой степени заполнения сцены объектами выполняется бесполезно из-за прохождения большинства лучей мимо всех объектов. Поэтому желательно как можно раньше исключить такие лучи из рассмотрения путём быстрой оценки их непересечения с простыми оболочками;

- **обработка столкновений** движущихся объектов с помощью тестов, проверяющих взаимное проникновение элементов одного объекта внутрь другого. С помощью простых оболочек можно быстро обнаружить заведомо не пересекающиеся объекты. В отсутствие столкновения оболочек отпадает необходимость трудоёмкой проверки столкновения самих объектов сложной формы;

- в системах анимации, моделирования и автоматизированного проектирования сложных многосвязных конструкций практикуется **эскизное изображение** звеньев сложной формы в виде охватывающих их оболочек-параллелепипедов (рис. 5.24,а). Это повышает эффективность проектирования и ускоряет вывод в сравнении с каркасной моделью (рис. 5.24,б), не говоря уж о сплошной (рис. 5.24,в), за счёт исключения трудоёмкой визуализации сложных поверхностей и оптических эффектов. Аналогичный приём используется в про-

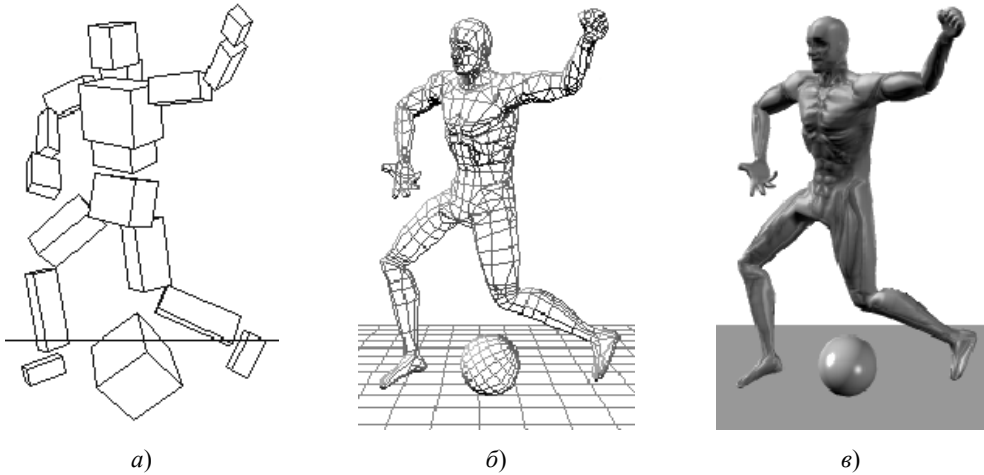


Рис. 5.24

граммах архитектурного и ландшафтного дизайна, заменяющих в эскизах сложные архитектурные сооружения условными «коробками» для ускорения композиционных, транспортных, акустических, инсоляционных (освещённость солнечным светом), аэродинамических и других инженерных расчётов.

Наиболее распространены три **типа оболочек**: сферические, габаритные и полигональные. Кроме геометрической формы они характеризуются;

- **способом построения** — ручным, полуавтоматическим или автоматическим;
- **плотностью** облегания объекта;
- **сложностью** модели — числом её параметров и трудоёмкостью применения;
- зависимостью параметров оболочки от геометрических преобразований охватываемого объекта.

Сферические оболочки

Сферическая оболочка (рис. 5.25,а) строится **автоматически** и отличается от других оболочек минимумом параметров, простотой построения и использования, но, с другой стороны, невысокой плотностью облегания. Положение сферы в пространстве однозначно задаётся точкой центра c и радиусом r . В первом приближении, достойном его простоте, центр сферической оболочки объекта, имеющего список вершин $P = \{p_1, p_2, \dots, p_n\}$, можно принять в геометрическом центре вершин $c = \text{mean}(P)$, а радиус оболочки равным $r = \max_i |p_i - c|$.

Аффинные преобразования вращения и переноса, не изменяющие форму объекта, не влияют на значение r , тогда как масштабирование изменяет радиус оболочки. Таким образом, у сферической оболочки любого движущегося жёсткого объекта изменяются только координаты геометрического центра c .

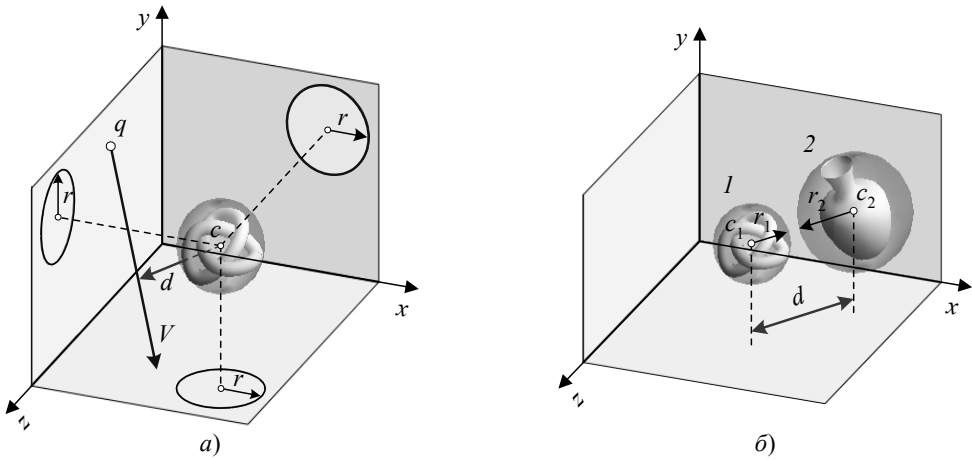


Рис. 5.25

Расчёт пересечения прямой линии $p(t)=q+Vt$ с квадратичной поверхностью (см. п. 1.5.9) сводится к решению квадратного уравнения (1.60а), для составления которого нужно иметь матрицу F квадратичной формы поверхности из (1.158б). Однако, тест пересечения луча со сферой интересуют не сами точки пересечения и даже не значения параметра t в этих точках, а лишь факт наличия или отсутствия пересечения. Достаточно вычислить расстояние от точки c до прямой $\{q, V\}$, равное по (1.113) значению $d=|(c-q) \times \bar{V}|$, и сравнить его с радиусом r . При $d \geq r$ луч и сфера гарантированно **не пересекаются**.

Так же просто с помощью двух сферических оболочек с параметрами $\{c_1, r_1\}$ и $\{c_2, r_2\}$ оценивается взаимная ориентация сложных объектов 1 и 2 (рис. 5.25, б). Тестом заведомого **непересечения** этих объектов является выполнение условия $|c_1 - c_2| \geq r_1 + r_2$.

Габаритные оболочки

Габаритная оболочка объекта с вершинами $p_i = [x_i \ y_i \ z_i]$ строится **автоматически** как описанный шестигранник, форма которого зависит от положения наблюдателя. В сцене с **дальним наблюдателем** и вектором направления $S = z^0$ оболочка имеет форму регулярного **параллелепипеда** с гранями, параллельными координатным плоскостям СКН (рис. 5.26). Габаритные координаты объекта

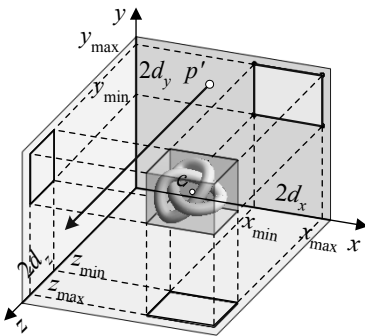


Рис. 5.26

$$\begin{aligned} x_{\min} &= \min\{x_i\}, & x_{\max} &= \max\{x_i\}, \\ y_{\min} &= \min\{y_i\}, & y_{\max} &= \max\{y_i\}, \\ z_{\min} &= \min\{z_i\}, & z_{\max} &= \max\{z_i\} \end{aligned}$$

определяют принадлежащее его оболочке замкнутое пространство

$$\{x \in [x_{\min}, x_{\max}]\} \wedge \{y \in [y_{\min}, y_{\max}]\} \wedge \{z \in [z_{\min}, z_{\max}]\}.$$

При диагональном расположении сильно вытянутого объекта его габаритная оболочка имеет очень малую плотность облегаания.

Для получения полиэдральной модели параллелепипеда $\{O, G\}$ из модели куба (1.179б) вычислим его центр и полудлины сторон:

$$c = 0.5[x_{\min} + x_{\max} \quad y_{\min} + y_{\max} \quad z_{\min} + z_{\max}], \\ d_x = 0.5(x_{\max} - x_{\min}), \quad d_y = 0.5(y_{\max} - y_{\min}), \quad d_z = 0.5(z_{\max} - z_{\min}).$$

Тогда матрица вершин оболочки и список граней принимают вид

$$\tilde{O} = \tilde{P}_{\square} \tilde{M}(d_x, d_y, d_z) T(c), \quad G = G_{\square}.$$

В алгоритмах удаления, использующих трассировку лучей и сортировку точек по глубине сцены, габаритные оболочки-параллелепипеды применяются для быстрого обнаружения объектов, не пересекаемых лучами. Габаритный тест пересечения объекта с прямой $p_3 + z^0 t$, проходящей через *экранную* точку p_3 параллельно боковым граням оболочки, состоит в вычислении по (5.17б) координат точки p' в системе координат наблюдателя и проверке условия

$$\{x_{\min} < x' < x_{\max}\} \wedge \{y_{\min} < y' < y_{\max}\} \quad (5.27)$$

их принадлежности *прямоугольной* проекции оболочки. Начало луча вне габаритного прямоугольника гарантирует непересечение объекта лучом.

Оболочки-параллелепипеды широко применяются в габаритных тестах взаимной ориентации графических объектов, сводящихся к простой проверке пересечения интервалов их координат. Примерами использования габаритных оболочек для быстрого обнаружения заведомо непересекающихся объектов служат функции *gab3_test* и *gab3_poly*, определённые в п. 1.5.6.

В сцене с *ближним наблюдателем*, находящимся в точке $S = Fz^0$, габаритные оболочки имеют форму *усечённых пирамид* с вершинами S (рис. 5.27). Задняя и передняя прямоугольные грани оболочки параллельны плоскости проецирования и лежат в плоскостях $z_{\min} = \min\{z_i\}$ и $z_{\max} = \max\{z_i\}$.

Плоскости боковых граней-трапеций образуют минимальные *углы раствора* по горизонтали и вертикали. В результате все точки объекта заключены между этими плоскостями. Тангенсы углов наклона $g_{\min}, g_{\max}, v_{\min}, v_{\max}$ плоскостей к оси z и габариты прямоугольной проекции оболочки на плоскость xy находятся так:

$$g_{\min} = \min_i (x_i / (F - z)), \quad g_{\max} = \max_i (x_i / (F - z)), \\ v_{\min} = \min_i (y_i / (F - z)), \quad v_{\max} = \max_i (y_i / (F - z)), \quad (5.28a)$$

$$x'_{\min} = Fg_{\min}, \quad x'_{\max} = Fg_{\max}, \quad y'_{\min} = Fv_{\min}, \quad y'_{\max} = Fv_{\max}.$$

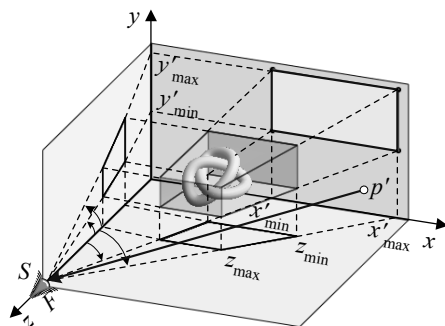


Рис. 5.27

Полиэдральная модель усечённой пирамиды $\{O, G\}$ включает список граней куба $G=G_{\square}$ из (1.179б) и матрицу вершин

$$O = \begin{bmatrix} g_{\min} H & g_{\min} h & g_{\max} h & g_{\max} H & g_{\min} H & g_{\min} h & g_{\max} h & g_{\max} H \\ v_{\max} H & v_{\max} h & v_{\max} h & v_{\max} H & v_{\min} H & v_{\min} h & v_{\min} h & v_{\min} H \\ z_{\min} & z_{\max} & z_{\max} & z_{\min} & z_{\min} & z_{\max} & z_{\max} & z_{\min} \end{bmatrix}^T,$$

где $H=F-z_{\min}$ и $h=F-z_{\max}$. Эти данные можно использовать для визуализации оболочки как любого другого полиэдра, а также в иных целях, например, при решении различных задач пересечения.

В алгоритмах, использующих трассировку лучей и сортировку точек по расстоянию до ближнего наблюдателя, габаритные оболочки-пирамиды применяются для быстрого обнаружения объектов, не пересекаемых сканирующими лучами, сходящимися в центре проецирования. Габаритный тест пересечения объекта с прямой $p' + (S - p')t$, соединяющей точку $p' \in xy$ с наблюдателем S , заключается в проверке условия

$$\{x'_{\min} < x' < x'_{\max}\} \wedge \{y'_{\min} < y' < y'_{\max}\} \quad (5.28б)$$

её принадлежности **прямоугольной** центральной проекции оболочки. Расположение начала луча вне габаритного прямоугольника означает гарантированное непересечение объекта с этим лучом.

Оболочки-пирамиды применяются в габаритных тестах взаимной ориентации графических объектов, сводящихся к проверке пересечения их **угловых интервалов** $[g_{\min}, g_{\max}]$, $[v_{\min}, v_{\max}]$ и координатных интервалов $[z_{\min}, z_{\max}]$. Например, аналогично тесту (1.154), использующему оболочку-параллелепипед, строится перспективный вариант габаритного теста локализации точки $q = [x \ y \ z]$ с угловыми координатами $g = x/(F - z)$ и $v = y/(F - z)$ относительно объекта P , имеющего пирамидальную оболочку с параметрами (5.28а):

$$gab3_test(q, P) = \{g \notin [g_{\min}, g_{\max}]\} \vee \{v \notin [v_{\min}, v_{\max}]\} \vee \{z \notin [z_{\min}, z_{\max}]\}.$$

Недостатки габаритной оболочки заключаются, во-первых, в зависимости её параметров от преобразований объекта; во-вторых, как уже говорилось, в малой плотности облегающего объекта вытянутой формы с диагональным расположением относительно координатных плоскостей.

Полиэдральные оболочки

Полиэдральная оболочка строится как выпуклый полиэдр $\{O, G\}$ несложной формы, плотно облегающий объект. Как правило, такая оболочка создаётся один раз и претерпевает синхронно с объектом все геометрические преобразования. Способ построения оболочки, в основном, **ручной** — путём интерактивного выставления вокруг объекта точек o_i (вершин оболочки) и формирования на них граней G_j с непрерывным тестированием:

- выпуклости получаемого полиэдра $\{O, G\}$ функцией $conv3(O, G)$, определённой в (1.147);

• нахождения всех вершин объекта p_i в пределах границ выпуклого полиэдра $\{O, G\}$ с помощью функции $\text{conv3_test}(p_i, O, G)$ из (1.149).

Автоматизация построения плотной полиэдральной оболочки возможна при наличии у объекта симметрии или если имеется алгоритм синтеза выпуклых полиэдральных оболочек массива точек. Для поверхности, полученной вращением некоторой образующей линии $p_o(\tau) = [x_o(\tau) \ y_o(\tau) \ 0]$ вокруг оси y , можно **интерактивно** задать на плоскости xu минимальное число точек o_i так, чтобы ломаная линия $o_1 o_2 \dots o_n$ огибала образующую, нигде с ней не пересекаясь (рис. 5.28, а). Непрерывное вращение этой ломаной линии вокруг оси y даёт **криволинейную** поликоническую боковую поверхность (рис. 5.28, б).

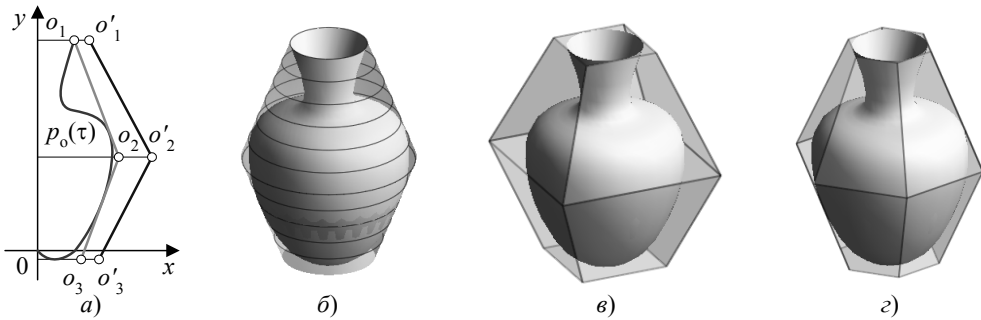


Рис. 5.28

Для получения полиэдральной оболочки впишем каждый усечённый конус в правильную усечённую пирамиду с $n_o \geq 3$ боковыми гранями. Поскольку у правильного n_o -угольника соотношение радиусов описанной и вписанной окружностей равно $\sec(\psi_o)$, где $\psi_o = \pi/n_o$ [21], то вершины оболочки лежат на меридианах поверхности вращения полилинии $o'_1 o'_2 \dots o'_n$, полученной растяжением полилинии $o_1 o_2 \dots o_n$ вдоль оси x в $\sec(\psi_o)$ раз. Угол между образующими меридианами равен $2\psi_o$ и вращение каждой неплюсовой точки o'_i даёт n_o вершин оболочки, рассчитываемых по формуле $o'_i R_y(2\psi_o j) \ \forall j = \overline{1, n_o}$.

Благодаря осевой симметрии расположения этих точек четырёхугольные грани оболочки являются планарными.

Таким образом, с учётом верхней и нижней n_o -угольных граней топологические параметры полиэдральной оболочки в полном соответствии с формулой Эйлера (1.141а) равны $n_b = nn_o$, $n_r = (n-1)n_o + 2$ и $n_p = (2n-1)n_o$. Если одна из крайних точек o_1 либо o_n является полюсом, то $n_b = (n-1)n_o + 1$, $n_r = (n-1)n_o + 1$ и $n_p = 2(n-1)n_o$. Наконец, если обе эти точки выбраны на оси вращения, то $n_b = (n-2)n_o + 2$, $n_r = (n-1)n_o$ и $n_p = (2n-3)n_o$.

Для иллюстрации метода на рис. 5.28 построены две полиэдральные оболочки одного и того же объекта вращения при $n_o = 4$ (в) и $n_o = 6$ (z).

Полностью **автоматический** синтез полиэдральной оболочки может быть выполнен с помощью алгоритма *cover3*, описанного в п. 1.5.8 (см. рис. 1.138).

Достаточно сформировать список P вершин объекта, пронумерованных в произвольном порядке, и алгоритм возвратит модель выпуклого полиэдра $\{O, G\} = \text{cover3}(P)$, плотно обтягивающего объект по выступающим вершинам.

Недостатки автоматического метода:

- задание большого числа точек криволинейной поверхности для увеличения плотности её облегания даёт слишком много граней оболочки, что нарушает принцип её простоты в сравнении с объектом;
- задание слишком малого числа точек криволинейного объекта приводит к выпячиванию его отдельных частей из построенной «оболочки», которая в этом случае таковой не является. Выходом из этой ситуации является добавление максимально выступающих точек к списку вершин построенного полиэдра и повторный расчёт оболочки.

Кластерные оболочки

Дальнейшее развитие метода оболочек, повышающее эффективность визуализации, основано на понятии **кластерных групп** пространственно связанных объектов и приводит к построению иерархической системы оболочек. Начальный уровень иерархии состоит из одного **кластера сцены**, оболочка которого охватывает всё её содержимое. А, например, сцена компьютерной игры в футбол состоит из объектов второго уровня иерархии — поля, двух ворот, игроков наподобие фигуры, изображённой на рис. 5.24, и мяча. Тело человека не является единым объектом и состоит из соединённых между собой относительно подвижных частей (головы, шеи, туловища, рук и ног) — объектов третьего уровня иерархии. **Кластер группы** конструктивно связанных объектов имеет свою собственную оболочку. Процедура кластеризации выполняется до получения терминальных вершин иерархического дерева — неделимых объектов со своими оболочками.

Построенные кластерные оболочки обрабатываются в иерархическом порядке, начиная с кластера сцены. Если луч не пересекает его оболочку, то он не может пересечь и ни одного из входящих в него объектов, что позволяет сразу исключить их из списка обработки. Иначе на пересечение с лучом проверяются оболочки кластеров следующего уровня иерархии. Если луч не пересекает некоторую оболочку, то её кластер и все содержащиеся в нём объекты и кластеры исключаются из дальнейшего анализа.

При пересечении кластера с лучом обход кластерного дерева рекурсивно повторяется, пока не будут рассмотрены все объекты. Когда луч пересекает оболочку некоего объекта, тот заносится в список активных объектов. По завершении обхода дерева число элементов в списке активных объектов, как правило, намного меньше общего числа объектов сцены, подвергаемых обработке.

Кластеризация резко сокращает расчёт пересечений лучей с оболочками и в целом повышает эффективность алгоритма визуализации сцены. Трудоемкость метода кластерных оболочек пропорциональна числу $\log(N_r) \ll N_r$ при $N_r \gg 1$.

5.5.4. Разбиение граней

В разд. 1.4 мы изучили алгоритмы разрезания плоских полигонов на выпуклые фрагменты и треугольники (*cut_poly*, *tri_poly*, *contri_poly*). В п. 1.4.3 было кратко обозначено место операции разрезания в арсенале инструментов компьютерной графики. Приведём ещё некоторые **аргументы в пользу выполнения разбиения** на предварительной стадии визуализации объектов:

- насколько это возможно, рекомендуется избегать операций с невыпуклыми объектами, существенно более сложных в сравнении с алгоритмами обработки выпуклых фигур. Однократные затраты на выпуклое разбиение, например, алгоритмом *cut_poly*, невыпуклых взаимно перекрывающихся или пересекающихся полигонов (см. рис. 1.129,а – в) несравнимы с многократным решением задачи визуализации этих конструкций в различных ракурсах;

- определённые трудности обычно вызывает визуализация неплоской грани. Во-первых, нормаль к её поверхности не имеет постоянного направления, из-за чего окраска этой грани индивидуальна в каждой её точке (см. рис. 4.82–4.84). Во-вторых, неплоская грань типа косой плоскости при определённом ракурсе наблюдения может экранировать саму себя и проецироваться на картинную плоскость невыпуклым и, возможно, самопересекающимся полигоном (см. рис. 4.68,в). Аппроксимация такой грани системой треугольников — всегда плоских полигонов — избавляет от перечисленных проблем. Для этого можно использовать, например, алгоритм *tri_poly*;

- некоторые алгоритмы удаления правильно работают с равновеликими гранями, близкими по форме к правильным полигонам (равносторонним треугольникам и квадратам), но не всегда верно отображают грани вытянутой формы, сильно отличающиеся друг от друга габаритными размерами (см. рис. 1.182). Решение становится возможным после разбиения таких граней на приблизительно равновеликие фрагменты желаемой формы. Для этого можно использовать, например, алгоритм *contri_poly*;

- в зависимости от удалённости наблюдателя один и тот же объект может выглядеть как мелко, так и крупно. Если в используемом алгоритме визуализации не применяется никаких методов сглаживания, то приближение наблюдателя к криволинейной поверхности делает её изображение крупнее, как бы сотканным из лоскутов с чётко различимыми границами. В этом случае целесообразно разбивать поверхность на фрагменты с постоянными **угловыми** размерами. При этом линейные размеры фрагментов будут изменяться пропорционально расстоянию от наблюдателя до поверхности;

- одним из столпов САПР объектов с распределёнными параметрами является **метод конечных элементов** [26, 27], основанный на разбиении объекта на большое число мелких частей. В области конечного элемента определённые свойства объекта считаются постоянными, а их изменения происходят на границах элементов. Таким образом, в системах проектирования сложных протяжённых объектов разбиение их на элементы играет основополагающую роль.

В сценах с подвижным наблюдателем весьма эффективным является **адаптивное разбиение** поверхности объекта, смысл которого заключается в следующем. С дальней дистанции мелкие детали объекта не различимы (сливаются друг с другом) из-за конечного углового разрешения глаза здорового человека, равного приблизительно $\delta_p \approx 1' = 0.00028$ рад [25] и обусловленного определённым расстоянием между светочувствительными рецепторами.

Для осмысления этой малой величины достаточно сказать, что круглый диск диаметром $d_\bullet = 0.25$ мм (размер пиксела) виден под углом δ_p с расстояния $d_\bullet / \delta_p \approx 1$ м. Значит, поверхности удалённых объектов можно разбивать на малое число крупных граней. При «наезде» наблюдателя на объект, что равносильно приближению объекта к наблюдателю, предельное линейное разрешение $h_p = r \delta_p$ — промежуток между различимыми точками — уменьшается пропорционально расстоянию r от наблюдателя до объекта (рис. 5.29).



Рис. 5.29

Вторым фактором, ограничивающим линейное разрешение изображения на экране дисплея, является конечный размер пиксела того графического режима, в котором работает прикладная программа. В режиме с максимальным разрешением видеоадаптера можно принять диаметр пиксела равным $d_\bullet = 0.25$ мм. Тогда с учётом масштабного коэффициента $\mu_\bullet$, вычисленного по (3.41) или (3.426), получим предельное линейное разрешение $h_p = \mu_\bullet d_\bullet$.

Объединяя обе оценки, получим следующую оценку **минимального габаритного параметра разбиения**:

$$h_{\min} = \max \{ r \delta_p, \mu_\bullet d_\bullet \}. \quad (5.29)$$

Таким образом, выбор размера ячеек разбиваемой поверхности должен осуществляться из условия $h \geq h_{\min}$, не забывая при этом, что значение $h_{\min}$ даёт предельно малые размеры различимых элементов изображения и делает визуализацию чрезвычайно трудоёмкой. На практике, конечно же, нужно задавать параметр h на порядок бóльшим.

Алгоритм адаптивного разбиения имеет явно выраженные фрактальные свойства в смысле бесконечности процесса приближения модели, составленной из плоских граней, к реальной гладкой поверхности.

Выделим основные **типы поверхностей разбиения**:

- **параметрические** $p(t, \tau)$, определённые в узловых точках $p_{ij} \equiv p(t_i, \tau_j)$ сетки отсчётов параметров $t \in \{t_0, \dots, t_n\}$, $\tau \in \{\tau_0, \dots, \tau_m\}$ (рис. 5.30, а), и **полигональные**, заданные списком вершин $P = p_1 p_2 \dots p_k p_1$ в порядке обхода границы поверхности (рис. 5.30, б).

Переход от параметрического описания к полигональному заключается в формировании замкнутого контура из $k = 2(n + m)$ граничных точек:

$$P = p_{00} p_{10} \dots p_{n0} p_{n1} \dots p_{nm} \dots p_{0m} \dots p_{00}. \quad (5.30)$$

При этом возможно слияние нескольких вершин в общем полюсе, порождающее кратность этих точек и необходимость её устранения перед началом разбиения.

Обратное преобразование полигональной поверхности в параметрическую, в общем случае, затруднительно из-за её произвольной формы и нерегулярного распределения вершин по контуру полигона;

- **плоские** и **объёмные** в зависимости от определяемой по (1.112б) размерности пространства, натянутого на точки поверхности. Разбиение плоских граней на более мелкие грани имеет целью приведение их размеров к одному порядку для облегчения решения задачи удаления, а также разрезания невыпуклых граней на совокупность выпуклых фрагментов.

Ещё одна и, пожалуй, главная причина, по которой объёмные поверхности подвергаются разбиению по алгоритму триангуляции, — это получение моделей с всегда плоскими треугольными гранями. Параметрические поверхности симметричных объектов — в основном, тел вращения — могут быть разбиты на плоские грани и более чем треугольной формы. Благодаря симметрии все вершины такой грани удовлетворяют в (1.112б) условию $m=2$.

Дробление **параметрической поверхности** целесообразнее всего выполнять в **пространстве параметров** $\{t, \tau\}$ с последующим прямым расчётом точек поверхности $p(t, \tau)$. Следует помнить, что нелинейности в координатных функциях $x(t, \tau) \div z(t, \tau)$ — источник искажений формы ячеек, построенных в пространстве $\{t, \tau\}$, при их отображении на криволинейную поверхность.

Рассмотрим некоторые **способы разбиения параметрической сетки**:

- уменьшение (не обязательно кратное) параметрических квантов $\Delta t_i = t_{i+1} - t_i$ и $\Delta \tau_j = \tau_{j+1} - \tau_j$. Фактически на прежнюю поверхность натягивается новая, более густая сетка, что увеличивает число ячеек и уменьшает их размер. Например, при кратном уменьшении Δt_i в N , а $\Delta \tau_j$ в M раз каждая ячейка дробится на NM прямоугольных ячеек (рис. 5.31, а);

- разбиение каждой прямоугольной ячейки диагоналями на два или четыре треугольника, причём во втором случае центр параметрической ячейки находится в точке $\theta_{ij} = 0.5[t_i + t_{i+1} \quad \tau_j + \tau_{j+1}]$ (рис. 5.31, б);

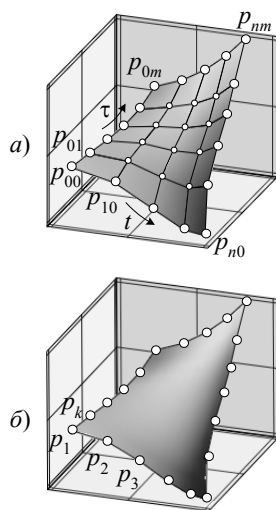


Рис. 5.30

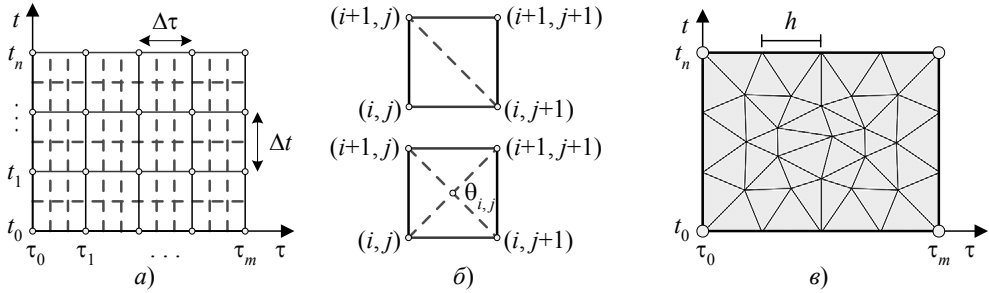


Рис. 5.31

• переход от параметрической сетки к её граничному прямоугольнику с четырьмя вершинами и его триангуляция по алгоритму *contri_poly* с желаемым значением h , выбираемым с учётом (5.29) (рис. 5.31, в).

Эффективность описанных методов разбиения объёмной параметрической поверхности обусловлена решением задачи фактически на плоскости её параметров и простотой прямого расчёта положения фрагментов в пространстве по известной зависимости $p(\tau, t)$. К **недостаткам** метода можно отнести искажения формы фрагментов при переходе из параметрического пространства на криволинейную поверхность. Особую осторожность нужно соблюдать при разбиении параметрических прямоугольных ячеек, примыкающих к полюсам поверхности, на которой они имеют форму треугольников (см. рис. 1.142, а, в, з, е). При разбиении такой ячейки на два треугольника один из них вырождается в отрезок с вытекающими отсюда сложностями его визуализации как грани.

Сложность разбиения **полигональной поверхности**, заданной граничным контуром $P = p_1 p_2 \dots p_k p_1$, зависит от её планарных свойств, т. е. близости к плоскости. Ориентация **плоской** грани характеризуется единым для всех её точек вектором нормали N , заданным в нормальном описании $\{o, N\}$, либо вычисленным по параметрическому описанию $\{o, V, W\}$ как $N = V \times W$, либо построенным по любым трём неколлинеарным точкам p_i, p_j и p_l как $N = \text{norm}(p_i p_j p_l)$.

Точки **объёмной** поверхности некопланарны, но если она не очень искривлена, то по **методу Ньюэла** [50] строится аппроксимирующая плоскость $\{p_1, N_{\text{cp}}\}$ с усреднённой нормалью

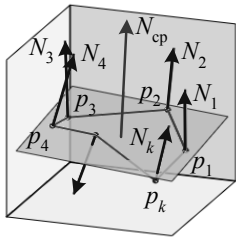


Рис. 5.32

$$N_{\text{cp}} = \sum_{i=1}^k N_i = (p_1 - p_k) \times (p_2 - p_1) + \sum_{i=2}^k (p_i - p_{i-1}) \times (p_{i+1} - p_i),$$

полученной сложением нормалей $N_i = \text{norm}(p_{i-1} p_i p_{i+1})$ к плоскостям треугольников при вершинах полигона (рис. 5.32). Оптимальная аппроксимирующая плоскость $\{o, N\}$, наименее удалённая от совокупности точек $\{p_1, \dots, p_k\}$, строится по алгоритму (1.121).

Промелькнувшее выше определение поверхности «не очень искривлённая» означает, что локальные нормали N_i к вершинным треугольникам составляют с

прямым и обратным направлениями усреднённой нормали $N_{\text{ср}}$ не слишком большие углы, скажем, не более 30° . Наличие нормалей, почти ортогональных к направлению $N_{\text{ср}}$, означает, что полигональная поверхность может иметь **складку**, описываемую **неоднозначной** функцией $z' = \varphi(x', y')$ в системе координат $x'y'z'$, в которой аппроксимирующая плоскость является фронтальной. Однозначное описание поверхности объёмной грани гарантирует, что любые ортогональные к плоскости $\{p_i, N_{\text{ср}}\}$ (или $\{o, N\}$) прямые пересекают поверхность один, а грань — не более одного раза.

Рассмотрим два метода, благодаря которым становится возможным применение **алгоритмов двухмерного разбиения объёмных граней**, как параметрических, так и полигональных.

1. С помощью матриц (3.75) или просто обнулением одной из координат множество точек поверхности S ортогонально проецируется на некоторую координатную плоскость МСК. Для минимизации проективных искажений выбирается такая плоскость, нормаль которой ω^0 (один из ортов) составляет с нормалью плоскости (аппроксимирующей плоскости) грани N **наименьший** угол, а ω -я координата нормали имеет **наибольший** модуль $|N_{\omega}|$. Второй причиной выбора плоскости, например, xu , может служить наличие явного описания поверхности, например, $z = \varphi(x, y)$.

Далее выполняется разбиение ортографической проекции S' и получение множества проекций точек разбиения Q' . Последнее, что нужно сделать, — это найти точки множества Q на исходной поверхности путём расчёта пересечения с ней проецирующих прямых $q'_i + \omega^0 t$ (рис. 5.33, а). При выборе проективной плоскости xu с нормалью $\omega^0 = z^0$ это делается по (5.18)–(5.20).

Недостатки ортографического проецирования объёмной грани заключаются, во-первых, в возможности получения на проекции самопересекающегося полигона, недопустимого для работы алгоритмов разбиения. Как иллюстрация сказанного на рис. 5.33, а построены три ортографические проекции кусочной полигональной поверхности, из которых одна — профильная проекция S'_p — является самопересекающейся. Отсутствие самопересечений — третье ограничение в выборе проективной плоскости. Тест $\text{self_test}(P)$ проверки плоского полигона P на самопересечение приведён в (1.40).

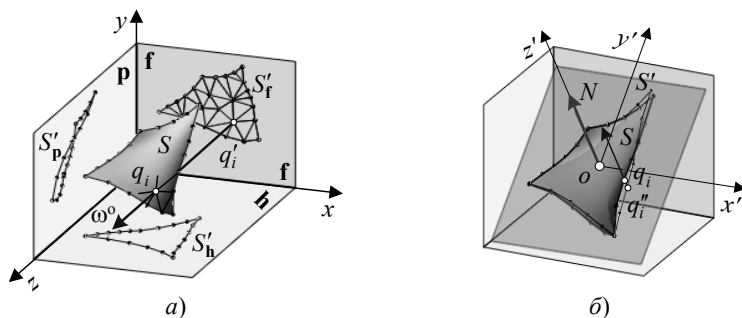


Рис. 5.33

Во-вторых, восстановление точек разбиения грани по их ортографическим проекциям вносит масштабные искажения тем большие, чем больше угол наклона нормали плоскости грани N к проективной плоскости.

2. Разбиение выполняется в плоскости (аппроксимирующей плоскости) самой грани $\{o, N\}$ (рис. 5.33,б). Преобразование поверхности S в систему координат $x'y'z'$, в которой плоскость разбиения является фронтальной, выполняется с помощью матрицы (3.25б): $C_k = MP2(o, \bar{N}, O_3, z^o)$.

Вычислив проекцию $\tilde{S}' = \tilde{S} C_k \tilde{O}_f$, делаем её разбиение, а полученное множество точек Q' на плоскости $x'y'$ преобразуем на плоскость $\{o, N\}$ по формуле $\tilde{Q}'' = \tilde{Q}' C_k^{-1}$. Если исходная поверхность S **плоская**, т. е. совпадает с $\{o, N\}$, то её разбиение фактически готово: $Q = Q''$.

В случае **объёмной** поверхности восстановление точек q_i множества Q на исходной поверхности производится путём расчёта пересечения с ней проецирующих прямых $q_i'' + Nt$:

- при **неявном** описании поверхности грани выражением $f(p) = 0$ решается скалярное уравнение $f(q_i'' + Nt) = 0$ относительно параметра t ;
- при **параметрическом** описании $p(\tau, \theta)$ решается система трёх скалярных уравнений, содержащихся в векторном уравнении $q_i'' + Nt = p(\tau, \theta)$, относительно параметров t , τ и θ .

Так или иначе, после нахождения значения t получим точку пересечения $q_i = q_i'' + Nt$.

С сожалением приходится констатировать, что применение алгоритмов разрезания плоских фигур для разбиения объёмной поверхности связано с численным решением нелинейных уравнений, что вкупе с проективными искажениями делает этот приближённый метод весьма трудоёмким.

Вот и подошёл неожиданный конец книги. Буду рад получить от читателей критические замечания по её материалу, сообщения об ошибках или опечатках, любые пожелания об улучшении содержания и оформления будущих изданий. Мой адрес e-mail: nea@mail.nnov.ru; почтовый адрес: Нижний Новгород, ул. Минина, 24, Нижегородский государственный технический университет им. Р. Е. Алексеева, кафедра «Вычислительные системы и технологии», Никулину Евгению Александровичу.

Всегда готов к взаимовыгодному сотрудничеству.

С уважением ко всем читателям, искренне ваш
Евгений Никулин.

БИБЛИОГРАФИЯ

1. Амерал Л. Машинная графика на языке C : в 4 кн. – М.: Сол Систем, 1992.
2. Ламот А. Программирование трехмерных игр для Windows. Советы профессионала по трехмерной графике и растеризации. – М.: Издательский дом «Вильямс», 2004. – 1414 с.
3. Божко А.Н. Компьютерная графика : учеб. пособие для вузов / А. Н. Божко, Д. М. Жук, В. Б. Маничев. – М.: Изд. МГТУ им. Н. Э. Баумана, 2007. – 392 с.
4. Борковский А. Б. Англо-русский словарь по программированию и информатике (с толкованиями). – М.: Русский язык, 1989. – 335 с.
5. Ватолин Д. Методы сжатия данных. Устройство архиваторов, сжатие изображений и видео / Д. Ватолин [и др.]. – М.: ДИАЛОГ-МИФИ, 2002. – 384 с.
6. Вельтмандер П. В. Машинная графика : учеб. пособие : в 3 кн. – Новосибирск: Новосибирский гос. ун-т, 1997.
7. Вернер А. В. Геометрия. Ч. I : учеб. пособие / А. В. Вернер, Б. Е. Кантор, С. А. Франгулов. – СПб.: Специальная литература, 1997. – 352 с.
8. Вернер А. В. Геометрия. Ч. II : учеб. пособие / А. В. Вернер, Б. Е. Кантор, С. А. Франгулов. – СПб.: Специальная литература, 1997. – 320 с.
9. Воеводин В. В. Линейная алгебра. – М.: Наука, 1980. – 400 с.
10. Воеводин В. В. Матрицы и вычисления / В. В. Воеводин, Ю. А. Кузнецов. – М.: Наука, 1984. – 318 с.
11. Волошинов А. В. Математика и искусство. – М.: Просвещение, 1992. – 335 с.
12. Гантмахер Ф. Р. Теория матриц. – М.: Наука, 1988. – 548 с.
13. Голованов Н. Н. Геометрическое моделирование. – М.: Физматлит, 2002. – 472 с.
14. Завьялов Ю. С. Методы сплайн-функций / Ю. С. Завьялов, Б. И. Квасов, В. Л. Мирошниченко. – М.: Наука, 1980. – 352 с.
15. Завьялов Ю. С. Сплаины в инженерной геометрии / Ю. С. Завьялов, В. А. Леус, В. А. Скороспелов. – М.: Машиностроение, 1985. – 224 с.
16. Захаров М. М. Симуляция и рендеринг фрактальных облаков воксельным методом / М. М. Захаров, Е. А. Никулин // Информационные процессы и технологии «Информатика–2013»: материалы VI Междунар. науч.-практ. конф. / СевНТУ. – Севастополь, Украина, 2013. – С. 136-137.
17. Иванов А. П. Трёхмерная компьютерная графика /А. П. Иванов, А. С. Батраков. – М.: Радио и связь, 1995. – 224 с.
18. Кирьянов Д. В. Самоучитель Mathcad 11. – СПб.: БХВ-Петербург, 2003. – 560 с.
19. Кнут Д. Искусство программирования : в 3 т. – М.: Издательский дом «Вильямс», 2001.
20. Компьютерная геометрия : учеб. пособие для студ. вузов / Н. Н. Голованов [и др.]. – М.: Издательский центр «Академия», 2006. – 512 с.
21. Корн Г. Справочник по математике (для научных работников и инженеров) / Г. Корн, Т. Корн. – М.: Наука, 1978. – 832 с.

22. Кухлинг Х. Справочник по физике. – М.: Мир, 1983. – 520 с.
23. Ласло М. Вычислительная геометрия и компьютерная графика на C++. – М.: Бином, 1997. – 304 с.
24. Лоусон Ч. Численное решение задач метода наименьших квадратов / Ч. Лоусон, Р. Хенсон. – М.: Наука, 1986. – 232 с.
25. Луизов А. В. Цвет и свет. – Л.: Энергоатомиздат, 1989. – 256 с.
26. Математика и САПР : в 2 кн. Кн. 1 / П. Шенен, М. Коснар, И. Гардан [и др.] – М.: Мир, 1988. – 204 с.
27. Математика и САПР : в 2 кн. Кн. 2 / П. Жермен-Лакур, П. Л. Жорж, П. Безье [и др.] – М.: Мир, 1989. – 264 с.
28. Никулин Е. А. Компьютерная геометрия и алгоритмы машинной графики : учеб. пособие для вузов. – СПб.: БХВ-Петербург, 2005. – 560 с.
29. Никулин Е. А. Построение составных линий с различными свойствами сопряжения и произвольными условиями фиксации сегментов // Системы обработки информации и управления: тр. НГТУ / Нижегород. гос. техн. ун-т. – Н. Новгород, 2007. – Т. 63. Вып. 13. – С. 38-46.
30. Никулин, Е. А. Построение гладких составных линий с перекрытием сегментов // Системы обработки информации и управления: тр. НГТУ / Нижегород. гос. техн. ун-т. – Н. Новгород, 2007. – Т. 63. Вып. 13. – С. 47-52.
31. Никулин Е. А. Картографические проекции: алгоритмы построения и анализ искажений // Системы обработки информации и управления: тр. НГТУ / Нижегород. гос. техн. ун-т. – Н. Новгород, 2007. – Т. 65. Вып. 14. – С. 107-118.
32. Никулин Е. А. Прямой параметрический синтез сплайновых линий // Системы обработки информации и управления: тр. НГТУ / Нижегород. гос. техн. ун-т. – Н. Новгород, 2007. – Т. 65. Вып. 14. – С. 123-130.
33. Никулин Е. А. Новый метод изменения порядков кривых и поверхностей Безье // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2011. – №1 (86). – С. 70-77.
34. Никулин Е. А. Модернизация алгоритма контролируемой триангуляции полигонов // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2011. – №2 (87). – С. 62-67.
35. Никулин Е. А. Компьютерное моделирование оптических эффектов // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2011. – №4 (91). – С. 77-87.
36. Никулин Е. А. Рекурсивный алгоритм трассировки лучей с приоритетным стеком // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2012. – №1 (94). – С. 60-69.
37. Никулин Е. А. Матричные методы рекластеризации составных объектов Безье // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2012. – №2 (95). – С. 92-101.
38. Никулин Е. А. Применение приоритетного стека в методе трассировки лучей // Информационные процессы и технологии «Информатика–2012»: материалы Междунар. науч.-практ. конф. / СевНТУ. – Севастополь, Украина, 2012. – С. 125-127.

39. Никулин Е. А. Генератор абсолютно случайных полигонов // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2013. – №2 (99). – С. 84-87.
40. Никулин Е. А. О разрешимости задачи циклической интерполяции // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2014. – №1 (103). – С. 59-65.
41. Никулин Е. А. О неразрешимости задачи циклической интерполяции // Информационные процессы и технологии «Информатика – 2014»: материалы VII Междунар. науч.-практ. конф. / СевНТУ. – Севастополь, Украина, 2014. – С. 88-89.
42. Никулин Е. А. Полиномиальная аппроксимация линиями и поверхностями // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2014. – №2 (104). – С. 61-69.
43. Никулин Е. А. Конгруэнтные преобразования полиномиальных моделей // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2014. – №3 (105). – С. 93-101.
44. Никулин Е. А. Синтез кубических сплайнов методом перекрытия парабол // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2015. – №1 (108). – С. 77-86.
45. Никулин Е. А. Процедурный шум и мультивариантный тайлинг // Тр. Нижегород. гос. техн. ун-та им. Р. Е. Алексеева / НГТУ. – Н. Новгород, 2016. – №1 (112). – С. 17-24.
46. Павлидис Т. Алгоритмы машинной графики и обработки изображений. – М.: Радио и связь, 1986. – 395 с.
47. Порев В. Н. Компьютерная графика. – СПб.: БХВ-Петербург, 2002. – 432 с.
48. Препарата Ф. Вычислительная геометрия: Введение / Ф. Препарата, М. Шеймос. – М.: Мир, 1989. – 478 с.
49. Пэрент Р. Компьютерная анимация. Теория и алгоритмы. – М.: КУДИЦ – ОБРАЗ, 2004. – 560 с.
50. Роджерс Д. Алгоритмические основы машинной графики. – М.: Мир, 1989. – 512 с.
51. Роджерс Д. Математические основы машинной графики / Д. Роджерс, Дж. Адамс. – М.: Мир, 2001. – 604 с.
52. Серапинас Б. Б. Математическая картография: учебник для вузов. – М.: Издательский центр «Академия», 2005. – 336 с.
53. Снук Г. Создание 3D-ландшафтов в реальном времени с использованием C++ и DirectX 9. – М.: КУДИЦ – ОБРАЗ, 2007. – 368 с.
54. Уилкинсон Дж. Х. Алгебраическая проблема собственных значений. – М.: Наука, 1970. – 564 с.
55. Физические величины: Справочник / А. П. Бабичев, Н. А. Бабушкина, А. М. Братковский и др.; под ред. И. С. Григорьева, Е. З. Мейлихова. – М.: Энергоатомиздат, 1991. – 1232 с.
56. Фокс А. Вычислительная геометрия. Применение в проектировании и на производстве / А. Фокс, М. Пратт. – М.: Мир, 1982. – 304 с.

57. Фостер Дж. Обработка списков. – М.: Мир, 1974. – 71 с.
58. Херн Д. Компьютерная графика и стандарт OpenGL. – 3-е изд. / Д. Херн, М. Бейкер. – М.: Издательский дом «Вильямс», 2005. – 1168 с.
59. Хорн Р. Матричный анализ / Р. Хорн, Ч. Джонсон. – М.: Мир, 1989. – 655 с.
60. Хэмминг Р. В. Численные методы (для научных работников и инженеров). – М.: Наука, 1968. – 400 с.
61. Шикин Е. В. Компьютерная графика. Динамика, реалистические изображения / Е. В. Шикин, А. В. Боресков. – М.: ДИАЛОГ-МИФИ, 1995. – 288 с.
62. Шикин Е. В. Компьютерная графика. Полигональные модели / Е. В. Шикин, А. В. Боресков. – М.: ДИАЛОГ-МИФИ, 2000. – 464 с.
63. Шикин Е. В. Кривые и поверхности на экране компьютера. Руководство по сплайнам для пользователей / Е. В. Шикин, А. И. Плис. – М.: ДИАЛОГ-МИФИ, 1996. – 240 с.
64. Эйнджел Й. Практическое введение в машинную графику. – М.: Радио и связь, 1984. – 136 с.
65. Эйнджел Э. Интерактивная компьютерная графика. Вводный курс на базе OpenGL ; 2-е изд. – М.: Издательский дом «Вильямс», 2001. – 592 с.
66. Ebert D. S., Musgrave F. K., Peachey D., Perlin K., Worley S. Texturing and modeling. A procedural approach. Third edition. – San Francisco, CA: Morgan Kaufmann Publishers Inc., 2002. – 600 p.

Евгений Александрович НИКУЛИН
КОМПЬЮТЕРНАЯ ГРАФИКА.
МОДЕЛИ И АЛГОРИТМЫ
Учебное пособие
Издание второе, стереотипное

Зав. редакцией
инженерно-технической литературы *Е. В. Баженова*

ЛР № 065466 от 21.10.97
Гигиенический сертификат 78.01.10.953.П.1028
от 14.04.2016 г., выдан ЦГСЭН в СПб

Издательство «ЛАНЬ»
lan@lanbook.ru; www.lanbook.com
196105, Санкт-Петербург, пр. Юрия Гагарина, д.1, лит. А.
Тел.: (812) 336-25-09, 412-92-72.
Бесплатный звонок по России: 8-800-700-40-71

Подписано в печать 03.07.18.
Бумага офсетная. Гарнитура Школьная. Формат 70×100^{1/16}.
Печать офсетная. Усл. п. л. 57,53. Тираж 100 экз.
Заказ № 451-18.
Отпечатано в полном соответствии
с качеством предоставленного оригинал-макета
в АО «Т8 Издательские технологии»
109316, г. Москва, Волгоградский пр., д. 42, к. 5.